차 례

1 머신러닝이란: 규칙을 짜지 않고 데이터로 배우기 1

들어가며: 스팸 메일, if 문으로 거를 수 있을까요? 1

1.1 규칙을 짜는 프로그램, 데이터로 배우는 프로그램 5

1.2 데이터를 표로 보기: 샘플, 특성, 레이블 6

1.3 머신러닝의 세 갈래 8

1.4 모델, 학습, 예측 12

1.5 첫 모델: 최근접 이웃 분류 14

1.6 숫자를 맞히는 이웃: 회귀 22

1.7 일반화와 과적합: 외우기와 이해하기 23

1.8 특성의 눈금 맞추기 29

1.9 얼마나 잘하나: 평가 맛보기 30

1.10 머신러닝 작업의 흐름 32

1.11 딥러닝은 무엇이 다른가요? 34

1.12 이 책의 지도 38

1.13 실습: 와인 생산자 맞히기 39

1.14 정리와 다음 장 미리보기 42

2 데이터 준비: 모델에 넣기 전에 할 일 45

들어가며: 장 봐 온 재료를 바로 냄비에 넣나요? 45

2.1 먼저 데이터를 살펴보기 47

2.2 학습·검증·테스트로 나누기 49

2.3 데이터 누수: 시험 문제가 새는 길 53

2.4 결측치 처리 58

2.5 범주형 인코딩: 글자를 숫자로 61

2.6 스케일링: 단위 맞추기 66

2.7 ColumnTransformer와 Pipeline으로 묶기 69

2.8 불균형 데이터 . . . 73
2.9 시간 순서가 있는 데이터 . . . 76
2.10 실습: 통신사 고객 이탈 데이터 손질하기 . . . 78
2.11 정리와 다음 장 미리보기 . . . 86

3 scikit-learn으로 보는 고전 머신러닝 89
들어가며: 엑셀 표 같은 데이터에도 딥러닝이 필요할까요? . . . 89
3.1 scikit-learn의 약속: fit, predict, score . . . 91
3.2 선형 회귀: 숫자를 예측하는 직선 . . . 93
3.3 로지스틱 회귀: 확률로 답하는 분류 . . . 96
3.4 결정 트리: 스무고개로 답 찾기 . . . 99
3.5 여럿이 함께: 랜덤 포레스트와 그래디언트 부스팅 . . . 102
3.6 정답 없이 묶기: k-means 군집 . . . 106
3.7 분류 모델 평가하기: 정확도만 보면 속는 이유 . . . 110
3.8 회귀 모델 평가하기: MAE, RMSE, R^2 . . . 116
3.9 교차 검증: 모의고사는 한 번보다 다섯 번 . . . 118
3.10 하이퍼파라미터 탐색 . . . 123
3.11 모델 저장하고 불러오기 . . . 126
3.12 언제 딥러닝 대신 고전 머신러닝을 쓸까? . . . 128
3.13 실습: 캘리포니아 집값 예측 모델 고르기 . . . 129
3.14 정리와 다음 장 미리보기 . . . 134

4 텐서: 숫자를 담는 그릇 137
들어가며: 사진 한 장은 숫자 몇 개일까요? . . . 137
4.1 텐서 만들기 . . . 139
4.2 모양과 차원 . . . 144
4.3 자료형 . . . 145
4.4 인덱싱과 슬라이싱 . . . 148
4.5 모양 바꾸기 . . . 155
4.6 한꺼번에 계산하기 . . . 160
4.7 제자리 연산과 메모리 . . . 167
4.8 장치: CPU와 GPU 사이 옮기기 . . . 168
4.9 텐서 저장하고 불러오기 . . . 171
4.10 NumPy와 다른 점, 자주 만나는 오류 . . . 174

4.11 실습: 도시별 기후 패턴 비교하기 . . . 178
4.12 정리와 다음 장 미리보기 . . . 181

5 자동 미분과 경사하강법 183
들어가며: 안개 낀 산에서 내려오려면? . . . 183
5.1 기울기: 조금 움직였을 때 얼마나 변하나 . . . 185
5.2 손으로 해 보는 경사하강법 . . . 189
5.3 선형 회귀를 손으로 학습하기 . . . 191
5.4 PyTorch에게 미분 맡기기 . . . 194
5.5 계산 그래프: PyTorch는 무엇을 기억하나 . . . 198
5.6 연쇄 법칙: 기울기가 거꾸로 흐르는 이유 . . . 200
5.7 기울기가 쌓이는 함정과 zero_grad . . . 203
5.8 기록 끄기: no_grad, detach, inference_mode . . . 205
5.9 제자리 연산과 자동 미분의 충돌 . . . 209
5.10 학습률: 너무 크면 튀고, 너무 작으면 늦다 . . . 210
5.11 미니배치 경사하강법 맛보기 . . . 214
5.12 torch.autograd.grad: 기울기를 직접 받기 . . . 217
5.13 실습: 당뇨병 진행 정도를 autograd만으로 예측하기 . . . 219
5.14 정리와 다음 장 미리보기 . . . 224

6 nn.Module과 학습 루프: 모든 모델이 따르는 표준형 227
들어가며: 모델이 달라도 학습하는 모양은 같을까요? . . . 227
6.1 층 하나: 입력 @ 가중치 + 편향 . . . 228
6.2 활성화 함수: 직선을 구부리는 장치 . . . 232
6.3 nn.Module로 모델 만들기 . . . 236
6.4 파라미터 들여다보기 . . . 241
6.5 손실 함수: 얼마나 틀렸나 . . . 242
6.6 옵티마이저: 어떻게 고칠까 . . . 246
6.7 표준 학습 루프: 다섯 줄의 반복 . . . 250
6.8 평가 루프: 학습 모드와 평가 모드 . . . 254
6.9 장치로 옮기기 . . . 257
6.10 모델 저장과 불러오기 . . . 259
6.11 실습: FashionMNIST 옷 사진 분류기 . . . 262
6.12 정리와 다음 장 미리보기 . . . 272

7 데이터 파이프라인: Dataset과 DataLoader 273

들어가며: 주방은 재료를 한꺼번에 꺼내지 않습니다 . . . 273

7.1 Dataset: 번호를 주면 샘플 하나를 주는 약속 . . . 275

7.2 TensorDataset: 이미 텐서로 있을 때 . . . 277

7.3 DataLoader: 배치로 묶어 나르기 . . . 278

7.4 직접 만든 Dataset ①: CSV 파일 . . . 283

7.5 직접 만든 Dataset ②: 이미지 폴더 . . . 287

7.6 transforms: 꺼낼 때 손질하기 . . . 291

7.7 collate_fn: 길이가 다른 샘플 묶기 . . . 297

7.8 학습/검증 나누기: random_split과 재현 가능한 분할 . . . 302

7.9 불균형 데이터: WeightedRandomSampler . . . 305

7.10 num_workers와 병목 찾기 . . . 307

7.11 실습: 도형 그림 파이프라인 한 바퀴 . . . 314

7.12 정리와 다음 장 미리보기 . . . 318

8 학습 잘 시키기 319

들어가며: 문제집 답을 통째로 외운 학생 . . . 319

8.1 과적합과 과소적합 진단하기 . . . 320

8.2 가중치 감쇠(weight decay) . . . 327

8.3 드롭아웃 . . . 330

8.4 배치 정규화 . . . 334

8.5 데이터 증강은 7장에서 . . . 337

8.6 가중치 초기화 . . . 337

8.7 학습률 스케줄러 . . . 340

8.8 조기 종료 . . . 344

8.9 그래디언트 클리핑 . . . 347

8.10 재현성 체크리스트 . . . 349

8.11 손실이 안 줄 때 디버깅 . . . 352

8.12 실습: 외우는 신경망에 대책을 하나씩 더하기 . . . 363

8.13 정리와 다음 장 미리보기 . . . 367

9 CNN과 이미지 분류: 돋보기를 옮기며 무늬 찾기 369

들어가며: 사진을 한 줄로 펴면 무엇을 잃을까요? . . . 369

9.1 합성곱: 돋보기를 옮기며 무늬 찾기 . . . 371

9.2 패딩과 보폭: 출력 크기 정하기 . . . 376
9.3 채널: 겹겹이 쌓인 그림 . . . 378
9.4 풀링: 줄여서 요약하기 . . . 380
9.5 첫 CNN으로 옷 사진 분류하기 . . . 382
9.6 그림을 밀어도 알아볼까요? . . . 389
9.7 CNN 속의 배치 정규화와 드롭아웃 . . . 392
9.8 유명한 구조의 아이디어 . . . 394
9.9 CNN은 무엇을 보는가 . . . 405
9.10 실습: 작은 ResNet으로 옷 분류기 다시 만들기 . . . 411
9.11 정리와 다음 장 미리보기 . . . 415

10 전이 학습: 남이 학습한 모델 빌려 쓰기 417
들어가며: 그림을 잘 그리는 사람은 지도도 빨리 읽습니다 . . . 417
10.1 사전학습 모델 불러오기 . . . 418
10.2 입력 전처리 맞추기 . . . 422
10.3 이 장의 데이터: EuroSAT 위성 사진 . . . 425
10.4 특징 추출: 눈은 빌리고 판단만 새로 . . . 431
10.5 처음부터 학습하면 어떨까 . . . 440
10.6 미세조정: 몸통까지 조금씩 고치기 . . . 442
10.7 다른 모델, 다른 입력 . . . 449
10.8 저장하고 불러오기 . . . 453
10.9 실습: 데이터가 적을수록 전이 학습이 빛난다 . . . 455
10.10 정리와 다음 장 미리보기 . . . 461

11 RNN과 LSTM: 순서가 있는 데이터 463
들어가며: 순서를 바꾸면 뜻이 바뀝니다 . . . 463
11.1 순서 데이터의 모양 . . . 465
11.2 RNN 셀: 메모장을 고쳐 쓰며 읽기 . . . 467
11.3 순서를 알아야 풀리는 문제: 괄호 짝 맞추기 . . . 471
11.4 기울기 소실과 폭주 . . . 477
11.5 LSTM: 기억을 따로 나르는 벨트 . . . 480
11.6 GRU와 파라미터 비용 . . . 483
11.7 긴 기억 시험: 멀리 떨어진 두 수 더하기 . . . 484
11.8 층 쌓기와 양방향 . . . 488

11.9 길이가 다른 순서열: pack_padded_sequence 490
11.10 시계열 예측: 마우나로아 이산화탄소 495
11.11 실습: 속담 짓는 글자 단위 LSTM 502
11.12 정리와 다음 장 미리보기 511

12 어텐션과 트랜스포머: 처음부터 직접 구현 513
들어가며: “그것”은 무엇을 가리킬까요? 513
12.1 어텐션: 부드럽게 꺼내는 딕셔너리 514
12.2 점수를 d의 제곱근으로 나누는 이유 518
12.3 셀프 어텐션: 문장이 자기 자신을 본다 519
12.4 마스크: 보면 안 되는 칸 가리기 522
12.5 멀티헤드 어텐션: 여러 관점으로 동시에 보기 525
12.6 순서 알려 주기: 위치 정보 530
12.7 트랜스포머 블록 조립하기 535
12.8 작은 GPT 만들기 541
12.9 글 만들기와 들여다보기 554
12.10 실습: 부품을 하나씩 빼 보기 559
12.11 정리와 다음 장 미리보기 561

13 한국어 NLP 실전: 토크나이저부터 감성 분류까지 563
들어가며: “아버지가방에들어가신다”를 어디서 끊을까요? 563
13.1 이 장의 데이터: 네이버 영화 리뷰 565
13.2 한국어가 어려운 이유 567
13.3 토큰화 ①: 글자와 어절 571
13.4 토큰화 ②: 형태소 분석 572
13.5 토큰화 ③: 서브워드 575
13.6 사전학습 토크나이저 쓰기 579
13.7 임베딩: 번호를 화살표로 582
13.8 처음부터 학습하는 기준 모델 586
13.9 사전학습 인코더 미세조정 589
13.10 평가: 숫자 너머를 보기 596
13.11 Hugging Face datasets로 데이터 다루기 600
13.12 실습: 뉴스 제목 주제 분류기 만들기 602
13.13 정리와 다음 장 미리보기 606

14 언어 모델 다루기: LLM 활용과 LoRA 609
들어가며: 휴대폰 자판은 어떻게 다음 단어를 알까요? . . . 609
14.1 사전학습 언어 모델 불러오기 . . . 610
14.2 다음 토큰 예측: LLM이 하는 일은 하나뿐 . . . 612
14.3 디코딩: 다음 토큰을 고르는 방법 . . . 615
14.4 프롬프트와 채팅 템플릿 . . . 620
14.5 작은 모델의 한계를 정직하게 보기 . . . 626
14.6 메모리 계산과 양자화 맛보기 . . . 628
14.7 LoRA의 아이디어: 고칠 부분만 작게 . . . 633
14.8 peft로 LLM에 LoRA 붙이기 . . . 638
14.9 실습: 1권 도우미로 미세조정하기 . . . 640
14.10 정리와 다음 장 미리보기 . . . 652

15 생성 모델: 오토인코더, VAE, 확산 모델 입문 655
들어가며: 고르는 모델, 그리는 모델 . . . 655
15.1 오토인코더: 줄였다가 되살리기 . . . 656
15.2 잠재 공간 들여다보기 . . . 664
15.3 합성곱 오토인코더로 잡음 지우기 . . . 666
15.4 재구성 오차로 이상한 것 찾기 . . . 669
15.5 오토인코더로 새 그림을 그릴 수 있을까 . . . 671
15.6 VAE: 점 대신 구름으로 . . . 673
15.7 GAN: 위조범과 감별사 . . . 682
15.8 확산 모델: 잡음을 걷어 내며 그리기 . . . 683
15.9 생성 모델은 어떻게 평가할까 . . . 698
15.10 실습: 주문한 옷을 그리는 조건부 확산 모델 . . . 702
15.11 정리와 다음 장 미리보기 . . . 707

16 GPU와 성능: 같은 모델을 더 빠르게, 더 가볍게 709
들어가며: 같은 요리, 다른 주방 . . . 709
16.1 GPU는 왜 빠를까요 . . . 713
16.2 GPU 시간 제대로 재기 . . . 715
16.3 CPU와 GPU, 언제 GPU가 이길까요 . . . 719
16.4 GPU 메모리 들여다보기 . . . 720
16.5 데이터 옮기는 비용: 고정 메모리와 non_blocking . . . 726

16.6 혼합 정밀도: 16비트로 계산하기 . . . 728
16.7 TF32: float32 행렬 곱을 빠르게 . . . 737
16.8 torch.compile: 모델을 미리 번역해 두기 . . . 739
16.9 어텐션은 scaled_dot_product_attention으로 . . . 746
16.10 프로파일러로 시간 새는 곳 찾기 . . . 749
16.11 배치 크기와 처리량 . . . 754
16.12 성능 체크리스트 . . . 756
16.13 실습: 옷 분류기 학습을 단계별로 빠르게 . . . 757
16.14 정리와 다음 장 미리보기 . . . 761

17 분산·대규모 학습: 메모리를 아끼고 GPU를 늘리기 763
들어가며: 이삿짐을 혼자 옮길까, 친구를 부를까? . . . 763
17.1 학습 중 GPU 메모리는 어디에 쓰일까 . . . 764
17.2 그래디언트 누적: 큰 배치를 나눠 담기 . . . 768
17.3 활성화 체크포인팅: 적어 두지 말고 다시 계산하기 . . . 772
17.4 여러 GPU에 일을 나누는 방법들 . . . 775
17.5 분산의 기본 낱말: 프로세스, rank, all-reduce . . . 776
17.6 통신은 공짜가 아니다 . . . 780
17.7 DistributedDataParallel로 학습하기 . . . 782
17.8 1 GPU와 같은 결과일까 . . . 786
17.9 GPU를 늘리면 몇 배 빨라질까 . . . 789
17.10 no_sync: 누적과 DDP 합치기 . . . 792
17.11 FSDP: 모델까지 나눠 담기 . . . 795
17.12 흔한 오류와 대처 . . . 798
17.13 실습: 옷 분류기를 세 가지 방법으로 학습하기 . . . 800
17.14 정리와 다음 장 미리보기 . . . 809

18 실험 관리와 재현성 811
들어가며: 그 86%는 어떤 설정이었을까요? . . . 811
18.1 실험 하나를 이루는 것 . . . 812
18.2 설정을 코드 밖으로 . . . 813
18.3 체크포인트와 재개 . . . 824
18.4 실험 기록 남기기 . . . 831
18.5 하이퍼파라미터 탐색 자동화 . . . 837

18.6 여러 씨앗으로 보고하기 . . . 843
18.7 데이터·코드·환경 버전 기록 . . . 846
18.8 실습: 작은 실험 묶음과 재현성 보고서 . . . 852
18.9 정리와 다음 장 미리보기 . . . 858

19 모델 내보내기와 경량화: ONNX와 양자화 861
들어가며: 모델을 다 만들었는데, 어디서 돌리죠? . . . 861
19.1 ONNX: 모델의 악보 . . . 863
19.2 torch.onnx.export로 내보내기 . . . 864
19.3 ONNX Runtime으로 추론하기 . . . 867
19.4 입력 모양 정하기 . . . 869
19.5 내보낼 수 있는 코드, 없는 코드 . . . 872
19.6 그래프 들여다보기 . . . 875
19.7 얼마나 빨라졌나: 속도 재기 . . . 883
19.8 양자화 다시 보기: 눈금, 영점, 정수 계산 . . . 885
19.9 PyTorch 안에서 양자화하기: torchao . . . 890
19.10 ONNX Runtime으로 양자화하기 . . . 892
19.11 더 작게 만드는 다른 방법: 가지치기와 지식 증류 . . . 897
19.12 휴대폰과 작은 기기에서: 런타임 고르기 . . . 901
19.13 실습: 작은 ResNet을 배포용으로 만들기 . . . 902
19.14 정리와 다음 장 미리보기 . . . 907

20 FastAPI로 모델 서빙: 내 모델을 웹 서비스로 909
들어가며: 모델 파일을 친구에게 보내면 끝일까요? . . . 909
20.1 모델을 서버에 올리기 . . . 910
20.2 서버가 켜질 때 한 번만: lifespan . . . 913
20.3 주문서 양식: 요청과 응답 스키마 . . . 916
20.4 사진 파일 받기 . . . 918
20.5 여러 장을 한 번에: 배치 추론 . . . 923
20.6 오래 걸리는 계산과 async . . . 928
20.7 살아 있나, 준비됐나, 무슨 모델인가 . . . 931
20.8 ONNX Runtime으로 백엔드 바꾸기 . . . 934
20.9 지연 시간 재기: 평균에 속지 않기 . . . 936
20.10 LLM의 답을 흘려보내기: 스트리밍 . . . 939

20.11 실습: 옷 사진 분류 서비스 완성하기 942
20.12 정리와 다음 장 미리보기 964

21 종합 프로젝트: 맛집 추천 AI 967

들어가며: “오늘 점심 뭐 먹지?” 967
21.1 좋은 추천이란 무엇일까요? 968
21.2 예제용 데이터 만들기 972
21.3 시간 순서로 나누기 978
21.4 기준선부터: 인기순과 동네 인기순 980
21.5 행렬 분해: 사용자와 식당을 화살표로 982
21.6 순위를 직접 배우기: BPR 987
21.7 학습된 임베딩 들여다보기 989
21.8 특징을 함께 쓰는 투-타워 모델 993
21.9 콜드 스타트: 처음 보는 식당과 사용자 997
21.10 실험 기록과 여러 씨앗 1002
21.11 서빙 준비: 벡터를 미리 계산해 두기 1005
21.12 실습: 맛집 추천 API 완성하기 1011
21.13 정리와 2권을 마치며 1025

부록 A. 실행 환경 만들기 1027

A.1 이 책에서 쓰는 패키지와 확인한 버전 1027
A.2 uv로 2권 프로젝트 만들기 1030
A.3 macOS: Apple 실리콘과 MPS 1031
A.4 리눅스·윈도우와 NVIDIA GPU: CUDA 1032
A.5 GPU가 없을 때: Colab과 클라우드 GPU 1035
A.6 장치 고르기와 설치 확인 스크립트 1036
A.7 자주 만나는 설치 문제 1039
A.8 내려받은 모델은 어디에 쌓일까: 캐시와 디스크 용량 1041
A.9 데이터 폴더 관례: BOOK_DATA_DIR 1044

부록 B. 꼭 필요한 수학 직관 1047

B.1 벡터: 숫자 묶음은 화살표다 1047
B.2 행렬 곱과 모양 규칙 1051
B.3 미분과 기울기 1054

B.4 지수와 로그 1057
B.5 확률과 소프트맥스 1059
B.6 교차 엔트로피: 정답 확률에 매기는 벌점 1063
B.7 평균·분산·표준편차와 표준화 1064
B.8 확률 분포에서 뽑기: 정규분포와 샘플링 1068
B.9 한눈에 보기 1070

부록 C. 용어집 **1073**
C.1 한글 용어 1073
C.2 영문으로 쓰는 용어 1099

01

머신러닝이란: 규칙을 짜지 않고 데이터로 배우기

학습 목표

- 규칙을 직접 짜는 프로그램과 데이터로 배우는 프로그램의 차이를 설명하기
- 샘플·특성·레이블, 모델·학습·예측이라는 기본 낱말로 머신러닝 문제를 읽기
- 지도 학습(분류·회귀), 비지도 학습, 강화 학습을 예로 구분하기
- 최근접 이웃 분류를 NumPy로 직접 만들고 scikit-learn과 같은 결과를 내기
- 과적합과 일반화를 "외우기와 이해하기"로 설명하고, 딥러닝이 무엇을 더 하는지 말하기

들어가며: 스팸 메일, if 문으로 거를 수 있을까요?

메일함에 쌓이는 스팸을 자동으로 거르는 프로그램을 만든다고 해 봅시다. 1권을 마친 여러분이라면 바로 떠오르는 방법이 있을 거예요. 1권 4장에서 배운 `if`와 `in`으로 "이런 단어가 들어 있으면 스팸"이라는 규칙을 짜는 겁니다.

```
def 스팸인가(제목: str) -> bool:
    """사람이 정해 둔 규칙(금지어)으로 스팸을 가려낸다."""
    금지어 = ["무료", "당첨", "대출"]
    for 단어 in 금지어:
```

```
        if 단어 in 제목:
            return True
    return False


메일들 = [
    "축하합니다 경품 당첨 안내",
    "무료 체험 쿠폰이 도착했어요",
    "내일 회의 자료 공유드립니다",
    "무.료 상담 지금 바로 전화 주세요",
    "저금리 대환 상품 한정 안내",
    "도서관 책 대출 기한이 내일까지예요",
]
for 제목 in 메일들:
    판정 = "스팸" if 스팸인가(제목) else "정상"
    print(f"[{판정}] {제목}")
```

```
[스팸] 축하합니다 경품 당첨 안내
[스팸] 무료 체험 쿠폰이 도착했어요
[정상] 내일 회의 자료 공유드립니다
[정상] 무.료 상담 지금 바로 전화 주세요
[정상] 저금리 대환 상품 한정 안내
[스팸] 도서관 책 대출 기한이 내일까지예요
```

처음 두 통은 잘 걸렀습니다. 그런데 나머지를 보세요. 스팸 보내는 사람이 "무료"를 "무.료"로 바꿔 쓰자 그냥 통과했고, 금지어 목록에 없는 "저금리 대환"도 빠져나갔어요. 반대로 도서관 책 **대출** 안내는 멀쩡한 메일인데 스팸으로 몰렸습니다.

규칙을 고치면 되지 않을까요? "무.료"도 넣고, "저금리"도 넣고, "도서관"이 있으면 대출이 들어 있어도 봐주고... 해 보면 금방 압니다. 스팸 쪽은 매일 새로운 표현을 만들어 내고, 규칙은 끝없이 늘어나며, 규칙끼리 서로 부딪히기 시작해요. 사람이 모든 경우를 미리 생각해서 적어 두는 방식에는 한계가 있습니다.

그럼 생각을 뒤집어 봅시다. 규칙을 사람이 적는 대신, 사람은 메일마다 "이건 스팸, 이건 정상"이라고 **표시만** 해 두고, 규칙은 프로그램이 그 예들을 보고 **스스로 찾게** 하면 어떨까요?

```
from collections import Counter

# 사람이 스팸/정상 표시를 붙여 둔 메일 제목 (예제용으로 만든 데이터)
학습_메일 = [
    ("경품 당첨 축하 안내", "스팸"),
    ("무료 쿠폰 지금 받기", "스팸"),
    ("무.료 상담 지금 전화", "스팸"),
    ("저금리 대환 대출 한정 안내", "스팸"),
    ("저금리 상품 지금 신청", "스팸"),
    ("한정 특가 무료 배송", "스팸"),
    ("내일 회의 자료 공유", "정상"),
    ("도서관 책 대출 기한 안내", "정상"),
    ("주말 등산 일정 공유", "정상"),
    ("회의 시간 변경 안내", "정상"),
    ("어머니 생신 선물 상의", "정상"),
    ("책 모임 자료 정리", "정상"),
]

# 1) 학습: 단어마다 스팸과 정상에 몇 번 나왔는지 센다
스팸_단어 = Counter()
정상_단어 = Counter()
for 제목, 표시 in 학습_메일:
    if 표시 == "스팸":
        스팸_단어.update(제목.split())
    else:
        정상_단어.update(제목.split())


def 스팸_점수(제목: str) -> int:
    """단어마다 (스팸 등장 수 - 정상 등장 수)를 더한다."""
    return sum(스팸_단어[w] - 정상_단어[w] for w in 제목.split())


# 2) 예측: 처음 보는 메일에 점수를 매긴다
새_메일 = [
    "무.료 체험 지금 신청",
    "저금리 대환 특가 안내",
```

```
    "도서관 대출 연장 안내",
    "다음 주 회의 자료",
]
for 제목 in 새_메일:
    점수 = 스팸_점수(제목)
    판정 = "스팸" if 점수 > 0 else "정상"
    print(f"[{판정}] 점수 {점수:+d}  {제목}")
```

```
[스팸] 점수 +5  무.료 체험 지금 신청
[스팸] 점수 +4  저금리 대환 특가 안내
[정상] 점수 -1  도서관 대출 연장 안내
[정상] 점수 -4  다음 주 회의 자료
```

이 프로그램에는 금지어 목록이 없습니다. 하는 일은 두 가지뿐이에요. 먼저 표시가 붙은 메일 12통을 읽으면서 단어마다 스팸 쪽에 몇 번, 정상 쪽에 몇 번 나왔는지 셉니다(1권 17장의 `Counter`). 그다음 새 메일이 오면 단어마다 "스팸 쪽 횟수 - 정상 쪽 횟수"를 더해 점수가 0보다 크면 스팸이라고 판정해요.

결과가 달라졌죠? "무.료"와 "저금리 대환"이 걸렸고, 도서관 대출 안내는 정상으로 통과했습니다. "대출"은 스팸에도 한 번, 정상에도 한 번 나와서 혼자서는 점수를 움직이지 못했고, "도서관"과 "안내"가 판정을 정상 쪽으로 기울였어요. 새로운 스팸 표현이 나타나면? 규칙을 고칠 필요 없이 표시한 예를 더 넣어 주면 됩니다.

이것이 이 책 전체를 꿰는 생각, **머신러닝(machine learning)** 입니다. 규칙을 짜는 대신 데이터를 주고, 규칙은 프로그램이 데이터에서 배우게 하는 거예요. 물론 위의 단어 세기는 아주 소박한 방법입니다. 이 장에서는 이 생각을 제대로 된 낱말로 정리하고, 첫 머신러닝 모델을 처음부터 직접 만든 다음, 같은 일을 scikit-learn 한 줄로 해 봅니다. 그리고 2권의 주인공인 딥러닝이 그 위에서 무엇을 더 하는지까지 살펴볼게요.

Q

Q. 이 장의 예제를 실행하려면 무엇을 설치해야 하나요? 1권에서 쓰던 uv 그대로입니다. 프로젝트 폴더에서 `uv add numpy scikit-learn`을 실행하세요. **scikit-learn**은 고전적인 머신러닝 방법을 거의 다 모아 둔 라이브러리로, 이 장과 2·3장에서 씁니다. 이 장의 예제는 파이썬 3.12, scikit-learn 1.9에서 실행해 확인했어요. 데이터는 scikit-learn에 들어 있는 것만 써서 따로 내려받을 필요가 없습니다.

1.1 규칙을 짜는 프로그램, 데이터로 배우는 프로그램

입력과 출력의 자리가 바뀝니다

1권에서 만든 프로그램은 모두 같은 모양이었습니다. 사람이 **규칙**(코드)을 쓰고, 컴퓨터는 **데이터**를 받아 그 규칙대로 **답**을 냈어요. 머신러닝은 이 그림에서 자리를 바꿉니다. 사람이 **데이터와 답**을 함께 주면, 컴퓨터가 둘을 잇는 **규칙**을 만들어 냅니다.

	전통적인 프로그래밍	머신러닝
사람이 주는 것	규칙(코드) + 데이터	데이터 + 정답
컴퓨터가 내놓는 것	답	규칙(모델)
규칙을 바꾸려면	코드를 고친다	데이터를 바꾸고 다시 학습한다
잘 맞는 문제	세금 계산, 정렬, 회원 가입	스팸 거르기, 사진 분류, 추천

머신러닝을 처음 정리한 사람 중 하나인 톰 미첼은 이렇게 정의했습니다. “어떤 작업에서 경험이 쌓일수록 성능이 좋아진다면, 그 프로그램은 경험으로부터 **배운다**고 말한다.” 스팸 예제에 대 보면 작업은 스팸 판정, 경험은 표시가 붙은 메일, 성능은 맞힌 비율이에요. 표시한 메일을 더 넣어 줄수록 판정이 좋아진다면 그 프로그램은 배우고 있는 겁니다.

언제 머신러닝을 쓰고, 언제 쓰지 말까요?

머신러닝이 모든 문제의 답은 아닙니다. 다음 두 질문에 모두 “그렇다”일 때 꺼내 드는 도구라고 생각하세요.

- **규칙을 사람이 적기 어렵다.** 고양이 사진과 개 사진을 가르는 규칙을 `if` 문으로 적을 수 있을까요? 우리는 둘을 한눈에 구별하지만 그 이유를 말로 다 적지는 못합니다. 이런 문제가 머신러닝의 무대예요.
- **예가 될 데이터가 충분히 있다.** 배울 거리가 없으면 배울 수 없습니다. 스팸 표시가 붙은 메일, 품종이 적힌 꽃 사진처럼 “입력과 정답의 짝”이 많이 모여 있어야 해요.

반대로 규칙이 분명한 일에는 쓰지 않는 것이 맞습니다. 부가세 10%를 계산하는 데 머신러닝을 쓰면 99.9%만 맞는 계산기가 생길 뿐이에요. 1권에서 배운 대로 규칙을 짜면 100% 맞습니다. 그리고 머신러닝은 **틀릴 수 있는** 프로그램이라는 점도 처음부터 받아들여야 합니다. 데이터에서 배운 규칙은 언제나 “대체로 맞는” 규칙이라서, 얼마나 자주 틀리는지를 재고 관리하는 일이 머신러닝 작업의 큰

부분을 차지해요(1.9절).

Q. 인공지능, 머신러닝, 딥러닝은 서로 어떤 관계인가요? 큰 원 안에 작은 원이 들어 있는 관계입니다. **인공지능(AI)** 은 사람처럼 판단하는 프로그램을 만들려는 분야 전체예요. 사람이 규칙을 일일이 적어 넣는 방식도 인공지능에 들어갑니다. 그 안에서 데이터로 규칙을 배우는 방법을 **머신러닝**이라 하고, 머신러닝 중에서 여러 층으로 쌓은 **신경망**으로 배우는 방법을 **딥러닝**이라고 해요. 요즘 화제인 대화형 AI(14장)도 딥러닝으로 만든 모델입니다. 1.11절에서 딥러닝이 무엇이 다른지 직접 확인해 봅니다.

1.2 데이터를 표로 보기: 샘플, 특성, 레이블

붓꽃 150송이

머신러닝을 이야기하려면 낱말 몇 개를 먼저 맞춰 둬야 합니다. 머신러닝 교과서마다 첫 예제로 나오는 **붓꽃(iris) 데이터**로 익혀 볼게요. 식물학자 에드거 앤더슨이 붓꽃 세 품종 150송이의 꽃받침과 꽃잎 크기를 잰 기록을 통계학자 로널드 피셔가 1936년 논문에 실은 것인데, scikit-learn에 들어 있어서 한 줄로 불러올 수 있습니다(UCI 머신러닝 저장소에도 CC BY 4.0으로 공개되어 있어요).

```
import numpy as np
from sklearn.datasets import load_iris

붓꽃 = load_iris()
X, y = 붓꽃.data, 붓꽃.target

print("X 모양:", X.shape, " y 모양:", y.shape)
print("특성 이름:", 붓꽃.feature_names)
print("레이블 이름:", 붓꽃.target_names.tolist())
for i in [0, 50, 100]:
    print(f"{i:3d}번 꽃: 특성 {X[i]} → 레이블 {y[i]} ({붓꽃.target_names[y[i]]})")
print("레이블별 개수:", np.bincount(y))
```

```
X 모양: (150, 4)  y 모양: (150,)
특성 이름: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
레이블 이름: ['setosa', 'versicolor', 'virginica']
  0번 꽃: 특성 [5.1 3.5 1.4 0.2] → 레이블 0 (setosa)
```

```
 50번 꽃: 특성 [7.  3.2 4.7 1.4] → 레이블 1 (versicolor)
100번 꽃: 특성 [6.3 3.3 6.  2.5] → 레이블 2 (virginica)
레이블별 개수: [50 50 50]
```

출력을 하나씩 읽어 보겠습니다.

- `X`는 **150행 4열 표**입니다. 한 행이 꽃 한 송이, 한 열이 잰 항목 하나예요. 이렇게 데이터 한 건을 **샘플(sample)** 이라고 부릅니다.
- 네 열은 꽃받침 길이·너비, 꽃잎 길이·너비(단위 cm)입니다. 샘플을 설명하는 이런 항목을 **특성(feature)** 이라고 해요. 0번 꽃은 `[5.1 3.5 1.4 0.2]`라는 특성 네 개로 설명됩니다.
- `y`는 꽃마다 붙은 정답, 즉 품종입니다. 이걸 **레이블(label)** 이라고 해요. 0은 setosa, 1은 versicolor, 2는 virginica로, 이름 대신 번호로 적었습니다. 컴퓨터는 글자보다 숫자를 다루기 편하니까요.
- 품종마다 50송이씩, 고르게 들어 있습니다.

표와 정답, 1권 20장에서 pandas로 다루던 바로 그 모양이죠. 머신러닝 데이터는 대부분 이렇게 **"샘플 × 특성" 표 하나와 레이블 목록 하나**로 정리됩니다.

X는 대문자, y는 소문자

코드에서 특성 표는 대문자 `X`, 레이블은 소문자 `y`로 쓰는 것이 관례예요. 수학에서 행렬(표)은 대문자, 벡터(한 줄)는 소문자로 쓰던 습관에서 왔습니다. 1권에서는 변수 이름을 소문자로 쓰라고 배웠는데, 머신러닝 코드에서만큼은 `X`를 대문자로 쓰는 사람이 대부분이라 이 책도 그 관례를 따릅니다. `X_학습`, `y_테스트`처럼 뒤에 쓰임새를 붙여 쓸게요.

낱말	뜻	붓꽃에서는
샘플	데이터 한 건, 표의 한 행	꽃 한 송이
특성	샘플을 설명하는 항목, 표의 한 열	꽃잎 길이 등 4개
레이블	맞히고 싶은 정답	품종(0, 1, 2)
`X`	샘플 × 특성 표	모양 `(150, 4)`
`y`	샘플마다의 레이블	모양 `(150,)`

샘플은 공간 속의 점입니다

특성 네 개짜리 샘플을 **4차원 공간의 점 하나**로 보는 눈도 길러 두면 좋습니다. 특성이 꽃잎 길이와 너비 두 개뿐이라면 가로축에 길이, 세로축에 너비를 놓고 꽃마다 점을 하나씩 찍을 수 있겠죠. 품종이 같은 꽃들은 크기가 비슷하니 점들이 품종별로 무리를 지을 거예요. 특성이 네 개면 축이 네 개라 그림으로 그릴 수는 없지만 생각은 똑같습니다. "비슷한 샘플은 가까이 있다", 이 한 문장이 1.5절에서 만들 첫 모델의 전부예요.

1.3 머신러닝의 세 갈래

머신러닝은 **무엇을 가지고 배우는가**에 따라 크게 세 갈래로 나뉩니다. 학교 공부에 빗대면 이해가 쉬워요.

지도 학습: 정답이 있는 문제집으로 배우기

정답이 달린 문제집을 풀고 채점하면서 배우는 방식입니다. 스팸 예제도, 붓꽃도 샘플마다 정답(레이블)이 붙어 있었죠. 이렇게 **입력과 정답의 짝**으로 배우는 것을 **지도 학습(supervised learning)** 이라고 해요. 선생님(정답)이 지도해 준다는 뜻입니다. 실무에서 쓰는 머신러닝의 대부분이 여기에 속하고, 이 책도 대부분 지도 학습을 다룹니다.

지도 학습은 맞히려는 정답의 종류에 따라 다시 둘로 나뉩니다.

```
import numpy as np
from sklearn.datasets import load_diabetes, load_iris

분류용 = load_iris()      # 꽃의 품종 맞히기
회귀용 = load_diabetes()  # 1년 뒤 당뇨 진행 정도(숫자) 맞히기

print("분류 레이블 예:", 분류용.target[[0, 1, 50, 51, 100]])
print("분류 레이블 종류:", np.unique(분류용.target))
print("회귀 레이블 예:", 회귀용.target[:5])
print(f"회귀 레이블은 {len(np.unique(회귀용.target))}가지 값, "
      f"범위 {회귀용.target.min():.0f}~{회귀용.target.max():.0f}")
```

```
분류 레이블 예: [0 0 1 1 2]
분류 레이블 종류: [0 1 2]
회귀 레이블 예: [151.  75. 141. 206. 135.]
회귀 레이블은 214가지 값, 범위 25~346
```

- **분류(classification)**: 정답이 **몇 개의 정해진 종류 중 하나**입니다. 붓꽃 레이블은 0, 1, 2 세 가지뿐이에요. 스팸인가 아닌가(두 종류), 사진 속 숫자가 0~9 중 무엇인가(열 종류)도 분류입니다.
- **회귀(regression)**: 정답이 **연속된 숫자**입니다. 두 번째 데이터는 당뇨 환자 442명의 검사 수치로 1년 뒤 병의 진행 정도를 맞히는 문제인데, 정답이 25에서 346 사이의 숫자로 214가지나 돼요. 이 데이터는 에프론(Efron) 등이 2004년 논문 "Least Angle Regression"에 공개한 것을 scikit-learn이 담아 둔 거예요. 집값, 내일 기온, 배달 도착 시간을 예측하는 것도 회귀입니다.

헷갈릴 때는 "답을 **고르는가**, **재는가**"를 물어보세요. 고르면 분류, 재면 회귀입니다. 이름 때문에 회귀를 어렵게 느끼는 분이 많은데, 그냥 "숫자 맞히기"라고 생각해도 충분해요.

비지도 학습: 정답 없이 스스로 묶어 보기

정답지 없이 문제들만 잔뜩 받았다고 해 봅시다. 채점은 못 하지만 "이 문제들은 비슷한 유형이네" 하고 묶어 볼 수는 있겠죠. 레이블 없이 데이터 속 구조를 찾는 것을 **비지도 학습(unsupervised learning)** 이라고 합니다. 붓꽃에서 품종 정보를 감추고 특성만 준 뒤 "비슷한 것끼리 세 묶음으로 나눠 봐"라고 시켜 보겠습니다.

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)

# 레이블 y는 주지 않고 특성 X만 주면서 "비슷한 것끼리 세 묶음으로" 나누라고 한다
묶음 = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(X)

# 다 나눈 뒤에야 실제 품종과 맞춰 본다
for 번호 in range(3):
    품종별 = np.bincount(y[묶음 == 번호], minlength=3)
```

```
    print(f"묶음 {번호}: setosa {품종별[0]:2d}, versicolor {품종별[1]:2d}, virginica {품종
    ↪ 별[2]:2d}")
```

```
묶음 0: setosa  0, versicolor 48, virginica 14
묶음 1: setosa 50, versicolor  0, virginica  0
묶음 2: setosa  0, versicolor  2, virginica 36
```

KMeans는 3장에서 자세히 다룰 군집(clustering) 방법인데, 여기서는 결과만 보세요. 품종을 한 번도 알려 주지 않았는데 묶음 1에 setosa 50송이가 정확히 모였습니다. setosa는 꽃잎이 유난히 작아서 다른 두 품종과 뚝 떨어져 있거든요. 반면 versicolor와 virginica는 크기가 겹치는 꽃이 있어서 묶음 0과 2에 조금씩 섞였습니다(묶음 0에 virginica 14송이, 묶음 2에 versicolor 2송이).

비지도 학습은 정답을 모르니 "맞혔다"고 말하기 어렵다는 약점이 있지만, 레이블을 붙이는 데 드는 비용이 없다는 큰 장점이 있습니다. 고객을 구매 패턴으로 묶어 보거나, 평소와 다른 이상한 거래를 찾거나, 수백 개의 특성을 몇 개로 줄여 그림으로 그려 볼 때 씁니다. 15장의 오토인코더처럼 딥러닝에서도 중요한 역할을 해요.

강화 학습: 해 보고 보상으로 배우기

마지막 갈래는 정답 대신 **보상**으로 배우는 방식입니다. 자전거 타기를 생각해 보세요. 누가 "지금 핸들을 3도 왼쪽으로"라고 정답을 알려 주지 않습니다. 이렇게 저렇게 해 보다가 넘어지면(나쁜 보상) 그 행동을 덜 하고, 잘 나아가면(좋은 보상) 그 행동을 더 하면서 익히죠. 이것이 **강화 학습**(reinforcement learning) 입니다.

가장 단순한 강화 학습 문제로, 당첨 확률이 서로 다른 뽑기 기계 세 대가 있다고 해 봅시다. 어느 기계가 좋은지는 모르고, 1000번을 누를 수 있어요. 당첨을 최대한 많이 받으려면 어떻게 해야 할까요?

```
import random

당첨_확률 = [0.3, 0.5, 0.7]   # 기계마다 다른 당첨 확률 (에이전트는 모른다)
탐험_비율 = 0.1               # 열 번에 한 번은 아무 기계나 눌러 본다
rng = random.Random(0)

누른_횟수 = [0, 0, 0]
받은_보상 = [0, 0, 0]
for 회차 in range(1000):
```

```
    if rng.random() < 탐험_비율 or 0 in 누른_횟수:
        기계 = rng.randrange(3)                                  # 탐험
    else:
        기계 = max(range(3), key=lambda i: 받은_보상[i] / 누른_횟수[i])  # 활용
    보상 = 1 if rng.random() < 당첨_확률[기계] else 0
    누른_횟수[기계] += 1
    받은_보상[기계] += 보상

for i in range(3):
    추정 = 받은_보상[i] / 누른_횟수[i]
    print(f"기계 {i}: {누른_횟수[i]:4d}번 누름, 추정 당첨률 {추정:.2f} (실제 {당첨_확률[i]})")
print("1000번 중 당첨:", sum(받은_보상))
```

```
기계 0:   33번 누름, 추정 당첨률 0.21 (실제 0.3)
기계 1:   82번 누름, 추정 당첨률 0.45 (실제 0.5)
기계 2:  885번 누름, 추정 당첨률 0.71 (실제 0.7)
1000번 중 당첨: 672
```

전략은 간단합니다. 대부분은 **지금까지 평균 당첨률이 가장 높았던 기계**를 누르고(활용), 열 번에 한 번쯤은 일부러 아무 기계나 눌러 봅니다(탐험). 결과를 보면 처음에는 몰랐던 2번 기계가 가장 좋다는 것을 알아내서 1000번 중 885번을 그 기계에 썼고, 추정한 당첨률(0.71)도 실제 값(0.7)과 거의 같아요. 덕분에 당첨을 672번 받았습니다. 세 기계를 똑같이 나눠 눌렀다면 평균 500번 정도였을 거예요.

탐험을 왜 할까요? 처음 몇 번 운 나쁘게 꽝이 나온 기계를 영영 안 누르게 되는 걸 막기 위해서입니다. "아는 것을 쓸 것인가, 모르는 것을 더 알아볼 것인가"는 강화 학습의 영원한 고민이에요. 바둑을 두는 AI, 로봇 제어, 대화형 AI를 사람 취향에 맞게 다듬는 과정에 강화 학습이 쓰입니다. 다만 이 책에서는 이 정도 맛보기에서 멈추고, 지도 학습과 비지도 학습에 집중합니다.

갈래	무엇으로 배우나	대표 문제	이 책에서
지도 학습 - 분류	입력 + 정답(종류)	스팸, 품종, 사진 속 물체	1, 3, 6, 9, 10장 등
지도 학습 - 회귀	입력 + 정답(숫자)	집값, 수요 예측	1, 3, 5장 등

갈래	무엇으로 배우나	대표 문제	이 책에서
비지도 학습	입력만	고객 묶기, 이상 탐지, 차원 줄이기	3, 15장
강화 학습	행동과 보상	게임, 로봇, 추천	이 절의 맛보기

1.4 모델, 학습, 예측

모델은 함수입니다

이제 머신러닝의 나머지 세 낱말을 맞춰 볼 차례입니다. **모델(model)** 은 거창한 것이 아니라 **입력(특성)을 받아 출력(예측)을 돌려주는 함수**예요. 1권 9장에서 만든 함수와 같습니다. 다만 그 함수 안의 일부 숫자를 사람이 정하지 않고 데이터에서 찾는다는 점이 달라요. 이렇게 데이터에서 찾아내는 숫자를 **파라미터(parameter)** 라고 하고, 좋은 파라미터를 찾는 과정을 **학습(learning)** 또는 **훈련(training)** 이라고 합니다. 학습이 끝난 모델에 새 입력을 넣어 답을 얻는 것이 **예측(prediction)** 이에요.

말로만 하면 막연하니 아주 작은 모델 하나를 학습시켜 보겠습니다. versicolor와 virginica 두 품종을 **꽃잎 길이 하나만으로** 가르는 모델이에요. 모델의 모양은 "꽃잎 길이가 기준보다 길면 virginica, 아니면 versicolor"로 정해 두고, 기준값은 데이터에서 찾게 합니다.

```
import numpy as np
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)
고름 = y > 0                        # versicolor(1)와 virginica(2)만
꽃잎_길이 = X[고름, 2]               # 특성 하나: 꽃잎 길이(cm)
정답 = y[고름]

최고_기준, 최고_정확도 = None, 0.0
for 기준 in np.unique(꽃잎_길이):                # 후보 기준을 하나씩 시험
    예측 = np.where(꽃잎_길이 > 기준, 2, 1)       # 기준보다 길면 virginica
    정확도 = (예측 == 정답).mean()
    if 정확도 > 최고_정확도:
        최고_기준, 최고_정확도 = 기준, 정확도
```

```
print("시험한 기준 수:", len(np.unique(꽃잎_길이)))
print(f"배운 규칙: 꽃잎 길이 > {최고_기준} 이면 virginica")
print(f"학습 데이터에서 정확도: {최고_정확도:.2f}")
```

```
시험한 기준 수: 34
배운 규칙: 꽃잎 길이 > 4.7 이면 virginica
학습 데이터에서 정확도: 0.93
```

이 짧은 코드에 학습의 뼈대가 모두 들어 있습니다.

1. **모델의 모양을 정한다.** "길이 > 기준이면 virginica". 기준이라는 빈칸 하나가 있는 함수예요.
2. **얼마나 좋은지 잴 잣대를 정한다.** 여기서는 정확도(맞힌 비율)입니다.
3. **잣대가 가장 좋아지는 빈칸 값을 찾는다.** 꽃잎 길이로 나올 수 있는 값 34개를 모두 기준으로 시험해 보고, 정확도가 가장 높은 4.7을 골랐습니다.

결과로 "꽃잎 길이 > 4.7이면 virginica"라는 규칙을 얻었어요. 이 규칙은 사람이 적은 게 아니라 **데이터가 정해 준** 것입니다. 들어가며의 스팸 예제에서 단어별 횟수를 센 것도 같은 일이었어요. 단어마다 점수라는 파라미터를 데이터로 정한 거죠.

빈칸이 수백만 개가 되면

딥러닝 모델도 뼈대는 똑같습니다. 다른 건 빈칸의 수예요. 위 모델은 빈칸이 하나라 후보를 전부 시험해 볼 수 있었지만, 신경망은 빈칸(파라미터)이 수백만에서 수십억 개라 그럴 수가 없습니다. 그래서 "지금 값에서 어느 쪽으로 조금 고치면 잣대가 좋아질까"를 계산해 가며 조금씩 고쳐 나가요. 그 방법이 5장의 **경사하강법**이고, "어느 쪽으로"를 계산해 주는 것이 같은 5장에서 만날 **자동 미분**입니다. 잣대 쪽은 보통 거꾸로 "얼마나 틀렸는가"를 재는데, 이를 **손실(loss)** 이라 부르고 손실을 줄이는 방향으로 학습해요. 6장에서 본격적으로 만납니다.

하나 더 구분해 둘 것이 있습니다. 위 모델에서 "꽃잎 길이 특성 하나만 쓴다"는 결정은 데이터가 아니라 **사람**이 내렸죠. 이렇게 학습 전에 사람이 정하는 설정을 **하이퍼파라미터(hyperparameter)** 라고 부릅니다. 파라미터는 학습이 찾고, 하이퍼파라미터는 사람이 고른다고 기억하세요. 다음 절의 최근접 이웃에서 "이웃을 몇 명 볼까"가 바로 하이퍼파라미터입니다.

Q

Q. 정확도가 0.93이면 7%는 틀린다는 건데, 이게 좋은 모델인가요? 그건 비교할 대상이 있어야 말할 수 있습니다. 두 품종이 50송이씩이니 아무렇게나 찍으면 0.5쯤 나오겠죠. 그에 비하면 특성 하나로 0.93은 꽤 좋습니다. 그런데 이 0.93은 **기준을 고를 때 쓴 바로 그 데이터**에서 잰 값이라 실제보다 좋게 나왔을 수 있어요. 시험 문제를 미리 보고 공부한 학생의 점수와 같습니다. 그래서 다음 절부터는 데이터 일부를 숨겨 두고 그걸로 따로 잽니다.

1.5 첫 모델: 최근접 이웃 분류

끼리끼리 모인다

새 동네로 이사 갔는데 근처 식당이 맛집인지 궁금하다고 해 봅시다. 가장 쉬운 방법은 그 식당과 **비슷한** 식당들이 어땠는지 보는 거예요. 같은 골목, 비슷한 가격, 비슷한 메뉴의 가게들이 다 맛있었다면 이 집도 그럴 가능성이 높겠죠.

최근접 이웃(k-nearest neighbors, k-NN) 분류가 바로 이 생각입니다. 정답을 모르는 샘플이 오면, 학습 데이터 중 그와 **가장 가까운 k개**를 찾아 그들의 레이블로 **다수결 투표**를 해요. 1.2절의 "샘플은 공간 속의 점"이라는 그림을 떠올리면, 새 점 주변에 어떤 색 점이 많은지 보는 겁니다. 이 단순한 방법이 생각보다 잘 맞아서 지금도 실무에서 쓰여요.

거리 재기

먼저 "가깝다"를 숫자로 정해야 합니다. 가장 흔한 것이 학교에서 배운 두 점 사이의 거리, **유클리드 거리**예요. 특성마다 차이를 구해 제곱하고, 모두 더한 뒤 제곱근을 씌웁니다. 특성이 두 개면 피타고라스 정리 그대로고, 네 개면 같은 계산을 네 자리로 늘리면 돼요.

품종을 모르는 꽃 한 송이(예제용으로 지어낸 크기)와 붓꽃 150송이 사이의 거리를 한꺼번에 재 보겠습니다. 1권 20장의 NumPy를 씁니다.

```
import numpy as np
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)
새_꽃 = np.array([6.0, 2.9, 4.6, 1.5])   # 정답을 모르는 꽃 한 송이 (예제용)
```

```
차이 = X - 새_꽃                          # (150, 4) - (4,) → 브로드캐스팅
거리 = np.sqrt((차이 ** 2).sum(axis=1))  # 꽃마다 거리 하나 → (150,)
print("거리 모양:", 거리.shape)

가까운_순 = np.argsort(거리)[:5]          # 가까운 다섯 송이의 번호
for 번호 in 가까운_순:
    print(f"{번호:3d}번 꽃  거리 {거리[번호]:.3f}  레이블 {y[번호]}")
print("다섯 이웃의 표:", np.bincount(y[가까운_순], minlength=3))
```

```
거리 모양: (150,)
 78번 꽃  거리 0.100  레이블 1
 91번 꽃  거리 0.173  레이블 1
 63번 꽃  거리 0.173  레이블 1
 73번 꽃  거리 0.346  레이블 1
138번 꽃  거리 0.374  레이블 2
다섯 이웃의 표: [0 4 1]
```

`X - 새_꽃`은 `(150, 4)` 표에서 `(4,)`짜리 한 줄을 뺍니다. 모양이 다른데 계산이 되는 건, NumPy가 한 줄짜리 `새_꽃`을 150행 모두에 똑같이 펼쳐서 빼 주기 때문이에요. 이 규칙을 **브로드캐스팅**이라고 하고, 4장에서 자세히 다룹니다. 그다음 `axis=1`(특성 방향)로 더해서 꽃마다 거리 하나씩, 150개를 얻었어요.

`np.argsort(거리)`는 거리를 작은 순서로 정렬했을 때의 **번호**를 돌려줍니다. 앞의 다섯 개가 가장 가까운 이웃 다섯 송이예요. 레이블을 보면 1(versicolor)이 넷, 2(virginica)가 하나입니다. `np.bincount`로 품종별 표를 세면 `[0 4 1]`, 다수결로 이 꽃은 versicolor라고 판정하면 되겠네요.

시험 문제는 따로 숨겨 둡니다

모델이 잘 맞히는지 알려면 정답을 아는 꽃으로 시험해 봐야 합니다. 그런데 1.4절의 Q&A에서 봤듯이, 배울 때 쓴 데이터로 시험하면 점수가 부풀려져요. 특히 최근접 이웃은 학습 데이터를 통째로 기억하니, 학습 데이터 속 꽃으로 시험하면 자기 자신이 가장 가까운 이웃이라 거의 다 맞힙니다. 그래서 데이터를 **학습용**과 **테스트용**으로 나눠, 테스트용은 학습이 끝날 때까지 꺼내 보지 않아요.

```
import numpy as np
from sklearn.datasets import load_iris
```

```
X, y = load_iris(return_X_y=True)

rng = np.random.default_rng(42)
섞은_번호 = rng.permutation(len(X))    # 0~149를 무작위 순서로
테스트_번호 = 섞은_번호[:30]           # 30송이는 시험용으로 숨겨 두고
학습_번호 = 섞은_번호[30:]             # 나머지 120송이로 배운다

X_학습, y_학습 = X[학습_번호], y[학습_번호]
X_테스트, y_테스트 = X[테스트_번호], y[테스트_번호]
print("학습:", X_학습.shape, "품종별", np.bincount(y_학습))
print("테스트:", X_테스트.shape, "품종별", np.bincount(y_테스트))
```

```
학습: (120, 4) 품종별 [40 41 39]
테스트: (30, 4) 품종별 [10  9 11]
```

150송이의 번호를 무작위로 섞은 뒤 앞의 30송이를 테스트용으로, 나머지 120송이를 학습용으로 나눴습니다. 원래 데이터는 품종 순서로 정렬되어 있어서(0번부터 49번까지가 setosa) 섞지 않고 앞에서부터 자르면 테스트에 setosa만 들어가는 사고가 납니다. 섞은 덕분에 양쪽 모두 품종이 고르게 들어갔어요. 씨앗(42)을 고정했으니 몇 번을 실행해도 같은 30송이가 골라집니다.

데이터를 나누는 일은 생각보다 함정이 많습니다. 시간 순서가 있는 데이터를 섞어 버리면 미래로 과거를 맞히는 셈이 되고, 나누기 전에 전체 데이터로 무언가를 계산하면 테스트 데이터의 정보가 학습에 새어 들어가요. 이런 이야기는 2장 데이터 준비에서 자세히 합니다.

NumPy로 직접 만들기

이제 재료가 다 모였습니다. 테스트 꽃 30송이 **모두**에 대해 한꺼번에 이웃을 찾는 함수를 만들어 보겠습니다.

```
import numpy as np
from sklearn.datasets import load_iris


def 최근접_이웃_예측(X_학습: np.ndarray, y_학습: np.ndarray,
                    X_새: np.ndarray, k: int) -> np.ndarray:
```

```
    """새 샘플마다 가장 가까운 학습 샘플 k개의 다수결로 레이블을 정한다."""
    # (새 개수, 1, 특성) - (1, 학습 개수, 특성) → (새 개수, 학습 개수, 특성)
    차이 = X_새[:, np.newaxis, :] - X_학습[np.newaxis, :, :]
    거리 = np.sqrt((차이 ** 2).sum(axis=2))          # (새 개수, 학습 개수)
    이웃 = np.argsort(거리, axis=1, kind="stable")[:, :k]  # 가까운 k개의 번호
    이웃_레이블 = y_학습[이웃]                        # (새 개수, k)
    return np.array([np.bincount(줄).argmax() for 줄 in 이웃_레이블])


X, y = load_iris(return_X_y=True)
섞은_번호 = np.random.default_rng(42).permutation(len(X))
X_학습, y_학습 = X[섞은_번호[30:]], y[섞은_번호[30:]]
X_테스트, y_테스트 = X[섞은_번호[:30]], y[섞은_번호[:30]]

예측 = 최근접_이웃_예측(X_학습, y_학습, X_테스트, k=5)
print("예측:", 예측[:10])
print("정답:", y_테스트[:10])
print(f"테스트 정확도: {(예측 == y_테스트).mean():.3f} ({(예측 == y_테스트).sum()}/{len(y_테스
↪ 트)})")
```

```
예측: [2 1 2 0 2 2 1 2 2 0]
정답: [1 1 2 0 2 2 1 2 2 0]
테스트 정확도: 0.967 (29/30)
```

핵심은 첫 줄의 거리 계산입니다. 앞에서는 꽃 한 송이와 150송이의 거리를 쟀는데, 이번에는 테스트 30송이 각각과 학습 120송이 각각, 모두 30 × 120 = 3600쌍의 거리를 한 번에 잽니다. `X_새[:, np.newaxis, :]`는 `(30, 1, 4)`, `X_학습[np.newaxis, :, :]`는 `(1, 120, 4)` 모양이 되고, 둘을 빼면 브로드캐스팅으로 `(30, 120, 4)`가 돼요. 특성 방향(`axis=2`)으로 접으면 `(30, 120)` 거리 표가 남습니다. 행은 테스트 꽃, 열은 학습 꽃이에요.

그다음은 앞에서 손으로 한 것과 같습니다. 행마다 `argsort`로 가까운 순서를 구해 앞의 k개를 자르고, 그 이웃들의 레이블을 `bincount`로 세어 가장 많은 품종(`argmax`)을 고릅니다. `kind="stable"`은 거리가 똑같은 이웃이 있을 때 원래 순서를 지키라는 뜻으로, 결과가 실행마다 흔들리지 않게 해 줘요.

결과는 30송이 중 29송이, **정확도 0.967**입니다. 예측과 정답의 앞 열 개를 비교해 보면 맨 처음 꽃

하나를 virginica(2)라고 했는데 실제는 versicolor(1)였네요. 1.3절에서 봤듯이 이 두 품종은 크기가 겹치는 꽃이 있어서 헷갈릴 만합니다. 열 줄 남짓한 함수로 처음 보는 꽃의 품종을 97% 맞혔으니, 첫 모델치고 훌륭하죠.

scikit-learn 한 줄로

같은 일을 scikit-learn으로 해 보겠습니다. 직접 만든 함수와 예측이 **하나도 빠짐없이** 같은지도 확인해요.

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier


def 최근접_이웃_예측(X_학습: np.ndarray, y_학습: np.ndarray,
                    X_새: np.ndarray, k: int) -> np.ndarray:
    """새 샘플마다 가장 가까운 학습 샘플 k개의 다수결로 레이블을 정한다."""
    거리 = np.sqrt(((X_새[:, None, :] - X_학습[None, :, :]) ** 2).sum(axis=2))
    이웃 = np.argsort(거리, axis=1, kind="stable")[:, :k]
    return np.array([np.bincount(줄).argmax() for 줄 in y_학습[이웃]])


X, y = load_iris(return_X_y=True)
섞은_번호 = np.random.default_rng(42).permutation(len(X))
X_학습, y_학습 = X[섞은_번호[30:]], y[섞은_번호[30:]]
X_테스트, y_테스트 = X[섞은_번호[:30]], y[섞은_번호[:30]]

모델 = KNeighborsClassifier(n_neighbors=5)   # 1) 모델 고르기
모델.fit(X_학습, y_학습)                      # 2) 학습
예측 = 모델.predict(X_테스트)                 # 3) 예측
print(f"scikit-learn 정확도: {모델.score(X_테스트, y_테스트):.3f}")

직접 = 최근접_이웃_예측(X_학습, y_학습, X_테스트, k=5)
print("직접 짠 것과 예측이 모두 같은가:", np.array_equal(예측, 직접))
```

```
scikit-learn 정확도: 0.967
직접 짠 것과 예측이 모두 같은가: True
```

모델을 만들고, `fit`으로 학습하고, `predict`로 예측하는 세 줄이면 끝입니다. 정확도는 똑같이 0.967이고, 30송이의 예측이 직접 짠 함수와 모두 같다는 `True`가 나왔어요. scikit-learn 안에서도 결국 우리가 한 계산, 즉 거리를 재고 가까운 k개를 골라 투표하는 일이 일어나는 겁니다. 다만 데이터가 수십만 건으로 늘어도 빠르게 이웃을 찾는 영리한 방법과, 여러 가지 거리 계산 방식과, 꼼꼼한 오류 검사가 들어 있어요. 원리는 직접 짜 보며 이해하고, 실무에서는 검증된 라이브러리를 쓰는 것이 이 책의 방식입니다.

`모델.score(X, y)`는 예측하고 정답과 비교해 정확도를 내는 일을 한 번에 해 주는 편의 메서드입니다.

얼마나 확신하나요?

다수결이니 "5 대 0으로 이겼는지, 3 대 2로 겨우 이겼는지"도 알 수 있겠죠? `predict_proba`는 바로 그 투표 비율을 돌려줍니다. 이번에는 붓꽃 150송이 전체로 학습하고, 지어낸 꽃 세 송이를 물어보겠습니다.

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier

붓꽃 = load_iris()
모델 = KNeighborsClassifier(n_neighbors=5).fit(붓꽃.data, 붓꽃.target)

새_꽃들 = np.array([[5.0, 3.4, 1.5, 0.2],    # 예제용으로 지어낸 꽃 세 송이
                    [6.0, 2.9, 4.6, 1.5],
                    [6.3, 2.8, 5.0, 1.7]])
확률 = 모델.predict_proba(새_꽃들)            # 이웃 다섯 중 품종별 비율
for 꽃, 줄 in zip(새_꽃들, 확률):
    이름 = 붓꽃.target_names[줄.argmax()]
    print(f"{꽃} → {줄.round(2)}  {이름}")
```

```
[5.  3.4 1.5 0.2] → [1. 0. 0.]  setosa
[6.  2.9 4.6 1.5] → [0.  0.8 0.2]  versicolor
[6.3 2.8 5.  1.7] → [0.  0.2 0.8]  virginica
```

첫 번째 꽃은 다섯 이웃이 모두 setosa라 `[1. 0. 0.]`, 망설임 없는 답입니다. 두 번째 꽃은 1.5절 처음에 거리를 쟀던 그 꽃인데, 다섯 중 넷이 versicolor라 `[0.  0.8 0.2]`예요. 세 번째 꽃은 반대로 virginica 쪽이 0.8입니다. 같은 "정답"이라도 확신의 정도가 다르다는 정보는 쓸모가 많습니다. 확신이 낮은 샘플만 사람에게 다시 확인받거나, "80% 이상일 때만 자동 처리" 같은 규칙을 세울 수 있거든요. 6장에서 만들 신경망 분류기도 마지막에 이렇게 품종별 확률을 내놓습니다.

fit과 predict라는 약속

scikit-learn의 모든 모델은 이름이 달라도 쓰는 법이 같습니다. `fit(X, y)`로 배우고 `predict(X)`로 답하고 `score(X, y)`로 채점해요. 이 약속을 우리 모델에도 지켜 주면 어떻게 될까요? 1권 11장의 클래스로 감싸 보겠습니다.

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier


class 최근접이웃분류기:
    """scikit-learn과 같은 fit/predict 모양으로 감싼 최근접 이웃 분류기."""

    def __init__(self, n_neighbors: int = 5) -> None:
        self.n_neighbors = n_neighbors

    def fit(self, X: np.ndarray, y: np.ndarray) -> "최근접이웃분류기":
        # 최근접 이웃의 "학습"은 데이터를 그대로 외워 두는 것뿐이다
        self.X_, self.y_ = X.copy(), y.copy()
        return self

    def predict(self, X: np.ndarray) -> np.ndarray:
        거리 = np.sqrt(((X[:, None, :] - self.X_[None, :, :]) ** 2).sum(axis=2))
        이웃 = np.argsort(거리, axis=1, kind="stable")[:, :self.n_neighbors]
        return np.array([np.bincount(줄).argmax() for 줄 in self.y_[이웃]])
```

```
    def score(self, X: np.ndarray, y: np.ndarray) -> float:
        return float((self.predict(X) == y).mean())


X, y = load_iris(return_X_y=True)
섞은_번호 = np.random.default_rng(42).permutation(len(X))
X_학습, y_학습 = X[섞은_번호[30:]], y[섞은_번호[30:]]
X_테스트, y_테스트 = X[섞은_번호[:30]], y[섞은_번호[:30]]

# 사용법이 같으니 같은 반복문에 넣어 돌릴 수 있다
for 모델 in [최근접이웃분류기(n_neighbors=3), KNeighborsClassifier(n_neighbors=3)]:
    모델.fit(X_학습, y_학습)
    print(f"{type(모델).__name__:22s} 정확도 {모델.score(X_테스트, y_테스트):.3f}")
```

```
최근접이웃분류기               정확도 0.967
KNeighborsClassifier   정확도 0.967
```

직접 만든 `최근접이웃분류기`와 scikit-learn의 `KNeighborsClassifier`를 **같은 반복문**에 넣고 돌렸습니다. 두 클래스는 아무 관계가 없지만 `fit`, `score`라는 같은 이름의 메서드를 가졌으니 반복문 입장에서는 구별할 필요가 없어요. 1권 13장의 **덕 타이핑**("오리처럼 걷고 오리처럼 울면 오리")이 여기서 빛을 발합니다. 이 약속 덕분에 scikit-learn에서는 모델 이름 하나만 바꿔 가며 여러 모델을 비교할 수 있어요. 3장에서 수많은 모델을 이 방식으로 갈아 끼웁니다.

`fit` 안을 보세요. 최근접 이웃의 학습은 데이터를 **복사해 두는 것**이 전부입니다. 진짜 계산은 예측할 때 일어나죠. 그래서 최근접 이웃을 "게으른 학습"이라고도 부릅니다. 학습은 순식간이지만 예측할 때마다 모든 학습 데이터와 거리를 재야 하니, 데이터가 커지면 예측이 느려져요. 반대로 신경망은 학습에 오랜 시간을 들이는 대신 예측은 빠릅니다. 이름 끝의 밑줄(`X_`, `y_`)은 "학습으로 생긴 값"이라는 scikit-learn의 관례를 따랐습니다.

Q

Q. k는 몇으로 해야 하나요? 정해진 답은 없고 데이터마다 다릅니다. k가 하이퍼파라미터인 이유예요. 다만 두 가지 요령이 있습니다. 두 품종을 가를 때 k를 짝수로 하면 2 대 2처럼 동점이 날 수 있으니 홀수를 많이 쓰고, 여러 k를 시험해서 **테스트 데이터가 아닌 별도의 검증 데이터**로 가장 좋은 값을 고릅니다. 왜 테스트로 고르면 안 되는지는 1.13절

실습에서 직접 해 봐요. k가 너무 작거나 크면 무슨 일이 생기는지는 바로 1.7절에서 봅니다.

1.6 숫자를 맞히는 이웃: 회귀

최근접 이웃은 회귀에도 그대로 쓸 수 있습니다. 투표 대신 이웃들의 레이블 **평균**을 답하면 되거든요. 이웃 동네 아파트 다섯 채의 시세 평균으로 우리 집 시세를 어림하는 것과 같습니다. 1.3절의 당뇨 데이터로 해 볼게요.

이번에는 데이터를 나눌 때 scikit-learn의 `train_test_split`을 썼습니다. 1.5절에서 손으로 한 "섞어서 자르기"를 한 줄로 해 주는 함수예요.

```
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor

X, y = load_diabetes(return_X_y=True)
X_학습, X_테스트, y_학습, y_테스트 = train_test_split(X, y, test_size=0.25, random_state=0)

# 비교 기준: 누구에게나 학습 데이터의 평균을 답한다
평균_답 = np.full(len(y_테스트), y_학습.mean())
print(f"평균으로 답하기: 평균 오차 {np.abs(평균_답 - y_테스트).mean():.1f}")

# 최근접 이웃 회귀: 가까운 k명의 레이블 평균을 답한다
for k in [1, 10, 30]:
    예측 = KNeighborsRegressor(n_neighbors=k).fit(X_학습, y_학습).predict(X_테스트)
    print(f"이웃 {k:2d}명의 평균: 평균 오차 {np.abs(예측 - y_테스트).mean():.1f}")
print("테스트 환자 셋의 정답:", y_테스트[:3], " k=30 예측:", 예측[:3].round(1))
```

```
평균으로 답하기: 평균 오차 58.3
이웃  1명의 평균: 평균 오차 64.9
이웃 10명의 평균: 평균 오차 46.5
이웃 30명의 평균: 평균 오차 43.1
테스트 환자 셋의 정답: [321. 215. 127.]  k=30 예측: [228.  209.6 146. ]
```

회귀에서는 맞았다, 틀렸다가 아니라 **얼마나 빗나갔는가**를 잽니다. 여기서는 예측과 정답의 차이를 절댓값으로 바꿔 평균 낸 "평균 오차"를 썼어요(3장에서 MAE라는 이름으로 다시 만납니다). 작을수록 좋습니다.

결과에서 세 가지를 보세요.

- **비교 기준부터 잽니다.** 환자가 누구든 학습 데이터의 평균을 답하는 "모델"도 평균 58.3만큼 빗나갑니다. 아무것도 배우지 않은 답이니, 이보다 못하면 모델이라 부르기 어려워요.
- **이웃 한 명만 보면 오히려 나빠집니다.** k=1일 때 평균 오차가 64.9로, 평균으로 답하기(58.3)보다도 못합니다. 가장 가까운 환자 한 명의 값을 그대로 베끼면 그 환자의 우연한 사정까지 따라 하게 되거든요.
- **이웃을 늘리면 좋아집니다.** 10명이면 46.5, 30명이면 43.1로 기준보다 확실히 낫습니다. 다만 마지막 줄의 예측을 보면 정답 321인 환자를 228로 예측하는 등 여전히 차이가 커요. 이 데이터의 특성 10개만으로는 병의 진행을 정확히 맞히기 어렵다는 뜻입니다. 모델이 할 수 있는 것은 데이터가 담고 있는 만큼까지예요.

k=1이 왜 나쁜지, 그리고 k를 무작정 키우면 어떻게 되는지가 다음 절의 주제입니다.

1.7 일반화와 과적합: 외우기와 이해하기

기출문제만 외운 학생

시험 공부를 하는 두 학생이 있습니다. 한 명은 기출문제의 답을 통째로 외웠고, 다른 한 명은 기출문제를 풀면서 원리를 이해했어요. 기출문제를 그대로 내면 둘 다 만점입니다. 그런데 숫자만 바꾼 새 문제를 내면? 외운 학생은 손도 못 대고, 이해한 학생은 풀어냅니다.

머신러닝이 바라는 것은 두 번째 학생입니다. 학습 데이터를 잘 맞히는 게 목적이 아니라, **처음 보는 데이터**를 잘 맞히는 게 목적이에요. 이 능력을 **일반화**(generalization) 라고 합니다. 극단적인 첫 번째 학생을 실제로 만들어 보겠습니다.

```
import numpy as np
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)
섞은_번호 = np.random.default_rng(42).permutation(len(X))
X_학습, y_학습 = X[섞은_번호[30:]], y[섞은_번호[30:]]
X_테스트, y_테스트 = X[섞은_번호[:30]], y[섞은_번호[:30]]
```

```
# 외우기 모델: 특성 네 개를 통째로 열쇠로 삼아 정답을 적어 둔다
암기장 = {tuple(특성): 정답 for 특성, 정답 in zip(X_학습, y_학습)}


def 외워서_답하기(특성: np.ndarray) -> int | None:
    return 암기장.get(tuple(특성))      # 처음 보는 꽃이면 None


학습_맞음 = sum(외워서_답하기(x) == t for x, t in zip(X_학습, y_학습))
테스트_답 = [외워서_답하기(x) for x in X_테스트]
테스트_맞음 = sum(답 == t for 답, t in zip(테스트_답, y_테스트))
print(f"학습 데이터: {학습_맞음}/{len(y_학습)} 맞힘")
print(f"테스트 데이터: {테스트_맞음}/{len(y_테스트)} 맞힘, "
      f"모른다(None): {테스트_답.count(None)}송이")
```

```
학습 데이터: 120/120 맞힘
테스트 데이터: 0/30 맞힘, 모른다(None): 30송이
```

암기장은 학습 데이터의 특성 네 개를 통째로 열쇠로 삼아 정답을 적어 둔 딕셔너리입니다(1권 7장). 학습 데이터에서는 120송이를 모두 맞혀 완벽해 보이지만, 테스트 꽃 30송이는 **한 송이도** 맞히지 못했어요. 학습 데이터와 소수점까지 똑같은 꽃이 하나도 없어서 30송이 모두 "모른다"는 답이 나왔습니다. 학습 정확도 100%가 쓸모없는 모델의 성적표일 수도 있다는 것, 그래서 반드시 따로 떼어 둔 데이터로 재야 한다는 것을 보여 주는 예예요.

k가 작으면 외우고, 너무 크면 뭉갭니다

최근접 이웃은 외우기 모델보다 훨씬 낫지만, k가 1이면 외우기에 가까워집니다. 이웃 한 명의 답을 그대로 베끼니까요. 이번에는 붓꽃보다 어려운 데이터로 확인해 보겠습니다. scikit-learn의 `make_moons`는 초승달 두 개가 맞물린 모양으로 점을 흩뿌려 주는 함수인데, 잡음(`noise`)을 섞어서 두 달의 경계가 꽤 뒤섞이게 만들었어요. 예제용으로 만든 데이터입니다.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

```
# 초승달 두 개가 맞물린 모양의 점들 (예제용으로 만든 데이터), 잡음을 섞었다
X, y = make_moons(n_samples=400, noise=0.35, random_state=0)
X_학습, X_테스트, y_학습, y_테스트 = train_test_split(
    X, y, test_size=0.5, random_state=0)

print("  k   학습 정확도   테스트 정확도")
for k in [1, 3, 7, 15, 31, 75, 199]:
    모델 = KNeighborsClassifier(n_neighbors=k).fit(X_학습, y_학습)
    print(f"{k:3d}      {모델.score(X_학습, y_학습):.3f}        {모델.score(X_테스트, y_테스
    ↪ 트):.3f}")
```

```
  k   학습 정확도   테스트 정확도
  1      1.000         0.845
  3      0.895         0.865
  7      0.875         0.880
 15      0.870         0.880
 31      0.875         0.890
 75      0.855         0.860
199      0.530         0.470
```

표를 위에서부터 읽어 보세요.

- **k=1**: 학습 정확도가 1.000입니다. 학습 데이터의 각 점은 자기 자신이 가장 가까운 이웃이니 당연하죠. 그런데 테스트는 0.845로 뚝 떨어집니다. 잡음으로 엉뚱한 자리에 찍힌 점까지 그대로 외워서, 그 근처에 온 새 점을 틀리게 맞히는 거예요. 이렇게 학습 데이터에만 지나치게 맞춰져 새 데이터에서 성능이 떨어지는 것을 **과적합(overfitting)** 이라고 합니다.
- **k=7~31**: 학습 정확도는 0.87~0.88로 내려갔지만 테스트 정확도는 0.88~0.89로 오히려 올라갔습니다. 여러 이웃의 의견을 들으니 잡음 몇 개에 휘둘리지 않아요. 학습과 테스트 점수가 비슷하다는 것도 좋은 신호입니다. 이 구간에서 k=31이 0.890으로 가장 좋았어요.
- **k=199**: 학습 데이터가 200개인데 이웃을 199명 보면 사실상 전체 다수결이라, 어디에 있든 거의 같은 답을 냅니다. 학습 0.530, 테스트 0.470으로 둘 다 동전 던지기 수준이에요. 데이터의 모양을 너무 뭉개서 학습 데이터조차 못 맞히는 것을 **과소적합(underfitting)** 이라고 합니다.

	과소적합	알맞음	과적합
비유	공부를 안 함	원리를 이해함	답만 외움
학습 성능	낮음	높음	아주 높음
테스트 성능	낮음	높음	학습보다 확 낮음
최근접 이웃에서	k가 너무 큼	적당한 k	k가 너무 작음

경계선을 그려 보면

k가 1일 때와 31일 때 모델이 평면을 어떻게 나눴는지 그림으로 보면 더 분명해집니다. 평면을 격자로 잘게 나누고 칸마다 모델에게 "여기는 어느 달이야?"라고 물어서, 답을 #과 .로 찍어 보겠습니다.

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X, y = make_moons(n_samples=400, noise=0.35, random_state=0)   # 예제용 데이터
X_학습, X_테스트, y_학습, y_테스트 = train_test_split(
    X, y, test_size=0.5, random_state=0)

# 평면을 가로 48칸 × 세로 14칸 격자로 나누고, 칸마다 모델의 답을 찍는다
가로 = np.linspace(-1.6, 2.6, 48)
세로 = np.linspace(1.6, -1.0, 14)
격자 = np.array([[x, 높이] for 높이 in 세로 for x in 가로])

for k in [1, 31]:
    모델 = KNeighborsClassifier(n_neighbors=k).fit(X_학습, y_학습)
    지도 = 모델.predict(격자).reshape(len(세로), len(가로))
    print(f"[k={k}]  # = 0번 달, . = 1번 달")
    for 줄 in 지도:
        print("".join("#" if 값 == 0 else "." for 값 in 줄))
```

```
[k=1]  # = 0번 달, . = 1번 달
####################################..........
####################################..........
###################.################..........
####################################..........
####################################..........
#############..##..##...##.######............
#############.######.###.#########............
################.......##.#########...##.......
###########............#...#..#.#.............
#########..................##.................
########.................##...##..............
######......................................
#####......................................
##.......................................
[k=31]  # = 0번 달, . = 1번 달
########################################......
####################################........
####################################..........
####################################..........
####################################..........
####################################..........
#################.....#########..............
##############............###.................
#############..............................
############...............................
##########.................................
#########..................................
########...................................
######.....................................
```

k=1의 지도를 보면 # 바다 한가운데 . 섬이 여기저기 떠 있고, 경계선이 들쭉날쭉합니다. 잡음으로 엉뚱한 자리에 찍힌 학습 점 하나하나에 자기 영역을 만들어 준 거예요. 이게 외우기가 그림으로 나타난 모습입니다. k=31의 지도는 경계가 매끄러운 곡선 하나로 정리됐어요. 초승달 두 개가 맞물린 원래 모양을 더 잘 따라갑니다. 과적합한 모델은 경계가 복잡하고, 알맞은 모델은 경계가 데이터의 큰 흐름을 따른다고 기억해 두세요. 8장에서 신경망의 과적합을 볼 때도 같은 그림이 나옵니다.

모델 성능을 볼 때 **학습 점수와 테스트 점수를 나란히** 보는 버릇을 들이세요. 둘 다 낮으면 과소적합, 학습만 높고 테스트가 낮으면 과적합입니다. 신경망은 파라미터가 많아서 학습 데이터를 통째로 외울 힘이 있고, 그래서 과적합과의 싸움이 딥러닝의 큰 숙제예요. 학습 곡선으로 과적합을 진단하고 막는 방법은 8장에서 본격적으로 다룹니다.

데이터가 많을수록 외우기 어렵습니다

과적합을 막는 가장 확실한 방법은 무엇일까요? 문제를 더 많이 풀어 보는 겁니다. 기출문제가 다섯 개뿐이면 외우는 게 쉽지만, 오천 개면 외우기보다 원리를 이해하는 편이 빠르죠.

```
from sklearn.datasets import make_moons
from sklearn.neighbors import KNeighborsClassifier

# 테스트용 1000개는 고정해 두고, 학습 데이터 양만 늘려 본다 (예제용 데이터)
X_테스트, y_테스트 = make_moons(n_samples=1000, noise=0.35, random_state=1)
X_전체, y_전체 = make_moons(n_samples=3000, noise=0.35, random_state=2)

print("학습 개수   k=1 테스트   k=15 테스트")
for 개수 in [20, 50, 200, 1000, 3000]:
    점수 = []
    for k in [1, 15]:
        k = min(k, 개수)
        모델 = KNeighborsClassifier(n_neighbors=k).fit(X_전체[:개수], y_전체[:개수])
        점수.append(모델.score(X_테스트, y_테스트))
    print(f"{개수:6d}       {점수[0]:.3f}        {점수[1]:.3f}")
```

```
학습 개수   k=1 테스트   k=15 테스트
    20       0.787        0.717
    50       0.822        0.809
   200       0.823        0.863
  1000       0.824        0.862
  3000       0.812        0.861
```

테스트용 1000개는 고정하고 학습 데이터만 20개에서 3000개까지 늘렸습니다. k=15 쪽을 보면 학습 데이터 20개(이때는 이웃이 15명이면 거의 전체 투표라 0.717)에서 200개일 때 0.863까지 올라갑니다. 데이터가 늘면 성능이 오르는 것, 머신러닝의 가장 기본적인 사실이에요.

그런데 200개를 넘어서면서 0.86 언저리에서 더는 오르지 않습니다. 이 데이터에는 잡음을 일부러 섞어서, 경계 근처에서는 같은 자리에 두 종류 점이 섞여 있어요. 그 부분은 아무리 좋은 모델도 맞힐 수 없는 **데이터 자체의 한계**입니다. 그리고 k=1은 데이터가 3000개가 되어도 0.81~0.82에 머뭅니다. 잡음까지 외우는 버릇은 데이터가 많아져도 고쳐지지 않는 거죠. 데이터를 늘리는 것과 모델을 알맞게 고르는 것, 둘 다 필요합니다.

Q

Q. 그럼 데이터는 무조건 많을수록 좋은가요? 같은 품질이라면 대체로 그렇습니다. 하지만 위 결과처럼 어느 지점부터는 늘려도 얻는 게 줄어들고, 레이블을 붙이는 비용은 계속 들어요. 더 중요한 건 **질**입니다. 레이블이 잘못 붙은 데이터, 실제 쓰임과 다른 상황에서 모은 데이터는 많을수록 오히려 해가 될 수 있어요. 데이터를 모으고 점검하는 일은 2장, 데이터와 실험을 버전으로 관리하는 일은 18장에서 다룹니다.

1.8 특성의 눈금 맞추기

최근접 이웃은 거리로 판단하니, 특성마다 **눈금(단위)** 이 크게 다르면 문제가 생깁니다. 고객 400명의 나이와 연소득으로 구매 여부를 맞히는 예제로 확인해 볼게요. 예제용으로 만든 데이터라, 구매 여부는 나이로만 정해지게(40세 초과면 구매) 일부러 꾸몄습니다.

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

# 고객 400명의 나이(세)와 연소득(만 원). 예제용으로 만든 데이터다.
rng = np.random.default_rng(0)
나이 = rng.uniform(20, 60, 400)
연소득 = rng.uniform(2000, 8000, 400)
X = np.column_stack([나이, 연소득])
y = (나이 > 40).astype(int)          # 구매 여부는 나이로만 정해진다고 하자

X_학습, X_테스트 = X[:300], X[300:]
y_학습, y_테스트 = y[:300], y[300:]

그대로 = KNeighborsClassifier(n_neighbors=5).fit(X_학습, y_학습)
print(f"그대로:    정확도 {그대로.score(X_테스트, y_테스트):.3f}")

# 학습 데이터의 평균과 표준편차로 두 특성의 눈금을 맞춘다 (표준화)
```

```
평균, 표준편차 = X_학습.mean(axis=0), X_학습.std(axis=0)
표준화 = KNeighborsClassifier(n_neighbors=5).fit((X_학습 - 평균) / 표준편차, y_학습)
print(f"표준화 후: 정확도 {표준화.score((X_테스트 - 평균) / 표준편차, y_테스트):.3f}")
print("특성별 표준편차(나이, 연소득):", 표준편차.round(1))
```

```
그대로:    정확도 0.510
표준화 후: 정확도 0.980
특성별 표준편차(나이, 연소득): [  11.9 1643.5]
```

나이만 보면 답이 나오는 쉬운 문제인데, 그대로 넣으니 정확도가 0.510, 동전 던지기 수준입니다. 이유는 마지막 줄에 있어요. 나이는 표준편차가 11.9세 정도인데 연소득은 1643.5만 원입니다. 두 고객의 거리를 재면 나이 차이 20세는 연소득 차이 수백만 원에 비해 티도 나지 않아요. 결국 이웃을 **연소득으로만** 고르게 되고, 연소득은 구매와 아무 관계가 없으니 맞힐 수가 없는 겁니다.

해결은 특성마다 평균을 빼고 표준편차로 나눠서 눈금을 맞추는 것, 즉 **표준화**입니다. 표준화하니 정확도가 0.980으로 뛰었어요. 두 특성이 같은 눈금이 되자 거리에 나이 차이가 제대로 반영된 거죠.

평균과 표준편차를 **학습 데이터에서만** 구했다는 점을 눈여겨보세요. 테스트 데이터는 "아직 오지 않은 미래의 데이터" 역할이라, 그 평균을 미리 알고 쓰면 반칙입니다. 이런 반칙을 **데이터 누수**라고 부르는데, 알아채기 어려워서 실무에서 자주 일어나요. 표준화를 비롯한 전처리를 scikit-learn으로 안전하게 하는 법과 데이터 누수는 2장의 주제입니다. 4장 실습에서는 같은 계산을 PyTorch 텐서로 다시 해 봅니다.

1.9 얼마나 잘하나: 평가 맛보기

지금까지 성능을 정확도로만 쟀습니다. 그런데 정확도 하나로 충분할까요? 두 가지만 맛보고, 자세한 평가 지표는 3장으로 넘기겠습니다.

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.metrics import confusion_matrix
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)
섞은_번호 = np.random.default_rng(42).permutation(len(X))
```

```
X_학습, y_학습 = X[섞은_번호[30:]], y[섞은_번호[30:]]
X_테스트, y_테스트 = X[섞은_번호[:30]], y[섞은_번호[:30]]

# 비교 기준: 학습 데이터에서 가장 많은 품종을 무조건 답하는 "모델"
가장_흔한 = np.bincount(y_학습).argmax()
print(f"늘 {가장_흔한}번이라고 답하면: {(y_테스트 == 가장_흔한).mean():.3f}")

예측 = KNeighborsClassifier(n_neighbors=5).fit(X_학습, y_학습).predict(X_테스트)
print(f"최근접 이웃(k=5):      {(예측 == y_테스트).mean():.3f}")
print("혼동 행렬 (행: 정답, 열: 예측)")
print(confusion_matrix(y_테스트, 예측))

# 정확도의 함정: 거래 1000건 중 사기가 10건뿐이라면?
사기 = np.zeros(1000, dtype=int)
사기[:10] = 1
늘_정상 = np.zeros(1000, dtype=int)
print(f"늘 '정상'이라고 답하는 모델의 정확도: {(늘_정상 == 사기).mean():.3f}, "
      f"찾아낸 사기: {((늘_정상 == 1) & (사기 == 1)).sum()}건")
```

```
늘 1번이라고 답하면: 0.300
최근접 이웃(k=5):      0.967
혼동 행렬 (행: 정답, 열: 예측)
[[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]
늘 '정상'이라고 답하는 모델의 정확도: 0.990, 찾아낸 사기: 0건
```

첫째, 모델의 점수는 **비교 기준과 나란히** 봐야 의미가 있습니다. 학습 데이터에서 가장 흔한 품종(1번)을 무조건 답하면 테스트에서 0.300이 나와요. 최근접 이웃의 0.967은 이 기준보다 훨씬 높으니 확실히 무언가를 배웠습니다. 1.6절 회귀에서 "평균으로 답하기"를 먼저 잰 것과 같은 습관이에요.

둘째, **어디서 틀렸는지**를 봐야 합니다. 혼동 행렬은 행이 정답, 열이 예측인 표로, 대각선이 맞힌 개수, 나머지가 틀린 개수예요. 둘째 줄 셋째 칸의 1은 "정답은 1번(versicolor)인데 2번(virginica)이라고 한 꽃이 한 송이"라는 뜻입니다. setosa는 한 송이도 안 틀렸고, 틀린 한 송이는 1.3절부터 계속 헷갈리던 그 두 품종 사이에서 나왔어요.

마지막 두 줄은 정확도의 함정입니다. 거래 1000건 중 사기가 10건뿐일 때, 무조건 "정상"이라고 답

하는 모델의 정확도는 0.990이에요. 숫자만 보면 훌륭하지만 사기를 **한 건도** 찾지 못했습니다. 사기 탐지, 질병 진단처럼 찾아야 할 쪽이 드문 문제에서는 정확도가 아니라 "진짜 사기 중 몇 건을 찾았나(재현율)" 같은 다른 잣대가 필요해요. 3장에서 정밀도, 재현율, F1 같은 지표로 이어지고, 이렇게 한 쪽이 드문 **불균형 데이터**를 다루는 방법은 2장에서 봅니다.

1.10 머신러닝 작업의 흐름

지금까지 데이터를 보고, 나누고, 모델을 학습하고, 평가했습니다. 실제 프로젝트는 그 앞뒤로 단계가 더 있어요. 대부분의 머신러닝 프로젝트는 다음 다섯 단계를 돌고 돕니다.

단계	하는 일	스팸 거르기라면	이 책에서
1. 문제 정의	무엇을 맞힐지, 무엇이 성공인지 정한다	스팸 분류, 정상 메일을 스팸으로 잘못 보내는 일은 1000통에 1통 이하	이 장
2. 데이터	모으고, 레이블을 붙이고, 정리하고, 나눈다	사용자가 누른 "스팸 신고" 모으기	2, 7장
3. 모델	모델을 고르고 학습한다	단어 기반 모델에서 신경망까지	3, 5~15장
4. 평가	따로 떼어 둔 데이터로 재고, 틀린 사례를 들여다본다	놓친 스팸, 잘못 막은 메일 살펴보기	3, 8장
5. 배포와 관찰	실제 서비스에 붙이고, 성능이 유지되는지 지켜본다	메일 서버에 붙이고 신고율 지켜보기	18~20장

몇 가지를 덧붙이겠습니다.

1단계가 가장 중요합니다. "스팸을 잘 거른다"는 목표로는 부족해요. 스팸 몇 통을 놓치는 것과 중요한 업무 메일을 스팸함에 넣는 것 중 무엇이 더 나쁜지 정해야 1.9절에서 본 것처럼 어떤 잣대로 잴지가 정해집니다. 그리고 1.1절의 질문, "이게 정말 머신러닝이 필요한 문제인가?"도 여기서 던져야 해요.

시간은 대부분 2단계에 듭니다. 머신러닝 실무자들은 일하는 시간의 대부분을 데이터를 모으고 다듬는 데 쓴다고 입을 모읍니다. 1.8절에서 봤듯이 눈금 하나 안 맞춰도 모델이 엉터리가 되니까요. 이 책이 모델 이야기에 앞서 2장 전체를 데이터 준비에 쓰는 이유예요.

한 바퀴로 끝나지 않습니다. 평가해 보고 성능이 모자라면 데이터를 더 모으거나(2단계), 모델을 바꾸거나(3단계), 아예 문제 정의를 고칩니다(1단계). 배포한 뒤에도 세상은 변해요. 스팸 보내는 사람들은 새 수법을 만들어 내고, 작년 데이터로 배운 모델은 서서히 낡아 갑니다. 그래서 배포 후에도 성능을 지켜보다가 새 데이터로 다시 학습하는 것이 머신러닝 프로젝트의 정상적인 모습입니다.

배포의 맛: 모델 앞에 창구 세우기

5단계의 배포가 막연하게 느껴진다면, 학습된 모델을 **함수 하나 뒤에 숨기는 것**에서 출발한다고 생각하면 됩니다. 사용자는 모델을 직접 만지지 않고 그 함수(창구)에 묻고, 창구는 입력을 점검한 뒤 모델에 전달해요. 그런데 왜 점검이 필요할까요?

```
from dataclasses import dataclass

import numpy as np
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier

붓꽃 = load_iris()
모델 = KNeighborsClassifier(n_neighbors=5).fit(붓꽃.data, 붓꽃.target)
학습_최소, 학습_최대 = 붓꽃.data.min(axis=0), 붓꽃.data.max(axis=0)


@dataclass
class 측정값:
    꽃받침_길이: float
    꽃받침_너비: float
    꽃잎_길이: float
    꽃잎_너비: float


def 품종_알려주기(측정: 측정값) -> str:
    """서비스 창구: 측정값을 받아 품종 이름을 돌려준다."""
    값 = np.array([[측정.꽃받침_길이, 측정.꽃받침_너비, 측정.꽃잎_길이, 측정.꽃잎_너비]])
```

```
    if ((값 < 학습_최소 * 0.5) | (값 > 학습_최대 * 1.5)).any():
        return "모르겠어요 (배운 적 없는 크기)"
    return str(붓꽃.target_names[모델.predict(값)[0]])


print("모델에 바로 묻기:", 붓꽃.target_names[모델.predict([[5.0, 3.4, 100.0, 0.2]])[0]])
print("창구로 묻기 1:", 품종_알려주기(측정값(5.0, 3.4, 1.5, 0.2)))
print("창구로 묻기 2:", 품종_알려주기(측정값(5.0, 3.4, 100.0, 0.2)))
```

```
모델에 바로 묻기: virginica
창구로 묻기 1: setosa
창구로 묻기 2: 모르겠어요 (배운 적 없는 크기)
```

첫 줄을 보세요. 꽃잎 길이 100cm인 꽃은 세상에 없는데, 모델은 망설임 없이 virginica라고 답합니다. 모델은 "모르겠다"고 말하는 법을 배운 적이 없어요. 최근접 이웃은 아무리 멀어도 가장 가까운 이웃을 찾아내고, 신경망도 처음 보는 입력에 그럴듯한 답을 내놓습니다. 그래서 서비스에서는 모델 앞에 창구를 두고, 학습 데이터의 범위에서 크게 벗어난 입력은 걸러 내야 해요. 여기서는 1권 8장의 `dataclass`로 입력 모양을 정하고, 학습 데이터 최솟값의 절반보다 작거나 최댓값의 1.5배보다 큰 값이 있으면 거절했습니다. 20장에서는 같은 생각을 FastAPI와 Pydantic으로 제대로 된 웹 서비스로 만듭니다.

1.11 딥러닝은 무엇이 다른가요?

특성은 누가 고르나요?

이 장의 모델들은 모두 사람이 정해 준 특성을 받아서 배웠습니다. 붓꽃은 사람이 꽃잎 길이를 재서 줬고, 스팸은 사람이 "띄어쓰기로 단어를 나눠 세자"고 정했어요. 그런데 특성을 잘못 고르면 어떻게 될까요?

손으로 쓴 숫자 이미지로 실험해 보겠습니다. scikit-learn에 들어 있는 숫자 데이터는 0~9를 손으로 쓴 이미지 1797장으로, 한 장이 8 × 8 = 64픽셀이에요(UCI 머신러닝 저장소의 손글씨 숫자 데이터 Optical Recognition of Handwritten Digits의 일부, CC BY 4.0). 먼저 한 장을 글자로 그려 보겠습니다.

```
from sklearn.datasets import load_digits

숫자 = load_digits()
print("이미지 모양:", 숫자.images.shape, " 특성 모양:", 숫자.data.shape)
print("픽셀 값 범위:", 숫자.data.min(), "~", 숫자.data.max())

그림 = 숫자.images[0]                  # 8 × 8 픽셀, 0(흰색)~16(검은색)
for 줄 in 그림:
    print("".join(" .:-=+*#%@"[int(값) * 9 // 16] * 2 for 값 in 줄))
print("레이블:", 숫자.target[0])
```

```
이미지 모양: (1797, 8, 8)  특성 모양: (1797, 64)
픽셀 값 범위: 0.0 ~ 16.0
    ::##++
    ##%%++%%::
  ..%%..  **==
  ::**    ====
  ::==    ++==
  ::**    **--
  ..##::++**
    --##++
레이블: 0
```

픽셀마다 0(흰색)에서 16(검은색) 사이의 숫자가 들어 있고, 진할수록 @에 가까운 글자로 그렸습니다. 거칠지만 0이라는 게 보이죠? 이제 두 가지 방법으로 최근접 이웃을 학습시켜 비교합니다. 하나는 사람이 "숫자를 구별하는 데 쓸 만하겠다"고 생각한 특성 네 개(전체 잉크 양, 위쪽 절반의 잉크 비율, 왼쪽 절반의 잉크 비율, 가운데의 잉크 비율)를 쓰고, 다른 하나는 픽셀 64개를 그대로 씁니다.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

숫자 = load_digits()
그림들 = 숫자.images                    # (1797, 8, 8)
```

```
def 손으로_만든_특성(그림들: np.ndarray) -> np.ndarray:
    """사람이 '숫자를 구별하는 데 쓸 만하다'고 생각한 특성 네 개."""
    전체_잉크 = 그림들.sum(axis=(1, 2))
    위_비율 = 그림들[:, :4, :].sum(axis=(1, 2)) / 전체_잉크
    왼쪽_비율 = 그림들[:, :, :4].sum(axis=(1, 2)) / 전체_잉크
    가운데_잉크 = 그림들[:, 2:6, 2:6].sum(axis=(1, 2)) / 전체_잉크
    return np.column_stack([전체_잉크, 위_비율, 왼쪽_비율, 가운데_잉크])


for 이름, X in [("손으로 만든 특성 4개", 손으로_만든_특성(그림들)),
                ("픽셀 64개 그대로", 숫자.data)]:
    X_학습, X_테스트, y_학습, y_테스트 = train_test_split(
        X, 숫자.target, test_size=0.3, random_state=0)
    평균, 표준편차 = X_학습.mean(axis=0), X_학습.std(axis=0) + 1e-9
    모델 = KNeighborsClassifier(n_neighbors=5).fit((X_학습 - 평균) / 표준편차, y_학습)
    print(f"{이름}: 정확도 {모델.score((X_테스트 - 평균) / 표준편차, y_테스트):.3f}")
```

```
손으로 만든 특성 4개: 정확도 0.635
픽셀 64개 그대로: 정확도 0.970
```

나름 머리를 써서 만든 특성 네 개로는 0.635, 픽셀 그대로는 0.970입니다. 사람이 고른 특성에는 숫자를 가르는 정보가 많이 빠져 있었던 거예요. 예를 들어 1과 7은 잉크 양도 비슷하고 위아래 비율도 비슷해서 이 특성들로는 잘 구별되지 않습니다. 특성을 고르는 일은 이렇게 어렵고, 고전적인 머신러닝에서는 좋은 특성을 만드는 일(**특성 공학**)에 전문가의 시간이 가장 많이 들었어요.

픽셀 그대로면 충분할까요?

그럼 특성을 고민하지 말고 늘 픽셀을 그대로 쓰면 될까요? 같은 모델에 테스트 이미지를 오른쪽으로 한두 칸 밀어서 넣어 보겠습니다. 사람 눈에는 똑같은 숫자예요.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

```
숫자 = load_digits()
X_학습, X_테스트, y_학습, y_테스트 = train_test_split(
    숫자.images, 숫자.target, test_size=0.3, random_state=0)

모델 = KNeighborsClassifier(n_neighbors=5).fit(X_학습.reshape(-1, 64), y_학습)


def 오른쪽으로_밀기(그림들: np.ndarray, 칸: int) -> np.ndarray:
    """그림 전체를 오른쪽으로 칸만큼 옮기고 빈자리는 0으로 채운다."""
    밀린 = np.zeros_like(그림들)
    밀린[:, :, 칸:] = 그림들[:, :, :8 - 칸]
    return 밀린


for 칸 in [0, 1, 2]:
    X_밀린 = 오른쪽으로_밀기(X_테스트, 칸) if 칸 else X_테스트
    print(f"오른쪽으로 {칸}칸 밀었을 때 정확도: {모델.score(X_밀린.reshape(-1, 64), y_테스
    ↪ 트):.3f}")
```

```
오른쪽으로 0칸 밀었을 때 정확도: 0.981
오른쪽으로 1칸 밀었을 때 정확도: 0.567
오른쪽으로 2칸 밀었을 때 정확도: 0.124
```

0.981이던 정확도가 한 칸 밀자 0.567, 두 칸 밀자 0.124로 무너졌습니다. 열 종류 중 아무거나 찍어도 0.1은 나오니 거의 찍기 수준이에요. 최근접 이웃은 "같은 자리의 픽셀이 비슷한가"로 거리를 재는데, 숫자를 한 칸 밀면 같은 자리의 픽셀이 완전히 달라지거든요. 픽셀 그대로의 거리는 "숫자의 **모양**이 비슷한가"를 재지 못한다는 뜻입니다. 사진이 8 × 8이 아니라 224 × 224 컬러라면(3 × 224 × 224, 숫자가 15만 개) 이 문제는 훨씬 심해져요.

여기서 **딥러닝**이 등장합니다. 딥러닝 모델은 픽셀을 그대로 받되, 여러 층을 거치면서 **쓸 만한 특성을 스스로 만들어** 냅니다. 첫 층은 짧은 선이나 모서리 같은 단순한 무늬를, 다음 층은 그 무늬를 조합한 곡선이나 고리를, 더 깊은 층은 "위에 고리가 있고 아래로 줄기가 내려오는 모양(9)" 같은 복잡한 특징을 잡아내는 식이에요. 사람이 특성을 설계하는 대신 **특성을 만드는 방법까지 데이터에서 배우는 것**, 이것이 딥러닝이 고전적인 머신러닝과 가장 다른 점입니다. 위치가 조금 달라도 같은 무늬를 찾아내는 구조인 합성곱 신경망(CNN)이 바로 위의 "밀기" 문제를 푸는 모델이고, 9장에서 직접 만듭

니다.

	고전적인 머신러닝 (2·3장)	딥러닝 (5장부터)
특성	사람이 설계한다	모델이 원본 데이터에서 배운다
잘 맞는 데이터	표 형태(고객, 거래, 센서 수치)	이미지, 소리, 글처럼 원본이 큰 데이터
필요한 데이터 양	수백~수만 건으로도 충분할 때가 많다	대체로 많을수록 좋다(전이 학습으로 줄일 수 있다)
계산	CPU로 충분	GPU가 있으면 크게 빨라진다
해석	비교적 쉽다	왜 그렇게 판단했는지 알기 어렵다

딥러닝이 늘 이기는 것은 아닙니다. 고객 정보나 거래 기록 같은 **표 데이터**에서는 3장의 그래디언트 부스팅 같은 고전적인 방법이 딥러닝과 비슷하거나 더 나은 경우가 많고, 학습도 빠르고 설명하기도 쉬워요. 그래서 이 책은 딥러닝으로 바로 뛰어들지 않고 2·3장에서 고전적인 머신러닝을 먼저 다룹니다. 문제에 맞는 도구를 고를 줄 아는 것이 목표예요.

1.12 이 책의 지도

2권은 다섯 부분, 21개 장으로 되어 있습니다. 이 장에서 맛본 낱말들이 어디서 다시 깊어지는지 지도로 보면 이렇습니다.

부분	장	무엇을 배우나
1부 머신러닝의 감각	1~3장	머신러닝의 기본 낱말, 데이터 준비(나누기·누수·전처리), scikit-learn으로 고전적인 모델과 평가 지표·교차 검증

부분	장	무엇을 배우나
2부 PyTorch 기초	4~8장	텐서, 자동 미분과 경사하강법, `nn.Module`과 학습 루프, 데이터 파이프라인, 과적합을 막고 학습을 잘 시키는 기술
3부 모델 구조	9~15장	CNN, 전이 학습, RNN·LSTM, 어텐션과 트랜스포머 직접 구현, 한국어 자연어 처리, LLM 활용과 LoRA, 생성 모델
4부 실무	16~20장	GPU와 성능, 분산·대규모 학습, 실험 관리와 재현성, ONNX와 양자화, FastAPI로 모델 서빙
5부 종합 프로젝트	21장	맛집 추천 AI를 처음부터 끝까지: 모델, API, 테스트, Docker

1부에서 머신러닝의 감각을 익히고, 2부에서 PyTorch라는 도구를 손에 익히고, 3부에서 그 도구로 여러 구조의 모델을 만들고, 4부에서 그 모델을 빠르고 믿을 수 있게 만들어 세상에 내놓습니다. 5부에서는 이 모든 것을 하나의 서비스로 묶어요. 1권에서 배운 파이썬(클래스, 제너레이터, FastAPI, pandas)이 곳곳에서 다시 쓰이니, 필요할 때마다 1권을 펼쳐 보세요.

1.13 실습: 와인 생산자 맞히기

이 장에서 배운 것을 모두 써서 처음부터 끝까지 한 바퀴 돌아 보겠습니다. scikit-learn에 들어 있는 와인 데이터는 이탈리아 한 지역의 와인 178병을 화학 성분 13가지로 분석한 기록이에요(UCI 머신러닝 저장소의 Wine 데이터, CC BY 4.0). 레이블은 그 와인을 만든 생산자(세 곳)입니다. 할 일은 다섯 가지예요.

1. 데이터의 모양과 특성의 범위 살펴보기
2. 학습·검증·테스트 세 덩어리로 나누기
3. 학습 데이터 기준으로 표준화하기

4. 검증 데이터로 “표준화 여부”와 k 고르기
5. 고른 설정으로 테스트 데이터에서 마지막에 한 번만 재고, scikit-learn과 결과 맞춰 보기

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler


def 최근접_이웃_예측(X_학습: np.ndarray, y_학습: np.ndarray,
                    X_새: np.ndarray, k: int) -> np.ndarray:
    """새 샘플마다 가장 가까운 학습 샘플 k개의 다수결로 레이블을 정한다."""
    거리 = np.sqrt(((X_새[:, None, :] - X_학습[None, :, :]) ** 2).sum(axis=2))
    이웃 = np.argsort(거리, axis=1, kind="stable")[:, :k]
    return np.array([np.bincount(줄).argmax() for 줄 in y_학습[이웃]])


# 1) 데이터: 와인 178병 × 화학 성분 13가지, 레이블은 생산자 3곳
와인 = load_wine()
X, y = 와인.data, 와인.target
print("모양:", X.shape, " 생산자별:", np.bincount(y))
print("특성별 범위 예:", 와인.feature_names[12], X[:, 12].min(), "~", X[:, 12].max(),
      "/", 와인.feature_names[7], X[:, 7].min(), "~", X[:, 7].max())

# 2) 나누기: 학습 60%, 검증 20%, 테스트 20%
번호 = np.random.default_rng(7).permutation(len(X))
학습, 검증, 테스트 = 번호[:106], 번호[106:142], 번호[142:]

# 3) 표준화: 평균과 표준편차는 학습 데이터에서만 구한다
평균, 표준편차 = X[학습].mean(axis=0), X[학습].std(axis=0)
X_표준 = (X - 평균) / 표준편차

# 4) 검증 데이터로 설정 고르기: 표준화 여부 × k
결과 = {}
for 이름, 데이터 in [("그대로", X), ("표준화", X_표준)]:
    for k in [1, 3, 5, 9, 15]:
        예측 = 최근접_이웃_예측(데이터[학습], y[학습], 데이터[검증], k)
```

```
        결과[(이름, k)] = (예측 == y[검증]).mean()
    print(이름, " ".join(f"k={k}:{결과[(이름, k)]:.3f}" for k in [1, 3, 5, 9, 15]))
# 검증 정확도가 같으면 k가 큰 쪽(덜 외우는 쪽)을 고른다
최고_이름, 최고_k = max(결과, key=lambda 설정: (결과[설정], 설정[1]))
print(f"고른 설정: {최고_이름}, k={최고_k}")

# 5) 테스트는 마지막에 딱 한 번
데이터 = X_표준 if 최고_이름 == "표준화" else X
예측 = 최근접_이웃_예측(데이터[학습], y[학습], 데이터[테스트], 최고_k)
print(f"테스트 정확도: {(예측 == y[테스트]).mean():.3f} ({(예측 == y[테스트]).sum()}/{len(테스
↪ 트)})")

# 6) scikit-learn으로 같은 결과가 나오는지 확인
척도 = StandardScaler().fit(X[학습])
모델 = KNeighborsClassifier(n_neighbors=최고_k).fit(척도.transform(X[학습]), y[학습])
print("scikit-learn과 예측이 같은가:",
      np.array_equal(모델.predict(척도.transform(X[테스트])), 예측))
```

```
모양: (178, 13)  생산자별: [59 71 48]
특성별 범위 예: proline 278.0 ~ 1680.0 / nonflavanoid_phenols 0.13 ~ 0.66
그대로 k=1:0.889 k=3:0.833 k=5:0.806 k=9:0.694 k=15:0.778
표준화 k=1:0.944 k=3:0.944 k=5:0.917 k=9:0.944 k=15:0.917
고른 설정: 표준화, k=9
테스트 정확도: 0.944 (34/36)
scikit-learn과 예측이 같은가: True
```

한 단계씩 짚어 보겠습니다.

- **데이터 살펴보기**: 특성 13개의 범위가 제각각입니다. 프롤린(proline)은 278~1680인데 비플라보노이드 페놀(nonflavanoid_phenols)은 0.13~0.66이에요. 1.8절에서 본 "눈금이 다른" 상황이니, 표준화가 중요하겠다고 미리 짐작할 수 있습니다.
- **세 덩어리로 나누기**: 이번에는 학습(106병), 검증(36병), 테스트(36병) 세 덩어리로 나눴습니다. 검증 데이터가 새로 생긴 이유는 다음 단계에 있어요.
- **표준화**: 평균과 표준편차는 학습 데이터에서만 구하고, 그 값으로 세 덩어리 모두를 바꿨습니다. 1.8절의 "테스트 데이터는 미래의 데이터" 원칙 그대로예요.
- **설정 고르기**: 표준화 여부 두 가지 × k 다섯 가지, 열 가지 설정을 **검증 데이터**로 시험했습니

다. 표준화하지 않으면 0.694~0.889, 표준화하면 0.917~0.944로 표준화 쪽이 고르게 좋아요. 표준화한 k=1, 3, 9가 0.944로 동점이라, 같으면 k가 큰 쪽(덜 외우는 쪽)을 고르도록 해서 **표준화, k=9**가 뽑혔습니다. 1.7절에서 k가 작을수록 잡음을 외우기 쉽다고 했죠.

- **테스트는 마지막에 한 번**: 고른 설정으로 테스트 데이터를 처음 꺼내 쟀더니 36병 중 34병, 정확도 0.944입니다. 직접 짠 함수와 scikit-learn(`StandardScaler`로 표준화하고 `KNeighborsClassifier`로 예측)의 예측도 모두 같았어요.

왜 테스트 데이터로 k를 고르면 안 될까요? 열 가지 설정 중 테스트 점수가 가장 높은 것을 고르면, 그 점수는 "테스트 데이터에 우연히 잘 맞은 설정"의 점수라 실제보다 부풀려집니다. 1.4절에서 기준을 고를 때 쓴 데이터로 정확도를 잰 것과 같은 실수예요. 그래서 고르는 일은 검증 데이터로 하고, 테스트 데이터는 모든 결정이 끝난 뒤 **딱 한 번** 씁니다. 실제로 이 실습에서도 검증에서 최고였던 0.944가 테스트에서 그대로 나왔지만, 항상 그렇지는 않아요. 데이터가 적을 때 검증 데이터를 여러 번 바꿔 가며 평가하는 **교차 검증**은 3장에서 배웁니다.

1.14 정리와 다음 장 미리보기

이 장에서 머신러닝의 기본 생각과 낱말을 익히고 첫 모델을 만들었습니다.

- **머신러닝**: 사람이 규칙을 짜는 대신 데이터와 정답을 주고, 규칙은 프로그램이 배운다. 규칙을 적기 어렵고 예가 충분할 때 쓴다
- **데이터의 낱말**: 샘플(행), 특성(열), 레이블(정답). 특성 표는 `X`, 레이블은 `y`. 샘플은 특성 공간 속의 점
- **세 갈래**: 지도 학습(분류는 고르기, 회귀는 재기), 비지도 학습(정답 없이 묶기), 강화 학습(해 보고 보상으로 배우기, 탐험과 활용)
- **모델·학습·예측**: 모델은 빈칸(파라미터)이 있는 함수, 학습은 잣대가 좋아지는 빈칸 값 찾기, 하이퍼파라미터는 사람이 고르는 설정
- **최근접 이웃**: 거리를 재고 가까운 k개로 투표(분류)하거나 평균(회귀). NumPy로 직접 짜고 scikit-learn의 `fit`/`predict`/`score`로 같은 결과 확인
- **일반화와 과적합**: 학습 점수가 아니라 처음 보는 데이터의 점수가 목표. k가 작으면 과적합, 너무 크면 과소적합. 데이터가 많을수록 외우기 어렵다
- **평가와 흐름**: 학습·검증·테스트로 나누고 테스트는 마지막에 한 번. 비교 기준과 함께 보고, 정확도의 함정을 조심. 문제 정의 → 데이터 → 모델 → 평가 → 배포를 돌고 돈다
- **딥러닝**: 특성을 사람이 설계하지 않고 원본 데이터에서 스스로 배운다. 표 데이터에서는 고전적인 방법도 강하다

1.8절과 실습에서 “표준화는 학습 데이터 기준으로”, “테스트 데이터는 미래의 데이터”라는 말을 여러 번 했습니다. 이 장의 데이터는 깔끔하게 정리된 숫자뿐이었지만, 실제 데이터에는 빈칸이 있고, “서울”, “부산” 같은 글자가 섞여 있고, 한쪽 레이블이 드물고, 시간 순서가 있어요. 다음 2장에서는 이런 날것의 데이터를 모델에 넣을 수 있게 다듬는 법을 배웁니다. 학습·검증·테스트를 제대로 나누는 법부터 시작해서, 알아채기 어려운 데이터 누수를 피하는 법, 그리고 그 모든 전처리를 scikit-learn의 `Pipeline` 하나로 묶어 실수를 막는 법까지 다룹니다.

02

데이터 준비: 모델에 넣기 전에 할 일

학습 목표

- 데이터를 받자마자 확인할 것들을 체크리스트로 점검하기
- 데이터를 학습·검증·테스트로 나누는 이유를 설명하고 직접 나누기
- 데이터 누수가 점수를 얼마나 부풀리는지 확인하고 막는 순서 익히기
- 결측치, 범주형 값, 단위가 다른 숫자를 모델이 먹을 수 있게 바꾸기
- scikit-learn의 ColumnTransformer와 Pipeline으로 전처리를 한 덩어리로 묶기
- 불균형 데이터와 시간 순서 데이터를 나눌 때의 함정 피하기

들어가며: 장 봐 온 재료를 바로 냄비에 넣나요?

마트에서 장을 봐 왔다고 해 봅시다. 감자는 흙이 묻어 있고, 대파는 뿌리가 달려 있고, 고기는 큰 덩어리째예요. 이걸 그대로 냄비에 넣는 사람은 없죠? 씻고, 다듬고, 먹기 좋은 크기로 썰어야 요리가 됩니다. 실제로 요리 시간의 대부분은 불 앞이 아니라 도마 앞에서 보내요.

머신러닝도 똑같습니다. 1장에서 모델에 특성과 레이블을 주면 규칙을 스스로 배운다고 했는데, 현실의 데이터는 그렇게 깔끔하게 오지 않아요. 칸이 비어 있고, "서울" 같은 글자가 섞여 있고, 단위가 제각각입니다. 이런 데이터를 모델에 그냥 넣으면 어떻게 될까요? 고객 다섯 명의 기록으로 해 보겠습니다.

```
import numpy as np
import pandas as pd
```

```
from sklearn.linear_model import LogisticRegression

# 고객 다섯 명의 기록 (예제용으로 만든 데이터)
고객 = pd.DataFrame({
    "나이": [34, 52, np.nan, 41, 29],
    "지역": ["서울", "부산", "서울", "대구", "부산"],
    "월요금": [55000, 89000, 32000, np.nan, 45000],
    "이탈": [0, 1, 0, 1, 0],
})
X = 고객.drop(columns="이탈")     # 문제(특성)
y = 고객["이탈"]                  # 정답(레이블)

모델 = LogisticRegression()
try:
    모델.fit(X, y)                # 그대로 넣어 보면?
except ValueError as e:
    print("오류 1:", str(e).splitlines()[0])

try:
    모델.fit(X[["나이", "월요금"]], y)   # 글자 열을 빼고 숫자만 넣어 보면?
except ValueError as e:
    print("오류 2:", str(e).splitlines()[0])
```

```
오류 1: could not convert string to float: '서울'
오류 2: Input X contains NaN.
```

`LogisticRegression`(로지스틱 회귀)은 1장의 최근접 이웃처럼 `fit`과 `predict`를 갖춘 분류 모델입니다. 이름에 "회귀"가 붙었지만 분류에 써요. 어떻게 동작하는지는 3장에서 자세히 보고, 이 장에서는 "숫자를 받아 예/아니요를 맞히는 모델" 정도로 알고 쓰겠습니다.

첫 번째 오류는 "'서울'이라는 글자를 실수로 바꿀 수 없다"는 뜻입니다. 모델은 숫자만 계산할 수 있어서 글자를 받으면 어쩔 줄 몰라 해요. 그래서 글자 열을 빼고 숫자만 넣었더니, 이번에는 "입력에 NaN이 들어 있다"며 또 멈췄습니다. 빈칸이 있는 숫자로는 계산을 할 수 없거든요.

이 장은 이런 재료 손질을 다룹니다. 그런데 손질만 하는 장은 아니에요. 더 중요한 이야기가 있습니다. **모델의 점수를 믿을 수 있게 만드는 일**이에요. 데이터를 어떻게 나누고 어떤 순서로 손질하느냐에 따라, 쓸모없는 모델이 80점짜리처럼 보이기도 하거든요. 이 장에서 실제로 그런 일이 일어나는 걸

눈으로 확인하고 막는 법을 익힙니다.

NumPy 배열과 pandas DataFrame을 만들고 고르고 묶는 기본기는 1권 20장에서 다뤘으니 여기서는 다시 설명하지 않습니다. `df.info()`, `df.describe()`, `dropna()`, `fillna()`, `groupby()`가 낯설다면 1권 20.2~20.3절과 20.6절을 먼저 훑어보고 오세요.

> **Q. 이 장에서 새로 설치할 것이 있나요?** 1장에서 설치한 scikit-learn에 pandas만 더하면 됩니다(`uv add pandas`). 이 장의 예제는 scikit-learn 1.9, pandas 3.0, 파이썬 3.12에서 실행해 확인했어요. 전처리는 딥러닝에서도 그대로 필요하지만 도구는 scikit-learn이 가장 잘 갖춰져 있어서, 이 장에서는 PyTorch 대신 scikit-learn을 씁니다.

2.1 먼저 데이터를 살펴보기

건강검진부터

병원에 가면 의사는 처방부터 하지 않고 먼저 키, 몸무게, 혈압을 잽니다. 데이터도 마찬가지예요. 모델을 고민하기 전에 데이터가 어떤 상태인지부터 재 봐야 합니다. 1권 20장에서 `info()`와 `describe()`로 건강검진을 했죠. 머신러닝을 하기 전에는 거기에 몇 가지를 더 봅니다. 일부러 문제를 잔뜩 심어 둔 회원 명단으로 해 보겠습니다.

```
import numpy as np
import pandas as pd

# 설문으로 모은 회원 기록 (예제용으로 만든 데이터)
회원 = pd.DataFrame({
    "회원번호": [101, 102, 103, 104, 105, 106, 107, 107, 108, 109],
    "나이": [27, 999, 45, np.nan, 38, 0, 52, 52, 31, 44],
    "지역": ["서울", "서울시", "부산", "서울 ", "대구", "부산", "서울", "서울", None, "대구"],
    "가입일": ["2025-01-03", "2025-01-10", "2025-02-01", "2025-02-14", "2025-03-02",
              "2025-03-09", "2025-03-20", "2025-03-20", "2025-04-01", "2025-04-11"],
    "탈퇴": [0, 0, 1, 0, 0, 0, 1, 1, 0, 0],
})

print("모양:", 회원.shape)
print("자료형:", dict(회원.dtypes.astype(str)))
print("빈칸 수:", dict(zip(회원.columns, 회원.isna().sum().tolist())))
```

```
print("중복 행:", 회원.duplicated().sum(), "개")
print("지역 값:", 회원["지역"].value_counts(dropna=False).to_dict())
print("나이 범위:", 회원["나이"].min(), "~", 회원["나이"].max())
print("정답 비율:", 회원["탈퇴"].value_counts(normalize=True).round(2).to_dict())
```

```
모양: (10, 5)
자료형: {'회원번호': 'int64', '나이': 'float64', '지역': 'str', '가입일': 'str', '탈퇴': 'int64'}
빈칸 수: {'회원번호': 0, '나이': 1, '지역': 1, '가입일': 0, '탈퇴': 0}
중복 행: 1 개
지역 값: {'서울': 3, '부산': 2, '대구': 2, '서울시': 1, '서울 ': 1, nan: 1}
나이 범위: 0.0 ~ 999.0
정답 비율: {0: 0.7, 1: 0.3}
```

몇 줄 안 되는 코드로 문제가 줄줄이 나왔습니다. 하나씩 볼까요?

- **빈칸**: 나이와 지역에 하나씩 비어 있습니다. 이건 쉽게 보이는 문제예요.
- **숨은 빈칸**: 나이 범위가 0~999입니다. 999살이나 0살 회원은 없죠. 입력하는 사람이 "모름"을 999나 0으로 적어 둔 겁니다. `isna()`로는 절대 안 잡히는, 빈칸이 숫자인 척하는 경우예요.
- **표기 흔들림**: "서울", "서울시", "서울 "(끝에 빈칸)이 따로 세어졌습니다. 사람 눈에는 같은 곳인데 컴퓨터에게는 세 지역이에요.
- **중복 행**: 107번 회원이 두 번 들어 있습니다. 같은 기록을 두 번 붙여 넣은 실수죠.
- **자료형**: 가입일이 날짜가 아니라 `str`(글자)입니다. 날짜로 계산하려면 바꿔야 해요.
- **정답 비율**: 탈퇴한 회원이 30%입니다. 이 비율이 한쪽으로 크게 쏠려 있으면 2.8절에서 볼 문제가 생겨요.

이 가운데 어느 것도 모델을 학습시키다가 오류로 알려 주지 않는다는 점이 무섭습니다. 999살은 그냥 큰 숫자로 계산되고, "서울시"는 그냥 다른 지역으로 학습돼요. 그래서 데이터를 받으면 **모델보다 먼저** 이런 점검을 하는 습관이 중요합니다.

데이터 살펴보기 체크리스트

앞으로 새 데이터를 받으면 아래 표를 위에서부터 차례로 확인하세요. 이 장의 실습(2.10절)도 이 순서를 따릅니다.

확인할 것	코드	이런 게 보이면
크기	`df.shape`	생각보다 적거나 많으면 불러오기를 의심
자료형	`df.dtypes`	숫자여야 할 열이 `str`이면 숫자 안에 글자가 섞인 것
빈칸	`df.isna().sum()`	2.4절의 결측치 처리
숨은 빈칸	`df.describe()`의 최솟값·최댓값	0, -1, 999, 9999 같은 수상한 값
범주 값	`df[열].value_counts()`	표기 흔들림, 오타, 너무 드문 값
중복	`df.duplicated().sum()`	같은 기록이 두 번 들어감
정답 비율	`y.value_counts(normalize=True)`	한쪽이 아주 적으면 2.8절
너무 좋은 특성	정답과의 상관	2.3절의 누수 의심
시간 열	날짜 범위	2.9절처럼 시간 순서로 나눌지 결정

마지막 두 줄은 아직 무슨 뜻인지 모르겠죠? 이 장을 다 읽고 나면 왜 이 표에 들어 있는지 알게 됩니다.

2.2 학습·검증·테스트로 나누기

모의고사와 수능

고3 수험생을 떠올려 보세요. 교과서와 문제집으로 **공부**하고, **모의고사**로 실력을 점검하면서 공부 방법을 고치고, 마지막에 **수능**을 봅니다. 수능 문제는 시험 당일까지 아무도 미리 볼 수 없어요. 그래야 수능 점수가 "진짜 실력"을 말해 주니까요.

머신러닝에서 데이터를 셋으로 나누는 이유가 정확히 이것입니다.

나눈 데이터	비유	하는 일
학습 데이터(training set)	교과서와 문제집	모델이 이걸 보고 규칙을 배움
검증 데이터(validation set)	모의고사	어떤 모델·설정이 나은지 비교하고 고름
테스트 데이터(test set)	수능	마지막에 딱 한 번, 진짜 실력을 잼

1장에서 기출문제만 외운 학생 이야기로 과적합을 봤죠. 문제집 답을 통째로 외운 학생은 문제집 점수는 100점이지만 처음 보는 문제는 못 풉니다. 그래서 모델의 실력은 반드시 **학습에 쓰지 않은 데이터**로 재야 해요. 1장 실습에서도 와인 데이터를 이렇게 셋으로 나눴고, "테스트 데이터로 설정을 고르면 점수가 부풀려진다"고 했습니다. 이 절에서는 그 말이 정말인지, 얼마나 부푸는지를 실험으로 확인합니다. 먼저 나누는 방법부터요.

train_test_split으로 나누기

1장 1.6절에서 써 본 scikit-learn의 `train_test_split`은 데이터를 무작위로 섞은 뒤 정해 준 비율로 한 번 자릅니다. 1장 실습에서는 섞은 번호를 손으로 세 토막 냈는데, 이 함수로 셋으로 나누려면 두 번 자르면 돼요.

```
import numpy as np
from sklearn.model_selection import train_test_split

# 손님 500명 × 특성 2개 (예제용). 0열에 손님 번호를 넣어 두면 누가 어디로 갔는지 보인다
X = np.column_stack([np.arange(500), np.random.default_rng(0).normal(size=500)])
y = np.arange(500) % 2

# 1단계: 테스트용 20%를 먼저 떼어 금고에 넣는다
X_나머지, X_테스트, y_나머지, y_테스트 = train_test_split(
    X, y, test_size=0.2, random_state=42)
# 2단계: 남은 80% 가운데 25%(전체의 20%)를 검증용으로
X_학습, X_검증, y_학습, y_검증 = train_test_split(
    X_나머지, y_나머지, test_size=0.25, random_state=42)

print("학습/검증/테스트:", len(X_학습), len(X_검증), len(X_테스트))
```

```
print("테스트에 들어간 손님 번호:", X_테스트[:6, 0].astype(int))

_, X_다시, _, _ = train_test_split(X, y, test_size=0.2, random_state=42)
print("같은 씨앗으로 다시:", X_다시[:6, 0].astype(int))
_, X_다른, _, _ = train_test_split(X, y, test_size=0.2, random_state=7)
print("씨앗을 바꾸면:    ", X_다른[:6, 0].astype(int))
```

```
학습/검증/테스트: 300 100 100
테스트에 들어간 손님 번호: [361  73 374 155 104 394]
같은 씨앗으로 다시: [361  73 374 155 104 394]
씨앗을 바꾸면:     [308  13 414  32 460 155]
```

순서를 눈여겨보세요. **테스트 데이터를 맨 먼저 떼어 냅니다.** 수능 문제를 금고에 넣어 두고 시작하는 거예요. 그다음 남은 80%를 다시 학습과 검증으로 나눴습니다. 남은 80%의 25%는 전체의 20%이니, 결과가 300 : 100 : 100, 즉 60 : 20 : 20이 됐어요. 데이터가 수만 건 이상으로 많으면 검증과 테스트를 10%씩만 떼어도 충분합니다.

`train_test_split`은 넣어 준 순서대로 짝을 맞춰서 돌려줍니다. `X`와 `y`를 넣었으니 `X_나머지, X_테스트, y_나머지, y_테스트` 네 개가 나와요. 순서를 헷갈리면 정답이 엉뚱한 문제에 붙으니 이 모양을 그대로 외워 두세요.

`random_state=42`는 1장에서 `np.random.default_rng(42)`에 넣었던 것과 같은 씨앗입니다. 같은 씨앗으로 다시 나누면 361번, 73번 손님이 똑같이 테스트로 가고, 씨앗을 바꾸면 다른 손님들이 갑니다. 씨앗을 고정해야 오늘 돌린 실험과 내일 돌린 실험을 같은 시험지로 비교할 수 있어요.

시험지를 보고 고르면 생기는 일

이제 모의고사가 왜 필요한지, 1장에서 말로만 했던 것을 실험으로 확인해 보겠습니다. 정답이 **동전 던지기로 정해진** 데이터를 만들 거예요. 특성과 정답 사이에 아무 관계가 없으니, 세상의 어떤 모델도 이 데이터에서는 50%보다 잘 맞힐 수 없습니다. 여기서 후보 모델 50개를 만들고, 시험 데이터 점수가 가장 높은 것을 고르면 어떻게 될까요?

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

rng = np.random.default_rng(0)
```

```
# 정답이 동전 던지기로 정해진 데이터: 어떤 모델도 50%보다 잘할 수 없다
X_학습, y_학습 = rng.normal(size=(200, 10)), rng.integers(0, 2, 200)
X_시험, y_시험 = rng.normal(size=(100, 10)), rng.integers(0, 2, 100)
X_새, y_새 = rng.normal(size=(20000, 10)), rng.integers(0, 2, 20000)

기록 = []
for 번호 in range(50):                              # 후보 모델 50개
    열 = rng.choice(10, size=3, replace=False)       # 특성 3개를 골라 쓰는 모델
    k = int(rng.integers(1, 30))
    모델 = KNeighborsClassifier(n_neighbors=k).fit(X_학습[:, 열], y_학습)
    기록.append((모델.score(X_시험[:, 열], y_시험), 열, 모델))

시험점수, 열, 최고 = max(기록, key=lambda 행: 행[0])
print(f"시험지를 보며 고른 최고 모델의 시험 점수: {시험점수:.0%}")
print(f"같은 모델을 처음 보는 데이터 2만 개로 채점: {최고.score(X_새[:, 열], y_새):.1%}")
```

```
시험지를 보며 고른 최고 모델의 시험 점수: 61%
같은 모델을 처음 보는 데이터 2만 개로 채점: 50.1%
```

시험 점수 61%. 동전 던지기를 61% 맞히는 모델이라니, 뭔가 배운 것 같죠? 하지만 같은 모델을 처음 보는 데이터 2만 개로 채점하니 50.1%, 그냥 동전 던지기였습니다. 61%는 실력이 아니라 **운**이에요. 50개 중에는 우연히 이 시험지 100문제와 잘 맞은 모델이 있기 마련이고, 우리는 시험지를 보면서 바로 그 운 좋은 모델을 골랐을 뿐입니다.

이게 테스트 데이터를 여러 번 들여다보면 안 되는 이유입니다. 테스트 점수를 보고 모델을 고르고, 설정을 바꾸고, 다시 테스트 점수를 보는 순간, 테스트 데이터는 수능이 아니라 **이미 풀어 본 모의고사**가 돼요. 점수가 실제보다 좋게 나옵니다. 그래서 고르고 고치는 일은 검증 데이터로 하고, 테스트 데이터는 모든 결정이 끝난 뒤 **딱 한 번만** 봅니다.

Q

Q. 검증 데이터로 고르는 것도 결국 같은 문제 아닌가요? 맞습니다. 검증 데이터로 여러 번 고르면 검증 점수도 조금씩 부풀어요. 위 실험의 61%가 바로 그 현상입니다. 그래서 검증 점수는 "고르는 데" 쓰고, 최종 성적표는 한 번도 고르는 데 쓰지 않은 테스트 데이터로 매기는 거예요. 데이터가 적어서 검증 데이터를 따로 떼기 아까울 때는 학습 데이터를 여러 조각으로 돌려 가며 검증하는 **교차 검증**을 씁니다. 3장에서 자세히 다룹니다.

2.3 데이터 누수: 시험 문제가 새는 길

시험지가 미리 새면

수능 전날 문제지가 유출됐다고 해 봅시다. 그 해 점수는 아무 의미가 없죠. 머신러닝에서 테스트 데이터의 정보가 어떤 길로든 학습 과정에 흘러 들어가는 것을 **데이터 누수(data leakage)** 라고 합니다. 1장 1.8절에서 이름만 나왔던 그 누수예요. 앞 절의 "시험지를 보고 고르기"도 누수의 한 종류예요.

누수가 무서운 이유는 **오류가 나지 않는다**는 데 있습니다. 코드는 잘 돌고 점수는 오히려 좋아져요. 좋아진 점수를 보고 기뻐하다가, 실제 서비스에 내놓은 뒤에야 엉터리였다는 걸 알게 됩니다. 가장 흔한 누수는 **나누기 전에 전체 데이터로 무언가를 계산하는 것**이에요. 얼마나 크게 부풀 수 있는지 실험해 보겠습니다.

나누기 전에 특성을 고르면

이번에도 의미 없는 데이터입니다. 100명에 대해 아무 뜻 없는 특성을 5000개 만들고, 정답은 동전 던지기로 정했어요. 특성이 너무 많으니 "정답과 가장 관련 있어 보이는 특성 20개"만 골라 쓰기로 합니다. 차이는 **언제** 고르느냐 하나뿐이에요.

```
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)
X = rng.normal(size=(100, 5000))     # 100명 × 특성 5000개, 전부 의미 없는 난수
y = rng.integers(0, 2, size=100)     # 정답도 동전 던지기

# 잘못된 순서: 전체 데이터로 "정답과 관련 있어 보이는" 특성 20개를 고른 뒤 나눈다
골라 = SelectKBest(f_classif, k=20).fit_transform(X, y)
X_학습, X_시험, y_학습, y_시험 = train_test_split(골라, y, test_size=0.3, random_state=0)
모델 = LogisticRegression().fit(X_학습, y_학습)
print(f"누수 있음: 시험 점수 {모델.score(X_시험, y_시험):.0%}")

# 올바른 순서: 먼저 나누고, 학습 데이터만 보고 고른다
X_학습, X_시험, y_학습, y_시험 = train_test_split(X, y, test_size=0.3, random_state=0)
```

```
선택기 = SelectKBest(f_classif, k=20).fit(X_학습, y_학습)
모델 = LogisticRegression().fit(선택기.transform(X_학습), y_학습)
print(f"누수 없음: 시험 점수 {모델.score(선택기.transform(X_시험), y_시험):.0%}")
```

```
누수 있음: 시험 점수 80%
누수 없음: 시험 점수 50%
```

같은 데이터, 같은 모델인데 순서만 바꿨더니 80%와 50%가 나왔습니다. 정답은 동전 던지기이니 진짜 실력은 50%예요. 80%는 완전히 가짜입니다.

어디서 새었을까요? 특성 5000개 중에는 순전히 우연으로 정답과 비슷하게 오르내리는 것들이 있습니다. 첫 번째 방법은 **시험 데이터의 정답까지 보면서** 그런 우연한 특성 20개를 골랐어요. 그러니 고른 특성들은 시험 데이터의 정답과도 우연히 잘 맞는 상태가 되고, 모델은 그 우연을 실력인 척 써먹었습니다. 두 번째 방법은 학습 데이터만 보고 골랐기 때문에, 그 우연이 시험 데이터에는 통하지 않아서 정직하게 50%가 나왔어요.

`SelectKBest`는 정답과의 관련성을 재서 상위 k개 특성을 고르는 도구입니다. 1장에서 모델에는 `fit`과 `predict`라는 약속이 있다고 했죠. 데이터를 바꾸는 전처리 도구에는 `predict` 대신 `transform`이 있습니다.

- `fit(X, y)`: 데이터를 보고 무언가를 **배웁니다**(여기서는 어떤 특성이 좋은지).
- `transform(X)`: 배운 대로 데이터를 **바꿉니다**(여기서는 고른 특성만 남기기).
- `fit_transform(X, y)`: 둘을 한 번에 합니다.

누수를 막는 규칙은 이 약속으로 한 줄에 정리됩니다. **`fit`은 학습 데이터로만, `transform`은 어디에나.** 스케일러, 결측치 채우기, 인코더, 특성 선택기 모두 마찬가지예요. 이 장의 나머지 절에서도 이 규칙이 계속 반복됩니다.

정답을 품은 특성

두 번째로 흔한 누수는 **정답이 정해진 뒤에야 생기는 정보**가 특성에 섞여 드는 경우입니다. 통신사 이탈 예측에 "위약금" 열이 들어 있다고 해 봅시다. 위약금은 해지할 때 내는 돈이니, 이탈한 고객에게만 찍혀 있겠죠.

```
import numpy as np
import pandas as pd
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(1)
n = 1000
# 통신사 고객 기록 (예제용으로 만든 데이터)
고객 = pd.DataFrame({
    "가입개월": rng.integers(1, 60, n),
    "상담횟수": rng.poisson(1.5, n),
})
확률 = 1 / (1 + np.exp(-(-1.5 - 0.03 * 고객["가입개월"] + 0.5 * 고객["상담횟수"])))
고객["이탈"] = (rng.random(n) < 확률).astype(int)
# 해지할 때만 생기는 기록: 이탈한 고객에게만 위약금이 찍힌다
고객["위약금"] = np.where(고객["이탈"] == 1, rng.integers(1, 20, n) * 10000, 0)


def 점수_보기(특성: list[str]) -> float:
    """주어진 특성으로 학습하고 시험 점수를 돌려준다."""
    X_학습, X_시험, y_학습, y_시험 = train_test_split(
        고객[특성], 고객["이탈"], test_size=0.3, random_state=0)
    모델 = LogisticRegression(max_iter=1000).fit(X_학습, y_학습)
    return 모델.score(X_시험, y_시험)


print(f"가입개월·상담횟수만:  {점수_보기(['가입개월', '상담횟수']):.1%}")
print(f"위약금까지 넣으면:    {점수_보기(['가입개월', '상담횟수', '위약금']):.1%}")
print("정답과의 상관계수:")
print(고객.corr()["이탈"].drop("이탈").round(2).to_string())
```

```
가입개월·상담횟수만:  80.7%
위약금까지 넣으면:    100.0%
정답과의 상관계수:
가입개월   -0.18
상담횟수    0.19
위약금     0.87
```

위약금을 넣자마자 100%가 나왔습니다. 완벽해 보이지만 이 모델은 쓸모가 없어요. 이탈을 **미리** 알고 싶은 시점에는 위약금 기록이 아직 없기 때문입니다. "위약금을 낸 사람이 이탈했다"는 건 예측이 아니라 사후 확인이죠.

이런 누수는 상관계수로 의심해 볼 수 있습니다. 다른 특성들은 정답과의 상관이 0.2 안팎인데 위약금만 0.87이에요. **너무 좋은 특성이 보이면 반가워하기 전에 의심부터** 하세요. 그리고 특성마다 "예측해야 하는 그 순간에 이 값을 정말 알 수 있나?"를 물어보는 게 가장 확실한 점검법입니다. 병원 데이터의 "처방된 약", 대출 데이터의 "연체 독촉 횟수"처럼 결과가 난 뒤에 쌓이는 기록은 모두 같은 함정이에요.

`점수_보기` 함수에 `list[str]` 같은 타입 힌트를 붙인 건 1권 2장에서 본 그대로입니다. 함수 하나로 특성 목록만 바꿔 가며 같은 실험을 되풀이하니 비교가 깔끔해졌죠.

같은 사람이 양쪽에

세 번째 누수는 데이터 한 줄이 한 사람이 아닐 때 생깁니다. 환자 한 명이 네 번 검사를 받아서 기록이 네 줄 있다면, 줄 단위로 섞어 나눌 때 같은 환자의 기록이 학습과 시험 양쪽에 들어갈 수 있어요.

```
import numpy as np
from sklearn.model_selection import GroupShuffleSplit, train_test_split

rng = np.random.default_rng(0)
# 환자 50명이 각자 4번씩 검사받은 기록 200줄 (예제용)
환자번호 = np.repeat(np.arange(50), 4)
X = rng.normal(size=(200, 3))

_, _, 학습_환자, 시험_환자 = train_test_split(X, 환자번호, test_size=0.25, random_state=0)
겹침 = set(학습_환자) & set(시험_환자)
print(f"줄 단위로 섞어 나누기: 양쪽에 다 나온 환자 {len(겹침)}명")

나누개 = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=0)
학습_위치, 시험_위치 = next(나누개.split(X, groups=환자번호))
겹침 = set(환자번호[학습_위치]) & set(환자번호[시험_위치])
print(f"환자 단위로 나누기:    양쪽에 다 나온 환자 {len(겹침)}명")
print("학습/시험 줄 수:", len(학습_위치), len(시험_위치))
```

```
줄 단위로 섞어 나누기: 양쪽에 다 나온 환자 33명
환자 단위로 나누기:    양쪽에 다 나온 환자 0명
학습/시험 줄 수: 148 52
```

줄 단위로 섞으니 시험 데이터에 나온 환자 33명이 **모두** 학습 데이터에도 나왔습니다. 모델이 "이 환자의 다른 검사 기록"을 이미 본 상태로 시험을 치르는 셈이라, 처음 보는 환자에 대한 실력보다 점수가 좋게 나와요. `GroupShuffleSplit`에 `groups=환자번호`를 주면 **환자 단위로** 통째로 나누므로 겹치는 환자가 0명이 됩니다. 환자 수로 나누기 때문에 줄 수는 정확히 75 : 25가 아니라 148 : 52가 됐어요.

누수가 들어오는 길 정리

누수	예	막는 법
전처리 통계	전체 데이터의 평균으로 빈칸 채우기·스케일링	나눈 뒤 학습 데이터로만 `fit` (2.4, 2.6절)
특성 선택	전체 데이터로 좋은 특성 고르기	나눈 뒤 학습 데이터로만 고르기
시험지 들여다보기	테스트 점수로 모델·설정 고르기	고르기는 검증 데이터로, 테스트는 한 번만
정답을 품은 특성	위약금, 처방된 약	예측 시점에 알 수 있는 값인지 확인
같은 대상의 여러 기록	한 환자의 여러 검사	그룹 단위로 나누기 (`GroupShuffleSplit`)
중복 행	같은 줄이 학습·시험에 하나씩	나누기 전에 `drop_duplicates()`
미래 정보	내일 데이터로 어제를 예측	시간 순서로 자르기 (2.9절)

이 표의 첫 줄, 전처리 통계 누수는 앞의 특성 선택처럼 점수를 크게 부풀리지는 않는 경우가 많습니다. 그래도 "시험 데이터를 미리 봤다"는 점은 같아서, 데이터가 적거나 특이한 값이 섞여 있을 때는 무시 못 할 차이를 냅니다. 그리고 무엇보다 **습관**의 문제예요. 작은 누수를 아무렇지 않게 넘기는 사람이 큰 누수도 놓칩니다. 2.7절의 Pipeline은 이 규칙을 사람이 기억하지 않아도 지켜지게 만드는 도

구입니다.

2.4 결측치 처리

버릴까, 채울까

비어 있는 값, **결측치**를 다루는 기본 방법은 1권 20.3절에서 봤습니다. 행을 버리는 `dropna()`와 그럴듯한 값으로 채우는 `fillna()`였죠. 머신러닝에서는 여기에 두 가지 생각을 더해야 합니다. 하나는 "버리면 얼마나 잃는가", 다른 하나는 "채울 값을 어느 데이터로 정하는가"예요.

먼저 버리는 쪽입니다. 빈칸이 열마다 10%씩이라면 조금 버리는 셈일까요?

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
n = 1000
# 열 다섯 개가 각자 10%씩 비어 있는 표 (예제용)
표 = pd.DataFrame(rng.normal(size=(n, 5)), columns=["가", "나", "다", "라", "마"])
for 열 in 표.columns:
    표.loc[rng.random(n) < 0.10, 열] = np.nan

print("열마다 빈 비율:", 표.isna().mean().round(3).tolist())
print("한 칸이라도 빈 행 비율:", round(표.isna().any(axis=1).mean(), 3))
print("dropna() 후 남은 행:", len(표.dropna()), "/", n)
```

```
열마다 빈 비율: [0.107, 0.106, 0.092, 0.101, 0.097]
한 칸이라도 빈 행 비율: 0.409
dropna() 후 남은 행: 591 / 1000
```

열 하나하나는 10% 정도만 비었는데, **한 칸이라도 빈 행**은 41%나 됩니다. 비는 칸이 행마다 제각각이라서, 다섯 열 중 하나라도 빈 행이 꽤 많은 거예요. `dropna()` 한 줄에 데이터 1000줄 중 409줄이 사라졌습니다. 열이 수십 개인 실제 데이터라면 남는 행이 거의 없을 수도 있어요. 그래서 머신러닝에서는 보통 **버리기보다 채우기**를 먼저 생각합니다. 정답(레이블)이 빈 행은 배울 거리가 없으니 버리는 게 맞지만요.

SimpleImputer: 학습 데이터로 배운 값으로 채우기

채울 값은 무엇으로 정할까요? 1권 20장에서는 `df["나이"].fillna(df["나이"].mean())`처럼 그 자리에서 평균을 냈습니다. 그런데 2.3절의 규칙을 떠올려 보세요. 전체 데이터의 평균은 시험 데이터를 본 값이에요. 채울 값은 **학습 데이터에서 배워서**, 학습·검증·시험 데이터에 똑같이 써야 합니다. scikit-learn의 `SimpleImputer`가 바로 그 일을 해요.

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer

# 연소득(만 원). 학습 데이터에 아주 큰 값이 하나 섞여 있다 (예제용)
학습 = pd.DataFrame({"연소득": [3200, 4100, np.nan, 3800, 4500, 250000, np.nan, 3600]})
시험 = pd.DataFrame({"연소득": [np.nan, 5000]})

평균_채우기 = SimpleImputer(strategy="mean").fit(학습)
중앙값_채우기 = SimpleImputer(strategy="median").fit(학습)
print("학습 데이터로 배운 평균:", 평균_채우기.statistics_.round(1))
print("학습 데이터로 배운 중앙값:", 중앙값_채우기.statistics_)
print("시험 데이터 채우기(중앙값):", 중앙값_채우기.transform(시험).ravel())

# 비어 있었다는 사실도 함께 남기기
표시 = SimpleImputer(strategy="median", add_indicator=True)
표시.set_output(transform="pandas")          # 결과를 DataFrame으로 받기
print(표시.fit(학습).transform(시험))

# 글자 열: 가장 흔한 값 또는 정해 둔 값으로
지역 = pd.DataFrame({"지역": ["서울", "부산", np.nan, "서울", "대구"]})
print(SimpleImputer(strategy="most_frequent").fit_transform(지역).ravel())
print(SimpleImputer(strategy="constant", fill_value="모름").fit_transform(지역).ravel())
```

```
학습 데이터로 배운 평균: [44866.7]
학습 데이터로 배운 중앙값: [3950.]
시험 데이터 채우기(중앙값): [3950. 5000.]
      연소득  missingindicator_연소득
0  3950.0                  1.0
```

```
1  5000.0                  0.0
['서울' '부산' '서울' '서울' '대구']
['서울' '부산' '모름' '서울' '대구']
```

결과에서 볼 점이 셋 있습니다.

첫째, `fit`이 끝나면 배운 값이 `statistics_`에 저장됩니다. 1장에서 본 대로 scikit-learn에서 `fit`으로 배운 값은 **이름 끝에 밑줄**이 붙은 속성에 들어가요. 4장에서 만날 PyTorch에서는 함수 이름 끝의 밑줄(`add_` 같은)이 "원본을 바로 바꾼다"는 전혀 다른 뜻이니, 그때 헷갈리지 마세요. 시험 데이터를 `transform`하면 시험 데이터의 평균이 아니라 학습 데이터에서 배운 3950이 들어갑니다.

둘째, **평균과 중앙값이 크게 다릅니다.** 연소득에 25억 원(250000만 원)인 사람이 한 명 섞여 있어서 평균이 약 4억 5천만 원(44866.7만 원)으로 치솟았어요. 이 값으로 빈칸을 채우면 평범한 사람이 하루아침에 부자가 됩니다. 1권 20장에서 "평균과 중앙값이 많이 다르면 한쪽으로 쏠린 값이 있다"고 했던 바로 그 상황이에요. 숫자 열은 **중앙값(median)** 으로 채우는 쪽이 대체로 안전합니다.

셋째, `add_indicator=True`를 주면 "여기가 원래 비어 있었다"는 표시 열(`missingindicator_연소득`)이 하나 더 붙습니다. 첫 번째 시험 손님은 소득을 채워 넣었지만 원래 비어 있었다는 사실(1.0)이 남아요. 이게 왜 필요한지는 바로 다음에 봅니다.

글자 열은 평균을 낼 수 없으니 가장 흔한 값(`most_frequent`)이나 정해 둔 값(`constant`)으로 채웁니다. "모름"으로 채우면 2.5절의 인코딩에서 "모름"이 하나의 범주로 취급돼요.

`set_output(transform="pandas")`는 결과를 NumPy 배열 대신 DataFrame으로 받는 설정입니다. 열 이름이 붙어서 무엇이 어디로 갔는지 보기 좋아요.

비어 있다는 것도 정보다

설문지에서 소득 칸을 비워 두는 사람은 어떤 사람일까요? 무작위로 비우는 게 아니라, 적기 싫은 이유가 있는 경우가 많습니다. 그렇다면 "비어 있다"는 사실 자체가 정답을 맞히는 단서가 될 수 있어요.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(3)
n = 2000
# 대출 신청 기록 (예제용): 형편이 어려운 사람일수록 소득 칸을 비워 두는 경향이 있다고 하자
```

```
형편 = rng.normal(size=n)
연체 = (rng.random(n) < 1 / (1 + np.exp(2 + 1.2 * 형편))).astype(int)
연소득 = np.where(rng.random(n) < 1 / (1 + np.exp(1.5 + 1.5 * 형편)), np.nan,
                  (4000 + 1200 * 형편).round())
신청 = pd.DataFrame({"연소득": 연소득, "연체": 연체})

비움 = 신청["연소득"].isna()
print(f"소득 칸을 비운 비율: {비움.mean():.1%}")
print(f"소득을 적은 사람의 연체율: {신청.loc[~비움, '연체'].mean():.1%}")
print(f"소득을 비운 사람의 연체율: {신청.loc[비움, '연체'].mean():.1%}")
```

```
소득 칸을 비운 비율: 24.9%
소득을 적은 사람의 연체율: 13.5%
소득을 비운 사람의 연체율: 30.9%
```

소득을 적은 사람의 연체율은 13.5%인데, 비운 사람은 30.9%로 두 배가 넘습니다. 이 데이터는 그렇게 되도록 만든 예제지만, 실제 데이터에서도 흔히 보이는 모습이에요. 이 빈칸을 중앙값으로 채우기만 하면 "소득을 비웠다"는 강력한 단서가 사라져 버립니다. 앞에서 본 `add_indicator=True`가 그 단서를 지켜 주는 장치예요. 빈칸이 많은 열이라면 표시 열을 함께 남겨 두는 게 좋습니다.

방법	언제	주의할 점
행 버리기	정답이 빈 행, 빈칸이 아주 드물 때	생각보다 많이 사라짐
열 버리기	열 대부분이 비었을 때	남은 값이 쓸모 있는지 먼저 확인
중앙값	숫자 열 기본값	튀는 값에 강함
평균	고르게 퍼진 숫자 열	튀는 값 하나에 끌려감
최빈값·"모름"	글자(범주) 열	"모름"은 하나의 범주가 됨
표시 열 추가	비어 있는 이유가 있을 법할 때	열이 하나씩 늘어남

2.5 범주형 인코딩: 글자를 숫자로

서울=1, 부산=2로 하면 안 되나요?

"서울", "부산" 같은 값을 **범주형(categorical)** 값이라고 합니다. 모델은 숫자만 먹으니 숫자로 바꿔줘야 해요. 가장 먼저 떠오르는 방법은 번호를 매기는 거겠죠. 서울=1, 부산=2, 대구=3처럼요.

그런데 모델은 이 숫자를 **진짜 숫자**로 받아들입니다. 부산(2)은 서울(1)과 대구(3)의 정확히 가운데이고, 대구는 서울의 세 배라고 생각해요. "서울과 대구의 평균은 부산"이라는 이상한 계산이 가능해지는 거죠. 지역 사이에는 그런 크기나 순서가 없습니다. 그래서 순서가 없는 범주는 다른 방법으로 바꿉니다.

원-핫 인코딩: 범주마다 스위치 하나

원-핫 인코딩(one-hot encoding) 은 범주마다 열을 하나씩 만들고, 해당하는 열만 1, 나머지는 0으로 채우는 방법입니다. 방마다 전등 스위치가 하나씩 있고, 지금 있는 방의 스위치만 켜는 모습을 떠올리면 돼요.

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

학습 = pd.DataFrame({"지역": ["서울", "부산", "대구", "서울", "부산"]})
시험 = pd.DataFrame({"지역": ["부산", "광주"]})      # 광주는 학습 때 못 본 값

인코더 = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
인코더.fit(학습)
print("배운 범주:", 인코더.categories_[0].tolist())
print("새 열 이름:", 인코더.get_feature_names_out().tolist())
print(인코더.transform(학습))
print("시험 데이터:")
print(인코더.transform(시험))
```

```
배운 범주: ['대구', '부산', '서울']
새 열 이름: ['지역_대구', '지역_부산', '지역_서울']
[[0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]]
```

```
시험 데이터:
[[0. 1. 0.]
 [0. 0. 0.]]
```

학습 데이터에서 범주 세 개를 배우고, 열 세 개(`지역_대구`, `지역_부산`, `지역_서울`)를 만들었습니다. 서울 손님은 `[0, 0, 1]`, 부산 손님은 `[0, 1, 0]`이에요. 이제 어느 두 지역 사이의 거리도 똑같아서, 지역 사이에 가짜 순서가 생기지 않습니다. 범주 순서가 가나다순인 건 scikit-learn이 배운 범주를 정렬해 두기 때문이에요.

시험 데이터에 학습 때 못 본 "광주"가 나왔습니다. `handle_unknown="ignore"`를 줬기 때문에 오류 없이 **모든 스위치를 끈** `[0, 0, 0]`이 됐어요. 이 설정이 없으면 모르는 값을 만나는 순간 오류가 나서, 서비스 중에 새 지역 손님이 들어오면 예측이 멈춰 버립니다. 실무에서는 거의 항상 켜 두세요.

`sparse_output=False`는 결과를 보기 좋은 일반 배열로 받는 설정입니다. 원-핫 결과는 대부분이 0이라서 기본값은 0을 저장하지 않는 **희소 행렬(sparse matrix)** 로 돌려줘요. 범주가 수천 개일 때 메모리를 크게 아껴 주니, 화면에 찍어 볼 때가 아니면 기본값 그대로 써도 됩니다.

pd.get_dummies를 조심하세요

pandas에도 원-핫 인코딩을 해 주는 `pd.get_dummies`가 있습니다. 1권 20장에서는 안 썼지만 인터넷 예제에 자주 나와요. 분석할 때는 편하지만 머신러닝에서는 함정이 있습니다.

```
import pandas as pd

학습 = pd.DataFrame({"지역": ["서울", "부산", "대구", "서울"]})
시험 = pd.DataFrame({"지역": ["부산", "광주"]})

print("학습:", pd.get_dummies(학습).columns.tolist())
print("시험:", pd.get_dummies(시험).columns.tolist())
```

```
학습: ['지역_대구', '지역_부산', '지역_서울']
시험: ['지역_광주', '지역_부산']
```

학습 데이터와 시험 데이터에 따로 쓰니 **열이 달라졌습니다.** 학습 데이터에는 대구·부산·서울 세 열, 시험 데이터에는 광주·부산 두 열이에요. 모델은 학습 때 본 열 순서 그대로 입력을 기대하므로, 이 시험 데이터는 넣을 수도 없고 억지로 맞춰 넣으면 엉뚱한 열끼리 짝이 맞습니다. `get_dummies`는

`fit`(배우기)과 `transform`(바꾸기)이 나뉘어 있지 않아서, 받은 데이터만 보고 그때그때 열을 만들기 때문이에요. 학습 때 배운 범주를 기억해 두는 `OneHotEncoder`를 쓰세요.

순서가 있는 범주

모든 범주에 순서가 없는 건 아닙니다. 만족도 "낮음 < 보통 < 높음", 옷 크기 "S < M < L", 학년처럼 **순서가 있는 범주**는 번호를 매기는 게 오히려 자연스러워요. 이런 값은 `OrdinalEncoder`로 바꿉니다. 그런데 하나 조심할 게 있어요.

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

만족도 = pd.DataFrame({"만족도": ["보통", "높음", "낮음", "보통", "높음"]})

알아서 = OrdinalEncoder().fit(만족도)
print("순서를 안 정하면:", 알아서.categories_[0].tolist())
print(알아서.transform(만족도).ravel())

순서대로 = OrdinalEncoder(categories=[["낮음", "보통", "높음"]],
                          handle_unknown="use_encoded_value", unknown_value=-1)
순서대로.fit(만족도)
print("순서를 정해 주면:", 순서대로.categories_[0].tolist())
print(순서대로.transform(만족도).ravel())
print("못 본 값:", 순서대로.transform(pd.DataFrame({"만족도": ["매우 높음"]})).ravel())
```

```
순서를 안 정하면: ['낮음', '높음', '보통']
[2. 1. 0. 2. 1.]
순서를 정해 주면: ['낮음', '보통', '높음']
[1. 2. 0. 1. 2.]
못 본 값: [-1.]
```

순서를 알려 주지 않으면 `OrdinalEncoder`는 가나다순으로 번호를 매깁니다. 그 결과 낮음=0, **높음=1, 보통=2**가 됐어요. "보통"이 "높음"보다 큰 숫자를 받은 겁니다. 컴퓨터는 낮음·보통·높음의 뜻을 모르니 글자 순서로 정렬할 수밖에 없어요. `categories=[["낮음", "보통", "높음"]]`처럼 **순서를 직접 적어 주면** 낮음=0, 보통=1, 높음=2로 제대로 매겨집니다. 대괄호가 두 겹인 이유는 열이 여러 개일 때 열마다 순서 목록을 하나씩 주기 때문이에요.

학습 때 없던 "매우 높음"은 `handle_unknown="use_encoded_value", unknown_value=-1` 설정 덕분에 -1을 받았습니다. 원-핫의 `"ignore"`와 같은 역할이에요.

종류가 너무 많을 때

동네 이름이나 상품 번호처럼 범주가 수백, 수천 가지라면 원-핫 인코딩은 열을 그만큼 만들어 냅니다. 대부분은 한두 번밖에 안 나와서 배울 거리도 거의 없는데요. 이럴 때는 드문 범주를 하나로 묶습니다.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

rng = np.random.default_rng(0)
# 동네 이름 200가지가 섞인 주소 1000건 (예제용). 앞 번호일수록 흔하다
비율 = 1 / np.arange(1, 201)
동네 = pd.DataFrame({"동네": rng.choice([f"동{i:03d}" for i in range(200)], 1000,
                                        p=비율 / 비율.sum())})
print("서로 다른 동네:", 동네["동네"].nunique(), "곳")

전부 = OneHotEncoder(handle_unknown="ignore").fit(동네)
print("그냥 원-핫:", len(전부.get_feature_names_out()), "열")

묶기 = OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=20).fit(동네)
이름 = 묶기.get_feature_names_out()
print("20번 미만은 묶기:", len(이름), "열 →", 이름[:3].tolist(), "...", 이름[-1])
```

```
서로 다른 동네: 163 곳
그냥 원-핫: 163 열
20번 미만은 묶기: 9 열 → ['동네_동000', '동네_동001', '동네_동002'] ... 동네_infrequent_sklearn
```

동네가 163곳이라 그냥 원-핫을 하면 163열이 생깁니다. `min_frequency=20`을 주면 20번 미만으로 나온 동네는 모두 `infrequent_sklearn`("드문 것들")이라는 열 하나로 묶여서 9열로 줄어들어요. `handle_unknown="infrequent_if_exist"`는 처음 보는 동네도 이 "드문 것들" 칸에 넣으라는 뜻입니다. 처음 본 동네는 드문 동네와 사정이 비슷하니 자연스러운 선택이죠. 범주를 몇 개까지만 남길지 정하는 `max_categories`도 있습니다. 13장에서 단어를 숫자로 바꿀 때는 범주가 수만 개가 되는

데, 그때는 원-핫 대신 **임베딩**이라는 전혀 다른 방법을 씁니다.

2.6 스케일링: 단위 맞추기

눈금이 다르면 생기는 일

1장 1.8절에서 나이(세)와 연소득(만 원)처럼 **눈금**이 다른 특성을 최근접 이웃에 그대로 넣었다가 정확도가 동전 던지기 수준으로 떨어지는 걸 봤습니다. 거리를 재는 모델은 큰 숫자를 가진 특성만 보게 되니까요. 그럼 거리를 재지 않는 모델은 괜찮을까요?

이번에는 이 장 첫머리에서 써 본 **로지스틱 회귀**로 확인해 보겠습니다. 로지스틱 회귀는 특성마다 가중치를 곱해 더하고, 정답에 가까워지도록 가중치를 **조금씩 여러 번 고쳐 가며** 배우는 모델이에요. 5장에서 손으로 해 볼 경사하강법과 비슷하게, 고치고 확인하기를 되풀이하는 방식입니다. 데이터는 scikit-learn에 들어 있는 유방 종양 측정값이라 따로 내려받지 않아도 돼요. 미국 위스콘신 대학 병원에서 모은 Breast Cancer Wisconsin (Diagnostic) 데이터로, UCI 머신러닝 저장소에 CC BY 4.0으로 공개되어 있습니다.

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 유방 종양 569건의 측정값 30가지, 양성/악성 (scikit-learn 내장 데이터)
X, y = load_breast_cancer(return_X_y=True, as_frame=True)
print(X[["mean area", "mean smoothness"]].describe().loc[["min", "max"]])
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

스케일러 = StandardScaler().fit(X_학습)      # 학습 데이터로만 평균·표준편차를 배운다
for 이름, 학습, 시험 in [("그대로", X_학습, X_시험),
                         ("표준화", 스케일러.transform(X_학습), 스케일러.transform(X_시험))]:
    모델 = LogisticRegression(max_iter=1000).fit(학습, y_학습)   # 최대 1000번까지 고쳐 본다
    print(f"{이름}: 고친 횟수 {모델.n_iter_[0]:4d}번, 시험 정확도 {모델.score(시험, y_시험):.1%}")
```

```
     mean area  mean smoothness
min      143.5          0.05263
max     2501.0          0.16340
그대로: 고친 횟수 1000번, 시험 정확도 93.7%
표준화: 고친 횟수   18번, 시험 정확도 95.8%
```

종양 면적(`mean area`)은 143.5~2501, 표면 매끄러움(`mean smoothness`)은 0.05~0.16입니다. 수만 배 차이가 나요. 이대로 학습하니 허락한 최대 횟수 1000번을 다 쓰고도 끝을 보지 못했고, 시험 정확도는 93.7%였습니다. 표준화하고 나니 **18번** 만에 학습이 끝났고 정확도도 95.8%로 올랐어요.

왜 이렇게 차이가 날까요? 산에서 가장 낮은 골짜기를 찾아 걸어 내려간다고 생각해 보세요. 눈금이 고르면 골짜기가 둥근 그릇 모양이라 곧장 바닥으로 내려갈 수 있습니다. 눈금이 크게 다르면 골짜기가 한쪽으로만 길쭉한 협곡이 돼서, 좁은 쪽 벽에 부딪히며 지그재그로 한참을 헤매야 해요. 고치는 보폭을 한 특성에 맞추면 다른 특성에는 너무 크거나 너무 작거든요. 이 그림은 5장 5.10절에서 경사하강법을 직접 짜서 표준화하지 않은 기온을 넣어 보면 훨씬 또렷해집니다. 신경망도 같은 방식으로 배우니, 2권 뒷부분에서 신경망에 넣는 숫자를 늘 표준화하는 이유가 바로 이것입니다.

"반복 1000번을 다 썼다"는 건 scikit-learn이 `ConvergenceWarning`이라는 경고로도 알려 줍니다. 이 경고를 보면 반복 횟수를 늘리기 전에 **스케일링을 했는지부터** 확인하세요.

세 가지 스케일러

자주 쓰는 스케일러는 셋입니다. 마지막 날만 방송을 타서 방문자가 확 튄 데이터에 모두 적용해 보겠습니다.

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler, RobustScaler, StandardScaler

# 하루 방문자 수 (예제용). 마지막 날만 방송을 타서 크게 튀었다
방문자 = np.array([[120.0], [135.0], [128.0], [142.0], [131.0], [1500.0]])

for 스케일러 in [StandardScaler(), MinMaxScaler(), RobustScaler()]:
    결과 = 스케일러.fit_transform(방문자).ravel()
    print(f"{type(스케일러).__name__:<15}", np.round(결과, 2))
```

```
StandardScaler  [-0.47 -0.44 -0.45 -0.43 -0.45  2.24]
MinMaxScaler    [0.   0.01 0.01 0.02 0.01 1.  ]
RobustScaler    [ -1.13   0.17  -0.43   0.78  -0.17 118.87]
```

- `StandardScaler`(**표준화**): 평균을 빼고 표준편차로 나눕니다. 결과는 평균 0, 표준편차 1 근처에 모여요. 1장 1.8절에서 손으로 했던 계산 그대로입니다. 가장 무난한 기본 선택이에요.
- `MinMaxScaler`(**정규화**): 최솟값을 0, 최댓값을 1로 맞춥니다. 범위가 분명해서 이해하기 쉽지만, 튀는 값 하나가 최댓값을 차지하면 나머지가 0 근처에 몰려 버려요. 위 결과에서 평범한 다섯 날이 모두 0~0.02에 뭉쳤습니다. 사진의 밝기값(0~255)처럼 **범위가 정해져 있는** 데이터에 잘 맞아요.
- `RobustScaler`: 평균·표준편차 대신 중앙값과 사분위 범위로 맞춥니다. 튀는 값의 영향을 덜 받아서 평범한 다섯 날이 -1.13~0.78 사이로 고르게 퍼졌고, 튀는 날은 118.87로 "확실히 튄다"는 게 드러나요. 튀는 값이 많은 데이터에 씁니다.

Q

Q. 모든 모델에 스케일링이 필요한가요? 아니요. 3장에서 볼 결정 트리와 랜덤 포레스트는 "이 값이 기준보다 큰가"만 물어서 눈금이 상관없습니다. 면적을 cm^2로 재든 mm^2로 재든 기준값만 같이 바뀔 뿐 나누는 방식은 똑같거든요. 반면 거리를 쓰는 최근접 이웃, 가중치를 조금씩 고쳐 배우는 선형 모델과 신경망에는 거의 필수예요. 어느 쪽인지 헷갈리면 표준화를 해 두세요. 필요 없는 모델에 해도 손해는 거의 없습니다.

시험 데이터는 학습 데이터의 자로 잽니다

스케일러도 `fit`으로 무언가를 배웁니다. 표준화는 평균과 표준편차를, 정규화는 최솟값과 최댓값을요. 그러니 2.3절의 규칙이 그대로 적용됩니다.

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler

학습 = np.array([[20.0], [35.0], [50.0], [65.0]])     # 학습 데이터의 나이
시험 = np.array([[30.0], [80.0]])                     # 시험 데이터의 나이

스케일러 = MinMaxScaler().fit(학습)                    # 최솟값 20, 최댓값 65를 배운다
print("배운 범위:", 스케일러.data_min_, "~", 스케일러.data_max_)
print("학습:", 스케일러.transform(학습).ravel().round(3))
print("시험:", 스케일러.transform(시험).ravel().round(3))
```

```
잘못 = MinMaxScaler().fit(np.vstack([학습, 시험]))     # 시험 데이터까지 보고 배우면
print("잘못 배운 범위:", 잘못.data_min_, "~", 잘못.data_max_)
print("학습:", 잘못.transform(학습).ravel().round(3))
```

```
배운 범위: [20.] ~ [65.]
학습: [0.    0.333 0.667 1.   ]
시험: [0.222 1.333]
잘못 배운 범위: [20.] ~ [80.]
학습: [0.   0.25 0.5  0.75]
```

학습 데이터로 배운 범위는 20~65살입니다. 시험 데이터의 80살은 이 범위를 벗어나니 1.333이 나왔어요. "정규화하면 0~1 사이"라고 배웠는데 1이 넘으니 잘못된 것 같죠? 아닙니다. 이게 **맞는** 결과예요. 학습 데이터로 만든 자로 시험 데이터를 재면, 처음 보는 큰 값은 자의 끝을 넘어갈 수 있습니다. 실제 서비스에서도 학습 때보다 나이 많은 손님이 언제든 올 수 있고요.

반대로 시험 데이터까지 넣고 `fit`하면 자 자체가 20~80으로 바뀌고, 학습 데이터의 65살이 1이 아니라 0.75가 됩니다. 시험 데이터를 몰래 보고 자를 만든 거예요. 결과가 0~1에 예쁘게 들어간다고 해서 맞는 방법이 아닙니다.

2.7 ColumnTransformer와 Pipeline으로 묶기

열마다 다른 손질

지금까지 손질 도구를 하나씩 따로 써 봤습니다. 실제 데이터에는 숫자 열, 범주 열, 순서 있는 범주 열이 섞여 있고, 열마다 필요한 손질이 다르죠. 숫자 열은 "빈칸 채우기 → 표준화", 범주 열은 "빈칸 채우기 → 원-핫"처럼요.

요리로 치면 재료마다 손질법이 다른 겁니다. 감자는 껍질을 벗겨 깍둑썰기, 대파는 어슷썰기, 고기는 핏물을 빼고 한입 크기로요. scikit-learn에서 이 "재료별 손질법"을 정리하는 도구가 둘 있습니다.

- `make_pipeline(A, B)`: A를 한 다음 B를 하는 **순서**를 묶습니다. "빈칸 채우기 → 표준화"처럼요.
- `ColumnTransformer`: **어느 열에 어떤 손질을** 할지 짝지어서, 결과를 옆으로 이어 붙입니다.

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler

# 아파트 매물 (예제용으로 만든 데이터)
매물 = pd.DataFrame({
    "면적": [84.9, 59.8, np.nan, 114.5, 84.9, 49.5],
    "층": [12, 3, 25, 7, np.nan, 15],
    "구": ["마포", "강서", "마포", np.nan, "송파", "강서"],
    "상태": ["보통", "좋음", "나쁨", "좋음", "보통", np.nan],
})

숫자_열 = ["면적", "층"]
범주_열 = ["구"]
순서_열 = ["상태"]

숫자_처리 = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
범주_처리 = make_pipeline(SimpleImputer(strategy="constant", fill_value="모름"),
                          OneHotEncoder(handle_unknown="ignore", sparse_output=False))
순서_처리 = make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OrdinalEncoder(categories=[["나쁨", "보통", "좋음"]]))

전처리 = ColumnTransformer([
    ("숫자", 숫자_처리, 숫자_열),
    ("범주", 범주_처리, 범주_열),
    ("순서", 순서_처리, 순서_열),
])
전처리.set_output(transform="pandas")
결과 = 전처리.fit_transform(매물)
print(결과.round(2).to_string())
```

```
   숫자__면적  숫자__층  범주__구_강서  범주__구_마포  범주__구_모름  범주__구_송파  순서__상태
0    0.25  -0.05       0.0       1.0       0.0       0.0     1.0
1   -0.96  -1.36       1.0       0.0       0.0       0.0     2.0
2    0.25   1.84       0.0       1.0       0.0       0.0     0.0
3    1.67  -0.78       0.0       0.0       1.0       0.0     2.0
4    0.25  -0.05       0.0       0.0       0.0       1.0     1.0
5   -1.45   0.39       1.0       0.0       0.0       0.0     1.0
```

`ColumnTransformer`에는 `(이름, 손질법, 열 목록)` 세 개짜리 튜플을 목록으로 줍니다. 결과 표를 보면 열 이름 앞에 `숫자__`, `범주__`, `순서__`처럼 우리가 붙인 이름이 달려 있어서 어느 손질을 거쳐 나온 열인지 바로 알 수 있어요.

결과에서 확인할 것들입니다.

- 비어 있던 면적(2번 행)은 중앙값으로 채워진 뒤 표준화돼서 0.25가 됐습니다. 층(4번 행)도 마찬가지예요.
- 구가 비어 있던 3번 행은 "모름"으로 채워졌고, 원-핫 결과에 `구_모름` 열이 생겼습니다.
- 상태는 나쁨=0, 보통=1, 좋음=2로 바뀌었습니다. 비어 있던 5번 행은 가장 흔한 값으로 채워졌는데, "보통"과 "좋음"이 두 번씩으로 같아서 정렬했을 때 앞에 오는 "보통"(1)이 골라졌어요.

4개였던 열이 7개가 됐습니다. 원-핫 인코딩이 구 하나를 네 열로 펼쳤기 때문이에요.

Pipeline: 손질부터 요리까지 한 번에

이제 마지막 단계입니다. 손질(`ColumnTransformer`)과 요리(모델)를 `Pipeline` 하나로 묶으면, 모델 하나처럼 `fit`과 `predict`를 부를 수 있어요.

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
rng = np.random.default_rng(0)
n = 400
# 헬스장 회원 (예제용): 방문이 뜸하고 나이가 많을수록 재등록을 안 한다고 하자
회원 = pd.DataFrame({
    "나이": rng.integers(20, 65, n).astype(float),
    "주간방문": rng.poisson(2.5, n).astype(float),
    "지점": rng.choice(["강남", "홍대", "잠실"], n),
})
확률 = 1 / (1 + np.exp(-(1.0 - 0.04 * (회원["나이"] - 40) + 0.6 * (회원["주간방문"] - 2.5))))
회원["재등록"] = (rng.random(n) < 확률).astype(int)
회원.loc[rng.random(n) < 0.1, "나이"] = np.nan          # 일부는 나이를 안 적었다

X, y = 회원.drop(columns="재등록"), 회원["재등록"]
X_학습, X_시험, y_학습, y_시험 = train_test_split(X, y, test_size=0.25, random_state=0)

전처리 = ColumnTransformer([
    ("숫자", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()),
     ["나이", "주간방문"]),
    ("범주", OneHotEncoder(handle_unknown="ignore"), ["지점"]),
])
모델 = Pipeline([("전처리", 전처리), ("분류", LogisticRegression())])

모델.fit(X_학습, y_학습)            # 전처리도 모델도 학습 데이터로만 배운다
print(f"시험 정확도: {모델.score(X_시험, y_시험):.1%}")

새_회원 = pd.DataFrame({"나이": [np.nan, 58.0], "주간방문": [4.0, 0.0], "지점": ["판교", "강남"]})
print("새 회원 재등록 확률:", 모델.predict_proba(새_회원)[:, 1].round(2))
print("배운 중앙값:", 모델.named_steps["전처리"].named_transformers_["숫자"][0].statistics_)
```

```
시험 정확도: 77.0%
새 회원 재등록 확률: [0.84 0.4 ]
배운 중앙값: [44.  2.]
```

`Pipeline`에는 `(이름, 단계)` 튜플을 순서대로 줍니다. `모델.fit(X_학습, y_학습)` 한 줄이 안에서 이런 일을 해요.

1. 전처리기가 학습 데이터로 `fit`합니다(중앙값, 평균·표준편차, 지점 목록을 배움).

2. 배운 대로 학습 데이터를 `transform`합니다.
3. 바뀐 데이터로 분류 모델이 `fit`합니다.

그리고 **`모델.score(X_시험, y_시험)`**이나 **`predict_proba(새_회원)`**을 부르면 시험·새 데이터에는 **`transform`만** 하고 모델에 넘깁니다. 2.3절의 규칙 "`fit`은 학습 데이터로만, `transform`은 어디에나"가 코드 구조 자체로 지켜지는 거예요. 사람이 매번 기억할 필요가 없습니다.

새 회원 두 명은 원본 모양 그대로 넣었습니다. 첫 번째 회원은 나이가 비어 있고 지점도 학습 때 없던 "판교"인데, 파이프라인이 알아서 중앙값으로 채우고 모르는 지점은 모든 스위치를 끈 채로 처리해서 확률 0.84를 내놓았어요. 주 4회나 오는 회원이니 재등록할 가능성이 높다고 본 거죠. 마지막 줄처럼 `named_steps`와 `named_transformers_`로 안을 들여다보면, 학습 데이터에서 배운 나이 중앙값 44와 주간방문 중앙값 2가 그대로 들어 있습니다.

Pipeline으로 묶으면 좋은 점은 누수 방지만이 아닙니다.

- **배포가 쉬워집니다.** 전처리와 모델이 한 덩어리라서, 파이프라인 하나만 저장하면 서비스에서 원본 데이터를 그대로 넣을 수 있어요(모델 저장은 3장).
- **실수가 줄어듭니다.** 학습 때와 예측 때 전처리를 따로 짜다가 순서나 설정이 어긋나는 일이 없습니다.
- **교차 검증이 정직해집니다.** 3장의 교차 검증은 데이터를 여러 번 나눠 가며 학습하는데, 파이프라인을 넘기면 조각마다 전처리도 새로 `fit`해서 누수가 생기지 않아요.

Q

Q. 딥러닝에서도 이 파이프라인을 쓰나요? 표 형태 데이터를 신경망에 넣을 때는 이 장의 `ColumnTransformer`로 전처리한 결과를 텐서로 바꿔 쓰는 경우가 많습니다. 4장에서 배울 `torch.from_numpy(...).float()`로요. 사진이나 글처럼 모양이 다른 데이터는 PyTorch 쪽 도구로 전처리하는데, 7장의 Dataset과 transforms가 그 역할을 합니다. 도구는 달라도 "학습 데이터로 배운 값으로 모든 데이터를 똑같이 바꾼다"는 원칙은 같아요.

2.8 불균형 데이터

드문 쪽이 중요한 문제

1장 1.9절에서 거래 1000건 중 사기가 10건뿐일 때 "늘 정상"이라고만 답하는 모델이 정확도 0.990을 받는 걸 봤습니다. 사기는 한 건도 못 찾았는데요. 화재 경보기가 1년 내내 한 번도 안 울려서 "정확도 99.9%"라고 자랑하는 것과 같아요. 불이 났을 때 울리지 않으면 그 정확도는 아무 의미가 없죠.

이렇게 한쪽 정답이 아주 드문 데이터를 **불균형 데이터(imbalanced data)** 라고 합니다. 부정 사용 탐지, 질병 진단, 고객 이탈, 불량품 검출처럼 실무에서 정말 중요한 문제일수록 불균형한 경우가 많아요. 불균형 데이터에서는 정확도만 보면 안 되고 "드문 쪽을 몇 개나 찾았나"를 함께 봐야 합니다. 이걸 재는 정밀도, 재현율 같은 지표는 3장에서 제대로 다루고, 이 장에서는 **나누기와 학습**에서 할 일 두 가지를 봅니다.

stratify: 비율을 지키며 나누기

양성이 10%뿐인 100명을 무작위로 나누면 시험 데이터 20명에 양성이 몇 명 들어갈까요? 기대값은 2명이지만, 운에 따라 달라집니다.

```
import numpy as np
from sklearn.model_selection import train_test_split

y = np.array([1] * 10 + [0] * 90)      # 100명 중 10명만 양성 (10%)
X = np.arange(100).reshape(-1, 1)

for 씨앗 in range(5):
    _, _, _, 그냥 = train_test_split(X, y, test_size=0.2, random_state=씨앗)
    _, _, _, 층화 = train_test_split(X, y, test_size=0.2, random_state=씨앗, stratify=y)
    print(f"씨앗 {씨앗}: 시험 20명 중 양성  그냥 {그냥.sum()}명, stratify {층화.sum()}명")
```

```
씨앗 0: 시험 20명 중 양성  그냥 3명, stratify 2명
씨앗 1: 시험 20명 중 양성  그냥 0명, stratify 2명
씨앗 2: 시험 20명 중 양성  그냥 3명, stratify 2명
씨앗 3: 시험 20명 중 양성  그냥 2명, stratify 2명
씨앗 4: 시험 20명 중 양성  그냥 1명, stratify 2명
```

그냥 나누면 씨앗에 따라 양성이 0명에서 3명까지 들쭉날쭉합니다. 씨앗 1처럼 0명이면 그 시험 데이터로는 "양성을 잘 찾는가"를 아예 잴 수가 없어요. `stratify=y`를 주면 정답 비율을 학습·시험 양쪽에서 똑같이 유지하므로 언제나 정확히 2명(10%)이 들어갑니다. 우리말로 **층화 추출**이라고 해요. 전국 여론조사에서 지역별 인구 비율에 맞춰 사람을 뽑는 것과 같은 원리입니다.

분류 문제라면 불균형하지 않더라도 `stratify=y`를 주는 습관을 들이세요. 손해 볼 일은 없고, 데이터가 적을 때 시험 결과가 훨씬 안정됩니다.

클래스 가중치 맛보기

나누기를 잘해도 학습 자체가 다수 쪽으로 쏠리는 문제는 남습니다. 모델 입장에서는 95%를 차지하는 쪽만 잘 맞혀도 점수가 높으니, 드문 쪽은 대충 넘어가도 손해가 적거든요. **클래스 가중치(class weight)** 는 드문 쪽을 틀렸을 때 벌점을 더 크게 매겨서 이 쏠림을 바로잡는 방법입니다. 시험에서 어려운 문제에 배점을 높게 주는 것과 같아요.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# 양성이 5%뿐인 분류 문제 (예제용 합성 데이터)
X, y = make_classification(n_samples=4000, n_features=8, n_informative=4,
                           weights=[0.95], flip_y=0.01, class_sep=1.5, random_state=0)
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)
print(f"시험 데이터 {len(y_시험)}건 중 양성 {y_시험.sum()}건")

for 가중치 in [None, "balanced"]:
    예측 = LogisticRegression(class_weight=가중치).fit(X_학습, y_학습).predict(X_시험)
    찾음 = ((예측 == 1) & (y_시험 == 1)).sum()
    헛경보 = ((예측 == 1) & (y_시험 == 0)).sum()
    print(f"class_weight={가중치!s:<8}  정확도 {(예측 == y_시험).mean():.1%}  "
          f"양성 {찾음}건 찾음, 헛경보 {헛경보}건")
```

```
시험 데이터 1000건 중 양성 55건
class_weight=None      정확도 96.0%  양성 15건 찾음, 헛경보 0건
class_weight=balanced  정확도 76.1%  양성 43건 찾음, 헛경보 227건
```

`make_classification`은 scikit-learn이 분류 연습용 가짜 데이터를 만들어 주는 함수예요. `weights=[0.95]`로 음성 95%, 양성 5%가 되게 했습니다.

가중치 없이 학습한 모델은 정확도 96.0%로 높지만, 양성 55건 중 15건밖에 못 찾았습니다. `class_weight="balanced"`를 주면 드문 쪽의 배점을 데이터 비율에 맞춰 자동으로 올리는데, 이번에는 43건을 찾아냈어요. 그 대신 정상인 걸 양성이라고 잘못 울린 헛경보가 0건에서 227건으로 늘었고, 정확도는 76.1%로 떨어졌습니다.

어느 쪽이 나을까요? **문제에 따라 다릅니다.** 암 검진이라면 헛경보가 좀 많더라도 환자를 놓치지 않는 쪽이 낫고, 스팸 필터라면 중요한 메일을 스팸함에 넣는 헛경보가 더 치명적이에요. 정확도 숫자 하나로는 이 판단을 할 수 없다는 것, 그래서 3장에서 다른 지표들이 필요하다는 것만 기억해 두세요. 신경망에서 같은 일을 하는 방법(손실 함수의 가중치, 7장의 샘플러)도 뒤에서 다시 만납니다.

2.9 시간 순서가 있는 데이터

내일 신문으로 어제를 맞히기

지금까지는 데이터를 무작위로 섞어서 나눴습니다. 그런데 날마다 쌓이는 판매량, 주가, 날씨처럼 **시간 순서가 있는 데이터**도 그렇게 해도 될까요?

모델을 만드는 목적을 생각해 보세요. 우리는 과거 데이터로 배워서 **미래**를 예측하고 싶습니다. 그런데 무작위로 섞으면 2025년 12월 데이터가 학습에, 2025년 3월 데이터가 시험에 들어갈 수 있어요. 내일 신문을 보고 어제 일을 맞히는 셈이죠. 실제 서비스에서는 절대 일어날 수 없는 상황이라, 점수가 실제보다 좋게 나옵니다. 1권 20.7절에서 만든 것 같은 날짜별 데이터로 확인해 보겠습니다.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor

rng = np.random.default_rng(0)
# 2년 동안 하루 판매량: 꾸준히 늘고 주말에 많이 팔린다 (예제용)
날짜 = pd.date_range("2024-01-01", "2025-12-31", freq="D")
일차 = np.arange(len(날짜))
판매 = 200 + 0.4 * 일차 + 60 * (날짜.dayofweek >= 5) + rng.normal(0, 15, len(날짜))
X = pd.DataFrame({"일차": 일차, "요일": 날짜.dayofweek})


def 평균_오차(학습_위치, 시험_위치) -> float:
    """나무 모델을 학습하고 시험 구간의 평균 절대 오차를 돌려준다."""
    모델 = DecisionTreeRegressor(min_samples_leaf=5, random_state=0)
    모델.fit(X.iloc[학습_위치], 판매[학습_위치])
    return np.abs(모델.predict(X.iloc[시험_위치]) - 판매[시험_위치]).mean()
```

```
섞은_학습, 섞은_시험 = train_test_split(일차, test_size=0.2, random_state=0)
print(f"마구 섞어 나누기: 평균 {평균_오차(섞은_학습, 섞은_시험):5.1f}개 틀림")

경계 = int(len(일차) * 0.8)                    # 앞 80%로 배우고 뒤 20%로 시험
print(f"시간 순서로 자르기: 평균 {평균_오차(일차[:경계], 일차[경계:]):5.1f}개 틀림")
print("시험 구간:", 날짜[경계].date(), "~", 날짜[-1].date())
```

```
마구 섞어 나누기: 평균  14.0개 틀림
시간 순서로 자르기: 평균  34.8개 틀림
시험 구간: 2025-08-07 ~ 2025-12-31
```

같은 모델인데 마구 섞어 나누면 하루 평균 14.0개, 시간 순서대로 자르면 34.8개를 틀렸습니다. 두 배 넘게 차이가 나요.

섞어서 나누면 시험 날짜의 **바로 앞뒤 날**이 학습 데이터에 들어 있습니다. 모델은 "8월 6일과 8일이 이 정도였으니 7일도 그쯤이겠지" 하고 사이를 메우기만 하면 돼요. 반면 시간 순서대로 자르면 2025년 8월 이후는 한 번도 본 적 없는 미래입니다. 판매량이 계속 늘어나는 추세라서, 과거 값만 기억하는 나무 모델은 미래의 더 큰 값을 따라가지 못해요. 진짜로 서비스할 때 겪을 상황은 뒤쪽이니, 34.8이 정직한 점수입니다.

그래서 시간 순서가 있는 데이터는 **섞지 않고 시점으로 자릅니다.** 어느 날짜를 기준으로 그 전은 학습, 그 뒤는 검증·테스트예요. `train_test_split`을 쓴다면 `shuffle=False`를 주면 앞쪽을 학습, 뒤쪽을 시험으로 순서대로 잘라 줍니다. 물론 그 전에 데이터를 날짜 순으로 정렬해 두어야 하고요.

시간을 따라 여러 번 나누기

검증을 여러 번 하고 싶을 때는 `TimeSeriesSplit`을 씁니다. 학습 구간을 조금씩 늘려 가면서 언제나 **그 바로 뒤**를 시험으로 쓰는 방식이에요.

```
import pandas as pd
from sklearn.model_selection import TimeSeriesSplit

달 = pd.period_range("2025-01", "2025-12", freq="M")

for 회차, (학습, 시험) in enumerate(TimeSeriesSplit(n_splits=4).split(달), start=1):
```

```
    print(f"{회차}회차  학습 {달[학습[0]]}~{달[학습[-1]]}  →  시험 {달[시험[0]]}~{달[시험[-1]]}")
```

```
1회차  학습 2025-01~2025-04  →  시험 2025-05~2025-06
2회차  학습 2025-01~2025-06  →  시험 2025-07~2025-08
3회차  학습 2025-01~2025-08  →  시험 2025-09~2025-10
4회차  학습 2025-01~2025-10  →  시험 2025-11~2025-12
```

1회차는 1~4월로 배워서 5~6월을, 2회차는 1~6월로 배워서 7~8월을 맞힙니다. 어느 회차에서도 학습 구간이 시험 구간보다 앞에 있어요. 실제로 모델을 운영하면서 새 데이터가 쌓일 때마다 다시 학습하는 모습과 같습니다. 여러 번 나눠 점수를 모으는 교차 검증 자체는 3장에서 다루니, 여기서는 "시간 데이터는 미래를 학습에 쓰지 않는다"는 원칙만 가져가세요.

시간 데이터에서는 특성을 만들 때도 같은 원칙을 지켜야 합니다. "최근 7일 평균 판매량" 같은 특성을 만들 때 오늘 값까지 포함해 버리면, 오늘을 예측하는 데 오늘 정답을 쓴 셈이에요. 2.3절 누수 표의 "미래 정보"가 바로 이것입니다.

2.10 실습: 통신사 고객 이탈 데이터 손질하기

이제 이 장에서 배운 것을 모두 써서 데이터 하나를 처음부터 끝까지 준비해 보겠습니다. 통신사가 "곧 해지할 것 같은 고객"을 미리 찾아 혜택을 주고 싶어 한다고 해 봅시다. 고객 3000명의 기록이 있고, 이탈 여부가 정답입니다. 실제 고객 정보가 아니라 이 책의 **예제용으로 만든 데이터**예요. 실제 데이터처럼 빈칸, 입력 실수, 표기 흔들림, 중복을 일부러 섞어 두었습니다.

할 일은 다섯 가지예요.

1. 데이터를 만들어 CSV 파일로 저장하기
2. 체크리스트로 살펴보고, 행 단위로 고칠 수 있는 것 고치기
3. 이탈 비율을 지키며 학습·검증·테스트로 나누기
4. 열 종류별 전처리와 모델을 파이프라인으로 묶고, 검증 데이터로 설정 고르기
5. 테스트 데이터로 딱 한 번 채점하고, 새 고객에게 써 보기

데이터 만들고 저장하기

```
import os
from pathlib import Path

import numpy as np
import pandas as pd

데이터_폴더 = Path(os.environ.get("BOOK_DATA_DIR", "data"))
데이터_폴더.mkdir(parents=True, exist_ok=True)

rng = np.random.default_rng(2025)
n = 3000
요금제 = rng.choice(["베이직", "스탠다드", "프리미엄"], n, p=[0.5, 0.35, 0.15])
약정 = rng.choice(["무약정", "1년", "2년"], n, p=[0.45, 0.35, 0.2])
기본료 = pd.Series(요금제).map({"베이직": 33000, "스탠다드": 55000, "프리미엄": 89000})
고객 = pd.DataFrame({
    "고객번호": np.arange(1, n + 1),
    "나이": rng.integers(19, 75, n).astype(float),
    "지역": rng.choice(["서울", "경기", "부산", "대구", "광주", "대전"], n,
                     p=[0.3, 0.3, 0.15, 0.1, 0.08, 0.07]),
    "요금제": 요금제,
    "약정": 약정,
    "가입개월": rng.integers(1, 73, n),
    "월요금": (기본료 + rng.normal(0, 5000, n)).round(-2),
    "상담횟수": rng.poisson(1.0, n),
})
# 이탈 확률: 무약정·짧은 가입·잦은 상담·젊은 나이일수록 높다고 정한다
점수 = (-2.6 + 1.1 * (약정 == "무약정") - 0.8 * (약정 == "2년")
        - 0.025 * 고객["가입개월"] + 0.55 * 고객["상담횟수"]
        - 0.015 * (고객["나이"] - 45) + 0.000015 * (고객["월요금"] - 50000))
고객["이탈"] = (rng.random(n) < 1 / (1 + np.exp(-점수))).astype(int)

# 실제 데이터처럼 지저분하게: 빈칸, 입력 실수, 표기 흔들림, 중복 행
고객.loc[rng.random(n) < 0.07, "나이"] = np.nan
고객.loc[rng.random(n) < 0.01, "나이"] = 999
고객.loc[rng.random(n) < 0.05, "월요금"] = np.nan
```

```
고객.loc[rng.random(n) < 0.04, "지역"] = np.nan
흔들림 = rng.random(n) < 0.03
고객.loc[흔들림, "지역"] = 고객.loc[흔들림, "지역"] + rng.choice(["시", " "], 흔들림.sum())
고객 = pd.concat([고객, 고객.sample(25, random_state=0)]).reset_index(drop=True)

고객.to_csv(데이터_폴더 / "통신사_고객.csv", index=False)
print("저장:", 고객.shape)
print(고객.head(3).to_string())
```

```
저장: (3025, 9)
   고객번호    나이   지역   요금제   약정  가입개월      월요금  상담횟수  이탈
0     1  55.0   서울  프리미엄   1년    62  87500.0     1   0
1     2  24.0   경기   베이직  무약정    50  39200.0     1   0
2     3  64.0  NaN  스탠다드   2년    24  49200.0     1   0
```

파일은 `data/통신사_고객.csv`에 저장됩니다. 저장할 폴더를 환경 변수 `BOOK_DATA_DIR`로 바꿀 수 있게 했는데, 따로 정하지 않으면 지금 폴더 아래의 `data`를 써요. `mkdir(parents=True, exist_ok=True)`는 1권 12장에서 본 대로 폴더가 없으면 만들고 있으면 넘어갑니다.

이탈 확률은 무약정, 짧은 가입 기간, 잦은 상담, 젊은 나이일수록 높아지도록 정해 두었습니다. 만드는 쪽은 규칙을 알지만, 모델은 이 규칙을 모른 채 데이터만 보고 찾아내야 해요. 마지막에 `sample(25)`로 25줄을 복사해 붙여 중복 행을 만들었습니다.

살펴보고 행 단위로 고치기

저장한 파일을 읽어서 2.1절의 체크리스트대로 살펴봅니다.

```
import os
from pathlib import Path

import numpy as np
import pandas as pd

데이터_폴더 = Path(os.environ.get("BOOK_DATA_DIR", "data"))
고객 = pd.read_csv(데이터_폴더 / "통신사_고객.csv")
```

```
print("1. 모양:", 고객.shape)
print("2. 빈칸:", {열: int(개수) for 열, 개수 in 고객.isna().sum().items() if 개수})
print("3. 중복 행:", int(고객.duplicated().sum()))
print("4. 지역 값:", sorted(고객["지역"].dropna().unique()))
print("5. 나이 범위:", 고객["나이"].min(), "~", 고객["나이"].max(),
      "/ 100살 넘는 사람:", int((고객["나이"] > 100).sum()))
print(f"6. 이탈 비율: {고객['이탈'].mean():.1%}")


def 정리하기(표: pd.DataFrame) -> pd.DataFrame:
    """행 하나만 보고 고칠 수 있는 문제를 고친다 (통계를 쓰지 않으므로 나누기 전에 해도 안전)."""
    표 = 표.drop_duplicates().copy()
    표["지역"] = 표["지역"].str.strip().str.removesuffix("시")
    표.loc[표["나이"] > 100, "나이"] = np.nan
    return 표


정리 = 정리하기(고객)
print("정리 후:", 정리.shape, sorted(정리["지역"].dropna().unique()),
      "나이 최대", 정리["나이"].max())
```

```
1. 모양: (3025, 9)
2. 빈칸: {'나이': 209, '지역': 129, '월요금': 157}
3. 중복 행: 25
4. 지역 값: ['경기', '경기 ', '경기시', '광주', '광주 ', '광주시', '대구', '대구 ', '대구시', '대
↪  전', '대전 ', '대전시', '부산', '부산 ', '부산시', '서울', '서울 ', '서울시']
5. 나이 범위: 19.0 ~ 999.0 / 100살 넘는 사람: 39
6. 이탈 비율: 10.7%
정리 후: (3000, 9) ['경기', '광주', '대구', '대전', '부산', '서울'] 나이 최대 74.0
```

체크리스트가 문제를 모두 찾아냈습니다. 빈칸이 세 열에 있고, 중복 행이 25개, 지역은 "서울", "서울", "서울시"처럼 흔들려서 6곳이 18가지로 보이고, 나이에는 999살이 39명 있어요. 이탈 비율은 10.7%로 불균형한 편입니다.

여기서 중요한 구분이 하나 있습니다. 이 문제들 중에는 **나누기 전에 고쳐도 되는 것**과 **나눈 뒤에 학습 데이터로 배워서 고쳐야 하는 것**이 있어요.

- 나누기 전에 고쳐도 되는 것: 중복 행 지우기, 지역 표기 통일, 999를 빈칸으로 바꾸기. 모두 **행 하나만 보고** 정해진 규칙대로 고치는 일이라 다른 데이터의 통계를 쓰지 않습니다. 시험 데이터가 섞여 있어도 새어 나갈 정보가 없어요.
- 나눈 뒤에 해야 하는 것: 빈칸을 중앙값으로 채우기, 표준화, 원-핫 인코딩의 범주 목록. 모두 **여러 행을 모아 계산한 값**을 쓰므로 학습 데이터로만 배워야 합니다.

정리하기 함수가 앞의 것을 맡았습니다. `drop_duplicates()`로 중복을 지우고, `.str.strip()`으로 앞뒤 빈칸을, `.str.removesuffix("시")`로 끝의 "시"를 떼었어요. `.str`은 pandas 열에 1권 2장의 문자열 메서드를 칸마다 적용하게 해 주는 통로이고, `removesuffix`는 끝이 그 글자로 끝날 때만 떼어 내는 문자열 메서드입니다. 마지막으로 100살이 넘는 나이를 `np.nan`으로 바꿔 "진짜 빈칸"으로 만들었어요. 이제 이 빈칸은 다음 단계의 `SimpleImputer`가 채웁니다. 정리 후 3000줄, 지역 6곳, 최대 나이 74살로 깔끔해졌습니다.

중복 행을 나누기 전에 지우는 건 2.3절 누수 표의 마지막 줄 때문이기도 해요. 같은 줄이 학습과 시험에 하나씩 들어가면, 모델은 시험에서 이미 외운 답을 쓰게 됩니다.

나누고, 묶고, 고르고, 채점하기

이제 핵심입니다. 앞 단계의 정리 규칙을 다시 적용한 뒤, 나누기부터 채점까지 한 번에 진행해요. 코드가 길지만 번호를 붙인 주석을 따라가면 이 장의 절 순서와 똑같습니다.

```
import os
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler

데이터_폴더 = Path(os.environ.get("BOOK_DATA_DIR", "data"))
고객 = pd.read_csv(데이터_폴더 / "통신사_고객.csv")

# 1) 행 단위 정리 (앞에서 살펴본 규칙 그대로)
```

```
고객 = 고객.drop_duplicates().copy()
고객["지역"] = 고객["지역"].str.strip().str.removesuffix("시")
고객.loc[고객["나이"] > 100, "나이"] = np.nan

# 2) 특성과 정답 나누기: 고객번호는 이름표일 뿐이라 뺀다
X = 고객.drop(columns=["고객번호", "이탈"])
y = 고객["이탈"]

# 3) 학습 60 / 검증 20 / 테스트 20, 이탈 비율을 똑같이 유지
X_나머지, X_테스트, y_나머지, y_테스트 = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
X_학습, X_검증, y_학습, y_검증 = train_test_split(
    X_나머지, y_나머지, test_size=0.25, stratify=y_나머지, random_state=42)
for 이름, 정답 in [("학습", y_학습), ("검증", y_검증), ("테스트", y_테스트)]:
    print(f"{이름}: {len(정답)}명, 이탈 {정답.mean():.1%}")


# 4) 열 종류마다 다른 전처리
def 전처리_만들기() -> ColumnTransformer:
    """숫자·범주·순서 열을 각자 맞는 방법으로 처리하는 전처리기를 만든다."""
    숫자 = make_pipeline(SimpleImputer(strategy="median", add_indicator=True),
                         StandardScaler())
    범주 = make_pipeline(SimpleImputer(strategy="constant", fill_value="모름"),
                         OneHotEncoder(handle_unknown="ignore"))
    순서 = OrdinalEncoder(categories=[["베이직", "스탠다드", "프리미엄"],
                                      ["무약정", "1년", "2년"]])
    return ColumnTransformer([
        ("숫자", 숫자, ["나이", "가입개월", "월요금", "상담횟수"]),
        ("범주", 범주, ["지역"]),
        ("순서", 순서, ["요금제", "약정"]),
    ])


def 성적(모델: Pipeline, X: pd.DataFrame, y: pd.Series) -> str:
    """정확도와 실제 이탈자 중 몇 명을 찾았는지를 한 줄로 만든다."""
    예측 = 모델.predict(X)
```

```
    찾음 = int(((예측 == 1) & (y == 1)).sum())
    return f"정확도 {(예측 == y).mean():.1%}, 이탈자 {int(y.sum())}명 중 {찾음}명 찾음"


# 5) 후보 둘을 검증 데이터로 비교 (테스트 데이터는 아직 금고 안에)
for 가중치 in [None, "balanced"]:
    후보 = Pipeline([("전처리", 전처리_만들기()),
                     ("분류", LogisticRegression(class_weight=가중치, max_iter=1000))])
    후보.fit(X_학습, y_학습)
    print(f"[검증] class_weight={가중치!s:<8} {성적(후보, X_검증, y_검증)}")

# 6) 고른 설정으로 학습+검증 데이터에 다시 학습하고, 테스트는 마지막에 딱 한 번
최종 = Pipeline([("전처리", 전처리_만들기()),
                 ("분류", LogisticRegression(class_weight="balanced", max_iter=1000))])
최종.fit(X_나머지, y_나머지)
print(f"[테스트] {성적(최종, X_테스트, y_테스트)}")
print("전처리 후 특성:", len(최종[:-1].get_feature_names_out()), "개")

# 7) 새 고객: 처음 보는 지역, 빈칸이 있어도 원본 모양 그대로 넣으면 된다
새_고객 = pd.DataFrame({
    "나이": [23, np.nan], "지역": ["세종", "서울"], "요금제": ["프리미엄", "베이직"],
    "약정": ["무약정", "2년"], "가입개월": [3, 60], "월요금": [91000, np.nan],
    "상담횟수": [4, 0],
})
print("새 고객 이탈 확률:", 최종.predict_proba(새_고객)[:, 1].round(2))
```

```
학습: 1800명, 이탈 10.7%
검증: 600명, 이탈 10.7%
테스트: 600명, 이탈 10.7%
[검증] class_weight=None     정확도 89.3%, 이탈자 64명 중 7명 찾음
[검증] class_weight=balanced 정확도 69.8%, 이탈자 64명 중 41명 찾음
[테스트] 정확도 72.8%, 이탈자 64명 중 47명 찾음
전처리 후 특성: 15 개
새 고객 이탈 확률: [0.97 0.06]
```

단계별로 짚어 보겠습니다.

- 2) **특성과 정답**: `고객번호`는 뺐습니다. 번호는 고객을 구별하는 이름표일 뿐 이탈과 아무 관계가 없어요. 그런데도 넣으면 모델이 번호에서 우연한 패턴을 찾아 외울 수 있습니다. 데이터를 시간 순으로 쌓았다면 번호가 가입 시점을 몰래 알려 주는 누수가 되기도 하고요.
- 3) **나누기**: `stratify`를 두 번 모두 줬더니 학습 1800명, 검증 600명, 테스트 600명이 모두 이탈 비율 10.7%로 똑같습니다. 두 번째 나누기의 `stratify`에는 `y`가 아니라 `y_나머지`를 줘야 한다는 점을 조심하세요. 지금 나누는 데이터의 정답이어야 하니까요.
- 4) **전처리**: 숫자 열은 중앙값으로 채우고(2.4절) 비었다는 표시를 남긴 뒤(`add_indicator=True`) 표준화합니다(2.6절). 지역은 "모름"으로 채워 원-핫(2.5절), 요금제와 약정은 순서를 적어 준 순서형 인코딩이에요. 요금제와 약정은 빈칸이 없어서 채우기 단계를 넣지 않았습니다. 전처리기를 함수로 만든 건 후보마다 **새 전처리기**를 쓰기 위해서예요. 같은 전처리기 객체를 여러 파이프라인이 나눠 쓰면 한쪽에서 `fit`할 때 다른 쪽의 것까지 바뀝니다.
- 5) **검증으로 고르기**: 가중치 없는 모델은 정확도 89.3%지만 이탈자 64명 중 7명밖에 못 찾았습니다. 이탈 비율이 10.7%이니 "아무도 이탈 안 함"이라고만 말해도 89.3%예요. 사실상 1장 1.9절의 "늘 정상" 모델에 가깝습니다. `balanced`는 정확도가 69.8%로 떨어졌지만 41명을 찾았어요. 우리 목적은 "떠날 고객을 미리 찾아 혜택 주기"이니 `balanced`를 고릅니다. 혜택을 받은 고객 중 일부가 원래 안 떠날 사람이었더라도, 떠날 고객을 놓치는 것보다는 손해가 적다고 판단한 거예요.
- 6) **최종 채점**: 설정을 골랐으니 학습과 검증 데이터를 합친 `X_나머지` 2400명으로 다시 학습했습니다. 고르기가 끝난 뒤에는 검증 데이터도 공부 재료로 쓰는 게 이득이에요. 그리고 금고에 있던 테스트 데이터로 **딱 한 번** 채점해서 이탈자 64명 중 47명을 찾았습니다. 검증에서 본 결과와 비슷한 수준이라, 이 점수를 믿을 만하다고 볼 수 있어요.
- **전처리 후 특성 15개**: 숫자 4개 + 빈칸 표시 2개(나이, 월요금) + 지역 7개(6곳과 "모름") + 순서형 2개입니다. 가입개월과 상담횟수는 학습 데이터에 빈칸이 없어서 표시 열이 생기지 않았어요. `최종[:-1]`은 파이프라인의 마지막 단계(모델)를 뺀 앞부분, 즉 전처리기만 잘라 낸 것입니다. 1권 6장의 슬라이싱 문법 그대로예요.
- 7) **새 고객**: 첫 번째 고객은 23살, 무약정, 가입 3개월, 상담 4회에 학습 때 없던 "세종" 지역입니다. 이탈 확률 0.97로 아주 위험하다고 나왔어요. 두 번째 고객은 나이와 월요금이 비었지만 2년 약정, 가입 60개월, 상담 0회라 0.06입니다. 원본 모양 그대로 넣었는데 빈칸 채우기, 모르는 지역 처리, 표준화가 모두 파이프라인 안에서 학습 때와 똑같이 이루어졌습니다.

이 실습의 모델은 가장 단순한 로지스틱 회귀 하나입니다. 다른 모델과 비교하고, 이탈자를 몇 명 찾았는지를 재현율이라는 이름의 지표로 정식으로 재고, 설정을 체계적으로 고르는 일은 3장에서 배웁니다. 3장의 도구를 익힌 뒤 이 데이터로 돌아와 직접 해 보면 좋은 연습이 될 거예요.

2.11 정리와 다음 장 미리보기

이 장에서 모델에 데이터를 넣기 전에 할 일을 익혔습니다.

- **살펴보기**: 크기, 자료형, 빈칸, 숨은 빈칸(0·999), 범주 표기 흔들림, 중복, 정답 비율을 모델보다 먼저 확인하기
- **나누기**: 학습(교과서), 검증(모의고사), 테스트(수능). 테스트를 먼저 떼어 금고에 넣고 마지막에 한 번만 보기. 테스트 점수를 보고 고르면 점수가 부푼다(동전 던지기가 61%)
- **누수 막기**: `fit`은 학습 데이터로만, `transform`은 어디에나. 나누기 전 특성 선택은 동전 던지기를 80%로 부풀렸다. 정답을 품은 특성, 같은 대상의 여러 기록, 중복 행, 미래 정보도 누수의 길
- **결측치**: 버리면 생각보다 많이 사라진다. 숫자는 중앙값, 글자는 최빈값이나 "모름", 비어 있다는 사실은 `add_indicator`로 남기기
- **범주형 인코딩**: 순서 없는 범주는 `OneHotEncoder(handle_unknown="ignore")`, 순서 있는 범주는 `OrdinalEncoder(categories=...)`로 순서를 직접 적기. `get_dummies`는 학습·시험 열이 어긋난다. 드문 범주는 `min_frequency`로 묶기
- **스케일링**: 거리를 재거나 가중치를 조금씩 고쳐 배우는 모델에는 필수(로지스틱 회귀가 1000번에도 못 끝낸 학습을 18번에). 표준화가 기본, 범위가 정해진 데이터는 정규화, 튀는 값이 많으면 RobustScaler. 시험 데이터가 0~1을 벗어나는 건 정상
- **묶기**: `ColumnTransformer`로 열마다 다른 손질, `Pipeline`으로 손질과 모델을 하나로. 누수 방지가 코드 구조로 보장된다
- **불균형**: 정확도만 보면 "늘 정상" 모델도 높은 점수. `stratify`로 비율 지키며 나누고, `class_weight="balanced"`로 드문 쪽 배점 올리기
- **시간 데이터**: 섞지 말고 시점으로 자르기. 섞으면 미래로 과거를 맞히는 셈이라 점수가 부푼다

2.1절의 체크리스트에 이 장의 교훈을 더해, 모델을 학습시키기 전 마지막 점검표로 정리해 둡니다.

순서	점검	이 장의 절
1	체크리스트로 살펴보고 행 단위로 고칠 것(중복, 표기, 숨은 빈칸) 고치기	2.1, 2.10
2	예측 시점에 알 수 없는 특성, 이름표 같은 특성 빼기	2.3, 2.10

순서	점검	이 장의 절
3	시간 데이터면 시점으로, 같은 대상이 여러 줄이면 그룹으로, 분류면 `stratify`로 나누기	2.2, 2.3, 2.8, 2.9
4	통계가 필요한 전처리는 전부 Pipeline 안에 넣기	2.4~2.7
5	고르기는 검증 데이터로, 테스트는 마지막에 한 번	2.2

다음 3장에서는 이렇게 준비한 데이터로 본격적으로 모델을 만들고 비교합니다. 선형 회귀와 로지스틱 회귀, 결정 트리와 랜덤 포레스트 같은 고전 머신러닝 모델을 써 보고, 이 장에서 "몇 명 찾았나"로 대충 셌던 것을 정밀도·재현율·F1 같은 지표로 제대로 재요. 검증 데이터를 여러 번 돌려 쓰는 교차검증과, 설정을 자동으로 찾아 주는 탐색 도구도 만납니다. 이 장의 Pipeline이 거기서 진가를 발휘합니다.

03

scikit-learn으로 보는 고전 머신러닝

학습 목표

- scikit-learn 모델이 공통으로 따르는 `fit`·`predict`·`score` 약속을 이해하고 모델을 바꿔 끼우기
- 선형 회귀·로지스틱 회귀가 무엇을 배우는지(계수) 읽고, 결정 트리·랜덤 포레스트·그래디언트 부스팅의 차이를 설명하기
- 정답 없는 데이터를 k-means로 묶고, 무리 수를 고르는 방법 알기
- 정확도·정밀도·재현율·F1·ROC-AUC와 MAE·RMSE·R^2를 상황에 맞게 고르기
- 교차 검증으로 모델을 믿을 만하게 비교하고, 하이퍼파라미터를 탐색하고, 완성한 모델을 파일로 저장하기

들어가며: 엑셀 표 같은 데이터에도 딥러닝이 필요할까요?

회사에서 받는 데이터의 상당수는 사진이나 문장이 아니라 **표**입니다. 고객 한 명이 한 줄이고, 나이·가입 기간·지난달 구매액 같은 값이 칸마다 들어 있는 엑셀 시트 말이에요. 병원의 검사 수치, 부동산의 면적과 위치, 공장의 센서 기록도 대개 이런 모양입니다.

이런 표 데이터라면 꼭 거대한 신경망을 꺼낼 필요가 없습니다. 수십 년 동안 다듬어진 **고전 머신러닝** 알고리즘이 몇 초 만에 학습하고, 결과를 사람이 읽을 수 있게 설명해 주고, 성능도 딥러닝에 뒤지지 않는 경우가 많거든요. 파이썬에서 이 알고리즘들을 한곳에 모아 둔 도구가 **scikit-learn**(사이킷런)입니다.

1장에서 최근접 이웃 모델을 scikit-learn 한 줄로 바꿔 봤고, 2장에서는 데이터를 나누고 다듬는 법

을 배웠습니다. 이번 장에서는 scikit-learn에 들어 있는 대표 모델들을 하나씩 열어 보고, 무엇보다 **모델을 공정하게 채점하는 법**을 익힙니다. 먼저 scikit-learn이 얼마나 일관된 도구인지부터 보겠습니다. 유방암 진단 데이터에 서로 전혀 다른 모델 세 개를 똑같은 코드로 학습시켜 볼게요.

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

모델들 = {
    "최근접 이웃": make_pipeline(StandardScaler(), KNeighborsClassifier()),
    "로지스틱 회귀": make_pipeline(StandardScaler(), LogisticRegression()),
    "결정 트리": DecisionTreeClassifier(random_state=42),
}
for 이름, 모델 in 모델들.items():
    모델.fit(X_학습, y_학습)              # 배우기: 모델마다 똑같은 한 줄
    정확도 = 모델.score(X_시험, y_시험)     # 처음 보는 문제로 채점하기
    print(f"{이름}: 시험 정확도 {정확도:.3f}")
```

```
최근접 이웃: 시험 정확도 0.979
로지스틱 회귀: 시험 정확도 0.986
결정 트리: 시험 정확도 0.923
```

최근접 이웃, 로지스틱 회귀, 결정 트리는 속에서 하는 일이 완전히 다릅니다. 그런데 쓰는 방법은 `fit`으로 배우고 `score`로 채점하는 두 줄로 똑같아요. 이 일관성 덕분에 "어느 모델이 내 데이터에 맞을까?"를 고민할 때 코드를 갈아엎지 않고 모델만 바꿔 끼우며 비교할 수 있습니다. 이 장이 끝날 즈음이면 위의 비교를 훨씬 믿을 만한 방법(교차 검증)으로 다시 하고, 고른 모델을 파일로 저장하는 데까지 가게 됩니다.

Q

Q. 따로 설치할 것이 있나요? 없습니다. 1·2장에서 설치한 scikit-learn과 pandas면 충분하고, 모델 저장에 쓰는 `joblib`과 계산에 쓰는 `scipy`는 scikit-learn을 설치할 때 함께 따라옵니다. 이 장의 예제는 scikit-learn 1.9, 파이썬 3.12에서 실행해 확인했습니다.

3.1 scikit-learn의 약속: fit, predict, score

모든 모델이 같은 모양의 손잡이를 갖고 있습니다

가전제품마다 전원 버튼 위치가 다르면 새 제품을 살 때마다 설명서를 읽어야겠죠. scikit-learn은 모든 모델에 **같은 손잡이**를 달아 두었습니다. 이 장에서 계속 쓸 약속은 다음과 같아요.

손잡이	하는 일
`모델 = 모델종류(설정=값)`	모델을 만들면서 **하이퍼파라미터**(사람이 정하는 설정)를 정한다
`모델.fit(X, y)`	데이터에서 배운다. 지도 학습은 정답 `y`도 함께 준다
`모델.predict(X)`	새 데이터의 답을 예측한다
`모델.predict_proba(X)`	분류 모델이 각 답일 **확률**을 돌려준다
`모델.score(X, y)`	예측해서 채점한다. 분류는 정확도, 회귀는 R^2(3.8절)
`모델.이름_`	끝에 밑줄이 붙은 속성은 `fit`으로 **배운 결과**다

마지막 줄이 특히 중요합니다. `coef_`, `feature_importances_`, `cluster_centers_`처럼 이름 끝에 밑줄이 붙은 속성은 `fit`을 하기 전에는 없다가 학습이 끝나면 생깁니다. 사람이 정해 주는 하이퍼파라미터(`max_depth=3` 같은)와, 모델이 데이터에서 스스로 찾아낸 **파라미터**(`coef_` 같은)를 이름만 보고 구분할 수 있게 해 둔 약속이에요.

데이터 `X`는 언제나 **(샘플 수, 특성 수)** 모양의 2차원 표입니다. 숫자만 보지 말고 "환자 569명, 검사 수치 30개"처럼 자리마다 이름을 붙여 소리 내어 읽어 보세요. 4장에서 텐서를 다룰 때도 이 습관이 큰 도움이 됩니다. 정답 `y`는 샘플 수만큼의 1차원 줄이고요. NumPy 배열이든 pandas 데이터프레임이든(1권 20장) 모두 받습니다.

이 장에서 쓰는 데이터

예제는 scikit-learn에 들어 있는 데이터를 주로 씁니다. 따로 받을 필요가 없고, 누가 실행해도 같은 숫자가 나오니까요.

```
import numpy as np
from sklearn.datasets import load_breast_cancer, load_diabetes

암 = load_breast_cancer()
print("유방암 진단 데이터:", 암.data.shape)
print("  특성 예:", 암.feature_names[:3].tolist())
print("  레이블 이름과 개수:", dict(zip(암.target_names.tolist(),
↪  np.bincount(암.target).tolist())))
y = 1 - 암.target                    # 0과 1을 뒤집어 1 = 악성으로
print("  뒤집은 뒤 악성(1) 개수:", y.sum())

당뇨 = load_diabetes()
print("당뇨병 진행도 데이터:", 당뇨.data.shape)
print("  특성:", list(당뇨.feature_names))
print("  목표값 범위:", 당뇨.target.min(), "~", 당뇨.target.max())
```

```
유방암 진단 데이터: (569, 30)
  특성 예: ['mean radius', 'mean texture', 'mean perimeter']
  레이블 이름과 개수: {'malignant': 212, 'benign': 357}
  뒤집은 뒤 악성(1) 개수: 212
당뇨병 진행도 데이터: (442, 10)
  특성: ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
  목표값 범위: 25.0 ~ 346.0
```

- **유방암 진단 데이터**: 환자 569명의 종양 세포를 측정한 값 30개(반지름, 질감, 둘레 등)와, 그 종양이 악성(malignant)인지 양성(benign)인지가 들어 있습니다. 분류 예제에 씁니다.
- **당뇨병 진행도 데이터**: 환자 442명의 나이·성별·체질량지수(bmi)·혈압(bp)과 혈액 검사 수치 6개(s1~s6), 그리고 1년 뒤 병이 얼마나 진행됐는지를 나타내는 숫자(25~346)가 있습니다. 회귀 예제에 씁니다. 특성 값은 원래 단위가 아니라 미리 평균 0 근처로 맞춰진 상태로 들어 있어요.

유방암 데이터는 원래 악성이 0, 양성이 1로 적혀 있습니다. 그런데 이 장에서는 **찾고 싶은 쪽**, 즉 악

성을 1로 두는 편이 3.7절의 평가 지표를 이해하기 훨씬 쉬워요. 그래서 `1 - 암.target`으로 0과 1을 뒤집어 씁니다. 앞으로 나오는 예제에서 `y = 1 - y`라는 줄이 보이면 "1이 악성"이라는 뜻입니다.

3.2 선형 회귀: 숫자를 예측하는 직선

집값의 숨은 공식 찾기

부동산 중개사무소 사장님이 "이 동네는 기본 1억에, 평당 1,500만 원쯤 해요"라고 말했다고 해 봅시다. 이 말을 식으로 쓰면 `가격 = 1.0 + 0.15 × 넓이`(억 원)입니다. 사장님은 수많은 거래를 지켜보면서 이 공식을 머릿속에 만들어 둔 거예요.

선형 회귀(linear regression) 는 이 일을 컴퓨터가 하게 합니다. 넓이와 가격 기록을 주면, 점들 사이를 가장 잘 지나가는 직선의 기울기와 절편을 찾아 줘요. 실제로 "기본 1억, 평당 0.15억"이라는 숨은 규칙에 잡음을 섞어 예제용 데이터를 만들고, 모델이 그 규칙을 찾아내는지 보겠습니다.

```
import numpy as np
from sklearn.linear_model import LinearRegression

# 예제용으로 만든 데이터: 집 넓이(평)와 가격(억 원)
rng = np.random.default_rng(0)
넓이 = rng.uniform(10, 40, size=50)
가격 = 1.0 + 0.15 * 넓이 + rng.normal(0, 0.3, size=50)   # 숨은 규칙 + 잡음

모델 = LinearRegression()
모델.fit(넓이.reshape(-1, 1), 가격)     # X는 언제나 (샘플 수, 특성 수) 2차원
print(f"기울기(평당 가격): {모델.coef_[0]:.3f}억 원")
print(f"절편(기본 가격):   {모델.intercept_:.3f}억 원")
print("30평, 35평 예상 가격:", 모델.predict([[30], [35]]).round(2))
```

```
기울기(평당 가격): 0.155억 원
절편(기본 가격):   0.875억 원
30평, 35평 예상 가격: [5.53 6.31]
```

모델이 찾은 기울기는 0.155, 절편은 0.875입니다. 숨겨 둔 0.15와 1.0에 꽤 가깝죠? 잡음 때문에 정확히 같지는 않지만, 점 50개만 보고 규칙의 모양을 거의 알아냈어요. 이제 이 모델에 30평을 물으면 "기본값 + 30 × 평당 가격"을 계산해서 5.53억이라고 답합니다.

여기서 "가장 잘 지나가는 직선"의 기준은 **예측과 실제의 차이를 제곱해서 모두 더한 값이 가장 작은 직선**입니다. 이 기준을 만족하는 기울기와 절편은 공식으로 한 번에 구할 수 있어서, `LinearRegression`은 학습이 눈 깜짝할 새에 끝나요. 같은 문제를 "조금씩 고쳐 가며" 푸는 방법이 5장의 경사하강법인데, 거기서 이 직선을 PyTorch로 직접 다시 찾아봅니다.

코드에서 `넓이.reshape(-1, 1)`을 눈여겨보세요. 특성이 하나뿐이어도 `X`는 (샘플 수, 특성 수) 2차원이어야 해서 `(50,)`을 `(50, 1)`로 세웠습니다. `reshape`의 `-1`은 "나머지는 알아서 계산하라"는 뜻이에요(4장에서 텐서로 자세히 봅니다). 예측할 때도 `[[30], [35]]`처럼 2차원으로 줍니다.

특성이 여러 개일 때: 계수 읽기

특성이 열 개라면 직선 대신 `예측 = 절편 + 계수1 × 특성1 + … + 계수10 × 특성10` 꼴이 됩니다. 계수 하나하나가 "그 특성이 1만큼 커지면 예측이 얼마나 바뀌는가"예요. 당뇨병 데이터로 어떤 특성이 진행도와 관련이 큰지 계수를 읽어 보겠습니다.

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

당뇨 = load_diabetes()
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    당뇨.data, 당뇨.target, test_size=0.25, random_state=42)

모델 = make_pipeline(StandardScaler(), LinearRegression())
모델.fit(X_학습, y_학습)

선형 = 모델[-1]                          # 파이프라인의 마지막 단계 꺼내기
계수 = sorted(zip(당뇨.feature_names, 선형.coef_), key=lambda 쌍: -abs(쌍[1]))
for 이름, 값 in 계수:
    print(f"{이름:>4}: {값:+7.1f}")
print(f"절편: {선형.intercept_:.1f}")
print(f"시험 세트 R²: {모델.score(X_시험, y_시험):.3f}")
```

```
  s1:   -44.1
  s5:   +33.4
 bmi:   +25.1
  s2:   +24.5
  bp:   +18.2
  s4:   +13.0
 sex:   -11.5
  s3:    +5.5
 age:    +2.2
  s6:    +1.2
절편: 154.3
시험 세트 R²: 0.485
```

`make_pipeline(StandardScaler(), LinearRegression())`은 2장에서 배운 파이프라인입니다. 특성을 먼저 **표준화**(평균 0, 표준편차 1)한 뒤 선형 회귀에 넘겨요. 계수를 비교하려면 이 단계가 꼭 필요합니다. 원래 단위 그대로라면(나이는 수십, 혈액 수치는 수백) 계수 크기가 단위에 휘둘리거든요. 이 데이터는 3.1절에서 말했듯 미리 눈금을 맞춰 둔 상태라 효과가 크지 않지만, 계수를 비교할 때는 습관처럼 표준화를 넣어 두는 게 안전합니다. 모두 같은 눈금으로 맞춰 두면 "표준편차 하나만큼 커질 때의 영향"으로 공평하게 비교할 수 있습니다. `모델[-1]`은 파이프라인의 마지막 단계, 즉 선형 회귀 모델을 꺼내는 방법이에요.

계수를 크기순으로 정렬한 결과(1권 10장의 `sorted`와 `key=lambda`)를 보면 bmi가 +25.1로, 체질량지수가 높을수록 진행도가 높게 예측됩니다. 의학 상식과도 맞죠. 그런데 혈액 수치 s1과 s2를 보면 이상합니다. s1은 -44.1로 모든 계수 중 가장 크고, s2는 +24.5로 부호가 반대예요. 둘은 서로 아주 비슷하게 움직이는 수치라서, 모델이 "한쪽은 크게 더하고 다른 쪽은 크게 빼는" 식으로 균형을 맞춘 결과입니다. 이렇게 서로 비슷하게 움직이는 특성이 함께 들어 있는 상태를 **다중공선성**(multicollinearity) 이라고 합니다. 다중공선성이 있으면 계수를 곧이곧대로 해석하기 어려워져요. 5장 5.13절에서는 같은 두 특성이 경사하강법에서 어떻게 보이는지 다시 만납니다. 시험 세트 R^2(3.8절)는 0.485로, 진행도 차이의 절반쯤을 설명하는 수준입니다.

계수에 고삐 매기: 릿지와 라소

계수가 서로 크게 밀고 당기는 상황을 막는 방법이 있습니다. 학습할 때 "예측을 잘하라"는 목표에 "**계수는 되도록 작게**"라는 벌점을 더하는 거예요. 고삐를 매어 계수가 멀리 뛰쳐나가지 못하게 하는 셈입니다. 벌점을 계수의 제곱으로 매기면 **릿지**(Ridge), 절댓값으로 매기면 **라소**(Lasso) 라고 부릅니다.

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import Lasso, LinearRegression, Ridge
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

당뇨 = load_diabetes()
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    당뇨.data, 당뇨.target, test_size=0.25, random_state=42)

print("     " + "".join(f"{이름:>6}" for 이름 in 당뇨.feature_names) + "   시험 R²")
for 이름, 회귀 in [("선형", LinearRegression()), ("릿지", Ridge(alpha=10.0)),
                   ("라소", Lasso(alpha=1.0))]:
    모델 = make_pipeline(StandardScaler(), 회귀).fit(X_학습, y_학습)
    계수 = 모델[-1].coef_.round(1) + 0.0          # + 0.0은 -0.0을 0.0으로 정리
    print(f"{이름} " + "".join(f"{값:6.1f}" for 값 in 계수)
          + f"   {모델.score(X_시험, y_시험):.3f}")
```

```
        age   sex   bmi    bp    s1    s2    s3    s4    s5    s6   시험 R²
선형    2.2 -11.5  25.1  18.2 -44.1  24.5   5.5  13.0  33.4   1.2   0.485
릿지    2.4 -11.0  25.1  17.6 -13.1   0.5  -7.8   8.5  21.2   1.9   0.488
라소    1.2  -9.3  25.7  17.0  -8.4   0.0 -10.8   3.3  21.1   1.1   0.496
```

alpha가 고삐의 세기입니다. 릿지에서는 -44.1과 +24.5로 싸우던 s1·s2가 -13.1과 0.5로 얌전해졌고, 라소는 s2의 계수를 아예 **0으로** 만들어 버렸어요. 라소는 쓸모가 적은 특성을 지워 버리는 성질이 있어서, 특성이 많을 때 "어떤 특성이 정말 필요한가"를 고르는 데도 씁니다. 그리고 계수를 눌렀는데도 시험 R^2는 오히려 조금씩 올랐습니다(0.485 → 0.488 → 0.496). 차이가 작아서 한 번 나눈 결과만으로 단정할 수는 없지만(3.9절), 계수가 과하게 커지는 것이 학습 데이터의 우연한 무늬까지 외운 결과일 수 있다는 신호예요.

이 "가중치를 작게 유지하라"는 벌점은 8장에서 신경망의 **가중치 감쇠(weight decay)** 라는 이름으로 다시 만납니다. 생각은 똑같아요.

3.3 로지스틱 회귀: 확률로 답하는 분류

점수를 확률로 바꾸는 S자 곡선

이번에는 숫자가 아니라 "악성인가, 아닌가"를 맞혀야 합니다. 선형 회귀처럼 **절편 + 계수 × 특성**으로 점수를 낼 수는 있는데, 이 점수는 -100이 될 수도 1000이 될 수도 있어요. 우리가 원하는 건 "악성일 확률 87%" 같은 0과 1 사이 값입니다.

그래서 점수를 0과 1 사이로 눌러 담는 함수를 하나 끼웁니다. 아무리 세게 눌러도 일정 높이 이상 올라가지 않는 스프링 저울을 떠올려 보세요. 이 함수를 **시그모이드(sigmoid)** 라고 해요.

```
import numpy as np


def 시그모이드(z: np.ndarray) -> np.ndarray:
    """어떤 실수든 0과 1 사이 값으로 눌러 담는다."""
    return 1 / (1 + np.exp(-z))


점수 = np.array([-6.0, -2.0, -0.5, 0.0, 0.5, 2.0, 6.0])
for z, p in zip(점수, 시그모이드(점수)):
    print(f"점수 {z:+5.1f} → 확률 {p:.3f}")
```

```
점수  -6.0 → 확률 0.002
점수  -2.0 → 확률 0.119
점수  -0.5 → 확률 0.378
점수  +0.0 → 확률 0.500
점수  +0.5 → 확률 0.622
점수  +2.0 → 확률 0.881
점수  +6.0 → 확률 0.998
```

점수가 0이면 확률은 딱 0.5, 점수가 커질수록 1에, 작아질수록 0에 다가가지만 넘지는 않습니다. 점수 +2만 돼도 확률 0.88, +6이면 0.998이에요. **로지스틱 회귀(logistic regression)** 는 "선형 회귀로 점수를 내고, 시그모이드로 확률을 만들고, 0.5를 넘으면 1이라고 답하는" 모델입니다. 이름에 회귀가 들어 있지만 분류 모델이라는 점이 처음엔 헷갈려요.

유방암 진단에 써 보기

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

암 = load_breast_cancer()
X, y = 암.data, 1 - 암.target            # 1 = 악성
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

모델 = make_pipeline(StandardScaler(), LogisticRegression())
모델.fit(X_학습, y_학습)

확률 = 모델.predict_proba(X_시험[:5])     # 열 0: 악성 아님, 열 1: 악성
print("예측 확률:\n", 확률.round(3))
print("예측 레이블:", 모델.predict(X_시험[:5]))
print("실제 레이블:", y_시험[:5])

계수 = 모델[-1].coef_[0]                  # 특성 30개에 하나씩
for i in 계수.argsort()[::-1][:3]:       # 큰 것부터 세 개
    print(f"{암.feature_names[i]:>15}: {계수[i]:+.2f}")
```

```
예측 확률:
 [[0.001 0.999]
 [0.996 0.004]
 [0.    1.   ]
 [1.    0.   ]
 [0.965 0.035]]
예측 레이블: [1 0 1 0 0]
실제 레이블: [1 0 1 0 0]
  worst texture: +1.37
   radius error: +1.27
 worst symmetry: +1.04
```

`predict_proba`는 환자마다 두 칸짜리 확률을 돌려줍니다. 앞 칸이 0(악성 아님), 뒤 칸이 1(악성)일 확률이고 둘을 더하면 1이에요. 첫 환자는 악성일 확률이 0.999라 1로, 다섯째 환자는 0.035라 0으로 예측했고, 다섯 명 모두 실제 레이블과 맞았습니다. `predict`는 이 확률이 0.5를 넘는지만 보고 답을 고르는 거예요. 3.7절에서 이 0.5라는 **문턱**을 바꾸면 무슨 일이 생기는지 봅니다.

계수의 읽는 법은 선형 회귀와 같습니다. 양수이면 그 값이 클수록 악성 쪽으로 점수를 밀고, 음수이면 반대쪽으로 밀어요. 여기서는 worst texture(가장 거친 부위의 질감)가 +1.37로 악성 쪽으로 가장 세게 밉니다. 다만 앞의 s1·s2처럼 비슷한 특성이 많은 데이터라, 계수 하나를 "원인"으로 해석하는 데는 조심해야 합니다.

> **Q. 로지스틱 회귀가 신경망과 관계가 있나요?** 아주 깊은 관계가 있습니다. 로지스틱 회귀는 "입력 × 가중치 + 편향"을 계산한 뒤 시그모이드를 거치는 계산이죠. 신경망의 층 하나도 결국 "입력 @ 가중치 + 편향"이라는 같은 계산이라(4장에서 행렬 곱으로 만나요), 로지스틱 회귀는 **층이 하나뿐인 신경망**과 같습니다. 6장에서 `nn.Linear`로 이 계산을 만들고, 층을 여러 겹 쌓으면 신경망이 돼요.

3.4 결정 트리: 스무고개로 답 찾기

질문을 이어 가며 좁혀 가기

스무고개 놀이를 해 본 적 있죠? "살아 있는 건가요?" "네." "다리가 네 개인가요?" "아니요." 이렇게 예/아니요 질문을 이어 가며 답을 좁혀 갑니다. 잘하는 사람은 **한 번에 후보를 가장 많이 떨어내는 질문**부터 해요.

결정 트리(decision tree) 가 바로 이 놀이를 하는 모델입니다. 학습할 때 "어떤 특성을 어떤 값에서 나누면 두 무리가 가장 깔끔하게 갈리는가"를 모든 특성에 대해 따져서 가장 좋은 질문을 고르고, 나뉜 무리 안에서 또 같은 일을 반복해요. 질문을 딱 두 번만 하도록(`max_depth=2`) 묶어 두고, 어떤 질문을 골랐는지 글로 뽑아 보겠습니다.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text

암 = load_breast_cancer()
X, y = 암.data, 1 - 암.target           # 1 = 악성
```

```
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

나무 = DecisionTreeClassifier(max_depth=2, random_state=42)
나무.fit(X_학습, y_학습)
print(export_text(나무, feature_names=암.feature_names.tolist(), show_weights=True))
print(f"질문 두 번만으로 시험 정확도 {나무.score(X_시험, y_시험):.3f}")
```

```
|--- worst perimeter <= 112.80
|   |--- worst concave points <= 0.18
|   |   |--- weights: [257.00, 12.00] class: 0
|   |--- worst concave points >  0.18
|   |   |--- weights: [0.00, 8.00] class: 1
|--- worst perimeter >  112.80
|   |--- mean texture <= 14.95
|   |   |--- weights: [5.00, 2.00] class: 0
|   |--- mean texture >  14.95
|   |   |--- weights: [5.00, 137.00] class: 1

질문 두 번만으로 시험 정확도 0.895
```

`export_text`가 나무를 글로 그려 줬습니다. 첫 질문은 "worst perimeter(가장 큰 부위의 둘레)가 112.80 이하인가?"예요. 둘레가 작은 쪽에서는 다시 오목한 부위 수치를 묻고, 큰 쪽에서는 질감을 묻습니다. `weights: [257.00, 12.00]`은 그 잎(마지막 칸)에 도착한 학습 데이터가 0(악성 아님) 257명, 1(악성) 12명이라는 뜻이고, 그래서 그 잎의 답은 `class: 0`입니다.

결정 트리의 가장 큰 장점은 이렇게 **사람이 읽을 수 있다**는 점이에요. "둘레가 112.8보다 크고 질감 값이 14.95보다 크면 악성 가능성이 높다"는 규칙은 의사에게 그대로 설명할 수 있습니다. 질문 두 번만으로도 시험 정확도가 0.895나 나왔죠. 또 각 질문이 "이 값보다 큰가 작은가"뿐이라, 3.2절과 달리 특성을 표준화할 필요가 없습니다.

너무 깊은 나무는 외워 버립니다

질문을 무한정 하게 두면 어떻게 될까요? 스무고개에 질문 제한이 없으면 "혹시 철수네 집 강아지 초코인가요?"처럼 아주 구체적인 질문까지 해서 결국 맞히겠죠. 결정 트리도 깊이 제한이 없으면 학습 데이터를 **한 명도 빠짐없이** 맞힐 때까지 자랍니다. 잡음이 섞인 예제용 데이터에서 깊이를 바꿔 보겠

습니다.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# 예제용으로 만든 데이터: 잡음 섞인 두 반달 모양의 점 400개
X, y = make_moons(n_samples=400, noise=0.35, random_state=0)
X_학습, X_시험, y_학습, y_시험 = train_test_split(X, y, test_size=0.3, random_state=0)

print("깊이  잎 수  학습 정확도  시험 정확도")
for 깊이 in [1, 2, 4, 6, 8, None]:
    나무 = DecisionTreeClassifier(max_depth=깊이, random_state=0).fit(X_학습, y_학습)
    print(f"{str(깊이):>4} {나무.get_n_leaves():>5} {나무.score(X_학습, y_학습):>12.3f}"
          f" {나무.score(X_시험, y_시험):>12.3f}")
```

```
깊이  잎 수  학습 정확도  시험 정확도
   1     2       0.800       0.800
   2     4       0.879       0.900
   4    12       0.882       0.900
   6    23       0.904       0.875
   8    32       0.950       0.875
None    51       1.000       0.867
```

`make_moons`는 두 반달이 맞물린 모양으로 점을 만드는 함수인데, `noise=0.35`로 점들을 꽤 흩뜨려서 경계가 지저분합니다. 깊이가 깊어질수록 잎이 늘고(2개 → 51개) 학습 정확도는 꾸준히 올라 제한이 없을 때(`None`) 1.000이 됩니다. 그런데 시험 정확도는 깊이 2~4에서 0.900으로 가장 높았다가 그 뒤로는 오히려 떨어져 0.867까지 내려가요. 깊은 나무가 반달의 진짜 모양이 아니라 **잡음으로 튀어나온 점 하나하나를 외우는 질문**까지 만든 결과입니다.

1장에서 본 **과적합**이 바로 이 모습이에요. 학습 점수와 시험 점수가 벌어지기 시작하면 모델이 외우고 있다는 신호입니다. 결정 트리에서는 `max_depth`(최대 깊이), `min_samples_leaf`(잎에 남아야 할 최소 샘플 수) 같은 하이퍼파라미터로 나무를 적당히 잘라 줘요. 어디서 자를지는 3.10절에서 자동으로 찾아봅니다.

3.5 여럿이 함께: 랜덤 포레스트와 그래디언트 부스팅

나무 한 그루는 이해하기 쉽지만 작은 변화에도 휘청입니다. 학습 데이터가 조금만 달라져도 첫 질문이 바뀌고, 그 아래가 통째로 달라지거든요. 그래서 실전에서는 나무를 **여러 그루** 모아 씁니다. 모으는 방법이 크게 두 가지예요.

랜덤 포레스트: 각자 풀고 다수결로

퀴즈 방송에서 "청중에게 묻기" 찬스를 쓰면 정답률이 꽤 높다고 하죠. 한 사람 한 사람은 자주 틀려도, 서로 다른 이유로 틀리기 때문에 다수결을 하면 틀린 답이 서로 상쇄되는 겁니다. 이것을 **집단 지성**이라고 해요.

랜덤 포레스트(random forest) 는 나무들을 일부러 서로 다르게 키웁니다. 나무마다 학습 데이터를 무작위로 다시 뽑아(같은 샘플이 여러 번 뽑히기도 합니다) 주고, 질문을 고를 때도 특성 일부만 후보로 보여 줘요. 그렇게 다르게 자란 나무 수백 그루가 투표해서 답을 정합니다. 3.4절에서 외우기만 하던 반달 데이터에 써 보겠습니다.

```
from sklearn.datasets import make_moons
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# 3.4절과 같은 예제용 두 반달 데이터
X, y = make_moons(n_samples=400, noise=0.35, random_state=0)
X_학습, X_시험, y_학습, y_시험 = train_test_split(X, y, test_size=0.3, random_state=0)

나무 = DecisionTreeClassifier(random_state=0).fit(X_학습, y_학습)
print(f"끝까지 자란 나무 한 그루: 시험 정확도 {나무.score(X_시험, y_시험):.3f}")
for 그루_수 in [1, 10, 50, 200]:
    숲 = RandomForestClassifier(n_estimators=그루_수, random_state=0).fit(X_학습, y_학습)
    print(f"나무 {그루_수:>3}그루짜리 숲: 학습 {숲.score(X_학습, y_학습):.3f},"
          f" 시험 {숲.score(X_시험, y_시험):.3f}")
```

```
끝까지 자란 나무 한 그루: 시험 정확도 0.867
나무   1그루짜리 숲: 학습 0.939, 시험 0.833
나무  10그루짜리 숲: 학습 0.986, 시험 0.908
```

```
나무  50그루짜리 숲: 학습 1.000, 시험 0.900
나무 200그루짜리 숲: 학습 1.000, 시험 0.900
```

끝까지 자란 나무 한 그루는 시험 정확도 0.867이었습니다. 숲 안의 나무도 깊이 제한 없이 끝까지 자라서, 나무가 50그루 넘게 모이면 학습 정확도는 1.000이에요. 그런데도 시험 정확도는 0.900으로, 3.4절에서 깊이를 공들여 골랐을 때의 최고 점수와 같습니다. 나무 하나하나는 외우고 있지만, 각자 다른 잡음을 외웠기 때문에 투표를 하면 그 실수들이 묻히는 거예요. 1그루짜리 숲(0.833)은 데이터 일부만 보고 자란 나무 하나라서 오히려 가장 나빴고, 10그루만 모여도 크게 좋아졌습니다.

숲의 투표를 직접 들여다보고, 숲이 어떤 특성을 중요하게 여겼는지도 보겠습니다.

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

암 = load_breast_cancer()
X, y = 암.data, 1 - 암.target            # 1 = 악성
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

숲 = RandomForestClassifier(n_estimators=200, random_state=42).fit(X_학습, y_학습)
print(f"숲의 시험 정확도: {숲.score(X_시험, y_시험):.3f}")

# 확률이 가장 애매한 시험 환자 한 명에 대한 나무들의 투표
애매도 = abs(숲.predict_proba(X_시험)[:, 1] - 0.5)
환자 = X_시험[애매도.argmin()].reshape(1, -1)
표 = sum(int(그루.predict(환자)[0]) for 그루 in 숲.estimators_)
print(f"가장 애매한 환자: 악성 {표}표, 악성 아님 {200 - 표}표")

중요도 = 숲.feature_importances_
for i in 중요도.argsort()[::-1][:4]:
    print(f"{암.feature_names[i]:>20}: {중요도[i]:.3f}")
```

```
숲의 시험 정확도: 0.958
가장 애매한 환자: 악성 94표, 악성 아님 106표
     worst perimeter: 0.150
```

```
          worst area: 0.121
worst concave points: 0.103
 mean concave points: 0.101
```

숲.estimators_에는 학습된 나무 200그루가 들어 있습니다. 확률이 0.5에 가장 가까운 애매한 환자 한 명을 골라 나무마다 답을 물었더니 악성 94표, 악성 아님 106표로 팽팽했어요. 이렇게 표가 갈리는 환자는 사람이 한 번 더 살펴봐야 할 대상이라는 정보가 되기도 합니다.

feature_importances_는 각 특성이 나무들의 질문에서 얼마나 큰 역할을 했는지를 모두 더하면 1이 되도록 나눠 준 값입니다. 둘레, 넓이, 오목한 부위 수치가 위쪽에 있죠. 3.4절의 나무 한 그루가 첫 질문으로 고른 worst perimeter가 여기서도 1등입니다.

그래디언트 부스팅: 오답 노트를 이어서 고치기

시험공부를 할 때 오답 노트를 만들어 본 적 있나요? 문제집을 한 번 풀고, 틀린 문제만 모아 다시 공부하고, 그래도 틀린 걸 또 모아서 공부합니다. **부스팅(boosting)** 은 나무들을 이렇게 **차례로** 세웁니다. 랜덤 포레스트가 나무를 각자 따로 키워 투표한다면, 부스팅은 앞 나무가 틀린 만큼을 다음 나무가 고쳐요.

말로만 들으면 감이 잘 안 오니, 당뇨병 데이터로 직접 해 보겠습니다. 처음엔 모두를 평균으로 찍고, 그다음부터는 "지금까지 틀린 양(**잔차**)"을 맞히는 작은 나무를 하나씩 더해 갑니다.

```
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.tree import DecisionTreeRegressor

X, y = load_diabetes(return_X_y=True)

예측 = np.full(len(y), y.mean())         # 0단계: 모두 평균으로 찍기
print(f"0단계(평균으로 찍기) 평균 오차(RMSE): {np.sqrt(np.mean((y - 예측) ** 2)):.1f}")
학습률 = 0.3
for 단계 in range(1, 6):
    잔차 = y - 예측                       # 지금까지 틀린 만큼 (오답 노트)
    작은_나무 = DecisionTreeRegressor(max_depth=2, random_state=0).fit(X, 잔차)
    예측 = 예측 + 학습률 * 작은_나무.predict(X)   # 오답을 조금만 고친다
    오차 = np.sqrt(np.mean((y - 예측) ** 2))
    print(f"{단계}단계 뒤 평균 오차(RMSE): {오차:.1f}")
```

```
0단계(평균으로 찍기) 평균 오차(RMSE): 77.0
1단계 뒤 평균 오차(RMSE): 68.0
2단계 뒤 평균 오차(RMSE): 62.2
3단계 뒤 평균 오차(RMSE): 58.6
4단계 뒤 평균 오차(RMSE): 56.1
5단계 뒤 평균 오차(RMSE): 54.3
```

평균으로만 찍었을 때 평균 오차는 77.0이었습니다. 질문 두 번째리 작은 나무로 잔차를 맞혀 더하니 68.0, 또 하나를 더하니 62.2, 다섯 그루 뒤에는 54.3까지 내려가요. 나무 하나하나는 아주 약한 모델인데, 서로의 빈틈을 이어서 메우면서 점점 강해집니다.

`학습률 = 0.3`은 나무가 알려 준 고칠 양을 그대로 다 반영하지 않고 30%만 반영한다는 뜻입니다. 오답 노트를 한 번에 통째로 믿으면 그 노트의 실수까지 따라 하게 되니, 조금씩 고치고 다음 나무에게 기회를 넘기는 거예요. 이 "틀린 만큼, 조금씩 고친다"는 생각은 5장의 경사하강법과 같은 뿌리를 가지고 있습니다. 부스팅의 이름에 **그래디언트**(기울기)가 붙은 이유이기도 해요.

실전에서는 이 과정을 손으로 짜지 않고 scikit-learn의 `HistGradientBoostingClassifier`(회귀는 `HistGradientBoostingRegressor`)를 씁니다. 특성 값을 최대 255개 구간으로 묶어서(히스토그램) 질문 후보를 줄인 덕분에 큰 데이터에서도 빠른 부스팅 모델이에요.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
y = 1 - y                                  # 1 = 악성
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

부스팅 = HistGradientBoostingClassifier(random_state=42).fit(X_학습, y_학습)
print(f"그래디언트 부스팅 시험 정확도: {부스팅.score(X_시험, y_시험):.3f}")
print("실제로 만든 나무 수:", 부스팅.n_iter_)

# 값의 10%를 일부러 지워도(결측치) 그대로 학습한다
rng = np.random.default_rng(0)
X_구멍 = X_학습.copy()
```

```
X_구멍[rng.random(X_구멍.shape) < 0.1] = np.nan
print("빈칸 개수:", np.isnan(X_구멍).sum())
부스팅.fit(X_구멍, y_학습)
print(f"빈칸 있는 데이터로 배운 뒤 시험 정확도: {부스팅.score(X_시험, y_시험):.3f}")
```

```
그래디언트 부스팅 시험 정확도: 0.965
실제로 만든 나무 수: 100
빈칸 개수: 1311
빈칸 있는 데이터로 배운 뒤 시험 정확도: 0.972
```

설정을 하나도 바꾸지 않았는데 시험 정확도 0.965가 나왔고, 나무 100그루를 차례로 세웠습니다(`max_iter`의 기본값). 아래 절반은 이 모델의 숨은 장점입니다. 학습 데이터의 10%를 일부러 지워 빈칸(`NaN`) 1,311개를 만들었는데도 오류 없이 학습했어요. 2장에서는 결측치를 채우는 단계를 파이프라인에 넣었지만, `HistGradientBoosting` 계열은 질문마다 "빈칸은 어느 쪽으로 보낼지"까지 스스로 배워서 빈칸을 그대로 받습니다. 시험 정확도가 0.972로 조금 오른 건 빈칸 때문에 좋아졌다기보다, 데이터가 조금 달라지면 결과도 이 정도는 흔들린다는 정도로 읽어 두세요. 그 흔들림을 다루는 법이 3.9절의 교차 검증입니다.

Q

Q. XGBoost나 LightGBM이라는 이름도 많이 들리던데요? 둘 다 그래디언트 부스팅을 아주 빠르게 구현한 별도 라이브러리입니다. 데이터 분석 대회에서 표 데이터 부문을 오랫동안 휩쓸어서 유명해졌어요. scikit-learn의 `HistGradientBoosting`은 LightGBM의 방법을 본떠 만든 것이라 생각이 같고, `fit`·`predict` 약속도 똑같이 따릅니다. 이 장에서 익힌 사용법과 하이퍼파라미터 감각은 그대로 옮겨 쓸 수 있으니, 데이터가 수백만 줄을 넘어갈 때 바꿔 보세요.

3.6 정답 없이 묶기: k-means 군집

깃발 세 개 주위로 모이기

지금까지는 정답 `y`를 주고 배우는 **지도 학습**이었습니다. 이번에는 정답이 없어요. 쇼핑몰 고객 데이터는 많은데 "이 고객은 어떤 유형"이라는 레이블은 아무도 붙여 두지 않은 상황입니다. 이럴 때 비슷한 것끼리 무리를 짓는 것이 **군집(clustering)** 이고, 가장 널리 쓰는 방법이 **k-means**입니다.

운동장에 학생들이 흩어져 있고 선생님이 깃발 세 개를 아무 데나 꽂았다고 해 봅시다.

1. 학생들은 **가장 가까운 깃발**로 모입니다.
2. 선생님은 각 무리의 **한가운데**로 깃발을 옮깁니다.
3. 깃발이 옮겨졌으니 학생들은 다시 가장 가까운 깃발을 찾아 움직입니다.

2와 3을 깃발이 더 움직이지 않을 때까지 반복하면, 서로 가까운 학생끼리 세 무리가 지어져요. 깃발의 개수가 k, 깃발 위치가 무리의 평균(mean)이라서 k-means입니다.

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# 예제용으로 만든 데이터: 정답 레이블 없이 점 300개만 있다고 하자
X, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=7)

군집 = KMeans(n_clusters=3, n_init=10, random_state=0)
번호 = 군집.fit_predict(X)              # 배우고 곧바로 각 점의 무리 번호 받기
print("처음 10개 점의 무리 번호:", 번호[:10])
print("무리별 점 개수:", np.bincount(번호))
print("무리 중심:\n", 군집.cluster_centers_.round(2))
print("새 점 두 개의 무리:", 군집.predict([[9.0, 1.0], [-8.0, 5.0]]))
```

```
처음 10개 점의 무리 번호: [0 2 2 0 1 2 0 1 2 0]
무리별 점 개수: [100 100 100]
무리 중심:
 [[-8.55  5.57]
 [ 9.63  0.54]
 [-1.26  4.38]]
새 점 두 개의 무리: [1 0]
```

`make_blobs`로 세 덩어리 모양의 예제용 점 300개를 만들었고, 정답 레이블(`_`)은 버리고 점의 위치만 줬습니다. k-means는 이 점들을 100개씩 세 무리로 나눴어요. `cluster_centers_`가 최종 깃발 위치이고, `predict`로 새 점을 주면 가장 가까운 깃발의 번호를 돌려줍니다. (9, 1)은 1번 무리, (-8, 5)는 0번 무리예요.

무리 번호 0, 1, 2에는 아무 의미가 없다는 점에 주의하세요. "0번이 우수 고객"처럼 정해진 게 아니라 그냥 이름표일 뿐이고, 씨앗을 바꾸면 번호가 뒤바뀔 수도 있습니다. 각 무리가 어떤 무리인지는 중심값과 소속 데이터를 보고 사람이 해석해야 해요. `n_init=10`은 깃발을 처음 꽂는 위치를 10번 바

꿔 시도해서 가장 좋은 결과를 고르라는 뜻입니다. 처음 위치가 나쁘면 이상한 무리로 굳어 버릴 수 있거든요.

깃발은 몇 개를 꽂아야 할까요?

위에서는 덩어리가 셋인 걸 알고 `n_clusters=3`을 줬지만, 실제 데이터에서는 무리가 몇 개인지 모릅니다. k를 바꿔 가며 두 가지 점수를 보겠습니다.

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score

X, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=7)

print(" k   관성(inertia)   실루엣 점수")
for k in range(2, 7):
    군집 = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    print(f"{k:>2} {군집.inertia_:>14.1f} {silhouette_score(X, 군집.labels_):>13.3f}")
```

```
 k   관성(inertia)   실루엣 점수
 2         3302.8         0.745
 3          573.4         0.790
 4          501.4         0.647
 5          434.8         0.471
 6          374.2         0.329
```

- **관성(inertia)**: 각 점에서 자기 깃발까지 거리의 제곱을 모두 더한 값입니다. 깃발을 많이 꽂을수록 누구나 가까운 깃발을 갖게 되니 k가 커지면 **항상** 줄어들어요. 그래서 가장 작은 값을 고르면 안 되고, **줄어드는 폭이 갑자기 완만해지는 곳**을 봅니다. 2에서 3으로 갈 때 3302.8에서 573.4로 크게 줄었다가, 그 뒤로는 조금씩만 줄죠. 그래프로 그리면 팔꿈치처럼 꺾이는 곳이라 **엘보(elbow) 방법**이라고 부릅니다.
- **실루엣 점수(silhouette score)**: 점마다 "우리 무리와는 얼마나 가깝고, 옆 무리와는 얼마나 먼가"를 -1~1 사이로 잰 값의 평균입니다. 클수록 무리가 또렷하게 갈렸다는 뜻이에요. k=3일 때 0.790으로 가장 높습니다.

두 점수 모두 k=3을 가리켰습니다. 실제 데이터에서는 이렇게 깔끔하지 않은 경우가 많아서, 점수는 참고로 삼고 "몇 무리로 나눠야 쓸모가 있는가"라는 업무 판단을 함께 해요.

거리를 쓰는 모델은 눈금을 맞춰야 합니다

k-means는 "가까운 깃발"을 찾으니 **거리**가 전부입니다. 그런데 특성마다 단위가 크게 다르면 어떻게 될까요? 와인 178병의 화학 성분 13가지로 무리를 지어 보고, 실제 품종(정답은 채점할 때만 씁니다)과 얼마나 맞는지 보겠습니다.

```
from sklearn.cluster import KMeans
from sklearn.datasets import load_wine
from sklearn.metrics import adjusted_rand_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

와인 = load_wine()
X, 품종 = 와인.data, 와인.target          # 품종은 채점할 때만 쓴다
print("특성 범위 예:", 와인.feature_names[12], X[:, 12].min(), "~", X[:, 12].max(),
      "/", 와인.feature_names[7], X[:, 7].min(), "~", X[:, 7].max())

그대로 = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(X)
맞춘_뒤 = make_pipeline(StandardScaler(),
                        KMeans(n_clusters=3, n_init=10, random_state=0)).fit_predict(X)
print(f"스케일링 없이: 품종과 일치도 {adjusted_rand_score(품종, 그대로):.3f}")
print(f"스케일링 후:   품종과 일치도 {adjusted_rand_score(품종, 맞춘_뒤):.3f}")
```

```
특성 범위 예: proline 278.0 ~ 1680.0 / nonflavanoid_phenols 0.13 ~ 0.66
스케일링 없이: 품종과 일치도 0.371
스케일링 후:   품종과 일치도 0.897
```

proline이라는 성분은 278~1680 범위인데, nonflavanoid_phenols는 0.13~0.66 범위입니다. 그대로 거리를 재면 proline의 차이가 거리를 거의 혼자 결정해 버려서, 나머지 12가지 성분은 무시되다시피 해요. 그 결과 품종과의 일치도가 0.371에 그쳤습니다. 표준화를 먼저 하니 모든 성분이 같은 눈금에서 목소리를 내게 되어 일치도가 0.897로 올랐어요.

`adjusted_rand_score`는 두 가지 묶음 방법이 얼마나 같은지를 재는 점수로, 1이면 완전히 같고 무작위로 묶으면 0 근처입니다. 무리 번호 자체는 의미가 없으니 "같은 품종끼리 같은 무리에 들어갔는가"만 봐요. 1장의 최근접 이웃도 거리를 쓰는 모델이라 같은 이유로 표준화가 중요했습니다. 반대로 결정 트리 계열은 "이 값보다 큰가"만 묻기 때문에 눈금에 영향을 받지 않아요.

3.7 분류 모델 평가하기: 정확도만 보면 속는 이유

99% 정확한 엉터리 검사기

코로나19가 유행할 때 신속 항원 검사를 많이 했죠. 어떤 검사기가 "정확도 99%"라고 광고한다면 믿고 써도 될까요? 감염자가 100명 중 1명뿐인 상황에서, **누구에게나 무조건 "음성"이라고 말하는 검사기**를 생각해 봅시다.

```
import numpy as np
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, recall_score

# 예제용으로 만든 데이터: 1,000명 중 10명(1%)만 감염자(1)
rng = np.random.default_rng(0)
y = np.zeros(1000, dtype=int)
y[rng.choice(1000, size=10, replace=False)] = 1
X = rng.normal(size=(1000, 3))          # 의미 없는 특성 (게으른 검사기는 보지도 않는다)

게으른_검사기 = DummyClassifier(strategy="most_frequent").fit(X, y)
예측 = 게으른_검사기.predict(X)          # 누구에게나 "음성"
print("양성 판정 수:", 예측.sum())
print(f"정확도: {accuracy_score(y, 예측):.3f}")
print(f"감염자 중 찾아낸 비율(재현율): {recall_score(y, 예측):.3f}")
```

```
양성 판정 수: 0
정확도: 0.990
감염자 중 찾아낸 비율(재현율): 0.000
```

`DummyClassifier(strategy="most_frequent")`는 특성을 보지도 않고 학습 데이터에서 가장 흔한 답만 말하는 "게으른" 모델입니다. 양성 판정을 한 번도 하지 않았는데 정확도는 0.990이에요. 1,000명 중 990명이 원래 음성이니까요. 그런데 정작 찾아야 할 감염자 10명은 **한 명도** 찾지 못했습니다(재현율 0). 찾고 싶은 쪽이 드물 때 정확도는 이렇게 사람을 속입니다. 그래서 다른 눈금들이 필요해요. 이런 불균형 데이터를 학습 단계에서 다루는 방법은 2장에서 봤으니, 여기서는 **채점하는 법**에 집중하겠습니다.

Q

Q. 이렇게 뻔한 모델을 왜 만들어 보나요? 기준선이 되기 때문입니다. 실제 프로젝트에서 어떤 모델의 정확도가 0.95라고 하면 좋아 보이지만, 아무것도 안 하는 `Dummy` 모델이 0.94라면 이야기가 완전히 달라져요. 새 모델을 만들면 언제나 `DummyClassifier`(회귀는 `DummyRegressor`)의 점수부터 재 두는 습관을 들이세요. 3.13절 실습에서도 그렇게 시작합니다.

혼동 행렬: 네 가지 칸

검사 결과는 네 가지 중 하나입니다. 감염자에게 양성(맞음), 감염자에게 음성(놓침), 비감염자에게 양성(헛경보), 비감염자에게 음성(맞음). 이 네 칸을 세어 놓은 표가 **혼동 행렬(confusion matrix)**이에요.

	예측: 음성(0)	예측: 양성(1)
실제: 음성(0)	TN 진짜 음성	FP 거짓 양성 (헛경보)
실제: 양성(1)	FN 거짓 음성 (놓침)	TP 진짜 양성

이름이 헷갈리면 뒤 글자부터 읽으세요. P/N은 모델이 **뭐라고 말했는지**(양성/음성), 앞의 T/F는 그 말이 **맞았는지**입니다. FN은 "음성이라고 말했는데(N) 틀렸다(F)", 즉 놓친 환자예요. 유방암 데이터의 로지스틱 회귀로 네 칸을 세고, 여기서 나오는 지표들을 손으로 계산해 보겠습니다.

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, f1_score, precision_score,
                             recall_score)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
y = 1 - y                                # 1 = 악성 (찾고 싶은 쪽 = 양성 판정)
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
모델 = make_pipeline(StandardScaler(), LogisticRegression()).fit(X_학습, y_학습)
예측 = 모델.predict(X_시험)
```

```
표 = confusion_matrix(y_시험, 예측)
print(표)
tn, fp, fn, tp = 표.ravel()
print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")

정밀도 = tp / (tp + fp)
재현율 = tp / (tp + fn)
f1 = 2 * 정밀도 * 재현율 / (정밀도 + 재현율)
print(f"직접 계산: 정밀도 {정밀도:.3f}, 재현율 {재현율:.3f}, F1 {f1:.3f}")
print(f"sklearn : 정밀도 {precision_score(y_시험, 예측):.3f}, "
      f"재현율 {recall_score(y_시험, 예측):.3f}, F1 {f1_score(y_시험, 예측):.3f}")
```

```
[[89  1]
 [ 4 49]]
TN=89, FP=1, FN=4, TP=49
직접 계산: 정밀도 0.980, 재현율 0.925, F1 0.951
sklearn : 정밀도 0.980, 재현율 0.925, F1 0.951
```

시험 환자 143명 중 악성 아님을 맞힌 TN이 89명, 악성을 맞힌 TP가 49명입니다. 틀린 건 헛경보(FP) 1명과 놓친 환자(FN) 4명이에요. 이 네 숫자에서 세 가지 지표가 나옵니다.

- **정밀도(precision)** = TP / (TP + FP): 모델이 **"양성"이라고 말한 것 중** 진짜 양성의 비율. "양성 판정을 믿어도 되는가?"에 답합니다. 50번 양성이라고 했는데 49번이 맞아서 0.980이에요.
- **재현율(recall)** = TP / (TP + FN): **진짜 양성 중** 모델이 찾아낸 비율. "환자를 빠뜨리지 않는가?"에 답합니다. 악성 53명 중 49명을 찾아 0.925예요. 앞의 게으른 검사기는 이 값이 0이었죠.
- **F1 점수** = 2 × 정밀도 × 재현율 / (정밀도 + 재현율): 두 값의 **조화 평균**입니다. 둘 중 하나라도 낮으면 F1이 크게 떨어져서, 한쪽만 좋은 모델을 걸러 냅니다. 여기서는 0.951이에요.

직접 계산한 값과 `precision_score`, `recall_score`, `f1_score`가 돌려준 값이 똑같습니다. 공식이 이게 전부라는 뜻이에요.

문턱을 움직이면 생기는 일

정밀도와 재현율은 시소처럼 움직입니다. 3.3절에서 로지스틱 회귀는 악성 확률이 0.5를 넘으면 양성이라고 답한다고 했죠. 이 **문턱**을 바꿔 보겠습니다.

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
y = 1 - y                                  # 1 = 악성
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
모델 = make_pipeline(StandardScaler(), LogisticRegression()).fit(X_학습, y_학습)
악성_확률 = 모델.predict_proba(X_시험)[:, 1]

print("문턱   FP   FN   정밀도   재현율")
for 문턱 in [0.9, 0.5, 0.2, 0.05]:
    예측 = (악성_확률 >= 문턱).astype(int)  # 문턱을 넘으면 양성 판정
    tn, fp, fn, tp = confusion_matrix(y_시험, 예측).ravel()
    print(f"{문턱:>4} {fp:>4} {fn:>4} {tp / (tp + fp):>8.3f} {tp / (tp + fn):>8.3f}")
```

```
문턱   FP   FN   정밀도   재현율
 0.9    0   10    1.000    0.811
 0.5    1    4    0.980    0.925
 0.2    3    2    0.944    0.962
0.05   11    0    0.828    1.000
```

문턱을 0.9로 높이면 아주 확실할 때만 양성이라고 해서 헛경보는 0건, 정밀도는 1.000이 됩니다. 대신 놓친 환자가 10명으로 늘어요. 문턱을 0.05로 낮추면 조금만 의심스러워도 양성이라고 해서 악성은 한 명도 놓치지 않지만(재현율 1.000) 헛경보가 11건으로 늘어납니다.

어느 쪽이 좋은지는 **틀렸을 때의 대가**가 정합니다. 암 검진에서 환자를 놓치면(FN) 치료 시기를 잃지만, 헛경보(FP)는 정밀 검사 한 번이면 바로잡혀요. 그래서 재현율을 중시해 문턱을 낮춥니다. 반

대로 스팸 필터는 중요한 메일을 스팸함에 넣는 것(FP)이 더 큰 문제라 정밀도를 중시하죠. 모델을 만든 다음 "문턱을 어디에 둘까"는 업무를 아는 사람과 함께 정해야 하는 질문입니다.

ROC-AUC: 문턱과 상관없이 매기는 점수

문턱마다 정밀도와 재현율이 달라지니, 문턱을 정하기 전에 "모델 자체가 얼마나 잘 가려내는가"를 재고 싶을 때가 있습니다. 문턱을 1에서 0까지 내리면서 **거짓 경보율**(FP / 실제 음성 수)을 가로축, **재현율**을 세로축에 찍은 선을 ROC **곡선**이라 하고, 그 아래 넓이를 ROC-AUC라고 해요.

```
from sklearn.datasets import load_breast_cancer
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, roc_curve
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)
y = 1 - y                                  # 1 = 악성
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

모델들 = {
    "아무 정보 없이 찍기": DummyClassifier(strategy="prior"),
    "질문 두 번짜리 나무": DecisionTreeClassifier(max_depth=2, random_state=42),
    "로지스틱 회귀": make_pipeline(StandardScaler(), LogisticRegression()),
}
for 이름, 모델 in 모델들.items():
    점수 = 모델.fit(X_학습, y_학습).predict_proba(X_시험)[:, 1]
    print(f"{이름}: ROC-AUC {roc_auc_score(y_시험, 점수):.3f}")

# AUC의 뜻: 악성 한 명과 악성 아닌 한 명을 뽑았을 때 악성 쪽 점수가 더 높을 확률
악성, 정상 = 점수[y_시험 == 1], 점수[y_시험 == 0]
이김 = (악성[:, None] > 정상[None, :]).mean() + 0.5 * (악성[:, None] == 정상[None, :]).mean()
print(f"짝 {len(악성) * len(정상)}개를 직접 비교한 비율: {이김:.3f}")
```

```
fpr, tpr, 문턱들 = roc_curve(y_시험, 점수)
print("FPR(거짓 경보율):", fpr.round(3).tolist())
print("TPR(재현율):     ", tpr.round(3).tolist())
```

```
아무 정보 없이 찍기: ROC-AUC 0.500
질문 두 번짜리 나무: ROC-AUC 0.880
로지스틱 회귀: ROC-AUC 0.996
짝 4770개를 직접 비교한 비율: 0.996
FPR(거짓 경보율): [0.0, 0.0, 0.0, 0.011, 0.011, 0.044, 0.044, 0.122, 0.122, 1.0]
TPR(재현율):      [0.0, 0.019, 0.906, 0.906, 0.962, 0.962, 0.981, 0.981, 1.0, 1.0]
```

아무 정보 없이 찍는 모델은 0.500, 질문 두 번짜리 나무는 0.880, 로지스틱 회귀는 0.996입니다. 넓이는 0.5(동전 던지기)에서 1(완벽)까지예요.

ROC-AUC에는 기억하기 쉬운 뜻이 있습니다. **악성 환자 한 명과 악성 아닌 환자 한 명을 무작위로 뽑았을 때, 모델이 악성 환자에게 더 높은 점수를 줄 확률**이에요. 코드 아랫부분에서 이 뜻대로 직접 계산해 봤습니다. 악성 53명 × 악성 아님 90명 = 4,770쌍을 모두 비교해서 악성 쪽 점수가 높은 비율을 구했더니(같으면 반 개로 셉니다) `roc_auc_score`와 같은 0.996이 나왔어요. `(53, 1)`과 `(1, 90)`을 비교해 4,770칸을 한 번에 만든 것은 1장 1.5절에서 거리를 한꺼번에 잴 때 쓴 브로드캐스팅 그대로입니다.

마지막 두 줄은 `roc_curve`가 돌려준 곡선 위의 점들입니다. 거짓 경보율이 0.011일 때 재현율이 이미 0.962에 이르러요. 왼쪽 위 모서리에 바짝 붙은 곡선일수록 넓이가 커집니다. 1권 20장의 matplotlib로 `plt.plot(fpr, tpr)`을 하면 곡선을 직접 그려 볼 수 있어요.

한눈에 보는 성적표

지표를 하나씩 부르기 번거로우면 `classification_report`가 클래스별 성적표를 한 번에 만들어 줍니다.

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
X, y = load_breast_cancer(return_X_y=True)
y = 1 - y                                  # 1 = 악성
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
모델 = make_pipeline(StandardScaler(), LogisticRegression()).fit(X_학습, y_학습)

print(classification_report(y_시험, 모델.predict(X_시험),
                            target_names=["benign(0)", "malignant(1)"], digits=3))
```

```
              precision    recall  f1-score   support

   benign(0)      0.957     0.989     0.973        90
malignant(1)      0.980     0.925     0.951        53

    accuracy                          0.965       143
   macro avg      0.968     0.957     0.962       143
weighted avg      0.966     0.965     0.965       143
```

malignant(1) 줄의 정밀도 0.980, 재현율 0.925, F1 0.951은 앞에서 계산한 값과 같습니다. benign(0) 줄은 "악성 아님"을 양성으로 보고 계산한 값이에요. `support`는 각 클래스의 실제 개수(90명, 53명)입니다. 아래쪽 `macro avg`는 클래스별 값을 그냥 평균한 것이고, `weighted avg`는 개수만큼 가중해서 평균한 것이에요. 클래스가 불균형할 때 작은 클래스의 성적을 놓치지 않으려면 macro 평균을 보세요. 표 이름을 영어로 단 것은 한글이 섞이면 출력 칸이 어긋나기 때문입니다.

3.8 회귀 모델 평가하기: MAE, RMSE, R^2

숫자를 예측하는 회귀에서는 "맞았다/틀렸다"가 아니라 **얼마나 빗나갔는가**를 잽니다. 날씨 예보로 생각해 보세요. 최고 기온 25도를 예보했는데 실제가 27도면 2도 빗나간 거죠. 이 빗나간 양(**오차**)을 모아서 요약하는 방법이 세 가지 있습니다.

```
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.dummy import DummyRegressor
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score, root_mean_squared_error
from sklearn.model_selection import train_test_split

X, y = load_diabetes(return_X_y=True)
X_학습, X_시험, y_학습, y_시험 = train_test_split(X, y, test_size=0.25, random_state=42)
예측 = LinearRegression().fit(X_학습, y_학습).predict(X_시험)

오차 = y_시험 - 예측
mae = np.abs(오차).mean()
rmse = np.sqrt((오차 ** 2).mean())
r2 = 1 - (오차 ** 2).sum() / ((y_시험 - y_시험.mean()) ** 2).sum()
print(f"직접 계산: MAE {mae:.1f}, RMSE {rmse:.1f}, R² {r2:.3f}")
print(f"sklearn : MAE {mean_absolute_error(y_시험, 예측):.1f}, "
      f"RMSE {root_mean_squared_error(y_시험, 예측):.1f}, R² {r2_score(y_시험, 예측):.3f}")

평균_찍기 = DummyRegressor(strategy="mean").fit(X_학습, y_학습).predict(X_시험)
print(f"평균으로 찍기: MAE {mean_absolute_error(y_시험, 평균_찍기):.1f}, "
      f"RMSE {root_mean_squared_error(y_시험, 평균_찍기):.1f}, R² {r2_score(y_시험, 평균_찍
      ↪ 기):.3f}")

# 큰 실수 하나가 섞이면?
망친_예측 = 예측.copy()
망친_예측[0] += 300
print(f"한 명을 300 틀리면: MAE {mean_absolute_error(y_시험, 망친_예측):.1f}, "
      f"RMSE {root_mean_squared_error(y_시험, 망친_예측):.1f}")
```

```
직접 계산: MAE 41.5, RMSE 53.4, R² 0.485
sklearn : MAE 41.5, RMSE 53.4, R² 0.485
평균으로 찍기: MAE 65.5, RMSE 74.9, R² -0.014
한 명을 300 틀리면: MAE 42.8, RMSE 56.8
```

- MAE(**평균 절대 오차**): 오차의 절댓값을 평균한 값입니다. "평균적으로 41.5만큼 빗나간다"처럼 목표값과 같은 단위로 바로 읽혀서 설명하기 가장 쉬워요.
- RMSE(**평균 제곱근 오차**): 오차를 제곱해서 평균한 뒤 제곱근을 씌운 값입니다. 단위는 MAE와 같지만, 제곱 때문에 **큰 실수에 더 무겁게 벌점**을 줘요. 마지막 줄에서 143명 중 한 명만

300 더 틀리게 했더니 MAE는 41.5에서 42.8로 조금 늘었지만 RMSE는 53.4에서 56.8로 훨씬 크게 뛰었습니다. 큰 실수 한 번이 치명적인 일(재고 예측, 전력 수요 예측 등)이면 RMSE를 봅니다.

- **R^2(결정 계수)**: "평균으로 찍는 것보다 얼마나 나은가"를 0~1 근처로 나타낸 값입니다. 1이면 완벽, 0이면 평균으로 찍는 것과 같은 수준, 음수면 평균보다도 못하다는 뜻이에요. 공식은 `1 - (모델의 오차 제곱합) / (평균으로 찍었을 때의 오차 제곱합)`으로, 코드의 `r2` 줄 그대로입니다.

평균으로 찍는 `DummyRegressor`의 R^2가 0 근처(-0.014)로 나온 것을 보세요. 정확히 0이 아닌 건, 학습 세트의 평균으로 시험 세트를 찍었기 때문이에요. 선형 회귀의 R^2 0.485는 "평균으로 찍었을 때의 오차 제곱합을 절반 가까이 줄였다"로 읽으면 됩니다. scikit-learn 회귀 모델의 `score`가 돌려주는 값이 바로 이 R^2예요.

지표	단위	큰 실수에	이럴 때
MAE	목표값과 같음	덜 민감	사람에게 "평균 몇 만큼 틀린다"고 설명할 때
RMSE	목표값과 같음	민감	큰 실수를 특히 피해야 할 때, 모델끼리 비교할 때
R^2	없음(1이 최고)	민감	데이터가 다른 문제끼리 "얼마나 설명하나"를 비교할 때

3.9 교차 검증: 모의고사는 한 번보다 다섯 번

시험 세트 한 번은 운이 섞입니다

학생 실력을 모의고사 한 번으로 판단하면 어떨까요? 그날 마침 아는 문제가 많이 나왔을 수도, 컨디션이 나빴을 수도 있습니다. 모델도 마찬가지예요. 2장에서 데이터를 학습/시험으로 한 번 나눠 채점했는데, **어떻게 나뉘었느냐**에 따라 점수가 얼마나 달라지는지 보겠습니다.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)
점수들 = []
for 씨앗 in range(10):                  # 나누는 방법만 바꿔 열 번
    X_학습, X_시험, y_학습, y_시험 = train_test_split(
        X, y, test_size=0.25, stratify=y, random_state=씨앗)
    나무 = DecisionTreeClassifier(max_depth=3, random_state=0).fit(X_학습, y_학습)
    점수들.append(나무.score(X_시험, y_시험))
print("시험 정확도:", [round(s, 3) for s in 점수들])
print(f"가장 낮을 때 {min(점수들):.3f}, 가장 높을 때 {max(점수들):.3f}")
```

```
시험 정확도: [0.916, 0.944, 0.93, 0.951, 0.937, 0.944, 0.951, 0.923, 0.916, 0.916]
가장 낮을 때 0.916, 가장 높을 때 0.951
```

모델도, 데이터도, 설정도 같습니다. 나누는 씨앗만 바꿨을 뿐인데 시험 정확도가 0.916에서 0.951까지 오르내려요. 두 모델의 차이가 0.02쯤이라면, 그 차이가 실력 차이인지 나누기 운인지 이 방법으로는 알 수 없습니다.

K-겹 교차 검증

해결책은 모의고사를 여러 번 보는 겁니다. 데이터를 똑같은 크기의 묶음 K개로 나누고, 한 묶음씩 돌아가며 채점용으로 쓰는 거예요. K=5라면 이렇습니다.

- 1회차: 묶음 2~5로 배우고 묶음 1로 채점
- 2회차: 묶음 1, 3~5로 배우고 묶음 2로 채점
- …
- 5회차: 묶음 1~4로 배우고 묶음 5로 채점

모든 샘플이 정확히 한 번씩 채점에 쓰이고, 점수 다섯 개의 평균과 흩어진 정도를 함께 봅니다. 이것이 **K-겹 교차 검증**(K-fold cross validation) 이에요. `KFold`로 묶음을 직접 나눠 보겠습니다.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import KFold
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)
접기 = KFold(n_splits=5, shuffle=True, random_state=0)

점수들 = []
for 번호, (학습_칸, 검증_칸) in enumerate(접기.split(X), start=1):
    나무 = DecisionTreeClassifier(max_depth=3, random_state=0)
    나무.fit(X[학습_칸], y[학습_칸])          # 네 묶음으로 배우고
    점수 = 나무.score(X[검증_칸], y[검증_칸])  # 남은 한 묶음으로 채점
    점수들.append(점수)
    print(f"{번호}번째: 학습 {len(학습_칸)}개, 검증 {len(검증_칸)}개 → 정확도 {점수:.3f}")
print(f"평균 {np.mean(점수들):.3f} ± {np.std(점수들):.3f}")
```

```
1번째: 학습 455개, 검증 114개 → 정확도 0.965
2번째: 학습 455개, 검증 114개 → 정확도 0.930
3번째: 학습 455개, 검증 114개 → 정확도 0.912
4번째: 학습 455개, 검증 114개 → 정확도 0.939
5번째: 학습 456개, 검증 113개 → 정확도 0.938
평균 0.937 ± 0.017
```

`접기.split(X)`는 회차마다 학습에 쓸 행 번호와 검증에 쓸 행 번호를 돌려줍니다. 569명을 다섯으로 나누니 검증 묶음이 114명 또는 113명이에요. 회차별 정확도가 0.912~0.965로 흔들리지만, 평균 0.937과 표준편차 0.017을 함께 보면 "이 모델은 대략 0.94 안팎, 위아래로 0.02쯤 흔들린다"는 훨씬 믿을 만한 그림이 나옵니다. `shuffle=True`는 나누기 전에 순서를 섞으라는 뜻인데, 왜 필요한지는 잠시 뒤에 봅니다.

cross_val_score와 cross_validate

반복문을 매번 짤 필요는 없습니다. 함수 하나로 같은 일을 해요.

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_validate
from sklearn.pipeline import make_pipeline
```

```
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
y = 1 - y                                  # 1 = 악성
모델 = make_pipeline(StandardScaler(), LogisticRegression())
접기 = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)

점수 = cross_val_score(모델, X, y, cv=접기)              # 기본 채점: 정확도
print("정확도 5개:", 점수.round(3), f"→ 평균 {점수.mean():.3f}")

결과 = cross_validate(모델, X, y, cv=접기, return_train_score=True,
                      scoring=["accuracy", "recall", "roc_auc"])
for 지표 in ["accuracy", "recall", "roc_auc"]:
    print(f"{지표:>8}: 학습 {결과['train_' + 지표].mean():.3f}, "
          f"검증 {결과['test_' + 지표].mean():.3f} ± {결과['test_' + 지표].std():.3f}")
```

```
정확도 5개: [0.956 0.974 0.982 1.    0.982] → 평균 0.979
accuracy: 학습 0.988, 검증 0.979 ± 0.014
  recall: 학습 0.976, 검증 0.958 ± 0.031
 roc_auc: 학습 0.998, 검증 0.995 ± 0.006
```

`cross_val_score`는 회차별 점수를 배열로 돌려줍니다. 여러 지표를 한 번에 보고 싶으면 `cross_validate`에 `scoring`으로 지표 이름 목록을 주면 돼요. `return_train_score=True`를 주면 학습 묶음의 점수도 함께 돌려주는데, 학습과 검증 점수의 차이로 과적합 정도를 볼 수 있습니다. 여기서는 정확도가 학습 0.988, 검증 0.979로 거의 붙어 있어서 로지스틱 회귀가 외우지 않고 있다는 걸 알 수 있어요. 재현율은 회차마다 ±0.031로 정확도보다 더 많이 흔들립니다. 악성 환자가 회차마다 40여 명뿐이라 한두 명만 놓쳐도 값이 크게 바뀌기 때문이에요.

여기서 꼭 짚을 점이 있습니다. 교차 검증에 `StandardScaler`만 따로 먼저 적용한 데이터를 넣지 않고, **스케일러가 들어 있는 파이프라인 통째로** 넣었어요. 그래야 회차마다 학습 묶음으로만 평균과 표준편차를 계산하고, 검증 묶음은 진짜 "처음 보는 데이터"로 남습니다. 2장에서 배운 데이터 누수를 교차 검증에서도 똑같이 막는 방법이에요.

StratifiedKFold: 묶음마다 비율 맞추기

그럼 `shuffle=True`는 왜 줬고, `StratifiedKFold`는 무엇이 다를까요? 극단적인 예를 보겠습니다. 붓꽃(iris) 데이터는 품종 세 가지가 50송이씩 **순서대로** 정렬되어 있어요.

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import KFold, StratifiedKFold, cross_val_score
from sklearn.tree import DecisionTreeClassifier

X, y = load_iris(return_X_y=True)
print("레이블 앞부분과 뒷부분:", y[:5], y[-5:])   # 품종 순서대로 정렬되어 있다

for 이름, 접기 in [("KFold(섞지 않음)", KFold(n_splits=3)),
                   ("StratifiedKFold", StratifiedKFold(n_splits=3))]:
    print(이름)
    for _, 검증_칸 in 접기.split(X, y):
        print("  검증 묶음의 품종별 개수:", np.bincount(y[검증_칸], minlength=3))
    점수 = cross_val_score(DecisionTreeClassifier(random_state=0), X, y, cv=접기)
    print("  정확도:", 점수.round(3))
```

```
레이블 앞부분과 뒷부분: [0 0 0 0 0] [2 2 2 2 2]
KFold(섞지 않음)
  검증 묶음의 품종별 개수: [50  0  0]
  검증 묶음의 품종별 개수: [ 0 50  0]
  검증 묶음의 품종별 개수: [ 0  0 50]
  정확도: [0. 0. 0.]
StratifiedKFold
  검증 묶음의 품종별 개수: [17 17 16]
  검증 묶음의 품종별 개수: [17 16 17]
  검증 묶음의 품종별 개수: [16 17 17]
  정확도: [0.98 0.94 0.98]
```

섞지 않은 `KFold(n_splits=3)`는 앞에서부터 50개씩 잘라서, 검증 묶음마다 한 품종만 들어갔습니다. 그러면 학습 묶음에는 그 품종이 하나도 없으니 모델은 그 품종을 배운 적이 없고, 정확도는 세 번 모두 0이에요. 시험 범위를 1단원만 공부시키고 3단원 시험을 본 꼴입니다.

`StratifiedKFold`는 묶음마다 **클래스 비율을 전체와 똑같이** 맞춰 나눕니다. 품종별로 16~17송이씩

고르게 들어갔고 정확도도 정상으로 돌아왔어요. 분류 문제에서는 `StratifiedKFold`를 기본으로 쓰세요. 특히 3.7절처럼 양성이 드문 데이터에서는 어떤 묶음에 양성이 하나도 안 들어가는 사고를 막아 줍니다. 사실 `cross_val_score(모델, X, y, cv=5)`처럼 숫자만 주면 scikit-learn은 분류 모델에 알아서 `StratifiedKFold`를 쓰지만, 섞지는 않습니다. 순서가 정렬된 데이터일 수 있으니 이 장의 예제처럼 `shuffle=True`와 `random_state`를 준 객체를 직접 넘기는 편이 안전해요. 시간 순서가 있는 데이터는 섞으면 안 된다는 점(미래로 과거를 맞히는 누수)은 2장에서 다뤘습니다.

3.10 하이퍼파라미터 탐색

손잡이 조합을 모두 돌려 보기: GridSearchCV

결정 트리의 깊이, 최근접 이웃의 이웃 수, 부스팅의 학습률처럼 사람이 정하는 설정을 **하이퍼파라미터**라고 했죠. 어떤 값이 좋은지는 데이터마다 다르니 결국 시험해 봐야 합니다. 라디오 주파수를 맞출 때 다이얼을 조금씩 돌려 가며 가장 깨끗하게 들리는 곳을 찾는 것과 같아요. 다이얼이 두 개라면 모든 조합을 바둑판(grid)처럼 다 시험해 볼 수 있습니다.

`GridSearchCV`는 후보 조합마다 교차 검증을 해서 가장 좋은 조합을 골라 줍니다. 1장에서 만난 최근접 이웃 모델의 이웃 수(`n_neighbors`)와 투표 방식(`weights`)을 탐색해 보겠습니다.

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
y = 1 - y                                  # 1 = 악성
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

파이프 = make_pipeline(StandardScaler(), KNeighborsClassifier())
후보 = {
    "kneighborsclassifier__n_neighbors": [1, 3, 5, 9, 15, 25],   # 단계이름__설정이름
    "kneighborsclassifier__weights": ["uniform", "distance"],
}
```

```
탐색 = GridSearchCV(파이프, 후보, scoring="f1",
                    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=0))
탐색.fit(X_학습, y_학습)                    # 6 × 2 = 12조합 × 5묶음 = 60번 학습

print("가장 좋은 조합:", 탐색.best_params_)
print(f"그 조합의 교차 검증 F1: {탐색.best_score_:.3f}")
표 = pd.DataFrame(탐색.cv_results_)[["param_kneighborsclassifier__n_neighbors",
                                    "param_kneighborsclassifier__weights",
                                    "mean_test_score", "rank_test_score"]]
표.columns = ["n_neighbors", "weights", "mean_f1", "rank"]
표 = 표.round(3)
print(표.sort_values("rank").head(4).to_string(index=False))
```

```
가장 좋은 조합: {'kneighborsclassifier__n_neighbors': 3, 'kneighborsclassifier__weights':
↪  'uniform'}
그 조합의 교차 검증 F1: 0.948
 n_neighbors  weights  mean_f1  rank
           3  uniform    0.948     1
           3 distance    0.948     1
           9 distance    0.943     3
           9  uniform    0.940     4
```

파이프라인 안 모델의 설정을 지정할 때는 **단계이름__설정이름** 처럼 밑줄 두 개로 잇습니다. `make_pipeline`은 단계 이름을 클래스 이름의 소문자로 자동으로 붙여서 `kneighborsclassifier`가 돼요. 이웃 수 6가지 × 투표 방식 2가지 = 12조합을 5겹 교차 검증으로 채점했으니 모델을 60번 학습한 셈입니다.

결과는 이웃 3명, `uniform`(모두 똑같이 한 표) 조합이 교차 검증 F1 0.948로 1등이었어요. 그런데 표를 보면 `distance`(가까운 이웃의 표를 더 무겁게)도 같은 점수로 공동 1등이고, 이웃 9명도 0.94대로 큰 차이가 없습니다. 3.9절에서 교차 검증 점수가 ±0.02쯤 흔들리는 걸 봤으니, 이 정도 차이는 사실상 비긴 거예요. 1등 조합에 너무 큰 의미를 두지 말고 "이웃 수 3~9 근처가 무난하다"는 감을 얻는 게 탐색의 진짜 수확입니다.

탐색에는 **학습 세트만** 넣었다는 점도 보세요. `GridSearchCV` 안의 교차 검증이 검증 역할을 하니, 시험 세트는 끝까지 아껴 둡니다. 탐색이 끝나면 `탐색` 객체가 자동으로 최고 조합을 학습 세트 전체에 다시 학습해 두어서(`refit=True`가 기본값), `탐색.predict()`처럼 바로 쓸 수 있어요.

조합이 너무 많을 때: RandomizedSearchCV

다이얼이 다섯 개이고 각각 10단계라면 조합이 10만 개입니다. 모두 돌리는 건 무리죠. 이럴 때는 다이얼 위치를 **무작위로** 몇십 번만 골라 시험하는 편이 효율적이에요. 하이퍼파라미터 중에는 결과를 크게 좌우하는 것과 별 영향이 없는 것이 섞여 있는데, 바둑판 탐색은 영향 없는 다이얼에도 똑같이 시간을 쓰지만 무작위 탐색은 중요한 다이얼의 여러 값을 골고루 시험하게 되거든요.

```
from scipy.stats import loguniform, randint
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold, train_test_split

X, y = load_breast_cancer(return_X_y=True)
y = 1 - y                                  # 1 = 악성
X_학습, X_시험, y_학습, y_시험 = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

분포 = {
    "learning_rate": loguniform(0.01, 0.3),   # 0.01~0.3을 자릿수 기준으로 고르게
    "max_leaf_nodes": randint(4, 32),         # 4 이상 32 미만 정수
    "min_samples_leaf": randint(5, 40),
}
탐색 = RandomizedSearchCV(HistGradientBoostingClassifier(random_state=0), 분포,
                          n_iter=15, scoring="roc_auc", random_state=0,
                          cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=0))
탐색.fit(X_학습, y_학습)                    # 15조합 × 5묶음 = 75번 학습

최고 = 탐색.best_params_
print(f"learning_rate={최고['learning_rate']:.4f}, max_leaf_nodes={최고['max_leaf_nodes']},"
      f" min_samples_leaf={최고['min_samples_leaf']}")
print(f"교차 검증 ROC-AUC: {탐색.best_score_:.4f}")
# best_estimator_는 최고 조합으로 학습 세트 전체를 다시 배운 모델
시험_점수 = roc_auc_score(y_시험, 탐색.predict_proba(X_시험)[:, 1])
print(f"시험 세트 ROC-AUC: {시험_점수:.4f}")
```

```
learning_rate=0.2790, max_leaf_nodes=28, min_samples_leaf=34
교차 검증 ROC-AUC: 0.9891
시험 세트 ROC-AUC: 0.9960
```

후보를 목록이 아니라 **분포**로 줍니다. `randint(4, 32)`는 4 이상 32 미만 정수 중 하나를, `loguniform(0.01, 0.3)`은 0.01~0.3 사이 값을 **자릿수 기준으로 고르게** 뽑아요. 학습률처럼 0.01과 0.1의 차이가 0.1과 0.2의 차이만큼 중요한 설정은 이렇게 뽑아야 작은 값 쪽도 충분히 시험됩니다. `n_iter=15`로 15조합만 뽑았으니 학습은 75번이에요.

찾은 조합의 교차 검증 ROC-AUC는 0.9891, 시험 세트에서 마지막에 한 번 잰 값은 0.9960입니다. 3.7절의 로지스틱 회귀(0.996)와 비슷한 수준이에요. 데이터가 작고 깔끔하면 단순한 모델이 이미 충분히 좋아서, 복잡한 모델을 탐색까지 해서 써도 크게 나아지지 않을 수 있습니다. 이것도 탐색이 알려 주는 중요한 정보예요.

Q

Q. 탐색이 너무 오래 걸리면 어떻게 하나요? `GridSearchCV`와 `RandomizedSearchCV`에 `n_jobs=-1`을 주면 CPU 코어를 모두 써서 여러 조합을 동시에 학습합니다. 그래도 느리면 데이터 일부로 넓게 탐색해서 범위를 좁힌 뒤 전체 데이터로 좁은 범위만 다시 탐색하세요. 더 영리하게 "지금까지 결과를 보고 다음에 시험할 조합을 고르는" 방법(Optuna 같은 라이브러리)도 있는데, 18장 실험 관리에서 딥러닝 학습과 함께 다룹니다.

3.11 모델 저장하고 불러오기

학습한 모델을 프로그램이 끝날 때마다 버리고 다시 학습할 수는 없겠죠. 1권 17장에서 파이썬 객체를 통째로 파일에 저장하는 pickle을 배웠습니다. scikit-learn 모델도 파이썬 객체라 pickle로 저장할 수 있는데, 큰 NumPy 배열을 더 효율적으로 저장해 주는 **joblib**을 쓰는 것이 관례예요.

```
from pathlib import Path

import joblib
import sklearn
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
X, y = load_breast_cancer(return_X_y=True)
y = 1 - y                                  # 1 = 악성
모델 = make_pipeline(StandardScaler(), LogisticRegression()).fit(X, y)

폴더 = Path("models")
폴더.mkdir(exist_ok=True)
경로 = 폴더 / "cancer_lr.joblib"
joblib.dump({"model": 모델, "sklearn_version": sklearn.__version__}, 경로)
print(f"저장: {경로} ({경로.stat().st_size:,}바이트)")

꾸러미 = joblib.load(경로)                  # 다른 프로그램에서 불러온다고 생각하자
불러온_모델 = 꾸러미["model"]
print("저장할 때 버전:", 꾸러미["sklearn_version"], "/ 지금 버전:", sklearn.__version__)
print("예측 비교(10명):", 모델.predict(X[::57]), 불러온_모델.predict(X[::57]))
print("확률까지 모두 같은가:", (모델.predict_proba(X) == 불러온_모델.predict_proba(X)).all())
print("단계 이름:", list(불러온_모델.named_steps))
```

```
저장: models/cancer_lr.joblib (2,342바이트)
저장할 때 버전: 1.9.1 / 지금 버전: 1.9.1
예측 비교(10명): [1 1 0 1 0 0 0 0 0 0] [1 1 0 1 0 0 0 0 0 0]
확률까지 모두 같은가: True
단계 이름: ['standardscaler', 'logisticregression']
```

세 가지를 눈여겨보세요.

- **파이프라인 통째로 저장했습니다.** 모델만 저장하면 불러온 쪽에서 표준화를 다시 해 줘야 하는데, 학습 때의 평균과 표준편차를 모르면 할 수가 없어요. 파이프라인을 저장하면 전처리 규칙까지 함께 들어 있어서 원본 데이터를 그대로 넣으면 됩니다. 불러온 뒤에도 단계 이름이 그대로 살아 있죠.
- **저장할 때의 scikit-learn 버전을 함께 적어 뒀습니다.** joblib 파일은 저장한 버전과 다른 버전에서 불러오면 경고가 나거나 동작이 달라질 수 있어요. 서비스 서버의 버전을 학습한 환경과 맞추는 것이 원칙이고, 그러려면 어떤 버전으로 저장했는지 알아야 합니다. 이렇게 딕셔너리로 모델과 정보를 함께 묶어 두면 나중에 편해요.
- **불러온 모델의 예측이 원래와 똑같습니다.** 확률값까지 모든 환자에 대해 같아요.

그리고 1권 17장의 경고가 여기에도 그대로 적용됩니다. joblib 파일도 내부는 pickle이라 **여는 순간 그 안의 코드가 실행될 수 있어요.** 인터넷에서 받은 모르는 모델 파일은 절대 `joblib.load`하지 마세요. 다른 언어나 다른 환경과 모델을 안전하게 주고받는 방법은 19장에서 ONNX로 다루고, 저장한 모델을 웹 서비스로 내보내는 일은 20장 FastAPI 서빙에서 합니다.

3.12 언제 딥러닝 대신 고전 머신러닝을 쓸까?

2권의 나머지는 거의 딥러닝 이야기입니다. 그럼 이 장에서 배운 모델들은 딥러닝을 배우기 전의 연습용일까요? 그렇지 않습니다. 현업에서는 오히려 고전 머신러닝이 먼저 나오는 경우가 많아요. 판단 기준을 표로 정리해 보겠습니다.

따져 볼 것	고전 머신러닝이 유리	딥러닝이 유리
데이터 모양	표(행 = 샘플, 열 = 특성)	사진, 소리, 글, 영상
데이터 양	수백~수십만 줄	수십만~수억 개
특성	사람이 의미를 아는 값(나이, 금액 등)	픽셀, 글자처럼 하나하나는 의미가 약한 값
설명 요구	결정 이유를 설명해야 함(대출 심사, 의료)	성능이 가장 중요함
자원	CPU 한 대, 몇 초~몇 분	GPU, 몇 시간~몇 주

표 데이터에서는 지금도 그래디언트 부스팅이 딥러닝과 대등하거나 더 나은 경우가 많다는 연구가 꾸준히 나옵니다. 특성 하나하나에 이미 의미가 담겨 있어서, 딥러닝이 잘하는 "의미 있는 특성을 스스로 만들어 내는 일"이 덜 필요하기 때문이에요. 반대로 사진의 픽셀 하나, 문장의 글자 하나는 그 자체로 의미가 거의 없고 주변과의 관계에서 의미가 생깁니다. 이런 데이터에서는 9장의 CNN이나 12장의 트랜스포머처럼 관계를 스스로 배우는 신경망이 압도적이에요.

그래서 실무의 순서는 대개 이렇습니다. **기준선(Dummy) → 간단한 모델(선형·로지스틱 회귀) → 트리 앙상블(랜덤 포레스트, 부스팅)** 순서로 빠르게 점수를 재 보고, 그래도 부족하고 데이터가 충분할 때 딥러닝을 시도해요. 고전 모델로 잰 점수는 딥러닝 모델이 "정말 나아졌는가"를 판단하는 기준선이 되기도 합니다. 그리고 이 장에서 배운 **평가 지표, 교차 검증, 하이퍼파라미터 탐색, 시험 세트는 마지막에 한 번**이라는 원칙은 딥러닝에서도 하나도 바뀌지 않아요. 모델만 바뀔 뿐입니다.

3.13 실습: 캘리포니아 집값 예측 모델 고르기

이 장의 흐름을 한 데이터셋으로 처음부터 끝까지 해 보겠습니다. 할 일은 다섯 가지예요.

1. 데이터를 받고 학습/시험 세트로 나누기
2. 후보 모델 다섯 개를 학습 세트에서 교차 검증으로 비교하기
3. 가장 좋은 모델의 하이퍼파라미터 탐색하기
4. 최종 모델을 시험 세트로 **딱 한 번** 평가하기
5. 저장하고 불러와서 써 보기

데이터는 1990년 미국 캘리포니아 인구 조사의 구역(블록 그룹) 20,640곳 자료입니다. 켈리 페이스(Pace)와 로널드 배리(Barry)가 1997년 논문에 쓰려고 정리해 StatLib 저장소에 공개한 데이터예요. scikit-learn이 제공하는 함수로 처음 한 번만 인터넷에서 받아요(1MB가 안 되는 작은 파일입니다).

1단계: 데이터 받고 나누기

```
import os

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

# 처음 한 번은 인터넷에서 받아 data/ 폴더에 저장하고, 다음부터는 거기서 읽는다
집 = fetch_california_housing(data_home=os.environ.get("BOOK_DATA_DIR", "data"), as_frame=True)
X, y = 집.data, 집.target                # y: 구역의 집값 중앙값(단위 10만 달러)
print(X.shape)
print(X.head(3).round(2).to_string())
print("집값 범위:", y.min(), "~", y.max(), "/ 평균", round(y.mean(), 3))

X_학습, X_시험, y_학습, y_시험 = train_test_split(X, y, test_size=0.2, random_state=42)
print("학습:", X_학습.shape, "시험:", X_시험.shape)
```

```
(20640, 8)
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  Longitude
0    8.33      41.0      6.98       1.02       322.0      2.56     37.88    -122.23
1    8.30      21.0      6.24       0.97      2401.0      2.11     37.86    -122.22
2    7.26      52.0      8.29       1.07       496.0      2.80     37.85    -122.24
```

```
집값 범위: 0.14999 ~ 5.00001 / 평균 2.069
학습: (16512, 8) 시험: (4128, 8)
```

특성 8개는 구역의 소득 중앙값(MedInc, 단위 1만 달러), 집의 나이 중앙값, 가구당 평균 방 수와 침실 수, 인구, 가구당 평균 인원, 위도와 경도입니다. 목표값은 구역의 집값 중앙값으로 단위가 10만 달러라서, 평균 2.069는 약 20만 7천 달러예요. 최댓값이 5.00001로 딱 떨어지는 건 원래 조사에서 50만 달러 이상을 모두 50만 달러로 적었기 때문입니다. 이런 "천장"은 실제 데이터에서 자주 만나는 함정이라 알아 두면 좋아요.

학습 16,512곳, 시험 4,128곳으로 나눴습니다. 시험 세트 4,128곳은 이제 4단계까지 **봉인**합니다.

2단계: 후보 모델 교차 검증으로 비교하기

```
import os

from sklearn.datasets import fetch_california_housing
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import HistGradientBoostingRegressor, RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold, cross_validate, train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = fetch_california_housing(data_home=os.environ.get("BOOK_DATA_DIR", "data"),
                                return_X_y=True, as_frame=True)
X_학습, X_시험, y_학습, y_시험 = train_test_split(X, y, test_size=0.2, random_state=42)

후보들 = {
    "평균 찍기": DummyRegressor(),
    "선형 회귀": make_pipeline(StandardScaler(), LinearRegression()),
    "최근접 이웃": make_pipeline(StandardScaler(), KNeighborsRegressor(n_neighbors=10)),
    "랜덤 포레스트": RandomForestRegressor(n_estimators=100, min_samples_leaf=2,
                                          n_jobs=-1, random_state=0),
    "부스팅": HistGradientBoostingRegressor(random_state=0),
}
```

```
접기 = KFold(n_splits=5, shuffle=True, random_state=0)
for 이름, 모델 in 후보들.items():                # 시험 세트는 여기서 쓰지 않는다
    결과 = cross_validate(모델, X_학습, y_학습, cv=접기,
                          scoring=["neg_root_mean_squared_error", "r2"])
    rmse = -결과["test_neg_root_mean_squared_error"]
    print(f"{이름}: RMSE {rmse.mean():.3f} ± {rmse.std():.3f}, R² {결과['test_r2'].mean():.3f}")
```

```
평균 찍기: RMSE 1.156 ± 0.024, R² -0.000
선형 회귀: RMSE 0.720 ± 0.020, R² 0.612
최근접 이웃: RMSE 0.641 ± 0.017, R² 0.692
랜덤 포레스트: RMSE 0.511 ± 0.013, R² 0.805
부스팅: RMSE 0.471 ± 0.010, R² 0.834
```

`cross_validate`의 `scoring`에 `"neg_root_mean_squared_error"`라는 긴 이름을 줬습니다. scikit-learn은 "점수는 클수록 좋다"는 규칙을 모든 지표에 맞추려고, 작을수록 좋은 오차 지표에는 마이너스를 붙여(neg) 돌려줘요. 그래서 결과에 다시 마이너스를 붙여 RMSE로 읽었습니다.

기준선인 평균 찍기의 RMSE는 1.156(약 11만 6천 달러), R^2는 정확히 0입니다(-0.000은 아주 작은 음수가 반올림된 표시예요). 선형 회귀가 0.720으로 크게 줄였고, 최근접 이웃이 0.641, 랜덤 포레스트가 0.511, 부스팅이 0.471로 가장 좋았어요. 회차별 흔들림(±0.010~0.024)보다 모델 사이의 차이가 훨씬 크니 이 순위는 믿을 만합니다. 위도·경도와 집값의 관계처럼 직선으로 나타낼 수 없는 패턴을 트리 계열이 잘 잡아낸 결과예요.

3단계: 부스팅의 하이퍼파라미터 탐색하기

1등인 부스팅을 기본 설정에서 더 다듬어 보겠습니다. 8조합 × 5겹이라 모델을 40번 학습하므로, 이 책의 실행 환경에서 2분 남짓 걸렸어요.

```
import os

from scipy.stats import loguniform, randint
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.model_selection import KFold, RandomizedSearchCV, train_test_split

X, y = fetch_california_housing(data_home=os.environ.get("BOOK_DATA_DIR", "data"),
```

```
                                return_X_y=True, as_frame=True)
X_학습, X_시험, y_학습, y_시험 = train_test_split(X, y, test_size=0.2, random_state=42)

분포 = {
    "learning_rate": loguniform(0.03, 0.3),
    "max_iter": randint(100, 500),          # 나무 수
    "max_leaf_nodes": randint(15, 63),
    "min_samples_leaf": randint(10, 60),
    "l2_regularization": loguniform(1e-3, 10),
}
탐색 = RandomizedSearchCV(HistGradientBoostingRegressor(random_state=0), 분포, n_iter=8,
                          scoring="neg_root_mean_squared_error", random_state=0,
                          cv=KFold(n_splits=5, shuffle=True, random_state=0))
탐색.fit(X_학습, y_학습)

for 이름, 값 in sorted(탐색.best_params_.items()):
    print(f"{이름}: {round(값, 4)}")
print(f"교차 검증 RMSE: {-탐색.best_score_:.3f}  (기본 설정은 0.471)")
```

```
l2_regularization: 0.0027
learning_rate: 0.0893
max_iter: 470
max_leaf_nodes: 43
min_samples_leaf: 44
교차 검증 RMSE: 0.457  (기본 설정은 0.471)
```

무작위로 고른 8조합 중에서는 학습률 0.0893, 나무 470그루, 잎 43개짜리 조합이 뽑혔고, 교차 검증 RMSE가 기본 설정의 0.471에서 0.457로 줄었습니다. 3% 남짓 나아진 셈이에요. 2단계에서 모델을 바꿨을 때(선형 회귀 0.720 → 부스팅 0.471)와 비교하면 작은 차이죠. 성능의 대부분은 **어떤 모델을 고르느냐**에서 나오고, 하이퍼파라미터 탐색은 마지막에 조금 더 짜내는 단계라고 생각하면 됩니다. 기본값(학습률 0.1, 나무 100그루)보다 학습률은 조금 낮추고 나무는 훨씬 많이 세운 조합이 뽑혔는데, "조금씩 고치되 더 여러 번" 고치는 쪽이 낫다는 3.5절의 이야기와도 맞아요. 다만 넓은 범위에서 8조합만 뽑아 본 것이라, 시간이 넉넉하면 `n_iter`를 늘려 보세요.

4단계: 시험 세트로 딱 한 번 평가하고 저장하기

이제 봉인을 풉니다. 3단계에서 찾은 설정을 그대로 적어 최종 모델을 만들고, **학습 세트 전체**로 학습한 뒤 시험 세트로 한 번만 채점해요. 이 블록은 앞 블록과 따로 실행할 수 있도록 탐색 결과를 숫자로 옮겨 적었습니다.

```
import os
from pathlib import Path

import joblib
import sklearn
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, r2_score, root_mean_squared_error
from sklearn.model_selection import train_test_split

X, y = fetch_california_housing(data_home=os.environ.get("BOOK_DATA_DIR", "data"),
                                return_X_y=True, as_frame=True)
X_학습, X_시험, y_학습, y_시험 = train_test_split(X, y, test_size=0.2, random_state=42)

# 3단계에서 찾은 설정을 옮겨 적는다
최종 = HistGradientBoostingRegressor(
    learning_rate=0.0893, max_iter=470, max_leaf_nodes=43,
    min_samples_leaf=44, l2_regularization=0.0027, random_state=0)
최종.fit(X_학습, y_학습)                  # 학습 세트 전체로 학습

예측 = 최종.predict(X_시험)               # 봉인했던 시험 세트, 딱 한 번
print(f"시험 RMSE {root_mean_squared_error(y_시험, 예측):.3f}, "
      f"MAE {mean_absolute_error(y_시험, 예측):.3f}, R² {r2_score(y_시험, 예측):.3f}")

경로 = Path("models") / "california_hgb.joblib"
경로.parent.mkdir(exist_ok=True)
joblib.dump({"model": 최종, "sklearn_version": sklearn.__version__,
             "features": list(X.columns)}, 경로)

꾸러미 = joblib.load(경로)
새_구역 = X_시험.head(3)
```

```
for 예상, 실제 in zip(꾸러미["model"].predict(새_구역), y_시험.head(3)):
    print(f"예상 {예상 * 10:5.2f}만 달러 / 실제 {실제 * 10:5.2f}만 달러")
```

```
시험 RMSE 0.450, MAE 0.296, R² 0.845
예상  5.40만 달러 / 실제  4.77만 달러
예상 10.59만 달러 / 실제  4.58만 달러
예상 50.79만 달러 / 실제 50.00만 달러
```

봉인했던 시험 세트에서 RMSE 0.450, MAE 0.296, R^2 0.845가 나왔습니다. 교차 검증에서 본 0.457과 거의 같으니, 교차 검증으로 짐작한 실력이 처음 보는 데이터에서도 그대로 나온 거예요. MAE 0.296은 구역마다 집값 중앙값을 평균 3만 달러쯤 빗나간다는 뜻입니다.

불러온 모델로 시험 구역 세 곳을 예측해 보면 첫째 구역은 5.40만 달러 대 실제 4.77만 달러로 가깝지만, 둘째 구역은 10.59만 달러로 실제(4.58만 달러)의 두 배 넘게 빗나갔어요. 평균 성적이 좋아도 하나하나의 예측은 크게 틀릴 수 있다는 것, 그래서 큰 실수를 드러내는 RMSE를 MAE와 함께 봐야 한다는 것을 보여 줍니다. 셋째 구역은 실제 값이 50.00만 달러로, 1단계에서 본 "천장"에 걸린 구역이에요. 진짜 집값은 이보다 높을 수도 있습니다.

왜 시험 세트는 한 번만 쓸까요?

실습에서 가장 중요한 규칙을 다시 짚겠습니다. 만약 시험 점수를 보고 "조금 아쉬운데 학습률을 바꿔서 다시 재 볼까?"를 반복하면 어떻게 될까요? 그 순간 시험 세트는 더 이상 처음 보는 문제가 아니라 **답을 맞춰 가며 고친 문제**가 됩니다. 기출문제를 외워서 받은 점수로 실력을 판단하는 것과 같아요. 그렇게 얻은 시험 점수는 실제 서비스에서 만날 새 데이터에서의 성능보다 좋게 나오기 마련입니다.

그래서 모델 고르기와 설정 고르기는 모두 학습 세트 안의 교차 검증으로 하고, 시험 세트는 "이 모델을 내보내도 되는가"를 마지막으로 확인하는 데 **한 번만** 씁니다. 시험 점수가 기대보다 나쁘게 나왔다면, 그 사실을 기록하고 원인을 따져 보되 같은 시험 세트로 다시 고르지는 않아요. 딥러닝에서도 학습/검증/시험 세트의 역할은 정확히 이대로입니다(6장, 8장).

3.14 정리와 다음 장 미리보기

이 장에서 scikit-learn으로 고전 머신러닝의 대표 모델과 평가 방법을 익혔습니다.

- **약속**: `fit`으로 배우고 `predict`·`predict_proba`로 예측하고 `score`로 채점. 끝에 밑줄이 붙은 속성(`coef_`, `feature_importances_`)은 배운 결과
- **선형 모델**: 선형 회귀는 계수와 절편을 배우고, 릿지·라소는 계수에 벌점을 줘 고삐를 맴. 로지스틱 회귀는 점수를 시그모이드로 확률로 바꾸는 분류 모델
- **트리 모델**: 결정 트리는 스무고개(깊으면 외움), 랜덤 포레스트는 서로 다른 나무들의 다수결, 그래디언트 부스팅은 앞 나무의 잔차를 조금씩 고쳐 가는 오답 노트
- **군집**: k-means는 깃발 옮기기를 반복. k는 관성의 엘보와 실루엣 점수로 고르고, 거리를 쓰니 표준화가 필수
- **분류 지표**: 드문 양성 앞에서 정확도는 속인다. 혼동 행렬의 네 칸에서 정밀도·재현율·F1, 문턱에 따라 움직이는 시소, 문턱과 상관없는 ROC-AUC
- **회귀 지표**: MAE는 설명하기 쉽고, RMSE는 큰 실수에 민감하고, R^2는 평균으로 찍기 대비 얼마나 나은지
- **교차 검증과 탐색**: 분류는 섞은 `StratifiedKFold`, 전처리는 파이프라인째로, `GridSearchCV`·`RandomizedSearchCV`는 학습 세트에서만, 시험 세트는 마지막에 한 번
- **저장**: 파이프라인 통째로 `joblib.dump`, 버전을 함께 기록, 모르는 파일은 열지 않기

이것으로 1부 "머신러닝의 감각"을 마칩니다. 다음 4장부터는 PyTorch로 넘어가 딥러닝의 바탕을 쌓아요. 먼저 숫자를 담는 그릇인 **텐서**를 손에 익히고, 5장에서는 이 장의 선형 회귀를 경사하강법으로 직접 풀면서 "틀린 만큼 조금씩 고친다"는 그래디언트 부스팅의 생각이 신경망 학습에서 어떻게 쓰이는지 보게 됩니다.

04

텐서: 숫자를 담는 그릇

학습 목표

- 텐서가 무엇이고 왜 리스트 대신 텐서를 쓰는지 설명하기
- 여러 방법으로 텐서를 만들고, 모양(shape)과 자료형(dtype)을 읽고 바꾸기
- 인덱싱·모양 바꾸기·브로드캐스팅·행렬 곱으로 숫자 묶음을 한 번에 계산하기
- 메모리를 함께 쓰는 경우와 복사하는 경우를 구분하기
- CPU와 GPU(CUDA, Apple MPS) 사이로 텐서를 옮기기
- 조건으로 바꾸고(`where`, `clamp`), 정렬하고(`sort`, `topk`), 번호표로 꺼내기(`gather`)
- 텐서를 파일로 저장하고 불러오며, 모양 오류 메시지를 읽고 고치기

들어가며: 사진 한 장은 숫자 몇 개일까요?

휴대폰으로 찍은 사진 한 장을 컴퓨터는 어떻게 기억할까요? 화면을 아주 작은 점(픽셀)으로 잘게 나누고, 점마다 빨강·초록·파랑이 얼마나 섞였는지를 숫자로 적어 둡니다. 세로 1080점, 가로 1920점인 사진이라면 1080 × 1920 × 3, 무려 **620만 개가 넘는 숫자**예요.

딥러닝은 이런 숫자 덩어리를 수천, 수만 장씩 받아서 계산합니다. 1권에서 배운 리스트로 이걸 다루면 어떻게 될까요? 사진을 20% 밝게 만드는 계산을 리스트와 이 장의 주인공인 **텐서(tensor)** 로 각각 해 보겠습니다.

```
import time
import torch
```

```
# 사진 한 장 크기의 숫자: 세로 1080 × 가로 1920 × 색 3개
리스트_사진 = [0.5] * (1080 * 1920 * 3)
텐서_사진 = torch.full((1080, 1920, 3), 0.5)

시작 = time.perf_counter()
밝게 = [값 * 1.2 for 값 in 리스트_사진]
리스트_시간 = time.perf_counter() - 시작

시작 = time.perf_counter()
밝게_텐서 = 텐서_사진 * 1.2
텐서_시간 = time.perf_counter() - 시작

print(f"숫자 개수: {len(리스트_사진):,}개")
print(f"리스트: {리스트_시간:.3f}초, 텐서: {텐서_시간:.4f}초")
print(f"텐서가 약 {리스트_시간 / 텐서_시간:.0f}배 빠름")
```

```
숫자 개수: 6,220,800개
리스트: 0.065초, 텐서: 0.0026초
텐서가 약 25배 빠름
```

같은 일을 했는데 텐서가 몇 배나 빠릅니다. 정확한 숫자는 컴퓨터마다 다르지만(위 결과는 이 책을 쓴 Apple M4 Max 노트북의 값입니다), 차이가 크게 난다는 점은 어디서나 같아요. 리스트 컴프리헨션은 숫자 620만 개를 **하나씩** 꺼내 곱하지만, 텐서는 같은 계산을 컴퓨터 깊숙한 곳의 빠른 코드로 **한꺼번에** 처리하기 때문입니다.

1권 20장에서 NumPy 배열을 써 봤다면 "그거 NumPy랑 같은 거 아니에요?" 싶을 거예요. 맞습니다. 텐서는 NumPy 배열과 거의 같은 그릇입니다. 다만 두 가지 능력이 더 있어요.

- **GPU에서 계산할 수 있습니다.** 그래픽 카드는 단순한 계산 수천 개를 동시에 하는 데 특화되어 있어서, 딥러닝 계산을 몇 배에서 수십 배 빠르게 해 줍니다. 4.8절에서 직접 옮겨 봅니다.
- **자기가 어떤 계산을 거쳐 만들어졌는지 기억할 수 있습니다.** 이 기억 덕분에 모델이 "어느 쪽으로 고쳐야 정답에 가까워지는지"를 자동으로 계산할 수 있어요. 이것이 다음 5장의 주제인 **자동 미분**입니다.

이 장에서는 그 그릇 자체를 손에 익힙니다. 만들고, 모양을 읽고, 잘라 내고, 접고 펴고, 한꺼번에 계산하는 법이에요. 2권의 모든 장이 이 위에서 돌아가니, 조금 많아 보여도 하나씩 직접 실행해 보세요.

Q. **PyTorch는 어떻게 설치하나요?** 1권에서 쓰던 uv 그대로입니다. 프로젝트 폴더에서 `uv add torch numpy`를 실행하면 운영체제에 맞는 PyTorch가 설치돼요. 이 장의 예제는 PyTorch 2.14, 파이썬 3.12에서 실행해 확인했습니다. NVIDIA GPU용 설치는 버전 조합이 까다로워서 부록에서 따로 다룹니다.

4.1 텐서 만들기

목록에서 바로 만들기

가장 쉬운 방법은 파이썬 목록을 `torch.tensor()`에 넣는 것입니다. 1권에서 리스트를 만들던 것과 똑같아요.

```
import torch

기온 = torch.tensor([12.5, 14.1, 13.0])      # 실수 목록 → 실수 텐서
인원 = torch.tensor([3, 5, 2])               # 정수 목록 → 정수 텐서
합격 = torch.tensor([True, False, True])     # 참/거짓 목록 → 불리언 텐서

print(기온, 기온.dtype)
print(인원, 인원.dtype)
print(합격, 합격.dtype)
```

```
tensor([12.5000, 14.1000, 13.0000]) torch.float32
tensor([3, 5, 2]) torch.int64
tensor([ True, False,  True]) torch.bool
```

출력 끝의 `torch.float32`, `torch.int64`, `torch.bool`이 **자료형(dtype)** 입니다. 넣은 값을 보고 PyTorch가 알아서 골랐어요. 소수점이 있으면 32비트 실수, 정수만 있으면 64비트 정수, 참/거짓이면 불리언입니다. 실수 쪽 기본이 64비트가 아니라 32비트라는 점은 기억해 두세요. 4.3절에서 이유를 봅니다.

`12.5000`처럼 소수점 아래가 넷째 자리까지 찍히는 건 **화면에 보여 줄 때만** 그렇게 맞춘 것입니다. 실제 값이 바뀐 건 아니에요.

모양만 정해서 만들기

값을 하나하나 적기보다 "2행 3열짜리 0 묶음" 같은 그릇이 먼저 필요할 때가 많습니다. 빈 그릇을 준비해 두고 나중에 채우는 거죠.

```
import torch

print(torch.zeros(2, 3))          # 0으로 채운 2행 3열
print(torch.ones(2, 3))           # 1로 채운 2행 3열
print(torch.full((2, 3), 7.0))    # 원하는 값으로 채우기
print(torch.arange(0, 10, 2))     # range처럼: 0부터 10 전까지 2씩
print(torch.linspace(0, 1, 5))    # 0부터 1까지 똑같은 간격으로 5개
print(torch.eye(3))               # 대각선만 1인 단위 행렬
```

```
tensor([[0., 0., 0.],
        [0., 0., 0.]])
tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[7., 7., 7.],
        [7., 7., 7.]])
tensor([0, 2, 4, 6, 8])
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
tensor([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.]])
```

`arange`는 1권 5장의 `range`와 같은 규칙(끝 값은 포함하지 않음)이고, `linspace`는 "처음과 끝 사이를 똑같은 간격으로 몇 개"라는 점이 다릅니다. `linspace(0, 1, 5)`는 끝인 1까지 포함해서 다섯 개예요. 둘을 헷갈리기 쉬우니 "개수를 정하고 싶으면 linspace, 간격을 정하고 싶으면 arange"라고 기억하세요.

난수로 만들기

딥러닝 모델은 처음에 아무것도 모르는 상태에서 시작합니다. 그래서 모델 안의 숫자들을 **무작위 값**으로 채워 두고 학습하면서 고쳐 나가요. 난수 텐서는 그래서 자주 씁니다.

```
import torch

torch.manual_seed(42)             # 같은 씨앗이면 같은 난수가 나온다
print(torch.rand(2, 3))           # 0 이상 1 미만에서 고르게
print(torch.randn(2, 3))          # 평균 0, 표준편차 1인 정규분포
print(torch.randint(1, 7, (5,)))  # 주사위 다섯 번: 1 이상 7 미만 정수

torch.manual_seed(42)
print(torch.rand(2, 3))           # 씨앗을 다시 심으면 처음과 똑같다
```

```
tensor([[0.8823, 0.9150, 0.3829],
        [0.9593, 0.3904, 0.6009]])
tensor([[ 1.1561,  0.3965, -2.4661],
        [ 0.3623,  0.3765, -0.1808]])
tensor([4, 5, 2, 3, 2])
tensor([[0.8823, 0.9150, 0.3829],
        [0.9593, 0.3904, 0.6009]])
```

`torch.manual_seed(42)`를 다시 부른 뒤의 결과가 첫 번째와 똑같죠? 컴퓨터의 난수는 사실 "씨앗(seed)"에서 정해진 순서대로 뽑히는 숫자라서, 씨앗이 같으면 같은 순서로 나옵니다. 그래서 이 책의 예제를 그대로 따라 하면 같은 숫자를 볼 수 있어요. 실험 결과를 다른 사람이 다시 확인할 수 있게 하는 **재현성**의 출발점이기도 합니다(18장에서 자세히 다룹니다).

`rand`와 `randn`은 이름이 비슷하지만 뽑는 방식이 다릅니다. `rand`는 0과 1 사이에서 고르게, `randn`은 0 근처가 많고 멀어질수록 드물게(정규분포) 뽑아요. `randn`의 결과에 음수와 -2.4 같은 큰 값이 섞여 있는 이유입니다.

나만 쓰는 난수 생성기

그런데 씨앗만 고정하면 정말 언제나 같은 숫자가 나올까요? 씨앗은 "몇 번째 난수부터 뽑을지"가 아니라 "난수 줄의 시작점"만 정합니다. 그래서 씨앗을 심은 뒤 **중간에 다른 코드가 난수를 먼저 꺼내 가면** 그다음부터는 순서가 밀려 버려요. 은행 번호표 기계를 떠올려 보세요. 기계를 1번부터 다시 시작하게 해도, 내 앞에 누가 표를 뽑아 가면 나는 1번이 아니라 2번을 받습니다.

```
import torch

torch.manual_seed(42)
첫번째 = torch.rand(3)

torch.manual_seed(42)
torch.randn(5)                    # 중간에 다른 코드가 난수를 먼저 쓰면
두번째 = torch.rand(3)
print(첫번째)
print(두번째)                     # 씨앗이 같아도 결과가 달라진다

# 나만 쓰는 난수 생성기: 전역 난수와 따로 돈다
생성기 = torch.Generator().manual_seed(7)
print(torch.rand(3, generator=생성기))
torch.randn(5)                    # 전역 난수를 얼마나 쓰든
생성기 = torch.Generator().manual_seed(7)
print(torch.rand(3, generator=생성기))   # 생성기의 순서는 그대로

# 섞는 순서 정하기: 0~9를 무작위 순서로
print(torch.randperm(10, generator=torch.Generator().manual_seed(1)))
```

```
tensor([0.8823, 0.9150, 0.3829])
tensor([0.8694, 0.5677, 0.7411])
tensor([0.5349, 0.1988, 0.6592])
tensor([0.5349, 0.1988, 0.6592])
tensor([5, 6, 1, 2, 0, 8, 9, 3, 7, 4])
```

첫 두 줄은 똑같이 `manual_seed(42)`를 심었는데도 결과가 다릅니다. 두 번째에는 중간에 `randn(5)`가 난수 다섯 개를 먼저 가져갔기 때문이에요. 지금은 한 줄이라 눈에 보이지만, 실제 프로젝트에서는 모델을 만들거나 데이터를 섞는 코드가 곳곳에서 난수를 꺼내 가서 "어제랑 똑같이 돌렸는데 결과가 다르다"는 일이 생깁니다.

`torch.Generator()`는 **나만 쓰는 번호표 기계**입니다. 함수에 `generator=생성기`를 넘기면 전역 난수가 아니라 그 기계에서 뽑아요. 그래서 중간에 전역 난수를 얼마나 쓰든 생성기의 순서는 흔들리지 않습니다. 마지막 줄의 `randperm(10)`은 0부터 9까지를 무작위 순서로 섞은 번호예요. 데이터를 섞어서 나눌 때 이 순서를 쓰는데, 7장의 `DataLoader`도 섞는 순서를 이런 생성기로 고정할 수 있습니다.

NumPy와 주고받기

1권 20장에서 만든 NumPy 배열이나 pandas 데이터를 텐서로 바꿔야 할 때가 많습니다. 방법이 두 가지인데, 결과가 아주 다릅니다.

```
import numpy as np
import torch

배열 = np.array([1.0, 2.0, 3.0])

복사본 = torch.tensor(배열)        # 새 그릇에 옮겨 담기 (복사)
공유본 = torch.from_numpy(배열)    # 같은 그릇을 함께 쓰기 (공유)

배열[0] = 100.0                    # 원래 NumPy 배열을 바꿔 보면?
print("NumPy  :", 배열)
print("복사본 :", 복사본)
print("공유본 :", 공유본)

print("다시 NumPy로:", 공유본.numpy())
```

```
NumPy  : [100.   2.   3.]
복사본 : tensor([1., 2., 3.], dtype=torch.float64)
공유본 : tensor([100.,   2.,   3.], dtype=torch.float64)
다시 NumPy로: [100.   2.   3.]
```

NumPy 배열의 첫 칸을 100으로 바꿨더니 **복사본**은 그대로인데 **공유본**은 같이 바뀌었습니다. 1권 3장의 쌍둥이 옷 이야기 기억나세요? `torch.tensor()`는 **똑같은 옷을 한 벌 더 사는 것**(복사)이고, `torch.from_numpy()`는 **같은 옷에 이름표를 하나 더 다는 것**(공유)입니다. 공유는 큰 데이터를 복사하지 않아도 되니 빠르고 메모리를 아끼지만, 한쪽을 바꾸면 다른 쪽도 바뀐다는 점을 꼭 기억해야 해요.

하나 더 보세요. 두 텐서 모두 `dtype=torch.float64`입니다. NumPy의 기본 실수가 64비트라서 그대로 따라온 거예요. 딥러닝에서는 보통 32비트를 쓰니, NumPy에서 가져온 텐서는 `.float()`로 바꿔 주는 습관을 들이세요.

4.2 모양과 차원

점, 선, 면, 그리고 책

텐서를 다룰 때 가장 먼저, 가장 자주 확인하는 것이 **모양(shape)** 입니다. 딥러닝 코드에서 나는 오류의 절반쯤이 모양이 안 맞아서 생긴다고 해도 과언이 아니에요. 체온 기록으로 차원을 하나씩 늘려 보겠습니다.

```
import torch

점 = torch.tensor(36.5)                          # 0차원: 체온 하나
선 = torch.tensor([36.5, 36.7, 36.6])            # 1차원: 하루 세 번 잰 체온
면 = torch.tensor([[36.5, 36.7, 36.6],
                   [36.4, 36.9, 36.8]])          # 2차원: 이틀치
책 = torch.zeros(7, 2, 3)                         # 3차원: 일주일치 표 7장

for 이름, t in [("점", 점), ("선", 선), ("면", 면), ("책", 책)]:
    print(f"{이름}: ndim={t.ndim}, shape={tuple(t.shape)}, 숫자 {t.numel()}개")
```

```
점: ndim=0, shape=(), 숫자 1개
선: ndim=1, shape=(3,), 숫자 3개
면: ndim=2, shape=(2, 3), 숫자 6개
책: ndim=3, shape=(7, 2, 3), 숫자 42개
```

- **0차원(스칼라)**: 숫자 하나. 점 하나와 같아서 모양이 빈 튜플 `()`입니다.
- **1차원(벡터)**: 한 줄로 늘어선 숫자. 모양 `(3,)`은 "숫자 3개짜리 줄"이라는 뜻이에요.
- **2차원(행렬)**: 가로세로가 있는 표. `(2, 3)`은 2행 3열입니다.
- **3차원 이상**: 표를 여러 장 쌓은 것. `(7, 2, 3)`은 2행 3열짜리 표가 7장 있는 책이에요.

`ndim`은 차원의 수, `shape`는 각 차원의 길이, `numel()`은 숫자의 총개수입니다. `numel()`은 언제나 `shape`의 숫자를 모두 곱한 값과 같아요(7 × 2 × 3 = 42).

딥러닝에서 만나는 모양

실제로 다룰 데이터는 대개 3차원, 4차원입니다. 대표적인 것이 사진 묶음이에요.

```
import torch

# 컬러 사진 32장 묶음: (장 수, 색 채널, 세로, 가로)
사진_묶음 = torch.zeros(32, 3, 224, 224)
print(사진_묶음.shape)
print(f"숫자 {사진_묶음.numel():,}개, 차원 {사진_묶음.ndim}개")
print("첫 사진의 모양:", 사진_묶음[0].shape)
```

```
torch.Size([32, 3, 224, 224])
숫자 4,816,896개, 차원 4개
첫 사진의 모양: torch.Size([3, 224, 224])
```

PyTorch에서는 사진 묶음을 **(장 수, 색 채널, 세로, 가로)** 순서로 놓는 것이 관례입니다. 첫 번째 차원은 "몇 장을 한꺼번에 처리하는가"로, **배치(batch)** 라고 불러요. 모델은 사진을 한 장씩이 아니라 수십 장씩 묶어서 처리하는데, 그래야 GPU가 쉬지 않고 일할 수 있기 때문입니다. 사진 32장 묶음 하나가 벌써 숫자 480만 개네요.

모양은 이렇게 읽는 버릇을 들이면 좋습니다. 숫자만 보지 말고 **각 자리에 이름을 붙여서** 읽는 거예요. `(32, 3, 224, 224)`를 보면 "32장, 3색, 224 × 224"라고 소리 내어 읽어 보세요. 이 습관 하나가 모양 오류를 크게 줄여 줍니다.

4.3 자료형

섞으면 넓은 쪽으로

자료형이 다른 텐서끼리 계산하면 어떻게 될까요?

```
import torch

a = torch.tensor([1, 2, 3])
b = torch.tensor([0.5, 0.5, 0.5])
print((a + b).dtype)             # 정수 + 실수 → 실수로 맞춰진다

print(torch.tensor([7, 8]) / 2)   # / 는 정수끼리여도 실수 결과
print(torch.tensor([7, 8]) // 2)  # // 는 몫, 정수 그대로
```

```
x = torch.tensor([1.5, 2.7])
print(x.int(), x.to(torch.int64).dtype)   # 실수 → 정수: 소수점 버림
print(x.half().dtype, x.double().dtype)   # float16, float64

print(torch.tensor(0.1) + torch.tensor(0.2))               # 화면에는 반올림해서 보인다
print((torch.tensor(0.1) + torch.tensor(0.2)).item())      # float32의 실제 값
print((torch.tensor(0.1, dtype=torch.float64)
       + torch.tensor(0.2, dtype=torch.float64)).item())   # float64의 실제 값
```

```
torch.float32
tensor([3.5000, 4.0000])
tensor([3, 4])
tensor([1, 2], dtype=torch.int32) torch.int64
torch.float16 torch.float64
tensor(0.3000)
0.30000001192092896
0.30000000000000004
```

정리하면 이렇습니다.

- 정수와 실수를 섞으면 **더 넓은 쪽(실수)** 으로 맞춰서 계산합니다. 1권 2장에서 `3 + 0.5`가 실수가 되던 것과 같아요.
- `/`는 정수끼리 나눠도 실수를 돌려주고, `//`는 몫만 정수로 돌려줍니다. 1권 3장 산술 연산자 규칙 그대로예요.
- 실수를 정수로 바꾸면(`.int()`, `.to(torch.int64)`) 소수점 아래는 **버려집니다**. 2.7이 3이 아니라 2가 됐죠. 반올림이 필요하면 `.round()`를 먼저 하세요.

왜 기본이 32비트일까요?

마지막 두 줄을 보세요. 1권 3장에서 `0.1 + 0.2`가 `0.30000000000000004`가 되던 것 기억나시죠? 파이썬 기본 실수는 64비트라 오차가 소수점 열여섯째 자리쯤에서 나타났는데, 32비트 텐서에서는 **여덟째 자리**부터 벌써 어긋납니다. 숫자 하나를 담는 칸이 절반이니 그만큼 거칠게 기억하는 거예요.

그런데도 딥러닝의 기본이 32비트인 이유는, 모델 안에 숫자가 수백만에서 수십억 개 있기 때문입니다. 칸 크기가 절반이면 메모리도 절반, 옮기고 계산하는 시간도 줄어들어요. 그리고 딥러닝은 "정답

쪽으로 조금씩 고쳐 가는" 계산이라, 여덟째 자리의 작은 오차는 결과에 거의 영향을 주지 않습니다. 오히려 요즘은 16비트(`float16`, `bfloat16`)까지 내려서 속도를 더 얻는 경우가 많아요. 이 이야기는 16장 GPU와 성능에서 이어집니다.

자료형	칸 크기	주로 쓰는 곳
`torch.float32`	4바이트	딥러닝 기본값, 모델의 숫자와 입력 데이터
`torch.float16`, `torch.bfloat16`	2바이트	학습·추론을 빠르게 할 때 (16장)
`torch.float64`	8바이트	정밀한 과학 계산, NumPy에서 넘어온 값
`torch.int64`	8바이트	정답 번호(분류의 클래스), 인덱스
`torch.bool`	1바이트	조건 마스크 (4.4절)

텐서 하나가 차지하는 메모리

표의 "칸 크기"는 실제로 얼마나 큰 차이를 만들까요? 텐서가 차지하는 메모리는 **칸 크기 × 칸 개수**로 바로 계산할 수 있습니다. 칸 크기는 `element_size()`, 칸 개수는 앞에서 본 `numel()`이에요.

```
import torch

MB = 1024 ** 2
사진묶음 = torch.zeros(32, 3, 224, 224)
print(사진묶음.element_size(), "바이트 ×", f"{사진묶음.numel():,}개")
print(f"{사진묶음.element_size() * 사진묶음.numel() / MB:.1f} MB")

for 자료형 in [torch.float64, torch.float32, torch.bfloat16, torch.int8]:
    t = torch.zeros(32, 3, 224, 224, dtype=자료형)
    크기 = t.element_size() * t.numel() / MB
    print(f"{str(자료형):15} 칸당 {t.element_size()}바이트 → {크기:6.1f} MB")

# 숫자 70억 개(7B 모델)를 float32로 담으면?
GB = 1024 ** 3
```

```
print(f"70억 × 4바이트 = {7_000_000_000 * 4 / GB:.1f} GB")
```

```
4 바이트 × 4,816,896개
18.4 MB
torch.float64   칸당 8바이트 →   36.8 MB
torch.float32   칸당 4바이트 →   18.4 MB
torch.bfloat16  칸당 2바이트 →    9.2 MB
torch.int8      칸당 1바이트 →    4.6 MB
70억 × 4바이트 = 26.1 GB
```

4.2절의 사진 32장 묶음은 숫자 약 480만 개라서 `float32`로 18.4MB입니다. 같은 묶음을 `float64`로 만들면 두 배인 36.8MB, `bfloat16`이면 절반인 9.2MB가 돼요. 자료형 하나 바꿨을 뿐인데 메모리가 두 배씩 오르내립니다.

마지막 줄은 요즘 자주 듣는 "7B 모델"을 계산해 본 것입니다. 숫자 70억 개를 `float32`로 담으면 26GB가 넘어서, 보통 노트북 GPU에는 올라가지도 않아요. 그래서 큰 모델은 16비트나 그보다 작은 칸에 담아 씁니다(14장, 19장). GPU 메모리가 부족하다는 오류를 만나면 이 곱셈부터 해 보세요. "내 텐서들이 몇 GB인가"를 알면 배치를 얼마나 줄여야 할지 감이 옵니다.

`1024 ** 2`로 나눈 것은 바이트를 메가바이트(MB)로 바꾸기 위해서입니다. 컴퓨터 메모리는 1000이 아니라 1024씩 묶어 세는 관례가 있어요.

4.4 인덱싱과 슬라이싱

리스트와 같은 문법, 차원만 늘어날 뿐

텐서에서 원하는 부분을 꺼내는 문법은 1권 6장의 리스트 인덱싱·슬라이싱과 같습니다. 다른 점은 쉼표로 **차원마다 따로** 지정할 수 있다는 것뿐이에요. 도시 셋의 분기별 기온표로 연습해 보겠습니다.

```
import torch

# 3개 도시 × 4분기 평균 기온 (예제용 숫자)
기온 = torch.tensor([[ 2.0, 13.5, 25.7, 15.0],
                     [ 4.5, 15.0, 25.0, 17.5],
                     [ 1.0, 14.0, 26.0, 15.5]])
```

```
print(기온[0])          # 첫 번째 도시의 네 분기
print(기온[0, 2])       # 첫 번째 도시의 3분기
print(기온[:, 2])       # 모든 도시의 3분기 (세로 한 줄)
print(기온[1:, :2])     # 두 번째 도시부터, 1~2분기
print(기온[-1])         # 마지막 도시
```

```
tensor([ 2.0000, 13.5000, 25.7000, 15.0000])
tensor(25.7000)
tensor([25.7000, 25.0000, 26.0000])
tensor([[ 4.5000, 15.0000],
        [ 1.0000, 14.0000]])
tensor([ 1.0000, 14.0000, 26.0000, 15.5000])
```

`기온[0, 2]`는 "0번째 줄의 2번째 칸", `기온[:, 2]`는 "모든 줄의 2번째 칸", 즉 세로 한 줄입니다. 쉼표 앞이 행(도시), 뒤가 열(분기)이에요. `:`는 "그 차원은 전부"라는 뜻이고, 음수 인덱스와 `1:` 같은 슬라이스도 리스트와 똑같이 동작합니다.

조건으로 고르기

"20도가 넘는 칸만" 처럼 조건으로 고를 수도 있습니다. 1권 20장 pandas에서 쓰던 방식과 같아요.

```
import torch

기온 = torch.tensor([[ 2.0, 13.5, 25.7, 15.0],
                     [ 4.5, 15.0, 25.0, 17.5],
                     [ 1.0, 14.0, 26.0, 15.5]])

더운가 = 기온 > 20
print(더운가)
print(기온[더운가])                # 참인 칸만 골라 1차원으로
print(기온[[0, 2]])                # 0번, 2번 도시만 (목록으로 고르기)

얼음 = 기온.clone()
얼음[얼음 < 5] = 0.0              # 5도 미만은 0으로 바꾸기
print(얼음)
```

```
tensor([[False, False,  True, False],
        [False, False,  True, False],
        [False, False,  True, False]])
tensor([25.7000, 25.0000, 26.0000])
tensor([[ 2.0000, 13.5000, 25.7000, 15.0000],
        [ 1.0000, 14.0000, 26.0000, 15.5000]])
tensor([[ 0.0000, 13.5000, 25.7000, 15.0000],
        [ 0.0000, 15.0000, 25.0000, 17.5000],
        [ 0.0000, 14.0000, 26.0000, 15.5000]])
```

`기온 > 20`은 같은 모양의 참/거짓 텐서를 만듭니다. 이걸 **마스크(mask)** 라고 불러요. 마스크로 인덱싱하면 참인 칸만 골라 **1차원으로** 늘어놓습니다. 어느 칸이 골라질지 미리 알 수 없으니 원래 모양을 유지할 수가 없거든요.

`기온[[0, 2]]`처럼 대괄호 안에 **목록**을 넣으면 그 번호의 줄만 골라 옵니다. 마지막 예제처럼 마스크 자리에 값을 대입하면 조건에 맞는 칸만 한꺼번에 바꿀 수 있어요. 반복문 없이 "5도 미만은 0으로" 같은 규칙을 표 전체에 적용하는 셈입니다.

그런데 마지막 예제에서 왜 `기온`을 바로 바꾸지 않고 `.clone()`으로 복사본부터 만들었을까요? 다음 예제가 그 이유입니다.

슬라이스는 복사가 아닙니다

```
import torch

원본 = torch.arange(6)
조각 = 원본[2:5]          # 슬라이싱은 복사하지 않는다
조각[0] = 100
print("조각:", 조각)
print("원본:", 원본)       # 원본도 바뀌었다!

안전 = 원본[2:5].clone()   # 따로 쓰고 싶으면 clone()
안전[0] = -1
print("원본:", 원본)       # 이번에는 그대로
```

```
조각: tensor([100,   3,   4])
원본: tensor([  0,   1, 100,   3,   4,   5])
원본: tensor([  0,   1, 100,   3,   4,   5])
```

조각만 바꿨는데 원본의 가운데가 100이 됐습니다. 1권 6장에서 리스트를 슬라이싱하면 **새 리스트**가 만들어졌던 것과 정반대예요. 텐서의 슬라이스는 원본 메모리의 일부를 **들여다보는 창(view)** 일 뿐이라서, 창을 통해 칠하면 원본이 칠해집니다.

텐서가 이렇게 동작하는 이유는 속도 때문입니다. 사진 묶음에서 한 장을 꺼낼 때마다 숫자 15만 개를 복사한다면 너무 느리겠죠. 그래서 꺼내기는 공짜로 하고, 정말 따로 쓰고 싶을 때만 `.clone()`으로 복사하게 해 둔 거예요.

Q

Q. 마스크로 고른 `기온[더운가]`도 원본과 연결되어 있나요? 아니요, 마스크나 목록으로 고른 결과는 **복사본**입니다. 어떤 칸이 골라질지 들쭉날쭉해서 창 하나로 보여 줄 수가 없기 때문이에요. 정리하면 `:`나 숫자로 자른 것은 창(공유), 마스크나 목록으로 고른 것은 복사입니다. 다만 `기온[더운가] = 0`처럼 **대입하는 쪽**에 쓰면 원본이 바뀌어요. 헷갈릴 때는 4.7절의 확인 방법을 쓰세요.

조건으로 바꾸기: where와 clamp

앞에서 `얼음[얼음 < 5] = 0.0`으로 조건에 맞는 칸을 바꿨습니다. 그런데 이 방법은 복사본을 따로 만들고, 그 복사본을 직접 고쳐야 했죠. "원본은 그대로 두고, 조건에 따라 값을 고른 **새 표**를 하나 받고 싶다"면 `torch.where`가 더 깔끔합니다. 1권 4장의 조건 표현식 `a if 조건 else b`를 표 전체에 한꺼번에 적용하는 셈이에요.

```
import torch

기온 = torch.tensor([[ 2.0, 13.5, 25.7, 15.0],
                     [ 4.5, 15.0, 25.0, 17.5],
                     [ 1.0, 14.0, 26.0, 15.5]])

# where(조건, 참일 때 값, 거짓일 때 값): 원본은 두고 새 텐서를 만든다
print(torch.where(기온 > 20, 1, 0))
print(torch.where(기온 < 5, 0.0, 기온))      # 5도 미만만 0, 나머지는 그대로
```

```
# clamp: 아래·위 한계를 벗어난 값을 한계에 붙인다
print(기온.clamp(min=5.0, max=20.0))
print(torch.tensor([-0.3, 0.5, 1.7]).clamp(0.0, 1.0))   # 0~1 사이로 묶기
print(torch.tensor([-2.0, 0.0, 3.0]).clamp(min=0.0))    # 음수만 0으로
```

```
tensor([[0, 0, 1, 0],
        [0, 0, 1, 0],
        [0, 0, 1, 0]])
tensor([[ 0.0000, 13.5000, 25.7000, 15.0000],
        [ 0.0000, 15.0000, 25.0000, 17.5000],
        [ 0.0000, 14.0000, 26.0000, 15.5000]])
tensor([[ 5.0000, 13.5000, 20.0000, 15.0000],
        [ 5.0000, 15.0000, 20.0000, 17.5000],
        [ 5.0000, 14.0000, 20.0000, 15.5000]])
tensor([0.0000, 0.5000, 1.0000])
tensor([0., 0., 3.])
```

`torch.where(조건, 참일 때 값, 거짓일 때 값)`은 칸마다 조건을 보고 둘 중 하나를 고릅니다. 첫 줄은 "20도가 넘으면 1, 아니면 0"이라는 표를 만들었고, 두 번째 줄은 참일 때 0, 거짓일 때 **원래 기온**을 골라서 앞의 **얼음** 예제와 같은 결과를 원본을 건드리지 않고 얻었어요. 값 자리에는 숫자 하나를 써도 되고 같은 모양의 텐서를 써도 됩니다. 브로드캐스팅(4.6절)이 모양을 맞춰 주거든요.

`clamp`는 값을 **울타리 안에 가둡니다.** `min`보다 작으면 `min`으로, `max`보다 크면 `max`로 붙이고, 그 사이는 그대로 둬요. 위 결과에서 1분기 기온은 모두 5로, 3분기 기온은 모두 20으로 바뀌었습니다. 확률처럼 0과 1 사이여야 하는 값이 계산 오차로 살짝 벗어났을 때 `clamp(0.0, 1.0)`으로 바로잡는 일이 많아요. 마지막 줄처럼 `min=0.0`만 주면 "음수만 0으로"가 되는데, 사실 이것이 6장에서 만날 활성화 함수 ReLU의 계산과 똑같습니다.

번호표로 꺼내기: gather

분류 모델은 사진 한 장마다 "고양이일 확률, 개일 확률, 새일 확률"처럼 클래스 수만큼 숫자를 냅니다. 학습할 때는 이 중에서 **정답 칸의 확률**만 꺼내 봐야 해요. "정답이 개인 사진에서 모델은 개일 확률을 얼마나 줬나?"를 알아야 잘했는지 못했는지 판단할 수 있으니까요. 줄마다 꺼낼 칸이 다르니 `확률[:, 1]` 같은 슬라이스로는 안 됩니다.

```
import torch

# 사진 4장에 대해 모델이 낸 클래스별 확률 (0: 고양이, 1: 개, 2: 새)
확률 = torch.tensor([[0.7, 0.2, 0.1],
                     [0.1, 0.8, 0.1],
                     [0.3, 0.3, 0.4],
                     [0.6, 0.3, 0.1]])
정답 = torch.tensor([0, 1, 2, 1])           # 장마다 정답 클래스 번호

예측 = 확률.argmax(dim=1)
print("예측:", 예측, "정확도:", (예측 == 정답).float().mean())

# 방법 1: 줄 번호 목록과 칸 번호 목록을 짝지어 꺼내기
print(확률[torch.arange(4), 정답])

# 방법 2: gather. dim=1 방향에서 index가 가리키는 칸을 꺼낸다
번호표 = 정답.unsqueeze(1)                  # (4,) → (4, 1): 확률과 차원 수를 맞춘다
정답_확률 = 확률.gather(dim=1, index=번호표)
print(정답_확률.shape)
print(정답_확률.squeeze(1))

# 정답 확률이 낮을수록 커지는 벌점: -log를 씌워 평균
print(정답_확률.log().neg().mean())
```

```
예측: tensor([0, 1, 2, 0]) 정확도: tensor(0.7500)
tensor([0.7000, 0.8000, 0.4000, 0.3000])
torch.Size([4, 1])
tensor([0.7000, 0.8000, 0.4000, 0.3000])
tensor(0.6750)
```

먼저 `argmax`로 예측을 구하고 정답과 비교해 **정확도**를 냈습니다. 같은 자리끼리 비교한 참/거짓을 `.float()`로 1과 0으로 바꾼 뒤 평균을 내면 "맞힌 비율"이 돼요. 넷 중 셋을 맞혀 0.75입니다. 마지막 사진은 정답이 개(1번)인데 고양이(0번)를 골랐네요.

정답 칸의 확률을 꺼내는 방법은 두 가지입니다. **방법 1**은 앞에서 본 목록 인덱싱을 두 차원에 동시에 쓰는 것이에요. `확률[[0, 1, 2, 3], [0, 1, 2, 1]]`은 (0, 0), (1, 1), (2, 2), (3, 1) 칸을 짝지어

꺼냅니다. **방법 2**인 `gather`는 "`dim=1` 방향에서, 줄마다 번호표(`index`)가 가리키는 칸을 꺼내라"는 뜻이에요. 번호표는 원본과 **차원 수가 같아야** 해서 `unsqueeze(1)`로 `(4,)`를 `(4, 1)`로 바꿔 넘겼고, 결과도 `(4, 1)`로 나오니 `squeeze(1)`로 풀었습니다.

꺼낸 정답 확률 0.7, 0.8, 0.4, 0.3을 보면 마지막 사진의 0.3이 가장 낮습니다. 마지막 줄은 여기에 `-log`를 씌워 평균한 값이에요. 확률이 1에 가까우면 0에 가깝고, 확률이 낮을수록 크게 커지는 "벌점"입니다. 6장에서 이 계산을 **교차 엔트로피 손실**이라는 이름으로 다시 만나요. 지금은 "정답 칸을 꺼내는 일이 손실 계산의 한가운데에 있다"는 것만 기억해 두세요.

번호표로 써 넣기: scatter

`gather`가 번호표로 **꺼내는** 것이라면, `scatter`는 반대로 번호표가 가리키는 칸에 **써 넣습니다.**

```
import torch

정답 = torch.tensor([0, 1, 2, 1])

# scatter_: gather의 반대. 번호표가 가리키는 칸에 값을 써 넣는다
원핫 = torch.zeros(4, 3)
원핫.scatter_(dim=1, index=정답.unsqueeze(1), value=1.0)
print(원핫)
print(torch.nn.functional.one_hot(정답, num_classes=3))   # 같은 일을 하는 지름길

# 같은 칸에 여러 번 오면 더하기: 클래스별 개수 세기
개수 = torch.zeros(3).scatter_add_(0, 정답, torch.ones(4))
print(개수)
print(torch.bincount(정답))                                # 개수만 필요하면 이쪽
```

```
tensor([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.],
        [0., 1., 0.]])
tensor([[1, 0, 0],
        [0, 1, 0],
        [0, 0, 1],
        [0, 1, 0]])
```

```
tensor([1., 2., 1.])
tensor([1, 2, 1])
```

0으로 채운 `(4, 3)` 표에 정답 번호 칸에만 1을 써 넣었습니다. 이렇게 정답 칸만 1이고 나머지는 0인 표현을 **원-핫(one-hot) 인코딩**이라고 해요. 2장에서 범주형 특성을 숫자로 바꿀 때 봤던 바로 그 모양입니다. 원-핫만 필요하면 `torch.nn.functional.one_hot`이라는 지름길이 있으니 그쪽을 쓰면 되고, 결과가 정수형으로 나온다는 점만 다릅니다.

`scatter_add_`는 같은 칸으로 여러 번 오면 덮어쓰지 않고 **더합니다.** 정답 `[0, 1, 2, 1]`에서 1번 칸에 두 번 1을 더했으니 클래스별 개수 `[1, 2, 1]`이 나왔어요. 개수 세기만 필요하면 `bincount`가 더 간단합니다. 7장에서 불균형 데이터를 다룰 때 클래스별 개수를 세는 데 이 함수를 씁니다.

4.5 모양 바꾸기

데이터를 모델에 넣다 보면 "숫자는 그대로인데 모양만 바꾸고 싶은" 순간이 계속 옵니다. 일렬로 늘어선 숫자를 표로 접거나, 사진의 차원 순서를 바꾸거나, 배치 차원을 하나 덧붙이는 일이에요.

reshape: 접고 펴기

```
import torch

x = torch.arange(12)
print(x.reshape(3, 4))
print(x.reshape(2, -1))       # -1: 나머지는 알아서 계산
print(x.reshape(2, 2, 3).shape)

try:
    x.reshape(5, -1)
except RuntimeError as e:
    print("오류:", e)
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
tensor([[ 0,  1,  2,  3,  4,  5],
        [ 6,  7,  8,  9, 10, 11]])
torch.Size([2, 2, 3])
오류: shape '[5, -1]' is invalid for input of size 12
```

reshape는 숫자 12개를 그대로 둔 채 **줄을 바꿔 다시 늘어놓습니다.** 왼쪽 위부터 오른쪽으로, 한 줄이 차면 다음 줄로 채워요. 한 자리에 -1을 쓰면 "나머지는 알아서 계산해 줘"라는 뜻입니다. 12개를 2줄로 나누면 한 줄에 6개라는 걸 PyTorch가 계산하는 거죠.

숫자 개수가 맞지 않으면 바로 오류가 납니다. 12개를 5줄로 똑같이 나눌 수는 없으니까요. 오류 메시지가 "입력 크기 12에 대해 모양 [5, -1]은 불가능"이라고 친절하게 알려 줍니다.

permute와 전치: 차원 순서 바꾸기

이번에는 숫자를 다시 늘어놓는 게 아니라 **차원의 순서**를 바꾸는 경우입니다.

```
import torch

# 사진 파일을 읽으면 보통 (세로, 가로, 색) 순서
사진 = torch.zeros(224, 224, 3)
파이토치용 = 사진.permute(2, 0, 1)     # (색, 세로, 가로)로 순서 바꾸기
print(사진.shape, "→", 파이토치용.shape)

행렬 = torch.arange(6).reshape(2, 3)
print(행렬.T)                          # 2차원은 .T로 전치
print(행렬.T.is_contiguous())          # 순서만 바꿔 본 것이라 메모리는 그대로

try:
    행렬.T.view(6)
except RuntimeError as e:
    print("view 오류:", str(e).split(".")[0])
print(행렬.T.reshape(6))               # reshape는 필요하면 복사해서 해결
```

```
torch.Size([224, 224, 3]) → torch.Size([3, 224, 224])
tensor([[0, 3],
        [1, 4],
```

```
       [2, 5]])
False
view 오류: view size is not compatible with input tensor's size and stride (at least one dimension
↪ spans across two contiguous subspaces)
tensor([0, 3, 1, 4, 2, 5])
```

사진 파일을 읽어 오면 보통 (세로, 가로, 색) 순서인데, PyTorch는 4.2절에서 본 것처럼 (색, 세로, 가로) 순서를 원합니다. `permute(2, 0, 1)`은 "새 0번 자리에 옛 2번 차원, 새 1번에 옛 0번, 새 2번에 옛 1번을 놓아라"라는 뜻이에요. 2차원 표의 행과 열을 바꾸는 **전치**는 `.T`로 짧게 쓸 수 있습니다.

그런데 전치한 텐서에 `.view(6)`을 하니 오류가 났습니다. `view`는 `reshape`와 비슷하지만 **절대로 복사하지 않는** 방법이에요. 전치는 숫자를 실제로 옮기지 않고 "읽는 순서"만 바꿔 놓은 상태라(`is_contiguous()`가 `False`), 메모리에 놓인 순서와 보이는 순서가 다릅니다. 이 상태에서는 복사 없이 한 줄로 펼 수가 없어서 `view`는 포기하고, `reshape`는 필요하면 조용히 복사해서 해결합니다.

> **Q. 그럼 항상 `reshape`만 쓰면 되지 않나요?** 대부분은 그래도 됩니다. 다만 `reshape`는 상황에 따라 원본과 메모리를 공유하기도 하고 복사본을 만들기도 해서, 결과를 바꿨을 때 원본이 바뀌는지 미리 알기 어렵습니다. "공유가 꼭 필요하다"면 `view`를 써서 안 될 때 오류로 알게 하고, 그런 의도가 없으면 `reshape`를 쓰세요. 다른 사람의 코드에서 `view`가 많이 보이는 건 예전 PyTorch에 `reshape`가 없던 시절의 습관이기도 해요.

메모리 속 모습: stride와 contiguous

앞에서 "전치는 숫자를 실제로 옮기지 않고 읽는 순서만 바꾼다"고 했습니다. 읽는 순서를 어떻게 바꾼다는 걸까요? 사실 메모리는 표가 아니라 **한 줄로 늘어선 칸**입니다. 2행 3열 표도 메모리 안에서는 0, 1, 2, 3, 4, 5가 일렬로 놓여 있어요. 텐서는 여기에 "다음 행으로 가려면 몇 칸 건너뛰고, 다음 열로 가려면 몇 칸 건너뛰는지"라는 **보폭(stride)** 을 함께 기억해서 표처럼 보여 줍니다.

```
import torch

행렬 = torch.arange(6).reshape(2, 3)
print(행렬)
print("보폭(stride):", 행렬.stride())       # 다음 행으로 3칸, 다음 열로 1칸

전치 = 행렬.T
```

```
print(전치)
print("보폭(stride):", 전치.stride())       # 같은 메모리를 다른 보폭으로 읽는다
print("같은 메모리?", 전치.data_ptr() == 행렬.data_ptr())
print("이어져 있나?", 전치.is_contiguous())

정리본 = 전치.contiguous()                  # 보이는 순서대로 새로 늘어놓은 복사본
print("보폭(stride):", 정리본.stride(), "이어져 있나?", 정리본.is_contiguous())
print("같은 메모리?", 정리본.data_ptr() == 행렬.data_ptr())
print(정리본.view(6))                       # 이제 view도 된다
```

```
tensor([[0, 1, 2],
        [3, 4, 5]])
보폭(stride): (3, 1)
tensor([[0, 3],
        [1, 4],
        [2, 5]])
보폭(stride): (1, 3)
같은 메모리? True
이어져 있나? False
보폭(stride): (2, 1) 이어져 있나? True
같은 메모리? False
tensor([0, 3, 1, 4, 2, 5])
```

그림으로 그리면 이렇습니다. 메모리의 줄은 하나인데, 보폭만 바꿔서 두 가지 표로 읽고 있어요.

```
메모리:  [0] [1] [2] [3] [4] [5]

행렬   보폭 (3, 1): 오른쪽 칸은 +1, 아랫줄은 +3
         0 1 2
         3 4 5

전치   보폭 (1, 3): 오른쪽 칸은 +3, 아랫줄은 +1
         0 3
         1 4
         2 5
```

행렬의 보폭 `(3, 1)`은 "아랫줄로 가려면 3칸, 오른쪽 칸으로 가려면 1칸 건너뛴다"는 뜻입니다. **전**

치는 같은 메모리(data_ptr가 같음)를 보폭 (1, 3)으로 읽어요. 아랫줄로 가면 1칸, 오른쪽으로 가면 3칸이니, 0 다음 줄은 1이고 0 오른쪽은 3이 됩니다. 숫자는 한 개도 움직이지 않고 보폭만 바꿨으니 전치가 공짜인 거예요.

이어져 있다(contiguous) 는 건 "보이는 순서대로 읽으면 메모리를 처음부터 한 칸씩 차례로 밟는다"는 뜻입니다. 전치를 보이는 순서(0, 3, 1, 4, 2, 5)대로 읽으면 메모리를 앞뒤로 건너뛰어야 하니 이어져 있지 않아요. view(6)이 실패한 이유가 이것입니다. view는 메모리를 그대로 두고 보폭만 바꾸는 방법이라, 메모리를 건너뛰며 읽어야 하는 순서는 한 줄짜리 보폭 하나로 나타낼 수 없거든요.

.contiguous()는 보이는 순서대로 숫자를 **새로 복사해서** 늘어놓습니다. 결과의 보폭이 (2, 1)로 바뀌고, 메모리 주소도 원본과 달라졌죠. 이제는 이어져 있으니 view(6)도 됩니다. 사실 reshape가 "필요하면 조용히 복사한다"고 할 때 하는 일이 바로 이 contiguous() 뒤의 view예요. 다른 사람의 코드에서 x.permute(...).contiguous().view(...) 같은 줄을 보면 이제 왜 그렇게 썼는지 읽을 수 있을 겁니다. 12장 트랜스포머를 직접 구현할 때 이 패턴이 그대로 나옵니다.

squeeze, unsqueeze, cat, stack, flatten

마지막으로 자주 쓰는 다섯 가지를 한꺼번에 보겠습니다.

```
import torch

사진 = torch.zeros(3, 224, 224)
묶음 = 사진.unsqueeze(0)            # 맨 앞에 크기 1인 차원 추가
print(묶음.shape)
print(묶음.squeeze(0).shape)        # 크기 1인 차원 빼기

a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
print(torch.cat([a, b]))            # 이어 붙이기: 차원 수 그대로
print(torch.stack([a, b]))          # 쌓기: 새 차원이 생긴다
print(torch.stack([a, b]).shape, torch.stack([a, b], dim=1).shape)

print(torch.zeros(32, 3, 4, 4).flatten(start_dim=1).shape)   # 사진마다 한 줄로
```

```
torch.Size([1, 3, 224, 224])
torch.Size([3, 224, 224])
```

```
tensor([1, 2, 3, 4, 5, 6])
tensor([[1, 2, 3],
        [4, 5, 6]])
torch.Size([2, 3]) torch.Size([3, 2])
torch.Size([32, 48])
```

- `unsqueeze(0)`: 맨 앞에 크기 1인 차원을 끼워 넣습니다. 모델은 항상 배치를 기대하므로, 사진 **한 장**을 넣을 때는 "1장짜리 묶음"으로 포장해야 해요. 실무에서 정말 자주 쓰입니다.
- `squeeze(0)`: 반대로 크기 1인 차원을 빼서 포장을 풉니다.
- `cat`: 이미 있는 차원 방향으로 **이어 붙입니다.** 숫자 3개 + 3개 = 6개, 차원 수는 그대로예요.
- `stack`: **새 차원을 만들어 쌓습니다.** 줄 두 개를 쌓으니 2행 3열 표가 됐죠. `dim=1`로 쌓으면 3행 2열이 됩니다.
- `flatten(start_dim=1)`: 지정한 차원부터 뒤를 전부 한 줄로 폅니다. 사진 묶음 `(32, 3, 4, 4)`에서 배치는 남기고 사진마다 48개짜리 한 줄로 만드는 데 써요. 9장 CNN에서 다시 만납니다.

4.6 한꺼번에 계산하기

같은 자리끼리

텐서 계산의 기본은 **같은 자리끼리** 하는 원소별 연산입니다.

```
import torch

국어 = torch.tensor([80.0, 92.0, 75.0])
수학 = torch.tensor([70.0, 88.0, 95.0])

print(국어 + 수학)          # 같은 자리끼리 더하기
print((국어 + 수학) / 2)    # 평균
print(국어 > 수학)          # 같은 자리끼리 비교
print(torch.sqrt(국어))     # 함수도 칸마다
```

```
tensor([150., 180., 170.])
tensor([75., 90., 85.])
```

```
tensor([ True,  True, False])
tensor([8.9443, 9.5917, 8.6603])
```

국어 + 수학은 학생별로 두 점수를 더합니다. 1권에서였다면 zip과 반복문을 썼을 일을 한 줄로 끝내는 거예요. 비교 연산도 같은 자리끼리 해서 참/거짓 텐서를 돌려주고, torch.sqrt 같은 수학 함수도 칸마다 적용됩니다.

브로드캐스팅: 모양이 달라도 계산되는 규칙

모양이 다른 텐서끼리는 어떻게 될까요? 학생 3명의 과목 2개 점수에 보너스를 줘 보겠습니다.

```
import torch

# 학생 3명 × 과목 2개 점수
점수 = torch.tensor([[80.0, 70.0],
                     [92.0, 88.0],
                     [75.0, 95.0]])

print(점수 + 5)                              # 모두에게 5점
과목별_보너스 = torch.tensor([3.0, 10.0])     # 국어 +3, 수학 +10
print(점수 + 과목별_보너스)                   # (3, 2) + (2,) → (3, 2)

학생별_보너스 = torch.tensor([[1.0], [2.0], [3.0]])   # (3, 1)
print(점수 + 학생별_보너스)                           # (3, 2) + (3, 1) → (3, 2)

try:
    점수 + torch.tensor([1.0, 2.0, 3.0])              # (3, 2) + (3,)
except RuntimeError as e:
    print("오류:", e)
```

```
tensor([[ 85.,  75.],
        [ 97.,  93.],
        [ 80., 100.]])
tensor([[ 83.,  80.],
        [ 95.,  98.],
        [ 78., 105.]])
```

```
tensor([[81., 71.],
        [94., 90.],
        [78., 98.]])
오류: The size of tensor a (2) must match the size of tensor b (3) at non-singleton dimension 1
```

`점수 + 5`는 숫자 하나를 **모든 칸에** 더합니다. `점수 + 과목별_보너스`에서는 `(2,)` 모양 보너스가 학생 세 명 줄마다 똑같이 반복되어 더해지고, `(3, 1)` 모양 학생별 보너스는 과목 두 칸에 똑같이 반복됩니다. 이렇게 **작은 쪽을 늘려서(펼쳐서) 모양을 맞추는 규칙**을 **브로드캐스팅(broadcasting)**이라고 해요. 방송(broadcast)처럼 한 값을 여러 곳에 퍼뜨린다는 뜻입니다.

규칙은 세 줄로 정리됩니다.

1. 두 모양을 **오른쪽 끝부터** 맞대어 비교합니다.
2. 각 자리의 길이가 **같거나, 한쪽이 1이면** 통과입니다. 1인 쪽이 상대 길이만큼 늘어나요.
3. 한쪽에 자리가 없으면 1이 있는 것으로 봅니다.

`(3, 2)`와 `(2,)`를 오른쪽부터 맞대면 2와 2가 같고, 왼쪽은 3과 "없음(1로 봄)"이라 통과입니다. `(3, 2)`와 `(3, 1)`은 2와 1(늘어남), 3과 3이라 통과예요. 마지막 `(3, 2)`와 `(3,)`은 오른쪽 끝이 2와 3으로 서로 다르고 둘 다 1이 아니라서 오류가 납니다. 사람 눈에는 "학생이 3명이니 학생별 보너스겠지" 싶지만, 브로드캐스팅은 **오른쪽부터** 맞추기 때문에 과목 자리에 3을 대 보고 실패한 거예요. 학생별로 주고 싶으면 `(3, 1)`처럼 모양을 분명히 해 주면 됩니다.

> **Q. 브로드캐스팅이 편하긴 한데 위험하지는 않나요?** 위험할 때가 있습니다. 위의 오류는 오히려 다행인 경우예요. 진짜 무서운 건 의도와 다르게 **조용히 계산되는** 경우입니다. 예를 들어 `(3,)`과 `(3, 1)`을 더하면 오류 없이 `(3, 3)`이 나와요. 셋씩 더하려던 게 아홉 칸짜리 표가 되는 거죠. 그래서 브로드캐스팅을 쓸 때는 결과의 `shape`를 한 번 찍어 보는 습관이 중요합니다.

행렬 곱: @의 정체

1권 3장에서 "신입 캐릭터" `@`를 소개하면서 "리스트끼리는 쓸 수 없고 NumPy 같은 배열에서 쓴다"고 했죠. 드디어 제대로 쓸 차례입니다.

```
import torch

# 학생 3명 × 과목 2개 점수, 과목별 반영 비율
```

```
점수 = torch.tensor([[80.0, 70.0],
                     [92.0, 88.0],
                     [75.0, 95.0]])
비율 = torch.tensor([0.4, 0.6])

print(점수 @ 비율)                    # (3, 2) @ (2,) → (3,)
print((점수 * 비율).sum(dim=1))       # 같은 계산을 풀어 쓰면

두_기준 = torch.tensor([[0.4, 0.5],
                        [0.6, 0.5]])  # 기준 A, 기준 B
print(점수 @ 두_기준)                 # (3, 2) @ (2, 2) → (3, 2)
```

```
tensor([74.0000, 89.6000, 87.0000])
tensor([74.0000, 89.6000, 87.0000])
tensor([[74.0000, 75.0000],
        [89.6000, 90.0000],
        [87.0000, 85.0000]])
```

학생 셋의 국어·수학 점수에 반영 비율 40%, 60%를 적용해 최종 점수를 구했습니다. `점수 @ 비율`은 학생마다 "점수와 비율을 같은 자리끼리 곱해서 모두 더한 값"을 계산해요. 풀어 쓴 두 번째 줄과 결과가 같죠? 행렬 곱은 이렇게 **곱하고 더하기**를 한꺼번에 하는 연산입니다.

비율 기준이 둘이면 `(2, 2)` 행렬을 곱해서 기준별 점수를 한 번에 얻을 수 있어요. 모양 규칙은 **`(a, b) @ (b, c) → (a, c)`**, 즉 왼쪽의 마지막 길이와 오른쪽의 첫 길이가 같아야 하고, 그 자리가 곱하고 더하면서 사라집니다. 앞으로 만날 신경망의 층 하나하나가 결국 "입력 @ 가중치 + 편향"이라는 이 계산이에요. 6장에서 바로 이 모양으로 첫 신경망을 만듭니다.

einsum: 행렬 곱을 글자로 쓰기

행렬 곱, 전치, 합계를 섞어 쓰다 보면 "어느 차원끼리 곱하고 어느 차원을 없애는지"가 코드에서 잘 안 보일 때가 있습니다. `torch.einsum`은 차원마다 **글자 이름**을 붙이고, 계산 전후의 모양을 글자로 적는 방법이에요.

```
import torch

점수 = torch.tensor([[80.0, 70.0],
```

```
                    [92.0, 88.0],
                    [75.0, 95.0]])          # (학생 s, 과목 k)
비율 = torch.tensor([0.4, 0.6])             # (과목 k)

print(torch.einsum("sk,k->s", 점수, 비율))   # 점수 @ 비율과 같다
print(torch.einsum("sk->ks", 점수))          # 전치: 글자 순서만 바꾸기
print(torch.einsum("sk->k", 점수))           # s가 사라짐 = sum(dim=0)

# 반 2개 × 학생 3명 × 과목 2개, 반마다 다른 반영 기준 2가지 (예제용)
반별점수 = torch.stack([점수, 점수 - 10])     # (b, s, k) = (2, 3, 2)
기준 = torch.tensor([[[0.4, 0.5], [0.6, 0.5]],
                    [[0.5, 0.3], [0.5, 0.7]]])   # (b, k, c) = (2, 2, 2)
결과 = torch.einsum("bsk,bkc->bsc", 반별점수, 기준)
print(결과.shape)
print(torch.allclose(결과, 반별점수 @ 기준))   # @도 앞 차원을 배치로 보고 똑같이 계산
```

```
tensor([74.0000, 89.6000, 87.0000])
tensor([[80., 92., 75.],
        [70., 88., 95.]])
tensor([247., 253.])
torch.Size([2, 3, 2])
True
```

`"sk,k->s"`는 이렇게 읽습니다. 첫 번째 텐서의 차원은 학생(s)과 과목(k), 두 번째는 과목(k)이고, 결과에는 s만 남긴다. 화살표 오른쪽에서 **사라진 글자(k)는 곱한 뒤 더해서 없앤다**는 것이 einsum의 유일한 규칙이에요. 그래서 이 줄은 `점수 @ 비율`과 같은 74, 89.6, 87을 냅니다. `"sk->ks"`는 글자 순서만 바꿨으니 전치이고, `"sk->k"`는 s가 사라졌으니 학생 방향으로 더한 `sum(dim=0)`입니다.

einsum이 빛나는 건 차원이 많을 때입니다. 마지막 예제는 반(b)이 둘 있고, 반마다 학생 셋의 점수에 **반마다 다른** 반영 기준을 곱합니다. `"bsk,bkc->bsc"`는 "반(b)은 짝을 맞춰 따로따로, 그 안에서 과목(k)을 곱하고 더해 없앤다"는 뜻이에요. 결과가 `@`와 같다는 것도 확인했습니다. `@`는 3차원 이상이면 **앞쪽 차원을 배치로 보고** 마지막 두 차원끼리 행렬 곱을 해 주거든요.

글자를 이름처럼 붙여 읽는 건 4.2절에서 권한 "모양을 자리마다 이름 붙여 읽기"와 같은 생각입니다. 12장에서 어텐션을 직접 구현할 때 (배치, 헤드, 단어, 차원)처럼 차원이 넷인 텐서를 곱하는데, 그때 einsum으로 쓰면 수식과 코드가 거의 그대로 겹쳐 보여요.

집계: 차원을 따라 접기

합계, 평균, 최댓값처럼 여러 숫자를 하나로 줄이는 계산을 **집계**라고 합니다. 텐서에서는 **어느 방향으로 접을지**를 `dim`으로 정해요.

```
import torch

점수 = torch.tensor([[80.0, 70.0],
                     [92.0, 88.0],
                     [75.0, 95.0]])

print(점수.sum(), 점수.mean())
print(점수.mean(dim=0))                # 세로로 접기: 과목별 평균
print(점수.mean(dim=1))                # 가로로 접기: 학생별 평균
print(점수.mean(dim=1, keepdim=True).shape)
최고 = 점수.max(dim=0)
print(최고.values, 최고.indices)       # 과목별 최고점과 그 학생 번호
print(점수.argmax(dim=1))              # 학생별로 더 잘한 과목
```

```
tensor(500.) tensor(83.3333)
tensor([82.3333, 84.3333])
tensor([75., 90., 85.])
torch.Size([3, 1])
tensor([92., 95.]) tensor([1, 2])
tensor([0, 0, 1])
```

`dim`을 이해하는 가장 쉬운 방법은 **"그 차원이 사라진다"** 고 생각하는 것입니다. `(3, 2)` 표에서 `dim=0`(학생 방향)으로 평균을 내면 학생 차원이 사라지고 과목별 평균 2개가 남습니다. `dim=1`(과목 방향)이면 과목 차원이 사라지고 학생별 평균 3개가 남아요. `dim`을 주지 않으면 전부 접어서 숫자 하나가 됩니다.

`keepdim=True`를 주면 사라질 차원을 크기 1로 남겨 둡니다. 결과 모양이 `(3,)`이 아니라 `(3, 1)`이 되는 거죠. 앞의 브로드캐스팅 규칙을 떠올려 보세요. 학생별 평균을 각 학생 줄에서 빼고 싶을 때 `(3, 1)`이면 바로 브로드캐스팅되지만 `(3,)`이면 오른쪽 끝에 맞춰지는 바람에 오류가 납니다. 4.11절 실습에서 이 조합을 씁니다.

`max(dim=0)`은 최댓값과 **그 위치**를 함께 돌려줍니다. 국어 최고점 92점은 1번 학생, 수학 최고점 95

점은 2번 학생이라는 뜻이에요. 위치만 필요하면 `argmax`를 씁니다. 분류 모델이 "몇 번 클래스일 확률이 가장 높은가"를 고를 때 바로 이 `argmax`를 써요.

정렬과 상위 k개

최댓값 하나가 아니라 "1등부터 꼴찌까지" 순서가 필요하거나, "상위 3명"만 필요할 때도 있습니다.

```
import torch

점수 = torch.tensor([72.0, 95.0, 88.0, 60.0, 91.0, 79.0])
정렬 = 점수.sort(descending=True)
print(정렬.values)
print(정렬.indices)                # 원래 몇 번째 학생이었는지
print(점수.argsort())              # 오름차순 순서(번호)만

상위 = 점수.topk(3)                # 큰 값 3개만
print(상위.values, 상위.indices)

# 사진 2장의 클래스별 확률에서 장마다 상위 2개 후보
확률 = torch.tensor([[0.05, 0.60, 0.10, 0.25],
                     [0.40, 0.05, 0.35, 0.20]])
값, 번호 = 확률.topk(2, dim=1)
print(번호)
print(값)
정답 = torch.tensor([3, 1])
print("상위 2개 안에 정답이 있나:", (번호 == 정답.unsqueeze(1)).any(dim=1))
```

```
tensor([95., 91., 88., 79., 72., 60.])
tensor([1, 4, 2, 5, 0, 3])
tensor([3, 0, 5, 2, 4, 1])
tensor([95., 91., 88.]) tensor([1, 4, 2])
tensor([[1, 3],
        [0, 2]])
tensor([[0.6000, 0.2500],
        [0.4000, 0.3500]])
상위 2개 안에 정답이 있나: tensor([ True, False])
```

`sort`는 정렬한 값(`values`)과 함께 그 값이 **원래 몇 번째였는지**(`indices`)를 돌려줍니다. 1권 6장의 `sorted()`는 값만 돌려줬지만, 데이터 분석에서는 "95점이 누구였나"가 더 중요할 때가 많아요. 1번 학생이 1등, 4번 학생이 2등이라는 걸 바로 알 수 있습니다. 번호만 필요하면 `argsort`를 쓰는데, 기본은 오름차순이라 꼴찌(3번 학생, 60점)부터 나왔어요.

`topk(3)`은 전체를 정렬하지 않고 **큰 값 k개만** 골라 줍니다. 데이터가 크면 전부 정렬하는 것보다 빠르고 코드도 짧아요. 2차원에서는 `dim`을 주면 줄마다 따로 고릅니다. 분류 대회에서 자주 쓰는 "상위 5개 정확도(top-5 accuracy)"가 바로 이 계산이에요. 모델이 첫 번째로 고른 답은 틀렸어도 상위 후보 안에 정답이 있으면 맞힌 것으로 칩니다. 마지막 줄에서 첫 번째 사진은 정답 3번이 후보 `[1, 3]` 안에 있어서 `True`, 두 번째 사진은 정답 1번이 후보 `[0, 2]`에 없어서 `False`가 나왔어요. `(2, 2)`와 `(2, 1)`을 비교하는 데 브로드캐스팅을 썼고, `any(dim=1)`은 "그 줄에 참이 하나라도 있나"를 묻는 집계입니다.

4.7 제자리 연산과 메모리

```
import torch

a = torch.tensor([1.0, 2.0, 3.0])
b = a.add(10)          # 새 텐서를 돌려준다
print(a, b)
a.add_(10)             # 밑줄: a 자신을 바꾼다
print(a)
```

```
tensor([1., 2., 3.]) tensor([11., 12., 13.])
tensor([11., 12., 13.])
```

PyTorch 함수 중에는 이름 끝에 밑줄(`_`)이 붙은 짝꿍이 있습니다. `add`는 새 텐서를 돌려주고 원본은 그대로 두지만, `add_`는 **원본 자체를 바꿉니다.** 1권 3장의 복합 할당 연산자(`+=`)와 비슷한 "제자리" 연산이에요. 메모리를 새로 잡지 않으니 아주 큰 텐서에서는 유리하지만, 5장에서 볼 자동 미분과 섞어 쓰면 "기울기 계산에 필요한 값이 제자리 연산으로 바뀌었다"는 오류를 만나기 쉬워서 익숙해지기 전까지는 밑줄 없는 쪽을 쓰는 것을 권합니다.

이 장에서 원본과 메모리를 **함께 쓰는** 경우가 여러 번 나왔으니 한곳에 모아 두겠습니다.

이렇게 만들면	원본과의 관계
`torch.from_numpy(배열)`, `텐서.numpy()` (CPU)	공유
`텐서[1:3]`, `텐서[:, 0]` 같은 슬라이스	공유
`view`, `permute`, `.T`, `unsqueeze`, `squeeze`	공유
`reshape`	될 수 있으면 공유, 안 되면 복사
`torch.tensor(...)`, `.clone()`	복사
마스크·목록으로 고른 결과, `cat`, `stack`	복사
`.contiguous()`	이미 이어져 있으면 그대로, 아니면 복사
`텐서.add_(...)` 같은 밑줄 함수	원본 자체를 바꿈

둘이 정말 같은 메모리를 쓰는지 궁금하면 `a.data_ptr() == b.data_ptr()`처럼 메모리 주소를 비교해 보면 됩니다. 1권 3장의 `is`가 "같은 물건인가"를 묻던 것과 비슷한 확인 방법이에요.

4.8 장치: CPU와 GPU 사이 옮기기

장치 고르기

이제 텐서의 두 번째 능력, GPU 계산입니다. PyTorch에서는 계산하는 곳을 **장치(device)** 라고 부르고, 크게 세 가지가 있어요.

- `cpu`: 모든 컴퓨터에 있는 기본 장치
- `cuda`: NVIDIA 그래픽 카드
- `mps`: Apple 실리콘(M1~M4 등) 맥의 GPU

```
import torch


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
```

```
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")


장치 = 장치_고르기()
print("고른 장치:", 장치)

x = torch.ones(3)                  # 처음에는 CPU에서 만들어진다
print(x.device)
y = x.to(장치)                     # 장치로 옮기기 (새 텐서를 돌려준다)
print(y.device)
z = torch.ones(3, device=장치)     # 처음부터 그 장치에 만들기
print((y + z).cpu())               # 결과를 다시 CPU로
```

```
고른 장치: mps
cpu
mps:0
tensor([2., 2., 2.])
```

`장치_고르기()`는 이 책의 모든 예제에서 쓸 함수입니다. 쓸 수 있는 가장 빠른 장치를 골라 주니, 같은 코드가 NVIDIA GPU가 있는 서버에서도, 맥북에서도, GPU 없는 노트북에서도 그대로 돌아가요. 이 책을 쓴 맥에서는 `mps`가 골라졌습니다.

텐서는 처음에 CPU에서 만들어지고, `.to(장치)`로 옮깁니다. `.to()`는 4.7절의 밑줄 함수와 달리 **새 텐서를 돌려주니** `y = x.to(장치)`처럼 받아야 해요. 처음부터 `device=장치`를 주고 만들면 옮기는 시간을 아낄 수 있습니다. 출력에 `mps:0`이라고 나온 것은 "0번 GPU"라는 뜻이에요. GPU가 여러 개인 서버에서는 `cuda:0`, `cuda:1`처럼 번호로 구분합니다(17장).

서로 다른 장치는 계산할 수 없습니다

```
import torch

if torch.backends.mps.is_available():
    gpu = torch.ones(3, device="mps")
    cpu = torch.ones(3)
```

```
    try:
        gpu + cpu
    except RuntimeError as e:
        print("오류:", str(e)[:120])
    print(gpu + cpu.to("mps"))
```

```
오류: Expected all tensors to be on the same device, but found at least two devices, mps:0 and cpu!
tensor([2., 2., 2.], device='mps:0')
```

GPU에 있는 텐서와 CPU에 있는 텐서를 더하려 하자 "모든 텐서가 같은 장치에 있어야 하는데 mps:0과 cpu 두 곳에서 발견됐다"는 오류가 났습니다. 처음 GPU를 쓸 때 가장 많이 만나는 오류예요. 모델은 GPU로 옮겼는데 데이터는 CPU에 두었을 때 주로 생깁니다. 해결법은 간단해요. 계산에 들어가는 텐서를 모두 같은 장치로 옮기면 됩니다.

Q

Q. GPU가 없으면 이 책을 못 따라가나요? 따라갈 수 있습니다. 이 책의 예제는 노트북 CPU나 맥의 MPS에서 몇 분 안에 끝나도록 크기를 맞췄어요. 시간이 오래 걸리는 몇몇 학습 예제는 GPU 서버에서 돌린 결과를 함께 실었고, 무료로 GPU를 빌려 쓰는 방법(Colab 등)은 부록에서 소개합니다. `장치_고르기()`를 쓰면 어느 환경에서든 코드를 바꿀 필요가 없어요.

얼마나 빨라질까요?

```
import time
import torch

크기 = 4096
a = torch.randn(크기, 크기)
b = torch.randn(크기, 크기)

시작 = time.perf_counter()
a @ b
print(f"CPU: {time.perf_counter() - 시작:.3f}초")

if torch.backends.mps.is_available():
```

```
    ga, gb = a.to("mps"), b.to("mps")
    ga @ gb                              # 처음 한 번은 준비 운동
    torch.mps.synchronize()
    시작 = time.perf_counter()
    ga @ gb
    torch.mps.synchronize()              # GPU 일이 끝날 때까지 기다린 뒤 잰다
    print(f"MPS: {time.perf_counter() - 시작:.3f}초")
```

```
CPU: 0.056초
MPS: 0.015초
```

4096 × 4096 행렬 두 개를 곱하는 데 이 맥에서 CPU는 0.05초 안팎, MPS는 0.01초대가 걸렸습니다. 숫자는 장비마다 크게 다르지만, 측정하는 **방법**에서 두 가지는 꼭 기억해 두세요.

- **처음 한 번은 빼고 잽니다.** GPU는 첫 계산 때 준비 작업을 해서 느립니다. 그래서 한 번 돌려 둔 뒤에 쟀어요.
- **GPU 일이 끝날 때까지 기다린 뒤 잽니다.** GPU에 일을 맡기면 파이썬은 끝나기를 기다리지 않고 바로 다음 줄로 넘어갑니다. `torch.mps.synchronize()`(NVIDIA는 `torch.cuda.synchronize()`)로 기다리지 않으면 "일을 맡긴 시간"만 재게 돼서 엉터리로 빠르게 나와요.

행렬이 작으면 오히려 CPU가 빠를 수도 있습니다. GPU로 옮기고 준비하는 시간이 계산 시간보다 커지기 때문이에요. 16장에서 언제 GPU가 이득인지 더 정확히 재 봅니다.

4.9 텐서 저장하고 불러오기

torch.save와 torch.load

몇 시간 걸려 전처리한 데이터나 계산 결과를 프로그램이 끝날 때마다 버린다면 아깝겠죠. 1권 12장에서 파이썬 변수를 `pickle`로 통째로 저장했던 것처럼, 텐서는 `torch.save`로 파일에 담고 `torch.load`로 꺼냅니다. 파일 이름 끝은 관례상 `.pt`를 붙여요.

```
import tempfile
from pathlib import Path
```

```
import torch

기온 = torch.tensor([[ 2.0, 13.5, 25.7, 15.0],
                     [ 4.5, 15.0, 25.0, 17.5],
                     [ 1.0, 14.0, 26.0, 15.5]])

with tempfile.TemporaryDirectory() as 폴더:      # 연습용 임시 폴더 (끝나면 지워진다)
    경로 = Path(폴더) / "기온.pt"
    torch.save(기온, 경로)
    다시 = torch.load(경로)
    print(다시.shape, torch.equal(기온, 다시))

    # 여러 개는 딕셔너리로 묶어 한 파일에
    torch.save({"기온": 기온, "도시": ["가람시", "나래시", "다솜시"], "단위": "°C"},
               Path(폴더) / "묶음.pt")
    묶음 = torch.load(Path(폴더) / "묶음.pt")
    print(묶음["도시"], 묶음["기온"][0])

    # 주의: 슬라이스를 저장하면 원본 메모리 전체가 같이 저장된다
    큰_텐서 = torch.zeros(1_000_000)
    torch.save(큰_텐서[:10], Path(폴더) / "조각.pt")
    torch.save(큰_텐서[:10].clone(), Path(폴더) / "복사.pt")
    for 이름 in ["조각.pt", "복사.pt"]:
        print(이름, f"{(Path(폴더) / 이름).stat().st_size:,}바이트")
```

```
torch.Size([3, 4]) True
['가람시', '나래시', '다솜시'] tensor([ 2.0000, 13.5000, 25.7000, 15.0000])
조각.pt 4,001,577바이트
복사.pt 1,577바이트
```

연습이라 1권 19장에서 썼던 `tempfile`로 임시 폴더를 만들고, `with` 블록이 끝나면 폴더째 지워지게 했습니다. 실제로는 `Path("data") / "기온.pt"`처럼 원하는 경로를 쓰면 돼요. 저장했다가 불러온 텐서가 원본과 모양도 값도 같다는 것을 `torch.equal`로 확인했습니다.

여러 개를 저장할 때는 1권 12장의 pickle 예제처럼 **딕셔너리로 묶어** 한 파일에 넣는 것이 관례입니다. 텐서뿐 아니라 도시 이름 목록, 단위 문자열 같은 파이썬 기본 값도 함께 들어가요. 꺼낼 때 이름

으로 찾으니 순서를 외울 필요도 없습니다. 6장에서 모델을 저장할 때도 모델 안의 숫자들을 이름표 붙은 딕셔너리(`state_dict`)로 꺼내 이 함수로 저장합니다.

마지막 실험은 꼭 기억해 두세요. 숫자 100만 개짜리 텐서에서 **10개만 슬라이스**해 저장했는데 파일이 4MB나 됩니다. 4.4절에서 슬라이스는 원본 메모리를 들여다보는 창이라고 했죠? `torch.save`는 창이 아니라 창 너머의 **메모리 전체**를 저장해서, 나중에 불러와도 창 관계가 유지되게 합니다. 작은 조각만 저장하고 싶으면 `.clone()`으로 복사해서 저장하세요. 같은 10개인데 파일이 1.5KB로 줄었습니다(나머지는 파일 형식의 기본 정보입니다).

Q

Q. 남이 준 .pt 파일을 열어도 안전한가요? 1권 12장에서 "믿을 수 없는 곳에서 받은 pickle 파일은 열지 말라"고 했죠. `.pt` 파일도 속은 pickle 방식이라 같은 위험이 있었습니다. 그래서 요즘 PyTorch의 `torch.load`는 기본으로 **텐서와 숫자·문자열·리스트·딕셔너리 같은 기본 값만** 꺼내고, 그 밖의 파이썬 객체가 들어 있으면 오류를 내며 거부합니다(`weights_only=True`가 기본값). 이 오류를 보고 `weights_only=False`로 바꿔 열고 싶어질 수 있는데, 내가 직접 만든 파일이 확실할 때만 그렇게 하세요.

다른 장치에서 불러오기: map_location

텐서는 자기가 어느 장치에 있었는지까지 함께 저장됩니다. 그러면 GPU 서버에서 저장한 파일을 GPU 없는 노트북에서 열면 어떻게 될까요?

```
import tempfile
from pathlib import Path

import torch


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")
```

```
x = torch.arange(3.0, device=장치_고르기())
with tempfile.TemporaryDirectory() as 폴더:
    경로 = Path(폴더) / "x.pt"
    torch.save(x, 경로)
    print(torch.load(경로).device)                      # 저장할 때의 장치로 돌아온다
    print(torch.load(경로, map_location="cpu").device)  # 불러오면서 CPU로
```

```
mps:0
cpu
```

그냥 불러오면 저장할 때의 장치(이 맥에서는 `mps:0`)로 돌아옵니다. 같은 컴퓨터에서는 문제가 없지만, CUDA로 저장한 파일을 CUDA가 없는 컴퓨터에서 이렇게 열면 "CUDA 장치가 없다"는 오류가 나요. `map_location="cpu"`를 주면 불러오면서 바로 CPU로 옮겨 줍니다. 다른 사람이 준 파일이나 서버에서 만든 파일은 일단 `map_location="cpu"`로 열고, 그다음에 4.8절의 `.to(장치)`로 원하는 곳에 옮기는 것이 가장 안전한 순서예요.

4.10 NumPy와 다른 점, 자주 만나는 오류

이름은 같은데 조금씩 다른 것들

1권 20장에서 NumPy를 익혔다면 PyTorch는 거의 그대로 손에 붙습니다. 그런데 "거의"가 함정이에요. 비슷하게 생겨서 그대로 썼다가 조용히 다른 답을 받는 경우가 있거든요. 같은 표로 나란히 계산해 보겠습니다.

```
import numpy as np
import torch

a = np.array([[1.0, 2.0], [3.0, 4.0]])
t = torch.tensor([[1.0, 2.0], [3.0, 4.0]])

print(a.sum(axis=0), t.sum(dim=0))          # NumPy는 axis, PyTorch는 dim
print(np.concatenate([a, a]).shape, torch.cat([t, t]).shape)
print(a.astype(np.int64).dtype, t.to(torch.int64).dtype)
print(a.max(axis=1))
```

```
print(t.max(dim=1))                          # PyTorch는 값과 위치를 함께 준다

# 표준편차: 이름은 같은데 기본 계산이 다르다
값 = [2.0, 4.0, 4.0, 6.0]
print(np.std(값), torch.tensor(값).std())
print(np.std(값, ddof=1), torch.tensor(값).std(correction=0))   # 서로 맞추기
```

```
[4. 6.] tensor([4., 6.])
(4, 2) torch.Size([4, 2])
int64 torch.int64
[2. 4.]
torch.return_types.max(
values=tensor([2., 4.]),
indices=tensor([1, 1]))
1.4142135623730951 tensor(1.6330)
1.632993161855452 tensor(1.4142)
```

합계, 이어 붙이기, 자료형 바꾸기는 이름만 다르고 결과는 같습니다. `max`는 NumPy가 값만 주는 데 비해 PyTorch는 4.6절에서 본 것처럼 값과 위치를 묶어서 줘요. 여기까지는 다르게 쓰면 오류가 나거나 결과 모양이 달라서 금방 알아챕니다.

진짜 조심할 것은 마지막 두 줄입니다. 같은 숫자 네 개의 표준편차를 구했는데 NumPy는 1.414, PyTorch는 1.633을 냈어요. NumPy는 기본으로 "개수 n으로 나누는" 표준편차를, PyTorch는 "n - 1로 나누는" 표준편차(표본 표준편차)를 계산하기 때문입니다. 숫자가 적을수록 차이가 커져요. NumPy에서 `ddof=1`을 주거나 PyTorch에서 `correction=0`을 주면 서로 맞출 수 있습니다. 4.11절 실습의 표준화에서 쓴 `std`도 n - 1로 나눈 값이에요. 전처리는 NumPy로, 학습은 PyTorch로 하다가 두 결과가 미세하게 어긋난다면 이것부터 의심해 보세요.

하는 일	NumPy	PyTorch
접는 방향	`axis=0`	`dim=0` (`axis=`도 받아 줌)
이어 붙이기	`np.concatenate`	`torch.cat`
자료형 바꾸기	`a.astype(np.float32)`	`t.to(torch.float32)`, `t.float()`

하는 일	NumPy	PyTorch
복사	`a.copy()`	`t.clone()`
최댓값	`a.max(axis=1)` → 값만	`t.max(dim=1)` → 값과 위치
표준편차 기본	n으로 나눔 (`ddof=0`)	n - 1로 나눔 (`correction=1`)
차원 순서 바꾸기	`a.transpose(2, 0, 1)`	`t.permute(2, 0, 1)`
차원 끼워 넣기	`np.expand_dims(a, 0)`	`t.unsqueeze(0)`
씨앗 고정	`np.random.seed(42)`	`torch.manual_seed(42)`
기본 실수	`float64`	`float32`

마지막 줄도 다시 짚어 둡니다. 4.1절에서 본 것처럼 NumPy에서 넘어온 텐서는 `float64`라서, `float32`인 모델에 그대로 넣으면 자료형이 안 맞는다는 오류가 나요. 바로 다음 예제의 두 번째 오류가 그런 종류입니다.

오류 메시지 읽는 법

4.2절에서 딥러닝 오류의 절반쯤이 모양 때문이라고 했습니다. 오류는 피할 수 없으니, 오류 메시지를 **빨리 읽는 법**을 익히는 게 훨씬 남는 장사예요. 자주 만나는 네 가지를 일부러 일으켜 보겠습니다.

```
import torch


def 오류_보기(설명: str, 계산) -> None:
    """계산을 실행해 보고, 오류가 나면 첫 줄만 보여 준다."""
    try:
        계산()
    except RuntimeError as e:
        print(f"[{설명}] {str(e).splitlines()[0]}")


torch.manual_seed(0)
x = torch.randn(4, 3)            # 사진 4장 × 특성 3개
w = torch.randn(4, 2)            # 실수로 (특성 3개 → 2개)가 아닌 (4, 2)로 만듦
```

```
오류_보기("행렬 곱 모양", lambda: x @ w)
오류_보기("자료형 섞기", lambda: torch.tensor([[1, 2]]) @ torch.tensor([[0.5], [0.5]]))
오류_보기("숫자 하나가 아님", lambda: x.sum(dim=0).item())
오류_보기("쌓을 크기가 다름", lambda: torch.stack([torch.zeros(3), torch.zeros(4)]))

# 고친 뒤: 모양을 찍어 보며 확인
w = torch.randn(3, 2)
print(x.shape, "@", w.shape, "→", (x @ w).shape)
print((torch.tensor([[1, 2]]).float() @ torch.tensor([[0.5], [0.5]])).item())
print(x.sum().item())
```

```
[행렬 곱 모양] mat1 and mat2 shapes cannot be multiplied (4x3 and 4x2)
[자료형 섞기] expected m1 and m2 to have the same dtype, but got: long long != float
[숫자 하나가 아님] a Tensor with 3 elements cannot be converted to Scalar
[쌓을 크기가 다름] stack expects each tensor to be equal size, but got [3] at entry 0 and [4] at
↪ entry 1
torch.Size([4, 3]) @ torch.Size([3, 2]) → torch.Size([4, 2])
1.5
-3.141735553741455
```

`오류_보기`는 1권 12장의 `try/except`로 오류를 잡아 메시지의 첫 줄만 보여 주는 작은 함수입니다. 1권 10장의 `lambda`로 계산을 감싸 넘겨서, 오류가 함수 안에서 일어나게 했어요. 메시지는 영어지만 읽는 요령은 단순합니다. **숫자와 자료형 이름부터 보세요.**

- **`mat1 and mat2 shapes cannot be multiplied (4x3 and 4x2)`**: 행렬 곱의 두 모양이 `4x3`과 `4x2`라는 뜻입니다. 4.6절의 규칙 `(a, b) @ (b, c)`에서 가운데 두 숫자 3과 4가 같아야 하는데 다르다는 거예요. 가중치를 `(3, 2)`로 만들면 해결됩니다.
- **`expected m1 and m2 to have the same dtype, but got: long long != float`**: 정수(`long long`, 즉 `int64`)와 실수(`float`)를 행렬 곱하려 했습니다. 원소별 연산은 4.3절처럼 알아서 넓은 쪽으로 맞춰 주지만, 행렬 곱은 자료형이 같아야 해요. 한쪽을 `.float()`로 바꿉니다. NumPy에서 온 `float64`와 `float32`를 곱할 때도 같은 오류가 납니다.
- **`a Tensor with 3 elements cannot be converted to Scalar`**: `.item()`은 숫자가 **딱 하나**인 텐서만 파이썬 숫자로 바꿀 수 있습니다. `sum(dim=0)`으로 과목별 합계 3개가 남았는데 `.item()`을 불렀어요. 전부 더한 `sum()`이나, 여러 개면 `.tolist()`를 씁니다.
- **`stack expects each tensor to be equal size, but got [3] at entry 0 and [4] at`**

`entry 1`: `stack`으로 쌓을 텐서들은 모양이 모두 같아야 하는데, 0번째는 `[3]`, 1번째는 `[4]`라는 뜻입니다. 길이가 다른 문장들을 묶을 때 흔히 만나는데, 7장에서 짧은 쪽을 채워(padding) 길이를 맞추는 방법을 배웁니다.

고친 뒤의 세 줄처럼, 모양 오류가 나면 계산 바로 앞에서 **관련된 텐서의 `shape`를 모두 찍어 보는 것**이 가장 빠른 해결법입니다. `(4, 3) @ (3, 2) → (4, 2)`처럼 한 줄로 찍어 보면 어디가 어긋났는지 눈에 바로 들어와요. 4.2절에서 권한 "자리마다 이름 붙여 읽기"를 여기에 더하면, `x`의 3이 "특성 3개"이고 `w`의 첫 자리도 "특성 개수"여야 한다는 것까지 보입니다.

4.11 실습: 도시별 기후 패턴 비교하기

지금까지 배운 것을 모두 써서 작은 분석을 해 보겠습니다. 다섯 도시의 12개월 평균 기온 표가 있다고 해 봅시다(도시 이름과 숫자는 실제 관측값이 아닌 **예제용**입니다). 할 일은 여섯 가지예요.

1. 도시별 연평균과 연교차 구하기
2. 도시마다 가장 더운 달 찾기
3. 도시마다 기온을 표준화하기
4. 기후 **패턴**이 가장 닮은 도시 짝 찾기
5. 도시마다 가장 더운 석 달 찾기
6. 겨울이 얼마나 추운지 하나의 숫자(난방 지수)로 나타내기

```
import torch

도시 = ["가람시", "나래시", "다솜시", "라온시", "마루시"]
# 도시 5곳 × 12개월 월평균 기온(°C). 실제 관측값이 아닌 예제용 숫자입니다.
기온 = torch.tensor([
    [-2.0,  0.5,  6.0, 12.5, 18.0, 22.5, 25.5, 26.0, 21.5, 14.5,  7.0,  0.5],
    [ 3.5,  5.0,  9.0, 14.0, 18.5, 21.5, 25.0, 26.5, 23.0, 18.0, 12.0,  6.0],
    [-4.0, -1.5,  4.5, 11.5, 17.5, 22.0, 24.5, 25.0, 19.5, 12.5,  5.0, -1.5],
    [ 6.0,  7.0, 10.0, 14.5, 18.5, 21.5, 26.0, 27.0, 23.5, 19.0, 13.5,  8.5],
    [-1.0,  1.0,  6.5, 13.0, 18.5, 23.0, 25.5, 26.5, 21.0, 14.0,  7.5,  1.0],
])
print("모양:", 기온.shape)

# 1) 도시별 연평균과 연교차(가장 더운 달 - 가장 추운 달)
연평균 = 기온.mean(dim=1)
```

```
연교차 = 기온.max(dim=1).values - 기온.min(dim=1).values
for 이름, 평균, 교차 in zip(도시, 연평균, 연교차):
    print(f"{이름}: 연평균 {평균:5.2f}°C, 연교차 {교차:4.1f}°C")

# 2) 도시마다 가장 더운 달 (argmax는 0부터 세므로 +1)
print("가장 더운 달:", (기온.argmax(dim=1) + 1).tolist())

# 3) 도시마다 표준화: (기온 - 그 도시 평균) / 그 도시 표준편차
평균 = 기온.mean(dim=1, keepdim=True)        # (5, 1)
표준편차 = 기온.std(dim=1, keepdim=True)     # (5, 1)
표준화 = (기온 - 평균) / 표준편차            # (5, 12) - (5, 1) → 브로드캐스팅
print("표준화 후 도시별 평균:", 표준화.mean(dim=1).abs().max() < 1e-6)

# 4) 기후 패턴이 가장 닮은 도시 찾기: 코사인 유사도 = 단위 벡터끼리의 행렬곱
단위 = 표준화 / 표준화.norm(dim=1, keepdim=True)
유사도 = 단위 @ 단위.T                       # (5, 12) @ (12, 5) → (5, 5)
유사도.fill_diagonal_(-1.0)                  # 자기 자신은 빼기
짝 = 유사도.argmax(dim=1)
for i, j in enumerate(짝.tolist()):
    print(f"{도시[i]}와 가장 닮은 도시: {도시[j]} (유사도 {유사도[i, j]:.4f})")

# 5) 도시마다 가장 더운 석 달 (topk는 큰 순서로 주므로 달 순서로 다시 정렬)
더운_석달 = 기온.topk(3, dim=1).indices.sort(dim=1).values + 1
for 이름, 달 in zip(도시, 더운_석달.tolist()):
    print(f"{이름}의 가장 더운 석 달: {달}월")

# 6) 난방 지수: 10도보다 추운 만큼을 달마다 더한 값 (예제용 지표)
난방 = torch.where(기온 < 10, 10 - 기온, 0.0)
print("난방이 필요한 달 수:", (기온 < 10).sum(dim=1).tolist())
print("난방 지수:", 난방.sum(dim=1).tolist())
print("clamp로 계산해도 같은가:", torch.equal(난방, (10 - 기온).clamp(min=0)))
```

```
모양: torch.Size([5, 12])
가람시: 연평균 12.71°C, 연교차 28.0°C
나래시: 연평균 15.17°C, 연교차 23.0°C
다솜시: 연평균 11.25°C, 연교차 29.0°C
```

```
라온시: 연평균 16.25°C, 연교차 21.0°C
마루시: 연평균 13.04°C, 연교차 27.5°C
가장 더운 달: [8, 8, 8, 8, 8]
표준화 후 도시별 평균: tensor(True)
가람시와 가장 닮은 도시: 마루시 (유사도 0.9992)
나래시와 가장 닮은 도시: 라온시 (유사도 0.9974)
다솜시와 가장 닮은 도시: 마루시 (유사도 0.9996)
라온시와 가장 닮은 도시: 나래시 (유사도 0.9974)
마루시와 가장 닮은 도시: 다솜시 (유사도 0.9996)
가람시의 가장 더운 석 달: [6, 7, 8]월
나래시의 가장 더운 석 달: [7, 8, 9]월
다솜시의 가장 더운 석 달: [6, 7, 8]월
라온시의 가장 더운 석 달: [7, 8, 9]월
마루시의 가장 더운 석 달: [6, 7, 8]월
난방이 필요한 달 수: [5, 4, 5, 3, 5]
난방 지수: [38.0, 16.5, 47.5, 8.5, 35.0]
clamp로 계산해도 같은가: True
```

한 줄씩 짚어 보겠습니다.

- **연평균과 연교차**: `mean(dim=1)`로 달 방향(12개월)을 접어 도시별 평균을 냈습니다. 연교차는 `max(dim=1).values`에서 `min(dim=1).values`를 뺀 값이에요. 두 결과 모두 모양이 `(5,)`라서 `zip`으로 도시 이름과 나란히 출력할 수 있었습니다. 텐서의 원소 하나는 `f"{평균:5.2f}"`처럼 1권의 f-string 서식을 그대로 쓸 수 있어요.
- **가장 더운 달**: `argmax(dim=1)`은 0부터 세므로 1을 더해 "몇 월"로 바꿨습니다. `.tolist()`는 텐서를 1권의 리스트로 돌려줍니다.
- **표준화**: 도시마다 평균과 표준편차가 다르니 각자의 기준으로 바꿔야 합니다. `keepdim=True`로 `(5, 1)`을 만든 덕분에 `(5, 12) - (5, 1)`이 브로드캐스팅되어 도시마다 자기 평균을 뺐어요. 결과를 확인하는 줄은 "표준화 후 도시별 평균의 절댓값 중 가장 큰 것이 0에 아주 가까운가"를 물어 `True`를 받았습니다. 실수 계산은 정확히 0이 되지 않을 수 있어서(4.3절) `== 0` 대신 `< 1e-6`으로 비교했어요.
- **닮은 도시**: 표준화한 12개월 기온을 도시마다 12차원 화살표로 보고, 화살표 방향이 얼마나 비슷한지(**코사인 유사도**)를 쟀습니다. 각 화살표를 길이 1로 맞춘 뒤(`norm`) 행렬 곱 `단위 @ 단위.T`를 하면 `(5, 12) @ (12, 5) → (5, 5)`, 모든 도시 쌍의 유사도가 한 번에 나와요. 자기 자신과의 유사도(대각선)는 언제나 1이라 `fill_diagonal_(-1.0)`로 지운 뒤 `argmax`로 짝

을 찾았습니다.

결과를 보면 겨울이 영하로 내려가는 가람시·다솜시·마루시가 서로 닮았고, 겨울에도 따뜻한 나래시·라온시가 짝이 됐습니다. 표준화를 했기 때문에 "얼마나 따뜻한가"가 아니라 "1년 동안 어떤 **모양**으로 오르내리는가"를 비교한 결과예요. 이렇게 숫자 묶음을 화살표로 보고 방향을 비교하는 생각은 13장에서 단어의 뜻을 비교할 때(임베딩) 그대로 다시 등장합니다.

이어지는 두 단계는 이 장 중간에 새로 배운 도구를 씁니다.

- **가장 더운 석 달**: `topk(3, dim=1)`은 도시마다 기온이 높은 순서로 세 달의 번호를 줍니다. 그런데 순서가 "가장 더운 달부터"라서 `[8, 7, 6]`처럼 거꾸로 나와요. 달 순서로 보고 싶어서 번호를 다시 `sort(dim=1)`로 정렬하고 1을 더했습니다. `.indices`, `.values`처럼 결과를 이어서 꺼내 쓸 수 있다는 것도 눈여겨보세요. 결과를 보면 겨울이 추운 세 도시는 6~8월, 겨울이 따뜻한 나래시·라온시는 7~9월이 가장 덥습니다. 따뜻한 도시는 더위가 한 달 늦게까지 이어진다는 뜻이고, 앞의 닮은 도시 짝과도 맞아떨어져요.
- **난방 지수**: 10도보다 추운 달마다 "10도에서 얼마나 모자란가"를 더한 값입니다(이 장에서 정한 예제용 지표예요). `torch.where(기온 < 10, 10 - 기온, 0.0)`으로 추운 달은 모자란 만큼, 나머지 달은 0을 골랐고, 이를 `sum(dim=1)`로 도시마다 더했습니다. 난방이 필요한 달 수는 참/거짓 마스크를 `sum`해서 셌어요. 참은 1, 거짓은 0으로 더해지기 때문입니다. 다솜시가 47.5로 가장 춥고 라온시가 8.5로 가장 따뜻하네요. 마지막 줄은 같은 계산을 `(10 - 기온).clamp(min=0)`으로 해도 똑같다는 확인입니다. "모자란 만큼, 단 음수는 0"이라는 말을 그대로 옮긴 셈이라 이쪽이 더 짧아요.

4.12 정리와 다음 장 미리보기

이 장에서 텐서라는 그릇을 손에 익혔습니다.

- **만들기**: `torch.tensor`, `zeros`/`ones`/`full`, `arange`/`linspace`, `rand`/`randn`과 `manual_seed`, 나만 쓰는 `torch.Generator`, NumPy와 주고받기(`tensor`는 복사, `from_numpy`는 공유)
- **읽기**: `shape`, `ndim`, `numel()`, `dtype`. 모양은 자리마다 이름을 붙여 읽기. 메모리 크기는 `element_size() × numel()`
- **자르기**: 리스트와 같은 인덱싱·슬라이싱, 마스크, 슬라이스는 창(view)이라는 점, 조건으로 바꾸는 `where`/`clamp`, 번호표로 꺼내고 써 넣는 `gather`/`scatter`
- **바꾸기**: `reshape`/`view`, `permute`/`.T`, `unsqueeze`/`squeeze`, `cat`/`stack`, `flatten`. 메모리는 한 줄이고 보폭(stride)으로 표처럼 읽는다는 점과 `contiguous()`
- **계산하기**: 원소별 연산, 브로드캐스팅(오른쪽부터, 같거나 1이면 통과), 행렬 곱 `(a, b) @ (b,`

`c) → (a, c)`와 글자로 쓰는 `einsum`, `dim`을 따라 접는 집계와 `keepdim`, `sort`/`topk`

- **옮기기**: `장치_고르기()`와 `.to(장치)`, 같은 장치끼리만 계산, GPU 시간은 준비 운동과 동기화 후에 재기
- **저장하기**: `torch.save`/`torch.load`, 여러 개는 딕셔너리로, 슬라이스는 `clone()` 후 저장, 다른 장치의 파일은 `map_location`
- **오류 읽기**: NumPy와 다른 점(`dim`, 표준편차 기본값, 기본 실수), 오류 메시지의 숫자와 자료형부터 보고 `shape` 찍어 보기

다음 5장에서는 텐서의 두 번째 능력인 **자동 미분**을 만납니다. 텐서에 `requires_grad=True`라는 표시 하나를 붙이면, 그 텐서가 거쳐 간 계산을 PyTorch가 모두 기억했다가 "정답에 가까워지려면 어느 쪽으로 얼마나 고쳐야 하는지"를 거꾸로 계산해 줘요. 먼저 그 계산을 손으로 해 보고, PyTorch가 같은 답을 내는지 확인하는 것으로 시작하겠습니다.

05

자동 미분과 경사하강법

학습 목표

- 기울기(미분)를 "조금 움직였을 때 얼마나 변하는가"로 이해하고 수치 미분으로 직접 재기
- 경사하강법으로 함수의 가장 낮은 곳과 선형 회귀의 답을 손으로 찾기
- `requires_grad`, `backward()`, `.grad`로 PyTorch가 같은 기울기를 내는지 확인하기
- 계산 그래프와 연쇄 법칙으로 기울기가 거꾸로 흐르는 원리를 설명하기
- 기울기 누적, `no_grad`·`detach`·`inference_mode`, 제자리 연산 충돌 같은 함정을 피하기
- 학습률과 미니배치가 학습에 미치는 영향을 실제 손실 값으로 확인하기

들어가며: 안개 낀 산에서 내려오려면?

짙은 안개가 낀 산 중턱에 혼자 서 있다고 해 봅시다. 앞이 한 치도 보이지 않으니 지도도, 저 아래 마을도 보이지 않아요. 그런데 꼭 골짜기 바닥까지 내려가야 합니다. 어떻게 하면 될까요?

방법이 하나 있습니다. 발끝으로 주변 땅을 더듬어 보고, **가장 많이 내리막인 쪽으로 한 걸음** 내딛는 거예요. 거기서 다시 더듬고, 또 한 걸음. 이걸 반복하면 산 전체를 보지 못해도 결국 바닥에 닿습니다. 이 단순한 방법이 이 장의 주인공인 **경사하강법**(gradient descent)이고, 오늘날 거의 모든 딥러닝 모델이 이 방법으로 학습합니다.

그럼 "산"은 뭐고 "바닥"은 뭘까요? 작은 예로 시작해 보겠습니다. 아이스크림 가게 사장님이 30일 동안 그날 기온과 판매량을 적어 두었어요. 기온을 보고 판매량을 예측하는 직선 `판매량 = w × 기`

온 + b를 찾고 싶습니다. 1장에서 본 **모델**이 바로 이 직선이고, w와 b는 모델이 정해야 할 숫자예요. 아무 직선이나 몇 개 그어 보고, 실제 판매량과 얼마나 어긋나는지 재 보겠습니다.

```
import numpy as np

# 30일 동안의 기온(°C)과 아이스크림 판매량(개). 예제용으로 만든 데이터입니다.
rng = np.random.default_rng(0)
기온 = rng.uniform(18, 34, size=30).round(1)
판매량 = (2.5 * 기온 - 15 + rng.normal(0, 4, size=30)).round()
print("기온  :", 기온[:5])
print("판매량:", 판매량[:5])


def 평균제곱오차(w: float, b: float) -> float:
    """직선 '판매량 = w × 기온 + b'가 실제 판매량에서 평균적으로 얼마나 벗어나는지."""
    예측 = w * 기온 + b
    return float(((예측 - 판매량) ** 2).mean())


for w, b in [(2.0, 0.0), (2.0, -5.0), (2.5, -10.0), (3.0, -20.0)]:
    print(f"w={w}, b={b:6.1f} → 오차 {평균제곱오차(w, b):7.2f}")
```

```
기온  : [28.2 22.3 18.7 18.3 31. ]
판매량: [51. 40. 31. 33. 63.]
w=2.0, b=   0.0 → 오차   20.76
w=2.0, b=  -5.0 → 오차   39.62
w=2.5, b= -10.0 → 오차   29.22
w=3.0, b= -20.0 → 오차   72.05
```

어긋난 정도는 **평균제곱오차**로 쟀습니다. 날마다 "예측 - 실제"를 제곱해서 평균을 낸 값이에요. 제곱을 하니 많이 틀린 날은 크게 벌을 받고, 음수와 양수가 서로 지워지지도 않습니다. 이렇게 "모델이 얼마나 틀렸는가"를 숫자 하나로 나타낸 것을 **손실(loss)** 이라고 불러요.

네 직선 중에서는 w=2.0, b=0.0이 가장 덜 틀렸습니다. 그런데 이게 최선일까요? w를 조금 키우고 b를 조금 줄이면 더 좋아질까요, 나빠질까요? w=2.5, b=-10.0은 오히려 나빠졌죠. 이렇게 찍어서는 끝이 없습니다. 숫자가 둘이 아니라 수백만 개인 딥러닝 모델이라면 더 말할 것도 없고요.

여기서 산 비유가 다시 등장합니다. w와 b를 바꿀 때마다 손실이 오르내리니, (w, b)를 땅 위의 위치

로, 손실을 그 자리의 **높이**로 보면 손실은 울퉁불퉁한 산이 됩니다. 가장 좋은 직선을 찾는 일은 이 산의 **가장 낮은 곳**을 찾는 일이에요. 그리고 안개 속에서 내리막을 알려 주는 발끝 감각이 바로 **기울기(미분)** 입니다.

4장 끝에서 약속한 대로, 이 장은 기울기를 **손으로** 재고 계산하는 데서 출발합니다. 그다음 PyTorch에게 같은 일을 맡겨서 **똑같은 답**이 나오는지 확인하고, 마지막에는 실제 의료 데이터로 회귀 모델을 처음부터 학습시켜 봅니다.

Q

Q. 미분을 몰라도 이 장을 따라갈 수 있나요? 네, 따라갈 수 있습니다. 이 장에서는 미분을 공식보다 "조금 움직여 보고 얼마나 변하는지 재기"로 설명하고, 공식이 나오면 반드시 직접 잰 값과 맞춰 봅니다. 고등학교 때 미분을 배웠다면 "아, 그게 이거였구나" 하고 다시 만나는 기분일 거예요. 그리고 좋은 소식은, 이 장의 뒷부분부터는 미분을 PyTorch가 대신 해 준다는 점입니다.

5.1 기울기: 조금 움직였을 때 얼마나 변하나

등산로의 경사

등산로 안내판에 "경사 10%"라고 적혀 있으면 무슨 뜻일까요? 앞으로 100m 걸어가면 높이가 10m 올라간다는 뜻입니다. 즉 **경사 = 높이 변화 ÷ 걸어간 거리**예요. 수학의 기울기도 똑같습니다. 함수 `f(x)`에서 `x`를 아주 조금(`h`만큼) 움직였을 때 `f(x)`가 얼마나 변하는지를 움직인 거리로 나눈 값이죠.

그런데 산길은 곳곳마다 경사가 다릅니다. 그래서 "지금 서 있는 바로 이 자리의 경사"를 알려면 아주 조금만 움직여 봐야 해요. 많이 움직이면 그 사이의 오르막 내리막이 섞여 버리니까요. `f(x) = x²`의 `x = 3` 자리에서 `h`를 점점 줄여 가며 재 보겠습니다.

```
def f(x: float) -> float:
    return x ** 2


x = 3.0
for h in [1.0, 0.1, 0.01, 0.001, 0.0001]:
    기울기 = (f(x + h) - f(x)) / h       # 조금(h) 움직였을 때 변한 양 ÷ 움직인 거리
    print(f"h={h:<7} 기울기 ≈ {기울기:.6f}")

print()
```

```
for h in [1e-8, 1e-10, 1e-12, 1e-14]:
    print(f"h={h:<7} 기울기 ≈ {(f(x + h) - f(x)) / h:.6f}")   # 너무 작게 움직이면?
```

```
h=1.0     기울기 ≈ 7.000000
h=0.1     기울기 ≈ 6.100000
h=0.01    기울기 ≈ 6.010000
h=0.001   기울기 ≈ 6.001000
h=0.0001  기울기 ≈ 6.000100

h=1e-08   기울기 ≈ 6.000000
h=1e-10   기울기 ≈ 6.000000
h=1e-12   기울기 ≈ 6.000533
h=1e-14   기울기 ≈ 6.039613
```

h를 1에서 0.0001까지 줄이니 잰 기울기가 7, 6.1, 6.01, 6.001, 6.0001로 **6에 점점 다가갑니다.** 이 "다가가는 값" 6이 x = 3에서 x^2의 기울기, 다른 말로 **미분값**이에요. 이렇게 조금 움직여 보고 재는 방법을 **수치 미분**이라고 합니다.

그렇다면 h를 끝없이 줄이면 더 정확해질까요? 아래 네 줄을 보세요. 1e-10까지는 6.000000이 잘 나오다가 1e-12, 1e-14로 가니 오히려 6.0005, 6.04로 **틀어집니다.** 4.3절에서 실수는 정해진 칸에 근사값으로 담긴다고 했죠. 아주 비슷한 두 수 f(x + h)와 f(x)를 빼면 앞자리가 모두 지워지고 근사 오차만 남는데, 그걸 아주 작은 h로 나누니 오차가 크게 부풀려지는 거예요. 수치 미분은 이렇게 **너무 크게 움직여도, 너무 작게 움직여도** 틀립니다.

양쪽으로 움직여 보기

조금 더 영리한 방법이 있습니다. 한쪽으로만 움직이지 말고 **앞뒤로 똑같이** 움직여 보는 거예요. 앞으로 갔을 때와 뒤로 갔을 때의 높이 차이를 재면 한쪽으로 치우친 오차가 서로 상쇄됩니다.

```
def f(x: float) -> float:
    return x ** 2


def 수치_미분(함수, x: float, h: float = 1e-5) -> float:
    """x 양쪽으로 h만큼 움직여 본 높이 차이로 기울기를 잰다(중앙 차분)."""
    return (함수(x + h) - 함수(x - h)) / (2 * h)
```

```
print(f"한쪽만 움직이기 (h=0.01): {(f(3.0 + 0.01) - f(3.0)) / 0.01:.6f}")
print(f"양쪽 움직이기   (h=0.01): {수치_미분(f, 3.0, h=0.01):.6f}")
print()
for x in [-2.0, 0.0, 1.0, 3.0, 5.0]:
    print(f"x={x:4}: 잰 기울기 {수치_미분(f, x):8.4f}, 공식 2x = {2 * x:5.1f}")
```

```
한쪽만 움직이기 (h=0.01): 6.010000
양쪽 움직이기   (h=0.01): 6.000000

x=-2.0: 잰 기울기  -4.0000, 공식 2x =  -4.0
x= 0.0: 잰 기울기   0.0000, 공식 2x =   0.0
x= 1.0: 잰 기울기   2.0000, 공식 2x =   2.0
x= 3.0: 잰 기울기   6.0000, 공식 2x =   6.0
x= 5.0: 잰 기울기  10.0000, 공식 2x =  10.0
```

같은 `h=0.01`인데 한쪽만 움직이면 6.01, 양쪽으로 움직이면 6.000000이 나왔습니다. 이 방법을 **중앙 차분**이라고 하고, 앞으로 이 장에서 "직접 잰 기울기"는 모두 이 방법으로 구합니다. 1권 10장에서 함수를 다른 함수의 인자로 넘길 수 있다고 했죠? `수치_미분(f, x)`처럼 함수 자체를 넘기니 어떤 함수든 기울기를 잴 수 있어요.

아래 다섯 줄은 여러 자리에서 잰 기울기입니다. 잰 값이 언제나 `2x`와 같네요. 그래서 "`x²`의 미분은 `2x`"라는 **공식**이 있는 겁니다. 공식은 매번 재지 않고도 기울기를 바로 알려 주는 지름길이에요. 이 장에서 쓸 공식은 몇 개뿐입니다.

함수	기울기(미분)	말로 풀면
`c` (상수)	`0`	움직여도 변하지 않는다
`a × x`	`a`	x가 1 움직이면 a배만큼 변한다
`x²`	`2x`	멀리 갈수록 가팔라진다
`x^n`	`n × x^(n-1)`	위 규칙을 일반화한 것
`f(x) + g(x)`	`f의 기울기 + g의 기울기`	더한 것의 기울기는 기울기의 합

기울기의 **부호**도 중요합니다. `x = -2`에서는 -4, `x = 3`에서는 6이었죠. 양수면 오른쪽으로 갈수록 올라가는 오르막, 음수면 오른쪽으로 갈수록 내려가는 내리막이에요. `x = 0`에서는 0, 평평한 바닥입니다. 기울기의 크기는 얼마나 가파른지, 부호는 어느 쪽이 오르막인지를 알려 줘요.

여러 방향의 기울기

들어가며의 손실 산은 `w`와 `b` 두 방향으로 펼쳐져 있었습니다. 이렇게 방향이 여럿이면 기울기는 어떻게 잴까요? 간단합니다. **한 방향씩만** 움직여 보면 돼요. 동서로 한 발 움직여 보고, 제자리로 돌아와서 남북으로 한 발 움직여 보는 거죠. 이렇게 다른 방향은 그대로 두고 한 방향으로만 잰 기울기를 **편미분**이라고 부르고, 방향마다 잰 기울기를 모은 묶음을 **그래디언트(gradient)** 라고 합니다. PyTorch에서 `grad`라는 이름을 계속 보게 될 텐데, 바로 이 그래디언트의 줄임말이에요.

```
def 높이(x: float, y: float) -> float:
    """두 방향(동서 x, 남북 y)으로 펼쳐진 언덕의 높이."""
    return x ** 2 + 3 * y ** 2


h = 1e-5
x, y = 1.0, 2.0
동서_기울기 = (높이(x + h, y) - 높이(x - h, y)) / (2 * h)   # y는 그대로 두고 x만 움직이기
남북_기울기 = (높이(x, y + h) - 높이(x, y - h)) / (2 * h)   # x는 그대로 두고 y만 움직이기
print(f"동서 방향 기울기: {동서_기울기:.4f}  (공식 2x = {2 * x})")
print(f"남북 방향 기울기: {남북_기울기:.4f}  (공식 6y = {6 * y})")

# 기울기 반대 방향으로 아주 조금 걸어 보면 높이가 내려가는지 확인
걸음 = 0.01
print(f"지금 높이: {높이(x, y):.4f}")
print(f"기울기 반대로 한 걸음: {높이(x - 걸음 * 동서_기울기, y - 걸음 * 남북_기울기):.4f}")
print(f"기울기 방향으로 한 걸음: {높이(x + 걸음 * 동서_기울기, y + 걸음 * 남북_기울기):.4f}")
```

```
동서 방향 기울기: 2.0000  (공식 2x = 2.0)
남북 방향 기울기: 12.0000  (공식 6y = 12.0)
지금 높이: 13.0000
기울기 반대로 한 걸음: 11.5636
기울기 방향으로 한 걸음: 14.5236
```

`(1, 2)` 자리에서 동서 방향 기울기는 2, 남북 방향 기울기는 12가 나왔습니다. 공식 `2x`, `6y`와 같아

요. 이 두 숫자를 묶은 `(2, 12)`가 이 자리의 그래디언트입니다.

마지막 세 줄이 경사하강법의 핵심입니다. 그래디언트는 **가장 가파른 오르막 방향**을 가리켜요. 그래서 그 방향으로 걸으면 높이가 13에서 14.52로 올라가고, **반대 방향**으로 걸으면 11.56으로 내려갑니다. 안개 속에서 발끝으로 더듬던 감각을 숫자로 바꾼 거예요. 내려가고 싶으면 그래디언트의 반대쪽으로 걸으면 됩니다.

5.2 손으로 해 보는 경사하강법

한 걸음씩 바닥으로

이제 실제로 내려가 보겠습니다. 함수 `f(x) = (x - 3)² + 1`은 `x = 3`에서 가장 낮은 값 1을 가집니다. 우리는 답을 알지만, 모르는 척하고 `x = -2`에서 출발해 볼게요. 규칙은 단 한 줄입니다.

> 새 위치 = 지금 위치 - 학습률 × 지금 자리의 기울기

학습률(learning rate)은 한 걸음을 얼마나 크게 내디딜지 정하는 숫자예요. 기울기가 가파르면 크게, 완만하면 작게 움직이게 됩니다.

```
def f(x: float) -> float:
    return (x - 3) ** 2 + 1          # x=3일 때 가장 낮은 1


def 기울기(x: float) -> float:
    return 2 * (x - 3)               # f를 미분한 공식


x = -2.0                             # 아무 데서나 출발
학습률 = 0.1                          # 한 번에 얼마나 크게 걸을지
for 걸음 in range(1, 31):
    x = x - 학습률 * 기울기(x)         # 경사하강법의 전부: 기울기 반대로 조금 이동
    if 걸음 in (1, 2, 3, 5, 10, 20, 30):
        print(f"{걸음:2}걸음: x = {x:.4f}, f(x) = {f(x):.6f}")
```

```
 1걸음: x = -1.0000, f(x) = 17.000000
 2걸음: x = -0.2000, f(x) = 11.240000
 3걸음: x = 0.4400, f(x) = 7.553600
```

```
 5걸음: x = 1.3616, f(x) = 3.684355
10걸음: x = 2.4631, f(x) = 1.288230
20걸음: x = 2.9424, f(x) = 1.003323
30걸음: x = 2.9938, f(x) = 1.000038
```

첫걸음에 -2에서 -1로, 다음에 -0.2로, 그다음 0.44로 3을 향해 움직이고, 30걸음 뒤에는 2.9938, 거의 3에 도착했습니다. `f(x)`도 17에서 1.000038까지 내려왔어요.

걸음 폭을 보세요. 처음에는 한 번에 1 가까이 움직이다가 3에 가까워질수록 걸음이 점점 작아집니다. 학습률은 0.1로 그대로인데 왜 그럴까요? **바닥에 가까울수록 기울기가 작아지기** 때문이에요. 바닥에서는 기울기가 0이니, 내려갈수록 저절로 조심조심 걷게 됩니다. 바닥을 지나치지 않게 해 주는 고마운 성질이에요.

공식이 없어도 내려갈 수 있습니다

위에서는 기울기 공식 `2(x - 3)`을 손으로 구해서 넣었습니다. 그런데 생각해 보면 경사하강법이 필요로 하는 건 **그 자리의 기울기 값**뿐이에요. 공식이 없어도 5.1절의 수치 미분으로 재면 됩니다. 이번에는 골짜기가 두 개인 함수에서 출발점만 바꿔 가며 내려가 보겠습니다.

```python
def f(x: float) -> float:
    return x ** 4 - 3 * x ** 2 + x   # 골짜기가 두 개인 함수


def 수치_미분(함수, x: float, h: float = 1e-5) -> float:
    return (함수(x + h) - 함수(x - h)) / (2 * h)


for 출발 in [2.0, -0.5, -2.0]:
    x = 출발
    for _ in range(200):
        x = x - 0.01 * 수치_미분(f, x)   # 공식 없이, 잰 기울기만으로 내려가기
    print(f"출발 {출발:4}: 도착 x = {x:7.4f}, f(x) = {f(x):8.4f}")
```

```
출발  2.0: 도착 x =  1.1309, f(x) =  -1.0702
출발 -0.5: 도착 x = -1.3008, f(x) =  -3.5139
출발 -2.0: 도착 x = -1.3008, f(x) =  -3.5139
```

공식 없이, 잰 기울기만으로 모두 바닥에 도착했습니다. 그런데 도착한 곳이 다릅니다. -0.5와 -2.0에서 출발하면 `x = -1.3008`(높이 -3.5139)에 도착하지만, 2.0에서 출발하면 `x = 1.1309`(높이 -1.0702)에 멈춰요. 오른쪽 골짜기는 왼쪽보다 얕은데도, 그 안에 들어간 순간 사방이 오르막이라 더 내려갈 곳이 없습니다.

이렇게 주변보다는 낮지만 전체에서 가장 낮지는 않은 곳을 **지역 최솟값**(local minimum), 진짜 가장 낮은 곳을 **전역 최솟값**(global minimum) 이라고 합니다. 안개 속 등산객은 자기가 선 골짜기가 산 전체에서 가장 낮은 곳인지 알 길이 없어요. 경사하강법의 근본적인 한계입니다.

다행히 이 장에서 다루는 선형 회귀의 손실은 골짜기가 **딱 하나**인 그릇 모양이라 이런 걱정이 없습니다. 골짜기가 여러 개인 신경망에서는 무작위 출발점, 미니배치(5.11절)가 주는 흔들림 같은 것들이 얕은 골짜기에서 빠져나오는 데 도움이 되는데, 실제로는 생각보다 큰 문제가 되지 않는다는 것이 경험적으로 알려져 있어요.

5.3 선형 회귀를 손으로 학습하기

손실을 w와 b로 미분하기

이제 들어가며의 아이스크림 문제로 돌아가겠습니다. 손실 산은 `w`와 `b` 두 방향이니, 두 방향의 기울기가 필요해요. 손실은 이렇습니다.

> 손실 = 평균($(w \times x + b - y)^2$)

여기서 `x`는 기온, `y`는 실제 판매량이고, `w × x + b − y`는 날마다의 **오차**입니다. 5.1절의 표와 다음 절에서 볼 연쇄 법칙으로 미분하면 두 방향의 기울기는 이렇게 나와요.

> w 방향 기울기 = 평균(2 × 오차 × x)
> b 방향 기울기 = 평균(2 × 오차)

식이 낯설면 뜻으로 읽어 보세요. 오차가 양수(너무 많이 예측)인 날이 많으면 `b` 방향 기울기가 양수가 되고, 경사하강법은 그 반대로 가니 `b`를 줄입니다. 너무 많이 예측했으니 줄이는 거죠. `w` 방향은 오차에 `x`가 곱해져 있어서, 기온이 높은 날의 오차일수록 `w`를 고치는 데 크게 반영됩니다. 상식과 맞죠?

하나 손볼 것이 있습니다. 기온은 18~34 사이의 큰 숫자라서 그대로 쓰면 `w` 방향과 `b` 방향의 기울기 크기가 크게 달라져 학습이 까다로워져요(5.10절에서 직접 봅니다). 그래서 2장에서 배운 **표준화**로 기온을 평균 0, 표준편차 1로 바꿔서 쓰겠습니다. 공식이 맞는지부터 수치 미분으로 확인해 볼게요.

```
import numpy as np

rng = np.random.default_rng(0)
기온 = rng.uniform(18, 34, size=30).round(1)
판매량 = (2.5 * 기온 - 15 + rng.normal(0, 4, size=30)).round()
x = (기온 - 기온.mean()) / 기온.std()          # 표준화: 평균 0, 표준편차 1
y = 판매량


def 손실(w: float, b: float) -> float:
    return float(((w * x + b - y) ** 2).mean())


def 손_기울기(w: float, b: float) -> tuple[float, float]:
    """손으로 미분한 공식: 오차 = 예측 - 정답."""
    오차 = w * x + b - y
    return float((2 * 오차 * x).mean()), float((2 * 오차).mean())


w, b = 0.0, 0.0
h = 1e-5
잰_w = (손실(w + h, b) - 손실(w - h, b)) / (2 * h)
잰_b = (손실(w, b + h) - 손실(w, b - h)) / (2 * h)
공식_w, 공식_b = 손_기울기(w, b)
print(f"w 방향: 수치 미분 {잰_w:.4f}, 공식 {공식_w:.4f}")
print(f"b 방향: 수치 미분 {잰_b:.4f}, 공식 {공식_b:.4f}")
```

```
w 방향: 수치 미분 -25.3119, 공식 -25.3119
b 방향: 수치 미분 -105.0000, 공식 -105.0000
```

`w = 0, b = 0`에서 공식으로 계산한 기울기와 직접 잰 기울기가 소수점 넷째 자리까지 똑같습니다. 이렇게 미분 공식을 수치 미분과 맞춰 보는 것을 **기울기 검사(gradient check)** 라고 해요. 공식을 손으로 유도할 때 실수하기 쉬워서, 예전에는 새 모델을 만들 때마다 꼭 하던 절차였습니다.

둘 다 음수죠? 지금 `w`와 `b`가 너무 작다는 뜻이에요(직선이 0에 붙어 있으니 판매량을 한참 적게 예측하고 있습니다). 그러니 기울기의 반대, 즉 `w`와 `b`를 **키우는** 쪽으로 가야 합니다.

경사하강법으로 직선 찾기

공식을 확인했으니 이제 경사하강법을 돌려 봅시다. 5.2절의 한 줄 규칙을 `w`와 `b`에 똑같이 적용하면 됩니다.

```
import numpy as np

rng = np.random.default_rng(0)
기온 = rng.uniform(18, 34, size=30).round(1)
판매량 = (2.5 * 기온 - 15 + rng.normal(0, 4, size=30)).round()
x = (기온 - 기온.mean()) / 기온.std()
y = 판매량

w, b = 0.0, 0.0
학습률 = 0.1
for 반복 in range(1, 101):
    오차 = w * x + b - y
    손실 = (오차 ** 2).mean()
    기울기_w = (2 * 오차 * x).mean()
    기울기_b = (2 * 오차).mean()
    w -= 학습률 * 기울기_w
    b -= 학습률 * 기울기_b
    if 반복 in (1, 5, 10, 20, 50, 100):
        print(f"{반복:3}번째: 손실 {손실:8.3f}, w = {w:.4f}, b = {b:.4f}")

정답_w, 정답_b = np.polyfit(x, y, deg=1)      # 한 번에 푸는 공식(최소제곱법)
print(f"polyfit 정답: w = {정답_w:.4f}, b = {정답_b:.4f}")
print(f"기온 1°C당 판매량 변화: {w / 기온.std():.3f}개")
```

```
  1번째: 손실 2930.500, w = 2.5312, b = 10.5000
  5번째: 손실  503.372, w = 8.5088, b = 35.2968
 10번째: 손실   66.615, w = 11.2970, b = 46.8629
 20번째: 손실   14.683, w = 12.5100, b = 51.8947
 50번째: 손실   14.077, w = 12.6558, b = 52.4993
100번째: 손실   14.077, w = 12.6559, b = 52.5000
polyfit 정답: w = 12.6559, b = 52.5000
기온 1°C당 판매량 변화: 2.495개
```

손실이 2930.5에서 시작해 10번째에 66.6, 20번째에 14.7로 줄고, 50번째부터는 14.077에서 더 내려가지 않습니다. 바닥에 도착한 거예요. 이때 `w = 12.6559`, `b = 52.5000`입니다.

마지막 두 줄이 이 결과가 맞다는 증거입니다. NumPy의 `polyfit`은 직선을 **한 번에 계산하는 공식**(최소제곱법)으로 답을 구하는데, 경사하강법이 100번 걸어서 찾은 답과 소수점 넷째 자리까지 같아요. 표준화를 되돌려 해석하면 "기온이 1°C 오를 때마다 약 2.495개 더 팔린다"는 뜻이고, 이 데이터를 만들 때 정한 2.5와 거의 같습니다. `b = 52.5`는 표준화한 기온이 0일 때, 즉 **평균 기온인 날**의 판매량이에요.

> **Q. 공식으로 한 번에 풀 수 있는데 왜 경사하강법을 쓰나요?** 선형 회귀처럼 단순한 모델은 공식이 있어서 그게 더 빠르고 정확합니다. 3장에서 쓴 scikit-learn의 `LinearRegression`도 공식으로 풀어요. 하지만 6장부터 만들 신경망은 그런 공식이 **아예 없습니다.** 기울기만 잴 수 있으면 어떤 모델이든 쓸 수 있다는 것, 그게 경사하강법의 힘이에요. 여기서 선형 회귀를 쓰는 건 정답을 알고 있어서 경사하강법이 제대로 동작하는지 확인하기 좋기 때문입니다.

5.4 PyTorch에게 미분 맡기기

requires_grad, backward, grad

5.3절에서 기울기 공식을 손으로 유도했습니다. 숫자 두 개짜리 모델이라 할 만했지만, 숫자가 수백만 개이고 계산이 수십 단계로 얽힌 신경망이라면 어떨까요? 손으로는 불가능합니다. 그래서 PyTorch가 대신 해 줘요. 4장 들어가며에서 말한 텐서의 두 번째 능력, **자동 미분**(autograd) 입니다. 가장 간단한 예부터 보겠습니다.

```
import torch

x = torch.tensor(3.0, requires_grad=True)   # "이 값에 대한 기울기가 필요해요" 표시
y = x ** 2                                  # 평소처럼 계산
print(y)

y.backward()                                # y를 x로 미분해 줘
print("PyTorch가 구한 기울기:", x.grad)
print("손으로 구한 기울기 2x:", 2 * 3.0)
```

```
h = 1e-3
print("수치 미분:", ((3.0 + h) ** 2 - (3.0 - h) ** 2) / (2 * h))
```

```
tensor(9., grad_fn=<PowBackward0>)
PyTorch가 구한 기울기: tensor(6.)
손으로 구한 기울기 2x: 6.0
수치 미분: 5.999999999999339
```

세 단계만 기억하면 됩니다.

1. **표시하기**: 기울기가 필요한 텐서를 만들 때 `requires_grad=True`를 붙입니다. "이 텐서가 거쳐 가는 계산을 기억해 줘"라는 뜻이에요.
2. **계산하기**: 평소처럼 계산합니다. 결과 `y`에 붙은 `grad_fn=<PowBackward0>`가 "거듭제곱으로 만들어졌다"는 기억이에요.
3. **거꾸로 계산하기**: `y.backward()`를 부르면 PyTorch가 기억을 거슬러 올라가며 기울기를 구해 `x.grad`에 넣어 둡니다.

`x.grad`가 6이죠. 손으로 구한 `2x = 6`, 수치 미분 5.999999...와 같습니다. 수치 미분은 근사값이라 끝자리가 조금 틀렸지만, 자동 미분은 공식을 쓴 것처럼 **정확한 값**을 줍니다. 이게 수치 미분과의 큰 차이예요. 자동 미분은 조금 움직여 보는 게 아니라, 각 계산 단계의 미분 공식(5.1절의 표 같은 것)을 PyTorch가 이미 다 알고 있어서 그걸 이어 붙여 계산합니다.

손으로 구한 기울기와 똑같을까요?

진짜 시험은 5.3절의 선형 회귀입니다. 같은 데이터, 같은 `w = 0, b = 0`에서 손으로 구한 기울기와 PyTorch의 기울기를 나란히 놓아 보겠습니다. 비교를 공정하게 하려고 PyTorch 쪽도 NumPy와 같은 64비트 실수를 쓰도록 `torch.from_numpy`로 가져왔어요(4.1절).

```
import numpy as np
import torch

rng = np.random.default_rng(0)
기온 = rng.uniform(18, 34, size=30).round(1)
판매량 = (2.5 * 기온 - 15 + rng.normal(0, 4, size=30)).round()
x_np = (기온 - 기온.mean()) / 기온.std()
y_np = 판매량
```

```
# 손으로 구한 기울기 (NumPy, w=0, b=0에서)
오차 = 0.0 * x_np + 0.0 - y_np
손_w, 손_b = (2 * 오차 * x_np).mean(), (2 * 오차).mean()

# PyTorch에게 맡긴 기울기
x = torch.from_numpy(x_np)                  # float64 그대로 (4.1절)
y = torch.from_numpy(y_np)
w = torch.tensor(0.0, dtype=torch.float64, requires_grad=True)
b = torch.tensor(0.0, dtype=torch.float64, requires_grad=True)
손실 = ((w * x + b - y) ** 2).mean()
손실.backward()

print(f"손으로  : w 기울기 {손_w:.10f}, b 기울기 {손_b:.10f}")
print(f"PyTorch : w 기울기 {w.grad.item():.10f}, b 기울기 {b.grad.item():.10f}")
print("같은가?", torch.allclose(w.grad, torch.tensor(손_w, dtype=torch.float64)),
      torch.allclose(b.grad, torch.tensor(손_b, dtype=torch.float64)))
```

```
손으로  : w 기울기 -25.3118789700, b 기울기 -105.0000000000
PyTorch : w 기울기 -25.3118789700, b 기울기 -105.0000000000
같은가? True True
```

소수점 열째 자리까지 똑같습니다. PyTorch 쪽 코드를 보면 손실을 계산하는 한 줄과 `backward()` 한 줄뿐, 기울기 공식은 어디에도 없어요. 그런데도 우리가 5.3절에서 공들여 유도한 공식과 같은 답을 냈습니다. `torch.allclose`는 두 텐서가 아주 작은 오차 안에서 같은지 확인하는 함수예요(실수는 정확히 같기 어려우니 `==` 대신 씁니다).

알아 둘 규칙 세 가지

`backward()`를 쓰다 보면 처음에 꼭 한 번씩 만나는 오류가 있습니다. 미리 만나 두죠.

```
import torch

x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = x ** 2                          # 결과가 숫자 3개인 벡터
try:
```

```
    y.backward()
except RuntimeError as e:
    print("오류 1:", e)

y.sum().backward()              # 하나로 모으면 된다
print("x.grad:", x.grad)        # 각 칸의 기울기 2x

try:
    torch.tensor([1, 2, 3], requires_grad=True)
except RuntimeError as e:
    print("오류 2:", e)

print("x는 잎인가?", x.is_leaf, "/ y는 잎인가?", y.is_leaf)
print("y.grad:", y.grad)        # 중간 결과의 기울기는 보관하지 않는다
```

```
오류 1: grad can be implicitly created only for scalar outputs
x.grad: tensor([2., 4., 6.])
오류 2: Only Tensors of floating point and complex dtype can require gradients
x는 잎인가? True / y는 잎인가? False
y.grad: None
```

- **`backward()`는 숫자 하나(스칼라)에서 부릅니다.** "기울기는 스칼라 출력에 대해서만 암묵적으로 만들 수 있다"는 오류가 났죠. 경사하강법은 언제나 **손실이라는 숫자 하나**를 줄이는 일이라, 결과가 여러 개면 `sum()`이나 `mean()`으로 하나로 모은 뒤 부릅니다. 덕분에 `x.grad`에 칸마다 기울기 `2x`(2, 4, 6)가 들어왔어요.
- **실수(또는 복소수) 텐서만 기울기를 가질 수 있습니다.** 정수는 "조금 움직이기"가 안 되니 당연하죠. 4.1절에서 본 것처럼 소수점 없는 숫자로 만들면 정수 텐서가 되니, 여기서 걸리기 쉬워요. `torch.tensor([1, 2, 3])` 대신 `[1.0, 2.0, 3.0]`으로 만드세요.
- **기울기는 잎(leaf) 텐서에만 남습니다.** 우리가 직접 만든 `x`는 잎이고, 계산으로 만들어진 `y`는 잎이 아니에요. `backward()`가 끝나면 `x.grad`에는 값이 남지만 `y.grad`는 `None`입니다(직접 실행하면 "잎이 아닌 텐서의 `.grad`에 접근하고 있다"는 경고도 함께 나와요. 5.6절에서 볼 `retain_grad()`를 쓰라는 안내입니다). 중간 결과의 기울기까지 다 보관하면 메모리가 부족해지기 때문이에요. 모델에서 우리가 고치고 싶은 건 잎인 `w`와 `b`뿐이니 그걸로 충분합니다.

5.5 계산 그래프: PyTorch는 무엇을 기억하나

grad_fn을 따라가 보기

PyTorch는 계산을 "기억한다"고 했는데, 정확히 무엇을 어떻게 기억할까요? 데이터 한 개짜리 선형 회귀 손실로 그 기억을 직접 꺼내 보겠습니다. `grad_fn`에는 `next_functions`라는 속성이 있어서, "나를 만든 계산의 재료는 누가 만들었나"를 따라 거슬러 올라갈 수 있어요. 1권 9장의 재귀 함수로 끝까지 따라가 봅시다.

```
import torch

w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
x = torch.tensor(3.0)                        # 데이터: 기울기가 필요 없다
y = torch.tensor(10.0)

예측 = w * x + b
손실 = (예측 - y) ** 2
print("손실:", 손실)


def 그래프_따라가기(노드, 깊이: int = 0) -> None:
    """grad_fn에서 출발해 계산을 거꾸로 거슬러 올라가며 출력한다(1권 9장의 재귀)."""
    if 노드 is None:
        return
    이름 = type(노드).__name__
    if 이름 == "AccumulateGrad":             # 잎(requires_grad=True인 텐서)에 도착
        이름 += f"  ← 잎 텐서 값 {노드.variable.item()}"
    print("    " * 깊이 + 이름)
    for 다음, _ in 노드.next_functions:
        그래프_따라가기(다음, 깊이 + 1)


그래프_따라가기(손실.grad_fn)
```

```
손실: tensor(9., grad_fn=<PowBackward0>)
PowBackward0
    SubBackward0
        AddBackward0
            MulBackward0
                AccumulateGrad  ← 잎 텐서 값 2.0
            AccumulateGrad  ← 잎 텐서 값 1.0
```

출력을 아래에서 위로 읽으면 우리가 쓴 계산 순서와 같습니다.

- `MulBackward0`: `w * x` (곱하기). 재료 중 `w`는 잎이라 `AccumulateGrad`로 이어지고, `x`는 `requires_grad`가 없어서 기억하지 않았어요.
- `AddBackward0`: `... + b` (더하기). `b`도 잎입니다.
- `SubBackward0`: `예측 - y` (빼기).
- `PowBackward0`: `(...) ** 2` (제곱). 손실이 여기서 만들어졌어요.

이렇게 계산 하나하나를 점으로, 재료와 결과 관계를 선으로 이은 것을 **계산 그래프**(computational graph) 라고 합니다. `backward()`는 이 그래프의 맨 위(손실)에서 출발해 아래로 내려가며 각 점의 미분 공식을 적용하고, 잎에 도착하면 `AccumulateGrad`가 그 값을 `.grad`에 **쌓아**(accumulate) 둡니다. 이름에 "쌓는다"가 들어간 건 우연이 아니에요. 5.7절에서 이 이름 때문에 생기는 함정을 봅니다.

`x`와 `y`(데이터)는 `requires_grad`가 없으니 그래프에 기울기를 받을 자리가 없습니다. 데이터는 고칠 대상이 아니니까요. 고칠 대상인 `w`, `b`에만 표시를 붙인다는 것이 모델 학습의 기본 모양입니다.

그래프는 계산할 때마다 새로 만들어집니다

PyTorch의 계산 그래프는 코드가 **실행되는 그 순간** 만들어집니다. 그래서 평범한 파이썬 `if`나 `for`를 그대로 써도 자동 미분이 잘 동작해요. 100을 넘게 쓰면 넘은 만큼은 단가가 두 배가 되는 누진 요금으로 확인해 보겠습니다.

```
import torch


def 요금(사용량: torch.Tensor) -> torch.Tensor:
    """100 넘게 쓰면 넘은 만큼은 단가가 두 배가 되는 누진 요금."""
    if 사용량 > 100:                          # 평범한 파이썬 if
        return 100 * 1.0 + (사용량 - 100) * 2.0
```

```
    return 사용량 * 1.0


for 값 in [80.0, 130.0]:
    u = torch.tensor(값, requires_grad=True)
    요금(u).backward()
    print(f"사용량 {값}: 요금 기울기 {u.grad.item()}")

u = torch.tensor(130.0, requires_grad=True)
결과 = 요금(u)
결과.backward()
try:
    결과.backward()                          # 같은 그래프로 한 번 더?
except RuntimeError as e:
    print("오류:", str(e).split(".")[0])
```

```
사용량 80.0: 요금 기울기 1.0
사용량 130.0: 요금 기울기 2.0
오류: Trying to backward through the graph a second time (or directly access saved tensors after
↪ they have already been freed)
```

사용량이 80일 때는 `if`의 아래쪽(단가 1)이 실행되어 기울기가 1, 130일 때는 위쪽(넘은 만큼 단가 2)이 실행되어 기울기가 2입니다. 실제로 지나간 길만 그래프에 기록되니, 어느 쪽 길로 갔느냐에 따라 기울기가 달라지는 거예요. 이런 방식을 **실행하면서 정의(define-by-run)** 한다고 하는데, PyTorch가 연구자들에게 사랑받은 가장 큰 이유입니다. 11장의 RNN처럼 입력 길이에 따라 반복 횟수가 달라지는 모델도 평범한 `for` 문으로 쓸 수 있어요.

마지막 오류도 보세요. 같은 결과에서 `backward()`를 두 번 부르니 "그래프를 두 번째로 거꾸로 지나가려 한다"는 오류가 났습니다. PyTorch는 `backward()`가 끝나면 메모리를 아끼려고 그래프에 저장해 둔 중간값을 **바로 버립니다.** 그래프는 한 번 쓰고 버리는 일회용이에요. 그래서 학습 반복마다 손실을 **새로 계산**해서 새 그래프를 만든 뒤 `backward()`를 부르는 겁니다. 꼭 두 번 써야 한다면 `backward(retain_graph=True)`로 남겨 둘 수 있지만, 그럴 일은 드뭅니다.

5.6 연쇄 법칙: 기울기가 거꾸로 흐르는 이유

톱니바퀴 세 개

backward()가 그래프를 거꾸로 따라가며 무엇을 하는지 좀 더 들여다보겠습니다. 톱니바퀴 세 개가 맞물려 있다고 해 봅시다. A를 한 바퀴 돌리면 B가 3바퀴 돌고, B를 한 바퀴 돌리면 C가 2바퀴 돕니다. 그럼 A를 한 바퀴 돌리면 C는 몇 바퀴 돌까요? 3 × 2 = 6바퀴죠.

이게 **연쇄 법칙(chain rule)** 의 전부입니다. 계산이 여러 단계로 이어져 있으면, 처음에서 끝까지의 기울기는 **단계마다의 기울기를 곱한 값**이에요. y = (3x + 1)²을 두 단계로 나눠서 확인해 보겠습니다. 중간값을 u = 3x + 1이라고 하면 y = u²입니다.

```
import torch

# y = (3x + 1)^2 를 두 단계로 나눠 보기: u = 3x + 1, y = u^2
x값 = 2.0
u값 = 3 * x값 + 1
du_dx = 3.0                # u = 3x + 1 에서 x가 1 움직이면 u는 3 움직인다
dy_du = 2 * u값            # y = u^2 에서 u가 1 움직이면 y는 2u 움직인다
print(f"연쇄 법칙(손): dy/dx = {dy_du} × {du_dx} = {dy_du * du_dx}")

x = torch.tensor(x값, requires_grad=True)
y = (3 * x + 1) ** 2
y.backward()
print(f"PyTorch      : dy/dx = {x.grad.item()}")

h = 1e-4
f = lambda t: (3 * t + 1) ** 2
print(f"수치 미분    : dy/dx = {(f(x값 + h) - f(x값 - h)) / (2 * h):.4f}")
```

```
연쇄 법칙(손): dy/dx = 14.0 × 3.0 = 42.0
PyTorch      : dy/dx = 42.0
수치 미분    : dy/dx = 42.0000
```

x = 2에서 u = 7입니다. x가 1 움직이면 u는 3 움직이고(첫째 톱니), u가 1 움직이면 y는 2u = 14 움직여요(둘째 톱니). 둘을 곱하면 42이고, PyTorch도 수치 미분도 42입니다. (3x + 1)²을 전개해서 미분하는 방법도 있지만, 연쇄 법칙을 쓰면 **각 단계의 간단한 미분만 알면** 아무리 긴 계산이라도 기울기를 구할 수 있어요.

역전파를 손으로 따라 하기

이제 5.5절의 계산 그래프와 연쇄 법칙을 합쳐 보겠습니다. 데이터 한 개짜리 선형 회귀 손실을 단계마다 이름을 붙여 계산하고, 끝에서부터 거꾸로 기울기를 곱해 나가 보죠. 중간 결과는 원래 `.grad`가 남지 않지만(5.4절), `retain_grad()`를 부르면 PyTorch가 남겨 줘서 단계마다 비교할 수 있습니다.

```
import torch

x, y = torch.tensor(3.0), torch.tensor(10.0)         # 데이터 한 개
w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)

# 앞으로 계산(forward): 중간값을 모두 이름 붙여 둔다
예측 = w * x + b              # 7
오차 = 예측 - y               # -3
손실 = 오차 ** 2              # 9
예측.retain_grad()            # 중간 결과의 기울기도 보관해 달라고 부탁
오차.retain_grad()
손실.backward()

# 거꾸로 계산(backward)을 손으로: 끝에서부터 '지역 기울기'를 곱해 나간다
d손실_d오차 = 2 * 오차.item()                # 손실 = 오차^2
d손실_d예측 = d손실_d오차 * 1                # 오차 = 예측 - y
d손실_dw = d손실_d예측 * x.item()            # 예측 = w*x + b
d손실_db = d손실_d예측 * 1

for 이름, 손, 파이토치 in [("오차", d손실_d오차, 오차.grad), ("예측", d손실_d예측, 예측.grad),
                          ("w", d손실_dw, w.grad), ("b", d손실_db, b.grad)]:
    print(f"d손실/d{이름}: 손으로 {손:6.1f}, PyTorch {파이토치.item():6.1f}")
```

```
d손실/d오차: 손으로   -6.0, PyTorch   -6.0
d손실/d예측: 손으로   -6.0, PyTorch   -6.0
d손실/dw: 손으로  -18.0, PyTorch  -18.0
d손실/db: 손으로   -6.0, PyTorch   -6.0
```

손으로 한 계산을 순서대로 읽어 보세요.

1. 손실 = 오차²이니 손실을 오차로 미분하면 2 × 오차 = 2 × (-3) = -6.

2. `오차 = 예측 - y`이니 오차를 예측으로 미분하면 1. 연쇄 법칙으로 곱하면 `-6 × 1 = -6`.
3. `예측 = w × x + b`이니 예측을 `w`로 미분하면 `x = 3`, `b`로 미분하면 1. 곱하면 `w` 쪽은 `-6 × 3 = -18`, `b` 쪽은 `-6 × 1 = -6`.

네 단계 모두 PyTorch와 같습니다. 이렇게 손실에서 출발해 **거꾸로** 단계마다 지역 기울기를 곱하며 내려가는 방법을 **역전파(backpropagation)** 라고 해요. `backward()`라는 이름이 여기서 왔습니다.

왜 굳이 거꾸로 갈까요? 앞에서부터 가면 `w`에 대해 한 번, `b`에 대해 한 번, 고칠 숫자마다 따로 계산해야 합니다. 거꾸로 가면 "손실을 예측으로 미분한 값 -6"처럼 **공통 부분을 한 번만 계산**해서 모든 갈래에 나눠 줄 수 있어요. 고칠 숫자가 수백만 개인 신경망에서도 `backward()` 한 번이면 전부의 기울기가 나오는 비결입니다. 대신 거꾸로 갈 때 앞에서 계산한 중간값(여기서는 `오차`, `x`)이 필요해서, PyTorch는 앞으로 계산하는 동안 그 값들을 그래프에 저장해 둡니다. 이 "저장해 둔 중간값"이 5.9절 제자리 연산 문제의 열쇠예요.

5.7 기울기가 쌓이는 함정과 zero_grad

backward를 부를 때마다 더해집니다

5.5절에서 잎에 도착한 기울기는 `AccumulateGrad`가 `.grad`에 **쌓는다**고 했습니다. "넣는다"가 아니라 "쌓는다"예요. 무슨 차이인지 직접 보겠습니다.

```
import torch

x = torch.tensor(3.0, requires_grad=True)
for 번째 in range(1, 4):
    y = x ** 2
    y.backward()
    print(f"{번째}번째 backward 뒤 x.grad = {x.grad.item()}")   # 6, 12, 18 ... 쌓인다

x.grad.zero_()                                  # 비우기
(x ** 2).backward()
print("비운 뒤 다시:", x.grad.item())
```

```
1번째 backward 뒤 x.grad = 6.0
2번째 backward 뒤 x.grad = 12.0
3번째 backward 뒤 x.grad = 18.0
```

```
비운 뒤 다시: 6.0
```

`x = 3`에서 x^2의 기울기는 언제나 6인데, `backward()`를 세 번 부르니 6, 12, 18로 **불어났습니다.** `backward()`는 새로 구한 기울기를 `.grad`에 덮어쓰지 않고 **더하기** 때문이에요. 다시 6을 얻으려면 `x.grad.zero_()`로 먼저 비워야 합니다(4.7절의 밑줄 함수, 제자리에서 0으로 만들어요).

PyTorch가 왜 이렇게 만들었을까요? 일부러입니다. 한 텐서가 계산 그래프의 여러 갈래에서 쓰이면, 갈래마다 오는 기울기를 **모두 더해야** 올바른 기울기가 되거든요(`y = x * x`에서 `x`가 두 번 쓰이는 것처럼). 또 GPU 메모리가 부족해 큰 배치를 한 번에 못 넣을 때, 작은 배치 여러 개의 기울기를 쌓아서 큰 배치처럼 쓰는 기법(그래디언트 누적, 17장)도 이 성질 덕분에 가능해요.

비우지 않으면 학습이 망가집니다

학습할 때 비우는 것을 깜빡하면 어떻게 될까요? 5.3절의 아이스크림 회귀를 PyTorch로 바꿔서, 매번 비우는 경우와 비우지 않는 경우를 비교해 보겠습니다. 값을 고치는 부분의 `with torch.no_grad():`는 5.8절에서 설명할 테니 지금은 "고치는 계산은 기록하지 말라"는 뜻으로만 봐 주세요.

```
import numpy as np
import torch

rng = np.random.default_rng(0)
기온 = rng.uniform(18, 34, size=30).round(1)
판매량 = (2.5 * 기온 - 15 + rng.normal(0, 4, size=30)).round()
x = torch.tensor((기온 - 기온.mean()) / 기온.std(), dtype=torch.float32)
y = torch.tensor(판매량, dtype=torch.float32)


def 학습(비우기: bool) -> list[float]:
    w = torch.zeros((), requires_grad=True)
    b = torch.zeros((), requires_grad=True)
    기록 = []
    for _ in range(30):
        손실 = ((w * x + b - y) ** 2).mean()
        손실.backward()
        with torch.no_grad():                # 값 고치기는 기록하지 않는다(5.8절)
```

```
            w -= 0.1 * w.grad
            b -= 0.1 * b.grad
        if 비우기:
            w.grad.zero_()
            b.grad.zero_()
        기록.append(손실.item())
    return 기록


for 비우기 in (True, False):
    기록 = 학습(비우기)
    print(f"grad 비우기={비우기!s:5}:", [round(v, 1) for v in 기록[::5]], "→", round(기록[-1], 1))
```

```
grad 비우기=True : [2930.5, 327.2, 47.7, 17.7, 14.5, 14.1] → 14.1
grad 비우기=False: [2930.5, 1927.2, 15.8, 1786.5, 2987.0, 734.2] → 1691.5
```

매번 비운 쪽은 5.3절의 NumPy와 똑같이 2930.5에서 14.1로 내려갑니다. 비우지 않은 쪽은 15.8까지 내려갔다가 1786.5, 2987.0으로 다시 치솟고, 30번 뒤에도 1691.5로 출렁여요. 이전 반복의 기울기가 계속 쌓이니 바닥을 지나쳐도 예전 기울기에 떠밀려 한참 더 가 버리는 겁니다. 오류 메시지 하나 없이 **조용히 학습이 망가진다**는 점이 이 함정의 무서운 점이에요.

그래서 학습 반복은 언제나 이 순서를 지킵니다.

1. 손실 계산 (새 그래프 만들기)
2. `backward()` (기울기 구하기)
3. 값 고치기 (`no_grad` 안에서)
4. 기울기 비우기

6장에서 옵티마이저를 쓰면 3번과 4번이 `optimizer.step()`과 `optimizer.zero_grad()` 두 줄이 되는데, 그 `zero_grad`가 바로 이 4번입니다. 비우는 시점을 2번 앞(반복의 맨 처음)에 두는 코드도 많은데, `backward()` 전에만 비우면 결과는 같아요.

5.8 기록 끄기: no_grad, detach, inference_mode

값을 고칠 때는 기록을 끕니다

5.7절에서 `w`와 `b`를 고칠 때 `with torch.no_grad():` 안에서 했습니다. 그냥 `w -= 학습률 * w.grad`라고 쓰면 어떻게 될까요?

```
import torch

w = torch.tensor(2.0, requires_grad=True)
손실 = (w * 3.0 - 10.0) ** 2
손실.backward()

try:
    w -= 0.01 * w.grad                    # 기록을 켠 채로 잎 텐서를 고치면?
except RuntimeError as e:
    print("오류:", e)

w2 = w - 0.01 * w.grad                    # 새 텐서를 만들면 오류는 없지만...
print("w2:", w2, "/ 잎인가?", w2.is_leaf)

with torch.no_grad():                     # 올바른 방법
    w -= 0.01 * w.grad
print("w:", w, "/ 잎인가?", w.is_leaf)
```

```
오류: a leaf Variable that requires grad is being used in an in-place operation.
w2: tensor(2.2400, grad_fn=<SubBackward0>) / 잎인가? False
w: tensor(2.2400, requires_grad=True) / 잎인가? True
```

"기울기가 필요한 잎 변수가 제자리 연산에 쓰이고 있다"는 오류가 났습니다. `-=`는 4.7절의 제자리 연산인데, PyTorch 입장에서 `w`는 계산 그래프의 출발점(잎)이에요. 출발점을 제자리에서 바꾸는 계산까지 기록하라고 하면 그래프가 엉망이 되니 막는 겁니다.

그럼 `w2 = w - 0.01 * w.grad`처럼 새 텐서를 만들면 어떨까요? 오류는 없지만 `w2`는 `grad_fn=<SubBackward0>`이 붙은 **계산 결과**라서 잎이 아닙니다. 다음 반복에서 `backward()`를 불러도 `w2.grad`에는 기울기가 남지 않아요(5.4절). 고치는 계산이 매번 그래프에 덧붙어서 그래프가 끝없이 길어지는 문제도 생깁니다.

올바른 방법은 마지막처럼 `torch.no_grad()` 안에서 고치는 것입니다. "이 안의 계산은 기록하지 마"라는 뜻이라, `w`는 잎인 채로 값만 2.24로 바뀌었어요. **값을 고치는 일은 학습의 일부이지 모델 계산**

의 일부가 아니라서 기록할 필요가 없는 겁니다.

기록을 끄는 세 가지 방법

기록을 끄고 싶은 경우는 값 고치기 말고도 많습니다. 학습이 끝난 모델로 예측만 할 때, 손실 값을 기록용으로 꺼낼 때, 텐서를 NumPy로 넘길 때 같은 경우예요. PyTorch는 세 가지 도구를 줍니다.

```
import torch

w = torch.tensor([1.0, 2.0], requires_grad=True)

with torch.no_grad():                          # 이 안의 계산은 기록하지 않는다
    a = w * 2
print("no_grad       :", a.requires_grad, a.grad_fn)

with torch.inference_mode():                   # 예측만 할 때: no_grad보다 더 가볍다
    b = w * 2
print("inference_mode:", b.requires_grad, "/ 추론 전용 텐서인가?", b.is_inference())

c = w.detach()                                 # 기록에서 떼어 낸 '창'(4.4절)
print("detach        :", c.requires_grad, "/ 메모리 공유:", c.data_ptr() == w.data_ptr())
c[0] = 100.0                                   # 떼어 낸 쪽을 바꾸면?
print("w도 바뀌었다  :", w)

try:
    w.numpy()
except RuntimeError as e:
    print("오류:", e)
print("detach 후 numpy:", w.detach().numpy())
```

```
no_grad       : False None
inference_mode: False / 추론 전용 텐서인가? True
detach        : False / 메모리 공유: True
w도 바뀌었다  : tensor([100.,   2.], requires_grad=True)
오류: Can't call numpy() on Tensor that requires grad. Use tensor.detach().numpy() instead.
detach 후 numpy: [100.   2.]
```

- `torch.no_grad()`: `with` 블록 안의 모든 계산을 기록하지 않습니다. 결과 `a`는 re-

quires_grad가 False이고 grad_fn도 None이에요. 그래프를 만들지 않으니 메모리와 시간이 절약됩니다. 1권 11장에서 배운 컨텍스트 매니저라, 블록을 나가면 기록이 다시 켜져요.

- **torch.inference_mode()**: no_grad보다 한 걸음 더 나아가 "이 텐서는 앞으로도 절대 기울기 계산에 안 쓴다"고 약속하는 방법입니다. 그만큼 내부 기록을 더 줄여서 조금 더 빠르지만, 여기서 만든 텐서(is_inference()가 True)는 나중에 자동 미분 계산에 섞을 수 없어요. 학습이 끝난 모델로 **예측만** 하는 곳(20장 서빙 등)에서 씁니다.
- **텐서.detach()**: 블록 전체가 아니라 **텐서 하나**를 기록에서 떼어 냅니다. 출력을 보면 w.detach()는 w와 같은 메모리를 쓰는 창(4.4절)이라, 떼어 낸 c의 첫 칸을 100으로 바꾸니 w도 바뀌었어요. 4.7절 표에 한 줄 더 보태야겠네요. 따로 쓰고 싶다면 w.detach().clone()입니다.

마지막 오류도 자주 만납니다. 기울기를 추적 중인 텐서는 .numpy()로 바로 바꿀 수 없어요. 오류 메시지가 해결책까지 친절하게 알려 주네요. detach().numpy()를 쓰면 됩니다. 손실 값 하나만 파이썬 숫자로 꺼낼 때는 지금까지 쓴 .item()이 가장 간단하고요.

방법	범위	주로 쓰는 곳
with torch.no_grad():	블록 안 전체	파라미터 고치기, 검증 데이터로 평가하기
with torch.inference_mode():	블록 안 전체	학습이 끝난 모델로 예측만 하기
텐서.detach()	텐서 하나	NumPy로 넘기기, 기록하거나 그래프 그리기, 그래프 일부만 끊기

Q

Q. 다른 코드에서 w.data를 고치는 걸 봤는데, 그것도 되나요? .data는 detach()가 생기기 전의 옛 방식으로, 기록 없이 값에 직접 접근합니다. 동작은 하지만 자동 미분이 모르는 사이에 값이 바뀌어도 오류로 알려 주지 않아서, 5.9절에서 볼 보호 장치가 꺼져 버려요. 새로 쓰는 코드에서는 no_grad와 detach()를 쓰세요.

5.9 제자리 연산과 자동 미분의 충돌

4.7절에서 “제자리 연산을 자동 미분과 섞어 쓰면 오류를 만나기 쉽다”고 했습니다. 이제 그 이유를 이해할 준비가 됐어요. 5.6절 끝에서, PyTorch는 거꾸로 계산할 때 쓰려고 중간값을 그래프에 저장해 둔다고 했습니다. 그런데 그 중간값을 제자리 연산으로 **바꿔 버리면** 어떻게 될까요?

```
import torch

a = torch.tensor([0.0, 1.0, 2.0], requires_grad=True)
b = a.exp()                    # exp는 기울기 계산에 '자기 결과'를 다시 쓴다
print("b의 버전:", b._version)
b.add_(1)                      # 결과를 제자리에서 바꿔 버리면?
print("b의 버전:", b._version)
try:
    b.sum().backward()
except RuntimeError as e:
    print("오류:", str(e).split(" Hint")[0])

a = torch.tensor([0.0, 1.0, 2.0], requires_grad=True)
b = a.exp()
c = b + 1                      # 밑줄 없는 연산: 새 텐서를 만든다
c.sum().backward()
print("밑줄 없이:", a.grad)

a = torch.tensor([0.0, 1.0, 2.0], requires_grad=True)
b = a * 3                      # 곱하기 3의 기울기는 b의 값과 상관없다
b.add_(1)
b.sum().backward()
print("상관없는 경우:", a.grad)
```

```
b의 버전: 0
b의 버전: 1
오류: one of the variables needed for gradient computation has been modified by an inplace
↪ operation: [torch.FloatTensor [3]], which is output 0 of Exp, is at version 1; expected version
↪ 0 instead.
밑줄 없이: tensor([1.0000, 2.7183, 7.3891])
```

```
상관없는 경우: tensor([3., 3., 3.])
```

`exp`(지수 함수)는 미분해도 자기 자신이라는 특별한 성질이 있어서, PyTorch는 기울기를 계산할 때 **결과 `b`를 다시 씁니다.** 그래서 `b`를 그래프에 저장해 두는데, `b.add_(1)`로 그 값을 바꿔 버렸어요. 그대로 계산하면 엉뚱한 기울기가 나올 테니, PyTorch는 `backward()`에서 "기울기 계산에 필요한 변수가 제자리 연산으로 바뀌었다"는 오류를 냅니다.

PyTorch가 이걸 알아채는 방법은 `_version`이라는 **수정 횟수 표시**예요. 텐서를 제자리에서 바꿀 때마다 1씩 올라가는데, 그래프에 저장할 때 0이었던 버전이 `backward()` 때 1이 되어 있으니 바뀐 걸 아는 거죠. 오류 메시지 끝의 "버전 1인데 버전 0이어야 한다"는 말이 바로 그 이야기입니다.

해결은 간단합니다. 두 번째 예제처럼 밑줄 없는 연산(`b + 1`)으로 새 텐서를 만들면 저장된 `b`는 그대로라 문제없어요. 기울기도 `e^0, e^1, e^2` 즉 1, 2.7183, 7.3891로 정확합니다.

세 번째 예제는 반대로 제자리 연산을 해도 괜찮은 경우입니다. `a * 3`의 기울기는 언제나 3이라 결과 `b`의 값이 필요 없거든요. 저장된 값을 건드리지 않았으니 오류가 나지 않습니다. 하지만 어떤 연산이 무엇을 저장하는지 일일이 외울 수는 없죠. 그래서 4장의 권고가 여기서도 그대로입니다. **기울기를 추적하는 텐서에는 제자리 연산을 쓰지 마세요.** 메모리를 아끼려는 제자리 연산은 대부분 기록을 끈 곳(5.8절)에서만 씁니다. 파라미터를 고치는 `w -= ...`이 `no_grad` 안에서만 허용되는 것처럼요.

5.10 학습률: 너무 크면 튀고, 너무 작으면 늦다

걸음 폭에 따라 달라지는 길

경사하강법에서 우리가 정해야 하는 숫자가 하나 있었습니다. 학습률이에요. 지금까지는 0.1을 썼는데, 다른 값을 쓰면 어떻게 될까요? 가장 단순한 그릇 모양 $f(x) = x^2$에서 $x = 5$로 출발해 학습률만 바꿔 보겠습니다.

```
# f(x) = x^2 (가장 낮은 곳은 x=0), 기울기는 2x, x=5에서 출발
for 학습률 in [0.01, 0.1, 0.5, 0.9, 1.1]:
    x = 5.0
    자취 = []
    for 걸음 in range(1, 21):
        x = x - 학습률 * 2 * x
        if 걸음 in (1, 2, 3, 10, 20):
            자취.append(f"{x:9.3f}")
```

```
    print(f"학습률 {학습률:<4}: 1, 2, 3, 10, 20걸음 뒤 x =", " ".join(자취))
```

```
학습률 0.01: 1, 2, 3, 10, 20걸음 뒤 x =     4.900     4.802     4.706     4.085     3.338
학습률 0.1 : 1, 2, 3, 10, 20걸음 뒤 x =     4.000     3.200     2.560     0.537     0.058
학습률 0.5 : 1, 2, 3, 10, 20걸음 뒤 x =     0.000     0.000     0.000     0.000     0.000
학습률 0.9 : 1, 2, 3, 10, 20걸음 뒤 x =    -4.000     3.200    -2.560     0.537     0.058
학습률 1.1 : 1, 2, 3, 10, 20걸음 뒤 x =    -6.000     7.200    -8.640    30.959   191.688
```

다섯 줄이 모두 다른 이야기를 합니다.

- **0.01 (너무 작음)**: 20걸음 걸어도 5에서 3.338까지밖에 못 왔습니다. 방향은 맞는데 너무 느려요.
- **0.1 (적당함)**: 20걸음 뒤 0.058, 바닥에 거의 도착했습니다.
- **0.5 (딱 맞음)**: 한 걸음에 정확히 0입니다. 이 함수에서만 생기는 우연한 경우예요. 기울기 2x에 0.5를 곱하면 정확히 x만큼 움직이거든요.
- **0.9 (큼)**: -4.0, 3.2, -2.56으로 바닥을 사이에 두고 **좌우로 튀면서** 다가갑니다. 걸음이 커서 바닥을 지나쳐 반대편 벽에 닿는 거예요. 거리만 보면 0.1과 똑같이 줄고 있죠.
- **1.1 (너무 큼)**: -6, 7.2, -8.64로 튈 때마다 **더 멀어지다가** 20걸음 뒤에는 191.688이 됐습니다. 바닥을 지나쳐서 처음보다 더 높은 반대편 벽에 떨어지고, 거기서 더 가파른 기울기로 더 멀리 튕겨 나가는 거예요. 이것을 **발산**이라고 합니다.

실제 손실로 보기

같은 일을 아이스크림 회귀에서 PyTorch로 확인해 보겠습니다. 학습률을 다섯 가지로 바꿔 가며 10번째, 50번째, 200번째 반복의 손실을 적었어요.

```
import numpy as np
import torch

rng = np.random.default_rng(0)
기온 = rng.uniform(18, 34, size=30).round(1)
판매량 = (2.5 * 기온 - 15 + rng.normal(0, 4, size=30)).round()
x = torch.tensor((기온 - 기온.mean()) / 기온.std(), dtype=torch.float32)
y = torch.tensor(판매량, dtype=torch.float32)
```

```
def 학습(x: torch.Tensor, y: torch.Tensor, 학습률: float, 횟수: int) -> list[float]:
    """경사하강법을 돌리며 정해 둔 시점의 손실을 모은다."""
    w = torch.zeros((), requires_grad=True)
    b = torch.zeros((), requires_grad=True)
    기록 = []
    for 반복 in range(1, 횟수 + 1):
        손실 = ((w * x + b - y) ** 2).mean()
        손실.backward()
        with torch.no_grad():
            w -= 학습률 * w.grad
            b -= 학습률 * b.grad
            w.grad.zero_()
            b.grad.zero_()
        if 반복 in (10, 50, 200):
            기록.append(손실.item())
    return 기록


print("학습률       10번째      50번째     200번째")
for 학습률 in [0.001, 0.01, 0.1, 0.9, 1.05]:
    print(f"{학습률:<6}", "  ".join(f"{v:11.6g}" for v in 학습(x, y, 학습률, 200)))
```

```
학습률       10번째      50번째     200번째
0.001      2827.27     2410.94     1328.71
0.01       2041.39       416.8     15.0165
0.1        66.6148     14.0772     14.0772
0.9        66.6148     14.0772     14.0772
1.05       16229.1  3.3215e+07 8.69239e+19
```

5.3절에서 본 바닥 손실은 14.077이었죠. 학습률 0.001은 200번 반복해도 1328.71로 바닥이 아득하고, 0.01은 200번째에야 15.0165로 바닥 근처에 옵니다. 0.1과 0.9는 50번째에 이미 14.0772, 바닥이에요. 0.9는 좌우로 튀면서도 0.1과 같은 속도로 다가가서 손실이 똑같이 나왔습니다(앞의 표에서 0.1과 0.9의 거리가 같았던 것처럼요). 그런데 0.9보다 조금 큰 1.05에서는 손실이 1만 6천, 3천3백만을 지나 200번째에 `8.69239e+19`, 무려 10의 19제곱 단위가 됐어요. 이 책의 실행 환경에서 더 돌려 보니 500번째에는 float32가 담을 수 있는 한계를 넘어 `inf`(무한대)가 됐습니다. 여기서 더 가면

무한대끼리 빼는 계산이 끼어들어 nan(숫자가 아님)이 돼요. 바로 다음 예제에서 그 nan을 직접 보게 됩니다.

학습률은 이렇게 **너무 작으면 시간을 낭비하고, 너무 크면 학습 자체가 망가지는** 숫자입니다. 게다가 "딱 좋은 값"은 데이터와 모델마다 달라서 미리 알 수 없어요. 그래서 실무에서는 0.1, 0.01, 0.001처럼 10배씩 바꿔 가며 손실이 어떻게 줄어드는지 보고 고릅니다. 손실이 nan이 됐다면 가장 먼저 의심할 범인이 학습률이에요. 학습 도중에 학습률을 줄여 가는 방법(스케줄러)은 8장에서 다룹니다.

표준화하지 않으면 생기는 일

5.3절에서 기온을 표준화한 이유를 이제 확인할 수 있습니다. 표준화하지 않은 18~34도 기온을 그대로 넣고 학습률 0.01과 0.001로 돌려 보겠습니다.

```
import numpy as np
import torch

rng = np.random.default_rng(0)
기온 = rng.uniform(18, 34, size=30).round(1)
판매량 = (2.5 * 기온 - 15 + rng.normal(0, 4, size=30)).round()
x_원래 = torch.tensor(기온, dtype=torch.float32)          # 표준화하지 않은 18~34
y = torch.tensor(판매량, dtype=torch.float32)

for 학습률 in [0.01, 0.001]:
    w = torch.zeros((), requires_grad=True)
    b = torch.zeros((), requires_grad=True)
    for 반복 in range(1, 5001):
        손실 = ((w * x_원래 + b - y) ** 2).mean()
        손실.backward()
        with torch.no_grad():
            w -= 학습률 * w.grad
            b -= 학습률 * b.grad
            w.grad.zero_()
            b.grad.zero_()
        if 반복 in (5, 100, 5000):
            print(f"학습률 {학습률}, {반복:4}번째: 손실 {손실.item():.4g}, w = {w.item():.3f}, b =
            ↪ {b.item():.3f}")
```

```
정답_w, 정답_b = np.polyfit(기온, 판매량, deg=1)
print(f"정답: w = {정답_w:.3f}, b = {정답_b:.3f}")
```

```
학습률 0.01,    5번째: 손실 3.484e+12, w = 940409.688, b = 34166.535
학습률 0.01,  100번째: 손실 nan, w = nan, b = nan
학습률 0.01, 5000번째: 손실 nan, w = nan, b = nan
학습률 0.001,    5번째: 손실 27.05, w = 2.035, b = 0.069
학습률 0.001,  100번째: 손실 20.72, w = 1.996, b = -0.025
학습률 0.001, 5000번째: 손실 17.41, w = 2.141, b = -4.027
정답: w = 2.495, b = -13.758
```

표준화한 데이터에서 무난했던 학습률 0.01이 여기서는 5번 만에 손실 3.484e+12, 100번째에는 `nan`이 됐습니다. 기온이 25 근처의 큰 값이라 `w` 방향의 경사가 표준화했을 때보다 수백 배 가팔라져서, 같은 학습률로도 걸음이 엄청나게 커진 거예요.

그럼 학습률을 0.001로 줄이면? 발산은 멈췄지만 이번에는 `b`가 문제입니다. 5000번을 돌렸는데도 `b`는 -4.027로 정답 -13.758에 한참 못 미치고, 손실도 17.41로 바닥(14.077)에 닿지 못했어요. `w` 방향은 너무 가팔라서 작은 학습률이 필요한데, `b` 방향은 완만해서 그 작은 학습률로는 한없이 느린 겁니다. 한 산에 아주 가파른 방향과 아주 완만한 방향이 섞여 있으면, 학습률 하나로는 둘 다 만족시킬 수 없어요.

표준화는 특성들의 크기를 비슷하게 맞춰서 산을 **둥근 그릇**에 가깝게 만들어 줍니다. 그러면 어느 방향이든 비슷한 학습률로 잘 내려갈 수 있죠. 2장 2.6절에서 로지스틱 회귀가 표준화하기 전에는 1000번을 고치고도 끝내지 못하다가 표준화하자 18번 만에 끝냈던 것, 그때 "눈금이 다르면 골짜기가 길쭉한 협곡이 된다"고 했던 것을 지금 숫자로 직접 본 셈입니다. 2장에서 배운 스케일링이 경사하강법을 쓰는 모델에서 특히 중요한 이유가 바로 이것이에요. 신경망에 넣는 데이터를 거의 언제나 정규화하는 이유이기도 합니다.

5.11 미니배치 경사하강법 맛보기

전부 볼 것인가, 조금씩 볼 것인가

지금까지는 기울기를 구할 때마다 **데이터 전체**로 손실을 계산했습니다. 30일치라면 문제없지만, 사진 100만 장이라면 어떨까요? 한 걸음 내딛기 위해 100만 장을 모두 계산해야 하니 한 걸음이 너무 비쌉니다.

여론 조사를 생각해 보세요. 전 국민에게 다 묻지 않아도 잘 뽑은 1,000명에게 물으면 대략적인 결과를 알 수 있죠. 기울기도 마찬가지예요. 데이터에서 일부만 무작위로 뽑아 계산한 기울기는 전체로 계산한 것과 **대략 같은 방향**을 가리킵니다. 조금 부정확한 대신 훨씬 싸니까 같은 시간에 훨씬 많은 걸음을 걸을 수 있어요. 이렇게 데이터를 작은 묶음(**미니배치**)으로 나눠 묶음마다 한 걸음씩 걷는 방법을 **미니배치 경사하강법**이라고 합니다. 데이터 전체를 한 바퀴 도는 것을 **에폭(epoch)** 이라고 불러요.

1,000일치 데이터로 세 가지 방식을 비교해 보겠습니다. 데이터 전체를 한 묶음으로 쓰는 방식(배치 1000), 32개씩 나누는 방식, 한 개씩 쓰는 방식(배치 1)입니다. 무작위로 뽑은 데이터로 걸음을 정한다고 해서 배치 1 방식을 **확률적 경사하강법(SGD)** 이라고 부르는데, 요즘은 미니배치로 걷는 방식까지 흔히 SGD라고 불러요. 6장에서 만날 `torch.optim.SGD`도 보통 미니배치와 함께 씁니다.

```
import numpy as np
import torch

# 1,000일치 예제용 데이터(앞의 30일과 같은 규칙으로 만들었다)
rng = np.random.default_rng(1)
기온 = rng.uniform(18, 34, size=1000)
판매량 = 2.5 * 기온 - 15 + rng.normal(0, 4, size=1000)
x = torch.tensor((기온 - 기온.mean()) / 기온.std(), dtype=torch.float32)
y = torch.tensor(판매량, dtype=torch.float32)


def 학습(배치_크기: int, 에폭: int = 5, 학습률: float = 0.05) -> None:
    torch.manual_seed(0)
    w = torch.zeros((), requires_grad=True)
    b = torch.zeros((), requires_grad=True)
    고친_횟수 = 0
    기록 = []
    for _ in range(에폭):                          # 에폭: 데이터 전체를 한 바퀴 도는 것
        순서 = torch.randperm(len(x))              # 매 에폭 섞기
        for 시작 in range(0, len(x), 배치_크기):
            고를 = 순서[시작:시작 + 배치_크기]
            손실 = ((w * x[고를] + b - y[고를]) ** 2).mean()   # 일부만 보고 손실 계산
            손실.backward()
            with torch.no_grad():
                w -= 학습률 * w.grad
```

```
                b -= 학습률 * b.grad
                w.grad.zero_()
                b.grad.zero_()
            고친_횟수 += 1
        with torch.no_grad():
            기록.append(((w * x + b - y) ** 2).mean().item())  # 에폭 끝: 전체 데이터 손실
    print(f"배치 {배치_크기:4} ¦ {고친_횟수:4}번 고침 ¦", " ".join(f"{v:7.2f}" for v in 기록))


print("                        에폭별 전체 손실: 1, 2, 3, 4, 5")
for 배치_크기 in [1000, 32, 1]:
    학습(배치_크기)

정답_w, 정답_b = np.polyfit(x.numpy(), y.numpy(), deg=1)
print(f"도달할 수 있는 가장 낮은 손실: {((정답_w * x + 정답_b - y) ** 2).mean().item():.2f}")
```

```
                        에폭별 전체 손실: 1, 2, 3, 4, 5
배치 1000 ¦    5번 고침 ¦ 2171.42 1762.06 1430.49 1161.91  944.36
배치   32 ¦  160번 고침 ¦   19.20   17.02   16.94   16.96   16.94
배치    1 ¦ 5000번 고침 ¦   17.20   19.89   17.57   19.83   17.34
도달할 수 있는 가장 낮은 손실: 16.93
```

똑같이 5에폭, 즉 데이터를 똑같이 다섯 번 봤습니다. 그런데 결과가 크게 다르죠.

- **배치 1000**: 에폭마다 한 걸음이라 5번밖에 못 고쳤습니다. 손실이 944.36으로 아직 한참 높아요. 기울기는 가장 정확하지만 걸음 수가 너무 적습니다.
- **배치 32**: 에폭마다 32번 가까이 고쳐서 총 160번 걸었고, 2에폭 만에 17.02, 그 뒤로 16.94 근처에 머뭅니다. 도달할 수 있는 가장 낮은 손실 16.93과 거의 같아요.
- **배치 1**: 5000번이나 고쳐서 첫 에폭부터 17.20까지 내려왔지만, 그 뒤로 19.89, 17.57, 19.83, 17.34로 **오르내리며 흔들립니다.** 데이터 하나로 잰 기울기는 그 데이터 하나의 사정에 크게 휘둘리거든요.

미니배치는 그 사이의 균형입니다. 기울기가 충분히 정확하면서도 걸음을 많이 걸을 수 있고, GPU가 32개를 한꺼번에 계산하니(4.2절의 배치) 하나씩 32번 계산하는 것보다 훨씬 빠르기도 해요. 실무에서 거의 언제나 미니배치를 쓰는 이유입니다. 배치 1의 흔들림이 나쁘기만 한 것도 아니에요. 5.2절의 얕은 골짜기에서 빠져나오게 해 주는 것이 바로 이런 흔들림입니다.

코드를 보면 `torch.randperm`으로 매 에폭 순서를 섞고, 그 순서를 32개씩 잘라 인덱싱(4.4절)으로 묶음을 꺼냈습니다. 섞는 이유는 매번 같은 묶음을 같은 순서로 보면 그 순서 자체에 모델이 휘둘릴 수 있기 때문이에요. 이렇게 섞고 자르는 일을 매번 직접 하기는 번거롭죠. 7장에서 `DataLoader`가 이 일을 대신 해 줍니다.

5.12 torch.autograd.grad: 기울기를 직접 받기

.grad에 쌓지 않고 돌려받기

`backward()`는 기울기를 잎 텐서의 `.grad`에 쌓아 둡니다. 모델을 학습할 때는 이게 편하지만, 가끔은 **기울기를 결과값으로 바로 받고 싶을 때**가 있어요. 그럴 때 쓰는 함수가 `torch.autograd.grad`입니다.

```
import torch

x = torch.tensor(3.0, requires_grad=True)
y = x ** 3

# backward()와 달리 x.grad에 쌓지 않고, 기울기를 결과로 돌려준다
(dy_dx,) = torch.autograd.grad(y, x, create_graph=True)   # 기울기 계산도 기록해 둔다
print("1번 미분 3x^2 :", dy_dx)          # 3 × 9 = 27
(d2y_dx2,) = torch.autograd.grad(dy_dx, x)                # 기울기의 기울기
print("2번 미분 6x   :", d2y_dx2)        # 6 × 3 = 18
print("x.grad        :", x.grad)                          # 건드리지 않았다

# 여러 텐서에 대한 기울기를 한꺼번에
w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
손실 = (w * 3.0 + b - 10.0) ** 2
기울기_w, 기울기_b = torch.autograd.grad(손실, [w, b])
print("손실의 w, b 기울기:", 기울기_w.item(), 기울기_b.item())
```

```
1번 미분 3x^2 : tensor(27., grad_fn=<MulBackward0>)
2번 미분 6x   : tensor(18.)
x.grad        : None
```

```
손실의 w, b 기울기: -18.0 -6.0
```

`torch.autograd.grad(y, x)`는 "y를 x로 미분한 값"을 튜플로 돌려줍니다. 결과가 튜플이라서 `(dy_dx,) = ...`처럼 하나짜리 튜플로 풀어 받았어요(1권 6장의 언패킹). `x = 3`에서 x^3의 기울기 $3x^2 = 27$이 나왔습니다.

`create_graph=True`를 주면 **기울기를 구하는 계산까지 그래프로 기록**합니다. 그래서 출력된 `dy_dx`에 `grad_fn`이 붙어 있고, 이걸 다시 미분하면 "기울기의 기울기", 즉 2번 미분한 값 `6x = 18`이 나와요. 2번 미분은 함수가 얼마나 휘어 있는지를 알려 주는데, 일부 고급 학습 기법(학습 안정화를 위한 기울기 벌점 등)에서 씁니다.

`x.grad`가 `None`인 것도 보세요. `autograd.grad`는 `.grad`를 건드리지 않아서, 5.7절의 쌓이는 함정과 무관합니다. 여러 텐서에 대한 기울기도 목록으로 넘겨서 한꺼번에 받을 수 있어요. 마지막 줄의 -18, -6은 5.6절에서 손으로 역전파한 값 그대로입니다.

입력에 대한 기울기: 무엇에 민감한가

지금까지는 늘 **모델의 숫자**(`w`, `b`)에 대한 기울기를 구했습니다. 그런데 자동 미분은 `requires_grad`만 붙이면 무엇에 대해서든 기울기를 구해 줘요. **입력**에 대한 기울기를 구하면 "입력이 조금 바뀌면 예측이 얼마나 바뀌나", 즉 모델이 어떤 입력에 민감한지 알 수 있습니다. 학습이 끝났다고 치는 판매량 예측 모델로 해 보겠습니다.

```
import torch


def 모델(입력: torch.Tensor) -> torch.Tensor:
    """학습이 끝났다고 치는 모델: 기온과 습도로 판매량을 예측한다(예제용 식)."""
    기온, 습도 = 입력[0], 입력[1]
    return 2.5 * 기온 + 0.8 * 습도 + 0.05 * 기온 * 습도 - 15


오늘 = torch.tensor([30.0, 60.0], requires_grad=True)     # 기온 30°C, 습도 60%
예측 = 모델(오늘)
(민감도,) = torch.autograd.grad(예측, 오늘)               # 입력에 대한 기울기
print(f"예측 판매량: {예측.item():.1f}개")
print(f"기온 1°C당 {민감도[0]:.2f}개 (손으로: 2.5 + 0.05 × 60 = {2.5 + 0.05 * 60:.2f})")
```

```
print(f"습도 1%p당 {민감도[1]:.2f}개 (손으로: 0.8 + 0.05 × 30 = {0.8 + 0.05 * 30:.2f})")
```

```
예측 판매량: 198.0개
기온 1°C당 5.50개 (손으로: 2.5 + 0.05 × 60 = 5.50)
습도 1%p당 2.30개 (손으로: 0.8 + 0.05 × 30 = 2.30)
```

기온 30°C, 습도 60%인 날의 예측은 198개입니다. 이 자리에서 기온이 1°C 오르면 약 5.5개, 습도가 1%p 오르면 약 2.3개 더 팔린다고 나왔어요. 괄호 안의 손 계산과 같죠. 이 모델에는 `기온 × 습도` 항이 있어서 기온의 효과가 습도에 따라 달라지는데(습한 날일수록 기온이 더 중요), 자동 미분은 그런 얽힌 관계까지 알아서 계산해 줍니다. 이미지 분류 모델에서 "어느 픽셀이 판단에 가장 큰 영향을 줬나"를 보여 주는 기법도 바로 이 입력에 대한 기울기에서 출발해요.

5.13 실습: 당뇨병 진행 정도를 autograd만으로 예측하기

이제 지금까지 배운 것을 모두 써서 실제 데이터로 회귀 모델을 처음부터 학습해 보겠습니다. 데이터는 scikit-learn에 들어 있는 **당뇨병 데이터셋**이에요. 당뇨병 환자 442명의 나이, 성별, 체질량지수(bmi), 평균 혈압(bp), 혈액 검사 수치 6가지(s1~s6)와, 1년 뒤 병이 얼마나 진행됐는지를 나타내는 점수(대략 25~346)가 들어 있습니다(Efron 등의 2004년 논문 「Least Angle Regression」에 실린 데이터로, scikit-learn(BSD-3 라이선스) 안에 함께 들어 있어서 따로 내려받지 않아도 돼요).

특성이 10개이니 모델은 `예측 = x1 × w1 + x2 × w2 + … + x10 × w10 + b`, 4.6절의 행렬 곱으로 쓰면 `X @ w + b`입니다. 할 일은 네 가지예요.

1. 데이터를 학습용 350명, 시험용 92명으로 나누고 표준화하기
2. 손으로 유도한 기울기와 autograd가 같은지 확인하기
3. 미니배치 경사하강법으로 학습하고 시험 데이터로 평가하기
4. 한 번에 푸는 정답(최소제곱법)과 비교하기

```
import torch
from sklearn.datasets import load_diabetes

# 1) 데이터: 당뇨병 환자 442명의 검사값 10가지와 1년 뒤 진행 정도
데이터 = load_diabetes(scaled=False)
X = torch.tensor(데이터.data, dtype=torch.float32)       # (442, 10)
```

```
y = torch.tensor(데이터.target, dtype=torch.float32)     # (442,)
이름 = 데이터.feature_names

torch.manual_seed(0)
순서 = torch.randperm(len(X))
학습_idx, 시험_idx = 순서[:350], 순서[350:]            # 350명으로 배우고 92명으로 시험
평균 = X[학습_idx].mean(dim=0)                          # 기준은 학습 데이터에서만 (2장)
표준편차 = X[학습_idx].std(dim=0)
X = (X - 평균) / 표준편차
X_학습, y_학습, X_시험, y_시험 = X[학습_idx], y[학습_idx], X[시험_idx], y[시험_idx]


def 예측(X: torch.Tensor, w: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    return X @ w + b                                    # (n, 10) @ (10,) → (n,)


def rmse(X: torch.Tensor, y: torch.Tensor, w: torch.Tensor, b: torch.Tensor) -> float:
    """평균적으로 몇 점 틀리는지: 평균제곱오차의 제곱근."""
    return ((예측(X, w, b) - y) ** 2).mean().sqrt().item()


# 2) 손으로 미분한 기울기와 autograd가 같은지 먼저 확인
w = torch.zeros(10, requires_grad=True)
b = torch.zeros((), requires_grad=True)
손실 = ((예측(X_학습, w, b) - y_학습) ** 2).mean()
손실.backward()
with torch.no_grad():
    오차 = 예측(X_학습, w, b) - y_학습
    손_w = 2 * X_학습.T @ 오차 / len(X_학습)             # 5.3절 공식을 10개 특성으로
    손_b = 2 * 오차.mean()
print("기울기 일치:", torch.allclose(w.grad, 손_w), torch.allclose(b.grad, 손_b))
w.grad.zero_()
b.grad.zero_()

# 3) 미니배치 경사하강법으로 학습
학습률, 배치_크기 = 0.05, 32
```

```
for 에폭 in range(1, 101):
    섞기 = torch.randperm(len(X_학습))
    for 시작 in range(0, len(X_학습), 배치_크기):
        고를 = 섞기[시작:시작 + 배치_크기]
        손실 = ((예측(X_학습[고를], w, b) - y_학습[고를]) ** 2).mean()
        손실.backward()
        with torch.no_grad():
            w -= 학습률 * w.grad
            b -= 학습률 * b.grad
        w.grad.zero_()
        b.grad.zero_()
    if 에폭 in (1, 5, 20, 100):
        with torch.no_grad():
            print(f"에폭 {에폭:3}: 학습 RMSE {rmse(X_학습, y_학습, w, b):6.2f}, "
                  f"시험 RMSE {rmse(X_시험, y_시험, w, b):6.2f}")
print(f"s1, s2 가중치: {w[4].item():.2f}, {w[5].item():.2f}")
```

```
기울기 일치: True True
에폭   1: 학습 RMSE  72.23, 시험 RMSE  69.83
에폭   5: 학습 RMSE  54.18, 시험 RMSE  52.52
에폭  20: 학습 RMSE  54.14, 시험 RMSE  52.79
에폭 100: 학습 RMSE  54.06, 시험 RMSE  52.25
s1, s2 가중치: -25.44, 16.54
```

한 단계씩 짚어 보겠습니다.

- **나누기와 표준화**: `torch.randperm`으로 순서를 섞어 앞 350명은 학습용, 나머지 92명은 시험용으로 나눴습니다. 표준화의 평균과 표준편차는 **학습 데이터에서만** 구했어요. 시험 데이터는 "아직 본 적 없는 새 환자"여야 하니까요. 2장의 데이터 누수 이야기 그대로입니다.
- **기울기 확인**: 5.3절의 공식 `평균(2 × 오차 × x)`를 특성 10개로 넓히면 `2 * X.T @ 오차 / n`이 됩니다(4.6절의 행렬 곱이 특성마다의 "곱하고 더하기"를 한 번에 해 줘요). 이 공식과 `backward()`의 결과가 `True True`, 같다고 나왔습니다. 특성이 10개로 늘어도 autograd는 우리가 손으로 한 계산과 같은 답을 냅니다.
- **학습**: 5.11절의 미니배치 반복에 5.7절의 순서(손실 → `backward()` → `no_grad`에서 고치기 → 비우기)를 그대로 썼어요. 성적은 RMSE(평균제곱오차의 제곱근)로 쟀는데, 제곱근을 씌우면 단위가 원래 점수와 같아져서 "평균적으로 몇 점쯤 틀리나"로 읽을 수 있습니다. 1에폭에 시험

RMSE 69.83, 5에폭에 52.52로 빠르게 좋아지고, 그 뒤로는 52~53 근처에 머물러요.

시험 RMSE 52점이 좋은 걸까요? 비교 대상이 있어야 판단할 수 있죠. 아래에서 볼 "학습 데이터 평균만 찍는" 가장 단순한 예측은 80.35점을 틀립니다. 우리 모델이 그보다 30점 가까이 덜 틀리니, 10가지 검사값에서 꽤 쓸모 있는 정보를 뽑아낸 거예요. 다만 1년 뒤의 병세를 검사값 몇 개로 정확히 맞히기는 원래 어려운 일이라, 직선 모델로는 이 정도가 한계입니다(평가 지표는 3장에서 자세히 다뤘어요).

마지막 줄의 s1, s2 가중치(-25.44, 16.54)는 잠시 기억해 두세요. 이제 이 답이 정말 바닥인지 확인하겠습니다. 미니배치의 흔들림 없이 끝까지 내려가도록 이번에는 **전체 데이터로** 5000번 걷고, 한 번에 푸는 공식인 최소제곱법(`torch.linalg.lstsq`)의 답과 특성마다 나란히 놓아 봅니다.

```
import torch
from sklearn.datasets import load_diabetes

# 1) 데이터: 당뇨병 환자 442명의 검사값 10가지와 1년 뒤 진행 정도
데이터 = load_diabetes(scaled=False)
X = torch.tensor(데이터.data, dtype=torch.float32)       # (442, 10)
y = torch.tensor(데이터.target, dtype=torch.float32)     # (442,)
이름 = 데이터.feature_names

torch.manual_seed(0)
순서 = torch.randperm(len(X))
학습_idx, 시험_idx = 순서[:350], 순서[350:]             # 350명으로 배우고 92명으로 시험
평균 = X[학습_idx].mean(dim=0)                           # 기준은 학습 데이터에서만 (2장)
표준편차 = X[학습_idx].std(dim=0)
X = (X - 평균) / 표준편차
X_학습, y_학습, X_시험, y_시험 = X[학습_idx], y[학습_idx], X[시험_idx], y[시험_idx]


def 예측(X: torch.Tensor, w: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    return X @ w + b                                     # (n, 10) @ (10,) → (n,)


def rmse(X: torch.Tensor, y: torch.Tensor, w: torch.Tensor, b: torch.Tensor) -> float:
    """평균적으로 몇 점 틀리는지: 평균제곱오차의 제곱근."""
    return ((예측(X, w, b) - y) ** 2).mean().sqrt().item()
```

```
# 2) 전체 데이터로, 학습률 0.2, 5000번 (미니배치의 흔들림 없이 끝까지 내려가기)
w = torch.zeros(10, requires_grad=True)
b = torch.zeros((), requires_grad=True)
for _ in range(5000):
    손실 = ((예측(X_학습, w, b) - y_학습) ** 2).mean()
    손실.backward()
    with torch.no_grad():
        w -= 0.2 * w.grad
        b -= 0.2 * b.grad
    w.grad.zero_()
    b.grad.zero_()

# 3) 한 번에 푸는 정답(최소제곱법)
A = torch.cat([X_학습, torch.ones(len(X_학습), 1)], dim=1)   # 마지막 열 1은 b 자리
정답 = torch.linalg.lstsq(A, y_학습.unsqueeze(1)).solution.squeeze(1)
정답_w, 정답_b = 정답[:10], 정답[10]

print("특성  경사하강법  최소제곱법")
for i in range(10):
    print(f"{이름[i]:<4} {w[i].item():10.3f} {정답_w[i].item():11.3f}")
print(f"b    {b.item():10.3f} {정답_b.item():11.3f}")
print(f"가장 큰 차이: {(w.detach() - 정답_w).abs().max().item():.4f}")
with torch.no_grad():
    print(f"학습 RMSE: 경사하강법 {rmse(X_학습, y_학습, w, b):.2f}, 최소제곱법 {rmse(X_학습, y_학
    ↪ 습, 정답_w, 정답_b):.2f}")
    print(f"시험 RMSE: 경사하강법 {rmse(X_시험, y_시험, w, b):.2f}, 최소제곱법 {rmse(X_시험, y_시
    ↪ 험, 정답_w, 정답_b):.2f}")
print(f"평균만 찍었을 때 시험 RMSE: {(y_시험 - y_학습.mean()).pow(2).mean().sqrt():.2f}")
```

```
특성  경사하강법  최소제곱법
age      -2.824      -2.824
sex     -10.222     -10.222
bmi      24.753      24.753
bp       14.775      14.775
```

```
s1      -43.177     -43.177
s2       28.161      28.162
s3        5.636       5.636
s4        6.552       6.552
s5       36.419      36.419
s6        3.911       3.911
b       152.483     152.483
가장 큰 차이: 0.0004
학습 RMSE: 경사하강법 53.88, 최소제곱법 53.88
시험 RMSE: 경사하강법 52.35, 최소제곱법 52.35
평균만 찍었을 때 시험 RMSE: 80.35
```

열 개 특성의 가중치와 `b`가 소수점 셋째 자리까지 거의 모두 같고, 가장 큰 차이가 0.0004입니다. 학습·시험 RMSE도 둘이 똑같아요. **기울기 공식을 한 줄도 쓰지 않고** `backward()`만으로 걸어 내려간 곳이 공식으로 푼 정답과 같은 자리라는 뜻입니다. `torch.linalg.lstsq`에 넣을 때 `X`에 1로 채운 열을 덧붙인 건, 그 열의 가중치가 `b` 역할을 하게 하려는 흔한 방법이에요.

그런데 앞의 미니배치 결과와 비교하면 이상한 점이 있습니다. 정답의 s1, s2 가중치는 -43.18, 28.16인데 미니배치로 100에폭 학습한 값은 -25.44, 16.54로 꽤 다릅니다. 그런데도 시험 RMSE는 52.25와 52.35로 거의 같았죠. 어떻게 된 걸까요?

s1(총콜레스테롤)과 s2(LDL 콜레스테롤)는 서로 아주 비슷하게 움직이는 검사값입니다. 3장 3.2절에서 scikit-learn 선형 회귀의 계수가 -44.1과 +24.5로 밀고 당기던 바로 그 둘이에요(다중공선성). 그래서 "s1을 많이 빼고 s2를 많이 더하는" 답이나 "둘 다 조금씩만" 쓰는 답이나 예측이 거의 같아요. 손실 산으로 말하면 이 방향은 **거의 평평한 긴 골짜기**라서, 5.10절의 `b`처럼 기울기가 아주 작아 천천히 움직입니다. 미니배치는 흔들리면서 골짜기 바닥 근처를 돌아다니느라 그 평평한 방향으로는 끝까지 가지 못한 거예요. 이렇게 손실이 거의 같은 답이 여럿일 때는 가중치 하나하나를 "이 특성이 얼마나 중요한가"로 곧이곧대로 읽으면 안 됩니다. 딥러닝 모델은 이런 평평한 방향이 훨씬 많아서, 가중치 하나보다 모델이 내는 **예측**을 보고 판단하는 습관이 중요해요.

5.14 정리와 다음 장 미리보기

이 장에서는 모델이 스스로 나아지는 원리, 기울기와 경사하강법을 손으로 계산하고 PyTorch로 확인했습니다.

- **기울기**: 조금 움직였을 때 얼마나 변하는지. 수치 미분(중앙 차분)으로 직접 잴 수 있고, 여러

방향의 기울기 묶음이 그래디언트. 그래디언트의 반대 방향이 가장 빠른 내리막

- **경사하강법**: `새 위치 = 지금 위치 - 학습률 × 기울기`를 반복. 지역 최솟값에 멈출 수 있음. 선형 회귀에서는 공식(최소제곱법)과 같은 답에 도착
- **자동 미분**: `requires_grad=True`로 표시 → 평소처럼 계산 → `backward()` → `.grad`. 스칼라에서만 부르고, 실수 텐서만, 잎에만 기울기가 남음
- **계산 그래프와 연쇄 법칙**: 실행하면서 그래프를 만들고(`grad_fn`), `backward()`는 끝에서부터 단계별 기울기를 곱해 내려감(역전파). 그래프는 한 번 쓰면 사라짐
- **함정 피하기**: `.grad`는 쌓이므로 매번 비우기, 값 고치기는 `no_grad` 안에서, 예측만 할 때는 `inference_mode`, NumPy로 넘길 때는 `detach()`, 추적 중인 텐서에 제자리 연산 쓰지 않기
- **학습률과 미니배치**: 너무 작으면 느리고 너무 크면 발산(`nan`). 특성을 표준화해야 학습률 하나로 잘 내려감. 미니배치는 정확도와 걸음 수 사이의 균형
- **`torch.autograd.grad`**: `.grad`에 쌓지 않고 기울기를 돌려받기, 2번 미분, 입력에 대한 기울기

이번 장의 실습 코드를 다시 보면, 학습 반복 안에 `w`와 `b`를 하나씩 고치고 하나씩 비우는 줄이 있었습니다. 숫자 묶음이 둘이라 할 만했지만, 층이 여러 개인 신경망은 이런 묶음이 수십 개예요. 모델을 `X @ w + b`처럼 직접 쓰는 것도 금방 한계가 옵니다.

다음 6장에서는 이 일을 PyTorch의 도구에게 맡깁니다. `X @ w + b`는 `nn.Linear` 한 줄이 되고, 모델 전체를 `nn.Module`이라는 클래스(1권 11장)로 묶어요. 손으로 쓴 평균제곱오차는 손실 함수로, `no_grad` 안에서 고치고 비우던 네 줄은 옵티마이저의 `step()`과 `zero_grad()` 두 줄로 바뀝니다. 겉모습은 달라지지만 그 안에서 일어나는 일은 이 장에서 손으로 해 본 것과 똑같아요. 그리고 그 도구들로 첫 분류 신경망을 만들어 봅니다.

06

nn.Module과 학습 루프: 모든 모델이 따르는 표준형

학습 목표

- 층 하나가 "입력 @ 가중치 + 편향"이라는 것을 손으로 계산해 `nn.Linear`와 비교하기
- 활성화 함수가 왜 필요한지 XOR 문제로 확인하고, ReLU·시그모이드·소프트맥스를 구분하기
- `nn.Module`을 상속해 모델을 만들고, `nn.Sequential`로 짧게 쓰고, 파라미터를 세기
- 문제에 맞는 손실 함수와 옵티마이저를 고르고, 로짓을 넣어야 하는 이유 설명하기
- 표준 학습 루프 다섯 줄과 평가 루프를 쓰고, 모델을 장치로 옮기고 저장·불러오기
- FashionMNIST 옷 사진 분류기를 처음부터 끝까지 만들어 시험 정확도 재기

들어가며: 모델이 달라도 학습하는 모양은 같을까요?

요리 학원에 가면 메뉴는 매일 바뀌어도 수업 순서는 늘 같습니다. 재료를 손질하고, 조리하고, 맛을 보고, 간을 고치고, 다시 맛을 보죠. 김치찌개든 파스타든 "맛보고 고치기"를 되풀이한다는 틀은 같아요.

딥러닝도 똑같습니다. 이 책 뒤쪽에서 사진을 보는 CNN(9장), 문장을 읽는 트랜스포머(12장), 그림을 그리는 생성 모델(15장)까지 만나게 되는데, 모델 생김새는 제각각이어도 **학습시키는 코드의 뼈대는 거의 한 글자도 다르지 않습니다.** 예측하고, 얼마나 틀렸는지 재고, 고칠 방향을 계산하고, 고칩니다. 이 장의 제목에 "표준형"이라는 말을 넣은 이유예요.

5장에서는 기울기를 직접 구하고 `w -= 학습률 * w.grad`처럼 파라미터를 손으로 고쳐 가며 선형 회귀를 학습했습니다. 원리를 이해하는 데는 그게 최고지만, 층이 수십 개이고 파라미터가 수백만 개인 모델을 그렇게 다룰 수는 없죠. PyTorch는 이 일을 네 가지 부품으로 나눠 줍니다.

- **모델**(`nn.Module`): 입력을 받아 예측을 내는 부품. 층을 쌓아서 만듭니다.
- **손실 함수**(`nn.MSELoss`, `nn.CrossEntropyLoss` 등): 예측이 얼마나 틀렸는지 숫자 하나로 재는 부품.
- **옵티마이저**(`torch.optim.SGD`, `torch.optim.Adam` 등): 기울기를 보고 파라미터를 고치는 부품.
- **학습 루프**: 위의 셋을 정해진 순서로 돌리는 다섯 줄짜리 반복문.

이 장에서는 이 부품을 하나씩 손에 익히고, 마지막에 옷 사진 7만 장으로 진짜 이미지 분류기를 만들어 봅니다. 4장에서 "신경망의 층 하나하나가 결국 입력 @ 가중치 + 편향"이라고 예고했었죠. 바로 그 계산에서 시작하겠습니다.

6.1 층 하나: 입력 @ 가중치 + 편향

손으로 만드는 층

온라인 가게에서 손님 네 명의 행동을 숫자 세 개(방문 횟수, 머문 시간, 장바구니 금액)로 정리했다고 해 봅시다. 이 손님들이 "다시 올 가능성"과 "쿠폰에 반응할 가능성", 두 가지 점수를 내고 싶어요. 4장 4.6절의 반영 비율 계산을 떠올려 보세요. 특성마다 비율(가중치)을 곱해 더하면 점수 하나가 나왔죠. 점수 두 개가 필요하면 비율 묶음을 두 벌 준비하면 됩니다.

```
import torch

# 손님 4명 × 특성 3개(방문 횟수, 머문 시간, 장바구니 금액). 예제용 숫자입니다.
x = torch.tensor([[0.2, 0.5, 0.1],
                  [0.9, 0.1, 0.4],
                  [0.4, 0.8, 0.7],
                  [0.6, 0.3, 0.2]])

torch.manual_seed(0)
가중치 = torch.randn(3, 2)          # 입력 3개 → 출력 2개
편향 = torch.zeros(2)               # 출력마다 하나씩
```

```
출력 = x @ 가중치 + 편향              # (4, 3) @ (3, 2) + (2,) → (4, 2)
print(가중치)
print(출력)
print("모양:", 출력.shape)

# 첫 손님의 첫 번째 출력을 풀어 쓰면
print((x[0] * 가중치[:, 0]).sum() + 편향[0])
```

```
tensor([[ 1.5410, -0.2934],
        [-2.1788,  0.5684],
        [-1.0845, -1.3986]])
tensor([[-0.8896,  0.0857],
        [ 0.7352, -0.7667],
        [-1.8858, -0.6416],
        [ 0.0541, -0.2852]])
모양: torch.Size([4, 2])
tensor(-0.8896)
```

`(4, 3) @ (3, 2)`라서 결과는 `(4, 2)`, 손님 네 명마다 점수가 두 개씩 나왔습니다. 4장의 모양 규칙 `(a, b) @ (b, c) → (a, c)` 그대로예요. 마지막 줄은 첫 손님의 첫 번째 점수를 "같은 자리끼리 곱해서 더하고 편향을 더하기"로 풀어 쓴 것인데, 행렬 곱 결과의 왼쪽 위 칸 `-0.8896`과 똑같습니다.

여기 나온 두 재료에 이름을 붙여 두겠습니다.

- **가중치(weight)**: 입력 하나하나를 얼마나 중요하게 볼지 정하는 숫자. 입력 3개 × 출력 2개라서 6개입니다.
- **편향(bias)**: 입력과 상관없이 출력에 더해 주는 기본값. 출력마다 하나씩, 2개예요. 일차함수 `y = ax + b`에서 `b`에 해당합니다.

신경망에서 이런 계산 한 덩어리를 **층(layer)** 이라고 부르고, 가중치와 편향처럼 학습으로 고쳐 나갈 숫자를 **파라미터(parameter)** 라고 부릅니다. 지금은 가중치를 아무렇게나(`randn`) 정했으니 점수에 아무 의미가 없어요. 학습이란 결국 이 숫자들을 "쓸모 있는 점수가 나오도록" 고쳐 가는 일입니다.

nn.Linear: 같은 일을 하는 부품

이 계산은 너무 자주 쓰여서 PyTorch에 부품으로 들어 있습니다. `nn.Linear(입력수, 출력수)`예요. `nn`은 neural network(신경망)의 줄임말로, 층과 손실 함수 같은 부품이 모여 있는 꾸러미입니다.

```
import torch
from torch import nn

torch.manual_seed(0)
층 = nn.Linear(3, 2)                # 입력 3개 → 출력 2개
print(층)
print("weight:", 층.weight.shape, "bias:", 층.bias.shape)

x = torch.rand(4, 3)
print(층(x))
직접 = x @ 층.weight.T + 층.bias      # 손으로 같은 계산
print("같은가:", torch.allclose(층(x), 직접))
print("학습 대상인가:", 층.weight.requires_grad)
```

```
Linear(in_features=3, out_features=2, bias=True)
weight: torch.Size([2, 3]) bias: torch.Size([2])
tensor([[ 0.0166,  0.1776],
        [-0.0865,  0.3083],
        [-0.1836,  0.3256],
        [-0.0992,  0.1257]], grad_fn=<AddmmBackward0>)
같은가: True
학습 대상인가: True
```

`nn.Linear(3, 2)`를 만드는 순간 가중치와 편향이 무작위 값으로 준비됩니다. 손으로 할 때는 `randn`과 `zeros`를 따로 불렀던 일을 부품이 알아서 해 주는 거예요.

눈여겨볼 곳이 두 군데 있습니다. 첫째, `weight`의 모양이 `(3, 2)`가 아니라 **`(2, 3)`**, 즉 (출력수, 입력수)입니다. 그래서 손으로 같은 계산을 하려면 **`x @ 층.weight.T + 층.bias`**처럼 전치(`.T`)를 해야 해요. PyTorch가 이렇게 저장하는 데는 계산 효율상의 이유가 있지만, 우리가 기억할 것은 "`nn.Linear`의 가중치는 (출력, 입력) 순서"라는 점 하나입니다. 남이 만든 모델의 가중치를 꺼내 볼 때 자주 헷갈리는 부분이에요.

둘째, 출력 끝에 `grad_fn=<AddmmBackward0>`가 붙어 있고 `requires_grad`가 `True`입니다. 5장에서 본 그 표시죠. `nn.Linear`의 파라미터는 처음부터 "기울기를 계산할 대상"으로 만들어져서, 이 층을 거친 계산은 모두 기록됩니다. 우리가 따로 `requires_grad=True`를 붙일 필요가 없어요.

층을 그냥 쌓으면 생기는 일

층 하나로는 표현할 수 있는 것이 적으니 여러 층을 쌓고 싶어집니다. 그런데 `nn.Linear`를 두 개 이어 붙이면 정말 더 똑똑해질까요?

```
import torch
from torch import nn

torch.manual_seed(0)
층1 = nn.Linear(3, 5)
층2 = nn.Linear(5, 2)
x = torch.rand(4, 3)

두_층 = 층2(층1(x))

# 두 층을 합친 가중치 하나, 편향 하나
W = 층2.weight @ 층1.weight                # (2, 5) @ (5, 3) → (2, 3)
b = 층2.weight @ 층1.bias + 층2.bias        # (2,)
한_층 = x @ W.T + b

print(두_층)
print(한_층)
print("같은가:", torch.allclose(두_층, 한_층, atol=1e-6))
```

```
tensor([[-0.2763,  0.4202],
        [-0.2881,  0.5460],
        [-0.3759,  0.3027],
        [-0.3098,  0.3416]], grad_fn=<AddmmBackward0>)
tensor([[-0.2763,  0.4202],
        [-0.2881,  0.5460],
        [-0.3759,  0.3027],
        [-0.3098,  0.3416]], grad_fn=<AddBackward0>)
같은가: True
```

`3 → 5 → 2`로 두 층을 거친 결과와, 두 층의 가중치를 미리 곱해 만든 `3 → 2`짜리 층 하나의 결과가 똑같습니다. 곱하고 더하는 계산은 몇 번을 거듭해도 결국 "곱하고 더하는 계산 한 번"으로 합칠 수 있기 때문이에요. `2 × (3 × x)`가 `6 × x`와 같은 것과 같은 이치입니다.

결국 선형층만 백 개를 쌓아도 선형층 하나와 능력이 같습니다. 층을 쌓는 의미가 생기려면 층과 층 사이에 무언가가 더 필요해요. 그것이 다음 절의 주제입니다.

> **Q. 가중치를 왜 0이 아니라 무작위 값으로 시작하나요?** 모든 가중치가 0이면 같은 층의 출력들이 모두 똑같은 값을 내고, 기울기도 똑같이 받아서 똑같이 고쳐집니다. 출력이 백 개여도 사실상 한 개처럼 움직이는 거죠. 무작위로 시작해야 각자 다른 역할을 맡아 갈 수 있습니다. 어떤 범위의 무작위 값이 좋은지는 8장 가중치 초기화에서 다룹니다.

6.2 활성화 함수: 직선을 구부리는 장치

ReLU, 시그모이드, tanh

층과 층 사이에 끼워 넣는 것이 **활성화 함수**(activation function) 입니다. 하는 일은 단순해요. 숫자 하나를 받아서 **곧지 않게(비선형으로)** 바꿔 돌려줍니다. 대표 선수 셋을 같은 입력에 적용해 보겠습니다.

```
import torch
from torch import nn

x = torch.linspace(-3, 3, 7)
print("입력      :", x)
print("ReLU      :", torch.relu(x))
print("시그모이드:", torch.sigmoid(x).round(decimals=3))
print("tanh      :", torch.tanh(x).round(decimals=3))

활성화 = nn.ReLU()                    # 층처럼 쓰는 모양
print("nn.ReLU   :", 활성화(x))
```

```
입력      : tensor([-3., -2., -1.,  0.,  1.,  2.,  3.])
ReLU      : tensor([0., 0., 0., 0., 1., 2., 3.])
시그모이드: tensor([0.0470, 0.1190, 0.2690, 0.5000, 0.7310, 0.8810, 0.9530])
tanh      : tensor([-0.9950, -0.9640, -0.7620,  0.0000,  0.7620,  0.9640,  0.9950])
nn.ReLU   : tensor([0., 0., 0., 0., 1., 2., 3.])
```

- **ReLU**: 음수는 0으로, 양수는 그대로. 규칙이 "0보다 작으면 0"뿐이라 계산이 아주 빠르고, 지금 대부분의 신경망이 은닉층에 기본으로 씁니다. 이름은 Rectified Linear Unit의 줄임말이

에요.

- **시그모이드(sigmoid)**: 어떤 수든 0과 1 사이로 눌러 넣습니다. 0이면 딱 0.5, 크면 1에 가깝고 작으면 0에 가까워요. "확률처럼 보이는 숫자"가 필요할 때 씁니다.
- **tanh**: 시그모이드와 모양이 같은데 범위가 -1에서 1입니다. 11장 RNN·LSTM 안에서 다시 만나요.

마지막 줄처럼 `nn.ReLU()`로 만들면 층과 똑같이 다룰 수 있습니다. 다음 절에서 층과 함께 줄지어 쌓을 때 이 모양을 써요. 활성화 함수에는 파라미터가 없어서 학습으로 바뀌는 것은 없습니다.

ReLU는 그래프로 그리면 0에서 한 번 꺾인 직선이에요. "겨우 한 번 꺾였을 뿐인데 그게 그렇게 대단한가?" 싶죠. 그런데 이 꺾인 조각 수백 개를 더하고 겹치면 어떤 굽은 선이든 흉내 낼 수 있습니다. 레고 블록이 모두 네모인데도 쌓으면 둥근 성을 만들 수 있는 것과 비슷해요.

소프트맥스: 점수를 확률로

분류 모델의 마지막 층은 클래스마다 점수를 하나씩 냅니다. 이 날것의 점수를 **로짓(logit)** 이라고 불러요. 로짓은 음수일 수도 있고 합이 1도 아니라서, 사람이 읽기 좋게 확률로 바꾸고 싶을 때 **소프트맥스(softmax)** 를 씁니다.

```
import torch

# 사진 2장 × 클래스 3개(고양이, 개, 새)에 대한 모델의 점수
로짓 = torch.tensor([[2.0, 1.0, 0.1],
                     [0.5, 0.5, 3.0]])

확률 = torch.softmax(로짓, dim=1)      # 클래스 방향(dim=1)으로 합이 1
print(확률)
print("줄마다 합:", 확률.sum(dim=1))
print("고른 클래스:", 확률.argmax(dim=1), 로짓.argmax(dim=1))

print(torch.softmax(로짓 * 10, dim=1).round(decimals=4))   # 점수 차이를 10배로
print(torch.softmax(torch.tensor([1000.0, 0.0]), dim=0))    # 큰 숫자도 안전
```

```
tensor([[0.6590, 0.2424, 0.0986],
        [0.0705, 0.0705, 0.8590]])
줄마다 합: tensor([1.0000, 1.0000])
```

```
고른 클래스: tensor([0, 2]) tensor([0, 2])
tensor([[1., 0., 0.],
        [0., 0., 1.]])
tensor([1., 0.])
```

소프트맥스는 각 점수에 지수 함수를 씌운 뒤 그 합으로 나눕니다. 그래서 결과는 모두 0과 1 사이이고 **줄마다 합이 1**이에요. `dim=1`은 "클래스 방향으로" 합이 1이 되게 하라는 뜻입니다. 4장 4.6절에서 본 `dim`과 같은 의미예요. `dim=0`으로 잘못 주면 사진 방향으로 합이 1이 되어 버리니, 분류에서는 거의 항상 `dim=1`(또는 마지막 차원 `dim=-1`)입니다.

세 가지를 더 보세요.

- 확률로 바꿔도 **순서는 그대로**입니다. 가장 큰 로짓이 가장 큰 확률이 되니, 정답 클래스만 고를 때는 소프트맥스 없이 로짓에 바로 `argmax`를 써도 결과가 같아요.
- 점수 차이를 10배로 키우면 확률이 거의 0과 1로 쏠립니다. 소프트맥스는 "가장 큰 값을 부드럽게 골라 주는" 함수라서 이름에 맥스(max)가 들어 있어요.
- `[1000, 0]`처럼 지수 함수에 넣으면 넘쳐 버릴 큰 숫자도 문제없이 계산됩니다. PyTorch 안에서 가장 큰 값을 먼저 빼고 계산하는 요령을 쓰기 때문이에요. 소프트맥스를 직접 짜지 말고 `torch.softmax`를 써야 하는 이유입니다.

직선으로는 못 긋는 경계: XOR

이제 활성화 함수가 정말 필요한지 실험으로 확인해 보겠습니다. 문제는 XOR입니다. 입력 두 개가 **서로 다를 때만** 1이고 같으면 0이에요. 스위치 두 개로 전등 하나를 켜는 계단 조명을 떠올리면 됩니다. 아래층 스위치든 위층 스위치든 하나만 바꾸면 불이 켜지고, 둘 다 바꾸면 다시 꺼지죠.

네 점을 종이에 찍어 보면 (0, 1)과 (1, 0)은 1, (0, 0)과 (1, 1)은 0입니다. 대각선으로 마주 보는 점끼리 같은 편이라서, **직선 하나로는** 1인 점과 0인 점을 가를 수가 없어요. 세 가지 모델에게 각각 학습을 시켜 보겠습니다. 학습 코드는 6.7절에서 한 줄씩 뜯어볼 테니 지금은 결과만 보세요.

```
import torch
from torch import nn

# XOR: 두 입력이 다를 때만 1
X = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
y = torch.tensor([[0.0], [1.0], [1.0], [0.0]])
```

```
def 학습하기(모델: nn.Module, 단계: int = 2000) -> nn.Module:
    """XOR 네 점으로 모델을 학습한다(한 줄씩은 6.7절에서 본다)."""
    손실함수 = nn.BCEWithLogitsLoss()
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=0.05)
    for _ in range(단계):
        옵티마이저.zero_grad()
        손실 = 손실함수(모델(X), y)
        손실.backward()
        옵티마이저.step()
    return 모델


후보 = {
    "선형 한 층": lambda: nn.Linear(2, 1),
    "선형 두 층": lambda: nn.Sequential(nn.Linear(2, 4), nn.Linear(4, 1)),
    "선형+ReLU+선형": lambda: nn.Sequential(nn.Linear(2, 4), nn.ReLU(), nn.Linear(4, 1)),
}
for 이름, 만들기 in 후보.items():
    torch.manual_seed(1)
    모델 = 학습하기(만들기())
    with torch.no_grad():                        # 결과만 볼 때는 계산 기록을 끈다(6.8절)
        확률 = torch.sigmoid(모델(X)).squeeze(1)
    맞힘 = ((확률 > 0.5).float() == y.squeeze(1)).sum().item()
    print(f"{이름}: 확률 {확률.round(decimals=3).tolist()}, 맞힘 {맞힘:.0f}/4")
print("정답:", y.squeeze(1).int().tolist())
```

```
선형 한 층: 확률 [0.5, 0.5, 0.5, 0.5], 맞힘 2/4
선형 두 층: 확률 [0.5, 0.5, 0.5, 0.5], 맞힘 2/4
선형+ReLU+선형: 확률 [0.0, 1.0, 1.0, 0.0], 맞힘 4/4
정답: [0, 1, 1, 0]
```

결과가 극명합니다.

- **선형 한 층**은 네 점 모두에 확률 0.5를 냈습니다. 직선 하나로는 어떻게 그어도 넷 중 하나 이상을 틀리니, 차라리 "모르겠다(0.5)"라고 답하는 것이 손실을 가장 작게 만드는 길이었던 거

예요. 맞힌 개수는 찍은 것과 같은 2개입니다.

- **선형 두 층**도 결과가 똑같습니다. 6.1절에서 확인했듯 선형층 둘은 선형층 하나와 같으니까요. 층을 늘렸는데도 아무 소용이 없었습니다.
- **선형 + ReLU + 선형**은 확률이 소수점 셋째 자리까지 0, 1, 1, 0으로 네 점을 모두 맞혔습니다. 가운데 ReLU 하나가 공간을 접고 구부려 줘서, 직선으로는 못 긋던 경계를 그을 수 있게 된 거예요.

이것이 활성화 함수가 하는 일의 전부이자, 딥러닝이 "깊은(deep)" 층을 쌓을 수 있는 이유입니다. 선형층과 활성화 함수를 번갈아 쌓으면 층이 늘어날수록 더 복잡한 경계를 그릴 수 있어요. 이렇게 선형층과 활성화 함수를 번갈아 쌓은 가장 기본적인 신경망을 **다층 퍼셉트론**(MLP, multi-layer perceptron) 이라고 부릅니다. 이 장의 실습에서 만들 옷 사진 분류기도 MLP예요.

Q

Q. 활성화 함수는 어디에 어떤 것을 넣어야 하나요? 은닉층 사이에는 일단 ReLU를 넣으세요. 대부분 그걸로 충분합니다. 시그모이드와 소프트맥스는 은닉층이 아니라 **맨 끝에서 확률을 읽을 때** 쓰는데, 학습할 때는 모델 안에 넣지 않는 것이 보통이에요. 왜 그런지는 6.5절의 함정에서 봅니다. 위 XOR 예제에서도 모델은 로짓만 내고, 확률을 보고 싶을 때만 바깥에서 `torch.sigmoid`를 씌웠죠.

6.3 nn.Module로 모델 만들기

클래스로 설계도 그리기

층과 활성화 함수를 모아 하나의 모델로 묶을 차례입니다. PyTorch에서 모든 모델은 **`nn.Module`을 상속한 클래스**로 만듭니다. 1권 11장의 붕어빵 틀 이야기를 떠올려 보세요. 클래스는 설계도(틀)이고, 거기서 찍어 낸 객체가 실제 모델(붕어빵)입니다. 1권 11장과 13장에서 배운 상속 덕분에, 우리는 "어떤 층을 쓰고 어떤 순서로 지나가는지"만 적으면 나머지 복잡한 일은 `nn.Module`이 물려줍니다.

```
import torch
from torch import nn


class 작은신경망(nn.Module):
    """입력 → 은닉층(ReLU) → 출력으로 이어지는 신경망."""

    def __init__(self, 입력수: int, 은닉수: int, 출력수: int) -> None:
```

```
        super().__init__()                    # nn.Module의 준비 작업부터
        self.층1 = nn.Linear(입력수, 은닉수)
        self.활성화 = nn.ReLU()
        self.층2 = nn.Linear(은닉수, 출력수)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """입력이 모델 안을 지나가는 길."""
        x = self.층1(x)
        x = self.활성화(x)
        return self.층2(x)


torch.manual_seed(0)
모델 = 작은신경망(입력수=4, 은닉수=8, 출력수=3)
print(모델)

x = torch.rand(5, 4)                    # 5개 묶음 × 특성 4개
출력 = 모델(x)                           # forward를 직접 부르지 않는다
print(출력.shape)
print("forward와 같은가:", torch.equal(출력, 모델.forward(x)))
```

```
작은신경망(
  (층1): Linear(in_features=4, out_features=8, bias=True)
  (활성화): ReLU()
  (층2): Linear(in_features=8, out_features=3, bias=True)
)
torch.Size([5, 3])
forward와 같은가: True
```

모델 클래스에는 메서드 두 개만 있으면 됩니다.

- `__init__`: 쓸 부품을 준비하는 곳입니다. 첫 줄의 `super().__init__()`은 1권 11장에서 본 대로 "부모(`nn.Module`)의 초기화를 먼저 해 줘"라는 뜻이에요. `nn.Module`은 여기서 "이 모델에 딸린 층과 파라미터를 기록할 장부"를 마련합니다. 이 줄을 빼먹으면 바로 다음 줄에서 `self.층1 = ...`을 하는 순간 오류가 나요. 장부도 없는데 기록하려 했으니까요.
- `forward`: 입력이 모델 안을 **어떤 순서로 지나가는지** 적는 곳입니다. 층1 → ReLU → 층2. 이

길을 **순전파**(forward pass) 라고 불러요.

`print(모델)`을 하면 `__init__`에서 `self.`에 담아 둔 부품이 이름과 함께 나열됩니다. 속성 이름을 `층1`, `활성화`처럼 한글로 지어도 잘 동작하는 것도 보이죠.

모델을 쓸 때는 `모델.forward(x)`가 아니라 `모델(x)` 처럼 객체를 함수처럼 부릅니다. 1권 11장 특수 메서드 표에 있던 `__call__` 기억나세요? `nn.Module`이 `__call__`을 만들어 두었기 때문에 객체를 함수처럼 부를 수 있고, `__call__`은 몇 가지 준비 작업을 한 뒤 우리가 만든 `forward`를 불러 줍니다. 결과는 같지만(`True`), 준비 작업을 건너뛰지 않도록 언제나 `모델(x)`로 부르는 것이 규칙이에요.

5개 묶음 × 특성 4개가 들어가서 `(5, 3)`이 나왔습니다. 모델을 만들 때 묶음 크기는 어디에도 적지 않았는데도요. `nn.Linear`는 마지막 차원만 보고 계산하기 때문에, 묶음이 5개든 5천 개든 같은 모델이 그대로 받아 줍니다.

nn.Sequential: 줄 세우기만 할 때

`forward`가 "앞 층의 출력을 다음 층에 넣기"를 반복할 뿐이라면, 클래스를 만들 필요도 없습니다. 층을 순서대로 넣기만 하면 되는 `nn.Sequential`이 있어요.

```
import torch
from torch import nn

torch.manual_seed(0)
모델 = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 3),
)
print(모델)
print("두 번째 층:", 모델[1], "/ 층 개수:", len(모델))
print(모델(torch.rand(5, 4)).shape)

# 사진을 받는 모델: 맨 앞에서 (1, 28, 28) 사진을 784개 한 줄로 편다
사진_모델 = nn.Sequential(
    nn.Flatten(),                       # (묶음, 1, 28, 28) → (묶음, 784)
    nn.Linear(28 * 28, 64),
    nn.ReLU(),
    nn.Linear(64, 10),
```

```
)
사진들 = torch.rand(32, 1, 28, 28)
print(사진_모델[0](사진들).shape, "→", 사진_모델(사진들).shape)
```

```
Sequential(
  (0): Linear(in_features=4, out_features=8, bias=True)
  (1): ReLU()
  (2): Linear(in_features=8, out_features=3, bias=True)
)
두 번째 층: ReLU() / 층 개수: 3
torch.Size([5, 3])
torch.Size([32, 784]) → torch.Size([32, 10])
```

앞의 작은신경망과 똑같은 구조를 세 줄로 만들었습니다. 이름 대신 `0`, `1`, `2` 번호가 붙고, 리스트처럼 `모델[1]`로 꺼내거나 `len(모델)`로 층 수를 셀 수 있어요.

두 번째 모델은 사진을 받는 모양입니다. 흑백 사진 묶음은 `(묶음, 1, 28, 28)`인데 `nn.Linear`는 한 줄로 늘어선 입력을 원하죠. 그래서 맨 앞에 `nn.Flatten()` 을 두어 사진마다 784개(28 × 28)짜리 한 줄로 폅니다. 4장 4.5절의 `flatten(start_dim=1)`을 층으로 만든 것이에요. 배치 차원은 그대로 두고 나머지만 편다는 점까지 똑같습니다. 이 모양이 6.11절 실습 모델의 뼈대가 됩니다.

그렇다면 클래스와 `nn.Sequential` 중 무엇을 써야 할까요? 입력이 한 줄로 흘러가기만 하면 `nn.Sequential`이 짧고 읽기 좋습니다. 중간에 갈림길이 있거나(입력 두 개를 따로 처리해서 합치기), 같은 층을 두 번 쓰거나, `if`로 경로를 바꾸는 것처럼 **흐름을 직접 짜야 하면 클래스**로 만드세요. 실무에서는 둘을 섞어서, 클래스 안에 `nn.Sequential` 덩어리 몇 개를 두고 `forward`에서 이어 붙이는 모양을 가장 많이 봅니다. 실습의 의류분류기가 그 모양이에요.

리스트에 담으면 사라지는 층

층 개수를 바꿔 가며 실험하고 싶을 때, 층들을 리스트에 담아 반복문으로 돌리고 싶어집니다. 그런데 여기에는 아주 조용한 함정이 있어요.

```
import torch
from torch import nn
```

```
class 리스트모델(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.층들 = [nn.Linear(4, 4) for _ in range(3)]              # 그냥 리스트

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        for 층 in self.층들:
            x = torch.relu(층(x))
        return x


class 모듈리스트모델(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.층들 = nn.ModuleList([nn.Linear(4, 4) for _ in range(3)])  # 등록되는 리스트

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        for 층 in self.층들:
            x = torch.relu(층(x))
        return x


for 모델 in [리스트모델(), 모듈리스트모델()]:
    개수 = sum(p.numel() for p in 모델.parameters())
    print(f"{type(모델).__name__}: 파라미터 {개수}개, 출력 {tuple(모델(torch.rand(2, 4)).shape)}")
```

```
리스트모델: 파라미터 0개, 출력 (2, 4)
모듈리스트모델: 파라미터 60개, 출력 (2, 4)
```

두 모델 모두 출력은 잘 나옵니다. 그런데 파이썬 리스트에 담은 쪽은 **파라미터가 0개**예요. `nn.Module`의 장부는 `self.`에 `nn.Module`이나 파라미터를 직접 담을 때만 기록되는데, 파이썬 리스트는 장부가 들여다보지 않는 평범한 상자이기 때문입니다. 이 상태로 학습하면 옵티마이저가 고칠 파라미터를 하나도 받지 못해서, 오류 없이 돌아가는데 **영원히 학습되지 않는** 모델이 됩니다. 6.9절에서 볼 `.to(장치)`나 6.10절의 저장에서도 이 층들은 빠져요.

해결책은 리스트 대신 **`nn.ModuleList`** 를 쓰는 것입니다. 리스트처럼 쓰면서 장부에도 올라가서

3층 × (가중치 16개 + 편향 4개) = 60개가 제대로 잡혔어요. 딕셔너리가 필요하면 같은 이유로 `nn.ModuleDict`를 씁니다.

6.4 파라미터 들여다보기

모델을 만들면 가장 먼저 "파라미터가 몇 개인가"를 확인하는 습관을 들이세요. 모델의 크기, 필요한 메모리, 학습에 걸릴 시간을 가늠하는 첫 숫자입니다.

```
import torch
from torch import nn

모델 = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Linear(256, 10),
)
for 이름, p in 모델.named_parameters():
    print(f"{이름:9} {str(tuple(p.shape)):11} {p.numel():>8,}개")

전체 = sum(p.numel() for p in 모델.parameters())
print(f"전체 {전체:,}개 = {784 * 256 + 256 + 256 * 10 + 10:,}개")
print(f"float32로 약 {전체 * 4 / 1024**2:.2f}MB")

# 첫 층을 얼려 두면(학습하지 않게 하면)?
for p in 모델[1].parameters():
    p.requires_grad_(False)
학습_대상 = sum(p.numel() for p in 모델.parameters() if p.requires_grad)
print(f"학습 대상 {학습_대상:,}개")
```

```
1.weight  (256, 784)   200,704개
1.bias    (256,)           256개
3.weight  (10, 256)      2,560개
3.bias    (10,)             10개
전체 203,530개 = 203,530개
float32로 약 0.78MB
```

```
학습 대상 2,570개
```

`named_parameters()`는 파라미터마다 이름과 텐서를 차례로 돌려줍니다. 1권 7장에서 딕셔너리를 `.items()`로 돌던 것과 비슷하죠. 이름의 `1.weight`는 "`nn.Sequential`의 1번 부품(`nn.Linear(784, 256)`)의 가중치"라는 뜻이에요. 0번 `Flatten`과 2번 `ReLU`는 파라미터가 없어서 목록에 없습니다.

개수를 손으로 세는 법도 알아 두면 좋습니다. `nn.Linear(입력, 출력)`의 파라미터는 **입력 × 출력 + 출력**개예요. 784 × 256 + 256에 256 × 10 + 10을 더하면 `203,530`개로, `parameters()`로 센 값과 같습니다. 파라미터 하나가 float32(4바이트)이니 이 모델은 약 0.78MB예요. 숫자를 보면 첫 층 하나가 전체의 98% 넘게 차지하고 있습니다. 사진을 한 줄로 펴서 784개 입력을 모두 은닉층 256개에 하나하나 연결했기 때문인데, 이렇게 큰 연결이 사진에 꼭 필요한지는 9장 CNN에서 다시 따져 봅니다.

마지막 부분은 첫 층의 파라미터에 `requires_grad_(False)`를 걸어 **얼린** 것입니다. 얼린 파라미터는 기울기를 계산하지 않아 학습해도 바뀌지 않아요. 학습 대상이 2,570개로 줄었죠. 남이 잘 학습시켜 둔 모델의 앞부분은 얼려 두고 뒷부분만 우리 데이터에 맞게 고치는 방법이 10장 전이 학습의 핵심 요령입니다. 그래서 파라미터를 셀 때는 "전체"와 "학습 대상"을 나눠 세는 것이 좋아요.

6.5 손실 함수: 얼마나 틀렸나

모델이 예측을 냈으면 얼마나 틀렸는지 재야 고칠 수 있습니다. 다트를 던졌으면 과녁 중심에서 얼마나 빗나갔는지 재는 것과 같아요. 이 "빗나간 정도"를 숫자 하나로 만드는 것이 **손실 함수**(loss function) 이고, 학습은 이 숫자를 줄이는 방향으로 파라미터를 고치는 일입니다. 어떤 손실 함수를 쓸지는 **무엇을 예측하는가**에 따라 정해집니다.

숫자를 맞힐 때: MSELoss

집값이나 기온처럼 연속된 숫자를 맞히는 문제를 **회귀**(regression) 라고 합니다. 5장에서 손으로 썼던 평균 제곱 오차가 `nn.MSELoss`로 들어 있어요.

```
import torch
from torch import nn

예측 = torch.tensor([2.5, 0.0, 2.0, 8.0])     # 집값 예측(억 원, 예제용)
정답 = torch.tensor([3.0, -0.5, 2.0, 7.0])
```

```
손실함수 = nn.MSELoss()
print("MSE :", 손실함수(예측, 정답))
print("손으로:", ((예측 - 정답) ** 2).mean())
print("MAE :", nn.L1Loss()(예측, 정답))

# 모양이 (4, 1)과 (4,)로 어긋나면?
어긋난_예측 = 예측.unsqueeze(1)                # 모델 출력은 보통 (묶음, 1)
print("어긋난 모양:", 손실함수(어긋난_예측, 정답))
print("맞춘 모양 :", 손실함수(어긋난_예측.squeeze(1), 정답))
```

```
MSE : tensor(0.3750)
손으로: tensor(0.3750)
MAE : tensor(0.5000)
어긋난 모양: tensor(16.1562)
맞춘 모양 : tensor(0.3750)
```

`nn.MSELoss()`는 "차이를 제곱해서 평균"을 낸 값과 똑같은 `0.3750`을 돌려줍니다. 손실 함수도 `nn.MSELoss()`로 객체를 만든 뒤 `손실함수(예측, 정답)`처럼 함수처럼 부르는 모양이에요. 모델과 같은 방식이죠. 차이의 절댓값을 평균하는 `nn.L1Loss`(평균 절대 오차)도 있습니다. 제곱을 하지 않으니 크게 빗나간 값 하나에 덜 휘둘려요.

마지막 두 줄이 회귀에서 정말 자주 나는 실수입니다. 출력이 하나인 모델은 보통 `(묶음, 1)` 모양을 내는데, 정답은 `(묶음,)` 모양으로 들고 있는 경우가 많아요. 이 둘을 그대로 넣었더니 오류 없이 `16.1562`라는 엉뚱한 값이 나왔습니다. 4장 4.6절 Q&A에서 경고했던 "조용한 브로드캐스팅"이에요. `(4, 1)`과 `(4,)`가 `(4, 4)`로 펼쳐져서 예측 하나하나를 정답 네 개 모두와 비교해 버린 겁니다. 이 상태로 학습하면 모델은 모든 집값의 평균만 외우게 돼요. PyTorch가 경고 문구를 띄워 주긴 하지만 쉽게 지나치니, **손실에 넣기 전에 두 모양이 같은지** 확인하세요. `squeeze(1)`로 맞추자 제대로 된 `0.3750`이 나왔습니다.

종류를 맞힐 때: CrossEntropyLoss

사진이 고양이인지 개인지 새인지처럼 **종류를 고르는** 문제를 **분류**(classification) 라고 합니다. 분류에는 **교차 엔트로피**(cross entropy) 손실을 씁니다.

```
import torch
from torch import nn

로짓 = torch.tensor([[2.0, 1.0, 0.1],          # 사진 2장 × 클래스 3개의 점수
                     [0.5, 0.5, 3.0]])
정답 = torch.tensor([0, 2])                    # 클래스 번호(int64)

손실함수 = nn.CrossEntropyLoss()
print("CrossEntropy:", 손실함수(로짓, 정답))

# 손으로: 정답 칸의 확률에 -log를 씌워 평균
확률 = torch.softmax(로짓, dim=1)
정답_확률 = 확률[torch.arange(2), 정답]
print("정답 칸 확률:", 정답_확률)
print("손으로      :", -torch.log(정답_확률).mean())

for p in [0.99, 0.9, 0.5, 0.1, 0.01]:
    print(f"정답 확률 {p:4}일 때 손실 {-torch.log(torch.tensor(p)):.3f}")
```

```
CrossEntropy: tensor(0.2845)
정답 칸 확률: tensor([0.6590, 0.8590])
손으로      : tensor(0.2845)
정답 확률 0.99일 때 손실 0.010
정답 확률  0.9일 때 손실 0.105
정답 확률  0.5일 때 손실 0.693
정답 확률  0.1일 때 손실 2.303
정답 확률 0.01일 때 손실 4.605
```

`nn.CrossEntropyLoss`에 넣는 것은 두 가지입니다.

- **모델의 로짓**: `(묶음, 클래스 수)` 모양의 날것 점수. 소프트맥스를 **씌우지 않은** 값입니다.
- **정답 클래스 번호**: `(묶음,)` 모양의 정수(`int64`) 텐서. 0번 클래스면 0, 2번이면 2예요. 4장 4.3절 표에서 `int64`를 "정답 번호"에 쓴다고 했던 게 바로 이것입니다.

손으로 풀어 보면 교차 엔트로피가 하는 일이 보입니다. 로짓에 소프트맥스를 씌워 확률로 만들고, **정답 칸의 확률만** 골라서(`0.6590`, `0.8590`) `-log`를 씌워 평균을 냈더니 `0.2845`로 똑같았어요. 정답 칸 확률을 고를 때 쓴 `확률[torch.arange(2), 정답]`은 4장의 목록 인덱싱을 두 차원에 동시에 쓴

것으로, "0번 줄의 0번 칸, 1번 줄의 2번 칸"을 꺼냅니다.

아래 표를 보면 `-log`를 쓰는 이유가 드러납니다. 정답 확률이 0.99면 손실이 0.010으로 거의 없고, 0.5면 0.693, 0.01이면 4.605로 치솟아요. 정답을 확신할수록 손실이 0에 가까워지고, **틀린 답을 확신할수록 손실이 아주 커집니다.** 모델에게 "정답에 높은 확률을 줘라, 특히 정답을 무시하지 마라"라고 가르치는 셈이에요.

소프트맥스를 미리 씌우면 생기는 일

`nn.CrossEntropyLoss`는 안에서 소프트맥스를 직접 계산합니다. 그런데 이걸 모르고 모델 끝에 소프트맥스를 넣으면 어떻게 될까요? 0번 클래스를 점점 더 확신하는 세 모델로 비교해 보겠습니다.

```
import torch
from torch import nn

손실함수 = nn.CrossEntropyLoss()
정답 = torch.tensor([0])                    # 0번 클래스가 정답

for 점수 in [1.0, 3.0, 10.0]:
    로짓 = torch.tensor([[점수, 0.0, 0.0]])     # 점점 더 확신하는 모델
    바르게 = 손실함수(로짓, 정답)
    잘못 = 손실함수(torch.softmax(로짓, dim=1), 정답)   # 소프트맥스를 미리 씌움
    print(f"정답 점수 {점수:4}: 로짓을 넣으면 {바르게:.4f}, 확률을 넣으면 {잘못:.4f}")
```

```
정답 점수  1.0: 로짓을 넣으면 0.5514, 확률을 넣으면 0.8711
정답 점수  3.0: 로짓을 넣으면 0.0949, 확률을 넣으면 0.6113
정답 점수 10.0: 로짓을 넣으면 0.0001, 확률을 넣으면 0.5515
```

로짓을 넣으면 확신이 커질수록 손실이 `0.5514 → 0.0949 → 0.0001`로 0을 향해 갑니다. 그런데 확률을 넣으면 `0.8711 → 0.6113 → 0.5515`에서 더 내려가지 않아요. 소프트맥스를 **두 번** 씌운 셈이 되어서, 0과 1 사이로 눌린 값을 다시 소프트맥스에 넣으니 차이가 아무리 커도 `[1, 0, 0]`의 소프트맥스 이상으로는 확신할 수가 없는 겁니다. 모델이 완벽하게 맞혀도 손실은 0.55 밑으로 떨어지지 않고, 그만큼 고칠 방향(기울기)도 흐려져서 학습이 눈에 띄게 느려져요.

오류가 나지 않아서 더 무서운 함정입니다. 규칙은 하나로 기억하세요. **모델은 로짓을 내고, 손실 함수가 알아서 처리하게 둡니다.** 확률이 필요하면 학습이 끝난 뒤 바깥에서 `torch.softmax`를 씌우면 돼요.

문제 종류별로 마지막 층과 손실 함수를 정리하면 이렇습니다.

문제	마지막 층의 출력	손실 함수	정답의 모양과 자료형
회귀(숫자 하나)	`nn.Linear(…, 1)` 그대로	`nn.MSELoss`, `nn.L1Loss`	`(묶음, 1)` 실수, 예측과 같은 모양
이진 분류(예/아니오)	`nn.Linear(…, 1)` 로짓	`nn.BCEWithLogitsLoss`	`(묶음, 1)` 실수 0. 또는 1.
다중 분류(여럿 중 하나)	`nn.Linear(…, 클래스 수)` 로짓	`nn.CrossEntropyLoss`	`(묶음,)` 정수 클래스 번호

6.2절 XOR 예제에서 쓴 `nn.BCEWithLogitsLoss`가 이진 분류용입니다. 이름의 "WithLogits"가 "시그모이드는 내가 할 테니 로짓을 달라"는 뜻이에요. 소프트맥스 함정과 똑같은 이유로, 모델 끝에 시그모이드를 넣지 않습니다.

Q

Q. 학습을 시작하자마자 손실이 2.3쯤 나왔는데 정상인가요? 클래스가 10개라면 아주 정상입니다. 학습 전의 모델은 아무것도 모르니 열 클래스에 비슷한 확률, 즉 정답에 약 0.1을 줍니다. 위 표에서 정답 확률 0.1의 손실이 2.303이었죠. 그래서 **첫 손실이 `-log(1/클래스 수)` 근처인지** 확인하면 모델과 손실 함수가 제대로 연결됐는지 바로 알 수 있어요. 이진 분류라면 0.693입니다. 첫 손실이 이 값과 크게 다르면 레이블이나 모양이 잘못됐을 가능성이 큽니다.

6.6 옵티마이저: 어떻게 고칠까

step()이 하는 일

손실을 재고 `backward()`로 기울기를 구했으면, 이제 그 기울기를 보고 파라미터를 고칠 차례입니다. 5장에서는 이걸 손으로 했죠. `torch.no_grad()` 안에서 `w -= 학습률 * w.grad`를 파라미터마다 쓰고, 끝나면 `w.grad.zero_()`로 지웠습니다. **옵티마이저**(optimizer)는 이 일을 모든 파라미터에 한꺼번에 해 주는 부품이에요.

```
import torch
from torch import nn
```

```
torch.manual_seed(0)
층 = nn.Linear(2, 1)
옵티마이저 = torch.optim.SGD(층.parameters(), lr=0.1)

x = torch.tensor([[1.0, 2.0]])
y = torch.tensor([[1.0]])
손실 = nn.MSELoss()(층(x), y)
손실.backward()                                  # 기울기 계산(5장)

이전 = 층.weight.detach().clone()
기울기 = 층.weight.grad.clone()
옵티마이저.step()                                 # 파라미터 고치기

print("고치기 전:", 이전)
print("기울기   :", 기울기)
print("고친 뒤  :", 층.weight.detach())
print("이전 - 0.1 × 기울기와 같은가:", torch.allclose(층.weight.detach(), 이전 - 0.1 * 기울기))
```

```
고치기 전: tensor([[-0.0053,  0.3793]])
기울기   : tensor([[-1.6573, -3.3145]])
고친 뒤  : tensor([[0.1604, 0.7108]])
이전 - 0.1 × 기울기와 같은가: True
```

옵티마이저를 만들 때 `층.parameters()`를 넘겨서 "네가 고칠 파라미터는 이것들이다"라고 알려 줍니다. 6.3절에서 리스트에 담은 층의 파라미터가 사라지면 안 되는 이유가 여기 있어요. 여기서 빠진 파라미터는 옵티마이저가 존재조차 모릅니다.

`옵티마이저.step()`을 부르자 가중치가 `[-0.0053, 0.3793]`에서 `[0.1604, 0.7108]`로 바뀌었고, 이 값은 "이전 값 − 0.1 × 기울기"와 정확히 같습니다. 가장 기본인 **SGD(확률적 경사하강법, stochastic gradient descent)** 는 5장에서 손으로 하던 경사하강법 그대로예요. 이름의 "확률적"은 데이터 전체가 아니라 무작위로 뽑은 일부(5장 5.11절의 배치 1이나 미니배치)로 기울기를 구한다는 뜻입니다.

골짜기를 내려가는 세 가지 방법

5장에서 경사하강법을 안개 낀 산에서 내려오는 일에 비유했죠. 그리고 5.10절에서 가파른 방향과 완만한 방향이 섞여 있으면 학습률 하나로는 둘 다 만족시킬 수 없다는 것도 봤습니다. 실제 손실의 지형은 이렇게 한쪽은 가파르고 한쪽은 완만한 **길쭉한 골짜기** 모양인 경우가 많아요. 그런 골짜기를 만들어 놓고 옵티마이저 넷이 얼마나 빨리 바닥에 닿는지 비교해 보겠습니다. 높이 함수는 $x^2 + 50y^2$로, 남북(y) 방향이 동서(x) 방향보다 50배 가파르고 바닥은 (0, 0)이에요.

```
import torch


def 골짜기(점: torch.Tensor) -> torch.Tensor:
    """동서로 길고 남북으로 가파른 골짜기. 가장 낮은 곳은 (0, 0)."""
    return 점[0] ** 2 + 50 * 점[1] ** 2


def 내려가기(옵티마이저_만들기, 단계: int = 100) -> tuple[list[float], list[float]]:
    """(-4, 1)에서 출발해 골짜기 바닥으로 내려가며 높이를 기록한다."""
    점 = torch.tensor([-4.0, 1.0], requires_grad=True)
    옵티마이저 = 옵티마이저_만들기([점])
    기록 = []
    for _ in range(단계):
        옵티마이저.zero_grad()
        높이 = 골짜기(점)
        높이.backward()
        옵티마이저.step()
        기록.append(골짜기(점).item())
    return 기록, 점.detach().tolist()


후보 = {
    "SGD lr=0.01         ": lambda p: torch.optim.SGD(p, lr=0.01),
    "SGD lr=0.03         ": lambda p: torch.optim.SGD(p, lr=0.03),
    "SGD+momentum lr=0.01": lambda p: torch.optim.SGD(p, lr=0.01, momentum=0.9),
    "Adam lr=0.1         ": lambda p: torch.optim.Adam(p, lr=0.1),
}
print(" " * 21, "  10단계    30단계   100단계  마지막 위치")
```

```
for 이름, 만들기 in 후보.items():
    기록, 위치 = 내려가기(만들기)
    print(f"{이름} {기록[9]:9.4g} {기록[29]:9.4g} {기록[99]:9.4g}  ({위치[0]:.3f}, {위치[1]:.3g})")
```

```
                          10단계    30단계   100단계  마지막 위치
SGD lr=0.01                10.68     4.761    0.2814  (-0.530, 0)
SGD lr=0.03           5.243e+07 5.765e+19       inf  (-0.008, 1.27e+30)
SGD+momentum lr=0.01      3.046    0.5975 0.0003772  (-0.017, 0.00135)
Adam lr=0.1                9.35     1.685 0.0008905  (0.021, 0.00294)
```

이 결과에 옵티마이저 이야기가 거의 다 들어 있습니다.

- SGD lr=0.01: 100단계 뒤 높이 `0.2814`, 마지막 위치가 `(-0.530, 0)`입니다. 가파른 y 방향은 금세 바닥(0)에 닿았는데, 완만한 x 방향은 출발점 -4에서 아직 -0.53까지밖에 못 왔어요. 완만한 곳에서는 기울기가 작으니 걸음도 작아지기 때문입니다.
- SGD lr=0.03: "그럼 학습률을 키우면 되지 않나?" 싶어 세 배로 키웠더니 높이가 `inf`(무한대)로 날아갔습니다. 가파른 방향에서 너무 큰 걸음을 내디뎌 골짜기 반대편 벽으로 튀어 오르고, 거기서 더 크게 튀기를 반복한 거예요. 5장에서 학습률이 너무 크면 발산한다고 했던 그 현상입니다. SGD는 **가장 가파른 방향에 맞춰** 학습률을 작게 잡을 수밖에 없고, 그래서 완만한 방향이 느려집니다.
- SGD + momentum: 같은 학습률 0.01에 `momentum=0.9`만 더했더니 10단계 만에 `3.046`, 100단계 뒤 `0.0003772`로 SGD보다 훨씬 빨리 내려갔습니다. **모멘텀(momentum, 관성)** 은 지난 걸음의 방향을 기억해서 이번 걸음에 더합니다. 비탈을 굴러 내려가는 공처럼, 같은 방향으로 계속 내려가면 점점 빨라지고 좌우로 흔들리는 움직임은 서로 상쇄돼요.
- Adam lr=0.1: 100단계 뒤 `0.0008905`로 역시 바닥 근처에 왔습니다. Adam은 모멘텀에 더해 **파라미터마다 걸음 크기를 따로 맞춥니다.** 기울기가 늘 큰 방향은 걸음을 줄이고 늘 작은 방향은 키워서, 가파른 방향과 완만한 방향을 비슷한 속도로 내려가요.

어떤 옵티마이저가 항상 이긴다는 정답은 없습니다. 이 골짜기에서는 모멘텀이 가장 빨랐지만, 6.11절 실습에서 실제 옷 사진으로 비교하면 결과가 또 달라요. 다만 처음 시작할 때는 **Adam에 학습률 0.001**이 무난한 출발점이라는 것이 많은 사람의 경험이라, 이 책의 예제도 대부분 그렇게 시작합니다.

Q. 학습률은 어떻게 정하나요? 정답은 데이터와 모델마다 달라서 결국 실험으로 찾습니다. 출발점으로는 Adam이면 0.001, SGD+momentum이면 0.01~0.1을 많이 써요. 손실이 `nan`이나 `inf`로 튀면 줄이고, 너무 천천히 줄면 키워 봅니다. 학습 도중에 학습률을 점점 줄여 가는 스케줄러는 8장, 여러 값을 체계적으로 시험하는 방법은 18장에서 다룹니다.

6.7 표준 학습 루프: 다섯 줄의 반복

다트 연습 한 번에 다섯 동작

이제 부품이 모두 모였습니다. 다트 연습을 떠올려 보세요. 한 번 연습할 때마다 이렇게 합니다.

1. 지난번에 고친 기억은 털어 버리고(**기울기 지우기**)
2. 다트를 던지고(**순전파**)
3. 과녁 중심에서 얼마나 빗나갔는지 재고(**손실 계산**)
4. 팔 각도와 힘을 어느 쪽으로 고쳐야 할지 생각하고(**역전파**)
5. 실제로 자세를 고칩니다(**파라미터 갱신**)

이 다섯 동작이 PyTorch 학습 루프의 다섯 줄입니다. 먼저 5장 5.7절에서 손으로 학습했던 아이스크림 판매 회귀를 이 모양으로 옮겨 보겠습니다. 5장에서는 `w`와 `b`를 하나씩 고치고 하나씩 비웠는데, 이번에는 이 장에서 배운 부품만 씁니다.

```
import numpy as np
import torch
from torch import nn

# 5장 5.7절의 아이스크림 판매 데이터(예제용으로 만든 데이터)
rng = np.random.default_rng(0)
기온 = rng.uniform(18, 34, size=30).round(1)
판매량 = (2.5 * 기온 - 15 + rng.normal(0, 4, size=30)).round()
x = torch.tensor((기온 - 기온.mean()) / 기온.std(), dtype=torch.float32).unsqueeze(1)  # (30, 1)
y = torch.tensor(판매량, dtype=torch.float32).unsqueeze(1)                             # (30, 1)

모델 = nn.Linear(1, 1)                          # w * x + b
nn.init.zeros_(모델.weight)                      # 5장처럼 0에서 출발
```

```
nn.init.zeros_(모델.bias)
손실함수 = nn.MSELoss()
옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.1)

기록 = []
for _ in range(30):
    옵티마이저.zero_grad()
    손실 = 손실함수(모델(x), y)
    손실.backward()
    옵티마이저.step()
    기록.append(손실.item())

print([round(v, 1) for v in 기록[::5]], "→", round(기록[-1], 1))
print(f"w = {모델.weight.item():.3f}, b = {모델.bias.item():.3f}")
```

```
[2930.5, 327.2, 47.7, 17.7, 14.5, 14.1] → 14.1
w = 12.640, b = 52.435
```

손실이 `2930.5 → 327.2 → 47.7 → 17.7 → 14.5 → 14.1`로, 5장 5.7절에서 손으로 비웠을 때의 숫자와 **한 자리도 다르지 않습니다.** `w * x + b`는 `nn.Linear(1, 1)`이 되었고(`nn.init.zeros_`로 5장처럼 0에서 출발시켰어요), 손으로 쓴 평균 제곱 오차는 `nn.MSELoss`, `no_grad` 안에서 고치고 비우던 네 줄은 `step()`과 `zero_grad()` 두 줄이 됐습니다. 겉모습만 바뀌었을 뿐 안에서 일어나는 계산은 똑같다는 뜻이에요. `x`와 `y`를 `unsqueeze(1)`로 `(30, 1)`로 만든 것도 보세요. `nn.Linear`는 `(묶음, 특성 수)` 모양을 받고, 6.5절에서 본 대로 예측과 정답의 모양을 맞춰야 하니까요.

`zero_grad()`를 다섯 줄의 맨 앞에 둔 것도 보세요. 5.7절에서는 비우기를 반복의 맨 끝에 뒀지만, `backward()` 전에만 비우면 결과가 같다고 했죠. 실무 코드는 대부분 이렇게 맨 앞에 둡니다. 5.7절에서 기울기를 비우지 않았더니 손실이 15.8까지 내려갔다가 2987.0으로 다시 치솟았죠. 옵티마이저를 써도 이 성질은 같아서, `zero_grad()`를 빼먹으면 오류 없이 학습만 조용히 망가집니다. 학습이 이상하게 출렁이면 이 줄부터 확인하세요.

미니배치로 분류 모델 학습하기

이번에는 분류 문제를 미니배치로 학습해 보겠습니다. 예제용으로 만든 동심원 데이터(안쪽 원은 0번, 바깥 고리는 1번)예요. 직선으로는 절대 가를 수 없는 모양이라 6.2절의 활성화 함수가 꼭 필요한 문제입니다.

```
import torch
from torch import nn


def 동심원_만들기(개수: int) -> tuple[torch.Tensor, torch.Tensor]:
    """안쪽 원(0번)과 바깥 고리(1번)에 흩어진 점. 예제용으로 만든 데이터."""
    반지름 = torch.cat([torch.rand(개수 // 2), 1.5 + torch.rand(개수 // 2)])
    각도 = torch.rand(개수) * 2 * torch.pi
    X = torch.stack([반지름 * torch.cos(각도), 반지름 * torch.sin(각도)], dim=1)
    X = X + 0.3 * torch.randn(개수, 2)               # 경계가 겹치도록 흔들기
    y = torch.cat([torch.zeros(개수 // 2), torch.ones(개수 // 2)]).long()
    return X, y


torch.manual_seed(0)
X, y = 동심원_만들기(1000)
순서 = torch.randperm(1000)                      # 섞은 뒤 800개 학습, 200개 시험
X_학습, y_학습 = X[순서[:800]], y[순서[:800]]
X_시험, y_시험 = X[순서[800:]], y[순서[800:]]

모델 = nn.Sequential(nn.Linear(2, 16), nn.ReLU(), nn.Linear(16, 16), nn.ReLU(), nn.Linear(16, 2))
손실함수 = nn.CrossEntropyLoss()
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=0.01)
묶음_크기 = 32

for 에폭 in range(1, 21):
    모델.train()
    섞기 = torch.randperm(800)
    합계 = 0.0
    for 시작 in range(0, 800, 묶음_크기):
        번호 = 섞기[시작:시작 + 묶음_크기]
        xb, yb = X_학습[번호], y_학습[번호]

        옵티마이저.zero_grad()            # 1) 지난 기울기 지우기
        출력 = 모델(xb)                    # 2) 순전파: 예측하기
        손실 = 손실함수(출력, yb)          # 3) 얼마나 틀렸나 재기
        손실.backward()                   # 4) 역전파: 고칠 방향 계산
```

```
        옵티마이저.step()                  # 5) 파라미터 고치기

        합계 += 손실.item() * len(번호)
    if 에폭 in (1, 5, 10, 20):
        모델.eval()
        with torch.no_grad():
            정확도 = (모델(X_시험).argmax(dim=1) == y_시험).float().mean().item()
        print(f"에폭 {에폭:2}: 학습 손실 {합계 / 800:.4f}, 시험 정확도 {정확도:.1%}")
```

```
에폭  1: 학습 손실 0.5459, 시험 정확도 95.5%
에폭  5: 학습 손실 0.0660, 시험 정확도 95.5%
에폭 10: 학습 손실 0.0619, 시험 정확도 96.5%
에폭 20: 학습 손실 0.0563, 시험 정확도 96.5%
```

1에폭 만에 시험 정확도 95.5%, 20에폭 뒤 학습 손실 `0.0563`, 시험 정확도 96.5%가 됐습니다. 점에 일부러 잡음을 섞어 두 고리가 조금 겹치게 만들었기 때문에, 겹친 부분의 점까지 다 맞히기는 어려워요. 그래도 원 모양 경계를 잘 찾아냈습니다.

코드의 짜임을 위에서부터 보겠습니다.

- **데이터 나누기**: `randperm`으로 순서를 섞은 뒤 800개는 학습, 200개는 시험용으로 나눴습니다. 모델이 학습 중에 한 번도 보지 못한 데이터로 정확도를 재야 "외웠는지, 배웠는지"를 알 수 있어요. 데이터를 나누는 원칙은 2장에서 다룬 그대로입니다.
- **준비 세 줄**: 모델, 손실 함수, 옵티마이저를 만듭니다. 분류 문제이니 `CrossEntropyLoss`, 옵티마이저는 Adam이에요.
- **에폭 반복**: 5장 5.11절에서 본 대로 학습 데이터 전체를 한 바퀴 도는 것이 **에폭(epoch)** 입니다. 20에폭이면 같은 데이터를 스무 번 본 거예요.
- **미니배치 반복**: 한 에폭 안에서 데이터를 32개씩 잘라 그 묶음마다 파라미터를 고칩니다(5장의 미니배치 경사하강법). 에폭마다 `randperm`으로 순서를 다시 섞어서, 모델이 데이터 순서를 외우지 않게 해요. 800개를 32개씩이면 한 에폭에 25번 고치게 됩니다.
- **다섯 줄**: 주석에 번호를 단 그 다섯 줄이 이 장의 핵심입니다. 이 책의 모든 학습 코드에서 이 다섯 줄을 다시 만나요.

`손실.item()`은 손실 텐서에서 파이썬 숫자 하나를 꺼냅니다. 에폭 평균 손실을 내려고 묶음 손실에 묶음 크기를 곱해 더해 두었어요. 손실 텐서를 그대로 더해 두면 계산 기록까지 줄줄이 붙들고 있게 되니, 기록용으로 모을 때는 꼭 `.item()`으로 숫자만 꺼내세요.

데이터를 직접 `randperm`으로 섞고 잘랐는데, 실무에서는 이 일을 `DataLoader`가 대신해 줍니다. 6.11절 실습에서 미리 한 번 써 보고, 자세한 사용법은 7장에서 다룹니다.

> **Q. 에폭, 배치, 스텝은 어떻게 다른가요?** **배치(묶음)** 는 한 번에 모델에 넣는 데이터 덩어리, **스텝** 은 옵티마이저가 파라미터를 한 번 고치는 것, **에폭** 은 학습 데이터 전체를 한 바퀴 도는 것입니다. 위 예제는 배치 크기 32, 한 에폭에 25스텝, 20에폭이니 모두 500스텝을 고쳤어요. 논문이나 다른 사람의 코드에서 "몇 스텝 학습했다"와 "몇 에폭 학습했다"가 섞여 나오니 구분해 두세요.

6.8 평가 루프: 학습 모드와 평가 모드

model.train()과 model.eval()

학습이 끝난 모델로 시험을 칠 때는 학습할 때와 다르게 동작해야 하는 층이 있습니다. 대표적인 것이 **드롭아웃(dropout)** 이에요. 학습 중에 일부 출력을 무작위로 꺼서, 모델이 특정 뉴런 몇 개에만 기대지 않게 하는 장치입니다. 왜 그게 도움이 되는지는 8장의 주제이니, 여기서는 모드에 따라 동작이 달라지는 것만 보겠습니다.

```
import torch
from torch import nn

torch.manual_seed(0)
모델 = nn.Sequential(nn.Linear(4, 4), nn.Dropout(p=0.5))   # 드롭아웃: 8장에서 자세히
x = torch.ones(1, 4)

모델.train()                                    # 학습 모드(처음 만들면 이 모드)
print("학습 모드:", 모델.training)
print(모델(x))
print(모델(x))                                   # 같은 입력인데 결과가 다르다

모델.eval()                                     # 평가 모드
print("평가 모드:", 모델.training, 모델[1].training)
print(모델(x))
print(모델(x))
```

```
학습 모드: True
tensor([[-0.6347,  1.2561, -1.0009, -0.0000]], grad_fn=<MulBackward0>)
tensor([[-0.6347,  1.2561, -1.0009, -2.4283]], grad_fn=<MulBackward0>)
평가 모드: False False
tensor([[-0.3173,  0.6281, -0.5004, -1.2141]], grad_fn=<AddmmBackward0>)
tensor([[-0.3173,  0.6281, -0.5004, -1.2141]], grad_fn=<AddmmBackward0>)
```

학습 모드에서는 같은 입력을 두 번 넣었는데 결과가 달랐습니다. 첫 번째는 마지막 칸이 0으로 꺼졌고, 두 번째는 모두 살아 있었죠. `p=0.5`라서 매번 절반쯤을 무작위로 끄기 때문이에요. 살아남은 값은 끈 만큼을 보충하려고 두 배로 커집니다(`-0.6347`은 평가 모드 값 `-0.3173`의 두 배예요).

`모델.eval()`로 평가 모드로 바꾸자 드롭아웃이 꺼져서 같은 입력에 언제나 같은 결과가 나왔습니다. 시험 문제를 풀 때마다 답이 바뀌면 곤란하니까요. `eval()`은 모델 안의 모든 층에 모드를 전달해서, 바깥 모델과 안쪽 `Dropout` 층의 `training`이 모두 `False`가 됐습니다.

규칙은 간단합니다. **학습 전에는 `모델.train()`, 평가 전에는 `모델.eval()`.** 6.7절의 학습 루프도 에폭마다 `모델.train()`으로 시작하고, 평가하기 전에 `모델.eval()`을 불렀어요. 지금 모델에 드롭아웃이 없어도 이 습관을 들여 두세요. 8장에서 배울 배치 정규화도 모드에 따라 다르게 동작해서, 나중에 층을 하나 추가했을 때 결과가 이상해지는 일을 막아 줍니다.

torch.no_grad()와 평가 함수

평가할 때 꺼야 할 것이 하나 더 있습니다. 바로 계산 기록이에요.

```
import torch
from torch import nn

torch.manual_seed(0)
모델 = nn.Linear(1000, 1000)
x = torch.rand(64, 1000)

출력 = 모델(x)
print("기록함   :", 출력.requires_grad, type(출력.grad_fn).__name__)

with torch.no_grad():
    출력 = 모델(x)
print("기록 안 함:", 출력.requires_grad, 출력.grad_fn)
```

```
def 정확도_재기(모델: nn.Module, X: torch.Tensor, y: torch.Tensor) -> float:
    """분류 모델의 정확도를 잰다. 평가 루프의 표준형."""
    모델.eval()                                  # 1) 평가 모드로
    with torch.no_grad():                        # 2) 계산 기록 끄기
        예측 = 모델(X).argmax(dim=1)             # 3) 가장 높은 점수의 클래스
    return (예측 == y).float().mean().item()


분류기 = nn.Sequential(nn.Linear(2, 8), nn.ReLU(), nn.Linear(8, 3))
X = torch.randn(100, 2)
y = torch.randint(0, 3, (100,))
print(f"학습 전 정확도: {정확도_재기(분류기, X, y):.0%}")
print("다시 학습 모드로 되돌렸나:", 분류기.training)
```

```
기록함   : True AddmmBackward0
기록 안 함: False None
학습 전 정확도: 30%
다시 학습 모드로 되돌렸나: False
```

평소에는 출력에 `AddmmBackward0` 같은 계산 기록이 붙지만, `with torch.no_grad():` 블록 안에서 계산하면 `requires_grad`가 `False`이고 기록(`grad_fn`)이 `None`입니다. 5장에서 본 그 `no_grad`예요. 평가할 때는 `backward()`를 하지 않으니 기록할 필요가 없고, 기록을 안 하면 메모리를 아끼고 계산도 빨라집니다. `with` 문은 1권 11장에서 본 컨텍스트 매니저라서, 블록을 벗어나면 기록이 자동으로 다시 켜져요.

여기서 꼭 구분할 것이 있습니다. **`eval()`과 `no_grad()`는 하는 일이 다릅니다.** `eval()`은 드롭아웃 같은 층의 **동작**을 바꾸고, `no_grad()`는 **기울기 기록**을 끕니다. 하나만 해서는 안 되고, 평가할 때는 둘 다 합니다. 그래서 평가 루프의 표준형은 `정확도_재기` 함수처럼 "평가 모드로 → 기록 끄기 → 예측"의 세 단계예요. 예측은 로짓에 `argmax(dim=1)`을 씌워 고르고, 정답과 같은 칸을 세어 평균을 냅니다.

학습 전 모델의 정확도는 30%로, 클래스 셋 중에 찍었을 때 기대할 수 있는 33%와 비슷합니다. 마지막 줄을 보면 `정확도_재기`가 끝난 뒤에도 모델이 평가 모드로 남아 있어요(`False`). 그래서 학습 루프는 에폭을 시작할 때마다 `모델.train()`을 다시 불러야 합니다. 평가 함수 안에서 모드를 바꿨

다는 사실을 잊고 학습을 이어 가면, 드롭아웃이 꺼진 채로 학습하게 돼요.

Q

Q. `no_grad()`를 깜빡하면 평가 결과가 틀리게 나오나요? 아니요, 숫자는 똑같이 나옵니다. 다만 쓸데없이 계산 기록을 쌓느라 메모리를 더 쓰고 느려져요. 큰 모델이나 큰 데이터로 평가하다 메모리가 모자라서 멈춘다면 `no_grad()`를 빼먹지 않았는지 의심해 보세요. 5장 5.8절의 `torch.inference_mode()`를 써도 되는데, 예측만 하는 서비스 코드(20장)에서 조금 더 빠르게 쓰입니다.

6.9 장치로 옮기기

4장 4.8절에서 텐서를 GPU로 옮겼죠. 모델도 똑같이 `.to(장치)`로 옮깁니다. 이 책의 모든 예제에서 쓰는 `장치_고르기()` 함수를 그대로 가져왔어요.

```
import torch
from torch import nn


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")


장치 = 장치_고르기()
torch.manual_seed(0)
모델 = nn.Linear(3, 1)
옮긴_모델 = 모델.to(장치)                    # 모델은 제자리에서 옮겨진다
print("같은 객체인가:", 옮긴_모델 is 모델)
print("파라미터 위치:", next(모델.parameters()).device)

x = torch.rand(2, 3)                      # 데이터는 아직 CPU에
try:
    모델(x)
```

```
except RuntimeError as e:
    print("오류:", e)

x = x.to(장치)                                  # 텐서는 새 텐서를 돌려주니 다시 받는다
print(모델(x).detach().cpu())       # 결과를 CPU로 가져와 보기
```

```
같은 객체인가: True
파라미터 위치: mps:0
오류: Tensor for argument input is on cpu but expected on mps
tensor([[-0.4627],
        [-0.5881]])
```

이 책을 쓴 맥에서는 `mps`가 골라져서 모델의 파라미터가 `mps:0`으로 옮겨졌습니다. GPU가 없는 컴퓨터라면 `cpu`로 나오고 오류 없이 그대로 지나가요.

여기서 4장의 텐서와 다른 점이 하나 있습니다. 텐서의 `.to()`는 새 텐서를 돌려줘서 `x = x.to(장치)`처럼 다시 받아야 했죠. 그런데 **모델의 `.to()`는 모델 자신을 옮기고** 자기 자신을 돌려줍니다. 그래서 `옮긴_모델 is 모델`이 `True`예요. `모델.to(장치)`라고만 써도 되지만, 텐서와 헷갈리지 않게 `모델 = 모델.to(장치)`나 `모델 = 작은신경망().to(장치)`처럼 쓰는 사람이 많습니다.

모델은 GPU에 있는데 입력 `x`가 CPU에 있으니, 4장에서 본 "같은 장치에 있어야 한다"는 오류가 났습니다. 그래서 학습 루프와 평가 루프 안에서 묶음을 꺼낼 때마다 `x, y = x.to(장치), y.to(장치)`를 해 주는 것이 표준형이에요. 실습 코드의 루프 첫 줄이 바로 이것입니다. 결과를 출력하거나 NumPy로 넘길 때는 `.cpu()`로 다시 가져오는데, 계산 기록이 붙은 텐서는 `.detach()`로 기록을 떼고 가져옵니다(5장).

Q

Q. 옵티마이저를 먼저 만들고 모델을 옮겨도 되나요? 순서는 **모델을 옮긴 다음 옵티마이저를 만드는 것**이 안전합니다. 옵티마이저는 만들 때 받은 파라미터 객체를 기억합니다. 지금의 PyTorch는 `.to()`로 옮겨도 대부분 같은 파라미터 객체를 그대로 두어서 순서를 바꿔도 돌아가는 경우가 많지만, 예전 버전이나 일부 특수한 설정에서는 옮기면서 새 객체가 만들어져 옵티마이저가 옛 파라미터를 붙잡고 있게 돼요. PyTorch 공식 문서도 모델을 옮긴 다음에 옵티마이저를 만들라고 권합니다. "모델 만들기 → 장치로 옮기기 → 옵티마이저 만들기" 순서를 습관으로 삼으세요. 실습 코드도 이 순서입니다.

6.10 모델 저장과 불러오기

state_dict: 모델의 숫자만 담은 딕셔너리

몇 시간 동안 학습시킨 모델을 프로그램이 끝날 때 버릴 수는 없겠죠. 학습한 결과는 결국 **파라미터의 숫자**이니, 그 숫자만 저장해 두면 됩니다. PyTorch는 모델의 숫자를 `state_dict()`라는 딕셔너리로 꺼내 줘요.

```
import torch
from torch import nn


class 작은신경망(nn.Module):
    """입력 → 은닉층(ReLU) → 출력으로 이어지는 신경망."""

    def __init__(self, 입력수: int, 은닉수: int, 출력수: int) -> None:
        super().__init__()
        self.층1 = nn.Linear(입력수, 은닉수)
        self.활성화 = nn.ReLU()
        self.층2 = nn.Linear(은닉수, 출력수)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.층2(self.활성화(self.층1(x)))


torch.manual_seed(0)
모델 = 작은신경망(4, 8, 3)
상태 = 모델.state_dict()                          # 이름 → 텐서 딕셔너리
for 이름, 값 in 상태.items():
    print(f"{이름:10} {tuple(값.shape)}")
torch.save(상태, "작은신경망.pt")

새_모델 = 작은신경망(4, 8, 3)                      # 같은 설계도로 새로 만들면 숫자가 다르다
x = torch.rand(2, 4)
print("불러오기 전 같은가:", torch.equal(새_모델(x), 모델(x)))
새_모델.load_state_dict(torch.load("작은신경망.pt", weights_only=True))
print("불러온 뒤 같은가  :", torch.equal(새_모델(x), 모델(x)))
```

```
다른_모델 = 작은신경망(4, 16, 3)                    # 설계도가 다르면?
try:
    다른_모델.load_state_dict(torch.load("작은신경망.pt", weights_only=True))
except RuntimeError as e:
    print("오류:", str(e).split("\n")[1].strip())
```

```
층1.weight  (8, 4)
층1.bias    (8,)
층2.weight  (3, 8)
층2.bias    (3,)
불러오기 전 같은가: False
불러온 뒤 같은가  : True
오류: size mismatch for 층1.weight: copying a param with shape torch.Size([8, 4]) from checkpoint,
↪  the shape in current model is torch.Size([16, 4]).
```

`state_dict()`는 "파라미터 이름 → 텐서" 딕셔너리입니다. 이름이 `층1.weight`처럼 `__init__`에서 붙인 속성 이름을 따라가서, 한글 이름이 그대로 열쇠가 됐어요. 이 딕셔너리를 `torch.save`로 파일에 씁니다.

불러올 때는 순서가 중요합니다. 파일에는 숫자만 있고 설계도(클래스)는 없으니, **먼저 같은 설계도로 모델을 만들고** 거기에 숫자를 `load_state_dict`로 채워 넣어요. 새로 만든 모델은 무작위 값이라 원래 모델과 출력이 달랐는데(`False`), 숫자를 채우자 똑같아졌습니다(`True`).

설계도가 다르면 어떻게 될까요? 은닉층을 8개가 아니라 16개로 만든 모델에 불러오려 하자, "`층1.weight`의 모양이 파일에서는 `[8, 4]`인데 지금 모델에서는 `[16, 4]`"라는 오류가 났습니다. 이름과 모양이 하나라도 어긋나면 이렇게 알려 주니, 오류 메시지의 이름과 모양을 보면 어느 층이 다른지 바로 찾을 수 있어요.

`torch.load`에 준 `weights_only=True`도 꼭 기억하세요. PyTorch의 저장 파일은 1권 12장에서 본 pickle을 바탕으로 하는데, 그때 "믿을 수 없는 곳에서 받은 pickle 파일은 절대 열지 말라"고 했죠. `weights_only=True`를 주면 텐서와 딕셔너리 같은 안전한 것만 읽고 실행될 수 있는 코드는 거부합니다. 최신 PyTorch는 이것이 기본값이지만, 의도를 분명히 하려고 이 책에서는 늘 적어 둡니다.

이어서 학습하기: 체크포인트

예측에만 쓸 거라면 모델의 `state_dict`로 충분합니다. 그런데 학습을 중간에 멈췄다가 **이어서** 하고 싶다면 더 필요한 것이 있어요.

```
import torch
from torch import nn

torch.manual_seed(0)
모델 = nn.Linear(3, 1)
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=0.01)
x, y = torch.rand(16, 3), torch.rand(16, 1)
for _ in range(5):                              # 5에폭 학습했다고 하자
    옵티마이저.zero_grad()
    nn.functional.mse_loss(모델(x), y).backward()
    옵티마이저.step()

# 이어서 학습할 수 있게 필요한 것을 한 상자에
torch.save({
    "에폭": 5,
    "모델": 모델.state_dict(),
    "옵티마이저": 옵티마이저.state_dict(),
}, "체크포인트.pt")

# 다음 날: 같은 설계도로 만들고 상자를 열어 채운다
새_모델 = nn.Linear(3, 1)
새_옵티마이저 = torch.optim.Adam(새_모델.parameters(), lr=0.01)
상자 = torch.load("체크포인트.pt", weights_only=True)
새_모델.load_state_dict(상자["모델"])
새_옵티마이저.load_state_dict(상자["옵티마이저"])
print("이어서 시작할 에폭:", 상자["에폭"] + 1)
print("옵티마이저가 기억하는 것:", list(상자["옵티마이저"]["state"][0].keys()))
```

```
이어서 시작할 에폭: 6
옵티마이저가 기억하는 것: ['step', 'exp_avg', 'exp_avg_sq']
```

옵티마이저에도 `state_dict()`가 있습니다. 마지막 줄을 보면 Adam은 파라미터마다 `exp_avg`(지난

기울기의 평균, 6.6절의 모멘텀에 해당)와 `exp_avg_sq`(기울기 크기의 평균, 걸음 크기 조절용)를 기억하고 있어요. 모델 숫자만 불러오고 옵티마이저를 새로 만들면 이 기억이 사라져서, 이어서 학습할 때 처음 몇 스텝이 출렁일 수 있습니다.

그래서 이어서 학습할 용도로는 **에폭 번호, 모델 상태, 옵티마이저 상태**를 딕셔너리 하나에 담아 저장합니다. 1권 12장에서 pickle로 변수 여러 개를 딕셔너리 하나에 묶어 저장하던 것과 같은 요령이에요. 이렇게 학습 도중에 저장한 파일을 **체크포인트(checkpoint)** 라고 부릅니다. 게임의 세이브 포인트처럼, 컴퓨터가 꺼져도 마지막 저장 지점부터 다시 시작할 수 있어요. 체크포인트를 언제, 무엇과 함께 남길지는 18장 실험 관리에서 더 다룹니다.

Q

Q. **`torch.save(모델)`로 모델을 통째로 저장하면 안 되나요?** 저장은 되지만 권하지 않습니다. 통째로 저장하면 클래스가 어느 파일에 있었는지까지 함께 기록돼서, 나중에 파일 위치나 클래스 이름을 바꾸면 불러오지 못해요. 게다가 `weights_only=True`로는 열 수 없어서 안전하게 불러오기도 어렵습니다. **숫자는 `state_dict`로 저장하고, 설계도는 코드로 남기는 것**이 PyTorch가 권하는 방법이에요.

6.11 실습: FashionMNIST 옷 사진 분류기

데이터 둘러보기

이제 진짜 데이터로 처음부터 끝까지 만들어 볼 차례입니다. FashionMNIST는 옷과 신발, 가방 사진 7만 장을 모은 데이터셋이에요. 의류 쇼핑몰 잘란도(Zalando)가 MIT 라이선스로 공개한 것으로, 28 × 28 크기의 흑백 사진에 열 가지 종류 중 하나가 정답으로 붙어 있습니다. PyTorch의 사진 도구 꾸러미인 `torchvision`으로 바로 받을 수 있어요(`uv add torchvision`). 처음 실행하면 `data/` 폴더에 약 30MB를 내려받습니다.

```
import torch
from torchvision import datasets, transforms

변환 = transforms.ToTensor()            # 사진 → (1, 28, 28) 실수 텐서, 값은 0~1
학습셋 = datasets.FashionMNIST("data", train=True, download=True, transform=변환)
시험셋 = datasets.FashionMNIST("data", train=False, download=True, transform=변환)
print("학습", len(학습셋), "장 / 시험", len(시험셋), "장")
print(학습셋.classes)
print("클래스별 학습 사진 수:", torch.bincount(학습셋.targets).tolist())
```

```
사진, 정답 = 학습셋[0]
print(사진.shape, 사진.dtype, f"값 범위 {사진.min():.1f}~{사진.max():.1f}", "정답:", 정답, 학습
↪ 셋.classes[정답])

# 글자로 그려 보기: 밝을수록 진한 글자, 두 줄에 한 줄만
글자 = " .:-=+*#%@"
for 줄 in 사진[0, ::2]:
    print("".join(글자[min(int(v * 10), 9)] for v in 줄))
```

```
학습 60000 장 / 시험 10000 장
['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle
↪ boot']
클래스별 학습 사진 수: [6000, 6000, 6000, 6000, 6000, 6000, 6000, 6000, 6000, 6000]
torch.Size([1, 28, 28]) torch.float32 값 범위 0.0~1.0 정답: 9 Ankle boot


              .+=::
              *@%*=***=: -+:
             #@@@%%%%%*==#%
             #%%%##%%%%%%@#
            -@%%%##%%%%%@=*:
            @%%%%%%%%%%%%@-
     .-=#%%%%%#%%@%@*#@@@@%
 #%%%%%%%%%@-+@%%#*#%%%%%%
:%%%#%%%%%##%#%@#%@@%%%%%%%@:
 =%##*##%%%%%%%%%%%%****%-
    :#%@@@@@@%%########***-:
```

학습용 6만 장, 시험용 1만 장이고, 열 클래스가 6,000장씩 고르게 들어 있습니다. 클래스가 고르니 정확도를 그대로 성적으로 봐도 괜찮아요(한쪽으로 쏠린 데이터는 2장과 7장에서 다룹니다). 클래스 이름은 순서대로 T셔츠/상의, 바지, 풀오버, 드레스, 코트, 샌들, 셔츠, 운동화, 가방, 앵클부츠예요.

`transforms.ToTensor()`는 사진 파일을 `(1, 28, 28)` 모양의 float32 텐서로 바꾸면서, 0~255였던 밝기를 0~1로 나눠 줍니다. 7장 7.6절에서 볼 `transforms.v2`가 요즘 권장하는 방식이지만, 이 장과

9장은 결과를 서로 비교하려고 같은 전처리(`ToTensor`)를 끝까지 씁니다. v2로 바꿔도 정확도는 소수점 아래만 조금 달라져요. 앞의 1은 흑백이라 색 채널이 하나라는 뜻이에요(4장 4.2절의 "장 수, 색 채널, 세로, 가로" 순서). 첫 사진을 밝기에 따라 글자로 찍어 보니 앵클부츠 모양이 보이죠? 정답 번호 9가 `Ankle boot`인 것과 맞습니다.

학습시키기

앞에서 만든 부품을 모두 조립합니다. 학습 한 에폭과 평가를 함수로 나눠 두면, 이 두 함수를 다음 장들에서도 거의 그대로 다시 쓸 수 있어요.

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms


class 의류분류기(nn.Module):
    """28×28 흑백 사진을 받아 옷 종류 10가지의 점수를 내는 다층 퍼셉트론."""

    def __init__(self) -> None:
        super().__init__()
        self.펴기 = nn.Flatten()
        self.층들 = nn.Sequential(
            nn.Linear(28 * 28, 256), nn.ReLU(),
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, 10),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.층들(self.펴기(x))


def 한_에폭_학습(모델: nn.Module, 로더: DataLoader, 손실함수: nn.Module,
                옵티마이저: torch.optim.Optimizer, 장치: torch.device) -> float:
    """학습 데이터를 한 바퀴 돌며 학습하고 평균 손실을 돌려준다."""
    모델.train()
    손실_합, 개수 = 0.0, 0
```

```
    for x, y in 로더:
        x, y = x.to(장치), y.to(장치)
        옵티마이저.zero_grad()
        손실 = 손실함수(모델(x), y)
        손실.backward()
        옵티마이저.step()
        손실_합 += 손실.item() * len(y)
        개수 += len(y)
    return 손실_합 / 개수


def 평가하기(모델: nn.Module, 로더: DataLoader, 손실함수: nn.Module,
            장치: torch.device) -> tuple[float, float]:
    """평균 손실과 정확도를 잰다. 모델은 고치지 않는다."""
    모델.eval()
    손실_합, 맞힘, 개수 = 0.0, 0, 0
    with torch.no_grad():
        for x, y in 로더:
            x, y = x.to(장치), y.to(장치)
            출력 = 모델(x)
            손실_합 += 손실함수(출력, y).item() * len(y)
            맞힘 += (출력.argmax(dim=1) == y).sum().item()
            개수 += len(y)
    return 손실_합 / 개수, 맞힘 / 개수


torch.manual_seed(0)
변환 = transforms.ToTensor()
학습셋 = datasets.FashionMNIST("data", train=True, download=True, transform=변환)
시험셋 = datasets.FashionMNIST("data", train=False, download=True, transform=변환)
학습_로더 = DataLoader(학습셋, batch_size=64, shuffle=True)    # 64장씩 섞어서(7장)
시험_로더 = DataLoader(시험셋, batch_size=256)

장치 = torch.device("cpu")      # 결과를 똑같이 재현하려고 CPU. 장치_고르기()로 바꿔도 된다
모델 = 의류분류기().to(장치)
손실함수 = nn.CrossEntropyLoss()
```

```
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
print(f"파라미터 {sum(p.numel() for p in 모델.parameters()):,}개")

for 에폭 in range(1, 11):
    학습_손실 = 한_에폭_학습(모델, 학습_로더, 손실함수, 옵티마이저, 장치)
    시험_손실, 정확도 = 평가하기(모델, 시험_로더, 손실함수, 장치)
    print(f"에폭 {에폭:2}: 학습 손실 {학습_손실:.4f}, 시험 손실 {시험_손실:.4f}, 시험 정확도 {정확
    ↪ 도:.2%}")

torch.save(모델.state_dict(), "fashion_mlp.pt")
print("저장했습니다: fashion_mlp.pt")
```

```
파라미터 235,146개
에폭  1: 학습 손실 0.5218, 시험 손실 0.4338, 시험 정확도 84.04%
에폭  2: 학습 손실 0.3679, 시험 손실 0.3914, 시험 정확도 86.26%
에폭  3: 학습 손실 0.3290, 시험 손실 0.3813, 시험 정확도 86.42%
에폭  4: 학습 손실 0.3061, 시험 손실 0.3415, 시험 정확도 87.84%
에폭  5: 학습 손실 0.2875, 시험 손실 0.3462, 시험 정확도 87.62%
에폭  6: 학습 손실 0.2749, 시험 손실 0.3492, 시험 정확도 87.97%
에폭  7: 학습 손실 0.2585, 시험 손실 0.3421, 시험 정확도 87.97%
에폭  8: 학습 손실 0.2480, 시험 손실 0.3521, 시험 정확도 88.05%
에폭  9: 학습 손실 0.2366, 시험 손실 0.3473, 시험 정확도 88.20%
에폭 10: 학습 손실 0.2264, 시험 손실 0.3294, 시험 정확도 88.58%
저장했습니다: fashion_mlp.pt
```

10에폭 뒤 시험 정확도 **88.58%** 가 나왔습니다. 아무것도 모르고 찍으면 10%였을 문제에서, 사진을 한 줄로 편 단순한 MLP로 열 장 중 아홉 장 가까이를 맞힌 거예요. 결과를 그대로 재현할 수 있게 씨앗을 고정하고 CPU에서 돌렸습니다. 이 정도 크기의 모델은 CPU로도 금방 끝나요. `장치_고르기()`로 바꿔 GPU에서 돌리면 계산 순서가 조금 달라져서 소수점 아래 숫자가 이 책과 약간 다를 수 있습니다.

코드에서 이 장에서 배운 것을 하나씩 찾아보세요.

- **모델**: `nn.Module`을 상속한 `의류분류기`가 `nn.Flatten`으로 사진을 펴고, `nn.Sequential`로 784 → 256 → 128 → 10을 쌓았습니다. 층 사이에는 ReLU, 끝에는 아무것도 없이 로짓을 냅니다. 파라미터는 784 × 256 + 256, 256 × 128 + 128, 128 × 10 + 10을 더한 235,146개예

요.

- **손실 함수와 옵티마이저**: 열 클래스 분류이니 `CrossEntropyLoss`, Adam에 학습률 0.001입니다.
- **`한_에폭_학습`**: `모델.train()`으로 시작해 묶음마다 장치로 옮기고 다섯 줄을 돕니다.
- **`평가하기`**: `모델.eval()`과 `torch.no_grad()` 안에서 손실과 맞힌 개수를 셉니다.
- `DataLoader`: 6.7절에서 `randperm`으로 직접 하던 "섞어서 64장씩 자르기"를 대신해 줍니다. `for x, y in 로더:`가 한 번 돌 때마다 사진 64장 `(64, 1, 28, 28)`과 정답 64개가 나와요. 7장에서 자세히 봅니다.
- **저장**: 학습이 끝나면 `state_dict`를 `fashion_mlp.pt`로 저장합니다.

결과에서 볼 점이 하나 더 있습니다. 학습 손실은 `0.5218`에서 `0.2264`로 에폭마다 꾸준히 줄었는데, 시험 손실은 4에폭쯤부터 `0.33~0.35` 언저리를 오르내리며 거의 줄지 않았어요. 모델이 학습 데이터에는 점점 더 잘 맞아 가는데 처음 보는 데이터에서는 그만큼 나아지지 않는다는 신호입니다. 1장에서 직관으로 만났던 **과적합**이 시작되는 모습이에요. 이 신호를 읽고 대처하는 방법은 8장의 주제입니다.

무엇을 틀렸을까

정확도 숫자 하나만 보고 끝내면 모델이 어디서 약한지 알 수 없습니다. 저장한 모델을 불러와서 틀린 것들을 들여다보겠습니다. 6.10절에서 배운 대로 같은 설계도를 만들고 숫자를 채워 넣는 데서 시작해요.

```
import torch
from torch import nn
from torchvision import datasets, transforms

이름 = ["T셔츠", "바지", "풀오버", "드레스", "코트", "샌들", "셔츠", "운동화", "가방", "앵클부츠"]


class 의류분류기(nn.Module):
    """6.11절에서 학습한 모델과 같은 설계도."""

    def __init__(self) -> None:
        super().__init__()
        self.펴기 = nn.Flatten()
        self.층들 = nn.Sequential(
```

```
            nn.Linear(28 * 28, 256), nn.ReLU(),
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, 10),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.층들(self.펴기(x))


모델 = 의류분류기()
모델.load_state_dict(torch.load("fashion_mlp.pt", weights_only=True))
모델.eval()

시험셋 = datasets.FashionMNIST("data", train=False, download=True, transform=transforms.ToTensor())
X = 시험셋.data.float().unsqueeze(1) / 255     # 1만 장을 한 번에: (10000, 1, 28, 28)
y = 시험셋.targets
with torch.no_grad():
    확률 = torch.softmax(모델(X), dim=1)
예측 = 확률.argmax(dim=1)
print(f"전체 정확도: {(예측 == y).float().mean():.2%}")

# 클래스별 정확도
for k in range(10):
    print(f"{이름[k]:6} {(예측[y == k] == k).float().mean():6.1%}", end="   " if k % 2 == 0 else
    ↪ "\n")

# 가장 많이 헷갈린 짝
혼동 = torch.zeros(10, 10, dtype=torch.int64)
for 정답, 답 in zip(y.tolist(), 예측.tolist()):
    혼동[정답, 답] += 1
혼동.fill_diagonal_(0)
for 순위, 위치 in enumerate(혼동.flatten().topk(3).indices.tolist(), start=1):
    정답, 답 = divmod(위치, 10)
    print(f"헷갈림 {순위}위: 정답 {이름[정답]} → 예측 {이름[답]}, {혼동[정답, 답]}번")

# 틀린 사진 세 장: 모델은 얼마나 확신했을까?
```

```
for i in (예측 != y).nonzero().flatten()[:3].tolist():
    print(f"{i}번 사진: 정답 {이름[y[i]]}, 예측 {이름[예측[i]]} (확률 {확률[i, 예측[i]]:.1%})")
```

```
전체 정확도: 88.58%
T셔츠      84.0%   바지       95.9%
풀오버      72.9%   드레스      87.7%
코트       87.3%   샌들       96.6%
셔츠       74.1%   운동화      97.4%
가방       96.3%   앵클부츠     93.6%
헷갈림 1위: 정답 풀오버 → 예측 코트, 172번
헷갈림 2위: 정답 T셔츠 → 예측 셔츠, 116번
헷갈림 3위: 정답 셔츠 → 예측 T셔츠, 85번
12번 사진: 정답 운동화, 예측 샌들 (확률 66.2%)
17번 사진: 정답 코트, 예측 풀오버 (확률 72.8%)
23번 사진: 정답 앵클부츠, 예측 샌들 (확률 99.6%)
```

시험 사진 1만 장은 크기가 작아서 `DataLoader` 없이 한 번에 텐서로 만들어 넣었습니다. `시험셋.data`는 0~255 정수 사진이라 `ToTensor()`가 하던 일(실수로 바꾸고 255로 나누고 채널 차원 붙이기)을 직접 했어요. 전체 정확도가 학습 직후와 같은 88.58%이니 저장과 불러오기가 제대로 된 겁니다.

클래스별로 보면 차이가 큽니다. 운동화(97.4%), 샌들(96.6%), 가방(96.3%)은 잘 맞히는데 **풀오버(72.9%)와 셔츠(74.1%)** 가 약해요. 가장 많이 헷갈린 짝을 보면 이유가 보입니다. 풀오버를 코트로 172번, T셔츠를 셔츠로 116번, 셔츠를 T셔츠로 85번 틀렸어요. 28 × 28 흑백 사진에서 소매 달린 상의들은 사람이 봐도 구별이 쉽지 않습니다. 모델은 사진을 한 줄로 펴서 보기 때문에 "칼라가 있다", "단추가 있다" 같은 모양의 특징을 잡는 데 약하고요. 사진의 모양을 살펴보는 9장 CNN이 바로 이 약점을 메웁니다.

마지막 세 줄은 틀린 사진들입니다. 23번 사진은 앵클부츠를 샌들이라고 하면서 확률이 **99.6%** 였어요. 틀렸는데도 아주 확신한 거죠. 소프트맥스 확률은 "모델이 얼마나 확신하는가"이지 "얼마나 맞을 가능성이 있는가"가 아니라는 점을 기억해 두세요. 모델이 높은 확률을 냈다고 해서 그 답을 그대로 믿으면 곤란할 때가 있습니다.

옵티마이저 바꿔 보기

6.6절에서는 골짜기 함수로 옵티마이저를 비교했죠. 진짜 데이터에서는 어떨까요? 같은 모델, 같은 씨앗에서 옵티마이저만 바꿔 2에폭씩 학습해 보겠습니다.

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms


def 모델_만들기() -> nn.Module:
    """6.11절과 같은 구조의 다층 퍼셉트론."""
    return nn.Sequential(
        nn.Flatten(),
        nn.Linear(28 * 28, 256), nn.ReLU(),
        nn.Linear(256, 128), nn.ReLU(),
        nn.Linear(128, 10),
    )


def 학습하고_재기(옵티마이저_만들기, 에폭수: int = 2) -> float:
    """같은 씨앗에서 출발해 학습한 뒤 시험 정확도를 돌려준다."""
    torch.manual_seed(0)
    모델 = 모델_만들기()
    옵티마이저 = 옵티마이저_만들기(모델.parameters())
    손실함수 = nn.CrossEntropyLoss()
    for _ in range(에폭수):
        모델.train()
        for x, y in 학습_로더:
            옵티마이저.zero_grad()
            손실함수(모델(x), y).backward()
            옵티마이저.step()
    모델.eval()
    with torch.no_grad():
        예측 = 모델(X_시험).argmax(dim=1)
    return (예측 == y_시험).float().mean().item()
```

```
학습셋 = datasets.FashionMNIST("data", train=True, download=True, transform=transforms.ToTensor())
시험셋 = datasets.FashionMNIST("data", train=False, download=True)
학습_로더 = DataLoader(학습셋, batch_size=64, shuffle=True)
X_시험 = 시험셋.data.float().unsqueeze(1) / 255
y_시험 = 시험셋.targets

후보 = {
    "SGD lr=0.01          ": lambda p: torch.optim.SGD(p, lr=0.01),
    "SGD+momentum lr=0.01 ": lambda p: torch.optim.SGD(p, lr=0.01, momentum=0.9),
    "Adam lr=0.001        ": lambda p: torch.optim.Adam(p, lr=0.001),
    "Adam lr=0.1          ": lambda p: torch.optim.Adam(p, lr=0.1),
}
for 이름, 만들기 in 후보.items():
    print(f"{이름} 2에폭 뒤 시험 정확도 {학습하고_재기(만들기):.2%}")
```

```
SGD lr=0.01           2에폭 뒤 시험 정확도 76.44%
SGD+momentum lr=0.01  2에폭 뒤 시험 정확도 84.12%
Adam lr=0.001         2에폭 뒤 시험 정확도 86.07%
Adam lr=0.1           2에폭 뒤 시험 정확도 33.35%
```

이번에는 **Adam(lr=0.001)이 86.07%** 로 가장 높고, 모멘텀을 더한 SGD가 84.12%, 그냥 SGD가 76.44%입니다. 골짜기에서는 모멘텀이 Adam보다 빨랐는데 여기서는 순서가 바뀌었어요. 6.6절에서 "항상 이기는 옵티마이저는 없다"고 한 이유입니다. 그래도 두 실험 모두 그냥 SGD보다 모멘텀이 확실히 나았다는 점은 같아요.

마지막 줄이 더 중요합니다. 같은 Adam이라도 학습률을 0.1로 키우자 33.35%로 무너졌어요. 걸음이 너무 커서 손실의 골짜기를 계속 넘나들며 제대로 내려가지 못한 겁니다. **옵티마이저의 종류보다 학습률이 결과를 더 크게 좌우할 때가 많습니다.** 옵티마이저를 바꿀 때는 그 옵티마이저에 맞는 학습률을 함께 찾아야 해요.

그런데 이 비교의 Adam 결과(86.07%)는 앞에서 학습한 모델의 2에폭째 정확도(86.26%)와 조금 다릅니다. 모델 구조도 씨앗도 같은데 왜일까요? 앞의 학습은 에폭마다 `시험_로더`로 평가했는데, `DataLoader`는 돌기 시작할 때마다 난수를 하나 꺼내 씁니다. 그 바람에 2에폭째에 학습 데이터를 섞는 순서가 달라진 거예요. 이번 비교는 학습이 다 끝난 뒤에 한 번만 평가했으니 섞는 순서가 앞과 다릅니다. 같은 실험이라도 이런 작은 차이로 숫자가 흔들리니, 옵티마이저 사이의 0.1~0.2%p 차이로

우열을 가리면 안 돼요. 의미 있는 비교를 하려면 씨앗을 바꿔 여러 번 돌려 봐야 하는데, 그 방법은 18장에서 다룹니다.

6.12 정리와 다음 장 미리보기

이 장에서 모든 PyTorch 모델이 따르는 표준형을 익혔습니다.

- **층**: `nn.Linear(입력, 출력)`은 `x @ weight.T + bias`. 가중치 모양은 (출력, 입력). 선형층만 쌓으면 선형층 하나와 같다
- **활성화 함수**: 은닉층 사이에는 ReLU. 직선으로 못 긋는 경계(XOR, 동심원)를 그리게 해 준다. 시그모이드와 소프트맥스는 바깥에서 확률을 읽을 때
- **모델**: `nn.Module` 상속, `__init__`에서 `super().__init__()` 후 부품 준비, `forward`에 흐름. 부를 때는 `모델(x)`. 줄 세우기만 하면 `nn.Sequential`, 층 목록은 `nn.ModuleList`
- **파라미터**: `named_parameters()`, `numel()`로 세기. `nn.Linear`는 입력 × 출력 + 출력개. `requires_grad_(False)`로 얼리기
- **손실 함수**: 회귀는 `MSELoss`(모양 맞추기 주의), 다중 분류는 `CrossEntropyLoss`에 **로짓**과 정수 정답, 이진 분류는 `BCEWithLogitsLoss`. 첫 손실은 `-log(1/클래스 수)` 근처
- **옵티마이저**: `step()`은 기울기 방향으로 고치기. SGD, 모멘텀, Adam. 학습률이 가장 중요한 손잡이
- **학습 루프 다섯 줄**: `zero_grad()` → `모델(x)` → `손실함수(출력, y)` → `backward()` → `step()`. 에폭과 미니배치로 반복
- **평가 루프**: `모델.eval()` + `torch.no_grad()`, 다시 학습할 때 `모델.train()`
- **장치와 저장**: 모델은 `.to(장치)`로 제자리에서 옮기고 데이터는 묶음마다 옮기기. `state_dict`로 저장하고 같은 설계도에 `load_state_dict(torch.load(..., weights_only=True))`. 이어서 학습하려면 옵티마이저 상태까지 체크포인트로

실습에서 쓴 `DataLoader`는 사실 그 자체로 한 장을 쓸 만한 도구입니다. FashionMNIST처럼 `torchvision`이 준비해 둔 데이터가 아니라 **내 컴퓨터의 CSV 파일이나 사진 폴더**를 학습에 쓰려면 어떻게 해야 할까요? 사진을 조금씩 돌리고 뒤집어서 데이터를 불리는 방법은 없을까요? 다음 7장에서는 데이터를 모델 앞까지 실어 나르는 파이프라인, `Dataset`과 `DataLoader`를 직접 만들어 봅니다.

07

데이터 파이프라인: Dataset과 DataLoader

학습 목표

- 데이터를 한꺼번에 메모리에 올리지 않고 필요할 때 꺼내 쓰는 이유 설명하기
- `__len__`과 `__getitem__`으로 CSV 파일, 이미지 폴더용 Dataset을 직접 만들기
- DataLoader의 `batch_size`, `shuffle`, `drop_last`, `num_workers`가 하는 일을 결과로 확인하기
- transforms로 정규화와 데이터 증강을 하고, 길이가 다른 샘플은 `collate_fn`으로 묶기
- 재현 가능하게 학습/검증을 나누고, 불균형 데이터는 샘플러로 고르게 뽑기

들어가며: 주방은 재료를 한꺼번에 꺼내지 않습니다

손님이 몰리는 식당 주방을 떠올려 보세요. 하루에 쓸 재료를 아침에 창고에서 전부 꺼내 조리대에 쌓아 두는 주방은 없습니다. 조리대가 감당할 수 없을 뿐 아니라, 재료가 상하고 정작 필요한 칼과 도마를 놓을 자리도 없어지니까요. 실제 주방은 **창고에 무엇이 어디 있는지 목록만 쥐고**, 주문이 들어오면 그때 필요한 재료를 꺼내 **손질하고**, 같은 테이블 음식은 **쟁반 하나에 담아** 내보냅니다.

딥러닝 학습도 똑같은 문제를 겪습니다. 4장에서 사진 한 장이 숫자 15만 개쯤 된다고 했죠. 사진이 많아지면 어떻게 될까요?

```
import torch

한장 = torch.zeros(3, 224, 224)              # 컬러 사진 한 장 (색, 세로, 가로)
한장_바이트 = 한장.numel() * 한장.element_size()  # 숫자 개수 × 숫자 하나의 크기(float32 = 4바이트)
print(f"사진 한 장: 숫자 {한장.numel():,}개, {한장_바이트 / 1024**2:.2f}MB")

for 장수 in [1_000, 100_000, 1_000_000]:
    print(f"사진 {장수:>9,}장: {장수 * 한장_바이트 / 1024**3:8.1f}GB")
```

```
사진 한 장: 숫자 150,528개, 0.57MB
사진     1,000장:      0.6GB
사진   100,000장:     56.1GB
사진 1,000,000장:    560.8GB
```

224 × 224 컬러 사진 한 장은 약 0.57MB지만, 10만 장이면 56GB, 100만 장이면 560GB가 넘습니다. 이 책을 쓴 노트북의 메모리로는 10만 장도 한꺼번에 올릴 수 없어요. 그래서 PyTorch는 데이터를 다루는 일을 주방처럼 세 역할로 나눕니다.

- **Dataset (창고 목록)**: 샘플이 몇 개인지 알고, "n번 샘플 주세요"라고 하면 그 하나만 꺼내 줍니다.
- **transform (손질)**: 꺼낸 샘플을 모델이 먹기 좋은 텐서로 바꾸고, 필요하면 살짝 변형(증강)합니다.
- **DataLoader (쟁반)**: 샘플을 여러 개 꺼내 배치 하나로 묶어 모델에 건넵니다. 순서를 섞고, 여러 일꾼에게 나눠 꺼내게 하는 일도 맡아요.

이렇게 데이터가 창고에서 모델까지 흘러가는 길을 **데이터 파이프라인**이라고 부릅니다. 5장에서는 `randperm`으로 직접 섞고 잘라 묶음을 만들었고, 6장 실습에서는 torchvision이 준비해 둔 FashionMNIST를 `DataLoader`에 넘겨 썼죠. 이번에는 그 안에서 무슨 일이 일어나는지 열어 보고, 내 파일로 직접 만들어 봅니다. 실제 프로젝트에서는 학습 코드보다 이 파이프라인을 짜는 데 시간이 더 들 때가 많아요. 모델이 아무리 좋아도 엉뚱한 데이터가 들어가면 소용없고, 데이터가 늦게 오면 비싼 GPU가 놀기 때문입니다.

이 장에서는 Dataset을 직접 만들어 보는 것부터 시작해서 CSV 파일과 이미지 폴더를 읽고, 증강을 걸고, 길이가 다른 문장을 묶고, 학습/검증을 나누고, 드문 클래스를 더 자주 뽑고, 마지막으로 파이프라인이 느린 곳을 찾는 법까지 다룹니다. 모델은 거의 등장하지 않아요. 모델 학습은 6장, 학습을 잘 시키는 요령은 8장의 몫이고, 이 장의 주인공은 오직 데이터가 흘러가는 길입니다.

Q

Q. 이 장의 예제 데이터는 어디서 받나요? 받을 필요가 없습니다. CSV 파일, 도형 그림, 짧은 리뷰 문장 모두 **예제용으로 만든 데이터**이고, 만드는 코드가 각 예제 안에 들어 있어요. 예제를 실행하면 현재 폴더 아래 `data/` 폴더에 파일이 생깁니다. 이 장의 예제는 PyTorch 2.14, torchvision 0.29, 파이썬 3.12에서 실행해 확인했고, `uv add torchvision pillow pandas`로 필요한 라이브러리를 더할 수 있어요.

7.1 Dataset: 번호를 주면 샘플 하나를 주는 약속

두 가지 특수 메서드면 충분합니다

PyTorch의 Dataset은 생각보다 단순한 약속입니다. 딱 두 가지만 할 줄 알면 돼요.

- `len(데이터셋)`이라고 물으면 샘플이 몇 개인지 답한다 → `__len__`
- `데이터셋[i]`라고 하면 i번 샘플 하나를 돌려준다 → `__getitem__`

1권 11장에서 `len()`과 `객체[번호]`가 사실 `__len__`과 `__getitem__`이라는 특수 메서드를 부르는 것이라고 배웠죠. Dataset은 바로 그 두 메서드를 구현한 클래스일 뿐입니다. 0부터 n-1까지의 수와 그 제곱을 (입력, 정답) 짝으로 돌려주는 가장 작은 데이터셋을 만들어 보겠습니다. 샘플이 언제 만들어지는지 보려고 `__getitem__` 안에 출력문을 하나 넣었어요.

```
from torch.utils.data import Dataset


class 제곱수_데이터셋(Dataset):
    """0부터 n-1까지의 수와 그 제곱을 (입력, 정답) 짝으로 돌려준다."""

    def __init__(self, n: int) -> None:
        self.n = n                          # 재료 목록만 기억해 둔다

    def __len__(self) -> int:
        return self.n                       # 샘플이 모두 몇 개인가

    def __getitem__(self, i: int) -> tuple[int, int]:
        if i >= self.n:
            raise IndexError(f"{i}번 샘플은 없습니다")
```

```
        print(f"  ({i}번 샘플을 지금 만듭니다)")
        return i, i * i                  # i번 샘플 하나를 그때 만들어 돌려준다


데이터 = 제곱수_데이터셋(4)
print("샘플 수:", len(데이터))
print("3번 샘플:", 데이터[3])
for 입력, 정답 in 데이터:
    print("반복:", 입력, 정답)
```

```
샘플 수: 4
  (3번 샘플을 지금 만듭니다)
3번 샘플: (3, 9)
  (0번 샘플을 지금 만듭니다)
반복: 0 0
  (1번 샘플을 지금 만듭니다)
반복: 1 1
  (2번 샘플을 지금 만듭니다)
반복: 2 4
  (3번 샘플을 지금 만듭니다)
반복: 3 9
```

출력을 순서대로 보세요. `len(데이터)`는 아무 샘플도 만들지 않고 바로 4를 답했습니다. `데이터[3]`을 하자 그제야 "(3번 샘플을 지금 만듭니다)"가 찍혔어요. 주방 비유 그대로입니다. `__init__`은 창고 목록(여기서는 개수 `n`)만 기억해 두고, 실제 재료는 `__getitem__`이 불릴 때 하나씩 꺼냅니다. 샘플이 100만 개여도 한 번에 메모리에 있는 것은 지금 꺼낸 하나뿐이에요.

`for 입력, 정답 in 데이터:`도 잘 돌았습니다. 우리는 `__iter__`를 만든 적이 없는데 말이죠. 1권 15장에서 `for`문은 `__iter__`와 `__next__`로 돈다고 배웠는데, 파이썬에는 오래된 방법이 하나 더 있습니다. `__iter__`가 없고 `__getitem__`만 있으면 0, 1, 2, ...를 차례로 넣어 보다가 `IndexError`가 나는 순간 멈추는 거예요. 그래서 범위를 벗어난 번호에 `IndexError`를 일으키는 줄이 중요합니다. 이 줄을 빼면 `for`문이 끝나지 않아요. 다만 실제 학습에서는 Dataset을 직접 `for`로 돌릴 일이 거의 없습니다. 7.3절의 DataLoader가 그 일을 대신하니까요.

Q. **꼭 `Dataset`을 상속해야 하나요?** 꼭 그렇지는 않습니다. DataLoader는 "`__len__`과 `__getitem__`이 있는가"만 보기 때문에, 7.7절에서처럼 평범한 파이썬 리스트를 넣어도 돌아가요. 1권 2장에서 본 "오리처럼 걸으면 오리"라는 생각 그대로입니다. 그래도 상속해 두면 읽는 사람이 "이건 데이터셋이구나" 하고 바로 알 수 있고, `데이터셋1 + 데이터셋2`로 두 데이터셋을 이어 붙이는 기능 같은 덤도 따라오니 상속하는 것이 관례예요.

무거운 일은 `__getitem__`에서

Dataset을 만들 때 기억할 원칙은 하나입니다. **`__init__`은 가볍게, 무거운 일은 `__getitem__`에서.** `__init__`에서는 파일 경로 목록이나 정답 표처럼 작은 정보만 준비하고, 그림 파일을 열거나 글을 숫자로 바꾸는 일은 샘플을 꺼내는 순간에 합니다. 그래야 데이터가 아무리 커도 메모리가 버티고, 7.10절에서 볼 것처럼 꺼내는 일을 여러 일꾼에게 나눠 맡길 수도 있어요. 반대로 데이터가 작다면 `__init__`에서 전부 읽어 두는 편이 빠릅니다. 다음 절의 TensorDataset이 그런 경우예요.

7.2 TensorDataset: 이미 텐서로 있을 때

특성과 정답이 이미 텐서로 메모리에 다 올라와 있다면, 클래스를 만들 필요도 없습니다. `TensorDataset`이 텐서들을 받아 같은 번호끼리 짝지어 줘요.

```
import torch
from torch.utils.data import TensorDataset

torch.manual_seed(0)
X = torch.randn(6, 3)                    # 샘플 6개 × 특성 3개
y = torch.tensor([0, 1, 0, 1, 1, 0])     # 샘플마다 정답 하나

데이터 = TensorDataset(X, y)
print("샘플 수:", len(데이터))
x0, y0 = 데이터[0]                         # 0번 샘플: (특성 3개, 정답)
print(x0, y0)
특성들, 정답들 = 데이터[1:3]                # 슬라이스로 여러 개도 꺼낼 수 있다
print(특성들.shape, 정답들)

try:
```

```
    TensorDataset(X, torch.tensor([0, 1]))   # 샘플 수가 다르면?
except AssertionError as e:
    print("오류:", e)
```

```
샘플 수: 6
tensor([-1.1258, -1.1524,  0.5667]) tensor(0)
torch.Size([2, 3]) tensor([1, 0])
오류: Size mismatch between tensors
```

`TensorDataset(X, y)`는 `X`의 0번 줄과 `y`의 0번 값을 짝지어 0번 샘플로 돌려줍니다. 샘플 수는 텐서의 첫 번째 차원 길이(6)가 되고, `데이터[1:3]`처럼 슬라이스로 여러 샘플을 한 번에 꺼낼 수도 있어요. 텐서를 몇 개든 넣을 수 있어서 `TensorDataset(X, y, 가중치)`처럼 세 개를 짝지어도 됩니다.

넣은 텐서들의 첫 번째 차원 길이가 다르면 바로 "Size mismatch between tensors" 오류를 냅니다. 특성은 6개 샘플인데 정답이 2개뿐이면 짝을 지을 수가 없으니까요. 이 검사 덕분에 특성과 정답의 개수가 어긋나는 실수를 일찍 잡을 수 있습니다.

TensorDataset은 데이터가 메모리에 다 들어갈 만큼 작을 때, 예를 들어 3장에서 다룬 것 같은 표 데이터나 이 장의 7.3절처럼 작은 실험을 할 때 가장 간편한 선택이에요.

7.3 DataLoader: 배치로 묶어 나르기

batch_size, drop_last, shuffle

Dataset이 샘플을 **하나씩** 꺼내 준다면, DataLoader는 그걸 **배치로 묶어** 나릅니다. 4장에서 모델은 사진을 한 장씩이 아니라 수십 장씩 묶어서 처리해야 GPU가 쉬지 않는다고 했죠. 샘플 10개짜리 데이터셋으로 DataLoader의 기본 옵션 세 가지를 확인해 보겠습니다. 정답을 샘플 번호와 같게 만들어서, 출력의 정답 목록이 곧 "몇 번 샘플들이 묶였는지"가 되게 했어요.

```
import torch
from torch.utils.data import DataLoader, TensorDataset

X = torch.arange(10).float().unsqueeze(1) * 10   # 샘플 10개 × 특성 1개: 0, 10, ..., 90
y = torch.arange(10)                             # 정답은 샘플 번호와 같게
데이터 = TensorDataset(X, y)
```

```
로더 = DataLoader(데이터, batch_size=4)
print("배치 수:", len(로더))
for xb, yb in 로더:
    print(tuple(xb.shape), yb.tolist())

버리는_로더 = DataLoader(데이터, batch_size=4, drop_last=True)
print("drop_last=True 배치 수:", len(버리는_로더))

g = torch.Generator().manual_seed(0)            # 섞는 순서도 씨앗으로 고정
섞는_로더 = DataLoader(데이터, batch_size=4, shuffle=True, generator=g)
for 에폭 in range(2):
    print(f"에폭 {에폭}:", [yb.tolist() for _, yb in 섞는_로더])
```

```
배치 수: 3
(4, 1) [0, 1, 2, 3]
(4, 1) [4, 5, 6, 7]
(2, 1) [8, 9]
drop_last=True 배치 수: 2
에폭 0: [[3, 7, 5, 2], [0, 8, 1, 6], [9, 4]]
에폭 1: [[1, 3, 2, 7], [5, 9, 0, 6], [4, 8]]
```

- `batch_size=4`: 샘플 4개씩 묶습니다. 입력은 `(1,)` 모양 샘플 4개가 쌓여 `(4, 1)`이 됐어요. 10개를 4개씩 묶으면 마지막 배치에는 2개만 남는데, 버리지 않고 작은 배치로 내보냅니다. 그래서 `len(로더)`는 3입니다.
- `drop_last=True`: 크기가 모자란 마지막 배치를 버립니다. 배치 수가 2로 줄었어요. 배치 정규화(8장)처럼 배치 크기가 너무 작으면 불안정해지는 경우에 씁니다.
- `shuffle=True`: 에폭마다 순서를 새로 섞습니다. **에폭(epoch)** 은 데이터 전체를 한 바퀴 도는 것을 말해요. 에폭 0과 에폭 1의 순서가 다르죠? 샘플 순서가 늘 같으면 모델이 순서의 버릇까지 배워 버릴 수 있어서, 학습할 때는 보통 섞습니다.

`generator=g`로 섞는 순서의 씨앗을 고정한 것도 눈여겨보세요. 4장에서 본 "나만 쓰는 번호표 기계" `torch.Generator`예요. DataLoader에 씨앗을 고정한 생성기를 넘기면 언제 실행해도 같은 순서로 섞입니다. 그래서 이 책의 출력과 여러분의 출력이 같게 나와요. 재현성 전반은 18장에서 정리합니다.

DataLoader의 옵션은 많지만 자주 쓰는 것은 이 정도입니다. 나머지는 이 장의 뒤쪽 절에서 하나씩

만나요.

옵션	하는 일	나오는 곳
`batch_size`	배치 하나에 넣을 샘플 수	7.3절
`shuffle`	에폭마다 순서 섞기 (학습용은 `True`, 검증용은 `False`)	7.3절
`drop_last`	모자란 마지막 배치 버리기	7.3절
`generator`	섞기·뽑기에 쓸 난수 씨앗	7.3절, 7.8절
`collate_fn`	샘플 목록을 배치 하나로 합치는 함수	7.7절
`sampler`, `batch_sampler`	어떤 순서로 몇 번 샘플을 뽑을지 직접 정하기	7.7절, 7.9절
`num_workers`	샘플을 꺼내는 일꾼 프로세스 수	7.10절
`persistent_workers`	에폭이 끝나도 일꾼을 살려 두기	7.10절
`pin_memory`	CPU→GPU 복사를 빠르게 하는 고정 메모리 (CUDA)	7.10절

샘플을 배치로 합치는 규칙

DataLoader는 샘플 여러 개를 **어떻게** 하나로 합칠까요? 샘플이 텐서 하나가 아니라 이름, 점수, 합격 여부가 섞인 딕셔너리라면요?

```
import torch
from torch.utils.data import DataLoader, Dataset


class 학생_데이터셋(Dataset):
    """샘플 하나를 딕셔너리로 돌려주는 데이터셋 (예제용 점수)."""

    def __init__(self) -> None:
```

```
        self.이름 = ["가온", "나래", "다온", "라희", "마루"]
        self.점수 = torch.tensor([[80.0, 70.0], [92.0, 88.0], [75.0, 95.0],
                                  [60.0, 65.0], [88.0, 91.0]])
        self.합격 = [1, 1, 1, 0, 1]

    def __len__(self) -> int:
        return len(self.이름)

    def __getitem__(self, i: int) -> dict:
        return {"이름": self.이름[i], "점수": self.점수[i], "합격": self.합격[i]}


print("샘플 하나:", 학생_데이터셋()[0])
로더 = DataLoader(학생_데이터셋(), batch_size=3)
배치 = next(iter(로더))                    # 첫 배치만 꺼내 보기
for 키, 값 in 배치.items():
    print(f"{키}: {type(값).__name__} {값}")
```

```
샘플 하나: {'이름': '가온', '점수': tensor([80., 70.]), '합격': 1}
이름: list ['가온', '나래', '다온']
점수: Tensor tensor([[80., 70.],
        [92., 88.],
        [75., 95.]])
합격: Tensor tensor([1, 1, 1])
```

DataLoader는 샘플의 **생김새를 그대로 유지하면서** 안쪽 값만 묶어 줍니다. 샘플이 딕셔너리면 배치도 같은 키를 가진 딕셔너리가 되고, 키마다 이렇게 처리해요.

- **텐서**(`점수`)는 4장의 `torch.stack`처럼 새 차원을 만들어 쌓습니다. `(2,)` 모양 3개가 `(3, 2)`가 됐어요.
- **숫자**(`합격`)는 텐서로 바꿔 모읍니다. 파이썬 정수 1, 1, 1이 `tensor([1, 1, 1])`이 됐죠.
- **문자열**(`이름`)은 텐서로 바꿀 수 없으니 리스트로 모읍니다.

샘플이 `(입력, 정답)` 튜플이면 배치도 `(입력 묶음, 정답 묶음)` 튜플이 됩니다. 앞의 예제에서 `for xb, yb in 로더:`로 바로 풀어 쓸 수 있었던 이유예요. 이 합치는 규칙을 **collate**(모아 맞추기)라고 부르고, 기본 규칙이 맞지 않을 때는 7.7절처럼 직접 바꿀 수 있습니다. 샘플에 파일 이름이나 원문

문장 같은 문자열을 함께 넣어 두면, 나중에 모델이 틀린 샘플을 찾아볼 때 아주 편해요.

번호가 없는 데이터: IterableDataset

지금까지의 Dataset은 "몇 번 샘플"이라고 번호로 꺼낼 수 있었습니다. 이런 것을 **맵 스타일(map-style)** 데이터셋이라고 해요. 그런데 끝없이 쌓이는 주문 기록이나, 한 줄씩 읽을 수밖에 없는 거대한 로그 파일처럼 **번호로 바로 찾아갈 수 없는** 데이터도 있습니다. 이럴 때는 1권 15장의 제너레이터처럼 처음부터 차례로 흘려보내는 `IterableDataset`을 씁니다.

```
from collections.abc import Iterator

import torch
from torch.utils.data import DataLoader, IterableDataset


class 주문_흐름(IterableDataset):
    """주문 기록을 처음부터 차례로 흘려보내는 데이터셋 (예제용)."""

    def __init__(self, 주문_수: int) -> None:
        self.주문_수 = 주문_수

    def __iter__(self) -> Iterator[torch.Tensor]:
        for 번호 in range(self.주문_수):   # 실제로는 파일을 한 줄씩 읽는 자리
            yield torch.tensor([번호, 번호 * 1000])


로더 = DataLoader(주문_흐름(7), batch_size=3)
for 배치 in 로더:
    print(배치.tolist())

try:
    DataLoader(주문_흐름(7), batch_size=3, shuffle=True)
except ValueError as e:
    print("오류:", str(e).split(",")[0])
```

```
[[0, 0], [1, 1000], [2, 2000]]
[[3, 3000], [4, 4000], [5, 5000]]
[[6, 6000]]
오류: DataLoader with IterableDataset: expected unspecified shuffle option
```

`__len__`과 `__getitem__` 대신 `__iter__` 하나만 만들었습니다. 안에서 `yield`로 샘플을 하나씩 내보내면 DataLoader가 3개씩 모아 배치로 만들어 줘요. 1권 15장에서 제너레이터를 파이프처럼 이어 붙이면 데이터가 아무리 커도 메모리에는 한 배치만 머문다고 했는데, 바로 그 생각입니다.

대신 잃는 것도 있습니다. `shuffle=True`를 주자 오류가 났죠. 번호가 없으니 DataLoader가 순서를 섞을 방법이 없기 때문입니다. 섞고 싶다면 `__iter__` 안에서 직접 일정량을 모아 섞어야 해요. 그리고 7.10절에서 볼 여러 일꾼과 함께 쓸 때 주의할 점이 하나 더 있습니다. 그래서 번호로 꺼낼 수 있는 데이터라면 맵 스타일이 기본이고, IterableDataset은 정말 흘려보낼 수밖에 없을 때 꺼내는 도구예요.

7.4 직접 만든 Dataset ①: CSV 파일

예제용 CSV 만들기

이제 실제 파일을 읽는 Dataset을 만들어 보겠습니다. 첫 번째는 표 데이터가 담긴 CSV 파일이에요. 귤의 무게, 당도, 껍질 두께를 재서 상·중·하 등급을 매긴 표가 있다고 해 봅시다. 실제 측정값이 아니라 **예제용으로 만든 데이터**로, 등급이 높을수록 무겁고 달고 껍질이 얇도록 만들었어요. 학습용 160개와 검증용 40개를 서로 다른 파일로 저장합니다.

```
from pathlib import Path

import numpy as np
import pandas as pd

# 예제용으로 만든 귤 품질 데이터: 무게(g), 당도(브릭스), 껍질 두께(mm) → 등급
rng = np.random.default_rng(7)
등급들 = np.array(["하", "중", "상"])


def 귤_표_만들기(개수: int) -> pd.DataFrame:
    """등급을 먼저 정하고, 등급에 따라 조금씩 다른 측정값을 만든다."""
```

```
    등급번호 = rng.integers(0, 3, 개수)
    return pd.DataFrame({
        "무게": (rng.normal(80, 12, 개수) + 등급번호 * 8).round(1),
        "당도": (rng.normal(9.5, 1.0, 개수) + 등급번호 * 1.2).round(1),
        "껍질": (rng.normal(3.0, 0.4, 개수) - 등급번호 * 0.3).round(2),
        "등급": 등급들[등급번호],
    })


폴더 = Path("data/귤")
폴더.mkdir(parents=True, exist_ok=True)
귤_표_만들기(160).to_csv(폴더 / "학습.csv", index=False)
귤_표_만들기(40).to_csv(폴더 / "검증.csv", index=False)
print((폴더 / "학습.csv").read_text(encoding="utf-8").splitlines()[:4])
```

```
['무게,당도,껍질,등급', '114.3,10.5,2.09,상', '82.9,10.2,2.77,중', '92.4,12.6,2.56,상']
```

첫 줄이 열 이름이고 그 아래로 귤 한 개가 한 줄씩 적혀 있습니다. 1권 12장에서 다룬 CSV 형식 그대로예요. pandas로 표를 만들고 `to_csv`로 저장하는 방법은 1권 20장에서 봤습니다.

CSV를 읽는 Dataset

이 파일을 읽는 Dataset을 만들어 보겠습니다. 표 데이터는 몇 MB를 넘는 경우가 드물어서, 이번에는 `__init__`에서 파일 전체를 읽어 텐서로 바꿔 두는 쪽을 택했어요. 대신 특성을 **표준화**하는 일은 꺼낼 때 합니다.

```
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from torch.utils.data import DataLoader, Dataset

# 7.4절 앞 블록과 같은 예제용 귤 데이터를 만든다(이미 있으면 그대로 쓴다)
폴더 = Path("data/귤")
if not (폴더 / "학습.csv").exists():
```

```
    rng = np.random.default_rng(7)
    폴더.mkdir(parents=True, exist_ok=True)
    for 이름, 개수 in [("학습", 160), ("검증", 40)]:
        n = rng.integers(0, 3, 개수)
        pd.DataFrame({"무게": (rng.normal(80, 12, 개수) + n * 8).round(1),
                      "당도": (rng.normal(9.5, 1.0, 개수) + n * 1.2).round(1),
                      "껍질": (rng.normal(3.0, 0.4, 개수) - n * 0.3).round(2),
                      "등급": np.array(["하", "중", "상"])[n]}
                     ).to_csv(폴더 / f"{이름}.csv", index=False)


class 귤_데이터셋(Dataset):
    """CSV 한 파일을 읽어 (특성 3개, 등급 번호)를 돌려준다."""

    특성_열 = ["무게", "당도", "껍질"]
    등급_번호 = {"하": 0, "중": 1, "상": 2}

    def __init__(self, 경로: Path, 평균: torch.Tensor | None = None,
                 표준편차: torch.Tensor | None = None) -> None:
        표 = pd.read_csv(경로)
        self.X = torch.tensor(표[self.특성_열].to_numpy(), dtype=torch.float32)
        self.y = torch.tensor(표["등급"].map(self.등급_번호).to_numpy())
        # 기준값을 받지 않았으면 이 파일에서 구한다(학습용일 때만 그래야 한다)
        self.평균 = self.X.mean(dim=0) if 평균 is None else 평균
        self.표준편차 = self.X.std(dim=0) if 표준편차 is None else 표준편차

    def __len__(self) -> int:
        return len(self.y)

    def __getitem__(self, i: int) -> tuple[torch.Tensor, torch.Tensor]:
        x = (self.X[i] - self.평균) / self.표준편차   # 꺼낼 때 표준화
        return x, self.y[i]


학습 = 귤_데이터셋(폴더 / "학습.csv")
검증 = 귤_데이터셋(폴더 / "검증.csv", 학습.평균, 학습.표준편차)  # 학습 기준을 그대로 넘긴다
```

```
print("샘플 수:", len(학습), len(검증))
print("학습 기준 평균:", 학습.평균)
x, y = 학습[0]
print("0번 샘플:", x, y)
xb, yb = next(iter(DataLoader(학습, batch_size=8)))
print("배치 모양:", tuple(xb.shape), tuple(yb.shape), yb.dtype)
print("등급별 개수:", torch.bincount(학습.y).tolist())
```

```
샘플 수: 160 40
학습 기준 평균: tensor([86.9087, 10.7006,  2.5896])
0번 샘플: tensor([ 2.0863, -0.1422, -1.1424]) tensor(2)
배치 모양: (8, 3) (8,) torch.int64
등급별 개수: [46, 62, 52]
```

앞부분의 `if not ... exists():` 덩어리는 이 예제만 따로 실행해도 돌아가도록 앞 블록과 같은 데이터를 다시 만드는 부분이니 건너뛰어도 됩니다. 볼 곳은 `귤_데이터셋` 클래스예요.

- `__init__`: `pd.read_csv`로 표를 읽고, 특성 세 열은 `float32` 텐서로, 등급 글자는 `map`으로 0·1·2 번호로 바꿔 정수 텐서로 만듭니다. 4장에서 NumPy에서 넘어온 값은 `float64`라 32비트로 바꾸라고 했던 것 기억나죠? `dtype=torch.float32`가 그 역할입니다. 정답을 정수(`int64`) 번호로 만든 것은 6장의 분류 손실 함수가 이 형태를 기대하기 때문이에요.
- `__getitem__`: 특성에서 평균을 빼고 표준편차로 나눠 표준화한 뒤 돌려줍니다. 0번 귤은 무게가 학습 데이터 평균보다 표준편차의 2배쯤 무거운 귤이라는 뜻이죠.
- **배치**: DataLoader로 8개를 묶으니 특성은 `(8, 3)`, 정답은 `(8,)`의 `int64`가 됐습니다.

여기서 가장 중요한 줄은 `검증 = 귤_데이터셋(..., 학습.평균, 학습.표준편차)`입니다. 검증 데이터를 표준화할 때 **검증 데이터의 평균이 아니라 학습 데이터의 평균**을 썼어요. 2장에서 본 **데이터 누수** 이야기입니다. 검증 데이터는 "모델이 처음 보는 데이터"를 흉내 내야 하는데, 표준화 기준에 검증 데이터가 섞이면 시험 문제를 미리 엿본 셈이 됩니다. 그래서 기준값은 학습 데이터에서만 구하고, 검증·테스트 데이터에는 그 값을 넘겨서 씁니다. scikit-learn에서 `fit`은 학습 데이터에만, `transform`은 양쪽에 하던 것과 같은 원칙이에요.

마지막 줄의 `torch.bincount`는 0, 1, 2가 각각 몇 번 나왔는지 세어 줍니다. 하·중·상이 46, 62, 52개로 적당히 섞여 있네요. 새 데이터셋을 만들면 클래스별 개수부터 세어 보는 습관을 들이세요. 7.9절에서 클래스 개수가 크게 차이 날 때의 대처를 다룹니다.

Q

Q. CSV 파일이 수십 GB라서 한 번에 못 읽으면요? 몇 가지 길이 있습니다. 한 번만 전처리해서 NumPy 파일로 저장해 두고 `np.load(..., mmap_mode="r")`로 열면, 디스크에 둔 채 필요한 줄만 읽어 올 수 있어요(메모리 매핑). 순서대로 읽어도 되는 데이터라면 pandas의 `read_csv(..., chunksize=...)`로 조금씩 읽어 7.3절의 IterableDataset으로 흘려보내는 방법도 있습니다. 어느 쪽이든 원칙은 같아요. 창고 목록만 들고 있다가 필요할 때 꺼내는 것이죠.

7.5 직접 만든 Dataset ②: 이미지 폴더

폴더 이름이 곧 정답

이미지 데이터는 보통 이런 모양으로 정리되어 있습니다. 클래스마다 폴더를 하나씩 만들고, 그 안에 해당하는 그림을 넣는 거예요. 폴더 이름이 곧 정답이 됩니다.

```
data/shapes/
├── circle/     000.png, 001.png, ...
├── square/     000.png, 001.png, ...
└── triangle/   000.png, 001.png, ...
```

예제용 그림을 직접 만들어 보겠습니다. 32 × 32 흰 바탕에 동그라미, 네모, 세모 중 하나를 무작위 색과 크기, 위치로 그려요. 이 코드는 7.11절 실습에서도 다시 쓰니 **`도형_만들기.py`라는 파일로 저장**해 두세요. 맨 아래의 `if __name__ == "__main__":`은 1권 14장에서 배운 장치입니다. 이 파일을 직접 실행하면 그림 폴더를 만들고 내용을 보여 주지만, 다른 파일에서 `import`하면 함수만 빌려줘요.

```python
# 도형_만들기.py — 예제용 도형 그림 폴더를 만드는 모듈
import random
from pathlib import Path

from PIL import Image, ImageDraw


def 도형_그리기(모양: str, rng: random.Random) -> Image.Image:
    """흰 바탕 32×32 그림에 무작위 색·크기·위치로 도형 하나를 그린다."""
    그림 = Image.new("RGB", (32, 32), "white")
    펜 = ImageDraw.Draw(그림)
```

```
    크기 = rng.randint(12, 22)
    x, y = rng.randint(1, 31 - 크기), rng.randint(1, 31 - 크기)
    색 = tuple(rng.randint(0, 200) for _ in range(3))
    if 모양 == "circle":
        펜.ellipse([x, y, x + 크기, y + 크기], fill=색)
    elif 모양 == "square":
        펜.rectangle([x, y, x + 크기, y + 크기], fill=색)
    else:
        펜.polygon([(x + 크기 // 2, y), (x, y + 크기), (x + 크기, y + 크기)], fill=색)
    return 그림


def 도형_폴더_만들기(뿌리: Path, 개수: dict[str, int], 씨앗: int = 0) -> None:
    """뿌리/모양/번호.png 구조로 그림을 저장한다. 폴더가 이미 있으면 건너뛴다."""
    if 뿌리.exists():
        return
    rng = random.Random(씨앗)
    for 모양, n in 개수.items():
        폴더 = 뿌리 / 모양                    # 폴더 이름이 곧 정답(클래스)
        폴더.mkdir(parents=True)
        for i in range(n):
            도형_그리기(모양, rng).save(폴더 / f"{i:03d}.png")


if __name__ == "__main__":
    뿌리 = Path("data/shapes")
    도형_폴더_만들기(뿌리, {"circle": 20, "square": 20, "triangle": 20})
    for 폴더 in sorted(뿌리.iterdir()):
        파일들 = sorted(폴더.glob("*.png"))
        print(f"{폴더.name}: {len(파일들)}개", [f.name for f in 파일들[:3]], "...")
```

```
circle: 20개 ['000.png', '001.png', '002.png'] ...
square: 20개 ['000.png', '001.png', '002.png'] ...
triangle: 20개 ['000.png', '001.png', '002.png'] ...
```

그림 그리기에는 파이썬 이미지 라이브러리 Pillow(`PIL`)를 썼습니다. `ImageDraw`의 `ellipse`, `rect-`

`angle`, `polygon`으로 도형을 그리고 `save`로 PNG 파일을 저장해요. `random.Random(씨앗)`으로 난수 생성기를 따로 만들어서, 언제 실행해도 같은 그림이 나오게 했습니다. 폴더 이름을 영어로 지은 것은 운영체제마다 한글 파일 이름을 저장하는 방식이 조금씩 달라서 생기는 문제를 피하려는 거예요.

그림 폴더를 읽는 Dataset

이제 이 폴더를 읽는 Dataset입니다. CSV와 달리 그림은 수가 많고 커질 수 있으니, `__init__`에서는 **(파일 경로, 정답) 목록만** 만들고 그림 파일은 `__getitem__`에서 엽니다. 직접 만든 것과 torchvision이 준비해 둔 `ImageFolder`를 나란히 비교해 볼게요.

```
from pathlib import Path

import numpy as np
import torch
from PIL import Image
from torch.utils.data import Dataset
from torchvision.datasets import ImageFolder

from 도형_만들기 import 도형_폴더_만들기

뿌리 = Path("data/shapes")
도형_폴더_만들기(뿌리, {"circle": 20, "square": 20, "triangle": 20})


class 그림폴더_데이터셋(Dataset):
    """뿌리/클래스이름/파일.png 구조의 폴더를 읽는다."""

    def __init__(self, 뿌리: Path) -> None:
        self.클래스들 = sorted(p.name for p in 뿌리.iterdir() if p.is_dir())
        self.클래스_번호 = {이름: i for i, 이름 in enumerate(self.클래스들)}
        # 그림을 읽지 않고 (경로, 정답) 목록만 만들어 둔다
        self.목록 = [(경로, self.클래스_번호[이름])
                     for 이름 in self.클래스들
                     for 경로 in sorted((뿌리 / 이름).glob("*.png"))]

    def __len__(self) -> int:
        return len(self.목록)
```

```
    def __getitem__(self, i: int) -> tuple[torch.Tensor, int]:
        경로, 정답 = self.목록[i]
        with Image.open(경로) as 그림:                  # 필요한 그 순간에 파일을 연다
            배열 = np.array(그림.convert("RGB"))         # (세로, 가로, 색), 0~255 정수
        텐서 = torch.from_numpy(배열).permute(2, 0, 1)   # (색, 세로, 가로)
        return 텐서.float() / 255, 정답


우리것 = 그림폴더_데이터셋(뿌리)
print("클래스:", 우리것.클래스_번호, "샘플 수:", len(우리것))
그림, 정답 = 우리것[25]
print("25번:", tuple(그림.shape), 그림.dtype, "정답", 정답, "경로", 우리것.목록[25][0].name)

기성품 = ImageFolder(뿌리)                              # torchvision이 준비해 둔 같은 기능
print("ImageFolder:", 기성품.class_to_idx, len(기성품))
그림2, 정답2 = 기성품[25]
print("25번:", type(그림2).__name__, 그림2.size, "정답", 정답2, 기성품.samples[25][0])
```

```
클래스: {'circle': 0, 'square': 1, 'triangle': 2} 샘플 수: 60
25번: (3, 32, 32) torch.float32 정답 1 경로 005.png
ImageFolder: {'circle': 0, 'square': 1, 'triangle': 2} 60
25번: Image (32, 32) 정답 1 data/shapes/square/005.png
```

직접 만든 `그림폴더_데이터셋`의 흐름을 따라가 보겠습니다.

1. **클래스 목록**: 하위 폴더 이름을 정렬해서 `circle`, `square`, `triangle`에 0, 1, 2 번호를 붙였습니다. **정렬**이 중요해요. 정렬하지 않으면 컴퓨터마다 폴더를 읽는 순서가 달라서 같은 그림의 정답 번호가 바뀔 수 있습니다.
2. **(경로, 정답) 목록**: 그림 60장의 경로와 정답만 담았습니다. 그림 자체는 아직 하나도 읽지 않았어요.
3. **꺼내기**: `Image.open`으로 파일을 열어 NumPy 배열로 바꾸면 모양이 (세로, 가로, 색)입니다. 4장 4.5절에서 본 것처럼 PyTorch는 (색, 세로, 가로)를 원하니 `permute(2, 0, 1)`로 순서를 바꾸고, 0~255 정수를 255로 나눠 0~1 실수로 만들었어요. 파일은 1권 12장에서 배운 `with` 문으로 열어서, 다 읽으면 저절로 닫히게 했습니다.

25번 샘플은 정답 1(`square`) 폴더의 `005.png`입니다. 동그라미 20장(0~19번) 다음이 네모의 0번(20번)이니 계산이 맞죠.

아래쪽 `ImageFolder`는 이 일을 한 줄로 해 줍니다. `class_to_idx`가 우리 것과 똑같고, 25번 샘플도 같은 파일을 가리켜요. 차이가 하나 있습니다. `ImageFolder`는 그림을 텐서로 바꾸지 않고 **Pillow 그림(`Image`) 그대로** 돌려줍니다. 텐서로 바꾸는 일은 다음 절의 transform에 맡기라는 뜻이에요. 실무에서는 폴더 구조가 이 모양이면 `ImageFolder`를 쓰고, 정답이 CSV에 따로 적혀 있는 식으로 구조가 다르면 위처럼 직접 만듭니다.

7.6 transforms: 꺼낼 때 손질하기

변환 단계를 숫자로 따라가기

창고에서 꺼낸 재료를 그대로 손님에게 내지는 않죠. 씻고, 썰고, 간을 맞춥니다. 그림도 모델에 넣기 전에 손질이 필요해요. Pillow 그림을 텐서로 바꾸고, 0~255 정수를 실수로 바꾸고, 값의 범위를 고르게 맞추는 일입니다. torchvision은 이런 손질 도구를 `transforms`에 모아 두었고, 이 책은 최신 버전인 `torchvision.transforms.v2`를 씁니다. 세로 2 × 가로 3짜리 아주 작은 그림으로 각 단계가 숫자를 어떻게 바꾸는지 따라가 보겠습니다.

```
import numpy as np
import torch
from PIL import Image
from torchvision.transforms import v2

# 세로 2 × 가로 3짜리 아주 작은 컬러 그림 (예제용 숫자)
배열 = np.array([[[255, 0, 0], [0, 255, 0], [0, 0, 255]],
                 [[255, 255, 255], [128, 128, 128], [0, 0, 0]]], dtype=np.uint8)
그림 = Image.fromarray(배열)

텐서 = v2.ToImage()(그림)                                    # PIL → (색, 세로, 가로) 텐서
print("ToImage :", tuple(텐서.shape), 텐서.dtype)
print(텐서[0])                                                # 빨강 채널만
실수 = v2.ToDtype(torch.float32, scale=True)(텐서)           # 0~255 → 0~1
print("ToDtype :", 실수.dtype)
print(실수[0])
정규화 = v2.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])(실수)
```

```
print("Normalize:")
print(정규화[0])

한번에 = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True),
                    v2.Normalize(mean=[0.5] * 3, std=[0.5] * 3)])
print("Compose와 같은가:", torch.equal(한번에(그림), 정규화))
```

```
ToImage : (3, 2, 3) torch.uint8
tensor([[255,   0,   0],
        [255, 128,   0]], dtype=torch.uint8)
ToDtype : torch.float32
tensor([[1.0000, 0.0000, 0.0000],
        [1.0000, 0.5020, 0.0000]])
Normalize:
tensor([[ 1.0000, -1.0000, -1.0000],
        [ 1.0000,  0.0039, -1.0000]])
Compose와 같은가: True
```

빨강 채널의 숫자만 따라가 보세요.

- **`ToImage()`**: Pillow 그림을 **`(색, 세로, 가로)`** 텐서로 바꿉니다. 앞 절에서 손으로 한 `permute`를 대신해 주는 거예요. 값은 아직 0~255의 `uint8`입니다. 첫 줄의 빨강·초록·파랑 칸에서 빨강 채널 값은 255, 0, 0이고, 둘째 줄의 흰색·회색·검은색 칸은 255, 128, 0이에요.
- **`ToDtype(torch.float32, scale=True)`**: `float32`로 바꾸면서 255로 나눠 0~1로 맞춥니다. 128은 0.5020이 됐어요. `scale=True`를 빼먹으면 자료형만 바뀌고 값은 0~255 그대로라서, 뒤의 정규화가 엉뚱한 결과를 냅니다. 자주 하는 실수예요.
- **`Normalize(mean, std)`**: 채널마다 **`(값 - 평균) / 표준편차`**를 계산합니다. 평균 0.5, 표준편차 0.5를 주면 0~1이 -1~1로 바뀌어요. 1은 1, 0은 -1, 0.502는 0.0039가 됐죠.

6장에서 쓴 `transforms.ToTensor()`는 앞의 두 단계(텐서로 바꾸기와 0~1로 맞추기)를 한 번에 하는 기존 방식입니다. 6장과 9장은 결과를 서로 비교하려고 이 방식을 끝까지 쓰지만, 새로 짜는 코드라면 v2처럼 두 단계를 나눠 쓰기를 권해요. 자료형과 크기 조절을 따로 정할 수 있어서 실수가 줄기 때문입니다.

`Compose`는 여러 변환을 순서대로 이어 붙여 하나의 변환으로 만듭니다. 결과가 단계별로 한 것과 똑같았어요. 실전에서는 이 `Compose`를 Dataset의 `transform`으로 넘겨서, `__getitem__`이 샘플을 꺼낼

때마다 적용되게 합니다.

정규화에 쓰는 평균과 표준편차는 7.4절의 표준화처럼 **학습 데이터에서 구한 값**을 씁니다. 7.11절 실습에서 직접 구해 볼 거예요. 인터넷의 예제 코드에서 `mean=[0.485, 0.456, 0.406]`이라는 숫자를 자주 보게 되는데, 이것은 ImageNet이라는 큰 사진 데이터셋에서 구한 값입니다. ImageNet으로 미리 학습된 모델을 가져다 쓸 때는 그 모델이 배운 기준에 맞춰야 해서 이 값을 그대로 써요. 10장 전이 학습에서 다시 만납니다.

자주 쓰는 변환을 정리하면 이렇습니다.

변환	하는 일	증강인가
`v2.ToImage()`	Pillow 그림·NumPy 배열 → `(색, 세로, 가로)` 텐서	아니요
`v2.ToDtype(torch.float32, scale=True)`	실수로 바꾸고 0~1로 맞추기	아니요
`v2.Normalize(mean, std)`	채널마다 `(값 - 평균) / 표준편차`	아니요
`v2.Resize(크기)`	크기 바꾸기	아니요
`v2.CenterCrop(크기)`	가운데 잘라 내기	아니요
`v2.RandomHorizontalFlip(p)`	확률 p로 좌우 뒤집기	예
`v2.RandomCrop(크기, padding)`	테두리를 두르고 무작위 위치 잘라 내기	예
`v2.RandomResizedCrop(크기)`	무작위 크기·위치로 잘라 원하는 크기로	예
`v2.RandomRotation(각도)`	무작위 각도로 돌리기	예
`v2.ColorJitter(...)`	밝기·대비·채도를 무작위로 바꾸기	예

데이터 증강: 같은 그림을 조금씩 다르게

표 아래쪽의 `Random`으로 시작하는 변환들이 **데이터 증강**(data augmentation) 입니다. 고양이 사진을 좌우로 뒤집어도 여전히 고양이죠. 조금 잘라 내도, 살짝 어둡게 해도 고양이입니다. 이렇게 **정답**

이 바뀌지 않는 범위에서 그림을 살짝 바꿔 보여 주면, 모델은 "고양이 귀가 꼭 왼쪽 위에 있어야 하는 건 아니구나"를 배웁니다. 같은 사진을 계속 똑같이 보여 줘서 사진을 통째로 외워 버리는 일도 줄어들고요. 증강이 정말 과적합을 줄이는지 숫자로 확인하는 일은 8장 실습에서 하고, 여기서는 증강이 숫자를 어떻게 바꾸는지만 정확히 보겠습니다.

```
import torch
from torchvision.transforms import v2

torch.manual_seed(0)
그림 = torch.arange(1, 13, dtype=torch.uint8).reshape(1, 3, 4)   # 흑백 1채널, 세로 3 × 가로 4
print(f"원본:\n{그림[0]}")

print(f"좌우 뒤집기:\n{v2.RandomHorizontalFlip(p=1.0)(그림)[0]}")
print(f"상하 뒤집기:\n{v2.RandomVerticalFlip(p=1.0)(그림)[0]}")

자르기 = v2.RandomCrop(size=(3, 4), padding=1)     # 테두리를 0으로 1칸 두른 뒤 원래 크기로 자르기
for i in range(2):
    print(f"무작위 자르기 {i + 1}:\n{자르기(그림)[0]}")

반반 = v2.RandomHorizontalFlip(p=0.5)
뒤집힌_횟수 = sum(int(not torch.equal(반반(그림), 그림)) for _ in range(1000))
print("p=0.5로 1000번 중 뒤집힌 횟수:", 뒤집힌_횟수)
```

```
원본:
tensor([[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12]], dtype=torch.uint8)
좌우 뒤집기:
tensor([[ 4,  3,  2,  1],
        [ 8,  7,  6,  5],
        [12, 11, 10,  9]], dtype=torch.uint8)
상하 뒤집기:
tensor([[ 9, 10, 11, 12],
        [ 5,  6,  7,  8],
        [ 1,  2,  3,  4]], dtype=torch.uint8)
무작위 자르기 1:
```

```
tensor([[ 0,  5,  6,  7],
        [ 0,  9, 10, 11],
        [ 0,  0,  0,  0]], dtype=torch.uint8)
무작위 자르기 2:
tensor([[ 0,  1,  2,  3],
        [ 0,  5,  6,  7],
        [ 0,  9, 10, 11]], dtype=torch.uint8)
p=0.5로 1000번 중 뒤집힌 횟수: 493
```

`p=1.0`으로 확률을 100%로 만들어 확실히 뒤집어 봤습니다. 좌우 뒤집기는 각 줄의 순서를 거꾸로 (1, 2, 3, 4 → 4, 3, 2, 1), 상하 뒤집기는 줄의 순서를 거꾸로 바꿔요.

`RandomCrop(size=(3, 4), padding=1)`은 먼저 그림 둘레에 0을 한 칸씩 둘러 5 × 6으로 키운 뒤, 그 안에서 원래 크기인 3 × 4를 **무작위 위치**에서 잘라 냅니다. 첫 번째는 오른쪽 위로, 두 번째는 오른쪽으로 한 칸 밀린 것처럼 보이죠. 그림 속 물체를 위아래, 좌우로 조금씩 옮기는 효과가 있어요. 테두리에 채울 값은 `fill`로 정할 수 있습니다. 실습에서는 바탕이 흰색이라 흰색으로 채울 거예요.

`p=0.5`로 1000번 적용했더니 493번 뒤집혔습니다. 반쯤 뒤집힌다는 뜻이 그대로 나타났죠. 증강 변환은 이렇게 **부를 때마다 새로 주사위를 굴립니다.** 이것이 Dataset 안에서 어떤 의미인지 그림으로 보겠습니다.

```
import torch
from torch.utils.data import Dataset
from torchvision.transforms import v2

ㄱ = torch.tensor([[0, 0, 0, 0, 0, 0],
                   [0, 1, 1, 1, 1, 0],
                   [0, 0, 0, 0, 1, 0],
                   [0, 0, 0, 0, 1, 0],
                   [0, 0, 0, 0, 1, 0],
                   [0, 0, 0, 0, 0, 0]], dtype=torch.uint8).unsqueeze(0)   # (1, 6, 6)


class 글자_데이터셋(Dataset):
    """그림 한 장짜리 데이터셋. 꺼낼 때마다 transform을 새로 적용한다."""

    def __init__(self, transform: v2.Transform | None = None) -> None:
```

```
        self.그림들 = [ㄱ]
        self.transform = transform

    def __len__(self) -> int:
        return len(self.그림들)

    def __getitem__(self, i: int) -> torch.Tensor:
        그림 = self.그림들[i]
        return self.transform(그림) if self.transform else 그림


def 그려_보기(그림들: list[torch.Tensor]) -> None:
    """여러 그림을 #과 . 으로 나란히 찍는다."""
    for 줄 in range(그림들[0].shape[1]):
        print("   ".join("".join("#" if v else "." for v in g[0, 줄].tolist()) for g in 그림들))


torch.manual_seed(1)
증강 = v2.Compose([v2.RandomHorizontalFlip(p=0.5),
                   v2.RandomCrop(size=6, padding=1)])
데이터 = 글자_데이터셋(transform=증강)
그려_보기([데이터[0] for _ in range(5)])        # 같은 0번 샘플을 다섯 번 꺼낸다
```

```
..####   ......   ####..   ..####   .####.
.....#   .####.   #.....   ..#...   ....#.
.....#   ....#.   #.....   ..#...   ....#.
.....#   ....#.   #.....   ..#...   ....#.
......   ....#.   ......   ......   ......
......   ......   ......   ......   ......
```

그림 한 장(한글 자음 ㄱ)뿐인 데이터셋에서 **같은 0번 샘플을 다섯 번** 꺼냈습니다. 그런데 결과가 매번 다르죠. 첫 번째는 오른쪽 위로 한 칸씩 밀렸고, 두 번째는 원래 모습 그대로이고, 세 번째와 네 번째는 좌우가 뒤집힌 채 위치도 옮겨졌고, 다섯 번째는 위로 한 칸 올라갔어요. `transform`이 `__getitem__` 안에서 꺼낼 때마다 새로 적용되기 때문입니다.

여기서 증강의 중요한 성질이 나옵니다. **증강은 데이터 개수를 늘리지 않습니다.** 데이터셋의 길이는

그대로 1이고, 한 에폭에 이 그림은 한 번만 나와요. 대신 에폭이 바뀔 때마다 조금씩 다른 모습으로 나옵니다. 100에폭을 학습하면 모델은 같은 그림의 100가지 변형을 보게 되는 셈이죠. 파일을 미리 100배로 불려 저장하지 않아도 되니 디스크도 아낍니다.

Q

Q. 증강은 많이 할수록 좋은가요? 아닙니다. 증강은 **정답이 바뀌지 않는 범위**에서만 해야 해요. 위 결과의 세 번째 그림을 보세요. 좌우가 뒤집힌 ㄱ은 더 이상 ㄱ이 아닙니다. 글자를 알아보는 모델이라면 좌우 뒤집기를 쓰면 안 되는 거죠. 숫자 6을 상하로 뒤집으면 9처럼 보이고, 왼쪽·오른쪽을 구분하는 표지판도 뒤집으면 뜻이 반대가 됩니다. 증강을 고를 때는 "이렇게 바꾼 그림에도 사람이 같은 정답을 붙일까?"를 먼저 물어보세요. 그리고 증강은 **학습 데이터에만** 합니다. 검증 데이터는 실제로 만날 그림 그대로여야 정확한 점수를 잴 수 있어요. 7.8절에서 둘을 나누는 방법을 봅니다.

7.7 collate_fn: 길이가 다른 샘플 묶기

기본 규칙이 실패할 때

그림은 미리 크기를 맞춰 두면 되지만, 문장은 길이가 제각각입니다. "최고"는 두 글자, "기다리다 지쳤어요"는 아홉 글자죠. 예제용으로 만든 짧은 식당 리뷰 여덟 개를 데이터셋으로 만들어 보겠습니다. 모델은 글자를 바로 읽지 못하니, 글자마다 번호를 붙여 숫자 목록으로 바꿉니다. 번호는 1부터 붙이고 0은 비워 두었어요. 잠시 뒤에 쓸 자리입니다.

```
import torch
from torch.utils.data import DataLoader, Dataset

# 예제용으로 만든 짧은 식당 리뷰와 라벨(1: 좋음, 0: 나쁨)
리뷰들 = [("맛있어요", 1), ("너무 짜요", 0), ("또 오고 싶어요", 1), ("별로", 0),
        ("친절하고 깔끔해요", 1), ("다시는 안 가요", 0), ("최고", 1), ("기다리다 지쳤어요", 0)]


class 리뷰_데이터셋(Dataset):
    """문장을 글자 번호 텐서로 바꿔 돌려준다. 0번은 빈칸 채우기(패딩)용으로 비워 둔다."""

    def __init__(self, 자료: list[tuple[str, int]]) -> None:
        self.자료 = 자료
        글자들 = sorted({글자 for 문장, _ in 자료 for 글자 in 문장})
```

```
        self.글자_번호 = {글자: i + 1 for i, 글자 in enumerate(글자들)}

    def __len__(self) -> int:
        return len(self.자료)

    def __getitem__(self, i: int) -> tuple[torch.Tensor, int]:
        문장, 라벨 = self.자료[i]
        return torch.tensor([self.글자_번호[글자] for 글자 in 문장]), 라벨


데이터 = 리뷰_데이터셋(리뷰들)
print("글자 종류:", len(데이터.글자_번호))
for i in range(3):
    번호들, 라벨 = 데이터[i]
    print(f"{리뷰들[i][0]} → {번호들.tolist()}, 라벨 {라벨}")

try:
    next(iter(DataLoader(데이터, batch_size=4)))
except RuntimeError as e:
    print("오류:", e)
```

```
글자 종류: 30
맛있어요 → [13, 22, 19, 21], 라벨 1
너무 짜요 → [7, 14, 1, 25, 21], 라벨 0
또 오고 싶어요 → [10, 1, 20, 3, 1, 17, 19, 21], 라벨 1
오류: stack expects each tensor to be equal size, but got [4] at entry 0 and [5] at entry 1
```

여덟 문장에 쓰인 서로 다른 글자는 띄어쓰기까지 30종류입니다. "맛있어요"는 네 글자라 숫자 4개, "너무 짜요"는 띄어쓰기 포함 다섯 글자라 숫자 5개가 됐어요. 띄어쓰기의 번호가 1번이라 "너무 짜요"의 세 번째 숫자가 1이죠.

그런데 이걸 DataLoader로 4개씩 묶으려 하자 오류가 났습니다. 7.3절에서 본 기본 규칙은 텐서를 `torch.stack`으로 쌓는데, 4장에서 배운 것처럼 `stack`은 모양이 같은 텐서만 쌓을 수 있어요. 4장 "오류 메시지 읽는 법"에서 미리 봤던 바로 그 오류로, "0번은 [4]인데 1번은 [5]"라고 정확히 짚어 줍니다.

글자를 번호로 바꾸는 이 방식은 가장 단순한 형태입니다. 실제 한국어 처리에서는 글자보다 큰 단

위로 자르는 토크나이저를 쓰는데, 13장에서 자세히 다뤄요. 어떤 방식으로 자르든 "문장마다 길이가 다르다"는 문제는 똑같이 남습니다.

패딩으로 길이 맞추기

해결책은 짧은 문장의 뒤를 **빈칸 번호(0)로 채워서** 가장 긴 문장에 길이를 맞추는 것입니다. 이것을 **패딩(padding)** 이라고 해요. 0번을 비워 둔 이유가 이것입니다. DataLoader의 `collate_fn` 자리에 이 일을 하는 함수를 넘기면 기본 규칙 대신 그 함수로 배치를 만들어요.

```
import torch
from torch.nn.utils.rnn import pad_sequence
from torch.utils.data import DataLoader

# 앞 블록의 리뷰 데이터셋이 돌려준 샘플 세 개를 그대로 옮겨 적었다
샘플들 = [(torch.tensor([13, 22, 19, 21]), 1),
         (torch.tensor([7, 14, 1, 25, 21]), 0),
         (torch.tensor([10, 1, 20, 3, 1, 17, 19, 21]), 1)]


def 패딩_묶기(배치: list[tuple[torch.Tensor, int]]) -> dict[str, torch.Tensor]:
    """길이가 다른 문장들을 가장 긴 문장에 맞춰 0으로 채워 묶는다."""
    문장들 = [문장 for 문장, _ in 배치]
    return {
        "입력": pad_sequence(문장들, batch_first=True, padding_value=0),
        "길이": torch.tensor([len(문장) for 문장 in 문장들]),
        "정답": torch.tensor([라벨 for _, 라벨 in 배치]),
    }


배치 = 패딩_묶기(샘플들)                    # collate_fn은 그냥 함수라 직접 불러 볼 수 있다
print(배치["입력"])
print("길이:", 배치["길이"].tolist(), "정답:", 배치["정답"].tolist())
print("진짜 글자 자리(마스크):")
print((배치["입력"] != 0).int())

로더 = DataLoader(샘플들, batch_size=2, collate_fn=패딩_묶기)   # 리스트도 데이터셋이 된다
for 배치 in 로더:
```

```
    print(tuple(배치["입력"].shape), 배치["길이"].tolist())
```

```
tensor([[13, 22, 19, 21,  0,  0,  0,  0],
        [ 7, 14,  1, 25, 21,  0,  0,  0],
        [10,  1, 20,  3,  1, 17, 19, 21]])
길이: [4, 5, 8] 정답: [1, 0, 1]
진짜 글자 자리(마스크):
tensor([[1, 1, 1, 1, 0, 0, 0, 0],
        [1, 1, 1, 1, 1, 0, 0, 0],
        [1, 1, 1, 1, 1, 1, 1, 1]], dtype=torch.int32)
(2, 5) [4, 5]
(1, 8) [8]
```

`collate_fn`은 특별한 것이 아니라 **샘플 목록을 받아 배치 하나를 돌려주는 평범한 함수**입니다. 그래서 DataLoader 없이 직접 불러 볼 수 있었어요. 안에서 하는 일은 세 가지입니다.

- **`pad_sequence`**: 길이가 4, 5, 8인 문장을 가장 긴 8에 맞춰 뒤를 0으로 채우고 쌓았습니다. `batch_first=True`를 줘야 `(문장 수, 길이)` 순서가 돼요. 주지 않으면 `(길이, 문장 수)`로 나옵니다.
- **길이**: 원래 길이 `[4, 5, 8]`을 따로 담아 두었습니다. 11장의 순환 신경망은 이 값을 받아 채운 칸을 건너뛸 수 있어요.
- **마스크**: `배치["입력"] != 0`은 진짜 글자 자리만 1인 표를 만듭니다. 4장에서 본 마스크 그대로예요. 12장의 어텐션은 이 마스크로 "채운 칸은 보지 마"라고 알려 줍니다.

마지막 부분에서는 DataLoader에 **평범한 리스트**를 데이터셋으로 넣었습니다. 7.1절 Q&A에서 말했듯 리스트도 `__len__`과 `__getitem__`이 있으니 데이터셋 노릇을 할 수 있어요. 배치마다 길이가 다른 것도 보세요. 첫 배치는 가장 긴 문장이 5글자라 `(2, 5)`, 둘째 배치는 문장 하나라 `(1, 8)`입니다. 패딩은 **배치 안에서만** 맞추기 때문에, 데이터 전체에서 가장 긴 문장에 맞출 필요가 없어요.

비슷한 길이끼리 묶기

그런데 패딩한 0도 모델은 계산합니다. 채운 칸이 많을수록 헛일이 늘어나는 거죠. 길이가 3~60글자로 제각각인 문장 1,000개로 얼마나 낭비되는지 재 보겠습니다.

```
import torch
from torch.nn.utils.rnn import pad_sequence
from torch.utils.data import DataLoader

torch.manual_seed(0)
# 예제용: 길이가 3~60글자로 제각각인 문장 1,000개 (글자 번호는 1~9)
문장들 = [torch.randint(1, 10, (int(n),)) for n in torch.randint(3, 61, (1000,))]


def 패딩_묶기(배치: list[torch.Tensor]) -> torch.Tensor:
    return pad_sequence(배치, batch_first=True, padding_value=0)


def 패딩_비율(로더: DataLoader) -> float:
    """배치에 들어간 칸 중 0으로 채운 칸의 비율."""
    전체 = 빈칸 = 0
    for 배치 in 로더:
        전체 += 배치.numel()
        빈칸 += (배치 == 0).sum().item()
    return 빈칸 / 전체


g = torch.Generator().manual_seed(0)
그냥 = DataLoader(문장들, batch_size=32, shuffle=True, collate_fn=패딩_묶기, generator=g)
print(f"그냥 섞었을 때 패딩 비율: {패딩_비율(그냥):.1%}")

# 길이순으로 줄 세운 뒤 32개씩 끊고, 묶음의 순서만 섞는다
길이순 = sorted(range(len(문장들)), key=lambda i: len(문장들[i]))
묶음들 = [길이순[i:i + 32] for i in range(0, len(길이순), 32)]
묶음들 = [묶음들[i] for i in torch.randperm(len(묶음들), generator=g).tolist()]
비슷한_길이끼리 = DataLoader(문장들, batch_sampler=묶음들, collate_fn=패딩_묶기)
print(f"비슷한 길이끼리 묶었을 때: {패딩_비율(비슷한_길이끼리):.1%}")
print("배치 수:", len(그냥), len(비슷한_길이끼리))
```

```
그냥 섞었을 때 패딩 비율: 46.1%
비슷한 길이끼리 묶었을 때: 2.6%
배치 수: 32 32
```

그냥 섞어서 32개씩 묶었더니 배치에 들어간 칸의 46.1%가 0으로 채운 칸이었습니다. 짧은 문장과 긴 문장이 한 배치에 섞이니 짧은 쪽이 대부분 빈칸으로 채워진 거예요. 계산의 절반 가까이가 헛일입니다.

두 번째 방법은 문장을 **길이순으로 줄 세운 뒤 32개씩 끊고**, 그렇게 만든 묶음들의 순서만 섞었습니다. 비슷한 길이끼리 한 배치에 모이니 패딩 비율이 2.6%로 줄었어요. 배치 수는 32개로 같습니다. DataLoader의 `batch_sampler`에 "배치마다 어떤 번호들을 묶을지" 목록을 직접 넘겨서 만든 거예요. `batch_sampler`를 쓰면 묶는 방법을 우리가 정했으니 `batch_size`와 `shuffle`은 주지 않습니다.

이 방법을 **버킷팅(bucketing)** 이라고 부르고, 문장을 다루는 학습에서 널리 씁니다. 다만 배치 안의 문장들이 너무 비슷해지면 섞는 효과가 줄어드니, 실무에서는 데이터를 큰 덩어리로 섞은 다음 덩어리 안에서만 길이순으로 정렬하는 식으로 절충해요.

7.8 학습/검증 나누기: random_split과 재현 가능한 분할

random_split과 씨앗

2장에서 데이터를 학습/검증/테스트로 나누는 이유를 봤죠. 모델이 처음 보는 데이터에서 얼마나 잘 하는지 재려면 학습에 쓰지 않은 데이터를 떼어 둬야 합니다. 7.4절의 귤 데이터처럼 처음부터 파일이 나뉘어 있으면 좋겠지만, 데이터셋 하나만 받았다면 PyTorch의 `random_split`으로 나눠요.

```
import torch
from torch.utils.data import TensorDataset, random_split

데이터 = TensorDataset(torch.arange(100).float(), torch.arange(100))   # 샘플 100개 (예제용)

학습, 검증 = random_split(데이터, [0.8, 0.2], generator=torch.Generator().manual_seed(42))
print("크기:", len(학습), len(검증), type(학습).__name__)
print("검증에 뽑힌 번호(앞 8개):", 검증.indices[:8])
print("검증 2번 샘플:", 검증[2], "= 원래 데이터의", 검증.indices[2], "번")
```

```
다시_학습, 다시_검증 = random_split(데이터, [0.8, 0.2],
↪  generator=torch.Generator().manual_seed(42))
print("같은 씨앗이면 같은 분할:", 검증.indices == 다시_검증.indices)
print("학습과 검증이 겹치나:", bool(set(학습.indices) & set(검증.indices)))

학, 검, 시 = random_split(데이터, [70, 15, 15], generator=torch.Generator().manual_seed(42))
print("세 갈래:", len(학), len(검), len(시))
```

```
크기: 80 20 Subset
검증에 뽑힌 번호(앞 8개): [2, 91, 8, 53, 30, 90, 26, 23]
검증 2번 샘플: (tensor(8.), tensor(8)) = 원래 데이터의 8 번
같은 씨앗이면 같은 분할: True
학습과 검증이 겹치나: False
세 갈래: 70 15 15
```

- `[0.8, 0.2]`: 비율로 나눕니다. 100개가 80개와 20개로 나뉘었어요. `[70, 15, 15]`처럼 개수로 줘도 되고, 세 갈래 이상으로도 나눌 수 있습니다.
- `Subset`: 결과는 데이터를 복사한 것이 아니라 **원래 데이터셋과 번호 목록**을 들고 있는 `Subset`입니다. 검증의 2번 샘플을 달라고 하면 `indices[2]`인 원래 8번 샘플을 꺼내 줘요. 4장에서 슬라이스가 원본을 들여다보는 창이었던 것처럼, Subset도 원래 데이터셋을 들여다보는 창입니다.
- `generator`: 씨앗을 고정한 `Generator`를 넘기면 언제 실행해도 같은 분할이 나옵니다. 이게 정말 중요해요. 씨앗 없이 나누면 실행할 때마다 검증 데이터가 바뀌어서, 어제 모델과 오늘 모델의 점수를 비교할 수가 없습니다. 어제는 쉬운 문제로, 오늘은 어려운 문제로 시험을 본 셈이니까요. 학습과 검증이 겹치지 않는다는 것도 확인했습니다.

transform이 다를 때의 함정

그런데 7.6절에서 증강은 **학습 데이터에만** 해야 한다고 했죠. `random_split`으로 나눈 학습용과 검증용은 같은 원래 데이터셋을 들여다보는 창이라서, 원래 데이터셋에 증강 transform을 걸면 검증용에도 증강이 걸립니다. 실제로 확인해 보겠습니다. 증강 흉내로 작은 잡음을 더하는 `흔들기`를 transform으로 썼어요.

```
from collections.abc import Callable

import torch
from torch.utils.data import Dataset, Subset, random_split


class 숫자_데이터셋(Dataset):
    """꺼낼 때 transform을 적용하는 아주 작은 데이터셋 (예제용)."""

    def __init__(self, transform: Callable[[torch.Tensor], torch.Tensor] | None = None) -> None:
        self.X = torch.arange(10).float()
        self.transform = transform

    def __len__(self) -> int:
        return len(self.X)

    def __getitem__(self, i: int) -> torch.Tensor:
        x = self.X[i]
        return self.transform(x) if self.transform else x


def 흔들기(x: torch.Tensor) -> torch.Tensor:
    """학습용 증강 흉내: 작은 잡음을 더한다."""
    return x + torch.randn(()) * 0.1


torch.manual_seed(0)
g = torch.Generator().manual_seed(42)

# 잘못된 방법: 하나를 나누면 검증 쪽도 같은 transform을 쓴다
학습, 검증 = random_split(숫자_데이터셋(transform=흔들기), [0.7, 0.3], generator=g)
print("잘못: 검증 0번을 두 번 꺼내면", round(검증[0].item(), 4), round(검증[0].item(), 4))

# 바른 방법: transform이 다른 두 데이터셋을 만들고, 같은 번호로 나눈다
번호 = torch.randperm(10, generator=g).tolist()
학습 = Subset(숫자_데이터셋(transform=흔들기), 번호[:7])
검증 = Subset(숫자_데이터셋(transform=None), 번호[7:])
```

```
print("바름: 검증 0번을 두 번 꺼내면", 검증[0].item(), 검증[0].item())
print("학습 0번을 두 번 꺼내면", round(학습[0].item(), 4), round(학습[0].item(), 4))
```

```
잘못: 검증 0번을 두 번 꺼내면 9.1541 8.9707
바름: 검증 0번을 두 번 꺼내면 0.0 0.0
학습 0번을 두 번 꺼내면 2.7821 3.0568
```

잘못된 방법에서는 검증 0번 샘플을 두 번 꺼냈더니 9.1541, 8.9707로 값이 매번 달라졌습니다. 원래 값은 9인데 잡음이 섞인 거예요. 검증 점수가 잴 때마다 흔들리면 믿을 수가 없죠.

바른 방법은 **transform만 다른 데이터셋 두 개**를 만들고, **같은 번호 목록**을 나눠서 각각 `Subset`으로 감싸는 것입니다. 번호는 `torch.randperm`으로 0~9를 씨앗 고정해 섞은 뒤 앞 7개는 학습, 뒤 3개는 검증으로 잘랐어요. 이제 검증 0번은 몇 번 꺼내도 0.0으로 같고, 학습 0번만 꺼낼 때마다 달라집니다. 번호를 한곳에서 섞어 잘랐으니 두 데이터셋이 겹칠 일도 없어요. 7.11절 실습이 바로 이 방법을 씁니다.

데이터셋을 두 번 만들면 메모리가 두 배로 들지 않을까 걱정될 수 있는데, 7.5절의 그림 폴더 데이터셋처럼 `__init__`에서 경로 목록만 들고 있다면 거의 차이가 없어요. 무거운 일은 `__getitem__`에서 한다는 7.1절의 원칙이 여기서도 도움이 됩니다.

Q

Q. 테스트 데이터도 이렇게 나누면 되나요? 나눌 수는 있지만, 테스트 데이터는 되도록 **처음부터 따로** 두고 마지막에 한 번만 쓰세요. 검증 점수를 보면서 모델을 이리저리 고치다 보면 모르는 사이에 모델이 검증 데이터에 맞춰지기 때문입니다. 날짜 순서가 있는 데이터라면 무작위로 섞어 나누면 안 되고 과거로 학습해 미래로 검증해야 한다는 점도 2장에서 본 그대로예요. 그럴 때는 `random_split` 대신 날짜순으로 정렬한 번호를 앞뒤로 잘라 `Subset`을 만들면 됩니다.

7.9 불균형 데이터: WeightedRandomSampler

공장에서 나온 제품 1,000개 중 불량이 50개뿐이라고 해 봅시다. 2장에서 `class_weight`로 다뤄 봤듯이 이런 **불균형 데이터**로 그냥 학습하면, 모델은 "전부 정상"이라고만 답해도 95%를 맞히니 불량을 찾는 법을 열심히 배우지 않습니다. 한 가지 대처는 **드문 클래스를 더 자주 뽑아서** 모델에게 자주 보여 주는 것이에요. DataLoader의 `sampler` 자리에 `WeightedRandomSampler`를 넘기면 됩니다.

```
import torch
from torch.utils.data import DataLoader, TensorDataset, WeightedRandomSampler

# 예제용 불균형 데이터: 정상 950개, 불량 50개
정답 = torch.cat([torch.zeros(950, dtype=torch.long), torch.ones(50, dtype=torch.long)])
데이터 = TensorDataset(torch.arange(1000), 정답)        # 입력 자리에는 샘플 번호를 넣어 둔다


def 한_에폭_세기(로더: DataLoader) -> None:
    """한 에폭 동안 나온 클래스별 개수와, 불량 샘플이 몇 종류였는지 센다."""
    개수 = torch.zeros(2, dtype=torch.long)
    본_불량 = set()
    배치별_불량 = []
    for 번호, y in 로더:
        개수 += torch.bincount(y, minlength=2)
        본_불량.update(번호[y == 1].tolist())
        배치별_불량.append(int(y.sum()))
    print(f"  정상 {개수[0]}개, 불량 {개수[1]}개 (서로 다른 불량 샘플 {len(본_불량)}종)")
    print(f"  배치마다 불량 개수(앞 8배치): {배치별_불량[:8]}")


g = torch.Generator().manual_seed(0)
print("그냥 섞기:")
한_에폭_세기(DataLoader(데이터, batch_size=50, shuffle=True, generator=g))

클래스별_개수 = torch.bincount(정답)                    # [950, 50]
샘플_가중치 = 1.0 / 클래스별_개수[정답].float()          # 드문 클래스일수록 크게
print(f"가중치: 정상 {샘플_가중치[0]:.5f}, 불량 {샘플_가중치[-1]:.5f}")
샘플러 = WeightedRandomSampler(샘플_가중치, num_samples=len(데이터),
                               replacement=True, generator=g)
print("WeightedRandomSampler:")
한_에폭_세기(DataLoader(데이터, batch_size=50, sampler=샘플러))
```

```
그냥 섞기:
  정상 950개, 불량 50개 (서로 다른 불량 샘플 50종)
  배치마다 불량 개수(앞 8배치): [2, 4, 2, 2, 2, 4, 5, 3]
```

```
가중치: 정상 0.00105, 불량 0.02000
WeightedRandomSampler:
  정상 500개, 불량 500개 (서로 다른 불량 샘플 50종)
  배치마다 불량 개수(앞 8배치): [27, 25, 24, 22, 29, 26, 23, 23]
```

먼저 그냥 섞은 경우입니다. 한 에폭에 정상 950개, 불량 50개가 정확히 한 번씩 나왔고, 50개씩 묶은 배치를 앞에서부터 8개 보면 불량이 2~5개뿐이었어요. 모델 입장에서는 배치마다 정상 품목만 잔뜩 보는 셈이죠.

`WeightedRandomSampler`는 샘플마다 **뽑힐 가중치**를 받습니다. 여기서는 "1 / 그 클래스의 개수"로 줬어요. 정상 샘플 하나는 1/950 ≈ 0.00105, 불량 샘플 하나는 1/50 = 0.02입니다. 이렇게 하면 정상 950개의 가중치 합과 불량 50개의 가중치 합이 둘 다 1이 되어, 두 클래스가 **같은 확률**로 뽑혀요. `클래스별_개수[정답]`은 4장에서 본 목록으로 고르기 인덱싱으로, 샘플마다 자기 클래스의 개수를 꺼내 1,000개짜리 텐서를 만든 것입니다.

결과를 보면 한 에폭 1,000번 뽑기에서 정상과 불량이 500개씩, 배치마다 불량이 22~29개로 거의 반반이 됐습니다. 무작위로 뽑으니 정확히 반반일 필요는 없고 씨앗에 따라 조금씩 달라져요.

여기서 놓치면 안 되는 숫자가 있습니다. **서로 다른 불량 샘플은 여전히 50종**이에요. 불량 500번은 같은 50개를 평균 10번씩 반복해서 뽑은 것입니다. `replacement=True`(뽑은 것을 다시 넣고 또 뽑기)여야 이렇게 같은 샘플을 여러 번 뽑을 수 있어요. 새로운 불량 사례가 생긴 것이 아니니, 모델이 그 50개를 통째로 외워 버릴 위험이 커집니다. 그래서 이 샘플러는 7.6절의 데이터 증강과 함께 쓰는 경우가 많아요. 같은 불량품이 뽑힐 때마다 조금씩 다른 모습으로 나오게 하는 거죠.

몇 가지를 더 기억해 두세요.

- `sampler`가 순서를 정하므로 `shuffle=True`와 함께 쓸 수 없습니다. 함께 주면 오류가 나요.
- 샘플러는 **학습용 로더에만** 씁니다. 검증 데이터는 실제 비율 그대로 평가해야 실제 상황의 성능을 알 수 있어요.
- 다른 대처로, 뽑는 비율은 그대로 두고 손실 함수에서 드문 클래스의 틀림에 더 큰 벌점을 주는 방법도 있습니다. 6장의 `CrossEntropyLoss`에 `weight`를 넘기는 방식이에요. 어느 쪽이 나은지는 데이터마다 달라서, 8장의 방법으로 검증 곡선을 보며 정합니다.

7.10 num_workers와 병목 찾기

일꾼 프로세스와 `__main__` 가드

지금까지는 샘플을 꺼내는 일을 학습하는 프로그램이 직접 했습니다. 주방으로 치면 요리사가 재료 손질까지 혼자 하는 거예요. 그동안 화구(GPU)는 놀고 있죠. `num_workers`를 1 이상으로 주면 DataLoader가 **일꾼 프로세스**를 따로 띄워서, 모델이 계산하는 동안 일꾼들이 다음 배치를 미리 꺼내 둡니다. 1권 15장에서 "요리사를 여러 명 고용한다"고 비유했던 멀티프로세싱이 바로 이것이에요.

그런데 macOS와 Windows에서 처음 `num_workers`를 쓰면 많은 사람이 이상한 오류를 만납니다. 직접 확인해 보죠. 똑같은 코드를 `if __name__ == "__main__":` 가드 없이 쓴 파일과 가드 안에 넣은 파일로 저장해서 따로 실행해 봤습니다.

```
import subprocess
import sys
from pathlib import Path

가드_없음 = """
import torch
from torch.utils.data import DataLoader, TensorDataset

로더 = DataLoader(TensorDataset(torch.arange(8)), batch_size=4, num_workers=2)
for (배치,) in 로더:
    print("배치:", 배치.tolist())
"""

가드_있음 = """
import torch
from torch.utils.data import DataLoader, TensorDataset


def main() -> None:
    로더 = DataLoader(TensorDataset(torch.arange(8)), batch_size=4, num_workers=2)
    for (배치,) in 로더:
        print("배치:", 배치.tolist())


if __name__ == "__main__":
    main()
```

```
"""

for 이름, 본문 in [("가드_없음.py", 가드_없음), ("가드_있음.py", 가드_있음)]:
    Path(이름).write_text(본문, encoding="utf-8")            # 파일로 저장해서
    결과 = subprocess.run([sys.executable, 이름],              # 따로 실행해 본다
                          capture_output=True, text=True, timeout=120)
    print(f"--- {이름} (종료 코드 {결과.returncode})")
    if 결과.stdout:
        print(결과.stdout.strip())
    줄들 = [줄.strip() for 줄 in 결과.stderr.splitlines()]
    첫줄 = "An attempt has been made to start a new process before the"
    if 첫줄 in 줄들:                                          # 일꾼 프로세스가 남긴 설명
        시작 = 줄들.index(첫줄)
        print("\n".join(줄들[시작:시작 + 6]))
```

```
--- 가드_없음.py (종료 코드 1)
An attempt has been made to start a new process before the
current process has finished its bootstrapping phase.

This probably means that you are not using fork to start your
child processes and you have forgotten to use the proper idiom
in the main module:
--- 가드_있음.py (종료 코드 0)
배치: [0, 1, 2, 3]
배치: [4, 5, 6, 7]
```

가드가 없는 파일은 종료 코드 1로 실패했고, 일꾼 프로세스가 "현재 프로세스가 준비 단계를 마치기 전에 새 프로세스를 띄우려 했다. 메인 모듈에 알맞은 관용구를 쓰지 않은 것 같다"는 설명을 남겼습니다. 가드 안에 넣은 파일은 배치 두 개를 잘 받았어요.

이유는 일꾼을 띄우는 방식에 있습니다. macOS와 Windows는 일꾼 프로세스를 새로 시작하면서 **우리 파일을 처음부터 다시 import** 합니다(이 방식을 spawn이라고 해요). 가드가 없으면 일꾼이 import하는 순간 파일 맨 아래의 DataLoader 반복이 또 실행되고, 일꾼이 다시 일꾼을 띄우려다 멈추는 거예요. 1권 14장에서 배운 대로 `if __name__ == "__main__":` 아래의 코드는 **직접 실행할 때만** 돌고 import할 때는 건너뛰니, 일꾼들은 함수와 클래스 정의만 가져가고 반복은 하지 않습니다. Linux는 파이썬 3.13까지 프로세스를 통째로 복제하는 방식(fork)이 기본이라 가드 없이도 돌아가

지만, 3.14부터는 Linux도 다른 방식(forkserver)이 기본이 되어 가드가 필요합니다. 어디서나 돌아가는 코드를 쓰려면 **`num_workers`를 쓸 때는 항상 가드 안에서** 실행하는 습관을 들이세요. 이 장의 나머지 예제도 모두 그렇게 씁니다.

같은 이유로 Jupyter 노트북에서 `num_workers`를 쓰면 데이터셋 클래스를 노트북 셀 안에 정의했을 때 일꾼이 그 클래스를 찾지 못해 실패할 수 있어요. 그럴 때는 데이터셋 클래스를 `.py` 파일로 옮겨 import하면 해결됩니다.

일꾼이 몇이면 빨라질까

일꾼 수에 따라 얼마나 달라지는지 재 보겠습니다. 실제 디스크와 모델을 쓰면 컴퓨터마다 결과가 너무 달라서, 샘플 하나를 읽는 데 10ms, 배치 하나로 학습하는 데 50ms가 걸린다고 정하고 `time.sleep`으로 흉내 냈어요. 그리고 학습 반복 중에 **다음 배치를 기다린 시간**을 따로 쟀습니다.

```
import time

import torch
from torch.utils.data import DataLoader, Dataset

읽기_시간 = 0.01      # 샘플 하나를 디스크에서 읽고 푸는 데 10ms 걸린다고 치자
계산_시간 = 0.05      # 배치 하나로 모델을 학습시키는 데 50ms 걸린다고 치자


class 느린_데이터셋(Dataset):
    """꺼낼 때마다 읽기_시간만큼 기다리는 데이터셋 (디스크 읽기 흉내)."""

    def __len__(self) -> int:
        return 160

    def __getitem__(self, i: int) -> torch.Tensor:
        time.sleep(읽기_시간)
        return torch.full((3, 32, 32), float(i))


def 한_에폭(로더: DataLoader) -> tuple[float, float]:
    """한 에폭 전체 시간과, 그중 다음 배치를 기다린 시간을 잰다."""
    대기 = 0.0
```

```
    시작 = time.perf_counter()
    반복자 = iter(로더)
    while True:
        t = time.perf_counter()
        배치 = next(반복자, None)              # 다음 배치가 올 때까지 기다린다
        대기 += time.perf_counter() - t
        if 배치 is None:
            break
        time.sleep(계산_시간)                  # 학습 계산 흉내
    return time.perf_counter() - 시작, 대기


def main() -> None:
    for 일꾼 in [0, 2, 4]:
        로더 = DataLoader(느린_데이터셋(), batch_size=16, num_workers=일꾼,
                          persistent_workers=일꾼 > 0)   # 에폭이 끝나도 일꾼을 살려 둔다
        첫_전체, _ = 한_에폭(로더)
        전체, 대기 = 한_에폭(로더)
        print(f"num_workers={일꾼}: 첫 에폭 {첫_전체:.1f}초, 둘째 에폭 {전체:.1f}초 "
              f"(그중 데이터 대기 {대기 / 전체:.0%})")


if __name__ == "__main__":
    main()
```

```
num_workers=0: 첫 에폭 2.5초, 둘째 에폭 2.5초 (그중 데이터 대기 78%)
num_workers=2: 첫 에폭 1.6초, 둘째 에폭 1.1초 (그중 데이터 대기 51%)
num_workers=4: 첫 에폭 1.3초, 둘째 에폭 0.7초 (그중 데이터 대기 27%)
```

이 책의 실행 환경에서 잰 값이라 여러분의 숫자는 조금 다를 수 있지만, 흐름은 같을 거예요.

- **num_workers=0**: 한 에폭에 약 2.5초가 걸렸고, 그중 78%가 데이터를 기다린 시간입니다. 배치 하나(16개)를 꺼내는 데 160ms, 계산은 50ms이니 대부분의 시간 동안 계산 쪽이 놀고 있었던 거예요. 이런 상태를 **데이터 병목**이라고 합니다.
- **num_workers=2, 4**: 일꾼이 늘수록 둘째 에폭이 1.1초, 0.7초로 줄고 대기 비율도 약 50%, 약 25%로 떨어졌습니다. 일꾼 넷이 동시에 배치를 준비하니, 계산 50ms 동안 다음 배치가 거의

준비되어 있는 거죠.

- **첫 에폭이 더 느린 이유**: 일꾼 프로세스를 띄우고 각자 torch를 import하는 준비 시간이 들기 때문입니다. `persistent_workers=True`를 주면 에폭이 끝나도 일꾼을 살려 두어서, 둘째 에폭부터는 이 비용이 없어요. 주지 않으면 에폭마다 일꾼을 새로 띄웁니다.

일꾼은 많을수록 좋은 게 아닙니다. 일꾼마다 프로세스 하나와 그만큼의 메모리를 쓰고, 계산보다 데이터 준비가 빨라지면 더 늘려도 소용이 없어요. 위 예에서 대기 비율이 이미 4분의 1 정도까지 떨어졌으니 8개로 늘려도 얻을 것이 크지 않습니다. 보통 CPU 코어 수의 절반 정도에서 시작해 **대기 비율을 재 보며** 조절하세요. 일꾼마다 미리 준비해 두는 배치 수는 `prefetch_factor`(기본 2)로 바꿀 수 있습니다.

NVIDIA GPU를 쓴다면 `pin_memory=True`도 함께 주는 경우가 많습니다. 배치를 GPU로 옮기기 좋은 특별한 메모리 영역에 담아 두어 복사를 빠르게 하는 옵션이에요. Apple MPS에서는 아직 지원되지 않아 "MPS에서는 쓰지 않는다"는 경고만 나오므로 이 책의 예제에서는 쓰지 않았습니다. GPU 쪽 성능은 16장에서 자세히 봅니다.

IterableDataset과 일꾼

7.3절에서 IterableDataset을 일꾼과 함께 쓸 때 주의할 점이 있다고 했죠. 주문 번호 0~5를 흘려보내는 데이터셋을 일꾼 2명으로 읽어 보겠습니다.

```
from collections.abc import Iterator

from torch.utils.data import DataLoader, IterableDataset, get_worker_info


class 주문_흐름(IterableDataset):
    """주문 번호를 차례로 흘려보낸다. 나눠_읽기=True면 일꾼끼리 몫을 나눈다."""

    def __init__(self, 주문_수: int, 나눠_읽기: bool) -> None:
        self.주문_수 = 주문_수
        self.나눠_읽기 = 나눠_읽기

    def __iter__(self) -> Iterator[int]:
        정보 = get_worker_info()                # 일꾼 안이면 (번호, 전체 수)를 알려 준다
        if not self.나눠_읽기 or 정보 is None:
            return iter(range(self.주문_수))
```

```
        # 일꾼 0은 0, 2, 4, ...  일꾼 1은 1, 3, 5, ... 처럼 건너뛰며 맡는다
        return iter(range(정보.id, self.주문_수, 정보.num_workers))


def main() -> None:
    for 나눠_읽기 in [False, True]:
        로더 = DataLoader(주문_흐름(6, 나눠_읽기), batch_size=None, num_workers=2)
        받은_것 = sorted(int(x) for x in 로더)
        print(f"나눠_읽기={나눠_읽기}: {받은_것}")


if __name__ == "__main__":
    main()
```

```
나눠_읽기=False: [0, 0, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5]
나눠_읽기=True: [0, 1, 2, 3, 4, 5]
```

`나눠_읽기=False`일 때 모든 주문이 **두 번씩** 나왔습니다. 일꾼마다 데이터셋을 통째로 복사해 가서 각자 처음부터 끝까지 흘려보냈기 때문이에요. 맵 스타일 데이터셋은 DataLoader가 "너는 3번, 너는 7번" 하고 번호를 나눠 주니 이런 일이 없지만, 번호가 없는 IterableDataset은 일꾼끼리 몫을 나누는 일을 **우리가 직접** 해야 합니다.

`get_worker_info()`는 일꾼 안에서 부르면 자기 번호(`id`)와 전체 일꾼 수(`num_workers`)를 알려 주고, 일꾼 없이 실행하면 `None`을 돌려줍니다. 이걸로 일꾼 0은 0, 2, 4번을, 일꾼 1은 1, 3, 5번을 맡게 하니 중복 없이 하나씩 나왔어요. `batch_size=None`은 배치로 묶지 않고 샘플을 하나씩 받겠다는 뜻입니다.

병목 찾기 체크리스트

학습이 생각보다 느리다면, 모델을 고치기 전에 데이터 파이프라인부터 의심해 보세요. 순서는 이렇습니다.

1. **기다린 시간을 잽니다.** 위의 `한_에폭`처럼 다음 배치를 기다린 시간과 계산 시간을 나눠 재면, 느린 쪽이 데이터인지 모델인지 바로 알 수 있어요. 대기 비율이 높으면 데이터 병목입니다.
2. **`num_workers`를 늘려 봅니다.** 0에서 2, 4, 8로 늘려 가며 대기 비율이 떨어지는지 봅니다. `persistent_workers=True`도 함께 주세요.

3. **`__getitem__`을 가볍게 합니다.** 큰 사진을 매번 읽어서 줄이고 있다면, 미리 작은 크기로 줄여 저장해 두세요. 매번 똑같이 나오는 손질(크기 바꾸기, 글자를 번호로 바꾸기)은 한 번만 해서 저장하고, 매번 달라야 하는 증강만 `__getitem__`에 남깁니다.
4. **배치 하나를 직접 들여다봅니다.** 속도와 별개로, 학습 전에 배치 하나를 꺼내 모양, 자료형, 값의 범위, 클래스 개수를 찍어 보는 습관이 많은 사고를 막아 줍니다. 다음 절 실습이 바로 그 일을 해요.

계산 쪽이 느린 경우, 즉 GPU가 일을 제대로 하고 있는지 자세히 들여다보는 프로파일러는 16장에서 다룹니다.

7.11 실습: 도형 그림 파이프라인 한 바퀴

이 장에서 배운 것을 모두 이어서 학습 직전까지의 파이프라인을 완성해 보겠습니다. 데이터는 7.5절의 `도형_만들기.py`로 만든 예제용 도형 그림인데, 이번에는 일부러 **불균형**하게 동그라미 120장, 네모 60장, 세모 20장을 만들어요. 할 일은 다섯 가지입니다.

1. 씨앗을 고정해 학습 80%, 검증 20%로 나누기
2. 학습용 그림에서만 색 채널별 평균과 표준편차 구하기
3. 학습용에는 증강과 정규화, 검증용에는 정규화만 걸기
4. 학습용 로더에는 WeightedRandomSampler, 둘 다 일꾼 2명
5. 한 에폭을 돌며 배치 모양, 값의 범위, 클래스 개수를 확인하고, 검증 데이터가 매번 같은지 확인하기

`도형_만들기.py`를 같은 폴더에 두고 실행하세요. 일꾼을 쓰니 7.10절에서 본 대로 `main()` 함수와 `__main__` 가드로 감쌌습니다.

```
from pathlib import Path

import torch
from torch.utils.data import DataLoader, Subset, WeightedRandomSampler
from torchvision.datasets import ImageFolder
from torchvision.transforms import v2

from 도형_만들기 import 도형_폴더_만들기

뿌리 = Path("data/shapes_practice")
```

```
씨앗 = 42


def 학습_검증_번호(전체_수: int, 검증_비율: float = 0.2) -> tuple[list[int], list[int]]:
    """씨앗을 고정해 섞은 번호를 학습용과 검증용으로 자른다."""
    번호 = torch.randperm(전체_수, generator=torch.Generator().manual_seed(씨앗)).tolist()
    검증_수 = int(전체_수 * 검증_비율)
    return 번호[검증_수:], 번호[:검증_수]


def 채널_통계(데이터: Subset) -> tuple[list[float], list[float]]:
    """학습용 그림만으로 색 채널별 평균과 표준편차를 구한다."""
    모두 = torch.stack([그림 for 그림, _ in 데이터])            # (N, 3, 32, 32)
    return 모두.mean(dim=(0, 2, 3)).tolist(), 모두.std(dim=(0, 2, 3)).tolist()


def 로더_만들기() -> tuple[DataLoader, DataLoader, list[str]]:
    학습_번호, 검증_번호 = 학습_검증_번호(len(ImageFolder(뿌리)))
    기본 = [v2.ToImage(), v2.ToDtype(torch.float32, scale=True)]
    평균, 표준편차 = 채널_통계(Subset(ImageFolder(뿌리, transform=v2.Compose(기본)), 학습_번호))
    print("학습용 채널 평균:", [round(m, 3) for m in 평균])

    정규화 = v2.Normalize(mean=평균, std=표준편차)
    학습_변환 = v2.Compose([v2.ToImage(),
                          v2.RandomHorizontalFlip(p=0.5),
                          v2.RandomCrop(size=32, padding=3, fill=255),   # 흰 바탕색으로 테두리
                          v2.ToDtype(torch.float32, scale=True), 정규화])
    검증_변환 = v2.Compose(기본 + [정규화])                              # 증강 없음

    학습 = Subset(ImageFolder(뿌리, transform=학습_변환), 학습_번호)
    검증 = Subset(ImageFolder(뿌리, transform=검증_변환), 검증_번호)

    정답 = torch.tensor(학습.dataset.targets)[학습_번호]                # 학습용 샘플의 정답들
    가중치 = 1.0 / torch.bincount(정답)[정답].float()
    샘플러 = WeightedRandomSampler(가중치, num_samples=len(학습), replacement=True,
                                   generator=torch.Generator().manual_seed(씨앗))
```

```
    print("학습용 클래스별 개수:", torch.bincount(정답).tolist())

    학습_로더 = DataLoader(학습, batch_size=32, sampler=샘플러, num_workers=2)
    검증_로더 = DataLoader(검증, batch_size=64, shuffle=False, num_workers=2)
    return 학습_로더, 검증_로더, 학습.dataset.classes


def main() -> None:
    torch.manual_seed(씨앗)
    도형_폴더_만들기(뿌리, {"circle": 120, "square": 60, "triangle": 20}, 씨앗=1)
    학습_로더, 검증_로더, 클래스들 = 로더_만들기()
    print("클래스:", 클래스들, "| 배치 수:", len(학습_로더), len(검증_로더))

    에폭_개수 = torch.zeros(len(클래스들), dtype=torch.long)
    for 단계, (그림, 정답) in enumerate(학습_로더):
        에폭_개수 += torch.bincount(정답, minlength=len(클래스들))
        if 단계 == 0:
            print("첫 배치:", tuple(그림.shape), 그림.dtype, 정답.dtype)
            print(f"첫 배치 평균 {그림.mean():.3f}, 표준편차 {그림.std():.3f}")
    print("한 에폭 동안 본 클래스별 개수:", 에폭_개수.tolist())

    첫번째 = torch.cat([그림 for 그림, _ in 검증_로더])
    두번째 = torch.cat([그림 for 그림, _ in 검증_로더])
    print("검증 데이터:", tuple(첫번째.shape), "| 두 번 읽어도 같은가:", torch.equal(첫번째, 두번
    ↪ 째))


if __name__ == "__main__":
    main()
```

```
학습용 채널 평균: [0.841, 0.842, 0.845]
학습용 클래스별 개수: [98, 46, 16]
클래스: ['circle', 'square', 'triangle'] | 배치 수: 5 1
첫 배치: (32, 3, 32, 32) torch.float32 torch.int64
첫 배치 평균 0.089, 표준편차 0.959
한 에폭 동안 본 클래스별 개수: [50, 52, 58]
```

```
검증 데이터: (40, 3, 32, 32) ¦ 두 번 읽어도 같은가: True
```

단계마다 짚어 보겠습니다.

- **나누기 (`학습_검증_번호`)**: 200장의 번호를 씨앗 42로 섞어 앞 40개를 검증, 나머지 160개를 학습으로 잘랐습니다. 7.8절의 바른 방법 그대로, 이 번호 목록을 transform이 다른 두 `ImageFolder`에 똑같이 써서 `Subset`을 만들었어요.
- **정규화 기준 (`채널_통계`)**: 증강 없이 텐서로만 바꾼 학습용 그림 160장을 `torch.stack`으로 쌓아 `(160, 3, 32, 32)`를 만들고, `dim=(0, 2, 3)`으로 그림·세로·가로 방향을 한꺼번에 접어 채널별 평균을 구했습니다. 4장에서 "`dim`으로 지정한 차원이 사라진다"고 했죠. 색 채널 3개만 남아요. 평균이 0.84 정도로 높은 것은 흰 바탕(1.0)이 그림의 대부분을 차지하기 때문입니다. 검증용 그림은 이 계산에 넣지 않았어요(데이터 누수 방지).
- **변환**: 학습용은 좌우 뒤집기와 무작위 자르기를 거쳐 정규화합니다. 도형은 좌우로 뒤집어도 같은 도형이니 뒤집기를 써도 괜찮아요. 자르기의 테두리는 `fill=255`로 흰색을 채워, 바탕과 똑같이 보이게 했습니다. 검증용은 `기본 + [정규화]`로 증강 없이 같은 정규화만 겁니다.
- **샘플러**: 학습용 160장 중 동그라미 98장, 네모 46장, 세모 16장이었습니다. `학습.dataset.targets`는 `ImageFolder`가 들고 있는 전체 정답 목록이고, 거기서 학습용 번호만 골라 가중치를 만들었어요.
- **한 에폭**: 학습용 로더는 160장을 32장씩 5배치, 검증용은 40장을 한 배치로 냅니다. 첫 배치는 `(32, 3, 32, 32)`의 `float32` 그림과 `int64` 정답으로, 6장에서 만든 것 같은 분류 모델에 바로 넣을 수 있는 모양이에요. 정규화한 값의 평균은 0.089, 표준편차는 0.959로 0과 1에 가깝습니다. 딱 0과 1이 아닌 것은 증강으로 흰 테두리가 섞이고 샘플러가 그림을 고르게 뽑지 않기 때문이에요.
- **클래스 균형**: 한 에폭 동안 동그라미 50, 네모 52, 세모 58장을 봤습니다. 원래 98 : 46 : 16이던 비율이 거의 1 : 1 : 1이 됐어요. 세모 16장이 평균 3번 넘게 반복해서 나왔지만, 증강 덕분에 나올 때마다 조금씩 다른 모습입니다.
- **검증 데이터**: 검증 로더를 두 번 돌려 받은 그림 40장이 완전히 같았습니다(`True`). 증강이 걸리지 않았고 섞지도 않았으니까요. 이래야 에폭마다 잰 검증 점수를 서로 비교할 수 있습니다.

이 파이프라인에 6장의 모델과 학습 루프를 붙이면 바로 학습을 시작할 수 있어요. 데이터 개수와 그림 크기만 바꾸면 실제 이미지 분류 프로젝트에서도 뼈대는 그대로 씁니다. 시간이 있다면 세 가지를 바꿔 보세요. 증강에 `v2.RandomRotation(15)`를 더하면 첫 배치의 평균이 어떻게 바뀌는지, 샘플러를 빼고 `shuffle=True`로 바꾸면 에폭의 클래스 개수가 어떻게 되는지, `num_workers=0`으로 바꾸면 가드 없이도 돌아가는지 직접 확인해 보는 겁니다.

7.12 정리와 다음 장 미리보기

이 장에서 데이터가 창고에서 모델까지 흘러가는 길을 만들었습니다.

- Dataset: `__len__`과 `__getitem__` 두 메서드의 약속. `__init__`은 목록만 가볍게, 무거운 일은 꺼낼 때. 메모리에 다 들어가면 `TensorDataset`, 번호 없이 흘려보내야 하면 `IterableDataset`
- **직접 만든 Dataset**: CSV는 읽어서 텐서로(표준화 기준은 학습 데이터에서만), 이미지 폴더는 (경로, 정답) 목록만 들고 꺼낼 때 열기. 폴더 구조가 맞으면 `ImageFolder`
- DataLoader: `batch_size`, `shuffle`(씨앗은 `generator`로), `drop_last`. 샘플의 생김새를 유지하며 묶는 기본 collate 규칙
- transforms: `ToImage` → `ToDtype(scale=True)` → `Normalize`를 `Compose`로. 증강은 꺼낼 때마다 새로 적용되고, 정답이 바뀌지 않는 범위에서 학습용에만
- collate_fn: 길이가 다른 샘플은 `pad_sequence`로 채우고 길이·마스크를 함께. 비슷한 길이끼리 묶으면 패딩 낭비가 크게 준다
- **분할과 샘플링**: `random_split`과 씨앗 고정 `Generator`, transform이 다르면 같은 번호로 두 `Subset`. 불균형에는 `WeightedRandomSampler`(같은 샘플 반복에 주의)
- **속도**: `num_workers`는 `__main__` 가드 안에서, `persistent_workers`로 일꾼 살려 두기, 기다린 시간을 재서 병목 찾기. IterableDataset은 일꾼끼리 몫을 직접 나누기

다음 8장에서는 이렇게 준비한 학습용·검증용 데이터를 가지고 **모델을 잘 학습시키는 법**을 다룹니다. 학습 손실은 계속 줄어드는데 검증 손실이 다시 올라가는 순간, 즉 과적합을 학습 곡선에서 알아보는 것부터 시작해요. 이 장에서 맛만 본 데이터 증강이 실제로 과적합을 얼마나 줄이는지도, 드롭아웃과 가중치 감쇠 같은 다른 처방과 함께 숫자로 비교해 보겠습니다.

08

학습 잘 시키기

학습 목표

- 학습 손실과 검증 손실 곡선을 읽고 과적합과 과소적합을 구분하기
- weight decay, 드롭아웃, 배치 정규화로 모델이 외우지 않고 배우게 만들기
- 가중치 초기화와 학습률 스케줄러가 학습에 주는 영향을 숫자로 확인하기
- 조기 종료를 직접 구현하고, 그래디언트 클리핑으로 폭주를 막기
- 같은 결과를 다시 얻는 재현성 체크리스트를 갖추기
- 손실이 줄지 않을 때 원인을 순서대로 좁혀 가는 디버깅 방법 익히기

들어가며: 문제집 답을 통째로 외운 학생

시험 기간에 친구가 이런 공부법을 쓴다고 해 봅시다. 문제집 200문제를 풀면서 원리를 이해하는 대신 **"3번 문제 답은 ②, 4번 문제 답은 ⑤"** 하고 답을 통째로 외우는 거예요. 문제집을 다시 풀게 하면 100점이 나옵니다. 그런데 진짜 시험에서 숫자 하나만 바뀌어도 손을 못 대죠. 게다가 그 문제집 해답지에 오타가 섞여 있었다면 오타까지 그대로 외웠을 겁니다.

6장에서 만든 신경망도 똑같은 짓을 합니다. 학습 데이터에서는 점수가 계속 오르는데, 처음 보는 데이터에서는 점수가 멈추거나 오히려 떨어져요. 1장에서 직관으로 만나 본 **과적합(overfitting)** 이 바로 이것입니다. 반대로 공부를 너무 대충 해서 문제집 점수조차 낮은 상태는 **과소적합(underfitting)** 이라고 불러요.

이 장은 "학습 루프는 돌릴 줄 아는데, 결과가 마음에 안 들 때 무엇을 해야 하는가"에 대한 이야기입니다. 순서는 이렇습니다.

1. 학습/검증 곡선으로 지금 모델이 외우는 중인지, 덜 배운 상태인지 **진단**합니다(8.1절).
2. 외우지 못하게 막는 **정규화** 도구 세 가지를 봅니다. weight decay, 드롭아웃, 배치 정규화예요(8.2~8.5절).
3. 학습이 시작부터 꼬이지 않게 하는 **가중치 초기화**와, 걸음 크기를 조절하는 **학습률 스케줄러**를 봅니다(8.6~8.7절).
4. 알맞은 때에 멈추는 **조기 종료**, 폭주를 막는 **그래디언트 클리핑**, 같은 결과를 다시 얻는 **재현성**을 챙깁니다(8.8~8.10절).
5. 마지막으로 **손실이 안 줄 때** 어디부터 의심해야 하는지 순서를 정리합니다(8.11절).

이 장의 모든 실험은 같은 재료를 씁니다. 그래서 결과를 서로 비교할 수 있어요. 마지막 8.12절 실습에서는 대책을 하나씩 더해 가며 검증 성적이 어떻게 변하는지 표로 정리합니다.

Q

Q. 이 장의 예제를 돌리려면 무엇을 더 설치해야 하나요? PyTorch 말고 세 가지가 필요해요. 데이터를 가져올 scikit-learn, 학습 곡선을 그릴 matplotlib, 8.12절 실습의 데이터 증강에 쓸 torchvision입니다. 프로젝트 폴더에서 `uv add scikit-learn matplotlib torchvision`을 실행하세요(3장, 6장, 1권 20장에서 이미 설치했다면 건너뛰어도 됩니다). 그래프의 한글 글꼴은 macOS는 `AppleGothic`, 윈도우는 `Malgun Gothic`을 쓰면 되고, Linux라면 나눔글꼴(`sudo apt install fonts-nanum`)을 설치하고 matplotlib 글꼴 캐시(`~/.cache/matplotlib`)를 지운 뒤 `NanumGothic`을 넣으세요.

8.1 과적합과 과소적합 진단하기

이 장의 실험 재료

실험 재료는 scikit-learn에 들어 있는 **손글씨 숫자 데이터(load_digits)** 입니다. 0부터 9까지의 손글씨를 가로세로 8칸으로 줄인 흑백 그림 1,797장이에요. UCI 머신러닝 저장소의 「Optical Recognition of Handwritten Digits」 데이터(CC BY 4.0) 가운데 시험용 묶음을 scikit-learn이 함께 넣어 둔 것이라, 인터넷에서 받을 필요가 없고 작아서 CPU로도 몇 초면 학습됩니다.

과적합을 눈으로 보려면 일부러 외우기 쉬운 상황을 만들어야 합니다. 그래서 두 가지 장치를 했어요.

- 학습 데이터를 **200장만** 씁니다. 문제집이 얇으면 외우기 쉽죠.
- 학습 데이터 정답의 **20%를 무작위 숫자로 바꿉니다.** 해답지에 오타를 섞어 둔 거예요. 실제 데이터에도 사람이 잘못 매긴 정답이 꽤 섞여 있어서, 이 장치는 생각보다 현실적입니다. 검증·테스트 데이터의 정답은 건드리지 않습니다.

이 장의 여러 예제가 같은 준비 코드를 쓰니, 1권 14장에서 배운 대로 한 파일(모듈)에 모아 두고 `import`해서 쓰겠습니다. 아래 내용을 **`학습도구.py`**로 저장하세요. 학습 루프 자체는 6장, `DataLoader`는 7장에서 본 그대로입니다.

```
"""8장 실험에서 함께 쓰는 데이터 준비·모델·학습 도구."""
import torch
from sklearn.datasets import load_digits
from torch import nn
from torch.utils.data import DataLoader, TensorDataset


def 데이터_준비(잡음_비율: float = 0.2, 학습_개수: int = 200):
    """손글씨 숫자를 학습·검증·테스트로 나누고, 학습 정답 일부를 일부러 틀리게 바꾼다."""
    X, y = load_digits(return_X_y=True)          # 8×8 흑백 숫자 1,797장
    X = torch.tensor(X, dtype=torch.float32) / 16.0   # 0~16 밝기를 0~1로
    y = torch.tensor(y)                          # 정답 0~9 (int64)

    g = torch.Generator().manual_seed(0)         # 분할을 언제나 같게
    순서 = torch.randperm(len(X), generator=g)
    학습, 검증, 테스트 = 순서[:학습_개수], 순서[학습_개수:학습_개수 + 500], 순서[학습_개수 + 500:]

    y_학습 = y[학습].clone()
    바꿀_곳 = torch.rand(학습_개수, generator=g) < 잡음_비율
    y_학습[바꿀_곳] = torch.randint(0, 10, (int(바꿀_곳.sum()),), generator=g)
    return (X[학습], y_학습), (X[검증], y[검증]), (X[테스트], y[테스트])


def 모델_만들기(은닉: int = 256, 드롭아웃: float = 0.0, 배치정규화: bool = False) -> nn.Sequential:
    """입력 64 → 은닉 두 층 → 출력 10인 다층 신경망."""
    층들: list[nn.Module] = []
    입력 = 64
    for _ in range(2):
        층들.append(nn.Linear(입력, 은닉))
        if 배치정규화:
            층들.append(nn.BatchNorm1d(은닉))
        층들.append(nn.ReLU())
        if 드롭아웃 > 0:
```

```
            층들.append(nn.Dropout(드롭아웃))
        입력 = 은닉
    층들.append(nn.Linear(입력, 10))
    return nn.Sequential(*층들)


def 깊은_모델(배치정규화: bool = False, 활성화: type[nn.Module] = nn.ReLU) -> nn.Sequential:
    """폭 64짜리 은닉층 8개를 쌓은 깊은 신경망 (8.4절, 8.6절, 8.11절에서 사용)."""
    층들: list[nn.Module] = []
    for _ in range(8):
        층들.append(nn.Linear(64, 64))
        if 배치정규화:
            층들.append(nn.BatchNorm1d(64))
        층들.append(활성화())
    층들.append(nn.Linear(64, 10))
    return nn.Sequential(*층들)


def 한_에폭_학습(모델: nn.Module, 로더: DataLoader, 옵티마이저: torch.optim.Optimizer) -> None:
    """학습 데이터를 한 바퀴 돌며 가중치를 고친다."""
    모델.train()
    손실_함수 = nn.CrossEntropyLoss()
    for X, y in 로더:
        옵티마이저.zero_grad()
        손실 = 손실_함수(모델(X), y)
        손실.backward()
        옵티마이저.step()


@torch.no_grad()
def 평가(모델: nn.Module, X: torch.Tensor, y: torch.Tensor) -> tuple[float, float]:
    """손실과 정확도를 돌려준다."""
    모델.eval()
    출력 = 모델(X)
    손실 = nn.functional.cross_entropy(출력, y).item()
    정확도 = (출력.argmax(dim=1) == y).float().mean().item()
```

```
    return 손실, 정확도


def 로더_만들기(X: torch.Tensor, y: torch.Tensor, 배치_크기: int = 32) -> DataLoader:
    """씨앗을 고정한 생성기로 섞는 DataLoader."""
    g = torch.Generator().manual_seed(0)
    return DataLoader(TensorDataset(X, y), batch_size=배치_크기, shuffle=True, generator=g)
```

`데이터_준비()`는 1,797장을 섞은 뒤 앞 200장은 학습, 다음 500장은 검증, 나머지 1,097장은 테스트로 나눕니다. 섞을 때 씨앗을 고정한 **생성기(Generator)** 를 써서 몇 번을 불러도 같은 분할이 나와요. 이렇게 해야 실험끼리 비교가 됩니다(8.10절에서 다시 봅니다). `모델_만들기()`는 숫자 64개(8×8 그림)를 받아 은닉층 두 개를 거쳐 10개 숫자의 점수를 내는 신경망이고, 드롭아웃과 배치 정규화를 켜고 끄는 스위치가 달려 있습니다. `깊은_모델()`은 층을 8개 쌓은 신경망으로, 8.4절부터 쓸 거예요.

학습 곡선 그리기

이제 대책 없이 150에폭 동안 학습하면서, 에폭마다 학습 데이터와 검증 데이터의 손실을 기록해 보겠습니다. 이 두 줄의 곡선을 **학습 곡선(learning curve)** 이라고 불러요.

```
import matplotlib.pyplot as plt
import torch

from 학습도구 import 데이터_준비, 로더_만들기, 모델_만들기, 평가, 한_에폭_학습

(X_학습, y_학습), (X_검증, y_검증), _ = 데이터_준비()
(_, y_진짜), _, _ = 데이터_준비(잡음_비율=0.0)       # 바꾸기 전의 진짜 정답
print("학습", tuple(X_학습.shape), "검증", tuple(X_검증.shape))
print(f"정답이 틀린 학습 데이터: {(y_학습 != y_진짜).sum().item()}개")

torch.manual_seed(0)
모델 = 모델_만들기()
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
로더 = 로더_만들기(X_학습, y_학습)

기록 = {"학습 손실": [], "검증 손실": []}
```

```
print("에폭  학습손실  검증손실  학습정확도  검증정확도")
for 에폭 in range(1, 151):
    한_에폭_학습(모델, 로더, 옵티마이저)
    학습_손실, 학습_정확도 = 평가(모델, X_학습, y_학습)
    검증_손실, 검증_정확도 = 평가(모델, X_검증, y_검증)
    기록["학습 손실"].append(학습_손실)
    기록["검증 손실"].append(검증_손실)
    if 에폭 in (1, 5, 10, 20, 30, 50, 100, 150):
        print(f"{에폭:4d}  {학습_손실:8.4f}  {검증_손실:8.4f}  "
              f"{학습_정확도:9.1%}  {검증_정확도:9.1%}")

최저 = min(range(150), key=lambda i: 기록["검증 손실"][i])
print(f"검증 손실이 가장 낮았던 에폭: {최저 + 1} ({기록['검증 손실'][최저]:.4f})")

plt.rcParams["font.family"] = "AppleGothic"      # macOS 한글 글꼴
plt.rcParams["axes.unicode_minus"] = False
plt.figure(figsize=(7, 4))
for 이름, 값들 in 기록.items():
    plt.plot(range(1, 151), 값들, label=이름)
plt.axvline(최저 + 1, color="gray", linestyle="--", label="검증 손실 최저")
plt.xlabel("에폭")
plt.ylabel("손실")
plt.legend()
plt.tight_layout()
plt.savefig("learning_curve.png", dpi=200)
```

```
학습 (200, 64) 검증 (500, 64)
정답이 틀린 학습 데이터: 34개
에폭  학습손실  검증손실  학습정확도  검증정확도
   1    2.1868    2.2160      26.5%      20.2%
   5    1.4117    1.3878      70.0%      72.6%
  10    0.8719    0.6512      81.5%      85.6%
  20    0.6275    0.5264      83.0%      86.4%
  30    0.4652    0.5128      86.5%      87.2%
  50    0.2556    0.5268      94.0%      85.6%
 100    0.0355    0.6496     100.0%      82.6%
```

```
 150    0.0104    0.7462     100.0%      82.0%
검증 손실이 가장 낮았던 에폭: 32 (0.4883)
```

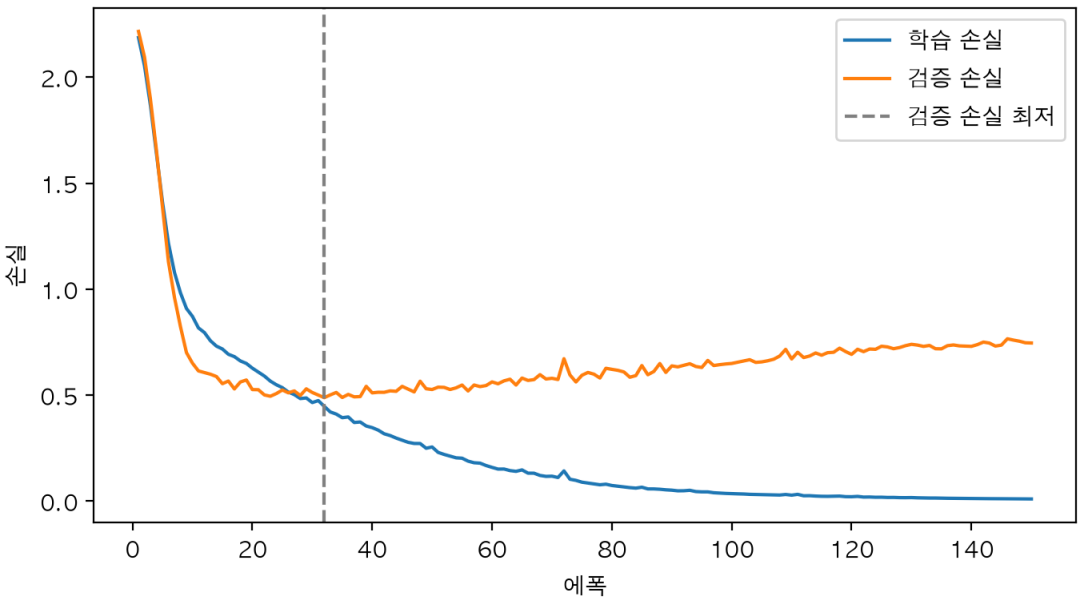


그림 8.1: 학습 손실과 검증 손실 곡선. 학습 손실은 0으로 내려가지만 검증 손실은 32에폭 근처에서 가장 낮고 그 뒤로 다시 오른다.

숫자와 그래프를 같이 읽어 보세요.

- **처음 30에폭쯤까지**는 학습 손실과 검증 손실이 함께 내려갑니다. 모델이 "숫자 0은 동그랗고, 1은 길쭉하다" 같은 쓸모 있는 규칙을 배우는 구간이에요. 검증 정확도도 87~90% 근처까지 올라갑니다.
- **검증 손실은 32에폭에서 가장 낮고(0.4883)**, 그 뒤로는 다시 오릅니다. 반면 학습 손실은 끝까지 내려가서 150에폭에는 0.0104가 돼요.
- **100에폭부터 학습 정확도가 100%** 입니다. 그런데 학습 데이터 200장 중 34장은 정답이 틀린 데이터였죠. 100%라는 건 그 틀린 정답 34개까지 모두 외웠다는 뜻입니다. 오타까지 외운 학생이에요. 그사이 검증 정확도는 87.2%에서 82.0%로 떨어졌습니다.

이렇게 **학습 손실은 계속 줄어드는데 검증 손실은 바닥을 찍고 올라가는 모양**이 과적합의 전형적인 신호입니다. 두 곡선 사이가 벌어지는 정도를 보면 얼마나 외우고 있는지 알 수 있어요.

그래프를 그리는 부분은 1권 20장의 matplotlib 그대로입니다. `AppleGothic`은 macOS의 한글 글

꼴이고, 윈도우라면 `Malgun Gothic`, Linux라면 `NanumGothic`(이 장 들어가며의 Q&A)을 넣으세요. `axes.unicode_minus`를 끄는 줄은 한글 글꼴에 없는 수학용 빼기 기호 대신 보통 하이픈(-)을 쓰게 하는 설정이에요.

> **Q. 검증 손실은 오르는데 검증 정확도는 그대로일 때도 있던데, 무엇을 믿어야 하나요?**
> 정확도는 "가장 높은 점수의 숫자가 정답인가"만 보고, 손실은 "정답에 얼마나 **확신**했는가"까지 봅니다. 과적합이 시작되면 모델은 틀린 문제에도 점점 더 강하게 확신하는데, 이때 정확도는 거의 그대로여도 손실은 오릅니다. 그래서 과적합의 시작은 손실이 먼저 알려 주는 경우가 많아요. 둘 다 기록해 두고, 멈출 시점은 검증 손실로, 최종 성적은 정확도처럼 사람이 이해하기 쉬운 지표로 보고하는 것이 보통입니다.

과소적합은 어떻게 생겼을까요?

반대쪽 실패도 보겠습니다. 모델이 너무 작거나, 학습률이 너무 작아서 거의 못 배운 경우입니다. 같은 150에폭 동안 세 가지 설정을 비교해 볼게요.

```
import torch

from 학습도구 import 데이터_준비, 로더_만들기, 모델_만들기, 평가, 한_에폭_학습

(X_학습, y_학습), (X_검증, y_검증), _ = 데이터_준비()
실험들 = {
    "은닉 2칸짜리 작은 모델": dict(은닉=2, lr=1e-3),
    "학습률 1e-5 (너무 작음)": dict(은닉=256, lr=1e-5),
    "은닉 256칸, 학습률 1e-3": dict(은닉=256, lr=1e-3),
}
for 이름, 설정 in 실험들.items():
    torch.manual_seed(0)
    로더 = 로더_만들기(X_학습, y_학습)
    모델 = 모델_만들기(은닉=설정["은닉"])
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=설정["lr"])
    for 에폭 in range(150):
        한_에폭_학습(모델, 로더, 옵티마이저)
    학습_손실, 학습_정확도 = 평가(모델, X_학습, y_학습)
    검증_손실, 검증_정확도 = 평가(모델, X_검증, y_검증)
    print(f"학습 {학습_손실:.3f} ({학습_정확도:6.1%})  "
```

```
        f"검증 {검증_손실:.3f} ({검증_정확도:5.1%})  <- {이름}")
```

```
학습 1.975 ( 25.0%)  검증 1.989 (21.0%)  <- 은닉 2칸짜리 작은 모델
학습 2.059 ( 48.5%)  검증 2.088 (42.4%)  <- 학습률 1e-5 (너무 작음)
학습 0.010 (100.0%)  검증 0.746 (82.0%)  <- 은닉 256칸, 학습률 1e-3
```

위의 두 줄은 **학습 손실부터 높습니다.** 은닉층이 2칸뿐인 모델은 64개 숫자를 2개로 줄였다가 다시 10개로 늘려야 하니, 아무리 공부해도 문제집조차 25%밖에 못 맞혀요. 학습률 1e-5는 걸음이 너무 작아서 150에폭 동안 산을 거의 내려오지 못했습니다(손실 2.059는 찍기 수준인 2.30과 큰 차이가 없어요). 세 번째 줄은 앞의 학습 곡선과 같은 설정이라 같은 숫자가 나왔습니다.

학습 곡선만 보고도 상태를 이렇게 판단할 수 있습니다.

학습 손실	검증 손실	진단	먼저 해 볼 일
높음	높음	과소적합	모델 키우기, 더 오래 학습, 학습률 조정, 8.11절의 버그 점검
낮음	높음(점점 오름)	과적합	데이터 늘리기·증강, 정규화(8.2~8.4절), 조기 종료(8.8절)
낮음	낮음(비슷)	좋음	테스트 데이터로 마지막 확인

순서도 중요합니다. **먼저 과적합할 수 있을 만큼 큰 모델로 학습 손실을 충분히 내린 다음, 정규화로 검증 손실을 끌어내리는 것**이 일반적인 순서예요. 학습 데이터조차 못 맞히는 모델에 정규화를 더 하면 더 못 배우게 될 뿐입니다.

8.2 가중치 감쇠(weight decay)

큰 가중치에 벌점 매기기

과적합한 모델의 속을 들여다보면 가중치가 아주 큰 경우가 많습니다. 틀린 정답 하나하나를 맞히려고 입력의 작은 차이에도 출력이 확 바뀌도록 가중치를 크게 키운 거예요. 그렇다면 **가중치가 커지는**

것 자체에 벌점을 주면 어떨까요? 모델은 정말 필요한 곳에만 큰 가중치를 쓰고, 나머지는 작게 유지하려 할 겁니다.

이 생각을 식으로 쓰면 원래 손실에 "모든 가중치 제곱의 합 × 벌점 세기 / 2"를 더하는 것입니다. 이를 **L2 정규화**라고 해요. 이 벌점을 미분하면 "벌점 세기 × 가중치"가 되므로, 경사하강법으로 한 걸음 갈 때마다 가중치가 자기 크기에 비례해 조금씩 0 쪽으로 줄어듭니다. 그래서 **가중치 감쇠**(weight decay) 라고도 불러요.

직접 벌점을 더하는 방법과 옵티마이저의 `weight_decay` 옵션이 정말 같은지 확인해 보겠습니다.

```
import torch
from torch import nn

torch.manual_seed(0)
X, y = torch.randn(8, 3), torch.randn(8, 1)
벌점_세기 = 0.1

# 방법 1: 손실에 (모든 가중치 제곱의 합) × 벌점_세기 / 2를 직접 더한다
torch.manual_seed(1)
모델1 = nn.Linear(3, 1)
옵티마이저1 = torch.optim.SGD(모델1.parameters(), lr=0.1)
벌점 = sum((p ** 2).sum() for p in 모델1.parameters())
손실1 = nn.functional.mse_loss(모델1(X), y) + 벌점_세기 / 2 * 벌점
손실1.backward()
옵티마이저1.step()

# 방법 2: 옵티마이저의 weight_decay 옵션에 맡긴다
torch.manual_seed(1)
모델2 = nn.Linear(3, 1)
옵티마이저2 = torch.optim.SGD(모델2.parameters(), lr=0.1, weight_decay=벌점_세기)
손실2 = nn.functional.mse_loss(모델2(X), y)
손실2.backward()
옵티마이저2.step()

print("방법 1 가중치:", 모델1.weight.data)
print("방법 2 가중치:", 모델2.weight.data)
print("같은가?", torch.allclose(모델1.weight, 모델2.weight))
```

```
방법 1 가중치: tensor([[ 0.1750, -0.1404,  0.0195]])
방법 2 가중치: tensor([[ 0.1750, -0.1404,  0.0195]])
같은가? True
```

두 방법으로 한 걸음 간 뒤의 가중치가 똑같습니다. 그러니 손실 식에 벌점을 직접 적을 필요 없이 옵티마이저를 만들 때 `weight_decay=벌점_세기`만 주면 돼요.

Adam에는 AdamW를

한 가지 주의할 점이 있습니다. 위의 동등함은 SGD에서만 성립해요. Adam은 파라미터마다 지금까지의 그래디언트 크기로 나눠서 걸음 크기를 따로 정하는데, 벌점의 그래디언트까지 같이 나눠 버리는 바람에 가중치마다 감쇠가 들쭉날쭉해집니다. 이 문제를 고친 것이 **AdamW**예요. AdamW는 벌점을 그래디언트에 섞지 않고, 매 걸음마다 가중치를 **`학습률 × weight_decay`**만큼 **따로** 줄입니다. 그래서 Adam 계열에서 weight decay를 쓸 때는 `torch.optim.Adam(..., weight_decay=...)`보다 `torch.optim.AdamW`를 쓰는 것이 표준입니다.

이제 우리 과적합 모델에 AdamW를 쓰고 `weight_decay`를 바꿔 가며 결과를 보겠습니다. 가중치 크기는 모든 가중치를 제곱해 더한 뒤 제곱근을 씌운 값(노름)으로 쟀어요.

```
import torch

from 학습도구 import 데이터_준비, 로더_만들기, 모델_만들기, 평가, 한_에폭_학습

(X_학습, y_학습), (X_검증, y_검증), _ = 데이터_준비()

print("weight_decay  가중치크기  학습손실  검증손실  검증정확도")
for wd in [0.0, 0.01, 0.1, 1.0, 3.0, 10.0]:
    torch.manual_seed(0)
    로더 = 로더_만들기(X_학습, y_학습)
    모델 = 모델_만들기()
    옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=1e-3, weight_decay=wd)
    for 에폭 in range(150):
        한_에폭_학습(모델, 로더, 옵티마이저)
    크기 = sum((p ** 2).sum() for p in 모델.parameters()).sqrt().item()
    학습_손실, _ = 평가(모델, X_학습, y_학습)
    검증_손실, 검증_정확도 = 평가(모델, X_검증, y_검증)
```

```
    print(f"{wd:12} {크기:10.2f} {학습_손실:8.3f} {검증_손실:8.3f} {검증_정확도:9.1%}")
```

```
weight_decay  가중치크기  학습손실  검증손실  검증정확도
         0.0      24.85     0.010     0.746      82.0%
        0.01      24.82     0.011     0.738      82.6%
         0.1      23.80     0.015     0.693      82.6%
         1.0      17.95     0.111     0.498      85.0%
         3.0      12.38     0.493     0.542      89.6%
        10.0       7.40     1.166     1.096      77.8%
```

표를 위에서부터 읽어 보세요.

- `weight_decay`가 커질수록 **가중치 크기가 작아집니다**(24.85 → 7.40). 벌점이 제 역할을 하고 있어요.
- 0.01과 0.1은 효과가 미미합니다. AdamW는 매 걸음 **학습률 × `weight_decay`**만큼 줄이는데 학습률이 0.001이라, 0.1이어도 한 걸음에 0.01%밖에 줄지 않거든요.
- **1.0과 3.0에서 검증 성적이 좋아집니다.** 3.0에서는 검증 정확도가 82.0%에서 89.6%까지 올랐어요. 학습 손실이 0.493으로 높아졌다는 건, 틀린 정답까지 외우지는 못하게 됐다는 뜻입니다.
- **10.0은 너무 셉니다.** 학습 손실과 검증 손실이 모두 1을 넘었어요. 앞 절의 표로 보면 과소적합입니다.

정규화는 이렇게 항상 **적당한 세기**가 있습니다. 이 데이터는 학습 데이터가 200장뿐이고 틀린 정답이 섞여 있어서 꽤 센 벌점이 잘 들었지만, 데이터가 많고 깨끗하면 0.01~0.1 정도가 흔히 쓰이는 값이에요. 정해진 정답은 없으니 검증 데이터로 몇 가지를 비교해서 고릅니다(3장의 하이퍼파라미터 탐색과 같은 일이에요).

8.3 드롭아웃

매번 몇 명씩 빠지는 팀 프로젝트

다섯 명이 하는 팀 프로젝트에서 한 사람이 일을 거의 다 한다고 해 봅시다. 그 사람이 아프면 프로젝트가 멈추죠. 그런데 회의 때마다 **무작위로 두세 명이 빠진다면** 어떨까요? 누가 빠져도 일이 돌아가도록 모두가 조금씩 다 할 줄 알게 됩니다. 특정 사람에게 기대는 팀이 아니라 튼튼한 팀이 되는 거예요.

드롭아웃(dropout) 이 신경망에 이 일을 합니다. 학습할 때 은닉층의 칸(뉴런) 일부를 매번 무작위로 0으로 꺼 버려요. 그러면 모델은 특정 칸 몇 개의 조합으로 학습 데이터를 외우기 어려워지고, 여러 칸에 골고루 퍼진 튼튼한 규칙을 배우게 됩니다.

학습 때와 평가 때가 다릅니다

드롭아웃에서 가장 중요한 것은 **학습 모드와 평가 모드의 동작이 다르다**는 점입니다. 직접 실행해서 보겠습니다.

```
import torch
from torch import nn

torch.manual_seed(0)
드롭아웃 = nn.Dropout(p=0.5)                # 칸마다 50% 확률로 끈다
x = torch.ones(10)

드롭아웃.train()                             # 학습 모드
print("학습 모드 1번째:", 드롭아웃(x))
print("학습 모드 2번째:", 드롭아웃(x))

드롭아웃.eval()                              # 평가 모드
print("평가 모드 1번째:", 드롭아웃(x))
print("평가 모드 2번째:", 드롭아웃(x))

큰_x = torch.ones(100_000)
드롭아웃.train()
print(f"학습 모드 평균: {드롭아웃(큰_x).mean().item():.4f}")
```

```
학습 모드 1번째: tensor([0., 0., 2., 0., 0., 0., 2., 2., 0., 2.])
학습 모드 2번째: tensor([2., 2., 2., 0., 2., 2., 2., 2., 0., 0.])
평가 모드 1번째: tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
평가 모드 2번째: tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
학습 모드 평균: 0.9985
```

- **학습 모드**에서는 부를 때마다 다른 칸이 꺼집니다. 첫 번째는 열 칸 중 여섯 칸, 두 번째는 세 칸이 0이 됐어요. 칸마다 50% 확률로 따로 정하기 때문에 정확히 절반씩 꺼지는 건 아닙니다.
- 살아남은 칸은 1이 아니라 **2가 됩니다.** 절반을 껐으니 남은 칸을 `1 / (1 - p)` = 2배로 키워서,

평균적으로는 원래 크기가 유지되게 한 거예요. 마지막 줄에서 숫자 10만 개의 평균이 0.9985로 거의 1인 것이 그 확인입니다.

- **평가 모드**에서는 아무것도 끄지 않고 입력을 그대로 내보냅니다. 학습 때 이미 크기를 맞춰 뒀기 때문에 평가 때는 손댈 필요가 없어요.

6장의 학습 루프와 평가 루프 앞에서 부른 `model.train()`과 `model.eval()`이 바로 이 모드를 바꾸는 메서드입니다. 이번에는 평가 때 `eval()`을 깜빡하면 어떻게 되는지 보겠습니다.

```
import torch

from 학습도구 import 데이터_준비, 로더_만들기, 모델_만들기, 한_에폭_학습

(X_학습, y_학습), (X_검증, y_검증), _ = 데이터_준비()
torch.manual_seed(0)
로더 = 로더_만들기(X_학습, y_학습)
모델 = 모델_만들기(드롭아웃=0.5)
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
for 에폭 in range(150):
    한_에폭_학습(모델, 로더, 옵티마이저)

숫자 = X_검증[:1]                          # 검증 데이터의 첫 숫자 하나
with torch.no_grad():
    모델.train()                           # eval()을 깜빡했다면
    print("train 모드:", [round(모델(숫자).softmax(dim=1).max().item(), 4) for _ in range(3)])
    모델.eval()
    print("eval 모드 :", [round(모델(숫자).softmax(dim=1).max().item(), 4) for _ in range(3)])

    for 모드 in ("train", "eval"):
        getattr(모델, 모드)()
        정확도 = (모델(X_검증).argmax(dim=1) == y_검증).float().mean().item()
        print(f"{모드} 모드 검증 정확도: {정확도:.1%}")
```

```
train 모드: [0.9913, 0.9398, 0.7024]
eval 모드 : [0.9911, 0.9911, 0.9911]
train 모드 검증 정확도: 82.8%
eval 모드 검증 정확도: 87.6%
```

같은 숫자 그림 하나를 세 번 넣었는데, `train` 모드에서는 매번 다른 확신도(0.9913, 0.9398, 0.7024)가 나옵니다. 부를 때마다 다른 칸이 꺼지니까요. `eval` 모드에서는 세 번 모두 같습니다. 검증 정확도도 `train` 모드로 재면 82.8%, `eval` 모드로 재면 87.6%예요. `eval()` 한 줄을 빠뜨리면 모델 성능을 **실제보다 낮게, 그리고 실행할 때마다 다르게** 재게 됩니다. 오류가 나지 않아서 알아채기 어려운 버그라 8.11절의 버그 목록에도 올라갑니다.

얼마나 꺼야 할까요?

드롭아웃 비율 `p`를 바꿔 가며 과적합 모델에 넣어 보겠습니다.

```
import torch

from 학습도구 import 데이터_준비, 로더_만들기, 모델_만들기, 평가, 한_에폭_학습

(X_학습, y_학습), (X_검증, y_검증), _ = 데이터_준비()

print("드롭아웃  학습손실  학습정확도  검증손실  검증정확도")
for p in [0.0, 0.2, 0.5, 0.7]:
    torch.manual_seed(0)
    로더 = 로더_만들기(X_학습, y_학습)
    모델 = 모델_만들기(드롭아웃=p)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
    for 에폭 in range(150):
        한_에폭_학습(모델, 로더, 옵티마이저)
    학습_손실, 학습_정확도 = 평가(모델, X_학습, y_학습)
    검증_손실, 검증_정확도 = 평가(모델, X_검증, y_검증)
    print(f"{p:8}  {학습_손실:8.3f}  {학습_정확도:9.1%}  {검증_손실:8.3f}  {검증_정확도:9.1%}")
```

```
드롭아웃  학습손실  학습정확도  검증손실  검증정확도
     0.0     0.010     100.0%     0.746      82.0%
     0.2     0.019     100.0%     0.629      83.0%
     0.5     0.092      97.0%     0.452      87.6%
     0.7     0.292      89.0%     0.401      90.8%
```

드롭아웃을 세게 걸수록 학습 성적은 떨어지고(학습 정확도 100% → 89.0%) 검증 성적은 좋아집니다(82.0% → 90.8%). 학습 정확도가 100% 아래로 내려왔다는 건, 틀린 정답을 더는 다 외우지 못한다는 뜻이에요. 이 데이터에서는 0.7까지도 좋아졌지만, 이건 학습 데이터가 아주 적고 지저분하기

때문입니다. 보통은 0.1~0.5 사이에서 시작하고, 칸이 많은 은닉층 뒤에 넣습니다. 드롭아웃을 쓰면 학습이 느려지므로 에폭을 조금 더 늘려야 할 때가 많다는 것도 기억해 두세요.

8.4 배치 정규화

반마다 다른 시험 난이도 맞추기

1반 시험은 쉬워서 평균이 90점이고, 2반 시험은 어려워서 평균이 50점이라면 두 반 학생의 점수를 그대로 비교할 수 없겠죠. 그래서 점수를 "반 평균에서 얼마나 떨어져 있는가"로 바꾸는 표준점수를 씁니다. 4장 실습에서 도시별 기온을 표준화한 것과 같은 계산이에요.

배치 정규화(batch normalization) 는 이 표준화를 신경망 **안에서**, 배치마다 합니다. 층을 지날 때마다 값의 크기가 들쭉날쭉해지면 다음 층이 매번 바뀌는 눈금에 적응해야 해서 학습이 느려지는데, 층 사이에서 값을 평균 0, 표준편차 1 근처로 맞춰 주면 학습이 훨씬 안정됩니다. 이것은 배치 정규화를 처음 제안한 논문의 설명이고, 뒤의 연구에서는 손실의 지형을 매끄럽게 만들어 주는 효과가 더 중요하다는 반론도 나왔어요. 정확히 왜 잘 되는지는 아직 연구가 이어지고 있지만, 잘 된다는 것만은 아래 실험처럼 분명합니다. 표준화한 뒤에는 학습하는 두 숫자(크기를 정하는 값과 위치를 정하는 값)를 곱하고 더해서, 모델이 필요하면 원래 눈금으로 되돌릴 수도 있게 해 둡니다.

```
import torch
from torch import nn

torch.manual_seed(0)
# 4개 샘플 × 특성 2개. 두 특성의 크기(눈금)가 아주 다르다
x = torch.tensor([[1.0, 100.0],
                  [2.0, 300.0],
                  [3.0, 200.0],
                  [4.0, 400.0]])
bn = nn.BatchNorm1d(2)                          # 특성 2개짜리 배치 정규화
print("처음 running_mean:", bn.running_mean)

bn.train()
출력 = bn(x)
print("학습 모드 출력:")
print(출력.detach())
손으로 = (x - x.mean(dim=0)) / torch.sqrt(x.var(dim=0, unbiased=False) + bn.eps)
```

```
print("손으로 계산과 같은가?", torch.allclose(출력, 손으로, atol=1e-5))
print("한 번 본 뒤 running_mean:", bn.running_mean)

bn.eval()                                   # 평가 모드는 저장해 둔 평균·분산을 쓴다
print("평가 모드 출력(첫 줄):", bn(x[:1]).detach())

bn.train()
try:
    bn(x[:1])                               # 학습 모드에 샘플 1개
except ValueError as e:
    print("오류:", e)
```

```
처음 running_mean: tensor([0., 0.])
학습 모드 출력:
tensor([[-1.3416, -1.3416],
        [-0.4472,  0.4472],
        [ 0.4472, -0.4472],
        [ 1.3416,  1.3416]])
손으로 계산과 같은가? True
한 번 본 뒤 running_mean: tensor([ 0.2500, 25.0000])
평가 모드 출력(첫 줄): tensor([[0.7262, 1.8366]])
오류: Expected more than 1 value per channel when training, got input size torch.Size([1, 2])
```

차근차근 보겠습니다.

- **학습 모드**에서는 지금 들어온 배치 4개의 평균과 분산으로 표준화합니다. 첫 특성(1~4)과 둘째 특성(100~400)은 눈금이 100배나 다른데, 표준화한 뒤에는 둘 다 -1.34~1.34 범위로 맞춰졌어요. 손으로 계산한 결과와도 같습니다.
- 학습 모드에서는 그와 동시에 **평균과 분산의 이동 평균**을 몰래 기록합니다. `running_mean`이 0에서 (0.25, 25)로 바뀐 게 보이죠. 이번 배치 평균 (2.5, 250)을 10%만 반영해서 기존 값과 섞은 결과입니다.
- **평가 모드**에서는 배치의 평균이 아니라 이 기록해 둔 값을 씁니다. 예측할 때는 데이터가 한 장만 올 수도 있는데, 한 장으로는 평균과 분산을 낼 수 없으니까요. 지금은 배치를 딱 한 번 봤을 뿐이라 기록이 아직 부정확해서 이상한 값(0.7262, 1.8366)이 나왔지만, 학습을 오래 하면 전체 데이터의 평균과 분산에 가까워집니다.

- **학습 모드에 샘플 한 개**를 넣으면 오류가 납니다. 숫자 하나의 분산은 0이라 표준화가 안 되거든요. 데이터 개수가 배치 크기로 나누어떨어지지 않아 마지막 배치에 한 개만 남는 경우 이 오류를 만날 수 있어요. `DataLoader(..., drop_last=True)`로 마지막 자투리 배치를 버리면 해결됩니다.

드롭아웃처럼 배치 정규화도 `train()`/`eval()`에 따라 동작이 다르다는 점을 꼭 기억하세요.

깊은 신경망이 살아납니다

배치 정규화의 진가는 층이 깊을 때 드러납니다. 은닉층 8개짜리 `깊은_모델()`을 SGD로 학습하면서, 배치 정규화가 있을 때와 없을 때를 비교해 보겠습니다. 이번에는 정답을 바꾸지 않은 깨끗한 데이터 1,000장을 씁니다.

```
import torch

from 학습도구 import 깊은_모델, 데이터_준비, 로더_만들기, 평가, 한_에폭_학습

# 이번에는 정답을 바꾸지 않은 깨끗한 데이터 1,000개로
(X_학습, y_학습), (X_검증, y_검증), _ = 데이터_준비(잡음_비율=0.0, 학습_개수=1000)
for 배치정규화 in (False, True):
    torch.manual_seed(0)
    로더 = 로더_만들기(X_학습, y_학습)
    모델 = 깊은_모델(배치정규화=배치정규화)
    옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.05)
    기록 = []
    for 에폭 in range(1, 21):
        한_에폭_학습(모델, 로더, 옵티마이저)
        if 에폭 in (1, 3, 5, 10, 20):
            기록.append(f"{에폭}에폭 {평가(모델, X_검증, y_검증)[1]:.0%}")
    print("배치 정규화", "있음:" if 배치정규화 else "없음:", ", ".join(기록))
```

```
배치 정규화 없음: 1에폭 9%, 3에폭 9%, 5에폭 9%, 10에폭 10%, 20에폭 10%
배치 정규화 있음: 1에폭 60%, 3에폭 86%, 5에폭 82%, 10에폭 91%, 20에폭 96%
```

차이가 극적입니다. 배치 정규화가 없는 모델은 20에폭 동안 검증 정확도가 10% 근처, 즉 10개 중 하나를 찍는 수준에서 한 발짝도 못 나갔어요. 배치 정규화를 넣으니 1에폭 만에 60%, 20에폭에 96%입니다. 왜 배치 정규화가 없으면 아예 못 배우는지는 8.6절 가중치 초기화에서 이유를 찾게 됩

니다.

Q. 배치 정규화는 정규화(규제) 효과도 있나요? 조금 있습니다. 배치마다 평균과 분산이 조금씩 달라서 같은 데이터도 함께 묶인 친구들에 따라 값이 살짝 흔들리는데, 이 흔들림이 약한 잡음 역할을 해요. 하지만 배치 정규화의 주된 목적은 학습을 빠르고 안정되게 하는 것이고, 과적합을 막는 힘은 weight decay나 드롭아웃보다 약합니다. 실제로 8.12절 실습에서는 배치 정규화를 더했더니 오히려 외우기가 쉬워져서 검증 성적이 떨어졌어요. 9장의 CNN에서는 거의 모든 층 뒤에 들어가는 기본 부품으로 다시 만납니다.

8.5 데이터 증강은 7장에서

과적합을 막는 가장 확실한 방법은 사실 **데이터를 늘리는 것**입니다. 문제집이 두꺼우면 외우기 어려우니까요. 진짜 데이터를 더 모으기 어려울 때는 있는 데이터를 살짝 비틀어 새 데이터처럼 쓰는 **데이터 증강(data augmentation)** 을 씁니다. 사진을 조금 돌리거나, 자르거나, 밝기를 바꿔도 정답은 그대로니까요.

데이터 증강은 7장에서 `transforms`로 이미 다뤘으므로 여기서 다시 설명하지 않습니다. 이 장의 정규화 도구들과 함께 쓸 수 있다는 것, 그리고 **학습 데이터에만** 적용하고 검증·테스트 데이터에는 적용하지 않는다는 것만 기억하세요. 증강이 이 장의 과적합 모델에서 실제로 얼마나 듣는지는 8.12절 실습에서 다른 대책들과 나란히 숫자로 비교합니다.

8.6 가중치 초기화

첫 단추가 잘못 끼워지면

8.4절에서 배치 정규화가 없는 깊은 모델은 전혀 배우지 못했습니다. 원인은 학습을 시작하는 순간, 가중치의 **처음 값**에 있어요.

층을 하나 지날 때마다 값에 가중치가 곱해집니다. 가중치가 조금만 작아도 10층을 지나면 0.5 × 0.5 × … 처럼 줄어들어 거의 0이 되고, 조금만 커도 2 × 2 × … 처럼 불어나 엄청나게 커져요. 직접 재 보겠습니다. 표준편차 1인 입력을 ReLU 층 10개에 통과시키면서, 가중치를 어떻게 초기화했느냐에 따라 출력의 표준편차가 어떻게 변하는지 보는 실험입니다.

```
import torch
from torch import nn

torch.manual_seed(0)
x = torch.randn(512, 256)                    # 입력: 평균 0, 표준편차 1


def 층별_표준편차(초기화) -> str:
    """ReLU 층 10개를 통과하며 출력의 표준편차가 어떻게 변하는지 잰다."""
    h = x
    결과 = []
    for 층 in range(1, 11):
        선형 = nn.Linear(256, 256, bias=False)
        초기화(선형.weight)
        h = torch.relu(선형(h))
        if 층 in (1, 2, 5, 10):
            결과.append(f"{층}층 {h.std().item():.2e}")
    return ", ".join(결과)


with torch.no_grad():
    print("표준편차 0.01:", 층별_표준편차(lambda w: nn.init.normal_(w, std=0.01)))
    print("표준편차 1.0 :", 층별_표준편차(lambda w: nn.init.normal_(w, std=1.0)))
    print("Kaiming      :", 층별_표준편차(lambda w: nn.init.kaiming_normal_(w,
    ↪  nonlinearity="relu")))
```

```
표준편차 0.01: 1층 9.33e-02, 2층 1.03e-02, 5층 1.39e-05, 10층 2.71e-10
표준편차 1.0 : 1층 9.38e+00, 2층 1.09e+02, 5층 1.62e+05, 10층 3.34e+10
Kaiming      : 1층 8.30e-01, 2층 8.29e-01, 5층 7.94e-01, 10층 8.87e-01
```

- **표준편차 0.01**로 초기화하면 층을 지날 때마다 값이 약 10분의 1로 줄어서, 10층에서는 0.000000000271입니다. 사실상 0이에요. 마지막 층은 입력이 무엇이든 거의 같은 값을 받게 됩니다.
- **표준편차 1.0**이면 반대로 층마다 10배씩 불어나서 10층에서는 330억이 넘습니다. 이런 값으로 손실을 계산하면 금방 `inf`나 `nan`이 나와요.
- **Kaiming 초기화**는 10층을 지나도 표준편차가 0.8 근처로 유지됩니다.

Kaiming 초기화(He 초기화라고도 불러요)는 "층을 지나도 값의 크기가 유지되려면 가중치의 표준편차가 얼마여야 하는가"를 계산해서 정한 값입니다. ReLU는 음수를 0으로 만들어 값의 절반을 버리니, 입력 개수가 n일 때 표준편차를 2/n의 제곱근으로 하면 크기가 유지돼요. 입력이 256개면 2/256의 제곱근, 약 0.0884입니다. 시그모이드나 tanh를 쓰는 층에는 비슷한 생각으로 만든 **Xavier(Glorot) 초기화**(`nn.init.xavier_normal_`)를 씁니다.

나쁜 초기화로 학습이 멈추는 모습

값이 사라지면 학습에는 어떤 일이 생길까요? 5장에서 본 연쇄 법칙을 떠올려 보세요. 그래디언트는 출력 쪽에서 입력 쪽으로 층마다 곱해지며 전해지기 때문에, 값이 사라지는 신경망에서는 그래디언트도 사라집니다. 고칠 방향을 알려 주는 신호가 앞쪽 층까지 오지 못하는 거예요. 8층짜리 `깊은_모델()`을 네 가지 방법으로 초기화해서 학습해 보겠습니다.

```
import torch
from torch import nn

from 학습도구 import 깊은_모델, 데이터_준비, 로더_만들기, 평가, 한_에폭_학습


def 초기화(모델: nn.Module, 방식: str) -> None:
    """모델 안의 모든 선형 층 가중치를 정한 방식으로 다시 채운다."""
    for 층 in 모델.modules():
        if not isinstance(층, nn.Linear):
            continue
        if 방식 == "모두 0":
            nn.init.zeros_(층.weight)
        elif 방식 == "표준편차 0.01":
            nn.init.normal_(층.weight, std=0.01)
        elif 방식 == "Kaiming":
            nn.init.kaiming_normal_(층.weight, nonlinearity="relu")
        nn.init.zeros_(층.bias)


(X_학습, y_학습), (X_검증, y_검증), _ = 데이터_준비(잡음_비율=0.0, 학습_개수=1000)
for 방식 in ["모두 0", "표준편차 0.01", "PyTorch 기본값", "Kaiming"]:
    torch.manual_seed(0)
```

```
    로더 = 로더_만들기(X_학습, y_학습)
    모델 = 깊은_모델()
    if 방식 != "PyTorch 기본값":
        초기화(모델, 방식)
    옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.05)
    for 에폭 in range(20):
        한_에폭_학습(모델, 로더, 옵티마이저)
    손실, 정확도 = 평가(모델, X_검증, y_검증)
    print(f"검증 손실 {손실:.4f}, 정확도 {정확도:5.1%}  <- {방식}")
```

```
검증 손실 2.3053, 정확도  9.8%  <- 모두 0
검증 손실 2.3053, 정확도  9.8%  <- 표준편차 0.01
검증 손실 2.3052, 정확도  9.8%  <- PyTorch 기본값
검증 손실 0.1364, 정확도 96.4%  <- Kaiming
```

`모델.modules()`는 모델 안의 모든 층을 하나씩 돌려주므로, 그중 `nn.Linear`인 것만 골라 가중치를 다시 채웠습니다. 결과를 보면 앞의 셋은 20에폭 내내 검증 손실이 2.305입니다. 10개 중 하나를 찍을 때의 손실이 ln(10), 약 2.3026이니, 아무것도 배우지 못한 거예요. Kaiming 초기화만 96.4%까지 학습했습니다.

- **모두 0**: 모든 칸의 가중치가 같으면 모든 칸이 같은 값을 계산하고 같은 그래디언트를 받아서, 학습해도 영원히 같은 값으로 움직입니다. 칸이 64개 있어도 한 칸짜리와 다를 게 없어요. 게다가 ReLU 신경망에서는 출력이 0이라 그래디언트도 0이 되어 아예 움직이지 않습니다. 그래서 가중치는 반드시 **무작위로** 시작해야 해요.
- **표준편차 0.01**: 바로 앞에서 본 것처럼 신호가 층마다 줄어서 사라집니다.
- **PyTorch 기본값**: 의외죠? `nn.Linear`의 기본 초기화는 표준편차가 1/(3n)의 제곱근 정도로, Kaiming보다 2.4배쯤 작습니다. 층이 두세 개인 보통 모델에서는 문제없지만, 8층을 쌓으니 신호가 줄어 버렸어요.

8.4절의 배치 정규화는 층마다 값을 표준화해서 이 문제를 강제로 해결한 것이었습니다. 초기값이 조금 어긋나도 층마다 다시 맞춰 주니까요. 실무에서는 9장 이후의 깊은 모델이 거의 모두 배치 정규화(또는 12장에서 볼 비슷한 정규화 층)와 알맞은 초기화를 함께 씁니다.

8.7 학습률 스케줄러

처음엔 크게, 가까워지면 작게

처음 가는 동네에서 약속 장소를 찾아간다고 해 봅시다. 멀리 있을 때는 큰길로 성큼성큼 가다가, 가까워지면 건물 번호를 보며 천천히 걷죠. 처음부터 끝까지 같은 보폭이면 멀리서는 너무 느리고, 가까이서는 지나쳐 버립니다.

학습률도 마찬가지입니다. 학습 초반에는 큰 학습률로 빠르게 내려가고, 후반에는 작은 학습률로 바닥 근처를 세밀하게 찾는 것이 유리해요. 에폭에 따라 학습률을 바꿔 주는 도구가 **학습률 스케줄러(learning rate scheduler)** 입니다. 많이 쓰는 세 가지가 학습률을 실제로 어떻게 바꾸는지 출력해 보겠습니다.

```
import torch
from torch.optim.lr_scheduler import CosineAnnealingLR, OneCycleLR, StepLR

에폭_수 = 10


def 학습률_기록(스케줄러_만들기) -> list[float]:
    """에폭마다 옵티마이저의 학습률을 적어 둔다."""
    가짜_가중치 = torch.zeros(1, requires_grad=True)   # 학습률만 볼 것이라 아무 값이나
    옵티마이저 = torch.optim.SGD([가짜_가중치], lr=0.1)
    스케줄러 = 스케줄러_만들기(옵티마이저)
    기록 = []
    for 에폭 in range(에폭_수):
        기록.append(옵티마이저.param_groups[0]["lr"])
        옵티마이저.step()                    # (실제로는 여기서 한 에폭 학습)
        스케줄러.step()                      # 그다음에 학습률 갱신
    return 기록


기록들 = {
    "StepLR": 학습률_기록(lambda o: StepLR(o, step_size=3, gamma=0.5)),
    "Cosine": 학습률_기록(lambda o: CosineAnnealingLR(o, T_max=에폭_수)),
    "OneCycle": 학습률_기록(lambda o: OneCycleLR(o, max_lr=0.1, total_steps=에폭_수)),
}
print("에폭  " + "".join(f"{이름:>10}" for 이름 in 기록들))
for 에폭 in range(에폭_수):
```

```
    print(f"{에폭:4d}  " + "".join(f"{값[에폭]:10.4f}" for 값 in 기록들.values()))
```

```
에폭      StepLR    Cosine  OneCycle
   0      0.1000    0.1000    0.0040
   1      0.1000    0.0976    0.0520
   2      0.1000    0.0905    0.1000
   3      0.0500    0.0794    0.0950
   4      0.0500    0.0655    0.0812
   5      0.0500    0.0500    0.0611
   6      0.0250    0.0345    0.0389
   7      0.0250    0.0206    0.0188
   8      0.0250    0.0095    0.0050
   9      0.0125    0.0024    0.0000
```

`학습률_기록()`은 스케줄러를 만드는 함수를 받아서, 에폭마다 옵티마이저에 들어 있는 현재 학습률을 적어 둡니다. 스케줄러마다 만드는 방법만 다르니 그 부분을 `lambda`(1권 10장)로 넘겼어요. 학습률은 `옵티마이저.param_groups[0]["lr"]`에 들어 있습니다.

- StepLR(step_size=3, gamma=0.5): 3에폭마다 학습률을 절반으로 줄입니다. 0.1 → 0.05 → 0.025 → 0.0125, 계단 모양이에요. 가장 단순하고 이해하기 쉽습니다.
- CosineAnnealingLR(T_max=10): 코사인 곡선을 따라 0.1에서 0 근처까지 부드럽게 줄입니다. 처음엔 천천히, 중간엔 빠르게, 끝에선 다시 천천히 줄어요. 요즘 가장 흔히 쓰는 방식입니다.
- OneCycleLR(max_lr=0.1): 아주 작은 값(0.004, 최댓값의 1/25)에서 시작해 전체의 30% 지점(2에폭)까지 최댓값 0.1로 **올렸다가**, 나머지 기간 동안 0 근처까지 내립니다. 처음부터 큰 학습률로 시작하면 초기화 직후의 불안정한 모델이 튕겨 나갈 수 있어서, 살살 올리는 **워밍업**(warm-up) 구간을 둔 거예요.

같은 함수로 100에폭 동안 기록해서 그래프로 그리면 모양 차이가 더 잘 보입니다(StepLR은 30에폭마다 절반으로 줄이게 했습니다).

스케줄러를 쓸 때 지켜야 할 규칙이 두 가지 있습니다.

1. `optimizer.step()` **다음에** `scheduler.step()` 을 부릅니다. 순서를 바꾸면 첫 학습률을 건너뛰게 되고, PyTorch가 경고를 띄워요.
2. **얼마마다 부를지** 확인합니다. StepLR과 Cosine은 보통 에폭마다 부르고 `T_max`도 에폭 수로

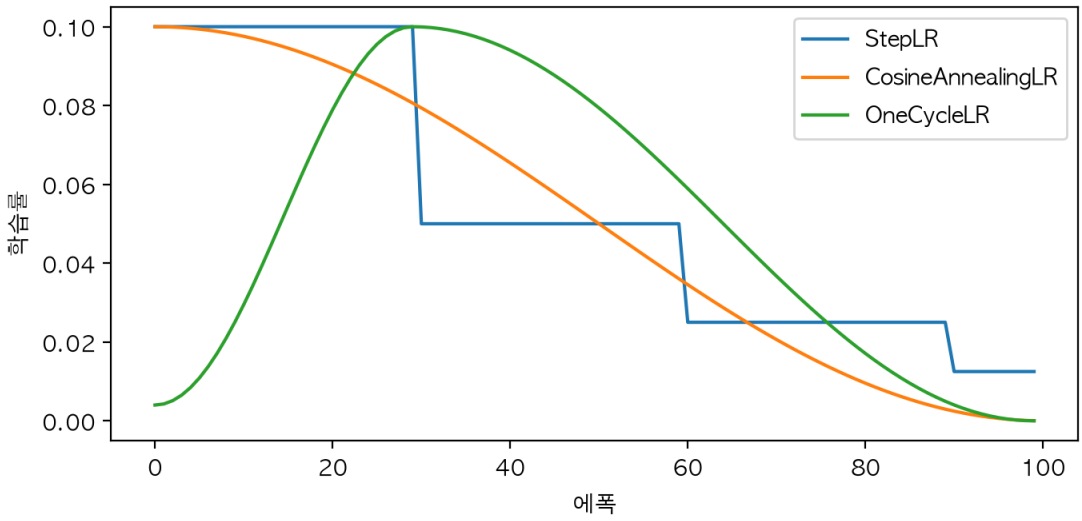


그림 8.2: StepLR, CosineAnnealingLR, OneCycleLR의 에폭별 학습률. StepLR은 계단, Cosine은 부드러운 감소, OneCycle은 올라갔다 내려오는 모양이다.

> 정하지만, OneCycleLR은 보통 **배치마다** 부르고 `total_steps`를 "에폭 수 × 에폭당 배치 수"로 정합니다. 단위를 헷갈리면 학습률이 엉뚱한 시점에 바닥으로 떨어져요.

검증 손실을 보고 줄이기

위의 스케줄러들은 시간표대로 학습률을 바꿉니다. 이와 달리 **검증 손실이 더 나아지지 않을 때만** 학습률을 줄이는 `ReduceLROnPlateau`도 있어요. "고원(plateau)에 올라서 평평해지면 줄인다"는 뜻입니다.

```
import torch
from torch.optim.lr_scheduler import ReduceLROnPlateau

가짜_가중치 = torch.zeros(1, requires_grad=True)
옵티마이저 = torch.optim.SGD([가짜_가중치], lr=0.1)
스케줄러 = ReduceLROnPlateau(옵티마이저, mode="min", factor=0.5, patience=2)

# 예제용 검증 손실: 처음엔 줄다가 0.50 근처에서 멈춘다
검증_손실들 = [0.90, 0.70, 0.55, 0.50, 0.51, 0.50, 0.52, 0.49, 0.50, 0.51, 0.50, 0.50]
for 에폭, 손실 in enumerate(검증_손실들, start=1):
```

```
    스케줄러.step(손실)                          # 검증 손실을 넘겨준다
    print(f"{에폭:2d}에폭 검증 손실 {손실:.2f} -> 학습률 {옵티마이저.param_groups[0]['lr']:.4f}")
```

```
 1에폭 검증 손실 0.90 -> 학습률 0.1000
 2에폭 검증 손실 0.70 -> 학습률 0.1000
 3에폭 검증 손실 0.55 -> 학습률 0.1000
 4에폭 검증 손실 0.50 -> 학습률 0.1000
 5에폭 검증 손실 0.51 -> 학습률 0.1000
 6에폭 검증 손실 0.50 -> 학습률 0.1000
 7에폭 검증 손실 0.52 -> 학습률 0.0500
 8에폭 검증 손실 0.49 -> 학습률 0.0500
 9에폭 검증 손실 0.50 -> 학습률 0.0500
10에폭 검증 손실 0.51 -> 학습률 0.0500
11에폭 검증 손실 0.50 -> 학습률 0.0250
12에폭 검증 손실 0.50 -> 학습률 0.0250
```

이 스케줄러는 `step()`에 검증 손실을 넘겨줘야 합니다. 4에폭에 0.50으로 최고 기록을 세운 뒤 5, 6, 7에폭 세 번 연속 기록을 넘지 못하자(`patience=2`, 즉 두 번까지는 참아 줌) 7에폭에 학습률이 절반이 됐어요. 6에폭의 0.50은 최고 기록과 같을 뿐 **더 나아지지는** 않았으니 못 넘은 것으로 셉니다. 8에폭에 0.49로 새 기록을 세웠다가 다시 세 번 못 넘어서 11에폭에 한 번 더 줄었습니다. 몇 에폭을 돌려야 할지 미리 정하기 어려울 때 편리해요.

8.8 조기 종료

가장 좋았던 순간에 멈추기

8.1절의 학습 곡선을 다시 떠올려 보세요. 검증 손실은 32에폭에 가장 낮았고 그 뒤로는 나빠지기만 했습니다. 그렇다면 **검증 손실이 한동안 나아지지 않으면 학습을 멈추고, 가장 좋았던 때의 가중치로 되돌리면** 되지 않을까요? 이것이 **조기 종료(early stopping)** 입니다. 시험 공부를 하다가 모의고사 점수가 더 오르지 않으면, 쓸데없이 답을 외우기 전에 멈추는 거예요.

다만 검증 손실은 에폭마다 조금씩 오르내리기 때문에, 한 번 나빠졌다고 바로 멈추면 안 됩니다. 몇 번까지 참아 줄지를 **참을성(patience)** 이라고 해요. 1권 11장에서 배운 클래스로 직접 만들어 보겠습니다.

```
import copy

import torch
from torch import nn

from 학습도구 import 데이터_준비, 로더_만들기, 모델_만들기, 평가, 한_에폭_학습


class 조기_종료:
    """검증 손실이 참을성(patience) 에폭 동안 나아지지 않으면 멈추라고 알려 준다."""

    def __init__(self, 참을성: int = 10, 최소_개선: float = 0.0) -> None:
        self.참을성 = 참을성
        self.최소_개선 = 최소_개선
        self.최고_손실 = float("inf")
        self.최고_에폭 = 0
        self.최고_가중치: dict | None = None
        self.나빠진_횟수 = 0

    def 확인(self, 에폭: int, 검증_손실: float, 모델: nn.Module) -> bool:
        """멈춰야 하면 True를 돌려준다."""
        if 검증_손실 < self.최고_손실 - self.최소_개선:
            self.최고_손실 = 검증_손실
            self.최고_에폭 = 에폭
            self.최고_가중치 = copy.deepcopy(모델.state_dict())   # 복사해서 보관
            self.나빠진_횟수 = 0
        else:
            self.나빠진_횟수 += 1
        return self.나빠진_횟수 >= self.참을성


(X_학습, y_학습), (X_검증, y_검증), _ = 데이터_준비()
torch.manual_seed(0)
로더 = 로더_만들기(X_학습, y_학습)
모델 = 모델_만들기()
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
멈춤이 = 조기_종료(참을성=20)
```

```
for 에폭 in range(1, 501):
    한_에폭_학습(모델, 로더, 옵티마이저)
    검증_손실, _ = 평가(모델, X_검증, y_검증)
    if 멈춤이.확인(에폭, 검증_손실, 모델):
        print(f"{에폭}에폭에서 멈춤 (최고 기록: {멈춤이.최고_에폭}에폭, 손실 {멈춤이.최고_손
        ↪ 실:.4f})")
        break

print("멈춘 시점 검증 (손실, 정확도):", [round(v, 4) for v in 평가(모델, X_검증, y_검증)])
모델.load_state_dict(멈춤이.최고_가중치)       # 가장 좋았던 가중치로 되돌리기
print("되돌린 뒤 검증 (손실, 정확도):", [round(v, 4) for v in 평가(모델, X_검증, y_검증)])
```

```
52에폭에서 멈춤 (최고 기록: 32에폭, 손실 0.4883)
멈춘 시점 검증 (손실, 정확도): [0.5369, 0.852]
되돌린 뒤 검증 (손실, 정확도): [0.4883, 0.896]
```

`조기_종료` 클래스는 지금까지의 최고 검증 손실과 그때의 가중치를 기억하고 있다가, 에폭마다 `확인()`으로 새 손실을 받습니다. 최고 기록을 `최소_개선`보다 더 낮추면 기록을 새로 쓰고 참은 횟수를 0으로 되돌리고, 아니면 참은 횟수를 하나 늘려요. `참을성`만큼 참았으면 `True`를 돌려줘서 멈추라고 알립니다.

결과를 보면 최고 기록은 32에폭(손실 0.4883)이었고, 그 뒤로 20에폭을 참아 준 52에폭에서 멈췄습니다. 최대 500에폭으로 잡아 뒀지만 52에폭이면 충분했던 거죠. 멈춘 시점의 모델은 이미 조금 외우기 시작해서 검증 정확도가 85.2%였는데, 32에폭의 가중치로 되돌리니 89.6%가 됐습니다.

여기서 꼭 짚을 부분이 `copy.deepcopy(모델.state_dict())`입니다. `state_dict()`는 6장에서 모델을 저장할 때 쓴 가중치 사전인데, 그 안의 텐서들은 지금 학습 중인 모델의 가중치와 **같은 메모리를 가리킵니다.** 1권 6장의 "이름표를 하나 더 붙이는" 상황이에요. 복사 없이 그냥 보관하면 학습이 계속되는 동안 보관해 둔 "최고 가중치"도 같이 바뀌어서, 마지막에 되돌려도 아무 일이 일어나지 않습니다. `deepcopy`는 사전 안의 텐서까지 통째로 새로 복사해요.

Q

Q. 검증 데이터 대신 테스트 데이터로 멈출 시점을 정하면 안 되나요? 안 됩니다. 멈출 시점을 고르는 순간 그 데이터는 "모델을 고르는 데 쓴 데이터"가 되고, 거기서 잰 점수는 실제보다 좋게 나옵니다. 2장에서 본 데이터 누수의 한 형태예요. 조기 종료, 하이퍼파라

미터 선택, 스케줄러 판단은 모두 **검증 데이터**로 하고, 테스트 데이터는 모든 결정이 끝난 뒤 마지막에 한 번만 씁니다. Q

8.9 그래디언트 클리핑

이상치 하나가 학습을 터뜨립니다

학습 중에 손실이 갑자기 `inf`나 `nan`이 되어 버리는 경우가 있습니다. 대개 어느 순간 그래디언트가 터무니없이 커져서, 가중치가 한 걸음에 엉뚱한 곳으로 날아갔기 때문이에요. 이를 **그래디언트 폭주**(exploding gradient) 라고 합니다. 11장의 RNN처럼 같은 계산을 여러 번 반복하는 모델에서 특히 자주 생기고, 데이터에 엉뚱하게 큰 값이 섞여 있어도 생깁니다.

예제용 회귀 데이터에 잘못 들어간 값(이상치) 두 개를 섞어서 재현해 보겠습니다. 정상적인 정답은 모두 -10~10 사이인데, 두 개만 300입니다.

```
import torch
from torch import nn

torch.manual_seed(0)
X = torch.randn(64, 10)
y = X @ torch.randn(10, 1)                    # 예제용 회귀 데이터
y[:2] = 300.0                                 # 잘못 들어간 값 두 개 (이상치)


def 학습(자를까: bool) -> None:
    torch.manual_seed(0)
    모델 = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 1))
    옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.05)
    for 단계 in range(1, 201):
        옵티마이저.zero_grad()
        손실 = nn.functional.mse_loss(모델(X), y)
        손실.backward()
        # 자르기 전의 전체 그래디언트 크기를 돌려준다 (자르지 않을 때는 max_norm을 무한대로)
        크기 = nn.utils.clip_grad_norm_(모델.parameters(),
                                        max_norm=1.0 if 자를까 else float("inf"))
        옵티마이저.step()
```

```
        if 단계 in (1, 10, 12, 13, 14, 15, 200):
            print(f"  {단계:3d}단계 손실 {손실.item():12.4g}  그래디언트 크기 {크기.item():10.4g}")


print("그래디언트 클리핑 없음")
학습(자를까=False)
print("그래디언트 클리핑 (max_norm=1.0)")
학습(자를까=True)
```

```
그래디언트 클리핑 없음
    1단계 손실         2817  그래디언트 크기      53.67
   10단계 손실         2199  그래디언트 크기        136
   12단계 손실    5.635e+04  그래디언트 크기  1.001e+04
   13단계 손실     3.69e+09  그래디언트 크기  4.499e+07
   14단계 손실    2.139e+24  그래디언트 크기   5.49e+18
   15단계 손실          inf  그래디언트 크기        nan
  200단계 손실          nan  그래디언트 크기        nan
그래디언트 클리핑 (max_norm=1.0)
    1단계 손실         2817  그래디언트 크기      53.67
   10단계 손실         2792  그래디언트 크기      56.96
   12단계 손실         2786  그래디언트 크기      59.72
   13단계 손실         2783  그래디언트 크기      61.01
   14단계 손실         2780  그래디언트 크기      62.36
   15단계 손실         2777  그래디언트 크기      63.78
  200단계 손실         2016  그래디언트 크기      58.86
```

클리핑이 없을 때는 10단계까지 그럭저럭 내려가던 손실이 12단계부터 갑자기 5만, 37억, 10의 24제곱으로 불어나고 15단계에 `inf`, 그 뒤로는 `nan`이 됩니다. 그래디언트 크기도 136 → 1만 → 4,500만으로 폭발했어요. 이상치 때문에 커진 그래디언트로 한 걸음 크게 갔더니 예측이 더 엉뚱해지고, 그래서 그래디언트가 더 커지는 악순환입니다. 한 번 `nan`이 되면 모든 가중치가 `nan`이 되어 되돌릴 수 없어요.

그래디언트 클리핑(gradient clipping) 은 이 악순환을 끊습니다. `clip_grad_norm_`은 모든 파라미터의 그래디언트를 하나로 이어 붙인 화살표의 길이(노름)를 재고, 그 길이가 `max_norm`보다 길면 **방향은 그대로 두고 길이만** `max_norm`으로 줄여요. 비탈이 아무리 가팔라도 한 걸음의 보폭에는 상한을 두는 셈입니다. 이 함수는 줄이기 **전의** 길이를 돌려주므로, 위 코드처럼 그래디언트 크기를 기록하는

데도 쓸 수 있어요. 클리핑을 켜니 그래디언트 크기가 수십 수준에서 더 커지지 않았고, 손실이 200단계 동안 2,817에서 2,016까지 꾸준히 내려갔습니다.

두 가지를 덧붙여 둡니다.

- `clip_grad_norm_`은 반드시 **`backward()`와 `optimizer.step()` 사이**에서 불러야 합니다. 그래디언트가 계산된 뒤, 가중치를 고치기 전이에요.
- 클리핑은 안전벨트이지 치료제가 아닙니다. 위 예제에서도 손실이 천천히 내려가는 건 이상치 두 개가 여전히 손실을 끌어올리고 있기 때문이에요. 그래디언트 크기를 기록하다가 자주 폭발한다면 데이터에 이상한 값이 없는지, 학습률이 너무 크지 않은지부터 확인하세요.

8.10 재현성 체크리스트

"어제 돌렸을 때는 89%였는데 오늘은 87%가 나와요." 딥러닝을 하다 보면 자주 하는 말입니다. 가중치 초기화, 데이터 섞기, 드롭아웃이 모두 난수를 쓰기 때문이에요. 정규화를 하나 더했을 때 성적이 1% 오른 게 그 덕분인지, 난수가 바뀐 탓인지 구분하려면 **같은 코드를 돌리면 같은 결과가 나오게** 만들어야 합니다. 4장에서 `torch.manual_seed`를 처음 봤고, 1권 8장에서도 `random.seed`를 썼죠. 학습 전체로 넓혀 보겠습니다.

```
import os
import random

import numpy as np
import torch
from torch.utils.data import random_split

from 학습도구 import 데이터_준비, 로더_만들기, 모델_만들기, 평가, 한_에폭_학습


def 씨앗_고정(씨앗: int = 42) -> None:
    """파이썬·NumPy·PyTorch의 난수 씨앗을 한꺼번에 고정한다."""
    random.seed(씨앗)
    np.random.seed(씨앗)
    torch.manual_seed(씨앗)                    # CPU와 모든 GPU의 씨앗
    os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"   # CUDA 행렬 곱 결정성용
    torch.use_deterministic_algorithms(True)   # 결정적이지 않은 연산은 오류로 알린다
```

```
def 한_번_학습(씨앗: int) -> float:
    씨앗_고정(씨앗)
    (X_학습, y_학습), (X_검증, y_검증), _ = 데이터_준비()
    로더 = 로더_만들기(X_학습, y_학습)
    모델 = 모델_만들기(드롭아웃=0.5)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
    for 에폭 in range(30):
        한_에폭_학습(모델, 로더, 옵티마이저)
    return 평가(모델, X_검증, y_검증)[0]


print("씨앗 42, 첫 번째:", 한_번_학습(42))
print("씨앗 42, 두 번째:", 한_번_학습(42))
print("씨앗 7          :", 한_번_학습(7))

# 데이터 분할도 생성기로 고정한다
for 번 in (1, 2):
    _, 검증용 = random_split(range(10), [7, 3], generator=torch.Generator().manual_seed(0))
    print(f"{번}번째 분할의 검증용 번호:", list(검증용))
```

```
씨앗 42, 첫 번째: 0.5159433484077454
씨앗 42, 두 번째: 0.5159433484077454
씨앗 7          : 0.5170934200286865
1번째 분할의 검증용 번호: [8, 6, 2]
2번째 분할의 검증용 번호: [8, 6, 2]
```

같은 씨앗 42로 두 번 학습하니 검증 손실이 소수점 끝자리까지 똑같습니다. 씨앗을 7로 바꾸면 결과가 달라져요. `씨앗_고정()` 함수가 하는 일을 하나씩 보겠습니다.

- `random.seed`, `np.random.seed`, `torch.manual_seed`: 파이썬, NumPy, PyTorch는 난수 발생기를 **따로따로** 갖고 있어서 셋 다 고정해야 합니다. 데이터 처리 코드 어딘가에서 `random.shuffle`을 쓰고 있을 수 있으니까요. `torch.manual_seed`는 CPU와 GPU의 씨앗을 함께 고정합니다.
- `torch.use_deterministic_algorithms(True)`: GPU의 일부 연산은 속도를 위해 계산 순서

가 매번 조금씩 달라서, 씨앗이 같아도 결과의 끝자리가 달라질 수 있습니다. 이 설정을 켜면 결정적인 방법으로 계산하고, 결정적인 방법이 없는 연산을 만나면 조용히 넘어가는 대신 오류로 알려 줘요. NVIDIA GPU에서는 행렬 곱을 결정적으로 하기 위해 `CUBLAS_WORKSPACE_CONFIG` 환경 변수도 함께 정해야 합니다. 이 환경 변수는 CUDA를 처음 쓰기 **전에** 정해야 효과가 있으니, 실제 코드에서는 `씨앗_고정()`을 스크립트 맨 처음에 부르세요.

마지막 부분은 7장에서 본 `random_split`입니다. `generator=`에 씨앗을 고정한 생성기를 넘기면 몇 번을 나눠도 같은 분할이 나와요. 이 장의 `데이터_준비()`와 `로더_만들기()`가 생성기를 따로 쓰는 것도 같은 이유입니다. 전역 씨앗만 믿으면, 중간에 난수를 쓰는 코드가 한 줄만 끼어들어도 그 뒤의 분할과 섞는 순서가 모두 바뀌어 버리거든요. 실제로 이 장의 예제를 만들 때도, 데이터 로더 하나를 여러 실험이 돌려 쓰는 바람에 같은 설정인데 8.1절과 결과가 다르게 나온 일이 있었습니다. 로더를 실험마다 새로 만들어서 해결했어요.

확인할 것	방법
세 가지 난수 씨앗	`random.seed`, `np.random.seed`, `torch.manual_seed`
데이터 분할 고정	`random_split(..., generator=...)`, 분할 결과(번호 목록)를 파일로 저장
섞는 순서 고정	`DataLoader(..., generator=...)`
결정적인 연산	`torch.use_deterministic_algorithms(True)` (+ CUDA는 `CUBLAS_WORKSPACE_CONFIG`)
워커 프로세스	`num_workers > 0`이면 `worker_init_fn`으로 워커 안의 `random`·NumPy 씨앗 정하기 (PyTorch 씨앗은 DataLoader가 워커마다 알아서 나눠 줌)
환경 기록	파이썬·PyTorch 버전, 장치(CPU/CUDA/MPS) 기록하기

마지막 줄도 중요합니다. 씨앗을 다 고정해도 **CPU와 GPU, 또는 PyTorch 버전이 다르면 결과가 조금 다를 수 있어요.** 그래서 이 장의 수치는 모두 CPU에서 잰 것입니다. 실험 기록을 남기는 도구와 체계적인 재현성 관리는 18장에서 본격적으로 다룹니다.

8.11 손실이 안 줄 때 디버깅

학습을 돌렸는데 손실이 꿈쩍도 하지 않거나, 줄다가 nan이 되거나, 줄긴 하는데 성적이 이상하게 나쁠 때가 있습니다. 이때 하이퍼파라미터부터 이것저것 바꿔 보는 것은 가장 시간이 많이 드는 방법이에요. 아래 순서대로 **싼 확인부터** 하나씩 지워 나가는 것이 훨씬 빠릅니다.

1. 학습 전 첫 손실이 예상한 값인가?
2. 배치 하나를 완벽하게 외울 수 있는가?
3. 학습률이 알맞은 범위인가?
4. 층마다 그래디언트가 제대로 흐르는가?
5. 흔한 버그 목록에 걸리는 것은 없는가?

1단계: 첫 손실 확인하기

학습을 시작하기 **전**의 손실은 계산으로 예측할 수 있습니다. 10개 클래스 분류에서 아무것도 모르는 모델은 10개에 거의 같은 확률(1/10)을 줄 테니, 교차 엔트로피 손실은 -ln(1/10) = ln(10), 약 2.3026 근처여야 해요.

```
import math

import torch
from torch import nn

from 학습도구 import 데이터_준비, 모델_만들기

(X_학습, y_학습), _, _ = 데이터_준비(잡음_비율=0.0)
print(f"찍기 수준의 손실 -ln(1/10) = {math.log(10):.4f}")

torch.manual_seed(0)
모델 = 모델_만들기()
with torch.no_grad():
    print(f"학습 전 손실(정상)       = {nn.functional.cross_entropy(모델(X_학습), y_학
    ↪ 습).item():.4f}")
    모델[-1].weight.mul_(100)                # 마지막 층 가중치를 100배로 (잘못된 초기화)
    print(f"학습 전 손실(출력이 큼)  = {nn.functional.cross_entropy(모델(X_학습), y_학
    ↪ 습).item():.4f}")
```

```
찍기 수준의 손실 -ln(1/10) = 2.3026
학습 전 손실(정상)       = 2.3190
학습 전 손실(출력이 큼)  = 7.1326
```

정상 모델의 첫 손실은 2.3190으로 예상값과 거의 같습니다. 마지막 층 가중치를 100배로 키운 모델은 7.1326이에요. 이렇게 첫 손실이 예상보다 훨씬 크다면, 모델이 아무것도 모르면서 엉뚱한 답에 **강하게 확신하고** 있다는 뜻입니다. 초기화나 입력 스케일(예를 들어 0~255 픽셀 값을 0~1로 바꾸지 않았다거나)을 의심해 보세요. 이 확인은 한 줄이면 되고 몇 초면 끝납니다.

2단계: 배치 하나에 과적합시켜 보기

가장 강력한 디버깅 방법입니다. 데이터 **배치 하나(32장)** 만 가지고 수백 번 학습시켜서, 손실이 0 가까이 내려가는지 보는 거예요. 8.1절에서 봤듯이 신경망은 적은 데이터를 외우는 데는 선수입니다. 32장조차 못 외운다면 데이터가 아니라 **코드에** 문제가 있는 거예요.

```
import torch
from torch import nn

from 학습도구 import 데이터_준비, 모델_만들기

(X_학습, y_학습), _, _ = 데이터_준비(잡음_비율=0.0)
X_한배치, y_한배치 = X_학습[:32], y_학습[:32]     # 배치 딱 하나


def 한_배치_과적합(버그: bool) -> None:
    torch.manual_seed(0)
    모델 = 모델_만들기()
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
    if 버그:
        모델 = 모델_만들기()                 # 옵티마이저를 만든 뒤 모델을 다시 만들었다!
    for 단계 in range(201):
        옵티마이저.zero_grad()
        손실 = nn.functional.cross_entropy(모델(X_한배치), y_한배치)
        손실.backward()
        옵티마이저.step()
        if 단계 in (0, 50, 100, 200):
```

```
            print(f"  {단계:3d}단계 손실 {손실.item():.4f}")


print("정상 코드")
한_배치_과적합(버그=False)
print("버그가 있는 코드")
한_배치_과적합(버그=True)
```

```
정상 코드
    0단계 손실 2.3140
   50단계 손실 0.0391
  100단계 손실 0.0035
  200단계 손실 0.0011
버그가 있는 코드
    0단계 손실 2.2960
   50단계 손실 2.2960
  100단계 손실 2.2960
  200단계 손실 2.2960
```

정상 코드는 200단계 만에 손실이 0.0011까지 내려갔습니다. 버그가 있는 코드는 처음부터 끝까지 2.2960에서 꼼짝도 하지 않아요. 옵티마이저를 만든 **뒤에** 모델을 다시 만드는 바람에, 옵티마이저는 버려진 옛 모델의 가중치를 고치고 있고 실제로 쓰는 새 모델은 그대로이기 때문입니다. 코드를 정리하다가 모델 만드는 줄을 아래로 옮겼을 때 흔히 생기는 실수예요. 오류 메시지도 없어서 전체 데이터로 오래 돌렸다면 "모델이 너무 작은가?" 하며 엉뚱한 데서 시간을 보냈을 겁니다. 배치 하나로 확인하면 몇 초 만에 "코드 문제"라는 것을 알 수 있어요.

3단계: 학습률 범위 찾기

5장에서 학습률이 너무 작으면 느리고, 너무 크면 발산하는 것을 봤습니다. 좋은 학습률을 찾는 빠른 방법이 **학습률 범위 테스트(learning rate range test)** 예요. 아주 작은 학습률에서 시작해 단계마다 조금씩 곱해서 키우면서 손실을 기록하고, 손실이 폭발하면 멈춥니다.

```
import math


import matplotlib.pyplot as plt
```

```
import torch
from torch import nn

from 학습도구 import 데이터_준비, 모델_만들기

(X_학습, y_학습), _, _ = 데이터_준비(잡음_비율=0.0, 학습_개수=1000)
torch.manual_seed(0)
모델 = 모델_만들기()
시작_lr, 끝_lr, 단계_수 = 1e-6, 1.0, 100
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=시작_lr)
배율 = (끝_lr / 시작_lr) ** (1 / 단계_수)      # 단계마다 학습률에 곱할 값

학습률들, 손실들 = [], []
평균 = None
for 단계 in range(단계_수):
    순서 = torch.randint(0, len(X_학습), (32,))
    옵티마이저.zero_grad()
    손실 = nn.functional.cross_entropy(모델(X_학습[순서]), y_학습[순서])
    손실.backward()
    옵티마이저.step()
    평균 = 손실.item() if 평균 is None else 0.9 * 평균 + 0.1 * 손실.item()   # 흔들림 줄이기
    학습률들.append(옵티마이저.param_groups[0]["lr"])
    손실들.append(평균)
    if not math.isfinite(평균) or 평균 > 4 * min(손실들):
        break                                  # 손실이 폭발하면 그만
    for 그룹 in 옵티마이저.param_groups:
        그룹["lr"] *= 배율

for i in range(0, len(학습률들), 20):
    print(f"학습률 {학습률들[i]:9.2e}  손실 {손실들[i]:.4f}")
print(f"마지막: 학습률 {학습률들[-1]:.2e}, 손실 {손실들[-1]:.4f}")
최저 = min(range(len(손실들)), key=손실들.__getitem__)
print(f"손실이 가장 낮았던 학습률: {학습률들[최저]:.2e}")

plt.rcParams["font.family"] = "AppleGothic"
plt.rcParams["axes.unicode_minus"] = False
```

```
plt.figure(figsize=(7, 3.5))
plt.plot(학습률들, 손실들)
plt.xscale("log")
눈금 = [1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1]
plt.xticks(눈금, [f"{v:g}" for v in 눈금])     # 눈금 글자를 1e-06, 0.01처럼
plt.xlabel("학습률 (로그 눈금)")
plt.ylabel("손실 (이동 평균)")
plt.tight_layout()
plt.savefig("lr_finder.png", dpi=200)
```

```
학습률  1.00e-06  손실 2.3093
학습률  1.58e-05  손실 2.3133
학습률  2.51e-04  손실 2.2987
학습률  3.98e-03  손실 1.9658
학습률  6.31e-02  손실 1.1300
마지막: 학습률 1.45e-01, 손실 6.0337
손실이 가장 낮았던 학습률: 6.31e-02
```

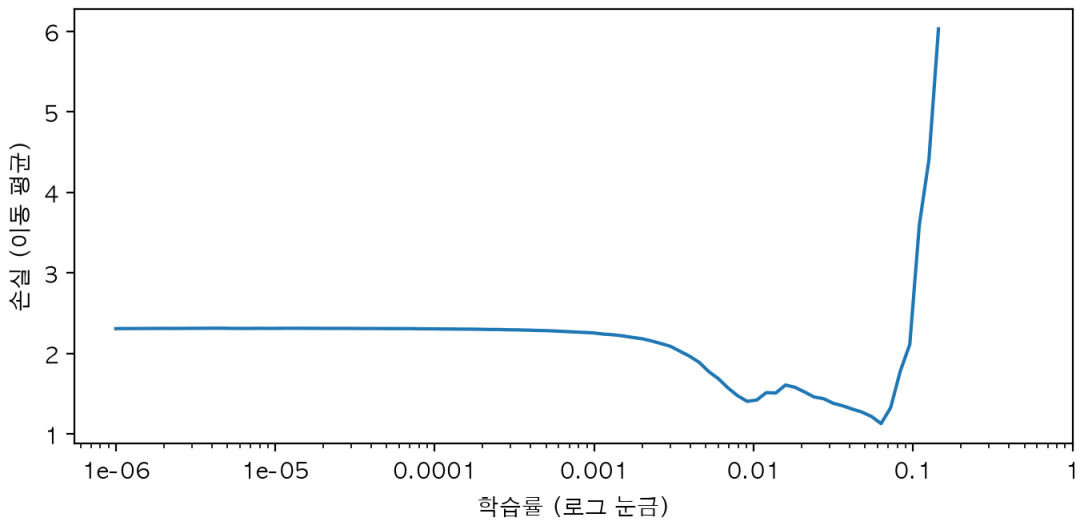


그림 8.3: 학습률 범위 테스트 결과. 학습률 0.001 근처부터 손실이 줄기 시작해 0.06 근처에서 가장 낮고, 0.1을 넘자 급격히 커진다.

100단계 동안 학습률을 1e-6에서 1까지 같은 비율(**배율**)로 키웠습니다. 단계마다 손실이 들쭉날쭉

해서 이전 값을 90% 섞은 이동 평균으로 부드럽게 만들었고, 손실이 최저치의 4배를 넘으면 그만두게 했어요. 결과를 읽는 법은 이렇습니다.

- 1e-4 아래에서는 손실이 거의 그대로입니다. 걸음이 너무 작아서 100단계로는 아무 변화가 없어요.
- 1e-3 근처부터 손실이 **줄기 시작하고**, 6.31e-02에서 가장 낮았습니다.
- 그 뒤로는 손실이 급격히 솟구치며 1.45e-01에서 멈췄습니다. 이 학습률은 너무 커요.

손실이 가장 낮은 지점의 학습률을 그대로 쓰면 이미 불안정한 경계에 가깝기 때문에, 보통 **그보다 10분의 1쯤 작은 값**, 또는 손실이 가장 가파르게 줄어드는 구간의 값을 고릅니다. 이 경우 0.005 안팎이 후보예요. 8.7절의 OneCycleLR을 쓴다면 손실이 가장 낮았던 근처를 `max_lr`로 잡기도 합니다. 그래프가 평평하기만 하고 어디서도 줄지 않는다면, 학습률이 아니라 2단계로 돌아가 코드를 의심하세요.

4단계: 그래디언트가 흐르는지 확인하기

손실이 줄지 않는다면 그래디언트가 앞쪽 층까지 제대로 전해지는지 직접 볼 수 있습니다. `named_parameters()`로 층마다 그래디언트의 크기를 출력하는 함수를 만들어, 8.6절의 깊은 모델에 시그모이드를 쓴 경우와 ReLU + Kaiming 초기화를 쓴 경우를 비교해 보겠습니다.

```
import torch
from torch import nn

from 학습도구 import 깊은_모델, 데이터_준비


def 그래디언트_크기_보기(모델: nn.Module) -> None:
    """층마다 가중치 그래디언트의 크기(노름)를 첫 층부터 차례로 출력한다."""
    크기들 = [f"{p.grad.norm().item():.1e}" for 이름, p in 모델.named_parameters()
            if 이름.endswith("weight") and p.grad is not None]
    print("  " + " ".join(크기들))


(X_학습, y_학습), _, _ = 데이터_준비(잡음_비율=0.0)

torch.manual_seed(0)
시그모이드_모델 = 깊은_모델(활성화=nn.Sigmoid)
```

```
nn.functional.cross_entropy(시그모이드_모델(X_학습), y_학습).backward()
print("Sigmoid 신경망")
그래디언트_크기_보기(시그모이드_모델)

torch.manual_seed(0)
렐루_모델 = 깊은_모델()
for 층 in 렐루_모델.modules():
    if isinstance(층, nn.Linear):
        nn.init.kaiming_normal_(층.weight, nonlinearity="relu")
nn.functional.cross_entropy(렐루_모델(X_학습), y_학습).backward()
print("ReLU 신경망 (Kaiming 초기화)")
그래디언트_크기_보기(렐루_모델)
```

```
Sigmoid 신경망
  9.2e-08 2.8e-07 1.9e-06 1.2e-05 9.2e-05 6.4e-04 4.5e-03 3.2e-02 2.1e-01
ReLU 신경망 (Kaiming 초기화)
  3.5e-01 3.2e-01 2.9e-01 3.3e-01 3.8e-01 3.8e-01 4.1e-01 4.1e-01 3.1e-01
```

숫자는 왼쪽이 첫 층, 오른쪽이 마지막 층입니다. 시그모이드 신경망은 마지막 층의 그래디언트가 0.21인데, 한 층 앞으로 갈 때마다 약 7분의 1씩 줄어서 첫 층에서는 0.000000092입니다. 첫 층은 사실상 배우지 않는다는 뜻이에요. 시그모이드의 기울기는 아무리 커도 0.25라서, 층을 지날 때마다 그래디언트가 깎여 나가는 **그래디언트 소실(vanishing gradient)** 이 생깁니다. ReLU와 Kaiming 초기화를 쓴 쪽은 모든 층의 그래디언트가 0.3~0.4 근처로 고르게 유지돼요.

이렇게 층마다 그래디언트 크기를 찍어 보면 문제가 어디서 생기는지 바로 보입니다. 앞쪽 층으로 갈수록 몇 자리씩 작아지면 소실, 반대로 터무니없이 크면 8.9절의 폭주예요. 어떤 파라미터의 `grad`가 `None`이라면 그 파라미터는 계산 그래프에 아예 연결되지 않은 것이니, 5장에서 본 `detach()`나 `no_grad()`가 엉뚱한 곳에 들어가 있지 않은지 확인하세요.

5단계: 흔한 버그 목록

마지막으로 실제로 자주 만나는 버그 네 가지를 재현해 보겠습니다. 모두 **오류 없이 실행된다**는 공통점이 있어서 더 위험해요.

버그 1: `zero_grad()` 누락. 5.7절에서 `backward()`는 그래디언트를 덮어쓰지 않고 **더한다**는 것과, 비우지 않으면 회귀 학습이 출렁인다는 것을 봤죠. 아래 앞부분은 그 성질을 다시 확인하는 세 줄이고,

이번에는 미니배치로 분류 모델을 학습할 때 빠뜨리면 어떻게 되는지 보겠습니다.

```
import torch
from torch import nn

from 학습도구 import 데이터_준비, 로더_만들기, 모델_만들기, 평가

# backward()는 그래디언트를 덮어쓰지 않고 더한다
w = torch.tensor(1.0, requires_grad=True)
for 번 in range(1, 4):
    (3 * w).backward()                          # d(3w)/dw = 3
    print(f"{번}번째 backward 뒤 w.grad = {w.grad.item()}")

(X_학습, y_학습), (X_검증, y_검증), _ = 데이터_준비(잡음_비율=0.0, 학습_개수=1000)
for 비울까 in (True, False):
    torch.manual_seed(0)
    로더 = 로더_만들기(X_학습, y_학습)
    모델 = 모델_만들기()
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
    기록 = []
    for 에폭 in range(1, 21):
        모델.train()
        for X, y in 로더:
            if 비울까:
                옵티마이저.zero_grad()          # 이 한 줄이 없으면?
            nn.functional.cross_entropy(모델(X), y).backward()
            옵티마이저.step()
        if 에폭 in (1, 2, 3, 5, 10, 15, 20):
            기록.append(f"{에폭}에폭 {평가(모델, X_검증, y_검증)[1]:.0%}")
    print("zero_grad 있음:" if 비울까 else "zero_grad 없음:", ", ".join(기록))
```

```
1번째 backward 뒤 w.grad = 3.0
2번째 backward 뒤 w.grad = 6.0
3번째 backward 뒤 w.grad = 9.0
zero_grad 있음: 1에폭 83%, 2에폭 88%, 3에폭 92%, 5에폭 94%, 10에폭 96%, 15에폭 97%, 20에폭 98%
zero_grad 없음: 1에폭 66%, 2에폭 49%, 3에폭 41%, 5에폭 52%, 10에폭 88%, 15에폭 80%, 20에폭 83%
```

`backward()`를 세 번 부르니 그래디언트가 3, 6, 9로 쌓입니다. 학습 루프에서 이것이 일어나면 매 걸음이 지금 배치의 그래디언트가 아니라 **지금까지 본 모든 배치의 그래디언트 합**을 따라가게 돼요. 결과를 보면 `zero_grad()`가 있을 때는 검증 정확도가 83% → 98%로 꾸준히 오르지만, 없을 때는 66%, 49%, 41%로 오히려 떨어졌다가 88%, 80%, 83%로 널뛰기합니다. 손실이 어느 정도 줄기는 해서 "학습이 좀 불안정하네" 하고 넘어가기 쉬운 버그예요.

버그 2: `eval()` 누락. 8.3절에서 이미 봤습니다. 드롭아웃이 있는 모델을 `train` 모드로 평가하니 같은 입력에도 매번 다른 답이 나오고, 검증 정확도가 87.6%에서 82.8%로 낮게 측정됐어요. 배치 정규화가 있는 모델이라면 예측할 때마다 그 배치의 평균으로 표준화하는 바람에, 배치에 어떤 데이터가 같이 묶였느냐에 따라 답이 바뀝니다. 평가 함수 맨 앞에 `model.eval()`을, 학습 함수 맨 앞에 `model.train()`을 두는 습관을 들이세요. 이 장의 `평가()`와 `한_에폭_학습()`이 그렇게 되어 있습니다.

버그 3: 소프트맥스 두 번. 6.5절에서 `nn.CrossEntropyLoss`는 안에서 소프트맥스(정확히는 로그 소프트맥스)를 계산하니 모델 끝에 소프트맥스를 붙이면 안 된다는 것을 손실 값 몇 개로 봤죠. 이번에는 붙인 채로 실제로 학습하면 어떻게 되는지 보겠습니다.

```
import torch
from torch import nn

from 학습도구 import 데이터_준비, 모델_만들기, 평가

(X_학습, y_학습), (X_검증, y_검증), _ = 데이터_준비(잡음_비율=0.0, 학습_개수=1000)

for 소프트맥스_추가 in (False, True):
    torch.manual_seed(0)
    모델 = 모델_만들기()
    if 소프트맥스_추가:
        모델.append(nn.Softmax(dim=1))        # 모델 끝에 소프트맥스를 붙였다
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
    for 단계 in range(300):
        옵티마이저.zero_grad()
        손실 = nn.CrossEntropyLoss()(모델(X_학습), y_학습)   # 안에서 또 소프트맥스
        손실.backward()
        옵티마이저.step()
    _, 정확도 = 평가(모델, X_검증, y_검증)
```

```
    print(f"소프트맥스 {'두 번' if 소프트맥스_추가 else '한 번'}: 학습 손실 {손실.item():.4f}, 검증
    ↪ 정확도 {정확도:.1%}")

# 확률이 완벽한 원-핫이어도 소프트맥스를 한 번 더 거치면 손실은 이 아래로 못 내려간다
완벽 = torch.zeros(1, 10)
완벽[0, 3] = 1.0
print(f"넘을 수 없는 바닥: {nn.CrossEntropyLoss()(완벽, torch.tensor([3])).item():.4f}")
```

```
소프트맥스 한 번: 학습 손실 0.0021, 검증 정확도 97.8%
소프트맥스 두 번: 학습 손실 1.5638, 검증 정확도 89.4%
넘을 수 없는 바닥: 1.4612
```

같은 300단계를 학습했는데, 정상 모델의 학습 손실은 0.0021, 소프트맥스를 두 번 거친 모델은 1.5638입니다. 검증 정확도도 97.8%와 89.4%로 차이가 나요. 이유는 마지막 줄에 있습니다. 소프트맥스의 출력은 0~1 사이의 확률이라, 이걸 다시 소프트맥스에 넣으면 아무리 완벽한 확률(정답 1, 나머지 0)이라도 e : 1(e는 약 2.718)의 비율밖에 못 만들어요. 그래서 손실이 **1.4612 아래로는 절대 내려갈 수 없습니다.** 바닥이 막혀 있으니 그래디언트도 약해져서 학습이 느려지고요. 손실이 이상하게 1점대에서 멈춰 있다면 이 버그를 의심하세요.

버그 4: 레이블 자료형과 모양. 정답 텐서의 자료형이나 모양이 어긋나는 경우입니다.

```
import torch
from torch import nn

출력 = torch.randn(4, 10)                    # 샘플 4개 × 클래스 10개
정답_실수 = torch.tensor([3.0, 1.0, 0.0, 7.0])   # 예: pandas에서 실수로 읽힌 정답
try:
    nn.CrossEntropyLoss()(출력, 정답_실수)
except RuntimeError as e:
    print("오류:", e)
print("고친 뒤:", nn.CrossEntropyLoss()(출력, 정답_실수.long()).item() > 0)

# 회귀에서는 모양이 어긋나도 오류 없이 계산된다
torch.manual_seed(0)
예측 = torch.randn(4, 1)                     # 모델 출력은 (4, 1)
정답 = torch.randn(4)                        # 정답은 (4,)
```

```
print(f"모양이 어긋난 손실: {nn.functional.mse_loss(예측, 정답).item():.4f}")
print(f"모양을 맞춘 손실  : {nn.functional.mse_loss(예측.squeeze(1), 정답).item():.4f}")
print("(예측 - 정답)의 모양:", tuple((예측 - 정답).shape))
```

```
오류: expected target dtype to be Long or Byte, but got Float
고친 뒤: True
모양이 어긋난 손실: 2.8257
모양을 맞춘 손실  : 3.7137
(예측 - 정답)의 모양: (4, 4)
```

분류의 정답은 클래스 번호이므로 `int64`(`long`)여야 합니다. pandas에서 읽은 정답 열에 결측치가 한 번이라도 있었다면 실수형이 되어 있기 쉬운데, 그대로 넣으면 "정답 자료형은 Long이나 Byte여야 하는데 Float이 왔다"는 오류가 나요. 4장 4.3절의 자료형 표에서 정답 번호는 `torch.int64`라고 했던 것을 떠올리고 `.long()`으로 바꾸면 됩니다. 이건 오류가 나니 차라리 다행인 경우예요.

정말 위험한 건 회귀의 모양 문제입니다. 모델 출력은 `(4, 1)`, 정답은 `(4,)`인 채로 `mse_loss`를 계산하면 오류 없이 2.8257이라는 그럴듯한 숫자가 나와요. 그런데 진짜 손실은 3.7137입니다. 6.5절에서 본 바로 그 함정이에요. `(4, 1) - (4,)`가 브로드캐스팅되어 `(4, 4)` 표가 됐고, 모든 예측과 모든 정답의 조합으로 평균을 냈습니다. 경고 한 줄은 학습 로그에 묻혀 지나치기 쉬우니, 손실을 계산하기 직전에 예측과 정답의 `shape`를 한 번 찍어 보는 습관이 가장 확실한 예방법이에요.

지금까지의 버그를 표로 정리해 두겠습니다.

증상	의심할 것	확인·해결
손실이 처음부터 꼼짝 안 함	옵티마이저와 모델이 연결 안 됨, `grad`가 `None`, 학습률이 너무 작음	배치 하나 과적합 테스트, 층별 그래디언트 출력
첫 손실이 ln(클래스 수)보다 훨씬 큼	초기화, 입력 스케일	첫 손실 확인, 입력을 0~1이나 표준화로
손실이 줄다가 `nan`	학습률이 너무 큼, 그래디언트 폭주, 데이터 이상치	학습률 범위 테스트, 그래디언트 클리핑
손실이 출렁이며 잘 안 줄어듦	`zero_grad()` 누락, 학습률이 큼	학습 루프 순서 확인

증상	의심할 것	확인·해결
손실이 1점대에서 멈춤(분류)	소프트맥스 두 번	모델 끝의 소프트맥스 제거
평가할 때마다 성적이 다름	`eval()` 누락	평가 함수 첫 줄에 `model.eval()`
회귀 손실이 그럴듯한데 성능이 나쁨	예측과 정답의 모양 불일치	손실 계산 전 `shape` 출력
앞쪽 층 그래디언트가 0에 가까움	그래디언트 소실	ReLU, Kaiming 초기화, 배치 정규화

8.12 실습: 외우는 신경망에 대책을 하나씩 더하기

이 장에서 배운 대책들을 8.1절의 과적합 모델에 **하나씩 더하면서** 검증 성적이 어떻게 변하는지 정리해 보겠습니다. 한 번 실행한 결과는 난수에 따라 1~2%씩 흔들릴 수 있으니, 씨앗 세 개(0, 1, 2)로 각각 학습해서 평균을 냈어요. 8.10절에서 말한 "그 대책 덕분인지, 난수 탓인지"를 조금이라도 가려내기 위해서입니다.

```
import copy

import torch
from torch.optim.lr_scheduler import CosineAnnealingLR
from torch.utils.data import DataLoader, Dataset
from torchvision.transforms import v2

from 학습도구 import 데이터_준비, 로더_만들기, 모델_만들기, 평가, 한_에폭_학습

(X_학습, y_학습), (X_검증, y_검증), (X_테스트, y_테스트) = 데이터_준비()
최대_에폭 = 150


class 이동_데이터셋(Dataset):
    """꺼낼 때마다 8×8 숫자를 위아래·좌우로 최대 한 칸씩 무작위로 옮긴다(7장의 RandomCrop)."""
```

```
    def __init__(self, X: torch.Tensor, y: torch.Tensor) -> None:
        self.X, self.y = X.reshape(-1, 1, 8, 8), y
        self.옮기기 = v2.RandomCrop(size=8, padding=1)    # 0으로 한 칸 두른 뒤 8×8로 자르기

    def __len__(self) -> int:
        return len(self.y)

    def __getitem__(self, i: int) -> tuple[torch.Tensor, torch.Tensor]:
        return self.옮기기(self.X[i]).flatten(), self.y[i]


def 실험(씨앗: int, wd: float = 0.0, 드롭아웃: float = 0.0, 배치정규화: bool = False,
         조기종료: bool = False, 코사인: bool = False, 증강: bool = False) -> tuple[dict,
↪ torch.nn.Module]:
    """설정 하나로 학습하고 결과 수치와 모델을 돌려준다."""
    torch.manual_seed(씨앗)
    로더 = 로더_만들기(X_학습, y_학습)
    if 증강:                                    # 학습 데이터에만 증강
        로더 = DataLoader(이동_데이터셋(X_학습, y_학습), batch_size=32, shuffle=True,
                          generator=torch.Generator().manual_seed(0))
    모델 = 모델_만들기(드롭아웃=드롭아웃, 배치정규화=배치정규화)
    옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=1e-3, weight_decay=wd)
    스케줄러 = CosineAnnealingLR(옵티마이저, T_max=최대_에폭) if 코사인 else None
    최고_손실, 최고_가중치, 참은_횟수, 멈춘_에폭 = float("inf"), None, 0, 최대_에폭
    for 에폭 in range(1, 최대_에폭 + 1):
        한_에폭_학습(모델, 로더, 옵티마이저)
        if 스케줄러:
            스케줄러.step()
        검증_손실, _ = 평가(모델, X_검증, y_검증)
        if 검증_손실 < 최고_손실:
            최고_손실, 최고_가중치, 참은_횟수 = 검증_손실, copy.deepcopy(모델.state_dict()), 0
        else:
            참은_횟수 += 1
        if 조기종료 and 참은_횟수 >= 20:
            멈춘_에폭 = 에폭
            break
```

```
    if 조기종료:
        모델.load_state_dict(최고_가중치)     # 가장 좋았던 순간으로
    _, 학습_정확도 = 평가(모델, X_학습, y_학습)
    검증_손실, 검증_정확도 = 평가(모델, X_검증, y_검증)
    return dict(학습=학습_정확도, 손실=검증_손실, 검증=검증_정확도, 에폭=멈춘_에폭), 모델


단계들 = [
    ("1. 기준 (대책 없음)", {}),
    ("2. + weight decay 1.0", dict(wd=1.0)),
    ("3. + 드롭아웃 0.5", dict(wd=1.0, 드롭아웃=0.5)),
    ("4. + 배치 정규화", dict(wd=1.0, 드롭아웃=0.5, 배치정규화=True)),
    ("5. 3번 + 조기 종료", dict(wd=1.0, 드롭아웃=0.5, 조기종료=True)),
    ("6. 5번 + 코사인 스케줄러", dict(wd=1.0, 드롭아웃=0.5, 조기종료=True, 코사인=True)),
    ("7. 5번 + 데이터 증강", dict(wd=1.0, 드롭아웃=0.5, 조기종료=True, 증강=True)),
    ("8. 기준 + 데이터 증강만", dict(증강=True)),
]
씨앗들 = [0, 1, 2]
print("학습정확도  검증손실  검증정확도  멈춘에폭  단계")
for 이름, 설정 in 단계들:
    결과들 = [실험(씨앗, **설정)[0] for 씨앗 in 씨앗들]
    평균 = {k: sum(r[k] for r in 결과들) / len(결과들) for k in 결과들[0]}
    print(f"{평균['학습']:9.1%}  {평균['손실']:8.3f}  {평균['검증']:9.1%}  {평균['에폭']:8.0f}  {이
    ↪ 름}")

# 검증 성적으로 고른 5번을, 한 번도 쓰지 않은 테스트 데이터로 마지막에 확인
for 이름, 설정 in [단계들[0], 단계들[4]]:
    정확도들 = [평가(실험(씨앗, **설정)[1], X_테스트, y_테스트)[1] for 씨앗 in 씨앗들]
    print(f"테스트 정확도 {sum(정확도들) / len(정확도들):.1%}  <- {이름}")
```

```
학습정확도  검증손실  검증정확도  멈춘에폭  단계
   100.0%    0.732     82.4%       150  1. 기준 (대책 없음)
    99.2%    0.494     84.7%       150  2. + weight decay 1.0
    90.8%    0.459     88.7%       150  3. + 드롭아웃 0.5
   100.0%    0.478     84.7%       150  4. + 배치 정규화
    88.7%    0.459     89.4%       128  5. 3번 + 조기 종료
```

```
    86.2%     0.482      89.3%       126  6. 5번 + 코사인 스케줄러
    75.7%     0.777      82.7%       134  7. 5번 + 데이터 증강
    84.8%     0.504      87.1%       150  8. 기준 + 데이터 증강만
테스트 정확도 76.8%  <- 1. 기준 (대책 없음)
테스트 정확도 86.5%  <- 5. 3번 + 조기 종료
```

`실험()` 함수 하나가 설정에 따라 weight decay, 드롭아웃, 배치 정규화, 조기 종료(참을성 20), 코사인 스케줄러, 데이터 증강을 켜고 끕니다. 증강은 `이동_데이터셋`이 맡아요. 7장의 `RandomCrop`으로 테두리에 0을 한 칸 두른 뒤 8×8로 잘라서, 꺼낼 때마다 숫자를 위아래·좌우로 최대 한 칸씩 옮깁니다. 학습용 로더에만 쓰고 검증·테스트 데이터는 그대로 둡니다. 조기 종료를 켜지 않은 단계도 최대 에폭은 150으로 같아요. 결과를 표로 옮기면 이렇습니다.

단계	학습 정확도	검증 손실	검증 정확도	바뀐 점
1. 기준	100.0%	0.732	82.4%	틀린 정답까지 모두 외움
2. + weight decay 1.0	99.2%	0.494	84.7%	검증 손실이 크게 줄어듦
3. + 드롭아웃 0.5	90.8%	0.459	88.7%	외우지 못하게 되면서 검증 정확도 4%p 상승
4. + 배치 정규화	100.0%	0.478	84.7%	다시 100% 외움, 검증 정확도 하락
5. 3번 + 조기 종료	88.7%	0.459	89.4%	평균 128에폭에서 멈춤
6. 5번 + 코사인 스케줄러	86.2%	0.482	89.3%	이 문제에서는 차이 없음
7. 5번 + 데이터 증강	75.7%	0.777	82.7%	대책이 겹쳐 과소적합
8. 기준 + 데이터 증강만	84.8%	0.504	87.1%	증강 하나로 검증 정확도 4.7%p 상승

몇 가지를 짚어 보겠습니다.

- **학습 정확도가 내려가면서 검증 정확도가 올라갑니다.** 1~3단계에서 학습 정확도는 100% → 90.8%, 검증 정확도는 82.4% → 88.7%예요. 학습 데이터에는 정답이 틀린 것이 17%쯤(200장 중 34장) 섞여 있으니, 학습 정확도가 100%라는 건 칭찬이 아니라 경고였던 셈입니다.
- **모든 대책이 늘 도움이 되지는 않습니다.** 배치 정규화를 더하자 학습 정확도가 다시 100%가 되고 검증 정확도가 84.7%로 떨어졌어요. 이 얕은 모델에서는 배치 정규화가 학습을 쉽게 만들어 준 덕분에 오히려 외우기가 쉬워진 것입니다. 8.4절에서 깊은 모델을 살려 낸 것과 정반대죠. 그래서 5단계부터는 배치 정규화를 빼고 3단계에 조기 종료를 더했습니다.
- **코사인 스케줄러는 여기서는 효과가 없었습니다.** 학습률 스케줄러는 큰 데이터로 오래 학습할 때 마지막 몇 %를 짜내는 데 강한데, 이 실험처럼 작고 지저분한 데이터에서는 차이가 잘 드러나지 않아요. 효과가 없으면 빼는 것이 맞습니다. 단순한 쪽이 이깁니다.
- **데이터 증강은 혼자서도 꽤 셉니다.** 7장에서 약속한 숫자예요. 기준 모델에 증강만 더한 8단계는 학습 정확도가 84.8%로 내려오고, 검증 정확도는 82.4%에서 87.1%로 올랐습니다. 같은 숫자를 매번 조금씩 다른 자리에서 보여 주니 틀린 정답 34개를 그대로 외우기 어려워진 거예요. 그런데 이미 weight decay와 드롭아웃을 건 5단계에 증강까지 겹친 7단계는 학습 정확도 75.7%, 검증 정확도 82.7%로 오히려 나빠졌습니다. 8.1절 표로 보면 과소적합이에요. 8×8짜리 작은 그림에서 한 칸은 폭의 8분의 1이나 되는 큰 이동이라, 규제 세 가지가 한꺼번에 걸리자 모델이 제대로 배우지 못한 겁니다. 대책은 많이 쌓을수록 좋은 게 아니라, 검증 데이터로 확인하며 골라야 해요.

마지막 두 줄은 여덟 단계 중 검증 정확도가 가장 높았던 5단계 설정을, 지금까지 한 번도 쓰지 않은 **테스트 데이터** 1,097장으로 확인한 결과입니다. 대책이 없는 모델은 76.8%, 대책을 더한 모델은 86.5%로 약 10%p 차이가 났어요. 두 모델 모두 테스트 점수가 검증 점수보다 낮다는 점도 보세요. 설정을 하나도 고르지 않은 기준 모델조차 82.4%에서 76.8%로 5.6%p나 낮으니, 차이의 대부분은 검증 500장과 테스트 1,097장이라는 두 묶음의 난이도 차이로 봐야 합니다. 데이터가 이렇게 적으면 어느 그림이 뽑혔느냐에 따라 점수가 이 정도는 흔들려요. 여기에 검증 점수는 설정을 고르는 데 이미 썼기 때문에 약간 좋게 나오는 효과(8.8절 Q&A)가 더해집니다. 그래서 대책의 효과는 같은 테스트 데이터 위에서 비교한 차이(약 10%p)로 읽고, 최종 실력은 테스트 점수로 보고합니다.

8.13 정리와 다음 장 미리보기

이 장에서 모델이 외우지 않고 배우게 만드는 도구들과, 학습이 꼬였을 때 원인을 찾는 방법을 익혔습니다.

- **진단**: 학습 손실과 검증 손실을 함께 그린다. 학습만 내려가고 검증이 오르면 과적합, 둘 다 높으면 과소적합. 먼저 과적합할 만큼 학습시킨 뒤 정규화한다
- **weight decay**: 큰 가중치에 벌점. Adam 계열에서는 `AdamW`. 세기는 검증 데이터로 고른다
- **드롭아웃**: 학습 때만 칸을 무작위로 끄고 남은 칸을 `1/(1-p)`배. 평가 때는 그대로 통과
- **배치 정규화**: 배치 평균·분산으로 표준화, 평가 때는 기록해 둔 이동 평균 사용. 깊은 모델의 학습을 살린다
- **가중치 초기화**: 0으로 시작하지 않는다. ReLU에는 Kaiming, 시그모이드·tanh에는 Xavier
- **학습률 스케줄러**: StepLR, CosineAnnealingLR, OneCycleLR(배치마다), ReduceLROnPlateau(검증 손실). `optimizer.step()` 다음에 `scheduler.step()`
- **조기 종료**: 참을성만큼 기다리고, `deepcopy`로 보관한 최고 가중치로 되돌린다. 기준은 검증 데이터
- **그래디언트 클리핑**: `backward()`와 `step()` 사이에 `clip_grad_norm_`. 돌려주는 값으로 그래디언트 크기 기록
- **재현성**: 씨앗 세 개, 분할·섞기용 생성기, 결정적 연산, 환경 기록
- **디버깅 순서**: 첫 손실 → 배치 하나 과적합 → 학습률 범위 → 층별 그래디언트 → 흔한 버그 (`zero_grad`, `eval`, 소프트맥스 두 번, 레이블 자료형·모양)

지금까지 다룬 모델은 모두 입력을 한 줄로 쭉 펴서 받는 다층 신경망이었습니다. 8×8 손글씨 그림도 숫자 64개짜리 한 줄로 폈죠. 그런데 그렇게 펴는 순간 "이 점의 바로 위아래에 어떤 점이 있는가" 하는 그림의 **공간 정보**가 사라집니다. 다음 9장에서는 그림을 그림 모양 그대로 받아서, 작은 돋보기를 옮겨 가며 무늬를 찾는 **합성곱 신경망**(CNN) 을 만납니다. 이 장에서 익힌 배치 정규화, 드롭아웃, 학습률 스케줄러, 조기 종료가 그 모델을 학습시키는 기본 도구로 그대로 쓰여요.

09

CNN과 이미지 분류: 돋보기를 옮기며 무늬 찾기

학습 목표

- 합성곱이 "작은 필터를 옮겨 가며 곱하고 더하는 계산"이라는 것을 손으로 계산해 `nn.Conv2d`와 비교하기
- 패딩과 보폭으로 출력 크기를 계산하고, 채널과 필터 묶음의 모양을 읽기
- 풀링과 전역 평균 풀링이 하는 일을 설명하기
- 작은 CNN으로 FashionMNIST를 학습해 6장의 MLP와 비교하고, 그림을 옆으로 밀었을 때의 정확도 재기
- CNN 속 배치 정규화의 자리를 알고, ResNet의 잔차 블록을 직접 구현하기
- 학습한 필터, 특징 맵, 입력 기울기를 그림으로 그려 CNN이 무엇을 보는지 살피기

들어가며: 사진을 한 줄로 펴면 무엇을 잃을까요?

친구에게 신발 사진을 보여 주며 "이거 무슨 신발이야?"라고 물으면 친구는 사진을 어떻게 볼까요? 픽셀 784개를 왼쪽 위부터 한 줄로 읽어 내려가지는 않겠죠. 발목이 올라온 부분, 굽, 신발 끈 같은 **부분 모양**을 먼저 알아보고, 그 조각들이 어떻게 모여 있는지를 보고 앵클부츠라고 답할 겁니다. 그 부분 모양이 사진의 왼쪽에 있든 오른쪽에 있든 상관없이요.

지금까지 만든 모델은 그렇게 보지 못했습니다. 6장 6.11절의 옷 분류기는 28 × 28 사진을 `nn.Flatten()`으로 784개짜리 한 줄로 편 다음 받아서, 시험 정확도 88.58% 를 냈어요. 꽤 잘한 것

같지만 문제가 두 가지 있었습니다.

- **연결이 너무 많습니다.** 6장 6.4절에서 센 것처럼 첫 층 하나가 파라미터의 98% 넘게 차지했어요. 입력 784칸 하나하나를 은닉층의 모든 칸에 따로 연결했기 때문입니다.
- **자리에 묶여 있습니다.** 1장 1.11절에서는 손글씨 숫자를 오른쪽으로 **한 칸만** 밀어도 최근접 이웃의 정확도가 0.981에서 0.567로 무너졌죠. 한 줄로 편 입력은 "몇 번째 칸에 잉크가 있나"만 알지 "어떤 모양이 있나"는 모릅니다.

첫 번째 문제가 사진이 커지면 얼마나 심각해지는지 숫자로 보겠습니다. 은닉 칸 256개짜리 완전 연결층과, 이 장에서 배울 **합성곱층**에 3 × 3 필터 256개를 둔 것을 비교했어요.

```
from torch import nn


def 파라미터_수(층: nn.Module) -> int:
    """층 하나에 들어 있는 학습할 숫자의 개수."""
    return sum(p.numel() for p in 층.parameters())


# (색 채널, 세로, 가로)
사진들 = {"FashionMNIST 옷 사진": (1, 28, 28), "작게 줄인 컬러 사진": (3, 224, 224)}
for 이름, (채널, 세로, 가로) in 사진들.items():
    완전연결 = nn.Linear(채널 * 세로 * 가로, 256)          # 한 줄로 펴서 256칸에 모두 연결
    합성곱 = nn.Conv2d(채널, 256, kernel_size=3)           # 3×3 돋보기 256개 (9.1절)
    print(f"{이름} {채널}×{세로}×{가로}")
    print(f"  완전 연결층: {파라미터_수(완전연결):>12,}개 ({파라미터_수(완전연결) * 4 /
    ↪ 1024**2:7.2f} MB)")
    print(f"  합성곱층   : {파라미터_수(합성곱):>12,}개 ({파라미터_수(합성곱) * 4 / 1024**2:7.2f}
    ↪ MB)")
```

```
FashionMNIST 옷 사진 1×28×28
  완전 연결층:      200,960개 (   0.77 MB)
  합성곱층   :        2,560개 (   0.01 MB)
작게 줄인 컬러 사진 3×224×224
  완전 연결층:   38,535,424개 ( 147.00 MB)
  합성곱층   :        7,168개 (   0.03 MB)
```

28 × 28 흑백 사진에서는 완전 연결층도 20만 개 정도라 견딜 만합니다. 그런데 작게 줄인 컬러 사진

224 × 224 × 3만 해도 첫 층 하나에 **3,850만 개**, 147MB가 필요해요. 사진이 조금만 커져도 감당이 안 됩니다. 반면 합성곱층은 사진이 커져도 파라미터 수가 그대로예요. 흑백일 때 2,560개, 컬러일 때 7,168개로 색 채널만큼만 늘었습니다. 사진 크기와 상관없이 **작은 돋보기 하나를 사진 위로 옮겨 가며 같은 숫자로 계산**하기 때문입니다.

이 장의 주인공은 이 돋보기를 쓰는 **합성곱 신경망**(Convolutional Neural Network, CNN) 입니다. 돋보기 계산을 손으로 해 보고, 작은 CNN으로 6장의 88.58%를 넘은 뒤, 1장에서 남겨 둔 "밀기" 문제가 어떻게 되는지 확인하겠습니다. 마지막으로 오늘날 이미지 모델의 뼈대인 ResNet의 아이디어를 직접 구현해서 옷 분류기를 한 번 더 개선해요.

9.1 합성곱: 돋보기를 옮기며 무늬 찾기

손으로 하는 합성곱

신문에서 특정 글자를 찾는 놀이를 떠올려 보세요. 작은 돋보기를 신문 왼쪽 위에 대고, 그 안에 찾는 글자가 있는지 보고, 한 칸 옆으로 옮겨서 또 보고, 줄 끝에 닿으면 다음 줄로 내려갑니다. **합성곱**(convolution) 이 바로 이 일이에요. 돋보기에 해당하는 작은 숫자 표를 **필터**(filter) 또는 **커널**(kernel) 이라고 부르고, 돋보기를 댈 때마다 "돋보기 아래 그림 조각과 필터를 같은 자리끼리 곱해서 모두 더한 값"을 적습니다.

'ㄱ' 모양이 그려진 6 × 6 그림에서 세로선을 찾는 3 × 3 필터를 반복문으로 직접 옮겨 보겠습니다.

```
import torch

# 6×6 흑백 그림: 'ㄱ' 모양 (1이 잉크, 0이 빈칸)
그림 = torch.tensor([
    [0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 0],
    [0, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 0, 0],
], dtype=torch.float32)

# 3×3 돋보기(필터): 가운데 세로줄은 +2, 양옆은 -1 → 세로선이 있으면 크게 반응
필터 = torch.tensor([
```

```
    [-1.0, 2.0, -1.0],
    [-1.0, 2.0, -1.0],
    [-1.0, 2.0, -1.0],
])

k = 필터.shape[0]
결과 = torch.zeros(6 - k + 1, 6 - k + 1)          # 돋보기를 놓을 수 있는 자리: 4×4
for i in range(6 - k + 1):                          # 위에서 아래로
    for j in range(6 - k + 1):                      # 왼쪽에서 오른쪽으로
        조각 = 그림[i:i + k, j:j + k]                # 돋보기 아래의 3×3
        결과[i, j] = (조각 * 필터).sum()             # 같은 자리끼리 곱해서 모두 더하기

print(결과)
print("가장 크게 반응한 자리:", divmod(결과.argmax().item(), 결과.shape[1]))
```

```
tensor([[ 1.,  0., -1.,  3.],
        [ 1.,  0., -2.,  5.],
        [ 0.,  0., -3.,  6.],
        [ 0.,  0., -2.,  4.]])
가장 크게 반응한 자리: (2, 3)
```

필터는 가운데 세로줄이 +2, 양옆이 -1입니다. 돋보기 아래에 세로선이 가운데로 지나가면 +2짜리 칸들이 잉크를 만나 값이 커지고, 잉크가 고르게 퍼져 있거나 없으면 +와 -가 상쇄되어 0 근처가 돼요. 결과 표의 오른쪽 열이 3, 5, 6, 4로 큰 것은 'ㄱ'의 세로획이 그 자리에 있기 때문입니다. 가장 크게 반응한 자리 `(2, 3)`은 돋보기의 왼쪽 위 모서리를 그림의 2행 3열에 댔을 때인데, 그때 돋보기 한가운데가 세로획의 아랫부분에 정확히 놓여요.

계산 자체는 4장 4.6절의 "같은 자리끼리 곱하고 모두 더하기"를 조각마다 반복한 것뿐입니다. 6 × 6 그림에 3 × 3 돋보기를 댈 수 있는 자리는 가로세로 4곳씩이라서 결과는 4 × 4로 조금 작아졌어요. 이렇게 나온 결과 표를 **특징 맵**(feature map) 이라고 부릅니다. "그림의 어느 자리에 그 무늬가 얼마나 있는가"를 적은 지도라는 뜻이에요.

F.conv2d와 nn.Conv2d

반복문은 이해하기에는 좋지만 느립니다. PyTorch에는 같은 계산을 하는 함수 `torch.nn.functional.conv2d`와, 필터를 학습할 파라미터로 품고 있는 층 `nn.Conv2d`가 있어요.

```
import torch
import torch.nn.functional as F
from torch import nn

그림 = torch.tensor([
    [0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 0],
    [0, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 0, 0],
], dtype=torch.float32)
필터 = torch.tensor([[-1.0, 2.0, -1.0]] * 3)

# 합성곱 함수는 4차원을 원한다: 그림 (장 수, 채널, 세로, 가로), 필터 (출력 채널, 입력 채널, 세로,
↪ 가로)
x = 그림.reshape(1, 1, 6, 6)
w = 필터.reshape(1, 1, 3, 3)
print(x.shape, "*", w.shape, "→", F.conv2d(x, w).shape)
print(F.conv2d(x, w)[0, 0])

# nn.Conv2d는 필터를 파라미터로 품고 있는 층이다
층 = nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3, bias=False)
print("층의 필터 모양:", 층.weight.shape)
with torch.no_grad():
    층.weight.copy_(w)                              # 무작위 필터를 우리 필터로 바꿔 끼우기
print("손 계산과 같은가?", torch.equal(층(x), F.conv2d(x, w)))
```

```
torch.Size([1, 1, 6, 6]) * torch.Size([1, 1, 3, 3]) → torch.Size([1, 1, 4, 4])
tensor([[ 1.,  0., -1.,  3.],
        [ 1.,  0., -2.,  5.],
        [ 0.,  0., -3.,  6.],
        [ 0.,  0., -2.,  4.]])
층의 필터 모양: torch.Size([1, 1, 3, 3])
손 계산과 같은가? True
```

합성곱 함수는 입력과 필터를 모두 **4차원**으로 받습니다. 입력은 4장 4.2절에서 본 사진 묶음 순서 그대로 (장 수, 채널, 세로, 가로)이고, 필터는 (출력 채널, 입력 채널, 세로, 가로)예요. 채널 이야기는 9.3절에서 하니 지금은 "그림 1장, 채널 1개, 필터 1개"라서 앞쪽이 모두 1이라고만 보면 됩니다. 결과는 반복문으로 구한 4 × 4와 똑같습니다.

`nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3)`은 같은 모양의 필터를 **무작위 값으로** 만들어 품고 있습니다. 6장의 `nn.Linear`가 가중치를 품고 있던 것과 같아요. 여기서는 확인을 위해 `copy_`로 우리 필터를 끼워 넣었지만, 실제로는 필터 값을 사람이 정하지 않습니다. **어떤 무늬를 찾을지를 학습으로 정하는 것**, 이것이 CNN의 핵심이에요. 1장 1.11절에서 말한 "특성을 만드는 방법까지 데이터에서 배운다"가 바로 이 뜻입니다.

Q

Q. 수학 시간에 배운 합성곱은 필터를 뒤집어서 곱한다던데요? 맞습니다. 수학과 신호처리에서 말하는 합성곱은 필터를 위아래, 좌우로 뒤집은 다음 곱하고 더해요. PyTorch를 비롯한 딥러닝 도구의 `conv2d`는 뒤집지 않고 그대로 곱하는데, 엄밀히는 이것을 **상호상관(cross-correlation)** 이라고 부릅니다. 필터 값을 어차피 학습으로 정하기 때문에, 뒤집힌 필터를 배우든 그대로인 필터를 배우든 결과가 같아서 이름만 관례대로 합성곱이라고 부르는 거예요.

필터가 여러 개면 특징 맵도 여러 장

세로선만 찾아서는 'ㄱ'인지 'ㄴ'인지 알 수 없습니다. 가로선, 꺾인 곳도 찾아야 하죠. 필터를 여러 개 쓰면 필터마다 특징 맵이 한 장씩 나옵니다.

```
import torch
import torch.nn.functional as F

그림 = torch.tensor([
    [0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 0],
    [0, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 0, 0],
], dtype=torch.float32).reshape(1, 1, 6, 6)

세로선 = torch.tensor([[-1.0, 2.0, -1.0]] * 3)
```

```
가로선 = 세로선.T                                    # 전치하면 가로선 찾기
모서리 = torch.tensor([[-1.0, -1.0, -1.0],
                       [ 1.0,  1.0, -1.0],
                       [-1.0,  1.0, -1.0]])          # 왼쪽에서 와서 아래로 꺾이는 곳
필터들 = torch.stack([세로선, 가로선, 모서리]).unsqueeze(1)   # (3, 1, 3, 3): 필터 3개
print("필터 묶음:", 필터들.shape)

특징맵 = F.conv2d(그림, 필터들)                       # (1, 3, 4, 4): 필터마다 한 장
print("특징 맵 묶음:", 특징맵.shape)
for 이름, 맵 in zip(["세로선", "가로선", "모서리"], 특징맵[0]):
    print(f"[{이름}] 가장 큰 값 {맵.max():.0f}, 자리 {divmod(맵.argmax().item(), 4)}")
    for 줄 in 맵:
        print("   " + " ".join("#" if v >= 맵.max() / 2 else "+" if v > 0 else "." for v in 줄))
```

```
필터 묶음: torch.Size([3, 1, 3, 3])
특징 맵 묶음: torch.Size([1, 3, 4, 4])
[세로선] 가장 큰 값 6, 자리 (2, 3)
   + . . #
   + . . #
   . . . #
   . . . #
[가로선] 가장 큰 값 6, 자리 (0, 1)
   # # # #
   . . . .
   . . . .
   . . + +
[모서리] 가장 큰 값 3, 자리 (0, 3)
   . + . #
   . . . .
   . . . +
   . . . .
```

필터 세 개를 `stack`으로 쌓아 `(3, 1, 3, 3)`, 즉 "출력 채널 3개"짜리 필터 묶음을 만들었습니다. 결과도 `(1, 3, 4, 4)`로 특징 맵이 세 장 나왔어요. 큰 값(#)이 나온 자리를 보면 세로선 필터는 오른쪽 열(세로획), 가로선 필터는 위쪽 행(가로획), 모서리 필터는 'ㄱ'이 꺾이는 오른쪽 위 한 곳에서만 크게 반응했습니다. 가로선 필터는 세로선 필터를 `.T`로 전치해서 만들었다는 것도 눈여겨보세요.

이제 그림은 "6 × 6 픽셀"이 아니라 "세로선 지도, 가로선 지도, 모서리 지도" 세 장으로 다시 쓰였습니다. 다음 층은 이 지도들을 입력으로 받아서 "위쪽에 가로선이 있고 오른쪽에 세로선이 있으며 그 둘이 만나는 모서리가 있다", 즉 'ㄱ'이라는 더 큰 무늬를 찾을 수 있어요. 1장 1.11절에서 말한 "첫 층은 짧은 선, 다음 층은 그 조합"이 이렇게 만들어집니다.

9.2 패딩과 보폭: 출력 크기 정하기

앞의 예제에서 6 × 6 그림이 4 × 4로 줄었습니다. 3 × 3 돋보기의 가운데는 그림의 가장자리 칸에 닿을 수가 없거든요. 합성곱층을 여러 번 쌓으면 그림이 층마다 깎여 나가서 금방 작아지고, 가장자리 정보도 덜 반영됩니다.

이때 쓰는 방법이 **패딩(padding)** 입니다. 액자 테두리처럼 그림 둘레에 0을 한 칸 둘러서, 돋보기 가운데가 원래 그림의 가장자리까지 닿게 하는 거예요. 반대로 그림을 일부러 줄이고 싶을 때는 돋보기를 한 칸씩이 아니라 두 칸씩 건너뛰며 옮깁니다. 이 건너뛰는 칸 수를 **보폭(stride)** 이라고 해요. 4장 4.5절의 stride와 같은 낱말인데, 여기서는 "돋보기의 걸음 폭"입니다.

```
import torch
import torch.nn.functional as F
from torch import nn


def 출력_크기(n: int, k: int, p: int = 0, s: int = 1) -> int:
    """입력 n, 필터 k, 패딩 p, 보폭 s일 때 합성곱 출력의 한 변 길이."""
    return (n + 2 * p - k) // s + 1


# 패딩: 테두리에 0을 한 칸 두르기
작은그림 = torch.ones(1, 1, 3, 3)
print(F.pad(작은그림, (1, 1, 1, 1))[0, 0])          # (왼, 오른, 위, 아래)로 한 칸씩

x = torch.zeros(1, 1, 28, 28)                        # 옷 사진 한 장 크기
설정들 = [dict(kernel_size=3), dict(kernel_size=3, padding=1), dict(kernel_size=5),
          dict(kernel_size=5, padding=2), dict(kernel_size=3, stride=2, padding=1),
          dict(kernel_size=2, stride=2), dict(kernel_size=3, padding="same")]
for 설정 in 설정들:
    실제 = nn.Conv2d(1, 1, **설정)(x).shape[-1]
```

```
    k, s = 설정["kernel_size"], 설정.get("stride", 1)
    p = (k - 1) // 2 if 설정.get("padding") == "same" else 설정.get("padding", 0)
    print(f"{str(설정):45} → {실제}×{실제}  (공식: {출력_크기(28, k, p, s)})")
```

```
tensor([[0., 0., 0., 0., 0.],
        [0., 1., 1., 1., 0.],
        [0., 1., 1., 1., 0.],
        [0., 1., 1., 1., 0.],
        [0., 0., 0., 0., 0.]])
{'kernel_size': 3}                          → 26×26  (공식: 26)
{'kernel_size': 3, 'padding': 1}            → 28×28  (공식: 28)
{'kernel_size': 5}                          → 24×24  (공식: 24)
{'kernel_size': 5, 'padding': 2}            → 28×28  (공식: 28)
{'kernel_size': 3, 'stride': 2, 'padding': 1} → 14×14  (공식: 14)
{'kernel_size': 2, 'stride': 2}             → 14×14  (공식: 14)
{'kernel_size': 3, 'padding': 'same'}       → 28×28  (공식: 28)
```

첫 출력은 3 × 3짜리 1 묶음에 `F.pad`로 0을 한 칸 두른 모습입니다. 그 아래는 28 × 28 옷 사진 한 장에 여러 설정의 합성곱을 적용한 결과예요. 출력의 한 변 길이는 언제나 이 공식으로 계산됩니다.

출력 = (입력 + 2 × 패딩 - 필터) ÷ 보폭 (몫만) + 1

여러 설정의 실제 결과가 모두 공식과 같았습니다. 실무에서 자주 쓰는 조합은 세 가지예요.

설정	28 × 28 입력	언제 쓰나
`kernel_size=3, padding=1`	28 × 28 그대로	크기를 유지하며 무늬 찾기. 가장 흔한 조합
`kernel_size=3, stride=2, padding=1`	14 × 14 (절반)	무늬를 찾으면서 크기를 절반으로 줄이기
`kernel_size=5, padding=2`	28 × 28 그대로	넓은 돋보기. 요즘은 3 × 3 두 번으로 대신하는 경우가 많다 (9.8절)

"크기 유지"는 홀수 필터 k에 패딩 (k - 1) / 2를 주면 됩니다. 계산하기 귀찮으면 `padding="same"`이

라고 쓸 수도 있어요. 보폭이 1일 때만 쓸 수 있다는 점은 기억해 두세요.

9.3 채널: 겹겹이 쌓인 그림

필터도 채널만큼 두껍다

지금까지는 흑백 그림이었습니다. 컬러 사진은 빨강·초록·파랑 세 장이 겹쳐 있죠(4장 4.2절). 이렇게 겹친 장 하나하나를 **채널(channel)** 이라고 합니다. 9.1절의 특징 맵 세 장도 다음 층 입장에서는 채널 세 개짜리 그림이에요. 입력에 채널이 여러 개면 필터는 어떻게 생겨야 할까요?

```
import torch
from torch import nn

torch.manual_seed(0)
컬러 = torch.rand(1, 3, 5, 5)                    # 컬러 사진 1장: 빨강·초록·파랑 3채널, 5×5
층 = nn.Conv2d(in_channels=3, out_channels=8, kernel_size=3)
print("필터 묶음:", 층.weight.shape, " 편향:", 층.bias.shape)
print("파라미터:", sum(p.numel() for p in 층.parameters()), "=", 8 * 3 * 3 * 3 + 8)

출력 = 층(컬러)
print("출력:", 출력.shape)

# 출력 0번 채널의 왼쪽 위 칸을 손으로: 세 채널의 3×3을 각자 곱해 모두 더하고 편향
조각 = 컬러[0, :, 0:3, 0:3]                      # (3, 3, 3): 세 채널의 왼쪽 위 3×3
손으로 = (조각 * 층.weight[0]).sum() + 층.bias[0]
print(f"손으로 {손으로.item():.6f}, 층이 낸 값 {출력[0, 0, 0, 0].item():.6f}")
```

```
필터 묶음: torch.Size([8, 3, 3, 3])  편향: torch.Size([8])
파라미터: 224 = 224
출력: torch.Size([1, 8, 3, 3])
손으로 0.111630, 층이 낸 값 0.111630
```

`nn.Conv2d(3, 8, kernel_size=3)`의 필터 묶음은 `(8, 3, 3, 3)`입니다. "필터 8개, 필터 하나는 3채널 × 3 × 3"이라고 읽어요. 필터 하나가 입력 채널 수만큼 **두께**를 갖고 있어서, 돋보기를 한 자리에 대면 세 채널의 3 × 3 조각을 각자의 필터 칸과 곱한 뒤 **27개를 모두 더하고** 편향을 더해 숫자 하나를 냅니다. 손으로 계산한 값과 층이 낸 값이 같죠. 그래서 출력 채널 0번 지도는 "빨강·초록·파랑

을 모두 보고 판단한 무늬 0번의 지도"가 됩니다.

파라미터 수는 **출력 채널 × 입력 채널 × 필터 세로 × 필터 가로 + 출력 채널(편향)** 이에요. 8 × 3 × 3 × 3 + 8 = 224개입니다. 입력이 5 × 5든 1000 × 1000이든 이 숫자는 그대로라는 점이 들어가며 의 표에서 본 차이를 만듭니다.

같은 돋보기를 어디서나: 파라미터 공유

합성곱층과 완전 연결층으로 "28 × 28 사진 → 28 × 28 지도 32장"이라는 같은 모양의 출력을 만들면 파라미터가 얼마나 다를까요? 그리고 필터가 1 × 1이면 어떤 일이 생길까요?

```
import torch
from torch import nn

# 28×28 사진 → 28×28 지도 32장을 만드는 두 방법
합성곱 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
완전연결 = nn.Linear(28 * 28, 32 * 28 * 28)
for 이름, 층 in [("합성곱", 합성곱), ("완전 연결", 완전연결)]:
    print(f"{이름:6} 파라미터 {sum(p.numel() for p in 층.parameters()):>12,}개")

# 1×1 합성곱: 자리마다 채널만 섞는 작은 완전 연결층
torch.manual_seed(0)
x = torch.randn(2, 64, 7, 7)                    # 2장, 채널 64, 7×7
일대일 = nn.Conv2d(64, 16, kernel_size=1)
print("1×1 합성곱:", x.shape, "→", 일대일(x).shape, "파라미터", sum(p.numel() for p in 일대
↪  일.parameters()))

선형 = nn.Linear(64, 16)
with torch.no_grad():
    선형.weight.copy_(일대일.weight.reshape(16, 64))
    선형.bias.copy_(일대일.bias)
자리마다 = 선형(x.permute(0, 2, 3, 1)).permute(0, 3, 1, 2)   # 채널을 맨 뒤로 보내 Linear, 다시 제
↪  자리
print("자리마다 Linear와 같은가?", torch.allclose(일대일(x), 자리마다, atol=1e-6))
```

```
합성곱     파라미터          320개
완전 연결  파라미터   19,694,080개
1×1 합성곱: torch.Size([2, 64, 7, 7]) → torch.Size([2, 16, 7, 7]) 파라미터 1040
자리마다 Linear와 같은가? True
```

출력 모양은 같은데 합성곱은 320개, 완전 연결은 **1,969만 개**입니다. 6만 배가 넘는 차이예요. 완전연결층은 출력 칸 25,088개 하나하나가 입력 784칸 전부와 따로 연결되지만, 합성곱층은 **같은 필터를 모든 자리에서 돌려 씁니다.** 이를 **파라미터 공유(parameter sharing)** 라고 해요. "세로선은 사진 왼쪽에 있든 오른쪽에 있든 세로선"이라는 사진의 성질을 구조에 미리 넣어 둔 셈입니다. 덕분에 배울 숫자가 적어서 데이터가 적어도 잘 배우고, 과적합(8장)도 덜 합니다. 그 대신 합성곱층 하나는 3 × 3 근처밖에 보지 못한다는 제약이 생기는데, 이 문제는 층을 쌓아서 풀어요(9.8절).

뒷부분은 **1 × 1 합성곱**입니다. 돋보기가 한 칸짜리라 주변을 보지는 못하고, **한 자리에서 채널끼리만 섞어요.** 채널 64개를 16개로 줄이는 데 파라미터가 64 × 16 + 16 = 1,040개 들었습니다. 같은 가중치를 넣은 `nn.Linear(64, 16)`을 자리마다(7 × 7 = 49곳) 따로 적용한 결과와 똑같다는 것도 확인했어요. 채널을 맨 뒤로 보내려고 4장 4.5절의 `permute`를 썼습니다. 1 × 1 합성곱은 채널 수를 늘리거나 줄일 때 싸게 쓰는 도구라서, 9.8절의 ResNet에서 다시 만납니다.

9.4 풀링: 줄여서 요약하기

합성곱으로 무늬 지도를 만들었으면, 이제 지도를 **요약**할 차례입니다. 동네 지도에서 골목 하나하나보다 "이 구역에 편의점이 있다"가 더 쓸모 있을 때가 많죠. 무늬가 정확히 어느 픽셀에 있는지보다 "이 근처에 있다"가 중요하다면, 지도를 2 × 2 구역으로 나눠 구역마다 숫자 하나만 남겨도 됩니다. 이것이 **풀링(pooling)** 이에요.

```
import torch
from torch import nn

지도 = torch.tensor([[[[1., 3., 0., 2.],
                       [4., 2., 1., 0.],
                       [0., 1., 5., 6.],
                       [2., 0., 7., 1.]]]])       # (1, 1, 4, 4) 특징 맵
print("최대 풀링:", nn.MaxPool2d(2)(지도)[0, 0].tolist())
print("평균 풀링:", nn.AvgPool2d(2)(지도)[0, 0].tolist())
```

```
# 반응 위치가 한 칸 움직여도 같은 칸 안이면 최대 풀링 결과는 그대로
가 = torch.zeros(1, 1, 4, 4); 가[0, 0, 0, 0] = 9.0
나 = torch.zeros(1, 1, 4, 4); 나[0, 0, 1, 1] = 9.0
print("한 칸 움직여도 같은가?", torch.equal(nn.MaxPool2d(2)(가), nn.MaxPool2d(2)(나)))

# 전역 평균 풀링: 크기와 상관없이 채널마다 평균 하나만 남긴다
torch.manual_seed(0)
특징맵 = torch.randn(8, 128, 7, 7)              # 8장, 채널 128, 7×7
전역 = nn.AdaptiveAvgPool2d(1)(특징맵)
print(특징맵.shape, "→", 전역.shape)
print("채널 평균과 같은가?", torch.allclose(전역.flatten(1), 특징맵.mean(dim=(2, 3))))
print("14×14를 넣어도:", nn.AdaptiveAvgPool2d(1)(torch.randn(8, 128, 14, 14)).shape)
```

```
최대 풀링: [[4.0, 2.0], [2.0, 7.0]]
평균 풀링: [[2.5, 0.75], [0.75, 4.75]]
한 칸 움직여도 같은가? True
torch.Size([8, 128, 7, 7]) → torch.Size([8, 128, 1, 1])
채널 평균과 같은가? True
14×14를 넣어도: torch.Size([8, 128, 1, 1])
```

- **최대 풀링**(`MaxPool2d(2)`)은 2 × 2 구역마다 가장 큰 값을 남깁니다. 왼쪽 위 구역 `[1, 3, 4, 2]`에서 4, 오른쪽 아래 구역 `[5, 6, 7, 1]`에서 7이 남았어요. "이 구역에서 무늬가 가장 강하게 나타난 정도"만 기억하는 거죠. 4 × 4가 2 × 2로 줄었고, 학습할 파라미터는 없습니다.
- **평균 풀링**(`AvgPool2d(2)`)은 구역의 평균을 남깁니다. 같은 왼쪽 위 구역의 평균은 2.5예요.
- 세 번째 줄은 최대 풀링의 장점입니다. 값 9가 (0, 0)에 있든 한 칸 대각선으로 옮겨 (1, 1)에 있든, 같은 2 × 2 구역 안이면 결과가 똑같아요. 무늬가 **조금** 움직여도 요약은 흔들리지 않습니다.

마지막 부분은 **전역 평균 풀링**(global average pooling) 입니다. `nn.AdaptiveAvgPool2d(1)`은 "출력을 1 × 1로 만들어라"라는 뜻이라서, 지도 크기가 7 × 7이든 14 × 14든 채널마다 **평균 하나만** 남깁니다. 결과가 `x.mean(dim=(2, 3))`과 같다는 것도 확인했어요. 지도 128장을 숫자 128개로 압축하면서 "어디에 있었나"는 완전히 잊고 "얼마나 있었나"만 남기는 셈입니다. 이 성질이 9.6절의 밀기 문제에서 중요한 역할을 해요.

9.5 첫 CNN으로 옷 사진 분류하기

모양을 따라가 보기

부품이 다 모였습니다. 합성곱으로 무늬를 찾고, ReLU로 구부리고(6장 6.2절), 풀링으로 줄이는 묶음을 두 번 반복한 다음, 6장처럼 완전 연결층으로 열 가지 옷 종류의 점수를 내 보겠습니다. 학습하기 전에 사진 64장 묶음이 층을 지날 때마다 모양이 어떻게 바뀌는지부터 따라가요. 모양을 모르고 CNN을 짜면 마지막 `nn.Linear`의 입력 크기를 틀리기 쉽거든요.

```
import torch
from torch import nn

모델 = nn.Sequential(
    nn.Conv2d(1, 32, kernel_size=3, padding=1), nn.ReLU(),   # 무늬 찾기 32가지
    nn.MaxPool2d(2),                                          # 28×28 → 14×14
    nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(),  # 무늬의 조합 64가지
    nn.MaxPool2d(2),                                          # 14×14 → 7×7
    nn.Flatten(),                                             # 64×7×7 = 3,136개를 한 줄로
    nn.Linear(64 * 7 * 7, 128), nn.ReLU(),
    nn.Linear(128, 10),
)

x = torch.zeros(64, 1, 28, 28)                               # 옷 사진 64장 묶음
print(f"{'입력':10} {str(tuple(x.shape)):17}")
for 층 in 모델:
    x = 층(x)
    개수 = sum(p.numel() for p in 층.parameters())
    print(f"{type(층).__name__:10} {str(tuple(x.shape)):17} {개수:>9,}")
print(f"전체 파라미터 {sum(p.numel() for p in 모델.parameters()):,}개")
```

```
입력         (64, 1, 28, 28)
Conv2d     (64, 32, 28, 28)        320
ReLU       (64, 32, 28, 28)          0
MaxPool2d  (64, 32, 14, 14)          0
Conv2d     (64, 64, 14, 14)     18,496
ReLU       (64, 64, 14, 14)          0
```

```
MaxPool2d  (64, 64, 7, 7)            0
Flatten    (64, 3136)                0
Linear     (64, 128)           401,536
ReLU       (64, 128)                 0
Linear     (64, 10)              1,290
전체 파라미터 421,642개
```

모양을 자리마다 이름 붙여 읽어 보세요(4장 4.2절). `(64, 32, 28, 28)`은 "64장, 무늬 지도 32장, 28 × 28"입니다. 합성곱은 `padding=1`이라 크기를 유지하면서 채널만 1 → 32 → 64로 늘리고, 최대 풀링이 28 → 14 → 7로 절반씩 줄여요. **크기는 줄이고 채널은 늘리는 것**이 CNN의 전형적인 모양입니다. 위치 정보는 조금씩 요약하고, 대신 찾는 무늬의 종류를 늘려 가는 거예요. 마지막에 `Flatten`이 64 × 7 × 7 = 3,136개를 한 줄로 펴서(4장 4.5절에서 예고한 그 `flatten`) 완전 연결층에 넘깁니다.

파라미터 수에서 재미있는 점이 보입니다. 합성곱층 둘은 합쳐서 2만 개도 안 되는데(320 + 18,496), 뒤쪽 `Linear(3136, 128)` 하나가 **40만 개**로 전체 421,642개의 95%를 차지해요. 6장의 MLP(235,146개)보다 전체 파라미터는 오히려 많습니다. 무늬를 찾는 일은 싸고, 펴서 연결하는 일이 비싼 거죠. 이 비싼 부분은 9.6절과 9.10절에서 손봅니다.

학습시키기

학습 코드는 6장 6.11절과 **모델만 다릅니다.** 6장 들어가며에서 "모델 생김새는 제각각이어도 학습시키는 코드의 뼈대는 거의 한 글자도 다르지 않다"고 했던 것을 직접 확인해 보세요. 이 장의 뒤쪽 예제들도 같은 모델과 함수를 계속 쓰니, 8장의 `학습도구.py`처럼 한 파일(모듈)에 모아 두고 `import`하겠습니다. 아래 내용을 `옷도구.py`로 저장하세요.

```python
"""9장 예제에서 함께 쓰는 장치 선택·모델·학습·시험 도구."""
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
```

```
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")


class 의류분류기(nn.Module):
    """6장 6.11절의 다층 퍼셉트론 설계도 (fashion_mlp.pt를 불러올 때 쓴다)."""

    def __init__(self) -> None:
        super().__init__()
        self.펴기 = nn.Flatten()
        self.층들 = nn.Sequential(
            nn.Linear(28 * 28, 256), nn.ReLU(),
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, 10),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.층들(self.펴기(x))


class 옷CNN(nn.Module):
    """합성곱 두 단으로 무늬를 찾고, 완전 연결층 두 개로 옷 종류를 고르는 작은 CNN."""

    def __init__(self) -> None:
        super().__init__()
        self.특징 = nn.Sequential(                      # 그림 모양 그대로 무늬 찾기
            nn.Conv2d(1, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        )
        self.분류 = nn.Sequential(                      # 찾은 무늬로 종류 고르기
            nn.Flatten(), nn.Linear(64 * 7 * 7, 128), nn.ReLU(), nn.Linear(128, 10),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.분류(self.특징(x))
```

```
def 한_에폭_학습(모델: nn.Module, 로더: DataLoader, 손실함수: nn.Module,
                옵티마이저: torch.optim.Optimizer, 장치: torch.device) -> float:
    """6장 6.11절과 같은 함수: 한 바퀴 학습하고 평균 손실을 돌려준다."""
    모델.train()
    손실_합, 개수 = 0.0, 0
    for x, y in 로더:
        x, y = x.to(장치), y.to(장치)
        옵티마이저.zero_grad()
        손실 = 손실함수(모델(x), y)
        손실.backward()
        옵티마이저.step()
        손실_합 += 손실.item() * len(y)
        개수 += len(y)
    return 손실_합 / 개수


def 평가하기(모델: nn.Module, 로더: DataLoader, 손실함수: nn.Module,
            장치: torch.device) -> tuple[float, float]:
    """6장 6.11절과 같은 함수: 평균 손실과 정확도를 잰다."""
    모델.eval()
    손실_합, 맞힘, 개수 = 0.0, 0, 0
    with torch.no_grad():
        for x, y in 로더:
            x, y = x.to(장치), y.to(장치)
            출력 = 모델(x)
            손실_합 += 손실함수(출력, y).item() * len(y)
            맞힘 += (출력.argmax(dim=1) == y).sum().item()
            개수 += len(y)
    return 손실_합 / 개수, 맞힘 / 개수


def 시험_데이터() -> tuple[torch.Tensor, torch.Tensor]:
    """FashionMNIST 시험 사진 1만 장을 (10000, 1, 28, 28) 실수 텐서와 정답으로."""
    시험셋 = datasets.FashionMNIST("data", train=False, download=True)
```

```
    return 시험셋.data.float().unsqueeze(1) / 255, 시험셋.targets


def 오른쪽으로_밀기(x: torch.Tensor, 칸: int) -> torch.Tensor:
    """사진 묶음을 오른쪽으로 칸만큼 옮기고 빈자리는 0으로 채운다 (1장 1.11절과 같은 일)."""
    밀린 = torch.zeros_like(x)
    밀린[..., 칸:] = x[..., :x.shape[-1] - 칸]
    return 밀린
```

한_에폭_학습과 **평가하기**는 6장 6.11절의 함수 그대로이고, **의류분류기**는 6장에서 저장한 **fashion_mlp.pt**를 다시 불러올 때 쓸 6장의 설계도입니다. 새로 만든 **옷CNN**은 앞에서 모양을 따라가 본 CNN과 같은 층을, 무늬를 찾는 **특징** 부분과 종류를 고르는 **분류** 부분으로 나눠 담았어요. torchvision의 유명한 모델들도 이렇게 나눠 두는 경우가 많아서, 10장에서 남이 만든 모델을 뜯어볼 때 이 구분이 도움이 됩니다. 나머지 두 함수는 시험 사진 1만 장을 한 번에 텐서로 꺼내는 **시험_데이터**(6장 6.11절에서 틀린 사진을 볼 때 쓴 방법)와, 9.6절에서 쓸 **오른쪽으로_밀기**예요. 이제 학습시켜 보겠습니다.

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

from 옷도구 import 옷CNN, 평가하기, 한_에폭_학습

torch.manual_seed(0)
변환 = transforms.ToTensor()
학습셋 = datasets.FashionMNIST("data", train=True, download=True, transform=변환)
시험셋 = datasets.FashionMNIST("data", train=False, download=True, transform=변환)
학습_로더 = DataLoader(학습셋, batch_size=64, shuffle=True)
시험_로더 = DataLoader(시험셋, batch_size=256)

장치 = torch.device("cpu")      # 6장처럼 재현을 위해 CPU. 장치_고르기()로 바꾸면 더 빠르다
모델 = 옷CNN().to(장치)
손실함수 = nn.CrossEntropyLoss()
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
print(f"파라미터 {sum(p.numel() for p in 모델.parameters()):,}개")
```

```
for 에폭 in range(1, 6):
    학습_손실 = 한_에폭_학습(모델, 학습_로더, 손실함수, 옵티마이저, 장치)
    시험_손실, 정확도 = 평가하기(모델, 시험_로더, 손실함수, 장치)
    print(f"에폭 {에폭}: 학습 손실 {학습_손실:.4f}, 시험 손실 {시험_손실:.4f}, 시험 정확도 {정확
    ↪ 도:.2%}")

torch.save(모델.state_dict(), "fashion_cnn.pt")
print("저장했습니다: fashion_cnn.pt")
```

```
파라미터 421,642개
에폭 1: 학습 손실 0.4494, 시험 손실 0.3506, 시험 정확도 87.50%
에폭 2: 학습 손실 0.2841, 시험 손실 0.3010, 시험 정확도 89.09%
에폭 3: 학습 손실 0.2409, 시험 손실 0.2618, 시험 정확도 90.54%
에폭 4: 학습 손실 0.2097, 시험 손실 0.2536, 시험 정확도 90.39%
에폭 5: 학습 손실 0.1853, 시험 손실 0.2479, 시험 정확도 91.01%
저장했습니다: fashion_cnn.pt
```

5에폭 만에 시험 정확도 **91.01%** 가 나왔습니다. 6장의 MLP는 10에폭을 학습하고 88.58%였으니, 절반의 에폭으로 2.4%p를 넘어섰어요. 첫 에폭부터 87.50%로, MLP가 4에폭째에 낸 87.84%에 벌써 가깝습니다. 6장처럼 결과를 재현하려고 씨앗을 고정하고 CPU에서 돌렸어요. 변환도 7장 7.6절의 v2 대신 6장의 `transforms.ToTensor()`를 그대로 썼습니다. MLP와 똑같은 전처리로 공정하게 비교하려는 것인데, v2(`v2.ToImage()`와 `v2.ToDtype(torch.float32, scale=True)`)로 바꿔 돌려봐도 시험 정확도가 91.2% 근처로 소수점 아래만 달라져요. 합성곱은 계산량이 많아서 MLP보다 한 에폭이 몇 배 오래 걸리니, 기다리기 지루하면 `장치_고르기()`로 GPU를 쓰세요(그 경우 소수점 아래 숫자는 조금 달라질 수 있습니다).

학습 손실과 시험 손실의 간격도 보세요. 5에폭째에 학습 손실 0.1853, 시험 손실 0.2479입니다. 학습 손실이 시험 손실보다 계속 빠르게 줄고 있으니 더 오래 학습하면 8장 8.1절의 과적합 신호가 나타날 거예요. 이 모델에는 아직 8장의 대책이 하나도 들어 있지 않습니다.

MLP와 무엇이 달라졌나

전체 정확도 2.4%p는 어디서 왔을까요? 6장 6.11절에서 저장한 `fashion_mlp.pt`와 방금 저장한 `fashion_cnn.pt`를 함께 불러와 클래스별로 비교해 보겠습니다. 6장 실습을 같은 폴더에서 따라 했다면 `fashion_mlp.pt`가 이미 있을 거예요. 없다면 6장 6.11절의 학습 코드를 먼저 실행해 만들어 두

세요.

```
import torch

from 옷도구 import 옷CNN, 시험_데이터, 의류분류기

이름 = ["T셔츠", "바지", "풀오버", "드레스", "코트", "샌들", "셔츠", "운동화", "가방", "앵클부츠"]
X, y = 시험_데이터()

예측 = {}
for 별명, 모델, 파일 in [("MLP", 의류분류기(), "fashion_mlp.pt"), ("CNN", 옷CNN(),
↪  "fashion_cnn.pt")]:
    모델.load_state_dict(torch.load(파일, weights_only=True))
    모델.eval()
    with torch.no_grad():
        예측[별명] = 모델(X).argmax(dim=1)
    print(f"{별명} 전체 정확도 {(예측[별명] == y).float().mean():.2%}")

print("클래스     MLP     CNN    차이")
for k in range(10):
    a, b = [(예측[m][y == k] == k).float().mean().item() for m in ("MLP", "CNN")]
    print(f"{이름[k]:6} {a:6.1%}  {b:6.1%}  {(b - a) * 100:+5.1f}%p")

for m in ("MLP", "CNN"):                          # 모델마다 가장 많이 헷갈린 짝 3개
    혼동 = torch.zeros(10, 10, dtype=torch.int64)
    혼동.index_put_((y, 예측[m]), torch.ones(len(y), dtype=torch.int64), accumulate=True)
    혼동.fill_diagonal_(0)
    짝들 = [divmod(i, 10) for i in 혼동.flatten().topk(3).indices.tolist()]
    print(m, "헷갈림:", ", ".join(f"{이름[a]}→{이름[b]} {혼동[a, b]}번" for a, b in 짝들))
```

```
MLP 전체 정확도 88.58%
CNN 전체 정확도 91.01%
클래스     MLP     CNN    차이
T셔츠     84.0%   83.7%   -0.3%p
바지      95.9%   99.4%   +3.5%p
풀오버     72.9%   89.1%  +16.2%p
드레스     87.7%   87.7%   +0.0%p
```

```
코트      87.3%   81.9%   -5.4%p
샌들      96.6%   98.2%   +1.6%p
셔츠      74.1%   77.9%   +3.8%p
운동화     97.4%   98.6%   +1.2%p
가방      96.3%   97.9%   +1.6%p
앵클부츠    93.6%   95.7%   +2.1%p
MLP 헷갈림: 풀오버→코트 172번, T셔츠→셔츠 116번, 셔츠→T셔츠 85번
CNN 헷갈림: T셔츠→셔츠 129번, 셔츠→T셔츠 93번, 코트→풀오버 80번
```

두 모델의 전체 정확도가 학습 직후와 같은 88.58%, 91.01%이니 불러오기는 제대로 됐습니다. 혼동 행렬은 6장에서는 반복문으로 칸마다 1씩 더해 만들었는데, 여기서는 `index_put_(..., accumulate=True)`로 (정답, 예측) 자리마다 1을 한꺼번에 더했어요. 4장 4.4절 `scatter_add_`의 2차원판이라고 보면 됩니다.

가장 크게 좋아진 것은 **풀오버(72.9% → 89.1%)** 입니다. MLP가 가장 많이 헷갈렸던 "풀오버를 코트로" 실수가 헷갈림 순위에서 사라졌어요. 바지(+3.5%p)와 셔츠(+3.8%p)도 좋아졌습니다. 다만 모든 클래스가 좋아진 것은 아니에요. **코트는 87.3%에서 81.9%로 오히려 떨어졌고**, CNN의 헷갈림 3위에 "코트를 풀오버로" 80번이 새로 올라왔습니다. 풀오버와 코트의 경계를 MLP와는 다른 곳에 그은 거죠. T셔츠와 셔츠 사이의 혼동은 두 모델 모두 비슷하게 많아서, CNN도 아직 풀지 못한 숙제로 남았습니다. 28 × 28 흑백 사진에서 칼라나 단추 같은 작은 차이는 사람 눈으로도 애매해요. 이 숙제는 9.10절 실습에서 다시 봅니다.

9.6 그림을 밀어도 알아볼까요?

1장의 약속: 밀어서 시험하기

이제 1장 1.11절에서 미뤄 둔 질문으로 돌아가겠습니다. 시험 사진을 오른쪽으로 0~4칸 밀어서 두 모델에 넣어 보면 어떨까요? 사람 눈에는 모두 같은 옷입니다. 미는 일은 `옷도구.py`의 `오른쪽으로_밀기`가 합니다. 1장에서 NumPy로 짰던 함수를 텐서 묶음용으로 옮긴 것으로, 빈자리는 배경과 같은 0으로 채워요.

```
import torch

from 옷도구 import 옷CNN, 시험_데이터, 오른쪽으로_밀기, 의류분류기
```

```
X, y = 시험_데이터()
print("밀기(칸)" + "".join(f"{칸:>8}" for 칸 in range(5)))
for 별명, 모델, 파일 in [("MLP", 의류분류기(), "fashion_mlp.pt"), ("CNN", 옷CNN(),
↪ "fashion_cnn.pt")]:
    모델.load_state_dict(torch.load(파일, weights_only=True))
    모델.eval()
    with torch.no_grad():
        정확도 = [(모델(오른쪽으로_밀기(X, 칸)).argmax(dim=1) == y).float().mean().item() for 칸 in
↪ range(5)]
    print(f"{별명:7}" + "".join(f"{a:>8.1%}" for a in 정확도))
```

```
밀기(칸)       0       1       2       3       4
MLP       88.6%   80.6%   62.1%   40.8%   25.2%
CNN       91.0%   88.9%   80.0%   58.2%   36.0%
```

MLP는 한 칸에 88.6% → 80.6%, 두 칸에 62.1%, 네 칸이면 25.2%까지 떨어졌습니다. 1장의 최근접 이웃과 같은 모습이에요. CNN은 한 칸에 88.9%, 두 칸에도 80.0%로 훨씬 잘 버텼습니다. 그런데 정직하게 보면 CNN도 세 칸부터는 58.2%, 36.0%로 크게 무너져요. "CNN은 위치가 달라도 알아본다"는 말은 반만 맞는 셈입니다. 왜 그런지 다음 실험으로 따져 보겠습니다.

밀린 만큼 밀리는 합성곱, 자리에 묶인 완전 연결층

```
import torch
from torch import nn

from 옷도구 import 오른쪽으로_밀기

torch.manual_seed(0)
x = torch.zeros(1, 1, 12, 12)
x[0, 0, 3:8, 2:6] = torch.rand(5, 4)            # 가운데 근처에 무늬 하나
합성곱 = nn.Conv2d(1, 4, kernel_size=3, padding=1)

# 1) 밀고 나서 합성곱 vs 합성곱하고 나서 밀기
먼저_밀기 = 합성곱(오른쪽으로_밀기(x, 2))
나중에_밀기 = 오른쪽으로_밀기(합성곱(x), 2)
```

```
안쪽 = (slice(None), slice(None), slice(None), slice(2, None))   # 밀려서 새로 생긴 왼쪽 두 줄은 빼
↪ 고
print("합성곱은 밀린 만큼 결과도 밀린다:", torch.allclose(먼저_밀기[안쪽], 나중에_밀기[안쪽],
↪ atol=1e-6))

# 2) 그런데 한 줄로 펴면 같은 무늬가 전혀 다른 칸에 들어간다
펴기 = nn.Flatten()
원래, 밀린 = 펴기(합성곱(x)), 펴기(합성곱(오른쪽으로_밀기(x, 2)))
print(f"한 줄로 편 뒤 같은 칸끼리의 차이 {(원래 - 밀린).abs().mean():.4f}, 값 자체의 크기 {원
↪ 래.abs().mean():.4f}")

# 3) 전역 평균 풀링은 자리를 잊고 채널마다 '얼마나 있나'만 남긴다
전역 = nn.AdaptiveAvgPool2d(1)
a, b = 전역(합성곱(x)).flatten(), 전역(합성곱(오른쪽으로_밀기(x, 2))).flatten()
print("전역 평균 풀링 뒤:", [round(v, 4) for v in a.tolist()])
print("밀고 나서      :", [round(v, 4) for v in b.tolist()])
```

```
합성곱은 밀린 만큼 결과도 밀린다: True
한 줄로 편 뒤 같은 칸끼리의 차이 0.0649, 값 자체의 크기 0.2040
전역 평균 풀링 뒤: [0.0826, -0.3031, -0.249, -0.1254]
밀고 나서      : [0.0826, -0.3031, -0.249, -0.1254]
```

세 가지를 확인했습니다.

1. **합성곱은 밀린 만큼 결과도 밀립니다.** 그림을 두 칸 밀고 합성곱한 결과와, 합성곱한 다음 결과를 두 칸 민 것이 같아요. 같은 돋보기를 모든 자리에서 돌려 쓰니 당연한 일이죠. 이런 성질을 **이동 등변성**(translation equivariance) 이라고 합니다. 무늬를 **찾는** 단계는 위치와 상관없이 공평하다는 뜻이에요.
2. **그런데 한 줄로 펴면 이야기가 달라집니다.** 같은 무늬 지도라도 두 칸 밀리면 `Flatten` 뒤에는 전혀 다른 칸에 숫자가 들어갑니다. 같은 칸끼리 비교한 차이가 0.0649로, 값 자체의 평균 크기(0.2040)의 3분의 1쯤 되네요. 그림은 같은데 다음 층이 받는 입력은 크게 달라진 거예요. 9.5절 CNN의 마지막 `Linear(3136, 128)`은 6장의 MLP처럼 "몇 번째 칸에 무엇이 있나"를 배웠기 때문에, 그 단계에서 다시 자리에 묶입니다. 최대 풀링 두 번이 한두 칸의 움직임은 흡수해 줬지만, 세 칸 이상은 감당하지 못한 거예요.
3. **전역 평균 풀링은 자리를 잊습니다.** 채널마다 지도 전체의 평균만 남기니 밀기 전과 후의 값이

완전히 같아요. 9.4절에서 본 대로 “어디에”를 버리고 “얼마나”만 남기기 때문입니다. 이렇게 입력을 밀어도 출력이 바뀌지 않는 성질을 **이동 불변성(translation invariance)** 이라고 해요.

정리하면, 합성곱은 무늬를 위치와 상관없이 찾아 주고(등변성), 그 결과를 전역 평균 풀링으로 요약하면 위치를 잊은 판단(불변성)을 할 수 있습니다. 그래서 요즘 CNN은 대부분 마지막 `Flatten` + 큰 `Linear` 대신 **전역 평균 풀링 + 작은 `Linear`** 를 써요. 9.5절에서 전체의 95%를 차지하던 비싼 `Linear`가 사라지는 덤도 있습니다.

Q

Q. 그럼 전역 평균 풀링만 쓰면 밀기 문제는 다 해결되나요? 완전히는 아닙니다. 그림 가장자리의 패딩이나 보폭 2인 층에서는 한 칸 차이로 계산 결과가 조금 달라지고, 그림이 밀려서 테두리 밖으로 잘려 나간 부분은 되찾을 수 없어요. 그래서 실무에서는 구조와 함께 **데이터 증강**도 씁니다. 7장 7.6절의 `RandomCrop(padding=...)`이 바로 “조금씩 밀린 그림”을 학습 때 보여 주는 방법이에요. 9.10절 실습에서 둘을 함께 써서 밀기에 얼마나 강해지는지 확인합니다.

9.7 CNN 속의 배치 정규화와 드롭아웃

8장에서 익힌 도구들은 CNN에서도 그대로 쓰입니다. 다만 그림 모양의 데이터에 맞게 조금 바뀐 것이 있어요. 8장 8.4절의 배치 정규화는 `nn.BatchNorm1d`로 **특성마다** 평균과 분산을 냈습니다. CNN에서는 `nn.BatchNorm2d`를 쓰는데, 무엇마다 평균을 낼까요?

```
import torch
from torch import nn

torch.manual_seed(0)
x = torch.randn(16, 3, 8, 8) * 5 + 2            # 16장, 채널 3, 8×8. 일부러 눈금을 크게
bn = nn.BatchNorm2d(3)                           # 채널 3개짜리 배치 정규화
print("학습할 숫자:", [tuple(p.shape) for p in bn.parameters()])

bn.train()
출력 = bn(x)
# 채널마다 (장 수, 세로, 가로) 전체에서 평균·분산을 낸다
평균 = x.mean(dim=(0, 2, 3), keepdim=True)
분산 = x.var(dim=(0, 2, 3), keepdim=True, unbiased=False)
```

```
print("손으로 계산과 같은가?", torch.allclose(출력, (x - 평균) / torch.sqrt(분산 + bn.eps),
↪ atol=1e-5))
print("채널별 평균:", [round(v, 4) + 0.0 for v in 출력.mean(dim=(0, 2, 3)).tolist()])
print("채널별 표준편차:", [round(v, 4) for v in 출력.std(dim=(0, 2, 3), unbiased=False).tolist()])

# 합성곱의 편향은 배치 정규화가 평균을 빼면서 지워 버린다
torch.manual_seed(1)
편향있음 = nn.Conv2d(3, 4, kernel_size=3, padding=1, bias=True)
편향없음 = nn.Conv2d(3, 4, kernel_size=3, padding=1, bias=False)
with torch.no_grad():
    편향없음.weight.copy_(편향있음.weight)
bn4 = nn.BatchNorm2d(4)
print("편향이 있으나 없으나 같은가?", torch.allclose(bn4(편향있음(x)), bn4(편향없음(x)),
↪ atol=1e-5))
```

```
학습할 숫자: [(3,), (3,)]
손으로 계산과 같은가? True
채널별 평균: [0.0, 0.0, 0.0]
채널별 표준편차: [1.0, 1.0, 1.0]
편향이 있으나 없으나 같은가? True
```

`BatchNorm2d(3)`은 **채널마다** 하나씩, 크기를 정하는 숫자 3개와 위치를 정하는 숫자 3개를 학습합니다. 평균과 분산은 채널마다 (장 수, 세로, 가로) 전체, 즉 16장 × 8 × 8 = 1,024개 값으로 내요. 손으로 `mean(dim=(0, 2, 3))`을 계산한 결과와 같고, 정규화한 뒤 채널마다 평균 0, 표준편차 1이 됐습니다. 같은 채널은 "같은 필터로 찾은 같은 무늬"이니 자리와 상관없이 한 눈금으로 맞추는 게 자연스럽죠. 9.3절의 파라미터 공유와 같은 생각입니다.

마지막 확인은 실무에서 자주 보는 `bias=False`의 이유입니다. 합성곱에 편향이 있든 없든, 바로 뒤에 배치 정규화가 오면 결과가 똑같아요. 배치 정규화가 채널 평균을 빼면서 편향(채널마다 더해진 같은 숫자)을 지워 버리기 때문입니다. 어차피 지워질 숫자를 학습할 필요가 없으니, 배치 정규화 앞의 합성곱은 `bias=False`로 만드는 것이 관례예요. 그 역할은 배치 정규화의 위치 숫자가 대신합니다.

CNN에서 이 도구들이 놓이는 자리를 정리하면 이렇습니다.

도구	놓이는 자리	비고
배치 정규화 `BatchNorm2d(채널)`	합성곱 바로 뒤, ReLU 앞: **합성곱 → 배치 정규화 → ReLU**	합성곱은 `bias=False`
드롭아웃 `Dropout(p)`	전역 평균 풀링(또는 `Flatten`) 뒤, 마지막 `Linear` 앞	합성곱 사이에는 잘 쓰지 않는다
데이터 증강	모델 밖, 학습 데이터의 transform(7장 7.6절)	검증·시험 데이터에는 쓰지 않는다
weight decay, 스케줄러, 조기 종료	옵티마이저와 학습 루프(8장)	MLP와 똑같이 쓴다

드롭아웃을 합성곱 사이에 잘 쓰지 않는 이유는, 특징 맵에서 이웃한 칸들은 값이 비슷해서 칸 하나를 꺼도 옆 칸이 같은 정보를 그대로 전해 주기 때문입니다. 칸 단위로 끄는 효과가 약한 거죠. 꼭 쓰고 싶다면 채널을 통째로 끄는 `nn.Dropout2d`가 있지만, 요즘 CNN은 배치 정규화와 데이터 증강만으로 충분한 경우가 많아서 드롭아웃은 마지막 분류층 앞에만 두는 편입니다.

9.8 유명한 구조의 아이디어

이미지 분류는 딥러닝이 가장 먼저 크게 성공한 분야라서, 이름난 구조가 많습니다. 구조 하나하나를 외울 필요는 없지만, 각 구조가 **어떤 문제를 어떤 아이디어로 풀었는지**는 알아 둘 만해요. 오늘날 쓰는 모델들이 그 아이디어를 조합해 만들어졌거든요.

LeNet: 합성곱, 풀링, 완전 연결의 원형

1998년 얀 르쿤(Yann LeCun)이 우편번호 손글씨를 읽으려고 만든 **LeNet-5**는 지금 CNN의 원형입니다. 28 × 28 입력에 맞춰 뼈대를 옮겨 보겠습니다.

```
import torch
from torch import nn

# LeNet-5 (1998)의 뼈대를 28×28 입력에 맞춘 것. 당시처럼 tanh와 평균 풀링을 쓴다
르넷 = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.Tanh(), nn.AvgPool2d(2),   # 28 → 28 → 14
```

```
    nn.Conv2d(6, 16, kernel_size=5), nn.Tanh(), nn.AvgPool2d(2),            # 14 → 10 → 5
    nn.Flatten(),
    nn.Linear(16 * 5 * 5, 120), nn.Tanh(),
    nn.Linear(120, 84), nn.Tanh(),
    nn.Linear(84, 10),
)
x = torch.zeros(1, 1, 28, 28)
for 층 in 르넷:
    x = 층(x)
    if not isinstance(층, (nn.Tanh, nn.Flatten)):
        print(f"{type(층).__name__:10} → {tuple(x.shape)}")
print(f"파라미터 {sum(p.numel() for p in 르넷.parameters()):,}개")
```

```
Conv2d     → (1, 6, 28, 28)
AvgPool2d  → (1, 6, 14, 14)
Conv2d     → (1, 16, 10, 10)
AvgPool2d  → (1, 16, 5, 5)
Linear     → (1, 120)
Linear     → (1, 84)
Linear     → (1, 10)
파라미터 61,706개
```

합성곱 → 풀링 → 합성곱 → 풀링 → 완전 연결층 셋. 9.5절에서 우리가 만든 CNN과 순서가 거의 같죠? 다른 점은 시대의 흔적입니다. 그때는 ReLU가 아니라 tanh를, 최대 풀링이 아니라 평균 풀링을 썼고, 두 번째 합성곱에 패딩이 없어서 14 → 10으로 깎였어요. 파라미터는 61,706개로 우리 CNN보다 훨씬 작습니다. 당시 컴퓨터로는 이 정도가 한계였어요.

점점 깊고 커진 모델들

그 뒤 14년쯤 조용하던 CNN은 2012년 **AlexNet**이 이미지 대회(ImageNet, 사진 120만 장을 1,000가지로 분류)에서 다른 방법들을 큰 차이로 이기면서 딥러닝 시대를 열었습니다. 대표적인 모델의 크기를 torchvision으로 세어 보겠습니다. `weights=None`으로 구조만 만들고 학습된 숫자는 받지 않으니 인터넷 연결도 필요 없어요.

```
from torchvision import models

# 구조만 만든다 (weights=None: 학습된 숫자는 받지 않는다. 받아서 쓰는 일은 10장에서)
for 이름, 만들기 in [("AlexNet (2012)", models.alexnet), ("VGG-16 (2014)", models.vgg16),
                     ("ResNet-18 (2015)", models.resnet18), ("ResNet-50 (2015)", models.resnet50)]:
    모델 = 만들기(weights=None)
    개수 = sum(p.numel() for p in 모델.parameters())
    합성곱수 = sum(1 for m in 모델.modules() if type(m).__name__ == "Conv2d")
    print(f"{이름:17} 파라미터 {개수 / 1e6:6.1f}백만개, 합성곱층 {합성곱수:2}개, float32로 {개수 *
    ↪ 4 / 1024**2:5.0f} MB")
```

```
AlexNet (2012)    파라미터   61.1백만개, 합성곱층  5개, float32로   233 MB
VGG-16 (2014)     파라미터  138.4백만개, 합성곱층 13개, float32로   528 MB
ResNet-18 (2015)  파라미터   11.7백만개, 합성곱층 20개, float32로    45 MB
ResNet-50 (2015)  파라미터   25.6백만개, 합성곱층 53개, float32로    97 MB
```

모델	해	새로 가져온 아이디어
LeNet-5	1998	합성곱 → 풀링 → 완전 연결이라는 기본 틀
AlexNet	2012	ReLU, 드롭아웃, 데이터 증강, GPU로 큰 데이터 학습
VGG-16	2014	3 × 3 필터만 깊게 쌓기
ResNet	2015	잔차 연결로 수십~수백 층 학습하기(전역 평균 풀링은 GoogLeNet(2014)에서 이어받음)

AlexNet이 들고 나온 ReLU, 드롭아웃, 데이터 증강은 이 책의 6~8장에서 이미 다 써 본 도구들이에요. VGG-16은 1억 3,800만 개로 가장 크고 float32로 528MB나 됩니다(4장 4.3절의 계산). 그런데 그보다 합성곱층이 네 배 많은 ResNet-50은 오히려 2,560만 개로 훨씬 작아요. VGG의 파라미터 대부분이 마지막 완전 연결층에 있는데, ResNet은 9.6절의 전역 평균 풀링을 써서 그 부분을 거의 없앴기 때문입니다. 깊이는 깊게, 크기는 작게. 이 차이를 만든 두 아이디어를 차례로 보겠습니다.

VGG: 3 × 3을 쌓는 이유

VGG는 5 × 5나 7 × 7 같은 큰 필터를 쓰지 않고 **3 × 3만 여러 번 쌓았습니다.** 3 × 3 필터는 이웃한 칸씩밖에 못 보는데 괜찮을까요?

```
import torch
from torch import nn

c = 64
큰필터 = nn.Conv2d(c, c, kernel_size=5, padding=2, bias=False)
작은필터_둘 = nn.Sequential(nn.Conv2d(c, c, kernel_size=3, padding=1, bias=False), nn.ReLU(),
                            nn.Conv2d(c, c, kernel_size=3, padding=1, bias=False))
for 이름, 층 in [("5×5 한 번", 큰필터), ("3×3 두 번", 작은필터_둘)]:
    print(f"{이름}: 파라미터 {sum(p.numel() for p in 층.parameters()):,}개")


def 보는_범위(층: nn.Module, 크기: int = 11) -> torch.Tensor:
    """출력 한가운데 칸이 입력의 어느 칸들에 영향을 받는지 기울기로 찾는다 (5장 5.12절)."""
    x = torch.randn(1, 1, 크기, 크기, requires_grad=True)
    출력 = 층(x)
    출력[0, 0, 크기 // 2, 크기 // 2].backward()       # 가운데 칸 하나만 거꾸로 따라가기
    return (x.grad[0, 0] != 0).int()


torch.manual_seed(0)
for 깊이 in (1, 2, 3):
    층 = nn.Sequential(*[nn.Conv2d(1, 1, kernel_size=3, padding=1) for _ in range(깊이)])
    범위 = 보는_범위(층)
    print(f"3×3을 {깊이}번 쌓으면 보는 범위: {범위.sum(dim=0).max().item()}×{범
    ↪ 위.sum(dim=1).max().item()}")
print(보는_범위(nn.Sequential(nn.Conv2d(1, 1, 3, padding=1), nn.Conv2d(1, 1, 3, padding=1))))
```

```
5×5 한 번: 파라미터 102,400개
3×3 두 번: 파라미터 73,728개
3×3을 1번 쌓으면 보는 범위: 3×3
3×3을 2번 쌓으면 보는 범위: 5×5
3×3을 3번 쌓으면 보는 범위: 7×7
```

```
tensor([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0],
        [0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0],
        [0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0],
        [0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0],
        [0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]], dtype=torch.int32)
```

채널 64개 기준으로 5 × 5 한 번은 102,400개, 3 × 3 두 번은 73,728개로 3 × 3 두 번이 28% 적습니다. 그런데 보는 범위는 어떨까요? 출력의 가운데 칸 하나에서 `backward()`로 거꾸로 따라가, 입력의 어느 칸에 기울기가 닿는지 찾아봤어요(5장 5.12절의 입력 기울기). 3 × 3 한 번은 3 × 3, 두 번 쌓으면 **5 × 5**, 세 번이면 7 × 7을 봅니다. 마지막 출력의 1이 가운데 5 × 5에 모여 있죠. 첫 층이 이웃 한 칸을 보고, 둘째 층은 "이웃의 이웃"까지 보게 되는 거예요. 출력 한 칸이 보는 입력의 범위를 **수용 영역(receptive field)** 이라고 합니다.

그러니 3 × 3 두 번은 5 × 5 한 번과 같은 범위를 보면서 파라미터는 적고, 중간에 ReLU가 한 번 더 들어가서 더 복잡한 무늬를 표현할 수 있습니다. 풀링도 수용 영역을 넓힙니다. 같은 방법으로 세어 보면 9.5절 CNN의 두 번째 풀링을 지난 한 칸은 입력의 10 × 10을 보고 있어요. 층을 쌓을수록 작은 돋보기들이 모여 사진 전체를 보게 됩니다.

ResNet: 고칠 양만 배우기

3 × 3을 많이 쌓으면 좋다면, 100층, 200층도 쌓으면 될까요? 실제로 해 보면 어느 깊이를 넘어서는 **층을 더 쌓을수록 학습 데이터에서조차 성적이 나빠지는** 일이 생겼습니다. 과적합이 아니라 학습 자체가 안 되는 거예요. 2015년 마이크로소프트 연구팀의 **ResNet**은 이 문제를 아주 간단한 덧셈 하나로 풀었습니다.

그림을 고쳐 그리는 일을 생각해 보세요. 매번 빈 종이에 처음부터 다시 그리는 것보다, 원래 그림을 그대로 두고 **고칠 부분만** 덧칠하는 편이 훨씬 쉽죠. 고칠 게 없으면 아무것도 안 하면 되고요. ResNet의 블록은 입력 x를 받아 합성곱 두 번으로 "고칠 양" F(x)를 계산하고, 마지막에 입력을 그대로 더해 **x + F(x)** 를 내보냅니다. 입력을 블록 뒤로 바로 건네주는 길을 **지름길(shortcut, skip connection)**, 이런 연결을 **잔차 연결(residual connection)** 이라고 해요. 직접 만들어 보겠습니다.

```
import torch
import torch.nn.functional as F
from torch import nn


class 잔차블록(nn.Module):
    """합성곱 두 번으로 '고칠 양'만 배우고, 입력을 그대로 더해 내보내는 블록."""

    def __init__(self, 입력: int, 출력: int, 보폭: int = 1) -> None:
        super().__init__()
        self.합성곱1 = nn.Conv2d(입력, 출력, 3, stride=보폭, padding=1, bias=False)
        self.정규화1 = nn.BatchNorm2d(출력)
        self.합성곱2 = nn.Conv2d(출력, 출력, 3, padding=1, bias=False)
        self.정규화2 = nn.BatchNorm2d(출력)
        # 모양이 달라지면 지름길도 1×1 합성곱으로 모양을 맞춘다
        if 보폭 != 1 or 입력 != 출력:
            self.지름길 = nn.Sequential(nn.Conv2d(입력, 출력, 1, stride=보폭, bias=False),
                                    nn.BatchNorm2d(출력))
        else:
            self.지름길 = nn.Identity()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        고칠양 = self.정규화2(self.합성곱2(F.relu(self.정규화1(self.합성곱1(x)))))
        return F.relu(고칠양 + self.지름길(x))         # 핵심은 이 덧셈 한 번


torch.manual_seed(0)
x = torch.randn(8, 32, 14, 14)
같은크기 = 잔차블록(32, 32)
줄이기 = 잔차블록(32, 64, 보폭=2)
print("같은 크기:", tuple(x.shape), "→", tuple(같은크기(x).shape), type(같은크기.지름길).__name__)
print("줄이기   :", tuple(x.shape), "→", tuple(줄이기(x).shape), type(줄이기.지름길).__name__)

# 고칠 양을 0으로 만들면 블록은 입력을 (ReLU만 거쳐) 그대로 내보낸다
nn.init.zeros_(같은크기.정규화2.weight)
같은크기.eval()
print("고칠 양이 0이면 그대로 통과:", torch.allclose(같은크기(x), F.relu(x)))
```

```
같은 크기: (8, 32, 14, 14) → (8, 32, 14, 14) Identity
줄이기   : (8, 32, 14, 14) → (8, 64, 7, 7) Sequential
고칠 양이 0이면 그대로 통과: True
```

`forward`의 마지막 줄 `고칠양 + self.지름길(x)`가 전부입니다. 나머지는 9.7절의 **합성곱 → 배치 정규화 → ReLU** 를 두 번 쌓은 것이에요(두 번째 ReLU는 덧셈 뒤로 옮겼습니다).

덧셈을 하려면 두 텐서의 모양이 같아야 하죠. 블록이 보폭 2로 크기를 절반으로 줄이거나 채널을 32 → 64로 늘리면 `x`와 `고칠양`의 모양이 달라집니다. 그때는 지름길에 9.3절의 **1 × 1 합성곱**(보폭 2)을 둬서 모양만 맞춰요. 결과를 보면 같은 크기 블록의 지름길은 `Identity`(아무것도 안 함), 줄이는 블록의 지름길은 `Sequential`(1 × 1 합성곱 + 배치 정규화)이고, `(8, 32, 14, 14)`가 `(8, 64, 7, 7)`로 바뀌었습니다.

마지막 확인이 ResNet의 아이디어를 잘 보여 줍니다. 두 번째 배치 정규화의 크기 숫자를 0으로 만들어 "고칠 양"을 0으로 만들면, 블록은 입력을 (ReLU만 거쳐) 그대로 내보내요. 블록 하나를 더 쌓아도 **최소한 아무것도 안 하는 것**은 쉽게 배울 수 있으니, 층을 더 쌓는다고 나빠질 이유가 없어지는 거죠. 그래서 실제 ResNet 구현은 학습 시작 때 일부러 이 숫자를 0으로 두기도 합니다.

깊어지면 기울기가 사라진다

덧셈 하나가 왜 그렇게 큰 차이를 만들까요? 5장 5.6절의 연쇄 법칙을 떠올려 보세요. 기울기는 출력에서 입력 쪽으로 층을 하나씩 거슬러 가며 **곱해집니다.** 층마다 1보다 작은 수가 곱해지면 깊은 곳에 닿을 즈음엔 0에 가까워져요. 배치 정규화 없이 평범하게 쌓은 CNN과 잔차 CNN에서 **첫 층**에 도착한 기울기의 크기를 재 보겠습니다.

```
import torch
import torch.nn.functional as F
from torch import nn


class 블록(nn.Module):
    """합성곱 두 번. 잔차=True면 입력을 더해 내보낸다 (비교를 위해 배치 정규화는 뺐다)."""

    def __init__(self, c: int, 잔차: bool) -> None:
        super().__init__()
        self.잔차 = 잔차
```

```
        self.합성곱1 = nn.Conv2d(c, c, 3, padding=1)
        self.합성곱2 = nn.Conv2d(c, c, 3, padding=1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        출력 = self.합성곱2(F.relu(self.합성곱1(x)))
        return F.relu(출력 + x) if self.잔차 else F.relu(출력)


def 깊은_CNN(블록수: int, 잔차: bool) -> nn.Sequential:
    """첫 합성곱 + 블록 여러 개 + 전역 평균 풀링 + 분류층."""
    return nn.Sequential(nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
                         *[블록(16, 잔차) for _ in range(블록수)],
                         nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(16, 10))


torch.manual_seed(0)
x = torch.randn(32, 1, 14, 14)                  # 예제용 무작위 입력과 정답
y = torch.randint(0, 10, (32,))
print("합성곱층 수   평범한 CNN   잔차 CNN   (첫 층 기울기 크기)")
for 블록수 in (2, 10, 25):
    크기 = []
    for 잔차 in (False, True):
        torch.manual_seed(0)
        모델 = 깊은_CNN(블록수, 잔차)
        F.cross_entropy(모델(x), y).backward()
        크기.append(모델[0].weight.grad.norm().item())
    print(f"{블록수 * 2 + 1:>8}     {크기[0]:10.2e}   {크기[1]:9.2e}")
```

```
합성곱층 수   평범한 CNN   잔차 CNN   (첫 층 기울기 크기)
       5      4.48e-04    3.54e-02
      21      3.94e-10    3.62e-02
      51      9.17e-23    3.58e-02
```

평범한 CNN은 합성곱 5층에서 4.48e-04이던 첫 층 기울기가 21층에서 **3.94e-10**, 51층에서는 **9.17e-23**이 됐습니다. 0이나 다름없어요. 이러면 첫 층은 아무리 학습해도 움직이지 않습니다. 8장 8.6절에서 본 "층을 지날 때마다 값이 줄어드는" 문제가 기울기 쪽에서 일어난 것이고, 이를 **기울기**

소실(vanishing gradient) 이라고 해요.

잔차 CNN은 5층이든 51층이든 첫 층 기울기가 0.03대로 거의 그대로입니다. x + F(x)를 x로 미분하면 **1 + (F의 기울기)** 가 되죠. 기울기가 지름길을 따라 "1을 곱하며" 그대로 흘러갈 수 있는 길이 생긴 거예요. 층이 아무리 깊어도 기울기가 끊기지 않는 고속도로가 깔린 셈입니다.

잔차 연결로 학습해 보기

기울기 크기만 보지 말고 실제로 학습해 보겠습니다. 합성곱 21층짜리 CNN을 평범하게 쌓은 것과 잔차 연결을 넣은 것, 그리고 각각에 배치 정규화를 더한 것, 네 가지를 비교해요. 빨리 보려고 학습 사진 5,000장을 14 × 14로 줄여 쓰고, 계산이 많아서 `장치_고르기()`로 GPU를 썼습니다. 그래서 숫자는 실행 환경에 따라 조금 다를 수 있어요(아래 결과는 이 책의 실행 환경인 Apple M4 Max의 MPS에서 나온 값입니다).

```
import torch
import torch.nn.functional as F
from torch import nn
from torchvision import datasets

from 옷도구 import 장치_고르기


class 블록(nn.Module):
    """합성곱 두 번. 잔차=True면 입력을 더하고, 정규화=True면 합성곱마다 배치 정규화."""

    def __init__(self, c: int, 잔차: bool, 정규화: bool) -> None:
        super().__init__()
        self.잔차 = 잔차
        self.합성곱1 = nn.Conv2d(c, c, 3, padding=1)
        self.합성곱2 = nn.Conv2d(c, c, 3, padding=1)
        self.정규화1 = nn.BatchNorm2d(c) if 정규화 else nn.Identity()
        self.정규화2 = nn.BatchNorm2d(c) if 정규화 else nn.Identity()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        출력 = self.정규화2(self.합성곱2(F.relu(self.정규화1(self.합성곱1(x)))))
        return F.relu(출력 + x) if self.잔차 else F.relu(출력)
```

```
def 깊은_CNN(블록수: int, 잔차: bool, 정규화: bool) -> nn.Sequential:
    return nn.Sequential(nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
                         *[블록(16, 잔차, 정규화) for _ in range(블록수)],
                         nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(16, 10))


# 빨리 보려고 학습 5,000장·시험 2,000장만, 사진은 14×14로 줄여서
장치 = 장치_고르기()
학습셋 = datasets.FashionMNIST("data", train=True, download=True)
시험셋 = datasets.FashionMNIST("data", train=False, download=True)
X = F.avg_pool2d(학습셋.data[:5000].float().unsqueeze(1) / 255, 2).to(장치)
y = 학습셋.targets[:5000].to(장치)
X_시험 = F.avg_pool2d(시험셋.data[:2000].float().unsqueeze(1) / 255, 2).to(장치)
y_시험 = 시험셋.targets[:2000].to(장치)

설정들 = [("평범한       ", False, False), ("평범한 + BN  ", False, True),
          ("잔차         ", True, False), ("잔차 + BN    ", True, True)]
for 이름, 잔차, 정규화 in 설정들:
    torch.manual_seed(0)
    모델 = 깊은_CNN(10, 잔차, 정규화).to(장치)   # 합성곱 21층
    옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.01, momentum=0.9)
    섞기 = torch.Generator().manual_seed(0)
    기록 = []
    for 에폭 in range(1, 6):
        모델.train()
        순서 = torch.randperm(len(X), generator=섞기).to(장치)
        손실들 = []
        for i in range(0, len(X), 64):
            묶음 = 순서[i:i + 64]
            옵티마이저.zero_grad()
            손실 = F.cross_entropy(모델(X[묶음]), y[묶음])
            손실.backward()
            옵티마이저.step()
            손실들.append(손실.item())
        모델.eval()
```

```
        with torch.no_grad():
            정확도 = (모델(X_시험).argmax(dim=1) == y_시험).float().mean().item()
        if 에폭 % 2 == 1:
            기록.append(f"{에폭}에폭 손실 {sum(손실들) / len(손실들):.3f}, 정확도 {정확도:5.1%}")
    print(이름, " / ".join(기록))
```

```
평범한        1에폭 손실 2.309, 정확도  9.7% / 3에폭 손실 2.302, 정확도 10.1% / 5에폭 손실 2.302, 정
↪ 확도 10.1%
평범한 + BN   1에폭 손실 1.945, 정확도 41.3% / 3에폭 손실 0.956, 정확도 41.7% / 5에폭 손실 0.778,
↪ 정확도 58.7%
잔차          1에폭 손실 2.274, 정확도 23.1% / 3에폭 손실 1.898, 정확도 35.9% / 5에폭 손실 1.153, 정
↪ 확도 49.5%
잔차 + BN     1에폭 손실 1.324, 정확도 52.9% / 3에폭 손실 0.663, 정확도 73.5% / 5에폭 손실 0.597,
↪ 정확도 69.2%
```

- **평범한 21층**은 손실이 처음부터 끝까지 2.30입니다. 열 클래스를 찍을 때의 손실 -log(1/10) = 2.30 그대로(6장 6.5절)라서 정확도도 10%예요. 앞에서 본 대로 기울기가 첫 층까지 닿지 않으니 아무것도 배우지 못했습니다.
- **잔차 21층**은 배치 정규화가 없어도 배웁니다. 손실이 2.274 → 1.153으로 내려갔어요. 덧셈 하나로 죽어 있던 모델이 살아났습니다.
- **배치 정규화**도 평범한 모델을 살려 냅니다(손실 0.778). 8장 8.4절에서 깊은 MLP를 살렸던 것과 같아요.
- **잔차 + 배치 정규화**가 손실이 가장 빨리, 가장 낮게(0.597) 내려갔습니다. 실제 ResNet이 바로 이 조합이에요.

정확도 열은 에폭마다 오르내림이 큽니다. 원인을 하나로 단정하기는 어렵지만, 학습 데이터가 5,000장뿐이고 시험 사진도 2,000장이라 몇십 장만 달리 맞혀도 숫자가 크게 움직이고, 학습률 0.01에 모멘텀 0.9로 꽤 큰 걸음을 걷고 있어서 에폭이 끝나는 순간의 가중치도 흔들리기 때문이에요. 이런 작은 실험에서는 손실 추세를 더 믿으세요. 이 정도 깊이에서는 배치 정규화만으로도 학습이 되지만, 50층, 100층으로 가면 배치 정규화만 쓴 평범한 모델은 다시 성적이 떨어지고 잔차 연결이 있는 모델만 깊이의 덕을 봅니다. ResNet 논문이 152층까지 쌓을 수 있었던 이유예요. 잔차 연결은 CNN을 넘어 12장의 트랜스포머에서도 블록마다 그대로 쓰입니다.

9.9 CNN은 무엇을 보는가

딥러닝 모델은 "왜 그렇게 판단했는지 알기 어렵다"고 1장 1.11절의 표에 적었습니다. 그래도 CNN은 그림을 그림 모양 그대로 다루니까, 안을 들여다볼 방법이 꽤 있어요. 9.5절에서 학습한 `fashion_cnn.pt`를 열어 세 가지를 그려 보겠습니다.

첫 층의 필터

```
import matplotlib
import matplotlib.pyplot as plt
import torch

matplotlib.use("Agg")                                  # 창을 띄우지 않고 파일로만 그리기
plt.rcParams["font.family"] = "AppleGothic"            # macOS 한글 글꼴
plt.rcParams["axes.unicode_minus"] = False

가중치 = torch.load("fashion_cnn.pt", weights_only=True)    # 9.5절에서 저장한 CNN
첫필터 = 가중치["특징.0.weight"]                             # 첫 합성곱층: (32, 1, 3, 3)
print("첫 층 필터 묶음:", tuple(첫필터.shape))
print("둘째 층 필터 묶음:", tuple(가중치["특징.3.weight"].shape))

그림, 칸들 = plt.subplots(4, 8, figsize=(8, 4.4))
for i, 칸 in enumerate(칸들.flat):
    필터 = 첫필터[i, 0]
    한계 = 필터.abs().max()
    칸.imshow(필터, cmap="bwr", vmin=-한계, vmax=한계)    # 빨강은 +, 파랑은 -
    칸.set_title(f"{i}번", fontsize=8)
    칸.axis("off")
그림.suptitle("첫 합성곱층의 3×3 필터 32개 (빨강 +, 파랑 -)")
그림.tight_layout()
그림.savefig("cnn_filters.png", dpi=200)

# 필터 숫자의 합이 0에 가까우면 '밝기의 변화(경계)'에, 한쪽으로 쏠리면 '밝기 자체'에 반응한다
쏠림 = 첫필터.sum(dim=(1, 2, 3)) / 첫필터.abs().sum(dim=(1, 2, 3))
경계형 = (쏠림.abs() < 0.3).nonzero().flatten().tolist()
print(f"경계에 반응하는 필터 {len(경계형)}개:", 경계형)
```

```
print("잉크에 반응:", (쏠림 >= 0.3).nonzero().flatten().tolist(),
      "/ 잉크 없는 곳에 반응:", (쏠림 <= -0.3).nonzero().flatten().tolist())
```

```
첫 층 필터 묶음: (32, 1, 3, 3)
둘째 층 필터 묶음: (64, 32, 3, 3)
경계에 반응하는 필터 25개: [0, 1, 2, 4, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 22,
↪  23, 24, 25, 26, 31]
잉크에 반응: [29] / 잉크 없는 곳에 반응: [3, 6, 16, 27, 28, 30]
```

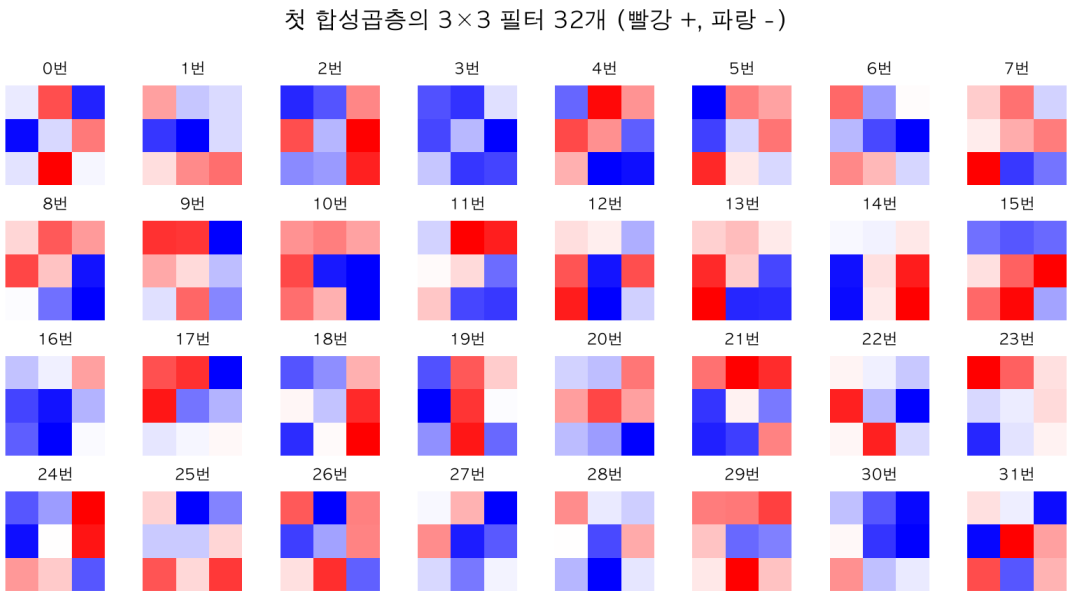


그림 9.1: 9.5절 CNN의 첫 합성곱층이 학습한 3×3 필터 32개. 빨강은 양수, 파랑은 음수이고, 대부분 한쪽은 빨갛고 반대쪽은 파란 경계 모양이다.

`state_dict`에서 첫 합성곱층의 가중치를 이름(`특징.0.weight`)으로 꺼냈습니다. 6장 6.10절에서 본 대로 이름은 "`특징` 부품의 0번 층의 가중치"라는 뜻이에요. 32개 필터를 빨강(+)과 파랑(-)으로 칠하니, 9.1절에서 우리가 손으로 만든 세로선·가로선 필터처럼 **한쪽은 빨갛고 반대쪽은 파란** 필터가 많습니다. 밝기가 변하는 곳, 즉 경계에 반응하는 모양이에요.

숫자로도 확인했습니다. 필터 숫자의 합이 0에 가까우면(+와 -가 상쇄) 고르게 밝은 곳에서는 0이 나오고 밝기가 **변하는** 곳에서만 반응하니 경계 탐지기이고, 한쪽으로 쏠려 있으면 밝기 **자체**에 반응합니다. 32개 중 25개가 경계형이었어요. 아무도 "경계를 찾아라"라고 시키지 않았는데, 옷을 구별

하려고 학습하다 보니 스스로 경계 탐지기를 만든 겁니다. 다만 3 × 3은 너무 작아서 필터 그림만으로 정확한 역할을 읽기는 어려워요. 실제로 무엇에 반응하는지는 특징 맵을 보는 편이 낫습니다.

특징 맵: 층마다 무엇이 켜지나

```
import matplotlib
import matplotlib.pyplot as plt
import torch

from 옷도구 import 옷CNN, 시험_데이터

matplotlib.use("Agg")
plt.rcParams["font.family"] = "AppleGothic"
plt.rcParams["axes.unicode_minus"] = False

모델 = 옷CNN()
모델.load_state_dict(torch.load("fashion_cnn.pt", weights_only=True))
모델.eval()
X, _ = 시험_데이터()
사진 = X[:1]                                        # 시험 사진 0번(앵클부츠): (1, 1, 28, 28)
with torch.no_grad():
    첫단 = 모델.특징[:2](사진)                       # 첫 합성곱 + ReLU: (1, 32, 28, 28)
    둘째단 = 모델.특징[:5](사진)                     # 둘째 합성곱 + ReLU까지: (1, 64, 14, 14)
print("첫째 단 특징 맵:", tuple(첫단.shape), " 둘째 단 특징 맵:", tuple(둘째단.shape))
print("첫째 단에서 0인 칸의 비율:", f"{(첫단 == 0).float().mean():.1%}")
print("둘째 단에서 0인 칸의 비율:", f"{(둘째단 == 0).float().mean():.1%}")

그림, 칸들 = plt.subplots(3, 8, figsize=(8, 3.6))
칸들[0, 0].imshow(사진[0, 0], cmap="gray_r")
칸들[0, 0].set_title("입력", fontsize=8)
for i in range(1, 8):                               # 첫째 단 7장
    칸들[0, i].imshow(첫단[0, i - 1], cmap="viridis")
    칸들[0, i].set_title(f"1단 {i - 1}번", fontsize=8)
for 번호 in range(16):                              # 둘째 단 16장
    칸 = 칸들[1 + 번호 // 8, 번호 % 8]
    칸.imshow(둘째단[0, 번호], cmap="viridis")
    칸.set_title(f"2단 {번호}번", fontsize=8)
```

```
for 칸 in 칸들.flat:
    칸.axis("off")
그림.tight_layout()
그림.savefig("cnn_feature_maps.png", dpi=200)
```

```
첫째 단 특징 맵: (1, 32, 28, 28)  둘째 단 특징 맵: (1, 64, 14, 14)
첫째 단에서 0인 칸의 비율: 50.3%
둘째 단에서 0인 칸의 비율: 79.6%
```

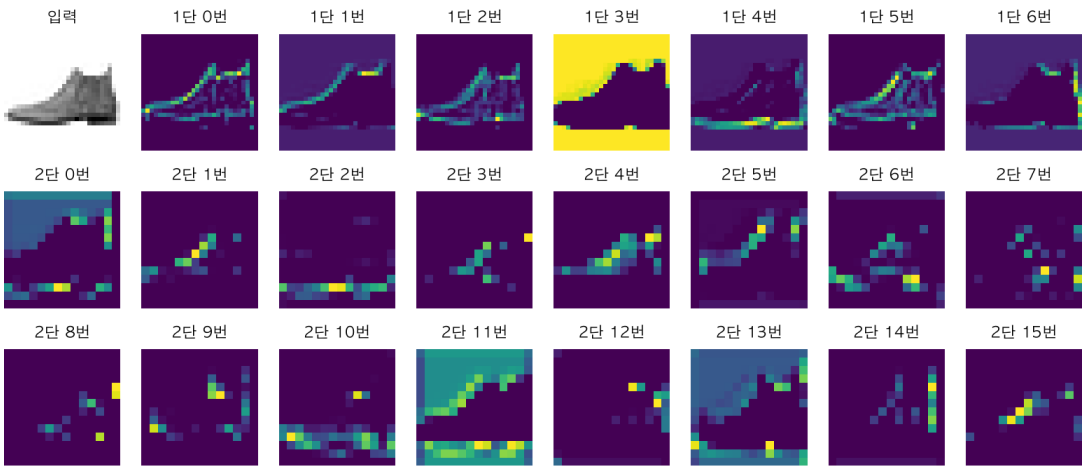


그림 9.2: 앵클부츠 사진과 첫째 단 특징 맵 7장, 둘째 단 특징 맵 16장. 첫째 단은 신발의 윤곽선과 배경을 그대로 따라가고, 둘째 단은 발등의 사선, 밑창의 가로줄처럼 일부 부위에서만 밝게 켜진다.

시험 사진 0번인 앵클부츠를 넣고, 첫째 단(합성곱 + ReLU)과 둘째 단의 출력을 그렸습니다. `모델.특징`은 `nn.Sequential`이라 리스트처럼 잘라 쓸 수 있어요. `특징[:2]`는 첫 두 층, `특징[:5]`는 다섯 층(둘째 합성곱의 ReLU)까지만 통과시킨 결과입니다. 모델을 `특징`과 `분류`로 나눠 둔 덕을 여기서 봅니다.

- **첫째 단**은 아직 그림과 많이 닮았습니다. 신발의 윤곽선, 발등의 사선, 밑창 같은 경계가 지도마다 다르게 켜져 있어요. 3번 지도는 신발이 아니라 **배경 전체**가 밝은데, 앞의 필터 분석에서 "잉크 없는 곳에 반응"으로 분류된 필터가 바로 3번입니다. 필터 숫자가 대부분 음수라서 잉크(큰 값)를 만나면 0 아래로 내려가 ReLU에 꺼지고, 잉크가 없는 배경(값 0)에서는 양수인 편향만 남아 켜진 채로 있는 거예요. 입력 그림은 `gray_r`로 그려서 배경이 흰색으로 보이지만, 값은 0이라는 점을 헷갈리지 마세요.

- **둘째 단**은 14 × 14로 작아졌고, 그림이라기보다 **부위 지도**에 가깝습니다. 발등의 사선만, 밑창의 가로줄만, 뒤꿈치 세로선만 켜진 지도가 따로 있어요.
- 0인 칸의 비율이 첫째 단 50.3%에서 둘째 단 79.6%로 늘었습니다. ReLU가 음수를 0으로 만드니, 깊은 층일수록 "내가 찾는 무늬가 있는 곳에서만 켜지는" 선택적인 지도가 된다는 뜻이에요.

어디를 보고 판단했나

마지막으로 "이 사진을 앵클부츠라고 할 때, 어느 픽셀이 판단에 영향을 줬나"를 보겠습니다. 5장 5.12절에서 입력에 대한 기울기로 "무엇에 민감한가"를 쟀던 방법을 그대로 써요. 예측한 클래스의 점수를 입력 사진으로 미분하면, 픽셀마다 "이 픽셀을 조금 바꾸면 점수가 얼마나 바뀌나"가 나옵니다. 이런 그림을 **돌출 맵(saliency map)** 이라고 해요.

```
import matplotlib
import matplotlib.pyplot as plt
import torch

from 옷도구 import 옷CNN, 시험_데이터

matplotlib.use("Agg")
plt.rcParams["font.family"] = "AppleGothic"
plt.rcParams["axes.unicode_minus"] = False
이름 = ["T셔츠", "바지", "풀오버", "드레스", "코트", "샌들", "셔츠", "운동화", "가방", "앵클부츠"]

모델 = 옷CNN()
모델.load_state_dict(torch.load("fashion_cnn.pt", weights_only=True))
모델.eval()
X, y = 시험_데이터()

번호들 = [0, 1, 2, 4]                                  # 앵클부츠, 풀오버, 바지, 셔츠
그림, 칸들 = plt.subplots(2, len(번호들), figsize=(7, 3.8))
for 열, 번호 in enumerate(번호들):
    사진 = X[번호:번호 + 1].clone().requires_grad_(True)
    점수 = 모델(사진)[0]
    예측 = 점수.argmax().item()
```

```
    점수[예측].backward()                                  # 예측한 클래스 점수를 입력으로 미분 (5장 5.12
↪   절)
    중요도 = 사진.grad[0, 0].abs()
    확률 = torch.softmax(점수.detach(), dim=0)[예측]
    print(f"{번호}번: 정답 {이름[y[번호]]}, 예측 {이름[예측]} ({확률:.1%})")
    칸들[0, 열].imshow(사진.detach()[0, 0], cmap="gray_r")
    칸들[0, 열].set_title(f"예측: {이름[예측]}", fontsize=9)
    칸들[1, 열].imshow(중요도, cmap="hot")
    for 칸 in 칸들[:, 열]:
        칸.axis("off")
그림.tight_layout()
그림.savefig("cnn_saliency.png", dpi=200)
```

```
0번: 정답 앵클부츠, 예측 앵클부츠 (99.7%)
1번: 정답 풀오버, 예측 풀오버 (100.0%)
2번: 정답 바지, 예측 바지 (100.0%)
4번: 정답 셔츠, 예측 셔츠 (93.1%)
```

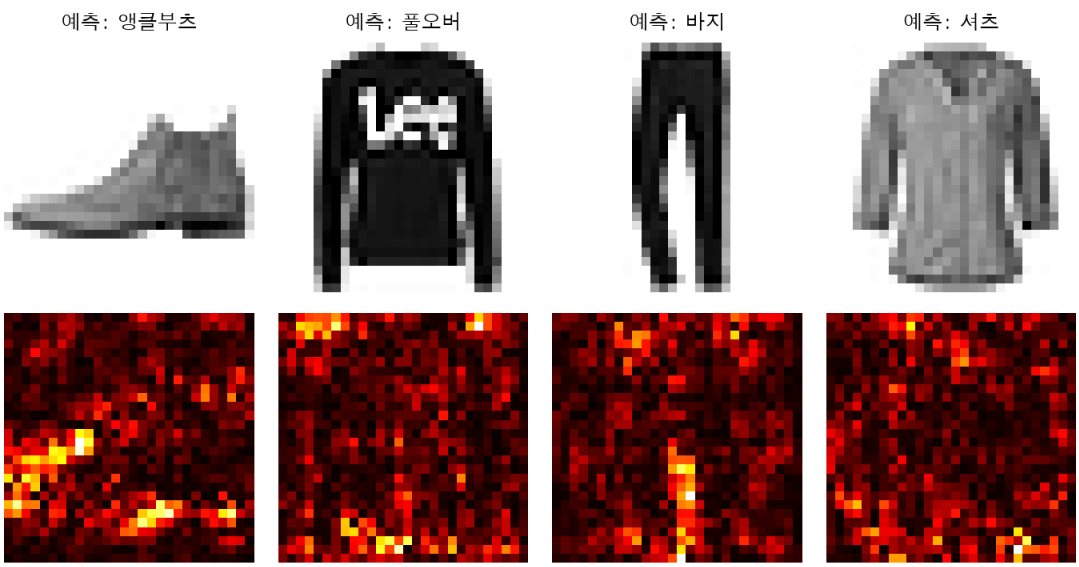


그림 9.3: 시험 사진 네 장(앵클부츠, 풀오버, 바지, 셔츠)과 그 아래 입력 기울기의 크기. 앵클부츠는 앞코와 굽, 바지는 두 다리 사이의 빈틈 주변이 밝다.

기울기를 받으려면 입력 사진에 `requires_grad_(True)`를 걸어야 하는데, `시험_데이터()`가 준 텐서를 그대로 건드리지 않으려고 `clone()`으로 복사한 뒤 걸었어요(4장 4.4절). 네 장 모두 맞혔고, 셔츠만 93.1%로 확신이 조금 낮았습니다.

밝은 픽셀일수록 판단에 큰 영향을 준 곳입니다. 앵클부츠는 앞코 쪽과 굽 근처가, 바지는 **두 다리 사이의 빈틈**이 가장 밝아요. 바지와 드레스를 가르는 결정적인 단서가 "가운데가 갈라져 있는가"라는 것을 모델이 스스로 찾은 셈입니다. 다만 그림 전체에 자잘한 점이 흩뿌려진 것처럼 돌출 맵은 잡음이 많아서, 하나만 보고 단정하기는 어려워요. 실무에서는 여러 번 잡음을 섞어 평균을 내는 등 다듬은 방법들을 씁니다. 그래도 "모델이 엉뚱한 곳(예를 들어 배경의 워터마크)을 보고 맞히고 있지는 않은가"를 점검하는 데는 이 정도로도 충분히 쓸모가 있어요.

9.10 실습: 작은 ResNet으로 옷 분류기 다시 만들기

이 장과 앞 장들에서 배운 것을 모두 모아 옷 분류기를 다시 만들겠습니다. 들어가는 재료는 이렇습니다.

1. **작은 ResNet**: 첫 합성곱 뒤에 9.8절의 잔차 블록 세 개(28 × 28에서 32채널 → 14 × 14에서 64채널 → 7 × 7에서 128채널)
2. **전역 평균 풀링 + 드롭아웃 + 작은 `Linear`**: 9.6절과 9.7절. 비싼 `Flatten` + 큰 `Linear`를 없앤다
3. **데이터 증강**: 7장 7.6절의 `RandomCrop(28, padding=2)`로 상하좌우 2칸 안에서 무작위로 밀고, 좌우 뒤집기(옷은 좌우를 뒤집어도 같은 옷)
4. **검증 데이터 따로 두기**: 학습 6만 장 중 5,000장을 검증용으로. 7장 7.8절처럼 transform만 다른 데이터셋 두 개를 같은 번호 목록으로 나눈다
5. **AdamW + OneCycleLR**: 8장 8.2절과 8.7절. OneCycleLR은 배치마다 `step()`

시험 데이터는 설계를 모두 마친 뒤 마지막에 **한 번만** 엽니다(7장 7.8절 Q&A). 계산이 많아 `장치_고르기()`로 GPU를 썼어요. 아래 숫자는 이 책의 실행 환경(Apple M4 Max, MPS)에서 나온 값이라, 다른 환경에서는 소수점 아래가 조금 다를 수 있습니다.

```
import torch
import torch.nn.functional as F
from torch import nn
from torch.utils.data import DataLoader, Subset
from torchvision import datasets
```

```
from torchvision.transforms import v2

from 옷도구 import 시험_데이터, 오른쪽으로_밀기, 장치_고르기


class 잔차블록(nn.Module):
    """9.8절의 잔차 블록."""

    def __init__(self, 입력: int, 출력: int, 보폭: int = 1) -> None:
        super().__init__()
        self.합성곱1 = nn.Conv2d(입력, 출력, 3, stride=보폭, padding=1, bias=False)
        self.정규화1 = nn.BatchNorm2d(출력)
        self.합성곱2 = nn.Conv2d(출력, 출력, 3, padding=1, bias=False)
        self.정규화2 = nn.BatchNorm2d(출력)
        if 보폭 != 1 or 입력 != 출력:
            self.지름길 = nn.Sequential(nn.Conv2d(입력, 출력, 1, stride=보폭, bias=False),
                                        nn.BatchNorm2d(출력))
        else:
            self.지름길 = nn.Identity()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        고칠양 = self.정규화2(self.합성곱2(F.relu(self.정규화1(self.합성곱1(x)))))
        return F.relu(고칠양 + self.지름길(x))


def 작은_ResNet() -> nn.Sequential:
    """28×28 → 28×28(32채널) → 14×14(64채널) → 7×7(128채널) → 전역 평균 풀링 → 10개 점수."""
    return nn.Sequential(
        nn.Conv2d(1, 32, 3, padding=1, bias=False), nn.BatchNorm2d(32), nn.ReLU(),
        잔차블록(32, 32), 잔차블록(32, 64, 보폭=2), 잔차블록(64, 128, 보폭=2),
        nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Dropout(0.3), nn.Linear(128, 10),
    )


@torch.no_grad()
def 정확도_재기(모델: nn.Module, 로더: DataLoader, 장치: torch.device) -> float:
```

```
    """로더의 모든 사진에 대한 정확도."""
    모델.eval()
    맞힘 = sum((모델(x.to(장치)).argmax(dim=1).cpu() == y).sum().item() for x, y in 로더)
    return 맞힘 / len(로더.dataset)


기본 = [v2.ToImage(), v2.ToDtype(torch.float32, scale=True)]
증강 = v2.Compose([v2.ToImage(), v2.RandomCrop(28, padding=2),        # 상하좌우 2칸 안에서 무작위
↪ 로 밀기
                   v2.RandomHorizontalFlip(), v2.ToDtype(torch.float32, scale=True)])
학습용 = datasets.FashionMNIST("data", train=True, download=True, transform=증강)
검증용 = datasets.FashionMNIST("data", train=True, download=True, transform=v2.Compose(기본))
순서 = torch.randperm(60000, generator=torch.Generator().manual_seed(0))
학습셋, 검증셋 = Subset(학습용, 순서[:55000]), Subset(검증용, 순서[55000:])   # 7장 7.8절의 방법

torch.manual_seed(0)
장치 = 장치_고르기()
학습_로더 = DataLoader(학습셋, batch_size=128, shuffle=True)
검증_로더 = DataLoader(검증셋, batch_size=1000)
모델 = 작은_ResNet().to(장치)
에폭수 = 12
옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=2e-3, weight_decay=5e-4)
스케줄러 = torch.optim.lr_scheduler.OneCycleLR(옵티마이저, max_lr=3e-3, total_steps=에폭수 * len(학
↪ 습_로더))
print(f"장치 {장치}, 파라미터 {sum(p.numel() for p in 모델.parameters()):,}개")

for 에폭 in range(1, 에폭수 + 1):
    모델.train()
    for x, y in 학습_로더:
        x, y = x.to(장치), y.to(장치)
        옵티마이저.zero_grad()
        F.cross_entropy(모델(x), y).backward()
        옵티마이저.step()
        스케줄러.step()                              # OneCycleLR은 배치마다 (8장 8.7절)
    if 에폭 % 3 == 0:
        print(f"에폭 {에폭:2}: 검증 정확도 {정확도_재기(모델, 검증_로더, 장치):.2%}")
```

```
torch.save(모델.state_dict(), "fashion_resnet.pt")

# 설계를 모두 마친 뒤 처음으로 시험 데이터를 연다
X, y = 시험_데이터()
이름 = ["T셔츠", "바지", "풀오버", "드레스", "코트", "샌들", "셔츠", "운동화", "가방", "앵클부츠"]
모델.eval()
with torch.no_grad():
    예측 = 모델(X.to(장치)).argmax(dim=1).cpu()
    밀기 = [(모델(오른쪽으로_밀기(X, 칸).to(장치)).argmax(dim=1).cpu() == y).float().mean().item()
            for 칸 in range(5)]
print(f"시험 정확도 {(예측 == y).float().mean():.2%}")
print("상의 네 종류:", ", ".join(f"{이름[k]} {(예측[y == k] == k).float().mean():.1%}" for k in (0,
↪  2, 4, 6)))
print("밀기 0~4칸:", "  ".join(f"{a:.1%}" for a in 밀기))
```

```
장치 mps, 파라미터 308,074개
에폭  3: 검증 정확도 85.26%
에폭  6: 검증 정확도 89.78%
에폭  9: 검증 정확도 92.56%
에폭 12: 검증 정확도 93.18%
시험 정확도 93.04%
상의 네 종류: T셔츠 89.9%, 풀오버 91.1%, 코트 88.9%, 셔츠 77.4%
밀기 0~4칸: 93.0%  93.1%  92.9%  92.4%  92.0%
```

결과를 하나씩 보겠습니다.

- **파라미터 308,074개**로 9.5절 CNN(421,642개)보다 작습니다. 합성곱층은 훨씬 많아졌지만 큰 `Linear`가 사라졌기 때문이에요.
- **검증 정확도**는 3에폭 85.26%에서 시작해 12에폭에 93.18%가 됐습니다. 처음 몇 에폭이 9.5절 CNN보다 낮은 것은 OneCycleLR이 학습률을 올리는 중이고(8장 8.7절의 워밍업), 증강 때문에 학습 문제가 더 어려워졌기 때문이에요. 학습률이 내려가는 후반에 크게 오릅니다.
- **시험 정확도 93.04%**. 검증 정확도와 거의 같으니 검증 데이터로 본 성적을 믿을 수 있었습니다. 6장 MLP의 88.58%에서 9.5절 CNN의 91.01%를 거쳐 4.5%p 올랐어요. 틀리던 사진의 40%쯤(11.42% → 6.96%)을 더 맞히게 된 셈입니다.
- **상의 네 종류**: 9.5절 CNN에서 81.9%로 떨어졌던 코트가 88.9%로, T셔츠가 83.7%에서

89.9%로 올랐고, 풀오버도 91.1%입니다. 그런데 **셔츠는 77.4%** 로 여전히 가장 약해요. 6장 74.1%, 9.5절 77.9%와 비교하면 사실상 제자리입니다. 모델을 바꿔도 셔츠가 안 오르는 것은 모델 탓보다 데이터 탓일 가능성이 큽니다. 28 × 28 흑백 사진에서 셔츠와 T셔츠, 풀오버를 가르는 칼라와 단추는 몇 픽셀밖에 안 되고, 사람이 봐도 정답이 애매한 사진이 섞여 있거든요. 이럴 때는 모델을 더 키우기보다 **틀린 사진을 직접 들여다보는 것**(6장 6.11절)이 먼저입니다.

- **밀기 0~4칸**: 93.0%, 93.1%, 92.9%, 92.4%, 92.0%. 네 칸을 밀어도 1%p밖에 떨어지지 않았습니다. 9.6절에서 MLP는 25.2%, 9.5절 CNN은 36.0%까지 무너졌던 시험이에요. 전역 평균 풀링(구조)과 무작위 밀기 증강(데이터)이 함께 1장 1.11절의 약속을 지켜 냈습니다.

모델	파라미터	시험 정확도	4칸 밀었을 때
6장 MLP	235,146	88.58%	25.2%
9.5절 CNN	421,642	91.01%	36.0%
9.10절 작은 ResNet + 증강	308,074	93.04%	92.0%

Q

Q. 더 오래, 더 크게 학습하면 얼마까지 오르나요? FashionMNIST에서 잘 다듬은 CNN은 시험 정확도 94~95% 근처까지 오른다고 알려져 있습니다. 에폭을 늘리고, 채널을 넓히고, 증강을 바꿔 보면 이 실습의 93%에서 조금 더 올라갈 거예요. 직접 해 보면 좋은 연습입니다. 다만 마지막 1%p를 짜내는 데는 지금까지 들인 것보다 훨씬 많은 계산이 들고, 셔츠처럼 데이터 자체가 애매한 부분은 남습니다. 실무에서는 그 시간에 데이터를 더 모으거나, 다음 10장처럼 **이미 큰 데이터로 학습된 모델을 빌려 오는** 편이 훨씬 효과적일 때가 많아요.

9.11 정리와 다음 장 미리보기

이 장에서 그림을 그림 모양 그대로 다루는 합성곱 신경망을 만들었습니다.

- **합성곱**: 작은 필터를 옮겨 가며 "같은 자리끼리 곱하고 모두 더하기". 결과는 무늬의 위치 지도(특징 맵). 필터 값은 학습으로 정한다
- **모양**: 입력 (장 수, 채널, 세로, 가로), 필터 묶음 (출력 채널, 입력 채널, 세로, 가로). 출력 크기 = (입력 + 2 × 패딩 - 필터) ÷ 보폭 + 1. 크기 유지는 3 × 3에 `padding=1`, 절반은 `stride=2`
- **파라미터 공유**: 같은 필터를 모든 자리에서 돌려 써서 완전 연결층보다 파라미터가 훨씬 적다. 1 × 1 합성곱은 자리마다 채널만 섞는다

- **풀링**: 최대·평균 풀링으로 줄여서 요약, 전역 평균 풀링(`AdaptiveAvgPool2d(1)`)은 채널마다 평균 하나
- **CNN의 모양**: 크기는 줄이고 채널은 늘린다. `nn.Sequential`의 특징 부분과 분류 부분
- **이동**: 합성곱은 등변(밀린 만큼 결과도 밀림), `Flatten` + `Linear`는 자리에 묶임, 전역 평균 풀링은 불변. 구조와 함께 데이터 증강을 쓴다
- **배치 정규화와 드롭아웃**: 합성곱 → `BatchNorm2d` → ReLU, 이때 합성곱은 `bias=False`. 드롭아웃은 마지막 `Linear` 앞
- **유명한 구조**: LeNet(기본 틀), AlexNet(ReLU·드롭아웃·GPU), VGG(3 × 3 쌓기와 수용 영역), ResNet(x + F(x), 지름길, 1 × 1 합성곱으로 모양 맞추기, 기울기 소실 해결)
- **들여다보기**: 첫 층 필터는 대부분 경계 탐지기, 깊은 층의 특징 맵은 부위 지도, 입력 기울기로 판단 근거 확인

이 장의 실습에서는 작은 ResNet을 FashionMNIST 6만 장으로 처음부터 학습시켰습니다. 그런데 실제 일에서 마주치는 데이터는 대개 이렇게 넉넉하지 않아요. 우리 가게 메뉴 사진 300장, 공장 불량품 사진 500장 같은 식이죠. 그 정도로는 ResNet을 처음부터 학습시키기 어렵습니다. 다음 10장에서는 9.8절에서 구조만 만들어 본 ResNet-18을, 이번에는 사진 120만 장으로 **이미 학습된 숫자까지** 받아 와서 우리 문제에 맞게 고쳐 쓰는 **전이 학습**을 합니다. 6장 6.4절에서 예고한 "앞부분은 얼리고 뒷부분만 고치기"를 드디어 해 볼 차례예요.

10

전이 학습: 남이 학습한 모델 빌려 쓰기

학습 목표

- 사전학습 모델이 무엇이고, 왜 적은 데이터로도 좋은 결과를 내는지 설명하기
- torchvision에서 사전학습 모델과 가중치 정보를 불러오고 몸통과 머리를 구분하기
- 모델이 배운 기준에 맞게 입력을 전처리하기(`weights.transforms()`, ImageNet 평균·표준편차)
- 몸통을 얼리고 머리만 학습하는 특징 추출과, 몸통까지 고치는 미세조정을 구분해 쓰기
- 얼린 모델에서도 배치 정규화 기록이 바뀌는 함정을 알고 피하기
- 층마다 학습률을 다르게 주고, 어디까지 풀어 학습할지 실험으로 정하기
- 머리를 바꾼 모델을 저장하고 다시 불러오기

들어가며: 그림을 잘 그리는 사람은 지도도 빨리 읽습니다

미술을 오래 한 친구에게 처음 보는 위성 사진을 보여 주고 "여기가 숲이야, 여기는 강이야" 하고 몇 장만 알려 주면 어떨까요? 아마 금세 구별할 겁니다. 위성 사진을 공부한 적은 없어도 선이 어떻게 이어지는지, 무늬가 반복되는지, 색이 고르게 퍼져 있는지를 보는 **눈**은 이미 있으니까요. 새로 배울 것은 "이런 무늬는 숲이라고 부른다"는 이름 붙이기뿐입니다.

딥러닝 모델도 똑같습니다. 사진 120만 장으로 1,000가지 물건을 구별하도록 학습한 모델은 그 과정에서 가장자리, 모서리, 질감, 모양 같은 "사진을 보는 눈"을 익혔어요. 이 눈을 그대로 빌려 와서 우리 문제에 맞게 이름 붙이는 부분만 새로 학습하는 방법이 **전이 학습(transfer learning)** 입니다.

남이 미리 학습해 둔 모델을 **사전학습 모델(pretrained model)** 이라고 불러요.

1장에서 딥러닝은 대체로 데이터가 많을수록 좋지만 전이 학습으로 필요한 양을 줄일 수 있다고 했죠. 이 장에서 그 말을 숫자로 확인합니다. 같은 모델 구조로 위성 사진 500장을 학습시켰을 때, 무작위 가중치에서 처음부터 시작하면 검증 정확도가 68.1%에 그치지만 사전학습 모델에서 시작하면 93%를 넘깁니다. 학습에 걸리는 시간도 훨씬 짧고요.

이 장에서 쓰는 사진 모델은 9장에서 배운 **합성곱 신경망(CNN)** 입니다. 9장 9.5절에서 CNN을 무늬를 찾는 **특징** 부분과 종류를 고르는 **분류** 부분으로 나눠 담았죠. 이 장에서는 같은 구분을 "사진을 받아 특징을 뽑는 몸통"과 "특징을 보고 답을 고르는 머리"라고 부르겠습니다. 6장에서 만든 학습 루프, 7장의 transforms, 8장의 배치 정규화가 모두 그대로 쓰입니다.

Q

Q. 무엇을 설치해야 하나요? 사전학습 모델은 PyTorch의 사진 도구 꾸러미인 torchvision에 들어 있습니다. 6장에서 이미 `uv add torchvision`을 했다면 그대로 쓰면 돼요. 이 장의 예제는 파이썬 3.12, PyTorch 2.14.1, torchvision 0.29.1에서 실행해 확인했습니다. 모델 가중치(약 10~45MB)와 데이터(약 90MB)는 처음 실행할 때 인터넷에서 내려받습니다.

10.1 사전학습 모델 불러오기

한 줄로 받는 1,170만 개의 숫자

torchvision의 `models`에는 유명한 사진 모델의 설계도와 학습된 가중치가 함께 들어 있습니다. 이 장의 주인공은 9장 9.8절에서 `weights=None`으로 구조만 만들어 봤던 ResNet18이에요. 잔차 연결을 쓰는 ResNet 가운데 가장 작은 것이라 노트북에서도 가볍게 돌아갑니다.

```
import torch
from torchvision import models

가중치 = models.ResNet18_Weights.DEFAULT        # ImageNet으로 미리 학습한 가중치
모델 = models.resnet18(weights=가중치)           # 처음 한 번은 약 45MB를 내려받는다
print(가중치)

개수 = sum(p.numel() for p in 모델.parameters())
바이트 = sum(p.numel() * p.element_size() for p in 모델.parameters())
print(f"파라미터 {개수:,}개, float32로 {바이트 / 1024**2:.1f}MB")
```

```
print(f"모델 카드에 적힌 개수 {가중치.meta['num_params']:,}개, 파일 {가중치.meta['_file_size']}MB")
print("ImageNet 검증 데이터 정확도(top-1):", 가중치.meta["_metrics"]["ImageNet-1K"]["acc@1"], "%")
범주 = 가중치.meta["categories"]
print(f"알아보는 범주 {len(범주)}가지:", 범주[:3], "...", 범주[-2:])
```

```
ResNet18_Weights.IMAGENET1K_V1
파라미터 11,689,512개, float32로 44.6MB
모델 카드에 적힌 개수 11,689,512개, 파일 44.661MB
ImageNet 검증 데이터 정확도(top-1): 69.758 %
알아보는 범주 1000가지: ['tench', 'goldfish', 'great white shark'] ... ['ear', 'toilet tissue']
```

`weights=` 자리에 가중치를 넘기면 설계도를 만든 뒤 학습된 숫자를 채워서 돌려줍니다. 6장 6.10절에서 `load_state_dict`로 하던 일을 torchvision이 대신해 주는 거예요. 처음 실행하면 파일을 내려받아 홈 폴더의 `.cache/torch/hub/checkpoints/`에 저장하고, 다음부터는 저장해 둔 파일을 씁니다. `weights=None`을 주면 같은 설계도에 무작위 숫자만 채운 모델이 나와요.

`ResNet18_Weights.DEFAULT`는 "이 모델에서 가장 좋은 가중치"를 가리키는 별명이고, 실제 이름은 `IMAGENET1K_V1`입니다. ImageNet이라는 사진 데이터셋에서 1,000개 범주(1K)를 구별하도록 학습한 첫 번째 판(V1)이라는 뜻이에요. 가중치 객체의 `meta`에는 모델 카드의 내용이 들어 있어서, 파라미터 개수, 파일 크기, ImageNet 검증 데이터 정확도, 범주 이름 1,000개를 바로 꺼내 볼 수 있습니다.

파라미터를 직접 세어 보니 11,689,512개로 모델 카드와 같습니다. 4장 4.3절의 계산법 그대로 칸 크기(`element_size()`, float32는 4바이트) × 칸 개수를 더하면 44.6MB이고, 내려받은 파일 크기 44.661MB와 거의 같아요. 파일에는 이 숫자들 외에 8장 8.4절의 배치 정규화가 기록해 둔 평균·분산(버퍼)이 조금 더 들어 있습니다. 6장의 옷 사진 MLP가 23만 개였으니 50배쯤 큰 모델인데, 이 숫자들이 모두 **이미 학습되어** 있다는 게 핵심입니다.

ImageNet top-1 정확도 69.758%는 "1,000개 범주 중 첫 번째로 고른 답이 맞은 비율"이에요. 열에 일곱이라니 낮아 보이지만, 범주 이름에 `tench`(잉어과 물고기)나 `great white shark`처럼 비슷한 것끼리 아주 잘게 나눈 1,000가지가 섞여 있다는 걸 생각하면 꽤 높은 숫자입니다.

어떤 모델이 있을까

ResNet18 말고도 고를 수 있는 모델이 많습니다. 모델 목록과 가중치 정보는 가중치를 내려받지 않고도 볼 수 있어요.

```
from torchvision import models

이름들 = models.list_models(module=models)     # torchvision의 사진 분류 모델 목록
print(f"사진 분류 모델 {len(이름들)}종:", 이름들[:4], "...")

for 이름 in ["mobilenet_v3_small", "resnet18", "efficientnet_b0", "resnet50", "vit_b_16"]:
    w = models.get_model_weights(이름).DEFAULT   # 가중치 정보만 읽는다. 내려받지 않는다
    정확도 = w.meta["_metrics"]["ImageNet-1K"]["acc@1"]
    print(f"{이름:19} 파라미터 {w.meta['num_params'] / 1e6:5.1f}백만, "
          f"파일 {w.meta['_file_size']:6.1f}MB, top-1 {정확도:.1f}%")
```

```
사진 분류 모델 80종: ['alexnet', 'convnext_base', 'convnext_large', 'convnext_small'] ...
mobilenet_v3_small  파라미터   2.5백만, 파일    9.8MB, top-1 67.7%
resnet18            파라미터  11.7백만, 파일   44.7MB, top-1 69.8%
efficientnet_b0     파라미터   5.3백만, 파일   20.5MB, top-1 77.7%
resnet50            파라미터  25.6백만, 파일   97.8MB, top-1 80.9%
vit_b_16            파라미터  86.6백만, 파일  330.3MB, top-1 81.1%
```

`list_models(module=models)`는 사진 분류 모델 이름을 모두 돌려주고, `get_model_weights(이름)`는 그 모델의 가중치 목록을 줍니다. 가중치 정보만 읽고 파일은 받지 않으니 금방 끝나요. 다섯 모델을 나란히 놓으면 고르는 기준이 보입니다.

모델	파라미터	파일	ImageNet top-1	이런 때
MobileNetV3-Small	250만	9.8MB	67.7%	휴대폰·작은 기기에서 빠르게
ResNet18	1,170만	44.7MB	69.8%	공부용, 무난한 출발점
EfficientNet-B0	530만	20.5MB	77.7%	크기 대비 정확도가 좋은 것
ResNet50	2,560만	97.8MB	80.9%	서버에서 더 높은 정확도

ViT-B/16	8,660만	330.3MB	81.1%	트랜스포머 구조 (12장), 큰 데이터

크기와 정확도가 꼭 비례하지 않는다는 점이 재미있습니다. EfficientNet-B0은 ResNet18의 절반도 안 되는 크기로 정확도가 8%p 가까이 높아요. 다만 ImageNet 정확도가 높다고 **우리 데이터에서도** 더 잘한다는 보장은 없습니다. 10.7절에서 직접 바꿔 끼워 보겠습니다.

Q

Q. 남이 학습한 가중치를 마음대로 써도 되나요? 모델 코드(torchvision)는 BSD 라이선스지만, 가중치는 학습에 쓴 데이터의 조건을 따를 수 있습니다. torchvision 문서도 "사전학습 모델은 학습 데이터에서 비롯된 별도의 조건을 가질 수 있으니 쓰려는 용도에 맞는지 직접 확인하라"고 안내해요. 공부와 연구에는 문제가 없지만, 회사 제품에 넣을 때는 모델 카드의 라이선스와 학습 데이터의 이용 조건을 꼭 확인하세요. 13장과 14장에서 Hugging Face의 언어 모델을 쓸 때도 같은 습관이 필요합니다.

몸통과 머리

이제 모델 안을 들여다보겠습니다. `named_children()`은 6장 6.4절의 `named_parameters()`처럼 이름과 함께 돌려주는데, 파라미터가 아니라 모델을 이루는 큰 부품(자식 모듈)을 차례로 꺼냅니다. 0으로 채운 가짜 사진 한 장을 부품마다 차례로 통과시키면서 모양이 어떻게 바뀌는지 찍어 보죠.

```
import torch
from torchvision import models

모델 = models.resnet18(weights=None)     # 구조만 볼 때는 가중치를 받지 않아도 된다
x = torch.zeros(1, 3, 224, 224)          # 사진 1장, 3색, 224 × 224
for 이름, 부품 in 모델.named_children():
    if 이름 == "fc":
        x = torch.flatten(x, 1)          # resnet18의 forward가 fc 바로 앞에서 하는 일
    x = 부품(x)
    개수 = sum(p.numel() for p in 부품.parameters())
    print(f"{이름:8} {type(부품).__name__:17} {개수:>10,}개 → {tuple(x.shape)}")
print(모델.fc)
```

```
conv1    Conv2d                9,408개 → (1, 64, 112, 112)
bn1      BatchNorm2d             128개 → (1, 64, 112, 112)
relu     ReLU                      0개 → (1, 64, 112, 112)
maxpool  MaxPool2d                 0개 → (1, 64, 56, 56)
layer1   Sequential           147,968개 → (1, 64, 56, 56)
layer2   Sequential           525,568개 → (1, 128, 28, 28)
layer3   Sequential         2,099,712개 → (1, 256, 14, 14)
layer4   Sequential         8,393,728개 → (1, 512, 7, 7)
avgpool  AdaptiveAvgPool2d          0개 → (1, 512, 1, 1)
fc       Linear               513,000개 → (1, 1000)
Linear(in_features=512, out_features=1000, bias=True)
```

`conv1`부터 `layer4`까지가 합성곱 층과 배치 정규화로 이루어진 부분입니다. 사진 `(1, 3, 224, 224)`가 들어가서 가로세로는 224 → 112 → 56 → 28 → 14 → 7로 줄고, 채널(특징의 종류)은 3 → 64 → 128 → 256 → 512로 늘어나요. `layer4`를 나오면 7 × 7 크기의 지도 512장이 됩니다. 9장 9.5절에서 본 "크기는 줄이고 채널은 늘리는" CNN의 전형적인 모양이고, `layer1`~`layer4` 안에는 9.8절의 잔차 블록이 두 개씩 들어 있어요.

`avgpool`은 9장 9.4절의 전역 평균 풀링으로, 7 × 7 지도마다 평균을 내서 숫자 하나로 줄입니다. 그러면 사진 한 장이 숫자 512개짜리 벡터가 되는데, 이것을 **특징 벡터**라고 불러요. 마지막 `fc`는 6장의 `nn.Linear(512, 1000)`로, 특징 512개를 보고 ImageNet 1,000개 범주의 점수를 냅니다. 파라미터 수 513,000개는 512 × 1,000 + 1,000이에요.

이 장에서는 `fc` 앞까지를 **몸통(backbone)**, `fc`를 **머리(head)** 라고 부르겠습니다. 몸통은 "사진을 보는 눈"이라 다른 문제에도 쓸모가 있지만, 머리는 ImageNet의 1,000개 범주에 딱 맞춰져 있어서 우리 문제에는 쓸 수 없어요. 그래서 전이 학습의 첫 단계는 언제나 **머리를 우리 문제에 맞는 새것으로 바꾸는 것**입니다.

`avgpool`의 이름에 붙은 `Adaptive`도 눈여겨보세요. 들어오는 지도의 크기가 7 × 7이든 4 × 4든 언제나 1 × 1로 줄여 줍니다. 덕분에 ResNet은 224 × 224가 아닌 사진도 받을 수 있어요. 10.4절에서 이 성질을 이용해 계산량을 줄입니다.

10.2 입력 전처리 맞추기

모델이 배운 기준 그대로

사전학습 모델을 쓸 때 가장 흔한 실수가 입력을 모델이 배울 때와 다르게 넣는 것입니다. 7장 7.6절에서 `Normalize`에 쓰는 평균과 표준편차는 학습 데이터에서 구한다고 했고, ImageNet으로 학습한 모델에는 `mean=[0.485, 0.456, 0.406]`을 그대로 쓴다고 예고했었죠. 모델은 이 기준으로 바꾼 숫자만 보며 자랐기 때문에, 다른 기준의 숫자를 넣으면 낯선 사진을 보는 것과 같습니다.

다행히 가중치 객체가 "나를 학습할 때 쓴 전처리"를 기억하고 있습니다.

```
import torch
from torchvision import datasets, models
from torchvision.transforms import v2

전처리 = models.ResNet18_Weights.DEFAULT.transforms()
print(전처리)

사진, 정답 = datasets.EuroSAT("data", download=True)[0]
print("원본:", 사진.size, 사진.mode)
x = 전처리(사진)
print("전처리 후:", tuple(x.shape), x.dtype)

# 같은 일을 7장의 v2 변환으로 풀어 쓰면
풀어쓰기 = v2.Compose([
    v2.ToImage(),
    v2.Resize(256),                # 짧은 변을 256으로
    v2.CenterCrop(224),            # 가운데 224 × 224
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
print("풀어 쓴 것과 같은가:", torch.allclose(풀어쓰기(사진), x, atol=1e-5))
```

```
ImageClassification(
    crop_size=[224]
    resize_size=[256]
    mean=[0.485, 0.456, 0.406]
    std=[0.229, 0.224, 0.225]
    interpolation=InterpolationMode.BILINEAR
```

```
)
원본: (64, 64) RGB
전처리 후: (3, 224, 224) torch.float32
풀어 쓴 것과 같은가: True
```

`weights.transforms()`가 돌려준 `ImageClassification`은 다섯 단계를 한 번에 합니다. 짧은 변을 256으로 늘리거나 줄이고(`resize_size`), 가운데 224 × 224를 잘라 내고(`crop_size`), 0~1 실수로 바꾼 뒤, ImageNet 평균과 표준편차로 정규화해요. 아래쪽에 7장의 v2 변환으로 같은 순서를 풀어 써 보니 결과가 같습니다. 숨은 마법이 아니라 7장에서 배운 변환을 차례로 이어 붙인 것뿐이에요.

여기서 쓸 데이터는 64 × 64짜리 작은 위성 사진입니다. 기본 전처리를 그대로 쓰면 64를 256으로 네 배 늘린 다음 가운데를 잘라서, 가장자리 12.5%를 버리고 224 × 224로 만들어요. 이렇게 크게 늘리는 게 꼭 필요한지는 10.4절에서 실험으로 따져 봅니다. 그때 크기는 바꾸더라도 **평균과 표준편차만은 반드시 ImageNet 값을 지킨다**는 것이 이 절의 요점입니다.

ImageNet의 눈으로 위성 사진 보기

전처리를 맞췄으니 사전학습 모델에 위성 사진을 그대로 보여 주면 뭐라고 할까요? 클래스마다 첫 사진 한 장씩, 확률이 높은 범주 셋을 찍어 보겠습니다.

```
import torch
from torchvision import datasets, models

가중치 = models.ResNet18_Weights.DEFAULT
모델 = models.resnet18(weights=가중치).eval()
전처리 = 가중치.transforms()
범주 = 가중치.meta["categories"]

원본 = datasets.EuroSAT("data", download=True)
정답들 = torch.tensor(원본.targets)
for k in [1, 3, 7, 8, 9]:                    # 숲, 도로, 주거지, 강, 바다·호수
    i = (정답들 == k).nonzero()[0].item()    # 그 클래스의 첫 사진
    with torch.no_grad():
        확률 = 모델(전처리(원본[i][0]).unsqueeze(0)).softmax(dim=1)[0]
    위 = 확률.topk(3)
```

```
    print(f"{원본.classes[k]:11} →", ", ".join(f"{범주[j]} {p:.0%}" for p, j in zip(위.values,
    ↪ 위.indices)))
```

```
Forest      → dugong 6%, velvet 4%, Petri dish 3%
Highway     → safety pin 11%, hand-held computer 10%, reel 4%
Residential → handkerchief 13%, stole 6%, terrapin 6%
River       → Petri dish 4%, Windsor tie 3%, hand-held computer 3%
SeaLake     → space shuttle 3%, nematode 3%, flagpole 3%
```

엉뚱하죠? 숲 사진을 보고 듀공(바다소)이라고 하고, 고속도로는 옷핀, 강은 페트리 접시라고 합니다. 확률도 가장 높은 것이 13%뿐이에요. 1,000개 범주에 "숲"이나 "고속도로"를 위에서 내려다본 사진이라는 범주가 없으니 당연한 결과입니다. 모델은 아는 단어 중에서 무늬가 가장 비슷한 것을 고를 수밖에 없어요. 고속도로를 옷핀이나 휴대용 컴퓨터라고 한 걸 보면, 길고 곧은 선과 회색 면을 보고 있다는 짐작은 할 수 있습니다.

이 결과가 전이 학습이 필요한 이유를 정확히 보여 줍니다. 이 모델은 **눈은 좋지만 우리 문제의 단어를 모릅니다.** 그러니 눈(몸통)은 남기고, 단어를 고르는 머리만 우리 데이터로 다시 가르치면 되겠죠. 그 전에 데이터부터 준비하겠습니다.

10.3 이 장의 데이터: EuroSAT 위성 사진

데이터 둘러보기

EuroSAT은 유럽 우주국의 지구 관측 위성 Sentinel-2가 찍은 사진을 64 × 64 조각으로 잘라 열 가지 땅의 쓰임새로 분류해 둔 데이터셋입니다. 사진 27,000장, 내려받는 크기는 약 90MB로 노트북에서 다루기 좋아요. 출처는 Helber 외(2019)의 논문 "EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification"이고, MIT 라이선스로 공개되어 있습니다. torchvision의 `datasets.EuroSAT`으로 바로 받을 수 있어요.

```
import torch
from torchvision import datasets
from torchvision.transforms import v2

원본 = datasets.EuroSAT("data", download=True)   # 처음 한 번은 약 90MB를 내려받는다
```

```
정답들 = torch.tensor(원본.targets)
print(f"사진 {len(원본):,}장, 크기 {원본[0][0].size}")
for k, (이름, 개수) in enumerate(zip(원본.classes, torch.bincount(정답들).tolist())):
    # 클래스마다 앞 100장의 평균 색(0~255)
    번호 = (정답들 == k).nonzero().flatten()[:100].tolist()
    평균색 = torch.stack([v2.functional.pil_to_tensor(원본[i][0]) for i in 번
↪   호]).float().mean(dim=(0, 2, 3))
    print(f"{k} {이름:20} {개수:,}장  평균 색 R·G·B = {평균색.round().int().tolist()}")
```

```
사진 27,000장, 크기 (64, 64)
0 AnnualCrop           3,000장  평균 색 R·G·B = [127, 119, 117]
1 Forest               3,000장  평균 색 R·G·B = [39, 64, 76]
2 HerbaceousVegetation 3,000장  평균 색 R·G·B = [100, 99, 104]
3 Highway              2,500장  평균 색 R·G·B = [89, 97, 103]
4 Industrial           2,500장  평균 색 R·G·B = [127, 129, 137]
5 Pasture              2,000장  평균 색 R·G·B = [62, 87, 92]
6 PermanentCrop        2,500장  평균 색 R·G·B = [118, 114, 113]
7 Residential          3,000장  평균 색 R·G·B = [95, 101, 110]
8 River                2,500장  평균 색 R·G·B = [66, 87, 97]
9 SeaLake              3,000장  평균 색 R·G·B = [45, 69, 86]
```

열 클래스의 뜻은 차례로 경작지(해마다 새로 심는 밭), 숲, 초지(풀과 덤불), 고속도로, 산업 지역, 목초지, 과수원(포도밭처럼 오래 두는 작물), 주거 지역, 강, 바다·호수입니다. 클래스마다 2,000~3,000장으로 조금 고르지 않지만, 이 장에서는 클래스마다 같은 수를 골라 쓸 거라 문제가 되지 않아요.

평균 색을 보면 숲(39, 64, 76)과 바다·호수(45, 69, 86)가 가장 어둡고, 산업 지역(127, 129, 137)과 경작지(127, 119, 117)가 밝습니다. 색만으로도 어느 정도 구별이 될 것 같지만, 숲과 바다·호수처럼 색이 비슷한 짝도 있고 고속도로·주거 지역·초지는 평균 색이 거의 같아요. 색이 아니라 **무늬와 모양**을 봐야 구별되는 클래스들입니다. 사전학습 모델의 눈이 필요한 곳이죠.

공통 준비 코드

이 장의 예제 대부분이 같은 데이터 준비와 학습 함수를 씁니다. 8장에서 `학습도구.py`를 만들었던 것처럼, 아래 내용을 `전이도구.py`로 저장해 두고 `import`해서 쓰겠습니다(1권 14장). 새로운 건 거의 없어요. 6장의 학습·평가 루프와 7장의 Dataset, transforms, 분할을 이 장의 데이터에 맞게 모은 것

입니다.

```
# 전이도구.py — 10장 예제가 함께 쓰는 준비 코드
import torch
from torch import nn
from torch.utils.data import DataLoader, Dataset
from torchvision import datasets
from torchvision.transforms import v2

IMAGENET_평균 = [0.485, 0.456, 0.406]      # 사전학습 모델이 배운 기준(10.2절)
IMAGENET_표준편차 = [0.229, 0.224, 0.225]
구간_시작 = {"학습": 0, "검증": 1000, "시험": 1500}   # 클래스마다 섞은 순서에서 겹치지 않는 구간


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")
    if torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")


def 유로샛_불러오기(구간: str, 클래스당: int) -> tuple[torch.Tensor, torch.Tensor]:
    """EuroSAT에서 클래스마다 같은 수를 골라 (사진 uint8 텐서, 정답)으로 돌려준다."""
    원본 = datasets.EuroSAT("data", download=True)
    정답들 = torch.tensor(원본.targets)
    번호 = []
    for k in range(len(원본.classes)):
        후보 = (정답들 == k).nonzero().flatten()
        섞기 = torch.randperm(len(후보), generator=torch.Generator().manual_seed(k))
        시작 = 구간_시작[구간]
        번호 += 후보[섞기[시작:시작 + 클래스당]].tolist()
    사진 = torch.stack([v2.functional.pil_to_tensor(원본[i][0]) for i in 번호])
    return 사진, 정답들[번호]
```

```
def 변환_만들기(크기: int = 112, 증강: bool = False) -> v2.Compose:
    """크기 맞추기 → (증강) → 0~1 실수 → ImageNet 기준으로 정규화."""
    단계 = [v2.Resize(크기)]
    if 증강:   # 위성 사진은 위아래가 따로 없으니 상하 뒤집기도 정답을 바꾸지 않는다
        단계 += [v2.RandomHorizontalFlip(), v2.RandomVerticalFlip()]
    단계 += [v2.ToDtype(torch.float32, scale=True), v2.Normalize(IMAGENET_평균, IMAGENET_표준편차)]
    return v2.Compose(단계)


class 사진_데이터셋(Dataset):
    """메모리에 올려 둔 사진 텐서를 꺼낼 때마다 변환한다(7장)."""

    def __init__(self, 사진: torch.Tensor, 정답: torch.Tensor, 변환: v2.Compose) -> None:
        self.사진, self.정답, self.변환 = 사진, 정답, 변환

    def __len__(self) -> int:
        return len(self.정답)

    def __getitem__(self, i: int) -> tuple[torch.Tensor, torch.Tensor]:
        return self.변환(self.사진[i]), self.정답[i]


def 로더_만들기(사진: torch.Tensor, 정답: torch.Tensor, 증강: bool = False,
              섞기: bool = False, 크기: int = 112) -> DataLoader:
    """사진 텐서로 DataLoader를 만든다. 섞는 순서는 씨앗으로 고정."""
    데이터셋 = 사진_데이터셋(사진, 정답, 변환_만들기(크기, 증강))
    return DataLoader(데이터셋, batch_size=64, shuffle=섞기,
                      generator=torch.Generator().manual_seed(0))


def 한_에폭_학습(모델: nn.Module, 로더: DataLoader, 옵티마이저: torch.optim.Optimizer,
                장치: torch.device) -> float:
    """한 바퀴 학습하고 평균 손실을 돌려준다(6장)."""
    모델.train()
    합, 개수 = 0.0, 0
    for x, y in 로더:
```

```
        x, y = x.to(장치), y.to(장치)
        옵티마이저.zero_grad()
        손실 = nn.functional.cross_entropy(모델(x), y)
        손실.backward()
        옵티마이저.step()
        합, 개수 = 합 + 손실.item() * len(y), 개수 + len(y)
    return 합 / 개수


@torch.no_grad()
def 정확도(모델: nn.Module, 로더: DataLoader, 장치: torch.device) -> float:
    """평가 모드에서 정확도를 잰다."""
    모델.eval()
    맞힘 = sum((모델(x.to(장치)).argmax(1).cpu() == y).sum().item() for x, y in 로더)
    return 맞힘 / len(로더.dataset)


@torch.no_grad()
def 특징_뽑기(몸통: nn.Module, 로더: DataLoader, 장치: torch.device) -> tuple[torch.Tensor,
↪ torch.Tensor]:
    """얼린 몸통에 사진을 한 번씩 통과시켜 특징 벡터를 모은다(10.4절)."""
    몸통.eval()
    특징, 정답 = zip(*[(몸통(x.to(장치)).cpu(), y) for x, y in 로더])
    return torch.cat(특징), torch.cat(정답)


def 머리_학습(특징: torch.Tensor, 정답: torch.Tensor, 에폭: int = 100) -> nn.Linear:
    """특징 벡터만으로 마지막 층 하나를 학습한다. 데이터가 작아 한 번에 다 넣는다."""
    torch.manual_seed(0)
    머리 = nn.Linear(특징.shape[1], int(정답.max()) + 1)
    옵티마이저 = torch.optim.Adam(머리.parameters(), lr=1e-3)
    for _ in range(에폭):
        옵티마이저.zero_grad()
        nn.functional.cross_entropy(머리(특징), 정답).backward()
        옵티마이저.step()
    return 머리
```

함수마다 짚어 보겠습니다.

- **유로샛_불러오기(구간, 클래스당)**: 클래스마다 사진 번호를 씨앗 고정으로 섞은 뒤, 섞인 순서에서 학습은 0번째부터, 검증은 1,000번째부터, 시험은 1,500번째부터 필요한 만큼 가져옵니다. 구간이 겹치지 않으니 학습·검증·시험에 같은 사진이 들어갈 일이 없어요. 사진 수가 많지 않아서 7장 7.1절의 원칙과 달리 처음에 전부 읽어 `uint8` 텐서로 메모리에 올려 둡니다. 클래스마다 씨앗을 `k`로 따로 쓴 덕분에, 학습 사진을 50장 고르든 100장 고르든 앞의 50장은 같은 사진이에요.
- **변환_만들기(크기, 증강)**: 크기 맞추기 → (학습용이면 증강) → 0~1 실수 → **ImageNet 기준 정규화**. 10.2절의 기본 전처리와 달리 가운데 자르기는 하지 않고, 크기는 기본 112로 정했습니다. 이유는 10.4절에서 실험으로 보여 드려요. 증강에는 상하 뒤집기를 넣었습니다. 7장 7.6절에서 "바꾼 그림에도 사람이 같은 정답을 붙일까?"를 먼저 물으라고 했죠. 위에서 내려다본 위성 사진은 위아래가 따로 없으니 뒤집어도 숲은 숲입니다.
- **사진_데이터셋과 로더_만들기**: 7장 7.8절에서 본 대로, 학습용과 검증용이 transform만 다른 별도의 데이터셋이 되게 했습니다. 섞는 순서도 `generator`로 고정했어요.
- **한_에폭_학습과 정확도**: 6장 6.11절과 같습니다. **정확도** 위의 `@torch.no_grad()`는 1권 10장 10.4절의 데코레이터로, 함수 전체를 5장 5.8절의 `with torch.no_grad():` 안에서 실행한 것과 같아요.
- **특징_뽑기와 머리_학습**: 10.4절에서 설명합니다.

학습·검증·시험으로 나눠 불러오기

준비 코드가 제대로 동작하는지 확인해 보겠습니다. 학습은 클래스당 50장, 검증과 시험은 클래스당 100장씩입니다.

```
import torch

from 전이도구 import 로더_만들기, 유로샛_불러오기

학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)
시험_사진, 시험_정답 = 유로샛_불러오기("시험", 100)
for 이름, 사진, 정답 in [("학습", 학습_사진, 학습_정답), ("검증", 검증_사진, 검증_정답), ("시험",
↪ 시험_사진, 시험_정답)]:
    print(f"{이름}: {tuple(사진.shape)} {사진.dtype}, 클래스별 {torch.bincount(정답).tolist()}")
```

```
print(f"학습 사진이 차지하는 메모리 {학습_사진.numel() * 학습_사진.element_size() /
↪  1024**2:.1f}MB")

x, y = next(iter(로더_만들기(학습_사진, 학습_정답, 증강=True, 섞기=True)))
print("배치 하나:", tuple(x.shape), x.dtype, y[:8].tolist())
print("채널별 평균:", [f"{v:.2f}" for v in x.mean(dim=(0, 2, 3)).tolist()],
      "표준편차:", [f"{v:.2f}" for v in x.std(dim=(0, 2, 3)).tolist()])
```

```
학습: (500, 3, 64, 64) torch.uint8, 클래스별 [50, 50, 50, 50, 50, 50, 50, 50, 50, 50]
검증: (1000, 3, 64, 64) torch.uint8, 클래스별 [100, 100, 100, 100, 100, 100, 100, 100, 100, 100]
시험: (1000, 3, 64, 64) torch.uint8, 클래스별 [100, 100, 100, 100, 100, 100, 100, 100, 100, 100]
학습 사진이 차지하는 메모리 5.9MB
배치 하나: (64, 3, 112, 112) torch.float32 [8, 9, 8, 8, 9, 5, 4, 3]
채널별 평균: ['-0.66', '-0.37', '-0.05'] 표준편차: ['0.86', '0.55', '0.44']
```

학습 500장, 검증 1,000장, 시험 1,000장이 클래스마다 고르게 나뉘었습니다. 일부러 학습 데이터를 **27,000장 중 500장만** 쓰는 거예요. 실제 현장에서는 이렇게 정답을 붙인 데이터가 몇백 장뿐인 경우가 흔하고, 전이 학습이 가장 빛나는 것도 그런 때입니다. 500장을 `uint8`로 담으니 5.9MB로 가볍네요.

배치 하나는 `(64, 3, 112, 112)`의 float32이고 정답 번호가 섞여서 나옵니다. 마지막 줄의 채널별 평균과 표준편차를 보세요. 정규화를 했는데도 평균이 0이 아니고(-0.66, -0.37, -0.05) 표준편차도 1이 아닙니다. 우리 데이터가 아니라 **ImageNet의 기준**으로 정규화했기 때문이에요. 위성 사진은 ImageNet의 평범한 사진보다 어둡고 빨간색이 적어서 이런 숫자가 나옵니다. 이게 틀린 걸까요? 10.4절에서 확인해 보겠습니다.

7장 7.8절의 Q&A에서 테스트 데이터는 처음부터 따로 두고 마지막에 한 번만 쓰라고 했죠. 이 장도 그렇게 합니다. 여러 방법을 비교하는 것은 모두 검증 데이터로 하고, 시험 데이터는 10.9절 실습의 마지막에 딱 한 번만 씁니다.

10.4 특징 추출: 눈은 빌리고 판단만 새로

몸통 얼리기

전이 학습의 첫 번째 방법은 **특징 추출(feature extraction)** 입니다. 몸통은 한 숫자도 고치지 않고 얼려 둔 채, 몸통이 뽑아 준 특징 벡터를 보고 답을 고르는 머리만 새로 학습하는 거예요. 6장 6.4절에서 `requires_grad_(False)`로 층을 얼려 봤던 것, 그리고 그때 "10장 전이 학습의 핵심 요령"이라고 예고했던 것이 바로 이 장면입니다.

```
import torch
from torch import nn
from torchvision import models

모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for p in 모델.parameters():
    p.requires_grad_(False)                # 몸통 전체를 얼린다(6장 6.4절)
모델.fc = nn.Linear(모델.fc.in_features, 10)  # 새로 만든 층은 얼어 있지 않다

전체 = sum(p.numel() for p in 모델.parameters())
학습_대상 = sum(p.numel() for p in 모델.parameters() if p.requires_grad)
print(f"전체 {전체:,}개 중 학습 대상 {학습_대상:,}개 ({학습_대상 / 전체:.3%})")
print("학습 대상 이름:", [이름 for 이름, p in 모델.named_parameters() if p.requires_grad])

# 가짜 배치로 한 번 거꾸로 계산해 보면 기울기가 어디에 생길까?
x = torch.randn(4, 3, 112, 112)
nn.functional.cross_entropy(모델(x), torch.tensor([0, 1, 2, 3])).backward()
print("conv1.weight.grad:", 모델.conv1.weight.grad)
print("fc.weight.grad 모양:", tuple(모델.fc.weight.grad.shape))

# 옵티마이저에는 학습 대상만 넘긴다
옵티마이저 = torch.optim.Adam(모델.fc.parameters(), lr=1e-3)
print("옵티마이저가 맡은 텐서 수:", len(옵티마이저.param_groups[0]["params"]))
```

```
전체 11,181,642개 중 학습 대상 5,130개 (0.046%)
학습 대상 이름: ['fc.weight', 'fc.bias']
conv1.weight.grad: None
fc.weight.grad 모양: (10, 512)
옵티마이저가 맡은 텐서 수: 2
```

순서가 중요합니다. 먼저 모든 파라미터를 얼리고, **그다음에** `fc`를 새 `nn.Linear(512, 10)`로 바꿨어요. 새로 만든 층의 파라미터는 기본값이 `requires_grad=True`라서 얼어 있지 않습니다. 순서를 거꾸로 하면 새 머리까지 얼어 버려서 아무것도 학습되지 않아요. `nn.Linear(모델.fc.in_features, 10)`처럼 입력 크기를 원래 층에서 읽어 오면 512를 외우지 않아도 됩니다.

전체 1,118만여 개 중 학습할 파라미터는 머리의 5,130개(512 × 10 + 10), **0.046%** 뿐입니다. 가짜 배치로 `backward()`를 해 보면 얼린 `conv1`의 기울기는 `None`이고 머리에만 `(10, 512)` 기울기가 생겨요. 5장에서 본 대로 `requires_grad=False`인 텐서는 기울기 계산에서 빠지기 때문입니다. 옵티마이저에도 `모델.fc.parameters()`만 넘겼습니다. 얼린 파라미터까지 넘겨도 기울기가 없어서 고쳐지진 않지만, 맡을 필요 없는 일을 맡기지 않는 편이 깔끔해요.

특징을 한 번만 뽑아 두기

이제 학습하면 되는데, 한 가지 꾀를 낼 수 있습니다. 몸통이 얼어 있으니 **같은 사진은 몇 번을 넣어도 같은 특징이 나옵니다.** 그렇다면 에폭마다 무거운 몸통을 다시 돌릴 필요가 없겠죠. 사진마다 특징을 한 번만 뽑아 두고, 그 특징으로 머리만 학습하면 됩니다.

```
import torch
from torch import nn
from torchvision import models

from 전이도구 import 로더_만들기, 유로샛_불러오기, 장치_고르기, 특징_뽑기

장치 = 장치_고르기()
몸통 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
몸통.fc = nn.Identity()                    # 마지막 층을 "그대로 통과"로 바꾸면 특징 추출기
몸통.requires_grad_(False)
몸통.to(장치)

학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)
학습_특징, 학습_정답 = 특징_뽑기(몸통, 로더_만들기(학습_사진, 학습_정답), 장치)
검증_특징, 검증_정답 = 특징_뽑기(몸통, 로더_만들기(검증_사진, 검증_정답), 장치)
print("특징:", tuple(학습_특징.shape), tuple(검증_특징.shape))

# 이제 학습은 특징 512개 → 10클래스, 층 하나뿐이다
torch.manual_seed(0)
```

```
머리 = nn.Linear(512, 10)
옵티마이저 = torch.optim.Adam(머리.parameters(), lr=1e-3)
for 에폭 in range(1, 101):
    옵티마이저.zero_grad()
    손실 = nn.functional.cross_entropy(머리(학습_특징), 학습_정답)
    손실.backward()
    옵티마이저.step()
    if 에폭 in (1, 10, 25, 50, 100):
        with torch.no_grad():
            학습_정확도 = (머리(학습_특징).argmax(1) == 학습_정답).float().mean()
            검증_정확도 = (머리(검증_특징).argmax(1) == 검증_정답).float().mean()
        print(f"에폭 {에폭:3}: 손실 {손실.item():.4f}, 학습 {학습_정확도:.1%}, 검증 {검증_정확
        ↪ 도:.1%}")
```

```
특징: (500, 512) (1000, 512)
에폭   1: 손실 2.3806, 학습 13.8%, 검증 13.9%
에폭  10: 손실 1.6413, 학습 70.6%, 검증 61.0%
에폭  25: 손실 0.9267, 학습 88.6%, 검증 82.7%
에폭  50: 손실 0.4790, 학습 94.2%, 검증 86.4%
에폭 100: 손실 0.2296, 학습 98.2%, 검증 87.0%
```

`몸통.fc = nn.Identity()`가 요령입니다. `nn.Identity`는 받은 것을 그대로 돌려주는 빈 층이에요. 마지막 층을 이걸로 바꾸면 모델이 1,000개 점수 대신 **특징 벡터 512개**를 내는 특징 추출기가 됩니다. 이렇게 학습 500장과 검증 1,000장의 특징을 한 번씩 뽑아 `(500, 512)`, `(1000, 512)` 표로 만들었어요. 그 뒤의 학습은 숫자 512개짜리 표로 10클래스를 맞히는 문제라, 6장에서 처음 만든 분류기만큼이나 간단합니다. 데이터가 500줄뿐이니 배치로 나누지 않고 한꺼번에 넣었어요. `전이도구.py`의 `머리_학습`이 이 과정을 함수로 만든 것입니다.

100에폭 뒤 검증 정확도는 **87.0%** 입니다. 무작위로 찍으면 10%인 문제에서, 학습 사진 500장과 층 하나만으로 열 장 중 아홉 장 가까이를 맞혔어요. 학습 정확도는 98.2%라서 과적합도 조금 보이지만(8장), 검증 정확도는 50에폭 이후에도 내려가지 않았습니다.

계산량을 따져 보면 이 꾀의 효과가 분명합니다. 몸통을 매번 돌리면 100에폭 동안 몸통을 500 × 100 = 50,000번 통과시켜야 하지만, 특징을 뽑아 두면 학습·검증을 합쳐 1,500번이면 끝나요. 남은 일은 작은 행렬 곱뿐이라 CPU로도 순식간입니다. 다만 이 꾀에는 대가가 하나 있어요. 사진을 미리 특징으로 바꿔 두었으니 **데이터 증강을 쓸 수 없습니다.** 증강은 꺼낼 때마다 그림을 새로 바꾸는 것

인데(7장 7.6절), 특징은 한 번 뽑으면 고정되니까요. 데이터가 아주 적어서 증강이 꼭 필요하면 몸통을 매번 돌리는 쪽을 택합니다.

얼렸는데도 바뀌는 것: 배치 정규화

여기서 많은 사람이 빠지는 함정이 하나 있습니다. 몸통을 매번 돌리는 방식으로 학습하면, 학습 루프는 습관대로 `모델.train()`을 부르겠죠. 파라미터를 모두 얼렸으니 아무것도 바뀌지 않을 것 같지만, 정말 그럴까요?

```
import torch
from torch import nn
from torchvision import models

from 전이도구 import 로더_만들기, 유로샛_불러오기

모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
모델.requires_grad_(False)                  # 전부 얼렸다
모델.fc = nn.Identity()
사진, 정답 = 유로샛_불러오기("학습", 10)
x, _ = next(iter(로더_만들기(사진, 정답)))

모델.eval()
with torch.no_grad():
    전 = 모델(x)
print("학습 대상 파라미터:", sum(p.numel() for p in 모델.parameters() if p.requires_grad))
print("bn1 기록 평균(전):", [f"{v:.3f}" for v in 모델.bn1.running_mean[:4].tolist()])

모델.train()                                # 습관대로 train 모드로 몇 배치 돌리면
with torch.no_grad():
    for _ in range(5):
        모델(x)
print("bn1 기록 평균(후):", [f"{v:.3f}" for v in 모델.bn1.running_mean[:4].tolist()])

모델.eval()
with torch.no_grad():
    후 = 모델(x)
유사도 = nn.functional.cosine_similarity(전, 후, dim=1)
```

```
print(f"같은 사진의 특징이 바뀌었나? 최대 차이 {(전 - 후).abs().max():.3f}, 코사인 유사도 평균 {유
↪ 사도.mean():.3f}")

# 해결: train 모드로 돌리더라도 배치 정규화 층만은 평가 모드로 묶어 둔다
def 배치정규화_고정(모델: nn.Module) -> None:
    for 층 in 모델.modules():
        if isinstance(층, nn.BatchNorm2d):
            층.eval()

새모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
새모델.fc = nn.Identity()
새모델.train()
배치정규화_고정(새모델)
with torch.no_grad():
    for _ in range(5):
        새모델(x)
print("고정한 bn1 기록 평균:", [f"{v:.3f}" for v in 새모델.bn1.running_mean[:4].tolist()])
```

```
학습 대상 파라미터: 0
bn1 기록 평균(전): ['0.003', '-0.026', '0.000', '-0.085']
bn1 기록 평균(후): ['-0.006', '0.070', '-0.000', '0.221']
같은 사진의 특징이 바뀌었나? 최대 차이 2.841, 코사인 유사도 평균 0.945
고정한 bn1 기록 평균: ['0.003', '-0.026', '0.000', '-0.085']
```

학습 대상 파라미터는 0개인데 `bn1`의 기록 평균이 바뀌었습니다. 8장 8.4절에서 본 대로, 배치 정규화는 **학습 모드에서 배치의 평균과 분산을 이동 평균으로 몰래 기록**합니다. 이 기록(`running_mean`, `running_var`)은 기울기로 고치는 파라미터가 아니라 버퍼라서 `requires_grad`와 상관없이 바뀌어요. 그 결과 같은 사진의 특징 벡터가 최대 2.841만큼 달라졌고, 방향을 비교한 코사인 유사도(4장 4.11절)도 평균 0.945로 떨어졌습니다. ImageNet 사진 120만 장으로 다듬어 둔 기록을 위성 사진 몇 배치로 덮어써 버린 거예요. 몸통의 "눈"이 조금 흐려진 셈입니다.

해결은 간단합니다. 학습 모드로 바꾼 직후에 배치 정규화 층만 다시 평가 모드로 돌려 두면 돼요. `배치정규화_고정`이 모든 부품을 돌며 `nn.BatchNorm2d`인 것만 `eval()`로 바꾸자 기록 평균이 처음 값 그대로 남았습니다. 특징 추출 방식에서는 `특징_뽑기`가 처음부터 `몸통.eval()`을 부르니 이 문제가 생기지 않아요.

사실 이 함정은 이 책을 쓰면서 직접 밟았습니다. 아래 "입력 크기" 실험에서 계산량을 세려고 모델에 가짜 사진을 한 번 통과시켰는데, 그 전에 `eval()`을 부르지 않았더니 0으로 채운 사진 한 장이 배치 정규화 기록을 흔들어서 검증 정확도가 87.0%에서 86.2%로 떨어졌어요. 파라미터를 얼리는 것(`requires_grad`)과 모드를 정하는 것(`train`/`eval`)은 **서로 다른 스위치**라는 점을 꼭 기억하세요.

전처리를 틀리면

10.2절에서 "평균과 표준편차만은 반드시 ImageNet 값을 지키라"고 했습니다. 정말 그렇게까지 중요할까요? 같은 특징 추출 실험을 전처리만 바꿔 네 번 해 보겠습니다. 우리 데이터에서 직접 구한 평균·표준편차로 정규화하는 것도 후보에 넣었어요. 7장에서는 그게 정석이었으니까요.

```
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
from torchvision import models
from torchvision.transforms import v2

from 전이도구 import 유로샛_불러오기, 장치_고르기, 머리_학습, 특징_뽑기

장치 = 장치_고르기()
몸통 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
몸통.fc = nn.Identity()
몸통.to(장치)
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)

# 우리 데이터(학습 사진)에서 직접 구한 채널별 평균·표준편차
실수 = 학습_사진.float() / 255
우리_평균, 우리_표준편차 = 실수.mean(dim=(0, 2, 3)).tolist(), 실수.std(dim=(0, 2, 3)).tolist()
print("우리 데이터 평균:", [f"{v:.3f}" for v in 우리_평균], "표준편차:", [f"{v:.3f}" for v in 우리_
↪ 표준편차])

크기_맞추기 = [v2.Resize(112), v2.ToDtype(torch.float32, scale=True)]
방법들 = {
    "ImageNet 기준 정규화": v2.Compose(크기_맞추기 + [v2.Normalize([0.485, 0.456, 0.406], [0.229,
    ↪ 0.224, 0.225])]),
    "우리 데이터 기준 정규화": v2.Compose(크기_맞추기 + [v2.Normalize(우리_평균, 우리_표준편차)]),
```

```
    "정규화 빼먹기(0~1)": v2.Compose(크기_맞추기),
    "scale 빼먹기(0~255)": v2.Compose([v2.Resize(112), v2.ToDtype(torch.float32),
                                       v2.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])]),
}


def 로더(사진: torch.Tensor, 정답: torch.Tensor, 변환: v2.Compose) -> DataLoader:
    return DataLoader(TensorDataset(변환(사진), 정답), batch_size=64)   # 묶음 전체를 한 번에 변환


for 이름, 변환 in 방법들.items():
    학습_특징, _ = 특징_뽑기(몸통, 로더(학습_사진, 학습_정답, 변환), 장치)
    검증_특징, _ = 특징_뽑기(몸통, 로더(검증_사진, 검증_정답, 변환), 장치)
    머리 = 머리_학습(학습_특징, 학습_정답)
    with torch.no_grad():
        정확도 = (머리(검증_특징).argmax(1) == 검증_정답).float().mean()
    print(f"검증 정확도 {정확도:6.1%}  {이름}")
```

```
우리 데이터 평균: ['0.341', '0.379', '0.406'] 표준편차: ['0.197', '0.132', '0.113']
검증 정확도  87.0%  ImageNet 기준 정규화
검증 정확도  84.4%  우리 데이터 기준 정규화
검증 정확도  80.1%  정규화 빼먹기(0~1)
검증 정확도  43.1%  scale 빼먹기(0~255)
```

결과가 분명합니다.

- **ImageNet 기준 정규화(87.0%)** 가 가장 좋습니다. 앞 절의 결과와 같은 숫자예요.
- **우리 데이터 기준 정규화(84.4%)** 는 2.6%p 낮습니다. 7장에서는 우리 데이터의 평균·표준편차를 쓰는 게 정석이었지만, 그건 모델을 **처음부터** 학습할 때 이야기예요. 사전학습 모델은 ImageNet 기준으로 바뀐 숫자를 보며 자랐으니, 우리 데이터의 평균을 0에 맞춰 버리면 오히려 모델이 아는 세상과 어긋납니다. 10.3절에서 본 "평균이 0이 아닌" 배치가 틀린 게 아니었던 거죠.
- **정규화를 빼먹으면(80.1%)** 7%p 가까이 떨어집니다. 오류는 하나도 나지 않아서 알아채기가 어려워요.
- **`scale=True`를 빼먹으면(43.1%)** 반토막이 납니다. 0~1이어야 할 값이 0~255로 들어가서 모델이 생전 처음 보는 크기의 숫자를 받은 거예요. 7장 7.6절에서 "자주 하는 실수"라고 했던

바로 그것입니다.

전처리가 틀려도 코드는 아무 오류 없이 돌아가고 정확도만 조용히 떨어집니다. 사전학습 모델을 쓰기 전에는 `weights.transforms()`를 출력해 보고, 내 변환이 같은 평균·표준편차를 쓰는지 꼭 확인하세요.

입력 크기는 얼마로 할까

10.1절에서 ResNet은 `AdaptiveAvgPool2d` 덕분에 224 × 224가 아닌 사진도 받을 수 있다고 했습니다. 그럼 64 × 64 위성 사진을 꼭 224로 늘려야 할까요? 크기를 바꿔 가며 정확도와 계산량을 함께 재 보겠습니다. 시간은 컴퓨터마다 다르니, 대신 사진 한 장을 처리하는 데 드는 연산 수를 세었어요.

```
import torch
from torch import nn
from torch.utils.flop_counter import FlopCounterMode
from torchvision import models

from 전이도구 import 로더_만들기, 유로샛_불러오기, 장치_고르기, 머리_학습, 특징_뽑기

장치 = 장치_고르기()
몸통 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
몸통.fc = nn.Identity()
몸통.eval()                                  # 세는 동안에도 배치 정규화 기록이 바뀌지 않게
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)

for 크기 in [64, 112, 160, 224]:
    with FlopCounterMode(display=False) as 계수기:      # 사진 한 장에 드는 곱셈·덧셈 수 세기
        몸통.cpu()(torch.zeros(1, 3, 크기, 크기))
    몸통.to(장치)
    학습_특징, _ = 특징_뽑기(몸통, 로더_만들기(학습_사진, 학습_정답, 크기=크기), 장치)
    검증_특징, _ = 특징_뽑기(몸통, 로더_만들기(검증_사진, 검증_정답, 크기=크기), 장치)
    머리 = 머리_학습(학습_특징, 학습_정답)
    with torch.no_grad():
        정확도 = (머리(검증_특징).argmax(1) == 검증_정답).float().mean()
    print(f"{크기:3} × {크기:3}: 한 장에 {계수기.get_total_flops() / 1e9:5.2f} GFLOP, 검증 정확도
    ↪ {정확도:.1%}")
```

```
 64 ×  64: 한 장에  0.30 GFLOP, 검증 정확도 84.3%
112 × 112: 한 장에  0.97 GFLOP, 검증 정확도 87.0%
160 × 160: 한 장에  1.85 GFLOP, 검증 정확도 89.1%
224 × 224: 한 장에  3.63 GFLOP, 검증 정확도 89.3%
```

`FlopCounterMode`는 `with` 블록 안에서 일어난 곱셈과 덧셈의 수를 세어 주는 도구입니다(곱셈과 덧셈을 따로 셉니다). 결과를 보면 크기를 키울수록 정확도가 오르지만 계산량은 그보다 훨씬 빨리 늘어요. 112에서 224로 키우면 정확도는 87.0%에서 89.3%로 2.3%p 오르는 대신 계산량은 0.97에서 3.63 GFLOP로 **3.7배**가 됩니다. 가로세로를 두 배로 하면 칸 수가 네 배가 되니 계산량도 거의 네 배가 되는 거예요.

사진을 키우면 정확도가 오르는 이유는, ImageNet 모델이 224 × 224 사진 속 물체 크기에 익숙하기 때문입니다. 64 × 64를 그대로 넣으면 `layer4`를 지난 특징 지도가 2 × 2밖에 안 돼서 무늬가 너무 빨리 뭉개져요. 어느 크기를 고를지는 정확도와 계산 비용의 저울질입니다. 이 장은 노트북에서 수많은 실험을 돌려야 하니 **112**로 정했어요. 최종 모델을 만들 때는 160이나 224로 키워서 몇 %p를 더 얻는 것도 좋은 선택입니다.

10.5 처음부터 학습하면 어떨까

전이 학습이 정말 그만한 값어치가 있는지 확인하려면 비교 대상이 필요합니다. **같은 ResNet18 구조**를 `weights=None`으로 무작위 가중치에서 시작해, 같은 500장으로 처음부터 학습해 보겠습니다. 이번에는 몸통 전체를 학습해야 하니 증강을 켜고 몸통을 매번 돌립니다. 학습할 양이 많아 GPU(`장치_고르기()`)를 썼어요.

```
import torch
from torchvision import models

from 전이도구 import 로더_만들기, 유로샛_불러오기, 장치_고르기, 정확도, 한_에폭_학습

장치 = 장치_고르기()
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)
학습_로더 = 로더_만들기(학습_사진, 학습_정답, 증강=True, 섞기=True)
학습_평가_로더 = 로더_만들기(학습_사진, 학습_정답)          # 증강 없이 학습 정확도를 잴 때
검증_로더 = 로더_만들기(검증_사진, 검증_정답)
```

```
torch.manual_seed(0)
모델 = models.resnet18(weights=None, num_classes=10).to(장치)   # 같은 구조, 무작위 가중치
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
for 에폭 in range(1, 31):
    손실 = 한_에폭_학습(모델, 학습_로더, 옵티마이저, 장치)
    if 에폭 % 5 == 0:
        print(f"에폭 {에폭:2}: 손실 {손실:.3f}, 학습 {정확도(모델, 학습_평가_로더, 장치):.1%}, "
              f"검증 {정확도(모델, 검증_로더, 장치):.1%}")
```

```
에폭  5: 손실 0.928, 학습 37.0%, 검증 34.2%
에폭 10: 손실 0.683, 학습 54.4%, 검증 46.8%
에폭 15: 손실 0.628, 학습 66.6%, 검증 54.2%
에폭 20: 손실 0.589, 학습 72.8%, 검증 63.4%
에폭 25: 손실 0.653, 학습 85.4%, 검증 69.9%
에폭 30: 손실 0.396, 학습 82.4%, 검증 68.1%
```

`models.resnet18(weights=None, num_classes=10)`처럼 `num_classes`를 주면 처음부터 머리가 10칸짜리인 모델을 만들어 줍니다. 무작위 가중치이니 머리를 바꿀 필요가 없어요.

30에폭 동안 1,100만 개의 파라미터를 모두 학습했는데 검증 정확도는 **68.1%** 입니다. 25에폭의 69.9%가 가장 높았고요. 앞 절에서 머리 5,130개만 학습한 특징 추출이 87.0%였으니 20%p 가까이 뒤집니다. 학습 정확도(82.4%)와 검증 정확도 사이의 차이도 크죠. 사진 500장으로는 1,100만 개의 숫자를 제대로 정하기에 정보가 턱없이 모자라서, 모델이 사진을 보는 눈을 스스로 기르지 못하고 학습 사진의 특징을 외우는 쪽으로 기운 거예요.

에폭을 더 늘리면 조금은 오르겠지만, 그만큼 시간이 들고 8장의 과적합 대책을 모두 동원해야 합니다. 반면 사전학습 모델은 ImageNet에서 이미 그 눈을 길러 왔어요. 사진 120만 장을 학습한 경험을 **공짜로** 물려받는 것, 이것이 전이 학습의 힘입니다.

Q

Q. 위성 사진은 ImageNet의 사진과 전혀 다른데 왜 도움이 되나요? 좋은 질문입니다. ImageNet에는 위에서 내려다본 땅 사진이 거의 없어요. 그래도 몸통의 앞부분이 배운 가장자리, 모서리, 색의 변화, 질감 같은 특징은 어떤 사진에나 있는 기본 재료라서 그대로 쓸모가 있습니다. 뒷부분으로 갈수록 "개의 귀", "자동차 바퀴"처럼 ImageNet에 특화된 특징을 배우는데, 이런 부분은 우리 데이터에 맞게 고쳐 주면 더 좋아지겠죠. 다음 절의

미세조정이 바로 그 일을 합니다. 다만 의료 영상이나 전파 신호처럼 일반 사진과 성질이 아주 다른 데이터에서는 효과가 작을 수 있으니, 이 장처럼 처음부터 학습한 결과와 비교해 보는 습관을 들이세요. Q

10.6 미세조정: 몸통까지 조금씩 고치기

학습률이 전부입니다

특징 추출은 몸통을 한 숫자도 고치지 않았습니다. 그런데 몸통도 우리 데이터에 맞게 **조금** 고칠 수 있다면 더 좋아지지 않을까요? 사전학습 가중치에서 출발해 모델 전체를 학습하는 방법을 **미세조정**(fine-tuning)이라고 합니다. 이름의 "미세"가 중요해요. 처음부터 학습할 때처럼 크게 고치면 애써 물려받은 눈이 망가지니, 아주 조심스럽게 고쳐야 합니다. 학습률만 바꿔 가며 확인해 보겠습니다.

```
import torch
from torch import nn
from torchvision import models

from 전이도구 import 로더_만들기, 유로샛_불러오기, 장치_고르기, 정확도, 한_에폭_학습

장치 = 장치_고르기()
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)
학습_로더 = 로더_만들기(학습_사진, 학습_정답, 증강=True, 섞기=True)
검증_로더 = 로더_만들기(검증_사진, 검증_정답)


def 사전학습_모델(클래스_수: int = 10) -> nn.Module:
    """ImageNet 가중치를 불러오고 마지막 층만 새것으로 바꾼다. 아무것도 얼리지 않는다."""
    모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    모델.fc = nn.Linear(모델.fc.in_features, 클래스_수)
    return 모델


for 학습률 in [1e-2, 1e-3, 1e-4]:
    torch.manual_seed(0)
```

```
    모델 = 사전학습_모델().to(장치)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=학습률)
    기록 = []
    for 에폭 in range(8):
        한_에폭_학습(모델, 학습_로더, 옵티마이저, 장치)
        기록.append(정확도(모델, 검증_로더, 장치))
    print(f"lr={학습률:g}: 에폭별 검증 정확도", " ".join(f"{a:.0%}" for a in 기록))
```

```
lr=0.01: 에폭별 검증 정확도 10% 17% 10% 16% 15% 23% 26% 22%
lr=0.001: 에폭별 검증 정확도 45% 50% 66% 65% 81% 82% 84% 85%
lr=0.0001: 에폭별 검증 정확도 60% 85% 89% 91% 92% 92% 93% 93%
```

같은 모델, 같은 데이터인데 학습률에 따라 결과가 완전히 다릅니다.

- **lr=0.01**: 정확도가 10~26%를 오갑니다. 무작위로 찍는 것(10%)과 크게 다르지 않아요. 첫 번째 고치기에서 큰 걸음으로 가중치를 휘저어 ImageNet에서 배운 것을 거의 다 잊어버린 겁니다. 처음부터 학습하는 것보다도 못한 결과예요.
- **lr=0.001**: 6장에서 Adam의 기본값으로 썼던 학습률입니다. 처음 두 에폭은 45~50%로 흔들리다가 8에폭에 85%까지 올라왔지만, 특징 추출(87.0%)보다도 낮아요. 처음 몇 걸음이 너무 커서 물려받은 눈을 한 번 흐려 놓고 다시 배우느라 시간이 걸린 겁니다.
- **lr=0.0001**: 첫 에폭부터 60%, 둘째 에폭에 85%, 넷째 에폭에 91%를 넘어 **93%** 에 이릅니다. 특징 추출보다 6%p 높아요.

미세조정의 학습률은 처음부터 학습할 때보다 **10배쯤 작게** 잡는 것이 출발점입니다. 6장 6.11절에서 "옵티마이저의 종류보다 학습률이 결과를 더 크게 좌우할 때가 많다"고 했는데, 미세조정에서는 그 말이 더 무겁습니다.

층마다 다른 학습률

그런데 고민이 하나 생깁니다. 몸통은 이미 잘 배웠으니 조심스럽게(작은 학습률) 고쳐야 하지만, 새로 붙인 머리는 무작위 숫자에서 시작하니 크게(큰 학습률) 배워야 하거든요. 한 모델 안에서 학습률을 다르게 줄 수 있을까요? 옵티마이저에 파라미터를 **그룹으로 나눠** 넘기면 됩니다.

```
import torch
from torch import nn
```

```
from torchvision import models

from 전이도구 import 로더_만들기, 유로샛_불러오기, 장치_고르기, 정확도, 한_에폭_학습

모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
모델.fc = nn.Linear(512, 10)

# 몸통과 머리를 나눠 학습률을 따로 준다
몸통_파라미터 = [p for 이름, p in 모델.named_parameters() if not 이름.startswith("fc.")]
옵티마이저 = torch.optim.Adam([
    {"params": 몸통_파라미터, "lr": 1e-4},          # 이미 잘 배운 부분은 조심스럽게
    {"params": 모델.fc.parameters(), "lr": 1e-3},   # 처음 배우는 머리는 크게
])
for i, 그룹 in enumerate(옵티마이저.param_groups):
    개수 = sum(p.numel() for p in 그룹["params"])
    print(f"그룹 {i}: 텐서 {len(그룹['params'])}개, 파라미터 {개수:,}개, lr={그룹['lr']:g}")

# 같은 조건에서 "전부 1e-4"와 비교한다
장치 = 장치_고르기()
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)
학습_로더 = 로더_만들기(학습_사진, 학습_정답, 증강=True, 섞기=True)
검증_로더 = 로더_만들기(검증_사진, 검증_정답)

for 방법 in ["전부 1e-4", "몸통 1e-4 + 머리 1e-3"]:
    torch.manual_seed(0)
    모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    모델.fc = nn.Linear(512, 10)
    모델.to(장치)
    if 방법 == "전부 1e-4":
        옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-4)
    else:
        몸통_파라미터 = [p for 이름, p in 모델.named_parameters() if not 이름.startswith("fc.")]
        옵티마이저 = torch.optim.Adam([{"params": 몸통_파라미터, "lr": 1e-4},
                                       {"params": 모델.fc.parameters(), "lr": 1e-3}])
    기록 = []
```

```
    for 에폭 in range(8):
        한_에폭_학습(모델, 학습_로더, 옵티마이저, 장치)
        기록.append(정확도(모델, 검증_로더, 장치))
    print(f"{방법}: 에폭별 검증 정확도", " ".join(f"{a:.1%}" for a in 기록))
```

```
그룹 0: 텐서 60개, 파라미터 11,176,512개, lr=0.0001
그룹 1: 텐서 2개, 파라미터 5,130개, lr=0.001
전부 1e-4: 에폭별 검증 정확도 61.7% 84.0% 89.4% 90.6% 92.2% 92.7% 92.8% 92.6%
몸통 1e-4 + 머리 1e-3: 에폭별 검증 정확도 66.7% 85.1% 87.7% 90.4% 91.8% 92.8% 93.2% 93.4%
```

옵티마이저에 파라미터 목록 대신 **딕셔너리의 목록**을 넘겼습니다. 딕셔너리마다 `"params"`에 파라미터를, `"lr"`에 그 그룹만의 학습률을 적어요. 몸통 파라미터는 이름이 `fc.`로 시작하지 않는 것들을 골라 모았습니다. 출력의 그룹 0이 몸통(텐서 60개, 파라미터 11,176,512개, lr=0.0001), 그룹 1이 머리(텐서 2개, 5,130개, lr=0.001)예요. 8장 8.7절에서 스케줄러가 바꾸는 학습률을 `옵티마이저.param_groups[0]["lr"]`에서 꺼냈던 것 기억나세요? 그때 `[0]`이 붙었던 이유가 이것입니다. 옵티마이저는 원래 그룹 목록을 들고 있고, 그냥 `모델.parameters()`를 넘기면 그룹이 하나뿐인 거였어요. 스케줄러도 그룹마다 학습률을 따로 조절해 줍니다.

비교 결과, 머리에 큰 학습률을 준 쪽이 첫 에폭부터 앞섭니다(61.7% → 66.7%). 머리가 빨리 자리를 잡으니 몸통으로 흘러가는 기울기도 처음부터 쓸모 있는 방향을 가리키기 때문이에요. 8에폭 뒤에는 92.6%와 93.4%로 차이가 0.8%p입니다. 6장에서 말했듯 이 정도 차이는 씨앗만 바꿔도 흔들릴 수 있으니 "언제나 더 낫다"고 단정할 수는 없어요. 다만 머리가 무작위에서 시작한다는 사정을 생각하면 그룹을 나누는 편이 자연스럽고, 실무에서도 흔히 쓰는 출발점입니다. 이 장의 남은 미세조정은 모두 이 설정을 씁니다.

어디까지 풀까

특징 추출(전부 얼리기)과 미세조정(전부 풀기) 사이에는 중간 단계가 있습니다. 입력에 가까운 앞부분은 어떤 사진에나 통하는 기본 특징을 배우고, 뒷부분일수록 ImageNet에 특화된 특징을 배운다고 했죠. 그렇다면 **뒷부분만 풀어서** 고쳐도 되지 않을까요? 뒤에서부터 푸는 부분을 하나씩 늘려 보겠습니다.

```
import torch
from torch import nn
```

```
from torchvision import models

from 전이도구 import 로더_만들기, 유로샛_불러오기, 장치_고르기, 정확도

장치 = 장치_고르기()
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)
학습_로더 = 로더_만들기(학습_사진, 학습_정답, 증강=True, 섞기=True)
검증_로더 = 로더_만들기(검증_사진, 검증_정답)

순서 = ["conv1", "bn1", "layer1", "layer2", "layer3", "layer4"]   # 입력에 가까운 쪽부터
for 풀_개수 in [0, 1, 2, 4, 6]:                                    # 뒤에서부터 몇 부분을 풀까
    얼린_부분, 풀_부분 = 순서[:len(순서) - 풀_개수], 순서[len(순서) - 풀_개수:]
    torch.manual_seed(0)
    모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    for 이름 in 얼린_부분:
        모델.get_submodule(이름).requires_grad_(False)
    모델.fc = nn.Linear(512, 10)
    모델.to(장치)
    몸통_파라미터 = [p for 이름, p in 모델.named_parameters() if p.requires_grad and not 이
↪   름.startswith("fc.")]
    옵티마이저 = torch.optim.Adam([{"params": 몸통_파라미터, "lr": 1e-4},
                                {"params": 모델.fc.parameters(), "lr": 1e-3}])
    for 에폭 in range(8):
        모델.train()
        for 이름 in 얼린_부분:                  # 얼린 부분은 배치 정규화 기록도 그대로 둔다(10.4절)
            모델.get_submodule(이름).eval()
        for x, y in 학습_로더:
            x, y = x.to(장치), y.to(장치)
            옵티마이저.zero_grad()
            nn.functional.cross_entropy(모델(x), y).backward()
            옵티마이저.step()
    학습_대상 = sum(p.numel() for p in 모델.parameters() if p.requires_grad)
    print(f"학습 대상 {학습_대상:>10,}개, 검증 {정확도(모델, 검증_로더, 장치):.1%} ← 푼 부분: {',
↪    '.join(풀_부분) or '없음(머리만)'}")
```

```
학습 대상      5,130개, 검증 83.6% ← 푼 부분: 없음(머리만)
학습 대상  8,398,858개, 검증 91.4% ← 푼 부분: layer4
학습 대상 10,498,570개, 검증 92.7% ← 푼 부분: layer3, layer4
학습 대상 11,172,106개, 검증 93.1% ← 푼 부분: layer1, layer2, layer3, layer4
학습 대상 11,181,642개, 검증 93.1% ← 푼 부분: conv1, bn1, layer1, layer2, layer3, layer4
```

`모델.get_submodule("layer4")`는 이름으로 부품을 꺼내는 메서드입니다. 6장의 `requires_grad_()`는 부품에도 쓸 수 있어서, 부품 안의 파라미터를 한꺼번에 얼리거나 풀어요. 얼린 부분은 앞 절의 함정을 피하려고 에폭마다 `eval()`로 돌려 두었습니다. 그래서 이번에는 `한_에폭_학습` 대신 학습 루프를 직접 썼어요.

결과를 보면 이렇습니다.

- **머리만(83.6%)**: 10.4절의 특징 추출(87.0%)보다 낮은데, 이번에는 8에폭 동안 64번만 고쳤기 때문입니다. 특징을 뽑아 두면 100에폭도 순식간이지만 몸통을 매번 돌리면 그만큼 비싸요.
- **`layer4`만 풀기(91.4%)**: 마지막 단계 하나만 풀었는데 8%p 가까이 뛰었습니다. 파라미터 수로는 전체의 75%를 차지하는 부분이에요.
- **`layer3`까지(92.7%), 전부(93.1%)**: 더 풀수록 조금씩 오르다가, `layer1`까지 푼 것과 `conv1`까지 전부 푼 것이 똑같이 93.1%입니다.

앞부분을 풀어도 거의 달라지지 않는 이유는 다음 실험이 보여 줍니다. 그전에 실무 요령을 정리하면, 데이터가 아주 적거나 ImageNet과 비슷한 사진이면 많이 얼리고, 데이터가 넉넉하거나 ImageNet과 다른 사진이면 많이 풉니다. 계산 자원이 빠듯하면 뒷부분만 풀어서 비용을 줄일 수 있어요. 앞부분을 얼리면 그 부분의 기울기를 계산하지 않으니 학습이 빨라지고 메모리도 덜 듭니다.

무엇이 얼마나 바뀌었을까

미세조정으로 몸통 전체를 학습하면, 실제로는 어느 부분이 많이 바뀔까요? 미세조정 전의 가중치를 복사해 두고, 학습이 끝난 뒤 부분마다 가중치가 처음보다 몇 % 달라졌는지 재 보겠습니다.

```
import copy


import torch
from torch import nn
from torchvision import models
```

```
from 전이도구 import 로더_만들기, 유로샛_불러오기, 장치_고르기, 정확도, 한_에폭_학습

장치 = 장치_고르기()
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)
학습_로더 = 로더_만들기(학습_사진, 학습_정답, 증강=True, 섞기=True)

torch.manual_seed(0)
모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
모델.fc = nn.Linear(512, 10)
처음 = copy.deepcopy(모델.state_dict())          # 미세조정 전 가중치를 복사해 둔다
모델.to(장치)
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-4)
for 에폭 in range(8):
    한_에폭_학습(모델, 학습_로더, 옵티마이저, 장치)
print(f"검증 정확도 {정확도(모델, 로더_만들기(검증_사진, 검증_정답), 장치):.1%}")

# 부분마다 가중치가 처음보다 몇 % 바뀌었나: ||나중 - 처음|| / ||처음||
나중 = {k: v.cpu() for k, v in 모델.state_dict().items()}
for 부분 in ["conv1", "layer1", "layer2", "layer3", "layer4"]:
    이름들 = [k for k in 처음 if k.startswith(부분 + ".") and k.endswith("weight")]
    전 = torch.cat([처음[k].flatten() for k in 이름들])
    후 = torch.cat([나중[k].flatten() for k in 이름들])
    print(f"{부분:7} 변화율 {(후 - 전).norm() / 전.norm():.2%}")
```

```
검증 정확도 92.6%
conv1   변화율 0.55%
layer1  변화율 1.54%
layer2  변화율 2.23%
layer3  변화율 3.38%
layer4  변화율 4.42%
```

`copy.deepcopy(모델.state_dict())`는 8장 8.8절의 조기 종료에서 최고 가중치를 보관할 때 쓴 방법입니다. 복사하지 않으면 학습하는 동안 보관본도 같이 바뀌어 버리니까요. 변화율은 "바뀐 양의 크기 ÷ 원래 크기"로, 부분마다 `weight`로 끝나는 텐서를 한 줄로 이어 붙여(`flatten`, `cat`) 크기

(norm)를 쟀습니다.

모든 부분에 같은 학습률 0.0001을 줬는데도 변화율이 conv1 0.55%에서 layer4 4.42%까지 **뒤로 갈수록 8배 커집니다.** 앞부분이 배운 가장자리·색 같은 기본 특징은 위성 사진에도 그대로 맞아서 기울기가 작았고, 뒷부분의 ImageNet 특화 특징은 많이 고쳐야 했다는 뜻이에요. 앞 절에서 앞부분을 얼려도 정확도가 거의 같았던 것과 맞아떨어집니다. 그리고 가장 많이 바뀐 layer4도 원래 크기의 4.42%만큼만 움직였어요. 미세조정은 정말 "미세하게" 조정할 뿐이고, 물려받은 가중치는 거의 그대로 남습니다. 검증 정확도 92.6%는 앞 절에서 "전부 1e-4"로 학습한 결과와 같은 설정이라 같은 숫자예요.

Q

Q. 층마다 학습률을 점점 다르게 줄 수도 있나요? 네. 앞부분일수록 작게, 뒷부분일수록 크게 주는 방법도 많이 씁니다. 예를 들어 layer1은 0.00001, layer2는 0.00003, … 이런 식으로 부품마다 그룹을 만들면 돼요. 위 실험처럼 앞부분은 원래 조금만 바뀌어야 하니 이치에 맞는 방법입니다. 다만 손잡이(하이퍼파라미터)가 늘어나는 만큼 검증 데이터로 확인하며 정해야 해요. 13장에서 한국어 언어 모델을 미세조정할 때도 같은 원리가 그대로 쓰이고, 14장의 LoRA는 "모델 전체를 고치지 않고 아주 작은 부분만 덧붙여 학습한다"는 생각을 한 걸음 더 밀고 나간 방법입니다.

10.7 다른 모델, 다른 입력

머리 바꾸기를 일반화하기

ResNet은 머리가 모델.fc 하나라서 바꾸기 쉬웠습니다. 그런데 모델마다 머리의 이름과 위치가 다릅니다. 어떤 모델이든 마지막 nn.Linear를 찾아 바꿔 주는 함수를 만들어 보겠습니다.

```
import torch
from torch import nn
from torchvision import models


def 머리_바꾸기(모델: nn.Module, 클래스_수: int) -> str:
    """모델에 등록된 마지막 nn.Linear를 찾아 클래스_수만큼 내는 새 층으로 바꾼다."""
    이름, 마지막 = [(n, m) for n, m in 모델.named_modules() if isinstance(m, nn.Linear)][-1]
    부모_이름, _, 자식_이름 = 이름.rpartition(".")        # "classifier.3" → "classifier", "3"
    setattr(모델.get_submodule(부모_이름), 자식_이름, nn.Linear(마지막.in_features, 클래스_수))
```

```
    return 이름


for 이름 in ["resnet18", "mobilenet_v3_small", "efficientnet_b0", "vit_b_16"]:
    모델 = models.get_model(이름, weights=None)           # 구조만 본다
    바꾼_곳 = 머리_바꾸기(모델, 10)
    with torch.no_grad():
        출력 = 모델.eval()(torch.zeros(1, 3, 224, 224))
    print(f"{이름:19} 마지막 층 '{바꾼_곳}' → {모델.get_submodule(바꾼_곳)}, 출력 {tuple(출
    ↪ 력.shape)}")
```

```
resnet18            마지막 층 'fc' → Linear(in_features=512, out_features=10, bias=True), 출력 (1,
↪ 10)
mobilenet_v3_small  마지막 층 'classifier.3' → Linear(in_features=1024, out_features=10,
↪ bias=True), 출력 (1, 10)
efficientnet_b0     마지막 층 'classifier.1' → Linear(in_features=1280, out_features=10,
↪ bias=True), 출력 (1, 10)
vit_b_16            마지막 층 'heads.head' → Linear(in_features=768, out_features=10, bias=True),
↪ 출력 (1, 10)
```

`named_modules()`는 모델 안의 모든 부품을 이름과 함께 차례로 돌려줍니다. 그중 `nn.Linear`만 골라 마지막 것을 꺼냈어요. 이름이 `"classifier.3"`처럼 점으로 이어져 있으면 앞은 부모 부품, 뒤는 그 안의 자식 이름입니다. 문자열의 `rpartition(".")`으로 마지막 점을 기준으로 나누고, `get_submodule`로 부모를 꺼내 `setattr`로 자식을 새 층으로 갈아 끼웠어요(1권 16장). 이름에 점이 없는 `"fc"`는 부모 이름이 빈 문자열이 되는데, `get_submodule("")`은 모델 자신을 돌려주니 그대로 동작합니다.

결과를 보면 ResNet은 `fc`, MobileNet은 `classifier`의 3번, EfficientNet은 `classifier`의 1번, ViT는 `heads.head`가 머리였습니다. 들어오는 특징의 수도 512, 1,024, 1,280, 768로 제각각이에요. 이 함수는 "부품을 등록한 순서의 마지막 `nn.Linear`가 머리"라고 가정하는데, 대부분의 모델에서 맞지만 직접 만든 모델에서는 아닐 수도 있습니다. 처음 쓰는 모델은 `print(모델)`로 구조를 한 번 훑어보는 습관을 들이세요.

몸통 바꿔 끼우기

10.1절의 표에서 MobileNetV3-Small과 EfficientNet-B0은 ImageNet 정확도나 크기에서 ResNet18과 달랐습니다. 우리 위성 사진에서는 어떨까요? 세 모델을 같은 조건(크기 112, 특징 추출)으로 비교해 보겠습니다. `models.get_model(이름, weights="DEFAULT")`은 이름 문자열로 모델과 가중치를 한 번에 불러오는 방법이에요.

```
import torch
from torch import nn
from torchvision import models

from 전이도구 import 로더_만들기, 유로샛_불러오기, 장치_고르기, 머리_학습, 특징_뽑기

장치 = 장치_고르기()
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)

for 이름 in ["mobilenet_v3_small", "resnet18", "efficientnet_b0"]:
    모델 = models.get_model(이름, weights="DEFAULT")          # 처음 한 번은 가중치를 내려받는다
    마지막 = [n for n, m in 모델.named_modules() if isinstance(m, nn.Linear)][-1]
    부모, _, 자식 = 마지막.rpartition(".")
    setattr(모델.get_submodule(부모), 자식, nn.Identity())   # 마지막 층을 떼면 특징 추출기
    모델.to(장치)
    학습_특징, _ = 특징_뽑기(모델, 로더_만들기(학습_사진, 학습_정답), 장치)
    검증_특징, _ = 특징_뽑기(모델, 로더_만들기(검증_사진, 검증_정답), 장치)
    머리 = 머리_학습(학습_특징, 학습_정답)
    with torch.no_grad():
        정확도 = (머리(검증_특징).argmax(1) == 검증_정답).float().mean()
    개수 = sum(p.numel() for p in 모델.parameters())
    print(f"{이름:19} 몸통 {개수 / 1e6:4.1f}백만 개, 특징 {학습_특징.shape[1]:4}개, 검증 정확도 {정
    ↪ 확도:.1%}")
```

```
mobilenet_v3_small  몸통  1.5백만 개, 특징 1024개, 검증 정확도 85.5%
resnet18            몸통 11.2백만 개, 특징  512개, 검증 정확도 87.0%
efficientnet_b0     몸통  4.0백만 개, 특징 1280개, 검증 정확도 85.6%
```

ImageNet 정확도는 EfficientNet-B0(77.7%)이 ResNet18(69.8%)보다 훨씬 높았는데, 우리 데이터

에서는 ResNet18이 87.0%로 가장 높고 EfficientNet-B0(85.6%)과 MobileNetV3-Small(85.5%)이 비슷합니다. **원래 문제에서 잘한 순서가 새 문제에서도 유지된다는 보장은 없다**는 걸 보여 주는 결과예요. 크기를 112로 줄인 것이 모델마다 다르게 영향을 줬을 수도 있고, 위성 사진처럼 ImageNet과 성질이 다른 데이터에서는 특히 그렇습니다.

한편 MobileNetV3-Small은 몸통이 150만 개로 ResNet18의 7분의 1인데 정확도 차이는 1.5%p뿐입니다. 휴대폰처럼 작은 기기에서 돌려야 한다면 이쪽이 더 나은 선택일 수 있어요. 결국 모델 고르기도 데이터로 정해야 합니다. 후보 두세 개를 특징 추출로 빠르게 비교해 보고, 가장 나은 것을 골라 미세조정하는 것이 효율적인 순서예요. 모델을 작게 줄여 기기에서 빠르게 돌리는 방법은 19장에서 다룹니다.

흑백 사진을 넣고 싶다면

사전학습 모델은 컬러(3채널) 사진으로 학습했습니다. 엑스레이나 6장의 FashionMNIST 같은 흑백(1채널) 사진은 어떻게 넣을까요? 방법이 두 가지 있는데, 사실 둘은 계산 결과가 똑같습니다.

```
import torch
from torch import nn
from torchvision import models

모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT).eval()
print("conv1 가중치:", tuple(모델.conv1.weight.shape))      # (출력 64, 입력 3색, 7, 7)

torch.manual_seed(0)
흑백 = torch.randn(2, 1, 64, 64)                            # 흑백 사진 2장 (예제용 난수)

# 방법 1: 흑백 한 채널을 세 번 복사해 컬러처럼 넣는다
with torch.no_grad():
    출력1 = 모델(흑백.repeat(1, 3, 1, 1))

# 방법 2: 첫 층을 한 채널짜리로 바꾸고, 세 색의 가중치를 더해 넣는다
새_첫층 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)
with torch.no_grad():
    새_첫층.weight.copy_(모델.conv1.weight.sum(dim=1, keepdim=True))   # (64, 3, 7, 7) → (64, 1, 7,
↪ 7)
모델.conv1 = 새_첫층
with torch.no_grad():
```

```
    출력2 = 모델(흑백)
print("conv1 가중치:", tuple(모델.conv1.weight.shape))
print("두 방법의 출력이 같은가:", torch.allclose(출력1, 출력2, atol=1e-4))
```

```
conv1 가중치: (64, 3, 7, 7)
conv1 가중치: (64, 1, 7, 7)
두 방법의 출력이 같은가: True
```

방법 1은 흑백 한 채널을 `repeat(1, 3, 1, 1)`로 세 번 복사해 컬러 사진처럼 만드는 것입니다. 가장 간단하고, 7장의 transforms에서는 `v2.Grayscale(num_output_channels=3)`이 같은 일을 해요. **방법 2**는 첫 층 `conv1`을 입력 1채널짜리로 새로 만들고, 원래 가중치 `(64, 3, 7, 7)`을 색 방향(`dim=1`)으로 더해 `(64, 1, 7, 7)`로 넣는 것입니다. 세 채널에 같은 값이 들어오면 세 가중치에 각각 곱해서 더하는 것과, 가중치를 먼저 더해 한 번 곱하는 것이 같으니(a × w1 + a × w2 + a × w3 = a × (w1 + w2 + w3)) 결과도 같아요.

방법 2는 첫 층의 계산과 메모리를 3분의 1로 줄이지만, 첫 층의 계산은 전체에서 작은 몫이라 대부분은 방법 1로 충분합니다. 어느 쪽이든 정규화는 신경 써야 해요. ImageNet 평균·표준편차는 채널마다 다르니, 흑백을 세 채널로 복사한 뒤에 정규화하면 됩니다.

10.8 저장하고 불러오기

전이 학습한 모델을 저장할 때도 6장 6.10절의 `state_dict`를 그대로 씁니다. 다만 머리를 바꿨다는 점 때문에 불러올 때 한 단계가 더 필요해요. 특징 추출로 만든 모델로 확인해 보겠습니다.

```
from pathlib import Path

import torch
from torch import nn
from torchvision import models

from 전이도구 import 로더_만들기, 유로샛_불러오기, 머리_학습, 정확도, 특징_뽑기

장치 = torch.device("cpu")
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
```

```
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)
검증_로더 = 로더_만들기(검증_사진, 검증_정답)

# 10.4절처럼 특징 추출로 머리를 학습해 완성 모델을 만든다
모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
모델.fc = nn.Identity()
학습_특징, _ = 특징_뽑기(모델, 로더_만들기(학습_사진, 학습_정답), 장치)
모델.fc = 머리_학습(학습_특징, 학습_정답)
print(f"완성 모델 검증 정확도 {정확도(모델, 검증_로더, 장치):.1%}")

torch.save(모델.state_dict(), "eurosat_resnet18.pt")      # 모델 전체
torch.save(모델.fc.state_dict(), "eurosat_head.pt")       # 머리만
for 파일 in ["eurosat_resnet18.pt", "eurosat_head.pt"]:
    print(f"{파일:20} {Path(파일).stat().st_size / 1024:10,.1f}KB")

# 불러오기 ①: 설계도를 그대로 만들지 않으면
새모델 = models.resnet18(weights=None)
try:
    새모델.load_state_dict(torch.load("eurosat_resnet18.pt", weights_only=True))
except RuntimeError as e:
    for 줄 in str(e).splitlines()[1:]:
        print("오류:", 줄.strip())

# 불러오기 ②: 머리를 바꾼 설계도에 전체를 채운다
새모델.fc = nn.Linear(512, 10)
새모델.load_state_dict(torch.load("eurosat_resnet18.pt", weights_only=True))
print(f"전체를 불러온 모델 {정확도(새모델, 검증_로더, 장치):.1%}")

# 불러오기 ③: 몸통은 torchvision에서 받고 머리만 채운다
가벼운모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
가벼운모델.fc = nn.Linear(512, 10)
가벼운모델.fc.load_state_dict(torch.load("eurosat_head.pt", weights_only=True))
print(f"머리만 불러온 모델 {정확도(가벼운모델, 검증_로더, 장치):.1%}")
```

```
완성 모델 검증 정확도 87.0%
eurosat_resnet18.pt    43,755.1KB
eurosat_head.pt            21.9KB
오류: size mismatch for fc.weight: copying a param with shape torch.Size([10, 512]) from
↪ checkpoint, the shape in current model is torch.Size([1000, 512]).
오류: size mismatch for fc.bias: copying a param with shape torch.Size([10]) from checkpoint, the
↪ shape in current model is torch.Size([1000]).
전체를 불러온 모델 87.0%
머리만 불러온 모델 87.0%
```

모델 전체를 저장하면 43,755KB(약 42.7MB), 머리만 저장하면 21.9KB입니다. 2,000배 차이죠. 특징 추출로 학습했다면 몸통은 torchvision에서 언제든 다시 받을 수 있으니 **머리만 저장해도 충분합니다.** 여러 문제에 같은 몸통을 쓴다면 몸통 하나에 문제마다 작은 머리 파일만 두면 되니 저장 공간도 아끼고요.

불러오기에서는 일부러 실수를 한 번 했습니다. `models.resnet18(weights=None)`으로 만든 설계도는 머리가 1,000칸인데, 파일 속 머리는 10칸이라 `size mismatch`(크기 불일치) 오류가 납니다. 오류 메시지를 4장 4.10절에서 배운 대로 숫자부터 읽어 보세요. 파일(checkpoint)의 `fc.weight`는 `[10, 512]`인데 지금 모델은 `[1000, 512]`라는 뜻입니다. 6장에서 말한 "같은 설계도를 만들고 숫자를 채운다"는 원칙대로, **머리까지 똑같이 바꾼 설계도**를 만든 뒤에 불러오면 해결돼요.

전체를 불러온 모델과, ImageNet 몸통에 머리만 채운 모델 모두 검증 정확도가 원래 모델과 같은 87.0%입니다. 미세조정한 모델은 몸통도 바뀌었으니 반드시 전체를 저장해야 한다는 점만 기억하세요. 저장한 모델을 웹 서비스로 내보내는 방법은 20장에서 다룹니다.

10.9 실습: 데이터가 적을수록 전이 학습이 빛난다

데이터 양을 바꿔 가며 비교하기

지금까지 클래스당 50장으로 세 가지 방법을 비교했습니다. 그런데 정답 붙은 사진이 클래스당 5장뿐이라면, 아니면 100장이라면 결과가 어떻게 달라질까요? 데이터 양을 바꿔 가며 세 방법을 한 표로 비교해 보겠습니다.

1. **처음부터**: 무작위 가중치의 ResNet18, lr=0.001, 배치 240번 고치기
2. **특징 추출**: 특징을 뽑아 두고 머리만 100에폭
3. **미세조정**: 몸통 lr=0.0001 + 머리 lr=0.001, 배치 64번 고치기

데이터가 5장이면 한 에폭이 배치 하나뿐이고 100장이면 배치 16개라서, 에폭 수로 맞추면 고치는 횟수가 크게 달라집니다. 그래서 이번에는 데이터 양과 상관없이 **배치를 꺼내 고치는 횟수**를 맞췄어요. 처음부터 학습하는 쪽은 10.5절과 같은 240번, 미세조정은 10.6절과 같은 64번입니다.

```
import itertools

import torch
from torch import nn
from torchvision import models

from 전이도구 import 로더_만들기, 유로샛_불러오기, 장치_고르기, 머리_학습, 정확도, 특징_뽑기

장치 = 장치_고르기()
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)
검증_로더 = 로더_만들기(검증_사진, 검증_정답)


def 끝없이(로더):
    """로더를 다 돌면 처음부터 다시 돈다. 돌 때마다 새로 섞고 새로 증강한다."""
    while True:
        yield from 로더


def 고치기(모델: nn.Module, 옵티마이저: torch.optim.Optimizer, 학습_로더, 횟수: int) -> None:
    """데이터 양과 상관없이 정해진 횟수만큼 배치를 꺼내 고친다."""
    모델.train()
    for x, y in itertools.islice(끝없이(학습_로더), 횟수):
        x, y = x.to(장치), y.to(장치)
        옵티마이저.zero_grad()
        nn.functional.cross_entropy(모델(x), y).backward()
        옵티마이저.step()


def 처음부터(학습_로더) -> float:
    torch.manual_seed(0)
    모델 = models.resnet18(weights=None, num_classes=10).to(장치)
    고치기(모델, torch.optim.Adam(모델.parameters(), lr=1e-3), 학습_로더, 240)
```

```
    return 정확도(모델, 검증_로더, 장치)


def 특징_추출(학습_사진, 학습_정답) -> float:
    몸통 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    몸통.fc = nn.Identity()
    몸통.to(장치)
    학습_특징, _ = 특징_뽑기(몸통, 로더_만들기(학습_사진, 학습_정답), 장치)
    검증_특징, _ = 특징_뽑기(몸통, 검증_로더, 장치)
    머리 = 머리_학습(학습_특징, 학습_정답)
    with torch.no_grad():
        return (머리(검증_특징).argmax(1) == 검증_정답).float().mean().item()


def 미세조정(학습_로더) -> float:
    torch.manual_seed(0)
    모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    모델.fc = nn.Linear(512, 10)
    모델.to(장치)
    몸통_파라미터 = [p for 이름, p in 모델.named_parameters() if not 이름.startswith("fc.")]
    옵티마이저 = torch.optim.Adam([{"params": 몸통_파라미터, "lr": 1e-4},
                                  {"params": 모델.fc.parameters(), "lr": 1e-3}])
    고치기(모델, 옵티마이저, 학습_로더, 64)
    return 정확도(모델, 검증_로더, 장치)


print("클래스당 ¦ 처음부터 ¦ 특징 추출 ¦ 미세조정")
for 클래스당 in [5, 20, 100]:
    학습_사진, 학습_정답 = 유로샛_불러오기("학습", 클래스당)
    학습_로더 = 로더_만들기(학습_사진, 학습_정답, 증강=True, 섞기=True)
    결과 = [처음부터(학습_로더), 특징_추출(학습_사진, 학습_정답), 미세조정(학습_로더)]
    print(f"{클래스당:6}장 ¦ " + " ¦ ".join(f"{a:7.1%}" for a in 결과))
```

```
클래스당 ¦ 처음부터 ¦ 특징 추출 ¦ 미세조정
     5장 ¦   51.7% ¦   72.0% ¦   81.5%
    20장 ¦   50.6% ¦   83.4% ¦   85.9%
```

```
100장 ¦   52.7% ¦   88.0% ¦   93.2%
```

`고치기`는 정해진 횟수만큼 배치를 꺼내 학습하는 함수입니다. `끝없이`는 로더를 다 돌면 처음부터 다시 도는 제너레이터(1권 15장 15.2절의 `yield from`)이고, `itertools.islice`로 앞에서부터 정해진 개수만 잘라 냈어요. 덕분에 데이터가 몇 장이든 같은 횟수만큼 고칩니다.

1권 17장의 `itertools.cycle`을 쓰면 더 짧을 것 같지만 여기서는 함정입니다. `cycle`은 첫 바퀴에서 꺼낸 배치를 저장해 두었다가 **똑같이** 되풀이하거든요. 그러면 둘째 바퀴부터는 새로 섞지도, 새로 뒤집지도 않아서 증강이 첫 바퀴에만 일어납니다. 클래스당 5장이면 배치가 하나뿐이니 240번을 모두 같은 사진 묶음으로 고치게 되죠. `while True: yield from 로더`는 바퀴마다 로더를 새로 돌리니 매번 새로 섞고 새로 증강합니다.

표를 한 줄씩 읽어 보겠습니다.

- **클래스당 5장(전체 50장)**: 처음부터 학습하면 51.7%, 특징 추출은 72.0%, 미세조정은 81.5%입니다. 사진 50장으로 열 가지 땅을 열에 여덟 넘게 맞힌다는 건, 사전학습 모델의 눈이 없었다면 불가능한 일이에요.
- **클래스당 20장**: 처음부터 50.6%, 특징 추출 83.4%, 미세조정 85.9%입니다.
- **클래스당 100장**: 처음부터 52.7%, 특징 추출 88.0%, 미세조정 **93.2%** 입니다.

이번 실험에서는 데이터가 5장뿐일 때도 미세조정이 특징 추출보다 9.5%p 앞섰습니다. 사진이 50장뿐이어도 미세조정은 바퀴마다 새로 뒤집힌 사진을 보며 몸통까지 조금씩 고칠 수 있지만, 특징을 미리 뽑아 두는 특징 추출은 증강을 쓸 수 없으니까요(10.4절). 다만 미세조정은 몸통 전체를 학습하는 만큼 계산이 훨씬 많이 들고, 학습률을 잘못 잡으면 10.6절처럼 물려받은 눈을 망가뜨릴 수 있습니다. 그래서 "먼저 특징 추출로 빠르게 기준을 재고, 시간이 허락하면 미세조정으로 더 올린다"는 순서가 실무에서 무난해요.

처음부터 학습한 쪽은 세 경우 모두 50% 초반에 머뭅니다. 데이터가 스무 배로 늘어도 고치는 횟수는 240번 그대로라서, 늘어난 데이터를 다 소화하지 못한 거예요. 10.5절에서 클래스당 50장으로 같은 240번을 고쳤을 때 68.1%였던 것과 비교하면 숫자가 들쭉날쭉한데, 무작위 가중치에서 시작한 모델은 고치는 횟수가 적으면 학습이 끝나는 순간에 따라 성적이 크게 흔들리기 때문입니다(10.5절 출력에서도 20에폭 63.4%, 25에폭 69.9%, 30에폭 68.1%로 오르내렸죠). 처음부터 학습하는 모델은 데이터가 늘면 학습 시간도 함께 늘려야 하고, 수만 장 이상이 모이면 전이 학습과의 차이도 줄어듭니다. 거꾸로 말하면 전이 학습은 **적은 데이터와 짧은 시간**이라는 두 가지를 한꺼번에 아껴 주는 방법이에요.

시험 데이터로 마지막 확인

검증 데이터로 고른 최종 방법은 “클래스당 100장 + 미세조정”입니다. 이제 아껴 둔 시험 데이터를 딱 한 번 꺼내 최종 성적을 재고, 6장 6.11절처럼 무엇을 틀렸는지 들여다보겠습니다.

```
import torch
from torch import nn
from torchvision import models

from 전이도구 import 로더_만들기, 유로샛_불러오기, 장치_고르기, 정확도, 한_에폭_학습

이름 = ["경작지", "숲", "초지", "고속도로", "산업지역", "목초지", "과수원", "주거지", "강", "바다·
↪ 호수"]
장치 = 장치_고르기()
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 100)
시험_사진, 시험_정답 = 유로샛_불러오기("시험", 100)     # 여기서 처음이자 마지막으로 쓴다
학습_로더 = 로더_만들기(학습_사진, 학습_정답, 증강=True, 섞기=True)

torch.manual_seed(0)
모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
모델.fc = nn.Linear(512, 10)
모델.to(장치)
몸통_파라미터 = [p for n, p in 모델.named_parameters() if not n.startswith("fc.")]
옵티마이저 = torch.optim.Adam([{"params": 몸통_파라미터, "lr": 1e-4},
                           {"params": 모델.fc.parameters(), "lr": 1e-3}])
for 에폭 in range(4):                       # 1,000장 ÷ 64 = 16배치 × 4에폭 = 64번 고치기
    한_에폭_학습(모델, 학습_로더, 옵티마이저, 장치)
print(f"시험 정확도 {정확도(모델, 로더_만들기(시험_사진, 시험_정답), 장치):.1%}")

# 클래스별 정확도와 헷갈린 짝(6장 6.11절과 같은 방법)
모델.eval()
with torch.no_grad():
    예측 = torch.cat([모델(x.to(장치)).argmax(1).cpu() for x, _ in 로더_만들기(시험_사진, 시험_정
↪ 답)])
for k in range(10):
    print(f"{이름[k]:5} {(예측[시험_정답 == k] == k).float().mean():5.0%}", end="  " if k % 5 != 4
    ↪ else "\n")
```

```
혼동 = torch.bincount(시험_정답 * 10 + 예측, minlength=100).reshape(10, 10)   # (정답, 예측) 칸마다
↪ 개수
혼동.fill_diagonal_(0)
for 위치 in 혼동.flatten().topk(3).indices.tolist():
    정답, 답 = divmod(위치, 10)
    print(f"정답 {이름[정답]} → 예측 {이름[답]}: {혼동[정답, 답]}번")
```

```
시험 정확도 92.9%
경작지     90%  숲       97%  초지      87%  고속도로    93%  산업지역    99%
목초지     92%  과수원     89%  주거지     95%  강       93%  바다·호수   94%
정답 초지 → 예측 숲: 6번
정답 과수원 → 예측 초지: 6번
정답 초지 → 예측 과수원: 5번
```

시험 정확도는 **92.9%** 로, 검증 정확도(93.2%)와 거의 같습니다. 검증 데이터로 이것저것 고른 결과가 처음 보는 시험 데이터에서도 그대로 통한다는 뜻이에요. 이 책의 실행 환경(Apple M4 Max, MPS)에서 잰 값이고, GPU에서 학습한 결과는 실행 환경에 따라 소수점 아래가 조금 다를 수 있어요.

헷갈린 짝을 셀 때는 4장 4.4절의 `bincount`를 썼습니다. `정답 × 10 + 예측`은 (정답, 예측) 짝마다 0~99 사이의 번호 하나가 되니, 번호마다 개수를 세서 10 × 10으로 접으면 6장에서 반복문으로 만들었던 혼동 표가 한 줄로 나와요.

클래스별로 보면 산업 지역(99%)과 숲(97%)이 가장 잘 맞고, 초지(87%)와 과수원(89%), 경작지(90%)가 상대적으로 약합니다. 가장 많이 헷갈린 짝은 초지 → 숲(6번), 과수원 → 초지(6번), 초지 → 과수원(5번)이에요. 모두 "식물로 덮인 땅"끼리입니다. 64 × 64 사진 한 장에서 풀숲과 과수원과 숲의 경계는 사람이 봐도 애매할 때가 많아요. 반면 10.3절에서 평균 색이 거의 같았던 고속도로·주거 지역은 93%, 95%로 잘 구별합니다. 색이 같아도 길게 뻗은 선과 촘촘한 지붕이라는 **무늬**의 차이를 사전학습 모델의 눈이 잡아낸 거죠.

여기서 더 올리고 싶다면 이 장에서 본 손잡이들을 차례로 돌려 보면 됩니다. 입력 크기를 160이나 224로 키우고(10.4절), 학습 데이터를 늘리고(10.9절), 8장의 학습률 스케줄러와 조기 종료를 더하는 것이죠. 이 실험들을 체계적으로 기록하고 비교하는 방법은 18장에서 다룹니다.

10.10 정리와 다음 장 미리보기

이 장에서 남이 학습한 모델을 빌려 내 문제를 푸는 법을 익혔습니다.

- **사전학습 모델**: `models.resnet18(weights=ResNet18_Weights.DEFAULT)`. `weights.meta`에 파라미터 수, 파일 크기, 정확도, 범주가 있다. `list_models`, `get_model_weights`, `get_model`로 다른 모델도 같은 방식으로. 가중치의 이용 조건은 직접 확인
- **몸통과 머리**: 몸통은 사진을 특징 벡터로 바꾸는 눈, 머리는 원래 문제의 범주에 맞춘 마지막 층. 전이 학습은 머리를 새것으로 바꾸는 데서 시작
- **전처리**: `weights.transforms()`를 출력해 확인. 크기는 바꿔도 되지만 평균·표준편차는 ImageNet 값 그대로. `scale=True` 누락과 정규화 누락은 오류 없이 정확도만 떨어뜨린다
- **특징 추출**: 몸통을 얼리고(`requires_grad_(False)`) 머리만 학습. 특징을 한 번 뽑아 두면(`fc = nn.Identity()`) 빠르지만 증강은 못 쓴다
- **배치 정규화 함정**: 얼린 몸통도 `train()` 모드에서는 기록 평균이 바뀐다. 얼린 부분은 `eval()`로 묶어 두기
- **미세조정**: 사전학습 가중치에서 전체를 학습. 학습률은 처음부터 학습할 때보다 10배쯤 작게. 몸통과 머리는 파라미터 그룹으로 학습률을 따로. 뒷부분일수록 많이 바뀐다
- **고르는 요령**: 특징 추출로 빠르게 기준을 재고, 시간이 허락하면 미세조정(증강을 쓸 수 있어 데이터가 적어도 유리할 수 있다). 고정 횟수로 반복할 때 `itertools.cycle`은 증강을 막으니 쓰지 않는다. 어디까지 풀지, 어떤 모델을 쓸지, 입력 크기를 얼마로 할지는 검증 데이터로 정한다
- **저장과 불러오기**: 머리를 바꾼 설계도를 똑같이 만든 뒤 `load_state_dict`. 특징 추출이면 머리만 저장해도 된다

지금까지 다룬 데이터는 표 한 줄이든 사진 한 장이든 **한 번에 통째로** 들어오는 것이었습니다. 그런데 문장이나 날씨 기록, 주가처럼 **순서가 있는** 데이터는 어떨까요? "나는 밥을 먹었다"와 "밥은 나를 먹었다"는 같은 단어로 되어 있지만 뜻이 전혀 다르죠. 다음 11장에서는 앞에서 본 것을 기억하며 하나씩 읽어 나가는 **순환 신경망**(RNN)과 LSTM을 직접 만들어 봅니다.

11

RNN과 LSTM: 순서가 있는 데이터

학습 목표

- 순서가 있는 데이터를 (배치, 길이, 특성) 모양으로 만들고 읽기
- RNN 셀을 손으로 구현하고 `nn.RNN`과 같은 결과가 나오는지 확인하기
- 순서를 알아야만 풀리는 문제에서 RNN이 MLP와 어떻게 다른지 숫자로 보기
- 기울기 소실과 폭주가 왜 생기는지 시점별 기울기를 재어 확인하기
- LSTM과 GRU의 문(gate) 구조를 이해하고, 긴 기억이 필요한 문제에서 비교하기
- 길이가 다른 순서열을 `pack_padded_sequence`로 올바르게 처리하기
- 실제 시계열(이산화탄소 농도) 예측과 글자 단위 한국어 생성 모델 만들기

들어가며: 순서를 바꾸면 뜻이 바뀝니다

식당 리뷰 두 개를 보세요. “가격은 별로지만 맛은 최고”와 “맛은 별로지만 가격은 최고”. 첫 번째 손님은 맛 때문에 또 올 것이고, 두 번째 손님은 아마 다시 오지 않을 거예요. 그런데 두 문장에 쓰인 글자를 세어 보면 어떨까요?

```
from collections import Counter

import torch

리뷰1 = "가격은 별로지만 맛은 최고"
리뷰2 = "맛은 별로지만 가격은 최고"
```

```
# 글자마다 몇 번 나왔는지만 센다: 어느 글자가 먼저 왔는지는 버린다
print(Counter(리뷰1) == Counter(리뷰2))

글자들 = sorted(set(리뷰1))
번호 = {글자: i for i, 글자 in enumerate(글자들)}


def 개수_벡터(문장: str) -> torch.Tensor:
    """문장을 글자 종류별 개수로 바꾼다."""
    v = torch.zeros(len(글자들))
    for 글자 in 문장:
        v[번호[글자]] += 1
    return v


print(torch.equal(개수_벡터(리뷰1), 개수_벡터(리뷰2)))

# 숫자 줄도 마찬가지: 닷새 동안의 주가 (예제용 숫자)
오르는_중 = torch.tensor([100.0, 102.0, 105.0, 109.0, 114.0])
내리는_중 = 오르는_중.flip(0)              # 같은 숫자를 거꾸로
print(오르는_중.mean().item(), 내리는_중.mean().item())
print(오르는_중.std().item() == 내리는_중.std().item())
print("마지막 날 - 첫날:", (오르는_중[-1] - 오르는_중[0]).item(),
      (내리는_중[-1] - 내리는_중[0]).item())
```

```
True
True
106.0 106.0
True
마지막 날 - 첫날: 14.0 -14.0
```

두 리뷰는 **글자의 종류와 개수가 완전히 같습니다.** 1권 7장의 `Counter`로 세어도, 글자 개수 벡터로 바꿔도 구별이 안 돼요. 다른 것은 오직 **순서**뿐입니다. 숫자 줄도 마찬가지예요. 계속 오르는 주가와 계속 내리는 주가는 평균도 표준편차도 똑같지만, 마지막 날에서 첫날을 빼면 +14와 -14로 정반대입니다. 투자자에게 이보다 중요한 차이는 없겠죠.

지금까지 만든 모델은 입력을 한 번에 통째로 받았습니다. 6장의 MLP는 숫자 묶음을 받아 곱하고 더했고, 9장의 CNN은 그림의 이웃한 칸을 함께 봤어요. 하지만 문장, 음성, 주가, 센서 기록처럼 **앞에서 뒤로 흘러가는 데이터**는 무엇이 먼저 오고 무엇이 나중에 오는지가 뜻의 절반입니다. 게다가 길이도 제각각이에요. 리뷰는 두 글자일 수도, 이백 글자일 수도 있습니다.

사람은 이런 데이터를 어떻게 읽을까요? 책을 읽을 때 우리는 한 글자씩 읽으면서 머릿속에 "지금까지의 줄거리"를 계속 고쳐 씁니다. 이 장의 주인공 **순환 신경망**(RNN, Recurrent Neural Network)이 정확히 그렇게 동작해요. 한 시점씩 읽으면서 작은 메모장을 고쳐 쓰고, 다 읽은 뒤 그 메모로 판단합니다.

이 장에서는 RNN을 손으로 만들어 보고, RNN이 긴 순서열에서 왜 기억을 잃는지 숫자로 확인한 뒤, 그 문제를 푼 LSTM과 GRU를 만납니다. 그리고 실제 대기 중 이산화탄소 측정값으로 미래를 예측하고, 마지막 실습에서는 우리 속담을 읽고 새 속담(?)을 지어내는 글자 단위 모델을 만들어 봅니다.

Q

Q. 이 장의 예제를 실행하려면 무엇을 설치해야 하나요? 4장부터 써 온 `torch`, `numpy`에 더해 `uv add statsmodels pandas matplotlib`를 실행하세요. statsmodels는 통계 분석 라이브러리인데, 이 장에서는 그 안에 들어 있는 이산화탄소 측정 데이터만 씁니다. 인터넷에서 받을 필요가 없어요. 이 장의 예제는 파이썬 3.12, PyTorch 2.14, statsmodels 0.15, pandas 3.0, matplotlib 3.11에서 실행해 확인했습니다. 모델이 작아서 모두 CPU로 돌렸고, 씨앗을 고정했으니 같은 버전이면 같은 숫자를 볼 수 있어요.

11.1 순서 데이터의 모양

창을 밀며 잘라 내기

순서가 있는 데이터를 모델에 넣으려면 먼저 모양부터 정해야 합니다. 열흘치 기온이 있고, "사흘을 보고 다음 날을 맞히는" 문제를 만든다고 해 봅시다. 사흘짜리 **창**(window)을 하루씩 밀면서 잘라 내면 문제와 정답 쌍이 여러 개 생겨요.

```
import torch

# 예제용으로 만든 열흘치 하루 평균 기온(°C)
기온 = torch.tensor([3.0, 4.5, 6.0, 5.5, 7.0, 8.5, 8.0, 9.5, 11.0, 10.5])

# 창 크기 3: 사흘을 보고 다음 날을 맞히는 문제로 바꾼다
```

```
창 = 3
조각들 = 기온.unfold(0, 창, 1)        # 길이 3짜리 창을 한 칸씩 밀며 잘라 낸다
print(조각들.shape)
print(조각들[:3])

X = 조각들[:-1]                       # 마지막 창은 다음 날 정답이 없으니 뺀다
y = 기온[창:]                         # 각 창의 바로 다음 날
for i in range(3):
    print(X[i].tolist(), "→", y[i].item())

# 순환 신경망이 받는 모양: (배치, 길이, 특성)
X = X.unsqueeze(-1)                   # 하루에 숫자 1개(기온)이니 특성 1
print("입력:", tuple(X.shape), "정답:", tuple(y.shape))
print("같은 메모리를 쓰나?", 조각들.data_ptr() == 기온.data_ptr())
```

```
torch.Size([8, 3])
tensor([[3.0000, 4.5000, 6.0000],
        [4.5000, 6.0000, 5.5000],
        [6.0000, 5.5000, 7.0000]])
[3.0, 4.5, 6.0] → 5.5
[4.5, 6.0, 5.5] → 7.0
[6.0, 5.5, 7.0] → 8.5
입력: (7, 3, 1) 정답: (7,)
같은 메모리를 쓰나? True
```

`unfold(0, 3, 1)`은 0번 차원을 따라 길이 3짜리 창을 1칸씩 밀면서 잘라 냅니다. 열흘에서 사흘짜리 창은 8개가 나와요(10 - 3 + 1). 마지막 창 `[9.5, 11.0, 10.5]`는 다음 날 기온이 없으니 문제로 쓸 수 없어서 빼고 7개를 썼습니다. 마지막 줄을 보면 `unfold`도 4장 4.5절의 `view`처럼 숫자를 복사하지 않고 같은 메모리를 다른 보폭으로 읽는 **창**이에요.

순환 신경망이 받는 입력은 **(배치, 길이, 특성)** 의 3차원입니다. 4장에서 권한 대로 자리마다 이름을 붙여 읽어 보세요. `(7, 3, 1)`은 "문제 7개, 각각 사흘, 하루마다 숫자 1개"예요. 기온 하나만 보니 특성이 1이고, 기온·습도·바람을 함께 본다면 특성이 3이 됩니다. 문장이라면 "문장 수, 글자 수, 글자 하나를 나타내는 숫자의 개수"가 되겠죠.

이렇게 과거의 창으로 다음 값을 맞히는 문제는 2장 2.9절에서 말한 시간 순서 데이터 그대로입니다.

학습·검증·테스트를 반드시 **시간 순서로** 잘라야 한다는 점도 같아요. 11.10절의 시계열 예측에서 그대로 지킵니다.

11.2 RNN 셀: 메모장을 고쳐 쓰며 읽기

손으로 만드는 RNN 셀

RNN이 하는 일은 한 문장으로 요약됩니다. **"지금 읽은 것과 지난 메모를 섞어서 새 메모를 쓴다."** 회의록을 쓰는 사람을 떠올려 보세요. 발언을 하나 들을 때마다 지금까지의 요약에 새 발언을 반영해 요약을 다시 씁니다. 요약 칸의 크기는 정해져 있어서, 중요한 것은 남기고 덜 중요한 것은 밀어내야 해요.

이 메모를 **은닉 상태(hidden state)** 라고 부르고 보통 `h`로 씁니다. 식으로 쓰면 이렇습니다.

```
새 메모 h_t = tanh( x_t @ W_x  +  h_(t-1) @ W_h  +  b )
              지금 읽은 것    지난 메모
```

6장 6.1절의 층 하나(입력 @ 가중치 + 편향)에 "지난 메모 @ 가중치"라는 항이 하나 더 붙었을 뿐이에요. 바깥의 `tanh`는 6장 6.2절에서 본 활성화 함수로, 메모의 값을 -1과 1 사이로 묶어 줍니다. 손으로 만들어 보겠습니다.

```
import torch

torch.manual_seed(0)
입력_크기, 메모_크기 = 3, 4                # 한 시점의 특성 3개, 메모장 칸 4개
W_x = torch.randn(입력_크기, 메모_크기) * 0.5   # 새로 읽은 것 → 메모
W_h = torch.randn(메모_크기, 메모_크기) * 0.5   # 지난 메모 → 새 메모
b = torch.zeros(메모_크기)


def rnn_셀(x: torch.Tensor, h: torch.Tensor) -> torch.Tensor:
    """지금 읽은 x와 지난 메모 h로 새 메모를 쓴다."""
    return torch.tanh(x @ W_x + h @ W_h + b)


순서열 = torch.randn(2, 5, 입력_크기)      # (배치 2, 길이 5, 특성 3)
h = torch.zeros(2, 메모_크기)              # 처음에는 빈 메모장
```

```
메모들 = []
for t in range(순서열.shape[1]):           # 한 시점씩 차례로 읽는다
    h = rnn_셀(순서열[:, t], h)            # (2, 3)과 (2, 4) → (2, 4)
    메모들.append(h)
    print(f"t={t}: 첫 문장의 메모 {[round(v, 3) for v in h[0].tolist()]}")

출력 = torch.stack(메모들, dim=1)          # 시점마다의 메모를 모으면 (2, 5, 4)
print("모든 시점의 메모:", tuple(출력.shape), "마지막 메모:", tuple(h.shape))
```

```
t=0: 첫 문장의 메모 [0.034, 0.412, -0.082, -0.311]
t=1: 첫 문장의 메모 [0.419, 0.388, -0.072, 0.271]
t=2: 첫 문장의 메모 [0.203, -0.596, -0.832, 0.29]
t=3: 첫 문장의 메모 [0.395, -0.032, -0.8, -0.651]
t=4: 첫 문장의 메모 [-0.498, -0.029, 0.382, 0.437]
모든 시점의 메모: (2, 5, 4) 마지막 메모: (2, 4)
```

핵심은 `for` 문입니다. 길이 5인 순서열을 한 시점씩 꺼내 `rnn_셀`에 넣고, 돌아온 새 메모를 다음 시점에 다시 넣었어요. 처음에는 0으로 채운 빈 메모장으로 시작합니다. 시점마다 메모의 숫자가 바뀌는 게 보이죠? 지금 읽은 입력과 지난 메모가 섞여서 새 메모가 됩니다.

눈여겨볼 점이 두 가지 있습니다. 첫째, **모든 시점이 같은 `W_x`, `W_h`, `b`를 씁니다.** 시점이 5개든 500개든 가중치는 한 벌이에요. 둘째, 시점마다 나온 메모를 모으면 `(2, 5, 4)`, 즉 (배치, 길이, 메모 크기)가 되고, 마지막 메모 `h`는 `(2, 4)`입니다. 앞의 것은 "읽어 가는 동안의 모든 메모", 뒤의 것은 "다 읽고 난 뒤의 최종 메모"예요.

5장 5.5절에서 PyTorch는 "실행하면서 그래프를 그린다"고 했습니다. 그래서 이런 평범한 `for` 문도 그대로 `backward()`를 할 수 있어요. 역전파는 이 반복을 거꾸로 거슬러 올라가는데, 이것을 **시간을 거슬러 가는 역전파**(BPTT, Backpropagation Through Time) 라고 부릅니다. 이름은 거창하지만 5장 5.6절의 연쇄 법칙을 시점 수만큼 반복하는 것뿐이에요.

nn.RNN: 같은 일을 하는 부품

PyTorch에는 이 반복을 빠르게 해 주는 `nn.RNN`이 있습니다. 정말 같은 계산인지, `nn.RNN`의 가중치를 꺼내 손으로 다시 계산해 보겠습니다.

```
import torch
from torch import nn

torch.manual_seed(0)
rnn = nn.RNN(input_size=3, hidden_size=4, batch_first=True)
for 이름, p in rnn.named_parameters():
    print(f"{이름:14} {tuple(p.shape)}")

# 같은 가중치로 손으로 계산해 보기 (PyTorch는 x @ W.T 꼴로 가중치를 보관한다)
순서열 = torch.randn(2, 5, 3)
h = torch.zeros(2, 4)
for t in range(5):
    h = torch.tanh(순서열[:, t] @ rnn.weight_ih_l0.T + rnn.bias_ih_l0
                   + h @ rnn.weight_hh_l0.T + rnn.bias_hh_l0)

출력, h_n = rnn(순서열)
print("출력:", tuple(출력.shape), "h_n:", tuple(h_n.shape))
print("손 계산과 차이:", (h - h_n[0]).abs().max().item())
print("출력의 마지막 시점 == h_n?", torch.allclose(출력[:, -1], h_n[0]))
```

```
weight_ih_l0   (4, 3)
weight_hh_l0   (4, 4)
bias_ih_l0     (4,)
bias_hh_l0     (4,)
출력: (2, 5, 4) h_n: (1, 2, 4)
손 계산과 차이: 0.0
출력의 마지막 시점 == h_n? True
```

손 계산과 차이가 0입니다. 같은 계산이에요. 가중치 이름의 `ih`는 입력(input)에서 메모(hidden)로, `hh`는 메모에서 메모로 가는 가중치라는 뜻이고, 끝의 `l0`은 0번 층이라는 뜻입니다(11.8절에서 층을 쌓아요). 6장 6.1절의 `nn.Linear`처럼 가중치를 `(출력, 입력)` 모양으로 보관하기 때문에 손 계산에서는 `.T`로 전치해서 곱했습니다. 편향이 `bias_ih`와 `bias_hh` 두 개인 것은 NVIDIA GPU용 계산 라이브러리(cuDNN)의 방식에 맞춘 것일 뿐, 둘을 더한 것이 앞의 `b`와 같은 역할을 해요.

`nn.RNN`은 두 가지를 돌려줍니다.

- **출력** `(2, 5, 4)`: 모든 시점의 메모. 시점마다 무언가를 내야 하는 문제(다음 글자 맞히기 등)

에 씁니다.

- `h_n (1, 2, 4)`: 마지막 시점의 메모. 다 읽고 한 번 판단하는 문제(문장 분류 등)에 씁니다. 맨 앞의 1은 층 수예요.

마지막 줄에서 확인한 것처럼, 층이 하나일 때 `출력[:, -1]`과 `h_n[0]`은 같은 값입니다.

길이가 달라도 같은 모델

메모장의 크기와 가중치가 시점 수와 상관없으니, 한 모델에 어떤 길이든 넣을 수 있어야 합니다. 확인해 보죠.

```
import torch
from torch import nn

torch.manual_seed(0)
rnn = nn.RNN(input_size=3, hidden_size=4, batch_first=True)
print("파라미터 수:", sum(p.numel() for p in rnn.parameters()))

for 길이 in [1, 5, 500]:                  # 같은 모델에 길이가 다른 입력
    출력, h_n = rnn(torch.randn(2, 길이, 3))
    print(f"길이 {길이:3d}: 출력 {tuple(출력.shape)}, h_n {tuple(h_n.shape)}")

# 처음 메모장을 직접 넘길 수도 있다: (층 수, 배치, 메모 크기)
h0 = torch.zeros(1, 2, 4)
앞부분, h_중간 = rnn(torch.randn(2, 3, 3), h0)
뒷부분, h_끝 = rnn(torch.randn(2, 2, 3), h_중간)   # 이어서 읽기
print("이어 읽은 출력:", tuple(앞부분.shape), tuple(뒷부분.shape))

# batch_first를 빼면 (길이, 배치, 특성) 순서를 기대한다
rnn_기본 = nn.RNN(input_size=3, hidden_size=4)
출력, _ = rnn_기본(torch.randn(5, 2, 3))
print("batch_first=False 출력:", tuple(출력.shape))
```

```
파라미터 수: 36
길이   1: 출력 (2, 1, 4), h_n (1, 2, 4)
길이   5: 출력 (2, 5, 4), h_n (1, 2, 4)
길이 500: 출력 (2, 500, 4), h_n (1, 2, 4)
```

```
이어 읽은 출력: (2, 3, 4) (2, 2, 4)
batch_first=False 출력: (5, 2, 4)
```

파라미터 36개짜리 모델이 길이 1, 5, 500을 모두 받았습니다. 출력의 가운데 자리만 길이를 따라 늘어나고 `h_n`의 모양은 언제나 같아요. 파라미터 36개는 `W_x`(4×3) 12개, `W_h`(4×4) 16개, 편향 두 벌 8개를 더한 값입니다.

`rnn(입력, h0)`처럼 처음 메모를 직접 넘길 수도 있습니다. 앞부분을 읽고 받은 `h_중간`을 넘겨서 뒷부분을 **이어서 읽는** 것도 가능해요. 긴 글을 조각조각 읽거나, 11.11절처럼 글자를 하나씩 생성할 때 이 방법을 씁니다.

마지막으로, `batch_first=True`를 빼면 `nn.RNN`은 **(길이, 배치, 특성)** 순서를 기대합니다. 옛날 관례가 남은 것인데, 이걸 깜빡하면 배치와 길이가 뒤바뀐 채로 오류 없이 돌아가서 찾기 어려운 버그가 됩니다. 이 책은 언제나 `batch_first=True`를 씁니다.

11.3 순서를 알아야 풀리는 문제: 괄호 짝 맞추기

문제와 데이터

RNN이 정말 순서를 이해하는지 시험해 볼 문제가 필요합니다. 코드 편집기는 괄호의 짝이 안 맞으면 빨간 줄을 긋죠. `(()())`는 짝이 맞고, `())(()`는 맞지 않습니다. 두 괄호 모두 여는 괄호 셋, 닫는 괄호 셋으로 개수가 같은데도요. 앞에서부터 읽으며 "지금 열린 괄호가 몇 개인가"를 세다가, 0 아래로 내려가는 순간이 있으면 틀린 겁니다. 순서를 모르면 절대 풀 수 없는 문제예요.

괄호 실험은 11.9절에서도 다시 쓰니, 8장의 `학습도구.py`처럼 데이터를 만드는 코드를 모듈로 모아 두겠습니다. 아래 내용을 `괄호도구.py`로 저장하세요.

```
"""11장 괄호 실험에서 함께 쓰는 데이터 도구."""
import random

import torch
from torch.nn.utils.rnn import pad_sequence


def 짝맞나(괄호: str) -> bool:
    """여는 괄호보다 닫는 괄호가 먼저 많아지는 순간이 없고, 끝에서 개수가 같으면 짝이 맞다."""
```

```
    깊이 = 0
    for 글자 in 괄호:
        깊이 += 1 if 글자 == "(" else -1
        if 깊이 < 0:
            return False
    return 깊이 == 0


def 괄호_만들기(개수: int, 길이들: list[int], 씨앗: int) -> tuple[list[str], torch.Tensor]:
    """짝이 맞는 괄호와 맞지 않는 괄호를 반반 만든다. 두 쪽 모두 '('와 ')'의 개수는 같다."""
    rng = random.Random(씨앗)
    괄호들 = []
    while len(괄호들) < 개수:
        길이 = rng.choice(길이들)
        글자들 = list("(" * (길이 // 2) + ")" * (길이 // 2))
        rng.shuffle(글자들)
        if not 짝맞나(글자들):
            continue                                # 먼저 짝이 맞는 괄호를 하나 얻고
        if len(괄호들) % 2 == 1:                    # 홀수 번째는 두 글자를 맞바꿔 짝을 깬다
            while True:
                i, j = rng.sample(range(길이), 2)
                바꾼 = 글자들.copy()
                바꾼[i], 바꾼[j] = 바꾼[j], 바꾼[i]
                if not 짝맞나(바꾼):
                    break
            글자들 = 바꾼
        괄호들.append("".join(글자들))
    정답 = torch.tensor([int(짝맞나(괄호)) for 괄호 in 괄호들])
    return 괄호들, 정답


def 원핫으로(괄호들: list[str]) -> tuple[torch.Tensor, torch.Tensor]:
    """'('는 1번, ')'는 2번, 빈칸(패딩)은 0번 칸이 1인 원-핫 텐서와 길이를 돌려준다."""
    번호들 = [torch.tensor([1 if 글자 == "(" else 2 for 글자 in 괄호]) for 괄호 in 괄호들]
    채운 = pad_sequence(번호들, batch_first=True, padding_value=0)
    길이 = torch.tensor([len(괄호) for 괄호 in 괄호들])
```

```
    return torch.nn.functional.one_hot(채운, 3).float(), 길이
```

괄호_만들기는 짝이 맞는 괄호와 맞지 않는 괄호를 반반씩 만듭니다. 맞지 않는 괄호는 짝이 맞는 괄호에서 두 글자를 맞바꿔 만들어서, **여는 괄호와 닫는 괄호의 개수가 언제나 같게** 했어요. 개수만 세서는 절대 구별할 수 없게 일부러 만든 것입니다. **원핫으로**는 7장 7.7절의 pad_sequence로 길이를 맞춘 뒤, 4장 4.4절의 원-핫으로 바꿉니다. 0번(빈칸)을 비워 둔 것도 7장과 같아요.

```
from collections import Counter

from 괄호도구 import 괄호_만들기, 원핫으로, 짝맞나

print(짝맞나("(()())"), 짝맞나("())(()"), 짝맞나("(()"))

괄호들, 정답 = 괄호_만들기(3000, 길이들=[20], 씨앗=0)
for 괄호, 답 in zip(괄호들[:4], 정답[:4].tolist()):
    print(괄호, "짝 맞음" if 답 else "짝 틀림")
print("짝 맞음 비율:", 정답.float().mean().item())
print("'('와 ')' 개수가 늘 같은가:", all(Counter(괄호)["("] == 10 for 괄호 in 괄호들))

X, 길이 = 원핫으로(괄호들[:2])
print("원-핫 모양:", tuple(X.shape))
print(X[0, :3])
```

```
True False False
(((()(((())(((((()) 짝 맞음
()()()()()()(((()()) 짝 틀림
()((((()))(())()(())) 짝 맞음
()())(()(()())))(()() 짝 틀림
짝 맞음 비율: 0.5
'('와 ')' 개수가 늘 같은가: True
원-핫 모양: (2, 20, 3)
tensor([[0., 1., 0.],
        [0., 1., 0.],
        [0., 1., 0.]])
```

길이 20짜리 괄호 3,000개를 만들었고, 짝이 맞는 것과 틀린 것이 정확히 반반입니다. 모든 괄호가

여는 괄호 10개, 닫는 괄호 10개라는 것도 확인했어요. 원-핫 텐서의 모양 `(2, 20, 3)`은 "괄호 2개, 각각 20글자, 글자마다 숫자 3개"이고, 첫 괄호의 처음 세 글자가 모두 `(`라서 1번 칸이 1입니다.

세 모델의 대결

이제 세 모델로 풀어 보겠습니다.

- **개수만 보는 MLP**: 글자를 모두 더해 "여는 괄호 몇 개, 닫는 괄호 몇 개"만 봅니다. 들어가며에서 본 글자 개수 벡터와 같은 방식이에요.
- **펼쳐서 보는 MLP**: 20글자 × 3칸을 한 줄 60칸으로 펴서 넣습니다. 6장에서 그림을 펴서 넣던 방식이에요.
- **RNN**: 한 글자씩 읽으며 메모를 고쳐 쓰고, 마지막 메모로 판단합니다.

```
import torch
from torch import nn

from 괄호도구 import 괄호_만들기, 원핫으로

괄호_학습, y_학습 = 괄호_만들기(3000, [20], 씨앗=0)      # 길이 20짜리 괄호만
괄호_검증, y_검증 = 괄호_만들기(1000, [20], 씨앗=1)
X_학습, _ = 원핫으로(괄호_학습)                          # (3000, 20, 3)
X_검증, _ = 원핫으로(괄호_검증)


class 개수_모델(nn.Module):
    """글자 개수만 보는 모델: 순서를 버린다."""
    def __init__(self) -> None:
        super().__init__()
        self.mlp = nn.Sequential(nn.Linear(3, 32), nn.ReLU(), nn.Linear(32, 2))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.mlp(x.sum(dim=1))             # (배치, 20, 3) → (배치, 3)


class 순환_모델(nn.Module):
    """한 글자씩 읽으며 메모를 고쳐 쓰고, 마지막 메모로 판단한다."""
    def __init__(self, 메모_크기: int = 32) -> None:
```

```
        super().__init__()
        self.rnn = nn.RNN(3, 메모_크기, batch_first=True)
        self.판단 = nn.Linear(메모_크기, 2)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        _, h_n = self.rnn(x)
        return self.판단(h_n[-1])                # 마지막 층의 마지막 메모


def 학습_평가(모델: nn.Module, 에폭_수: int = 30) -> tuple[float, float]:
    torch.manual_seed(0)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=3e-3)
    for _ in range(에폭_수):
        for 묶음 in torch.randperm(len(X_학습)).split(64):
            손실 = nn.functional.cross_entropy(모델(X_학습[묶음]), y_학습[묶음])
            옵티마이저.zero_grad()
            손실.backward()
            옵티마이저.step()
    with torch.no_grad():
        정확도 = lambda X, y: (모델(X).argmax(dim=1) == y).float().mean().item()
        return 정확도(X_학습, y_학습), 정확도(X_검증, y_검증)


torch.manual_seed(0)
모델들 = {"개수만 보는 MLP": 개수_모델(),
          "펼쳐서 보는 MLP": nn.Sequential(nn.Flatten(), nn.Linear(20 * 3, 64), nn.ReLU(),
          ↪  nn.Linear(64, 2)),
          "RNN": 순환_모델()}
for 이름, 모델 in 모델들.items():
    학습, 검증 = 학습_평가(모델)
    개수 = sum(p.numel() for p in 모델.parameters())
    print(f"{이름}: 파라미터 {개수:,}개, 학습 {학습:.1%}, 검증 {검증:.1%}")
```

```
개수만 보는 MLP: 파라미터 194개, 학습 50.0%, 검증 50.0%
펼쳐서 보는 MLP: 파라미터 4,034개, 학습 100.0%, 검증 99.7%
RNN: 파라미터 1,250개, 학습 100.0%, 검증 100.0%
```

개수만 보는 MLP는 정확히 50.0%, 동전 던지기입니다. 모든 괄호의 개수가 같으니 이 모델 눈에는 3,000개가 모두 똑같아 보여요. 순서를 버리면 이 문제는 원리적으로 풀 수 없습니다.

RNN은 학습·검증 모두 100.0%입니다. 한 글자씩 읽으며 메모에 "지금 열린 괄호의 깊이"와 "이미 0 아래로 내려간 적이 있는가" 같은 정보를 담는 법을 스스로 배운 거예요.

그런데 **펼쳐서 보는 MLP**도 검증 99.7%로 잘 풉니다. 60칸이 각각 "몇 번째 자리에 무엇이 있는가"를 담고 있으니, 순서 정보가 사라지지 않았기 때문이에요. 그럼 RNN이 필요 없는 걸까요? 파라미터 수를 보세요. 펼친 MLP는 4,034개, RNN은 1,250개입니다. 펼친 MLP는 1번 자리, 2번 자리, ..., 20번 자리를 위한 가중치를 **따로따로** 배워야 하지만, RNN은 모든 자리에 같은 가중치를 돌려 씁니다. 그 차이가 길이를 바꿨을 때 드러납니다.

처음 보는 길이

학습할 때는 길이 20만 봤습니다. 길이 24, 30, 40짜리 괄호를 주면 어떻게 될까요?

```
import torch
from torch import nn

from 괄호도구 import 괄호_만들기, 원핫으로

괄호_학습, y_학습 = 괄호_만들기(3000, [20], 씨앗=0)
X_학습, _ = 원핫으로(괄호_학습)

torch.manual_seed(0)
rnn = nn.RNN(3, 32, batch_first=True)
판단 = nn.Linear(32, 2)
모델 = lambda x: 판단(rnn(x)[1][-1])              # 마지막 메모로 판단
옵티마이저 = torch.optim.Adam([*rnn.parameters(), *판단.parameters()], lr=3e-3)
for _ in range(30):                                # 앞 예제와 같은 학습
    for 묶음 in torch.randperm(len(X_학습)).split(64):
        손실 = nn.functional.cross_entropy(모델(X_학습[묶음]), y_학습[묶음])
        옵티마이저.zero_grad()
        손실.backward()
        옵티마이저.step()

펼친_MLP = nn.Sequential(nn.Flatten(), nn.Linear(20 * 3, 64), nn.ReLU(), nn.Linear(64, 2))
for 길이 in [20, 24, 30, 40]:
```

```
    괄호들, 정답 = 괄호_만들기(1000, [길이], 씨앗=2)
    X, _ = 원핫으로(괄호들)
    with torch.no_grad():
        정확도 = (모델(X).argmax(dim=1) == 정답).float().mean().item()
    try:
        펼친_MLP(X)
        mlp_결과 = "실행됨"
    except RuntimeError as e:
        mlp_결과 = "모양 오류 " + str(e)[str(e).index("("):]
    print(f"길이 {길이}: RNN 정확도 {정확도:.1%} / 펼친 MLP {mlp_결과}")
```

```
길이 20: RNN 정확도 100.0% / 펼친 MLP 실행됨
길이 24: RNN 정확도 86.6% / 펼친 MLP 모양 오류 (1000x72 and 60x64)
길이 30: RNN 정확도 57.5% / 펼친 MLP 모양 오류 (1000x90 and 60x64)
길이 40: RNN 정확도 48.2% / 펼친 MLP 모양 오류 (1000x120 and 60x64)
```

펼친 MLP는 길이 24부터 아예 실행이 안 됩니다. 첫 층이 60칸짜리 입력만 받도록 만들어졌는데 72칸(24 × 3)이 들어왔다는 모양 오류예요(4장 4.10절의 오류 읽는 법 그대로입니다). 문장 길이가 바뀔 때마다 모델을 새로 만들어야 한다면 쓸 수가 없겠죠.

RNN은 어떤 길이든 받아 줍니다. 하지만 결과는 정직하게 봐야 해요. 길이 24에서는 86.6%로 꽤 맞히지만, 30에서 57.5%, 40에서는 48.2%로 동전 던지기가 됩니다. **길이를 받아 줄 수 있는 것과 처음 보는 길이에서 잘하는 것은 다른 문제**예요. 길이 20만 본 모델은 "열린 괄호가 열 개 넘게 쌓이는" 상황을 겪어 본 적이 없으니 당연합니다. 11.9절에서 여러 길이를 섞어 학습하면 이 숫자가 어떻게 바뀌는지 다시 봅니다.

11.4 기울기 소실과 폭주

거듭 곱하기의 함정

RNN이 길이 20의 괄호는 잘 풀었습니다. 그럼 길이 100, 1000인 문장도 잘 기억할까요? 여기에 RNN의 가장 큰 약점이 숨어 있습니다.

어릴 때 하던 귓속말 전달 놀이를 떠올려 보세요. 열 명을 거치면 처음 말이 엉뚱하게 바뀌어 있죠. 한 사람이 전할 때마다 조금씩 흐려지는 것이 쌓이기 때문입니다. RNN의 역전파도 똑같아요. 5장 5.6절에서 연쇄 법칙은 **단계마다의 기울기를 곱하는 것**이라고 했습니다. 100번째 시점의 손실이 1번째 시

점의 입력까지 거슬러 가려면 100단계의 기울기를 곱해야 해요. 같은 크기의 수를 100번 곱하면 어떻게 될까요?

```
import torch

# 같은 수를 거듭 곱하면: 1보다 조금 작으면 사라지고, 조금 크면 터진다
for 배율 in [0.5, 0.9, 1.0, 1.1, 1.5]:
    결과 = [배율 ** 횟수 for 횟수 in (10, 50, 100)]
    print(f"{배율} 거듭 곱하기  10번 {결과[0]:9.3g}  50번 {결과[1]:9.3g}  100번 {결과[2]:9.3g}")

# 역전파의 한 단계에는 tanh의 기울기(1 - tanh²)가 곱해진다. 언제나 1 이하
z = torch.tensor([0.0, 1.0, 2.0, 3.0], requires_grad=True)
torch.tanh(z).sum().backward()
print("tanh의 기울기:", [round(v, 4) for v in z.grad.tolist()])
```

```
0.5 거듭 곱하기  10번  0.000977  50번  8.88e-16  100번  7.89e-31
0.9 거듭 곱하기  10번     0.349  50번   0.00515  100번  2.66e-05
1.0 거듭 곱하기  10번         1  50번         1  100번         1
1.1 거듭 곱하기  10번      2.59  50번       117  100번  1.38e+04
1.5 거듭 곱하기  10번      57.7  50번  6.38e+08  100번  4.07e+17
tanh의 기울기: [1.0, 0.42, 0.0707, 0.0099]
```

0.9를 100번 곱하면 0.0000266, 0.5를 100번 곱하면 10의 -31제곱입니다. 사실상 0이에요. 반대로 1.1을 100번 곱하면 1만 3천, 1.5는 10의 17제곱으로 터져 버립니다. 정확히 1일 때만 그대로 유지되는데, 학습 중에 모든 단계가 정확히 1이기를 바랄 수는 없죠.

게다가 RNN의 한 단계에는 `tanh`의 기울기가 곱해집니다. 마지막 줄처럼 `tanh`의 기울기는 0에서 1이 최대이고, 입력이 커질수록 0.42, 0.07, 0.01로 빠르게 작아져요. 1 이하의 수를 계속 곱하니 기울기가 줄어드는 쪽으로 기울어 있는 셈입니다.

시점별 기울기를 재 보기

실제 RNN에서 확인해 보겠습니다. 길이 100인 순서열을 넣고, 마지막 출력이 각 시점의 입력에 얼마나 민감한지를 5장 5.12절의 `torch.autograd.grad`로 재 보죠. 이 값이 작다는 것은 "그 시점의 입력을 바꿔도 마지막 출력이 거의 안 변한다", 즉 **그 시점을 기억하지 못한다**는 뜻이고, 학습할 때 그 시점까지 고칠 신호가 닿지 않는다는 뜻이기도 합니다.

```
import torch
from torch import nn

torch.manual_seed(0)
길이 = 100
x = torch.randn(1, 길이, 8, requires_grad=True)    # 길이 100짜리 순서열 하나


def 시점별_기울기(rnn: nn.RNN) -> torch.Tensor:
    """마지막 출력이 각 시점의 입력에 얼마나 민감한지(기울기의 크기)를 잰다."""
    출력, _ = rnn(x)
    기울기, = torch.autograd.grad(출력[0, -1].sum(), x)
    return 기울기[0].norm(dim=1)                    # 시점마다 하나의 크기


torch.manual_seed(0)
기본 = nn.RNN(8, 32, batch_first=True)
torch.manual_seed(0)
큰_가중치 = nn.RNN(8, 32, batch_first=True)
with torch.no_grad():
    큰_가중치.weight_hh_l0.mul_(4)                # 메모 → 메모 가중치만 4배로

시점들 = [99, 95, 90, 80, 50, 0]
print("시점      " + "".join(f"{t:>10}" for t in 시점들))
for 이름, rnn in [("기본 RNN", 기본), ("W_h 4배", 큰_가중치)]:
    크기 = 시점별_기울기(rnn)
    print(f"{이름:8}" + "".join(f"{크기[t].item():10.2e}" for t in 시점들))
```

```
시점              99        95        90        80        50         0
기본 RNN    2.02e+00  1.85e-01  5.55e-03  9.52e-06  3.01e-14  0.00e+00
W_h 4배    7.35e-01  1.43e+00  9.16e+00  2.64e+01  5.81e+02  1.67e+07
```

기본 RNN의 줄을 오른쪽으로 읽어 보세요. 마지막 시점(99)에서 2.02이던 기울기가 4시점 전에 0.185, 9시점 전에 0.0056, 19시점 전에 0.0000095로 줄어들고, 49시점 전에는 10의 -14제곱, 맨 처음 시점에서는 0이 됩니다. 이 RNN에게 100시점 전의 입력은 없었던 것과 같아요. 9장 9.8절에서 층을 깊게 쌓을 때 본 **기울기 소실(vanishing gradient)** 이, 여기서는 층 대신 시간 방향으로 일어난

거예요. 학습해도 먼 과거와 지금을 잇는 규칙을 배울 수가 없어요.

메모 → 메모 가중치 `W_h`만 4배로 키우면 반대 현상이 납니다. 과거로 갈수록 기울기가 커져서 맨 처음 시점에서는 1,670만이 돼요. 이것이 8장 8.9절에서 본 **기울기 폭주(exploding gradient)** 입니다. 8장에서 예고한 대로 RNN처럼 같은 계산을 반복하는 모델에서 특히 잘 생기는 이유가 바로 이 거듭곱하기예요.

두 문제의 처방은 다릅니다.

- **폭주**는 비교적 쉽게 막습니다. 8장 8.9절의 `clip_grad_norm_`으로 한 걸음의 크기에 상한을 두면 돼요. 그래서 순환 신경망을 학습할 때는 클리핑을 거의 기본으로 켭니다. 이 장의 학습 예제에도 대부분 들어 있어요.
- **소실**은 클리핑으로 해결되지 않습니다. 작아진 기울기를 키울 방법이 없으니까요. 구조 자체를 바꿔야 합니다. 그 답이 다음 절의 LSTM입니다.

Q. 활성화 함수를 ReLU로 바꾸면 기울기가 1이니 해결되지 않나요? `nn.RNN(..., nonlinearity="relu")`로 바꿀 수 있고, 기울기가 줄어드는 정도는 덜합니다. 하지만 ReLU는 위로 상한이 없어서 메모의 값이 시점마다 커질 수 있고, 그러면 이번에는 폭주가 쉽게 일어나요. 결국 `W_h`를 거듭 곱하는 구조가 그대로인 한, 줄어들거나 터지거나 둘 중 하나를 피하기 어렵습니다. 그래서 실무에서는 활성화 함수를 바꾸기보다 LSTM이나 GRU를 씁니다.

11.5 LSTM: 기억을 따로 나르는 벨트

메모장과 벨트

LSTM(Long Short-Term Memory) 의 아이디어는 간단합니다. 메모를 매번 **새로 쓰지 말고, 지울 것만 지우고 더할 것만 더하자**는 거예요.

공장의 컨베이어 벨트를 떠올려 보세요. 벨트 위의 물건은 그대로 다음 칸으로 흘러갑니다. 각 작업대에서는 벨트에서 필요 없는 물건을 **덜어 내고**, 새 물건을 **올려놓고**, 지금 필요한 물건만 **꺼내서** 씁니다. 벨트 위의 물건을 매번 몽땅 다시 만드는 게 아니라 조금씩 고치기만 하니, 아무도 건드리지 않은 물건은 먼 곳까지 그대로 실려 갑니다.

LSTM에서 이 벨트를 **셀 상태(cell state)** `c`라고 부르고, 덜어 내기·올려놓기·꺼내기의 양을 정하는 장치를 **문(gate)** 이라고 부릅니다. 문은 0에서 1 사이의 값으로, 0이면 닫힘, 1이면 활짝 열림이에요. 6장 6.2절의 시그모이드가 바로 0~1 사이 값을 만드는 함수라서 문마다 시그모이드를 씁니다.

이름	하는 일	값의 범위
망각문 `f`	벨트 위의 기억을 얼마나 **남길지**	0~1 (시그모이드)
입력문 `i`	새 후보를 얼마나 **올려놓을지**	0~1 (시그모이드)
후보 `g`	올려놓을 새 내용	-1~1 (tanh)
출력문 `o`	벨트에서 지금 얼마나 **꺼내 보일지**	0~1 (시그모이드)

그리고 한 시점의 계산은 이 두 줄이 전부입니다.

```
c_t = f * c_(t-1) + i * g          벨트: 남길 만큼 남기고, 올릴 만큼 더한다
h_t = o * tanh(c_t)                메모: 벨트에서 꺼내 보일 만큼만
```

손으로 만드는 LSTM 셀

RNN 때처럼 `nn.LSTM`의 가중치를 꺼내 손으로 계산하고, 결과를 비교해 보겠습니다.

```
import torch
from torch import nn

torch.manual_seed(0)
lstm = nn.LSTM(input_size=3, hidden_size=4, batch_first=True)
print("weight_ih_l0:", tuple(lstm.weight_ih_l0.shape), "weight_hh_l0:",
↪  tuple(lstm.weight_hh_l0.shape))



def lstm_셀(x: torch.Tensor, h: torch.Tensor, c: torch.Tensor) -> tuple[torch.Tensor,
↪  torch.Tensor]:
    """문 세 개와 새 내용 후보로 기억 c와 메모 h를 고쳐 쓴다."""
    점수 = (x @ lstm.weight_ih_l0.T + lstm.bias_ih_l0
            + h @ lstm.weight_hh_l0.T + lstm.bias_hh_l0)      # (배치, 4 × 4)
    i, f, g, o = 점수.chunk(4, dim=1)          # PyTorch의 순서: 입력·망각·후보·출력
    입력문, 망각문, 출력문 = torch.sigmoid(i), torch.sigmoid(f), torch.sigmoid(o)
    후보 = torch.tanh(g)
    c = 망각문 * c + 입력문 * 후보            # 지울 만큼 지우고, 적을 만큼 더한다
    h = 출력문 * torch.tanh(c)                # 기억 중 지금 꺼내 보일 만큼만
```

```
    return h, c


순서열 = torch.randn(2, 5, 3)
h, c = torch.zeros(2, 4), torch.zeros(2, 4)
for t in range(5):
    h, c = lstm_셀(순서열[:, t], h, c)

출력, (h_n, c_n) = lstm(순서열)            # LSTM은 (h_n, c_n) 두 개를 돌려준다
print("출력:", tuple(출력.shape), "h_n:", tuple(h_n.shape), "c_n:", tuple(c_n.shape))
print("h 차이:", (h - h_n[0]).abs().max().item(), " c 차이:", (c - c_n[0]).abs().max().item())
print("기억 c :", [round(v, 3) for v in c[0].tolist()])
print("메모 h :", [round(v, 3) for v in h[0].tolist()])
```

```
weight_ih_l0: (16, 3) weight_hh_l0: (16, 4)
출력: (2, 5, 4) h_n: (1, 2, 4) c_n: (1, 2, 4)
h 차이: 0.0  c 차이: 0.0
기억 c : [0.123, 0.341, -0.288, 0.479]
메모 h : [0.076, 0.158, -0.147, 0.244]
```

이번에도 차이가 0입니다. 코드를 보면 문 넷의 점수를 한꺼번에 계산했어요. 가중치의 줄 수가 메모 크기 4의 4배인 16인 이유입니다. 이렇게 계산한 `(배치, 16)`을 `chunk(4, dim=1)`로 네 조각으로 나누면 차례대로 입력문, 망각문, 후보, 출력문의 점수가 돼요. 이 순서(i, f, g, o)는 PyTorch가 정한 약속이라, 손으로 가중치를 다룰 때 꼭 지켜야 합니다.

`nn.LSTM`은 `(h_n, c_n)` 두 개를 묶어서 돌려줍니다. 메모 `h`와 벨트 `c`를 모두 다음으로 넘겨야 이어서 읽을 수 있기 때문이에요. 마지막 두 줄을 보면 `h`의 값이 `c`보다 작습니다. 벨트의 내용에 `tanh`를 씌우고 출력문으로 한 번 더 걸렀으니까요.

왜 기울기가 살아남을까

핵심은 벨트의 식 `c_t = f * c_(t-1) + i * g`에 있습니다. RNN은 매번 `tanh(… h @ W_h …)`로 메모를 **새로 만들어서**, 거꾸로 갈 때마다 `W_h`와 `tanh`의 기울기가 곱해졌어요. LSTM의 벨트는 이전 값에 망각문 `f`를 곱하고 무언가를 **더할** 뿐입니다. 거꾸로 갈 때 한 단계마다 곱해지는 것은 사실상 `f` 하나예요. 모델이 "이건 기억해야 해"라고 판단하면 망각문을 1 가까이 열어 둘 수 있고, 그러면 기울기도 거의 줄지 않고 먼 과거까지 흘러갑니다.

중요한 건 망각문의 값을 **모델이 배운다**는 점이에요. 처음부터 기억력이 좋은 게 아니라, 기억이 필요하면 문을 열어 두는 법을 배울 수 있는 구조입니다. 11.7절에서 이 차이를 숫자로 확인합니다.

11.6 GRU와 파라미터 비용

문을 두 개로 줄인 GRU

LSTM은 잘 동작하지만 문이 셋에 벨트까지 따로 있어서 복잡합니다. 2014년에 나온 GRU(Gated Recurrent Unit)는 이걸 줄였어요. 벨트와 메모를 하나로 합치고, 문을 둘만 둡니다.

- **갱신문** `z`: 지난 메모를 얼마나 남기고 새 후보로 얼마나 바꿀지. LSTM의 망각문과 입력문을 하나로 묶은 것입니다. `새 메모 = (1 - z) * 새 후보 + z * 지난 메모`처럼 둘을 섞어요.
- **리셋문** `r`: 새 후보를 만들 때 지난 메모를 얼마나 참고할지.

"남기는 양"과 "바꾸는 양"을 합쳐서 1이 되게 묶은 것이 GRU의 요령입니다. 뭔가를 남기면 그만큼 덜 바꾸는 거죠. 세 가지의 크기를 비교해 보겠습니다.

```
import torch
from torch import nn

입력_크기, 메모_크기 = 8, 32
for 종류 in [nn.RNN, nn.GRU, nn.LSTM]:
    torch.manual_seed(0)
    층 = 종류(입력_크기, 메모_크기, batch_first=True)
    개수 = sum(p.numel() for p in 층.parameters())
    출력, 상태 = 층(torch.randn(2, 10, 입력_크기))
    상태_모양 = [tuple(s.shape) for s in 상태] if isinstance(상태, tuple) else tuple(상태.shape)
    print(f"{종류.__name__:4} 파라미터 {개수:5,}개  가중치 줄 수 {층.weight_ih_l0.shape[0]:3}  상태
    ↪ {상태_모양}")

기본 = 메모_크기 * 입력_크기 + 메모_크기 * 메모_크기 + 2 * 메모_크기
print(f"RNN 한 벌: {기본:,}개 → GRU는 3벌 {3 * 기본:,}개, LSTM은 4벌 {4 * 기본:,}개")
```

```
RNN  파라미터 1,344개  가중치 줄 수  32  상태 (1, 2, 32)
GRU  파라미터 4,032개  가중치 줄 수  96  상태 (1, 2, 32)
LSTM 파라미터 5,376개  가중치 줄 수 128  상태 [(1, 2, 32), (1, 2, 32)]
```

```
RNN 한 벌: 1,344개 → GRU는 3벌 4,032개, LSTM은 4벌 5,376개
```

같은 메모 크기 32에서 RNN은 1,344개, GRU는 그 3배인 4,032개, LSTM은 4배인 5,376개입니다. RNN 한 벌의 계산을 GRU는 세 번(문 둘과 후보 하나), LSTM은 네 번(문 셋과 후보 하나) 하기 때문이에요. 가중치의 줄 수 32, 96, 128에 그대로 드러납니다.

상태의 모양도 보세요. RNN과 GRU는 `h_n` 하나, LSTM만 `(h_n, c_n)` 두 개입니다. 그래서 RNN이나 GRU로 짠 코드를 LSTM으로 바꿀 때 `_, h_n = 모델(x)`가 `_, (h_n, c_n) = 모델(x)`로 바뀌어야 해요. 이 장의 예제들이 `출력, _ = 순환(x)`처럼 출력만 받거나 `isinstance(상태, tuple)`로 확인하는 이유입니다.

Q

Q. LSTM과 GRU 중 무엇을 써야 하나요? 정답은 없습니다. 여러 연구에서 둘의 성능은 문제마다 엎치락뒤치락했어요. GRU는 파라미터가 적어 조금 빠르고 작은 데이터에서 유리할 때가 있고, LSTM은 더 오래 쓰여서 참고할 자료가 많습니다. 8장에서 말한 원칙대로, 둘 다 해 보고 **검증 데이터로** 고르세요. 바로 다음 절에서 셋을 한 문제로 비교해 봅니다.

11.7 긴 기억 시험: 멀리 떨어진 두 수 더하기

더하기 문제

11.4절에서 RNN의 기울기가 과거로 갈수록 사라지는 것을 봤습니다. 그럼 실제로 **멀리 떨어진 정보를 써야 하는 문제**를 RNN이 못 푸는지, LSTM과 GRU는 푸는지 확인해 보겠습니다. LSTM을 처음 제안한 논문에서도 쓴 고전적인 시험인 **더하기 문제(adding problem)** 입니다.

0~1 사이 숫자가 한 줄로 늘어서 있고, 그중 두 칸에만 표시가 붙어 있습니다. 하나는 앞 절반에, 하나는 뒤 절반에 있어요. 모델은 한 칸씩 읽으며 숫자와 표시를 보고, 다 읽은 뒤 **표시된 두 숫자의 합**을 말해야 합니다. 앞 절반의 숫자를 줄 끝까지 기억하고 있어야 풀 수 있죠.

```
import torch


def 더하기_문제(개수: int, 길이: int, g: torch.Generator) -> tuple[torch.Tensor, torch.Tensor]:
    """0~1 사이 숫자 줄에서 표시된 두 칸(앞 절반에 하나, 뒤 절반에 하나)의 합을 맞히는 문제."""
    값 = torch.rand(개수, 길이, generator=g)
```

```
    표시 = torch.zeros(개수, 길이)
    앞 = torch.randint(0, 길이 // 2, (개수,), generator=g)
    뒤 = torch.randint(길이 // 2, 길이, (개수,), generator=g)
    표시[torch.arange(개수), 앞] = 1.0
    표시[torch.arange(개수), 뒤] = 1.0
    x = torch.stack([값, 표시], dim=-1)          # (개수, 길이, 특성 2): 숫자와 표시
    y = (값 * 표시).sum(dim=1, keepdim=True)      # 표시된 두 숫자의 합
    return x, y


g = torch.Generator().manual_seed(0)
x, y = 더하기_문제(1, 10, g)
print("숫자:", [round(v, 2) for v in x[0, :, 0].tolist()])
print("표시:", x[0, :, 1].int().tolist())
print("정답:", round(y.item(), 4))

# 아무것도 못 배운 모델은 늘 평균(1.0)을 말한다. 그때의 MSE는?
_, y = 더하기_문제(100_000, 50, g)
print(f"늘 1.0이라고 답할 때 MSE: {((y - 1.0) ** 2).mean().item():.4f}")
```

```
숫자: [0.5, 0.77, 0.09, 0.13, 0.31, 0.63, 0.49, 0.9, 0.46, 0.63]
표시: [0, 1, 0, 0, 0, 0, 1, 0, 0, 0]
정답: 1.2583
늘 1.0이라고 답할 때 MSE: 0.1663
```

길이 10짜리 문제 하나를 보면 1번과 6번 칸에 표시가 있고, 정답은 0.77 + 0.49 = 1.2583입니다. 입력의 모양은 `(개수, 길이, 2)`로, 한 시점마다 "숫자, 표시" 두 개를 봐요.

마지막 줄은 이 문제의 **기준선**입니다. 아무것도 못 배운 모델은 늘 정답의 평균인 1.0 근처를 말하게 되는데, 그때의 MSE가 약 0.166이에요. 학습한 모델의 MSE가 0.166 근처라면 **아무것도 못 배운 것**이고, 0에 가까울수록 잘 배운 것입니다. 8장 8.11절에서 첫 손실을 확인하던 것처럼, 결과를 보기 전에 "못 배우면 얼마인가"를 먼저 알아 두는 습관이에요.

RNN, LSTM, GRU 비교

길이 10과 50에서 세 모델을 같은 조건으로 학습합니다. 매번 새 문제 64개를 만들어 1,500번 학습하고, 처음 보는 시험 문제 1,000개로 MSE를 잽니다. 시험 문제에서 첫 시점까지 기울기가 얼마나 닿는지도 함께 재 보겠습니다.

```
import torch
from torch import nn


def 더하기_문제(개수: int, 길이: int, g: torch.Generator) -> tuple[torch.Tensor, torch.Tensor]:
    """앞 예제와 같은 문제: 표시된 두 숫자의 합 맞히기."""
    값 = torch.rand(개수, 길이, generator=g)
    표시 = torch.zeros(개수, 길이)
    표시[torch.arange(개수), torch.randint(0, 길이 // 2, (개수,), generator=g)] = 1.0
    표시[torch.arange(개수), torch.randint(길이 // 2, 길이, (개수,), generator=g)] = 1.0
    return torch.stack([값, 표시], dim=-1), (값 * 표시).sum(dim=1, keepdim=True)


class 더하기_모델(nn.Module):
    def __init__(self, 종류: type[nn.Module]) -> None:
        super().__init__()
        self.순환 = 종류(2, 32, batch_first=True)    # nn.RNN, nn.LSTM, nn.GRU 중 하나
        self.답 = nn.Linear(32, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        출력, _ = self.순환(x)                       # LSTM의 (h, c)도 _로 받아 넘긴다
        return self.답(출력[:, -1])                  # 마지막 시점의 메모로 답한다


def 실험(종류: type[nn.Module], 길이: int) -> tuple[float, float, list[float]]:
    torch.manual_seed(0)
    모델 = 더하기_모델(종류)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=3e-3)
    g = torch.Generator().manual_seed(0)
    기록 = []
    for 단계 in range(1, 1501):                      # 매번 새 문제 64개로 1,500번
```

```
        x, y = 더하기_문제(64, 길이, g)
        손실 = nn.functional.mse_loss(모델(x), y)
        옵티마이저.zero_grad()
        손실.backward()
        nn.utils.clip_grad_norm_(모델.parameters(), max_norm=1.0)   # 8.9절의 안전벨트
        옵티마이저.step()
        if 단계 % 300 == 0:
            기록.append(round(손실.item(), 3))
    x, y = 더하기_문제(1000, 길이, torch.Generator().manual_seed(99))  # 시험 문제
    x.requires_grad_(True)
    예측 = 모델(x)
    기울기, = torch.autograd.grad(예측.sum(), x)        # 첫 시점까지 기울기가 닿는가
    return nn.functional.mse_loss(예측, y).item(), 기울기[:, 0].norm(dim=1).mean().item(), 기록


for 길이 in [10, 50]:
    for 종류 in [nn.RNN, nn.LSTM, nn.GRU]:
        mse, 첫_기울기, 기록 = 실험(종류, 길이)
        print(f"길이 {길이:2d}  {종류.__name__:4}  시험 MSE {mse:.4f}  첫 시점 기울기 {첫_기울
        ↪  기:.1e}")
        print(f"          300번마다 학습 손실 {기록}")
```

```
길이 10  RNN   시험 MSE 0.0107  첫 시점 기울기 3.0e-01
          300번마다 학습 손실 [0.143, 0.062, 0.033, 0.014, 0.011]
길이 10  LSTM  시험 MSE 0.0002  첫 시점 기울기 3.5e-01
          300번마다 학습 손실 [0.088, 0.008, 0.001, 0.0, 0.0]
길이 10  GRU   시험 MSE 0.0002  첫 시점 기울기 3.4e-01
          300번마다 학습 손실 [0.008, 0.002, 0.001, 0.0, 0.0]
길이 50  RNN   시험 MSE 0.1582  첫 시점 기울기 2.5e-08
          300번마다 학습 손실 [0.164, 0.156, 0.182, 0.16, 0.174]
길이 50  LSTM  시험 MSE 0.0026  첫 시점 기울기 2.7e-01
          300번마다 학습 손실 [0.16, 0.154, 0.025, 0.007, 0.003]
길이 50  GRU   시험 MSE 0.0005  첫 시점 기울기 8.7e-02
          300번마다 학습 손실 [0.15, 0.004, 0.002, 0.001, 0.001]
```

길이 10에서는 셋 다 잘 풉니다. RNN의 MSE 0.0107도 기준선 0.166에 비하면 아주 작아요. 표시된

숫자가 기껏해야 9칸 전에 있으니 RNN의 기억으로도 충분합니다.

길이 50에서 결과가 갈립니다. RNN의 MSE는 0.1582로 기준선 0.166과 거의 같아요. 1,500번을 학습하고도 **아무것도 배우지 못한** 것입니다. 반면 LSTM은 0.0026, GRU는 0.0005로 길이 10일 때와 크게 다르지 않게 풀었습니다.

이유는 마지막 열에 있습니다. 학습을 마친 모델에서 첫 시점까지 닿는 기울기의 크기가 RNN은 0.000000025(2.5e-08)로 사실상 0인데, LSTM은 0.27, GRU는 0.087이에요. 11.4절에서 학습 전의 RNN으로 잰 기울기 소실이 학습 뒤에도 그대로 남아 있고, LSTM과 GRU는 먼 과거까지 고칠 신호를 전달하고 있습니다. 이 신호가 없으면 "앞 절반에 표시된 숫자를 기억해야 한다"는 규칙 자체를 배울 수가 없어요.

300번마다 찍은 학습 손실도 보세요. 길이 50의 RNN은 1,500번 내내 0.16~0.18에 머물렀습니다. 그런데 LSTM도 300번째, 600번째에는 0.16, 0.154로 RNN과 다를 바 없다가, 900번째에 0.025로 뚝 떨어져요. 망각문을 여는 법을 배우는 데 시간이 걸린 거예요. 그래서 순환 신경망의 학습 곡선은 한동안 평평하다가 갑자기 내려가는 모양이 흔합니다. 초반에 손실이 안 줄어든다고 너무 일찍 포기하지 마세요.

11.8 층 쌓기와 양방향

두 층, 그리고 거꾸로 읽기

6장에서 층을 쌓아 MLP를 깊게 만들었듯, 순환 신경망도 층을 쌓을 수 있습니다. 첫 층이 시점마다 낸 메모 줄 전체를 두 번째 층이 다시 순서대로 읽는 방식이에요. 또 하나의 선택지는 **양방향(bidirectional)** 입니다. 문장을 앞에서 뒤로 한 번, 뒤에서 앞으로 한 번 읽고 두 결과를 이어 붙여요. "배가 아파서 병원에 갔다"의 "배"가 과일인지 몸인지는 **뒤에 나오는** "아파서"를 봐야 알 수 있죠. 양방향은 시점마다 앞뒤 문맥을 모두 보게 해 줍니다.

```
import torch
from torch import nn

torch.manual_seed(0)
x = torch.randn(2, 7, 8)                        # (배치 2, 길이 7, 특성 8)

설정들 = {"한 층": {}, "두 층": dict(num_layers=2, dropout=0.2),
```

```
        "양방향": dict(bidirectional=True), "두 층 양방향": dict(num_layers=2,
        ↪ bidirectional=True)}
for 이름, 설정 in 설정들.items():
    lstm = nn.LSTM(8, 16, batch_first=True, **설정)
    출력, (h_n, c_n) = lstm(x)
    개수 = sum(p.numel() for p in lstm.parameters())
    print(f"{이름}: 출력 {tuple(출력.shape)}, h_n {tuple(h_n.shape)}, 파라미터 {개수:,}개")

# 양방향 출력의 앞 16칸은 정방향, 뒤 16칸은 역방향
양방향 = nn.LSTM(8, 16, batch_first=True, bidirectional=True)
출력, (h_n, _) = 양방향(x)
print("정방향 마지막 = 출력[:, -1, :16]?", torch.allclose(h_n[0], 출력[:, -1, :16]))
print("역방향 마지막 = 출력[:, 0, 16:]?", torch.allclose(h_n[1], 출력[:, 0, 16:]))
양쪽_요약 = torch.cat([h_n[-2], h_n[-1]], dim=1)   # 분류에 쓸 때는 둘을 이어 붙인다
print("양쪽 요약:", tuple(양쪽_요약.shape))
```

```
한 층: 출력 (2, 7, 16), h_n (1, 2, 16), 파라미터 1,664개
두 층: 출력 (2, 7, 16), h_n (2, 2, 16), 파라미터 3,840개
양방향: 출력 (2, 7, 32), h_n (2, 2, 16), 파라미터 3,328개
두 층 양방향: 출력 (2, 7, 32), h_n (4, 2, 16), 파라미터 9,728개
정방향 마지막 = 출력[:, -1, :16]? True
역방향 마지막 = 출력[:, 0, 16:]? True
양쪽 요약: (2, 32)
```

모양을 하나씩 읽어 보세요.

- **두 층**: 출력은 마지막 층의 메모라서 `(2, 7, 16)` 그대로이고, `h_n`의 첫 자리가 층 수 2가 됐습니다. `dropout=0.2`는 8장 8.3절의 드롭아웃을 **층과 층 사이**에 넣으라는 뜻이에요. 층이 하나면 넣을 자리가 없어서 쓸 수 없습니다.
- **양방향**: 시점마다 정방향 메모 16칸과 역방향 메모 16칸을 이어 붙여 출력이 `(2, 7, 32)`가 됩니다. `h_n`의 첫 자리 2는 층 수가 아니라 방향 수예요.
- **두 층 양방향**: `h_n`의 첫 자리가 층 수 × 방향 수 = 4입니다. 순서는 [0층 정방향, 0층 역방향, 1층 정방향, 1층 역방향]이에요.

양방향에는 함정이 하나 있습니다. 정방향의 "마지막"은 맨 끝 시점이지만, 역방향의 "마지막"은 **맨 처음 시점**이에요. 거꾸로 읽었으니까요. 그래서 역방향의 `h_n`은 `출력[:, -1]`이 아니라 `출력[:, 0,`

16:]과 같습니다. 문장 하나를 분류할 때는 보통 마지막 층의 두 방향 h_n[-2]와 h_n[-1]을 이어 붙여 씁니다.

양방향을 **쓰면 안 되는 곳**도 기억하세요. 다음 달 이산화탄소를 예측하거나 다음 글자를 생성할 때 역방향은 미래를 미리 보는 셈입니다. 2장 2.9절에서 경고한 미래 정보 누수가 모델 안에서 일어나는 거예요. 양방향은 문장 전체가 이미 주어진 분류나 태깅 문제에만 씁니다.

11.9 길이가 다른 순서열: pack_padded_sequence

패딩이 메모를 오염시킵니다

7장 7.7절에서 길이가 다른 문장을 pad_sequence로 0을 채워 묶었고, "11장의 순환 신경망은 길이를 받아 채운 칸을 건너뛸 수 있다"고 예고했습니다. 그런데 왜 건너뛰어야 할까요? 채운 0을 RNN이 그냥 읽으면 무슨 일이 생기는지 보겠습니다.

```
import torch
from torch import nn
from torch.nn.utils.rnn import pad_sequence

torch.manual_seed(0)
rnn = nn.RNN(2, 4, batch_first=True)
짧은 = torch.randn(3, 2)                          # 길이 3
긴 = torch.randn(6, 2)                            # 길이 6
배치 = pad_sequence([짧은, 긴], batch_first=True)  # 짧은 쪽 뒤를 0으로 채워 (2, 6, 2)
print("패딩한 배치:", tuple(배치.shape))

출력, h_n = rnn(배치)
_, h_혼자 = rnn(짧은.unsqueeze(0))                 # 짧은 문장만 따로 읽었을 때
print("혼자 읽은 마지막 메모:", [round(v, 3) for v in h_혼자[0, 0].tolist()])
print("배치의 h_n          :", [round(v, 3) for v in h_n[0, 0].tolist()])
print("배치의 3번째 시점    :", [round(v, 3) for v in 출력[0, 2].tolist()])
```

```
패딩한 배치: (2, 6, 2)
혼자 읽은 마지막 메모: [0.172, -0.593, 0.522, 0.458]
배치의 h_n          : [-0.349, -0.04, 0.226, -0.086]
배치의 3번째 시점    : [0.172, -0.593, 0.522, 0.458]
```

길이 3인 문장을 혼자 읽었을 때의 마지막 메모와, 길이 6에 맞춰 0을 채운 배치에서 받은 h_n이 다릅니다. RNN이 진짜 문장 3글자를 읽은 뒤에도 채운 0을 세 번 더 읽으며 메모를 계속 고쳐 썼기 때문이에요. 셋째 줄을 보면 배치의 3번째 시점 출력은 혼자 읽은 결과와 같습니다. 진짜 마지막 메모는 거기 있었는데, h_n은 그 뒤로 더 흘러간 메모를 준 거죠. 회의가 끝났는데 녹음기가 계속 돌아가서 정적까지 회의록에 반영된 셈입니다.

길이를 알려 주고 꾸러미로 싸기

pack_padded_sequence는 길이 정보를 받아서, 채운 칸을 빼고 **진짜 칸만 모은 꾸러미**를 만듭니다. RNN은 이 꾸러미를 받으면 문장마다 자기 길이까지만 읽어요.

```
import torch
from torch import nn
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence, pad_sequence

torch.manual_seed(0)
rnn = nn.RNN(2, 4, batch_first=True)
문장들 = [torch.randn(3, 2), torch.randn(6, 2), torch.randn(4, 2)]
길이 = torch.tensor([len(s) for s in 문장들])
배치 = pad_sequence(문장들, batch_first=True)

# 길이를 알려 주고 '꾸러미'로 싸면, 채운 칸은 아예 계산하지 않는다
꾸러미 = pack_padded_sequence(배치, 길이, batch_first=True, enforce_sorted=False)
print("실제 계산할 칸:", tuple(꾸러미.data.shape), "시점별 문장 수:", 꾸러미.batch_sizes.tolist())

출력_꾸러미, h_n = rnn(꾸러미)
for i, 문장 in enumerate(문장들):
    _, h_혼자 = rnn(문장.unsqueeze(0))
    print(f"문장 {i} (길이 {len(문장)}): 혼자 읽은 것과 같은가?", torch.allclose(h_n[0, i], h_혼자
    ↪ [0, 0], atol=1e-6))

출력, 출력_길이 = pad_packed_sequence(출력_꾸러미, batch_first=True)  # 다시 (배치, 길이, 특성)으
↪ 로
print("되돌린 출력:", tuple(출력.shape), 출력_길이.tolist())
print("채운 칸의 출력은 0:", 출력[0, 3:].abs().sum().item())
```

```
실제 계산할 칸: (13, 2) 시점별 문장 수: [3, 3, 3, 2, 1, 1]
문장 0 (길이 3): 혼자 읽은 것과 같은가? True
문장 1 (길이 6): 혼자 읽은 것과 같은가? True
문장 2 (길이 4): 혼자 읽은 것과 같은가? True
되돌린 출력: (3, 6, 4) [3, 6, 4]
채운 칸의 출력은 0: 0.0
```

꾸러미의 `data`는 `(13, 2)`입니다. 세 문장의 길이 3 + 6 + 4 = 13, 진짜 칸만 모았어요. `batch_sizes`는 시점마다 아직 읽을 문장이 몇 개 남았는지입니다. 처음 세 시점은 세 문장 모두, 4번째 시점은 길이 4 이상인 두 문장, 5·6번째 시점은 가장 긴 문장 하나뿐이에요. 이 정보 덕분에 RNN은 끝난 문장을 더 읽지 않습니다.

결과도 확인했습니다. 꾸러미로 읽은 `h_n`이 세 문장 모두 **혼자 읽은 결과와 같아요.** 꾸러미로 받은 출력은 `pad_packed_sequence`로 다시 `(배치, 길이, 특성)` 텐서로 펼 수 있고, 이때 채운 칸의 출력은 0이 됩니다. 원래 `pack_padded_sequence`는 문장을 길이순으로 정렬해서 넣어야 하는데, `enforce_sorted=False`를 주면 알아서 정렬했다가 원래 순서로 돌려줘요.

여러 길이로 학습하고, 더 긴 괄호로 시험하기

이제 11.3절의 괄호 문제를 다시 풉니다. 이번에는 길이 2부터 20까지를 섞어 학습하고, 꾸러미를 쓴 모델과 안 쓴 모델을 비교합니다. 그리고 학습 때 본 적 없는 길이 30, 40, 60으로 시험해요.

```
import torch
from torch import nn
from torch.nn.utils.rnn import pack_padded_sequence

from 괄호도구 import 괄호_만들기, 원핫으로

학습_길이들 = list(range(2, 21, 2))                # 2, 4, ..., 20 글자를 섞어서
괄호_학습, y_학습 = 괄호_만들기(6000, 학습_길이들, 씨앗=0)
X_학습, 길이_학습 = 원핫으로(괄호_학습)
print("학습 배치 모양:", tuple(X_학습.shape), "가장 짧은/긴 길이:", 길이_학습.min().item(), 길이_학
↪ 습.max().item())


class 괄호_판정기(nn.Module):
```

```
    def __init__(self, 종류: type[nn.Module], 꾸릴까: bool) -> None:
        super().__init__()
        self.꾸릴까 = 꾸릴까
        self.순환 = 종류(3, 32, batch_first=True)
        self.판단 = nn.Linear(32, 2)

    def forward(self, x: torch.Tensor, 길이: torch.Tensor) -> torch.Tensor:
        if self.꾸릴까:
            x = pack_padded_sequence(x, 길이, batch_first=True, enforce_sorted=False)
        _, 상태 = self.순환(x)
        h_n = 상태[0] if isinstance(상태, tuple) else 상태   # LSTM은 (h_n, c_n)
        return self.판단(h_n[-1])


def 정확도(모델: nn.Module, 길이들: list[int], 씨앗: int) -> float:
    괄호들, 정답 = 괄호_만들기(1000, 길이들, 씨앗)
    X, 길이 = 원핫으로(괄호들)
    with torch.no_grad():
        return (모델(X, 길이).argmax(dim=1) == 정답).float().mean().item()


print("모델         꾸러미   검증(2~20)  길이 30  길이 40  길이 60")
for 종류 in [nn.RNN, nn.LSTM]:
    for 꾸릴까 in [False, True]:
        torch.manual_seed(0)
        모델 = 괄호_판정기(종류, 꾸릴까)
        옵티마이저 = torch.optim.Adam(모델.parameters(), lr=3e-3)
        for _ in range(15):
            for 묶음 in torch.randperm(len(X_학습)).split(64):
                손실 = nn.functional.cross_entropy(모델(X_학습[묶음], 길이_학습[묶음]), y_학습[묶음])
                옵티마이저.zero_grad()
                손실.backward()
                옵티마이저.step()
        결과 = [정확도(모델, 학습_길이들, 1)] + [정확도(모델, [n], 2) for n in (30, 40, 60)]
        print(f"{종류.__name__:5}  {'쓺' if 꾸릴까 else '안 쓺':4}  " + "  ".join(f"{r:8.1%}" for r
        ↪  in 결과))
```

```
학습 배치 모양: (6000, 20, 3) 가장 짧은/긴 길이: 2 20
모델        꾸러미   검증(2~20)  길이 30  길이 40  길이 60
RNN    안 씀      78.3%     62.0%     60.3%     58.1%
RNN    씀        99.9%     92.1%     79.7%     66.5%
LSTM   안 씀     100.0%     75.5%     55.0%     43.9%
LSTM   씀       100.0%     98.2%     89.6%     70.0%
```

표로 정리하면 이렇습니다.

모델	꾸러미	검증(길이 2~20)	길이 30	길이 40	길이 60
RNN	안 씀	78.3%	62.0%	60.3%	58.1%
RNN	씀	99.9%	92.1%	79.7%	66.5%
LSTM	안 씀	100.0%	75.5%	55.0%	43.9%
LSTM	씀	100.0%	98.2%	89.6%	70.0%

검증 열은 조금 후하게 봐야 합니다. 짧은 괄호는 만들 수 있는 가짓수가 적어서(길이 2는 `()`와 `)(` 두 가지뿐이고, 길이 10도 252가지예요) 검증 문제 상당수가 학습 문제와 똑같거든요. 처음 보는 길이 30, 40, 60 열이 진짜 일반화 성적입니다.

- **RNN은 꾸러미가 없으면 학습한 길이에서도 78.3%에 그칩니다.** 길이 2짜리 괄호는 진짜 글자 2개를 읽은 뒤 채운 칸을 18개나 더 읽어야 하니, 11.4절의 기울기 소실 때문에 정작 중요한 앞부분이 흐려집니다. 꾸러미를 쓰면 99.9%가 돼요.
- **LSTM은 꾸러미 없이도 학습한 길이에서 100%** 입니다. 채운 칸이 오면 문을 닫아 벨트를 보존하는 법을 배운 거예요. 하지만 처음 보는 길이에서는 꾸러미를 쓴 쪽이 훨씬 낫습니다(길이 40에서 55.0% 대 89.6%). 채운 칸을 견디는 요령을 배우느라 쓴 힘을, 꾸러미가 있으면 괄호 자체를 배우는 데 쓸 수 있으니까요.
- **여러 길이로 학습하자 처음 보는 길이에도 강해졌습니다.** 11.3절에서 길이 20만 보고 학습한 RNN은 길이 30에서 57.5%였는데, 길이를 섞어 학습하고 꾸러미를 쓴 RNN은 92.1%, LSTM은 98.2%예요. 그래도 길이 60에서는 66.5%, 70.0%로 떨어집니다. 학습 때 본 것보다 세 배 깊게 쌓인 괄호까지 일반화하기는 여전히 어려워요.

정리하면, 길이가 다른 순서열을 패딩해서 넣을 때는 **언제나 꾸러미로 싸서** 넣으세요. 7장의 `collate_fn`에서 길이를 함께 돌려주게 만든 이유가 여기 있습니다. LSTM이 패딩을 견뎌 준다고 해서 그 일을 모델에 떠넘길 이유는 없습니다.

11.10 시계열 예측: 마우나로아 이산화탄소

실제 측정 데이터 둘러보기

이제 진짜 데이터로 미래를 예측해 보겠습니다. 하와이 마우나로아 관측소는 1958년부터 대기 중 이산화탄소(CO2) 농도를 재 왔어요. 지구 온난화 이야기에 늘 나오는 "톱니 모양으로 오르는 곡선"이 바로 이 측정값입니다. statsmodels에는 1958년 3월부터 2001년 12월까지의 주간 측정값이 들어 있습니다(출처: Keeling, C.D. and T.P. Whorf, Scripps 해양연구소. 데이터 설명에 public domain으로 밝혀져 있어요).

```
from statsmodels.datasets import co2

주간 = co2.load_pandas().data["co2"]                # 1958~2001년 주간 측정값 (ppm)
print(주간.shape, "빈 값:", 주간.isna().sum())

월간 = 주간.resample("MS").mean()                   # 달마다 평균 (MS: 매달 1일 기준)
print("달 수:", len(월간), "빈 달:", 월간.isna().sum())
월간 = 월간.interpolate()                           # 빈 달은 앞뒤 달 사이 값으로 메운다
print(월간.index[0].date(), "~", 월간.index[-1].date())

연평균 = 월간.groupby(월간.index.year).mean()
print("연평균:", 연평균[[1960, 1970, 1980, 1990, 2000]].round(1).to_dict())
print("2000년 달마다:", 월간["2000"].round(1).tolist())
```

```
(2284,) 빈 값: 59
달 수: 526 빈 달: 5
1958-03-01 ~ 2001-12-01
연평균: {1960: 316.9, 1970: 325.7, 1980: 338.6, 1990: 354.1, 2000: 369.4}
2000년 달마다: [369.0, 369.4, 370.4, 371.5, 371.6, 371.6, 369.9, 368.0, 366.5, 366.7, 368.1, 369.4]
```

주간 측정값 2,284개 중 59개가 비어 있습니다. 기계 고장 같은 이유로 못 잰 주예요. pandas의 `resample("MS")`로 달마다 평균을 내니 526개월이 됐고, 그래도 빈 달이 5개 남아 `interpolate()`로 앞뒤 달 사이의 값을 채웠습니다. `groupby`는 1권 20장에서 본 그대로예요.

숫자 두 줄이 이 데이터의 성격을 말해 줍니다. 연평균은 1960년 316.9ppm에서 2000년 369.4ppm까지 **꾸준히 오르고(추세)**, 2000년의 달별 값은 5~6월에 가장 높고 9월에 가장 낮게 **해마다 오르내립니다(계절성)**. 북반구의 식물이 여름에 이산화탄소를 빨아들이기 때문이에요. 좋은 예측 모델이라

면 이 두 가지를 모두 잡아야 합니다.

기준선부터 세우기

모델을 만들기 전에 할 일이 있습니다. **아주 단순한 방법으로 얼마나 맞히는지** 먼저 재 두는 거예요. 이걸 못 이기면 신경망을 쓸 이유가 없습니다. 지난 24개월을 보고 다음 달을 맞히는 문제로 만들고, 2장 2.9절대로 시간 순서로 자릅니다.

```
import torch
from statsmodels.datasets import co2

월간 = co2.load_pandas().data["co2"].resample("MS").mean().interpolate()
값 = torch.tensor(월간.values, dtype=torch.float32)

창 = 24                                          # 지난 24개월을 보고 다음 달을 맞힌다
X = 값.unfold(0, 창, 1)[:-1]                      # (502, 24)
y = 값[창:]                                       # (502,)
날짜 = 월간.index[창:]                            # 정답 달의 날짜

# 시간 순서로 자르기(2장 2.9절): 1990년까지 학습, 1991~1995년 검증, 1996년부터 테스트
학습 = torch.tensor(날짜 < "1991-01-01")
검증 = torch.tensor((날짜 >= "1991-01-01") & (날짜 < "1996-01-01"))
테스트 = torch.tensor(날짜 >= "1996-01-01")
print("학습", 학습.sum().item(), "검증", 검증.sum().item(), "테스트", 테스트.sum().item())


def rmse(예측: torch.Tensor, 정답: torch.Tensor) -> float:
    return ((예측 - 정답) ** 2).mean().sqrt().item()


지난달 = X[:, -1]                                # 기준선 1: 지난달 값 그대로
작년_같은달 = X[:, -12] + (X[:, -1] - X[:, -13])  # 기준선 2: 작년 같은 달 + 1년 동안 오른 만큼
for 이름, 예측 in [("지난달 그대로", 지난달), ("작년 같은 달 + 1년 증가분", 작년_같은달)]:
    print(f"{이름}: 검증 RMSE {rmse(예측[검증], y[검증]):.3f}, 테스트 RMSE {rmse(예측[테스트], y[테
    ↪ 스트]):.3f} ppm")
```

```
학습 370 검증 60 테스트 72
지난달 그대로: 검증 RMSE 1.305, 테스트 RMSE 1.253 ppm
작년 같은 달 + 1년 증가분: 검증 RMSE 0.340, 테스트 RMSE 0.428 ppm
```

1990년까지가 학습(370개월), 1991~1995년이 검증(60개월), 1996~2001년이 테스트(72개월)입니다. 오차는 RMSE로 쟀어요. 3장에서 본 것처럼 단위가 원래 값과 같아서 "평균 몇 ppm쯤 틀린다"로 읽을 수 있습니다.

- **지난달 그대로**: 다음 달도 지난달과 같을 거라고 말하는 방법입니다. 테스트 RMSE 1.253ppm. 계절에 따라 한 달에 1~2ppm씩 움직이니 꽤 틀려요.
- **작년 같은 달 + 1년 증가분**: "작년 이맘때 값에, 최근 1년 동안 오른 만큼을 더한다"는 방법입니다. 계절성과 추세를 모두 반영해서 테스트 RMSE가 0.428ppm으로 확 줄었어요.

두 번째 기준선은 이 데이터의 성격을 사람이 알고 만든 규칙입니다. 이제 신경망이 이 규칙을 **데이터에서 스스로** 배워서 이길 수 있는지 보겠습니다.

RNN, LSTM, GRU로 예측하기

신경망에 넣을 때 한 가지 손질을 합니다. 315ppm, 370ppm 같은 절대값을 그대로 넣으면, 1990년까지만 본 모델은 1996년 이후의 처음 보는 큰 값을 다루지 못해요(2장 2.9절에서 나무 모델이 미래의 큰 값을 못 따라간 것과 같은 이야기입니다). 그래서 창마다 **마지막 달을 0으로 맞춘 상대값**을 넣고, **다음 달에 얼마나 변할지**를 맞히게 했습니다. 크기는 학습 구간에서만 잰 표준편차로 나눴어요. 학습 구간의 정보만 쓰는 것은 2장의 데이터 누수 규칙 그대로입니다.

```
import copy

import torch
from statsmodels.datasets import co2
from torch import nn

월간 = co2.load_pandas().data["co2"].resample("MS").mean().interpolate()
값 = torch.tensor(월간.values, dtype=torch.float32)
창 = 24
X, y, 날짜 = 값.unfold(0, 창, 1)[:-1], 값[창:], 월간.index[창:]
학습 = torch.tensor(날짜 < "1991-01-01")
검증 = torch.tensor((날짜 >= "1991-01-01") & (날짜 < "1996-01-01"))
```

```
테스트 = torch.tensor(날짜 >= "1996-01-01")

# 절대값 대신 '마지막 달과의 차이'를 보고 '다음 달에 얼마나 변할지'를 맞힌다
마지막 = X[:, -1:]
입력, 목표 = X - 마지막, y - 마지막[:, 0]
척도 = 입력[학습].std()                          # 학습 구간에서만 잰 크기로 나눈다
입력, 목표 = 입력 / 척도, 목표 / 척도


class 예측기(nn.Module):
    def __init__(self, 종류: type[nn.Module]) -> None:
        super().__init__()
        self.순환 = 종류(1, 32, batch_first=True)
        self.답 = nn.Linear(32, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        출력, _ = self.순환(x.unsqueeze(-1))       # (배치, 24) → (배치, 24, 1)
        return self.답(출력[:, -1]).squeeze(-1)


def ppm_rmse(모델: nn.Module, 구간: torch.Tensor) -> float:
    with torch.no_grad():
        예측 = 모델(입력[구간]) * 척도 + 마지막[구간, 0]   # 차이를 다시 ppm 값으로
    return ((예측 - y[구간]) ** 2).mean().sqrt().item()


가장_좋은 = (float("inf"), None)
for 종류 in [nn.RNN, nn.LSTM, nn.GRU]:
    torch.manual_seed(0)
    모델 = 예측기(종류)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=3e-3)
    X_학습, y_학습 = 입력[학습], 목표[학습]
    최고 = (float("inf"), 0, None)
    for 에폭 in range(1, 201):
        for 묶음 in torch.randperm(len(X_학습)).split(32):
            손실 = nn.functional.mse_loss(모델(X_학습[묶음]), y_학습[묶음])
```

```
            옵티마이저.zero_grad()
            손실.backward()
            옵티마이저.step()
        검증_오차 = ppm_rmse(모델, 검증)
        if 검증_오차 < 최고[0]:                    # 검증 오차가 가장 낮았던 순간을 보관(8.8절)
            최고 = (검증_오차, 에폭, copy.deepcopy(모델.state_dict()))
    모델.load_state_dict(최고[2])
    print(f"{종류.__name__:4}: {최고[1]:3d}에폭 선택, 검증 RMSE {최고[0]:.3f}, 테스트 RMSE
    ↪ {ppm_rmse(모델, 테스트):.3f} ppm")
    if 최고[0] < 가장_좋은[0]:                     # 검증 성적으로 하나를 골라 저장
        가장_좋은 = (최고[0], 종류.__name__)
        torch.save({"종류": 종류.__name__, "가중치": 최고[2], "척도": 척도}, "co2_모델.pt")
print("저장한 모델:", 가장_좋은[1])
```

```
RNN : 137에폭 선택, 검증 RMSE 0.276, 테스트 RMSE 0.325 ppm
LSTM:  47에폭 선택, 검증 RMSE 0.315, 테스트 RMSE 0.358 ppm
GRU : 148에폭 선택, 검증 RMSE 0.286, 테스트 RMSE 0.321 ppm
저장한 모델: RNN
```

에폭마다 검증 RMSE를 재서 가장 낮았던 순간의 가중치를 보관했다가 되돌렸습니다. 8장 8.8절의 조기 종료와 같은 생각이에요. 세 모델 모두 테스트 RMSE가 0.32~0.36ppm으로, 사람이 만든 기준선(0.428ppm)보다 16~25% 낮습니다. 계절의 오르내림과 추세를 데이터에서 배운 거예요.

그런데 셋 중 검증 성적이 가장 좋은 것은 놀랍게도 **평범한 RNN**(0.276ppm)이고, LSTM이 가장 뒤처졌습니다(0.315ppm). 이상한 결과가 아니에요. 이 문제의 창은 24개월이라, 11.7절의 길이 50처럼 먼 과거를 기억할 필요가 없습니다. 기억 장치가 필요 없는 문제에서는 단순한 모델이 충분하고, 오히려 파라미터가 적어 학습이 쉬울 수 있어요. LSTM이 언제나 이기는 것이 아니니, 늘 검증 데이터로 고르세요. 코드는 검증 성적이 가장 좋은 모델을 `co2_모델.pt`로 저장했습니다.

여러 달을 이어서 내다보기

지금까지는 "실제 지난 24개월을 보고 다음 한 달"을 맞혔습니다. 그런데 1995년 말에 서서 앞으로 6년을 내다본다면 1996~2001년의 실제 값은 아직 없겠죠. 이럴 때는 모델의 예측을 다시 입력에 이어 붙여, 그다음 달을 또 예측하는 식으로 **여러 달을 이어서** 내다봅니다. 1995년 12월까지만 알고 그 뒤 72개월을 이렇게 예측해 보겠습니다.

```
import matplotlib.pyplot as plt
import torch
from statsmodels.datasets import co2
from torch import nn

월간 = co2.load_pandas().data["co2"].resample("MS").mean().interpolate()
값 = torch.tensor(월간.values, dtype=torch.float32)
시작 = 월간.index.get_loc(월간["1996-01"].index[0])  # 테스트 첫 달의 위치
개월 = len(값) - 시작                                  # 1996년 1월부터 2001년 12월까지 72개월


class 예측기(nn.Module):                              # 앞 예제와 같은 구조
    def __init__(self, 종류: type[nn.Module]) -> None:
        super().__init__()
        self.순환 = 종류(1, 32, batch_first=True)
        self.답 = nn.Linear(32, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        출력, _ = self.순환(x.unsqueeze(-1))
        return self.답(출력[:, -1]).squeeze(-1)


저장본 = torch.load("co2_모델.pt", weights_only=True)   # 앞 예제에서 검증 성적으로 고른 모델
모델 = 예측기(getattr(nn, 저장본["종류"]))
모델.load_state_dict(저장본["가중치"])
척도 = 저장본["척도"]

# 1995년 12월까지만 알고, 그 뒤 72개월을 자기 예측을 이어 붙이며 내다본다
모델_예측 = 값[:시작].clone()
기준선 = 값[:시작].clone()
with torch.no_grad():
    for _ in range(개월):
        창 = 모델_예측[-24:]
        변화 = 모델((창 - 창[-1]).unsqueeze(0) / 척도) * 척도
        모델_예측 = torch.cat([모델_예측, 창[-1:] + 변화])
        다음 = 기준선[-12] + (기준선[-1] - 기준선[-13])   # 작년 같은 달 + 1년 증가분
        기준선 = torch.cat([기준선, 다음.unsqueeze(0)])
```

```
실제 = 값[시작:]
for 이름, 예측 in [(저장본["종류"] + " 이어 예측", 모델_예측[시작:]), ("기준선 이어 예측", 기준선
↪ [시작:])]:
    오차 = 예측 - 실제
    print(f"{이름}: 첫 12개월 RMSE {오차[:12].pow(2).mean().sqrt():.3f}, "
          f"72개월 RMSE {오차.pow(2).mean().sqrt():.3f}, 2001년 12월 오차 {오차[-1]:+.2f} ppm")

연평균 = 월간.groupby(월간.index.year).mean()
print(f"연간 증가량: 1959~1990년 평균 {(연평균[1990] - 연평균[1959]) / 31:.2f}ppm, "
      f"1995~2001년 평균 {(연평균[2001] - 연평균[1995]) / 6:.2f}ppm")

plt.rcParams["font.family"] = "AppleGothic"
plt.rcParams["axes.unicode_minus"] = False
plt.figure(figsize=(7, 3.5))
보기 = slice(시작 - 48, None)                          # 1992년부터 그린다
plt.plot(월간.index[보기], 값[보기], color="black", label="실제 측정값")
plt.plot(월간.index[시작:], 모델_예측[시작:], label=f"{저장본['종류']} 이어 예측")
plt.plot(월간.index[시작:], 기준선[시작:], linestyle="--", label="기준선 이어 예측")
plt.axvline(월간.index[시작], color="gray", linestyle=":")
plt.ylabel("CO2 (ppm)")
plt.legend()
plt.tight_layout()
plt.savefig("co2_forecast.png", dpi=200)
```

```
RNN 이어 예측: 첫 12개월 RMSE 0.353, 72개월 RMSE 1.934, 2001년 12월 오차 -2.86 ppm
기준선 이어 예측: 첫 12개월 RMSE 0.372, 72개월 RMSE 0.656, 2001년 12월 오차 -0.48 ppm
연간 증가량: 1959~1990년 평균 1.23ppm, 1995~2001년 평균 1.67ppm
```

결과가 흥미롭습니다. **첫 12개월**은 RNN 이어 예측(RMSE 0.353ppm)이 기준선 이어 예측(0.372ppm)보다 조금 낫습니다. 그런데 **72개월 전체**로 보면 RNN은 1.934ppm, 기준선은 0.656ppm으로 순위가 뒤집혀요. 그래프에서도 RNN의 예측은 계절의 오르내림은 잘 따라가지만, 해가 갈수록 실제 값보다 아래로 처집니다. 2001년 12월에는 2.86ppm이나 낮게 예측했어요.

이유는 이어 예측의 구조에 있습니다. 한 달 예측이 0.1ppm만 낮아도, 그 값이 다음 달 입력이 되어 다음 예측도 조금 낮아지고, 그렇게 **작은 오차가 72번 쌓입니다.** 출력의 마지막 줄을 보면 학습 구

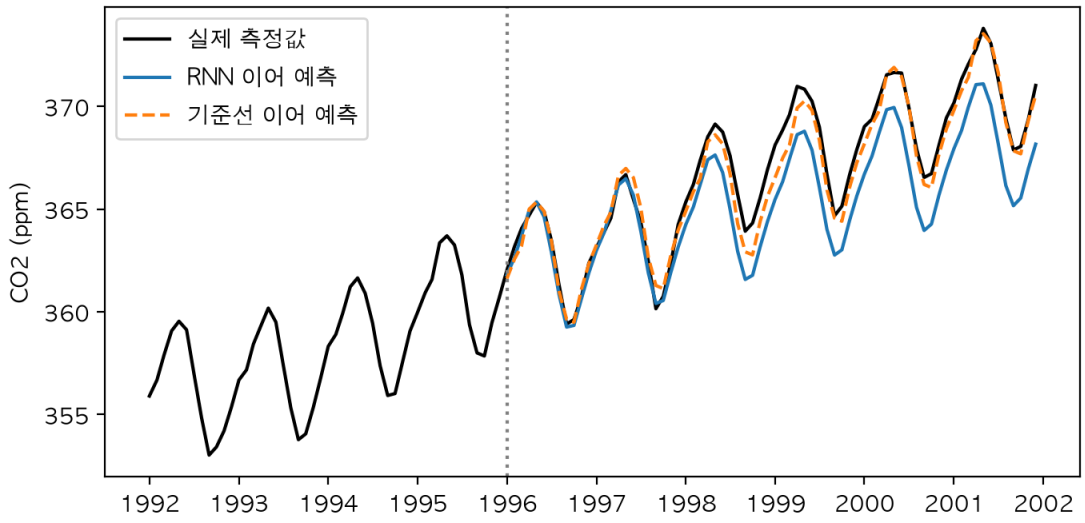


그림 11.1: 1992년부터 2001년까지 이산화탄소 농도의 실제 측정값과 두 가지 이어 예측. 기준선 이어 예측은 실제 값을 거의 따라가지만, RNN 이어 예측은 해가 갈수록 실제 값보다 아래로 처진다.

간(1959~1990년)의 연간 증가량은 평균 1.23ppm인데, 1995~2001년은 1.67ppm으로 더 빨랐어요. 모델은 학습 구간의 느린 증가에 맞춰 "한 달에 오르는 양"을 배웠으니 조금씩 모자라게 예측했고, 그 작은 차이가 6년 동안 쌓인 것입니다. 기준선은 "최근 1년 동안 오른 만큼"을 매번 그대로 이어 쓰니 이런 처짐이 덜했어요.

두 가지를 기억해 두세요. 첫째, **한 달 뒤를 잘 맞히는 모델이 6년 뒤도 잘 맞히는 것은 아닙니다.** 평가는 실제로 쓸 방식(몇 달 앞을 내다볼 것인가) 그대로 해야 해요. 둘째, 시계열에서는 단순한 기준선이 생각보다 강합니다. 신경망을 꺼내기 전에 기준선부터 세우는 습관이 이 결과로 정당화된 셈이에요.

11.11 실습: 속담 짓는 글자 단위 LSTM

이 장의 마지막 실습으로, 우리 속담을 한 글자씩 읽고 **다음 글자를 맞히는** LSTM을 학습시켜 보겠습니다. 다음 글자를 맞힐 수 있는 모델은 글을 **지어낼** 수도 있어요. 첫 글자를 주고, 모델이 고른 다음 글자를 다시 입력으로 넣기를 반복하면 됩니다. 11.10절에서 다음 달을 이어서 예측한 것과 같은 방법이에요. 단계는 넷입니다.

1. 속담 데이터를 파일로 만들기
2. 글자를 번호로 바꾸고, 입력과 정답을 한 칸 밀어 만들기

3. LSTM 학습하기
4. 새 속담 지어 보기

1단계: 데이터 만들기

속담은 오래전부터 전해 내려온 말이라 저작권이 없습니다. 잘 알려진 속담 117개를 `data/속담.txt`에 한 줄에 하나씩 저장합니다.

```
from pathlib import Path

# 저작권이 없는 옛 속담 117개. " / "로 나눠 한 줄에 몇 개씩 적었다
속담 = """
가는 말이 고와야 오는 말이 곱다 / 가는 날이 장날
가랑비에 옷 젖는 줄 모른다 / 가재는 게 편
가지 많은 나무에 바람 잘 날 없다 / 간에 붙었다 쓸개에 붙었다 한다
갈수록 태산 / 개구리 올챙이 적 생각 못 한다 / 개똥도 약에 쓰려면 없다
고래 싸움에 새우 등 터진다 / 고생 끝에 낙이 온다 / 고양이 목에 방울 달기
고양이한테 생선을 맡기다 / 공든 탑이 무너지랴 / 구관이 명관이다
구더기 무서워 장 못 담글까 / 구슬이 서 말이라도 꿰어야 보배
굼벵이도 구르는 재주가 있다 / 궁하면 통한다 / 그림의 떡 / 금강산도 식후경
급하게 먹는 밥이 체한다 / 급할수록 돌아가라 / 기는 놈 위에 나는 놈 있다
길고 짧은 것은 대어 보아야 안다 / 까마귀 날자 배 떨어진다
꿩 먹고 알 먹는다 / 꿩 대신 닭 / 낮말은 새가 듣고 밤말은 쥐가 듣는다
낫 놓고 기역 자도 모른다 / 내 코가 석 자 / 누워서 떡 먹기 / 누워서 침 뱉기
눈 가리고 아웅 / 눈엣가시 / 늦게 배운 도둑이 날 새는 줄 모른다
다 된 죽에 코 빠뜨리기 / 닭 쫓던 개 지붕 쳐다보듯
달면 삼키고 쓰면 뱉는다 / 도둑이 제 발 저린다 / 도토리 키 재기
독 안에 든 쥐 / 돌다리도 두들겨 보고 건너라 / 동에 번쩍 서에 번쩍
되로 주고 말로 받는다 / 등잔 밑이 어둡다 / 땅 짚고 헤엄치기
떡 줄 사람은 생각도 않는데 김칫국부터 마신다 / 뛰는 놈 위에 나는 놈 있다
말 한마디로 천 냥 빚을 갚는다 / 말이 씨가 된다 / 모난 돌이 정 맞는다
모로 가도 서울만 가면 된다 / 목마른 사람이 우물 판다
못 먹는 감 찔러나 본다 / 무소식이 희소식
물에 빠진 놈 건져 놓으니 보따리 내놓으라 한다 / 믿는 도끼에 발등 찍힌다
밑 빠진 독에 물 붓기 / 바늘 도둑이 소도둑 된다 / 바늘 가는 데 실 간다
발 없는 말이 천 리 간다 / 배보다 배꼽이 더 크다 / 백문이 불여일견
백지장도 맞들면 낫다 / 뱁새가 황새를 따라가면 다리가 찢어진다
```

```
벼는 익을수록 고개를 숙인다 / 병 주고 약 준다 / 보기 좋은 떡이 먹기도 좋다
비 온 뒤에 땅이 굳어진다 / 빈 수레가 요란하다
사공이 많으면 배가 산으로 간다 / 사촌이 땅을 사면 배가 아프다 / 산 넘어 산
서당 개 삼 년이면 풍월을 읊는다 / 선무당이 사람 잡는다
세 살 버릇 여든까지 간다 / 소 잃고 외양간 고친다
소문난 잔치에 먹을 것 없다 / 쇠뿔도 단김에 빼라 / 수박 겉 핥기
시작이 반이다 / 식은 죽 먹기 / 싼 것이 비지떡 / 아니 땐 굴뚝에 연기 날까
아는 길도 물어 가라 / 아닌 밤중에 홍두깨 / 약방에 감초
얌전한 고양이 부뚜막에 먼저 올라간다 / 어물전 망신은 꼴뚜기가 시킨다
열 길 물속은 알아도 한 길 사람 속은 모른다
열 번 찍어 안 넘어가는 나무 없다 / 옥에도 티가 있다
올라가지 못할 나무는 쳐다보지도 마라 / 우물 안 개구리
우물에 가 숭늉 찾는다 / 원숭이도 나무에서 떨어진다
윗물이 맑아야 아랫물이 맑다 / 될성부른 나무는 떡잎부터 알아본다
작은 고추가 더 맵다 / 잘되면 제 탓 못되면 조상 탓 / 제 눈에 안경
종로에서 뺨 맞고 한강에서 눈 흘긴다 / 지렁이도 밟으면 꿈틀한다
짚신도 제짝이 있다 / 천 리 길도 한 걸음부터 / 첫술에 배부르랴
콩 심은 데 콩 나고 팥 심은 데 팥 난다 / 티끌 모아 태산
하늘은 스스로 돕는 자를 돕는다 / 하늘이 무너져도 솟아날 구멍이 있다
하룻강아지 범 무서운 줄 모른다 / 호랑이도 제 말 하면 온다
호랑이 굴에 가야 호랑이 새끼를 잡는다
호랑이는 죽어서 가죽을 남기고 사람은 죽어서 이름을 남긴다
혹 떼러 갔다가 혹 붙여 온다 / 홍시 먹다가 이 빠진다
"""
속담들 = [s.strip() for 줄 in 속담.strip().splitlines() for s in 줄.split(" / ")]
경로 = Path("data/속담.txt")
경로.parent.mkdir(exist_ok=True)
경로.write_text("\n".join(속담들) + "\n", encoding="utf-8")   # 파일에는 한 줄에 하나씩

글 = 경로.read_text(encoding="utf-8")
print("속담", len(속담들), "개, 글자", len(글), "개 (줄바꿈 포함)")
print("가장 짧은 속담:", min(속담들, key=len))
print("가장 긴 속담:", max(속담들, key=len))
```

```
속담 117 개, 글자 1642 개 (줄바꿈 포함)
가장 짧은 속담: 눈엣가시
가장 긴 속담: 호랑이는 죽어서 가죽을 남기고 사람은 죽어서 이름을 남긴다
```

글자 수로는 1,642개입니다. 4장 들어가며의 사진 한 장이 숫자 620만 개였던 것과 비교하면 아주 작은 데이터예요. 이 크기가 결과에 어떤 영향을 주는지 4단계에서 봅니다.

2단계: 글자를 번호로, 입력과 정답 만들기

7장 7.7절에서 리뷰의 글자에 번호를 붙였던 것처럼, 글자마다 번호를 붙여 글 전체를 번호 한 줄로 바꿉니다. 이번에는 줄바꿈 "\n"도 한 글자로 쳐요. 모델이 "여기서 속담이 끝난다"는 것도 배우게 하기 위해서입니다.

```
from pathlib import Path

import torch
from torch import nn

글 = Path("data/속담.txt").read_text(encoding="utf-8")
글자들 = sorted(set(글))                          # 줄바꿈 "\n"도 한 글자로 친다
번호 = {글자: i for i, 글자 in enumerate(글자들)}
print("글자 종류:", len(글자들), "앞쪽 다섯:", 글자들[:5])

전체 = torch.tensor([번호[글자] for 글자 in 글])   # 글 전체를 번호 한 줄로
print("번호로 바꾼 글:", tuple(전체.shape), 전체[:8].tolist())

# 다음 글자 맞히기: 입력을 한 칸 밀면 정답이 된다
조각 = 전체[:9]
입력, 정답 = 조각[:-1], 조각[1:]
print("입력:", "".join(글자들[i] for i in 입력))
print("정답:", "".join(글자들[i] for i in 정답))

# nn.Embedding: 번호를 받아 그 번호 줄의 벡터를 꺼내 주는 표
torch.manual_seed(0)
임베딩 = nn.Embedding(len(글자들), 8)             # 글자마다 숫자 8개짜리 벡터
벡터 = 임베딩(입력)
```

```
print("임베딩 표:", tuple(임베딩.weight.shape), "→ 꺼낸 벡터:", tuple(벡터.shape))
원핫 = nn.functional.one_hot(입력, len(글자들)).float()
print("원-핫 @ 표와 같은가?", torch.allclose(벡터, 원핫 @ 임베딩.weight))
```

```
글자 종류: 346 앞쪽 다섯: ['\n', ' ', '가', '각', '간']
번호로 바꾼 글: (1642,) [2, 65, 1, 135, 261, 1, 19, 242]
입력: 가는 말이 고와
정답: 는 말이 고와야
임베딩 표: (346, 8) → 꺼낸 벡터: (8, 8)
원-핫 @ 표와 같은가? True
```

글자 종류는 346개이고, 정렬하면 줄바꿈과 띄어쓰기가 맨 앞에 옵니다. "가는 말이 고와야..."는 `[2, 65, 1, 135, …]`가 돼요. "가"가 2번, "는"이 65번, 띄어쓰기가 1번입니다.

입력과 정답을 보세요. 정답은 입력을 한 칸 민 것입니다. 입력의 첫 글자 "가"를 읽었을 때의 정답은 "는", "가는"까지 읽었을 때의 정답은 띄어쓰기... 이런 식으로 **모든 시점이 다음 글자를 맞히는 문제**가 됩니다. 정답을 따로 매길 필요 없이 글 자체가 문제와 정답이 되는 거예요. 14장에서 만날 거대 언어 모델(LLM)도 바로 이 방식으로 학습합니다.

마지막 부분은 **`nn.Embedding`** 입니다. 11.3절에서는 괄호를 원-핫으로 바꿔 넣었는데, 글자가 346종류면 글자 하나가 숫자 346개짜리 원-핫이 되고 거의 전부 0이에요. `nn.Embedding(346, 8)`은 `(346, 8)` 모양의 표를 하나 두고, 번호를 받으면 **그 번호의 줄을 꺼내 줍니다.** 원-핫 벡터에 이 표를 곱한 것과 결과가 같다는 것을 마지막 줄에서 확인했어요(원-핫은 한 칸만 1이니 곱하면 그 줄만 남죠). 계산은 곱셈 없이 줄을 꺼내기만 하니 훨씬 빠르고, 표의 숫자는 학습하면서 함께 고쳐집니다. 비슷하게 쓰이는 글자는 비슷한 벡터를 갖게 되는데, 이 이야기는 13장에서 자세히 합니다.

3단계: 학습하기

모델은 `임베딩 → LSTM → 다음 글자 점수` 순서입니다. LSTM의 **출력**(모든 시점의 메모)을 써서 시점마다 다음 글자를 맞혀요. 11.2절에서 "시점마다 무언가를 내야 하는 문제에는 출력을 쓴다"고 한 경우입니다. 학습 데이터는 글 전체에서 41글자짜리 조각을 아무 곳에서나 잘라 앞 40글자는 입력, 한 칸 민 40글자는 정답으로 씁니다. 손실은 6장의 교차 엔트로피인데, `(배치, 길이, 글자 수)`인 점수를 `(배치 × 길이, 글자 수)`로 펴서 "글자 하나 맞히기 문제 2,560개"로 계산해요.

글을 지을 때는 점수를 소프트맥스(6장 6.2절)로 확률로 바꾸고, `torch.multinomial`로 **그 확률대로 하나를 뽑습니다.** 가장 높은 것만 고르지 않고 확률대로 뽑는 이유는, 그래야 같은 첫 글자에서

도 매번 다른 글이 나오기 때문이에요. 뽑는 방법을 조절하는 여러 요령(온도, top-p 등)은 14장에서 자세히 다룹니다. 학습 도중 몇 번 글을 지어 보며 변화를 보겠습니다. CPU에서 1분 안팎 걸립니다.

```
from pathlib import Path

import torch
from torch import nn

글 = Path("data/속담.txt").read_text(encoding="utf-8")
글자들 = sorted(set(글))
번호 = {글자: i for i, 글자 in enumerate(글자들)}
전체 = torch.tensor([번호[글자] for 글자 in 글])


class 속담_LSTM(nn.Module):
    def __init__(self, 글자_수: int, 임베딩_크기: int = 32, 메모_크기: int = 128) -> None:
        super().__init__()
        self.임베딩 = nn.Embedding(글자_수, 임베딩_크기)
        self.lstm = nn.LSTM(임베딩_크기, 메모_크기, batch_first=True)
        self.다음_글자 = nn.Linear(메모_크기, 글자_수)

    def forward(self, x: torch.Tensor, 상태: tuple | None = None) -> tuple[torch.Tensor, tuple]:
        출력, 상태 = self.lstm(self.임베딩(x), 상태)
        return self.다음_글자(출력), 상태          # 모든 시점에서 다음 글자 점수를 낸다


def 짓기(모델: nn.Module, 시작: str, g: torch.Generator, 최대: int = 40) -> str:
    """시작 글자들을 읽힌 뒤, 줄바꿈이 나올 때까지 한 글자씩 뽑아 잇는다."""
    모델.eval()
    x, 상태, 결과 = torch.tensor([[번호[c] for c in 시작]]), None, 시작
    with torch.no_grad():
        for _ in range(최대):
            점수, 상태 = 모델(x, 상태)              # 상태를 넘겨 이어서 읽는다
            확률 = 점수[0, -1].softmax(dim=0)
            다음 = torch.multinomial(확률, 1, generator=g).item()   # 확률대로 하나 뽑기
            if 글자들[다음] == "\n":
                break
```

```
            결과 += 글자들[다음]
            x = torch.tensor([[다음]])
    모델.train()
    return 결과.strip()


torch.manual_seed(0)
모델 = 속담_LSTM(len(글자들))
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=3e-3)
길이 = 40
시작점들 = torch.arange(len(전체) - 길이)          # 40글자 조각을 어디서든 잘라 쓴다
print("파라미터:", f"{sum(p.numel() for p in 모델.parameters()):,}개")
for 에폭 in range(1, 61):
    합, 횟수 = 0.0, 0
    for 묶음 in 시작점들[torch.randperm(len(시작점들))].split(64):
        조각 = 전체[묶음.unsqueeze(1) + torch.arange(길이 + 1)]   # (64, 41)
        점수, _ = 모델(조각[:, :-1])
        손실 = nn.functional.cross_entropy(점수.reshape(-1, len(글자들)), 조각[:, 1:].reshape(-1))
        옵티마이저.zero_grad()
        손실.backward()
        nn.utils.clip_grad_norm_(모델.parameters(), max_norm=1.0)
        옵티마이저.step()
        합, 횟수 = 합 + 손실.item(), 횟수 + 1
    if 에폭 in (1, 5, 10, 20, 40, 60):
        g = torch.Generator().manual_seed(0)
        print(f"{에폭:2d}에폭 손실 {합 / 횟수:.3f} ┆ {짓기(모델, '가', g)} ┆ {짓기(모델, '호랑이',
        ↪ g)}")

torch.save({"가중치": 모델.state_dict(), "글자들": 글자들}, "속담_lstm.pt")
```

```
파라미터: 138,650개
 1에폭 손실 4.649 ┆ 가을둡싼기 만 굳다글 ┆ 호랑이한서
 5에폭 손실 2.467 ┆ 가을 배기 있다 ┆ 호랑이다글
10에폭 손실 0.893 ┆ 가면 배가 있다 ┆ 호랑이다
20에폭 손실 0.201 ┆ 가면 배가 있다 ┆ 호랑이다
40에폭 손실 0.105 ┆ 가는 데 실 간다 ┆ 호랑이는 죽어서 가죽을 남기고 사람은 죽어서 이름을 남긴다
```

```
60에폭 손실 0.091 ¦ 가는 길 먹기 ¦ 호랑이 굴에 가야 호랑이 새끼를 잡는다
```

파라미터는 13만 8천 개 남짓입니다. 손실과 지은 글을 차례로 읽어 보세요.

- **1에폭**(손실 4.649): "가을둡싼기 만 굳다글". 아무 글자나 늘어놓은 수준입니다. 참고로 346글자 중 아무거나 고르는 모델의 손실은 ln 346 ≈ 5.85예요.
- **5~10에폭**(2.467 → 0.893): "가을 배기 있다", "가면 배가 있다". 띄어쓰기 위치와 "~다"로 끝나는 말투를 배웠습니다. 뜻은 없지만 속담처럼 생겼어요.
- **40~60에폭**(0.105 → 0.091): "가는 데 실 간다", "호랑이 굴에 가야 호랑이 새끼를 잡는다". 진짜 속담이 나오기 시작합니다. "가는 데 실 간다"는 "바늘 가는 데 실 간다"의 뒷부분이에요.

학습 루프에서 새로 본 것은 두 가지입니다. 하나는 `nn.utils.clip_grad_norm_`으로, 11.4절에서 말한 대로 순환 신경망 학습의 기본 안전벨트예요. 다른 하나는 `짓기` 함수가 LSTM의 상태를 `모델(x, 상태)`로 넘기며 **한 글자씩 이어서** 읽는다는 점입니다. 11.2절에서 `h_중간`을 넘겨 이어 읽은 것과 같아요. 이렇게 하면 지금까지 지은 글 전체를 매번 다시 읽지 않아도 됩니다.

4단계: 새 속담 지어 보기

저장한 모델을 불러서(6장 6.10절) 속담을 30개 지어 보고, 그중 원래 있던 속담과 새로 지은 것이 몇 개인지 세어 보겠습니다. 시작 글자로 줄바꿈 `"\n"`을 주면, 모델이 "속담이 막 시작되는 자리"에서 첫 글자부터 스스로 고릅니다.

```
from pathlib import Path

import torch
from torch import nn



class 속담_LSTM(nn.Module):                          # 앞 예제와 같은 구조
    def __init__(self, 글자_수: int, 임베딩_크기: int = 32, 메모_크기: int = 128) -> None:
        super().__init__()
        self.임베딩 = nn.Embedding(글자_수, 임베딩_크기)
        self.lstm = nn.LSTM(임베딩_크기, 메모_크기, batch_first=True)
        self.다음_글자 = nn.Linear(메모_크기, 글자_수)

    def forward(self, x: torch.Tensor, 상태: tuple ¦ None = None) -> tuple[torch.Tensor, tuple]:
```

```
        출력, 상태 = self.lstm(self.임베딩(x), 상태)
        return self.다음_글자(출력), 상태


저장본 = torch.load("속담_lstm.pt")
글자들 = 저장본["글자들"]
번호 = {글자: i for i, 글자 in enumerate(글자들)}
모델 = 속담_LSTM(len(글자들))
모델.load_state_dict(저장본["가중치"])
모델.eval()


def 짓기(시작: str, g: torch.Generator, 최대: int = 40) -> str:
    x, 상태, 결과 = torch.tensor([[번호[c] for c in 시작]]), None, 시작
    with torch.no_grad():
        for _ in range(최대):
            점수, 상태 = 모델(x, 상태)
            다음 = torch.multinomial(점수[0, -1].softmax(dim=0), 1, generator=g).item()
            if 글자들[다음] == "\n":
                break
            결과 += 글자들[다음]
            x = torch.tensor([[다음]])
    return 결과.strip()


원래_속담 = set(Path("data/속담.txt").read_text(encoding="utf-8").splitlines())
g = torch.Generator().manual_seed(1)
지은_것 = [짓기("\n", g) for _ in range(30)]        # 줄바꿈 다음, 즉 속담의 첫 글자부터 짓는다
새것 = [s for s in 지은_것 if s not in 원래_속담]
print(f"30개 중 원래 있던 속담 {30 - len(새것)}개, 새로 지은 것 {len(새것)}개")
for s in dict.fromkeys(새것):                    # 겹치는 것은 한 번만
    print(" -", s)

print("시작 글자를 정해 주면:")
for 시작 in ["원숭이", "고양이", "티끌", "사람"]:
    print(" -", " / ".join(짓기(시작, g) for _ in range(3)))
```

```
30개 중 원래 있던 속담 24개, 새로 지은 것 6개
 - 잘되면 제 탓 못되면 조상 리 간다
 - 호랑이도 제짝 말이 천 리 간다
 - 구는 도끼에 발등 찍힌다
 - 금은 사람은 생각도 않는데 김칫국부터 마신다
 - 땅 줄 고추가 더 맵다
 - 기는 등 건져 없는 놈 건져 놓으니 보따리 내놓으라 한다
시작 글자를 정해 주면:
 - 원숭이도 나무에서 떨어진다 / 원숭이도 나무에서 떨어진다 / 원숭이도 나무에서 떨어진다
 - 고양이 부뚜막에 먼저 올라간다 / 고양이도 밟어야 오는 말이 천 리 간다 / 고양이 목에 방울 달기
 - 티끌 모아 태산 / 티끌 모아 태산 / 티끌 모아 태산
 - 사람은 죽어서 이름을 남긴다 / 사람 속은 모른다 / 사람 속은 모른다
```

30개 중 24개는 원래 있던 속담을 그대로 지었고, 6개만 새것입니다. 새로 지은 것들을 보면 웃음이 나요. "잘되면 제 탓 못되면 조상 리 간다"는 "잘되면 제 탓 못되면 조상 탓"과 "발 없는 말이 천 리 간다"가 섞였고, "호랑이도 제짝 말이 천 리 간다"는 "호랑이도 제 말 하면 온다", "짚신도 제짝이 있다", "천 리 간다"를 이어 붙였습니다. "금은 사람은 생각도 않는데 김칫국부터 마신다"는 진짜 속담 "떡 줄 사람은 생각도 않는데..."와 거의 같아요.

모델이 하는 일이 보이죠? **두 속담에 같은 글자 조각이 나오는 자리**("제", "말", "천 리")에서 다른 속담으로 갈아탑니다. 시작 글자를 정해 줘도 마찬가지예요. "원숭이"와 "티끌"은 세 번 모두 원래 속담 그대로 나왔고, "고양이"에서는 "고양이도 밟어야 오는 말이 천 리 간다"처럼 여러 속담이 이어 붙은 문장이 나왔습니다.

이 결과를 정직하게 평가하면, 이 모델은 속담을 **이해해서 짓는 것이 아니라 외운 것을 이어 붙이는** 수준입니다. 학습 데이터가 1,642글자뿐이니 당연해요. 8장의 말로 하면 과적합이고, 실제로 손실 0.091은 거의 외웠다는 뜻입니다. 데이터가 수백만 글자로 늘어나면 같은 구조에서도 처음 보는 그럴듯한 문장이 나오기 시작해요. 그리고 11.12절에서 말할 순환 신경망의 한계 때문에, 요즘 글을 짓는 모델은 대부분 다음 장의 트랜스포머로 만듭니다.

11.12 정리와 다음 장 미리보기

이 장에서 순서가 있는 데이터를 다루는 순환 신경망을 익혔습니다.

- **순서 데이터의 모양**: (배치, 길이, 특성). `unfold`로 창을 밀며 잘라 문제를 만들고, 시간 순서로 나눈다. `batch_first=True`를 잊지 않는다

- RNN: `h_t = tanh(x_t @ W_x + h_(t-1) @ W_h + b)`. 모든 시점이 같은 가중치를 쓰므로 어떤 길이든 받는다. 출력은 모든 시점의 메모, `h_n`은 마지막 메모
- **순서가 필요한 문제**: 개수만 보는 모델은 괄호 짝 문제에서 50%. 펼친 MLP는 길이가 바뀌면 쓸 수 없다
- **기울기 소실과 폭주**: 시점마다 기울기를 곱하니 사라지거나 터진다. 폭주는 `clip_grad_norm_`으로, 소실은 구조(LSTM·GRU)로 해결한다
- LSTM: 벨트 `c`와 문 셋(망각·입력·출력). `c_t = f * c_(t-1) + i * g`의 덧셈 덕분에 기울기가 먼 과거까지 흐른다. 상태는 `(h_n, c_n)`
- GRU: 문 둘(갱신·리셋), 파라미터는 RNN의 3배(LSTM은 4배). 둘 중 무엇이 나은지는 검증 데이터로 고른다
- **층 쌓기·양방향**: `h_n`의 첫 자리는 층 수 × 방향 수. 양방향은 미래를 보므로 예측·생성에는 쓰지 않는다
- **길이가 다른 순서열**: 패딩만 하면 채운 칸까지 읽어 메모가 오염된다. `pack_padded_sequence(…, enforce_sorted=False)`로 꾸러미를 만든다
- **시계열 예측**: 기준선부터 세우고, 상대값으로 바꿔 넣고, 검증 데이터로 모델을 고른다. 여러 달을 이어 예측하면 오차가 쌓인다
- **글자 생성**: 다음 글자 맞히기로 학습하고, 확률대로 뽑아 이어 붙인다. 작은 데이터에서는 외우고 이어 붙이는 수준에 머문다

이 장의 모델들은 모두 **한 시점씩 차례로** 읽었습니다. 여기에는 두 가지 한계가 있어요. 첫째, 문장 전체의 내용을 크기가 정해진 메모 하나에 눌러 담아야 합니다. 긴 문장일수록 앞부분의 내용이 흐려지죠. 둘째, 100번째 글자를 읽으려면 99번째까지 다 읽어야 해서 GPU가 잘하는 "한꺼번에 계산하기"를 쓸 수 없습니다. 다음 12장에서는 메모장 하나에 의지하는 대신, 필요할 때마다 **문장의 모든 자리를 한꺼번에 돌아보는** 어텐션을 직접 구현합니다. 그리고 어텐션만으로 쌓은 트랜스포머로, 이 장의 속담 생성기보다 훨씬 강력한 작은 문자 단위 GPT를 처음부터 학습시켜 봅니다.

12

어텐션과 트랜스포머: 처음부터 직접 구현

학습 목표

- 어텐션을 “부드럽게 꺼내는 딕셔너리”로 이해하고, 쿼리·키·값으로 직접 계산하기
- 점수를 d의 제곱근으로 나누는 이유와 마스크(패딩·인과)가 하는 일을 숫자로 확인하기
- 헤드를 나누고 합치는 모양 바꾸기를 따라가며 멀티헤드 어텐션을 구현하고, PyTorch 내장 부품과 결과 맞춰 보기
- 어텐션이 순서를 모른다는 것을 확인하고, 위치 정보를 더하는 두 가지 방법 익히기
- 층 정규화·잔차 연결·피드포워드로 트랜스포머 블록을 조립하고 파라미터 세기
- 헌법과 민법 조문으로 작은 문자 단위 GPT를 처음부터 학습시켜 한국어 문장을 만들고, 어텐션이 어디를 보는지 들여다보기

들어가며: “그것”은 무엇을 가리킬까요?

다음 두 문장을 읽어 보세요.

- 고양이가 생선을 먹지 않았다. **그것**이 너무 배불렀기 때문이다.
- 고양이가 생선을 먹지 않았다. **그것**이 너무 상했기 때문이다.

첫 문장의 “그것”은 고양이고, 두 번째 문장의 “그것”은 생선이죠. 사람은 “배불렀다”, “상했다”라는 말을 보고 거꾸로 올라가서 “그것”이 무엇인지 바로 압니다. 한 단어의 뜻을 정하려면 **문장 안의 다른 단어 중 어디를 봐야 하는지** 알아야 하는 거예요. 그리고 봐야 할 곳은 바로 옆일 수도 있고, 한참 떨어진 곳일 수도 있습니다.

11장의 RNN은 한 글자씩 순서대로 읽어야 했습니다. 앞에서 읽은 내용을 크기가 정해진 메모장(은닉 상태) 하나에 눌러 담아 다음 글자로 넘기는 방식이라, 멀리 떨어진 단어의 정보는 넘겨지는 동안 점점 흐려지고, 앞 글자를 다 읽기 전에는 다음 글자를 계산할 수도 없었죠. 2017년에 나온 **트랜스포머(Transformer)** 는 생각을 바꿨습니다. 차례로 넘겨받지 말고, **모든 글자가 다른 모든 글자를 한 번에 직접 보게 하자.** 그리고 어디를 얼마나 볼지는 모델이 데이터에서 배우게 하자. 이 "어디를 얼마나 볼지 고르는 장치"가 **어텐션(attention)** 입니다.

오늘날 ChatGPT 같은 대형 언어 모델(LLM)은 거의 모두 트랜스포머이고, 그림을 다루는 모델에도 트랜스포머가 널리 쓰입니다. 그래서 이 장은 이 책에서 가장 중요한 장 가운데 하나예요. 라이브러리 한 줄로 부르고 끝내지 않고, 어텐션 계산 한 줄부터 직접 짜서 쌓아 올릴 겁니다. 마지막에는 대한민국 헌법과 민법 조문을 읽고 "제850조(…" 같은 조문을 스스로 지어내는 작은 GPT를 노트북에서 처음부터 학습시켜요.

필요한 도구는 거의 다 앞에서 익혔습니다. 행렬 곱과 `einsum`, `permute().contiguous().view()`(4장), 소프트맥스와 `nn.Module`(6장), 패딩과 마스크(7장), 드롭아웃·weight decay·스케줄러·그래디언트 클리핑(8장)이 이 장에서 전부 다시 나와요. 이 장의 예제는 PyTorch 2.14, 파이썬 3.12에서 실행해 확인했습니다.

12.1 어텐션: 부드럽게 꺼내는 딕셔너리

키가 정확히 같지 않아도 꺼낼 수 있다면

1권 7장의 딕셔너리를 떠올려 보세요. `가격표["떡볶이"]`처럼 **키가 정확히 같아야** 값을 꺼낼 수 있었죠. "매콤한 거 있어요?"라고 물으면 딕셔너리는 `KeyError`를 냅니다.

분식집 사장님은 다릅니다. "맵고, 조금 달았으면 좋겠어요"라고 하면 메뉴판을 훑어보고 떡볶이를 가장 먼저 떠올리고, 김치찌개도 조금, 마카롱은 거의 안 떠올리죠. 어텐션은 이 사장님처럼 동작합니다. 키를 **숫자 묶음(벡터)** 으로 두고, 질문과 각 키가 **얼마나 닮았는지**를 재서 그 비율대로 값을 섞어 꺼내요. 음식마다 (매운 정도, 단 정도)를 키로 적어 두고 해 보겠습니다.

```
import torch

torch.set_printoptions(precision=3, sci_mode=False)   # 소수점 셋째 자리까지, 지수 표기 없이

# 1권 7장의 딕셔너리: 키가 정확히 같아야 값을 꺼낸다
가격표 = {"떡볶이": 4500, "마카롱": 2500, "김치찌개": 9000}
```

```
print("딕셔너리:", 가격표["떡볶이"])

# 어텐션: 키를 숫자 묶음으로 두고, 질문과 닮은 정도에 따라 값을 섞어 꺼낸다
키 = torch.tensor([[0.9, 0.6],      # 떡볶이: (매운 정도, 단 정도)
                   [0.0, 1.0],      # 마카롱
                   [1.0, 0.1]])     # 김치찌개
값 = torch.tensor([4500.0, 2500.0, 9000.0])
질문 = torch.tensor([1.0, 0.5])     # "맵고, 조금 달았으면 좋겠어요"

점수 = 키 @ 질문                    # 질문과 키마다 곱하고 더하기(내적) → 닮은 정도
가중치 = torch.softmax(점수, dim=0)  # 합이 1인 비율로
print("점수  :", 점수)
print("가중치:", 가중치)
print("섞어 꺼낸 값:", (가중치 @ 값).round())

# 점수 차이를 키우면 가장 닮은 키 하나에 쏠린다: 딕셔너리처럼 동작
가중치 = torch.softmax(점수 * 20, dim=0)
print("20배 가중치:", 가중치, "→", (가중치 @ 값).round())
```

```
딕셔너리: 4500
점수  : tensor([1.200, 0.500, 1.050])
가중치: tensor([0.424, 0.211, 0.365])
섞어 꺼낸 값: tensor(5722.)
20배 가중치: tensor([0.953, 0.000, 0.047]) → tensor(4713.)
```

한 줄씩 보겠습니다.

- **점수**: `키 @ 질문`은 키마다 "질문과 같은 자리끼리 곱해서 더한 값", 즉 **내적**을 계산합니다(4장 4.6절의 행렬 곱). 방향이 비슷한 두 벡터일수록 내적이 커서 닮은 정도로 쓸 수 있어요. 떡볶이 1.2, 김치찌개 1.05, 마카롱 0.5가 나왔습니다.
- **가중치**: 점수에 소프트맥스(6장 6.2절)를 씌워 합이 1인 비율 0.424, 0.365, 0.211로 바꿨습니다.
- **섞어 꺼낸 값**: 비율대로 가격을 섞으니 5,722원이 나왔어요. 메뉴판에 없는 값이지만 "그 질문에 어울리는 음식들의 가격을 닮은 정도만큼 섞은 것"입니다.

마지막 두 줄이 재미있습니다. 점수를 20배로 키워서 소프트맥스에 넣으면 가중치가 0.953으로 떡볶

이 하나에 거의 몰리고, 꺼낸 값도 떡볶이 가격에 가까운 4,713원이 됩니다. 6장에서 "점수 차이를 10배로 키우면 확률이 거의 0과 1로 쏠린다"고 했던 바로 그 성질이에요. 점수 차이가 아주 크면 어텐션은 딕셔너리처럼 하나를 콕 집어 꺼내고, 차이가 작으면 여러 개를 골고루 섞습니다. 그래서 어텐션을 **부드러운 딕셔너리**라고 부르기도 해요.

어텐션의 세 재료에는 이름이 있습니다.

재료	비유	하는 일
쿼리(query, q)	손님의 주문	"나는 이런 것을 찾는다"
키(key, k)	메뉴판에 적힌 특징	"나는 이런 특징을 가졌다". 쿼리와 내적해서 점수를 낸다
값(value, v)	실제로 내주는 음식	가중치대로 섞여서 결과가 된다

질문 여러 개를 한꺼번에

질문이 하나가 아니라 여러 개라면 어떻게 할까요? 4장에서 배운 대로 반복문을 돌리지 않고 질문들을 한 행렬로 쌓아 행렬 곱 한 번에 계산합니다. 이것이 트랜스포머 논문의 어텐션 공식 그대로예요.

```
import math

import torch


def 어텐션(q: torch.Tensor, k: torch.Tensor, v: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
    """질문 q마다 키 k와 닮은 정도를 재서, 그 비율대로 값 v를 섞는다."""
    점수 = q @ k.transpose(-2, -1) / math.sqrt(k.size(-1))   # (질문 수, 키 수)
    가중치 = 점수.softmax(dim=-1)                              # 질문마다 합이 1
    return 가중치 @ v, 가중치                                   # (질문 수, 값 차원)


torch.manual_seed(0)
torch.set_printoptions(precision=3)
q = torch.randn(2, 4)       # 질문 2개, 각각 숫자 4개
k = torch.randn(5, 4)       # 키 5개 (질문과 길이가 같아야 내적이 된다)
v = torch.randn(5, 3)       # 값 5개 (키와 개수가 같아야 한다), 각각 숫자 3개
```

```
출력, 가중치 = 어텐션(q, k, v)
print("가중치:", tuple(가중치.shape), "출력:", tuple(출력.shape))
print(가중치)
print("질문마다 가중치 합:", 가중치.sum(dim=-1))

# 같은 계산을 einsum으로: 사라지는 글자(d, k)가 곱하고 더해 없어지는 차원
점수 = torch.einsum("qd,kd->qk", q, k) / math.sqrt(4)
print(torch.allclose(torch.einsum("qk,kv->qv", 점수.softmax(dim=-1), v), 출력))
```

```
가중치: (2, 5) 출력: (2, 3)
tensor([[0.022, 0.055, 0.039, 0.237, 0.648],
        [0.737, 0.069, 0.163, 0.004, 0.027]])
질문마다 가중치 합: tensor([1.000, 1.000])
True
```

`어텐션()` 함수 세 줄이 이 장 전체의 심장입니다. 모양을 따라가 보세요.

- `q @ k.transpose(-2, -1)`: `(질문 2, 숫자 4) @ (숫자 4, 키 5) → (2, 5)`. 질문마다 키 다섯 개와의 점수가 한 줄씩 나옵니다. `k`를 전치한 것은 4장 4.6절의 규칙 `(a, b) @ (b, c)`에 맞추려는 거예요. `transpose(-2, -1)`은 "마지막 두 차원을 바꿔라"라는 뜻이라, 앞에 배치 차원이 붙어도 그대로 쓸 수 있습니다.
- `softmax(dim=-1)`: 키 방향으로 합이 1. 그래서 줄마다 합이 1.000입니다.
- `가중치 @ v`: `(2, 5) @ (5, 3) → (2, 3)`. 질문마다 값 다섯 개를 비율대로 섞은 결과가 하나씩 나와요.

첫 번째 질문은 4번 키에 0.648, 두 번째 질문은 0번 키에 0.737을 줬습니다. 질문마다 보는 곳이 다른 거죠. 모양에서 지켜야 할 규칙은 두 가지예요. 내적을 해야 하니 **쿼리와 키는 숫자 개수가 같아야** 하고, 가중치로 값을 섞어야 하니 **키와 값은 개수가 같아야** 합니다. 값의 숫자 개수(여기서는 3)는 자유롭고요.

마지막 줄은 같은 계산을 4장 4.6절의 `einsum`으로 쓴 것입니다. `"qd,kd->qk"`는 "질문(q)과 키(k)마다 숫자(d)를 곱해 더해 없앤다"는 뜻이라 전치를 따로 신경 쓰지 않아도 돼요. 숫자 4의 제곱근으로 나눈 부분은 바로 다음 절의 주제입니다.

12.2 점수를 d의 제곱근으로 나누는 이유

`어텐션()` 함수에서 점수를 `math.sqrt(k.size(-1))`, 즉 **키 숫자 개수의 제곱근**으로 나눴습니다. 이 나눗셈이 왜 필요할까요? 숫자 개수 d를 4, 64, 512로 바꿔 가며 점수의 크기와 소프트맥스의 모양, 그리고 기울기를 재 보겠습니다. 쿼리와 키는 평균 0, 표준편차 1인 난수이고, 쿼리 하나마다 키 8개를 봅니다.

```
import torch

torch.manual_seed(0)
for d in [4, 64, 512]:
    q = torch.randn(1000, 1, d)             # 질문 1000개
    k = torch.randn(1000, 8, d)             # 질문마다 키 8개
    점수 = (q * k).sum(dim=-1)              # 내적: (1000, 8)
    결과 = []
    for 나눌까 in (False, True):
        s = (점수 / d ** 0.5 if 나눌까 else 점수).detach().requires_grad_()
        가중치 = s.softmax(dim=-1)
        (가중치 * torch.randn(1000, 8)).sum().backward()    # 아무 손실이나 흘려 보기
        결과.append((s.std().item(), 가중치.max(dim=-1).values.mean().item(),
↪   s.grad.abs().mean().item()))
    (표1, 최대1, 기울기1), (표2, 최대2, 기울기2) = 결과
    print(f"d={d:3d}  점수 표준편차 {표1:5.2f} → {표2:.2f},  가장 큰 가중치 {최대1:.3f} → {최
    ↪ 대2:.3f},"
          f"  기울기 {기울기1:.4f} → {기울기2:.4f}")
```

```
d=  4  점수 표준편차  2.00 → 1.00,  가장 큰 가중치 0.533 → 0.347,  기울기 0.0686 → 0.0839
d= 64  점수 표준편차  8.08 → 1.01,  가장 큰 가중치 0.878 → 0.361,  기울기 0.0226 → 0.0867
d=512  점수 표준편차 22.69 → 1.00,  가장 큰 가중치 0.960 → 0.362,  기울기 0.0079 → 0.0861
```

세 가지를 차례로 보세요.

1. **나누지 않은 점수의 표준편차**가 2.00, 8.08, 22.69로 d가 커질수록 커집니다. 내적은 숫자 d개를 곱해 **더한** 값이라, 더하는 개수가 많을수록 결과가 크게 흔들리거든요. 정확히는 표준편차가 d의 제곱근배(2, 8, 22.6)가 됩니다. 제곱근으로 나누면 어느 d에서든 1 근처로 돌아와요.
2. 점수가 크게 흔들리면 **소프트맥스가 한 칸으로 쏠립니다.** 12.1절의 20배 실험과 같은 일이에

요. d가 512면 키 8개 중 가장 큰 가중치가 평균 0.960이라, 거의 언제나 키 하나만 봅니다. 나눈 뒤에는 0.36 근처로, 여러 키를 섞어 볼 여유가 생겨요.

3. 쏠린 소프트맥스는 **기울기가 거의 0**입니다. 이미 확률이 0과 1에 붙어 있으면 점수를 조금 바꿔도 결과가 거의 변하지 않으니까요. 나누지 않으면 d=512에서 기울기가 0.0079로, 나눈 경우(0.0861)의 10분의 1도 안 됩니다. 학습 초기부터 기울기가 이렇게 작으면 모델이 "어디를 볼지"를 고쳐 나갈 수가 없어요.

그래서 트랜스포머 논문은 이 어텐션을 **스케일드 닷 프로덕트 어텐션**(scaled dot-product attention), "크기를 맞춘 내적 어텐션"이라고 불렀습니다. 코드로 쓰면 `softmax(Q @ K.T / sqrt(d)) @ V`이고, 앞의 `어텐션()` 함수 세 줄이 정확히 이 수식이에요.

12.3 셀프 어텐션: 문장이 자기 자신을 본다

같은 입력에서 쿼리·키·값 만들기

지금까지는 쿼리, 키, 값을 따로 준비했습니다. 그런데 처음의 "그것" 문제로 돌아가 보면, 문장 속 글자 하나하나가 **쿼리**("나와 관련된 글자는 어디 있지?")이면서 동시에 다른 글자들의 **키**("나는 이런 글자야")이고 **값**("나를 보면 이 정보를 줄게")이어야 해요. 이렇게 한 문장이 자기 자신을 상대로 어텐션하는 것을 **셀프 어텐션**(self-attention) 이라고 합니다.

같은 글자가 세 역할을 하려면 역할마다 다른 모습이 필요합니다. 그래서 글자 벡터에 서로 다른 `nn.Linear` 세 개를 곱해 쿼리, 키, 값을 따로 만들어요. 어떤 모습으로 바꿀지는 학습으로 정해집니다.

```
import math

import torch
from torch import nn


class 셀프_어텐션(nn.Module):
    """같은 입력에서 쿼리·키·값을 따로 만들어, 글자마다 다른 글자들을 섞어 본다."""

    def __init__(self, 차원: int) -> None:
        super().__init__()
        self.쿼리 = nn.Linear(차원, 차원, bias=False)   # "나는 무엇을 찾나"
```

```
        self.키 = nn.Linear(차원, 차원, bias=False)     # "나는 이런 정보를 가졌다"
        self.값 = nn.Linear(차원, 차원, bias=False)     # "나를 고르면 이걸 준다"

    def forward(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
        q, k, v = self.쿼리(x), self.키(x), self.값(x)              # 모두 (B, T, C)
        점수 = q @ k.transpose(-2, -1) / math.sqrt(x.size(-1))      # (B, T, T)
        가중치 = 점수.softmax(dim=-1)
        return 가중치 @ v, 가중치


torch.manual_seed(0)
torch.set_printoptions(precision=2)          # 텐서를 소수점 둘째 자리까지만 보여 주기
x = torch.randn(1, 5, 8)            # 문장 1개 × 글자 5개 × 글자마다 숫자 8개
층 = 셀프_어텐션(8)
출력, 가중치 = 층(x)
print("입력", tuple(x.shape), "→ 출력", tuple(출력.shape), "/ 가중치", tuple(가중치.shape))
print(가중치[0].detach())
print("파라미터 수:", sum(p.numel() for p in 층.parameters()))
```

```
입력 (1, 5, 8) → 출력 (1, 5, 8) / 가중치 (1, 5, 5)
tensor([[0.25, 0.12, 0.23, 0.18, 0.22],
        [0.17, 0.20, 0.15, 0.20, 0.28],
        [0.18, 0.20, 0.20, 0.19, 0.23],
        [0.18, 0.25, 0.17, 0.21, 0.19],
        [0.14, 0.16, 0.17, 0.16, 0.36]])
파라미터 수: 192
```

입력 `(1, 5, 8)`은 문장 1개, 글자 5개, 글자마다 숫자 8개입니다. 앞으로 이 장의 텐서는 거의 모두 이 **(배치 B, 글자 수 T, 숫자 개수 C)** 모양이에요. 4장 4.2절에서 권한 대로 자리마다 이름을 붙여 읽으세요.

- 출력의 모양은 입력과 똑같은 `(1, 5, 8)`입니다. 글자 다섯 개가 저마다 "다른 글자들을 비율대로 섞은 새 벡터"로 바뀌었어요. 모양이 그대로라서 이 층을 여러 번 겹쳐 쌓을 수 있습니다.
- 가중치는 `(1, 5, 5)`, 글자 다섯 개가 글자 다섯 개를 보는 표입니다. 0번 글자의 줄 `[0.25, 0.12, 0.23, 0.18, 0.22]`는 "0번 글자의 새 벡터는 자기 자신을 25%, 1번 글자를 12%, … 섞어 만든다"는 뜻이에요. 아직 학습 전이라 비율이 고만고만합니다.

- 파라미터는 `nn.Linear(8, 8)` 세 개라 8 × 8 × 3 = 192개입니다. 편향은 빼고 만들었어요 (`bias=False`).

`셀프_어텐션`은 6장 6.3절에서 배운 대로 `nn.Module`을 상속한 클래스이고, `forward`에서 출력과 가중치를 함께 돌려줍니다. 가중치는 학습에는 필요 없지만, 모델이 어디를 보는지 들여다볼 때 쓸 거예요.

어텐션은 순서를 모릅니다

셀프 어텐션에는 이상한 성질이 하나 있습니다. 글자들의 **순서를 뒤섞어서** 넣어 보겠습니다.

```
import math

import torch
from torch import nn

torch.manual_seed(0)
차원 = 8
쿼리, 키, 값 = (nn.Linear(차원, 차원, bias=False) for _ in range(3))


def 셀프_어텐션(x: torch.Tensor) -> torch.Tensor:
    점수 = 쿼리(x) @ 키(x).transpose(-2, -1) / math.sqrt(차원)
    return 점수.softmax(dim=-1) @ 값(x)


x = torch.randn(1, 5, 차원)             # 글자 5개
순서 = torch.tensor([4, 2, 0, 3, 1])     # 글자 순서를 뒤섞는다
with torch.no_grad():
    원래 = 셀프_어텐션(x)
    섞은 = 셀프_어텐션(x[:, 순서])
print("섞은 입력의 출력 == 원래 출력을 같은 순서로 섞은 것:",
      torch.allclose(섞은, 원래[:, 순서], atol=1e-6))
print("0번 글자의 출력, 원래    :", 원래[0, 0, :4].round(decimals=4))
print("0번 글자의 출력, 섞은 뒤 :", 섞은[0, 2, :4].round(decimals=4))
```

```
섞은 입력의 출력 == 원래 출력을 같은 순서로 섞은 것: True
0번 글자의 출력, 원래    : tensor([ 0.0907, -0.0069,  0.1461, -0.1893])
0번 글자의 출력, 섞은 뒤 : tensor([ 0.0907, -0.0069,  0.1461, -0.1893])
```

순서를 `[4, 2, 0, 3, 1]`로 섞어 넣었더니, 출력은 원래 출력을 **같은 순서로 섞은 것과 완전히 같았습니다.** 0번 글자는 섞은 입력에서 2번 자리에 있는데, 그 자리의 출력이 원래 0번 출력과 숫자 하나까지 같아요.

생각해 보면 당연합니다. 셀프 어텐션은 "어떤 글자들이 있는가"만 보고 점수를 매기고 섞을 뿐, "그 글자가 몇 번째에 있는가"는 계산 어디에도 들어가지 않거든요. 글자들을 주머니에 넣고 흔들어도 결과는 같습니다. "고양이가 쥐를 쫓는다"와 "쥐가 고양이를 쫓는다"를 구분하지 못하는 셈이에요. RNN은 한 글자씩 순서대로 읽었으니 순서를 저절로 알았지만, 모든 글자를 한꺼번에 보는 어텐션은 이 정보를 잃어버렸습니다. 12.6절에서 위치 정보를 따로 넣어 이 문제를 해결합니다.

12.4 마스크: 보면 안 되는 칸 가리기

패딩 칸 가리기

7장 7.7절에서 길이가 다른 문장을 한 배치로 묶으려고 짧은 문장 뒤를 0번(빈칸)으로 채웠죠. 그리고 `입력 != 0`으로 진짜 글자 자리를 표시한 마스크를 만들면서 "12장의 어텐션은 이 마스크로 채운 칸은 보지 말라고 알려 준다"고 했습니다. 바로 그 일을 해 보겠습니다.

어떻게 하면 소프트맥스가 특정 칸에 0을 주게 만들 수 있을까요? 소프트맥스는 점수에 지수 함수를 씌운 뒤 합으로 나누는데, 지수 함수에 음의 무한대(`-inf`)를 넣으면 정확히 0이 됩니다. 그러니 **소프트맥스 전에** 가릴 칸의 점수를 `-inf`로 바꾸면 돼요.

```
import math

import torch

torch.manual_seed(0)
torch.set_printoptions(precision=3)
# 7장의 패딩 배치: 진짜 글자 3개인 문장과 5개인 문장. 0번은 채운 칸
입력 = torch.tensor([[13, 22, 19,  0,  0],
                     [ 7, 14,  1, 25, 21]])
진짜 = 입력 != 0                                  # (2, 5) 진짜 글자 자리
```

```
x = torch.randn(2, 5, 4)                          # 글자마다 숫자 4개라고 하자
점수 = x @ x.transpose(-2, -1) / math.sqrt(4)      # (2, 5, 5)

# 키 쪽(열)이 채운 칸이면 -inf로: softmax에서 e^(-inf) = 0이 된다
가린_점수 = 점수.masked_fill(~진짜[:, None, :], float("-inf"))
가중치 = 가린_점수.softmax(dim=-1)
print("첫 문장의 가중치(채운 칸 열은 0):")
print(가중치[0])
print("줄마다 합:", 가중치[0].sum(dim=-1))

# 잘못된 방법: softmax를 먼저 하고 채운 칸을 0으로 지우면 합이 1이 아니다
틀린 = 점수.softmax(dim=-1) * 진짜[:, None, :]
print("softmax 뒤에 지우면 줄마다 합:", 틀린[0].sum(dim=-1))
```

```
첫 문장의 가중치(채운 칸 열은 0):
tensor([[0.650, 0.107, 0.242, 0.000, 0.000],
        [0.036, 0.938, 0.026, 0.000, 0.000],
        [0.332, 0.109, 0.560, 0.000, 0.000],
        [0.154, 0.645, 0.200, 0.000, 0.000],
        [0.594, 0.016, 0.390, 0.000, 0.000]])
줄마다 합: tensor([1.000, 1.000, 1.000, 1.000, 1.000])
softmax 뒤에 지우면 줄마다 합: tensor([0.567, 0.910, 0.573, 0.374, 0.303])
```

`masked_fill(조건, 값)`은 조건이 참인 칸을 값으로 바꾼 새 텐서를 돌려줍니다. 4장 4.4절의 `torch.where`와 같은 일을 하는 지름길이에요. 마스크 모양 `~진짜[:, None, :]`를 읽어 보세요. `진짜`는 `(2, 5)`인데 가운데에 `None`으로 차원을 끼워 넣어 `(2, 1, 5)`로 만들었습니다. 그러면 브로드캐스팅(4장 4.6절)으로 `(2, 5, 5)` 점수표의 **모든 줄에 같은 열 마스크**가 적용돼요. "어떤 글자가 묻든(줄), 채운 칸(열)은 보지 마"라는 뜻입니다. `~`는 참과 거짓을 뒤집어 "채운 칸"을 고른 것이고요.

결과를 보면 첫 문장의 3번, 4번 열이 모두 0.000이고, 줄마다 합은 여전히 1입니다. 진짜 글자 세 개만으로 비율을 다시 나눠 가진 거예요.

마지막 줄은 흔한 실수입니다. 소프트맥스를 먼저 하고 채운 칸을 0으로 지우면, 지운 만큼 합이 모자라서 줄마다 합이 0.567, 0.910처럼 1보다 작아져요. 가릴 칸은 반드시 **소프트맥스 전에 `-inf`로** 가리세요.

앞으로 올 글자 가리기: 인과 마스크

GPT 같은 모델은 글을 한 글자씩 이어 씁니다. "대한민국"까지 보고 다음 글자를 맞히는 식이죠. 그런데 학습할 때는 문장 전체를 한 번에 넣으니, 셀프 어텐션이 **아직 나오지 않은 뒤쪽 글자**까지 훔쳐볼 수 있습니다. 시험 문제를 풀면서 답안지를 보는 셈이에요. 그래서 글자마다 **자기와 자기 앞의 글자만** 보도록 가립니다. 원인이 결과보다 앞선다는 뜻에서 **인과 마스크**(causal mask) 라고 불러요.

```
import math

import torch

T = 5
보임 = torch.tril(torch.ones(T, T, dtype=torch.bool))    # 아래 삼각형: 나와 내 앞만 보임
print(보임.int())

torch.manual_seed(0)
torch.set_printoptions(precision=2)
x = torch.randn(1, T, 4)


def 인과_어텐션(x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
    점수 = x @ x.transpose(-2, -1) / math.sqrt(x.size(-1))
    가중치 = 점수.masked_fill(~보임, float("-inf")).softmax(dim=-1)
    return 가중치 @ x, 가중치


출력, 가중치 = 인과_어텐션(x)
print(가중치[0])

# 마지막 글자를 바꿔도 앞 글자들의 출력은 그대로일까?
바꾼_x = x.clone()
바꾼_x[0, -1] = 100.0
바꾼_출력, _ = 인과_어텐션(바꾼_x)
print("앞 네 글자 출력이 같은가:", torch.allclose(출력[:, :4], 바꾼_출력[:, :4]))
print("마지막 글자 출력이 같은가:", torch.allclose(출력[:, 4], 바꾼_출력[:, 4]))
```

```
tensor([[1, 0, 0, 0, 0],
        [1, 1, 0, 0, 0],
        [1, 1, 1, 0, 0],
        [1, 1, 1, 1, 0],
        [1, 1, 1, 1, 1]], dtype=torch.int32)
tensor([[1.00, 0.00, 0.00, 0.00, 0.00],
        [0.05, 0.95, 0.00, 0.00, 0.00],
        [0.13, 0.25, 0.63, 0.00, 0.00],
        [0.01, 0.01, 0.09, 0.89, 0.00],
        [0.01, 0.01, 0.11, 0.30, 0.58]])
앞 네 글자 출력이 같은가: True
마지막 글자 출력이 같은가: False
```

`torch.tril`은 행렬의 아래 삼각형(대각선 포함)만 남기는 함수입니다. 0번 글자는 자기만, 1번 글자는 0~1번만, 4번 글자는 0~4번 모두를 볼 수 있어요. 가중치 표도 대각선 위가 모두 0이 됐고, 0번 글자는 볼 수 있는 게 자기뿐이라 1.00입니다.

마지막 확인이 핵심입니다. 마지막 글자를 100이라는 엉뚱한 값으로 바꿨는데 **앞 네 글자의 출력은 하나도 바뀌지 않았어요.** 앞 글자들은 마지막 글자를 아예 볼 수 없으니까요. 이 성질 덕분에 문장 하나를 넣으면 "첫 글자 보고 둘째 맞히기", "두 글자 보고 셋째 맞히기", ...를 **한 번의 계산으로 동시에** 학습할 수 있습니다. RNN처럼 차례를 기다릴 필요가 없어서 GPU로 훨씬 빨리 학습할 수 있어요. 12.8절에서 이 장점을 그대로 씁니다.

Q

Q. 모든 칸이 가려진 줄이 생기면 어떻게 되나요? 소프트맥스에 `-inf`만 들어가면 0 ÷ 0이 되어 그 줄 전체가 `nan`이 됩니다. 패딩 칸을 **줄(쿼리) 쪽까지** 가리거나, 문장 전체가 채운 칸이라 볼 수 있는 키가 하나도 없을 때 생겨요. 그 `nan`이 다음 층으로 퍼지면 학습 전체가 망가집니다. 그래서 패딩 마스크는 위 예제처럼 **열(키) 쪽에만** 적용해요. 위 결과에서 채운 칸인 3·4번 줄도 진짜 글자 세 개를 보며 멀쩡한 가중치를 받았죠. 그 줄들의 출력은 의미 없는 값이니 손실을 계산할 때 빼면 됩니다. 인과 마스크는 대각선(자기 자신)이 언제나 보이니 이런 일이 생기지 않아요.

12.5 멀티헤드 어텐션: 여러 관점으로 동시에 보기

헤드를 나누고 합치기

"고양이가 생선을 먹지 않았다"에서 "먹지"라는 글자가 봐야 할 곳은 하나가 아닙니다. 누가 먹는지(고양이), 무엇을 먹는지(생선), 바로 앞 글자가 무엇인지가 모두 필요하죠. 그런데 어텐션 한 번은 결국 비율 하나로 섞기 때문에, 여러 곳을 서로 다른 이유로 보기가 어렵습니다.

멀티헤드 어텐션(multi-head attention) 은 이 문제를 아주 단순하게 풉니다. 숫자 C개를 H조각으로 나눠서, **조각마다 따로 어텐션**을 하는 거예요. 조각 하나를 **헤드(head)** 라고 부릅니다. 한 문장을 여러 사람이 각자 다른 관점으로 읽고 의견을 모으는 셈이죠. 숫자 8개를 헤드 2개로 나누면 헤드마다 숫자 4개씩입니다. 이 나누고 합치는 모양 바꾸기가 4장 4.5절에서 예고한 바로 그 패턴이에요.

```
import torch

B, T, C, H = 2, 5, 8, 2                # 문장 2개, 글자 5개, 숫자 8개, 헤드 2개
D = C // H                             # 헤드마다 숫자 4개씩
x = torch.arange(B * T * C, dtype=torch.float32).view(B, T, C)

나눔 = x.view(B, T, H, D).transpose(1, 2)        # (B, T, C) → (B, T, H, D) → (B, H, T, D)
print("나눈 모양:", tuple(나눔.shape), "이어져 있나?", 나눔.is_contiguous())
print("0번 헤드 = 숫자 0~3번 칸:", torch.equal(나눔[:, 0], x[..., :D]))
print("1번 헤드 = 숫자 4~7번 칸:", torch.equal(나눔[:, 1], x[..., D:]))

결과 = 나눔 @ torch.eye(D)                       # 행렬 곱의 결과는 (B, H, T, D) 순서로 새로 놓인다
되돌림 = 결과.transpose(1, 2)                     # (B, H, T, D) → (B, T, H, D)
try:
    되돌림.view(B, T, C)
except RuntimeError as e:
    print("view 오류:", str(e).split("(")[0].strip())
합침 = 되돌림.contiguous().view(B, T, C)          # 4장의 permute → contiguous → view
print("다시 합친 모양:", tuple(합침.shape), "원래와 같은가?", torch.equal(합침, x))
```

```
나눈 모양: (2, 2, 5, 4) 이어져 있나? False
0번 헤드 = 숫자 0~3번 칸: True
1번 헤드 = 숫자 4~7번 칸: True
view 오류: view size is not compatible with input tensor's size and stride
다시 합친 모양: (2, 5, 8) 원래와 같은가? True
```

모양 바꾸기를 두 단계로 나눠 보세요.

1. `view(B, T, H, D)`: 마지막 차원 C = 8을 (헤드 2, 숫자 4)로 쪼갭니다. 숫자는 하나도 움직이지 않고 읽는 법만 바뀌어요.
2. `transpose(1, 2)`: 글자 차원과 헤드 차원을 바꿔 `(B, H, T, D)`로 만듭니다. 헤드를 배치처럼 앞으로 빼 두면, 4장 4.6절에서 본 대로 `@`가 **앞쪽 차원(B, H)을 배치로 보고** 헤드마다 따로 `(T, D) @ (D, T)`를 계산해 줘요.

확인해 보니 0번 헤드는 원래 숫자 0~3번 칸, 1번 헤드는 4~7번 칸과 똑같습니다. 헤드를 나눈다는 건 결국 **숫자 칸을 몇 묶음으로 나눠 따로 쓰는 것**이에요.

다시 합칠 때가 함정입니다. 어텐션 계산(행렬 곱)의 결과는 `(B, H, T, D)` 순서로 새로 놓이는데, 이걸 `transpose(1, 2)`로 `(B, T, H, D)`로 돌린 뒤 바로 `view(B, T, C)`를 하면 오류가 나요. 4장 4.5절에서 본 그대로, 전치한 텐서는 메모리가 보이는 순서대로 이어져 있지 않아서(`is_contiguous()`가 `False`) `view`로 펼 수 없거든요. `.contiguous()`로 보이는 순서대로 새로 늘어놓은 뒤에 `view`하면 됩니다. 이것이 트랜스포머 코드마다 나오는 `transpose(1, 2).contiguous().view(B, T, C)`의 정체예요.

멀티헤드 어텐션 직접 만들기

이제 조각을 모아 멀티헤드 어텐션을 완성하고, PyTorch에 준비된 `nn.MultiheadAttention`에 **같은 가중치**를 넣어서 결과가 똑같은지 확인하겠습니다.

```
import math

import torch
from torch import nn



class 멀티헤드_어텐션(nn.Module):
    def __init__(self, 차원: int, 헤드_수: int) -> None:
        super().__init__()
        self.헤드_수 = 헤드_수
        self.qkv = nn.Linear(차원, 3 * 차원)        # 쿼리·키·값을 한 번에 만든다
        self.출력 = nn.Linear(차원, 차원)            # 헤드들의 결과를 섞는다

    def forward(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
```

```
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(C, dim=2)                    # 각각 (B, T, C)
        q, k, v = (t.view(B, T, self.헤드_수, -1).transpose(1, 2) for t in (q, k, v))
        점수 = torch.einsum("bhqd,bhkd->bhqk", q, k) / math.sqrt(k.size(-1))   # (B, H, T, T)
        미래 = torch.ones(T, T, dtype=torch.bool).triu(1)          # 대각선 위 = 앞으로 올 글자
        가중치 = 점수.masked_fill(미래, float("-inf")).softmax(dim=-1)
        y = (가중치 @ v).transpose(1, 2).contiguous().view(B, T, C)
        return self.출력(y), 가중치


torch.manual_segment(0)
x = torch.randn(2, 6, 16)                    # 문장 2개 × 글자 6개 × 숫자 16개
우리 = 멀티헤드_어텐션(16, 헤드_수=4)
출력, 가중치 = 우리(x)
print("출력", tuple(출력.shape), "가중치", tuple(가중치.shape))
print("파라미터 수:", sum(p.numel() for p in 우리.parameters()))

# PyTorch의 nn.MultiheadAttention에 같은 가중치를 넣고 결과를 비교한다
파이토치 = nn.MultiheadAttention(16, num_heads=4, batch_first=True)
with torch.no_grad():
    파이토치.in_proj_weight.copy_(우리.qkv.weight)      # (3C, C): 쿼리·키·값 순서로 쌓여 있다
    파이토치.in_proj_bias.copy_(우리.qkv.bias)
    파이토치.out_proj.weight.copy_(우리.출력.weight)
    파이토치.out_proj.bias.copy_(우리.출력.bias)
미래 = torch.ones(6, 6, dtype=torch.bool).triu(1)        # True = 보지 마
그쪽_출력, 그쪽_가중치 = 파이토치(x, x, x, attn_mask=미래, average_attn_weights=False)
print("파라미터 수:", sum(p.numel() for p in 파이토치.parameters()))
print("출력이 같은가:", torch.allclose(출력, 그쪽_출력, atol=1e-6))
print("가중치가 같은가:", torch.allclose(가중치, 그쪽_가중치, atol=1e-6))
```

```
출력 (2, 6, 16) 가중치 (2, 4, 6, 6)
파라미터 수: 1088
파라미터 수: 1088
출력이 같은가: True
가중치가 같은가: True
```

12.3절의 **셀프_어텐션**과 달라진 점을 보세요.

- **qkv**: 쿼리·키·값용 `nn.Linear` 세 개를 `nn.Linear(차원, 3 * 차원)` 하나로 합쳤습니다. 한 번 곱한 뒤 `split(C, dim=2)`로 세 토막으로 자르면 따로 곱한 것과 같고, 큰 행렬 곱 한 번이 작은 곱 세 번보다 빨라요.
- **헤드 나누기**: 앞에서 본 `view` → `transpose`를 q, k, v에 똑같이 적용했습니다. `view`의 마지막 자리에 `-1`을 써서 헤드 차원은 알아서 계산하게 했어요(C ÷ H = 4).
- **점수**: `einsum("bhqd,bhkd->bhqk")`로 썼습니다. 4장 4.6절 끝에서 "(배치, 헤드, 단어, 차원)처럼 차원이 넷인 텐서를 곱할 때 einsum으로 쓰면 수식과 코드가 거의 그대로 겹쳐 보인다"고 했던 곳이 여기예요. 배치(b)와 헤드(h)는 짝을 맞춰 따로따로, 숫자(d)는 곱해서 없애고, 쿼리 글자(q)와 키 글자(k)가 남습니다.
- **인과 마스크**: `triu(1)`은 대각선 **위**만 참인 표라서, 12.4절의 `tril`을 뒤집은 "가릴 칸" 표입니다. `(T, T)` 마스크가 `(B, H, T, T)` 점수표에 브로드캐스팅돼요.
- **출력 층**: 헤드들의 결과를 이어 붙인 뒤 `nn.Linear` 하나를 더 통과시킵니다. 헤드마다 따로 본 것을 섞어 주는 단계예요.

파라미터는 `qkv`가 16 × 48 + 48 = 816개, **출력**이 16 × 16 + 16 = 272개로 모두 1,088개입니다. 헤드를 나눠도 파라미터 수는 늘지 않아요. 같은 숫자를 나눠 쓸 뿐이니까요.

`nn.MultiheadAttention`도 파라미터가 1,088개이고, 가중치를 옮겨 넣었더니 출력과 가중치가 모두 같았습니다. `in_proj_weight`가 우리 `qkv.weight`와 똑같이 쿼리·키·값 순서로 쌓인 `(3C, C)` 행렬이라 그대로 복사할 수 있었어요. 직접 짠 코드가 PyTorch 내부와 같은 계산이라는 확인입니다. 이렇게 **내장 부품과 결과를 맞춰 보는 것**은 직접 구현한 모델을 검증하는 가장 확실한 방법이에요.

한 줄짜리 함수: scaled_dot_product_attention

실무에서는 점수 계산부터 소프트맥스, 값 섞기까지를 함수 하나로 부르는 경우가 많습니다.

```
import math


import torch
from torch.nn import functional as F


torch.manual_seed(0)
B, H, T, D = 2, 4, 6, 8
q, k, v = torch.randn(B, H, T, D), torch.randn(B, H, T, D), torch.randn(B, H, T, D)


# 이 장에서 직접 쓴 계산
```

```
점수 = q @ k.transpose(-2, -1) / math.sqrt(D)
미래 = torch.ones(T, T, dtype=torch.bool).triu(1)
직접 = 점수.masked_fill(미래, float("-inf")).softmax(dim=-1) @ v

# PyTorch가 준비해 둔 한 줄짜리 함수
한줄 = F.scaled_dot_product_attention(q, k, v, is_causal=True)
print("모양:", tuple(한줄.shape), "같은가:", torch.allclose(직접, 한줄, atol=1e-6))

# 패딩 마스크도 받는다: True인 칸만 본다 (nn.MultiheadAttention과 뜻이 반대이니 주의)
보임 = torch.tensor([[True, True, True, True, False, False]])   # (1, T): 뒤의 두 칸은 채운 칸
직접2 = 점수.masked_fill(~보임, float("-inf")).softmax(dim=-1) @ v
한줄2 = F.scaled_dot_product_attention(q, k, v, attn_mask=보임)
print("패딩 마스크도 같은가:", torch.allclose(직접2, 한줄2, atol=1e-6))
```

```
모양: (2, 4, 6, 8) 같은가: True
패딩 마스크도 같은가: True
```

`F.scaled_dot_product_attention`은 이름 그대로 12.2절의 "크기를 맞춘 내적 어텐션"을 한 번에 계산합니다. `is_causal=True`를 주면 인과 마스크를 알아서 적용하고, `attn_mask`로 패딩 마스크를 넘길 수도 있어요. 두 경우 모두 직접 쓴 계산과 같았습니다.

결과가 같은데 왜 이 함수를 쓸까요? 장치에 따라 훨씬 빠르고 메모리를 덜 쓰는 계산법(예: FlashAttention)을 알아서 골라 주기 때문입니다. 12.9절에서 보겠지만 어텐션 가중치 표는 문장이 길어지면 엄청나게 커지는데, 이 함수는 그 표를 메모리에 통째로 만들지 않고도 같은 답을 내요. 이 장에서는 가중치를 들여다보려고 직접 계산을 쓰지만, 내 모델을 실제로 학습시킬 때는 이 함수로 바꾸는 것을 권합니다(16장).

주석에 적은 함정 하나는 꼭 기억하세요. 마스크의 `True`가 이 함수에서는 **"본다"**, `nn.MultiheadAttention`의 `attn_mask`에서는 **"가린다"** 입니다. 뜻이 정반대라서, 함수를 바꿀 때 마스크를 뒤집지 않으면 오류 없이 엉뚱한 칸만 보게 돼요.

12.6 순서 알려 주기: 위치 정보

사인·코사인 위치 인코딩

12.3절에서 어텐션은 글자 순서를 모른다는 것을 확인했습니다. 해결책은 간단해요. 글자 벡터에 **"나는 몇 번째 자리다"라는 정보를 더해** 넣는 겁니다. 자리마다 서로 다른 숫자 묶음을 하나씩 정해 두고, 그 자리의 글자 벡터에 더하는 거죠.

원래 트랜스포머 논문은 빠르기가 서로 다른 물결(사인·코사인)로 이 숫자 묶음을 만들었습니다. 시계를 떠올려 보세요. 초침은 빨리, 분침은 천천히, 시침은 더 천천히 돕니다. 세 바늘의 위치를 함께 보면 하루 중 어느 순간인지 정확히 알 수 있죠. 위치 인코딩도 칸마다 빠르기가 다른 바늘을 두고, 자리마다 그 바늘들이 가리키는 값을 적어 둡니다.

```
import matplotlib
import matplotlib.pyplot as plt
import torch

matplotlib.use("Agg")                              # 창을 띄우지 않고 파일로만 그리기
plt.rcParams["font.family"] = "AppleGothic"        # macOS 한글 글꼴
plt.rcParams["axes.unicode_minus"] = False
torch.set_printoptions(precision=2, sci_mode=False)


def 사인_위치_인코딩(길이: int, 차원: int) -> torch.Tensor:
    """위치마다 빠르기가 다른 사인·코사인 물결의 값을 이어 붙인 표 (길이, 차원)."""
    위치 = torch.arange(길이).unsqueeze(1)                        # (길이, 1)
    빠르기 = 10000 ** (-torch.arange(0, 차원, 2) / 차원)          # (차원/2,): 1부터 점점 느리게
    표 = torch.zeros(길이, 차원)
    표[:, 0::2] = torch.sin(위치 * 빠르기)                         # 짝수 칸: 사인
    표[:, 1::2] = torch.cos(위치 * 빠르기)                         # 홀수 칸: 코사인
    return 표


표 = 사인_위치_인코딩(100, 64)
print("모양:", tuple(표.shape))
print(표[:4, :6])                                  # 위치 0~3의 앞 칸 여섯 개

# 위치 20과 다른 위치들의 코사인 유사도: 가까울수록 닮았다
단위 = 표 / 표.norm(dim=1, keepdim=True)
```

```
닮음 = 단위 @ 단위[20]
for 위치 in [20, 21, 22, 25, 30, 40, 60, 99]:
    print(f"위치 20 vs {위치:2d}: {닮음[위치]:.3f}")

# 표 전체를 그림으로: 가로는 위치, 세로는 칸 번호
그림, 축 = plt.subplots(figsize=(7, 3.6))
색 = 축.imshow(표.T, aspect="auto", cmap="RdBu", vmin=-1, vmax=1)
축.set_xlabel("위치 (몇 번째 글자)")
축.set_ylabel("칸 번호 (0~63)")
그림.colorbar(색, ax=축, fraction=0.03)
그림.tight_layout()
그림.savefig("positional_encoding.png", dpi=200)
```

```
모양: (100, 64)
tensor([[ 0.00,  1.00,  0.00,  1.00,  0.00,  1.00],
        [ 0.84,  0.54,  0.68,  0.73,  0.53,  0.85],
        [ 0.91, -0.42,  1.00,  0.07,  0.90,  0.43],
        [ 0.14, -0.99,  0.78, -0.63,  0.99, -0.12]])
위치 20 vs 20: 1.000
위치 20 vs 21: 0.966
위치 20 vs 22: 0.884
위치 20 vs 25: 0.734
위치 20 vs 30: 0.658
위치 20 vs 40: 0.590
위치 20 vs 60: 0.481
위치 20 vs 99: 0.466
```

`빠르기`는 첫 칸이 1이고 뒤 칸으로 갈수록 점점 느려집니다. 출력의 첫 두 칸(가장 빠른 바늘)은 위치 0, 1, 2, 3에서 0.00 → 0.84 → 0.91 → 0.14로 크게 출렁이지만, 다섯째·여섯째 칸(느린 바늘)은 0.00 → 0.53 → 0.90 → 0.99로 천천히 움직이죠. 짝수 칸에 사인, 홀수 칸에 코사인을 넣은 것은 `표[:, 0::2]`, `표[:, 1::2]`처럼 1권 6장의 슬라이싱 간격으로 칸을 골라 채웠습니다.

아래쪽 결과가 이 방법의 좋은 점입니다. 위치 20과 다른 위치의 코사인 유사도(4장 4.11절 실습의 그 계산)를 쟀더니 21은 0.966, 25는 0.734, 60은 0.481로 **가까운 위치끼리 더 닮았어요.** 모델이 "바로 옆 글자", "조금 떨어진 글자"를 구분하는 데 쓸 수 있는 성질입니다. 코드 마지막 부분은 표 전체를 `imshow`로 그려 `positional_encoding.png`에 저장합니다. 위쪽 칸(빠른 바늘)은 촘촘하게, 아래

쪽 칸(느린 바늘)은 느리게 출렁이는 모습이 보여요.

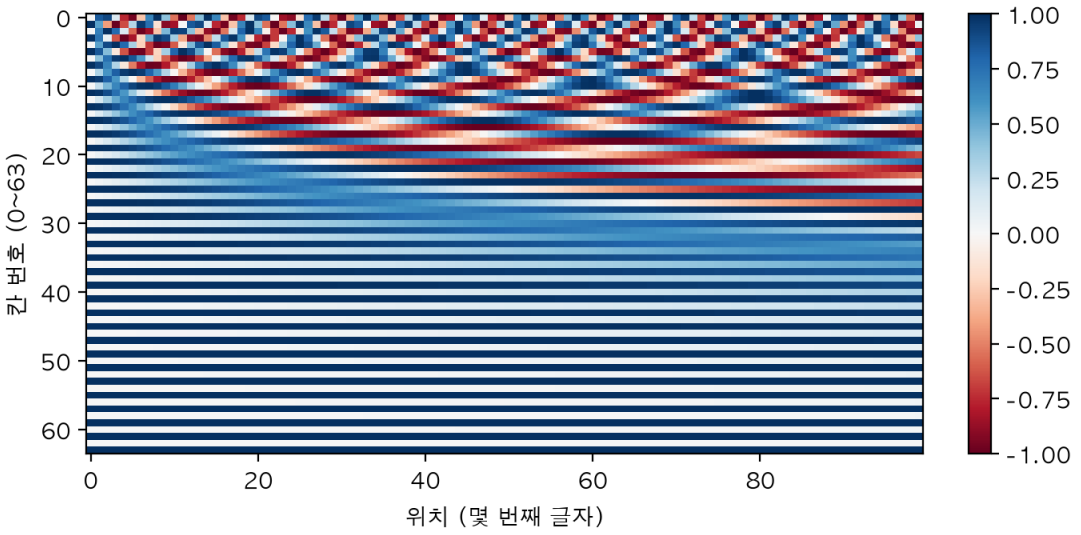


그림 12.1: 사인·코사인 위치 인코딩 표. 가로축은 위치, 세로축은 칸 번호다. 위쪽 칸은 위치에 따라 빠르게, 아래쪽 칸은 천천히 값이 바뀐다.

학습하는 위치 임베딩

GPT는 더 단순한 방법을 씁니다. 자리마다 숫자 묶음을 **무작위로 시작해서 학습으로 정하게** 하는 거예요. 이때 쓰는 부품이 11장 11.11절에서 글자 번호를 벡터로 바꿀 때 쓴 `nn.Embedding`입니다. 글자 번호 대신 **자리 번호**를 넣으면 위치 임베딩이 돼요.

```
import math

import torch
from torch import nn

torch.manual_seed(0)
torch.set_printoptions(precision=3)
글자_임베딩 = nn.Embedding(10, 4)          # 글자 10종류, 글자마다 숫자 4개 (학습되는 표)
번호들 = torch.tensor([[3, 7, 3]])         # 글자 번호 세 개 (첫째와 셋째가 같은 글자)
x = 글자_임베딩(번호들)
print("모양:", tuple(번호들.shape), "→", tuple(x.shape))
print(x[0].detach())
```

```
print("표의 3번 줄을 꺼낸 것과 같은가:", torch.equal(x[0, 0], 글자_임베딩.weight[3]))

# 위치 임베딩을 더하면 같은 글자라도 자리에 따라 다른 숫자가 된다
위치_임베딩 = nn.Embedding(8, 4)            # 위치 0~7, 위치마다 숫자 4개
x = x + 위치_임베딩(torch.arange(3))
print("위치를 더한 뒤 0번 == 2번 자리:", torch.allclose(x[0, 0], x[0, 2]))

# 12.3절의 실험을 다시: 순서를 섞으면 이제 결과가 달라진다
층 = nn.Linear(4, 4, bias=False)


def 셀프_어텐션(x: torch.Tensor) -> torch.Tensor:
    return (층(x) @ 층(x).transpose(-2, -1) / math.sqrt(4)).softmax(dim=-1) @ x


with torch.no_grad():
    섞은_번호 = 번호들[:, [1, 0, 2]]
    원래 = 셀프_어텐션(글자_임베딩(번호들) + 위치_임베딩(torch.arange(3)))
    섞음 = 셀프_어텐션(글자_임베딩(섞은_번호) + 위치_임베딩(torch.arange(3)))
print("위치가 있으면 섞은 결과가 원래를 섞은 것과 같은가:", torch.allclose(섞음, 원래[:, [1, 0,
↪  2]]))
```

```
모양: (1, 3) → (1, 3, 4)
tensor([[ 0.120,  1.238,  1.117, -0.247],
        [-0.174, -0.679,  0.938,  0.489],
        [ 0.120,  1.238,  1.117, -0.247]])
표의 3번 줄을 꺼낸 것과 같은가: True
위치를 더한 뒤 0번 == 2번 자리: False
위치가 있으면 섞은 결과가 원래를 섞은 것과 같은가: False
```

11장에서 본 대로 `nn.Embedding(10, 4)`는 **10줄 4칸짜리 표**이고, 번호를 주면 그 번호의 줄을 꺼내 줍니다(원-핫 @ 표와 같은 계산이었죠). 번호 `[3, 7, 3]`을 넣으니 `(1, 3, 4)`가 나왔고, 첫째와 셋째 줄이 같은 3번 줄이라 숫자도 똑같아요.

위치도 똑같이 `nn.Embedding(최대 길이, 숫자 개수)`로 만들어 `torch.arange(글자 수)`로 꺼낸 뒤 글자 벡터에 더합니다. 그러자 같은 글자였던 0번과 2번 자리가 더 이상 같지 않아요. 같은 글자

라도 **자리에 따라 다른 벡터**가 된 거죠. 12.3절의 섞기 실험을 다시 해 보니 이번에는 `False`가 나왔습니다. 순서를 바꾸면 결과가 달라지는, 순서를 아는 모델이 된 거예요.

Q. 사인·코사인과 학습하는 위치 임베딩 중 무엇이 좋나요? 작은 모델에서는 성능 차이가 크지 않습니다. 학습하는 방식은 단순하고 데이터에 맞춰지지만, 표에 없는 길이(예: 최대 128자로 만든 모델에 200자)는 다룰 수 없어요. 사인·코사인은 공식이라 어떤 길이든 계산할 수 있습니다. 요즘 LLM은 쿼리와 키를 위치만큼 회전시키는 RoPE 같은 방법을 많이 쓰는데, "자리 정보를 어텐션 점수에 섞어 넣는다"는 생각은 같아요. 이 장의 작은 GPT는 GPT-2처럼 학습하는 위치 임베딩을 씁니다.

12.7 트랜스포머 블록 조립하기

어텐션만으로는 모델이 되지 않습니다. 트랜스포머는 어텐션에 세 가지 부품을 더한 **블록**을 여러 층 쌓아 만들어요. 층 정규화, 잔차 연결, 피드포워드입니다.

층 정규화: 글자마다 따로 표준화하기

8장 8.4절의 배치 정규화는 **배치 안의 여러 샘플**을 모아 특성마다 평균과 분산을 냈습니다. 그런데 문장에서는 이 방식이 불편해요. 문장마다 길이가 달라 패딩 칸이 평균에 섞이고, 글을 한 글자씩 만들어 낼 때는 배치가 1개뿐이기도 하니까요. 그래서 트랜스포머는 **글자 하나의 숫자들만으로** 평균과 분산을 내는 **층 정규화**(layer normalization) 를 씁니다.

```
import torch
from torch import nn

torch.set_printoptions(precision=3)
# 문장 2개 × 글자 3개 × 숫자 4개. 둘째 문장의 마지막 글자는 채운 칸(0)이다
x = torch.tensor([[[1.0, 2.0, 3.0, 4.0], [10.0, 20.0, 30.0, 40.0], [2.0, 2.0, 2.0, 6.0]],
                  [[0.5, 0.0, 0.5, 1.0], [3.0, 1.0, 4.0, 1.0], [0.0, 0.0, 0.0, 0.0]]])
층정규화 = nn.LayerNorm(4)                # 글자 하나의 숫자 4개를 표준화한다
출력 = 층정규화(x)
print(출력[0].detach())

# 손으로: 글자마다(마지막 차원 방향으로) 평균과 분산을 낸다
평균 = x.mean(dim=-1, keepdim=True)
```

```
분산 = x.var(dim=-1, keepdim=True, unbiased=False)
print("손 계산과 같은가:", torch.allclose(출력, (x - 평균) / torch.sqrt(분산 + 층정규화.eps),
↪  atol=1e-5))
print("글자 1개만 넣어도 된다:", tuple(층정규화(x[:1, :1]).shape))
print("다른 글자와 섞이지 않는다:", torch.allclose(층정규화(x[:1, :1]), 출력[:1, :1]))
```

```
tensor([[-1.342, -0.447,  0.447,  1.342],
        [-1.342, -0.447,  0.447,  1.342],
        [-0.577, -0.577, -0.577,  1.732]])
손 계산과 같은가: True
글자 1개만 넣어도 된다: (1, 1, 4)
다른 글자와 섞이지 않는다: True
```

첫 문장의 첫 두 글자 `[1, 2, 3, 4]`와 `[10, 20, 30, 40]`은 크기가 10배 다른데, 층 정규화를 하니 똑같이 `[-1.342, -0.447, 0.447, 1.342]`가 됐습니다. 글자마다 **자기 숫자 4개**의 평균을 빼고 표준편차로 나눴기 때문이에요. 손 계산에서도 `dim=-1`, 즉 마지막 차원(숫자 방향)으로 평균과 분산을 냈습니다. 8장의 배치 정규화가 `dim=0`(샘플 방향)으로 냈던 것과 방향이 다르죠.

그래서 아래 두 줄이 가능합니다. 글자 **하나만** 넣어도 잘 동작하고(8장에서 배치 정규화는 샘플 하나를 넣으면 오류가 났죠), 그 결과가 배치 전체를 넣었을 때와 같아요. 다른 글자나 다른 문장, 패딩 칸과 전혀 섞이지 않는다는 뜻입니다. 학습할 때와 평가할 때 계산이 같아서 `train()`/`eval()`에 따라 동작이 달라지지도 않아요. `nn.LayerNorm(4)`에도 배치 정규화처럼 표준화 뒤에 곱하고 더하는 학습 숫자가 4개씩 들어 있습니다.

잔차 연결: 깊이 쌓아도 기울기가 흐르게

9장에서 ResNet의 잔차 연결을 만들었죠. 층의 출력에 **입력을 그대로 더해 주는** 지름길입니다. 트랜스포머도 블록 안의 어텐션과 피드포워드마다 잔차 연결을 둡니다. 효과를 다시 한번 숫자로 보겠습니다. `nn.Linear`와 tanh를 32층 쌓고, 첫 층과 마지막 층의 기울기 크기를 재요.

```
import torch
from torch import nn

torch.manual_seed(0)
깊이 = 32
```

```
층들 = nn.ModuleList(nn.Sequential(nn.Linear(64, 64), nn.Tanh()) for _ in range(깊이))
x = torch.randn(16, 64)

for 잔차 in (False, True):
    층들.zero_grad()
    h = x
    for 층 in 층들:
        h = h + 층(h) if 잔차 else 층(h)        # 잔차 연결: 입력을 출력에 더해 준다
    h.pow(2).mean().backward()
    첫층 = 층들[0][0].weight.grad.norm().item()
    끝층 = 층들[-1][0].weight.grad.norm().item()
    print(f"잔차 {'있음' if 잔차 else '없음'}: 첫 층 기울기 {첫층:.2e}, 마지막 층 기울기 {끝
    ↪ 층:.2e}")
```

```
잔차 없음: 첫 층 기울기 1.44e-09, 마지막 층 기울기 1.53e-02
잔차 있음: 첫 층 기울기 1.10e+01, 마지막 층 기울기 5.30e+00
```

잔차가 없으면 마지막 층의 기울기는 0.0153인데 첫 층은 1.44e-09, 즉 0.00000000144입니다. 역전파(5장 5.6절)는 층마다 기울기를 곱해 가며 거꾸로 내려가는데, 32번을 곱하는 동안 기울기가 거의 0으로 줄어든 거예요. 이러면 앞쪽 층은 사실상 학습되지 않습니다. 잔차 연결을 넣으니 첫 층의 기울기가 11.0으로 오히려 마지막 층보다 커졌어요. `h + 층(h)`를 미분하면 "1 + 층의 기울기"가 되어, 층의 기울기가 아무리 작아도 **1이라는 직통 길**이 남기 때문입니다. 트랜스포머가 수십, 수백 층으로 깊어질 수 있는 이유예요.

블록 하나 완성하기

이제 부품을 모읍니다. **피드포워드(feed-forward)** 는 6장에서 만든 그 다층 신경망(`Linear` → 활성화 함수 → `Linear`)으로, 숫자 개수를 4배로 넓혔다가 다시 줄입니다. 활성화 함수로는 ReLU를 부드럽게 만든 GELU를 많이 써요. 어텐션이 **글자끼리 정보를 주고받는** 단계라면, 피드포워드는 모은 정보를 가지고 **글자마다 혼자 생각하는** 단계입니다. 회의(어텐션)를 하고 나서 각자 자리로 돌아가 정리하는(피드포워드) 것과 비슷해요.

```
import math


import torch
```

```
from torch import nn


class 멀티헤드_어텐션(nn.Module):
    def __init__(self, 차원: int, 헤드_수: int) -> None:
        super().__init__()
        self.헤드_수 = 헤드_수
        self.qkv = nn.Linear(차원, 3 * 차원)
        self.출력 = nn.Linear(차원, 차원)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, T, C = x.shape
        q, k, v = (t.view(B, T, self.헤드_수, -1).transpose(1, 2) for t in self.qkv(x).split(C,
↪   dim=2))
        점수 = q @ k.transpose(-2, -1) / math.sqrt(k.size(-1))
        점수 = 점수.masked_fill(torch.ones(T, T, dtype=torch.bool).triu(1), float("-inf"))
        return self.출력((점수.softmax(dim=-1) @ v).transpose(1, 2).contiguous().view(B, T, C))


class 블록(nn.Module):
    def __init__(self, 차원: int, 헤드_수: int) -> None:
        super().__init__()
        self.정규화1 = nn.LayerNorm(차원)
        self.어텐션 = 멀티헤드_어텐션(차원, 헤드_수)
        self.정규화2 = nn.LayerNorm(차원)
        self.피드포워드 = nn.Sequential(nn.Linear(차원, 4 * 차원), nn.GELU(), nn.Linear(4 * 차원,
        ↪   차원))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x + self.어텐션(self.정규화1(x))        # 글자끼리 정보 주고받기
        x = x + self.피드포워드(self.정규화2(x))    # 글자마다 혼자 생각하기
        return x


C = 128
b = 블록(C, 헤드_수=4)
```

```
x = torch.randn(2, 10, C)
print("입력", tuple(x.shape), "→ 출력", tuple(b(x).shape))
for 이름, 부품 in b.named_children():
    print(f"  {이름}: {sum(p.numel() for p in 부품.parameters()):,}개")
print(f"합계: {sum(p.numel() for p in b.parameters()):,}개 (12 × C² = {12 * C * C:,})")

내장 = nn.TransformerEncoderLayer(C, nhead=4, dim_feedforward=4 * C, activation="gelu",
                                  batch_first=True, norm_first=True)
print(f"nn.TransformerEncoderLayer: {sum(p.numel() for p in 내장.parameters()):,}개")
```

```
입력 (2, 10, 128) → 출력 (2, 10, 128)
  정규화1: 256개
  어텐션: 66,048개
  정규화2: 256개
  피드포워드: 131,712개
합계: 198,272개 (12 × C² = 196,608)
nn.TransformerEncoderLayer: 198,272개
```

`블록.forward`의 두 줄이 트랜스포머 블록의 전부입니다.

```
x = x + 어텐션(층정규화(x))        # 회의: 글자끼리 정보 주고받기
x = x + 피드포워드(층정규화(x))    # 정리: 글자마다 혼자 생각하기
```

정규화를 어텐션과 피드포워드 **앞에** 두는 이 방식을 Pre-LN이라고 합니다. 원래 논문은 더한 **뒤에** 정규화했지만(Post-LN), 앞에 두는 쪽이 깊게 쌓아도 학습이 안정적이어서 GPT-2 이후 대부분의 모델이 이 순서를 씁니다. 입력 `(2, 10, 128)`이 출력 `(2, 10, 128)`로 모양이 그대로이니, 블록을 원하는 만큼 겹쳐 쌓을 수 있어요.

파라미터를 세어 보면 어텐션이 66,048개, 피드포워드가 131,712개로 **피드포워드가 두 배**입니다. 어텐션은 C × 3C와 C × C 행렬, 피드포워드는 C × 4C와 4C × C 행렬이라 대략 $4C^2$와 $8C^2$, 합쳐서 **$12 \times C^2$** 입니다. C = 128이면 196,608개이고, 편향과 층 정규화의 숫자를 더하면 198,272개예요. 모델 크기를 어림할 때 "블록 하나에 $12C^2$"라는 계산은 GPT 계열 어디에나 통합니다.

마지막 줄은 PyTorch에 준비된 `nn.TransformerEncoderLayer`를 같은 설정(정규화를 앞에, GELU, 피드포워드 4배)으로 만든 것입니다. 파라미터 수가 198,272개로 정확히 같아요. 이름은 "인코더"지만 인과 마스크를 넘기면 GPT의 블록으로도 쓸 수 있습니다. 우리 블록이 이 내장 부품과 같은 구조

라는 뜻이니, 나중에 내 모델을 만들 때 직접 짠 블록 대신 이 부품을 가져다 써도 됩니다.

인코더, 디코더, 그리고 교차 어텐션

"인코더"라는 이름이 나왔으니 트랜스포머의 가족 관계를 정리해 두겠습니다. 원래 트랜스포머는 번역기로 만들어졌고 두 부분으로 되어 있었어요. 원문을 읽는 **인코더**는 마스크 없이 문장 전체를 앞뒤로 다 보고, 번역문을 쓰는 **디코더**는 인과 마스크로 앞만 보면서 한 글자씩 써 나갑니다. 디코더가 원문을 참고할 때는 **쿼리는 번역문에서, 키와 값은 원문에서** 가져오는 어텐션을 하는데, 이것을 **교차 어텐션**(cross-attention) 이라고 해요.

```
import torch
from torch import nn

torch.manual_seed(0)
torch.set_printoptions(precision=2)
교차_어텐션 = nn.MultiheadAttention(16, num_heads=4, batch_first=True)
원문 = torch.randn(1, 4, 16)          # 인코더가 읽은 원문 단어 4개 (예: 영어 문장)
번역문 = torch.randn(1, 7, 16)        # 디코더가 지금까지 쓴 번역문 7글자

# 쿼리는 번역문에서, 키와 값은 원문에서: "다음 글자를 쓰려면 원문의 어디를 봐야 하나"
출력, 가중치 = 교차_어텐션(번역문, 원문, 원문)
print("출력", tuple(출력.shape), "가중치", tuple(가중치.shape))
print("번역문 0번 글자가 원문 단어들을 보는 비율:", 가중치[0, 0].detach())
```

```
출력 (1, 7, 16) 가중치 (1, 7, 4)
번역문 0번 글자가 원문 단어들을 보는 비율: tensor([0.25, 0.36, 0.18, 0.21])
```

쿼리(번역문 7글자)와 키·값(원문 4단어)의 개수가 달라도 계산이 됩니다. 12.1절에서 정리한 규칙 그대로, 쿼리와 키는 숫자 개수만 같으면 되고 키와 값은 개수가 같으면 되니까요. 가중치 모양 `(1, 7, 4)`는 "번역문 글자마다 원문 단어 4개를 얼마씩 보는가"입니다. 셀프 어텐션은 이 함수에 같은 텐서를 세 번 넣는 특별한 경우였던 셈이에요.

오늘날의 모델은 이 중 한쪽만 쓰는 경우가 많습니다.

종류	구조	마스크	대표 모델	잘하는 일
인코더만	셀프 어텐션 블록	없음(앞뒤 다 봄)	BERT	문장 이해, 분류 (13장)
디코더만	셀프 어텐션 블록	인과 마스크	GPT	글 이어 쓰기, 대화 (14장)
인코더-디코더	둘 다 + 교차 어텐션	디코더에만 인과	원래 트랜스포머, T5	번역, 요약

이 장에서 만들 작은 GPT는 둘째 줄, 디코더만 쓰는 모델입니다.

12.8 작은 GPT 만들기

학습 재료: 헌법과 민법

부품이 다 모였으니 진짜 글을 학습시킬 차례입니다. 재료로는 **대한민국헌법(1987년 전부개정)과 민법(법률 제17503호)의 조문**을 쓰겠습니다. 헌법과 법률은 저작권법 제7조에 따라 저작권 보호를 받지 않는 저작물이라 누구나 자유롭게 쓸 수 있고, 띄어쓰기와 맞춤법이 반듯한 요즘 한국어인 데다, "제N조(제목) ① …"처럼 구조가 뚜렷해서 모델이 무엇을 배웠는지 눈으로 확인하기 좋아요. 원문은 위키문헌(ko.wikisource.org)에서 받습니다.

이 절과 다음 절의 코드는 여러 예제가 함께 쓰니, 8장에서 `학습도구.py`를 만들었던 것처럼 두 파일로 모아 두겠습니다(1권 14장의 모듈). 먼저 데이터를 준비하는 `법전.py`입니다. 예제를 실행하는 폴더에 저장하세요.

```
"""12장 작은 GPT의 학습 재료: 대한민국헌법과 민법 조문을 받아 글자 번호로 바꾼다."""
import os
import re
import urllib.parse
import urllib.request
from pathlib import Path

import torch
```

```
데이터_폴더 = Path(os.environ.get("BOOK_DATA_DIR", "data"))
위키문헌 = "https://ko.wikisource.org/w/index.php?action=raw&title="
# 위키문헌 문서 제목: (본문이 시작하는 제목, 끝나는 제목). 부칙은 뺀다
원문들 = {"대한민국헌법 (제10호)": ("== 전문 ==", "== 부칙 =="),
          "민법 (대한민국, 제17503호)": ("== 본칙 ==", "== 부칙 ==")}


def 위키문헌_조문(제목: str, 시작: str, 끝: str) -> list[str]:
    """위키문헌에서 법령 원문을 받아 위키 문법을 지우고 줄 목록으로 돌려준다."""
    요청 = urllib.request.Request(위키문헌 + urllib.parse.quote(제목),
                                  headers={"User-Agent": "python-book-example"})
    with urllib.request.urlopen(요청, timeout=30) as 응답:
        원문 = 응답.read().decode("utf-8")
    줄들 = []
    for 줄 in 원문.split(시작)[1].split(끝)[0].splitlines():
        if re.match(r"==[^=]", 줄):                                  # 큰 제목(조문, 본칙)은 버린다
            continue
        줄 = re.sub(r"<[^>]+>|'''|\{\{[^}]*\}\}|^[*:]+\s*", "", 줄)    # 태그, 굵게, 틀, 목록 기호
        줄 = re.sub(r"\[\[(?:[^]|]*\|)?([^]]*)\]{1,2}", r"\1", 줄)    # [[#1001|제1001조]] → 제1001
↪ 조
        줄 = re.sub(r"\([\u4e00-\u9fff·]+\)", "", 줄)                # 괄호 속 한자 풀이
        줄 = re.sub(r"^=+\s*(.*?)\s*=+$", r"\1", 줄).strip()         # === 제1장 총강 === → 제1장
↪ 총강
        if 줄 and not 줄.startswith("["):                            # [본조신설 …] 같은 개정 기록
            줄들.append(줄)
    return 줄들


def 법전_읽기() -> str:
    """헌법과 민법 조문을 이어 붙인 글. 처음 한 번만 받아서 data/법전.txt에 저장한다."""
    경로 = 데이터_폴더 / "법전.txt"
    if not 경로.exists():
        줄들 = [줄 for 제목, (시작, 끝) in 원문들.items() for 줄 in 위키문헌_조문(제목, 시작, 끝)]
        데이터_폴더.mkdir(parents=True, exist_ok=True)
        경로.write_text("\n".join(줄들) + "\n", encoding="utf-8")
    return 경로.read_text(encoding="utf-8")
```

```
class 글자표:
    """글자와 번호를 서로 바꿔 주는 표."""

    def __init__(self, 글: str) -> None:
        self.글자들 = sorted(set(글))
        self.번호 = {글자: i for i, 글자 in enumerate(self.글자들)}

    def __len__(self) -> int:
        return len(self.글자들)

    def 인코딩(self, 글: str) -> torch.Tensor:
        return torch.tensor([self.번호[글자] for 글자 in 글])

    def 디코딩(self, 번호들: torch.Tensor) -> str:
        return "".join(self.글자들[i] for i in 번호들.tolist())


def 준비() -> tuple[글자표, torch.Tensor, torch.Tensor]:
    """글 전체를 번호로 바꿔 앞 90%는 학습용, 뒤 10%는 검증용으로 나눈다."""
    글 = 법전_읽기()
    표 = 글자표(글)
    전체 = 표.인코딩(글)
    경계 = int(len(전체) * 0.9)
    return 표, 전체[:경계], 전체[경계:]


def 배치_뽑기(데이터: torch.Tensor, 문맥_길이: int, 배치_크기: int,
              생성기: torch.Generator) -> tuple[torch.Tensor, torch.Tensor]:
    """아무 곳에서나 문맥_길이만큼 잘라 입력으로, 한 글자 밀린 조각을 정답으로."""
    시작 = torch.randint(len(데이터) - 문맥_길이, (배치_크기,), generator=생성기)
    x = torch.stack([데이터[i:i + 문맥_길이] for i in 시작])
    y = torch.stack([데이터[i + 1:i + 1 + 문맥_길이] for i in 시작])
    return x, y
```

하는 일을 순서대로 짚어 보겠습니다.

- **위키문헌_조문**: 위키문헌의 원문 보기 주소에 문서 제목을 붙여 위키 문법이 섞인 원문을 받습니다. 1권 17장의 `urllib.request`를 썼고, 위키문헌은 어떤 프로그램이 요청하는지 밝히라고 해서 `User-Agent` 머리글을 붙였어요. 받은 글에서 본문만 잘라 낸 뒤, 1권 17장의 정규표현식으로 태그, 굵게 표시, 링크 기호, 괄호 속 한자 풀이, "[본조신설 ...]" 같은 개정 기록을 지웁니다.
- **법전_읽기**: 헌법과 민법을 이어 붙여 `data/법전.txt`에 저장합니다. 파일이 있으면 다시 받지 않아요. 맨 위의 **데이터_폴더**는 환경 변수 `BOOK_DATA_DIR`이 있으면 그 폴더를, 없으면 `data`를 씁니다. 여러 장이 데이터를 한 폴더에서 같이 쓰게 하는 설정이라 따로 정하지 않았다면 신경 쓰지 않아도 돼요(부록 A.9절).
- **글자표**: 7장 7.7절에서 리뷰 데이터셋을 만들 때 했던 "글자마다 번호 붙이기"를 클래스로 묶었습니다. **인코딩**은 글을 번호 텐서로, **디코딩**은 번호를 다시 글로 바꿔요.
- **준비**와 **배치_뽑기**: 잠시 뒤에 자세히 봅니다.

데이터를 받아 둘러보겠습니다.

```
from collections import Counter

from 법전 import 글자표, 법전_읽기

글 = 법전_읽기()                      # 처음 한 번은 위키문헌에서 받아 data/법전.txt로 저장
줄들 = 글.splitlines()
print(f"글자 {len(글):,}개, 줄 {len(줄들):,}개")
for 줄 in 줄들[1:4] + 줄들[150:152]:
    print(" ", 줄[:60])

표 = 글자표(글)
print("글자 종류:", len(표))
print("자주 나오는 글자:", Counter(글).most_common(12))
번호들 = 표.인코딩("제1조 ① 대한민국은")
print(번호들.tolist())
print(표.디코딩(번호들))
```

```
글자 165,422개, 줄 2,454개
  제1장 총강
  제1조 ① 대한민국은 민주공화국이다.
```

```
  ② 대한민국의 주권은 국민에게 있고, 모든 권력은 국민으로부터 나온다.
  ② 제1항의 탄핵소추는 국회재적의원 3분의 1 이상의 발의가 있어야 하며, 그 의결은 국회재적의원 과반
↪ 수의
  ③ 탄핵소추의 의결을 받은 자는 탄핵심판이 있을 때까지 그 권한행사가 정지된다.
글자 종류: 478
자주 나오는 글자: [(' ', 34984), ('의', 5817), ('에', 3931), ('한', 3481), ('는', 3357), ('제',
↪ 3192), ('하', 3111), ('다', 2676), ('.', 2578), ('\n', 2454), ('자', 2387), ('이', 2367)]
[348, 9, 350, 1, 20, 1, 114, 439, 196, 63, 321]
제1조 ① 대한민국은
```

헌법과 민법을 합쳐 165,422글자, 2,454줄입니다. 책으로 치면 100쪽 남짓이에요. 쓰인 글자는 띄어쓰기와 줄바꿈, 숫자, ①② 같은 기호까지 478종류입니다. 1권 7장의 `Counter`로 세어 보니 띄어쓰기가 가장 많고, 그다음이 "의", "에", "한", "는", "제" 순이에요. "제"가 많은 건 "제1조", "제2항"처럼 조문 번호마다 붙기 때문입니다. 마지막 두 줄은 "제1조 ① 대한민국은"을 번호 11개로 바꿨다가 다시 글로 되돌린 것이에요.

다음 글자 맞히기 문제 만들기

GPT가 배우는 것은 단 하나, **"지금까지의 글자를 보고 다음 글자 맞히기"** 입니다. 정답을 따로 달 필요가 없어요. 글 자체가 문제이자 정답이니까요. `배치_뽑기`가 이 문제를 어떻게 만드는지 짧은 조각으로 보겠습니다.

```
import torch

from 법전 import 배치_뽑기, 준비

표, 학습_데이터, 검증_데이터 = 준비()
print(f"학습 {len(학습_데이터):,}글자, 검증 {len(검증_데이터):,}글자")

x, y = 배치_뽑기(학습_데이터, 문맥_길이=12, 배치_크기=2, 생성기=torch.Generator().manual_seed(3))
print("입력 모양:", tuple(x.shape), "정답 모양:", tuple(y.shape))
for i in range(2):
    print(f"입력 {i}: {표.디코딩(x[i])!r}")
    print(f"정답 {i}: {표.디코딩(y[i])!r}")

# 조각 하나에 '다음 글자 맞히기' 문제가 문맥_길이만큼 들어 있다
```

```
for t in range(5):
    print(f"  {표.디코딩(x[0, :t + 1])!r} 다음은 → {표.디코딩(y[0, t:t + 1])!r}")
```

```
학습 148,879글자, 검증 16,543글자
입력 모양: (2, 12) 정답 모양: (2, 12)
입력 0: '대법원과 각급법원으로 '
정답 0: '법원과 각급법원으로 조'
입력 1: '재로 주채무를 소멸하게'
정답 1: '로 주채무를 소멸하게 '
  '대' 다음은 → '법'
  '대법' 다음은 → '원'
  '대법원' 다음은 → '과'
  '대법원과' 다음은 → ' '
  '대법원과 ' 다음은 → '각'
```

준비()는 글 전체를 번호로 바꾼 뒤 앞 90%(148,879글자)는 학습용, 뒤 10%(16,543글자)는 검증용으로 나눴습니다. 2장에서 배운 대로 무작위로 섞지 않고 **앞뒤로 잘랐어요.** 글자 단위로 섞어 버리면 학습용 조각과 검증용 조각이 같은 조문에서 나와 사실상 답을 미리 보게 되거든요. 뒤쪽 10%는 민법의 상속 편이라, 검증 손실은 "한 번도 못 본 조문을 얼마나 잘 이어 쓰는가"를 잽니다.

배치_뽑기는 아무 자리에서나 문맥_길이만큼 잘라 입력 x로, **한 글자 뒤로 밀린** 조각을 정답 y로 만듭니다. "대법원과 각급법원으로"의 정답이 "법원과 각급법원으로 조"인 거죠. 입력의 t번째 글자 자리에서는 정답의 t번째 글자, 즉 **바로 다음 글자**를 맞혀야 합니다.

마지막 다섯 줄이 중요합니다. 조각 하나에는 "'대' 다음은 '법'", "'대법' 다음은 '원'"처럼 **문제가 문맥_길이만큼** 들어 있어요. 12.4절의 인과 마스크 덕분에 t번째 자리는 앞의 t개 글자만 볼 수 있으니, 이 문제들을 한 번의 계산으로 동시에 풀고 동시에 채점할 수 있습니다. 문맥 길이 128, 배치 32면 한 번의 학습 단계에서 다음 글자 맞히기 문제 4,096개를 푸는 셈이에요.

모델 모듈

이제 모델을 담은 작은GPT.py입니다. 이 장에서 만든 부품을 그대로 모았어요.

```
"""12장의 작은 문자 단위 GPT: 모델과 학습 함수."""
import math
from dataclasses import dataclass
```

```
import torch
from torch import nn
from torch.nn import functional as F

from 법전 import 데이터_폴더, 배치_뽑기, 준비, 글자표


@dataclass
class 설정:
    어휘_크기: int
    문맥_길이: int = 128       # 한 번에 보는 글자 수
    차원: int = 128            # 글자 하나를 나타내는 숫자 개수
    헤드_수: int = 4
    층_수: int = 4
    드롭아웃: float = 0.1
    위치_사용: bool = True     # 실습용 스위치: 위치 임베딩을 더할까
    인과_마스크: bool = True   # 실습용 스위치: 앞으로 올 글자를 가릴까


class 멀티헤드_어텐션(nn.Module):
    def __init__(self, s: 설정) -> None:
        super().__init__()
        self.헤드_수, self.인과_마스크 = s.헤드_수, s.인과_마스크
        self.qkv = nn.Linear(s.차원, 3 * s.차원)        # 쿼리·키·값을 한 번에
        self.출력 = nn.Linear(s.차원, s.차원)
        self.드롭아웃 = nn.Dropout(s.드롭아웃)
        self.가중치 = None                              # 들여다보기용으로 남겨 두는 어텐션 가중치

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(C, dim=2)
        # (B, T, C) → (B, 헤드, T, 헤드 차원)
        q, k, v = (t.view(B, T, self.헤드_수, -1).transpose(1, 2) for t in (q, k, v))
        점수 = q @ k.transpose(-2, -1) / math.sqrt(k.size(-1))     # (B, 헤드, T, T)
        if self.인과_마스크:
```

```
            미래 = torch.ones(T, T, dtype=torch.bool, device=x.device).triu(1)
            점수 = 점수.masked_fill(미래, float("-inf"))
        가중치 = 점수.softmax(dim=-1)
        self.가중치 = 가중치.detach()
        y = self.드롭아웃(가중치) @ v                          # (B, 헤드, T, 헤드 차원)
        y = y.transpose(1, 2).contiguous().view(B, T, C)       # 헤드를 다시 이어 붙이기
        return self.출력(y)


class 블록(nn.Module):
    """트랜스포머 블록: (층 정규화 → 어텐션) + 잔차, (층 정규화 → 피드포워드) + 잔차."""

    def __init__(self, s: 설정) -> None:
        super().__init__()
        self.정규화1 = nn.LayerNorm(s.차원)
        self.어텐션 = 멀티헤드_어텐션(s)
        self.정규화2 = nn.LayerNorm(s.차원)
        self.피드포워드 = nn.Sequential(
            nn.Linear(s.차원, 4 * s.차원), nn.GELU(),
            nn.Linear(4 * s.차원, s.차원), nn.Dropout(s.드롭아웃))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x + self.어텐션(self.정규화1(x))        # 글자끼리 정보 주고받기
        x = x + self.피드포워드(self.정규화2(x))    # 글자마다 혼자 생각하기
        return x


class 작은GPT(nn.Module):
    def __init__(self, s: 설정) -> None:
        super().__init__()
        self.s = s
        self.글자_임베딩 = nn.Embedding(s.어휘_크기, s.차원)
        self.위치_임베딩 = nn.Embedding(s.문맥_길이, s.차원)
        self.드롭아웃 = nn.Dropout(s.드롭아웃)
        self.블록들 = nn.Sequential(*[블록(s) for _ in range(s.층_수)])
        self.마지막_정규화 = nn.LayerNorm(s.차원)
```

```
        self.머리 = nn.Linear(s.차원, s.어휘_크기)       # 다음 글자의 점수(로짓)

    def forward(self, 번호들: torch.Tensor) -> torch.Tensor:
        B, T = 번호들.shape
        x = self.글자_임베딩(번호들)                              # (B, T, C)
        if self.s.위치_사용:
            x = x + self.위치_임베딩(torch.arange(T, device=번호들.device))
        x = self.블록들(self.드롭아웃(x))
        return self.머리(self.마지막_정규화(x))                  # (B, T, 어휘 크기)

    @torch.no_grad()
    def 생성(self, 번호들: torch.Tensor, 개수: int) -> torch.Tensor:
        """마지막 자리의 예측 확률대로 한 글자를 뽑아 붙이기를 개수만큼 되풀이한다."""
        for _ in range(개수):
            로짓 = self(번호들[:, -self.s.문맥_길이:])[:, -1, :]      # (B, 어휘 크기)
            다음 = torch.multinomial(로짓.softmax(dim=-1), num_samples=1)
            번호들 = torch.cat([번호들, 다음], dim=1)
        return 번호들


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")


@torch.no_grad()
def 손실_재기(모델: 작은GPT, 데이터: torch.Tensor, 장치: torch.device) -> float:
    """늘 같은 자리 640곳(20배치 × 32)에서 잰 평균 손실."""
    모델.eval()
    생성기 = torch.Generator().manual_seed(0)
    합 = 0.0
    for _ in range(20):
```

```
        x, y = 배치_뽑기(데이터, 모델.s.문맥_길이, 32, 생성기)
        합 += F.cross_entropy(모델(x.to(장치)).flatten(0, 1), y.to(장치).flatten()).item()
    모델.train()
    return 합 / 20


def 학습(s: 설정, 학습_데이터: torch.Tensor, 검증_데이터: torch.Tensor,
         단계_수: int, 장치: torch.device, 보고_간격: int = 500) -> 작은GPT:
    torch.manual_seed(0)
    모델 = 작은GPT(s).to(장치)
    옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=1e-3, weight_decay=0.1)
    스케줄러 = torch.optim.lr_scheduler.CosineAnnealingLR(옵티마이저, T_max=단계_수)
    생성기 = torch.Generator().manual_seed(0)
    for 단계 in range(1, 단계_수 + 1):
        x, y = 배치_뽑기(학습_데이터, s.문맥_길이, 32, 생성기)
        로짓 = 모델(x.to(장치))                                    # (32, 문맥_길이, 어휘 크기)
        손실 = F.cross_entropy(로짓.flatten(0, 1), y.to(장치).flatten())
        옵티마이저.zero_grad()
        손실.backward()
        nn.utils.clip_grad_norm_(모델.parameters(), 1.0)
        옵티마이저.step()
        스케줄러.step()
        if 단계 % 보고_간격 == 0:
            print(f"{단계:5d}단계  학습 손실 {손실_재기(모델, 학습_데이터, 장치):.3f}"
                  f"  검증 손실 {손실_재기(모델, 검증_데이터, 장치):.3f}")
    return 모델.cpu().eval()


def 학습된_모델(단계_수: int = 3000) -> tuple[작은GPT, 글자표]:
    """저장해 둔 모델이 있으면 불러오고, 없으면 학습해서 data/작은GPT.pt에 저장한다."""
    표, 학습_데이터, 검증_데이터 = 준비()
    s = 설정(어휘_크기=len(표))
    경로 = 데이터_폴더 / "작은GPT.pt"
    if 경로.exists():
        모델 = 작은GPT(s)
        모델.load_state_dict(torch.load(경로, weights_only=True))
```

```
    return 모델.eval(), 표
모델 = 학습(s, 학습_데이터, 검증_데이터, 단계_수, 장치_고르기())
torch.save(모델.state_dict(), 경로)
return 모델, 표
```

부품은 모두 앞에서 만든 것이고, 새로 보이는 것은 이렇습니다.

- **설정**: 모델 크기를 정하는 숫자들을 1권 8장의 `dataclass`로 묶었습니다. **위치_사용**과 **인과_마스크**는 12.10절 실습에서 부품을 하나씩 빼 볼 스위치예요.
- **작은GPT.forward**: 글자 번호 → 글자 임베딩 + 위치 임베딩 → 블록 4개 → 마지막 층 정규화 → **머리**(`nn.Linear`) 순서입니다. **머리**는 자리마다 "다음 글자가 478종류 중 무엇일지"의 점수(로짓)를 냅니다. 6장 6.5절의 분류 모델과 똑같이, 자리마다 478개 클래스 중 하나를 고르는 분류 문제예요.
- **학습**: 6장의 다섯 줄 학습 루프에 8장의 도구를 더했습니다. AdamW와 weight decay(8.2절), 코사인 스케줄러(8.7절), 그래디언트 클리핑(8.9절)이에요. 손실을 계산할 때 로짓 `(32, 128, 478)`을 `flatten(0, 1)`로 `(4096, 478)`, 정답 `(32, 128)`을 `(4096,)`로 펴서 `cross_entropy`에 넣습니다. 문제 4,096개짜리 분류 배치로 보는 거죠.
- **손실_재기**: 매번 같은 자리 640곳에서 손실을 재도록 생성기의 씨앗을 고정했습니다(4장 4.1절). 그래야 단계마다 잰 손실을 공정하게 비교할 수 있어요.
- **생성**과 **학습된_모델**: 12.9절에서 씁니다.

학습 전 점검

학습을 시작하기 전에 8장 8.11절의 디버깅 1단계부터 해 봅시다. 모델 크기와 첫 손실을 확인하는 거예요.

```
import math

import torch
from torch.nn import functional as F

from 법전 import 배치_뽑기, 준비
from 작은GPT import 설정, 작은GPT

표, 학습_데이터, _ = 준비()
```

```
s = 설정(어휘_크기=len(표))
print(s)
torch.manual_seed(0)
모델 = 작은GPT(s)
for 이름, 부품 in 모델.named_children():
    개수 = sum(p.numel() for p in 부품.parameters())
    if 개수:
        print(f"  {이름}: {개수:,}개")
print(f"모두 {sum(p.numel() for p in 모델.parameters()):,}개")

x, y = 배치_뽑기(학습_데이터, s.문맥_길이, 32, torch.Generator().manual_seed(0))
with torch.no_grad():
    로짓 = 모델(x)
print("입력", tuple(x.shape), "→ 로짓", tuple(로짓.shape))
print(f"학습 전 손실 {F.cross_entropy(로짓.flatten(0, 1), y.flatten()):.4f}, "
      f"찍기 수준 -ln(1/{len(표)}) = {math.log(len(표)):.4f}")
```

```
설정(어휘_크기=478, 문맥_길이=128, 차원=128, 헤드_수=4, 층_수=4, 드롭아웃=0.1, 위치_사용=True, 인과
↪  _마스크=True)
  글자_임베딩: 61,184개
  위치_임베딩: 16,384개
  블록들: 793,088개
  마지막_정규화: 256개
  머리: 61,662개
모두 932,574개
입력 (32, 128) → 로짓 (32, 128, 478)
학습 전 손실 6.3229, 찍기 수준 -ln(1/478) = 6.1696
```

파라미터는 모두 932,574개, 100만 개가 조금 안 됩니다. 4장 4.3절의 계산(칸 크기 × 개수)으로 float32에서 약 3.6MB예요. 70억 개짜리 모델이 26GB였던 것과 비교하면 정말 작은 모델이죠. 대부분은 블록 4개(793,088 = 198,272 × 4)이고, 글자 임베딩(478 × 128)과 머리(128 × 478 + 478)가 6만여 개씩, 위치 임베딩이 128 × 128 = 16,384개입니다.

로짓의 모양 `(32, 128, 478)`은 "조각 32개 × 자리 128개 × 글자 478종류의 점수"예요. 학습 전 손실은 6.3229로, 478종류를 고르게 찍을 때의 손실 -ln(1/478) = 6.1696과 거의 같습니다. 모델이 아직 아무것도 모르고, 엉뚱한 글자를 강하게 확신하지도 않는 건강한 출발이에요.

학습시키기

이제 3,000단계를 학습시킵니다. 이 블록만 계산량이 많아서 `장치_고르기()`로 GPU를 씁니다. 이 책의 실행 환경(Apple M4 Max, MPS)에서 몇 분이 걸렸고, CPU에서는 그보다 몇 배 더 걸려요. 결과 숫자는 MPS에서 나온 값이라 다른 장치에서는 조금 다를 수 있습니다.

```
import torch

from 법전 import 데이터_폴더, 준비
from 작은GPT import 설정, 장치_고르기, 학습

표, 학습_데이터, 검증_데이터 = 준비()
장치 = 장치_고르기()
print("장치:", 장치)
모델 = 학습(설정(어휘_크기=len(표)), 학습_데이터, 검증_데이터, 단계_수=3000, 장치=장치)
torch.save(모델.state_dict(), 데이터_폴더 / "작은GPT.pt")
print("저장:", 데이터_폴더 / "작은GPT.pt")
```

```
장치: mps
  500단계  학습 손실 1.929  검증 손실 2.341
 1000단계  학습 손실 1.471  검증 손실 2.079
 1500단계  학습 손실 1.243  검증 손실 2.029
 2000단계  학습 손실 1.113  검증 손실 2.045
 2500단계  학습 손실 1.047  검증 손실 2.036
 3000단계  학습 손실 1.036  검증 손실 2.039
저장: data/작은GPT.pt
```

학습 손실은 6.3에서 시작해 500단계에 1.929, 3,000단계에 1.036까지 내려갔습니다. 검증 손실도 2.341에서 2.029까지 내려갔지만, 1,500단계 뒤로는 2.03~2.05에서 더 내려가지 않아요. 학습 손실과 검증 손실의 차이가 1 가까이 벌어진 것은 8장 8.1절에서 본 **과적합**입니다. 3,000단계 동안 본 글자는 3,000 × 4,096, 약 1,200만 개로 학습 데이터 15만 글자를 80번 넘게 본 셈이니, 조문을 상당 부분 외워 버린 거예요. 데이터가 작을 때는 피할 수 없는 일이고, 8장의 조기 종료를 쓴다면 1,500단계 근처에서 멈추면 됩니다.

손실 2.03을 확률로 바꾸면 e^(-2.03) ≈ 0.13, 즉 한 번도 못 본 조문에서도 다음 글자의 정답에 평균적으로 13% 안팎의 확률을 준다는 뜻입니다(정확히는 확률의 기하 평균이에요). 478종류 중에서 고르는 문제라는 걸 생각하면 꽤 잘 맞히는 셈이죠. 진짜로 쓸 만한지는 글을 만들어 봐야 알 수

있습니다.

12.9 글 만들기와 들여다보기

한 글자씩 이어 쓰기

학습한 모델로 글을 만드는 방법은 단순합니다. 시작 글을 넣고, **마지막 자리의 예측**에서 다음 글자를 하나 뽑아 뒤에 붙이고, 늘어난 글을 다시 넣는 것을 되풀이해요. `작은GPT.생성`이 바로 이 일을 합니다. 다음 글자는 6장의 `argmax`처럼 가장 높은 것 하나만 고르지 않고, `torch.multinomial`로 **확률대로 뽑습니다.** 그래야 같은 시작에서도 다양한 글이 나와요. 글이 문맥 길이(128자)보다 길어지면 `번호들[:, -128:]`로 최근 128자만 봅니다.

```
import torch

from 법전 import 준비
from 작은GPT import 학습된_모델

모델, 표 = 학습된_모델()                    # data/작은GPT.pt를 불러온다 (없으면 학습부터)
torch.manual_seed(0)
for 시작 in ["대한민국은", "제850조(", "국민은"]:
    번호들 = 모델.생성(표.인코딩(시작).unsqueeze(0), 개수=100)
    print(표.디코딩(번호들[0]))
    print("-" * 20)

학습_글 = 표.디코딩(준비()[1])               # 학습용 앞 90%를 다시 글로
for 구절 in ["제83조제3호, 제851조 및 제925조의 규정에 위반하거나", "혼인신고의 방법"]:
    print(f"학습 데이터에 있나? {구절 in 학습_글}  ← {구절!r}")
```

```
대한민국은 평등을 관한다.
② 국가는 국가에 대한무를 받은 경계의 중요한 조국에서 조직을 받거나 직무범위 또는 정부위원은 법률로
↪ 정한다.
③ 대안은 국군사무총리의 자의 복리에 관한 특별을
--------------------
제850조(혼인신고의 방법, 성년후견인의 정이해관) 제83조제3호, 제851조 및 제925조의 규정에 위반하거나
↪ 부양을 할 수 있다. 이 경우 가정법원은 본인, 친족, 그 취소 및 청구, 제
```

```
--------------------
국민은 안전보장에 위보장된다.
제13조 ① 한정안은 주각부의 대법원·비속에 관하여 관한 조약되음에 관하여 법률로 정한다.
② 제1항 제2항의 규정은 미성년자의 직권으로 지정한다.
제41
--------------------
학습 데이터에 있나? False ← '제83조제3호, 제851조 및 제925조의 규정에 위반하거나'
학습 데이터에 있나? False ← '혼인신고의 방법'
```

`학습된_모델()`은 앞 블록이 저장해 둔 `data/작은GPT.pt`를 6장 6.10절의 `state_dict` 방식으로 불러옵니다(파일이 없으면 학습부터 해요). 결과를 읽어 보세요.

- "제850조("로 시작했더니 "혼인신고의 방법, ...)"처럼 **괄호 안에 조문 제목**을 짓고 닫는 괄호를 붙인 뒤 본문을 이어 씁니다. "제83조제3호, 제851조 및 제925조의 규정에 위반하거나"처럼 조문 번호를 인용하는 법률 문체도 흉내 내요.
- "대한민국은"과 "국민은"으로 시작한 글은 헌법 말투로 이어지고, 줄을 바꾼 뒤 "② ...", "제13조 ① ..."처럼 **항 번호와 조 번호의 형식**을 지킵니다. "법률로 정한다", "법률이 정하는 바에 의하여" 같은 자주 나오는 표현도 정확히 씁니다.
- 반면 뜻은 엉망입니다. "평등을 관한다", "국가에 대한무를 받은"처럼 없는 말도 만들어요. 글자 하나하나를 보고 다음 글자를 고를 뿐, 문장의 뜻을 이해하는 건 아니니까요.

11장의 속담 LSTM은 속담 117개(1,642글자)로 학습해서, 지어낸 속담 대부분이 원래 있던 속담 그대로였죠. 이번 모델이 지은 "제83조제3호, 제851조 및 제925조의 규정에 위반하거나"나 "혼인신고의 방법" 같은 구절은 학습 데이터 어디에도 그대로 있지 않습니다. 출력 마지막 두 줄처럼 `in`으로 찾아보면 `False`가 나와요. 데이터가 100배쯤 크고 앞의 128글자를 한꺼번에 볼 수 있으니, 외운 문장을 꺼내는 수준을 넘어 조문의 **형식**을 새로 조립하는 데까지 온 거예요.

100만 개도 안 되는 파라미터로 글자만 보고 배운 것치고는 놀라운 결과입니다. 이 모델과 ChatGPT의 차이는 원리가 아니라 **크기**예요. 파라미터가 수만 배, 학습 글이 수백만 배 많고, 글자 대신 13장에서 볼 토큰 단위로 읽는다는 점이 다를 뿐, "앞의 글을 보고 다음 조각을 확률대로 뽑아 붙인다"는 생성 방식은 똑같습니다. 확률을 그대로 쓸지, 날카롭게 하거나 뭉툭하게 할지(온도), 후보를 몇 개로 좁힐지 같은 뽑는 방법은 14장에서 자세히 다뤄요.

어텐션은 어디를 보고 있을까

12.3절에서 어텐션 가중치를 돌려받게 만들어 둔 이유가 여기 있습니다. 학습된 모델이 글자마다 어디를 보는지 들여다볼 수 있거든요. 학습 때 못 본 검증 데이터에서 128글자를 넣어 보겠습니다.

```
import matplotlib
import matplotlib.pyplot as plt
import torch

from 법전 import 준비
from 작은GPT import 학습된_모델

matplotlib.use("Agg")                              # 창을 띄우지 않고 파일로만 그리기
plt.rcParams["font.family"] = "AppleGothic"        # macOS 한글 글꼴

모델, 표 = 학습된_모델()
_, _, 검증_데이터 = 준비()
x = 검증_데이터[1000:1128].unsqueeze(0)       # 학습 때 못 본 128글자
글 = 표.디코딩(x[0])
with torch.no_grad():
    로짓 = 모델(x)

# 1) 첫 층의 헤드들은 평균 몇 칸 앞을 보고, 바로 앞 글자에 얼마를 줄까 (20번째 글자부터 평균)
거리 = (torch.arange(128)[:, None] - torch.arange(128)[None, :]).clamp(min=0).float()
가중치 = 모델.블록들[0].어텐션.가중치[0]                    # (헤드, 128, 128)
for h in range(4):
    평균_거리 = (가중치[h] * 거리).sum(dim=-1)[20:].mean()
    바로_앞 = 가중치[h].diagonal(-1)[20:].mean()
    print(f"1층 {h}번 헤드: 평균 {평균_거리:4.1f}칸 앞, 바로 앞 글자에 {바로_앞:.2f}")

# 2) "제1006" 다음 글자를 맞힐 때, 층마다 각 헤드가 가장 많이 본 글자
자리 = 글.index("제1006") + 4                                # '6'의 자리
값, 번호 = 로짓[0, 자리].softmax(dim=-1).topk(2)
print(repr(글[자리 - 30:자리 + 1]), "다음 글자 예측:",
      [(표.글자들[i], round(p, 3)) for p, i in zip(값.tolist(), 번호.tolist())])
for 층, 블 in enumerate(모델.블록들, start=1):
    본_곳 = []
    for h, 줄 in enumerate(블.어텐션.가중치[0, :, 자리]):
```

```
        j = 줄.argmax().item()
        본_곳.append(f"{글[j - 2:j]!r}[{글[j]!r}] {줄[j]:.2f}")
    print(f"{층}층:", " ¦ ".join(본_곳))

# 3) 1층 3번 헤드와 3층 2번 헤드의 가중치 표 일부(뒤쪽 44글자)를 그림으로
시작 = 84
라벨 = ["\\n" if c == "\n" else "_" if c == " " else c for c in 글[시작:]]  # 줄바꿈·띄어쓰기를 보
↪ 이게
그림, 축들 = plt.subplots(1, 2, figsize=(11, 5.6))
for 축, (층, h) in zip(축들, [(0, 3), (2, 2)]):
    축.imshow(모델.블록들[층].어텐션.가중치[0, h, 시작:, 시작:], cmap="Greys", vmin=0, vmax=1)
    축.set_title(f"{층 + 1}층 {h}번 헤드")
    축.set_xticks(range(len(라벨)), 라벨, fontsize=6)
    축.set_yticks(range(len(라벨)), 라벨, fontsize=6)
    축.set_xlabel("보는 글자 (키)")
    축.set_ylabel("지금 글자 (쿼리)")
그림.tight_layout()
그림.savefig("attention_heads.png", dpi=200)
```

```
1층 0번 헤드: 평균 34.9칸 앞, 바로 앞 글자에 0.02
1층 1번 헤드: 평균 30.1칸 앞, 바로 앞 글자에 0.05
1층 2번 헤드: 평균  8.5칸 앞, 바로 앞 글자에 0.74
1층 3번 헤드: 평균  9.4칸 앞, 바로 앞 글자에 0.71
'속인의 일신에 전속한 것은 그러하지 아니하다.\n제1006' 다음 글자 예측: [('조', 0.999), ('(', 0.0)]
1층: ' 승'['계'] 0.03 ¦ '에 '['전'] 0.06 ¦ '10'['0'] 0.80 ¦ '10'['0'] 0.96
2층: '다.'['\n'] 0.41 ¦ ' 아'['니'] 0.10 ¦ '10'['0'] 0.16 ¦ '다.'['\n'] 0.19
3층: '10'['0'] 0.36 ¦ '10'['0'] 0.17 ¦ '.\n'['제'] 0.98 ¦ '10'['0'] 0.29
4층: '10'['0'] 0.34 ¦ '다.'['\n'] 0.60 ¦ '제1'['0'] 0.51 ¦ '\n제'['1'] 0.33
```

첫 부분은 1층 헤드 넷의 습관을 잰 것입니다. 2번, 3번 헤드는 **바로 앞 글자**에 평균 0.74, 0.71을 줍니다. 아무도 그렇게 하라고 가르치지 않았는데, 다음 글자를 맞히는 데 바로 앞 글자가 가장 중요하다는 것을 스스로 배운 거예요. 반면 0번, 1번 헤드는 평균 30~35칸 앞, 넓은 범위를 두루 봅니다. 같은 층 안에서도 헤드마다 맡은 일이 나뉘었어요. 멀티헤드를 쓰는 이유를 모델이 직접 보여 준 셈입니다.

둘째 부분은 "…아니하다.\n제1006" 다음 글자를 맞히는 순간입니다. 모델은 "조"를 0.999로 확신

했어요. 숫자 뒤에는 "조"가 온다는 것을 안 거죠. 이때 층마다 각 헤드가 가장 많이 본 글자를 보면, 1층의 2번·3번 헤드는 바로 앞 글자 "0"을, 3층의 2번 헤드는 조문 번호가 시작된 **"제"** 를 0.98로 보고 있습니다. "지금 쓰는 것이 '제…조'의 숫자 부분"이라는 것을 확인하려고 "제"를 찾아간 것처럼 보여요. 2층과 4층의 헤드들은 줄바꿈 기호나 숫자 여러 개에 주의를 나눠 줍니다.

코드의 마지막 부분은 두 헤드의 가중치 표 가운데 뒤쪽 44글자를 `imshow`로 그려 `attention_heads.png`에 저장합니다. 그림으로 보면 차이가 더 분명해요(검은 칸일수록 많이 본 곳).

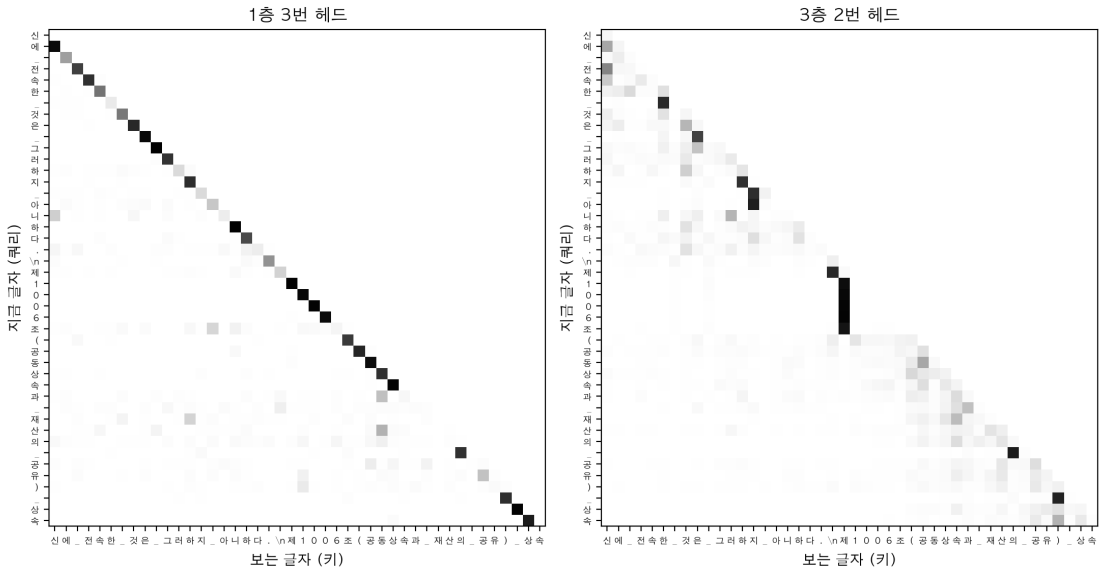


그림 12.2: 학습된 작은 GPT의 어텐션 가중치 일부. 왼쪽 1층 3번 헤드는 거의 모든 글자가 바로 앞 글자를 보는 대각선 무늬다. 오른쪽 3층 2번 헤드는 "제1006조"의 숫자들과 "조"가 모두 "제"를 보는 세로 줄 무늬를 보인다.

왼쪽 1층 3번 헤드는 대각선 바로 아래 칸이 진한 **"바로 앞 글자 보기"** 무늬이고, 오른쪽 3층 2번 헤드는 "제1006조"의 숫자와 "조"가 모두 "제" 하나를 보는 **세로 줄** 무늬입니다. 다만 이런 해석은 조심해야 해요. 가중치는 "어디서 정보를 가져왔는가"를 보여 줄 뿐, 모델이 그 정보로 무엇을 했는지까지 말해 주지는 않습니다. 그래도 수십만 개의 숫자 덩어리 속에서 이런 규칙이 저절로 생겨난다는 것은 충분히 신기한 일이죠.

문맥 길이의 대가

우리 모델은 한 번에 128글자만 봅니다. 요즘 LLM이 수만~수십만 토큰을 한 번에 읽는 것과 비교하면 아주 짧은데, 그냥 늘리면 안 될까요? 어텐션 가중치 표의 모양을 떠올려 보세요. `(헤드, T, T)`, 즉 글자 수의 **제곱**입니다. 헤드 32개짜리 층 하나의 가중치 표를 2바이트 칸(bfloat16)에 담는다고

하고 4장 4.3절의 계산(칸 크기 × 개수)을 해 보겠습니다.

```
# 어텐션 가중치 표 (배치 1, 헤드 32, T × T)를 bfloat16(2바이트)으로 담으면 몇 바이트일까
헤드_수, 칸_크기 = 32, 2
for T in [128, 1_024, 8_192, 131_072]:
    바이트 = 헤드_수 * T * T * 칸_크기
    print(f"문맥 {T:>7,}글자: 가중치 표 {바이트 / 1024 ** 3:10.3f} GB (층 하나에)")
```

```
문맥     128글자: 가중치 표      0.001 GB (층 하나에)
문맥   1,024글자: 가중치 표      0.062 GB (층 하나에)
문맥   8,192글자: 가중치 표      4.000 GB (층 하나에)
문맥 131,072글자: 가중치 표   1024.000 GB (층 하나에)
```

글자 수가 8배(1,024 → 8,192)가 되면 표는 64배(0.062GB → 4GB)가 됩니다. 131,072글자(128K)면 층 하나에 1,024GB, 1TB예요. 어떤 GPU에도 올라가지 않는 크기입니다. 그래서 긴 글을 다루는 모델은 12.5절의 `scaled_dot_product_attention`처럼 이 표를 통째로 만들지 않고 조각조각 계산하는 방법(FlashAttention)을 쓰고, 글을 만들 때는 앞에서 계산한 키와 값을 저장해 두고 다시 쓰는 요령(KV 캐시)도 씁니다. 계산량도 글자 수의 제곱으로 늘어나는 건 피할 수 없어서, 문맥 길이를 늘리는 일은 지금도 LLM 연구의 큰 숙제예요. GPU 메모리를 아끼는 방법은 16장과 17장에서 더 다룹니다.

12.10 실습: 부품을 하나씩 빼 보기

이 장에서 만든 부품이 정말 필요한지 확인해 보겠습니다. 기본 모델과 함께, 12.6절의 **위치 임베딩을 뺀 모델**, 12.4절의 **인과 마스크를 뺀 모델**을 똑같이 1,000단계씩 학습시키고, 손실과 생성 결과를 비교해요. `설정`의 스위치만 바꾸면 됩니다. 이번에도 MPS에서 돌린 결과이고, 세 모델을 합쳐 몇 분이 걸립니다.

```
import torch

from 법전 import 준비
from 작은GPT import 설정, 장치_고르기, 학습

표, 학습_데이터, 검증_데이터 = 준비()
장치 = 장치_고르기()
```

```
실험들 = {
    "기본": 설정(어휘_크기=len(표)),
    "위치 임베딩 없음": 설정(어휘_크기=len(표), 위치_사용=False),
    "인과 마스크 없음": 설정(어휘_크기=len(표), 인과_마스크=False),
}
for 이름, s in 실험들.items():
    print(f"[{이름}]")
    모델 = 학습(s, 학습_데이터, 검증_데이터, 단계_수=1000, 장치=장치, 보고_간격=1000)
    torch.manual_seed(0)
    글 = 표.디코딩(모델.생성(표.인코딩("제5조 ").unsqueeze(0), 개수=60)[0])
    print("  생성:", 글.replace("\n", " / "))
```

```
[기본]
 1000단계  학습 손실 1.824  검증 손실 2.244
  생성: 제5조 ① 기간내에 대하여 타인의 최고가 있어야 한다. / ⑤ 이사생활 기타 국무소) 총리인령의
↪  이나 ① 자총리자정이
[위치 임베딩 없음]
 1000단계  학습 손실 1.767  검증 손실 2.313
  생성: 제5조 ① 및 이를 주장래한 때에는 그 그 대통무를 받은 경과 생활에 때에 책임있다. / ② 법률행
↪  위를 정하여 그 얻어
[인과 마스크 없음]
 1000단계  학습 손실 0.043  검증 손실 0.056
  생성: 제5조 5조조55조조조555조조5조조5조조조조조조55조조조5조555555조조55조조조조조5555조5조55
↪  조조55조
```

먼저 **인과 마스크를 뺀 모델**부터 보세요. 학습 손실 0.043, 검증 손실 0.056으로 기본 모델(1.824, 2.244)보다 수십 배 좋은, 거의 완벽한 점수입니다. 그런데 만든 글은 "5조조55조조조555…"로 완전히 망가졌어요. 마스크가 없으니 학습할 때 t번째 자리가 **t + 1번째 글자, 즉 정답을 그대로 볼 수 있었던** 겁니다. 모델은 "다음 글자를 맞히는 법" 대신 "바로 오른쪽 글자를 베끼는 법"을 배웠어요. 그런데 글을 만들 때는 마지막 자리 오른쪽에 아무것도 없으니 베낄 게 없고, 엉터리 글자만 나옵니다. 검증 손실까지 낮은 것은 검증할 때도 똑같이 답을 볼 수 있었기 때문이에요.

이 실험은 꼭 기억해 둘 교훈을 줍니다. **손실이 믿기 어려울 만큼 좋으면 먼저 답이 새고 있지 않은지 의심하세요.** 2장의 데이터 누수가 데이터를 나눌 때의 실수였다면, 이것은 모델 구조 안에서 생기는 누수입니다. 직접 구현한 GPT에서 마스크를 빠뜨리거나 `tril`과 `triu`를 헷갈리는 일은 아주 흔하고, 오류 메시지도 없어요. 12.4절처럼 "뒤 글자를 바꿔도 앞 출력이 그대로인가"를 테스트로 확인

하는 습관이 가장 확실한 예방법입니다.

위치 임베딩을 뺀 모델은 예상과 조금 다릅니다. 학습 손실은 1.767로 기본 모델(1.824)보다 오히려 낮고, 검증 손실은 2.313으로 기본 모델(2.244)보다 높아요. 만든 글도 "제5조 ① 및 이를 주장래한 때에는 그 그 대통무를…"처럼 법률 문체 흉내는 냅니다. 12.3절에서 어텐션은 순서를 모른다고 했는데, 어떻게 된 걸까요?

비밀은 인과 마스크에 있습니다. 12.3절의 섞기 실험은 마스크가 **없는** 셀프 어텐션이었어요. 인과 마스크가 있으면 0번 자리는 1글자, 100번 자리는 101글자를 보고 평균을 냅니다. 몇 글자를 섞었는지에 따라 결과의 성질이 달라지니, 모델이 그 차이에서 "대략 몇 번째쯤인지"를 짐작할 수 있는 거예요. 게다가 바로 앞 글자를 찾는 것처럼 "가까운 글자"를 구분하는 일은 이 단서만으로는 어렵습니다. 검증 손실이 더 나쁜 것은 그 때문이라고 볼 수 있어요. 위치 정보가 없어도 완전히 망가지지 않지만, 있는 쪽이 한 번도 못 본 글에서 더 잘 맞힙니다. 그래서 거의 모든 언어 모델이 위치 정보를 따로 넣어요.

기본 모델의 1,000단계 결과(검증 2.244)를 12.8절의 3,000단계 결과(검증 2.029 ~ 2.045)와 비교해 보면, 더 오래 학습한 쪽이 낫다는 것도 보입니다. 직접 더 해 볼 만한 실험을 몇 가지 남겨 둘게요. 실행 시간이 늘어나니 단계 수를 조절해 가며 해 보세요.

- `헤드_수`를 1, 2, 8로 바꿔 보세요. 차원(128)은 헤드 수로 나누어떨어져야 합니다.
- `층_수`를 2나 6으로, `문맥_길이`를 32나 256으로 바꾸고 검증 손실과 생성 결과를 비교해 보세요.
- `드롭아웃`을 0.2로 올리면 학습 손실과 검증 손실의 차이(과적합)가 줄어드는지 확인해 보세요.
- `법전.py`의 `원문들`에 다른 법률을 더해 학습 데이터를 늘려 보세요. 위키문헌에서 문서 제목과 본문의 시작·끝 제목만 확인하면 됩니다.

12.11 정리와 다음 장 미리보기

이 장에서 트랜스포머를 부품 하나하나부터 직접 만들었습니다.

- **어텐션**: 쿼리와 키의 내적으로 닮은 정도를 재고, 소프트맥스로 비율을 만들어 값을 섞는다. 부드러운 딕셔너리. `softmax(Q @ K.T / sqrt(d)) @ V`
- **d의 제곱근으로 나누기**: 내적의 크기가 d의 제곱근배로 커져 소프트맥스가 쏠리고 기울기가 사라지는 것을 막는다
- **셀프 어텐션**: 같은 입력에서 `nn.Linear`로 쿼리·키·값을 만든다. 마스크가 없으면 순서를 모른

다

- **마스크**: 소프트맥스 **전에** 점수를 `-inf`로. 패딩 마스크는 키(열) 쪽, 인과 마스크는 `tril`로 앞만 보게. `scaled_dot_product_attention`과 `nn.MultiheadAttention`은 마스크의 `True` 뜻이 반대
- **멀티헤드**: `view(B, T, H, D).transpose(1, 2)`로 나누고 `transpose(1, 2).contiguous().view(B, T, C)`로 합친다. 파라미터 수는 그대로. 직접 구현은 내장 부품에 같은 가중치를 넣어 결과를 맞춰 검증한다
- **위치 정보**: 사인·코사인 위치 인코딩 또는 학습하는 위치 임베딩(11장 11.11절의 `nn.Embedding`에 자리 번호를 넣는다)을 글자 벡터에 더한다
- **블록**: `x + 어텐션(층정규화(x))`, `x + 피드포워드(층정규화(x))`. 층 정규화는 글자마다 표준화, 잔차 연결은 깊어도 기울기가 흐르게. 블록 하나에 약 $12C^2$개의 파라미터
- **인코더·디코더**: 인코더는 앞뒤를 다 보고(BERT), 디코더는 앞만 보며(GPT), 교차 어텐션은 쿼리와 키·값을 다른 문장에서 가져온다
- **작은 GPT**: 한 글자 밀린 조각이 정답인 다음 글자 맞히기. 첫 손실은 -ln(1/어휘 크기). 만들 때는 마지막 자리의 확률대로 한 글자씩 뽑아 붙인다. 문맥 길이의 대가는 제곱으로 커진다
- **실습의 교훈**: 손실이 너무 좋으면 누수를 의심한다. 인과 마스크가 빠지면 정답을 베낀다

우리 GPT는 글자 단위로 읽었습니다. 글자 478종류로 시작하니 간단하지만, "대한민국"을 네 글자로 쪼개 읽으니 128자 문맥에 담을 수 있는 내용이 적고, 모델이 글자를 모아 단어를 이해하는 데까지 힘을 써야 해요. 그렇다고 단어 단위로 자르면 "대한민국은", "대한민국의", "대한민국에서"가 모두 다른 단어가 되어 어휘가 끝없이 늘어나죠. 조사와 어미가 붙는 한국어에서는 특히 그렇습니다. 다음 13장에서는 글자와 단어 사이 어딘가에서 글을 자르는 **토크나이저**를 만나고, 한국어 문장을 미리 학습한 트랜스포머 인코더를 가져와 감성 분류 모델로 미세조정해 봅니다.

13

한국어 NLP 실전: 토크나이저부터 감성 분류까지

학습 목표

- 한국어 문장을 컴퓨터가 다루기 어려운 이유(교착어, 들쭉날쭉한 띄어쓰기, 유니코드)를 데이터로 확인하기
- 글자·어절·형태소·서브워드(BPE, WordPiece) 토큰화를 같은 문장으로 비교하고 장단점 설명하기
- Hugging Face 토크나이저의 출력(`input_ids`, `attention_mask`, 특수 토큰)을 읽고 패딩·자르기 다루기
- 임베딩이 번호를 화살표로 바꾸는 표라는 것을 이해하고, 코사인 유사도로 단어와 문장을 비교하기
- 처음부터 학습한 기준 모델과 사전학습 한국어 인코더(`klue/bert-base`)를 미세조정한 모델을 비교하기
- 혼동 행렬·F1과 틀린 예제 읽기로 모델을 평가하고, 모델·데이터의 라이선스를 확인하는 습관 들이기

들어가며: "아버지가방에들어가신다"를 어디서 끊을까요?

띄어쓰기 없이 적힌 이 문장을 소리 내어 읽어 보세요. 아마 "아버지가 방에 들어가신다"로 읽었을 거예요. 그런데 "아버지 가방에 들어가신다"로 읽을 수도 있죠. 사람은 아버지가 가방에 들어갈 리 없다

는 걸 알기 때문에 고민 없이 앞쪽을 고릅니다. 컴퓨터는 어떻게 끊을까요? 네 가지 방법으로 잘라 보겠습니다.

```
from kiwipiepy import Kiwi
from transformers import AutoTokenizer

문장 = "아버지가방에들어가신다"
print("띄어쓰기로:", 문장.split())
print("글자로    :", list(문장))
print("형태소로  :", [t.form for t in Kiwi().tokenize(문장, compatible_jamo=True)])
print("KLUE BERT :", AutoTokenizer.from_pretrained("klue/bert-base").tokenize(문장))
```

```
띄어쓰기로: ['아버지가방에들어가신다']
글자로    : ['아', '버', '지', '가', '방', '에', '들', '어', '가', '신', '다']
형태소로  : ['아버지', '가', '방', '에', '들어가', '시', 'ㄴ다']
KLUE BERT : ['아버지', '##가방', '##에', '##들', '##어가', '##신', '##다']
```

띄어쓰기로 자르면 문장 전체가 한 덩어리로 남고, 글자로 자르면 열한 조각이 됩니다. 셋째 줄은 한국어 **형태소 분석기**가 자른 결과로, "아버지 / 가 / 방 / 에 / 들어가 / 시 / ㄴ다"라고 사람처럼 바르게 끊었어요. 넷째 줄이 재미있습니다. 한국어 문장 수십 GB로 학습한 KLUE BERT의 토크나이저는 "아버지 / ##가방 / ##에"로, 그러니까 **가방**으로 끊었어요. 띄어쓰기가 없으니 자주 본 조각을 앞에서부터 길게 붙잡다가 이렇게 된 겁니다. 그래도 이 토크나이저로 학습한 모델은 한국어 문장을 꽤 잘 이해합니다. 조각을 어떻게 자르든 모델이 문맥으로 메워 주기 때문이에요.

7장 7.7절에서 리뷰 문장을 글자마다 번호로 바꿔 패딩해 봤죠. 그때 "실제 한국어 처리에서는 글자보다 큰 단위로 자르는 토크나이저를 쓴다"고 미뤄 둔 이야기를 이 장에서 합니다. 순서는 이렇습니다.

1. 영화 리뷰 15만 개로 한국어가 왜 다루기 어려운지 숫자로 확인합니다.
2. 글자, 어절, 형태소, 서브워드로 잘라 보며 "사전 크기와 문장 길이의 줄다리기"를 봅니다.
3. 잘라 낸 토큰 번호를 **임베딩**으로 화살표로 바꾸고, 4장 4.11절의 코사인 유사도로 뜻을 비교합니다.
4. 처음부터 학습한 작은 모델과, 사전학습 한국어 인코더를 **미세조정**한 모델을 겨뤄 봅니다.
5. 같은 방법으로 뉴스 제목의 주제를 맞히는 분류기를 실습으로 만듭니다.

Q. 이 장에 필요한 패키지는 무엇인가요? PyTorch 말고 다섯 가지를 더 씁니다. `uv add transformers datasets tokenizers kiwipiepy scikit-learn`을 실행하세요. 이 장의 예제는 파이썬 3.12, PyTorch 2.14.1, transformers 5.20.2, datasets 5.1.0, tokenizers 0.23.3, kiwipiepy 0.24.1, scikit-learn 1.9.1에서 실행해 확인했습니다. Hugging Face에서 내려받는 모델과 데이터는 처음 한 번만 받아서 사용자 폴더의 `.cache/huggingface`에 보관하는데, 위치를 바꾸고 싶으면 환경 변수 `HF_HOME`을 지정하면 됩니다. 이 장에서 쓰는 모델과 데이터는 모두 로그인 없이 받을 수 있어요.

13.1 이 장의 데이터: 네이버 영화 리뷰

받아서 읽는 도구 만들기

이 장의 예제 대부분은 NSMC(Naver Sentiment Movie Corpus)를 씁니다. 네이버 영화에 달린 리뷰 20만 개에 긍정(1)·부정(0) 라벨을 붙인 데이터로, 한국어 감성 분류 연습에 가장 널리 쓰여요. 원본 저장소(github.com/e9t/nsmc)에서 CC0 1.0(퍼블릭 도메인)으로 공개되어 있어서 자유롭게 쓸 수 있습니다. 파일은 탭으로 구분된 텍스트 두 개(학습 15만, 시험 5만)예요.

여러 예제가 같은 데이터를 읽으니, 8장의 `학습도구.py`처럼 받기와 읽기를 한 모듈에 모아 두겠습니다. 아래 내용을 `한국어도구.py`로 저장하세요.

```
"""13장 예제들이 함께 쓰는 NSMC(네이버 영화 리뷰) 받기·읽기 도구."""
import csv
import random
import urllib.request
from pathlib import Path

# 원본 저장소: github.com/e9t/nsmc (CC0 1.0, 퍼블릭 도메인)
주소 = "https://raw.githubusercontent.com/e9t/nsmc/master/"
폴더 = Path("data/nsmc")


def NSMC_받기() -> None:
    """학습·시험 파일이 없으면 내려받는다(합쳐서 약 19MB)."""
    폴더.mkdir(parents=True, exist_ok=True)
    for 파일 in ["ratings_train.txt", "ratings_test.txt"]:
```

```
        if not (폴더 / 파일).exists():
            urllib.request.urlretrieve(주소 + 파일, 폴더 / 파일)


def NSMC_읽기(파일: str, 개수: int | None = None, 씨앗: int = 0) -> list[tuple[str, int]]:
    """(문장, 라벨) 목록을 돌려준다. 빈 문장은 버리고, 개수를 주면 무작위로 그만큼 고른다."""
    NSMC_받기()
    with open(폴더 / 파일, encoding="utf-8", newline="") as f:
        줄들 = list(csv.reader(f, delimiter="\t"))[1:]       # 첫 줄은 열 이름
    자료 = [(문장.strip(), int(라벨)) for _, 문장, 라벨 in 줄들 if 문장.strip()]
    if 개수 is not None:
        자료 = random.Random(씨앗).sample(자료, 개수)
    return 자료
```

`NSMC_받기`는 파일이 없을 때만 1권 17장의 `urllib.request`로 내려받고, `NSMC_읽기`는 1권 12장 12.5절의 `csv` 모듈로 탭 구분 파일을 읽습니다. `split("\t")`로 직접 자르지 않고 `csv`를 쓴 이유는 큰따옴표 때문이에요. 이 파일은 큰따옴표가 든 리뷰를 `"..""비밀""..."`처럼 따옴표로 감싸고 안쪽 따옴표를 두 번 겹쳐 적어 두었는데, `csv`가 이 표기를 원래 문장으로 풀어 줍니다. 빈 리뷰는 버리고, `개수`를 주면 `random.Random(씨앗)`으로 매번 같은 리뷰들을 골라요. 4장에서 본 "나만 쓰는 난수 생성기"와 같은 생각입니다.

둘러보기

먼저 몇 개인지, 긍정과 부정이 얼마나 섞였는지, 리뷰가 얼마나 긴지 봅니다. 2장 2.1절의 "건강검진부터" 원칙 그대로예요.

```
from 한국어도구 import NSMC_읽기

학습 = NSMC_읽기("ratings_train.txt")
시험 = NSMC_읽기("ratings_test.txt")
print(f"학습 {len(학습):,}개, 시험 {len(시험):,}개")
print(f"긍정(1) 비율: 학습 {sum(y for _, y in 학습) / len(학습):.3f}, "
      f"시험 {sum(y for _, y in 시험) / len(시험):.3f}")

# 원문 대신 리뷰가 어떤 모습인지 세어 본다
문장들 = [문장 for 문장, _ in 학습]
```

```
for 이름, 조건 in [("띄어쓰기가 하나도 없음", lambda s: " " not in s),
                  ("'ㅋ'나 'ㅎ'가 들어 있음", lambda s: "ㅋ" in s or "ㅎ" in s),
                  ("마침표를 두 개 이상 이어 씀", lambda s: ".." in s),
                  ("영어 알파벳이 들어 있음", lambda s: any("a" <= c.lower() <= "z" for c in s))]:
    print(f"{이름}: {sum(map(조건, 문장들)) / len(문장들):.1%}")

길이 = sorted(len(문장) for 문장, _ in 학습)
print("글자 수: 중앙값", 길이[len(길이) // 2], "/ 90% 지점", 길이[int(len(길이) * 0.9)],
      "/ 가장 긴 것", 길이[-1])
```

```
학습 149,995개, 시험 49,997개
긍정(1) 비율: 학습 0.499, 시험 0.503
띄어쓰기가 하나도 없음: 8.3%
'ㅋ'나 'ㅎ'가 들어 있음: 8.1%
마침표를 두 개 이상 이어 씀: 28.3%
영어 알파벳이 들어 있음: 5.9%
글자 수: 중앙값 27 / 90% 지점 75 / 가장 긴 것 146
```

빈 리뷰를 빼고 학습 149,995개, 시험 49,997개입니다. 긍정 비율이 두 쪽 모두 0.5 근처라 2장 2.8절에서 걱정했던 불균형은 없어요. 리뷰는 짧은 편이라 글자 수 중앙값이 27, 열에 아홉은 75글자 이하이고, 가장 긴 것도 146글자예요. 한 줄 평이 대부분이라는 뜻입니다.

리뷰가 어떤 모습인지는 원문 대신 숫자로 보겠습니다. 리뷰는 네이버 사용자들이 쓴 글이라 이 책에는 원문을 싣지 않고, 본문에 나오는 리뷰 같은 문장은 모두 예제용으로 직접 지은 것이에요. 여러분 컴퓨터에서는 `print(학습[:5])`로 원문을 직접 읽어 보세요. 열에 하나 가까이(8.3%)는 띄어쓰기가 하나도 없고, 8.1%는 "ㅋㅋ", "ㅎㅎ"를 달고 있으며, 28.3%는 ".."처럼 마침표를 이어 씁니다. 원문을 읽어 보면 맞춤법이 틀리거나 줄임말이 섞인 리뷰도 흔해요. 교과서 같은 문장이 아니라 사람들이 실제로 쓰는 말이라는 뜻입니다. 그리고 읽다 보면 "추천한다"는 칭찬인데 라벨이 부정(0)인 것처럼 글과 라벨이 어긋난 리뷰도 눈에 띕니다. 사람이 단 라벨에도 **잡음**이 섞여 있다는 걸 기억해 두세요. 13.10절에서 다시 만납니다.

13.2 한국어가 어려운 이유

말끝이 끝없이 바뀝니다

영어는 “fun”이 문장 어디에 오든 대개 “fun”입니다. 한국어는 어떨까요? “재밌다, 재밌어요, 재밌네요, 재밌었다, 재밌고, 재밌는, 재밌게…” 뜻의 중심(재밌)에 조사와 어미가 끝없이 붙어 모양을 바꿉니다. 이렇게 말 뒤에 문법 조각을 붙여 쓰는 언어를 **교착어**라고 해요. 띄어쓰기로 자른 덩어리를 **어절**이라 부르는데, 어절을 단어처럼 다루면 어떤 일이 생기는지 세어 보겠습니다. 1권 7장 7.7절의 `Counter`를 씁니다.

```
from collections import Counter

from 한국어도구 import NSMC_읽기

학습 = NSMC_읽기("ratings_train.txt")
어절_수 = Counter(어절 for 문장, _ in 학습 for 어절 in 문장.split())
한번 = sum(1 for n in 어절_수.values() if n == 1)
print(f"서로 다른 어절: {len(어절_수):,}개")
print(f"딱 한 번 나온 어절: {한번:,}개 ({한번 / len(어절_수):.1%})")

재밌_계열 = sorted((a for a in 어절_수 if a.startswith("재밌")), key=lambda a: -어절_수[a])
print(f"'재밌'으로 시작하는 어절: {len(재밌_계열):,}가지")
print("  자주 나온 것:", ", ".join(f"{a}({어절_수[a]})" for a in 재밌_계열[:8]))

# 학습 데이터로 만든 어절 사전으로 시험 데이터를 읽으면?
시험_어절 = [a for 문장, _ in NSMC_읽기("ratings_test.txt") for a in 문장.split()]
모름 = sum(1 for a in 시험_어절 if a not in 어절_수)
print(f"시험 어절 중 사전에 없는 것: {모름 / len(시험_어절):.1%}")
```

```
서로 다른 어절: 357,862개
딱 한 번 나온 어절: 284,772개 (79.6%)
'재밌'으로 시작하는 어절: 1,778가지
  자주 나온 것: 재밌게(1343), 재밌다(375), 재밌는(374), 재밌고(339), 재밌어요(271), 재밌음(238), 재
↪ 밌다.(193), 재밌는데(182)
시험 어절 중 사전에 없는 것: 25.2%
```

리뷰 15만 개에 서로 다른 어절이 35만 개가 넘습니다. 그중 79.6%는 **딱 한 번**만 나왔어요. 한 번 본 말로는 그 말의 뜻을 배우기 어렵습니다. “재밌”으로 시작하는 어절만 1,778가지인데, “재밌다”와 “재밌다.”가 따로 세어진 것도 보이죠? 문장부호 하나만 붙어도 다른 어절이 됩니다.

마지막 줄이 가장 아픕니다. 학습 데이터의 어절을 모두 사전에 넣어도, 시험 리뷰의 어절 넷 중 하나(25.2%)는 사전에 없어요. 모델 입장에서는 시험 문장의 네 단어 중 하나가 처음 보는 외국어인 셈입니다. 이렇게 사전에 없는 말을 OOV(out-of-vocabulary) 라고 부르고, 한국어를 어절 단위로 다루면 이 문제가 특히 심해요.

띄어쓰기가 들쭉날쭉합니다

한국어 맞춤법은 띄어쓰기 규칙이 까다롭고, 보조 용언처럼 붙여 써도 되고 띄어 써도 되는 경우도 있습니다. 인터넷 글은 여기에 귀찮음까지 더해지죠.

```
from kiwipiepy import Kiwi

from 한국어도구 import NSMC_읽기

학습 = NSMC_읽기("ratings_train.txt")
for 변형들 in [["볼만한", "볼 만한"], ["재미없다", "재미 없다"], ["시간가는줄", "시간 가는 줄"]]:
    print(" / ".join(f"'{v}' {sum(v in 문장 for 문장, _ in 학습)}번" for v in 변형들))

붙여쓴 = [문장 for 문장, _ in 학습 if " " not in 문장 and len(문장) >= 15]
print(f"15글자 이상인데 띄어쓰기가 하나도 없는 리뷰: {len(붙여쓴):,}개")

kiwi = Kiwi()
for 문장 in ["어제동생이랑보러갔는데생각보다훨씬재밌었어요", "배우들연기는좋은데결말이너무허무해서
↪ 아쉬움"]:  # 예제용으로 지은 문장
    print(문장, "→", kiwi.space(문장))
```

```
'볼만한' 406번 / '볼 만한' 28번
'재미없다' 997번 / '재미 없다' 65번
'시간가는줄' 109번 / '시간 가는 줄' 27번
15글자 이상인데 띄어쓰기가 하나도 없는 리뷰: 2,336개
어제동생이랑보러갔는데생각보다훨씬재밌었어요 → 어 제 동생이랑 보러 갔는데 생각보다 훨씬 재밌었어요
배우들연기는좋은데결말이너무허무해서아쉬움 → 배우들 연기는 좋은 데 결말이 너무 허무해서 아쉬움
```

"볼만한"과 "볼 만한"은 둘 다 맞는 표기이고 뜻도 같지만, 띄어쓰기로 자르면 전혀 다른 토큰이 됩니다. "재미없다"와 "재미 없다", "시간가는줄"과 "시간 가는 줄"도 마찬가지예요. 띄어쓰기가 아예 없는 15글자 이상의 리뷰도 2,336개나 됩니다.

아래 두 줄은 예제용으로 지은 붙여 쓴 문장에 형태소 분석기 Kiwi의 `space()`로 띄어쓰기를 다시 넣어 본 결과입니다. 대부분 그럴듯하게 고쳐졌지만, "어제"를 "어 제"로, "좋은데"를 "좋은 데"로 잘못 띄운 곳도 보이죠. 띄어쓰기 교정도 완벽하지는 않아요. 이런 손질을 **전처리**로 먼저 하는 방법도 있지만, 뒤에서 볼 서브워드 토크나이저는 애초에 띄어쓰기에 덜 흔들리도록 만들어져 있습니다.

눈에는 같은데 다른 글자

마지막 어려움은 눈에 보이지 않습니다. 1권 17장에서 `unicodedata.normalize('NFD', ...)`로 한글 한 글자를 초성·중성·종성으로 분해해 봤죠. 문제는 **분해된 채로 저장된 글**이 실제로 돌아다닌다는 점입니다. 맥에서 만든 파일 이름이나 압축 파일, 일부 웹 페이지에서 가져온 글이 그래요.

```
import unicodedata

완성형 = "한글"
분해형 = unicodedata.normalize("NFD", 완성형)     # 초성·중성·종성으로 쪼갠 형태
print(완성형 == 분해형, len(완성형), len(분해형))
print("분해형이 '글'로 끝나나?", 분해형.endswith("글"))
print(unicodedata.normalize("NFC", 분해형) == 완성형)   # 다시 합치면 같아진다


def 정규화(문장: str) -> str:
    """처리하기 전에 언제나 완성형(NFC)으로 맞춘다."""
    return unicodedata.normalize("NFC", 문장)


print([정규화(s) == "한글" for s in [완성형, 분해형]])

# 자판으로 친 'ㅋ'(호환용 자모)과 NFKC로 바뀐 'ㅋ'(첫소리 자모)도 다른 문자다
print(hex(ord("ㅋ")), "→", hex(ord(unicodedata.normalize("NFKC", "ㅋ"))))
```

```
False 2 6
분해형이 '글'로 끝나나? False
True
[True, True]
0x314b → 0x110f
```

화면에는 둘 다 "한글"로 보이는데, 분해형은 글자 수가 6이고 `==`로 비교하면 `False`입니다. "글"로

끝나는지 물어도 `False`예요. 분해형의 마지막 글자는 "글"이 아니라 받침 "ㄹ"이니까요. 사전에서 단어를 찾을 때 이런 문장이 섞여 들어오면, 멀쩡한 단어가 OOV가 되는 원인을 한참 찾게 됩니다. 그래서 한국어 문장을 처리할 때는 맨 처음에 `정규화()`처럼 **NFC(완성형)로 맞추는 한 줄**을 넣어 두는 것이 좋아요.

마지막 줄은 자판으로 친 "ㅋ"(0x314B)과, NFKC라는 더 강한 정규화를 거친 "ㅋ"(0x110F)이 다른 문자라는 확인입니다. 13.6절의 XLM-R처럼 내부에서 NFKC 정규화를 하는 토크나이저는 "ㅋㅋ"를 이 첫소리 자모로 바꿔 돌려줘서, 글꼴에 따라 화면에 깨져 보이기도 해요.

13.3 토큰화 ①: 글자와 어절

사전 크기와 문장 길이의 줄다리기

문장을 모델에 넣으려면 먼저 조각(**토큰**)으로 자르고, 조각마다 번호를 붙여야 합니다. 이 일을 하는 도구가 **토크나이저(tokenizer)** 예요. 어떤 크기로 자를지가 첫 번째 결정입니다. 레고로 생각해 보세요. 1×1 블록만 있으면 무엇이든 만들 수 있지만 블록을 엄청 많이 끼워야 하고, 완성된 집 모양 블록만 있으면 한 번에 끝나지만 블록 종류가 끝도 없이 필요합니다.

리뷰 2만 개로 사전을 만들고 시험 리뷰 5천 개를 잘라서, 두 극단인 글자와 어절을 비교해 보겠습니다.

```
from collections.abc import Callable

from 한국어도구 import NSMC_읽기

학습 = [문장 for 문장, _ in NSMC_읽기("ratings_train.txt", 20000)]
시험 = [문장 for 문장, _ in NSMC_읽기("ratings_test.txt", 5000)]


def 비교(이름: str, 자르기: Callable[[str], list[str]]) -> None:
    """학습 문장으로 사전을 만들고, 시험 문장의 평균 길이와 모르는 토큰 비율을 잰다."""
    사전 = {토큰 for 문장 in 학습 for 토큰 in 자르기(문장)}
    토큰들 = [자르기(문장) for 문장 in 시험]
    전체 = sum(len(t) for t in 토큰들)
    모름 = sum(1 for t in 토큰들 for 토큰 in t if 토큰 not in 사전)
```

```
    print(f"{이름}: 사전 {len(사전):>6,}개, 문장당 {전체 / len(시험):5.1f}토큰, 모르는 토큰 {모름 /
    ↪ 전체:5.1%}")


비교("글자", lambda s: list(s.replace(" ", "")))
비교("어절", str.split)
```

```
글자: 사전  1,942개, 문장당  29.0토큰, 모르는 토큰  0.1%
어절: 사전 69,651개, 문장당   7.7토큰, 모르는 토큰 38.3%
```

- **글자**: 사전이 1,942개로 작고 모르는 토큰은 0.1%뿐입니다. 한글 음절은 11,172자나 되지만 리뷰 2만 개에 실제로 쓰인 글자는 기호까지 합쳐 2천 개가 안 돼요. 대신 문장 하나가 평균 29토큰으로 길어지고, "재"나 "밌" 같은 글자 하나에는 뜻이 거의 없어서 모델이 뜻을 조립하는 일을 모두 떠안습니다.
- **어절**: 문장은 7.7토큰으로 짧지만, 2만 문장만으로 사전이 7만 개 가까이 되고 시험 토큰의 38.3%를 모릅니다. 13.2절에서 15만 문장으로 만든 사전의 25.2%보다도 나쁘죠. 데이터가 적으면 OOV가 더 심해집니다.

이 줄다리기에서 좋은 자리를 찾는 방법이 두 가지 있습니다. 하나는 언어학 지식으로 뜻의 최소 단위를 찾는 **형태소 분석**이고, 다른 하나는 데이터에서 자주 붙어 다니는 글자를 찾아 묶는 **서브워드**예요.

13.4 토큰화 ②: 형태소 분석

뜻의 최소 단위로 자르기

형태소는 뜻을 가진 가장 작은 말의 단위입니다. "재밌었어요"는 "재밌(형용사) + 었(과거) + 어요(높임 끝맺음)"로 나뉘어요. 형태소 분석기는 문장을 형태소로 자르고 각 조각의 **품사 태그**를 붙여 줍니다. 한국어 분석기는 여러 가지가 있는데, 이 장에서는 설치가 `uv add kiwipiepy` 한 줄로 끝나고 다른 프로그램(자바 등)이 필요 없는 **Kiwi**(kiwipiepy, Apache 2.0)를 씁니다.

```
from kiwipiepy import Kiwi


kiwi = Kiwi()
```

```
for 문장 in ["이 영화 진짜 재밌었어요ㅋㅋ", "재미없지는 않았다", "스토리가 산으로 감", "존잼 노잼
↪ 꿀잼"]:
    결과 = kiwi.tokenize(문장, compatible_jamo=True)
    print(" ".join(f"{t.form}/{t.tag}" for t in 결과))

# 뜻을 가진 말(명사·동사·형용사·어근)만 남기기
뜻말 = ("NNG", "NNP", "VV", "VA", "XR")
print([t.form for t in kiwi.tokenize("배우들 연기가 너무 어색해서 몰입이 안 됨") if t.tag in 뜻말])
```

```
이/MM 영화/NNG 진짜/MAG 재밌/VA 었/EP 어요/EF ㅋㅋ/SW
재미없/VA 지/EC 는/JX 않/VX 었/EP 다/EF
스토리/NNG 가/JKS 산/NNG 으로/JKB 가/VV ㅁ/EF
존/NNP 잼/NNG 노/NNG 잼/NNG 꿀/NNG 잼/NNG
['배우', '연기', '어색', '몰입', '되']
```

`compatible_jamo=True`는 "ㄴ다"의 "ㄴ"처럼 홀로 쓰인 자모를 자판으로 치는 문자로 돌려 달라는 옵션이에요(13.2절의 그 문제입니다). 태그는 이렇게 읽습니다.

태그	뜻	예
`NNG`, `NNP`	일반 명사, 고유 명사	영화, 스토리
`VV`, `VA`	동사, 형용사	가, 재밌, 재미없
`XR`	어근(하다가 붙어 형용사가 되는 말)	어색
`EP`, `EF`, `EC`	선어말 어미(었), 끝맺는 어미(어요), 잇는 어미(지)	었, 어요, 지
`JKS`, `JKB`, `JX`	주격 조사, 부사격 조사, 보조사	가, 으로, 는
`SW`, `SF`	기호, 마침표·느낌표	ㅋㅋ, !

"재미없지는 않았다"를 "재미없 / 지 / 는 / 않 / 었 / 다"로 나눈 것을 보세요. 이중 부정의 뼈대가 그대로 드러납니다. "스토리가 산으로 감"의 "감"도 "가(가다) + ㅁ"으로 바르게 풀었어요.

하지만 분석기가 배운 적 없는 말에는 약합니다. "존잼 노잼 꿀잼"은 "존(고유 명사) / 잼"처럼 엉뚱하게 잘렸어요. 사전과 규칙으로 학습된 분석기는 새로 생긴 인터넷 말을 따라가기 어렵습니다. 마지

막 줄처럼 명사·동사·형용사·어근만 남기면 “배우 연기 어색 몰입 되”라는 핵심 단어만 뽑을 수 있어서, 검색 색인이나 키워드 분석에 많이 써요.

사전이 얼마나 줄어들까요?

13.3절과 같은 리뷰로 형태소 사전을 만들어 보겠습니다. 같은 형태라도 품사가 다르면(“가/JKS”와 “가/VV”) 다른 토큰으로 셉니다. 덤으로 긍정·부정 리뷰에서 유난히 많이 나오는 형용사도 찾아봅니다.

```
from collections import Counter

from kiwipiepy import Kiwi

from 한국어도구 import NSMC_읽기

kiwi = Kiwi()
학습 = NSMC_읽기("ratings_train.txt", 20000)
시험 = [문장 for 문장, _ in NSMC_읽기("ratings_test.txt", 5000)]


def 형태소(문장들: list[str]) -> list[list[str]]:
    """문장마다 '형태/품사' 목록. 문장 목록을 한꺼번에 넘기면 kiwi가 여러 개를 함께 처리한다."""
    return [[f"{t.form}/{t.tag}" for t in 결과]
            for 결과 in kiwi.tokenize(문장들, compatible_jamo=True)]


학습_형태소 = 형태소([문장 for 문장, _ in 학습])
사전 = {토큰 for 토큰들 in 학습_형태소 for 토큰 in 토큰들}
시험_형태소 = 형태소(시험)
전체 = sum(len(t) for t in 시험_형태소)
모름 = sum(1 for t in 시험_형태소 for 토큰 in t if 토큰 not in 사전)
print(f"형태소: 사전 {len(사전):>6,}개, 문장당 {전체 / len(시험):5.1f}토큰, 모르는 토큰 {모름 / 전
↪  체:5.1%}")

# 긍정·부정 리뷰에서 유난히 자주 나오는 형용사(VA)
세기 = {0: Counter(), 1: Counter()}
for (_, 라벨), 토큰들 in zip(학습, 학습_형태소):
```

```
    세기[라벨].update(t for t in 토큰들 if t.endswith("/VA"))
for 라벨, 이름 in [(1, "긍정"), (0, "부정")]:
    상대 = 세기[1 - 라벨]
    점수 = {w: n / (상대[w] + 1) for w, n in 세기[라벨].items() if n >= 30}
    print(이름, [w.split("/")[0] for w in sorted(점수, key=점수.get, reverse=True)[:6]])
```

```
형태소: 사전 18,793개, 문장당  20.0토큰, 모르는 토큰  2.6%
긍정 ['멋지', '깊', '재미있', '아프', '슬프', '멋있']
부정 ['재미없', '심하', '어설프', '어이없', '뻔하', '지루하']
```

사전이 18,793개로 어절(69,651개)의 4분의 1 정도로 줄었고, 모르는 토큰도 38.3%에서 2.6%로 크게 줄었습니다. 문장 길이는 20토큰으로 글자와 어절 사이에 놓였어요. `kiwi.tokenize`에 문장 목록을 통째로 넘기면 여러 문장을 함께 처리해서 빠릅니다.

긍정·부정 형용사는 "부정 리뷰보다 긍정 리뷰에 몇 배 더 나왔나"로 줄 세운 것입니다(30번 이상 나온 것만, 분모에 1을 더해 0으로 나누지 않게 했어요). 긍정 쪽에 "슬프", "아프"가 있는 게 재미있죠? 영화 리뷰에서 "슬프다"는 칭찬입니다. 감성은 단어 하나가 아니라 **분야와 문맥**에 달렸다는 뜻이고, 사람이 만든 감성 사전보다 데이터로 배우는 모델이 필요한 이유이기도 해요.

Q

Q. 형태소 분석기는 왜 이렇게 종류가 많고, 어떤 걸 골라야 하나요? 한국어 분석기는 Kiwi 말고도 Mecab-ko, Okt, Komoran 등이 있고, 품사 태그 체계와 속도, 신조어 처리, 설치 난이도가 조금씩 다릅니다. 정답은 없어서 내 데이터에서 몇십 문장을 직접 잘라 보고 고르는 게 가장 확실해요. 다만 이 장의 뒤쪽처럼 **사전학습 모델을 쓸 때는 분석기를 고를 필요가 없습니다.** 모델과 짝을 이루는 토크나이저가 정해져 있기 때문이에요(13.6절).

13.5 토큰화 ③: 서브워드

자주 붙어 다니는 글자를 묶기

형태소 분석기는 사람이 만든 언어 지식에 기대므로 새 말에 약하고, 언어마다 따로 만들어야 합니다. 지식 없이 **데이터만 보고** 좋은 조각을 찾을 수는 없을까요? 아이디어는 단순합니다. 글자에서 시작해서, 데이터에서 **가장 자주 붙어 나오는 두 조각을 하나로 합치는 일**을 정해진 횟수만큼 되풀이하는 거예요. 이 방법을 BPE(Byte Pair Encoding) 라고 합니다. 예제용 어절 여섯 가지로 직접 해 보

겠습니다.

```
from collections import Counter

# 예제용 어절과 등장 횟수. 처음에는 모든 어절을 글자 단위로 쪼개 둔다
어절_수 = {"재밌다": 5, "재밌어요": 6, "재밌네요": 3, "재미없다": 4, "재미없어요": 3, "좋네요": 4}
쪼갬 = {어절: list(어절) for 어절 in 어절_수}


def 짝_세기() -> Counter:
    """붙어 있는 두 조각의 쌍이 전체에서 몇 번 나오는지 센다(어절의 등장 횟수만큼)."""
    세기 = Counter()
    for 어절, 조각 in 쪼갬.items():
        for 짝 in zip(조각, 조각[1:]):
            세기[짝] += 어절_수[어절]
    return 세기


def 합치기(짝: tuple[str, str]) -> None:
    """모든 어절에서 그 쌍을 한 조각으로 붙인다."""
    for 어절, 조각 in 쪼갬.items():
        새 = []
        i = 0
        while i < len(조각):
            if i + 1 < len(조각) and (조각[i], 조각[i + 1]) == 짝:
                새.append(조각[i] + 조각[i + 1])
                i += 2
            else:
                새.append(조각[i])
                i += 1
        쪼갬[어절] = 새


for 단계 in range(1, 7):
    짝, 횟수 = 짝_세기().most_common(1)[0]
    합치기(짝)
    print(f"{단계}단계: {짝[0]} + {짝[1]} ({횟수}번)")
```

```
for 어절, 조각 in 쪼갬.items():
    print(f"{어절} → {조각}")
```

```
1단계: 재 + 밌 (14번)
2단계: 어 + 요 (9번)
3단계: 네 + 요 (7번)
4단계: 재 + 미 (7번)
5단계: 재미 + 없 (7번)
6단계: 재밌 + 어요 (6번)
재밌다 → ['재밌', '다']
재밌어요 → ['재밌어요']
재밌네요 → ['재밌', '네요']
재미없다 → ['재미없', '다']
재미없어요 → ['재미없', '어요']
좋네요 → ['좋', '네요']
```

1단계에서 "재"와 "밌"이 14번 붙어 나와(재밌다 5 + 재밌어요 6 + 재밌네요 3) 가장 먼저 합쳐졌습니다. 이어서 "어요", "네요", "재미", "재미없"이 차례로 생겼어요. 여섯 번 합친 결과를 보면 "재밌 / 다", "재미없 / 어요", "좋 / 네요"처럼 형태소 분석과 꽤 비슷하게 잘렸습니다. 문법을 하나도 가르치지 않았는데, 자주 나오는 뼈대와 어미가 저절로 조각이 된 거예요. 가장 많이 나온 "재밌어요"는 아예 한 조각이 됐습니다. 자주 쓰는 말은 통째로, 드문 말은 잘게 자르는 것이 서브워드의 핵심이에요.

새 문장이 들어오면 배운 합치기 규칙을 **같은 순서대로** 적용해서 자릅니다. 처음 보는 어절도 글자까지는 쪼갤 수 있으니, 글자 사전에 있는 한 OOV가 생기지 않아요.

tokenizers로 진짜 BPE 학습하기

손으로 짠 코드는 원리를 보여 주기에는 좋지만 느립니다. Hugging Face의 `tokenizers` 라이브러리는 같은 일을 러스트로 빠르게 해 줘요. 리뷰 15만 개로 사전 크기를 바꿔 가며 학습해 보겠습니다.

```
from tokenizers import Tokenizer, models, pre_tokenizers, trainers

from 한국어도구 import NSMC_읽기

학습 = [문장 for 문장, _ in NSMC_읽기("ratings_train.txt")]
```

```
시험 = [문장 for 문장, _ in NSMC_읽기("ratings_test.txt", 5000)]
예문 = "시간 가는 줄 모르고 봤네요 끝까지 재밌었어요"

for 크기 in [500, 4000, 32000]:
    토크나이저 = Tokenizer(models.BPE(unk_token="[UNK]"))
    토크나이저.pre_tokenizer = pre_tokenizers.Whitespace()       # 먼저 공백·문장부호로 자른다
    훈련 = trainers.BpeTrainer(vocab_size=크기, special_tokens=["[PAD]", "[UNK]"],
                               show_progress=False)
    토크나이저.train_from_iterator(학습, 훈련)
    평균 = sum(len(토크나이저.encode(s).ids) for s in 시험) / len(시험)
    print(f"사전 {토크나이저.get_vocab_size():>6,}개: 문장당 {평균:4.1f}토큰 ¦ "
          + " ".join(토크나이저.encode(예문).tokens))

토크나이저.save("data/nsmc_bpe.json")            # 마지막(32,000개) 토크나이저를 파일로 저장
print(Tokenizer.from_file("data/nsmc_bpe.json").encode("재미없어요").tokens)
```

```
사전  3,005개: 문장당 29.0토큰 ¦ 시 간 가 는 줄 모 르 고 봤 네 요 끝 까 지 재 밌 었 어 요
사전  4,000개: 문장당 19.7토큰 ¦ 시간 가는 줄 모르 고 봤 네요 끝까지 재밌 었어요
사전 32,000개: 문장당 13.1토큰 ¦ 시간 가는 줄 모르고 봤네요 끝까지 재밌었어요
['재미없어요']
```

15만 문장으로 BPE를 학습하는 데 몇 초면 충분합니다. 결과에서 세 가지를 보세요.

- **사전 크기 500을 요청했는데 3,005개**가 됐습니다. BPE는 처음 출발점인 글자들을 모두 사전에 넣는데, 리뷰에 쓰인 글자 종류만 이미 3천 개가 넘거든요. 그래서 합치기를 한 번도 하지 못하고 글자 토크나이저와 같은 29.0토큰이 나왔습니다. 한국어는 글자 종류가 많아서 영어보다 사전이 커야 한다는 뜻이에요.
- **4,000개**에서는 "모르 / 고", "재밌 / 었어요"처럼 형태소에 가깝게 잘리고 문장당 19.7토큰입니다. 13.4절의 형태소(20.0토큰)와 거의 같은 길이를 분석기 없이 얻었어요.
- **32,000개**로 키우면 "모르고", "봤네요", "재밌었어요"가 통째로 한 토큰이 되고 문장이 13.1토큰으로 짧아집니다. 사전이 클수록 문장은 짧아지고, 각 토큰이 드물게 나와 배우기는 어려워져요. 13.3절의 줄다리기가 여기서도 그대로입니다.

`pre_tokenizers.Whitespace()`는 BPE 전에 공백과 문장부호에서 먼저 자르라는 설정입니다. 그래서 조각이 어절 경계를 넘어 합쳐지지 않아요. 학습한 토크나이저는 `save`로 JSON 파일 하나에 저장하고 `Tokenizer.from_file`로 불러옵니다.

WordPiece와 SentencePiece

실제 사전학습 모델의 토크나이저는 BPE의 친척을 씁니다. 이름이 자주 나오니 구분해 두세요.

방식	합칠 짝 고르는 기준	표시	쓰는 모델 예
BPE	가장 자주 붙어 나오는 짝	따로 없음	GPT 계열
WordPiece	합쳤을 때 데이터의 확률을 가장 많이 올리는 짝	어절 중간 조각 앞에 `##`	BERT, KLUE BERT
SentencePiece(Unigram)	큰 사전에서 시작해 덜 중요한 조각을 지워 나감	어절 첫 조각 앞에 `▁`	XLM-R, T5

들어가며의 "아버지 / ##가방 / ##에"에서 `##`는 "앞 조각에 붙어 있던 조각"이라는 WordPiece의 표시였습니다. 이 표시 덕분에 토큰을 다시 문장으로 붙일 때 어디를 띄어 쓸지 알 수 있어요.

13.6 사전학습 토크나이저 쓰기

세 모델의 토크나이저 비교

사전학습 모델은 학습할 때 쓴 토크나이저와 한 몸입니다. 모델이 "번호 7478은 '재밌'"이라고 배웠으니, 다른 토크나이저로 자른 번호를 넣으면 엉뚱한 단어를 읽게 돼요. 그래서 모델을 고르는 순간 토크나이저도 정해집니다. Hugging Face의 `AutoTokenizer.from_pretrained(모델 이름)`은 그 짝을 찾아서 내려받아 줘요.

세 가지를 비교해 보겠습니다. 한국어 전용인 **KLUE BERT**(`klue/bert-base`), 104개 언어를 한 사전에 담은 **다국어 BERT**(`google-bert/bert-base-multilingual-cased`), 100개 언어로 학습한 **XLM-R**(`FacebookAI/xlm-roberta-base`)입니다. 라이선스는 각각 CC BY-SA 4.0, Apache 2.0, MIT예요(13.9절에서 확인하는 방법을 봅니다).

```
from transformers import AutoTokenizer

from 한국어도구 import NSMC_읽기
```

```
시험 = [문장 for 문장, _ in NSMC_읽기("ratings_test.txt", 1000)]
예문 = "이 영화 진짜 재밌었어요 배우들 연기도 최고!"

for 이름 in ["klue/bert-base", "google-bert/bert-base-multilingual-cased", "FacebookAI/
↪ xlm-roberta-base"]:
    토크나이저 = AutoTokenizer.from_pretrained(이름)        # 처음 한 번만 내려받고, 다음부터는 캐
↪ 시에서
    번호들 = [토크나이저(s, add_special_tokens=False)["input_ids"] for s in 시험]
    전체 = sum(len(b) for b in 번호들)
    모름 = sum(1 for b in 번호들 for i in b if i == 토크나이저.unk_token_id)
    print(f"{이름} (사전 {토크나이저.vocab_size:,}개)")
    print(f"  문장당 {전체 / len(시험):.1f}토큰, [UNK] {모름 / 전체:.2%}")
    print(" ", 토크나이저.tokenize(예문))
```

```
klue/bert-base (사전 32,000개)
  문장당 20.3토큰, [UNK] 0.32%
  ['이', '영화', '진짜', '재밌', '##었', '##어요', '배우', '##들', '연기', '##도', '최고', '!']
google-bert/bert-base-multilingual-cased (사전 119,547개)
  문장당 24.1토큰, [UNK] 1.77%
  ['이', '영화', '진', '##짜', '[UNK]', '배우', '##들', '연', '##기도', '최고', '!']
FacebookAI/xlm-roberta-base (사전 250,002개)
  문장당 21.9토큰, [UNK] 0.11%
  ['▁이', '▁영화', '▁진짜', '▁재', '밌', '었어요', '▁배우', '들', '▁연기', '도', '▁최고', '!']
```

사전 크기만 보면 다국어 BERT(약 12만)와 XLM-R(약 25만)이 KLUE BERT(3만 2천)보다 훨씬 큽니다. 그런데 한국어 리뷰를 자르면 KLUE BERT가 문장당 20.3토큰으로 가장 짧아요. 큰 사전을 100여 개 언어가 나눠 쓰니 한국어 몫은 오히려 작은 거죠. 같은 내용을 더 적은 토큰으로 담으면 모델이 한 번에 더 긴 글을 읽을 수 있고 계산도 줄어듭니다.

다국어 BERT의 결과를 보세요. "재밌었어요"가 통째로 `[UNK]`(모르는 토큰)가 됐습니다. 사전에 "밌"이라는 글자가 없어서, WordPiece가 이 어절을 끝까지 쪼개지 못하고 **어절 전체**를 모르는 토큰으로 바꿔 버린 거예요. 감성 분류에서 가장 중요한 단어가 사라졌습니다. 리뷰 전체로 보면 토큰의 1.77%가 `[UNK]`예요. XLM-R은 "▁재 / 밌 / 었어요"로 잘게 쪼개서라도 살려 냈고, KLUE BERT는 "재밌 / ##었 / ##어요"로 형태소와 거의 같게 잘랐습니다. 한국어를 다룬다면 한국어 데이터로 만든 토크나이저를 먼저 고려하세요.

토크나이저가 돌려주는 것

이제 KLUE BERT 토크나이저 하나만 깊이 보겠습니다. 토크나이저를 함수처럼 부르면 무엇이 나올까요?

```
from transformers import AutoTokenizer

토크나이저 = AutoTokenizer.from_pretrained("klue/bert-base")

인코딩 = 토크나이저("재밌었어요")
print(인코딩)
print(토크나이저.convert_ids_to_tokens(인코딩["input_ids"]))
print(토크나이저.decode(인코딩["input_ids"]))
print(토크나이저.decode(인코딩["input_ids"], skip_special_tokens=True))

# 여러 문장을 한꺼번에: 패딩·자르기·텐서로 바꾸기까지
배치 = 토크나이저(["최고", "배우들 연기가 너무 어색해서 몰입이 안 됨", "두 시간이 순식간에 지나갔다"],
                padding=True, truncation=True, max_length=10, return_tensors="pt")
print(배치["input_ids"])
print(배치["attention_mask"])
print(토크나이저.convert_ids_to_tokens(배치["input_ids"][1]))
```

```
{'input_ids': [2, 7478, 2359, 10283, 3], 'token_type_ids': [0, 0, 0, 0, 0], 'attention_mask': [1, 1, 1, 1, 1]}
['[CLS]', '재밌', '##었', '##어요', '[SEP]']
[CLS] 재밌었어요 [SEP]
재밌었어요
tensor([[    2,  3841,     3,     0,     0,     0,     0,     0,     0,     0],
        [    2,  4165,  2031,  4483,  2116,  3760,  9977,  6396, 11306,     3],
        [    2,   864,  3641,  2052,  9390,  2170, 12613,  2062,     3,     0]])
tensor([[1, 1, 1, 0, 0, 0, 0, 0, 0, 0],
        [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1, 1, 0]])
['[CLS]', '배우', '##들', '연기', '##가', '너무', '어색', '##해서', '몰입', '[SEP]']
```

한 문장을 넣으면 딕셔너리 같은 결과에 세 가지가 들어 있습니다.

- `input_ids`: 토큰 번호입니다. `[2, 7478, 2359, 10283, 3]`을 토큰으로 돌려 보면 "재밌 / ## 었 / ##어요" 앞뒤에 `[CLS]`(2번)와 `[SEP]`(3번)가 붙어 있어요. 이런 **특수 토큰**은 모델이 학습할 때부터 쓰던 약속입니다. `[CLS]`는 문장 맨 앞에서 "문장 전체를 대표하는 자리" 역할을 하고, 13.9절의 분류 모델이 바로 이 자리의 출력을 보고 판단해요. `[SEP]`은 문장의 끝(두 문장을 넣을 때는 경계)을 표시합니다.
- `token_type_ids`: 문장 두 개를 한꺼번에 넣을 때 첫 문장은 0, 둘째 문장은 1로 구분하는 표시예요. 한 문장이면 모두 0입니다.
- `attention_mask`: 진짜 토큰은 1, 패딩은 0입니다.

`decode`는 번호를 다시 문장으로 돌립니다. `##` 표시를 보고 조각을 이어 붙여서 "재밌었어요"가 정확히 복원됐어요.

여러 문장을 넣는 아래쪽이 7장 7.7절의 `collate_fn`이 하던 일입니다. `padding=True`는 가장 긴 문장에 맞춰 0번(`[PAD]`)으로 채우고, `truncation=True`와 `max_length=10`은 10토큰을 넘으면 자르고, `return_tensors="pt"`는 PyTorch 텐서로 돌려줍니다. `attention_mask`는 7.7절에서 `(배치 != 0)`으로 직접 만들었던 그 마스크예요. 둘째 문장은 10토큰에서 잘려서 "어색 / ##해서 / 몰입 / [SEP]"으로 끝났습니다. 자를 때도 `[SEP]` 자리는 남겨 둔다는 점을 보세요. 몇 토큰에서 자를지는 13.9절에서 데이터를 보고 정합니다.

13.7 임베딩: 번호를 화살표로

번호표로 꺼내는 표

토큰 번호 7478은 그냥 이름표입니다. 7478이 7477보다 크다고 더 재밌다는 뜻은 아니죠. 2장 2.5절에서 "서울=1, 부산=2"로 하면 안 되는 이유와 같습니다. 그때의 해결책은 원-핫이었는데, 사전이 3만 2천 개면 토큰 하나가 3만 2천 칸짜리 원-핫이 돼요. 너무 크고, 모든 토큰 사이의 거리가 똑같아서 "재밌"과 "흥미"가 비슷하다는 걸 담을 수도 없습니다.

그래서 토큰마다 작은 화살표(숫자 몇십~몇백 개)를 하나씩 주는 표, **임베딩(embedding)** 을 씁니다. 11장 11.11절에서 처음 본 `nn.Embedding`이 바로 이 표예요. 원-핫 벡터에 표를 곱한 것(편향 없는 `nn.Linear`에 원-핫을 넣은 것)과 결과가 같은데, 곱셈 대신 그 번호의 줄을 꺼내기만 해서 훨씬 빠릅니다. 12장 12.6절의 작은 GPT도 이 표로 글자와 위치를 화살표로 바꿨죠. 처음에는 무작위 숫자로 시작해서 학습하면서 고쳐집니다. 이 장에서 하나만 덧붙이면, 13.8절처럼 패딩(0번)을 쓰는 모델은 `nn.Embedding(사전 크기, 칸 수, padding_idx=0)`으로 만들어요. 그러면 0번 줄은 0으로 채워지고 학습해도 바뀌지 않아서, 채워 넣은 칸이 아무 뜻도 갖지 않습니다.

그럼 학습이 끝난 표의 숫자들은 어떤 뜻을 갖게 될까요? 12장에서 "13장에서 자세히 본다"고 미뤄 둔 이야기입니다.

KLUE BERT의 임베딩 표 들여다보기

사전학습 모델의 임베딩 표에는 수십 GB의 문장을 읽으며 배운 것이 담겨 있습니다. 4장 4.11절에서 도시의 기온 패턴을 화살표로 보고 코사인 유사도로 닮은 도시를 찾았죠? 같은 계산으로 닮은 토큰을 찾아보겠습니다.

```
import torch
from transformers import AutoModel, AutoTokenizer

토크나이저 = AutoTokenizer.from_pretrained("klue/bert-base")
모델 = AutoModel.from_pretrained("klue/bert-base")
표 = 모델.get_input_embeddings().weight.detach()      # (사전 크기, 768): 토큰마다 화살표 하나
print(tuple(표.shape), f"{표.element_size() * 표.numel() / 1024**2:.1f} MB")

단위 = 표 / 표.norm(dim=1, keepdim=True)              # 4장 4.11절처럼 길이를 1로 맞추고
for 단어 in ["영화", "최고", "지루", "배우", "서울"]:
    번호 = 토크나이저.convert_tokens_to_ids(단어)
    유사도 = 단위 @ 단위[번호]                          # 모든 토큰과의 코사인 유사도를 한 번에
    상위 = 유사도.topk(6).indices[1:]                   # 1등은 자기 자신이라 뺀다
    print(단어, "→", 토크나이저.convert_ids_to_tokens(상위.tolist()))
```

```
(32000, 768) 93.8 MB
영화 → ['드라마', '##영화', '소설', '연극', '영상']
최고 → ['최대', '최상', '가장', '최악', '최저']
지루 → ['따분', '식상', '지겹', '밋밋', '재미없']
배우 → ['##배우', '배울', '배웠', '배운다', '연기자']
서울 → ['부산', '인천', '서울시', '한국', '대구']
```

표는 32,000 × 768로, `float32`라면 4장 4.3절의 `element_size() × numel()` 계산으로 93.8MB입니다. 모델 전체 크기의 5분의 1쯤이 이 표예요.

이웃을 보면 "영화"는 드라마·소설·연극, "지루"는 따분·식상·지겹·밋밋, "서울"은 부산·인천·대구와 가깝습니다. 사람이 하나도 알려 주지 않았는데 비슷한 자리에 쓰이는 말끼리 모였어요. 그런데 두 가지 함정도 보입니다.

- “최고”의 이웃에 **“최악”** 이 있습니다. “최고의 연기”, “최악의 연기”처럼 같은 자리에 쓰이니 화살표가 가까워진 거예요. 임베딩이 가깝다는 건 “쓰이는 자리가 비슷하다”는 뜻이지 “뜻이 같다”는 뜻이 아닙니다. 감성 분류에는 치명적이죠. 이 차이는 미세조정(13.9절)으로 배워야 합니다.
- “배우”의 이웃에 “배울”, “배웠”이 있습니다. 연기하는 배우와 공부하는 배우다는 글자가 같아서 **같은 행 하나**를 나눠 씁니다. 임베딩 표만으로는 둘을 구분할 수 없어요.

문맥이 뜻을 정합니다

그렇다면 같은 글자, 다른 뜻은 어떻게 구분할까요? 트랜스포머 인코더는 임베딩 표에서 꺼낸 화살표를 출발점으로, 문장의 다른 토큰들을 보며 화살표를 계속 고쳐 나갑니다(그 블록은 12장에서 직접 조립했어요). 마지막 층에서 나온 화살표는 **문맥이 반영된** 화살표예요. “배”라는 토큰이 세 가지 뜻(몸, 탈것, 과일)으로 쓰인 문장 여섯 개로 확인해 보겠습니다.

```
import torch
from transformers import AutoModel, AutoTokenizer

토크나이저 = AutoTokenizer.from_pretrained("klue/bert-base")
모델 = AutoModel.from_pretrained("klue/bert-base").eval()

문장들 = ["배가 고파서 밥을 먹었다", "점심을 굶었더니 배가 아프다",      # 몸
         "배를 타고 섬에 갔다", "항구에 큰 배가 들어왔다",              # 탈것
         "깎아 놓은 배가 달고 시원하다", "사과와 배를 한 상자씩 샀다"]   # 과일
배_번호 = 토크나이저.convert_tokens_to_ids("배")

벡터들 = []
with torch.no_grad():
    for 문장 in 문장들:
        입력 = 토크나이저(문장, return_tensors="pt")
        출력 = 모델(**입력).last_hidden_state[0]                # (토큰 수, 768)
        자리 = (입력["input_ids"][0] == 배_번호).nonzero()[0, 0]  # '배' 토큰의 위치
        벡터들.append(출력[자리])

단위 = torch.stack(벡터들)
단위 = 단위 / 단위.norm(dim=1, keepdim=True)
print("문맥을 거친 '배'끼리의 코사인 유사도:")
```

```
print((단위 @ 단위.T).numpy().round(2))
```

```
문맥을 거친 '배'끼리의 코사인 유사도:
[[1.   0.89 0.69 0.65 0.64 0.65]
 [0.89 1.   0.61 0.62 0.67 0.65]
 [0.69 0.61 1.   0.82 0.58 0.66]
 [0.65 0.62 0.82 1.   0.6  0.64]
 [0.64 0.67 0.58 0.6  1.   0.71]
 [0.65 0.65 0.66 0.64 0.71 1.  ]]
```

입력 임베딩만 보면 여섯 문장의 “배”는 모두 같은 행에서 꺼낸 같은 화살표라 유사도가 전부 1이었을 겁니다. 그런데 인코더를 거친 뒤에는 결과 표가 2 × 2 덩어리 세 개로 나뉩니다. 몸의 배끼리 0.89, 탈것끼리 0.82, 과일끼리 0.71로, 다른 뜻끼리(0.58~0.69)보다 높아요. 모델이 “고파서”, “섬”, “달고” 같은 주변 말을 보고 같은 토큰의 화살표를 다른 방향으로 돌려놓은 겁니다. 이렇게 문맥에 따라 달라지는 화살표를 **문맥 임베딩**이라고 부릅니다.

`last_hidden_state`는 마지막 층의 출력으로 모양이 (배치, 토큰 수, 768)이고, `nonzero()`로 “배” 토큰의 자리를 찾아 그 줄만 꺼냈어요. 모델은 `eval()`로 평가 모드로, 계산은 `torch.no_grad()` 안에서 했습니다. 6장 6.8절에서 배운 추론의 기본 자세 그대로예요.

문장 하나를 화살표 하나로

토큰마다 화살표가 있으니, 평균을 내면 문장 전체를 화살표 하나로 나타낼 수 있습니다. 이때 패딩 칸은 평균에서 빼야 하니 `attention_mask`를 곱해서 진짜 토큰만 더해요.

```
import torch
from transformers import AutoModel, AutoTokenizer

토크나이저 = AutoTokenizer.from_pretrained("klue/bert-base")
모델 = AutoModel.from_pretrained("klue/bert-base").eval()


def 문장_벡터(문장들: list[str]) -> torch.Tensor:
    """토큰 벡터를 평균 내어 문장마다 화살표 하나를 만든다. 패딩 칸은 마스크로 뺀다."""
    입력 = 토크나이저(문장들, padding=True, return_tensors="pt")
    with torch.no_grad():
```

```
        출력 = 모델(**입력).last_hidden_state                # (문장, 토큰, 768)
    마스크 = 입력["attention_mask"].unsqueeze(-1).float()     # (문장, 토큰, 1)
    평균 = (출력 * 마스크).sum(dim=1) / 마스크.sum(dim=1)
    return 평균 / 평균.norm(dim=1, keepdim=True)


문장들 = ["연기도 좋고 스토리도 탄탄하다", "배우들 연기가 훌륭하고 줄거리도 짜임새 있다",
        "연기도 별로고 스토리도 엉성하다", "오늘 저녁 메뉴는 김치찌개다"]
벡터 = 문장_벡터(문장들)
print((벡터 @ 벡터.T).numpy().round(2))
```

```
[[1.   0.86 0.8  0.47]
 [0.86 1.   0.74 0.45]
 [0.8  0.74 1.   0.46]
 [0.47 0.45 0.46 1.  ]]
```

영화 리뷰 셋끼리는 0.74~0.86인데, "오늘 저녁 메뉴는 김치찌개다"와는 0.45~0.47로 뚝 떨어집니다. **무엇에 대한 문장인지**는 잘 잡았어요. 그런데 칭찬하는 첫 문장과 깎아내리는 셋째 문장의 유사도가 0.80으로, 둘째 칭찬 문장과의 0.86과 별로 차이 나지 않습니다. 사전학습만 한 모델은 "연기, 스토리" 같은 화제가 겹치면 비슷하다고 보고, **좋다·나쁘다**의 차이는 크게 신경 쓰지 않아요. 우리가 원하는 감성을 구분하려면 그 일에 맞게 모델을 더 가르쳐야 합니다. 다음 두 절의 주제예요.

13.8 처음부터 학습하는 기준 모델

평균 임베딩 분류기

사전학습 모델을 쓰기 전에, 아무것도 미리 배우지 않은 작은 모델로 점수를 먼저 재 두겠습니다. 이렇게 비교의 출발점이 되는 모델을 **기준 모델(baseline)** 이라고 해요. 3장에서 고전 머신러닝 모델을 먼저 돌려 본 것과 같은 이유입니다. 기준이 없으면 비싼 모델이 얼마나 좋아졌는지 말할 수 없으니까요.

모델은 가장 단순하게 만듭니다. 13.5절처럼 리뷰 15만 개로 학습한 BPE 토크나이저(사전 8,000개)로 자르고, 토큰마다 64칸 화살표를 꺼내서, 진짜 토큰들의 **평균** 하나를 선형층에 넣어 긍정·부정 점수를 냅니다. 13.7절의 문장 화살표 만들기와 같은 계산이에요. 학습 데이터 양을 5천, 2만, 15만 개로 바꿔 가며 같은 시험 리뷰 5천 개로 평가합니다. 작은 모델이라 CPU로 충분해요.

```
import torch
from tokenizers import Tokenizer, models, pre_tokenizers, trainers
from torch import nn
from torch.nn.utils.rnn import pad_sequence
from torch.utils.data import DataLoader

from 한국어도구 import NSMC_읽기

전체 = NSMC_읽기("ratings_train.txt")
시험 = NSMC_읽기("ratings_test.txt", 5000)

토크나이저 = Tokenizer(models.BPE(unk_token="[UNK]"))
토크나이저.pre_tokenizer = pre_tokenizers.Whitespace()
토크나이저.train_from_iterator([s for s, _ in 전체], trainers.BpeTrainer(
    vocab_size=8000, special_tokens=["[PAD]", "[UNK]"], show_progress=False))


class 평균_임베딩_분류기(nn.Module):
    """토큰 화살표들의 평균 하나로 긍정/부정을 가린다. 순서는 보지 않는다."""

    def __init__(self, 사전_크기: int, 칸수: int = 64) -> None:
        super().__init__()
        self.임베딩 = nn.Embedding(사전_크기, 칸수, padding_idx=0)
        self.출력 = nn.Linear(칸수, 2)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        마스크 = (x != 0).unsqueeze(-1).float()                    # 패딩(0번)은 평균에서 뺀다
        평균 = (self.임베딩(x) * 마스크).sum(1) / 마스크.sum(1).clamp(min=1)
        return self.출력(평균)


def 묶기(배치: list[tuple[str, int]]) -> tuple[torch.Tensor, torch.Tensor]:
    번호들 = [torch.tensor(토크나이저.encode(s).ids[:64]) for s, _ in 배치]
    return pad_sequence(번호들, batch_first=True), torch.tensor([y for _, y in 배치])


def 학습하고_평가(학습: list[tuple[str, int]], 에폭: int = 3) -> 평균_임베딩_분류기:
```

```
    torch.manual_seed(0)
    모델 = 평균_임베딩_분류기(토크나이저.get_vocab_size())
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=3e-3)
    로더 = DataLoader(학습, batch_size=64, shuffle=True, collate_fn=묶기,
                      generator=torch.Generator().manual_seed(0))
    for _ in range(에폭):
        모델.train()
        for x, y in 로더:
            옵티마이저.zero_grad()
            nn.functional.cross_entropy(모델(x), y).backward()
            옵티마이저.step()
    모델.eval()
    with torch.no_grad():
        x, y = 묶기(시험)
        정확도 = (모델(x).argmax(1) == y).float().mean().item()
    print(f"학습 {len(학습):>7,}개 → 시험 정확도 {정확도:.4f}")
    return 모델


학습하고_평가(전체[:5000])
학습하고_평가(전체[:20000])
모델 = 학습하고_평가(전체)

# 임베딩이 배운 '긍정 쪽 방향': 출력층의 (긍정 가중치 - 부정 가중치)
방향 = 모델.출력.weight[1] - 모델.출력.weight[0]
점수 = (모델.임베딩.weight @ 방향).detach()
번호_토큰 = {i: t for t, i in 토크나이저.get_vocab().items()}
print("긍정 쪽 끝:", [번호_토큰[i] for i in 점수.topk(8).indices.tolist()])
print("부정 쪽 끝:", [번호_토큰[i] for i in (-점수).topk(8).indices.tolist()])
```

```
학습   5,000개 → 시험 정확도 0.7416
학습  20,000개 → 시험 정확도 0.8154
학습 149,995개 → 시험 정확도 0.8500
긍정 쪽 끝: ['울컥', '돌아보게', '탄탄하고', '좋았어요', '질리지', '여운이', '좋았고', '저평가']
부정 쪽 끝: ['최악의', '최악', '최악이다', '높아서', '엉성하고', '시간낭비', '개판', '아깝고']
```

학습 데이터가 5천 개면 74.16%, 2만 개면 81.54%, 15만 개를 모두 쓰면 85.00%입니다. 데이터가

늘수록 좋아지지만 늘어나는 폭은 점점 줄어들어요. 이 모델은 토큰의 **순서를 전혀 보지 않습니다**. "재미없지는 않았다"와 "않았다 재미없지는"이 같은 평균이 되죠. 그런데도 85%를 맞히는 건 영화 리뷰의 감성이 상당 부분 단어 몇 개로 결정되기 때문입니다.

마지막 두 줄은 학습된 임베딩을 들여다본 결과입니다. 출력층의 "긍정 가중치 - 부정 가중치"는 화살표 공간에서 **긍정 쪽을 가리키는 방향**이에요. 토큰마다 이 방향으로 얼마나 뻗어 있는지(내적)를 재서 양 끝을 뽑았습니다. "울컥", "좋았어요", "여운이"와 "최악", "시간낭비", "아깝고"는 예상대로예요.

눈여겨볼 것은 "저평가"가 긍정, "높아서"가 부정 쪽 끝에 있다는 점입니다. 이 데이터에는 다른 사람들이 매긴 평점이 지나치게 높거나 낮다고 불평하는 **평점에 대한 불만**이 많습니다. 그래서 모델이 "평점이 높다고 투덜대면 이 영화가 싫다는 뜻, 저평가됐다고 하면 좋다는 뜻"이라고 배운 거예요. 실제로 "높아서"가 들어간 리뷰를 찾아보면 거의 모두 부정 리뷰입니다. 13.7절의 사전학습 임베딩은 "최고"와 "최악"을 이웃으로 뒀지만, 감성 분류를 위해 처음부터 배운 이 작은 임베딩은 "최악"을 부정 쪽 맨 끝에 놓았습니다. 무엇을 위해 배웠느냐에 따라 같은 단어의 화살표가 전혀 다른 곳을 가리키는 거예요.

13.9 사전학습 인코더 미세조정

남이 공부해 둔 것을 빌려 쓰기

10장 전이 학습에서 ImageNet으로 미리 학습한 ResNet18을 가져와 위성 사진(EuroSAT)을 분류했죠. 문장도 똑같이 할 수 있습니다. 한국어 문장을 엄청나게 많이 읽고 "가려 둔 단어 맞히기"를 연습한 모델을 가져와서, 우리 일(감성 분류)에 맞게 조금만 더 가르치는 거예요. 이것을 **미세조정(fine-tuning)** 이라고 합니다. 외국어를 처음부터 배우는 사람보다, 한국어를 잘하는 사람에게 영화 평론 요령만 알려 주는 쪽이 훨씬 빠른 것과 같아요.

이 장에서 쓸 모델은 KLUE BERT(`klue/bert-base`)입니다. 한국어 이해 평가 데이터셋인 KLUE를 만든 연구진이 뉴스, 나무위키, 국립국어원 말뭉치 등 한국어 문장 약 63GB로 학습해 공개했어요. 12장에서 조립한 트랜스포머 블록을 마스크 없이(앞뒤를 모두 보게) 12층 쌓은 인코더 구조예요(12장 12.7절의 표에서 "인코더만"에 해당합니다).

쓰기 전에 라이선스부터

남이 만든 모델이나 데이터를 쓸 때는 **라이선스**부터 확인해야 합니다. 공개되어 있다고 아무렇게나 써도 되는 건 아니에요. 비상업 용도만 허락하는 모델을 회사 서비스에 넣으면 문제가 되고, 어떤 모

델은 사용 동의나 로그인을 해야 받을 수 있습니다. Hugging Face의 모델 페이지(모델 카드)에 라이선스가 적혀 있고, 코드로도 확인할 수 있어요.

```
from huggingface_hub import model_info

for 이름 in ["klue/bert-base", "google-bert/bert-base-multilingual-cased", "FacebookAI/
↪ xlm-roberta-base"]:
    정보 = model_info(이름)
    라이선스 = 정보.card_data.license if 정보.card_data else "카드에 없음"
    print(f"{이름}: 라이선스 {라이선스}, 로그인 필요 {bool(정보.gated)}, "
          f"파라미터 {정보.safetensors.total:,}개")
```

```
klue/bert-base: 라이선스 cc-by-sa-4.0, 로그인 필요 False, 파라미터 111,243,522개
google-bert/bert-base-multilingual-cased: 라이선스 apache-2.0, 로그인 필요 False, 파라미터
↪ 178,566,653개
FacebookAI/xlm-roberta-base: 라이선스 mit, 로그인 필요 False, 파라미터 278,885,778개
```

KLUE BERT는 CC BY-SA 4.0입니다. 상업적 이용을 포함해 자유롭게 쓸 수 있지만, 출처를 밝혀야 하고(BY), 이 모델을 고쳐서 배포할 때는 같은 라이선스로 공개해야(SA) 해요. 미세조정한 모델을 다른 사람에게 배포하려면 이 조건을 따라야 한다는 뜻입니다. 이 장의 데이터인 NSMC는 CC0 1.0(퍼블릭 도메인), 실습에서 쓸 KLUE YNAT는 CC BY-SA 4.0이에요. `gated`가 `False`인 것은 로그인 없이 받을 수 있다는 뜻입니다.

모델 크기 재기

분류용 모델을 불러와서 생김새와 크기를 보겠습니다. `AutoModelForSequenceClassification`은 사전학습 인코더 위에 분류용 머리를 얹어 주는 클래스예요.

```
from transformers import AutoModelForSequenceClassification

모델 = AutoModelForSequenceClassification.from_pretrained("klue/bert-base", num_labels=2)

for 이름, 부품 in 모델.named_children():
    print(f"{이름}: {type(부품).__name__}")
for 이름, 부품 in 모델.bert.named_children():
    print(f"  {'bert.' + 이름:16}{sum(p.numel() for p in 부품.parameters()):>12,}개")
```

```
print(f"  {'classifier':16}{sum(p.numel() for p in 모델.classifier.parameters()):>12,}개")
print("인코더 층 수:", len(모델.bert.encoder.layer))

전체 = sum(p.numel() for p in 모델.parameters())
바이트 = sum(p.element_size() * p.numel() for p in 모델.parameters())
print(f"전체 {전체:,}개, float32로 {바이트 / 1024**2:.0f} MB")
# AdamW로 학습하면 파라미터 + 기울기 + 옵티마이저 상태 2개 = 파라미터의 4배
print(f"학습 중 최소 {4 * 바이트 / 1024**3:.2f} GB (중간 계산값은 별도)")
```

```
bert: BertModel
dropout: Dropout
classifier: Linear
  bert.embeddings   24,972,288개
  bert.encoder      85,054,464개
  bert.pooler          590,592개
  classifier             1,538개
인코더 층 수: 12
전체 110,618,882개, float32로 422 MB
학습 중 최소 1.65 GB (중간 계산값은 별도)
```

모델은 세 부분입니다. 문장을 읽는 `bert`, 8장 8.3절의 드롭아웃, 그리고 `classifier`라는 `nn.Linear(768, 2)` 하나예요. 768 × 2 + 2 = 1,538개의 파라미터만 새로 만든 것이고, 나머지 1억 1천만 개는 사전학습에서 가져왔습니다. 임베딩 표(13.7절의 3만 2천 × 768에 위치·문장 구분 임베딩을 더한 것)가 약 2,500만 개, 12층 인코더가 약 8,500만 개예요. `pooler`는 `[CLS]` 자리의 출력을 한 번 더 다듬는 층입니다. 13.6절에서 "`[CLS]` 자리를 보고 판단한다"고 한 것이 이 경로예요.

앞에서 `model_info`로 Hub에서 확인한 파라미터 수(111,243,522개)와 조금 다른 이유는, 사전학습 때 쓰던 "빈칸 맞히기" 머리가 빠지고 우리 분류 머리가 붙었기 때문입니다. 이렇게 머리를 바꿔 끼우면 transformers가 "새로 초기화된 가중치가 있다"는 경고를 띄우는데, 정상이에요. 그 머리를 지금부터 학습시킬 테니까요.

`float32`로 422MB이니 노트북에서도 넉넉히 올라갑니다. 다만 학습할 때는 파라미터마다 기울기 하나, AdamW가 기억하는 값 두 개(8장 8.2절)가 더 붙어서 최소 네 배인 1.65GB가 필요하고, 여기에 중간 계산값이 배치 크기와 문장 길이에 비례해 더해져요. 14장에서 더 큰 언어 모델을 다룰 때 이 계산이 다시 중요해집니다.

몇 토큰에서 자를까요?

문장 길이는 메모리와 시간에 바로 영향을 줍니다. 모델은 최대 512토큰까지 받을 수 있지만, 리뷰는 대부분 짧으니 그렇게 길게 둘 필요가 없어요. 데이터를 보고 정하겠습니다.

```
from transformers import AutoTokenizer

from 한국어도구 import NSMC_읽기

토크나이저 = AutoTokenizer.from_pretrained("klue/bert-base")
학습 = [문장 for 문장, _ in NSMC_읽기("ratings_train.txt", 20000)]
길이 = sorted(len(토크나이저(s)["input_ids"]) for s in 학습)     # [CLS], [SEP] 포함

for 비율 in [0.5, 0.9, 0.99]:
    print(f"{비율:.0%} 지점: {길이[int(len(길이) * 비율)]}토큰")
print("가장 긴 리뷰:", 길이[-1], "토큰")
for 자르기 in [32, 64, 128]:
    잘림 = sum(1 for n in 길이 if n > 자르기) / len(길이)
    print(f"max_length={자르기}: 잘리는 리뷰 {잘림:.1%}")
print("모델이 받을 수 있는 최대 길이:", 토크나이저.model_max_length)
```

```
50% 지점: 17토큰
90% 지점: 44토큰
99% 지점: 80토큰
가장 긴 리뷰: 127 토큰
max_length=32: 잘리는 리뷰 16.6%
max_length=64: 잘리는 리뷰 4.2%
max_length=128: 잘리는 리뷰 0.0%
모델이 받을 수 있는 최대 길이: 512
```

절반의 리뷰가 17토큰 이하이고, 90%가 44토큰, 99%가 80토큰 이하입니다. `max_length=64`로 자르면 4.2%의 리뷰만 뒷부분이 잘려요. 긴 리뷰의 뒷부분이 조금 잘려도 감성은 대개 앞부분에서 드러나니, 이 장에서는 64를 쓰겠습니다. 128로 하면 아무것도 잘리지 않지만, 패딩은 배치 안의 가장 긴 문장에 맞추니(7장 7.7절) 긴 리뷰가 하나만 섞여도 배치 전체가 길어져 계산이 늘어납니다.

미세조정하기

이제 학습시킬 차례입니다. 학습 루프는 6장의 다섯 줄 그대로이고, 8장에서 익힌 도구 세 가지를 얹습니다.

- **AdamW**와 weight decay(8장 8.2절)
- **선형 워밍업 스케줄러**: 처음 10% 동안은 학습률을 0에서 천천히 올리고, 그 뒤로는 0까지 곧게 내립니다. 8장 8.7절의 스케줄러와 같은 일을 하는 transformers의 함수예요. 처음부터 큰 걸음을 떼면 새로 붙인 머리가 내는 엉뚱한 기울기가 사전학습된 가중치를 망가뜨릴 수 있어서 워밍업을 둡니다.
- **그래디언트 클리핑**(8장 8.9절)

데이터는 노트북 규모로 줄여서 학습 리뷰 5천 개로 두 에폭만 돌리고, 시험 리뷰 5천 개로 평가합니다. 1억 개가 넘는 파라미터를 학습하니 GPU(이 책에서는 Apple MPS)를 씁니다.

```
import torch
from torch.utils.data import DataLoader
from transformers import (AutoModelForSequenceClassification, AutoTokenizer,
                          get_linear_schedule_with_warmup)

from 한국어도구 import NSMC_읽기


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")


torch.manual_seed(0)
장치 = 장치_고르기()
이름 = "klue/bert-base"
토크나이저 = AutoTokenizer.from_pretrained(이름)
모델 = AutoModelForSequenceClassification.from_pretrained(
    이름, num_labels=2, id2label={0: "부정", 1: "긍정"}, label2id={"부정": 0, "긍정": 1}).to(장치)
```

```
학습 = NSMC_읽기("ratings_train.txt", 5000)
시험 = NSMC_읽기("ratings_test.txt", 5000)


def 묶기(배치: list[tuple[str, int]]) -> dict[str, torch.Tensor]:
    """토크나이저가 패딩·자르기·마스크를 한 번에 해 준다(7장 collate_fn의 역할)."""
    입력 = 토크나이저([s for s, _ in 배치], padding=True, truncation=True,
                      max_length=64, return_tensors="pt")
    입력["labels"] = torch.tensor([y for _, y in 배치])
    return 입력


학습_로더 = DataLoader(학습, batch_size=32, shuffle=True, collate_fn=묶기,
                       generator=torch.Generator().manual_seed(0))
시험_로더 = DataLoader(시험, batch_size=128, collate_fn=묶기)


def 정확도() -> float:
    모델.eval()
    맞음 = 0
    with torch.no_grad():
        for 배치 in 시험_로더:
            배치 = {k: v.to(장치) for k, v in 배치.items()}
            맞음 += (모델(**배치).logits.argmax(-1) == 배치["labels"]).sum().item()
    return 맞음 / len(시험)


에폭 = 2
옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=3e-5, weight_decay=0.01)
전체_걸음 = 에폭 * len(학습_로더)
스케줄러 = get_linear_schedule_with_warmup(옵티마이저, int(0.1 * 전체_걸음), 전체_걸음)

print(f"미세조정 전 시험 정확도: {정확도():.4f}")
for 에폭_번호 in range(1, 에폭 + 1):
    모델.train()
```

```
    손실_합 = 0.0
    for 배치 in 학습_로더:
        배치 = {k: v.to(장치) for k, v in 배치.items()}
        출력 = 모델(**배치)                    # labels를 함께 주면 손실까지 계산해 준다
        출력.loss.backward()
        torch.nn.utils.clip_grad_norm_(모델.parameters(), 1.0)
        옵티마이저.step()
        스케줄러.step()
        옵티마이저.zero_grad()
        손실_합 += 출력.loss.item()
    print(f"에폭 {에폭_번호}: 학습 손실 {손실_합 / len(학습_로더):.4f}, 시험 정확도 {정확도():.4f}")

모델.save_pretrained("data/감성모델")
토크나이저.save_pretrained("data/감성모델")
```

```
미세조정 전 시험 정확도: 0.4182
에폭 1: 학습 손실 0.4201, 시험 정확도 0.8518
에폭 2: 학습 손실 0.2249, 시험 정확도 0.8620
```

미세조정 전에는 41.82%로, 동전 던지기보다도 못합니다. 분류 머리가 무작위 값이니 당연해요. 한 에폭 뒤 85.18%, 두 에폭 뒤 **86.20%** 가 됐습니다. 13.8절의 기준 모델과 나란히 놓아 보세요.

모델	학습 리뷰 수	시험 정확도
평균 임베딩(처음부터)	5,000	74.16%
평균 임베딩(처음부터)	149,995	85.00%
KLUE BERT 미세조정	5,000	86.20%

사전학습 모델은 **30분의 1의 데이터**로 처음부터 학습한 모델을 넘어섰습니다. 라벨을 붙이는 데 돈과 시간이 드는 실무에서 이 차이는 아주 큽니다. 리뷰 5천 개에 라벨을 다는 것과 15만 개에 다는 것은 전혀 다른 일이니까요. 데이터를 더 쓰고 에폭을 늘리면 점수는 더 오릅니다. NSMC 전체로 미세조정하면 90% 안팎까지 올라간다고 알려져 있지만, 노트북에서 몇 분 안에 끝나는 규모로 맞췄어요. 이 숫자들은 이 책의 실행 환경(Apple M4 Max, MPS)에서 나온 값이고, MPS의 계산 순서 때문에 실행할 때마다 소수점 아래 둘째 자리 정도가 달라질 수 있습니다.

코드에서 눈여겨볼 곳이 몇 군데 있습니다.

- 토크나이저가 패딩·자르기·마스크를 모두 해 주니, `묶기` 함수는 토크나이저를 부르고 정답만 `labels`라는 이름으로 덧붙이면 끝입니다. 7장 7.7절에서 직접 짰던 `collate_fn`과 비교해 보세요.
- `모델(**배치)`는 딕셔너리의 키를 인자 이름으로 풀어서 넘기는 1권 9장의 `**` 문법입니다. 배치에 `labels`가 있으면 모델이 교차 엔트로피 손실(6장 6.5절)까지 계산해서 `출력.loss`로 돌려줘요. 점수(로짓)는 `출력.logits`입니다.
- 학습률이 `3e-5`(0.00003)로 6장보다 훨씬 작습니다. 사전학습으로 이미 좋은 자리에 있는 가중치를 조금만 옮기려는 거예요. 학습률을 크게 하면 사전학습으로 배운 것을 잊어버려 성능이 오히려 떨어집니다.
- 마지막의 `save_pretrained`는 가중치, 설정(`id2label` 포함), 토크나이저를 폴더 하나에 저장합니다. 6장 6.10절의 `state_dict` 저장보다 한 걸음 더 나간 것으로, 이 폴더 이름만 있으면 설계도 클래스를 몰라도 `from_pretrained`로 그대로 불러올 수 있어요.

Q

Q. 인코더는 얼리고 분류 머리만 학습하면 안 되나요? 됩니다. 10장의 "특징 추출" 방식이에요. `모델.bert.requires_grad_(False)`로 인코더를 얼리면 학습할 파라미터가 1,538개로 줄어 훨씬 빠르고 메모리도 적게 듭니다. 다만 13.7절에서 봤듯 사전학습 인코더의 문장 화살표는 감성을 잘 구분하지 못해서, 이 장의 설정 그대로 해 보면 정확도가 전체 미세조정보다 몇 포인트 낮게 나옵니다. 그리고 Apple MPS에서 인코더를 얼린 채 학습 모드로 돌리면 이 책의 환경(PyTorch 2.14.1)에서는 `NotImplementedError: scaled_dot_product_attention for MPS does not support dropout.` 오류가 났습니다. 인코더를 얼리지 않은 13.9절의 학습에서는 나지 않던 오류예요. 모델을 불러올 때 `attn_implementation="eager"`를 주면 어텐션을 다른 방식으로 계산해서 해결됩니다. 문장 분류에서는 인코더까지 함께 고치는 전체 미세조정이 기본이고, 전부 고치기에는 모델이 너무 클 때 쓰는 절충안이 14장의 LoRA입니다.

13.10 평가: 숫자 너머를 보기

혼동 행렬과 F1

정확도 86.2%만으로는 모델이 어떻게 틀리는지 알 수 없습니다. 3장 3.7절에서 배운 혼동 행렬과 정밀도·재현율·F1로 나눠 보고, 무엇보다 **틀린 리뷰를 직접 읽어** 보겠습니다. 앞 절에서 저장한 폴더를 불러와요(이 블록은 앞 블록을 먼저 실행해야 합니다). 평가만 하니 CPU에서도 충분합니다.

```
import torch
from sklearn.metrics import classification_report, confusion_matrix
from transformers import AutoModelForSequenceClassification, AutoTokenizer

from 한국어도구 import NSMC_읽기

토크나이저 = AutoTokenizer.from_pretrained("data/감성모델")          # 13.9절에서 저장한 폴더
모델 = AutoModelForSequenceClassification.from_pretrained("data/감성모델").eval()
시험 = NSMC_읽기("ratings_test.txt", 5000)

확률들 = []
with torch.no_grad():
    for i in range(0, len(시험), 250):
        입력 = 토크나이저([s for s, _ in 시험[i:i + 250]], padding=True, truncation=True,
                          max_length=64, return_tensors="pt")
        확률들.append(모델(**입력).logits.softmax(-1))
확률 = torch.cat(확률들)
예측 = 확률.argmax(-1)
정답 = torch.tensor([y for _, y in 시험])

print(confusion_matrix(정답, 예측))
print(classification_report(정답, 예측, target_names=["부정", "긍정"], digits=3))

# 가장 자신 있게 틀린 리뷰 다섯 개 (원문은 시험[i][0]을 출력해 직접 읽어 보세요)
틀림 = (예측 != 정답).nonzero().squeeze(1)
자신감 = 확률[틀림].max(-1).values
for i in 틀림[자신감.argsort(descending=True)[:5]].tolist():
    print(f"{i}번: 정답 {정답[i].item()} / 예측 {예측[i].item()} ({확률[i].max():.3f}), {len(시험
    ↪ [i][0])}글자")
print(f"틀린 리뷰 {len(틀림)}개 중 확률 0.95 이상으로 틀린 것: {(자신감 >= 0.95).sum().item()}개")
```

```
[[2192  346]
 [ 344 2118]]
              precision    recall  f1-score   support

          부정      0.864     0.864     0.864      2538
```

```
          긍정      0.860     0.860     0.860      2462

    accuracy                          0.862      5000
   macro avg      0.862     0.862     0.862      5000
weighted avg      0.862     0.862     0.862      5000

3475번: 정답 0 / 예측 1 (0.995), 13글자
345번: 정답 0 / 예측 1 (0.995), 12글자
668번: 정답 0 / 예측 1 (0.994), 10글자
853번: 정답 0 / 예측 1 (0.994), 19글자
2666번: 정답 0 / 예측 1 (0.994), 10글자
틀린 리뷰 690개 중 확률 0.95 이상으로 틀린 것: 145개
```

혼동 행렬의 윗줄은 진짜 부정 리뷰 2,538개 중 2,192개를 부정으로, 346개를 긍정으로 판단했다는 뜻입니다. 아랫줄은 진짜 긍정 2,462개 중 344개를 부정으로 잘못 봤어요. 두 방향의 실수가 거의 같아서 부정·긍정의 정밀도, 재현율, F1이 모두 0.86 언저리로 고릅니다. 데이터가 균형 잡혀 있으니 정확도만 봐도 크게 속지는 않는 경우예요. 3.7절의 99% 엉터리 검사기처럼 한쪽이 드문 문제라면 이 표가 훨씬 중요해집니다.

이제 가장 흥미로운 부분입니다. 모델이 **가장 자신 있게 틀린** 리뷰 다섯 개를 골랐습니다. 책에는 번호와 길이만 실었으니, 여러분은 `print(시험[i][0])`으로 원문을 직접 읽어 보세요. 다섯 개 모두 정답은 부정인데 모델은 0.99가 넘는 확률로 긍정이라고 했고, 10~19글자의 짧은 리뷰입니다.

이 책의 실행에서 다섯 개를 읽어 보니 세 개는 "재밌어요, 또 보고 싶어요"처럼(예제용으로 지은 문장입니다) 누가 읽어도 칭찬인데 라벨이 부정이었습니다. 모델이 틀린 게 아니라 **라벨이 틀린** 거예요. 별점을 실수로 낮게 줬거나 별점과 글이 따로 노는 경우입니다. 하나는 말줄임표가 붙은 칭찬이라 비꼬는 말인지 사람도 판단이 갈렸고, 마지막 하나는 지루하다는 불평이 분명해서 모델이 진짜로 틀린 경우였어요.

이렇게 하나씩 읽어 가며 "라벨이 틀렸나, 애매한가, 모델이 틀렸나"를 나눠 보면 모델이 실제로 얼마나 잘하는지 감이 옵니다. 마지막 줄을 보면 틀린 690개 중 145개는 0.95 이상의 확률로 틀렸으니, 읽어 볼 거리는 충분해요.

자신 있게 틀린 예제를 읽는 일은 평가에서 가장 값진 습관입니다. 모델의 약점뿐 아니라 **데이터의 약점**이 보이거든요. 라벨 잡음이 많다면 모델을 키우기보다 데이터를 고치는 것이 먼저이고, 시험 데이터의 라벨이 틀렸다면 "정확도 86.2%"라는 숫자 자체에도 그만큼 오차가 섞여 있는 겁니다.

pipeline으로 바로 써 보기

저장한 모델을 서비스에 쓸 때는 토크나이저 부르기, 모델 부르기, 소프트맥스, 라벨 이름 붙이기를 매번 짜는 게 번거롭습니다. transformers의 `pipeline`이 이 과정을 한 줄로 묶어 줘요. 일부러 까다로운 문장들을 넣어 보겠습니다.

```
from transformers import pipeline

분류기 = pipeline("text-classification", model="data/감성모델")
예문들 = ["배우들 연기가 정말 좋았어요",
        "표값이 아까운 두 시간이었다",
        "재미없지는 않았다",                    # 이중 부정
        "와 정말 대단하다 두 시간을 이렇게 버리게 해 주다니",   # 비꼬기
        "ㄹㅇ 개꿀잼 인정"]                     # 줄임말
for 예문, 결과 in zip(예문들, 분류기(예문들)):
    print(f"{결과['label']} {결과['score']:.3f}  {예문}")
```

```
긍정 0.994  배우들 연기가 정말 좋았어요
부정 0.972  표값이 아까운 두 시간이었다
긍정 0.882  재미없지는 않았다
긍정 0.673  와 정말 대단하다 두 시간을 이렇게 버리게 해 주다니
긍정 0.983  ㄹㅇ 개꿀잼 인정
```

학습할 때 넣어 둔 `id2label` 덕분에 결과가 "긍정", "부정"으로 나옵니다. 첫 두 문장은 쉽게 맞혔고, "재미없지는 않았다"의 이중 부정도 긍정(0.882)으로 읽었어요. 13.8절의 평균 임베딩 모델은 순서를 보지 않아 이런 문장에 약한데, 트랜스포머는 단어 사이의 관계를 보기 때문에 가능한 일입니다. "ㄹㅇ 개꿀잼 인정" 같은 줄임말도 맞혔어요.

하지만 "와 정말 대단하다 두 시간을 이렇게 버리게 해 주다니"는 0.673으로 긍정이라고 답했습니다. 자신은 없어 보이지만 틀렸어요. "대단하다"를 칭찬으로 읽은 거예요. **비꼬기**는 감성 분류가 가장 어려워하는 문장입니다. 사람도 말투와 상황을 보고 알아채는 것이라, 리뷰 5천 개로 배운 모델에게는 무리예요. 이런 한계를 숨기지 말고, 모델을 쓸 곳에서 어떤 문장이 들어올지 생각해 보고 그런 예제를 시험 데이터에 일부러 넣어 점검하세요.

13.11 Hugging Face datasets로 데이터 다루기

지금까지는 한국어도구.py로 리뷰를 파이썬 리스트에 읽어 왔습니다. 데이터가 작을 때는 이게 가장 이해하기 쉬워요. 그런데 Hugging Face Hub에는 수많은 공개 데이터셋이 올라와 있고, 이걸 받고 손질하는 데는 datasets 라이브러리가 표준처럼 쓰입니다. 같은 NSMC 파일을 datasets로 다뤄 보겠습니다.

```
from datasets import load_dataset
from torch.utils.data import DataLoader
from transformers import AutoTokenizer, DataCollatorWithPadding

from 한국어도구 import NSMC_받기

NSMC_받기()
자료 = load_dataset("csv", data_files={"train": "data/nsmc/ratings_train.txt",
                                       "test": "data/nsmc/ratings_test.txt"},
                    delimiter="\t")
print(자료)

자료 = 자료.filter(lambda 줄: 줄["document"] is not None and 줄["document"].strip() != "")
print("빈 리뷰를 걸러 낸 뒤:", 자료["train"].num_rows, 자료["test"].num_rows)
작은_학습 = 자료["train"].shuffle(seed=0).select(range(5000))
print(작은_학습.features)

토크나이저 = AutoTokenizer.from_pretrained("klue/bert-base")
토큰화 = 작은_학습.map(lambda 줄: 토크나이저(줄["document"], truncation=True, max_length=64),
                      batched=True, remove_columns=["id", "document"])
토큰화 = 토큰화.rename_column("label", "labels")
print(토큰화)

로더 = DataLoader(토큰화, batch_size=4, collate_fn=DataCollatorWithPadding(토크나이저))
배치 = next(iter(로더))
print({k: tuple(v.shape) for k, v in 배치.items()})
print("문장별 실제 토큰 수:", 배치["attention_mask"].sum(1).tolist())
```

```
DatasetDict({
    train: Dataset({
        features: ['id', 'document', 'label'],
        num_rows: 150000
    })
    test: Dataset({
        features: ['id', 'document', 'label'],
        num_rows: 50000
    })
})
빈 리뷰를 걸러 낸 뒤: 149995 49997
{'id': Value('int64'), 'document': Value('large_string'), 'label': Value('int64')}
Dataset({
    features: ['labels', 'input_ids', 'token_type_ids', 'attention_mask'],
    num_rows: 5000
})
{'labels': (4,), 'input_ids': (4, 64), 'token_type_ids': (4, 64), 'attention_mask': (4, 64)}
문장별 실제 토큰 수: [18, 64, 16, 15]
```

`load_dataset("csv", ...)`는 내 컴퓨터의 CSV·TSV 파일을 읽어서 `train`, `test` 두 갈래로 담은 `DatasetDict`를 돌려줍니다. Hub에 올라온 데이터셋은 파일 경로 대신 `load_dataset("klue/klue", "ynat")`처럼 이름을 주면 돼요(실습에서 씁니다). 이어지는 손질은 1권 10장의 함수형 도구와 닮았습니다.

- **`filter`**: 조건을 만족하는 줄만 남깁니다. 빈 리뷰 5개와 3개를 걸러서 `한국어도구.py`와 같은 149,995개, 49,997개가 됐어요.
- **`shuffle(seed=0).select(range(5000))`**: 섞어서 앞의 5천 개만 고릅니다. 씨앗을 주면 언제나 같은 5천 개예요.
- **`map(..., batched=True)`**: 함수를 모든 줄에 적용합니다. `batched=True`면 1,000줄씩 묶어서 넘겨주는데, 토크나이저는 여러 문장을 한꺼번에 처리할 때 훨씬 빠르니 꼭 켜 두세요. `remove_columns`로 더 필요 없는 열을 지웠고, 정답 열은 모델이 기대하는 이름인 `labels`로 바꿨습니다.

`datasets`의 데이터셋은 7장의 `Dataset`처럼 `len()`과 번호 인덱싱이 되니 `DataLoader`에 바로 넣을 수 있습니다. 이때 `collate_fn`으로 쓴 `DataCollatorWithPadding`은 "배치 안의 가장 긴 문장에 맞춰 패딩하라"는 transformers의 묶기 함수예요. 첫 배치는 실제 토큰 수가 18, 64, 16, 15라서 64에

맞춰졌습니다. 토큰화할 때 패딩하지 않고(`map` 안에는 `padding`이 없죠) 배치를 묶을 때 패딩하는 이 방식을 **동적 패딩**이라고 해요. 7장 7.7절의 "패딩은 배치 안에서만"과 같은 생각입니다.

또 하나의 장점은 디스크 캐시입니다. `datasets`는 데이터를 Apache Arrow 형식으로 디스크에 두고 필요한 부분만 꺼내 써서, 메모리보다 큰 데이터도 다룰 수 있어요. 한 번 `map`한 결과도 캐시해 두었다가 같은 코드를 다시 실행하면 바로 꺼내 줍니다.

13.12 실습: 뉴스 제목 주제 분류기 만들기

배운 것을 모두 엮어서 새 문제를 풀어 보겠습니다. KLUE YNAT는 연합뉴스 기사 제목에 일곱 가지 주제(IT과학, 경제, 사회, 생활문화, 세계, 스포츠, 정치) 중 하나를 붙인 데이터입니다. 학습 4만 5천여 개, 검증 9천여 개이고, KLUE 벤치마크의 일부로 CC BY-SA 4.0으로 공개되어 있어요. 할 일은 다섯 가지입니다.

1. `datasets`로 Hub에서 YNAT를 받고, 학습 6,000개와 검증 3,000개를 고르기
2. KLUE BERT 토크나이저로 제목 토큰화하기(제목은 짧으니 48토큰에서 자르기)
3. 분류 머리를 주제 7개짜리로 바꿔 붙이기
4. 13.9절의 학습 루프로 두 에폭 미세조정하기
5. 주제별 정밀도·재현율·F1로 평가하고, 새 제목을 넣어 보기

```
import torch
from datasets import load_dataset
from sklearn.metrics import classification_report
from torch.utils.data import DataLoader
from transformers import (AutoModelForSequenceClassification, AutoTokenizer,
                          DataCollatorWithPadding, get_linear_schedule_with_warmup)


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")
```

```
# 1) 데이터: KLUE YNAT 뉴스 제목 주제 분류 (CC BY-SA 4.0)
자료 = load_dataset("klue/klue", "ynat")
주제들 = 자료["train"].features["label"].names
print("주제:", 주제들)
학습 = 자료["train"].shuffle(seed=0).select(range(6000))
검증 = 자료["validation"].shuffle(seed=0).select(range(3000))
print(학습[0]["title"], "→", 주제들[학습[0]["label"]])

# 2) 토큰화: 제목만 남기고 나머지 열은 버린다
이름 = "klue/bert-base"
토크나이저 = AutoTokenizer.from_pretrained(이름)


def 토큰화(묶음: dict) -> dict:
    return 토크나이저(묶음["title"], truncation=True, max_length=48)


열 = ["guid", "title", "url", "date"]
학습 = 학습.map(토큰화, batched=True, remove_columns=열).rename_column("label", "labels")
검증 = 검증.map(토큰화, batched=True, remove_columns=열).rename_column("label", "labels")
묶기 = DataCollatorWithPadding(토크나이저)
학습_로더 = DataLoader(학습, batch_size=32, shuffle=True, collate_fn=묶기,
                       generator=torch.Generator().manual_seed(0))
검증_로더 = DataLoader(검증, batch_size=128, collate_fn=묶기)

# 3) 모델: 주제 7개짜리 분류 머리를 붙인다
torch.manual_seed(0)
장치 = 장치_고르기()
모델 = AutoModelForSequenceClassification.from_pretrained(
    이름, num_labels=len(주제들), id2label=dict(enumerate(주제들)),
    label2id={t: i for i, t in enumerate(주제들)}).to(장치)

# 4) 학습: 13.9절과 같은 루프
에폭 = 2
옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=5e-5, weight_decay=0.01)
```

```
전체_걸음 = 에폭 * len(학습_로더)
스케줄러 = get_linear_schedule_with_warmup(옵티마이저, int(0.1 * 전체_걸음), 전체_걸음)
for 에폭_번호 in range(1, 에폭 + 1):
    모델.train()
    손실_합 = 0.0
    for 배치 in 학습_로더:
        배치 = {k: v.to(장치) for k, v in 배치.items()}
        출력 = 모델(**배치)
        출력.loss.backward()
        torch.nn.utils.clip_grad_norm_(모델.parameters(), 1.0)
        옵티마이저.step()
        스케줄러.step()
        옵티마이저.zero_grad()
        손실_합 += 출력.loss.item()
    print(f"에폭 {에폭_번호}: 학습 손실 {손실_합 / len(학습_로더):.4f}")

# 5) 평가: 주제별 정밀도·재현율·F1
모델.eval()
예측, 정답 = [], []
with torch.no_grad():
    for 배치 in 검증_로더:
        배치 = {k: v.to(장치) for k, v in 배치.items()}
        예측 += 모델(**배치).logits.argmax(-1).tolist()
        정답 += 배치["labels"].tolist()
print(classification_report(정답, 예측, target_names=주제들, digits=3))

# 6) 새 제목 넣어 보기 (예제용으로 지어낸 제목)
for 제목 in ["반도체 수출 석 달 연속 증가", "국가대표팀 월드컵 예선 첫 승",
             "주말 전국에 비 소식 기온 뚝", "새 스마트폰 인공지능 비서 탑재"]:
    입력 = 토크나이저(제목, return_tensors="pt").to(장치)
    with torch.no_grad():
        확률 = 모델(**입력).logits.softmax(-1)[0]
    print(f"{제목} → {주제들[확률.argmax()]} ({확률.max():.2f})")
```

```
주제: ['IT과학', '경제', '사회', '생활문화', '세계', '스포츠', '정치']
트럼프 앙숙 워런 트럼프 2020년엔 자유인 아닐 수도 → 세계
에폭 1: 학습 손실 0.7238
에폭 2: 학습 손실 0.2734
              precision    recall  f1-score   support

       IT과학      0.778     0.836     0.806       189
          경제      0.772     0.896     0.829       461
          사회      0.887     0.828     0.857      1186
       생활문화       0.859     0.867     0.863       451
          세계      0.895     0.875     0.885       264
         스포츠       0.934     0.947     0.940       208
          정치      0.861     0.826     0.843       241

    accuracy                          0.857      3000
   macro avg      0.855     0.868     0.861      3000
weighted avg      0.860     0.857     0.857      3000

반도체 수출 석 달 연속 증가 → 경제 (0.99)
국가대표팀 월드컵 예선 첫 승 → 스포츠 (0.98)
주말 전국에 비 소식 기온 뚝 → 생활문화 (0.99)
새 스마트폰 인공지능 비서 탑재 → IT과학 (0.98)
```

한 단계씩 짚어 보겠습니다.

- **데이터**: `features["label"].names`로 번호와 주제 이름의 짝을 바로 얻었습니다. Hub의 데이터셋은 이렇게 라벨 정보까지 함께 담고 있어요. 첫 학습 예제는 "트럼프 앙숙 워런 트럼프 2020년엔 자유인 아닐 수도"로, 주제는 세계입니다(KLUE YNAT에 실린 연합뉴스 기사 제목을 CC BY-SA 4.0에 따라 그대로 인용했어요).
- **모델**: `num_labels=len(주제들)`로 분류 머리가 `nn.Linear(768, 7)`이 됐고, 인코더는 13.9절과 똑같은 KLUE BERT입니다. 문제가 바뀌어도 머리만 갈아 끼우면 되는 것이 사전학습 모델의 큰 장점이에요. 학습률은 미세조정에서 흔히 쓰는 범위(`2e-5`~`5e-5`)의 위쪽인 `5e-5`로 잡았어요.
- **결과**: 검증 정확도 85.7%입니다. 일곱 개 중 하나를 고르는 문제라 아무렇게나 찍으면 14% 정도이니 꽤 잘 배웠어요. 주제별로 보면 스포츠(F1 0.940)가 가장 쉽고, IT과학(0.806)과 경제(0.829)가 가장 어렵습니다. "반도체", "스마트폰" 같은 말이 경제 기사와 IT 기사에 모두

나오니 헷갈릴 만하죠. 이렇게 주제별로 나눠 보면 어디에 데이터를 더 모아야 할지 보입니다.

- **검증 데이터의 분포**: support 열을 보면 검증 데이터는 사회(1,186개)가 압도적으로 많습니다. 학습 데이터는 일곱 주제가 비교적 고른데 검증 데이터는 그렇지 않아요. 그래서 주제별 점수를 그냥 평균 낸 `macro avg`(0.861)와 개수로 가중한 `weighted avg`(0.857)가 다릅니다. 3장 3.7절에서 본 것처럼, 실제로 들어올 데이터의 분포를 생각하고 어느 평균을 볼지 정해야 해요.
- **새 제목**: 예제용으로 지어낸 제목 네 개를 모두 그럴듯한 주제로 분류했습니다. "주말 전국에 비 소식"이 생활문화로 분류된 것은 이 데이터가 날씨 기사를 생활문화로 묶기 때문이에요. 모델은 우리 상식이 아니라 **데이터를 만든 사람의 기준**을 배웁니다.

이 실습의 학습은 이 책의 실행 환경(Apple M4 Max, MPS)에서 몇 분 걸렸고, 결과 수치는 실행마다 조금씩 다를 수 있습니다. 시간이 넉넉하다면 학습 데이터를 4만 5천 개 전부로 늘려 보세요. 점수가 얼마나 오르는지, 어느 주제가 가장 좋아지는지 확인해 보는 것도 좋은 연습입니다.

13.13 정리와 다음 장 미리보기

이 장에서 한국어 문장을 모델이 읽을 수 있게 만들고, 사전학습 모델로 분류기를 만들었습니다.

- **한국어의 어려움**: 교착어라 어절 종류가 폭발한다(15만 리뷰에 35만 어절, 79.6%가 한 번만 등장). 띄어쓰기가 들쭉날쭉하고, 분해형(NFD) 문자열은 눈에 같아도 다르다. 처리 전에 NFC로 맞춘다
- **토큰화의 줄다리기**: 글자는 사전이 작고 문장이 길다, 어절은 반대에 OOV가 심하다. 형태소 분석(Kiwi)은 뜻 단위로 자르지만 신조어에 약하다. 서브워드(BPE, WordPiece, SentencePiece)는 데이터에서 자주 붙는 조각을 묶어 OOV 없이 중간을 찾는다
- **사전학습 토크나이저**: 모델과 한 몸이니 `AutoTokenizer.from_pretrained(모델 이름)`으로 짝을 맞춘다. 출력은 `input_ids`, `token_type_ids`, `attention_mask`, 특수 토큰 `[CLS]`·`[SEP]`·`[PAD]`. 한국어에는 한국어 사전이 유리하다
- **임베딩**: 번호로 행을 꺼내는 표(`nn.Embedding`, 원-핫 @ 가중치와 같다). 코사인 유사도로 이웃을 찾으면 쓰이는 자리가 비슷한 말이 모인다("최고"와 "최악"도). 인코더를 거친 문맥 임베딩은 같은 토큰도 문맥에 따라 다르다
- **기준 모델과 미세조정**: 평균 임베딩 분류기는 5천 개로 74.2%, 15만 개로 85.0%. KLUE BERT를 5천 개로 미세조정하면 86.2%. 작은 학습률, 워밍업 스케줄러, 클리핑, `save_pretrained`
- **평가**: 혼동 행렬과 F1에 더해 자신 있게 틀린 예제를 읽는다. 라벨 잡음과 비꼬기 같은 한계가 보인다
- **도구와 습관**: `datasets`의 `load_dataset`·`filter`·`map(batched=True)`과 동적 패딩,

`pipeline`으로 추론, 그리고 쓰기 전에 모델·데이터의 라이선스 확인

이 장의 KLUE BERT는 문장을 **읽고 이해하는** 인코더였습니다. 문장을 받아 화살표로 바꾸고, 그 화살표로 분류했죠. 그렇다면 요즘 모두가 이야기하는 ChatGPT 같은 모델은 어떻게 문장을 **써 내는** 걸까요? 다음 14장에서는 12장에서 맛본 "다음 토큰 맞히기"를 수십억 번 연습한 공개 대형 언어 모델(LLM)을 노트북에서 직접 돌려 봅니다. 그리고 1억 개 파라미터를 모두 고친 이 장의 미세조정과 달리, 아주 작은 행렬 두 개만 덧붙여 학습하는 LoRA로 큰 모델을 가볍게 길들이는 방법을 배웁니다.

14

언어 모델 다루기: LLM 활용과 LoRA

학습 목표

- 사전학습된 작은 언어 모델을 내려받아 불러오고, 크기와 구조를 숫자로 읽기
- 언어 모델이 하는 일이 "다음 토큰 맞히기" 하나뿐이라는 것을 직접 생성 루프를 짜서 확인하기
- 탐욕·샘플링·온도·top-k·top-p가 다음 토큰 고르기를 어떻게 바꾸는지 설명하기
- 채팅 템플릿과 시스템 프롬프트, 예시(few-shot), 대화 기록을 다루기
- 모델 크기·자료형·KV 캐시로 메모리를 계산하고, 양자화를 직접 흉내 내 보기
- LoRA를 손으로 구현한 뒤 peft로 작은 LLM에 붙여 미세조정하고, 무엇을 배우고 무엇을 못 배우는지 측정하기

들어가며: 휴대폰 자판은 어떻게 다음 단어를 알까요?

휴대폰으로 "오늘 저녁에"까지 치면 자판 위에 "뭐", "같이", "시간" 같은 단어가 떠오릅니다. 자판은 내 마음을 읽은 게 아니라, 수많은 문장에서 "오늘 저녁에" 다음에 자주 나온 말을 기억해 두었다가 보여 줄 뿐이에요. 그중 하나를 누르면 다시 그다음 단어를 추천하죠.

요즘 화제인 **대규모 언어 모델(LLM, Large Language Model)** 도 놀랍게도 같은 놀이를 합니다. 다른 점은 규모예요. 휴대폰 자판이 단어 몇 개의 짝을 기억한다면, LLM은 인터넷 문서와 책 수조 토큰을 읽으며 수억에서 수천억 개의 숫자에 "이 글 다음에는 무엇이 올까"를 새겨 둡니다. 그 놀이를 아주 잘하게 되면 질문에 답하고, 요약하고, 코드를 짜는 것처럼 보이게 돼요.

12장에서는 트랜스포머를 직접 만들고 작은 문자 단위 GPT를 처음부터 학습시켰습니다. 노트북에

서 몇 분 학습한 모델이니 문장이 그럴듯해지는 정도에서 멈췄죠. 이 장에서는 방향을 바꿔, 남이 큰 컴퓨터로 오래 학습해 공개한 **사전학습 모델**을 내려받아 씁니다. 순서는 이렇습니다.

1. 모델을 불러와 크기를 재고, 다음 토큰을 어떻게 고르는지 직접 들여다봅니다.
2. 프롬프트와 채팅 템플릿으로 모델에게 일을 시키고, 작은 모델의 한계를 정직하게 확인합니다.
3. 모델이 메모리를 얼마나 먹는지 계산하고, 줄이는 방법(양자화)을 맛봅니다.
4. 모델 전체가 아니라 아주 작은 부분만 학습하는 LoRA를 손으로 만든 뒤, 그 방법으로 모델을 우리 데이터에 맞게 고쳐 봅니다.

이 장에서 쓰는 모델은 알리바바 Qwen 팀이 공개한 **Qwen2.5-0.5B-Instruct**입니다. 라이선스는 Apache-2.0이고(모델 카드: huggingface.co/Qwen/Qwen2.5-0.5B-Instruct), 로그인이나 동의 절차 없이 받을 수 있어요. 이름의 0.5B는 파라미터가 약 5억(0.5 billion) 개라는 뜻이고, Instruct는 질문에 답하도록 추가 학습한 판이라는 뜻입니다. 노트북에서 돌릴 수 있는 가장 작은 축에 드는 LLM이에요.

Q

Q. 무엇을 설치해야 하나요? 4장의 PyTorch에 더해 `uv add transformers peft`를 실행하세요. `transformers`는 사전학습 모델을 불러오고 쓰는 Hugging Face의 라이브러리(13장에서 처음 썼습니다), `peft`는 LoRA 같은 "적은 파라미터만 학습하는" 방법을 모아 둔 라이브러리예요. `peft`를 설치하면 함께 쓰는 `accelerate`도 따라 설치됩니다. 이 장의 예제는 파이썬 3.12, PyTorch 2.14.1, transformers 5.20.2, peft 0.21.2, accelerate 1.15.0에서 실행해 확인했습니다. 모델 파일은 처음 실행할 때 약 1GB를 내려받아 사용자 폴더의 캐시(`~/.cache/huggingface`)에 저장하고, 그다음부터는 다시 받지 않아요.

14.1 사전학습 언어 모델 불러오기

이름 하나로 모델과 토크나이저 받기

Hugging Face 허브에는 모델이 "만든 곳/모델 이름" 형식의 이름으로 올라 있습니다. `transformers`는 이 이름만 주면 설정 파일과 가중치를 내려받아 알맞은 PyTorch 모델을 만들어 줘요. 언어 모델은 글을 왼쪽에서 오른쪽으로 이어 쓰는 모델이라 `AutoModelForCausalLM`("앞만 보고 다음을 맞히는 언어 모델")으로 불러옵니다.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```
이름 = "Qwen/Qwen2.5-0.5B-Instruct"                # Hugging Face에 올라 있는 모델 이름
토크나이저 = AutoTokenizer.from_pretrained(이름)     # 글자 ↔ 번호 변환기
모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32)

print(type(모델).__name__)
print(f"파라미터: {sum(p.numel() for p in 모델.parameters()):,}개")
설정 = 모델.config
print("층 수:", 설정.num_hidden_layers, "/ 은닉 크기:", 설정.hidden_size,
      "/ 어텐션 헤드:", 설정.num_attention_heads)
print("어휘 크기:", 설정.vocab_size, "/ 한 번에 읽는 최대 길이:", 설정.max_position_embeddings)
print(isinstance(모델, torch.nn.Module))           # 6장의 nn.Module 그대로
```

```
Qwen2ForCausalLM
파라미터: 494,032,768개
층 수: 24 / 은닉 크기: 896 / 어텐션 헤드: 14
어휘 크기: 151936 / 한 번에 읽는 최대 길이: 32768
True
```

파라미터가 494,032,768개, 약 4억 9천만 개입니다. 6장 6.11절의 FashionMNIST 분류기가 235,146개였으니 2천 배쯤 큰 모델이에요. 설정(`config`)을 읽어 보면 구조가 보입니다. 트랜스포머 블록 24개를 쌓았고, 토큰 하나를 숫자 896개로 표현하며, 어텐션 헤드는 14개예요. 블록, 은닉 크기, 헤드는 모두 12장에서 직접 만든 부품의 이름 그대로입니다. 크기만 다를 뿐 같은 설계도라는 뜻이죠.

어휘 크기 151,936은 이 모델이 아는 토큰의 종류 수입니다. 마지막 층이 토큰마다 점수를 하나씩 내니, 다음 토큰을 고를 때마다 15만 개가 넘는 후보 중에서 하나를 고르는 셈이에요. 최대 길이 32,768은 한 번에 읽을 수 있는 토큰 수의 한계입니다. 마지막 줄의 `True`는 이 거대한 모델도 결국 6장에서 배운 `nn.Module`이라는 확인이에요. `parameters()`, `eval()`, `.to(장치)`, `state_dict()`가 모두 그대로 통합니다.

글자를 번호로: 토크나이저

모델은 글자를 직접 읽지 못하고 번호만 받습니다. 글을 조각(토큰)으로 자르고 번호를 붙이는 일은 모델과 짝을 이루는 **토크나이저**가 해요. 자르는 방법은 13장에서 자세히 다뤘으니 여기서는 이 모델이 한국어를 어떻게 자르는지만 확인하겠습니다.

```
from transformers import AutoTokenizer

토크나이저 = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-0.5B-Instruct")

문장 = "파이썬은 재미있다"
번호 = 토크나이저.encode(문장)
print(번호)
print([토크나이저.decode([n]) for n in 번호])      # 번호 하나하나가 어떤 조각인지
print(토크나이저.decode(번호))                      # 번호를 다시 글자로

# 같은 뜻의 문장, 토큰은 몇 개일까?
for 글 in ["오늘 날씨가 정말 좋네요.", "The weather is really nice today."]:
    print(f"{len(글):2}글자 → {len(토크나이저.encode(글)):2}토큰: {글}")
```

```
[126793, 12802, 144773, 33704, 129242, 56039, 13165, 19946]
['파', '이', '썬', '은', ' 재', '미', ' ', ' 다']
파이썬은 재미있다
14글자 → 10토큰: 오늘 날씨가 정말 좋네요.
33글자 →  7토큰: The weather is really nice today.
```

"파이썬은 재미있다"가 번호 8개가 됐습니다. 조각을 하나씩 글자로 되돌려 보면 "파", "이", "썬"처럼 거의 한 글자씩 잘렸고, "있"은 깨진 조각(�) 두 개에 나뉘어 들어갔는데 둘째 조각은 "다"와 한 토큰이 됐어요. 이 토크나이저는 글자를 컴퓨터 내부 표현인 **바이트**까지 쪼갤 수 있게 만들어져서, 어휘에 통째로 들어 있지 않은 글자는 바이트 단위 조각으로 나눕니다. 조각 하나만 글자로 바꾸면 깨져 보이지만, 번호를 모두 이어서 되돌리면 원래 문장이 그대로 나와요.

아래 두 줄은 같은 뜻의 문장입니다. 한국어는 14글자에 토큰 10개, 영어는 33글자에 토큰 7개가 들었어요. 영어는 단어 하나가 토큰 하나인 경우가 많지만, 한국어는 한두 글자마다 토큰이 하나씩 필요합니다. 모델이 학습할 때 영어 글을 훨씬 많이 봐서 영어 조각을 크게 만들어 두었기 때문이에요. 같은 내용을 말하는 데 토큰이 더 필요하다는 건, 한국어가 같은 길이 한도 안에 덜 들어가고 생성도 더 오래 걸린다는 뜻입니다. 14.5절에서 이 이야기를 다시 합니다.

14.2 다음 토큰 예측: LLM이 하는 일은 하나뿐

모델이 내는 것은 점수표

번호를 모델에 넣으면 무엇이 나올까요? 6장의 분류 모델이 클래스마다 점수(로짓)를 냈던 것처럼, 언어 모델은 **어휘에 있는 토큰마다** 점수를 냅니다.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

이름 = "Qwen/Qwen2.5-0.5B-Instruct"
토크나이저 = AutoTokenizer.from_pretrained(이름)
모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32).eval()

입력 = 토크나이저("대한민국의 수도는", return_tensors="pt")
print("입력 번호:", 입력["input_ids"])
with torch.no_grad():
    로짓 = 모델(**입력).logits                          # (배치, 토큰 수, 어휘 크기)
print("로짓 모양:", tuple(로짓.shape))

마지막 = 로짓[0, -1]                                   # 마지막 자리의 점수 = 다음 토큰 후보
확률 = torch.softmax(마지막, dim=-1)
상위 = 확률.topk(5)
for p, n in zip(상위.values, 상위.indices):
    print(f"{p.item():.3f}  {토크나이저.decode([n.item()])!r}")
```

```
입력 번호: tensor([[ 66845,  23573, 125496, 124785,  20401, 134013,  16560]])
로짓 모양: (1, 7, 151936)
0.110  '?\n'
0.089  ' 어디'
0.077  ' 무엇'
0.057  ' __'
0.052  ' 서울'
```

"대한민국의 수도는"이 토큰 7개로 들어갔고, 로짓 모양은 `(1, 7, 151936)`입니다. 4장 4.2절에서 권한 대로 자리마다 이름을 붙여 읽으면 "문장 1개, 토큰 7자리, 자리마다 어휘 151,936개의 점수"예요. 언어 모델은 **모든 자리에서 동시에** "이 자리 다음에 올 토큰"의 점수를 냅니다. 학습할 때는 7자리 모두의 예측을 정답과 비교해 한꺼번에 배우고, 생성할 때는 마지막 자리의 점수만 쓰면 돼요. 12장에서 어텐션 마스크로 뒤쪽을 가렸던 이유가 바로 이것입니다.

마지막 자리의 점수를 6장 6.2절의 소프트맥스로 확률로 바꾸고 4장 4.6절의 `topk`로 상위 다섯 개를 뽑았습니다. 1등은 `'?\n'`(물음표와 줄바꿈)이고, ' 어디', ' 무엇'이 뒤를 잇고, ' 서울'은 5등으로 5.2%입니다. 의외죠? 모델은 "대한민국의 수도는"을 **질문의 앞부분**으로 읽은 거예요. 인터넷에는 "대한민국의 수도는?"으로 끝나는 퀴즈 문장이 많으니까요. 모델은 사실을 묻는 게 아니라 "이런 글 다음에는 보통 무엇이 오는가"를 답합니다. 이 차이를 기억해 두면 이 장 후반의 이상한 대답들이 이해가 됩니다.

생성 루프를 직접 짜 보기

그럼 문장은 어떻게 길게 만들까요? 다음 토큰 하나를 고르고, 그것을 입력 뒤에 붙여서 다시 모델에 넣고, 또 하나를 고르기를 되풀이하면 됩니다. 휴대폰 자판에서 추천 단어를 계속 누르는 것과 똑같아요.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

이름 = "Qwen/Qwen2.5-0.5B-Instruct"
토크나이저 = AutoTokenizer.from_pretrained(이름)
모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32).eval()

번호 = 토크나이저("대한민국의 수도는", return_tensors="pt")["input_ids"]
시작_길이 = 번호.shape[1]

# 직접 만든 생성 루프: 가장 점수가 높은 토큰을 골라 뒤에 붙이고 다시 넣기
with torch.no_grad():
    for _ in range(15):
        다음 = 모델(번호).logits[0, -1].argmax()        # 탐욕(greedy): 1등만 고른다
        번호 = torch.cat([번호, 다음.view(1, 1)], dim=1)
직접 = 토크나이저.decode(번호[0, 시작_길이:])
print(repr(직접))

# 같은 일을 generate 한 줄로 (반복 억제 기본값 1.1을 꺼서 직접 만든 루프와 똑같은 규칙으로)
입력 = 토크나이저("대한민국의 수도는", return_tensors="pt")
with torch.no_grad():
    결과 = 모델.generate(**입력, max_new_tokens=15, do_sample=False, repetition_penalty=1.0)
자동 = 토크나이저.decode(결과[0, 시작_길이:])
print(repr(자동))
```

```
print("같은가:", 직접 == 자동)
```

```
'?\nA. 서울\nB. 인천\nC. 대구\n'
'?\nA. 서울\nB. 인천\nC. 대구\n'
같은가: True
```

15번 반복하면서 매번 1등 토큰(`argmax`)을 골라 4장의 `torch.cat`으로 이어 붙였습니다. 매번 가장 점수가 높은 것만 고르는 방법을 **탐욕(greedy) 디코딩**이라고 불러요. 결과를 보면 앞에서 확인한 1등 `'?\n'`을 고른 뒤, 정말로 객관식 퀴즈를 만들어 냈습니다. "A. 서울"이 정답 자리에 있긴 하네요.

아래의 `generate` 한 줄은 같은 일을 하는 지름길이고, 결과가 글자 하나까지 같습니다. `repetition_penalty=1.0`은 14.3절에서 볼 이 모델의 기본값 하나(반복 억제)를 꺼서, 직접 짠 루프와 규칙을 똑같이 맞춘 거예요. 다만 `generate`는 훨씬 효율적으로 돌아요. 직접 짠 루프는 토큰을 하나 붙일 때마다 문장 전체를 처음부터 다시 계산하지만, `generate`는 앞 토큰들의 어텐션 계산 결과(키와 값)를 저장해 두었다가 새 토큰 하나만 계산합니다. 이 저장소를 **KV 캐시**라고 하는데, 14.6절에서 이것이 메모리를 얼마나 쓰는지 계산해 봐요.

14.3 디코딩: 다음 토큰을 고르는 방법

온도: 확률 분포를 날카롭게, 또는 평평하게

탐욕 디코딩은 언제나 같은 답을 냅니다. 사실을 물을 때는 좋지만, 이야기나 아이디어처럼 다양한 답이 필요할 때는 지루하죠. 그래서 확률대로 **뽑는(sampling)** 방법을 쓰는데, 이때 가장 먼저 만지는 손잡이가 **온도(temperature)** 입니다. 소프트맥스를 하기 전에 점수를 온도로 나누는 단순한 계산이에요.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

이름 = "Qwen/Qwen2.5-0.5B-Instruct"
토크나이저 = AutoTokenizer.from_pretrained(이름)
모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32).eval()

입력 = 토크나이저("점심 메뉴로 추천하는 음식은", return_tensors="pt")
```

```
with torch.no_grad():
    로짓 = 모델(**입력).logits[0, -1]

후보 = 로짓.topk(5).indices                          # 원래 점수 기준 상위 5개 후보
print("후보:", [토크나이저.decode([n]) for n in 후보.tolist()])
for 온도 in [0.5, 1.0, 2.0]:
    확률 = torch.softmax(로짓 / 온도, dim=-1)        # 점수를 온도로 나눈 뒤 확률로
    print(f"온도 {온도}: 상위 5개 {[round(p, 3) for p in 확률[후보].tolist()]}"
          f"  나머지 합 {1 - 확률[후보].sum().item():.3f}")
```

```
후보: [' 무엇', '?', ' 다음과', " '", ' 어떤']
온도 0.5: 상위 5개 [0.967, 0.016, 0.004, 0.002, 0.001]  나머지 합 0.010
온도 1.0: 상위 5개 [0.414, 0.054, 0.027, 0.016, 0.016]  나머지 합 0.473
온도 2.0: 상위 5개 [0.014, 0.005, 0.004, 0.003, 0.003]  나머지 합 0.971
```

"점심 메뉴로 추천하는 음식은" 다음 토큰의 상위 다섯 후보입니다. 온도 1.0이 모델이 원래 낸 확률이에요. 1등 ' 무엇'이 41.4%이고, 상위 다섯 개 밖의 나머지 15만여 개가 모두 합쳐 47.3%를 나눠 갖습니다.

- **온도를 낮추면(0.5)** 점수 차이가 두 배로 벌어져서 1등이 96.7%를 가져갑니다. 거의 탐욕 디코딩처럼 늘 같은 답을 내게 돼요. 온도를 0에 가깝게 하면 탐욕과 같아집니다.
- **온도를 높이면(2.0)** 점수 차이가 반으로 줄어 분포가 평평해집니다. 1등도 1.4%뿐이고, 나머지가 97.1%예요. 15만 개 중 아무거나 뽑힐 수 있다는 뜻이라, 엉뚱한 토큰이 자주 섞입니다.

온도는 이렇게 "모델이 자신 있는 쪽을 얼마나 믿을지"를 정하는 손잡이입니다.

top-k와 top-p: 엉뚱한 후보는 아예 빼기

온도가 1이어도 문제가 남습니다. 확률이 아주 낮은 후보 수만 개를 모두 합치면 큰 확률이 되기 때문이에요. 앞에서 "나머지 합"이 47.3%였죠. 뽑을 때마다 그중 엉뚱한 것이 걸릴 수 있습니다. 그래서 뽑기 전에 후보를 추리는 두 가지 방법을 함께 씁니다. 원리를 보기 쉽게 후보 여섯 개짜리 예제로 직접 구현해 보겠습니다.

```
import torch

# 다음 토큰 후보 6개의 확률 (예제용 숫자)
```

```
후보 = ["김밥", "라면", "비빔밥", "떡볶이", "냉면", "우주선"]
확률 = torch.tensor([0.50, 0.20, 0.12, 0.08, 0.06, 0.04])


def top_k_거르기(확률: torch.Tensor, k: int) -> torch.Tensor:
    """확률이 높은 k개만 남기고 다시 합이 1이 되게 맞춘다."""
    남길 = torch.zeros_like(확률)
    값, 번호 = 확률.topk(k)
    남길[번호] = 값
    return 남길 / 남길.sum()


def top_p_거르기(확률: torch.Tensor, p: float) -> torch.Tensor:
    """높은 순서로 더해 가다 누적 확률이 p에 닿을 때까지만 남긴다."""
    정렬값, 정렬번호 = 확률.sort(descending=True)
    누적 = 정렬값.cumsum(dim=0)
    버릴 = (누적 - 정렬값) >= p                    # 앞까지의 합이 이미 p 이상이면 버린다
    정렬값[버릴] = 0.0
    남길 = torch.zeros_like(확률).scatter(0, 정렬번호, 정렬값)
    return 남길 / 남길.sum()


print("top-k=3 :", [round(v, 3) for v in top_k_거르기(확률, 3).tolist()])
print("top-p=0.85:", [round(v, 3) for v in top_p_거르기(확률, 0.85).tolist()])

생성기 = torch.Generator().manual_seed(0)
for 이름, 분포 in [("그대로", 확률), ("top-p=0.85", top_p_거르기(확률, 0.85))]:
    뽑기 = torch.multinomial(분포, 1000, replacement=True, generator=생성기)
    횟수 = torch.bincount(뽑기, minlength=6)
    print(f"{이름:10} 1000번 뽑기:", dict(zip(후보, 횟수.tolist())))
```

```
top-k=3 : [0.61, 0.244, 0.146, 0.0, 0.0, 0.0]
top-p=0.85: [0.556, 0.222, 0.133, 0.089, 0.0, 0.0]
그대로        1000번 뽑기: {'김밥': 500, '라면': 206, '비빔밥': 125, '떡볶이': 59, '냉면': 55, '우
↪ 주선': 55}
```

```
top-p=0.85 1000번 뽑기: {'김밥': 566, '라면': 228, '비빔밥': 117, '떡볶이': 89, '냉면': 0, '우주
↪ 선': 0}
```

- **top-k**는 "확률이 높은 k개만 남긴다"입니다. k=3이면 김밥·라면·비빔밥만 남고, 셋의 합이 1이 되게 다시 나눠서 김밥이 0.61이 됐어요.
- **top-p**(nucleus sampling이라고도 해요)는 "높은 순서로 더해서 누적 확률이 p에 닿을 때까지만 남긴다"입니다. 0.50, 0.70, 0.82까지는 아직 0.85에 못 미치니 떡볶이까지 넷이 남고, 냉면부터는 앞까지의 합이 이미 0.90이라 버려졌어요. 코드에서는 4장의 `sort`로 정렬하고, `cumsum`(누적 합)으로 더해 가고, `scatter`로 원래 자리에 되돌려 놓았습니다.

두 방법의 차이는 "몇 개 남길지"를 고정하느냐, 분포 모양에 맡기느냐입니다. 모델이 확신할 때(1등이 0.9쯤)는 top-p가 한두 개만 남기고, 헷갈릴 때는 여러 개를 남겨요. 그래서 실무에서는 top-p를 더 많이 씁니다.

마지막 두 줄은 `torch.multinomial`로 1000번 뽑아 센 결과입니다. 그대로 뽑으면 확률 4%짜리 "우주선"도 55번 나와요. 점심 메뉴를 묻는데 1000번 중 55번은 우주선이라고 답하는 셈이죠. top-p로 거르면 우주선과 냉면은 한 번도 나오지 않습니다. 4장의 `torch.Generator`로 뽑기 순서를 고정해서 몇 번을 실행해도 같은 숫자가 나와요.

generate로 비교하기

실제 모델에서 세 가지 방법을 나란히 돌려 보겠습니다. 방법마다 씨앗을 두 번 바꿔 봐요.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

이름 = "Qwen/Qwen2.5-0.5B-Instruct"
토크나이저 = AutoTokenizer.from_pretrained(이름)
모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32).eval()
입력 = 토크나이저("비 오는 날 창밖을 보며", return_tensors="pt")
길이 = 입력["input_ids"].shape[1]

# 모델의 기본값(top_k=20, top_p=0.8, 반복 억제 1.1)이 끼어들지 않게 손잡이를 모두 직접 적는다
끄기 = dict(top_k=0, top_p=1.0, repetition_penalty=1.0)
설정들 = {
    "탐욕": dict(do_sample=False, repetition_penalty=1.0),
    "온도 0.7, top-p 0.9": dict(끄기, do_sample=True, temperature=0.7, top_p=0.9),
```

```
    "온도 1.5": dict(끄기, do_sample=True, temperature=1.5),
}
for 설명, 옵션 in 설정들.items():
    for 씨앗 in [1, 2]:
        torch.manual_seed(씨앗)
        결과 = 모델.generate(**입력, max_new_tokens=25, **옵션)
        글 = 토크나이저.decode(결과[0, 길이:], skip_special_tokens=True)
        print(f"[{설명} / 씨앗 {씨앗}] {글!r}")
```

```
[탐욕 / 씨앗 1] ', 그는 놀라운 일들을 보았습니다. 그는 놀라운 일들을 보았'
[탐욕 / 씨앗 2] ', 그는 놀라운 일들을 보았습니다. 그는 놀라운 일들을 보았'
[온도 0.7, top-p 0.9 / 씨앗 1] ', 이어질 수 없는 우주의 진화와 재생의 문화가 있습니다. 2012년'
[온도 0.7, top-p 0.9 / 씨앗 2] ' "오늘 밤은 달빛이 매력적이고, 움직이는 생명체들의 수요를'
[온도 1.5 / 씨앗 1] ' 목야()})\n{\\hangfmt}`, ห้องที่ 관});\n\n\n.perform_pipe("DMETHOD 皋 design
↪ score efficiently prioritize detail activity cap'
[온도 1.5 / 씨앗 2] ' 통해 弼 t åt reacts bc 달 에Congress 리 Toni compens forever منح Ted ǀ Failed
↪ doğ의 buz Accessed быстро כל fullWidth'
```

- **탐욕**은 씨앗과 상관없이 두 번 모두 같은 문장입니다. 뽑기를 하지 않으니 씨앗이 할 일이 없죠. 게다가 "그는 놀라운 일들을 보았습니다"를 벌써 되풀이하기 시작했어요. 매번 1등만 고르면 한번 들어선 흐름에서 빠져나오지 못하는데, 작은 모델의 탐욕 디코딩에서 흔히 보이는 모습입니다.
- **온도 0.7, top-p 0.9**는 씨앗마다 다른 문장이 나오면서도 한국어 문장의 꼴은 유지합니다. 이야기나 대화에 흔히 쓰는 설정이에요.
- **온도 1.5**(`top_k=0`, `top_p=1.0`으로 후보 거르기를 모두 끈 상태)는 처음 한두 토큰 뒤로 코드 조각과 태국어, 베트남어, 아랍어, 러시아어, 중국어, 영어가 뒤섞인 글자 더미가 됐습니다. 평평해진 분포에서 엉뚱한 토큰이 한 번 뽑히면, 모델은 그 엉뚱한 글을 이어 쓰려고 또 엉뚱한 토큰을 고르게 돼요. 한 번 미끄러지면 계속 미끄러지는 겁니다.

`generate`는 `do_sample=False`이면 탐욕, `True`이면 샘플링이고, `temperature`, `top_k`, `top_p`로 손잡이를 정합니다. 무엇을 정하지 않으면 모델과 함께 배포된 기본값(`generation_config.json`)을 따르는데, 이 모델의 기본값은 `do_sample=True, temperature=0.7, top_k=20, top_p=0.8`에, 이미 나온 토큰의 점수를 깎아 같은 말을 덜 되풀이하게 하는 **반복 억제** `repetition_penalty=1.1`까지 켜져 있어요(`print(모델.generation_config)`로 확인할 수 있습니다). 그래서 이 예제는 쓰지 않을 손잡이까지 `끄기`로 모두 직접 적었습니다. 결과를 비교할 때는 이렇게 직접 적어 주는 것이 안전해요.

다만 위의 탐욕처럼 작은 모델은 반복 억제 없이 쓰면 같은 말을 되풀이하기 쉬워서, 14.4절부터 모델에게 답을 받을 때는 탐욕 디코딩에 기본값의 반복 억제(1.1)를 켜 두고 씁니다.

Q

Q. 씨앗을 고정했는데 다른 컴퓨터에서는 다른 문장이 나와요. 왜 그런가요? 샘플링은 확률 계산의 아주 작은 차이에도 다른 토큰을 뽑을 수 있습니다. CPU, CUDA, Apple MPS는 같은 계산도 더하는 순서나 내부 정밀도가 조금씩 달라서, 소수점 아래 끝자리가 달라지면 거기서부터 다른 문장으로 갈라져요. MPS에서는 같은 컴퓨터에서도 실행마다 달라질 수 있습니다. 그래서 이 장의 예제는 모두 CPU에서 실행했고, 본문의 출력은 이 책의 실행 환경(Apple M4 Max의 CPU)에서 나온 결과입니다. 여러분의 결과가 글자 하나까지 같지 않아도 이상한 게 아니에요. 4장의 장치_고르기()로 GPU에 올려도 코드는 그대로 돌아가지만, 그때 나오는 문장은 이 책과 다를 수 있다는 점만 기억해 두세요.

14.4 프롬프트와 채팅 템플릿

대화는 사실 하나의 긴 글입니다

지금까지는 "대한민국의 수도는"처럼 글의 앞부분을 주고 이어 쓰게 했습니다. 그런데 우리가 쓰는 챗봇은 질문을 하면 대답을 하죠. Instruct 모델은 어떻게 "이건 질문이니 대답해야 한다"를 알까요? 비밀은 **채팅 템플릿**에 있습니다. 대화를 정해진 표시가 붙은 하나의 글로 바꿔서 넣는 거예요.

```
from transformers import AutoTokenizer

토크나이저 = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-0.5B-Instruct")

대화 = [
    {"role": "system", "content": "너는 친절한 파이썬 선생님이야."},
    {"role": "user", "content": "리스트가 뭐예요?"},
]
글 = 토크나이저.apply_chat_template(대화, tokenize=False, add_generation_prompt=True)
print(글)
print("특수 토큰:", 토크나이저.convert_ids_to_tokens(토크나이저.encode("<|im_start|><|im_end|>")))
```

```
<|im_start|>system
너는 친절한 파이썬 선생님이야.<|im_end|>
```

```
<|im_start|>user
리스트가 뭐예요?<|im_end|>
<|im_start|>assistant

특수 토큰: ['<|im_start|>', '<|im_end|>']
```

대화는 1권 7장의 딕셔너리를 1권 6장의 리스트에 담은 형태입니다. 메시지마다 `role`(누가 말했나)과 `content`(무슨 말인가)가 있고, 역할은 셋이에요.

- **system**: 모델이 지켜야 할 태도나 규칙. 대화 전체에 걸리는 지시입니다.
- **user**: 사용자가 한 말.
- **assistant**: 모델이 한 말.

`apply_chat_template`은 이 리스트를 모델이 학습 때 본 형식의 글로 바꿉니다. `<|im_start|>`와 `<|im_end|>`는 메시지의 시작과 끝을 알리는 **특수 토큰**이에요. 마지막 줄의 `<|im_start|>assistant`는 `add_generation_prompt=True`가 붙여 준 것으로, "이제 모델이 말할 차례"라는 표시입니다. 모델은 여전히 다음 토큰을 이어 쓸 뿐인데, 이 표시 뒤에는 대답이 오고 대답이 끝나면 `<|im_end|>`가 온다는 것을 추가 학습에서 배웠어요. 그래서 질문에 대답하는 것처럼 보입니다.

템플릿은 모델마다 다릅니다. 다른 모델의 템플릿으로 글을 만들어 넣으면 모델이 엉뚱하게 반응해요. 그래서 문자열을 손으로 조립하지 말고 언제나 그 모델 토크나이저의 `apply_chat_template`을 쓰세요.

시스템 프롬프트로 태도 정하기

같은 질문에 시스템 지시만 바꿔 보겠습니다. 답만 꺼내 주는 작은 함수 `묻기`를 만들어 이 절 내내 씁니다.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

이름 = "Qwen/Qwen2.5-0.5B-Instruct"
토크나이저 = AutoTokenizer.from_pretrained(이름)
모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32).eval()


def 묻기(대화: list[dict], 최대: int = 50) -> str:
```

```
    """대화 기록을 채팅 템플릿에 맞춰 넣고 모델의 답만 돌려준다."""
    입력 = 토크나이저.apply_chat_template(대화, add_generation_prompt=True,
                                          return_tensors="pt", return_dict=True)
    with torch.no_grad():
        결과 = 모델.generate(**입력, max_new_tokens=최대, do_sample=False,
                             repetition_penalty=1.1)      # 모델 기본값의 반복 억제는 켜 둔다
    return 토크나이저.decode(결과[0, 입력["input_ids"].shape[1]:], skip_special_tokens=True)


질문 = {"role": "user", "content": "고양이에 대해 알려 줘."}
for 시스템 in ["답은 반드시 한 문장으로 해.",
               "모든 답을 JSON으로 해. 키는 '요약' 하나뿐이야.",
               "모든 답을 영어로만 해.",
               "너는 해적이야. 해적 말투로 답해."]:
    print(f"[{시스템}] {묻기([{'role': 'system', 'content': 시스템}, 질문])!r}")
```

```
[답은 반드시 한 문장으로 해.] '고양이는 사람의 식사와 휴식을 즐기며, 농담과 이야기를 나누는 동물입
↪ 니다.'
[모든 답을 JSON으로 해. 키는 '요약' 하나뿐이야.] '{\n  "요약": "고양이는 고유한 특징과 장점을 가지
↪ 고 있으며, 특히 농사와 식물栽培에 유명합니다. 고양이는 다양한 종류의 식물과 동물로'
[모든 답을 영어로만 해.] '고양이는 매우 유명한 동물입니다. 그들은 다양한 종류가 있으며, 특히 고유의
↪ 특징과 행동은 다음과 같습니다:\n\n1. 고양이는 빨간색 또는 검은색으로 주요 색'
[너는 해적이야. 해적 말투로 답해.] '고양이는 매우 건강하고 유쾌한 동물입니다. 고양이는 편안하게 웃
↪ 으며, 다양한 활동을 즐길 수 있습니다. 고양이는 높은 체중과 강력한'
```

`return_dict=True`를 주면 번호(`input_ids`)와 마스크(`attention_mask`)가 딕셔너리로 나오고, `**입력`으로 풀어 `generate`에 넘깁니다. 결과에는 질문까지 포함되어 있으니 질문 길이만큼 잘라 내고 대답만 글자로 바꿨어요.

결과는 반반입니다. "한 문장으로"는 지켰고, "JSON으로"도 `{"요약": ...` 꼴로 시작했어요(50토큰에서 잘라서 닫는 괄호는 보이지 않고, 내용이 이상하고 중간에 한자가 섞인 건 다음 절에서 이야기합니다). 그런데 "영어로만"과 "해적 말투로"는 무시했습니다. 작은 모델은 학습 때 자주 본 종류의 지시(길이, 형식)는 잘 따르지만, 질문 언어를 바꾸거나 역할극을 하라는 지시처럼 자기 습관과 부딪히는 지시는 놓치기 쉽습니다. 시스템 프롬프트는 부탁이지 명령이 아니에요. 큰 모델일수록 잘 따르지만, 어떤 모델이든 **결과를 확인하는 코드**를 따로 두는 습관이 필요합니다.

예시를 보여 주기: few-shot

말로 설명하는 대신 예시를 보여 주면 어떨까요? 상품 후기를 “긍정”과 “부정”으로 나누는 일을 시켜 보겠습니다. 예시는 **이미 주고받은 대화처럼** 넣는 것이 요령이에요.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

이름 = "Qwen/Qwen2.5-0.5B-Instruct"
토크나이저 = AutoTokenizer.from_pretrained(이름)
모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32).eval()


def 묻기(대화: list[dict], 최대: int) -> str:
    """대화 기록을 채팅 템플릿에 맞춰 넣고 모델의 답만 돌려준다."""
    입력 = 토크나이저.apply_chat_template(대화, add_generation_prompt=True,
                                       return_tensors="pt", return_dict=True)
    with torch.no_grad():
        결과 = 모델.generate(**입력, max_new_tokens=최대, do_sample=False,
                           repetition_penalty=1.1)      # 모델 기본값의 반복 억제는 켜 둔다
    return 토크나이저.decode(결과[0, 입력["input_ids"].shape[1]:], skip_special_tokens=True)


# 예시 두 개를 "이미 주고받은 대화"처럼 넣어 둔다 (예제용 후기)
예시 = [
    {"role": "system", "content": "상품 후기를 읽고 '긍정' 또는 '부정' 한 단어로만 답해."},
    {"role": "user", "content": "배송이 빠르고 품질도 좋아요."},
    {"role": "assistant", "content": "긍정"},
    {"role": "user", "content": "한 번 빨았더니 색이 다 빠졌어요."},
    {"role": "assistant", "content": "부정"},
]
후기들 = ["가격 대비 정말 만족합니다.", "사이즈가 너무 작아서 반품했어요.",
        "포장은 꼼꼼했는데 맛이 별로네요.", "생각보다 튼튼하고 예뻐요."]
for 후기 in 후기들:
    예시없이 = 묻기([{"role": "user", "content": f"다음 후기는 긍정이야 부정이야? {후기}"}], 20)
    예시있게 = 묻기(예시 + [{"role": "user", "content": 후기}], 5)
    print(f"{후기}\n  예시 없이: {예시없이!r}\n  예시 두 개: {예시있게!r}")
```

```
가격 대비 정말 만족합니다.
  예시 없이: '물론입니다! "가격 대비 정말 만족합니다."는 긍정적인 표현'
  예시 두 개: '긍정'
사이즈가 너무 작아서 반품했어요.
  예시 없이: '죄송합니다, 저는 인공지능으로서 사이트를 운영하고 있습니다. 하지만 개인'
  예시 두 개: '부정'
포장은 꼼꼼했는데 맛이 별로네요.
  예시 없이: '물론입니다! "다음 후기는 긍정이야 부정이야?"와'
  예시 두 개: '긍정'
생각보다 튼튼하고 예뻐요.
  예시 없이: '물론입니다! "다음 후기는 긍정이야"라는 문장은 "'
  예시 두 개: '긍정'
```

예시 없이 물으면 "물론입니다!"로 시작하는 긴 설명을 늘어놓거나, 아예 엉뚱한 답을 합니다. 답을 고르는 대신 대화를 하려 드는 거죠. 예시 두 개를 보여 주자 네 번 모두 딱 한 단어로 답했고, 그중 셋을 맞혔어요. 이렇게 예시 몇 개를 프롬프트에 넣는 방법을 **few-shot 프롬프팅**이라고 부릅니다(예시가 없으면 zero-shot). 모델의 가중치는 하나도 바뀌지 않았는데 행동이 바뀌었다는 점이 중요해요. 모델은 프롬프트 속 예시의 **형식**을 그대로 이어 쓰려고 합니다.

틀린 하나는 "포장은 꼼꼼했는데 맛이 별로네요"입니다. 앞은 칭찬, 뒤는 불만이라 "~는데" 뒤를 읽어야 하는 문장이에요. 작은 모델은 이런 문장에서 쉽게 틀립니다. 이런 분류를 제대로 하려면 13장처럼 분류 모델을 데이터로 미세조정하는 편이 훨씬 정확해요.

모델은 대화를 기억하지 않습니다

챗봇이 앞에서 한 말을 기억하는 것처럼 보이는 건 어떻게 된 걸까요?

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

이름 = "Qwen/Qwen2.5-0.5B-Instruct"
토크나이저 = AutoTokenizer.from_pretrained(이름)
모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32).eval()


def 묻기(대화: list[dict], 최대: int = 40) -> str:
```

```
    """대화 기록을 채팅 템플릿에 맞춰 넣고 모델의 답만 돌려준다."""
    입력 = 토크나이저.apply_chat_template(대화, add_generation_prompt=True,
                                          return_tensors="pt", return_dict=True)
    with torch.no_grad():
        결과 = 모델.generate(**입력, max_new_tokens=최대, do_sample=False,
                             repetition_penalty=1.1)      # 모델 기본값의 반복 억제는 켜 둔다
    return 토크나이저.decode(결과[0, 입력["input_ids"].shape[1]:], skip_special_tokens=True)


기록 = [{"role": "user", "content": "내 이름은 민준이야."}]
기록.append({"role": "assistant", "content": 묻기(기록)})
print("첫 답:", 기록[-1]["content"])

새_질문 = {"role": "user", "content": "내 이름이 뭐라고 했지?"}
print("기록 없이:", 묻기([새_질문]))
print("기록 함께:", 묻기(기록 + [새_질문]))
print("모델에 들어간 메시지 수:", len(기록 + [새_질문]))
```

```
첫 답: 안녕하세요! 저는 AI 어시스턴트인 Qwen입니다. 어떻게 도와드릴까요?
기록 없이: 저는 "Qwen"라는 이름을 가지고 있습니다. 이 이름은 "Quantum Neural Network"의 약자로, AI
↪ 분야에서 중요한 역할을 하는 인공지능
기록 함께: 당신의 이름은 "민준"이라고 합니다.
모델에 들어간 메시지 수: 3
```

첫 답은 인사만 하고 이름은 따라 말하지 않았네요. 그다음이 중요합니다. 새 질문만 보내면 모델은 민준이라는 이름을 모른 채, 자기 이름이 "Qwen"이라며 "Quantum Neural Network"의 약자라는 지어낸 설명까지 덧붙였어요(실제로 그런 뜻이 아닙니다). 앞의 대화를 리스트에 쌓아 **함께 보내야** 비로소 "민준"이라고 답합니다.

모델 자체에는 기억 장치가 없습니다. 매번 처음 보는 글을 받아 다음 토큰을 이어 쓸 뿐이에요. 챗봇이 대화를 기억하는 것처럼 보이는 건, 프로그램이 지금까지의 대화를 모두 이어 붙여 매번 다시 넣어 주기 때문입니다. 그래서 대화가 길어질수록 넣는 토큰이 늘어나 느려지고, 14.1절의 최대 길이를 넘으면 앞부분을 잘라 내야 해요.

14.5 작은 모델의 한계를 정직하게 보기

지금까지 이상한 대답을 몇 번 봤습니다. 이번에는 일부러 사실을 묻는 질문을 해서 한계를 똑바로 보겠습니다.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

이름 = "Qwen/Qwen2.5-0.5B-Instruct"
토크나이저 = AutoTokenizer.from_pretrained(이름)
모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32).eval()


def 묻기(질문: str, 최대: int = 45) -> str:
    """질문 하나를 채팅 형식으로 묻고 답을 한 줄로 돌려준다."""
    입력 = 토크나이저.apply_chat_template([{"role": "user", "content": 질문}],
                                          add_generation_prompt=True,
                                          return_tensors="pt", return_dict=True)
    with torch.no_grad():
        결과 = 모델.generate(**입력, max_new_tokens=최대, do_sample=False,
                             repetition_penalty=1.1)      # 모델 기본값의 반복 억제는 켜 둔다
    답 = 토크나이저.decode(결과[0, 입력["input_ids"].shape[1]:], skip_special_tokens=True)
    return 답.replace("\n", " ")


for 질문 in ["파이썬에서 리스트와 튜플의 차이를 한 문장으로 설명해 줘.",
           "In one sentence, what is the difference between a list and a tuple in Python?",
           "37 곱하기 24는 얼마야? 숫자만 답해.",
           "세종대왕이 만든 문자는?",
           "대한민국에서 가장 높은 산은?"]:
    print(f"Q. {질문}\nA. {묻기(질문)}\n")
```

```
Q. 파이썬에서 리스트와 튜플의 차이를 한 문장으로 설명해 줘.
A. 파이썬에서 리스트와 튜플의 차이는 다음과 같습니다:  1. 리스트는 원소가 변경될 수 있는 자료형입니
↪  다. 2. 튜플은 원소가 변경되지 않아서
```

```
Q. In one sentence, what is the difference between a list and a tuple in Python?
A. A list and a tuple in Python are both used to store collections of items, but they differ in
↪  their structure and operations: lists are mutable (changeable) and can be indexed, while tuples
↪  are immutable (unchangeable

Q. 37 곱하기 24는 얼마야? 숫자만 답해.
A. 37 곡 24은 105입니다.

Q. 세종대왕이 만든 문자는?
A. 세종대왕이 만든 문자는 "서울"입니다.  세종대왕은 19세기의 대한민국의 세종도시에 거주한 대한민국
↪  의 천년

Q. 대한민국에서 가장 높은 산은?
A. 대한민국에서 가장 높은 산은 울릉산입니다.  물론, 이는 대략적인 기준으로 생각해 보시면 좋을 것 같
↪  습니다:  1. 울릉
```

- **리스트와 튜플(한국어)**: 바꿀 수 있는지 없는지는 맞혔지만, "한 문장으로"라는 요청을 무시하고 목록을 시작했습니다.
- **리스트와 튜플(영어)**: 같은 질문을 영어로 하자 한 문장으로, 정확하게 답합니다.
- **곱셈**: 37 × 24는 888인데 105라고 했고, "곱하기"를 "곡"으로 잘못 썼어요.
- **세종대왕이 만든 문자**: 한글이 아니라 "서울"이라 하고, 세종대왕이 19세기 사람이라는 이야기까지 지어냈습니다.
- **가장 높은 산**: 한라산(약 1,950m)이 아니라 있지도 않은 "울릉산"이라고 답했어요.

이렇게 그럴듯한 말투로 사실이 아닌 내용을 지어내는 것을 **환각(hallucination)** 이라고 부릅니다. 이유는 앞에서 본 것들로 설명할 수 있어요.

- **모델이 작습니다.** 사실을 기억해 둘 숫자가 5억 개 남짓뿐입니다. 큰 모델이 수천억 개를 쓰는 것과 비교하면 세상의 지식을 담기에 턱없이 작아요. 문장을 매끄럽게 잇는 능력은 남아도, 정확한 사실은 자주 빠집니다.
- **한국어 글을 적게 봤습니다.** 이 모델은 주로 영어와 중국어 글로 학습했습니다. 모델 카드는 한국어를 포함해 29개가 넘는 언어를 지원한다고 밝히지만, 비중은 훨씬 작아요. 한국어 답에 한자가 섞여 나오는 것(14.4절의 "栽培")도, 같은 질문인데 영어 답이 훨씬 나은 것도 이 때문이에요. 14.1절에서 본 것처럼 한국어는 토큰도 더 잘게 쪼개져서, 같은 길이 안에 담기는 내용도 적습니다.
- **모델은 사실이 아니라 그럴듯함을 배웁니다.** 14.2절에서 "대한민국의 수도는" 다음에 물음표

> 를 1등으로 고른 것처럼, 모델의 목표는 처음부터 끝까지 "다음에 올 법한 토큰"입니다. 계산기처럼 곱셈을 하지 않고, 사전처럼 사실을 찾아보지 않아요.

그러면 이 모델은 쓸모가 없을까요? 그렇지 않습니다. 형식을 맞추고, 짧게 정리하고, 정해진 꼴로 답하는 일은 잘해요. 그리고 노트북에서 몇 분 안에 학습시켜 볼 수 있을 만큼 작다는 것이 이 장에서는 가장 큰 장점입니다. 같은 코드에 모델 이름만 큰 모델로 바꾸면 답의 질은 확 올라가지만, 그만큼 메모리가 필요해요. 라이선스도 다시 봐야 합니다. 같은 Qwen2.5 계열이라도 크기마다 라이선스가 달라서, 이 책을 쓸 때 모델 카드를 확인해 보니 0.5B·1.5B·7B·14B·32B는 Apache-2.0이지만 3B는 비상업 연구용으로만 쓸 수 있는 Qwen Research 라이선스, 72B는 월 사용자가 1억 명을 넘는 서비스라면 따로 허락을 받아야 하는 Qwen 라이선스였어요. 크기를 바꿀 때는 모델 카드의 라이선스부터 다시 확인하세요. 얼마나 필요한지 계산해 보겠습니다.

14.6 메모리 계산과 양자화 맛보기

내 노트북에 들어갈까?

4장 4.3절에서 "70억 × 4바이트 = 26.1GB"를 계산하며 7B 모델은 보통 노트북 GPU에 올라가지도 않는다고 했습니다. 이제 진짜 모델로 같은 계산을 해 봐요.

```
import torch
from transformers import AutoConfig, AutoModelForCausalLM

이름 = "Qwen/Qwen2.5-0.5B-Instruct"
모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32)
GB = 1024 ** 3

# 4장 4.3절: 메모리 = 칸 크기(element_size) × 칸 개수(numel)
바이트 = sum(p.element_size() * p.numel() for p in 모델.parameters())
print(f"float32로 올린 이 모델: {바이트 / GB:.2f} GB")
print("입력 임베딩과 출력층이 같은 가중치인가:",
      모델.get_input_embeddings().weight is 모델.get_output_embeddings().weight)

# 같은 계산을 크기별·자료형별로: 칸당 바이트 수만 바꾸면 된다
for 크기, 개수 in [("0.5B", 494_032_768), ("7B", 7_000_000_000), ("70B", 70_000_000_000)]:
    칸 = {"float32": 4, "bfloat16": 2, "int8": 1, "int4": 0.5}
    print(f"{크기:>4}: " + ", ".join(f"{k} {개수 * v / GB:6.1f}GB" for k, v in 칸.items()))
```

```
# 생성할 때 따로 쌓이는 메모리: KV 캐시 (토큰마다 층마다 키와 값을 저장)
설정 = AutoConfig.from_pretrained(이름)
헤드_크기 = 설정.hidden_size // 설정.num_attention_heads
토큰당 = 2 * 설정.num_hidden_layers * 설정.num_key_value_heads * 헤드_크기 * 2   # bfloat16 기준
print(f"키·값 헤드 {설정.num_key_value_heads}개, 헤드 크기 {헤드_크기}, 토큰당 {토큰당:,}바이트")
print(f"토큰 32,768개를 기억하면 KV 캐시 {토큰당 * 32_768 / GB:.2f} GB")
print(f"키·값 헤드가 쿼리 헤드처럼 {설정.num_attention_heads}개였다면 "
      f"{토큰당 * 설정.num_attention_heads // 설정.num_key_value_heads * 32_768 / GB:.2f} GB")
```

```
float32로 올린 이 모델: 1.84 GB
입력 임베딩과 출력층이 같은 가중치인가: True
0.5B: float32    1.8GB, bfloat16    0.9GB, int8    0.5GB, int4    0.2GB
  7B: float32   26.1GB, bfloat16   13.0GB, int8    6.5GB, int4    3.3GB
 70B: float32  260.8GB, bfloat16  130.4GB, int8   65.2GB, int4   32.6GB
키·값 헤드 2개, 헤드 크기 64, 토큰당 12,288바이트
토큰 32,768개를 기억하면 KV 캐시 0.38 GB
키·값 헤드가 쿼리 헤드처럼 14개였다면 2.62 GB
```

파라미터마다 `element_size() × numel()`을 더하니 1.84GB입니다. 4억 9천만 × 4바이트를 1024^3으로 나눈 값과 같아요. 두 번째 줄은 하나 짚어 둘 점입니다. 토큰 번호를 896개 숫자로 바꾸는 입력 임베딩과, 896개 숫자를 다시 15만 개 점수로 바꾸는 출력층이 **같은 가중치를 함께 쓰고** 있어요. 어휘가 15만 개라 이 행렬만 1억 3천만 개가 넘는데, 두 번 두지 않고 한 번만 둬서 아낀 겁니다. 작은 모델에서 자주 쓰는 방법이에요.

가운데 표는 같은 계산을 크기와 자료형만 바꿔 한 것입니다. 7B 모델은 `float32`로 26.1GB, `bfloat16`으로 13.0GB, 숫자 하나를 4비트(0.5바이트)에 담는 `int4`로는 3.3GB예요. 70B 모델도 `int4`라면 32.6GB까지 내려옵니다. 큰 모델을 노트북에서 돌린다는 이야기는 대부분 이렇게 **칸을 줄여서** 담는다는 뜻이에요.

마지막 세 줄은 모델 가중치 말고 따로 쌓이는 메모리, 14.2절의 **KV 캐시**입니다. 생성하는 동안 층마다, 토큰마다 키와 값을 저장하니 크기는 `2(키와 값) × 층 수 × 키·값 헤드 수 × 헤드 크기 × 칸 크기`예요. 이 모델은 토큰 하나에 12,288바이트, 최대 길이 32,768토큰을 다 채우면 0.38GB입니다. 그런데 어텐션 헤드는 14개인데 키·값 헤드는 2개뿐이죠? 쿼리 헤드 일곱 개가 키·값 헤드 하나를 함께 쓰는 설계라서 그래요. 이런 설계를 **그룹 쿼리 어텐션**(GQA, Grouped-Query Attention)이라고 부릅니다. 키·값 헤드가 14개였다면 2.62GB가 필요했을 테니 일곱 배를 아낀 셈입니다. 긴 글

을 다루는 모델에서는 가중치보다 이 캐시가 더 커지기도 해요.

bfloat16으로 절반만 쓰기

칸을 줄이면 답이 달라질까요? 가장 쉬운 방법인 `bfloat16`부터 비교해 보겠습니다. 불러올 때 `dtype`만 바꾸면 돼요.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

이름 = "Qwen/Qwen2.5-0.5B-Instruct"
토크나이저 = AutoTokenizer.from_pretrained(이름)
입력 = 토크나이저.apply_chat_template([{"role": "user", "content": "1 더하기 1은?"}],
                                      add_generation_prompt=True,
                                      return_tensors="pt", return_dict=True)
결과 = {}
for 자료형 in [torch.float32, torch.bfloat16]:
    모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=자료형).eval()
    with torch.no_grad():
        로짓 = 모델(**입력).logits[0, -1].float()
        생성 = 모델.generate(**입력, max_new_tokens=20, do_sample=False,
                             repetition_penalty=1.1)
    결과[자료형] = 로짓
    print(f"{str(자료형):15} {모델.get_memory_footprint() / 1024**3:.2f} GB  "
          f"{토크나이저.decode(생성[0, 입력['input_ids'].shape[1]:], skip_special_tokens=True)!r}")

차이 = (결과[torch.float32] - 결과[torch.bfloat16]).abs()
print(f"다음 토큰 점수 차이: 평균 {차이.mean():.3f}, 최대 {차이.max():.3f}")
print("1등 토큰이 같은가:", (결과[torch.float32].argmax() == 결과[torch.bfloat16].argmax()).item())
```

```
torch.float32   1.84 GB  '1 더하기 1은 2입니다.\n\n더하기 연산자는 두 수를 더합니다'
torch.bfloat16  0.92 GB  '1 더하기 1은 2입니다.\n\n더하기 연산자는 두 수를 더합니다'
다음 토큰 점수 차이: 평균 0.089, 최대 0.458
1등 토큰이 같은가: True
```

`get_memory_footprint()`로 잰 메모리가 1.84GB에서 0.92GB로 정확히 절반이 됐고, 생성된 문장은 똑같습니다. 다음 토큰 점수를 비교하면 평균 0.089, 큰 곳은 0.458까지 차이가 나지만, 1등 토큰은

같아요. 4장 4.3절에서 말한 대로 딥러닝은 끝자리의 작은 오차에 둔감합니다. 그래서 요즘 LLM은 대부분 `bfloat16`으로 배포되고 쓰입니다. 학습까지 16비트로 빠르게 하는 방법은 16장에서 다룹니다.

양자화 맛보기: 숫자를 정수 칸에 담기

`bfloat16`보다 더 줄이려면 실수를 **정수**로 바꿔 담습니다. 이것을 **양자화(quantization)** 라고 해요. 원리는 자에 눈금을 새기는 것과 같습니다. 행마다 가장 큰 절댓값을 찾아 그 값이 127이 되게 눈금 간격(배율)을 정하고, 모든 값을 가장 가까운 눈금에 맞춰 정수로 기록해요. 쓸 때는 정수에 배율을 곱해 실수로 되돌립니다. 그 원리를 직접 구현해서, 트랜스포머 층 안의 모든 선형층 가중치를 "정수로 바꿨다가 되돌린 값"으로 바꿔치기해 보겠습니다.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer


def 양자화_왕복(w: torch.Tensor, 비트: int) -> torch.Tensor:
    """행마다 가장 큰 절댓값을 기준으로 정수 칸에 맞춰 반올림했다가 다시 실수로 되돌린다."""
    최대정수 = 2 ** (비트 - 1) - 1                          # int8이면 127, int4면 7
    배율 = w.abs().amax(dim=1, keepdim=True) / 최대정수       # 행마다 하나 (keepdim: 4장 4.6절)
    정수 = (w / 배율).round().clamp(-최대정수, 최대정수)      # 이 정수만 저장하면 된다
    return 정수 * 배율


w = torch.tensor([[0.12, -0.50, 0.33, 0.07]])
print("원래  :", w)
print("int8  :", 양자화_왕복(w, 8))
print("int4  :", 양자화_왕복(w, 4))

이름 = "Qwen/Qwen2.5-0.5B-Instruct"
토크나이저 = AutoTokenizer.from_pretrained(이름)
문장 = ("파이썬은 배우기 쉬운 프로그래밍 언어입니다. "
        "많은 사람들이 데이터 분석과 인공지능에 파이썬을 씁니다.")
번호 = 토크나이저(문장, return_tensors="pt")["input_ids"]
for 비트 in [None, 8, 4]:
    모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32).eval()
    if 비트:
```

```
        with torch.no_grad():
            for 층 in 모델.model.layers.modules():          # 트랜스포머 층 안의 선형층만
                if isinstance(층, torch.nn.Linear):
                    층.weight.copy_(양자화_왕복(층.weight, 비트))
    with torch.no_grad():
        손실 = 모델(번호, labels=번호).loss.item()
    print(f"{'원래 float32' if 비트 is None else f'int{비트} 흉내':12} 손실 {손실:.3f}  "
          f"(다음 토큰 정답 확률의 기하평균 {torch.exp(torch.tensor(-손실)):.3f})")
```

```
원래   : tensor([[ 0.1200, -0.5000,  0.3300,  0.0700]])
int8   : tensor([[ 0.1181, -0.5000,  0.3307,  0.0709]])
int4   : tensor([[ 0.1429, -0.5000,  0.3571,  0.0714]])
원래 float32   손실 2.189  (다음 토큰 정답 확률의 기하평균 0.112)
int8 흉내      손실 2.200  (다음 토큰 정답 확률의 기하평균 0.111)
int4 흉내      손실 3.968  (다음 토큰 정답 확률의 기하평균 0.019)
```

먼저 숫자 네 개짜리 예제입니다. 가장 큰 절댓값이 0.5라서 `int8`이면 눈금 간격이 0.5/127, `int4`면 0.5/7입니다. `int8`은 0.12가 0.1181이 되는 정도로 거의 원래 값이지만, `int4`는 눈금이 열다섯 개(-7~7)뿐이라 0.12가 0.1429로 꽤 멀리 밀렸어요. 가장 큰 값 -0.5는 눈금 위에 정확히 있으니 둘 다 그대로입니다.

이것을 모델 전체에 적용하고 문장 하나의 손실(6장의 교차 엔트로피)을 쟀습니다. 괄호 안의 값은 손실을 확률로 되돌린 것으로, 정답 토큰에 평균적으로 얼마의 확률을 줬는지를 뜻해요. `int8`은 손실이 2.189에서 2.200으로 거의 그대로지만, `int4`는 3.968로 크게 나빠졌습니다. 정답 토큰에 주는 확률이 11%에서 2%로 떨어졌으니 문장을 제대로 이어 쓰기 어려운 수준이에요.

여기서는 원리만 보이려고 정수를 다시 실수로 되돌려 넣었으니 메모리는 줄지 않았습니다. 실제 양자화 도구는 정수와 배율만 저장하고 계산할 때 되돌려요. 그리고 `int4`가 망가진 건 행마다 배율을 **하나만** 썼기 때문입니다. 실제 4비트 양자화는 숫자 수십 개씩 작은 묶음마다 배율을 따로 두고, 결과가 덜 나빠지도록 눈금을 고르는 요령을 더해서 7B 모델도 쓸 만한 품질을 지켜요. 그 방법들과 모델을 실제로 작게 만들어 배포하는 과정은 19장에서 다룹니다.

Q

Q. 양자화하면 계산도 빨라지나요? 장비와 방법에 따라 다릅니다. 정수 계산을 빠르게 하는 하드웨어에서 정수 그대로 계산하면 빨라지고, 정수를 실수로 되돌려 계산하는 방식이면 오히려 조금 느려질 수도 있어요. 그래도 LLM 생성은 대개 "계산"보다 "가중치를

메모리에서 읽어 오는 시간"이 병목이라, 읽을 바이트가 줄어드는 것만으로 빨라지는 경우가 많습니다. 확실한 이득은 메모리예요. 들어가지 않던 모델이 들어가게 됩니다. 속도를 재는 법은 16장, 양자화 도구는 19장에서 다룹니다.

Q

14.7 LoRA의 아이디어: 고칠 부분만 작게

교과서를 다시 쓰지 말고 포스트잇을 붙이기

이제 모델을 우리 일에 맞게 고칠 차례입니다. 6장의 방법대로라면 5억 개 숫자 모두에 기울기를 구하고 Adam으로 고치면 되죠. 그런데 계산해 보면 만만치 않습니다. Adam은 숫자마다 기울기와 이동 평균 두 개를 더 들고 있어야 해서, `float32`라면 가중치 1.84GB에 더해 약 5.5GB가 더 필요해요. 7B 모델이면 100GB가 넘습니다. 고친 모델을 저장할 때도 매번 모델 전체 크기의 파일이 생기고요.

두꺼운 교과서에 틀린 곳이 몇 군데 있다고 책을 통째로 다시 쓰지는 않습니다. 고칠 곳에 포스트잇을 붙이죠. LoRA(Low-Rank Adaptation)가 바로 그 포스트잇입니다. 원래 가중치 `W`는 그대로 얼려 두고, 옆에 고칠 양 `ΔW`를 더하는데, 이 `ΔW`를 아주 작은 행렬 두 개의 곱 `B @ A`로 만듭니다.

작은 행렬 두 개로 큰 행렬 만들기

작은 행렬 두 개를 곱하면 어떤 행렬이 될까요? 4장 4.6절의 행렬 곱 모양 규칙부터 떠올려 봅시다.

```
import torch

# 세로 벡터 하나 × 가로 벡터 하나 = 표 전체 (외적)
a = torch.tensor([[1.0], [2.0], [3.0]])      # (3, 1)
b = torch.tensor([[10.0, 20.0, 30.0, 40.0]]) # (1, 4)
표 = a @ b                                    # (3, 4): 숫자 7개로 12개짜리 표를 만든다
print(표)
print("랭크:", torch.linalg.matrix_rank(표).item())

torch.manual_seed(0)
B, A = torch.randn(896, 8), torch.randn(8, 896)
print("(896, 8) @ (8, 896) 의 랭크:", torch.linalg.matrix_rank(B @ A).item())

# 가중치 하나를 통째로 고칠 때와 작은 행렬 두 개로 고칠 때의 숫자 개수
```

```
d = 896                                          # Qwen2.5-0.5B의 은닉 크기
print(f"통째로: {d * d:,}개")
for r in [1, 4, 8, 16, 64]:
    print(f"랭크 {r:2}: {r * (d + d):7,}개 ({r * (d + d) / (d * d):.1%})")
```

```
tensor([[ 10.,  20.,  30.,  40.],
        [ 20.,  40.,  60.,  80.],
        [ 30.,  60.,  90., 120.]])
랭크: 1
(896, 8) @ (8, 896) 의 랭크: 8
통째로: 802,816개
랭크  1:   1,792개 (0.2%)
랭크  4:   7,168개 (0.9%)
랭크  8:  14,336개 (1.8%)
랭크 16:  28,672개 (3.6%)
랭크 64: 114,688개 (14.3%)
```

`(3, 1) @ (1, 4)`는 `(3, 4)` 표가 됩니다. 숫자 7개로 숫자 12개짜리 표를 만든 거예요. 다만 결과를 보면 모든 줄이 첫 줄의 1배, 2배, 3배일 뿐입니다. 이렇게 "서로 독립인 줄이 몇 개인가"를 **랭크**(rank) 라고 해요. 이 표는 랭크가 1입니다. 마찬가지로 `(896, 8) @ (8, 896)`은 896 × 896 크기지만 랭크가 8을 넘을 수 없어요. 가운데의 8이 통로처럼 좁아서, 그 폭만큼만 정보가 지나가기 때문입니다.

LoRA는 "모델을 새 일에 맞게 고칠 때 필요한 변화는 이렇게 **랭크가 낮은** 행렬로도 충분하다"는 관찰에서 출발했습니다. 그러면 숫자가 크게 줄어요. 은닉 크기 896짜리 가중치 하나를 통째로 고치면 802,816개를 학습해야 하지만, 랭크 8이면 `8 × (896 + 896)` = 14,336개, 1.8%만 학습하면 됩니다.

LoRA 층을 손으로 만들기

6장의 `nn.Module`로 LoRA 층을 직접 만들어 보겠습니다. 원래 `nn.Linear`를 받아 얼리고, 옆에 `A`와 `B`를 붙여요.

```
import torch
from torch import nn
```

```
class LoRA선형층(nn.Module):
    """얼린 nn.Linear 옆에 작은 행렬 두 개(A, B)를 붙인 층."""

    def __init__(self, 원래: nn.Linear, r: int = 4, alpha: int = 8) -> None:
        super().__init__()
        self.원래 = 원래
        self.원래.requires_grad_(False)                # 원래 가중치는 얼린다 (6장 6.4절)
        self.A = nn.Parameter(torch.randn(r, 원래.in_features) * 0.01)   # 작은 무작위 값
        self.B = nn.Parameter(torch.zeros(원래.out_features, r))       # 0에서 시작
        self.배율 = alpha / r

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.원래(x) + (x @ self.A.T @ self.B.T) * self.배율


torch.manual_seed(0)
원래 = nn.Linear(64, 64)
층 = LoRA선형층(원래, r=4)
x = torch.randn(5, 64)
print("처음에는 원래 층과 출력이 같은가:", torch.allclose(층(x), 원래(x)))
for 이름, p in 층.named_parameters():
    print(f"{이름:14} {tuple(p.shape)!s:10} 학습함={p.requires_grad}")
학습 = sum(p.numel() for p in 층.parameters() if p.requires_grad)
전체 = sum(p.numel() for p in 층.parameters())
print(f"학습하는 숫자 {학습:,}개 / 전체 {전체:,}개")
```

```
처음에는 원래 층과 출력이 같은가: True
A              (4, 64)    학습함=True
B              (64, 4)    학습함=True
원래.weight      (64, 64)   학습함=False
원래.bias        (64,)      학습함=False
학습하는 숫자 512개 / 전체 4,672개
```

`forward`는 원래 층의 출력에 `x @ A.T @ B.T`(입력을 랭크 r의 좁은 통로로 줄였다가 다시 펴는 길)를 더합니다. 6장에서 본 `nn.Linear`의 계산 `x @ weight.T`와 같은 모양이에요. 눈여겨볼 곳은 세 군데입니다.

- **`B`는 0에서 시작합니다.** 그래서 처음에는 `B @ A`가 0이고, LoRA를 붙인 층의 출력이 원래 층과 똑같아요(`True`). 학습을 시작하는 순간 모델이 망가지지 않고, 원래 실력에서 출발해 조금씩 고쳐 나갑니다.
- **`A`는 작은 무작위 값입니다.** 둘 다 0이면 어떻게 될까요? 5장의 연쇄 법칙으로 따져 보면 `B`의 기울기에는 `A`가 곱해지고 `A`의 기울기에는 `B`가 곱해져서, 둘 다 0이면 기울기도 영원히 0이에요. 한쪽만 0으로 둬야 학습이 시작됩니다.
- **배율 `alpha / r`**: 랭크를 바꿔도 고치는 세기가 크게 달라지지 않도록 곱하는 숫자입니다. 보통 `alpha`를 `r`의 1~2배로 둬요.

이 작은 층에서도 학습하는 숫자는 512개, 전체 4,672개의 11% 정도입니다. 층이 클수록 이 비율은 훨씬 작아져요.

정말 낮은 랭크로 배워질까?

정답을 알고 있는 장난감 문제로 확인해 보겠습니다. 원래 층에 "랭크 2짜리 변화"를 더한 층을 정답으로 만들어 두고, LoRA로 그 변화를 배울 수 있는지 랭크를 바꿔 가며 봅니다.

```
import torch
from torch import nn


class LoRA선형층(nn.Module):
    """얼린 nn.Linear 옆에 작은 행렬 두 개(A, B)를 붙인 층 (앞 예제와 같음)."""

    def __init__(self, 원래: nn.Linear, r: int, alpha: int) -> None:
        super().__init__()
        self.원래 = 원래.requires_grad_(False)
        self.A = nn.Parameter(torch.randn(r, 원래.in_features) * 0.01)
        self.B = nn.Parameter(torch.zeros(원래.out_features, r))
        self.배율 = alpha / r

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.원래(x) + (x @ self.A.T @ self.B.T) * self.배율


torch.manual_seed(0)
원래 = nn.Linear(64, 64)
```

```
# 목표: 원래 가중치에 "랭크 2짜리 변화"를 더한 층 (예제용으로 만든 정답)
진짜_변화 = torch.randn(64, 2) @ torch.randn(2, 64) * 0.1
x = torch.randn(512, 64)
with torch.no_grad():
    y = x @ (원래.weight + 진짜_변화).T + 원래.bias

for r in [1, 2, 4]:
    torch.manual_seed(1)
    층 = LoRA선형층(원래, r=r, alpha=r)                    # 배율 1로 맞춰 비교
    옵티마이저 = torch.optim.Adam([층.A, 층.B], lr=0.01)
    for 단계 in range(1, 1501):
        손실 = nn.functional.mse_loss(층(x), y)
        옵티마이저.zero_grad()
        손실.backward()
        옵티마이저.step()
    배운_변화 = 층.배율 * 층.B @ 층.A
    오차 = (배운_변화 - 진짜_변화).norm() / 진짜_변화.norm()
    print(f"r={r}: 손실 {손실.item():.6f}, 배운 변화와 진짜 변화의 상대 오차 {오차.item():.3f}")

# 다 배운 뒤에는 W + 배율·(B @ A)를 미리 더해 보통 층 하나로 합칠 수 있다
with torch.no_grad():
    합친 = nn.Linear(64, 64)
    합친.weight.copy_(원래.weight + 층.배율 * 층.B @ 층.A)
    합친.bias.copy_(원래.bias)
    print("합친 층과 LoRA 층의 출력이 같은가:", torch.allclose(합친(x), 층(x), atol=1e-5))
```

```
r=1: 손실 0.502039, 배운 변화와 진짜 변화의 상대 오차 0.658
r=2: 손실 0.000000, 배운 변화와 진짜 변화의 상대 오차 0.000
r=4: 손실 0.000000, 배운 변화와 진짜 변화의 상대 오차 0.000
합친 층과 LoRA 층의 출력이 같은가: True
```

진짜 변화의 랭크가 2이니, 랭크 1인 LoRA는 손실이 0.502에서 더 내려가지 못하고 배운 변화도 진짜와 66%나 어긋납니다. 통로가 너무 좁은 거예요. 랭크 2와 4는 손실이 0이 되고, 배운 변화가 진짜 변화와 거의 정확히 같아졌습니다. 원래 가중치 4,096개는 한 번도 건드리지 않고, `A`와 `B`만 고쳐서 얻은 결과예요.

마지막 줄도 중요합니다. 다 배운 뒤에는 `W + 배율 × (B @ A)`를 미리 계산해 **보통 `nn.Linear` 하나**

로 합칠 수 있어요. 합친 층의 출력은 LoRA 층과 같습니다. 그래서 LoRA로 학습한 모델은 배포할 때 추가 계산이 전혀 없습니다. 이것이 LoRA가 널리 쓰이는 또 하나의 이유예요.

Q

Q. 랭크는 얼마로 정하면 되나요? 정답은 데이터로 정하지만, LLM 미세조정에서는 4~16에서 시작하는 경우가 많습니다. 말투나 답의 형식처럼 바꿀 것이 단순하면 작은 랭크로 충분하고, 새로운 일을 많이 가르칠수록 큰 랭크가 도움이 돼요. 다만 위 실험처럼 필요한 만큼을 넘으면 더 좋아지지 않고 학습할 숫자만 늘어납니다. 8장처럼 검증 데이터로 몇 가지를 비교해 고르세요.

14.8 peft로 LLM에 LoRA 붙이기

손으로 만든 LoRA선형층을 모델의 선형층 수백 개에 하나씩 끼워 넣을 수도 있지만, `peft` 라이브러리가 그 일을 해 줍니다. 어느 층에 붙일지, 랭크는 얼마인지 설정만 적으면 돼요.

```
import torch
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM, AutoTokenizer

이름 = "Qwen/Qwen2.5-0.5B-Instruct"
토크나이저 = AutoTokenizer.from_pretrained(이름)
모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32).eval()
입력 = 토크나이저("LoRA를 붙여도 처음에는 출력이 같을까?", return_tensors="pt")
with torch.no_grad():
    붙이기_전 = 모델(**입력).logits

설정 = LoraConfig(
    r=8,                                                    # 작은 행렬의 랭크
    lora_alpha=16,                                          # 배율 = lora_alpha / r = 2
    lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],  # 어텐션의 선형층 네 개
    task_type="CAUSAL_LM",
)
모델 = get_peft_model(모델, 설정)
모델.print_trainable_parameters()
```

```
층0 = 모델.base_model.model.model.layers[0].self_attn
print(층0.k_proj)

# 손으로 세어 보기: 층마다 r × (입력 + 출력)
손계산 = sum(8 * (층.in_features + 층.out_features)
           for 층 in [층0.q_proj.base_layer, 층0.k_proj.base_layer,
                      층0.v_proj.base_layer, 층0.o_proj.base_layer]) * 24
print(f"손으로 센 LoRA 숫자: {손계산:,}개")

모델.eval()
with torch.no_grad():
    붙인_뒤 = 모델(**입력).logits
print("붙이기 전후 출력이 같은가:", torch.allclose(붙이기_전, 붙인_뒤))
```

```
trainable params: 1,081,344 ¦¦ all params: 495,114,112 ¦¦ trainable%: 0.2184
lora.Linear(
  (base_layer): Linear(in_features=896, out_features=128, bias=True)
  (lora_dropout): ModuleDict(
    (default): Dropout(p=0.05, inplace=False)
  )
  (lora_A): ModuleDict(
    (default): Linear(in_features=896, out_features=8, bias=False)
  )
  (lora_B): ModuleDict(
    (default): Linear(in_features=8, out_features=128, bias=False)
  )
  (lora_embedding_A): ParameterDict()
  (lora_embedding_B): ParameterDict()
  (lora_magnitude_vector): ModuleDict()
)
손으로 센 LoRA 숫자: 1,081,344개
붙이기 전후 출력이 같은가: True
```

`target_modules`에 이름을 적은 선형층만 LoRA 층으로 바뀝니다. 여기서는 트랜스포머 블록마다 있는 어텐션의 네 선형층, 즉 쿼리(`q_proj`)·키(`k_proj`)·값(`v_proj`)을 만드는 층과 출력(`o_proj`) 층을 골랐어요. 12장에서 직접 만든 어텐션의 그 부품들입니다.

`print_trainable_parameters()`를 보면 학습하는 숫자는 1,081,344개, 전체의 0.22%예요. 5억 개 중 108만 개만 고치는 겁니다. 바뀐 `k_proj`를 찍어 보면 원래 층(`base_layer`)을 감싸고 옆에 `lora_A`(896 → 8), `lora_B`(8 → 128)가 붙어 있죠. 앞에서 손으로 만든 구조 그대로이고, `lora_dropout`은 8장 8.3절의 드롭아웃을 LoRA 쪽 입력에만 거는 것입니다.

손으로도 세어 봤습니다. 층마다 `r × (입력 + 출력)`이니 첫 블록의 네 층은 `8 × (896 + 896) + 8 × (896 + 128) × 2 + 8 × (896 + 896)` = 45,056개이고, 블록이 24개라 1,081,344개입니다. `peft`가 센 숫자와 정확히 같아요. 키와 값의 출력이 896이 아니라 128인 것은 14.6절에서 본 대로 키·값 헤드가 2개(2 × 64)뿐이기 때문입니다. 마지막 줄은 LoRA를 붙이기 전과 후의 출력이 똑같다는 확인이에요. `B`가 0에서 시작하는 덕분입니다.

Q

Q. `target_modules`는 어떻게 고르나요? 모델 구조를 `print(모델)`로 찍어 보면 선형층들의 이름이 나옵니다. 이 장처럼 어텐션 층만 고르는 것이 가장 가볍고, 피드포워드 층(Qwen에서는 `gate_proj`, `up_proj`, `down_proj`)까지 넣으면 학습할 숫자가 늘어나는 대신 더 많은 것을 바꿀 수 있어요. 이름은 모델마다 다르니(GPT-2는 `c_attn` 등) 새 모델을 쓸 때는 꼭 찍어 보고 정하세요.

14.9 실습: 1권 도우미로 미세조정하기

목표와 데이터

마지막으로 이 장의 모델을 우리 일에 맞게 고쳐 보겠습니다. 목표는 "이 책 1권의 내용을 묻는 질문에, 짧게 답하고 끝에 `(참고: 1권 N장)`을 붙이는 도우미"예요. 학습 데이터는 이 책에서 직접 만든 **예제용** 질문과 답 42쌍이고, 학습에 쓰지 않은 시험 문답 6쌍을 따로 두었습니다. 시험 문답 중 넷은 학습 문답과 같은 내용을 다르게 묻고, 둘(f-string, 정규 표현식)은 학습 문답에 아예 없는 주제예요. 8장처럼 "본 적 없는 문제도 잘 푸는가"를 확인하려는 겁니다.

1권 14장에서 배운 대로 데이터는 모듈 하나에 담아 두겠습니다. 아래 내용을 **`일권문답.py`**로 저장하세요.

```
"""LoRA 실습용 지시-응답 데이터: 1권 내용에 대한 질문과 답 (이 책에서 직접 만든 예제용 데이터)."""

시스템 = "너는 파이썬 입문서의 질문에 짧게 답하는 도우미야."

# 학습용 (질문, 답) 짝 42개. 답은 모두 "참고: 1권 N장"으로 끝나게 맞췄다.
```

```
학습_문답 = [
    ("변수란 무엇인가요?", "값에 붙이는 이름표예요. `나이 = 20`처럼 등호로 이름과 값을 묶어 두고 이
↪ 름으로 다시 꺼내 씁니다. (참고: 1권 2장)"),
    ("파이썬에서 문자열은 어떻게 만드나요?", "글자를 작은따옴표나 큰따옴표로 감싸면 됩니다. 여러 줄
↪ 이면 따옴표 세 개로 감싸요. (참고: 1권 2장)"),
    ("type 함수는 무엇을 알려 주나요?", "값의 자료형을 알려 줍니다. `type(3.14)`는 float를 돌려줘
↪ 요. (참고: 1권 2장)"),
    ("7 // 2와 7 % 2의 결과는 무엇인가요?", "`7 // 2`는 몫인 3, `7 % 2`는 나머지인 1입니다. (참고:
↪ 1권 3장)"),
    ("정수끼리 / 로 나누면 결과도 정수인가요?", "아니요. `/`는 언제나 실수를 돌려줘서 `6 / 3`은 2.0
↪ 입니다. 몫만 원하면 `//`를 씁니다. (참고: 1권 3장)"),
    ("==와 is는 어떻게 다른가요?", "`==`는 값이 같은지, `is`는 같은 물건(객체)인지를 묻습니다. (참
↪ 고: 1권 3장)"),
    ("if 문은 어떻게 쓰나요?", "`if 조건:`처럼 조건 뒤에 콜론을 찍고, 실행할 문장을 들여 써서 씁니
↪ 다. (참고: 1권 4장)"),
    ("elif는 언제 쓰나요?", "조건이 여러 개일 때 if 다음에 이어서 다른 조건을 차례로 검사할 때 씁니
↪ 다. (참고: 1권 4장)"),
    ("for 문과 while 문은 어떻게 다른가요?", "for는 정해진 묶음을 처음부터 끝까지 돌고, while은 조
↪ 건이 참인 동안 계속 돕니다. (참고: 1권 5장)"),
    ("break와 continue의 차이는 무엇인가요?", "break는 반복을 완전히 끝내고, continue는 이번 차례만
↪ 건너뛰고 다음 차례로 갑니다. (참고: 1권 5장)"),
    ("range(5)는 어떤 숫자를 만드나요?", "0부터 4까지 정수 다섯 개를 만듭니다. 끝 값 5는 들어가지
↪ 않아요. (참고: 1권 5장)"),
    ("리스트와 튜플의 차이는 무엇인가요?", "리스트는 만든 뒤에도 바꿀 수 있고, 튜플은 한번 만들면
↪ 바꿀 수 없습니다. (참고: 1권 6장)"),
    ("리스트 a에서 a[1:3]은 무엇을 꺼내나요?", "1번부터 3번 앞까지, 즉 1번과 2번 항목을 새 리스트로
↪ 꺼냅니다. (참고: 1권 6장)"),
    ("리스트 컴프리헨션이란 무엇인가요?", "`[x * 2 for x in 숫자]`처럼 반복과 리스트 만들기를 한 줄
↪ 로 쓰는 문법이에요. (참고: 1권 6장)"),
    ("딕셔너리란 무엇인가요?", "키와 값을 짝지어 담는 자료형입니다. 키로 값을 바로 찾을 수 있어요.
↪ (참고: 1권 7장)"),
    ("딕셔너리에 없는 키를 오류 없이 읽으려면 어떻게 하나요?", "`get` 메서드를 쓰면 키가 없을 때 오
↪ 류 대신 None이나 정해 둔 기본값을 돌려줍니다. (참고: 1권 7장)"),
    ("집합(set)의 특징은 무엇인가요?", "순서가 없고 같은 값을 한 번만 담습니다. 중복을 없앨 때 편해
↪ 요. (참고: 1권 7장)"),
```

```
    ("dataclass는 무엇을 해 주나요?", "필드만 적으면 `__init__`과 `__repr__` 같은 메서드를 자동으로
↪ 만들어 줍니다. (참고: 1권 8장)"),
    ("Pydantic은 언제 쓰나요?", "들어온 데이터의 자료형과 값을 검사하고 알맞게 바꿀 때 씁니다. (참
↪ 고: 1권 8장)"),
    ("함수는 어떻게 만드나요?", "`def` 뒤에 이름과 괄호를 쓰고, 들여 쓴 본문에서 `return`으로 결과
↪ 를 돌려줍니다. (참고: 1권 9장)"),
    ("기본값 매개변수란 무엇인가요?", "`def 인사(이름=\"손님\")`처럼 값을 넘기지 않으면 대신 쓸 값
↪ 을 정해 둔 매개변수예요. (참고: 1권 9장)"),
    ("*args와 **kwargs는 무엇인가요?", "`*args`는 위치 인자를 튜플로, `**kwargs`는 키워드 인자를 딕
↪ 셔너리로 모아 받습니다. (참고: 1권 9장)"),
    ("재귀 함수란 무엇인가요?", "자기 자신을 다시 부르는 함수입니다. 멈추는 조건을 꼭 넣어야 해요.
↪ (참고: 1권 9장)"),
    ("lambda는 무엇인가요?", "이름 없이 한 줄로 만드는 작은 함수입니다. `lambda x: x * 2`처럼 씁니
↪ 다. (참고: 1권 10장)"),
    ("map과 filter는 무엇을 하나요?", "map은 모든 항목에 함수를 적용하고, filter는 조건을 만족하는
↪ 항목만 남깁니다. (참고: 1권 10장)"),
    ("데코레이터란 무엇인가요?", "함수를 받아 기능을 덧붙인 새 함수를 돌려주는 함수로, 함수 위에 `@
↪ 이름`을 붙여 씁니다. (참고: 1권 10장)"),
    ("클래스와 객체는 어떤 관계인가요?", "클래스는 설계도이고, 객체는 그 설계도로 만든 실제 물건입
↪ 니다. (참고: 1권 11장)"),
    ("메서드의 self는 무엇인가요?", "메서드를 부른 객체 자신을 가리키는 첫 번째 매개변수예요. (참
↪ 고: 1권 11장)"),
    ("__init__ 메서드는 언제 실행되나요?", "객체를 만들 때 자동으로 실행되어 처음 값을 채웁니다.
↪ (참고: 1권 11장)"),
    ("파일을 안전하게 여는 방법은 무엇인가요?", "`with open(...) as f:`로 열면 블록이 끝날 때 파일
↪ 이 저절로 닫힙니다. (참고: 1권 12장)"),
    ("try와 except는 무엇에 쓰나요?", "오류가 날 수 있는 코드를 try에 넣고, 오류가 나면 except에서
↪ 처리합니다. (참고: 1권 12장)"),
    ("pathlib은 무엇에 쓰나요?", "파일 경로를 객체로 다룹니다. `Path(\"data\") / \"a.txt\"`처럼 경
↪ 로를 이어 붙여요. (참고: 1권 12장)"),
    ("상속이란 무엇인가요?", "기존 클래스의 기능을 물려받아 새 클래스를 만드는 방법입니다. `class
↪ 자식(부모):`처럼 씁니다. (참고: 1권 13장)"),
    ("super()는 무엇에 쓰나요?", "부모 클래스의 메서드를 부를 때 씁니다. 자식의 `__init__`에서
↪ `super().__init__()`을 자주 불러요. (참고: 1권 13장)"),
    ("모듈이란 무엇인가요?", "함수와 클래스를 담아 둔 파이썬 파일로, `import`로 불러 씁니다. (참고:
↪ 1권 14장)"),
```

```
    ("패키지는 모듈과 어떻게 다른가요?", "패키지는 여러 모듈을 폴더 하나로 묶은 것입니다. (참고: 1
↪ 권 14장)"),
    ("제너레이터란 무엇인가요?", "`yield`로 값을 하나씩 만들어 주는 함수입니다. 모든 값을 한꺼번에
↪ 메모리에 올리지 않아요. (참고: 1권 15장)"),
    ("반복자(iterator)란 무엇인가요?", "`next()`를 부를 때마다 다음 값을 하나씩 주는 객체입니다.
↪ (참고: 1권 15장)"),
    ("SQLite는 무엇인가요?", "파일 하나에 저장되는 가벼운 데이터베이스로, 파이썬에 `sqlite3` 모듈이
↪ 들어 있습니다. (참고: 1권 18장)"),
    ("FastAPI는 무엇인가요?", "파이썬으로 웹 API를 빠르게 만드는 프레임워크입니다. (참고: 1권 19
↪ 장)"),
    ("pandas의 DataFrame은 무엇인가요?", "행과 열로 된 표 데이터를 다루는 자료형입니다. (참고: 1권
↪ 20장)"),
    ("uv는 무엇인가요?", "파이썬과 패키지를 설치하고 프로젝트 환경을 관리하는 도구예요. `uv add`로
↪ 패키지를 더합니다. (참고: 1권 1장)"),
]

# 학습에 쓰지 않은 문답: 위 내용을 다르게 묻거나(앞의 넷), 아예 없는 주제(뒤의 둘)
시험_문답 = [
    ("튜플은 값을 바꿀 수 있나요?", "아니요. 튜플은 한번 만들면 바꿀 수 없습니다. 바꿔야 하면 리스
↪ 트를 씁니다. (참고: 1권 6장)"),
    ("with 문으로 파일을 열면 무엇이 좋은가요?", "블록이 끝나면 파일이 저절로 닫혀서 닫기를 잊을 걱
↪ 정이 없습니다. (참고: 1권 12장)"),
    ("yield는 무엇을 하나요?", "값을 하나 내보내고 그 자리에서 멈췄다가, 다음 값을 달라고 하면 이어
↪ 서 실행합니다. (참고: 1권 15장)"),
    ("부모 클래스의 메서드를 자식 클래스에서 부르려면 어떻게 하나요?", "`super().메서드이름()`처럼
↪ super()를 거쳐 부릅니다. (참고: 1권 13장)"),
    ("f-string은 무엇인가요?", "문자열 앞에 f를 붙이고 중괄호 안에 변수를 넣어 값을 끼워 넣는 문법
↪ 입니다. (참고: 1권 2장)"),
    ("정규 표현식은 무엇인가요?", "글자의 규칙(패턴)을 적어 문자열을 찾거나 바꾸는 방법으로, `re`
↪ 모듈로 씁니다. (참고: 1권 17장)"),
]
```

실습의 여러 예제가 같은 준비 코드를 쓰니, 8장의 **학습도구.py**처럼 모듈로 모아 둡니다. 아래 내용을 **문답도우미.py**로 저장하세요.

```
"""14.9절 실습에서 함께 쓰는 준비 코드."""
import re

import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

from 일권문답 import 시스템

모델_이름 = "Qwen/Qwen2.5-0.5B-Instruct"
어댑터_경로 = "data/lora_1권도우미"


def 불러오기() -> tuple:
    """토크나이저와 float32 모델을 불러온다. 결과를 재현하기 쉽게 CPU에서 쓴다."""
    토크나이저 = AutoTokenizer.from_pretrained(모델_이름)
    모델 = AutoModelForCausalLM.from_pretrained(모델_이름, dtype=torch.float32).eval()
    return 토크나이저, 모델


def 프롬프트(토크나이저, 질문: str) -> str:
    """시스템 지시와 질문을 채팅 템플릿에 맞춘 글로 만든다."""
    대화 = [{"role": "system", "content": 시스템}, {"role": "user", "content": 질문}]
    return 토크나이저.apply_chat_template(대화, tokenize=False, add_generation_prompt=True)


def 예제_만들기(토크나이저, 질문: str, 답: str) -> tuple[torch.Tensor, torch.Tensor]:
    """입력 번호와 정답 번호를 만든다. 질문 부분의 정답은 -100으로 가려 손실에서 뺀다."""
    앞 = 토크나이저(프롬프트(토크나이저, 질문), add_special_tokens=False)["input_ids"]
    뒤 = 토크나이저(답 + "<|im_end|>", add_special_tokens=False)["input_ids"]
    return torch.tensor(앞 + 뒤), torch.tensor([-100] * len(앞) + 뒤)


def 답하기(모델, 토크나이저, 질문: str, 최대: int = 60) -> str:
    """탐욕 디코딩(반복 억제 1.1)으로 답을 만든다."""
    입력 = 토크나이저(프롬프트(토크나이저, 질문), return_tensors="pt", add_special_tokens=False)
    with torch.no_grad():
        결과 = 모델.generate(**입력, max_new_tokens=최대, do_sample=False,
```

```
                            repetition_penalty=1.1)      # 모델 기본값의 반복 억제는 켜 둔다
    return 토크나이저.decode(결과[0, 입력["input_ids"].shape[1]:], skip_special_tokens=True)


def 평가(모델, 토크나이저, 문답: list[tuple[str, str]]) -> tuple[float, float, float]:
    """정답을 보여 주며(교사 강요) 세 가지를 잰다.

    답 전체의 평균 손실, '1권 N장'의 N 자리 평균 손실, N을 1등으로 맞힌 비율.
    """
    손실합, 번호손실합, 맞힘 = 0.0, 0.0, 0
    for 질문, 답 in 문답:
        입력, 정답 = 예제_만들기(토크나이저, 질문, 답)
        with torch.no_grad():
            출력 = 모델(input_ids=입력[None], labels=정답[None])
        손실합 += 출력.loss.item()
        # 답 속 "1권 N장"의 N이 몇 번째 토큰인지 찾는다 (숫자는 한 글자씩 토큰이 된다)
        시작 = (정답 != -100).nonzero()[0].item()
        조각 = 토크나이저(답, add_special_tokens=False, return_offsets_mapping=True)
        처음, 끝 = re.search(r"1권 (\d+)장", 답).span(1)
        자리 = [시작 + i for i, (s, e) in enumerate(조각["offset_mapping"])
                if s < 끝 and e > 처음]
        점수 = 출력.logits[0, [j - 1 for j in 자리]]          # 그 토큰을 맞혀야 하는 자리의 점수
        번호손실합 += torch.nn.functional.cross_entropy(점수, 입력[자리]).item()
        맞힘 += bool((점수.argmax(dim=-1) == 입력[자리]).all())
    n = len(문답)
    return 손실합 / n, 번호손실합 / n, 맞힘 / n
```

함수 다섯 개가 들어 있습니다.

- **불러오기**: 14.1절과 같습니다. 결과를 재현하기 쉽도록 이 실습은 CPU에서 돌려요. 모델이 작고 데이터가 적어서 CPU로도 충분합니다.
- **프롬프트**: 14.4절의 채팅 템플릿으로 시스템 지시와 질문을 하나의 글로 만듭니다.
- **예제_만들기**: 학습 예제 하나를 만듭니다. 질문 부분과 답 부분의 번호를 이어 붙이되, 정답 번호에서 질문 부분은 -100으로 가려요. 교차 엔트로피 손실(6장 6.5절)은 정답이 -100인 자리를 계산에서 빼도록 정해져 있어서(`ignore_index=-100`이 기본값), 모델은 질문을 따라 쓰는 법이 아니라 **답을 쓰는 법만** 배웁니다. 답 끝에는 `<|im_end|>`를 붙여 "답이 끝나면 멈춘다"까지 배우게 했어요.

- **답하기**: 14.4절과 같이 반복 억제(1.1)를 켠 탐욕 디코딩으로 답을 만듭니다.
- **평가**: 정답을 한 토큰씩 보여 주며(이를 **교사 강요**라고 해요) 세 가지를 잽니다. 답 전체의 평균 손실, 그중 "1권 N장"의 **N 자리**만의 손실, 그리고 N 자리에서 1등으로 고른 숫자가 정답과 같은 비율입니다. 장 번호를 따로 재는 이유는 곧 결과에서 드러나요. N이 몇 번째 토큰인지는 토크나이저의 `return_offsets_mapping`(토큰마다 원래 글의 몇 번째 글자부터 몇 번째까지인지)으로 찾았습니다.

미세조정 전

먼저 고치기 전의 실력을 재 둡니다. 비교할 기준선이 없으면 나중에 좋아졌는지 알 수 없으니까요.

```
from 문답도우미 import 답하기, 불러오기, 평가
from 일권문답 import 시험_문답, 학습_문답

토크나이저, 모델 = 불러오기()
print(f"학습 문답 {len(학습_문답)}개, 시험 문답 {len(시험_문답)}개")
for 질문, _ in 시험_문답[:3]:
    첫줄 = 답하기(모델, 토크나이저, 질문).strip().splitlines()[0]      # 길어서 첫 줄만
    print(f"Q. {질문}\nA. {첫줄}")

for 이름, 문답 in [("학습", 학습_문답), ("시험", 시험_문답)]:
    손실, 번호손실, 맞힘 = 평가(모델, 토크나이저, 문답)
    print(f"[미세조정 전] {이름} 문답: 답 손실 {손실:.3f}, "
          f"장 번호 자리 손실 {번호손실:.3f}, 장 번호 맞힘 {맞힘:.0%}")
```

```
학습 문답 42개, 시험 문답 6개
Q. 튜플은 값을 바꿀 수 있나요?
A. 네, 템플을 사용하여 값을 변경할 수 있습니다. 예를 들어, 다음과 같은 코드를 작성하면 `a`와 `b`의
↪  값이 서로 바뀝니다:
Q. with 문으로 파일을 열면 무엇이 좋은가요?
A. with 문은 파일을 항상 닫아주어야 합니다. 이로 인해 파일이 닫히지 않으면 오류를 발생시킵니다. 따
↪  라서 파일을 닫아주는 코드를 작성해야 합니다.
Q. yield는 무엇을 하나요?
A. yield는 Python에서 함수를 호출하거나 반복할 때 사용하는 특별한 기능입니다.
[미세조정 전] 학습 문답: 답 손실 2.782, 장 번호 자리 손실 2.179, 장 번호 맞힘 10%
[미세조정 전] 시험 문답: 답 손실 2.974, 장 번호 자리 손실 2.024, 장 번호 맞힘 0%
```

답이 길어서 첫 줄만 보였는데, 질문 셋 모두 틀렸습니다. 튜플을 바꿀 수 있다고 하고("템플"이라는 오타까지 냈어요), with 문은 파일을 직접 닫아 줘야 한다며 정반대로 말해요. 첫 줄부터 길게 늘어놓고, 당연히 `(참고: 1권 N장)`도 없습니다. 답 손실은 학습 문답 2.782, 시험 문답 2.974이고, 장 번호는 학습 문답에서 10%만 맞혔어요. "1권" 다음에 올 숫자를 거의 찍는 수준입니다.

LoRA로 학습하기

이제 14.8절의 설정 그대로 LoRA를 붙이고 학습합니다. 학습 루프는 6장의 다섯 줄 그대로이고, 길이가 다른 예제를 묶는 방법은 7장 7.7절의 `pad_sequence`예요.

```
from pathlib import Path

import torch
from peft import LoraConfig, get_peft_model
from torch.nn.utils.rnn import pad_sequence

from 문답도우미 import 불러오기, 어댑터_경로, 예제_만들기
from 일권문답 import 학습_문답

torch.manual_seed(0)
토크나이저, 모델 = 불러오기()
설정 = LoraConfig(r=8, lora_alpha=16, lora_dropout=0.05, task_type="CAUSAL_LM",
                  target_modules=["q_proj", "k_proj", "v_proj", "o_proj"])
모델 = get_peft_model(모델, 설정)
예제들 = [예제_만들기(토크나이저, 질문, 답) for 질문, 답 in 학습_문답]
print("가장 긴 예제:", max(len(입력) for 입력, _ in 예제들), "토큰")

옵티마이저 = torch.optim.AdamW([p for p in 모델.parameters() if p.requires_grad], lr=1e-3)
생성기 = torch.Generator().manual_seed(0)
배치_크기 = 8
for 에폭 in range(1, 9):
    모델.train()
    손실합 = 0.0
    순서 = torch.randperm(len(예제들), generator=생성기).tolist()
    for i in range(0, len(순서), 배치_크기):
        묶음 = [예제들[j] for j in 순서[i:i + 배치_크기]]
        # 7장 7.7절처럼 길이가 다른 예제를 채워서(padding) 묶는다
```

```
        입력 = pad_sequence([x for x, _ in 묶음], batch_first=True,
                            padding_value=토크나이저.pad_token_id)
        정답 = pad_sequence([y for _, y in 묶음], batch_first=True, padding_value=-100)
        마스크 = pad_sequence([torch.ones(len(x), dtype=torch.long) for x, _ in 묶음],
                              batch_first=True)
        손실 = 모델(input_ids=입력, attention_mask=마스크, labels=정답).loss
        옵티마이저.zero_grad()
        손실.backward()
        옵티마이저.step()
        손실합 += 손실.item() * len(묶음)
    print(f"에폭 {에폭}: 학습 손실 {손실합 / len(예제들):.3f}")

모델.save_pretrained(어댑터_경로)                    # LoRA 부분(A, B)만 저장된다
for 파일 in sorted(Path(어댑터_경로).iterdir()):
    print(f"{파일.name}: {파일.stat().st_size / 1024**2:.2f} MB")
```

```
가장 긴 예제: 106 토큰
에폭 1: 학습 손실 2.176
에폭 2: 학습 손실 1.257
에폭 3: 학습 손실 0.947
에폭 4: 학습 손실 0.681
에폭 5: 학습 손실 0.476
에폭 6: 학습 손실 0.317
에폭 7: 학습 손실 0.219
에폭 8: 학습 손실 0.169
README.md: 0.00 MB
adapter_config.json: 0.00 MB
adapter_model.safetensors: 4.15 MB
```

예제 42개를 8개씩 묶어 8에폭, 모두 48번 고쳤습니다. 이 책의 실행 환경(Apple M4 Max의 CPU)에서 1~2분쯤 걸렸어요. 손실은 2.176에서 0.169까지 꾸준히 내려갔습니다. 채운 자리는 정답을 -100으로, 마스크(`attention_mask`)를 0으로 둬서 손실에도 어텐션에도 끼어들지 않게 했어요.

`save_pretrained`로 저장한 결과도 보세요. 모델 전체가 아니라 LoRA의 `A`와 `B`만 저장하는 **어댑터** 파일이 4.15MB입니다. 원래 모델 1.84GB의 0.2%쯤이에요. 같은 원래 모델 하나에 일마다 다른 어댑터를 여러 개 만들어 두고 바꿔 끼울 수 있습니다.

미세조정 후: 무엇을 배웠고 무엇을 못 배웠나

저장한 어댑터를 원래 모델에 끼워서 같은 측정을 해 보겠습니다.

```
import torch
from peft import PeftModel

from 문답도우미 import 답하기, 불러오기, 어댑터_경로, 평가
from 일권문답 import 시험_문답, 학습_문답

토크나이저, 모델 = 불러오기()
모델 = PeftModel.from_pretrained(모델, 어댑터_경로).eval()   # 원래 모델 + 저장한 LoRA

for 이름, 문답 in [("학습", 학습_문답), ("시험", 시험_문답)]:
    손실, 번호손실, 맞힘 = 평가(모델, 토크나이저, 문답)
    print(f"[미세조정 후] {이름} 문답: 답 손실 {손실:.3f}, "
          f"장 번호 자리 손실 {번호손실:.3f}, 장 번호 맞힘 {맞힘:.0%}")

for 질문, 기준 in 학습_문답[:2] + 시험_문답:
    print(f"\nQ. {질문}\n기준. {기준}\n모델. {답하기(모델, 토크나이저, 질문)}")

# LoRA를 원래 가중치에 합쳐 보통 모델로 만들기 (14.7절의 W + 배율·BA)
합친_모델 = 모델.merge_and_unload()
print("\n합친 뒤에도 같은 답인가:",
      답하기(합친_모델, 토크나이저, 시험_문답[0][0]) == 답하기(모델, 토크나이저, 시험_문답[0][0]))
print(type(합친_모델).__name__)
```

```
[미세조정 후] 학습 문답: 답 손실 0.123, 장 번호 자리 손실 1.862, 장 번호 맞힘 10%
[미세조정 후] 시험 문답: 답 손실 1.976, 장 번호 자리 손실 1.906, 장 번호 맞힘 17%

Q. 변수란 무엇인가요?
기준. 값에 붙이는 이름표예요. `나이 = 20`처럼 등호로 이름과 값을 묶어 두고 이름으로 다시 꺼내 씁니
↪ 다. (참고: 1권 2장)
모델. 값에 붙이는 이름표예요. `나이 = 20`처럼 등호로 이름과 값을 묶어 두고 이름으로 다시 꺼내 씁니
↪ 다. (참고: 1권 6장)

Q. 파이썬에서 문자열은 어떻게 만드나요?
```

```
기준. 글자를 작은따옴표나 큰따옴표로 감싸면 됩니다. 여러 줄이면 따옴표 세 개로 감싸요. (참고: 1권 2
↪ 장)
모델. `input()`로 입력을 받아서 문자열로 만듭니다. (참고: 1권 2장)

Q. 튜플은 값을 바꿀 수 있나요?
기준. 아니요. 튜플은 한번 만들면 바꿀 수 없습니다. 바꿔야 하면 리스트를 씁니다. (참고: 1권 6장)
모델. 네, 값을 바꿀 수 있습니다. `tup`를 객체 인스턴스로 만들어 주면 값을 바꿀 수 있어요. (참고: 1
↪ 권 9장)

Q. with 문으로 파일을 열면 무엇이 좋은가요?
기준. 블록이 끝나면 파일이 저절로 닫혀서 닫기를 잊을 걱정이 없습니다. (참고: 1권 12장)
모델. `with`로 열려 있는 파일에서 쓴 내용이 바로 닫힙니다. (참고: 1권 13장)

Q. yield는 무엇을 하나요?
기준. 값을 하나 내보내고 그 자리에서 멈췄다가, 다음 값을 달라고 하면 이어서 실행합니다. (참고: 1권
↪ 15장)
모델. `yield`를 돌려주는 메서드입니다. `yield`를 한 번만 돌려주면 다음으로 이어 붙여 씁니다. (참고:
↪ 1권 9장)

Q. 부모 클래스의 메서드를 자식 클래스에서 부르려면 어떻게 하나요?
기준. `super().메서드이름()`처럼 super()를 거쳐 부릅니다. (참고: 1권 13장)
모델. `__class__` 속성을 물어 둔 다음 `__class__`로 부른 자식을 다시 물어 둔 방법입니다. (참고: 1권
↪ 9장)

Q. f-string은 무엇인가요?
기준. 문자열 앞에 f를 붙이고 중괄호 안에 변수를 넣어 값을 끼워 넣는 문법입니다. (참고: 1권 2장)
모델. `f-string`로 씁니다면 자동으로 f와 {}를 바꾸고, 값을 꼭 넣어야 해요. (참고: 1권 6장)

Q. 정규 표현식은 무엇인가요?
기준. 글자의 규칙(패턴)을 적어 문자열을 찾거나 바꾸는 방법으로, `re` 모듈로 씁니다. (참고: 1권 17장)
모델. `re` 모듈을 사용해 정규표현식을 만드는 자료형입니다. (참고: 1권 9장)

합친 뒤에도 같은 답인가: True
Qwen2ForCausalLM
```

`PeftModel.from_pretrained`는 원래 모델에 저장한 어댑터를 끼워 줍니다. 결과를 하나씩 읽어 보죠.

말투와 형식은 완벽하게 배웠습니다. 여덟 개 답 모두 한두 문장으로 짧게 끝나고, 모두 `(참고: 1권 N장)`으로 마무리해요. 학습에 없던 f-string과 정규 표현식 질문에도 같은 형식으로 답했습니다. 답 손실도 학습 문답은 2.782에서 0.123으로, 시험 문답도 2.974에서 1.976으로 줄었어요. 첫 학습 문답 "변수란 무엇인가요?"에는 데이터의 답을 글자 그대로 되풀이합니다.

그런데 사실은 거의 배우지 못했습니다. 그 똑같은 답의 끝이 "1권 2장"이 아니라 "1권 6장"이에요. 장 번호를 맞힌 비율은 학습 문답에서도 10%로 전과 같고, 장 번호 자리의 손실은 2.179에서 1.862로 조금 줄었을 뿐입니다. 답 전체 손실이 20분의 1보다 작아지는 동안 장 번호 자리는 거의 그대로인 거예요. 시험 문답의 내용도 여전히 틀립니다. 튜플은 바꿀 수 있다고 하고, `super()` 대신 `__class__` 이야기를 지어냈어요. 장 번호 맞힘 17%는 여섯 문항 중 하나가 맞은 것으로, 정답 문장을 보여 주며 잰 값이라 위의 생성 답(여섯 개 모두 장 번호가 틀림)과는 조금 다릅니다. 우연에 가까운 숫자예요.

왜 이런 차이가 날까요? 답 문장의 토큰 대부분은 **바로 앞 몇 토큰만 보면** 이어 쓸 수 있습니다. "값에 붙이는 이름" 다음에 "표예요"가 오는 식이죠. 게다가 "(참고: 1권"처럼 42개 답에 똑같이 나오는 부분은 에폭마다 42번씩 반복해서 배웁니다. 반면 장 번호는 질문 내용에 따라 정해지는 **임의의 사실**이에요. "변수"와 "2"를 잇는 규칙은 어디에도 없고, 데이터에 단 한 번씩만 나옵니다. 8에폭이면 그 짝을 여덟 번 본 셈인데, 5억 개짜리 모델이 0.2%의 숫자만 고쳐서 마흔두 개의 짝을 외우기에는 부족했어요.

평균 손실만 봤다면 "다 배웠다"고 착각했을 겁니다. 6장 6.11절에서 정확도 하나로 끝내지 않고 무엇을 틀렸는지 들여다봤던 것처럼, **숫자 하나로 판단하지 말고 무엇을 틀리는지 봐야** 해요.

마지막 두 줄은 `merge_and_unload()`로 어댑터를 원래 가중치에 합친 결과입니다. 14.7절의 `W + 배율 × (B @ A)`를 모든 층에 적용한 것으로, 합친 뒤에는 다시 보통 `Qwen2ForCausalLM`이 되고 같은 답을 냅니다. 배포할 때는 이렇게 합쳐서 추가 계산 없이 쓸 수 있어요.

이 실습에서 얻은 결론은 실무에서도 그대로 통합니다. **적은 데이터의 LoRA 미세조정은 "어떻게 말할지"(말투, 형식, 답의 길이, 따라야 할 규칙)를 가르치는 데 강하고, "무엇을 알지"(새로운 사실)를 넣는 데는 약합니다.** 사실을 정확히 답하게 하려면 훨씬 많은 데이터와 반복, 더 큰 모델이 필요해요. 또는 모델이 외우게 하는 대신, 질문과 관련된 문서를 찾아서 프롬프트에 넣어 주고 그 문서를 근거로 답하게 하는 방법(RAG, 검색 증강 생성)을 씁니다. RAG와, 모델이 도구를 써 가며 일하는 에이전트는 3권에서 다룹니다.

Q. 장 번호까지 외우게 하려면 어떻게 해야 하나요? 가장 확실한 건 데이터입니다. 같은 사실을 여러 가지 질문과 말투로 바꿔 쓴 예제를 많이 만들면, 모델이 "이 주제면 이 장"이라는 짝을 여러 번 보게 돼요. 에폭을 늘리면 학습 문답은 더 외우지만 시험 문답은

Q 오히려 나빠질 수 있어요. 위 학습 코드에서 `range(1, 9)`를 `range(1, 17)`로만 바꿔 16에폭을 돌려 보니, 학습 문답의 장 번호 맞힘은 10%에서 43%로 올랐지만 시험 문답은 17%에서 0%로 떨어졌고 시험 문답의 답 손실도 1.976에서 2.236으로 커졌습니다. 8장 8.1절에서 본 과적합 그대로죠. 그러니 시험 문답의 지표를 함께 지켜봐야 합니다. LoRA 랭크를 키우거나 `target_modules`에 피드포워드 층을 넣는 것도 방법이에요. 그래도 이 장 크기의 모델로는 한계가 뚜렷하니, 정확한 사실이 중요한 일이라면 위의 RAG처럼 사실을 프롬프트로 건네는 쪽을 먼저 생각하세요.

14.10 정리와 다음 장 미리보기

이 장에서 사전학습된 언어 모델을 불러와 쓰고, 재고, 고쳐 봤습니다.

- **불러오기**: `AutoTokenizer`, `AutoModelForCausalLM.from_pretrained(이름, dtype=...)`. 모델은 `nn.Module`이라 6장의 도구가 그대로 통한다. 쓰기 전에 모델 카드에서 라이선스를 확인한다
- **토크나이저**: 글 → 번호 → 글. 바이트 단위 조각, 같은 뜻이어도 한국어는 토큰이 더 많다
- **다음 토큰 예측**: 로짓 모양은 (배치, 토큰 수, 어휘 크기). 생성은 "고르고, 붙이고, 다시 넣기"의 반복. `generate`는 KV 캐시로 빠르게 한다
- **디코딩**: 탐욕은 늘 같은 답. 온도로 분포를 날카롭게/평평하게, top-k·top-p로 엉뚱한 후보를 거른 뒤 샘플링. 장치가 다르면 같은 씨앗도 다른 문장이 될 수 있다
- **프롬프트**: `apply_chat_template`으로 system·user·assistant 메시지를 모델의 형식에 맞춘다. 예시를 대화처럼 넣는 few-shot. 모델은 기억이 없으니 대화 기록을 매번 함께 보낸다
- **한계**: 작은 모델은 그럴듯한 거짓(환각)을 만든다. 모델 크기, 학습 언어 비중, "그럴듯함"이 목표라는 점이 이유
- **메모리**: 파라미터 수 × 칸 크기, KV 캐시는 2 × 층 × 키·값 헤드 × 헤드 크기 × 칸 크기 × 토큰 수. `bfloat16`은 절반에 거의 같은 답, 양자화는 정수 칸과 배율. 비트를 너무 줄이면 망가진다
- **LoRA**: 원래 `W`는 얼리고 `ΔW = B @ A`(랭크 r)만 학습. `B`는 0에서 시작, 배율 `alpha / r`, 다 배우면 `W`에 합칠 수 있다
- **peft**: `LoraConfig(r, lora_alpha, target_modules)`, `get_peft_model`, `print_trainable_parameters`, `save_pretrained`(어댑터만), `PeftModel.from_pretrained`, `merge_and_unload`
- **미세조정의 교훈**: 질문 부분 정답은 -100으로 가린다. 적은 데이터의 LoRA는 말투·형식에 강하고 사실 주입에 약하다. 평균 손실 말고 중요한 자리를 따로 잰다

이 장의 모델은 "다음 토큰"을 하나씩 뽑아 글을 만들었습니다. 그렇다면 그림은 어떻게 만들까요? 그림은 28 × 28만 해도 숫자 784개가 한꺼번에 어울려야 해서, 왼쪽 위 점부터 하나씩 뽑는 방법은 잘 맞지 않아요. 다음 15장에서는 그림을 작은 숫자 묶음으로 줄였다가 되살리는 **오토인코더**와 VAE, 그리고 잡음에서 출발해 조금씩 잡음을 걷어 내며 그림을 그리는 **확산 모델**을 노트북 크기로 직접 만들어 봅니다.

15

생성 모델: 오토인코더, VAE, 확산 모델 입문

학습 목표

- 고르는 모델(판별 모델)과 그리는 모델(생성 모델)의 차이를 설명하기
- 오토인코더로 사진을 작게 줄였다가 되살리고, 잠재 공간을 그림으로 들여다보기
- 합성곱 오토인코더로 잡음을 지우고, 재구성 오차로 낯선 데이터를 찾아내기
- VAE의 재매개변수화와 KL 발산을 직접 계산하고, 잠재 공간에서 새 그림을 뽑고 보간하기
- 잡음을 더하고 빼는 확산 모델(DDPM)을 처음부터 구현해 노트북에서 옷 그림 그리기
- 생성 모델을 평가하기가 왜 어려운지 알고, 분류기와 최근접 이웃으로 결과를 점검하기

들어가며: 고르는 모델, 그리는 모델

지금까지 만든 모델은 모두 **고르는** 모델이었습니다. 6장의 분류기는 옷 사진을 받아 "이건 앵클부츠"라고 열 가지 중 하나를 골랐고, 9장의 CNN도 같은 일을 더 잘했죠. 이런 모델을 **판별 모델**(discriminative model)이라고 부릅니다. 사진이 주어지면 답을 고르는 거예요.

그럼 거꾸로는 어떨까요? "앵클부츠를 하나 그려 줘"라고 하면 세상에 없던 앵클부츠 사진을 새로 만들어 내는 모델이요. 이런 모델이 **생성 모델**(generative model)입니다. 미술 시간을 떠올려 보세요. 그림을 보고 "이건 고흐 그림"이라고 맞히는 감상자와, 고흐 그림을 수백 장 보고 나서 고흐풍의 새 그림을 그리는 화가는 하는 일이 전혀 다릅니다. 감상자는 그림 사이의 **경계**만 알면 되지만, 화가는 고흐 그림이 **어떻게 생겼는지** 자체를 알아야 해요. 붓 터치는 어떤지, 색은 어떻게 쓰는지, 무엇이 그럴듯하고 무엇이 어색한지를요.

생성 모델이 배우는 것이 바로 이 "어떻게 생겼는가"입니다. 조금 어려운 말로 하면 데이터의 **분포**를 배운다고 해요. 옷 사진 6만 장을 보면서 "옷 사진이라면 대개 이렇게 생겼다"는 감각을 익히고, 그 감각에서 새 사진을 하나 뽑아내는 거죠. 요즘 화제가 되는 그림 생성 AI와 채팅 AI가 모두 이 갈래에 속합니다(채팅 AI의 원리는 14장에서 봤죠).

이 장에서는 세 걸음으로 화가를 키워 봅니다.

1. **오토인코더**: 사진을 작은 요약으로 줄였다가 다시 되살리는 모델입니다. 아직 화가는 아니지만, "요약"이라는 중요한 생각을 줍니다(15.1~15.5절).
2. **VAE(변분 오토인코더)**: 요약들이 놓이는 자리를 고르게 정리해서, 아무 자리나 골라도 그럴듯한 그림이 나오게 만든 오토인코더입니다. 처음으로 새 그림을 그립니다(15.6절).
3. **확산 모델**: 사진에 잡음을 조금씩 섞어 완전히 망가뜨리는 과정을 거꾸로 배우는 모델입니다. 요즘 그림 생성 AI의 바탕이 되는 방법이고, 이 장에서 작은 버전을 처음부터 만들어 노트북에서 학습시킵니다(15.8절).

중간에 한때 생성 모델의 대표였던 GAN의 생각도 짧게 짚고(15.7절), 마지막으로 "그린 그림이 좋은지 어떻게 아는가"라는 까다로운 질문을 다룹니다(15.9절). 재료는 6장에서 쓴 FashionMNIST 옷 사진입니다. 의류 쇼핑몰 잘란도(Zalando)가 MIT 라이선스로 공개한 데이터라 자유롭게 쓸 수 있어요.

Q

Q. 그림 생성 모델이라면 GPU가 꼭 있어야 하나요? 이 장의 오토인코더와 VAE는 CPU로 수 초에서 수십 초면 학습됩니다. 확산 모델은 계산이 훨씬 많아서, 이 책의 실행 환경(Apple M4 Max 노트북의 MPS)에서 학습에 2~5분(동시에 돌던 다른 작업에 따라 달랐어요), 그림 1,000장을 그리는 데 1~2분쯤 걸렸습니다. CPU만 있어도 돌아가지만 몇 배 더 걸리니, 확산 모델 예제는 걸음 수를 줄여서 먼저 돌려 보세요. 필요한 패키지는 `uv add torch torchvision matplotlib`입니다. 이 장의 예제는 파이썬 3.12, PyTorch 2.14.1, torchvision 0.29.1, matplotlib 3.11.2에서 실행해 확인했습니다.

15.1 오토인코더: 줄였다가 되살리기

요약 노트로 원본 다시 쓰기

시험공부를 할 때 교과서 한 단원을 요약 노트 한 장으로 줄여 본 적이 있죠? 요약을 잘했다면, 나중에 그 노트만 보고도 단원 내용을 거의 다시 설명할 수 있습니다. 반대로 중요한 것을 빼먹고 요약했다면 다시 설명할 때 구멍이 나요. 그러니 "요약만 보고 원본을 얼마나 잘 되살리는가"가 요약 실력의

성적표가 됩니다.

오토인코더(autoencoder) 가 바로 이 방법으로 배웁니다. 모델이 두 부분으로 되어 있어요.

- **인코더(encoder)**: 사진 한 장(784칸)을 받아 아주 짧은 숫자 묶음으로 줄입니다. 이 짧은 묶음을 **잠재 벡터(latent vector)**, 잠재 벡터가 놓이는 공간을 **잠재 공간**이라고 불러요. 요약 노트에 해당합니다.
- **디코더(decoder)**: 잠재 벡터만 보고 원래 사진(784칸)을 다시 그려 냅니다. 요약 노트를 보고 단원을 다시 설명하는 쪽이에요.

학습할 때 정답은 **입력 사진 자기 자신**입니다. 그래서 정답 레이블이 필요 없어요. 1장 1.3절에서 본 비지도 학습의 한 종류이고, 이름의 "오토(auto)"도 "스스로"라는 뜻입니다. 가운데가 잘록하게 좁아서 모든 정보를 다 담을 수는 없으니, 모델은 어쩔 수 없이 **무엇이 중요한지**를 골라 담는 법을 배우게 됩니다. 이 잘록한 부분을 병의 목에 빗대 **병목(bottleneck)** 이라고 해요.

이 장의 도구 모음

이 장의 여러 예제가 같은 데이터 불러오기와 그림 저장 코드를 씁니다. 8장에서 `학습도구.py`를 만들었던 것처럼(1권 14장의 모듈), 아래 내용을 `생성도구.py`로 저장해 두고 `import`해서 쓰겠습니다. 이 장의 예제 파일들은 모두 이 파일과 같은 폴더에 두세요.

```
"""15장에서 함께 쓰는 데이터·그림·오토인코더 도구."""
import torch
import torch.nn.functional as F
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
from torchvision import datasets
from torchvision.utils import save_image

이름 = ["T셔츠", "바지", "풀오버", "드레스", "코트", "샌들", "셔츠", "운동화", "가방", "앵클부츠"]


def 사진_불러오기(학습: bool = True) -> tuple[torch.Tensor, torch.Tensor]:
    """FashionMNIST를 통째로 (N, 1, 28, 28) 텐서(값 0~1)와 정답 텐서로 돌려준다."""
    셋 = datasets.FashionMNIST("data", train=학습, download=True)
    return 셋.data.unsqueeze(1).float() / 255, 셋.targets
```

```
def 로더_만들기(X: torch.Tensor, 배치: int = 256, 씨앗: int = 0) -> DataLoader:
    """사진만 담은 DataLoader. 정답이 필요 없으니 사진 하나만 넘긴다."""
    return DataLoader(TensorDataset(X), batch_size=배치, shuffle=True,
                      generator=torch.Generator().manual_seed(씨앗))


def 격자_저장(사진들: torch.Tensor, 파일: str, 한_줄: int = 8) -> None:
    """(N, 1, 28, 28) 사진 묶음을 격자 한 장으로 저장한다. 인쇄용으로 3배 키운다."""
    큰_사진 = F.interpolate(사진들.detach().cpu().clamp(0, 1), scale_factor=3, mode="nearest")
    save_image(큰_사진, 파일, nrow=한_줄, padding=6, pad_value=1.0)


class 오토인코더(nn.Module):
    """784칸 사진을 잠재_크기 칸으로 줄였다가(인코더) 다시 784칸으로 되살린다(디코더)."""

    def __init__(self, 잠재_크기: int = 32) -> None:
        super().__init__()
        self.인코더 = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 256), nn.ReLU(),
            nn.Linear(256, 잠재_크기),
        )
        self.디코더 = nn.Sequential(
            nn.Linear(잠재_크기, 256), nn.ReLU(),
            nn.Linear(256, 784), nn.Sigmoid(),      # 밝기를 0~1로
            nn.Unflatten(1, (1, 28, 28)),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.디코더(self.인코더(x))


def 오토인코더_학습(모델: nn.Module, X: torch.Tensor, 에폭: int = 5,
                    잡음: float = 0.0, 출력: bool = True) -> nn.Module:
    """입력을 그대로 되살리도록 학습한다. 잡음 > 0이면 잡음 섞은 입력에서 깨끗한 원본을 되살린다."""
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
```

```
    for 에폭_번호 in range(1, 에폭 + 1):
        손실_합 = 0.0
        for (x,) in 로더_만들기(X, 씨앗=에폭_번호):
            입력 = (x + 잡음 * torch.randn_like(x)).clamp(0, 1) if 잡음 > 0 else x
            손실 = F.mse_loss(모델(입력), x)        # 정답은 언제나 깨끗한 원본
            옵티마이저.zero_grad()
            손실.backward()
            옵티마이저.step()
            손실_합 += 손실.item() * len(x)
        if 출력:
            print(f"에폭 {에폭_번호}: 재구성 손실 {손실_합 / len(X):.4f}")
    return 모델


def 채점기_만들기(장치: torch.device, 에폭: int = 2) -> nn.Module:
    """생성한 그림을 채점할 작은 CNN 옷 분류기를 진짜 사진으로 학습해 돌려준다."""
    torch.manual_seed(0)
    X, y = 사진_불러오기(학습=True)
    모델 = nn.Sequential(
        nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        nn.Flatten(), nn.Linear(64 * 7 * 7, 128), nn.ReLU(), nn.Linear(128, 10),
    ).to(장치)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
    로더 = DataLoader(TensorDataset(X, y), batch_size=128, shuffle=True,
                      generator=torch.Generator().manual_seed(0))
    for _ in range(에폭):
        for x, 정답 in 로더:
            손실 = F.cross_entropy(모델(x.to(장치)), 정답.to(장치))
            옵티마이저.zero_grad()
            손실.backward()
            옵티마이저.step()
    return 모델.eval()
```

하나씩 보겠습니다.

- **사진_불러오기**: 6장 6.11절에서 쓴 FashionMNIST를 `DataLoader` 없이 **통째로** 텐서 하나로 꺼냅니다. `셋.data`는 `(60000, 28, 28)` 모양의 0~255 정수 텐서라서, `unsqueeze(1)`로 채널

자리를 끼워 넣고 255로 나눠 0~1 실수로 바꿨어요. 7만 장을 다 합쳐도 200MB 남짓이라 노트북 메모리에 넉넉히 들어갑니다. 처음 실행하면 `data/` 폴더에 내려받아요.

- **로더_만들기**: 7장 7.2절의 `TensorDataset`에 사진만 담습니다. 오토인코더는 정답이 필요 없으니까요. 섞는 순서는 7장 7.8절처럼 생성기로 고정했습니다.
- **격자_저장**: 사진 여러 장을 바둑판처럼 붙여 PNG 한 장으로 저장합니다. torchvision의 `save_image`가 이 일을 해 줘요. 28 × 28은 인쇄하면 너무 작아서 `F.interpolate`로 가로세로 세 배씩 키웠습니다(`mode="nearest"`는 픽셀을 그대로 복제해 키우는 방식).
- **오토인코더**: 인코더는 784 → 256 → 잠재_크기, 디코더는 잠재_크기 → 256 → 784로 거울처럼 대칭입니다. 디코더 끝의 `Sigmoid`는 밝기를 0~1 사이로 맞춰 주고, `nn.Unflatten`은 784칸을 다시 `(1, 28, 28)` 사진 모양으로 접어요. `nn.Flatten`의 반대입니다.
- **오토인코더_학습**: 6장의 학습 루프 그대로인데, 손실이 `F.mse_loss(모델(입력), x)`라는 점만 다릅니다. **출력과 입력 자신**을 비교하는 거예요. **잡음** 인자는 15.3절에서 씁니다.
- **채점기_만들기**: 15.9절에서 모델이 그린 그림을 채점할 작은 CNN 분류기입니다. 합성곱과 풀링은 9장에서 배운 그대로라 여기서는 쓰기만 해요.

첫 오토인코더

784칸을 32칸으로, 약 25분의 1로 줄이는 오토인코더를 학습해 보겠습니다.

```
import torch
import torch.nn.functional as F

from 생성도구 import 격자_저장, 사진_불러오기, 오토인코더, 오토인코더_학습

torch.manual_seed(0)
X_학습, _ = 사진_불러오기(학습=True)
X_시험, _ = 사진_불러오기(학습=False)
print("학습 사진:", tuple(X_학습.shape), "시험 사진:", tuple(X_시험.shape))

모델 = 오토인코더(잠재_크기=32)
print(f"파라미터 {sum(p.numel() for p in 모델.parameters()):,}개")
오토인코더_학습(모델, X_학습, 에폭=5)

모델.eval()
with torch.no_grad():
    잠재 = 모델.인코더(X_시험[:8])          # 사진 8장을 32칸으로 줄이고
```

```
    복원 = 모델.디코더(잠재)                # 다시 784칸으로 되살린다
    시험_손실 = F.mse_loss(모델(X_시험), X_시험).item()
print("잠재 벡터 모양:", tuple(잠재.shape))
print("첫 사진의 잠재 벡터 앞 6칸:", [round(v, 2) for v in 잠재[0, :6].tolist()])
print(f"시험 사진 재구성 손실: {시험_손실:.4f}")
print(f"줄인 비율: 784 → 32 (약 {784 / 32:.1f}분의 1)")

격자_저장(torch.cat([X_시험[:8], 복원]), "ae_recon.png")   # 위: 원본, 아래: 복원
```

```
학습 사진: (60000, 1, 28, 28) 시험 사진: (10000, 1, 28, 28)
파라미터 419,120개
에폭 1: 재구성 손실 0.0405
에폭 2: 재구성 손실 0.0193
에폭 3: 재구성 손실 0.0164
에폭 4: 재구성 손실 0.0148
에폭 5: 재구성 손실 0.0137
잠재 벡터 모양: (8, 32)
첫 사진의 잠재 벡터 앞 6칸: [1.37, 1.2, 1.24, 1.37, -3.48, -4.76]
시험 사진 재구성 손실: 0.0134
줄인 비율: 784 → 32 (약 24.5분의 1)
```

다섯 에폭 만에 재구성 손실이 0.0137까지 내려갔고, 한 번도 보지 못한 시험 사진에서도 0.0134로 비슷합니다. 외운 게 아니라 옷 사진 일반을 요약하는 법을 배웠다는 뜻이에요. 픽셀마다 평균 제곱 오차가 0.0134라면, 0~1 밝기에서 한 칸당 평균 0.12쯤 어긋난다는 뜻입니다($\sqrt{0.0134} \approx 0.12$).

`모델.인코더(...)`와 `모델.디코더(...)`를 따로 부를 수 있다는 점도 보세요. 6장 6.3절처럼 `nn.Module` 안에 넣은 층은 속성 이름으로 꺼내 쓸 수 있습니다. 사진 8장이 `(8, 32)`, 한 장에 숫자 32개로 줄었어요. 첫 사진의 잠재 벡터 `[1.37, 1.2, ...]`는 사람이 읽을 수 있는 뜻은 없지만, 디코더는 이 32개만 보고 784칸을 다시 그려 냅니다.

위 줄이 원본, 아래 줄이 되살린 사진입니다. 신발, 바지, 셔츠의 **전체 모양**은 거의 그대로 살아났어요. 반면 두 번째 티셔츠의 로고 글자, 마지막 셔츠의 체크무늬처럼 **잔무늬**는 흐릿하게 뭉개졌습니다. 숫자 32개짜리 요약 노트에 무엇을 적을지 고르다 보니, 대부분의 사진에 공통으로 중요한 "윤곽과 밝기"를 먼저 적고 드문 무늬는 버린 거예요. 오토인코더의 그림이 대체로 흐릿한 이유가 이것입니다.

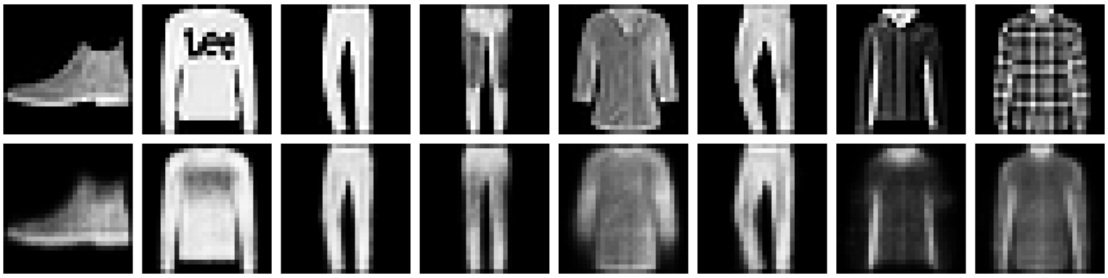


그림 15.1: 위 줄은 시험 사진 원본 8장, 아래 줄은 오토인코더가 잠재 벡터 32칸만 보고 되살린 사진이다. 모양은 잘 살아나지만 로고, 체크무늬 같은 잔무늬는 흐릿하게 뭉개진다.

얼마나 줄일 수 있을까

요약 노트를 짧게 쓸수록 되살리기는 어려워집니다. 잠재 크기를 2, 8, 32, 128로 바꿔 가며 재구성 손실을 비교해 보겠습니다. 비교 상대로 **PCA(주성분 분석)** 도 함께 놓았어요. PCA는 데이터가 가장 많이 퍼져 있는 방향 k개를 찾아 그 방향으로만 그림자를 내리는 고전적인 차원 축소 방법인데, 계산이 모두 행렬 곱이라 **직선(선형)으로만** 줄이고 되살립니다. 오토인코더에서 ReLU를 모두 빼면 PCA와 거의 같은 일을 하게 돼요.

```
import torch
import torch.nn.functional as F

from 생성도구 import 사진_불러오기, 오토인코더, 오토인코더_학습

X_학습, _ = 사진_불러오기(학습=True)
X_시험, _ = 사진_불러오기(학습=False)

# 비교 대상: 직선(선형)으로만 줄이는 PCA. 학습 사진의 평균과 주성분 방향을 구한다
평균 = X_학습.flatten(1).mean(dim=0)
_, _, V = torch.linalg.svd(X_학습.flatten(1) - 평균, full_matrices=False)


def pca_재구성_손실(k: int) -> float:
    """주성분 k개로 줄였다가 되살렸을 때의 시험 사진 MSE."""
    방향 = V[:k].T                                  # (784, k)
    줄임 = (X_시험.flatten(1) - 평균) @ 방향         # (10000, k)
    복원 = 줄임 @ 방향.T + 평균                      # (10000, 784)
```

```
    return F.mse_loss(복원, X_시험.flatten(1)).item()


print("잠재 크기  오토인코더  PCA")
for k in [2, 8, 32, 128]:
    torch.manual_seed(0)
    모델 = 오토인코더_학습(오토인코더(잠재_크기=k), X_학습, 에폭=5, 출력=False).eval()
    with torch.no_grad():
        손실 = F.mse_loss(모델(X_시험), X_시험).item()
    print(f"{k:8}  {손실:10.4f}  {pca_재구성_손실(k):.4f}")
```

```
잠재 크기  오토인코더  PCA
       2      0.0307  0.0461
       8      0.0165  0.0266
      32      0.0134  0.0151
     128      0.0127  0.0063
```

PCA 부분은 4장의 도구만으로 계산했습니다. 평균을 뺀 사진들을 `torch.linalg.svd`로 분해하면 `V`의 앞쪽 줄들이 데이터가 가장 많이 퍼진 방향입니다. 그 방향 k개로 행렬 곱을 해서 줄이고(`@ 방향`), 다시 행렬 곱으로 되살렸어요(`@ 방향.T`). 4장 4.6절의 `(a, b) @ (b, c)` 규칙대로 `(10000, 784) @ (784, k) → (10000, k)`입니다.

결과에서 볼 점이 두 가지예요.

- **아주 작게 줄일수록 오토인코더가 유리합니다.** 2칸과 8칸에서는 오토인코더의 손실이 PCA보다 30% 넘게 낮습니다. 직선으로는 접을 수 없는 구부러진 모양을 ReLU가 접어 주기 때문이에요. 6장 6.2절에서 활성화 함수가 직선을 구부린다고 했던 것과 같은 이야기입니다.
- **넉넉하게 줄이면 PCA도 훌륭합니다.** 128칸에서는 오히려 PCA가 0.0063으로 오토인코더(0.0127)보다 두 배 정확해요. PCA는 선형 방법 중에서는 수학적으로 가장 좋은 답을 한 번에 구하지만, 오토인코더는 경사 하강법으로 다섯 에폭만 학습해서 가장 좋은 답에 닿았다는 보장이 없기 때문입니다. 차원을 줄이는 게 목적이라면 3장의 교훈처럼 **단순한 방법(PCA)부터** 해 보고, 그걸로 부족할 때 오토인코더를 꺼내는 게 순서예요.

15.2 잠재 공간 들여다보기

잠재 크기를 2로 하면 재구성은 흐려지지만 좋은 점이 하나 생깁니다. 사진 한 장이 점 하나(가로, 세로)가 되니 **종이에 그대로 찍어 볼 수 있어요.** 1장 1.3절에서 "수백 개의 특성을 몇 개로 줄여 그림으로 그려 본다"고 했던 것을 직접 해 보겠습니다. 시험 사진 1만 장을 2차원 점으로 바꿔 종류별로 색을 칠합니다. 이 그림을 그릴 때 오토인코더는 정답(종류)을 한 번도 보지 못했다는 점을 기억하세요.

```
import matplotlib.pyplot as plt
import torch

from 생성도구 import 사진_불러오기, 오토인코더, 오토인코더_학습, 이름

X_학습, _ = 사진_불러오기(학습=True)
X_시험, y_시험 = 사진_불러오기(학습=False)

torch.manual_seed(0)
모델 = 오토인코더_학습(오토인코더(잠재_크기=2), X_학습, 에폭=5, 출력=False).eval()
with torch.no_grad():
    점 = 모델.인코더(X_시험)                # 시험 사진 1만 장 → 점 1만 개 (10000, 2)

for 번호, 클래스 in enumerate(이름):
    중심 = 점[y_시험 == 번호].mean(dim=0)
    print(f"{클래스:5}: 중심 ({중심[0]:6.1f}, {중심[1]:6.1f})")

plt.rcParams["font.family"] = "AppleGothic"     # macOS 한글 글꼴 (윈도우는 8장 참고)
plt.rcParams["axes.unicode_minus"] = False
plt.figure(figsize=(8, 6))
for 번호, 클래스 in enumerate(이름):
    고른 = 점[y_시험 == 번호]
    plt.scatter(고른[:, 0], 고른[:, 1], s=2, alpha=0.5, label=클래스)
plt.legend(markerscale=6, fontsize=9, loc="upper left", bbox_to_anchor=(1.01, 1))
plt.xlabel("잠재 1번 칸")
plt.ylabel("잠재 2번 칸")
plt.tight_layout()
plt.savefig("latent_2d.png", dpi=200)
```

```
T셔츠  : 중심 (  9.4,  -2.7)
바지   : 중심 ( 10.8,   3.6)
풀오버  : 중심 (  4.2,  -5.6)
드레스  : 중심 ( 10.3,  -0.3)
코트   : 중심 (  6.0,  -5.5)
샌들   : 중심 ( -4.0,  -0.8)
셔츠   : 중심 (  5.7,  -4.1)
운동화  : 중심 ( -7.5,   0.4)
가방   : 중심 ( -2.6,  -4.6)
앵클부츠 : 중심 ( -5.7,  -4.4)
```

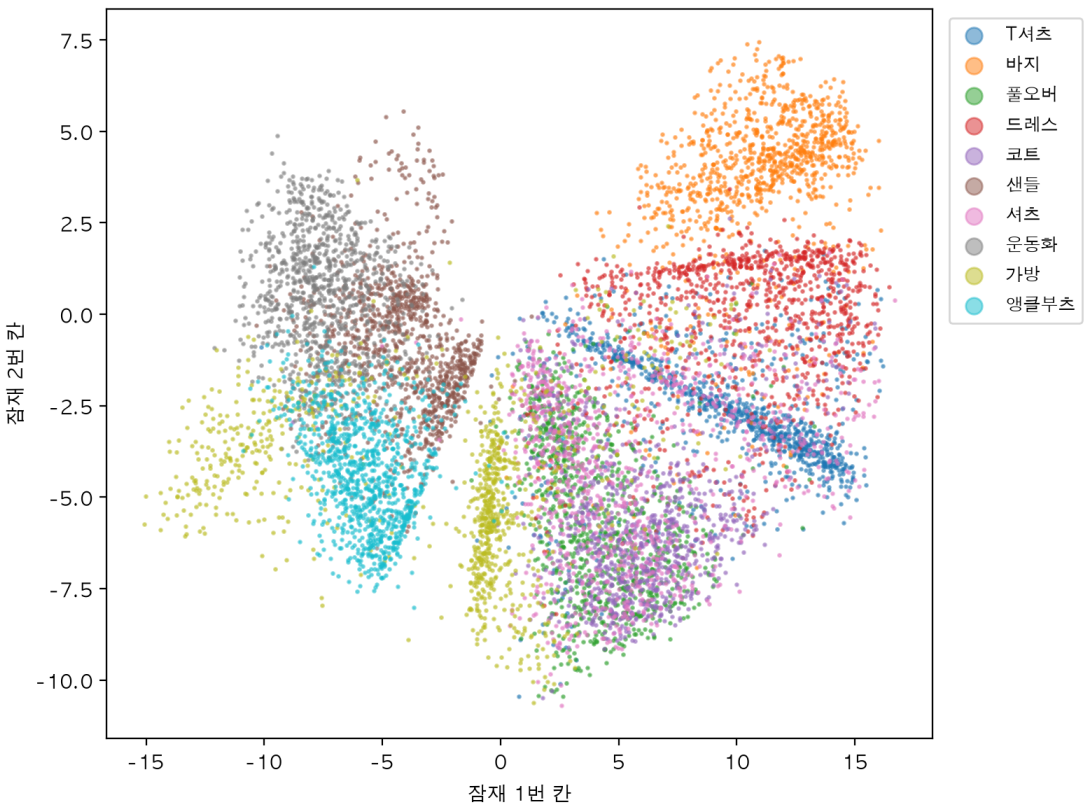


그림 15.2: 2차원 잠재 공간에 시험 사진 1만 장을 찍은 산점도. 왼쪽에는 운동화·샌들·앵클부츠·가방이, 오른쪽에는 바지·드레스·T셔츠가 모이고, 오른쪽 아래에는 풀오버·코트·셔츠가 뒤섞여 있다.

정답을 모르고 배웠는데도 비슷한 것끼리 모였습니다. 신발 셋(운동화, 샌들, 앵클부츠)은 왼쪽에, 옷은 오른쪽에 자리를 잡았어요. 바지는 오른쪽 위에 따로 떨어져 있고, T셔츠와 드레스도 비교적 자

기 자리가 있습니다. 반면 풀오버, 코트, 셔츠는 오른쪽 아래에 한데 뒤섞여 있어요. 6장 6.11절에서 분류기가 가장 약했던 풀오버·셔츠와, 가장 많이 헷갈린 짝인 풀오버·코트가 바로 이 셋입니다. 사진 자체가 비슷하게 생겼으니, 정답 없이 요약하는 오토인코더 눈에도 비슷해 보이는 거죠.

중심 좌표를 보면 그림에서 받은 인상을 숫자로 확인할 수 있습니다. 운동화 (-7.5, 0.4)와 바지 (10.8, 3.6)는 멀리 떨어져 있고, 풀오버 (4.2, -5.6), 코트 (6.0, -5.5), 셔츠 (5.7, -4.1)는 서로 2칸 남짓밖에 떨어져 있지 않아요. 가방은 특이하게 두 무리로 나뉘었는데, 손잡이가 있는 가방과 없는 가방이 꽤 다르게 생겼기 때문으로 보입니다.

Q. 3장의 k-means와 무엇이 다른가요? 3장 3.6절의 k-means는 정해진 개수의 무리로 **묶는** 방법이고, 오토인코더는 무리를 정하지 않고 데이터를 **짧은 좌표로 바꾸는** 방법입니다. 둘을 이어서 쓸 수도 있어요. 784칸 사진에 바로 k-means를 쓰면 픽셀 하나하나의 차이에 휘둘리지만, 오토인코더로 줄인 잠재 벡터에 k-means를 쓰면 "모양이 비슷한 것끼리" 묶이는 경우가 많습니다.

15.3 합성곱 오토인코더로 잡음 지우기

줄이는 합성곱, 키우는 전치 합성곱

지금까지의 오토인코더는 사진을 784칸 한 줄로 펴서 받았습니다. 9장에서 본 것처럼 사진은 펴는 순간 "바로 위아래 픽셀이 무엇인가"라는 공간 정보를 잃어요. 그래서 사진을 다루는 오토인코더는 보통 인코더에 합성곱을 씁니다. `stride=2`인 합성곱은 가로세로를 절반으로 줄이죠.

그럼 디코더는요? 7 × 7을 다시 28 × 28로 **키워야** 합니다. 이때 쓰는 층이 **전치 합성곱**(`nn.ConvTranspose2d`) 이에요. 합성곱이 여러 칸을 보고 한 칸을 만든다면, 전치 합성곱은 한 칸을 여러 칸에 나눠 뿌려서 크기를 키웁니다. 도장을 찍는 것과 비슷해요. 입력 한 칸의 값만큼 진하게 도장(커널)을 찍되, 도장 사이 간격을 `stride`만큼 벌리면 결과가 그만큼 커집니다. 모양부터 확인해 보겠습니다.

```
import torch
from torch import nn

x = torch.zeros(1, 1, 28, 28)                    # 사진 한 장 (장 수, 채널, 세로, 가로)

줄이기 = nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1)
```

```
키우기 = nn.ConvTranspose2d(16, 1, kernel_size=4, stride=2, padding=1)

작아진 = 줄이기(x)
print("Conv2d(stride=2)         :", tuple(x.shape), "→", tuple(작아진.shape))
print("ConvTranspose2d(stride=2):", tuple(작아진.shape), "→", tuple(키우기(작아진).shape))

# 출력 크기 공식: (입력 - 1) × stride - 2 × padding + kernel_size
print("공식으로 계산:", (14 - 1) * 2 - 2 * 1 + 4)
```

```
Conv2d(stride=2)         : (1, 1, 28, 28) → (1, 16, 14, 14)
ConvTranspose2d(stride=2): (1, 16, 14, 14) → (1, 1, 28, 28)
공식으로 계산: 28
```

`stride=2` 합성곱이 28을 14로 줄였고, 같은 `stride=2`의 전치 합성곱이 14를 다시 28로 키웠습니다. 출력 크기는 마지막 줄의 공식으로 계산할 수 있어요. 커널 4, 패딩 1, 보폭 2 조합이 "정확히 두 배"를 만드는 단골 조합이라 이 장에서 계속 씁니다. 채널 수가 16에서 1로 바뀐 것은 합성곱과 같은 규칙이에요. 채널 이야기는 9장을 참고하세요.

잡음 제거 오토인코더

오토인코더를 조금 비틀면 쓸모 있는 도구가 하나 생깁니다. 입력에는 **잡음을 섞은 사진**을 주고, 정답으로는 **깨끗한 원본**을 주는 거예요. 그러면 모델은 "잡음 섞인 사진에서 원래 사진이 무엇이었을지"를 배우게 됩니다. 이것이 **잡음 제거 오토인코더(denoising autoencoder)** 입니다. 지직거리는 옛날 사진을 복원하는 일을 떠올리면 돼요. 그리고 이 생각, "잡음을 걷어 내는 법을 배운다"는 15.8절 확산 모델의 씨앗이기도 합니다.

```
import torch
import torch.nn.functional as F
from torch import nn

from 생성도구 import 격자_저장, 사진_불러오기, 오토인코더_학습


class 합성곱_오토인코더(nn.Module):
    """합성곱으로 28×28 → 7×7로 줄였다가, 전치 합성곱으로 다시 28×28로 키운다."""
```

```
    def __init__(self) -> None:
        super().__init__()
        self.인코더 = nn.Sequential(
            nn.Conv2d(1, 16, 3, stride=2, padding=1), nn.ReLU(),     # (16, 14, 14)
            nn.Conv2d(16, 32, 3, stride=2, padding=1), nn.ReLU(),    # (32, 7, 7)
        )
        self.디코더 = nn.Sequential(
            nn.ConvTranspose2d(32, 16, 4, stride=2, padding=1), nn.ReLU(),  # (16, 14, 14)
            nn.ConvTranspose2d(16, 1, 4, stride=2, padding=1), nn.Sigmoid(),  # (1, 28, 28)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.디코더(self.인코더(x))


torch.manual_seed(0)
X_학습, _ = 사진_불러오기(학습=True)
X_시험, _ = 사진_불러오기(학습=False)

모델 = 합성곱_오토인코더()
print(f"파라미터 {sum(p.numel() for p in 모델.parameters()):,}개")
오토인코더_학습(모델, X_학습, 에폭=3, 잡음=0.4)      # 잡음 섞인 입력 → 깨끗한 원본

모델.eval()
잡음_사진 = (X_시험 + 0.4 * torch.randn_like(X_시험)).clamp(0, 1)
with torch.no_grad():
    고친_사진 = 모델(잡음_사진)
print(f"잡음 사진과 원본의 차이(MSE): {F.mse_loss(잡음_사진, X_시험):.4f}")
print(f"고친 사진과 원본의 차이(MSE): {F.mse_loss(고친_사진, X_시험):.4f}")

격자_저장(torch.cat([잡음_사진[:8], 고친_사진[:8], X_시험[:8]]), "denoise.png")
```

```
파라미터 13,265개
에폭 1: 재구성 손실 0.0444
에폭 2: 재구성 손실 0.0179
```

```
에폭 3: 재구성 손실 0.0164
잡음 사진과 원본의 차이(MSE): 0.0849
고친 사진과 원본의 차이(MSE): 0.0161
```

모델 정의가 9장의 CNN과 다른 점은 디코더에 전치 합성곱을 쓴다는 것뿐입니다. 학습은 `생성도구.py`의 `오토인코더_학습`에 `잡음=0.4`만 넘겼어요. 함수 안에서 배치마다 `x + 0.4 * torch.randn_like(x)`로 잡음을 섞고 `clamp(0, 1)`로 밝기 범위 안에 가둔 뒤, 손실은 **깨끗한 `x`**와 비교합니다.

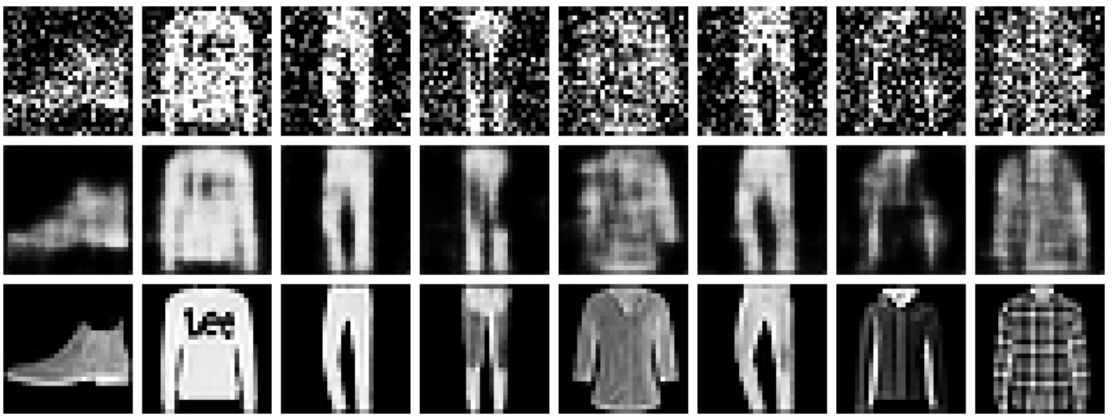


그림 15.3: 위 줄은 잡음을 섞은 시험 사진, 가운데 줄은 잡음 제거 오토인코더가 고친 사진, 아래 줄은 깨끗한 원본이다. 잡음에 묻혀 거의 보이지 않던 옷 모양이 가운데 줄에서 또렷하게 돌아온다.

잡음 사진(위)은 사람 눈에도 무엇인지 알아보기 어려운데, 고친 사진(가운데)은 원본(아래)의 모양을 거의 되찾았습니다. 숫자로도 원본과의 차이가 0.0849에서 0.0161로 5분의 1 남짓으로 줄었어요. 모델이 잡음 아래에 숨은 옷 모양을 "아마 이랬을 것"이라고 추측해서 채운 것입니다. 이 모델의 파라미터는 13,265개로, 15.1절의 완전 연결 오토인코더(약 42만 개)의 30분의 1밖에 안 됩니다. 9장에서 본 합성곱의 장점, 같은 커널을 사진 전체에 돌려 쓴다는 점 덕분이에요.

15.4 재구성 오차로 이상한 것 찾기

오토인코더는 **자주 본 것**을 잘 되살립니다. 그렇다면 한 번도 본 적 없는 것을 넣으면 어떻게 될까요? 요약하는 법을 모르니 잘 되살리지 못하고, 재구성 오차가 커질 겁니다. 이 성질을 이용하면 정답 없이도 **평소와 다른 것(이상치)** 을 찾아낼 수 있어요. 공장에서 정상 제품 사진만 잔뜩 모아 오토인코더를 학습시킨 다음, 재구성이 유난히 안 되는 제품을 불량 후보로 골라내는 식입니다. 불량품은 드

물어서 정답을 모으기 어려운데, 이 방법은 정상 제품만 있으면 되죠.

가방을 "불량품" 역할로 정해서 실험해 보겠습니다. 가방 사진을 한 장도 보여 주지 않고 나머지 아홉 종류로만 학습시켜요.

```
import torch

from 생성도구 import 사진_불러오기, 오토인코더, 오토인코더_학습, 이름

X_학습, y_학습 = 사진_불러오기(학습=True)
X_시험, y_시험 = 사진_불러오기(학습=False)

# 가방(8번)을 한 장도 보여 주지 않고 나머지 아홉 종류로만 학습한다
torch.manual_seed(0)
모델 = 오토인코더_학습(오토인코더(잠재_크기=8), X_학습[y_학습 != 8], 에폭=5, 출력=False).eval()

with torch.no_grad():
    오차 = ((모델(X_시험) - X_시험) ** 2).flatten(1).mean(dim=1)   # 사진 한 장마다 재구성 오차

for 번호 in range(10):
    print(f"{이름[번호]:5} 평균 재구성 오차 {오차[y_시험 == 번호].mean():.4f}")

# 문턱: 학습한 종류의 시험 사진 중 95%가 넘지 않는 오차
문턱 = 오차[y_시험 != 8].quantile(0.95)
print(f"\n문턱 {문턱:.4f}를 넘으면 '이상'으로 판정")
print(f"가방이 이상으로 잡힌 비율: {(오차[y_시험 == 8] > 문턱).float().mean():.1%}")
print(f"다른 옷이 이상으로 잡힌 비율: {(오차[y_시험 != 8] > 문턱).float().mean():.1%}")
```

```
T셔츠   평균 재구성 오차 0.0159
바지    평균 재구성 오차 0.0097
풀오버   평균 재구성 오차 0.0163
드레스   평균 재구성 오차 0.0157
코트    평균 재구성 오차 0.0152
샌들    평균 재구성 오차 0.0247
셔츠    평균 재구성 오차 0.0165
운동화   평균 재구성 오차 0.0125
가방    평균 재구성 오차 0.0395
```

```
앵클부츠  평균 재구성 오차 0.0169

문턱 0.0359를 넘으면 '이상'으로 판정
가방이 이상으로 잡힌 비율: 58.6%
다른 옷이 이상으로 잡힌 비율: 5.0%
```

예상대로 처음 보는 가방의 재구성 오차가 0.0395로 가장 큽니다. 학습한 종류들은 0.0097~0.0247 사이인데, 가방은 그 어느 것보다 크고 대부분 종류의 2~4배예요. 학습한 종류 중에서는 샌들(0.0247)이 가장 큰데, 끈 사이로 바닥이 비치는 복잡한 모양이라 원래 되살리기 어려운 종류입니다.

판정 문턱은 "학습한 종류의 시험 사진 중 95%가 넘지 않는 값"으로 정했습니다. `quantile(0.95)`가 그 값을 구해 줘요. 이 문턱으로 자르면 정상 제품의 5%를 괜히 의심하는 대신, 가방의 58.6%를 이상으로 잡아냅니다. 완벽하지는 않죠. 가방 중에는 네모난 모양이 셔츠나 풀오버와 비슷해서 그럭저럭 되살아나는 것도 많기 때문이에요. 실제로 쓸 때는 문턱을 올리고 내리며 "놓치는 불량"과 "괜한 의심" 사이에서 고르게 되는데, 3장 3.7절에서 본 정밀도와 재현율의 줄다리기와 같은 문제입니다.

15.5 오토인코더로 새 그림을 그릴 수 있을까

이제 이 장의 진짜 질문으로 갑니다. 디코더는 잠재 벡터를 받아 사진을 그리는 부품이에요. 그렇다면 인코더 없이 **아무 잠재 벡터나 만들어서** 디코더에 넣으면, 세상에 없던 새 옷 사진이 나오지 않을까요? 15.2절의 2차원 오토인코더로 해 보겠습니다.

```
import torch

from 생성도구 import 격자_저장, 사진_불러오기, 오토인코더, 오토인코더_학습

torch.manual_seed(0)
X_학습, _ = 사진_불러오기(학습=True)
모델 = 오토인코더_학습(오토인코더(잠재_크기=2), X_학습, 에폭=5, 출력=False).eval()

with torch.no_grad():
    점 = 모델.인코더(X_학습)
    print("잠재 공간 범위: 1번 칸", [round(v, 1) for v in (점[:, 0].min().item(), 점[:,
    ↪ 0].max().item())],
          "/ 2번 칸", [round(v, 1) for v in (점[:, 1].min().item(), 점[:, 1].max().item())])
```

```
    print("잠재 벡터의 평균:", [round(v, 2) for v in 점.mean(dim=0).tolist()],
          "표준편차:", [round(v, 2) for v in 점.std(dim=0).tolist()])

    # 범위 안의 점이라도 '빈 땅'이면? 1번 칸 0, 2번 칸 7 근처는 데이터가 거의 없다
    빈_땅 = torch.tensor([[0.0, 7.0]])
    가까운_개수 = ((점 - 빈_땅).norm(dim=1) < 1.0).sum().item()
    print("(0, 7)에서 거리 1 안에 있는 학습 사진:", 가까운_개수, "장")

    # 표준정규분포에서 아무 점이나 15개 골라 디코더에 넣고, 마지막 칸에는 빈 땅을 넣어 보기
    아무_점 = torch.randn(15, 2)
    격자_저장(모델.디코더(torch.cat([아무_점, 빈_땅])), "ae_random.png")
```

```
잠재 공간 범위: 1번 칸 [-15.8, 17.1] / 2번 칸 [-11.3, 7.7]
잠재 벡터의 평균: [2.7, -2.37] 표준편차: [7.39, 3.53]
(0, 7)에서 거리 1 안에 있는 학습 사진: 0 장
```

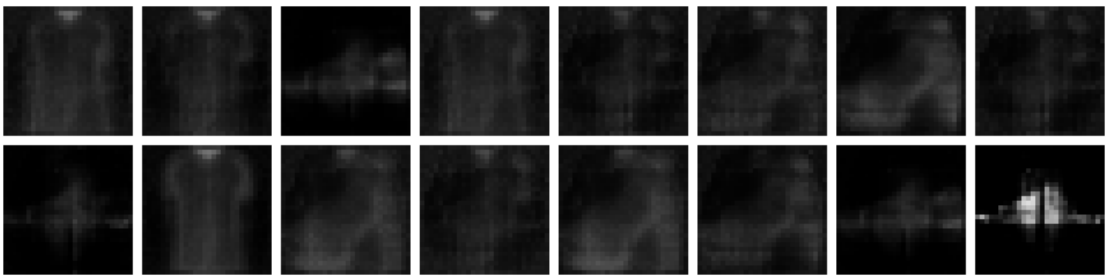


그림 15.4: 오토인코더의 디코더에 표준정규분포에서 뽑은 점 15개와 빈 땅의 점 하나를 넣어 그린 그림 16장. 대부분 어둡고 흐릿한 얼룩이라 무슨 옷인지 알아보기 어렵다.

결과는 실망스럽습니다. 대부분 어두운 얼룩이고, 무엇을 그린 건지 알아보기 어려워요. 마지막 칸, 빈 땅 (0, 7)에서 그린 그림도 마찬가지입니다. 이유는 출력의 숫자에 있어요.

- **잠재 공간의 범위가 제멋대로입니다.** 1번 칸은 -15.8에서 17.1까지, 2번 칸은 -11.3에서 7.7까지 퍼져 있고, 평균은 (2.7, -2.37), 표준편차는 (7.39, 3.53)입니다. 그런데 우리는 `torch.randn`으로 평균 0, 표준편차 1인 좁은 곳에서 점을 뽑았어요. 데이터가 사는 동네와 엉뚱한 곳을 뒤진 셈입니다.
- **범위 안에도 빈 땅이 많습니다.** 15.2절의 그림을 보면 점들이 몇 무리로 뭉쳐 있고 사이사이가 비어 있죠. 좌표 (0, 7) 근처에는 학습 사진이 0장입니다. 디코더는 학습하는 동안 이런 곳의

점을 한 번도 받아 본 적이 없으니, 무엇을 그려야 할지 모릅니다.

오토인코더는 "되살리기"만 배웠지, 잠재 공간을 **고르게 쓰라는** 요구는 받은 적이 없습니다. 그래서 요약들이 제멋대로 흩어지고, 그 사이 빈 땅은 아무 의미도 없는 곳이 돼요. 그림을 그리려면 잠재 공간이 이렇게 생겨야 합니다.

1. 데이터가 사는 곳이 **정해진 동네**(예: 평균 0, 표준편차 1 근처)여야 한다. 그래야 어디서 점을 뽑을지 안다.
2. 동네 안에 **빈 땅이 없어야** 한다. 어느 점을 뽑아도 그럴듯한 그림이 나와야 한다.

이 두 가지를 손실 함수로 강제한 것이 다음 절의 VAE입니다.

15.6 VAE: 점 대신 구름으로

아이디어: 정확한 주소 대신 동네

오토인코더의 인코더는 사진마다 **정확한 주소 하나**(점)를 줍니다. 디코더는 그 주소만 정확히 외우면 되니, 주소와 주소 사이는 신경 쓸 필요가 없었죠. 그래서 빈 땅이 생겼습니다.

VAE(Variational AutoEncoder, 변분 오토인코더) 는 인코더가 점 대신 **"이 근처 어딘가"라는 동네**를 줍니다. 구체적으로는 평균(동네의 중심)과 분산(동네의 넓이)을 내고, 학습할 때마다 그 동네 안에서 점을 **무작위로 하나 뽑아** 디코더에 넘겨요. 같은 사진이라도 매번 조금씩 다른 점이 디코더에 들어가니, 디코더는 중심점뿐 아니라 **그 주변 전체**에서 비슷한 그림을 그리는 법을 배우게 됩니다. 사진마다 점이 아니라 작은 구름을 차지하는 셈이고, 구름들이 서로 겹치면서 빈 땅이 메워져요.

여기에 벌점을 하나 더합니다. 각 사진의 구름이 **평균 0, 표준편차 1인 표준정규분포**에서 너무 멀어지지 않게 하는 벌점이에요. 이 벌점을 **KL 발산(Kullback-Leibler divergence)** 이라고 부르는데, 두 분포가 얼마나 다른지를 재는 값입니다. 이 벌점 덕분에 구름들이 원점 근처로 모이고(조건 1), 너무 작게 쪼그라들지도 못합니다(조건 2). 구름을 점처럼 작게 만들면 표준편차 1에서 멀어지니 벌점이 커지거든요.

그래서 VAE의 손실은 두 항의 합입니다.

항	하는 일	혼자 두면
재구성 손실	원본을 잘 되살리게 한다	구름을 점으로 줄이고 제각각 흩어 놓는다 (오토인코더)

항	하는 일	혼자 두면
KL 발산	구름을 원점 근처로 모으고 넓게 편다	모든 사진의 구름이 똑같아져서 무엇을 넣든 같은 그림

두 힘이 줄다리기를 한 끝에, 비슷한 사진의 구름은 가까이 모이고 전체는 원점 근처를 빈틈없이 채우는 잠재 공간이 만들어집니다. 그러면 그림을 그릴 때는 인코더가 필요 없어요. 표준정규분포에서 점을 하나 뽑아 디코더에 넣기만 하면 됩니다.

무작위로 뽑으면서 미분하기: 재매개변수화

그런데 문제가 하나 있습니다. 5장에서 본 자동 미분은 **계산**을 거꾸로 따라가며 기울기를 구했어요. "동네 안에서 무작위로 하나 뽑기"는 계산이 아니라 제비뽑기라서, 뽑힌 점이 평균과 분산에 어떻게 달려 있는지 알 길이 없습니다. 그러면 인코더까지 기울기가 흐르지 못해요.

해결책은 뽑기를 두 단계로 나누는 겁니다. 먼저 평균 0, 표준편차 1인 표준정규분포에서 **잡음**을 하나 뽑아요. 그다음 `z = 평균 + 표준편차 × 잡음`으로 **계산**합니다. 결과는 "평균, 표준편차인 정규분포에서 뽑은 것"과 똑같은 분포가 되는데, 이번에는 무작위성이 전부 잡음 쪽으로 빠져나가고 `z`는 평균과 표준편차의 평범한 계산 결과가 돼요. 이 요령을 **재매개변수화(reparameterization trick)**라고 합니다. 확인해 보겠습니다.

```
import torch
from torch.distributions import Normal, kl_divergence

평균 = torch.tensor([0.5, -1.0], requires_grad=True)
로그분산 = torch.tensor([0.0, -2.0], requires_grad=True)
표준편차 = torch.exp(0.5 * 로그분산)              # 로그분산 → 표준편차
분포 = Normal(평균, 표준편차)

# 1) 그냥 뽑기: 뽑힌 값은 계산 기록이 없는 '상수'다
torch.manual_seed(1)
z = 분포.sample()
print("sample()  :", z, "¦ grad_fn:", z.grad_fn)

# 2) 재매개변수화: 무작위는 잡음에 맡기고, z는 평균과 표준편차로 '계산'한다
torch.manual_seed(0)
```

```
잡음 = torch.randn(2)
z = 평균 + 표준편차 * 잡음
print("직접 계산 :", z.detach(), "¦ grad_fn:", type(z.grad_fn).__name__)
torch.manual_seed(0)
print("rsample() :", 분포.rsample().detach())     # 같은 씨앗이면 같은 값

z.sum().backward()
print("평균의 기울기:", 평균.grad)
print("로그분산의 기울기:", 로그분산.grad)

# KL 발산: N(평균, 표준편차²)가 표준정규분포 N(0, 1)에서 얼마나 떨어졌나
kl_공식 = -0.5 * (1 + 로그분산 - 평균 ** 2 - 로그분산.exp())
print("KL (공식)             :", kl_공식.detach())
print("KL (torch.distributions):", kl_divergence(분포, Normal(0.0, 1.0)).detach())
```

```
sample()  : tensor([ 1.1614, -0.9018]) ¦ grad_fn: None
직접 계산 : tensor([ 2.0410, -1.1079]) ¦ grad_fn: AddBackward0
rsample() : tensor([ 2.0410, -1.1079])
평균의 기울기: tensor([1., 1.])
로그분산의 기울기: tensor([ 0.7705, -0.0540])
KL (공식)             : tensor([0.1250, 1.0677])
KL (torch.distributions): tensor([0.1250, 1.0677])
```

`torch.distributions`의 `Normal`은 정규분포 하나를 나타내는 객체입니다. `sample()`로 뽑은 값은 `grad_fn`이 `None`이에요. 5장 5.5절에서 본 계산 그래프가 거기서 끊겨 있다는 뜻입니다. 반면 직접 계산한 `z`에는 `AddBackward0`가 붙어 있고, `backward()`를 하니 평균과 로그분산 모두에 기울기가 생겼습니다. 평균의 기울기가 1인 것은 `z = 평균 + …`이니 당연하고, 로그분산의 기울기는 뽑힌 잡음에 따라 달라져요. `rsample()`은 PyTorch가 같은 요령을 대신 해 주는 함수라서, 같은 씨앗이면 직접 계산한 값과 똑같습니다.

분산을 바로 내지 않고 **로그분산**을 내는 데도 이유가 있습니다. 분산은 언제나 양수여야 하는데, 신경망 출력은 음수도 나오거든요. 로그분산은 어떤 실수여도 괜찮고, `exp(0.5 × 로그분산)`을 하면 언제나 양수인 표준편차가 됩니다.

마지막 두 줄은 KL 발산입니다. 정규분포와 표준정규분포 사이의 KL 발산은 `-0.5 × (1 + 로그분산 - 평균² - 분산)`이라는 간단한 공식으로 계산할 수 있고, `torch.distributions.kl_divergence`로

구한 값과 똑같이 나왔어요. 첫 번째 칸은 평균이 0.5만큼 벗어나서 0.125, 두 번째 칸은 평균이 -1이고 분산도 0.135로 쪼그라들어서 1.068로 벌점이 큽니다. 평균 0, 로그분산 0(분산 1)이면 이 값은 정확히 0이에요.

VAE 학습하고 새 그림 그리기

이제 VAE를 만듭니다. 15.5절과 비교하기 쉽게 잠재 크기는 2로 했어요.

```
import torch
import torch.nn.functional as F
from torch import nn

from 생성도구 import 격자_저장, 로더_만들기, 사진_불러오기


class VAE(nn.Module):
    """인코더가 점 하나 대신 '평균과 로그분산'을 내는 변분 오토인코더."""

    def __init__(self, 잠재_크기: int = 2) -> None:
        super().__init__()
        self.몸통 = nn.Sequential(nn.Flatten(), nn.Linear(784, 256), nn.ReLU())
        self.평균 = nn.Linear(256, 잠재_크기)
        self.로그분산 = nn.Linear(256, 잠재_크기)
        self.디코더 = nn.Sequential(
            nn.Linear(잠재_크기, 256), nn.ReLU(),
            nn.Linear(256, 784),                    # 로짓. 시그모이드는 손실 함수와 그림 그릴 때
            nn.Unflatten(1, (1, 28, 28)),
        )

    def forward(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
        h = self.몸통(x)
        평균, 로그분산 = self.평균(h), self.로그분산(h)
        z = 평균 + torch.exp(0.5 * 로그분산) * torch.randn_like(평균)   # 재매개변수화
        return self.디코더(z), 평균, 로그분산


def vae_손실(로짓: torch.Tensor, x: torch.Tensor, 평균: torch.Tensor,
```

```
            로그분산: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
    """사진 한 장당 재구성 손실과 KL 발산 (둘 다 칸별 값을 더한 뒤 배치 평균)."""
    재구성 = F.binary_cross_entropy_with_logits(로짓, x, reduction="sum") / len(x)
    kl = -0.5 * (1 + 로그분산 - 평균 ** 2 - 로그분산.exp()).sum() / len(x)
    return 재구성, kl


torch.manual_seed(0)
X_학습, _ = 사진_불러오기(학습=True)
모델 = VAE(잠재_크기=2)
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)

for 에폭 in range(1, 11):
    합계 = torch.zeros(2)
    for (x,) in 로더_만들기(X_학습, 씨앗=에폭):
        로짓, 평균, 로그분산 = 모델(x)
        재구성, kl = vae_손실(로짓, x, 평균, 로그분산)
        옵티마이저.zero_grad()
        (재구성 + kl).backward()
        옵티마이저.step()
        합계 += torch.tensor([재구성.item(), kl.item()]) * len(x)
    재구성_평균, kl_평균 = (합계 / len(X_학습)).tolist()
    if 에폭 in (1, 2, 5, 10):
        print(f"에폭 {에폭:2}: 재구성 {재구성_평균:6.1f}, KL {kl_평균:4.2f}")

torch.save(모델.state_dict(), "vae.pt")

# 표준정규분포에서 점을 뽑아 디코더에 넣으면 새 사진이 나온다
모델.eval()
with torch.no_grad():
    새_사진 = torch.sigmoid(모델.디코더(torch.randn(1000, 2)))
격자_저장(새_사진[:32], "vae_samples.png")
torch.save(새_사진, "vae_1000.pt")                # 15.9절에서 채점할 1,000장
```

```
에폭  1: 재구성  304.1, KL 7.86
에폭  2: 재구성  272.3, KL 6.58
에폭  5: 재구성  263.7, KL 6.41
에폭 10: 재구성  259.2, KL 6.46
```

오토인코더와 달라진 곳은 세 군데입니다.

- **인코더가 둘로 갈립니다.** 공통 **몸통**을 지난 뒤 **평균** 층과 **로그분산** 층이 따로 값을 냅니다.
- **`forward`에서 재매개변수화로 `z`를 뽑습니다.** 그래서 같은 사진을 넣어도 매번 조금 다른 그림이 나와요.
- **손실이 두 항입니다.** 재구성 손실은 MSE 대신 6장 6.5절의 이진 교차 엔트로피(`binary_cross_entropy_with_logits`)를 썼습니다. 픽셀 밝기 0~1을 "그 칸이 밝을 확률"처럼 보고 맞히는 거예요. 6장의 조언대로 디코더 끝에는 시그모이드를 넣지 않고 로짓을 내게 했습니다. 두 항 모두 칸별 값을 **더한** 다음 배치 크기로 나눠서 "사진 한 장당" 값으로 맞췄어요. 한쪽만 평균을 내면 두 힘의 균형이 무너집니다.

재구성 손실은 304.1에서 259.2로 줄었고, KL 발산은 6.4 근처에서 자리를 잡았습니다. KL이 0이 아니라는 건 구름들이 표준정규분포와 완전히 같지는 않다는 뜻이에요. 같아지면 사진마다 구별이 안 되니 당연합니다. 학습한 모델은 `vae.pt`로 저장하고, 표준정규분포에서 점 1,000개를 뽑아 그린 그림도 15.9절에서 채점하려고 `vae_1000.pt`로 저장했어요.

15.5절의 얼룩과 비교해 보세요. 똑같이 `torch.randn`으로 점을 뽑았는데, 이번에는 앵클부츠, 운동화, T셔츠, 바지, 긴소매 옷이 알아볼 수 있게 나왔습니다. 처음으로 모델이 **새 그림**을 그린 거예요. 다만 모두 흐릿합니다. 잠재 공간이 2칸뿐이라 담을 수 있는 정보가 적은 데다, VAE는 구름 속 여러 점이 비슷한 그림을 내도록 배우다 보니 그 점들의 "평균적인 그림"을 그리는 경향이 있거든요. 이 흐릿함은 VAE의 대표적인 약점이고, 15.8절의 확산 모델이 이 문제를 크게 줄입니다.

잠재 공간 지도와 보간

잠재 공간이 2칸이면 재미있는 그림을 하나 더 그릴 수 있습니다. 표준정규분포의 대부분이 들어 있는 -2.5~2.5 범위를 바둑판처럼 나눠서, **모든 칸의 점을 디코더에 넣어** 보는 거예요. 잠재 공간의 지도를 그리는 셈입니다. 저장한 `vae.pt`를 불러와서 해 보겠습니다. 6장 6.11절에서 한 것처럼 같은 설계도를 다시 만들고 숫자를 채웠어요. 그리기에는 디코더만 쓰니 `forward`는 생략했습니다.

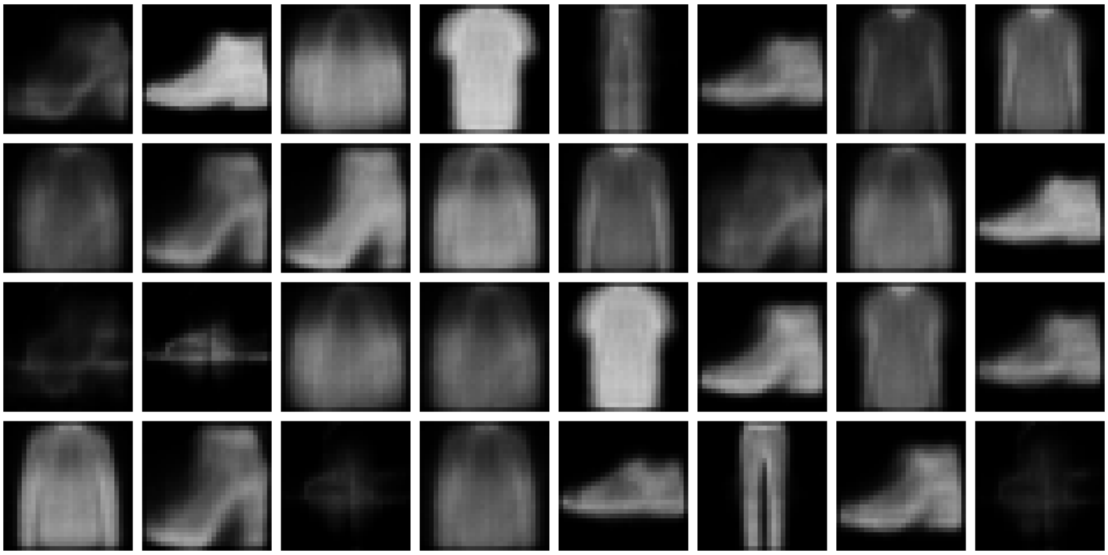


그림 15.5: VAE의 디코더에 표준정규분포에서 뽑은 점 32개를 넣어 그린 그림. 흐릿하지만 앵클부츠, 운동화, T셔츠, 바지, 긴소매 옷처럼 무엇인지 알아볼 수 있는 모양이 나온다.

```
import torch
from torch import nn

from 생성도구 import 격자_저장, 사진_불러오기, 이름


class VAE(nn.Module):
    """15.6절에서 학습한 VAE와 같은 설계도."""

    def __init__(self, 잠재_크기: int = 2) -> None:
        super().__init__()
        self.몸통 = nn.Sequential(nn.Flatten(), nn.Linear(784, 256), nn.ReLU())
        self.평균 = nn.Linear(256, 잠재_크기)
        self.로그분산 = nn.Linear(256, 잠재_크기)
        self.디코더 = nn.Sequential(nn.Linear(잠재_크기, 256), nn.ReLU(),
                                  nn.Linear(256, 784), nn.Unflatten(1, (1, 28, 28)))


모델 = VAE()
```

```
모델.load_state_dict(torch.load("vae.pt"))
모델.eval()

# 1) 잠재 공간 지도: -2.5~2.5 사이를 가로세로 12칸씩 나눠 모든 점을 그려 본다
눈금 = torch.linspace(-2.5, 2.5, 12)
격자_점 = torch.stack(torch.meshgrid(눈금, -눈금, indexing="xy"), dim=-1).reshape(-1, 2)
with torch.no_grad():
    격자_저장(torch.sigmoid(모델.디코더(격자_점)), "vae_map.png", 한_줄=12)

# 2) 보간: T셔츠 한 장에서 바지 한 장으로 조금씩 옮겨 가기
X, y = 사진_불러오기(학습=False)
A, B = X[y == 0][0:1], X[y == 1][0:1]
with torch.no_grad():
    za, zb = 모델.평균(모델.몸통(A)), 모델.평균(모델.몸통(B))
    print(f"{이름[0]}의 잠재 위치: {[round(v, 2) for v in za[0].tolist()]}")
    print(f"{이름[1]}의 잠재 위치: {[round(v, 2) for v in zb[0].tolist()]}")
    비율 = torch.linspace(0, 1, 8).view(-1, 1)
    픽셀_섞기 = (1 - 비율.view(-1, 1, 1, 1)) * A + 비율.view(-1, 1, 1, 1) * B
    잠재_보간 = torch.sigmoid(모델.디코더((1 - 비율) * za + 비율 * zb))
격자_저장(torch.cat([픽셀_섞기, 잠재_보간]), "interp.png")
```

```
T셔츠의 잠재 위치: [-1.15, 1.8]
바지의 잠재 위치: [-2.86, -0.01]
```

`torch.meshgrid`는 가로 눈금과 세로 눈금으로 바둑판의 모든 교차점을 만들어 줍니다. 세로 눈금에 `-`를 붙인 건 그림의 위쪽이 2번 칸의 큰 값이 되게 하려는 거예요. 결과 지도에서는 **빈 땅이 보이지 않습니다.** 어느 칸에서 그려도 무언가 옷이나 신발이 나오고, 이웃한 칸끼리는 모양이 조금씩만 달라요. T셔츠가 오른쪽으로 가며 소매가 길어져 풀오버가 되고, 아래로 내려가며 드레스, 바지로 바뀌고, 오른쪽 아래로 가면 앵클부츠와 운동화가 됩니다. KL 벌점이 구름들을 원점 근처에 빈틈없이 모아 놓은 결과예요.

이 성질 덕분에 **보간(interpolation)** 도 할 수 있습니다. 두 사진의 잠재 위치 사이를 직선으로 조금씩 옮겨 가며 그리면, 한 사진이 다른 사진으로 부드럽게 바뀌어 가는 모습을 볼 수 있어요.

위 줄은 비교를 위해 **픽셀을 직접 섞은** 것입니다. 두 사진을 반투명하게 겹친 것일 뿐이라, 중간 그림은 T셔츠와 바지가 동시에 보이는 이상한 사진이에요. 아래 줄은 잠재 공간에서 T셔츠의 위치 (-1.15,

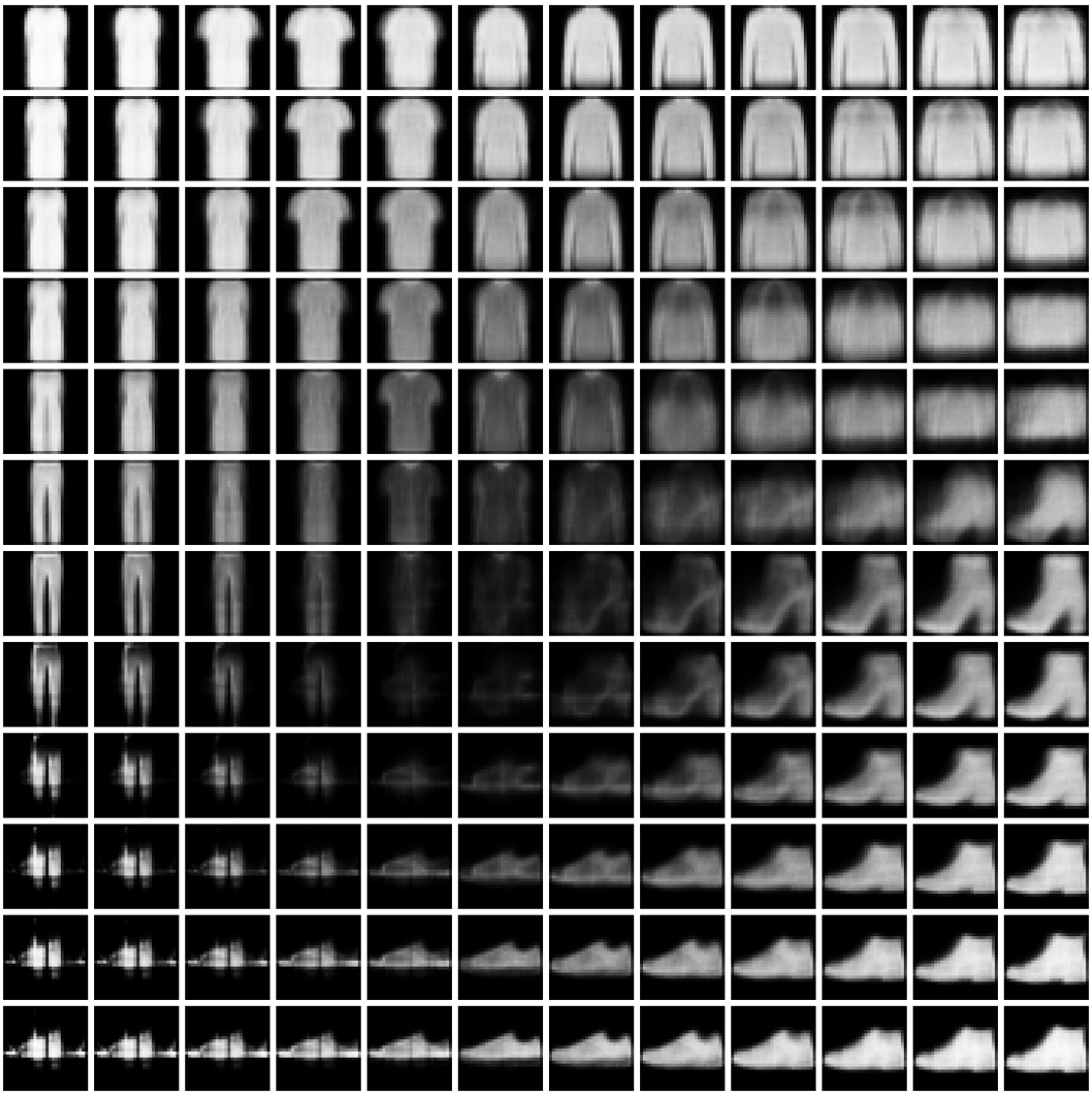


그림 15.6: VAE 잠재 공간을 12 × 12 바둑판으로 나눠 각 점을 디코더로 그린 지도. 왼쪽 위의 T셔츠가 오른쪽으로 갈수록 긴소매 옷으로, 아래로 갈수록 바지와 가방으로 바뀌고, 오른쪽 아래에는 운동화와 앵클부츠가 자리 잡는다. 이웃한 칸끼리는 모양이 조금씩만 다르다.

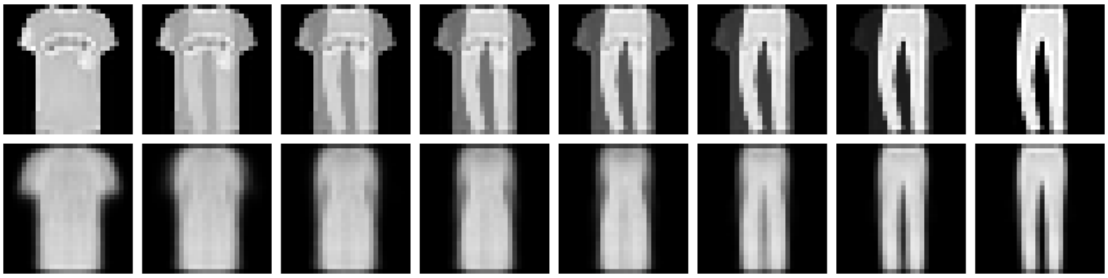


그림 15.7: 위 줄은 T셔츠 사진과 바지 사진의 픽셀을 비율대로 섞은 것, 아래 줄은 VAE 잠재 공간에서 두 사진 사이를 8단계로 옮겨 가며 그린 것이다. 위 줄은 두 사진이 겹쳐 보이는 반투명 합성이고, 아래 줄은 T셔츠가 드레스를 거쳐 바지로 모양 자체가 바뀐다.

1.8)부터 바지의 위치 (-2.86, -0.01)까지 옮겨 가며 그린 것입니다. 중간 그림들이 **그 자체로 그럴듯한 옷**이에요. T셔츠가 길어져 드레스가 되고, 드레스가 갈라져 바지가 됩니다. 잠재 공간에서 가깝다는 것이 "모양이 비슷하다"는 뜻이 되었기 때문이에요.

Q

Q. KL 항에 곱하는 비율을 바꾸면 어떻게 되나요? 손실을 `재구성 + 베타 × KL`로 두고 베타를 바꾸는 방법이 있습니다(베타-VAE라고 불러요). 베타를 키우면 잠재 공간이 더 고르게 정리되어 지도와 보간은 매끄러워지지만, 사진마다의 개성이 지워져서 재구성은 더 흐려져요. 줄이면 반대로 오토인코더에 가까워집니다. 이 장의 예제에서 `(재구성 + kl)`을 `(재구성 + 4 * kl)`, `(재구성 + 0.25 * kl)`로 바꿔 지도를 다시 그려 보면 차이를 바로 볼 수 있어요.

15.7 GAN: 위조범과 감별사

확산 모델로 가기 전에, 한동안 그림 생성의 대표였던 GAN(Generative Adversarial Network, **생성적 적대 신경망**)의 생각을 짧게 짚고 가겠습니다. 이 장에서 직접 구현하지는 않아요. 학습이 까다로워서 노트북 규모 예제로는 원리보다 요령 이야기가 길어지기 때문입니다.

GAN은 두 모델을 **경쟁**시킵니다. 위조지폐범과 감별사를 떠올려 보세요.

- **생성자(generator)**: 잡음을 받아 그림을 그립니다. VAE의 디코더와 같은 자리예요. 목표는 감별사를 속이는 것입니다.
- **판별자(discriminator)**: 그림 한 장을 받아 진짜인지 생성자가 그린 가짜인지 맞힙니다. 6장의 이진 분류기와 같은 모양이에요.

둘은 번갈아 학습합니다. 판별자는 진짜와 가짜를 더 잘 가려내도록, 생성자는 판별자가 "진짜"라고 답하도록 고쳐 나가요. 위조범이 실력을 키우면 감별사도 눈이 높아지고, 감별사가 깐깐해지면 위조범은 더 정교해집니다. 이 경쟁이 잘 굴러가면 생성자는 진짜와 구별하기 어려운 **선명한** 그림을 그리게 돼요. VAE처럼 "평균적인 그림"으로 도망갈 수가 없거든요. 흐릿한 그림은 판별자가 바로 가짜라고 알아보니까요.

문제는 이 경쟁이 **잘 굴러가기가 어렵다**는 것입니다. 한쪽이 너무 강해지면 다른 쪽이 배울 거리를 잃어 학습이 멈추고, 생성자가 판별자를 속이기 쉬운 그림 몇 가지만 반복해서 그리는 일(**모드 붕괴**)도 자주 생깁니다. 운동화만 잘 그리면 속일 수 있으니 운동화만 그리는 식이에요. 그래서 GAN은 학습률, 구조, 정규화를 섬세하게 맞추는 요령이 많이 필요합니다.

	VAE	GAN	확산 모델
학습 방식	되살리기 + KL 벌점	생성자와 판별자의 경쟁	잡음 맞히기 (MSE 하나)
학습 안정성	안정적	까다로움	안정적
그림 품질	흐릿한 편	선명함	선명함
다양성	좋음	모드 붕괴 위험	좋음
그리는 속도	디코더 한 번	생성자 한 번	수십~수천 단계 (느림)

확산 모델이 GAN의 자리를 넘겨받은 이유가 표에 보입니다. 학습은 MSE 하나로 안정적인데 그림은 선명하고 다양해요. 대신 그림 한 장을 그리는 데 시간이 오래 걸린다는 약점이 있습니다. 이제 그 확산 모델을 만들어 봅시다.

15.8 확산 모델: 잡음을 걷어 내며 그리기

잉크 한 방울이 퍼지는 과정을 거꾸로

맑은 물컵에 잉크를 한 방울 떨어뜨리면, 잉크는 조금씩 퍼지다가 결국 물 전체가 고르게 흐려집니다. 이 과정을 거꾸로 돌리는 영상을 상상해 보세요. 흐린 물에서 잉크가 조금씩 모여 처음의 방울 모양으로 돌아옵니다. 현실에서는 일어나지 않지만, "한 걸음 전에는 어땠을까"를 아주 잘 추측하는 모델이 있다면 흉내는 낼 수 있겠죠.

확산 모델(diffusion model) 이 이 생각을 사진에 씁니다. 두 과정으로 되어 있어요.

- **앞으로 가는 과정(잡음 더하기)**: 사진에 잡음을 아주 조금씩, 수백 수천 번 섞어서 결국 순수한 잡음으로 만듭니다. 학습이 필요 없는 정해진 계산이에요.
- **거꾸로 가는 과정(잡음 빼기)**: 순수한 잡음에서 출발해 잡음을 조금씩 걷어 내며 사진으로 돌아옵니다. 이 "조금 걷어 내기"를 신경망이 배웁니다.

거꾸로 가는 과정을 잘 배우면, 아무 잡음에서나 출발해도 그럴듯한 사진에 도착합니다. 출발점인 잡음이 매번 다르니 도착하는 사진도 매번 다르고요. 이 장에서 구현하는 방법은 2020년에 발표된 DDPM(Denoising Diffusion Probabilistic Models) 의 가장 기본적인 형태입니다.

앞으로 가는 과정: 잡음을 한 번에 섞는 공식

잡음을 섞는 방법은 이렇습니다. 단계 t마다 사진을 아주 조금 줄이고(√알파 배), 그만큼 잡음을 조금 섞어요(√베타 배). 알파는 1 - 베타입니다. 베타는 단계마다 섞는 잡음의 양으로, 처음엔 0.0001처럼 아주 작게, 끝으로 갈수록 0.02까지 조금씩 키웁니다. 이 방식에는 고마운 성질이 있어요. 이걸 t번 되풀이한 결과를 **한 번에** 계산할 수 있습니다.

> 섞인 사진 = √a × 원본 + √(1 - a) × 잡음 (a는 0단계부터 t단계까지의 알파를 모두 곱한 값)

학습할 때 사진마다 1,000단계 중 아무 단계나 골라야 하는데, 그때마다 잡음을 수백 번 섞을 필요 없이 이 공식 한 줄이면 됩니다. 확인해 보겠습니다. 아래 코드는 바로 뒤에서 소개할 `확산도구.py`에서 잡음 일정과 `잡음_더하기` 함수를 가져다 써요. 파일 내용은 조금 뒤에 보여 드릴 테니 지금은 결과부터 보세요.

```
import torch

from 생성도구 import 격자_저장, 사진_불러오기
from 확산도구 import T, 베타, 알파, 알파_누적, 잡음_더하기

X, _ = 사진_불러오기(학습=False)
x0 = X[:1] * 2 - 1                              # 밝기 0~1 → -1~1 (잡음과 눈금을 맞춘다)

print("단계 t   원본 비율 √a   잡음 비율 √(1-a)   (a = 알파_누적[t])")
for t in [0, 50, 100, 200, 400, 700, 999]:
    a = 알파_누적[t]
    print(f"{t:6}   {a.sqrt():12.3f}   {(1 - a).sqrt():16.3f}")

# 같은 사진에 단계별로 잡음을 섞어 한 줄로 저장
```

```
torch.manual_seed(0)
단계들 = torch.tensor([0, 50, 100, 200, 300, 500, 700, 999])
섞인 = 잡음_더하기(x0.expand(8, -1, -1, -1), 단계들, torch.randn(8, 1, 28, 28))
격자_저장((섞인 + 1) / 2, "forward.png")

# 한 걸음씩 300번 섞은 것과 공식으로 한 번에 섞은 것은 같은 분포일까?
torch.manual_seed(0)
x = x0.expand(5000, -1, -1, -1).clone()          # 같은 사진 5,000장
for t in range(300):
    x = 알파[t].sqrt() * x + 베타[t].sqrt() * torch.randn_like(x)
a = 알파_누적[299]
print(f"한 걸음씩: 평균의 평균 오차 {(x.mean(0) - a.sqrt() * x0[0]).abs().mean():.4f}, "
      f"표준편차 {x.std(0).mean():.4f}")
print(f"공식     : 표준편차 √(1-a) = {(1 - a).sqrt():.4f}")
```

```
단계 t   원본 비율 √a   잡음 비율 √(1-a)   (a = 알파_누적[t])
     0          1.000              0.010
    50          0.985              0.173
   100          0.946              0.324
   200          0.810              0.586
   400          0.440              0.898
   700          0.083              0.997
   999          0.006              1.000
한 걸음씩: 평균의 평균 오차 0.0088, 표준편차 0.7766
공식     : 표준편차 √(1-a) = 0.7769
```

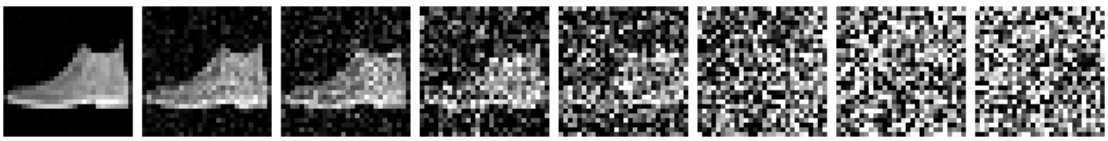


그림 15.8: 앵클부츠 사진 한 장에 0, 50, 100, 200, 300, 500, 700, 999단계만큼 잡음을 섞은 모습. 왼쪽에서 오른쪽으로 갈수록 부츠가 잡음에 묻혀, 오른쪽 끝에서는 순수한 잡음만 남는다.

확산 모델에서는 사진의 밝기를 0~1이 아니라 **-1~1**로 바꿔 씁니다. 섞을 잡음이 평균 0, 표준편차 1이라서 눈금을 비슷하게 맞추는 거예요. 그림으로 저장할 때만 `(x + 1) / 2`로 0~1로 되돌립니다.

표를 보면 원본 비율 √a는 단계가 갈수록 줄어 999단계에는 0.006, 원본이 거의 남지 않아요. 잡음 비율 √(1 - a)는 1에 가까워집니다. 그림에서도 200단계쯤까지는 부츠가 보이다가, 500단계를 넘으면 사람 눈에는 잡음만 보여요. 마지막 두 줄은 공식이 맞는지 확인한 실험입니다. 같은 사진 5,000장에 한 걸음씩 300번 잡음을 섞었더니 픽셀별 표준편차가 0.7766으로, 공식 √(1 - a) = 0.7769와 소수점 셋째 자리까지 같아요. 평균도 공식의 √a × 원본과 평균 0.0088밖에 차이 나지 않습니다. 300번 걸어간 것과 공식으로 한 번에 뛴 것이 같은 분포라는 뜻이에요.

거꾸로 가는 과정: 섞인 잡음 맞히기

그럼 신경망에게는 무엇을 배우라고 할까요? 15.3절의 잡음 제거 오토인코더처럼 "깨끗한 원본을 그려라"라고 할 수도 있지만, DDPM은 조금 다른 문제를 냅니다. **"이 사진에 섞인 잡음이 무엇이었는지 맞혀라."** 잡음을 알면 섞인 사진에서 빼서 원본 쪽으로 돌아갈 수 있으니 결국 같은 일인데, 이렇게 물어야 학습이 더 잘 된다는 것이 실험으로 알려져 있어요.

학습 한 걸음은 이렇습니다.

1. 학습 사진을 몇 장 고르고, 사진마다 단계 t를 1,000 중에서 무작위로 고른다.
2. 잡음을 뽑아서 위의 공식으로 t단계만큼 섞는다.
3. 신경망에 **섞인 사진과 t**를 주고, 섞은 잡음을 맞히게 한다.
4. 맞힌 잡음과 실제 잡음의 MSE를 손실로 학습한다.

손실은 6장 6.5절의 MSE 하나뿐입니다. GAN처럼 두 모델을 경쟁시킬 필요가 없어요. 신경망이 t를 함께 받는 이유는, 같은 사진이라도 몇 단계인지에 따라 섞인 잡음의 양이 다르기 때문입니다. 10단계 사진에서는 잡음이 살짝 묻었을 뿐이고, 900단계 사진은 거의 잡음이죠.

그림을 그릴 때는 반대로 갑니다. 순수한 잡음 x에서 출발해, 999단계부터 0단계까지 한 걸음씩 신경망이 맞힌 잡음의 일부를 빼고, 약간의 무작위를 다시 섞어요. 한 걸음의 계산은 아래 `그리기` 함수 안의 두 줄인데, 공식이 어떻게 나왔는지는 확률 이론이 필요해서 이 책에서는 다루지 않습니다. "맞힌 잡음을 이번 단계에 맞는 만큼 빼고, 사진 크기를 되돌리는 계산"이라고 읽으면 충분해요.

확산도구.py: 잡음 일정, 잡음 예측기, 그리기

확산 모델의 부품을 한 파일에 모았습니다. `확산도구.py`로 저장하세요.

```
"""15장 확산 모델 도구: 잡음 일정, 잡음 예측기(작은 U-Net), 그리기(거꾸로 가는 과정)."""
import math
```

```
import torch
import torch.nn.functional as F
from torch import nn

T = 1000                                        # 잡음을 더하는 단계 수
베타 = torch.linspace(1e-4, 0.02, T)            # 단계마다 섞는 잡음의 양
알파 = 1 - 베타
알파_누적 = torch.cumprod(알파, dim=0)          # 0단계부터 t단계까지 알파를 모두 곱한 값


def 잡음_더하기(x0: torch.Tensor, t: torch.Tensor, 잡음: torch.Tensor) -> torch.Tensor:
    """깨끗한 사진 x0에 t단계만큼의 잡음을 한 번에 섞는다."""
    a = 알파_누적.to(x0.device)[t].view(-1, 1, 1, 1)
    return a.sqrt() * x0 + (1 - a).sqrt() * 잡음


def 시간_임베딩(t: torch.Tensor, 크기: int) -> torch.Tensor:
    """단계 번호 t를 여러 빠르기의 sin·cos 값으로 펼친다. (배치,) → (배치, 크기)"""
    반 = 크기 // 2
    빠르기 = torch.exp(-math.log(10000) * torch.arange(반, device=t.device) / 반)
    각도 = t.float()[:, None] * 빠르기[None, :]
    return torch.cat([각도.sin(), 각도.cos()], dim=1)


class 블록(nn.Module):
    """합성곱 두 번 + 시간 정보 더하기 + 지름길(잔차 연결)."""

    def __init__(self, 입력: int, 출력: int, 시간_크기: int) -> None:
        super().__init__()
        self.합성곱1 = nn.Conv2d(입력, 출력, 3, padding=1)
        self.정규화1 = nn.GroupNorm(8, 출력)
        self.시간 = nn.Linear(시간_크기, 출력)
        self.합성곱2 = nn.Conv2d(출력, 출력, 3, padding=1)
        self.정규화2 = nn.GroupNorm(8, 출력)
        self.지름길 = nn.Conv2d(입력, 출력, 1) if 입력 != 출력 else nn.Identity()
```

```
    def forward(self, x: torch.Tensor, 시간: torch.Tensor) -> torch.Tensor:
        h = F.silu(self.정규화1(self.합성곱1(x)))
        h = h + self.시간(시간)[:, :, None, None]    # 채널마다 '지금 몇 단계인지'를 더한다
        h = F.silu(self.정규화2(self.합성곱2(h)))
        return h + self.지름길(x)


class 작은UNet(nn.Module):
    """28×28 → 14×14 → 7×7로 줄였다가 다시 키우며, 같은 크기끼리 이어 붙이는 잡음 예측기."""

    def __init__(self, 채널: int = 32, 시간_크기: int = 128, 클래스_수: int = 0) -> None:
        super().__init__()
        c = 채널
        self.시간_크기 = 시간_크기
        self.시간_층 = nn.Sequential(nn.Linear(시간_크기, 시간_크기), nn.SiLU(),
                                  nn.Linear(시간_크기, 시간_크기))
        self.클래스_층 = nn.Embedding(클래스_수, 시간_크기) if 클래스_수 > 0 else None
        self.입구 = nn.Conv2d(1, c, 3, padding=1)
        self.내려1, self.줄이기1 = 블록(c, c, 시간_크기), nn.Conv2d(c, c, 4, 2, 1)
        self.내려2, self.줄이기2 = 블록(c, 2 * c, 시간_크기), nn.Conv2d(2 * c, 2 * c, 4, 2, 1)
        self.가운데 = 블록(2 * c, 2 * c, 시간_크기)
        self.키우기2, self.올라2 = nn.ConvTranspose2d(2 * c, 2 * c, 4, 2, 1), 블록(4 * c, 2 * c, 시
        ↪ 간_크기)
        self.키우기1, self.올라1 = nn.ConvTranspose2d(2 * c, c, 4, 2, 1), 블록(2 * c, c, 시간_크기)
        self.출구 = nn.Conv2d(c, 1, 3, padding=1)

    def forward(self, x: torch.Tensor, t: torch.Tensor,
                클래스: torch.Tensor | None = None) -> torch.Tensor:
        시간 = self.시간_층(시간_임베딩(t, self.시간_크기))
        if self.클래스_층 is not None:
            시간 = 시간 + self.클래스_층(클래스)         # 무엇을 그릴지도 함께 알려 준다
        h = self.입구(x)
        h1 = self.내려1(h, 시간)                        # (c, 28, 28)
        h2 = self.내려2(self.줄이기1(h1), 시간)         # (2c, 14, 14)
        h = self.가운데(self.줄이기2(h2), 시간)         # (2c, 7, 7)
        h = self.올라2(torch.cat([self.키우기2(h), h2], dim=1), 시간)   # 14×14끼리 이어 붙이기
```

```
        h = self.올라1(torch.cat([self.키우기1(h), h1], dim=1), 시간)   # 28×28끼리 이어 붙이기
        return self.출구(h)                                  # 예측한 잡음 (1, 28, 28)


@torch.no_grad()
def 그리기(모델: nn.Module, 개수: int, 장치: torch.device, 클래스: torch.Tensor | None = None,
           기록_단계: tuple[int, ...] = ()) -> tuple[torch.Tensor, list[torch.Tensor]]:
    """순수한 잡음에서 출발해 T단계에 걸쳐 잡음을 덜어 낸다. 결과 값은 -1~1."""
    모델.eval()
    베타_, 알파_, 누적_ = 베타.to(장치), 알파.to(장치), 알파_누적.to(장치)
    x = torch.randn(개수, 1, 28, 28, device=장치)
    기록 = []
    for i in reversed(range(T)):
        t = torch.full((개수,), i, device=장치)
        예측_잡음 = 모델(x, t, 클래스)
        x = (x - 베타_[i] / (1 - 누적_[i]).sqrt() * 예측_잡음) / 알파_[i].sqrt()
        if i > 0:
            x = x + 베타_[i].sqrt() * torch.randn_like(x)   # 마지막 단계만 빼고 약간의 무작위를 다
↪   시 섞는다
        if i in 기록_단계:
            기록.append(x.cpu())
    return x, 기록
```

부품이 많아 보이지만, 앞 절들에서 배운 것의 조합입니다.

- **잡음 일정**: `베타`는 0.0001부터 0.02까지 1,000개로 고르게 늘어놓고(4장 `linspace`), `알파_누적`은 `torch.cumprod`로 "처음부터 t단계까지 알파를 모두 곱한 값"을 한꺼번에 구했습니다. 누적 합 `cumsum`의 곱하기 버전이에요.
- **`잡음_더하기`**: 앞에서 확인한 공식 그대로입니다. `t`가 사진마다 다르므로 `알파_누적[t]`로 사진마다 다른 값을 꺼내고, `view(-1, 1, 1, 1)`로 모양을 `(배치, 1, 1, 1)`로 만들어 `(배치, 1, 28, 28)` 사진과 브로드캐스팅했어요(4장 4.6절).
- **`시간_임베딩`**: 단계 번호 t를 숫자 하나 그대로 주면 신경망이 쓰기 어렵습니다. 그래서 여러 빠르기로 도는 sin과 cos 값으로 펼쳐 128칸짜리 벡터로 만들어요. 12장에서 트랜스포머가 단어의 위치를 알려 줄 때 쓰는 위치 인코딩과 같은 생각입니다. 바로 아래에서 확인해 봅니다.
- **`블록`**: 합성곱 두 번 사이에 시간 정보를 더합니다. 시간 벡터를 `nn.Linear`로 채널 수에 맞춘 뒤 `[:, :, None, None]`으로 `(배치, 채널, 1, 1)` 모양을 만들어 더하면, 모든 픽셀에 "지금 몇

단계인지"가 똑같이 더해져요. 마지막의 `h + self.지름길(x)`은 9장에서 직접 구현한 ResNet의 **잔차 연결**입니다. 입력과 출력의 채널 수가 다를 때만 1 × 1 합성곱으로 채널을 맞춰요.

- `nn.GroupNorm`: 8장 8.4절의 배치 정규화와 같은 목적의 정규화인데, 배치가 아니라 **사진 한 장 안에서** 채널을 8묶음으로 나눠 평균과 분산을 맞춥니다. 배치에 섞인 다른 사진의 영향을 받지 않고, 학습 모드와 평가 모드의 차이도 없어서 생성 모델에서 많이 써요. `F.silu`는 ReLU를 부드럽게 만든 활성화 함수입니다.
- **`작은UNet`**: 이름의 U는 모양에서 왔습니다. 28 × 28 → 14 × 14 → 7 × 7로 내려갔다가(합성곱 `stride=2`) 다시 14 × 14 → 28 × 28로 올라오는데(15.3절의 전치 합성곱), 올라올 때 **내려갈 때 같은 크기였던 결과를 `torch.cat`으로 옆에 이어 붙입니다.** 7 × 7까지 줄이면 큰 모양은 잘 보이지만 세밀한 위치 정보는 흐려지는데, 이렇게 이어 붙이면 디코더 쪽이 원래 크기의 세부 정보를 바로 받아 쓸 수 있어요. 잡음은 픽셀 하나하나에 섞여 있으니 그걸 맞히려면 세부 정보가 꼭 필요합니다. 15.10절에서 쓸 **`클래스_수`**는 지금은 0이라 꺼져 있어요.
- **`그리기`**: 순수한 잡음 `torch.randn`에서 출발해 999단계부터 0단계까지 거꾸로 걸어옵니다. `@torch.no_grad()`는 5장 5.8절의 `with torch.no_grad():`를 함수 전체에 씌우는 데코레이터예요(데코레이터는 1권 10장). 1,000걸음 동안 기울기 기록이 쌓이면 메모리가 금방 바닥나니까요. `기록_단계`에 적은 단계를 지날 때는 중간 그림을 남겨 둡니다.

시간 임베딩이 정말 "가까운 단계는 비슷하게, 먼 단계는 다르게" 펼쳐지는지 확인해 보겠습니다. 4장 4.11절의 코사인 유사도로 재요.

```
import torch
import torch.nn.functional as F

from 확산도구 import 시간_임베딩

t = torch.tensor([0, 10, 11, 500, 999])
e = 시간_임베딩(t, 128)
print("모양:", tuple(t.shape), "→", tuple(e.shape))
print("t=10의 앞 4칸:", [round(v, 3) for v in e[1, :4].tolist()])

# 가까운 단계끼리는 비슷하고, 먼 단계끼리는 다르다 (코사인 유사도)
for i, j in [(1, 2), (1, 3), (3, 4), (1, 4)]:
    print(f"t={t[i].item():3} 와 t={t[j].item():3}: {F.cosine_similarity(e[i], e[j], dim=0):.3f}")
```

```
모양: (5,) → (5, 128)
t=10의 앞 4칸: [-0.544, 0.693, 0.938, 0.209]
t= 10 와 t= 11: 0.970
t= 10 와 t=500: 0.285
t=500 와 t=999: 0.261
t= 10 와 t=999: 0.167
```

10단계와 11단계의 임베딩은 유사도 0.970으로 거의 같고, 10단계와 500단계는 0.285, 10단계와 999단계는 0.167로 멀어집니다. 신경망 입장에서는 이웃한 단계를 비슷하게 다루면서도 처음과 끝은 분명히 구별할 수 있는 입력을 받는 거예요.

학습시키기

이제 학습입니다. 모든 사진을 장치에 통째로 올려 두고, 걸음마다 128장을 무작위로 꺼내 씁니다. `DataLoader`의 섞기·묶기 과정이 없으니 GPU가 쉬지 않고 일할 수 있어요. 학습 중간중간(200걸음, 1,000걸음, 5,000걸음) 그림을 8장씩 그려서 모델이 어떻게 나아지는지도 남깁니다.

```
import torch
import torch.nn.functional as F

from 생성도구 import 격자_저장, 사진_불러오기
from 확산도구 import T, 그리기, 작은UNet, 잡음_더하기


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")


장치 = 장치_고르기()
torch.manual_seed(0)
X, _ = 사진_불러오기(학습=True)
```

```
X = (X * 2 - 1).to(장치)                          # -1~1로 바꿔 통째로 장치에 올린다 (약 188MB)

모델 = 작은UNet().to(장치)
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=2e-3)
스케줄러 = torch.optim.lr_scheduler.CosineAnnealingLR(옵티마이저, T_max=5000)   # 8장 8.7절
print(f"장치 {장치}, 파라미터 {sum(p.numel() for p in 모델.parameters()):,}개")

중간_그림 = []
손실_합 = 0.0
for 걸음 in range(1, 5001):
    x0 = X[torch.randint(0, len(X), (128,), device=장치)]     # 사진 128장 무작위로
    t = torch.randint(0, T, (128,), device=장치)              # 사진마다 단계도 무작위로
    잡음 = torch.randn_like(x0)
    손실 = F.mse_loss(모델(잡음_더하기(x0, t, 잡음), t), 잡음)   # 섞은 잡음을 맞혀라
    옵티마이저.zero_grad()
    손실.backward()
    옵티마이저.step()
    스케줄러.step()                                  # 걸음마다 학습률을 조금씩 줄인다
    손실_합 += 손실.item()
    if 걸음 % 1000 == 0:
        print(f"{걸음:5}걸음: 최근 1000걸음 평균 손실 {손실_합 / 1000:.4f}")
        손실_합 = 0.0
    if 걸음 in (200, 1000, 5000):                  # 학습 중간중간 그림 8장씩 그려 보기
        torch.manual_seed(1)
        그림, _ = 그리기(모델, 8, 장치)
        중간_그림.append(그림.cpu())
        모델.train()

torch.save(모델.state_dict(), "확산.pt")
격자_저장((torch.cat(중간_그림) + 1) / 2, "diffusion_progress.png")
```

```
장치 mps, 파라미터 546,753개
 1000걸음: 최근 1000걸음 평균 손실 0.0641
 2000걸음: 최근 1000걸음 평균 손실 0.0452
 3000걸음: 최근 1000걸음 평균 손실 0.0423
 4000걸음: 최근 1000걸음 평균 손실 0.0402
```

```
5000걸음: 최근 1000걸음 평균 손실 0.0388
```

파라미터는 약 55만 개로, 6장의 MLP(약 24만 개)보다 두 배 남짓 큰 정도입니다. 손실은 처음 1,000걸음 평균 0.0641에서 5,000걸음 무렵 0.0388까지 내려갔어요. 잡음 표준편차가 1이니 아무것도 모르고 0이라고 답하면 MSE가 1인데, 그 25분의 1 아래로 내려간 겁니다. 학습률은 8장 8.7절의 `CosineAnnealingLR`로 걸음마다 조금씩 줄여서 마지막에는 0에 가깝게 만들었어요. 확산 모델은 마지막 걸음의 가중치로 그림을 그리는데, 학습률이 큰 채로 끝나면 가중치가 흔들리던 중간 어딘가에서 멈춰서 그림이 거칠어지기 쉽습니다.

이 블록은 MPS에서 돌렸기 때문에 손실의 소수점 아래 숫자는 실행할 때마다 조금씩 다를 수 있습니다(이 책의 실행 환경인 Apple M4 Max에서 나온 값이에요). CPU에서도 돌아가지만 몇 배 오래 걸리니, 처음에는 `range(1, 5001)`을 `range(1, 1001)`로 줄여서 흐름부터 확인해 보세요.

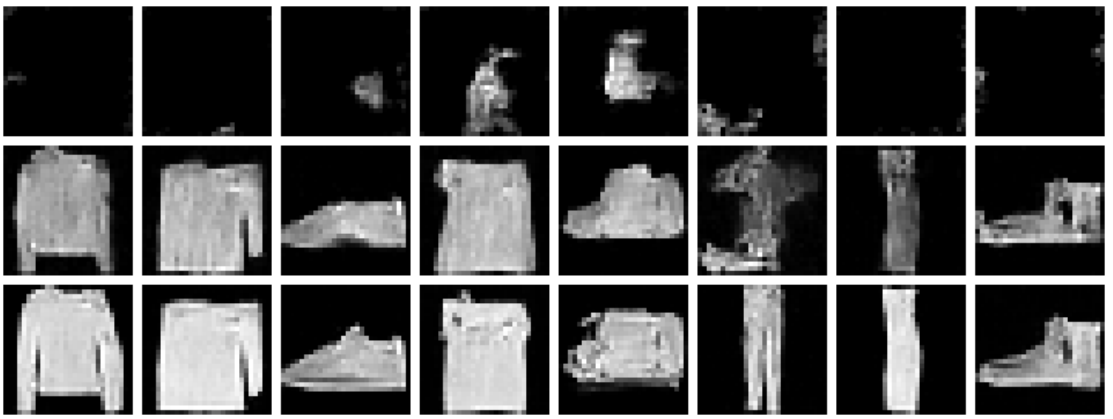


그림 15.9: 확산 모델을 200걸음, 1,000걸음, 5,000걸음 학습한 뒤 각각 8장씩 그린 그림. 200걸음 뒤에는 검은 바탕에 얼룩뿐이고, 1,000걸음 뒤에는 거친 옷과 신발 모양이 나타나며, 5,000걸음 뒤에는 윤곽이 매끈한 옷과 신발이 나온다.

학습 중간에 그린 그림입니다. 200걸음 뒤(첫 줄)에는 검은 바탕에 얼룩뿐이에요. 1,000걸음 뒤(둘째 줄)에는 벌써 풀오버, 운동화, 앵클부츠 같은 모양이 나오지만 가장자리가 거칠고 군데군데 지저분합니다. 5,000걸음 뒤(셋째 줄)에는 윤곽이 매끈해지고 바지처럼 가는 모양도 제대로 나와요. 같은 씨앗으로 그렸기 때문에 같은 칸끼리는 같은 잡음에서 출발한 그림입니다. 손실 숫자보다 이 그림이 학습 상태를 훨씬 잘 말해 줍니다. 생성 모델에서는 손실이 조금 줄었다고 그림이 좋아졌다는 보장이 없어서, 이렇게 학습 중간에 그림을 그려 눈으로 확인하는 것이 기본 습관이에요.

그리기: 잡음에서 옷이 나오기까지

저장한 모델로 그림을 그려 보겠습니다. 먼저 8장을 그리면서 잡음이 사진으로 바뀌어 가는 중간 과정을 기록하고, 이어서 1,000장을 그려 64장은 그림으로, 전부는 15.9절의 채점용으로 저장합니다.

```
import torch

from 생성도구 import 격자_저장
from 확산도구 import 그리기, 작은UNet


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")
    if torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")


장치 = 장치_고르기()
모델 = 작은UNet().to(장치)
모델.load_state_dict(torch.load("확산.pt", map_location=장치))

# 1) 잡음이 사진으로 바뀌어 가는 과정: 8장을 그리며 중간 단계를 기록
torch.manual_seed(3)
기록_단계 = (999, 800, 600, 400, 300, 200, 100, 0)
_, 기록 = 그리기(모델, 8, 장치, 기록_단계=기록_단계)
for 단계, x in zip(기록_단계, 기록):
    print(f"t={단계:3}: 값 범위 {x.min():6.2f} ~ {x.max():5.2f}, 표준편차 {x.std():.2f}")
과정 = torch.stack(기록, dim=1).flatten(0, 1)      # (8장 × 8단계, 1, 28, 28): 한 줄이 한 장의 과정
격자_저장((과정 + 1) / 2, "diffusion_reverse.png")

# 2) 새 그림 1,000장: 64장은 그림으로, 전부는 15.9절 채점용으로 저장
torch.manual_seed(0)
그림, _ = 그리기(모델, 1000, 장치)
그림 = ((그림.cpu() + 1) / 2).clamp(0, 1)
격자_저장(그림[:64], "diffusion_samples.png")
```

```
torch.save(그림, "확산_1000.pt")
print("저장한 그림:", tuple(그림.shape))
```

```
t=999: 값 범위  -3.67 ~  3.56, 표준편차 1.01
t=800: 값 범위  -3.86 ~  4.53, 표준편차 1.00
t=600: 값 범위  -3.28 ~  3.44, 표준편차 0.99
t=400: 값 범위  -3.65 ~  3.43, 표준편차 0.93
t=300: 값 범위  -3.29 ~  2.88, 표준편차 0.87
t=200: 값 범위  -3.11 ~  2.36, 표준편차 0.78
t=100: 값 범위  -2.14 ~  1.64, 표준편차 0.67
t=  0: 값 범위  -1.05 ~  1.31, 표준편차 0.62
저장한 그림: (1000, 1, 28, 28)
```

한 줄이 그림 한 장의 일생입니다. 왼쪽 999단계는 순수한 잡음이고, 400단계까지도 사람 눈에는 별 차이가 없어요. 그러다 300~200단계쯤 갑자기 윤곽이 떠오르고, 100단계를 지나면 잔잡음만 남았다가 0단계에서 깨끗해집니다. 출력의 표준편차도 600단계까지는 1.0 근처에서 버티다가 400단계부터 0.93, 0.87, 0.78, 0.67로 줄어 0단계에서 0.62가 돼요. 값 범위도 -1.05~1.31로 좁혀져서 사진의 밝기 범위(-1~1) 근처로 돌아왔습니다. 살짝 벗어난 값은 그림으로 저장할 때 `clamp`로 잘라요. 정리하면 1,000걸음 중 999~400단계는 사람 눈에 거의 잡음 그대로이고, 300~200단계에서 큰 윤곽이 정해지며, 100단계부터는 세부를 다듬는 시간입니다. 앞으로 가는 과정에서 500단계를 넘으면 원본이 거의 보이지 않았던 것과 짝을 이루는 모습이에요.

64장을 보면 운동화, 샌들, 앵클부츠, 가방, 바지, 드레스, T셔츠, 긴소매 상의까지 여러 종류가 섞여 나왔습니다. VAE의 그림보다 훨씬 또렷해서 가방 손잡이, 샌들 끈, 상의의 소매 같은 세부도 보여요. 그렇다고 완벽하지는 않습니다. 일부 그림은 무늬가 얼룩덜룩하게 지저분하고, 몇 장은 상의인지 가방인지 모호하며, 샌들처럼 가는 끈이 많은 신발은 모양이 무너지기도 해요. 진짜 사진의 매끈한 면과 깔끔한 윤곽을 흉내 내기에는 모델도 작고 학습도 짧았기 때문입니다. 숫자로 따져 보는 일은 15.9절에서 합니다.

무엇을 키우면 좋아질까

이 정도면 "알아볼 수 있는 모양이 나오기 시작한" 단계입니다. 요즘 그림 생성 AI의 결과와는 거리가 멀죠. 이 장의 모델은 노트북에서 몇 분 안에 끝나도록 일부러 작게 만들었으니, 무엇을 키우면 좋아지는지 알아 두면 직접 실험해 볼 수 있습니다.

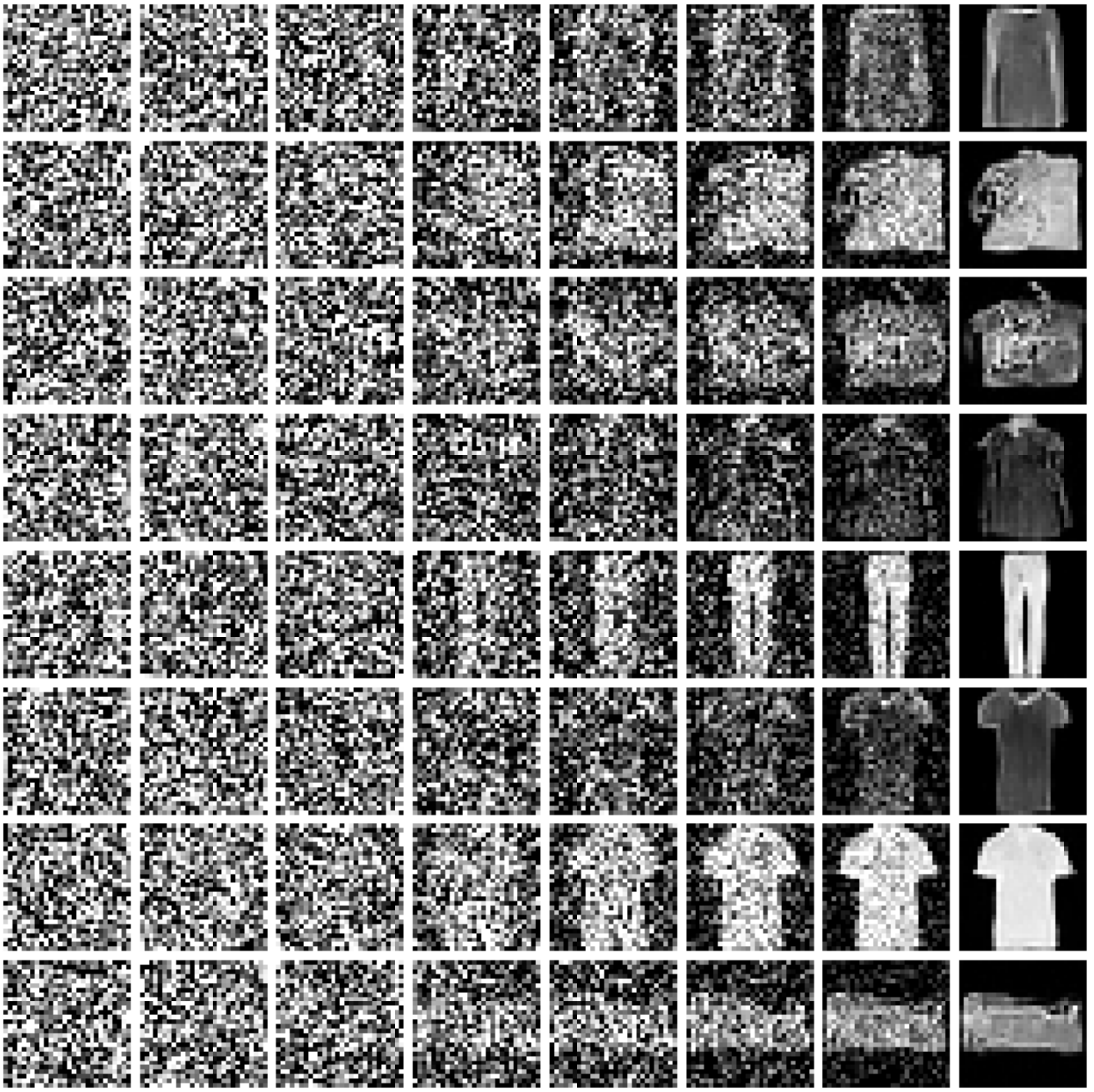


그림 15.10: 확산 모델이 그림 8장을 그리는 과정. 한 줄이 한 장이고, 왼쪽부터 999, 800, 600, 400, 300, 200, 100, 0단계의 모습이다. 처음 몇 칸은 순수한 잡음이다가 300~200단계쯤 옷과 신발의 윤곽이 떠오르고, 마지막 칸에서 깨끗한 그림이 된다.



그림 15.11: 확산 모델이 순수한 잡음에서 출발해 그린 새 그림 64장. 운동화, 샌들, 앵클부츠, 바지, T셔츠, 드레스, 가방 등이 알아볼 수 있게 나오지만, 일부는 무늬가 지저분하거나 무엇인지 모호하다.

손잡이	이 장의 값	키우면	주의할 점
학습 걸음 수	5,000걸음 (약 11에폭)	가장 확실하게 좋아진다. 수만 걸음 이상이 보통	시간에 그대로 비례
모델 크기(**채널**)	32	세부 무늬가 좋아진다. 64나 128로	메모리와 시간이 늘어남
가중치 이동 평균(EMA)	없음	그림이 깨끗해진다. 최근 가중치들의 평균으로 그린다	코드 몇 줄 추가
어텐션 층	없음	멀리 떨어진 부분끼리의 관계(좌우 대칭 등)를 잘 맞춘다	12장에서 배운 그 층
잡음 일정	직선(linear)	코사인 일정 등으로 바꾸면 작은 사진에서 효과가 크다	공식만 바꾸면 됨
그리는 단계 수	1,000	줄일수록 빠르다. DDIM 같은 방법은 50단계로도 그린다	너무 줄이면 품질이 떨어짐

가장 먼저 해 볼 것은 걸음 수입니다. 같은 코드에서 `range(1, 5001)`과 `T_max=5000`을 함께 늘리기만 하면 되고, 시간은 걸음 수에 비례해 늘어나요. 두 번째는 EMA입니다. 학습 중 가중치의 이동 평균을 따로 들고 있다가 그걸로 그리는 방법인데, 8장의 학습률 스케줄러처럼 "마지막 순간의 흔들림"을 줄여 줍니다. 그다음이 모델 크기와 어텐션이에요. 실제 그림 생성 모델은 여기에 사진 대신 작은 잠재 공간(15.1절의 오토인코더로 줄인 공간!)에서 확산을 돌리는 방법까지 더해서, 큰 사진도 감당할 수 있는 속도를 얻습니다.

15.9 생성 모델은 어떻게 평가할까

정답이 없는 시험

분류 모델은 평가가 쉬웠습니다. 시험 데이터의 정답과 비교해 정확도를 내면 되니까요. 그런데 생성 모델이 그린 앵클부츠는 **비교할 정답이 없습니다.** 세상에 없던 그림을 그리는 게 목적이니까요. 그렇다고 손실 값을 믿을 수도 없어요. 확산 모델의 손실은 "잡음을 얼마나 잘 맞히나"이지 "그림이 얼마나 좋은가"가 아니고, VAE의 손실도 흐릿한 그림을 막아 주지 못했습니다.

그래서 생성 모델의 평가는 여러 질문으로 쪼개서 합니다.

- **그럴듯한가(품질)**: 진짜 사진처럼 보이는가?
- **골고루 그리는가(다양성)**: 한 종류만 그리지 않고 데이터의 여러 모습을 다 그리는가? 15.7절의 모드 붕괴를 잡아내는 질문이에요.
- **베낀 건 아닌가(새로움)**: 학습 사진을 외워서 그대로 내놓는 건 아닌가?

연구에서 가장 많이 쓰는 지표는 **FID(Fréchet Inception Distance)** 입니다. 진짜 사진들과 생성한 사진들을 미리 학습된 큰 분류 모델(Inception)에 통과시켜 특징을 뽑고, 두 묶음의 특징 분포가 얼마나 다른지 재요. 품질과 다양성을 한 숫자로 함께 보는 지표인데, 큰 사전학습 모델과 수만 장의 그림이 필요합니다. 여기서는 같은 생각을 작게 따라 해 보겠습니다. 9장에서 배운 작은 CNN을 **채점기**로 학습시키고, 진짜 시험 사진·VAE·확산 모델의 그림 1,000장씩을 채점해요.

```
import torch

from 생성도구 import 격자_저장, 사진_불러오기, 채점기_만들기


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")
    if torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")


장치 = 장치_고르기()
X_학습, _ = 사진_불러오기(학습=True)
X_시험, y_시험 = 사진_불러오기(학습=False)
채점기 = 채점기_만들기(장치)
```

```
with torch.no_grad():
    정확도 = (채점기(X_시험.to(장치)).argmax(1).cpu() == y_시험).float().mean()
print(f"채점기의 시험 정확도: {정확도:.2%}\n")

묶음 = {
    "진짜 사진": X_시험[:1000],
    "VAE": torch.load("vae_1000.pt"),
    "확산 모델": torch.load("확산_1000.pt"),
}
print("            확신도   종류별 개수 (T셔츠 … 앵클부츠)")
for 이름_, 사진 in 묶음.items():
    with torch.no_grad():
        확률 = 채점기(사진.to(장치)).softmax(dim=1).cpu()
    확신도 = 확률.max(dim=1).values.mean()           # 채점기가 얼마나 뚜렷하게 알아보나
    개수 = torch.bincount(확률.argmax(dim=1), minlength=10)   # 종류가 골고루 나오나
    print(f"{이름_:8}  {확신도:6.1%}   {개수.tolist()}")


# 외워서 베낀 건 아닐까? 가장 가까운 학습 사진까지의 거리
def 가장_가까운(사진: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
    """사진마다 픽셀 거리가 가장 가까운 학습 사진의 거리와 번호를 돌려준다."""
    거리 = torch.cdist(사진.flatten(1).to(장치), X_학습.flatten(1).to(장치))   # (200, 60000)
    최소 = 거리.min(dim=1)
    return 최소.values.cpu(), 최소.indices.cpu()


진짜_거리, _ = 가장_가까운(X_시험[:200])
확산_거리, 번호 = 가장_가까운(묶음["확산 모델"][:200])
print(f"\n가장 가까운 학습 사진까지 평균 거리: 진짜 시험 사진 {진짜_거리.mean():.2f}, "
      f"확산 모델 {확산_거리.mean():.2f}")
격자_저장(torch.cat([묶음["확산 모델"][:8], X_학습[번호[:8]]]), "nearest.png")
```

```
채점기의 시험 정확도: 88.15%

            확신도   종류별 개수 (T셔츠 … 앵클부츠)
진짜 사진      88.1%   [110, 102, 141, 92, 101, 81, 80, 95, 98, 100]
```

```
VAE        61.2%   [127, 47, 193, 80, 0, 0, 96, 182, 89, 186]
확산 모델      80.3%   [96, 72, 158, 121, 81, 81, 74, 91, 99, 127]

가장 가까운 학습 사진까지 평균 거리: 진짜 시험 사진 3.43, 확산 모델 4.25
```

채점기는 진짜 시험 사진을 88.15% 맞힙니다. 두 에폭만 학습해서 6장의 MLP와 비슷한 수준이라 완벽한 채점관은 아니지만, 큰 경향을 보기에는 충분해요. 세 묶음의 결과를 질문별로 읽어 보겠습니다.

- **확신도(품질)**: 채점기가 "가장 그럴듯한 종류"에 준 확률의 평균입니다. 그림이 또렷할수록 채점기도 자신 있게 답해요. 진짜 사진이 88.1%, 확산 모델이 80.3%, VAE가 61.2%입니다. 흐릿한 VAE 그림은 채점기도 무엇인지 확신하지 못하고, 확산 모델 그림은 진짜에 꽤 가깝지만 아직 차이가 있어요. 그림을 보고 받은 인상과 같은 순서입니다.
- **종류별 개수(다양성)**: 채점기가 판정한 종류를 세었습니다. 진짜 시험 사진은 종류마다 80~141장으로 100장 안팎이에요. 확산 모델도 모든 종류가 72~158장 사이로 나와서 모양이 비슷합니다. 풀오버가 많은 것도 진짜 사진과 같은데, 채점기가 상의를 풀오버로 판정하는 버릇이 있어서예요. 반면 VAE는 코트와 샌들이 **0장**입니다. 잠재 공간이 2칸뿐이라 좁은 지도 안에서 코트는 풀오버에, 샌들은 다른 신발에 자리를 내준 것으로 보여요. 표준정규분포에서 아무리 뽑아도 이 두 종류는 나오지 않는다는 뜻이니, 다양성 문제를 숫자로 잡아낸 셈입니다.
- **가장 가까운 학습 사진(새로움)**: 그림마다 학습 사진 6만 장 중 픽셀 거리가 가장 가까운 것을 찾았습니다. `torch.cdist`는 두 묶음 사이의 모든 쌍의 거리를 한 번에 구하는 함수로, 결과가 `(200, 60000)` 표예요. 진짜 시험 사진도 학습 사진과 똑같은 것은 없으니 가장 가까운 학습 사진까지 평균 3.43만큼 떨어져 있습니다. 확산 모델의 그림은 4.25로 그보다 더 멀어요. 학습 사진을 베꼈다면 이 거리가 0에 가까웠을 테니 베끼지는 않았다는 뜻이고, 동시에 진짜 사진들보다 조금 '바깥'에 있다는 뜻이기도 합니다. 아래 그림에서 위 줄(그린 그림)과 아래 줄(가장 가까운 학습 사진)을 비교해 보면 종류와 큰 모양은 닮았지만 무늬와 세부는 달라요.

이 채점은 어디까지나 **간이 점검**입니다. 채점기 자신이 틀릴 수 있고, 확신도가 높다고 사람 눈에 좋은 그림이라는 보장도 없어요. 실제로 생성 모델의 논문들은 FID 같은 자동 지표와 함께 **사람 평가**(여러 사람에게 두 그림을 보여 주고 더 나은 쪽을 고르게 하기)를 꼭 함께 싣습니다. 결국 생성 모델 평가의 마지막 심사위원은 사람의 눈이에요. 이 절에서 숫자를 뽑기 전에 그림부터 들여다본 이유이기도 합니다.

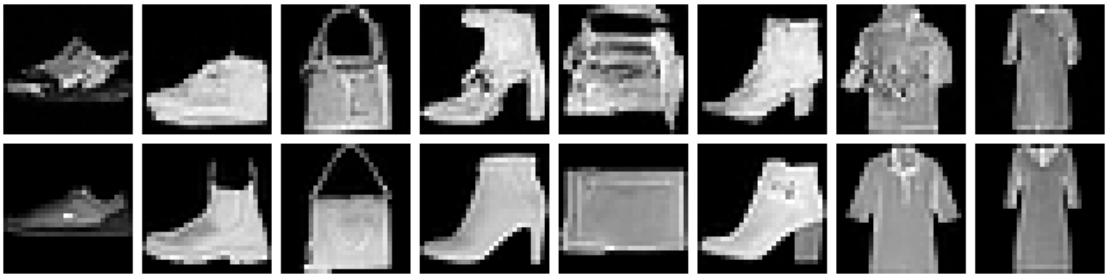


그림 15.12: 위 줄은 확산 모델이 그린 그림 8장, 아래 줄은 각 그림과 픽셀 거리가 가장 가까운 학습 사진이다. 종류와 큰 모양은 비슷하지만 세부 모양은 서로 달라, 학습 사진을 그대로 베낀 것이 아님을 보여 준다.

15.10 실습: 주문한 옷을 그리는 조건부 확산 모델

지금까지의 확산 모델은 무엇을 그릴지 우리가 정할 수 없었습니다. 잡음에서 출발해 운동화가 나올지 바지가 나올지는 운이에요. 이번 실습에서는 "바지를 그려 줘"처럼 **종류를 주문할 수 있는** 모델로 바꿔 봅니다. 이런 모델을 **조건부(conditional) 생성 모델**이라고 해요. 그림 생성 AI에 글로 원하는 그림을 설명하는 것도, 주문서가 "종류 번호" 대신 "문장"일 뿐 같은 생각입니다.

바꿀 곳은 놀랍도록 적습니다. `확산도구.py`의 `작은UNet`에는 이미 `클래스_수`라는 손잡이를 만들어 두었어요. `클래스_수=10`을 주면 `nn.Embedding(10, 128)`이 종류 번호마다 128칸짜리 벡터를 하나씩 배우고, 이 벡터를 시간 임베딩에 **더합니다.** 그러면 모든 블록이 "지금 몇 단계인지"와 함께 "무엇을 그리는 중인지"도 받게 돼요. `nn.Embedding`은 11장 11.11절에서 처음 본, 번호를 벡터로 바꾸는 표예요. 13장에서는 이 표로 단어를 벡터로 바꿨죠.

```
import torch
import torch.nn.functional as F

from 생성도구 import 격자_저장, 사진_불러오기
from 확산도구 import T, 그리기, 작은UNet, 잡음_더하기


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")
```

```
    if torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")


장치 = 장치_고르기()
torch.manual_seed(0)
X, y = 사진_불러오기(학습=True)
X, y = (X * 2 - 1).to(장치), y.to(장치)

모델 = 작은UNet(클래스_수=10).to(장치)            # 바뀐 곳 (1): 종류 10가지를 알려 줄 수 있게
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=2e-3)
손실_합 = 0.0
for 걸음 in range(1, 5001):
    번호 = torch.randint(0, len(X), (128,), device=장치)
    x0, 종류 = X[번호], y[번호]
    t = torch.randint(0, T, (128,), device=장치)
    잡음 = torch.randn_like(x0)
    손실 = F.mse_loss(모델(잡음_더하기(x0, t, 잡음), t, 종류), 잡음)   # 바뀐 곳 (2): 정답 종류도
↪   넘긴다
    옵티마이저.zero_grad()
    손실.backward()
    옵티마이저.step()
    손실_합 += 손실.item()
    if 걸음 % 1000 == 0:
        print(f"{걸음:5}걸음: 평균 손실 {손실_합 / 1000:.4f}")
        손실_합 = 0.0
torch.save(모델.state_dict(), "조건부_확산.pt")

# 종류마다 8장씩: 한 줄이 한 종류 (T셔츠, 바지, … 앵클부츠)
torch.manual_seed(0)
종류 = torch.arange(10, device=장치).repeat_interleave(8)
그림, _ = 그리기(모델, 80, 장치, 클래스=종류)
격자_저장((그림.cpu() + 1) / 2, "conditional.png")
```

```
1000걸음: 평균 손실 0.0622
2000걸음: 평균 손실 0.0448
3000걸음: 평균 손실 0.0427
4000걸음: 평균 손실 0.0411
5000걸음: 평균 손실 0.0406
```

학습 코드에서 바뀐 곳은 주석의 (1), (2) 두 군데뿐입니다. 모델을 만들 때 `클래스_수=10`을 주고, 학습할 때 사진의 정답 종류를 함께 넘겼어요. 그릴 때는 `torch.arange(10).repeat_interleave(8)`로 `[0, 0, …, 0, 1, 1, …, 9]`처럼 종류 번호를 8번씩 늘어놓은 주문서를 넘겼습니다. 그림을 보면 줄마다 주문한 종류가 나왔어요. 바지, 운동화, 가방, 앵클부츠 줄은 거의 모두 주문대로이고, T셔츠 줄에는 소매 짧은 상의가, 풀오버와 코트 줄에는 긴소매 상의가 나옵니다. 셔츠 줄(일곱째)은 T셔츠 같기도 하고 풀오버 같기도 한 그림이 섞여 있고, 샌들 줄은 끈 모양이 어지럽게 엉킨 것이 많아요.

주문을 얼마나 잘 지키는지 채점기로 확인해 보겠습니다. 종류마다 100장씩 주문해서 그리고, 채점기가 주문한 종류로 알아보는 비율을 셉니다. 채점기 자신의 실력을 함께 보려고 진짜 시험 사진에서의 종류별 정확도도 옆에 놓았어요.

```
import torch

from 생성도구 import 사진_불러오기, 채점기_만들기, 이름
from 확산도구 import 그리기, 작은UNet


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")
    if torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")


장치 = 장치_고르기()
모델 = 작은UNet(클래스_수=10).to(장치)
모델.load_state_dict(torch.load("조건부_확산.pt", map_location=장치))
채점기 = 채점기_만들기(장치)
```



그림 15.13: 조건부 확산 모델에 종류를 주문해 그린 그림. 한 줄이 한 종류로, 위에서부터 T셔츠, 바지, 풀오버, 드레스, 코트, 샌들, 셔츠, 운동화, 가방, 앵클부츠를 8장씩 그렸다.

```
# 종류마다 100장씩 주문해서 그리고, 채점기가 주문한 종류로 알아보는지 센다
torch.manual_seed(0)
주문 = torch.arange(10, device=장치).repeat_interleave(100)
그림, _ = 그리기(모델, 1000, 장치, 클래스=주문)
with torch.no_grad():
    판정 = 채점기((그림 + 1) / 2).argmax(dim=1)

X_시험, y_시험 = 사진_불러오기(학습=False)
with torch.no_grad():
    진짜_판정 = 채점기(X_시험.to(장치)).argmax(dim=1).cpu()

print("종류       주문대로 그린 비율   (참고) 진짜 사진 정확도")
for 번호 in range(10):
    맞음 = (판정[주문 == 번호] == 번호).float().mean()
    진짜 = (진짜_판정[y_시험 == 번호] == 번호).float().mean()
    print(f"{이름[번호]:5}     {맞음:10.0%}              {진짜:6.0%}")
print(f"전체       {(판정 == 주문).float().mean():10.1%}              "
      f"{(진짜_판정 == y_시험).float().mean():6.1%}")
```

```
종류       주문대로 그린 비율   (참고) 진짜 사진 정확도
T셔츠              80%                85%
바지              100%                97%
풀오버              82%                91%
드레스              90%                89%
코트               68%                80%
샌들               69%                94%
셔츠               25%                60%
운동화              76%                91%
가방               98%                98%
앵클부츠             97%                98%
전체            78.5%              88.2%
```

전체로는 주문한 종류로 알아본 비율이 78.5%입니다. 채점기가 진짜 사진에서도 88.2%만 맞히니, 채점기의 실수를 감안하면 주문을 꽤 잘 지킨 셈이에요. 종류별로 보면 바지(100%), 가방(98%), 앵클부츠(97%)처럼 모양이 독특한 것은 거의 완벽합니다. 가장 약한 것은 셔츠(25%)인데, 채점기 자

신도 진짜 셔츠를 60%밖에 못 맞히는 종류라 그림 탓만은 아니에요. 6장부터 계속 말썽인 바로 그 상의들이죠. 반면 샌들은 진짜 사진은 94%를 맞히는데 그린 그림은 69%에 그쳤습니다. 가는 끈이 얽힌 모양을 작은 모델이 5,000걸음 만에 그려 내기는 아직 어렵다는 뜻이고, 15.8절의 표에서 걸음 수나 채널을 키워 볼 첫 번째 대상이에요.

더 해 볼 만한 것을 몇 가지 남겨 둡니다.

- **주문을 더 잘 지키게 하기**: 학습할 때 10% 정도는 종류를 일부러 "없음"(11번째 번호)으로 바꿔 넣으면, 한 모델이 주문 있는 그림과 없는 그림을 모두 배웁니다. 그릴 때 두 예측의 차이를 키워서 쓰면 주문을 훨씬 확실하게 따르는데, 이것이 그림 생성 AI에서 널리 쓰는 **분류기 없는 가이던스**(classifier-free guidance) 예요.
- **15.8절의 표 적용하기**: 걸음 수와 채널을 늘리고 EMA를 더해서, 같은 채점기로 점수가 얼마나 오르는지 재 보세요. 실험마다 설정과 점수를 기록해 두는 방법은 18장에서 다룹니다.
- **MNIST로 바꿔 보기**: `datasets.FashionMNIST`를 `datasets.MNIST`로 바꾸면 손글씨 숫자를 주문해서 그리는 모델이 됩니다. 숫자는 무늬가 없어서 같은 학습량으로도 더 깔끔하게 나와요. MNIST는 미국 국립표준기술연구소(NIST)의 손글씨 자료를 얀 르쿤 등이 정리한 데이터인데, 이렇게 데이터를 바꿀 때도 출처와 라이선스부터 확인하는 습관을 들이세요.

15.11 정리와 다음 장 미리보기

이 장에서 고르는 모델에서 그리는 모델로 넘어갔습니다.

- **생성 모델**: 경계가 아니라 데이터가 어떻게 생겼는지(분포)를 배워서, 새 샘플을 뽑는다
- **오토인코더**: 인코더로 줄이고 디코더로 되살린다. 정답은 입력 자신. 아주 작게 줄일 때는 PCA보다 낫지만, 단순한 방법부터 비교한다
- **잠재 공간**: 정답 없이도 비슷한 것끼리 모인다. 2차원이면 그림으로 그려 볼 수 있다
- **합성곱 오토인코더**: `stride=2` 합성곱으로 줄이고 `ConvTranspose2d`로 키운다. 잡음 섞인 입력 → 깨끗한 정답으로 학습하면 잡음 제거기
- **이상 탐지**: 정상만 보고 배운 오토인코더는 낯선 것을 잘 못 되살린다. 재구성 오차와 문턱으로 판정
- **VAE**: 점 대신 평균·로그분산의 구름. 재매개변수화 `z = 평균 + 표준편차 × 잡음`으로 미분을 통과시키고, KL 벌점으로 잠재 공간을 표준정규분포 근처에 빈틈없이 모은다. 아무 점에서나 그리고, 보간이 매끄럽지만 그림이 흐린 편
- **GAN**: 생성자와 판별자의 경쟁. 선명하지만 학습이 까다롭고 모드 붕괴 위험
- **확산 모델**(DDPM): 잡음을 한 번에 섞는 공식으로 학습 데이터를 만들고, 섞인 잡음을 MSE

로 맞힌다. 시간 임베딩 + U-Net(같은 크기끼리 이어 붙이기, 잔차 연결, GroupNorm). 그릴 때는 1,000단계를 거꾸로 걷는다

- **평가**: 정답이 없으니 품질·다양성·새로움으로 쪼갠다. FID의 생각을 채점기로 따라 하고, 최근접 학습 사진으로 베끼기를 점검하고, 마지막은 사람의 눈
- **조건부 생성**: 종류 임베딩을 시간 임베딩에 더하기만 해도 주문한 것을 그린다

이 장으로 3부의 모델 구조 여행이 끝납니다. 사진을 보는 CNN, 순서를 읽는 RNN, 모든 것을 서로 비교하는 트랜스포머, 그리고 그림을 그리는 생성 모델까지 왔어요. 그런데 이 장에서 처음으로 "학습에 몇 분, 그리는 데 또 몇 분"을 기다려야 했죠. 확산 모델은 그림 한 장에 신경망을 1,000번 돌리고, 모델을 조금만 키워도 시간이 금방 몇 배로 늘어납니다. 다음 16장에서는 같은 모델을 더 빠르게, 더 적은 메모리로 돌리는 방법을 다룹니다. 16비트 숫자로 계산하는 혼합 정밀도, 모델을 미리 번역해 두는 `torch.compile`, 그리고 어디서 시간이 새는지 찾아내는 프로파일링이에요. 이 장의 확산 모델 학습이 좋은 실험 재료가 될 겁니다.

16

GPU와 성능: 같은 모델을 더 빠르게, 더 가볍게

학습 목표

- GPU가 빠른 이유와, GPU가 오히려 느린 경우를 설명하기
- 비동기 실행을 이해하고 GPU 시간을 올바르게 재기(동기화, 준비 운동, CUDA 이벤트)
- GPU 메모리를 들여다보고, 메모리 부족(OOM) 오류를 재현하고 대처하기
- 고정 메모리와 `non_blocking`으로 CPU와 GPU 사이 복사 비용 줄이기
- 혼합 정밀도(`autocast`, `bfloat16`, `float16`과 `GradScaler`)와 TF32로 학습을 빠르게 하기
- `torch.compile`의 준비 시간과 이득, 그래프 끊김을 이해하기
- `scaled_dot_product_attention`과 프로파일러로 시간이 새는 곳을 찾아 고치기

들어가며: 같은 요리, 다른 주방

15장의 확산 모델을 학습시키면서 몇 분씩 기다렸던 것 기억나세요? 모델을 조금만 키워도 기다리는 시간은 금방 몇 배가 됩니다. 이럴 때 가장 먼저 떠오르는 생각은 "더 좋은 GPU를 사야 하나?"예요. 그런데 같은 GPU, 같은 모델로도 **몇 배씩 빨라지는 경우가 아주 흔합니다.** 느렸던 이유가 GPU가 아니라 다른 곳에 있었던 거죠.

식당 주방을 떠올려 보세요. 화구가 아무리 좋아도 재료 손질하는 사람이 한 명뿐이면 화구는 대부

분 꺼져 있고, 볶음밥 한 그릇마다 불을 켰다 껐다 하면 시간이 다 갑니다. 딥러닝 학습도 똑같아요. 데이터를 준비하는 쪽, GPU에 일을 맡기는 방식, 숫자를 담는 칸의 크기, 계산을 묶는 방식이 모두 속도를 좌우합니다.

이 장에서는 이 모든 것을 **직접 재 보면서** 하나씩 고칩니다. 그래서 이 장의 예제는 지금까지와 달리 NVIDIA GPU가 달린 서버에서 실행했어요. 이 책의 실행 환경은 이렇습니다.

```
import torch

print("PyTorch    :", torch.__version__)
print("CUDA 빌드  :", torch.version.cuda)
print("GPU 개수   :", torch.cuda.device_count())
속성 = torch.cuda.get_device_properties(0)
print("GPU 이름   :", 속성.name)
print(f"GPU 메모리 : {속성.total_memory / 1024**3:.1f} GB")
print("SM 개수    :", 속성.multi_processor_count)
print("계산 능력  :", f"{속성.major}.{속성.minor}")
print("bfloat16   :", torch.cuda.is_bf16_supported())
```

```
PyTorch    : 2.14.1+cu126
CUDA 빌드  : 12.6
GPU 개수   : 2
GPU 이름   : NVIDIA GeForce RTX 4090
GPU 메모리 : 23.6 GB
SM 개수    : 128
계산 능력  : 8.9
bfloat16   : True
```

RTX 4090 두 장이 달린 리눅스 서버이고, 이 장에서는 그중 한 장(0번)만 씁니다. 두 장을 함께 쓰는 법은 17장의 주제예요. `계산 능력 8.9`는 NVIDIA가 GPU 세대마다 붙인 번호로, 8.0 이상이면 이 장에서 다룰 `bfloat16`과 TF32를 하드웨어가 직접 지원합니다.

`CUDA 빌드: 12.6`도 눈여겨보세요. 처음에는 `uv add torch`가 기본으로 설치하는 CUDA 13용 PyTorch를 썼는데, `NVIDIA driver on your system is too old`라는 오류가 나며 GPU를 찾지 못했습니다. 이 서버의 드라이버가 CUDA 12.4까지만 지원했기 때문이에요. 그래서 부록 A.4절의 방법대로 PyTorch 인덱스를 CUDA 12.6용(`cu126`)으로 지정해 설치했습니다. "드라이버가 지원하는 CUDA 버전에 맞는 빌드를 고른다"는 말이 실제로 이런 뜻입니다.

Q

Q. NVIDIA GPU가 없으면 이 장은 어떻게 따라 하나요? 이 장의 예제는 대부분 `cuda` 장치를 직접 씁니다. GPU 메모리 통계, 고정 메모리, `GradScaler`, 플래시 어텐션처럼 NVIDIA GPU에서만 의미가 있는 기능을 재 보는 장이라서요. 가장 쉬운 방법은 부록 A.5절의 Colab에서 무료 GPU를 빌려 실행하는 것입니다. 다만 Colab에서 흔히 받는 T4는 계산 능력이 7.5라서, 이 장의 기능 몇 가지는 하드웨어가 지원하지 않아요. 절마다 어디서 돌아가는지 정리하면 이렇습니다("그대로는 안 됨"은 코드가 `cuda`를 직접 써서 고치지 않으면 멈춘다는 뜻이에요).

Q

절	Colab T4	맥(MPS)	CPU만
16.1~16.5 시간 재기, 메모리, 고정 메모리	돌아감	그대로는 안 됨	그대로는 안 됨
16.6 자료형 범위, `autocast` 예제	돌아감	돌아감	돌아감(`bfloat16`으로)
16.6 `GradScaler`, 세 방식 비교	돌아감. `bfloat16`은 하드웨어 지원이 없어 빨라지지 않음	그대로는 안 됨	그대로는 안 됨
16.7 TF32	돌아가지만 빨라지지 않음(계산 능력 8.0 미만)	그대로는 안 됨	그대로는 안 됨
16.8 `torch.compile` 앞의 두 예제	돌아감	`"cuda"`를 `"mps"`로, `torch.cuda.synchronize`를 `torch.mps.synchronize`로 바꾸면 돌아감	장치를 `"cpu"`로 바꾸면 돌아가지만 아주 느림
16.9 SDPA	플래시만 빼고 돌아감(계산 능력 8.0 이상이 필요해서 코드가 알아서 뺌)	그대로는 안 됨	그대로는 안 됨
16.10~16.13 프로파일러, 데이터 로더, 실습	돌아감. CPU 코어가 적으니 `num_workers`는 `os.cpu_count()`보다 크게 잡지 않기	그대로는 안 됨	그대로는 안 됨

맥과 CPU 칸은 이 책의 맥(M4 Max)에서 직접 돌려 확인했고, Colab 칸은 T4의 계산 능력과 PyTorch가 지원하는 범위로 정리했습니다. GPU 종류가 다르니 숫자는 다르게 나오지만, 무엇이 왜 빨라지는지는 같은 방법으로 확인할 수 있어요.

16.1 GPU는 왜 빠를까요

교수 한 명과 학생 만 명

수학 교수 한 명과 초등학생 만 명이 계산 시합을 한다고 해 봅시다. 문제가 "숫자 1억 개에 각각 2를 곱하고 1을 더하라"라면 누가 이길까요? 학생들입니다. 문제 하나하나는 쉽고, 서로 상관이 없으니 만 명이 나눠서 동시에 풀면 끝나거든요. 하지만 문제가 "앞의 답을 받아서 다음 계산을 하라"처럼 줄줄이 이어진다면 어떨까요? 한 번에 한 명밖에 일할 수 없으니, 학생 만 명이 있어도 소용이 없고 똑똑한 교수 한 명이 훨씬 빠릅니다.

CPU는 교수, GPU는 학생 만 명입니다. CPU에는 복잡한 일을 아주 빨리 처리하는 코어가 수십 개, GPU에는 단순한 계산을 하는 작은 코어가 수천~수만 개 있어요. 앞에서 본 `SM 개수 128`이 GPU 안의 작은 공장 수이고, 공장마다 계산 일꾼이 수십 명씩 있습니다. 딥러닝 계산은 대부분 "숫자 수백만 개에 같은 계산을 한꺼번에"라 GPU와 궁합이 좋아요. 두 종류의 일을 시켜 보겠습니다.

```
import time

import torch


def 평균_시간(함수, 반복: int = 10) -> float:
    """준비 운동 한 번 뒤 여러 번 돌려, 한 번에 걸린 평균 시간(ms)을 잰다."""
    함수()
    torch.cuda.synchronize()                # GPU 일이 끝날 때까지 기다린다 (16.2절)
    시작 = time.perf_counter()
    for _ in range(반복):
        함수()
    torch.cuda.synchronize()
    return (time.perf_counter() - 시작) / 반복 * 1000
```

```
def 작은_일_천_번(x: torch.Tensor) -> torch.Tensor:
    """앞의 결과가 있어야 다음을 할 수 있는 작은 계산을 1,000번."""
    for _ in range(1000):
        x = (x * 2 + 1).sqrt()
    return x


큰_cpu = torch.rand(100_000_000)              # 숫자 1억 개
큰_gpu = 큰_cpu.to("cuda")
작은_cpu = torch.rand(10)                     # 숫자 10개
작은_gpu = 작은_cpu.to("cuda")

print(f"큰 일 한 번   (1억 개)   CPU {평균_시간(lambda: (큰_cpu * 2 + 1).sqrt()):7.2f} ms"
      f" ¦ GPU {평균_시간(lambda: (큰_gpu * 2 + 1).sqrt()):7.2f} ms")
print(f"작은 일 천 번 (10개씩)   CPU {평균_시간(lambda: 작은_일_천_번(작은_cpu)):7.2f} ms"
      f" ¦ GPU {평균_시간(lambda: 작은_일_천_번(작은_gpu)):7.2f} ms")
```

```
큰 일 한 번   (1억 개)   CPU   60.69 ms ¦ GPU    2.61 ms
작은 일 천 번 (10개씩)   CPU    3.78 ms ¦ GPU    7.73 ms
```

숫자 1억 개에 같은 계산을 하는 **큰 일 한 번**은 CPU가 60ms, GPU가 2.6ms로 GPU가 20배 넘게 빠릅니다. 그런데 숫자 10개짜리 계산을 앞의 결과를 받아 1,000번 이어 하는 **작은 일 천 번**은 CPU가 3.8ms, GPU가 7.7ms로 오히려 GPU가 두 배쯤 느려요.

GPU가 느린 이유는 계산 때문이 아닙니다. 파이썬이 GPU에 "이 계산을 해라"라고 일감을 하나 맡길 때마다 몇 마이크로초(μs)의 준비 비용이 들어요. 계산 한 줄(`x * 2`, `+ 1`, `.sqrt()`)이 일감 하나이니 1,000번 반복하면 일감이 3,000개, 7.7ms를 3,000으로 나누면 일감 하나에 약 2.6μs입니다. 숫자 10개 계산은 GPU에게 눈 깜짝할 일이라, 시간이 거의 전부 **일감을 맡기는 비용**으로 나간 거예요. 이 비용을 **커널 실행 비용(kernel launch overhead)** 이라고 부릅니다. GPU에서 실행되는 계산 하나를 **커널(kernel)** 이라고 하거든요.

여기서 이 장 전체를 꿰뚫는 두 가지 교훈이 나옵니다.

- **GPU에는 큰 일을 한꺼번에 맡겨야 합니다.** 배치를 키우고(16.11절), 작은 계산 여러 개를 하나로 합치는 것(16.8절)이 모두 이 원칙에서 나와요.
- **GPU가 쉬지 않게 해야 합니다.** 일감을 맡기는 쪽(파이썬, 데이터 준비)이 느리면 GPU는 기다

립니다(16.10절).

`평균_시간` 함수 안의 `torch.cuda.synchronize()`가 궁금했을 텐데, 이것 없이 재면 숫자가 엉터리가 됩니다. 바로 다음 절에서 그 이유를 봐요.

16.2 GPU 시간 제대로 재기

맡기고 돌아오는 파이썬

4장 4.8절에서 "GPU 일이 끝날 때까지 기다린 뒤 재야 한다"고 했습니다. 왜 그래야 하는지 이번에는 일부러 틀리게 재서 확인해 보겠습니다. 8192 × 8192 행렬 곱을 다섯 가지 방법으로 잽어요.

```
import time

import torch

a = torch.randn(8192, 8192, device="cuda")
b = torch.randn(8192, 8192, device="cuda")

시작 = time.perf_counter()
c = a @ b                                   # 이 크기의 행렬 곱은 처음
torch.cuda.synchronize()
print(f"① 첫 호출 (준비 포함)   : {(time.perf_counter() - 시작) * 1000:8.2f} ms")

시작 = time.perf_counter()
c = a @ b
print(f"② 기다리지 않고 잼      : {(time.perf_counter() - 시작) * 1000:8.2f} ms")
torch.cuda.synchronize()

시작 = time.perf_counter()
c = a @ b
torch.cuda.synchronize()                    # GPU가 다 끝낼 때까지 기다린 뒤 잰다
print(f"③ synchronize 후 잼     : {(time.perf_counter() - 시작) * 1000:8.2f} ms")

# ④ CUDA 이벤트: GPU의 작업 줄에 시작·끝 표시를 끼워 넣는다
시작_표시 = torch.cuda.Event(enable_timing=True)
끝_표시 = torch.cuda.Event(enable_timing=True)
```

```
시작_표시.record()
c = a @ b
끝_표시.record()
끝_표시.synchronize()                          # 끝 표시까지 지나갈 때까지 기다린다
print(f"④ CUDA 이벤트           : {시작_표시.elapsed_time(끝_표시):8.2f} ms")

# ⑤ 결과를 CPU로 꺼내는 순간(.item(), print, .cpu()) 저절로 기다리게 된다
시작 = time.perf_counter()
c = a @ b
합계 = c.sum().item()
print(f"⑤ .item()까지 잼        : {(time.perf_counter() - 시작) * 1000:8.2f} ms")
```

```
① 첫 호출 (준비 포함)   :    49.09 ms
② 기다리지 않고 잼      :     0.16 ms
③ synchronize 후 잼     :    19.67 ms
④ CUDA 이벤트           :    20.69 ms
⑤ .item()까지 잼        :    27.38 ms
```

같은 행렬 곱인데 0.16ms부터 49ms까지 숫자가 300배 넘게 차이 납니다. 하나씩 보죠.

- ② **기다리지 않고 잼: 0.16ms.** 곱셈과 덧셈이 1조 번 넘게 필요한 계산이 0.16ms에 끝났다면 초당 7,000조 번 계산한 셈이니 말이 안 되죠. 파이썬은 GPU에 일을 **맡기기만 하고 바로 돌아와서** 시계를 멈춰 버렸습니다. 이것을 **비동기 실행**이라고 해요. 주문서를 주방에 넣는 시간만 재고 "요리 0.2초 만에 완성!"이라고 하는 셈입니다.
- ③ **synchronize 후 잼: 19.7ms.** `torch.cuda.synchronize()`는 "GPU가 맡은 일을 모두 끝낼 때까지 기다려라"라는 뜻입니다. 이렇게 재야 진짜 계산 시간이에요.
- ① **첫 호출: 49ms.** 처음 계산할 때는 행렬 곱을 담당하는 라이브러리(cuBLAS)가 준비 작업을 하고, 이 크기에 맞는 계산 방법을 고르느라 시간이 더 듭니다. 그래서 측정 전에 몇 번 미리 돌려 두는 **준비 운동(warm-up)** 이 필요해요.
- ④ **CUDA 이벤트: 20.7ms.** GPU의 작업 줄 사이에 "여기서 시작", "여기서 끝"이라는 표시를 끼워 넣고, GPU가 그 표시를 지나간 시각의 차이를 잽니다. 파이썬 시계를 쓰지 않으니 GPU 안에서 걸린 시간만 정확히 나와요.
- ⑤ **`.item()`까지 잼: 27.4ms.** 결과를 CPU로 꺼내는 순간에는 파이썬이 저절로 기다립니다. 값이 있어야 꺼낼 수 있으니까요. `.item()`, `.cpu()`, `print(텐서)`가 모두 그렇습니다. ③보다 7ms쯤 더 걸렸는데, 숫자 6,700만 개의 합계는 1ms도 안 걸리니 대부분은 합계 때문이 아니

에요. 한 번만 잰 값이라 그때그때의 GPU 상태에 따라 흔들린 것으로 보여요. 그래서 바로 다음에 만들 측정 도구는 여러 번 재서 중앙값을 씁니다.

⑤는 학습 코드에도 중요합니다. 반복문 안에서 `손실.item()`을 매 걸음 부르면 그때마다 CPU가 GPU를 기다려서, 다음 일감을 미리 맡겨 둘 기회가 사라져요. 손실은 몇 걸음에 한 번만 꺼내면 됩니다.

재기 도구 만들기

그럼 이 장에서 쓸 측정 함수를 하나 만들어 두겠습니다. 준비 운동을 몇 번 하고, 매번 동기화하며 여러 번 잰 뒤 **중앙값**을 돌려줍니다. 평균 대신 중앙값을 쓰는 이유는, 다른 프로그램이 잠깐 끼어들어 한 번 크게 튄 값이 결과를 흔들지 않게 하기 위해서예요. PyTorch에 들어 있는 측정 도구 `torch.utils.benchmark`와도 비교해 봅니다.

```
import statistics
import time

import torch
from torch.utils import benchmark


def 재기(함수, 반복: int = 50, 준비: int = 5) -> float:
    """준비 운동 뒤 한 번씩 동기화하며 재고, 중앙값(ms)을 돌려준다."""
    for _ in range(준비):
        함수()
    torch.cuda.synchronize()
    시간들 = []
    for _ in range(반복):
        시작 = time.perf_counter()
        함수()
        torch.cuda.synchronize()
        시간들.append((time.perf_counter() - 시작) * 1000)
    return statistics.median(시간들)


n = 4096
a = torch.randn(n, n, device="cuda")
```

```
b = torch.randn(n, n, device="cuda")

직접 = 재기(lambda: a @ b)
타이머 = benchmark.Timer(stmt="a @ b", globals={"a": a, "b": b})
측정 = 타이머.blocked_autorange(min_run_time=1.0)   # 1초 이상 알아서 반복해 잰다

print(f"직접 만든 재기()   : 중앙값 {직접:.3f} ms")
print(f"benchmark.Timer    : 중앙값 {측정.median * 1000:.3f} ms ({측정.number_per_run}번씩 {len(측
↪ 정.times)}묶음)")
연산수 = 2 * n ** 3                                 # 곱하기와 더하기 각각 n³번
print(f"초당 {연산수 / (직접 / 1000) / 1e12:.1f}조 번 계산 (TFLOPS)")
```

```
직접 만든 재기()   : 중앙값 2.588 ms
benchmark.Timer    : 중앙값 2.601 ms (10번씩 39묶음)
초당 53.1조 번 계산 (TFLOPS)
```

직접 만든 `재기()`와 `benchmark.Timer`가 2.59ms, 2.60ms로 거의 같습니다. `blocked_autorange`는 몇 번씩 몇 묶음을 잴지를 정해진 시간(여기서는 1초) 안에서 알아서 정하고 동기화도 해 줘요.

마지막 줄은 행렬 곱의 계산 횟수를 시간으로 나눈 값입니다. n × n 행렬 두 개를 곱하면 곱셈과 덧셈이 각각 n^3번이라 모두 $2n^3$번이에요. 이걸 초당으로 바꾸면 이 GPU가 `float32`로 초당 53조 번(53 TFLOPS) 계산했다는 뜻입니다. **TFLOPS**(테라플롭스)는 "초당 1조 번의 실수 계산"이라는 단위로, GPU 성능을 비교할 때 자주 써요. 16.7절에서 같은 행렬 곱이 이보다 세 배 넘게 빨라지는 것을 보게 됩니다.

Q. 4장에서는 `torch.mps.synchronize()`를 썼는데, 맥에서도 같은 원리인가요? 네, 같습니다. Apple MPS도 일을 맡기고 바로 돌아오는 비동기 방식이라 `torch.mps.synchronize()`로 기다린 뒤 재야 해요. 장치에 상관없이 쓰고 싶다면 `if 장치.type == "cuda": torch.cuda.synchronize()`처럼 장치 종류에 따라 골라 부르는 작은 함수를 만들어 두면 됩니다. CPU 계산은 처음부터 끝날 때까지 기다리므로 동기화가 필요 없어요.

16.3 CPU와 GPU, 언제 GPU가 이길까요

4장 4.8절 끝에서 “행렬이 작으면 오히려 CPU가 빠를 수도 있다. 16장에서 더 정확히 재 본다”고 약속했습니다. 이제 그 약속을 지킬 차례예요. 행렬 크기를 16부터 4096까지 바꿔 가며 세 가지를 쟀습니다. CPU로 곱하기, 이미 GPU에 있는 행렬을 GPU로 곱하기(계산만), 그리고 CPU에 있는 행렬을 GPU로 옮겨서 곱하고 결과를 다시 CPU로 가져오기(왕복)입니다.

```
import statistics
import time

import torch


def 재기(함수, 반복: int = 30) -> float:
    """준비 운동 뒤 동기화하며 재고 중앙값(ms)을 돌려준다."""
    for _ in range(5):
        함수()
    torch.cuda.synchronize()
    시간들 = []
    for _ in range(반복):
        시작 = time.perf_counter()
        함수()
        torch.cuda.synchronize()
        시간들.append((time.perf_counter() - 시작) * 1000)
    return statistics.median(시간들)


print(f"CPU 스레드 {torch.get_num_threads()}개")
for n in [16, 64, 256, 1024, 4096]:
    a, b = torch.randn(n, n), torch.randn(n, n)
    ga, gb = a.to("cuda"), b.to("cuda")
    cpu = 재기(lambda: a @ b)
    gpu = 재기(lambda: ga @ gb)
    왕복 = 재기(lambda: (a.to("cuda") @ b.to("cuda")).cpu())   # 옮기고, 곱하고, 되가져오기
    print(f"{n:>4}×{n:<4} CPU {cpu:7.3f} ¦ GPU 계산만 {gpu:6.3f} ¦ GPU 왕복 {왕복:6.3f} ms"
          f" → GPU가 {cpu / gpu:4.1f}배 / {cpu / 왕복:3.1f}배")
```

```
CPU 스레드 24개
  16×16   CPU   0.004 ¦ GPU 계산만  0.007 ¦ GPU 왕복  0.024 ms → GPU가  0.5배 / 0.1배
  64×64   CPU   0.007 ¦ GPU 계산만  0.024 ¦ GPU 왕복  0.074 ms → GPU가  0.3배 / 0.1배
 256×256  CPU   0.049 ¦ GPU 계산만  0.024 ¦ GPU 왕복  0.176 ms → GPU가  2.0배 / 0.3배
1024×1024 CPU   1.748 ¦ GPU 계산만  0.049 ¦ GPU 왕복  1.151 ms → GPU가 35.8배 / 1.5배
4096×4096 CPU 105.370 ¦ GPU 계산만  2.506 ¦ GPU 왕복 30.134 ms → GPU가 42.0배 / 3.5배
```

맨 오른쪽의 두 숫자는 "GPU가 CPU보다 몇 배 빠른가"를 계산만 했을 때와 왕복까지 했을 때로 나눈 것입니다. 1보다 작으면 CPU가 빠르다는 뜻이에요. 이 서버의 CPU는 스레드 24개로 행렬 곱을 나눠 계산합니다(1권 17장에서 본 스레드와 같은 뜻이에요).

- **16 × 16, 64 × 64**: GPU가 계산만 해도 CPU보다 느립니다. 숫자가 몇천 개뿐이라 CPU는 순식간에 끝내는데, GPU는 16.1절의 일감 맡기는 비용만으로 수~수십 μs가 들어요.
- **256 × 256**: 계산만 보면 GPU가 2배 빨라지기 시작합니다. 이 근처가 교차점이에요. 하지만 왕복까지 하면 여전히 CPU가 3배쯤 빠릅니다.
- **1024 × 1024**: 계산만은 36배 빠른데, 왕복까지 하면 1.5배밖에 안 됩니다. 1.15ms 중 대부분이 행렬을 옮기는 시간이라는 뜻이에요.
- **4096 × 4096**: 계산만 하면 CPU 105ms 대 GPU 2.5ms로 42배. 왕복해도 3.5배 빠릅니다.

이 표에서 기억할 것은 숫자 자체보다 **모양**입니다. 작은 계산은 CPU가 빠르고, 클수록 GPU가 압도적이며, 그 사이에는 **옮기는 비용**이 이득을 갉아먹는 구간이 있어요. 그래서 실무에서 지키는 원칙이 두 가지 나옵니다.

1. **한 번 GPU에 올린 것은 GPU에 둡니다.** 모델은 학습 시작 전에 한 번만 옮기고, 중간 결과를 CPU로 가져왔다 다시 보내는 왕복을 만들지 않아요.
2. **옮겨야 한다면 한꺼번에, 그리고 숨겨서 옮깁니다.** 데이터는 배치 단위로 묶어 옮기고, GPU가 계산하는 동안 다음 배치를 옮겨 둡니다. 16.5절의 고정 메모리와 `non_blocking`이 그 방법이에요.

그래서 특성 몇 개짜리 표 데이터를 다루는 아주 작은 신경망은 GPU 서버에서도 CPU가 더 빠를 수 있습니다. 직접 재 보고 고르세요.

16.4 GPU 메모리 들여다보기

할당과 예약: PyTorch의 메모리 창고

GPU로 학습하다 가장 자주 만나는 오류는 "메모리가 부족하다"는 오류입니다. 메모리를 다루려면 먼저 PyTorch가 GPU 메모리를 어떻게 관리하는지 알아야 해요. 텐서를 만들고 지우면서 두 가지 숫자를 찍어 보겠습니다.

```
import torch

MB = 1024 ** 2


def 보기(때: str) -> None:
    """지금 할당된 메모리와, PyTorch가 GPU에서 맡아 둔(예약한) 메모리를 보여 준다."""
    print(f"할당 {torch.cuda.memory_allocated() / MB:6.1f} MB ¦ "
          f"예약 {torch.cuda.memory_reserved() / MB:6.1f} MB  ← {때}")


보기("처음")
x = torch.zeros(256, 1024, 1024, device="cuda")      # 2억 6천만 칸 × 4바이트 = 1GB
보기("1GB 텐서를 만든 뒤")
y = x + 1
보기("계산 결과까지")
del x, y
보기("del 한 뒤")
z = torch.zeros(128, 1024, 1024, device="cuda")      # 0.5GB: 맡아 둔 자리를 다시 쓴다
보기("0.5GB를 다시 만든 뒤")
del z
torch.cuda.empty_cache()                              # 맡아 둔 자리를 GPU에 돌려준다
보기("empty_cache 뒤")
print(f"지금까지의 최고 할당: {torch.cuda.max_memory_allocated() / MB:.1f} MB")
```

```
할당    0.0 MB ¦ 예약    0.0 MB  ← 처음
할당 1024.0 MB ¦ 예약 1024.0 MB  ← 1GB 텐서를 만든 뒤
할당 2048.0 MB ¦ 예약 2048.0 MB  ← 계산 결과까지
할당    0.0 MB ¦ 예약 2048.0 MB  ← del 한 뒤
할당  512.0 MB ¦ 예약 2048.0 MB  ← 0.5GB를 다시 만든 뒤
할당    0.0 MB ¦ 예약    0.0 MB  ← empty_cache 뒤
```

```
지금까지의 최고 할당: 2048.0 MB
```

`memory_allocated()`는 지금 **텐서들이 실제로 쓰고 있는** 메모리, `memory_reserved()`는 PyTorch가 GPU에서 **받아 두고 관리하는** 메모리입니다.

1GB 텐서를 만들면 둘 다 1GB가 되고, `x + 1`의 결과까지 만들면 2GB가 됩니다. 4장 4.3절의 계산 그대로예요. 칸 2억 6천만 개(256 × 1024 × 1024) × 4바이트 = 1GB입니다. 그런데 `del`로 지운 뒤를 보세요. 할당은 0인데 예약은 2GB 그대로입니다. 그다음 0.5GB를 만들어도 예약은 늘지 않았어요.

이것이 PyTorch의 **캐싱 할당자(caching allocator)** 입니다. GPU에서 메모리를 받았다 돌려주는 일은 느려서, PyTorch는 다 쓴 메모리를 자기 창고에 두었다가 다음 텐서에 다시 내줘요. 학습은 같은 크기의 텐서를 매 걸음 만들고 버리니, 두 번째 걸음부터는 메모리를 새로 받을 일이 거의 없습니다. `torch.cuda.empty_cache()`는 창고의 빈자리를 GPU에 돌려주는데, 속도에는 도움이 안 되고 다른 프로그램에 자리를 내줄 때나 씁니다.

마지막 줄의 `max_memory_allocated()`는 지금까지 **가장 많이 쓴 순간**의 할당량입니다. 배치를 얼마나 키울 수 있을지는 이 값으로 판단해요. `reset_peak_memory_stats()`로 기록을 지우고 다시 잴 수 있습니다.

> **Q. `nvidia-smi`에 보이는 사용량은 `memory_allocated()`보다 훨씬 큰데요?** `nvidia-smi`는 GPU 입장에서 본 사용량이라, PyTorch가 창고에 받아 둔 예약 메모리 전체에 더해 CUDA 자체가 쓰는 기본 메모리(보통 수백 MB)까지 보여 줍니다. 텐서를 다 지웠는데도 `nvidia-smi` 숫자가 줄지 않는 건 대개 캐싱 할당자 때문이니 걱정하지 않아도 돼요. 내 텐서가 실제로 얼마를 쓰는지는 `memory_allocated()`, 창고까지 포함하면 `memory_reserved()`를 보세요.

학습 한 걸음은 메모리를 얼마나 쓸까

이제 진짜 모델로 재 보겠습니다. 10장에서 전이 학습에 썼던 ResNet18을 224 × 224 크기의 가짜 사진으로 학습시키며, 배치 크기에 따라 메모리가 어떻게 변하는지 봅니다. 배치 크기와 상관없이 쓰는 파라미터 메모리는 14장 14.6절의 계산법(파라미터 수 × 칸 크기)으로 먼저 구했어요.

```
import torch
import torch.nn.functional as F
from torchvision.models import resnet18
```

```
MB = 1024 ** 2
모델 = resnet18(num_classes=10).to("cuda")
개수 = sum(p.numel() for p in 모델.parameters())
print(f"파라미터 {개수:,}개 × 4바이트 = {개수 * 4 / MB:.1f} MB (14장 14.6절의 계산)")
print(f"모델을 올린 뒤 실제 할당   = {torch.cuda.memory_allocated() / MB:.1f} MB")

for 배치 in [16, 32, 64, 128]:
    x = torch.randn(배치, 3, 224, 224, device="cuda")
    y = torch.randint(0, 10, (배치,), device="cuda")
    전 = torch.cuda.memory_allocated()
    torch.cuda.reset_peak_memory_stats()            # 최고 기록을 지금부터 다시 잰다
    손실 = F.cross_entropy(모델(x), y)
    순전파_뒤 = torch.cuda.memory_allocated()       # backward를 위해 기억해 둔 중간값(활성화)
    손실.backward()
    최고 = torch.cuda.max_memory_allocated()
    모델.zero_grad(set_to_none=True)
    print(f"배치 {배치:>3}: 순전파가 남긴 것 {(순전파_뒤 - 전) / MB:7.1f} MB, "
          f"이 걸음의 최고 {최고 / MB:7.1f} MB")
    del x, y, 손실
```

```
파라미터 11,181,642개 × 4바이트 = 42.7 MB (14장 14.6절의 계산)
모델을 올린 뒤 실제 할당   = 42.7 MB
배치  16: 순전파가 남긴 것   341.0 MB, 이 걸음의 최고   439.9 MB
배치  32: 순전파가 남긴 것   662.5 MB, 이 걸음의 최고   812.4 MB
배치  64: 순전파가 남긴 것  1321.0 MB, 이 걸음의 최고  1453.0 MB
배치 128: 순전파가 남긴 것  2638.0 MB, 이 걸음의 최고  2828.5 MB
```

파라미터 1,118만 개 × 4바이트 = 42.7MB라는 계산이 실제 할당량과 정확히 맞습니다. 그런데 배치 16장을 순전파하자 메모리가 341MB나 더 늘었어요. 모델 크기의 8배입니다.

5장 5.5절에서 계산 그래프는 backward 때 쓰려고 **순전파의 중간 결과를 기억해 둔다**고 했습니다. 이 중간값을 **활성화(activation)** 라고 불러요. ResNet18은 224 × 224 사진 한 장을 지나가며 층마다 큰 특징 맵을 만들고, 이걸 전부 기억하니 사진 한 장에 20MB쯤 듭니다. 그래서 배치를 두 배로 늘릴 때마다 341 → 663 → 1,321 → 2,638MB로 활성화 메모리도 정확히 두 배씩 늘어요. 최고 메모리는 여기에 backward에서 만드는 기울기와 임시 텐서가 더해진 값입니다.

이 결과가 알려 주는 것은 분명합니다. **학습할 때 메모리를 가장 많이 먹는 것은 모델이 아니라 활성화이고, 활성화는 배치 크기에 비례합니다.** 학습 중 메모리가 파라미터, 기울기, 옵티마이저 상태, 활성화로 어떻게 나뉘는지, 그리고 활성화를 줄이는 방법(활성화 체크포인팅)은 17장에서 자세히 다룹니다.

메모리 부족 오류 재현하고 대처하기

그럼 배치를 계속 키우면 어떻게 될까요? 두 배씩 키우다가 메모리 부족 오류(OOM, Out Of Memory)가 나면 잡아서 멈추는 코드입니다.

```
import torch
import torch.nn.functional as F
from torchvision.models import resnet18

GB = 1024 ** 3
모델 = resnet18(num_classes=10).to("cuda")


def 한_걸음(배치: int) -> None:
    """배치 크기만큼 가짜 사진을 만들어 순전파·역전파를 한 번 한다."""
    x = torch.randn(배치, 3, 224, 224, device="cuda")
    y = torch.randint(0, 10, (배치,), device="cuda")
    F.cross_entropy(모델(x), y).backward()
    모델.zero_grad(set_to_none=True)


배치, 성공한_배치 = 64, None
while True:
    try:
        torch.cuda.reset_peak_memory_stats()
        한_걸음(배치)
        print(f"배치 {배치:>5}: 성공, 최고 {torch.cuda.max_memory_allocated() / GB:5.1f} GB")
        성공한_배치 = 배치
        배치 *= 2
    except torch.OutOfMemoryError as 오류:              # GPU 메모리가 모자랄 때 나는 오류
        print(f"배치 {배치:>5}: 메모리 부족!")
        print("   ", ". ".join(str(오류).split(". ")[:3]) + ".")   # 메시지의 앞 세 문장
```

```
        break

torch.cuda.empty_cache()                                    # 실패한 걸음이 잡아 둔 자리 정리
print(f"예약 메모리 정리 뒤: {torch.cuda.memory_reserved() / GB:.1f} GB")
한_걸음(성공한_배치)                                        # 정리하고 나면 다시 계속할 수 있다
print(f"배치 {성공한_배치}로 다시 학습: 문제없음")
```

```
배치    64: 성공, 최고   1.4 GB
배치   128: 성공, 최고   2.8 GB
배치   256: 성공, 최고   5.4 GB
배치   512: 성공, 최고  10.8 GB
배치  1024: 성공, 최고  21.5 GB
배치  2048: 메모리 부족!
    CUDA out of memory. Tried to allocate 1.53 GiB. GPU 0 has a total capacity of 23.64 GiB of
↪  which 1.46 GiB is free.
예약 메모리 정리 뒤: 0.1 GB
배치 1024로 다시 학습: 문제없음
```

배치 1,024까지는 21.5GB로 버텼고, 2,048에서 메모리가 모자랐습니다. 오류 메시지는 "1.53GiB를 받으려 했는데 전체 23.64GiB 중 1.46GiB만 비어 있다"는 뜻이에요. 4장 4.10절의 요령대로 숫자부터, **받으려던 양**과 **남은 양**을 보면 됩니다. 메시지 뒷부분에는 PyTorch가 쓰는 양과 메모리 조각 문제를 줄이는 설정(`PYTORCH_CUDA_ALLOC_CONF`) 안내가 이어져요.

`torch.OutOfMemoryError`는 다른 오류처럼 `try`/`except`로 잡을 수 있습니다(1권 12장). 잡은 뒤 `empty_cache()`로 실패한 걸음이 잡아 둔 자리를 정리하면 예약 메모리가 0.1GB로 돌아오고, 배치 1,024로 학습을 다시 이어 갈 수 있었어요. 다만 실제 학습 코드에서 OOM을 잡아 넘기는 일은 드뭅니다. 보통은 OOM이 나면 멈추고 다음 순서로 대처해요.

1. **배치를 줄입니다.** 가장 확실한 방법입니다. 위 결과처럼 메모리는 배치에 거의 비례하니, 절반으로 줄이면 대개 들어갑니다.
2. **16비트로 계산합니다.** 다음 16.6절의 혼합 정밀도는 활성화를 절반 크기로 저장해서 메모리를 크게 줄여 줘요.
3. **큰 배치의 효과는 나눠서 냅니다.** 작은 배치 여러 개의 기울기를 쌓아 큰 배치처럼 쓰는 그래디언트 누적(5장에서 예고한 기법)과, 활성화를 덜 저장하는 활성화 체크포인팅은 17장에서 다룹니다.

4. **평가 때는 `torch.no_grad()`를 씁니다.** 5장 5.8절의 기록 끄기를 하면 활성화를 저장하지 않아서 메모리가 크게 줄어요.

16.5 데이터 옮기는 비용: 고정 메모리와 non_blocking

고정 메모리는 왜 빠를까

16.3절에서 옮기는 비용이 GPU의 이득을 갉아먹는 것을 봤습니다. 7장 7.10절에서는 `DataLoader`의 `pin_memory=True`를 "배치를 GPU로 옮기기 좋은 특별한 메모리 영역에 담아 두는 옵션"이라고만 소개하고 넘어갔죠. 이제 정말 그런지 재 보겠습니다.

운영체제는 메모리가 모자라면 오래 안 쓴 내용을 디스크로 잠시 내보내기 때문에, 보통 메모리는 내용이 **어디에 있을지 장담할 수 없습니다.** 그래서 GPU로 보낼 때는 움직이지 않는 임시 공간에 한 번 복사한 뒤 보내요. **고정 메모리(pinned memory)** 는 처음부터 "이 자리에서 움직이지 않는다"고 못 박아 둔 메모리라 이 중간 복사가 필요 없습니다. 택배로 치면 물건을 이미 출고장에 내놓은 것이에요.

```
import time

import torch

GB = 1024 ** 3
일반 = torch.randn(64, 1024, 1024)                # 256MB, 보통의 CPU 메모리
고정 = 일반.pin_memory()                           # 같은 내용을 고정(pinned) 메모리에
gpu_텐서 = 일반.to("cuda")
고정_받을_곳 = torch.empty_like(일반).pin_memory()  # GPU에서 받아 올 고정 메모리 자리


def 초당_GB(옮기기, 반복: int = 10) -> float:
    """옮기기를 여러 번 해 보고 초당 몇 GB를 옮겼는지 잰다."""
    옮기기()
    torch.cuda.synchronize()
    시작 = time.perf_counter()
    for _ in range(반복):
        옮기기()
    torch.cuda.synchronize()
```

```
    return 일반.numel() * 일반.element_size() * 반복 / (time.perf_counter() - 시작) / GB


print(f"CPU(일반) → GPU: {초당_GB(lambda: 일반.to('cuda')):5.1f} GB/s")
print(f"CPU(고정) → GPU: {초당_GB(lambda: 고정.to('cuda')):5.1f} GB/s")
print(f"GPU → CPU(일반): {초당_GB(lambda: gpu_텐서.to('cpu')):5.1f} GB/s")
print(f"GPU → CPU(고정): {초당_GB(lambda: 고정_받을_곳.copy_(gpu_텐서)):5.1f} GB/s")

# non_blocking=True: 옮기라고 맡기고 기다리지 않고 바로 다음 줄로 간다
for 이름, 원본 in [("일반", 일반), ("고정", 고정)]:
    torch.cuda.synchronize()
    시작 = time.perf_counter()
    도착 = 원본.to("cuda", non_blocking=True)
    맡긴_시간 = (time.perf_counter() - 시작) * 1000
    torch.cuda.synchronize()
    끝난_시간 = (time.perf_counter() - 시작) * 1000
    print(f"{이름} 메모리 + non_blocking: 맡기고 돌아오기까지 {맡긴_시간:6.2f} ms, "
          f"다 옮기기까지 {끝난_시간:6.2f} ms")
```

```
CPU(일반) → GPU:  11.5 GB/s
CPU(고정) → GPU:  12.2 GB/s
GPU → CPU(일반):   3.7 GB/s
GPU → CPU(고정):  12.3 GB/s
일반 메모리 + non_blocking: 맡기고 돌아오기까지  21.54 ms, 다 옮기기까지  21.65 ms
고정 메모리 + non_blocking: 맡기고 돌아오기까지   0.15 ms, 다 옮기기까지  20.61 ms
```

256MB를 옮겨 본 결과입니다. CPU에서 GPU로 갈 때는 보통 메모리 11.5GB/s, 고정 메모리 12.2GB/s로 차이가 크지 않았어요. 이 방향은 중간 복사가 옮기는 일과 겹쳐 진행되면서 비용이 꽤 가려지는 것으로 보여요. 반대 방향은 달랐습니다. GPU에서 보통 메모리로 받아 올 때는 3.7GB/s, 고정 메모리로 받아 오면 12.3GB/s로 세 배 넘게 빨라요.

12GB/s 근처라는 숫자에도 이유가 있습니다. 이 서버에서 GPU 0번은 CPU와 **PCIe 4.0 x8** 통로로 연결되어 있는데, 이 통로의 이론상 최고 속도가 약 16GB/s예요. 실제로는 그 75% 정도가 나온 것입니다. GPU 안에서 메모리를 읽고 쓰는 속도(메모리 대역폭)는 1,000GB/s 수준이니(16.8절), **GPU 밖으로 나가는 길은 GPU 안보다 수십 배 좁습니다.** 16.3절에서 왕복이 그렇게 비쌌던 이유예요. 17장에서 GPU 두 장이 서로 기울기를 주고받을 때도 이 좁은 길이 다시 문제가 됩니다.

non_blocking: 맡기고 다른 일 하기

마지막 두 줄이 이 절의 핵심입니다. `.to("cuda", non_blocking=True)`는 "옮기라고 맡기고 기다리지 말고 돌아와라"는 뜻이에요. 그런데 보통 메모리에서는 맡기고 돌아오기까지 21.5ms, 즉 다 옮길 때까지 그대로 기다렸습니다. 중간 복사를 CPU가 직접 해야 하니 비동기로 할 수가 없거든요. 고정 메모리에서는 0.15ms 만에 돌아왔고, 실제 복사는 그 뒤 20ms 동안 GPU 쪽에서 따로 진행됐습니다. **`non_blocking`은 고정 메모리와 함께 써야 효과가 있습니다.**

CPU가 0.15ms 만에 풀려나면 무엇이 좋을까요? 그동안 다음 배치를 준비하거나 GPU에 다음 계산을 맡겨 둘 수 있습니다. 그래서 GPU 학습 코드는 흔히 이렇게 씁니다.

```
로더 = DataLoader(데이터셋, batch_size=256, shuffle=True,
                  num_workers=8, pin_memory=True)      # 일꾼이 배치를 고정 메모리에 담아 준다
for x, y in 로더:
    x = x.to(장치, non_blocking=True)                   # 맡기고 바로 다음 줄로
    y = y.to(장치, non_blocking=True)
    ...
```

`pin_memory=True`를 주면 DataLoader가 배치를 고정 메모리에 담아서 건네주고, 우리는 `non_blocking=True`로 옮기기만 하면 됩니다. 같은 GPU 작업 줄 안에서는 순서가 지켜지므로, 복사가 끝나기 전에 모델 계산이 그 텐서를 읽는 일은 없어요. 반대 방향(GPU → CPU)에 `non_blocking=True`를 쓰면 복사가 끝나기 전에 값을 읽을 위험이 있으니, 결과를 가져올 때는 그냥 `.cpu()`나 `.item()`을 쓰세요.

Q

Q. 그럼 모든 텐서를 고정 메모리에 두면 더 빠르지 않나요? 고정 메모리는 운영체제가 디스크로 내보낼 수 없는 메모리라, 많이 잡으면 컴퓨터 전체의 다른 프로그램이 쓸 메모리가 줄어듭니다. 그래서 데이터셋 전체가 아니라 **GPU로 보낼 배치만** 고정 메모리에 두는 것이 원칙이고, `pin_memory=True`가 바로 그렇게 해 줘요. 7장 7.10절에서 말했듯 Apple MPS에서는 고정 메모리가 지원되지 않아 경고만 나오니 맥에서는 이 옵션을 빼세요.

16.6 혼합 정밀도: 16비트로 계산하기

16비트 두 가지: float16과 bfloat16

4장 4.3절에서 딥러닝의 기본이 32비트인 이유를 이야기하면서 "요즘은 16비트까지 내려서 속도를 더 얻는다, 16장에서 이어진다"고 했습니다. 칸이 절반이면 메모리도 절반, 옮기는 시간도 절반이고, 무엇보다 요즘 GPU에는 16비트 행렬 곱만 전담하는 **텐서 코어**(Tensor Core) 라는 계산 장치가 따로 있어서 계산 자체가 몇 배 빨라집니다.

그런데 16비트 실수에는 두 종류가 있어요. 칸 16개를 어떻게 나눠 쓰느냐가 다릅니다. 실수는 "몇 자리 숫자(가수)를 10의 몇 제곱(지수)만큼 키운다"는 식으로 저장하는데, `float16`은 가수에 칸을 많이 주고, `bfloat16`은 지수에 칸을 많이 줍니다. 각각이 담을 수 있는 수의 범위를 보죠.

```
import torch

for 자료형 in [torch.float32, torch.float16, torch.bfloat16]:
    정보 = torch.finfo(자료형)                        # 자료형이 담을 수 있는 수의 범위
    print(f"{str(자료형):<15} 최대 {정보.max:8.2e} ¦ 가장 작은 정규 수 {정보.tiny:8.2e}"
          f" ¦ 1 옆의 간격 {정보.eps:8.2e}")

값 = torch.tensor([70000.0, 0.00000001, 1.001, 0.1])
for 자료형 in [torch.float32, torch.float16, torch.bfloat16]:
    바꾼_값 = 값.to(자료형).tolist()
    print(f"{str(자료형):<15}", " ".join(f"{v:>13.7g}" for v in 바꾼_값))
```

```
torch.float32   최대 3.40e+38 ¦ 가장 작은 정규 수 1.18e-38 ¦ 1 옆의 간격 1.19e-07
torch.float16   최대 6.55e+04 ¦ 가장 작은 정규 수 6.10e-05 ¦ 1 옆의 간격 9.77e-04
torch.bfloat16  최대 3.39e+38 ¦ 가장 작은 정규 수 1.18e-38 ¦ 1 옆의 간격 7.81e-03
torch.float32           70000         1e-08         1.001           0.1
torch.float16             inf             0      1.000977    0.09997559
torch.bfloat16          70144  1.001172e-08             1     0.1000977
```

위의 세 줄은 `torch.finfo`로 자료형마다 담을 수 있는 가장 큰 수, 가장 작은 (정규) 수, 그리고 1 바로 옆의 수와의 간격을 본 것입니다. 아래 세 줄은 같은 네 숫자 70000, 0.00000001, 1.001, 0.1을 각 자료형으로 바꿔 본 결과예요.

- `float16`: 가장 큰 수가 65,504라서 **70000은 `inf`(무한대)가 됩니다.** 아주 작은 수 쪽도 좁아서 0.00000001은 0이 돼 버렸어요. 대신 1 옆의 간격이 0.001이라 1.001을 1.000977로 꽤 가깝게 담았습니다.

- `bfloat16`: 가장 큰 수와 가장 작은 수가 `float32`와 같습니다. 70000도, 0.00000001도 담겼어요. 대신 1 옆의 간격이 0.0078이라 거칩니다. 1.001은 그냥 1이 되고, 70000은 70144가 됐어요.

정리하면 `float16`은 **범위가 좁고 촘촘한** 자, `bfloat16`은 **범위가 넓고 성긴** 자입니다. 딥러닝에서는 대부분 범위가 더 중요해요. 4장에서 말했듯 끝자리의 작은 오차는 학습에 큰 영향이 없지만, 값이 `inf`가 되거나 0이 되어 사라지면 학습이 망가지거든요. 그래서 `bfloat16`을 지원하는 GPU(NVIDIA는 계산 능력 8.0 이상, 앞에서 본 `is_bf16_supported()`)라면 `bfloat16`을 먼저 고르는 것이 요즘의 관례입니다. 14장 14.6절에서 LLM을 `bfloat16`으로 올린 것도 같은 이유예요.

autocast: 알아서 골라 주는 16비트

그럼 모델 전체를 `.half()`로 바꾸면 될까요? 그러면 손실이나 소프트맥스처럼 작은 값을 많이 더하는 계산에서 오차가 크게 쌓입니다. 그래서 **계산마다 알맞은 자료형을 골라 쓰는** 방법을 써요. 이것이 **혼합 정밀도**(mixed precision) 이고, `torch.autocast`가 그 고르기를 대신해 줍니다.

```
import torch
import torch.nn.functional as F
from torch import nn


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")


장치 = 장치_고르기()
반정밀도 = torch.bfloat16 if 장치.type == "cpu" else torch.float16
torch.manual_seed(0)
모델 = nn.Sequential(nn.Linear(64, 64), nn.ReLU(), nn.Linear(64, 10)).to(장치)
x = torch.randn(8, 64, device=장치)
y = torch.randint(0, 10, (8,), device=장치)
```

```
with torch.autocast(device_type=장치.type, dtype=반정밀도):   # 이 안에서만 16비트로
    은닉 = 모델[0](x)
    점수 = 모델(x)
    확률 = 점수.softmax(dim=1)
    손실 = F.cross_entropy(점수, y)
print("입력:", x.dtype, "| Linear 출력:", 은닉.dtype, "| 점수:", 점수.dtype)
print("softmax:", 확률.dtype, "| 손실:", 손실.dtype)

손실.backward()                                              # backward는 with 밖에서
print("가중치:", 모델[0].weight.dtype, "| 기울기:", 모델[0].weight.grad.dtype)
print("autocast 밖의 Linear 출력:", 모델[0](x).dtype)
```

```
입력: torch.float32 | Linear 출력: torch.float16 | 점수: torch.float16
softmax: torch.float32 | 손실: torch.float32
가중치: torch.float32 | 기울기: torch.float32
autocast 밖의 Linear 출력: torch.float32
```

`with torch.autocast(...)` 블록 안에서 일어난 일을 보세요.

- 입력은 `float32`인데 **`Linear`의 출력은 `float16`** 입니다. 행렬 곱은 16비트로 해도 안전하고, 텐서 코어 덕분에 가장 많이 빨라지는 계산이라 autocast가 16비트를 골랐어요.
- **`softmax`와 손실은 `float32`** 입니다. 지수 함수와 큰 합계가 들어가 16비트에서 오차가 크게 나는 계산이라, 입력이 16비트여도 autocast가 32비트로 올려서 계산했습니다.
- **가중치와 기울기는 여전히 `float32`** 입니다. 모델 안의 진짜 숫자는 32비트로 보관하고, 계산할 때만 16비트 사본을 만들어 쓰는 거예요. 이렇게 해야 학습률 × 기울기처럼 아주 작은 변화도 가중치에 제대로 더해집니다.
- autocast 블록 **밖**에서는 같은 `Linear`가 다시 `float32`를 냅니다. 모델을 바꾼 게 아니라 블록 안에서만 계산 방식을 바꾼 거죠.

`backward()`는 블록 밖에서 부릅니다. 역전파는 순전파 때 정해진 자료형을 그대로 따라가요. 이 코드는 `장치_고르기()`를 쓰므로 이 책의 맥(MPS)에서도 똑같은 자료형이 나왔고, CPU에서는 `bfloat16`을 쓰도록 했습니다.

GradScaler: float16의 사라지는 기울기 지키기

`float16`에는 한 가지 문제가 더 있습니다. 학습 막바지에는 손실이 작아지고 기울기도 아주 작아지는데, `float16`은 위 표의 가장 작은 정규 수(6×10^{-5})보다 작아지면 정밀도를 잃기 시작하고, 6×10^{-8}보다 작으면 아예 0이 되어 버려요(위에서 0.00000001이 0이 된 것처럼). 이것을 **언더플로(underflow)** 라고 합니다. 기울기가 0이면 학습이 멈추겠죠.

해결법은 간단합니다. backward 하기 전에 손실에 큰 수(예: 65,536)를 곱해 기울기를 통째로 키워서 계산하고, 가중치를 고치기 직전에 같은 수로 다시 나누는 거예요. 아주 작은 글씨를 확대 복사했다가 읽은 뒤 원래 크기로 되돌리는 셈입니다. 정말 효과가 있는지 일부러 아주 작은 손실을 만들어 확인해 보겠습니다.

```
import math

import torch
from torch import nn

torch.manual_seed(0)
모델 = nn.Linear(1024, 1024).to("cuda")
x = torch.randn(256, 1024, device="cuda")


def 기울기_구하기(배율: float) -> torch.Tensor:
    """float16 autocast로 계산하고, 손실에 배율을 곱해 backward한 뒤 배율로 다시 나눈다."""
    모델.zero_grad()
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        출력 = 모델(x)
    손실 = 출력.float().pow(2).mean() * 1e-3        # 학습 막바지처럼 아주 작은 손실
    (손실 * 배율).backward()
    return 모델.weight.grad / 배율


# 기준: autocast 없이 float32로만 계산한 기울기
모델.zero_grad()
(모델(x).pow(2).mean() * 1e-3).backward()
기준 = 모델.weight.grad.clone()
```

```
for 배율 in [1.0, 2.0 ** 16]:
    기울기 = 기울기_구하기(배율)
    print(f"배율 {배율:>7.0f}: 0이 된 기울기 {(기울기 == 0).float().mean():6.1%}, "
          f"float32 기준과의 상대 오차 {((기울기 - 기준).norm() / 기준.norm()).item():.4f}")

# 실제 학습에서는 GradScaler가 배율을 알아서 관리한다
옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.1)
스케일러 = torch.amp.GradScaler("cuda", init_scale=2.0 ** 40)   # 일부러 너무 큰 배율로 시작
걸음, 갱신_횟수 = 0, 0
while 갱신_횟수 < 2:                                # 가중치를 두 번 갱신할 때까지
    옵티마이저.zero_grad()
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        손실 = 모델(x).float().pow(2).mean() * 1e-3
    스케일러.scale(손실).backward()                 # 손실에 배율을 곱해 backward
    배율 = 스케일러.get_scale()
    스케일러.step(옵티마이저)                       # 기울기를 배율로 나누고, inf가 있으면 건너뛴다
    스케일러.update()                               # inf가 있었으면 배율을 반으로
    결과 = "inf 발견, 이번 걸음은 건너뜀" if 스케일러.get_scale() < 배율 else "정상, 가중치 갱신"
    print(f"걸음 {걸음}: 배율 2^{math.log2(배율):.0f} → {결과}")
    갱신_횟수 += 결과.startswith("정상")
    걸음 += 1
```

```
배율       1: 0이 된 기울기 100.0%, float32 기준과의 상대 오차 1.0000
배율   65536: 0이 된 기울기   0.0%, float32 기준과의 상대 오차 0.0005
걸음 0: 배율 2^40 → inf 발견, 이번 걸음은 건너뜀
걸음 1: 배율 2^39 → inf 발견, 이번 걸음은 건너뜀
걸음 2: 배율 2^38 → inf 발견, 이번 걸음은 건너뜀
걸음 3: 배율 2^37 → 정상, 가중치 갱신
걸음 4: 배율 2^37 → 정상, 가중치 갱신
```

배율 없이 `float16`으로 계산하자 가중치 기울기가 **100% 0이 됐습니다.** `float32` 기준과의 상대 오차 1.0은 "완전히 틀렸다"는 뜻이에요. 배율 65,536을 곱했더니 0이 된 기울기가 하나도 없고, 오차도 0.0005로 `float32`와 거의 같아졌습니다.

배율이 너무 크면 이번에는 반대로 기울기가 65,504를 넘어 `inf`가 됩니다. 그래서 적당한 배율을 사람이 정하기는 어려워요. **`torch.amp.GradScaler`** 가 이 일을 자동으로 해 줍니다. 위 결과에서는 일

부터 2^{40}이라는 너무 큰 배율로 시작했어요. `GradScaler`는 기울기에서 `inf`를 발견하면 **그 걸음은 가중치를 고치지 않고 건너뛰고** 배율을 절반으로 줄입니다. 2^{40}, 2^{39}, 2^{38}에서 세 번 건너뛰고 2^{37}에서 정상으로 갱신하기 시작했죠. 반대로 오랫동안(기본 2,000걸음) `inf`가 안 나오면 배율을 두 배로 키워서, 언제나 넘치지 않는 선에서 가장 큰 배율을 유지합니다.

쓰는 법은 학습 루프에서 세 줄만 바꾸면 됩니다.

- `손실.backward()` → `스케일러.scale(손실).backward()`
- `옵티마이저.step()` → `스케일러.step(옵티마이저)`
- 그 뒤에 `스케일러.update()` 한 줄 추가

8장 8.9절의 그래디언트 클리핑과 함께 쓸 때는 주의할 점이 하나 있습니다. 기울기가 배율만큼 커져 있는 상태에서 자르면 엉뚱한 기준으로 자르게 되니, 자르기 전에 `스케일러.unscale_(옵티마이저)`로 배율을 먼저 나눠 줘야 해요. 그리고 **`bfloat16`은 범위가 `float32`와 같아서 `GradScaler`가 필요 없습니다.** 이것도 `bfloat16`을 먼저 고르는 이유예요.

속도, 메모리, 정확도 비교

이제 진짜 학습에서 비교해 보겠습니다. 12장에서 만든 작은 GPT와 같은 모양의 모델(층 6개, 차원 512, 헤드 8개)을 PyTorch의 `nn.TransformerEncoder` 부품으로 짧게 다시 만들었어요. 12장에서 한 줄 한 줄 만든 블록과 같은 구조를 PyTorch가 미리 만들어 둔 것입니다. 학습 데이터는 예제용으로 만든 규칙 데이터예요. 줄마다 시작 번호와 간격을 무작위로 정해서 7, 19, 31, 43, …처럼 일정하게 늘어나는 번호(1,000에서 넘치면 0으로 돌아감)를 만들었으니, 모델은 앞의 번호들을 보고 간격을 알아내야 다음 번호를 맞힐 수 있습니다.

```
import statistics
import time

import torch
import torch.nn.functional as F
from torch import nn

어휘, 길이, 배치 = 1000, 256, 32


class 작은_GPT(nn.Module):
    """12장의 작은 GPT와 같은 모양을 PyTorch 부품(nn.TransformerEncoder)으로 짧게 쓴 것."""
```

```
    def __init__(self, 차원: int = 512, 층: int = 6, 헤드: int = 8) -> None:
        super().__init__()
        self.글자 = nn.Embedding(어휘, 차원)
        self.위치 = nn.Embedding(길이, 차원)
        블록 = nn.TransformerEncoderLayer(차원, 헤드, 4 * 차원, dropout=0.0,
                                          batch_first=True, norm_first=True)
        self.몸통 = nn.TransformerEncoder(블록, 층, enable_nested_tensor=False)
        self.머리 = nn.Linear(차원, 어휘)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        T = x.shape[1]
        h = self.글자(x) + self.위치(torch.arange(T, device=x.device))
        마스크 = nn.Transformer.generate_square_subsequent_mask(T, device=x.device)
        return self.머리(self.몸통(h, mask=마스크, is_causal=True))


def 배치_만들기() -> torch.Tensor:
    """예제용으로 만든 규칙 데이터: 줄마다 시작 글자와 간격을 정해 일정하게 늘어나는 번호."""
    시작 = torch.randint(0, 어휘, (배치, 1), device="cuda")
    간격 = torch.randint(1, 20, (배치, 1), device="cuda")
    return (시작 + 간격 * torch.arange(길이 + 1, device="cuda")) % 어휘


def 학습(방식: str, 걸음_수: int = 200) -> tuple[float, float, float]:
    """방식대로 학습하고 (걸음당 ms, 최고 메모리 GB, 마지막 20걸음 평균 손실)을 돌려준다."""
    torch.manual_seed(0)
    모델 = 작은_GPT().to("cuda")
    옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=3e-4)
    자료형 = {"float32": None, "float16": torch.float16, "bfloat16": torch.bfloat16}[방식]
    스케일러 = torch.amp.GradScaler("cuda", enabled=(방식 == "float16"))  # float16일 때만 켠다
    torch.cuda.reset_peak_memory_stats()
    시간들, 손실들 = [], []
    for _ in range(걸음_수):
        시작 = time.perf_counter()
        데이터 = 배치_만들기()
```

```
        x, y = 데이터[:, :-1], 데이터[:, 1:]
        with torch.autocast("cuda", dtype=자료형, enabled=자료형 is not None):
            손실 = F.cross_entropy(모델(x).flatten(0, 1), y.flatten())
        옵티마이저.zero_grad(set_to_none=True)
        스케일러.scale(손실).backward()
        스케일러.step(옵티마이저)
        스케일러.update()
        손실들.append(손실.item())                  # .item()이 GPU를 기다려 준다
        시간들.append((time.perf_counter() - 시작) * 1000)
    return (statistics.median(시간들[10:]), torch.cuda.max_memory_allocated() / 1024 ** 3,
            statistics.mean(손실들[-20:]))


for 방식 in ["float32", "float16", "bfloat16"]:
    ms, 메모리, 손실 = 학습(방식)
    print(f"{방식:<9} 걸음당 {ms:5.1f} ms ¦ 최고 메모리 {메모리:5.2f} GB ¦ 마지막 손실 {손실:.4f}")
```

```
float32   걸음당  36.1 ms ¦ 최고 메모리  1.62 GB ¦ 마지막 손실 1.1455
float16   걸음당  14.7 ms ¦ 최고 메모리  1.09 GB ¦ 마지막 손실 1.1615
bfloat16  걸음당  14.4 ms ¦ 최고 메모리  1.09 GB ¦ 마지막 손실 1.1410
```

`float32`로 한 걸음에 36.1ms 걸리던 학습이 `float16`과 `bfloat16`에서는 14.7ms, 14.4ms로 **2.5배 빨라졌습니다.** 최고 메모리도 1.62GB에서 1.09GB로 줄었어요. 활성화를 16비트로 저장하니 그만큼 줄어든 것입니다. 가중치와 옵티마이저 상태는 `float32` 그대로라 정확히 절반까지 줄지는 않았어요.

200걸음 뒤의 손실도 1.1455, 1.1615, 1.1410으로 거의 같습니다. 16비트의 작은 오차가 학습 결과를 바꾸지 않았어요(실행마다 끝자리는 조금 달라질 수 있습니다). `GradScaler`와 `autocast`는 `enabled=`로 끄고 켤 수 있어서, 학습 함수 하나로 세 방식을 모두 돌렸습니다. 학습 함수에서 걸음마다 `손실.item()`을 부른 것은 16.2절에서 피하라고 한 일이지만, 여기서는 걸음마다 시간을 재려고 일부러 GPU를 기다리게 한 거예요.

Q

Q. float16과 bfloat16 중 무엇을 써야 하나요? GPU가 지원한다면 `bfloat16`을 권합니다. 위 결과처럼 속도는 거의 같은데, `GradScaler`가 필요 없고 값이 넘칠 걱정도 적어요. NVIDIA GPU 중에서는 A100(계산 능력 8.0)과 RTX 30 시리즈(8.6) 이후부터 하드웨어가 직접 지원합니다. Colab의 T4(계산 능력 7.5)처럼 그보다 오래된

Q GPU에서는 `bfloat16` 계산이 느리니 `float16` + `GradScaler`를 쓰세요. 확인할 때는 `torch.cuda.is_bf16_supported(including_emulation=False)`처럼 **흉내 내기를 빼고** 물어야 합니다. 그냥 `is_bf16_supported()`라고 부르면 하드웨어가 지원하지 않아도 흉내 내서 계산할 수 있으면 `True`를 돌려줘서, T4에서도 `True`가 나와요.

16.7 TF32: float32 행렬 곱을 빠르게

혼합 정밀도는 코드를 조금 고쳐야 합니다. 코드를 한 줄도 안 고치고 빨라지는 방법은 없을까요? 텐서 코어에는 TF32라는 계산 방식이 하나 더 있습니다. 입력과 출력은 `float32` 그대로 두고, 텐서 코어 안에서 곱할 때만 가수를 `bfloat16`보다 조금 긴 10비트로 줄여서 계산해요. 범위는 `float32`와 같고 정밀도만 조금 낮춘 셈입니다. `torch.set_float32_matmul_precision`으로 켜고 끕니다. 같은 행렬 곱을 네 가지 방식으로 비교하고, `float64`로 구한 답과의 오차도 쟀어요.

```
import statistics
import time

import torch


def 재기(함수, 반복: int = 20) -> float:
    """준비 운동 뒤 동기화하며 재고 중앙값(ms)을 돌려준다."""
    for _ in range(3):
        함수()
    torch.cuda.synchronize()
    시간들 = []
    for _ in range(반복):
        시작 = time.perf_counter()
        함수()
        torch.cuda.synchronize()
        시간들.append((time.perf_counter() - 시작) * 1000)
    return statistics.median(시간들)


n = 8192
torch.manual_seed(0)
```

```
a64 = torch.randn(n, n, device="cuda", dtype=torch.float64)
b64 = torch.randn(n, n, device="cuda", dtype=torch.float64)
정답 = a64 @ b64                                        # float64로 구한 기준 답

for 이름, 자료형, 정밀도 in [("float32 (highest)", torch.float32, "highest"),
                            ("float32 + TF32 (high)", torch.float32, "high"),
                            ("bfloat16", torch.bfloat16, "highest"),
                            ("float16", torch.float16, "highest")]:
    torch.set_float32_matmul_precision(정밀도)          # float32 행렬 곱에 TF32를 쓸지
    a, b = a64.to(자료형), b64.to(자료형)
    ms = 재기(lambda: a @ b)
    오차 = ((a @ b).double() - 정답).norm() / 정답.norm()
    print(f"{이름:<22} {ms:6.2f} ms ¦ {2 * n ** 3 / ms / 1e9:5.1f} TFLOPS ¦ 상대 오차 {오
    ↪ 차.item():.1e}")
torch.set_float32_matmul_precision("highest")           # 기본값으로 되돌리기
```

```
float32 (highest)       20.22 ms ¦  54.4 TFLOPS ¦ 상대 오차 1.6e-06
float32 + TF32 (high)   13.00 ms ¦  84.6 TFLOPS ¦ 상대 오차 2.9e-04
bfloat16                 6.63 ms ¦ 165.8 TFLOPS ¦ 상대 오차 3.1e-03
float16                  6.76 ms ¦ 162.6 TFLOPS ¦ 상대 오차 3.6e-04
```

- **`float32 (highest)`**: PyTorch의 기본값입니다. 20.2ms, 54 TFLOPS이고 오차는 1.6×10^{-6}으로 가장 정확해요.
- **`float32 + TF32 (high)`**: 한 줄 바꿨을 뿐인데 13.0ms로 1.6배 빨라졌습니다. 오차는 2.9×10^{-4}로 커졌지만, 딥러닝 학습에는 충분한 정밀도예요.
- **`bfloat16`, `float16`**: 6.6~6.8ms, 160 TFLOPS를 넘습니다. `float32`보다 3배 빨라요. 정밀도는 `float16`(3.6×10^{-4})이 `bfloat16`(3.1×10^{-3})보다 열 배쯤 좋습니다. 16.6절에서 본 "촘촘한 자와 성긴 자"의 차이가 그대로 나타났죠.

그래서 NVIDIA GPU로 학습하는 코드 맨 위에 다음 한 줄을 넣는 경우가 많습니다.

```
torch.set_float32_matmul_precision("high")   # float32 행렬 곱에 TF32 허용
```

TF32는 계산 능력 8.0 이상의 NVIDIA GPU에서만 효과가 있고, autocast로 이미 16비트로 계산하는 부분에는 영향이 없습니다. autocast가 32비트로 남겨 둔 행렬 곱이나 autocast를 쓰지 않는 코드가 빨라져요. 정밀도가 정말 중요한 과학 계산이라면 켜지 않는 것이 맞고, 켜고 끄면 결과의 끝자

리가 달라진다는 점도 기억해 두세요(8장 8.10절).

16.8 torch.compile: 모델을 미리 번역해 두기

첫 실행은 느리고, 그다음부터 빠르다

지금까지 PyTorch는 코드를 **한 줄씩 읽고 바로 실행**했어요. 이것을 **즉시 실행(eager) 모드**라고 하고, 중간에 `print`를 넣어 들여다보기 쉬워서 배우고 실험하기에 좋습니다.

그런데 레시피를 한 줄 읽고 재료 하나 넣는 요리사는 빠르지 않습니다. 레시피 전체를 미리 읽으면 "이 두 소스는 섞어서 한 번에 끓이기"처럼 순서를 다듬을 수 있죠. `torch.compile`이 그 일을 합니다. 처음 실행할 때 계산 전체를 **그래프**로 따 두고, 합칠 수 있는 계산을 합쳐 GPU용 코드를 새로 만들어요. 16.6절의 작은 GPT로 재 보겠습니다.

```
import statistics
import time

import torch
import torch.nn.functional as F
from torch import nn

어휘, 길이, 배치 = 1000, 256, 32


class 작은_GPT(nn.Module):
    """16.6절과 같은 작은 GPT."""

    def __init__(self, 차원: int = 512, 층: int = 6, 헤드: int = 8) -> None:
        super().__init__()
        self.글자 = nn.Embedding(어휘, 차원)
        self.위치 = nn.Embedding(길이, 차원)
        블록 = nn.TransformerEncoderLayer(차원, 헤드, 4 * 차원, dropout=0.0,
                                          batch_first=True, norm_first=True)
        self.몸통 = nn.TransformerEncoder(블록, 층, enable_nested_tensor=False)
        self.머리 = nn.Linear(차원, 어휘)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```
        T = x.shape[1]
        h = self.글자(x) + self.위치(torch.arange(T, device=x.device))
        마스크 = nn.Transformer.generate_square_subsequent_mask(T, device=x.device)
        return self.머리(self.몸통(h, mask=마스크, is_causal=True))


def 걸음_시간들(모델: nn.Module, 걸음_수: int = 60) -> list[float]:
    """bfloat16 autocast로 학습 걸음을 밟으며 걸음마다 걸린 시간(ms)을 모은다."""
    옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=3e-4)
    생성기 = torch.Generator(device="cuda").manual_seed(0)
    시간들 = []
    for _ in range(걸음_수):
        시작 = time.perf_counter()
        데이터 = torch.randint(0, 어휘, (배치, 길이 + 1), device="cuda", generator=생성기)
        with torch.autocast("cuda", dtype=torch.bfloat16):
            손실 = F.cross_entropy(모델(데이터[:, :-1]).flatten(0, 1), 데이터[:, 1:].flatten())
        옵티마이저.zero_grad(set_to_none=True)
        손실.backward()
        옵티마이저.step()
        torch.cuda.synchronize()
        시간들.append((time.perf_counter() - 시작) * 1000)
    return 시간들


torch.manual_seed(0)
보통 = 걸음_시간들(작은_GPT().to("cuda"))
torch.manual_seed(0)
컴파일 = 걸음_시간들(torch.compile(작은_GPT().to("cuda")))   # 모델을 감싸기만 하면 된다

print(f"보통(eager) : 첫 걸음 {보통[0] / 1000:5.1f}초, 이후 걸음당 {statistics.median(보통
↪ [10:]):5.1f} ms")
print(f"compile     : 첫 걸음 {컴파일[0] / 1000:5.1f}초, 이후 걸음당 {statistics.median(컴파일
↪ [10:]):5.1f} ms")
print(f"걸음당 {statistics.median(보통[10:]) / statistics.median(컴파일[10:]):.2f}배 빨라짐")
```

```
보통(eager) : 첫 걸음   0.2초, 이후 걸음당  14.2 ms
compile     : 첫 걸음   6.6초, 이후 걸음당  12.5 ms
걸음당 1.14배 빨라짐
```

쓰는 법은 정말 간단해서, 모델을 `torch.compile()`로 감싸기만 했습니다. 결과를 보면 두 가지가 눈에 띕니다.

- **첫 걸음이 6.6초**나 걸렸습니다. 보통 실행은 0.2초였어요. 이 시간 동안 모델을 그래프로 따고, 계산을 합치고, GPU 코드를 만들어 컴파일했습니다. 이 시간을 **컴파일 시간**이라고 해요.
- **그다음부터는 걸음당 14.2ms → 12.5ms로 1.14배 빨라졌습니다.**

1.14배는 생각보다 작죠? 이 모델은 시간 대부분이 큰 행렬 곱이고, 행렬 곱은 이미 cuBLAS가 가장 빠른 방법으로 하고 있어서 합칠 것이 많지 않았기 때문이에요. `nn.TransformerEncoder`는 어텐션에도 이미 빠른 함수(16.9절)를 쓰고 있고요. 그럼 `torch.compile`은 어디서 빛날까요?

계산 합치기의 힘

16.1절에서 계산 한 줄이 커널 하나라고 했습니다. 원소별 계산(곱하기, 더하기, tanh 같은)이 여러 줄 이어지면, 커널마다 **메모리에서 숫자를 읽어 와 계산하고 다시 메모리에 써 둡니다.** 숫자 6,700만 개짜리 텐서라면 한 줄마다 256MB를 읽고 256MB를 쓰는 거예요. 정작 계산은 금방 끝나는데 읽고 쓰는 데 시간이 다 갑니다. 이 계산들을 **커널 하나로 합치면** 한 번 읽어서 모든 계산을 다 하고 한 번만 쓰면 되죠. 이것을 **커널 융합(kernel fusion)** 이라고 합니다. GELU 활성화 함수의 근사식을 손으로 써서 확인해 보겠습니다.

```
import statistics
import time

import torch


def 재기(함수, 반복: int = 30) -> float:
    """준비 운동 뒤 동기화하며 재고 중앙값(ms)을 돌려준다."""
    for _ in range(3):
        함수()
    torch.cuda.synchronize()
    시간들 = []
```

```
    for _ in range(반복):
        시작 = time.perf_counter()
        함수()
        torch.cuda.synchronize()
        시간들.append((time.perf_counter() - 시작) * 1000)
    return statistics.median(시간들)


def 손으로_쓴_GELU(x: torch.Tensor) -> torch.Tensor:
    """GELU 근사식: 곱하기·더하기·tanh가 여러 번 이어지는 원소별 계산."""
    return 0.5 * x * (1 + torch.tanh(0.7978845608 * (x + 0.044715 * x ** 3)))


x = torch.randn(64 * 1024 * 1024, device="cuda")       # 숫자 6,700만 개 (256MB)
합친_GELU = torch.compile(손으로_쓴_GELU)
print("결과 같음:", torch.allclose(손으로_쓴_GELU(x), 합친_GELU(x), atol=1e-6))

보통 = 재기(lambda: 손으로_쓴_GELU(x))
합침 = 재기(lambda: 합친_GELU(x))
내장 = 재기(lambda: torch.nn.functional.gelu(x, approximate="tanh"))
print(f"보통(eager)   : {보통:6.2f} ms")
print(f"torch.compile : {합침:6.2f} ms ({보통 / 합침:.1f}배)")
print(f"내장 F.gelu   : {내장:6.2f} ms")
대역폭 = 1008e9                                          # RTX 4090 메모리 대역폭 사양: 초당 1008GB
print(f"256MB를 한 번 읽고 한 번 쓰는 최소 시간: {2 * x.numel() * 4 / 대역폭 * 1000:.2f} ms")
```

```
결과 같음: True
보통(eager)   :   5.25 ms
torch.compile :   0.60 ms (8.8배)
내장 F.gelu   :   0.59 ms
256MB를 한 번 읽고 한 번 쓰는 최소 시간: 0.53 ms
```

손으로 쓴 GELU는 5.25ms, `torch.compile`로 합친 것은 0.60ms로 **8.8배 빨라졌습니다.** 결과도 같아요. 마지막 줄은 이 GPU의 메모리 대역폭(제조사 사양 1,008GB/s)으로 256MB를 한 번 읽고 한 번 쓰는 데 필요한 최소 시간인데, 0.53ms입니다. 합친 버전(0.60ms)이 이 한계에 거의 붙었어요. 원소별 계산은 계산보다 **메모리를 읽고 쓰는 시간이 병목**이라, 읽고 쓰는 횟수를 줄이는 것이 곧 속도

입니다. PyTorch에 들어 있는 `F.gelu`(0.59ms)도 사람이 미리 하나의 커널로 짜 둔 것이라 같은 속도가 나왔어요.

그러니 `torch.compile`은 **원소별 계산이 많은 모델**, 그리고 내장 함수 대신 직접 쓴 수식이 많은 모델에서 효과가 큽니다. 연구 코드에서 새로운 활성화 함수나 정규화를 직접 만들었다면 `torch.compile` 한 줄로 내장 함수 수준의 속도를 얻을 수 있다는 뜻이에요.

모드: 일감 맡기는 비용까지 줄이기

`torch.compile`에는 `mode=`로 고를 수 있는 방식이 몇 가지 있습니다. 16.1절의 "작은 일 천 번"처럼 일감 맡기는 비용이 큰 경우를 보기 위해, 작은 층 24개(Linear, LayerNorm, GELU 8번 반복)에 작은 배치 16개를 넣어 비교했어요.

```
import statistics
import time

import torch
from torch import nn


def 재기(함수, 반복: int = 200) -> float:
    """준비 운동 뒤 동기화하며 재고 중앙값(ms)을 돌려준다."""
    for _ in range(10):
        함수()
    torch.cuda.synchronize()
    시간들 = []
    for _ in range(반복):
        시작 = time.perf_counter()
        함수()
        torch.cuda.synchronize()
        시간들.append((time.perf_counter() - 시작) * 1000)
    return statistics.median(시간들)


# 작은 모델, 작은 배치: 계산보다 "GPU에 일을 맡기는 비용"이 큰 경우
torch.manual_seed(0)
층들 = []
```

```
for _ in range(8):
    층들 += [nn.Linear(256, 256), nn.LayerNorm(256), nn.GELU()]
모델 = nn.Sequential(*층들).to("cuda").eval()
x = torch.randn(16, 256, device="cuda")

for 모드 in [None, "default", "reduce-overhead"]:
    torch.compiler.reset()                          # 앞에서 컴파일한 것을 지운다
    실행 = 모델 if 모드 is None else torch.compile(모델, mode=모드)
    with torch.no_grad():
        시작 = time.perf_counter()
        실행(x)
        torch.cuda.synchronize()
        준비 = time.perf_counter() - 시작
        ms = 재기(lambda: 실행(x))
    print(f"{모드 or 'eager':<16} 첫 호출 {준비:5.2f}초 ¦ 그다음부터 한 번에 {ms:6.3f} ms")
```

```
eager            첫 호출  0.04초 ¦ 그다음부터 한 번에  0.116 ms
default          첫 호출  0.83초 ¦ 그다음부터 한 번에  0.081 ms
reduce-overhead  첫 호출  0.08초 ¦ 그다음부터 한 번에  0.067 ms
```

- eager: 0.115ms. 커널 수십 개를 하나씩 맡기는 비용이 대부분입니다.
- `default`: 기본 모드입니다. 커널을 합쳐서 수를 줄이니 0.080ms가 됐어요.
- `reduce-overhead`: 0.068ms로 가장 빠릅니다. **CUDA 그래프**라는 기능으로, GPU에 맡길 일감 전체를 한 번 녹화해 두었다가 다음부터는 "녹화한 것 다시 재생"이라는 일감 하나만 맡겨요. 일감 맡기는 비용이 사실상 사라집니다. 작은 모델의 추론이나 작은 배치 학습에 효과가 크지만, 입력 모양이 바뀌면 다시 녹화해야 하고 메모리를 조금 더 씁니다.

가장 빠른 계산 방법을 직접 실험해서 고르는 `max-autotune` 모드도 있지만, 컴파일에 몇 분씩 걸릴 수 있어 이 장에서는 재지 않았습니다.

첫 호출 시간을 보면 `default`가 0.83초, `reduce-overhead`가 0.08초였습니다. `reduce-overhead`가 더 빨리 준비된 것은, 앞에서 같은 모델을 컴파일하며 만든 결과물 일부를 PyTorch가 디스크에 **캐시**해 두었다가 다시 쓸 수 있었기 때문으로 보여요. PyTorch는 컴파일 결과를 디스크에 저장해 두기 때문에, 같은 모델을 다시 실행할 때는 컴파일 시간이 크게 줄어듭니다.

그래프가 끊기는 곳

`torch.compile`은 파이썬 코드를 읽어서 그래프로 바꾸는데, 그래프로 바꿀 수 없는 코드를 만나면 거기서 그래프를 **끊고** 그 부분은 보통 파이썬으로 실행한 뒤 다시 그래프를 시작합니다. 이것을 **그래프 끊김**(graph break) 이라고 해요. 끊긴 곳이 많으면 합칠 기회가 줄어서 빨라지지 않습니다. 어디서 끊기는지 `torch._dynamo.explain`으로 들여다볼 수 있어요. 이름에 밑줄이 붙은 `_dynamo`는 PyTorch 내부 도구라 버전에 따라 바뀔 수 있습니다. 앞에서 컴파일한 것을 지우는 일은 공개된 `torch.compiler.reset()`으로 했어요.

```
import torch


def 깐깐한_함수(x: torch.Tensor) -> torch.Tensor:
    y = x.sin() * 2
    print("중간 점검")                # 파이썬의 부수 효과
    if y.sum() > 0:                  # 텐서 값을 보고 갈림길을 고른다
        return y.cos()
    return y + 1


설명 = torch._dynamo.explain(깐깐한_함수)(torch.randn(8))     # 내부 도구(_dynamo)라 버전마다 바뀔
↪ 수 있다
print("그래프 수:", 설명.graph_count, "¦ 끊긴 곳:", 설명.graph_break_count)
for 이유 in 설명.break_reasons:
    print(" -", 이유.reason.splitlines()[0][:70])

torch.compiler.reset()
try:
    torch.compile(깐깐한_함수, fullgraph=True)(torch.randn(8))   # 끊기면 오류를 내라
except Exception as 오류:
    print("fullgraph=True:", type(오류).__name__)
```

```
중간 점검
그래프 수: 3 ¦ 끊긴 곳: 2
 - Failed to trace builtin operator
 - generic_jump TensorVariable()
```

```
fullgraph=True: Unsupported
```

작은 함수 하나가 그래프 3개로 쪼개졌고 끊긴 곳이 2군데입니다. 이유도 알려 줘요.

- `Failed to trace builtin operator`: `print`처럼 화면에 무언가를 하는 파이썬 내장 함수는 그래프에 담을 수 없습니다. 그래프는 "텐서 계산의 흐름"만 담는 그림이니까요.
- `generic_jump TensorVariable()`: `if y.sum() > 0:`처럼 **텐서의 값**을 보고 갈림길을 고르는 코드입니다. 그래프를 따는 시점에는 값을 모르니 어느 길로 갈지 정할 수가 없어요.

`fullgraph=True`를 주면 그래프가 끊길 때 조용히 넘어가지 않고 오류(`Unsupported`)를 냅니다. 모델 전체가 그래프 하나로 따지는지 확인하고 싶을 때 써요. 그래프 끊김을 줄이려면 학습 걸음 안에서 `print`나 `.item()`을 빼고, 텐서 값에 따른 `if`는 `torch.where`(4장 4.4절)처럼 텐서 계산으로 바꾸면 됩니다. 실무에서는 우선 모델 전체를 `torch.compile`로 감싸 보고, 기대만큼 빨라지지 않을 때 `explain`으로 끊긴 곳을 찾는 순서가 좋아요.

입력 모양이 바뀌어도 다시 컴파일이 일어납니다. 7장 7.7절처럼 문장 길이가 배치마다 다르면 처음 몇 번은 컴파일하느라 느려질 수 있으니, 길이를 몇 가지로 맞춰 채우는 것이 좋아요.

> **Q. 그럼 언제나 torch.compile을 켜는 게 좋나요?** 오래 돌릴 학습이라면 대개 켜는 것이 이득입니다. 몇 초에서 몇십 초의 컴파일 시간은 수천 걸음에 나눠지니까요. 반대로 코드를 자주 고치며 실험하는 단계, 몇 걸음만 돌려 보는 디버깅, 모델 안을 `print`로 들여다볼 때는 끄는 편이 편합니다. 오류 메시지도 eager 모드 쪽이 훨씬 읽기 쉬워요. "먼저 eager로 맞게 만들고, 다 되면 compile로 빠르게"가 좋은 순서입니다.

16.9 어텐션은 scaled_dot_product_attention으로

12장 12.5절에서 `F.scaled_dot_product_attention`(줄여서 SDPA)이 직접 쓴 어텐션과 같은 답을 낸다는 것을 확인하고, "장치에 따라 훨씬 빠르고 메모리를 덜 쓰는 계산법(예: FlashAttention)을 알아서 골라 준다, 실제로 학습시킬 때는 이 함수로 바꾸길 권한다(16장)"고 했습니다. 12.9절에서는 어텐션 점수 표가 글자 수의 제곱으로 커지는 것을 계산으로 봤고요. 이제 실제로 얼마나 다른지 재 보겠습니다.

SDPA 안에는 계산법(**백엔드**)이 여러 개 들어 있습니다. `sdpa_kernel()`로 하나를 골라 쓰게 할 수 있어요.

- **수학(MATH)**: 12장에서 직접 쓴 것과 같은 방법을 PyTorch 코드로 한 것. 어디서나 돌아가

는 기본 방법입니다.

- **메모리 효율(EFFICIENT_ATTENTION)**: 점수 표를 통째로 만들지 않고 조각조각 계산하는 방법입니다.
- **플래시(FLASH_ATTENTION)**: 같은 생각을 GPU 구조에 맞춰 더 다듬은 FlashAttention 방법입니다. 16비트 입력이 필요하고, 계산 능력 8.0 이상의 GPU에서만 돌아요. 그래서 아래 코드는 Colab의 T4처럼 그보다 오래된 GPU에서는 플래시를 빼고 잽니다.

```
import math
import statistics
import time

import torch
import torch.nn.functional as F
from torch.nn.attention import SDPBackend, sdpa_kernel


def 재기(함수, 반복: int = 10) -> float:
    """준비 운동 뒤 동기화하며 재고 중앙값(ms)을 돌려준다."""
    함수()
    torch.cuda.synchronize()
    시간들 = []
    for _ in range(반복):
        시작 = time.perf_counter()
        함수()
        torch.cuda.synchronize()
        시간들.append((time.perf_counter() - 시작) * 1000)
    return statistics.median(시간들)


def 직접_어텐션(q: torch.Tensor, k: torch.Tensor, v: torch.Tensor) -> torch.Tensor:
    """12장에서 직접 쓴 인과 어텐션: 점수 표를 통째로 만든다."""
    점수 = q @ k.transpose(-2, -1) / math.sqrt(q.shape[-1])
    T = q.shape[-2]
    미래 = torch.ones(T, T, dtype=torch.bool, device=q.device).triu(1)
    return 점수.masked_fill(미래, float("-inf")).softmax(dim=-1) @ v
```

```
def SDPA(백엔드: SDPBackend):
    """계산법(백엔드)을 하나로 정해서 scaled_dot_product_attention을 부르는 함수."""
    def 실행(q, k, v):
        with sdpa_kernel(백엔드):
            return F.scaled_dot_product_attention(q, k, v, is_causal=True)
    return 실행


방법들 = {"직접 계산": 직접_어텐션, "SDPA 수학": SDPA(SDPBackend.MATH),
          "SDPA 메모리 효율": SDPA(SDPBackend.EFFICIENT_ATTENTION),
          "SDPA 플래시": SDPA(SDPBackend.FLASH_ATTENTION)}
if torch.cuda.get_device_capability() < (8, 0):     # 플래시는 계산 능력 8.0 이상에서만 돈다
    del 방법들["SDPA 플래시"]                         # (Colab의 T4 같은 GPU에서는 빼고 잰다)
GB = 1024 ** 3
torch.manual_seed(0)
for T in [1024, 4096]:
    q, k, v = (torch.randn(8, 16, T, 64, device="cuda", dtype=torch.float16) for _ in range(3))
    기준 = 직접_어텐션(q, k, v)
    print(f"[글자 {T}개] 배치 8, 헤드 16, 헤드 크기 64, float16")
    for 이름, 함수 in 방법들.items():
        torch.cuda.reset_peak_memory_stats()
        전 = torch.cuda.memory_allocated()
        ms = 재기(lambda: 함수(q, k, v))
        추가 = (torch.cuda.max_memory_allocated() - 전) / GB
        같은가 = torch.allclose(함수(q, k, v), 기준, atol=1e-2)
        print(f"  {ms:7.2f} ms ¦ 추가 메모리 {추가:5.2f} GB ¦ 결과 같음 {같은가} ← {이름}")
```

```
[글자 1024개] 배치 8, 헤드 16, 헤드 크기 64, float16
     3.07 ms ¦ 추가 메모리  0.75 GB ¦ 결과 같음 True ← 직접 계산
     7.05 ms ¦ 추가 메모리  1.25 GB ¦ 결과 같음 True ← SDPA 수학
     0.19 ms ¦ 추가 메모리  0.02 GB ¦ 결과 같음 True ← SDPA 메모리 효율
     0.17 ms ¦ 추가 메모리  0.02 GB ¦ 결과 같음 True ← SDPA 플래시
[글자 4096개] 배치 8, 헤드 16, 헤드 크기 64, float16
    48.13 ms ¦ 추가 메모리 12.02 GB ¦ 결과 같음 True ← 직접 계산
   114.60 ms ¦ 추가 메모리 18.56 GB ¦ 결과 같음 True ← SDPA 수학
     2.28 ms ¦ 추가 메모리  0.06 GB ¦ 결과 같음 True ← SDPA 메모리 효율
```

```
1.91 ms ¦ 추가 메모리  0.06 GB ¦ 결과 같음 True ← SDPA 플래시
```

글자 1,024개에서도 차이가 큽니다. 직접 계산은 3.07ms에 메모리 0.75GB를 더 썼는데, 플래시는 0.17ms에 0.02GB로 **18배 빠르고 메모리는 수십 분의 일**이에요. 글자 4,096개가 되자 차이가 더 벌어집니다. 직접 계산은 48ms에 메모리를 12GB나 썼고, 플래시는 1.91ms에 0.06GB입니다. 25배 빠르고, 메모리는 200분의 1이에요. 네 방법 모두 결과는 같았습니다.

12.9절의 계산을 떠올려 보세요. 배치 8 × 헤드 16 × 4,096 × 4,096칸 × 2바이트면 점수 표 하나가 4GB입니다. 직접 계산은 점수 표, 마스크를 씌운 표, 소프트맥스를 거친 표를 차례로 만드니 12GB가 들었어요. 플래시와 메모리 효율 방법은 이 표를 **한 번도 통째로 만들지 않습니다.** 작은 조각을 GPU 안의 아주 빠른 메모리에 올려 계산을 끝내고 결과만 내보내요. 16.8절에서 본 "메모리를 읽고 쓰는 횟수가 곧 속도"라는 원리가 여기서도 그대로입니다.

`SDPA 수학`은 직접 계산보다도 느리고 메모리를 더 썼습니다(4,096글자에서 18.6GB). 수학 백엔드는 정확도를 지키려고 `float16` 입력을 안에서 `float32`로 올려 계산하기 때문이에요(`torch.backends.cuda.fp16_bf16_reduction_math_sdp_allowed()`가 기본값 `False`). 점수 표 한 칸이 2바이트가 아니라 4바이트가 되니 메모리도 시간도 더 듭니다. 속도보다 어디서나 정확히 돌아가는 것을 우선한, 다른 방법을 쓸 수 없을 때를 위한 대비책이에요. 백엔드를 고르지 않으면 SDPA는 입력의 모양, 자료형, 장치를 보고 쓸 수 있는 것 중 가장 빠른 것을 알아서 고르니, 보통은 `sdpa_kernel` 없이 그냥 부르면 됩니다. 이 표의 교훈은 "어텐션을 직접 쓰지 말고 SDPA를 부르자"예요. 16.6절의 `nn.TransformerEncoder`도 안에서 SDPA를 씁니다.

Q

Q. FlashAttention을 쓰려면 따로 설치해야 하나요? 이 장의 결과는 PyTorch만 설치해서 얻은 것입니다. PyTorch에 FlashAttention 계열의 계산법이 들어 있어서 SDPA가 알아서 골라 써요. 다만 플래시 방법은 지원하는 GPU 세대, 자료형(16비트), 헤드 크기에 조건이 있어서, 조건이 안 맞으면 SDPA가 다른 방법을 고릅니다. 최신 버전의 FlashAttention을 쓰려고 별도 패키지를 설치하는 경우도 있지만, 대부분은 SDPA로 충분합니다.

16.10 프로파일러로 시간 새는 곳 찾기

어디서 시간이 드는지 표로 보기

지금까지는 “이걸 바꾸면 빨라질까?”를 하나씩 재 봤습니다. 그런데 처음 보는 학습 코드가 느리다면 어디부터 고쳐야 할까요? 짐작으로 고치기 전에 **프로파일러(profiler)** 로 시간이 어디서 드는지부터 봐야 합니다. 건강 검진을 받고 나서 처방을 내리는 것과 같아요. ResNet18 학습 다섯 걸음을 `torch.profiler`로 기록해 보겠습니다.

```
import torch
import torch.nn.functional as F
from torch.autograd import DeviceType
from torch.profiler import ProfilerActivity, profile, record_function
from torchvision.models import resnet18

torch.manual_seed(0)
모델 = resnet18(num_classes=10).to("cuda")
옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.01)
x = torch.randn(64, 3, 224, 224, device="cuda")
y = torch.randint(0, 10, (64,), device="cuda")


def 한_걸음() -> None:
    with record_function("순전파"):                    # 프로파일 기록에 이름표를 붙인다
        손실 = F.cross_entropy(모델(x), y)
    with record_function("역전파"):
        옵티마이저.zero_grad(set_to_none=True)
        손실.backward()
    with record_function("갱신"):
        옵티마이저.step()


for _ in range(3):                                     # 준비 운동은 기록하지 않는다
    한_걸음()
torch.cuda.synchronize()

with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA]) as 기록:
    for _ in range(5):
        한_걸음()
```

```
    torch.cuda.synchronize()

이름표 = ("순전파", "역전파", "갱신")
# PyTorch 연산(aten::...)마다, 그 연산이 GPU에 맡긴 일의 시간을 모은다 (이름표 구간은 따로)
통계 = [e for e in 기록.key_averages() if e.device_type == DeviceType.CPU and e.key not in 이름표]
GPU_전체 = sum(e.self_device_time_total for e in 통계) / 1000
print(f"5걸음 동안 GPU가 일한 시간: {GPU_전체:.1f} ms")
for e in sorted(통계, key=lambda e: e.self_device_time_total, reverse=True)[:8]:
    ms = e.self_device_time_total / 1000
    print(f"{ms:7.2f} ms {ms / GPU_전체:6.1%} {e.count:4}번  {e.key[:48]}")
for e in 기록.key_averages():
    if e.key in 이름표 and e.device_type == DeviceType.CPU:    # 파이썬 쪽에서 걸린 시간
        print(f"이름표 '{e.key}': CPU에서 {e.cpu_time_total / 1000:5.1f} ms")
```

```
5걸음 동안 GPU가 일한 시간: 125.8 ms
  55.98 ms  44.5%  100번  aten::convolution_backward
  23.27 ms  18.5%  100번  aten::cudnn_convolution
  15.85 ms  12.6%  100번  aten::cudnn_batch_norm_backward
   7.83 ms   6.2%   85번  aten::threshold_backward
   7.53 ms   6.0%  100번  aten::cudnn_batch_norm
   5.19 ms   4.1%  180번  aten::add_
   3.63 ms   2.9%   85번  aten::clamp_min_
   2.76 ms   2.2%    5번  aten::max_pool2d_with_indices_backward
이름표 '순전파': CPU에서  11.1 ms
이름표 '역전파': CPU에서  43.7 ms
이름표 '갱신': CPU에서   0.5 ms
```

`with profile(...) as 기록:` 블록 안에서 일어난 일이 모두 기록됩니다. `activities`에 `CPU`와 `CUDA`를 주면 파이썬 쪽 PyTorch 연산과 GPU 쪽 커널을 함께 기록해요. `기록.key_averages()`는 같은 이름끼리 묶어서 시간을 합친 표인데, 여기서는 PyTorch 연산(`aten::`으로 시작하는 이름)마다 **그 연산이 GPU에 맡긴 일의 시간**을 골라 큰 순서로 찍었습니다. `aten`은 PyTorch 안쪽의 C++ 연산 모음 이름이에요.

5걸음 동안 GPU가 일한 125.8ms 중 **합성곱의 역전파(`convolution_backward`)가 44.5%, 순전파(`cudnn_convolution`)가 18.5%**, 합성곱 둘만 63%입니다. 그다음이 배치 정규화(12.6% + 6.0%)이고, ReLU의 역전파(`threshold_backward`)와 순전파(`clamp_min_`), 잔차 연결의 덧셈(`add_`)이 뒤를

잇습니다. 그러니 이 모델은 합성곱을 빠르게 하는 방법(16비트로 텐서 코어 쓰기)이 가장 효과가 크고, ReLU를 바꾸는 고민은 많아야 몇 %의 이득이에요.

`record_function("순전파")`처럼 내 코드의 구간에 이름표를 붙일 수도 있습니다. 마지막 세 줄은 이름표마다 **CPU에서 걸린 시간**인데, 순전파가 11.1ms, 역전파가 43.7ms, 갱신이 0.5ms예요. GPU가 125ms 동안 일하는 사이에 파이썬은 훨씬 짧은 시간 만에 일감을 다 맡겼다는 뜻입니다. 16.2절의 비동기 실행을 프로파일러로 다시 확인한 셈이에요. CPU 시간이 GPU 시간보다 길어지면 그때는 일감 맡기는 쪽이 병목이라는 신호입니다.

`기록.key_averages().table(...)`은 PyTorch가 만들어 주는 자세한 표를, `기록.export_chrome_trace("기록.json")`은 웹 브라우저의 추적 도구(Perfetto 등)에서 CPU와 GPU의 시간 줄을 그림으로 볼 수 있는 파일을 만들어 줍니다. GPU 줄에 빈틈이 많으면 GPU가 놀고 있다는 뜻이에요.

데이터 로더 병목 잡기

7장 7.10절에서 일꾼 수에 따라 데이터 대기 시간이 어떻게 바뀌는지 `time.sleep`으로 흉내 냈고, "GPU가 일을 제대로 하는지 들여다보는 프로파일러는 16장에서"라고 했습니다. 이번에는 진짜 GPU와 진짜 데이터로 확인해요. 9장의 FashionMNIST(7만 장, 처음 한 번 `data/` 폴더에 내려받음)에 7장 7.6절의 증강을 걸고, 작은 CNN을 100걸음 학습하면서 **벽시계 시간**과 **그동안 GPU가 실제로 일한 시간**을 비교합니다. 둘을 나누면 GPU가 시간의 몇 %를 일했는지 알 수 있어요.

```
import os
import time

import torch
import torch.nn.functional as F
from torch import nn
from torch.autograd import DeviceType
from torch.profiler import ProfilerActivity, profile
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import v2

데이터_폴더 = os.environ.get("BOOK_DATA_DIR", "data")


def 작은_CNN() -> nn.Sequential:
```

```
    return nn.Sequential(
        nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        nn.Flatten(), nn.Linear(64 * 7 * 7, 10))


def 재기(일꾼: int, 걸음_수: int = 100) -> tuple[float, float]:
    """걸음_수만큼 학습하며 (벽시계 시간, 그동안 GPU가 실제로 일한 시간)을 초로 잰다."""
    증강 = v2.Compose([v2.ToImage(), v2.RandomCrop(28, padding=2), v2.RandomHorizontalFlip(),
                       v2.ToDtype(torch.float32, scale=True)])
    데이터 = datasets.FashionMNIST(데이터_폴더, train=True, download=True, transform=증강)
    로더 = DataLoader(데이터, batch_size=256, shuffle=True, num_workers=일꾼,
                      pin_memory=True, persistent_workers=일꾼 > 0)
    모델 = 작은_CNN().to("cuda")
    옵티마이저 = torch.optim.Adam(모델.parameters())
    반복자 = iter(로더)
    next(반복자)                                          # 일꾼을 띄우는 시간은 빼고 잰다
    with profile(activities=[ProfilerActivity.CUDA]) as 기록:
        시작 = time.perf_counter()
        for _ in range(걸음_수):
            x, y = next(반복자)
            x, y = x.to("cuda", non_blocking=True), y.to("cuda", non_blocking=True)
            손실 = F.cross_entropy(모델(x), y)
            옵티마이저.zero_grad(set_to_none=True)
            손실.backward()
            옵티마이저.step()
        torch.cuda.synchronize()
        전체 = time.perf_counter() - 시작
    GPU_시간 = sum(e.self_device_time_total for e in 기록.key_averages()
                   if e.device_type == DeviceType.CUDA) / 1e6        # GPU에서 실제로 돈 시간
    return 전체, GPU_시간


if __name__ == "__main__":
    for 일꾼 in [0, 8]:
        전체, GPU = 재기(일꾼)
```

```
        print(f"num_workers={일꾼}: 100걸음 {전체:5.2f}초, GPU가 일한 시간 {GPU:4.2f}초 "
              f"→ GPU 사용률 {GPU / 전체:5.1%}")
```

```
num_workers=0: 100걸음  2.49초, GPU가 일한 시간 0.09초 → GPU 사용률  3.4%
num_workers=8: 100걸음  0.31초, GPU가 일한 시간 0.09초 → GPU 사용률 27.9%
```

일꾼 없이(`num_workers=0`) 학습하면 100걸음에 2.49초가 걸리는데, GPU가 실제로 일한 시간은 0.09초뿐입니다. **GPU 사용률 3.4%**, 시간의 97%를 놀았어요. 사진 25,600장을 하나씩 꺼내 자르고 뒤집는 일을 파이썬 프로세스 하나가 다 하는 동안 GPU는 기다린 거죠. 일꾼 8명을 쓰자 같은 100걸음이 0.31초로 **8배 빨라졌습니다.** GPU가 한 일(0.09초)은 똑같아요. 기다리는 시간만 줄었습니다.

일꾼 8명으로도 GPU 사용률은 27.9%입니다. 남은 시간은 어디로 갔을까요? 이 CNN은 너무 작아서 걸음마다 GPU 일이 1ms도 안 되는데, 파이썬이 커널 수십 개를 맡기고 옵티마이저를 돌리는 데 그보다 긴 시간이 듭니다. 16.1절의 "작은 일" 문제예요. 이럴 때는 배치를 키우거나(다음 절), `torch.compile`로 커널 수를 줄이는 것이 다음 처방입니다.

`nvidia-smi`의 `GPU-Util`이 학습 중에 낮게 오르내릴 때도 데이터 쪽부터 의심하세요. 다만 이 숫자는 "커널이 하나라도 돈 시간의 비율"이라 100%여도 계산 장치를 다 쓴다는 뜻은 아니니, 정확히 보려면 프로파일러가 낫습니다.

16.11 배치 크기와 처리량

16.1절에서 "GPU에는 큰 일을 한꺼번에 맡겨야 한다"고 했습니다. 배치를 키우면 정말 계속 빨라질까요? ResNet18을 `bfloat16` autocast로 학습하면서 배치 크기에 따라 걸음당 시간과 **처리량**(throughput), 즉 초당 몇 장을 학습하는지를 쟀습니다.

```
import time

import torch
import torch.nn.functional as F
from torchvision.models import resnet18

GB = 1024 ** 3
```

```
torch.manual_seed(0)
모델 = resnet18(num_classes=10).to("cuda")
옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.01)

for 배치 in [8, 32, 128, 512]:
    x = torch.randn(배치, 3, 224, 224, device="cuda")
    y = torch.randint(0, 10, (배치,), device="cuda")
    torch.cuda.reset_peak_memory_stats()
    for 걸음 in range(13):
        if 걸음 == 3:                                  # 준비 운동 세 걸음 뒤부터 잰다
            torch.cuda.synchronize()
            시작 = time.perf_counter()
        with torch.autocast("cuda", dtype=torch.bfloat16):
            손실 = F.cross_entropy(모델(x), y)
        옵티마이저.zero_grad(set_to_none=True)
        손실.backward()
        옵티마이저.step()
    torch.cuda.synchronize()
    ms = (time.perf_counter() - 시작) / 10 * 1000
    print(f"배치 {배치:>3}: 걸음당 {ms:6.1f} ms ¦ 초당 사진 {배치 / ms * 1000:5.0f}장"
          f" ¦ 최고 메모리 {torch.cuda.max_memory_allocated() / GB:5.2f} GB")
    del x, y, 손실
```

```
배치   8: 걸음당    3.1 ms ¦ 초당 사진  2559장 ¦ 최고 메모리  0.22 GB
배치  32: 걸음당    8.4 ms ¦ 초당 사진  3818장 ¦ 최고 메모리  0.51 GB
배치 128: 걸음당   33.6 ms ¦ 초당 사진  3808장 ¦ 최고 메모리  1.63 GB
배치 512: 걸음당  141.0 ms ¦ 초당 사진  3632장 ¦ 최고 메모리  6.13 GB
```

배치 8에서 32로 키우자 처리량이 초당 2,559장에서 3,818장으로 1.5배 늘었습니다. 배치 8은 GPU를 다 채우기에 일이 작았던 거예요. 그런데 32에서 128, 512로 키워도 처리량은 3,800장 근처에서 더 늘지 않습니다. 걸음당 시간이 배치에 정확히 비례해서 늘었어요(8.4 → 33.6 → 141.0ms). **GPU가 이미 꽉 찼다**는 뜻입니다. 메모리만 0.51GB에서 6.13GB로 늘었죠.

그러니 배치 크기는 "들어가는 만큼 최대한"이 아니라 **처리량이 더 늘지 않는 지점** 근처가 적당합니다. 그보다 크게 해도 속도 이득은 없고 메모리만 쓰며, 무엇보다 학습 결과가 달라질 수 있어요. 배치를 키우면 한 에폭의 걸음 수가 줄어 가중치를 고칠 기회가 적어지고, 보통은 학습률도 함께 조정

해야 합니다. 배치 크기를 바꾸면 8장의 과적합 진단과 18장의 실험 기록으로 결과가 정말 같은지 확인하세요.

GPU가 꽉 차는 배치는 모델과 입력 크기마다 다르니, 새 모델은 이 절처럼 처리량부터 재 보세요.

Q

Q. 배치를 키울 때 학습률은 어떻게 바꾸나요? 정해진 공식은 없지만, 많이 쓰는 경험칙은 "배치를 k배로 키우면 학습률도 k배로"(SGD)입니다. Adam 계열은 그보다 덜 키우는 경우가 많고(예: √k배), 처음 몇백 걸음은 학습률을 천천히 올리는 워밍업(warm-up)을 함께 써요. 8장 8.7절의 스케줄러로 할 수 있습니다. 어느 쪽이든 검증 손실로 확인하는 것이 정답이에요.

16.12 성능 체크리스트

이 장에서 잰 것을 순서대로 정리하면, 느린 학습을 빠르게 만드는 체크리스트가 됩니다. 위에서부터 차례로 확인하세요. 위쪽일수록 고치기 쉽고 효과가 큰 것들이에요.

순서	확인할 것	방법	이 장
1	재는 방법이 맞는가	준비 운동, `synchronize`, 중앙값, `benchmark.Timer`	16.2
2	GPU가 놀고 있지 않은가	프로파일러로 GPU 사용률 보기, `num_workers`·`pin_memory`	16.10, 7.10
3	불필요한 왕복·동기화가 없는가	모델·데이터는 GPU에 두기, 매 걸음 `.item()` 줄이기, `non_blocking`	16.3, 16.5
4	16비트로 계산하는가	`autocast` + `bfloat16` (또는 `float16` + `GradScaler`)	16.6
5	`float32` 행렬 곱에 TF32를 쓰는가	`set_float32_matmul_precision("high")`	16.7

순서	확인할 것	방법	이 장
6	어텐션은 SDPA를 쓰는가	`F.scaled_dot_product_attention`	16.9
7	커널을 합쳤는가	`torch.compile`, 모드, 그래프 끊김 확인	16.8
8	배치 크기가 알맞은가	처리량이 더 늘지 않는 지점, 메모리 여유	16.11, 16.4
9	시간이 어디서 드는가	`torch.profiler`로 큰 순서 보기	16.10

이 밖에 합성곱 모델에서 `torch.backends.cudnn.benchmark = True`를 켜면 cuDNN이 입력 크기에 맞는 가장 빠른 합성곱 방법을 처음에 실험해서 고릅니다. 입력 크기가 일정한 이미지 학습에서 흔히 쓰는 설정인데, 8장 8.10절의 재현성과는 잘 맞지 않아요. 빠른 쪽을 고르는 실험이 실행마다 다른 방법을 고를 수 있어서, 결과의 끝자리가 달라질 수 있기 때문입니다.

한 가지 원칙을 덧붙이면, **한 번에 하나씩 바꾸고 매번 잽니다.** 여러 개를 한꺼번에 바꾸면 무엇이 효과가 있었는지, 무엇이 오히려 느리게 했는지 알 수 없어요. 그리고 속도를 고친 뒤에는 **정확도가 그대로인지** 꼭 확인합니다. 다음 실습이 바로 그 순서를 따릅니다.

16.13 실습: 옷 분류기 학습을 단계별로 빠르게

9장 9.10절에서 만든 작은 ResNet 옷 분류기를 다시 가져오겠습니다. FashionMNIST 6만 장으로 학습하고 1만 장으로 시험하며, 7장의 증강(상하좌우 2칸 안에서 밀기, 좌우 뒤집기)과 8장 8.7절의 OneCycleLR을 썼어요. 이 학습을 이 장에서 배운 방법으로 한 단계씩 빠르게 만듭니다.

1. ① **기본**: `float32`, 일꾼 없음. 지금까지 이 책에서 해 온 방식입니다.
2. ② **+ bfloat16 autocast**: 16.6절
3. ③ **+ torch.compile**: 16.8절
4. ④ **+ 데이터 로더 일꾼 8**: 16.10절과 7장 7.10절. `pin_memory`와 `persistent_workers`도 함께 켜고, `non_blocking`으로 옮깁니다.
5. ⑤ **④에서 autocast·compile 빼기**: 데이터 병목을 없앤 뒤 ②, ③이 얼마나 기여하는지 따로 보기 위한 비교입니다.

단계마다 4에폭씩 학습하고, 첫 에폭(준비 시간이 섞임)과 나머지 에폭의 평균 시간, 그리고 시험 정확도를 기록합니다.

```
import os
import time
from contextlib import nullcontext

import torch
import torch.nn.functional as F
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import v2

데이터_폴더 = os.environ.get("BOOK_DATA_DIR", "data")


class 잔차블록(nn.Module):
    """9장 9.8절의 잔차 블록."""

    def __init__(self, 입력: int, 출력: int, 보폭: int = 1) -> None:
        super().__init__()
        self.합성곱1 = nn.Conv2d(입력, 출력, 3, stride=보폭, padding=1, bias=False)
        self.정규화1 = nn.BatchNorm2d(출력)
        self.합성곱2 = nn.Conv2d(출력, 출력, 3, padding=1, bias=False)
        self.정규화2 = nn.BatchNorm2d(출력)
        self.지름길 = nn.Identity()
        if 보폭 != 1 or 입력 != 출력:
            self.지름길 = nn.Sequential(nn.Conv2d(입력, 출력, 1, stride=보폭, bias=False),
                                        nn.BatchNorm2d(출력))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        고칠양 = self.정규화2(self.합성곱2(F.relu(self.정규화1(self.합성곱1(x)))))
        return F.relu(고칠양 + self.지름길(x))


def 작은_ResNet() -> nn.Sequential:
```

```
    """9장 9.10절의 작은 ResNet."""
    return nn.Sequential(
        nn.Conv2d(1, 32, 3, padding=1, bias=False), nn.BatchNorm2d(32), nn.ReLU(),
        잔차블록(32, 32), 잔차블록(32, 64, 보폭=2), 잔차블록(64, 128, 보폭=2),
        nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Dropout(0.3), nn.Linear(128, 10))


def 로더들(일꾼: int) -> tuple[DataLoader, DataLoader]:
    증강 = v2.Compose([v2.ToImage(), v2.RandomCrop(28, padding=2), v2.RandomHorizontalFlip(),
                       v2.ToDtype(torch.float32, scale=True)])
    기본 = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)])
    학습 = datasets.FashionMNIST(데이터_폴더, train=True, download=True, transform=증강)
    시험 = datasets.FashionMNIST(데이터_폴더, train=False, download=True, transform=기본)
    옵션 = dict(num_workers=일꾼, pin_memory=일꾼 > 0, persistent_workers=일꾼 > 0)
    return (DataLoader(학습, batch_size=256, shuffle=True, **옵션),
            DataLoader(시험, batch_size=1024, **옵션))


def 실험(이름: str, amp: bool, 컴파일: bool, 일꾼: int, 에폭_수: int = 4) -> None:
    torch.manual_seed(0)
    학습_로더, 시험_로더 = 로더들(일꾼)
    모델 = 작은_ResNet().to("cuda")
    옵티마이저 = torch.optim.AdamW(모델.parameters())        # 학습률은 아래 OneCycleLR이 정한다
    스케줄러 = torch.optim.lr_scheduler.OneCycleLR(          # 8장 8.7절, 배치마다 step()
        옵티마이저, max_lr=4e-3, total_steps=에폭_수 * len(학습_로더))
    실행 = torch.compile(모델) if 컴파일 else 모델

    def 정밀도():
        """amp를 켰으면 bfloat16 autocast, 아니면 아무것도 안 하는 with 블록."""
        return torch.autocast("cuda", dtype=torch.bfloat16) if amp else nullcontext()

    에폭_시간 = []
    for _ in range(에폭_수):
        모델.train()
        torch.cuda.synchronize()
        시작 = time.perf_counter()
```

```
        for x, y in 학습_로더:
            x, y = x.to("cuda", non_blocking=True), y.to("cuda", non_blocking=True)
            with 정밀도():
                손실 = F.cross_entropy(실행(x), y)
            옵티마이저.zero_grad(set_to_none=True)
            손실.backward()
            옵티마이저.step()
            스케줄러.step()
        torch.cuda.synchronize()
        에폭_시간.append(time.perf_counter() - 시작)
    모델.eval()
    맞힘 = 0
    with torch.no_grad(), 정밀도():
        for x, y in 시험_로더:
            맞힘 += (실행(x.to("cuda")).argmax(dim=1).cpu() == y).sum().item()
    이후 = sum(에폭_시간[1:]) / (에폭_수 - 1)              # 첫 에폭은 준비 시간이 섞여 따로 본다
    print(f"첫 에폭 {에폭_시간[0]:4.1f}초 ¦ 이후 에폭 평균 {이후:4.1f}초"
          f" ¦ 시험 정확도 {맞힘 / 100:5.2f}% ← {이름}")


if __name__ == "__main__":
    실험("① 기본", amp=False, 컴파일=False, 일꾼=0)
    실험("② + bfloat16 autocast", amp=True, 컴파일=False, 일꾼=0)
    실험("③ + torch.compile", amp=True, 컴파일=True, 일꾼=0)
    실험("④ + 데이터 로더 일꾼 8", amp=True, 컴파일=True, 일꾼=8)
    실험("⑤ ④에서 autocast·compile 빼기", amp=False, 컴파일=False, 일꾼=8)
```

```
첫 에폭  5.3초 ¦ 이후 에폭 평균  5.1초 ¦ 시험 정확도 90.94% ← ① 기본
첫 에폭  5.2초 ¦ 이후 에폭 평균  5.1초 ¦ 시험 정확도 90.85% ← ② + bfloat16 autocast
첫 에폭  6.2초 ¦ 이후 에폭 평균  5.2초 ¦ 시험 정확도 90.97% ← ③ + torch.compile
첫 에폭  1.0초 ¦ 이후 에폭 평균  1.0초 ¦ 시험 정확도 90.78% ← ④ + 데이터 로더 일꾼 8
첫 에폭  1.1초 ¦ 이후 에폭 평균  1.1초 ¦ 시험 정확도 90.86% ← ⑤ ④에서 autocast·compile 빼기
```

결과를 순서대로 읽어 보겠습니다.

- **①에서 ②, ③으로: 전혀 빨라지지 않았습니다.** 에폭당 5.1초 → 5.1초 → 5.2초예요. 16.10절에서 일꾼 없이는 GPU가 시간의 97%를 놀았던 것을 떠올려 보세요. 데이터가 오지 않는데 계

산을 빨리 해 봐야 기다리는 시간만 늘어납니다. ③의 첫 에폭 6.2초에는 컴파일 시간이 더해졌어요(컴파일 캐시가 비어 있던 첫 실행에서는 17초쯤 걸렸습니다).

- **④: 에폭당 1.0초로 5.2배 빨라졌습니다.** 데이터 로더에 일꾼 8명을 붙인 것 하나로요. 병목은 처음부터 데이터였던 거예요.
- **⑤: 1.1초.** 데이터 대기를 크게 줄인 상태에서 autocast와 compile을 빼면 1.0초에서 1.1초로 10%쯤 느려집니다. 16.6절의 작은 GPT에서 2.5배였던 것과 비교하면 작은데, 28 × 28 흑백 사진을 다루는 작은 CNN은 계산 자체가 작아서 줄일 것이 적기 때문이에요. 다만 이 10%를 두 기능의 몫 전부라고 보기는 어렵습니다. 16.10절에서 일꾼 8명을 붙인 작은 CNN도 GPU 사용률이 28%에 그쳤듯이, ④에서도 GPU는 여전히 데이터와 일감 맡기기를 기다리는 시간이 있을 거예요. 그 시간이 남아 있는 한 GPU 계산을 줄여도 전체 시간에는 덜 드러납니다.
- **정확도는 다섯 단계 모두 90.8~91.0%로 같습니다.** 빠르게 만드느라 학습 결과를 잃지 않았다는 확인이에요. 단계마다 끝자리가 조금씩 다른 것은 16비트 계산의 작은 오차와, 일꾼 수에 따라 증강의 무작위 순서가 달라지는 것 때문입니다.

이 실습에서 가장 중요한 교훈은 순서입니다. 만약 ②, ③만 해 보고 "혼합 정밀도도 compile도 효과가 없네"라고 결론 내렸다면 크게 틀렸을 거예요. **가장 느린 곳을 먼저 고쳐야 나머지 처방이 효과를 냅니다.** 그래서 16.12절의 체크리스트에서 "GPU가 놀고 있지 않은가"를 위쪽에 두었어요. 실제 프로젝트에서는 ① 상태에서 16.10절의 프로파일러부터 돌려 보고 병목을 확인한 뒤 고치는 순서가 맞습니다.

16.14 정리와 다음 장 미리보기

이 장에서는 같은 GPU로 같은 모델을 더 빠르고 가볍게 돌리는 방법을 직접 재면서 익혔습니다.

- **GPU가 빠른 이유**: 단순한 계산을 수천 개씩 동시에. 대신 일감 하나를 맡기는 데 몇 μs가 들어, 작은 일을 여러 번 시키면 CPU보다 느리다
- **시간 재기**: GPU는 비동기라 `torch.cuda.synchronize()`나 CUDA 이벤트로 기다린 뒤 재고, 준비 운동 후 중앙값. `torch.utils.benchmark.Timer`. `.item()`은 숨은 동기화
- **CPU와 GPU의 교차점**: 작은 행렬은 CPU, 큰 행렬은 GPU. 옮기는 비용이 그 사이 이득을 갉아먹으니 GPU에 올린 것은 GPU에 둔다
- **메모리**: `memory_allocated`(실제 사용)와 `memory_reserved`(캐싱 할당자의 창고), `max_memory_allocated`. 학습 메모리의 대부분은 배치에 비례하는 활성화. `torch.OutOfMemoryError`와 대처 순서
- **옮기기**: GPU 밖으로 나가는 길(PCIe)은 GPU 안보다 수십 배 좁다. 고정 메모리 +

`non_blocking=True`, DataLoader의 `pin_memory=True`

- **혼합 정밀도**: `float16`은 좁고 촘촘, `bfloat16`은 넓고 성기다. `autocast`가 계산마다 자료형을 고르고, `float16`이면 `GradScaler`로 언더플로를 막는다. 학습 2.5배, 메모리 감소, 정확도는 그대로
- TF32: `set_float32_matmul_precision("high")` 한 줄로 `float32` 행렬 곱 1.6배
- torch.compile: 첫 실행에 컴파일 시간, 그다음부터 커널 융합으로 빨라진다. 원소별 계산에서 특히 크다. `reduce-overhead`(CUDA 그래프), 그래프 끊김과 `explain`, `fullgraph=True`
- SDPA: 플래시·메모리 효율 백엔드는 점수 표를 만들지 않아 수십 배 빠르고 메모리는 수백 분의 일
- **프로파일러**: `torch.profiler`로 시간이 드는 연산을 큰 순서로, `record_function`으로 구간 이름표, GPU 사용률로 데이터 병목 찾기
- **배치와 처리량**: 처리량이 더 늘지 않는 지점이 GPU가 꽉 찬 곳. 그 너머는 메모리만 쓴다
- **순서**: 가장 느린 곳부터. 한 번에 하나씩 바꾸고, 매번 재고, 정확도를 확인한다

이 장에서는 GPU 한 장의 성능을 끝까지 끌어냈습니다. 그런데 모델이 GPU 한 장의 메모리에 다 들어가지 않거나, 한 장으로는 학습이 며칠씩 걸린다면 어떻게 할까요? 다음 17장에서는 먼저 한 장 안에서 메모리를 아끼는 방법(그래디언트 누적, 활성화 체크포인팅)을 보고, 이 서버의 GPU 두 장을 함께 써서 학습하는 분산 학습(DDP)으로 넘어갑니다. 16.5절에서 본 "GPU 밖으로 나가는 좁은 길"이 두 GPU가 기울기를 주고받을 때 어떤 비용이 되는지도 직접 재 볼 거예요.

17

분산·대규모 학습: 메모리를 아끼고 GPU를 늘리기

학습 목표

- 학습 중 GPU 메모리가 파라미터·기울기·옵티마이저 상태·활성화에 얼마씩 쓰이는지 재고 계산하기
- 그래디언트 누적과 활성화 체크포인팅으로 GPU 하나에서 메모리 부족을 먼저 해결하기
- 프로세스·rank·world size·all-reduce의 뜻을 알고 `torchrun`으로 여러 프로세스를 띄우기
- `DistributedDataParallel`로 학습 코드를 여러 GPU용으로 바꾸고, 1 GPU와 같은 결과가 나오는지 확인하기
- GPU 사이 통신 비용을 재고, 모델과 배치 크기에 따라 GPU를 늘린 이득이 달라지는 이유를 설명하기
- FSDP·텐서 병렬·파이프라인 병렬이 무엇이고 언제 쓰는지 알고, 분산 학습의 흔한 오류를 피하기

들어가며: 이삿짐을 혼자 옮길까, 친구를 부를까?

이사 날을 떠올려 보세요. 짐이 많으면 방법은 두 가지입니다. 한 번에 드는 양을 줄여서 **여러 번 나르거나**, 친구를 불러 **여럿이 나눠 나르거나**요. 그런데 친구를 부르면 공짜로 빨라질까요? 아닙니다.

누가 어느 상자를 들지 정해야 하고, 다 나른 뒤에는 "다 옮겼어?" 하고 서로 확인해야 해요. 엘리베이터가 하나뿐인 아파트라면 친구가 셋이어도 엘리베이터 앞에서 줄을 서게 됩니다.

딥러닝 학습도 똑같습니다. 15장의 확산 모델처럼 모델이 조금만 커져도 GPU 메모리가 모자라거나 시간이 너무 오래 걸리죠. 16장에서 GPU 하나를 최대한 빠르게 쓰는 법을 봤다면, 이 장은 그다음 두 질문을 다룹니다.

- **GPU 메모리가 모자라면?** 먼저 GPU 하나에서 할 수 있는 일부터 합니다. 메모리가 어디에 쓰이는지 재 보고, 한 번에 드는 양을 줄여 여러 번 나르고(그래디언트 누적), 기억해 둘 것을 줄이고 필요할 때 다시 계산합니다(활성화 체크포인팅).
- **GPU를 여러 개 쓰면?** 친구를 부르는 방법입니다. 데이터를 나눠 맡기고(DDP), 그래도 모델이 안 들어가면 모델까지 나눠 맡깁니다(FSDP). 그리고 "서로 확인하는 시간", 즉 **통신 비용**이 얼마인지 직접 잽니다.

이 장의 예제는 지금까지와 달리 **NVIDIA GPU 두 장이 달린 리눅스 서버**에서 실행했습니다. RTX 4090(메모리 24GB) 두 장인데, 두 GPU를 잇는 전용 다리(NVLink)가 없어서 데이터를 주고받으려면 메인보드의 PCIe 통로와 CPU를 거쳐야 해요. 게다가 두 번째 GPU는 통로가 좁은 자리(PCIe 3세대 4차선)에 꽂혀 있습니다. 비싼 서버가 아니라 연구실이나 집에서 흔히 보는 구성이죠. 그래서 이 장의 숫자는 "GPU를 늘리면 무조건 빨라진다"는 기대가 어디서 깨지는지를 솔직하게 보여 줍니다.

Q

Q. GPU가 없거나 하나뿐이면 이 장을 못 따라 하나요? 반은 따라 할 수 있습니다. 17.2절의 누적이 큰 배치와 같은지 확인하는 예제와 17.5절의 첫 통신 예제, 17.12절의 멈춤 예제, 17.13절 실습은 CPU에서도 돌아가요. 이 책의 맥에서 확인해 보니 맥용 PyTorch에는 NVIDIA용 통신 라이브러리(NCCL)가 없지만, CPU끼리 통신하는 `gloo`로 프로세스 두 개를 띄워 DDP 학습까지 돌아갔습니다. 메모리를 재는 예제와 처리량 비교는 NVIDIA GPU가 있어야 하니, 숫자는 본문의 출력으로 읽어 주세요. GPU 한 장짜리 환경은 부록 A.5절의 Colab으로 빌릴 수 있습니다. 이 장의 예제는 파이썬 3.12, PyTorch 2.14.1(CUDA 12.6용 빌드), torchvision 0.29.1에서 실행해 확인했어요.

17.1 학습 중 GPU 메모리는 어디에 쓰일까

네 가지 손님

14장 14.6절에서 "모델이 내 노트북에 들어갈까?"를 계산했습니다. 파라미터 수 × 칸 크기였죠. 그런데 그건 **추론**, 즉 모델을 쓰기만 할 때의 이야기예요. 학습할 때는 GPU 메모리에 손님이 넷으로 늘

어납니다.

- **파라미터**: 모델의 숫자 그 자체
- **기울기**: `backward()`가 파라미터마다 하나씩 만드는 `.grad` (5장)
- **옵티마이저 상태**: Adam이 파라미터마다 기억해 두는 값 (6장 6.6절)
- **활성화**: forward 동안 층마다 저장해 두는 중간 결과. backward가 기울기를 계산할 때 다시 꺼내 씁니다(5장 5.6절의 연쇄 법칙)

말로만 들으면 감이 안 오니, 선형층 8개짜리 모델로 학습 한 걸음 한 걸음마다 메모리를 재 보겠습니다. `torch.cuda.memory_allocated()`는 지금 텐서들이 차지한 GPU 메모리입니다. 메모리를 재는 도구 자체는 16장 16.4절에서 자세히 다뤘으니, 여기서는 **무엇이 메모리를 차지하는지**에 집중할 게요.

```
import torch
from torch import nn

assert torch.cuda.is_available(), "이 예제는 NVIDIA GPU가 있어야 합니다"
장치 = torch.device("cuda")
MB = 1024 ** 2


def 찍기(시점: str) -> None:
    """지금 텐서들이 차지한 GPU 메모리를 MB로 보여 준다."""
    print(f"{시점:>16}: {torch.cuda.memory_allocated() / MB:6,.0f} MB")


torch.manual_seed(0)
# 4096 → 4096 선형층 8개: 파라미터 약 1억 3천만 개
모델 = nn.Sequential(*[nn.Sequential(nn.Linear(4096, 4096), nn.ReLU()) for _ in range(8)]).to(장치)
개수 = sum(p.numel() for p in 모델.parameters())
print(f"파라미터 {개수:,}개 × 4바이트 = {개수 * 4 / MB:,.0f} MB")

옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-4)
x = torch.randn(2048, 4096, device=장치)          # 입력 2048개 묶음 (32MB)
찍기("모델·입력 올린 뒤")
for 번째 in (1, 2):
    torch.cuda.reset_peak_memory_stats()
```

```
    손실 = 모델(x).pow(2).mean()
    찍기(f"{번째}회 forward 뒤")       # 활성화가 쌓였다
    손실.backward()
    찍기(f"{번째}회 backward 뒤")      # 활성화는 풀리고 기울기가 생겼다
    옵티마이저.step()
    찍기(f"{번째}회 step 뒤")          # 첫 step에서 Adam 상태가 생긴다
    옵티마이저.zero_grad()
    찍기(f"{번째}회 zero_grad 뒤")     # 기울기를 None으로 비운다
    print(f"{f'{번째}회 중 최고':>16}: {torch.cuda.max_memory_allocated() / MB:6,.0f} MB")

상태 = 옵티마이저.state[모델[0][0].weight]
print("Adam이 가중치마다 기억하는 것:", [k for k, v in 상태.items() if v.numel() > 1])
```

```
파라미터 134,250,496개 × 4바이트 = 512 MB
      모델·입력 올린 뒤:    544 MB
    1회 forward 뒤:    808 MB
   1회 backward 뒤:  1,073 MB
       1회 step 뒤:  2,097 MB
  1회 zero_grad 뒤:  1,585 MB
         1회 중 최고:  2,609 MB
    2회 forward 뒤:  1,841 MB
   2회 backward 뒤:  2,097 MB
       2회 step 뒤:  2,097 MB
  2회 zero_grad 뒤:  1,585 MB
         2회 중 최고:  2,609 MB
Adam이 가중치마다 기억하는 것: ['exp_avg', 'exp_avg_sq']
```

이 책의 실행 환경(RTX 4090)에서 잰 값입니다. 숫자를 위에서부터 따라가 보세요.

- **모델·입력 올린 뒤 544MB**: 파라미터 1억 3천만 개 × 4바이트 = 512MB에 입력 32MB를 더한 값입니다.
- **1회 forward 뒤 808MB**: 264MB가 늘었어요. 층마다 backward 때 쓸 중간 결과를 저장해 둔 **활성화**입니다. 입력 2048개 × 4096칸짜리 텐서(32MB)가 층마다 쌓인 거예요.
- **1회 backward 뒤 1,073MB**: 활성화는 다 쓰고 풀렸지만, 파라미터와 같은 크기의 **기울기** 512MB가 새로 생겼습니다.
- **1회 step 뒤 2,097MB**: 1,024MB가 늘었습니다. 마지막 줄에 보이는 `exp_avg`와 `exp_avg_sq`,

즉 Adam이 파라미터마다 기억하는 두 값(기울기의 이동 평균과 제곱의 이동 평균)이 **처음 step 할 때** 만들어졌어요. 파라미터 크기의 두 배입니다.

- **zero_grad 뒤 1,585MB**: `zero_grad()`는 기본으로 기울기를 0으로 채우는 대신 `None`으로 **버려서** 512MB를 돌려줍니다.

두 번째 걸음부터는 같은 일이 반복돼요. forward로 활성화가 쌓였다(1,841MB) 풀리고, 기울기가 생겼다(2,097MB) 버려집니다. 그리고 두 번 다 **최고 2,609MB**를 찍었는데, 이건 `step()` 안에서 Adam이 계산하느라 잠깐 쓰는 임시 공간까지 더한 값입니다. GPU 메모리가 모자라다는 오류는 평균이 아니라 이 **가장 높은 순간**에 납니다.

정리하면, Adam으로 학습할 때 활성화를 빼고도 **파라미터의 4배**(파라미터 1 + 기울기 1 + Adam 상태 2)가 늘 자리를 차지합니다. 512MB짜리 모델이 2GB를 쓰는 거예요.

7B 모델을 학습하려면

14장 14.6절의 표를 학습용으로 다시 계산해 보겠습니다. 파라미터 하나당 몇 바이트가 드는지만 바꾸면 됩니다.

```
GB = 1024 ** 3
# 파라미터 하나당 바이트 수 (float32로 학습할 때)
내역 = {"파라미터": 4, "기울기": 4, "Adam 1차 모멘트": 4, "Adam 2차 모멘트": 4}
학습_바이트 = sum(내역.values())
print(" + ".join(f"{k} {v}" for k, v in 내역.items()), f"= 파라미터당 {학습_바이트}바이트")

for 이름, 개수 in [("0.5B", 494_032_768), ("1B", 1_000_000_000), ("7B", 7_000_000_000)]:
    print(f"{이름:>4}: 추론(bfloat16) {개수 * 2 / GB:5.1f}GB, "
          f"학습(float32+Adam) {개수 * 학습_바이트 / GB:6.1f}GB, "
          f"학습(float32+SGD) {개수 * 8 / GB:5.1f}GB")
print("활성화는 따로: 배치 크기와 입력 길이에 비례해서 더 든다")
```

```
파라미터 4 + 기울기 4 + Adam 1차 모멘트 4 + Adam 2차 모멘트 4 = 파라미터당 16바이트
0.5B: 추론(bfloat16)   0.9GB, 학습(float32+Adam)    7.4GB, 학습(float32+SGD)   3.7GB
  1B: 추론(bfloat16)   1.9GB, 학습(float32+Adam)   14.9GB, 학습(float32+SGD)   7.5GB
  7B: 추론(bfloat16)  13.0GB, 학습(float32+Adam)  104.3GB, 학습(float32+SGD)  52.2GB
활성화는 따로: 배치 크기와 입력 길이에 비례해서 더 든다
```

`bfloat16`으로 쓰기만 하면 13GB인 7B 모델이, `float32`와 Adam으로 학습하려면 104GB가 필요합

니다. 24GB짜리 GPU 네 장을 붙여도 활성화를 넣을 자리가 없어요. 모멘텀 없는 SGD라면 옵티마이저 상태가 없어서 절반인 52GB입니다. 옵티마이저 하나만 바꿔도 메모리가 두 배 차이 나는 셈이에요.

실제 대형 모델 학습은 16장 16.6절의 혼합 정밀도로 계산을 16비트로 하고, 이 장 뒤에서 볼 FSDP로 이 104GB를 여러 GPU에 나눠 담고, 14장의 LoRA처럼 고칠 파라미터 자체를 줄이는 방법을 함께 씁니다. 그 전에 먼저 GPU 하나에서 할 수 있는 일부터 보겠습니다.

17.2 그래디언트 누적: 큰 배치를 나눠 담기

장바구니에 담아 두었다가 한 번에 계산하기

마트에서 물건을 하나 집을 때마다 계산대로 가지는 않죠. 장바구니에 담아 두었다가 다 고른 뒤 한 번에 계산합니다. **그래디언트 누적(gradient accumulation)** 이 이 방법이에요. 배치 256장이 메모리에 안 들어가면 64장씩 네 번 forward와 backward를 하면서 기울기를 장바구니(`.grad`)에 쌓고, 네 번째가 끝난 뒤에 한 번만 `step()`으로 고칩니다.

5장 5.7절에서 `backward()`는 기울기를 덮어쓰지 않고 **더한다**고 했고, 그때 "이 성질 덕분에 그래디언트 누적이 가능하다"고 예고했어요. 정말 큰 배치와 같은 결과가 나오는지 숫자로 확인해 보겠습니다. 이 예제는 CPU에서 돌아가니 어떤 컴퓨터에서든 같은 결과를 볼 수 있어요.

```
import torch
from torch import nn

torch.manual_seed(0)
X = torch.randn(64, 10)                # 예제용으로 만든 데이터 64개
y = torch.randn(64, 1)
손실함수 = nn.MSELoss()


def 새_모델() -> nn.Module:
    """매번 똑같은 초깃값으로 시작하는 작은 모델."""
    torch.manual_seed(1)
    return nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 1))
```

```
def 기울기_모으기(모델: nn.Module, 단계: int, 나누기: bool = True) -> None:
    """64개를 단계 수만큼 나눠 forward/backward하고 기울기를 .grad에 쌓는다."""
    for xb, yb in zip(X.chunk(단계), y.chunk(단계)):
        손실 = 손실함수(모델(xb), yb)
        (손실 / 단계 if 나누기 else 손실).backward()


큰배치 = 새_모델()
기울기_모으기(큰배치, 단계=1)            # 64개를 한 번에
누적 = 새_모델()
기울기_모으기(누적, 단계=4)              # 16개씩 네 번
잊음 = 새_모델()
기울기_모으기(잊음, 단계=4, 나누기=False)  # 손실을 4로 나누는 것을 잊으면

차이 = max((a.grad - b.grad).abs().max().item()
         for a, b in zip(큰배치.parameters(), 누적.parameters()))
print(f"64개 한 번 vs 16개×4번 누적: 기울기 최대 차이 {차이:.1e}")
print(f"나누기를 잊으면 기울기가 {(잊음[0].weight.grad / 큰배치[0].weight.grad).mean():.2f}배")


def 학습(단계: int) -> list[float]:
    """같은 64개로 다섯 걸음 학습하고, 걸음마다 전체 손실을 기록한다."""
    모델 = 새_모델()
    옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.1)
    기록 = []
    for _ in range(5):
        옵티마이저.zero_grad()
        기울기_모으기(모델, 단계)
        옵티마이저.step()                    # 누적이 끝난 뒤에 한 번만 고친다
        with torch.no_grad():
            기록.append(round(손실함수(모델(X), y).item(), 6))
    return 기록


for 단계 in (1, 4, 8):
    print(f"{64 // 단계:2}개씩 {단계}번 누적:", 학습(단계))
```

```
64개 한 번 vs 16개×4번 누적: 기울기 최대 차이 4.5e-08
나누기를 잊으면 기울기가 4.00배
64개씩 1번 누적: [1.152792, 1.109052, 1.072198, 1.040522, 1.012927]
16개씩 4번 누적: [1.152792, 1.109052, 1.072198, 1.040522, 1.012927]
 8개씩 8번 누적: [1.152792, 1.109052, 1.072198, 1.040522, 1.012927]
```

64개를 한 번에 넣은 기울기와 16개씩 네 번 나눠 쌓은 기울기의 차이가 4.5e-08, 즉 0.00000005 수준입니다. 4장 4.3절에서 본 `float32`의 끝자리 오차일 뿐 사실상 같아요. 다섯 걸음 학습한 손실도 64개씩, 16개씩 4번, 8개씩 8번 모두 소수점 여섯째 자리까지 똑같습니다.

핵심은 **손실 / 단계**입니다. `MSELoss`와 `CrossEntropyLoss`는 기본으로 배치의 **평균**을 내요. 16개짜리 평균 네 개를 그냥 더하면 64개짜리 평균의 4배가 되죠. 두 번째 줄처럼 나누기를 잊으면 기울기가 정확히 4.00배가 됩니다. 학습률을 4배로 올린 것과 같아서 학습이 불안정해져요. 누적할 때는 **손실을 누적 횟수로 나누고, step과 zero_grad는 누적이 끝난 뒤에 한 번만** 한다는 두 가지를 기억하세요.

Q

Q. 누적하면 언제나 큰 배치와 똑같나요? 아닙니다. 배치 안의 샘플끼리 섞이는 계산이 있으면 달라져요. 대표적인 것이 8장 8.4절의 **배치 정규화**입니다. 배치의 평균과 분산으로 표준화하니, 샘플 64개의 평균과 샘플 16개의 평균은 다르죠. 그래서 배치 정규화가 있는 모델에서 누적은 "큰 배치와 거의 비슷한" 결과를 냅니다. 또 8장 8.9절의 그래디언트 클리핑은 누적이 **끝난 뒤**, `step()` 바로 앞에서 해야 합니다. 중간에 자르면 일부 조각의 기울기만 잘려요.

메모리는 줄고, 시간은?

이번에는 GPU에서 실제로 메모리가 얼마나 줄고 시간은 얼마나 드는지 재 보겠습니다. 10장의 ResNet18에 224 × 224 사진 256장(예제용 난수 입력)을 넣고, 1번에서 16번까지 나눠 담아 봅니다.

```
import statistics
import time

import torch
from torch import nn
from torchvision.models import resnet18
```

```
assert torch.cuda.is_available(), "이 예제는 NVIDIA GPU가 있어야 합니다"
장치 = torch.device("cuda")
GB = 1024 ** 3

torch.manual_seed(0)
모델 = resnet18(num_classes=10).to(장치)
옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.01, momentum=0.9)
손실함수 = nn.CrossEntropyLoss()
X = torch.randn(256, 3, 224, 224, device=장치)   # 예제용 입력: 224×224 사진 256장
y = torch.randint(0, 10, (256,), device=장치)


def 한_걸음(단계: int) -> None:
    """256장을 단계 수만큼 나눠 기울기를 누적한 뒤 한 번 고친다."""
    옵티마이저.zero_grad()
    for xb, yb in zip(X.chunk(단계), y.chunk(단계)):
        (손실함수(모델(xb), yb) / 단계).backward()
    옵티마이저.step()


for 단계 in (1, 2, 4, 8, 16):
    한_걸음(단계)                                  # 준비 운동
    torch.cuda.synchronize()
    torch.cuda.reset_peak_memory_stats()
    시간들 = []
    for _ in range(5):
        시작 = time.perf_counter()
        한_걸음(단계)
        torch.cuda.synchronize()                   # GPU 일이 끝날 때까지 기다린 뒤 잰다
        시간들.append(time.perf_counter() - 시작)
    print(f"{256 // 단계:3}장 × {단계:2}번: 최고 메모리 {torch.cuda.max_memory_allocated() /
    ↪  GB:5.2f} GB, "
          f"256장 한 걸음 {statistics.median(시간들) * 1000:5.0f} ms")
```

```
256장 ×  1번: 최고 메모리  5.48 GB, 256장 한 걸음   108 ms
128장 ×  2번: 최고 메모리  2.92 GB, 256장 한 걸음   104 ms
 64장 ×  4번: 최고 메모리  1.61 GB, 256장 한 걸음    99 ms
 32장 ×  8번: 최고 메모리  0.96 GB, 256장 한 걸음    97 ms
 16장 × 16번: 최고 메모리  0.64 GB, 256장 한 걸음   101 ms
```

나누는 횟수를 두 배로 늘릴 때마다 최고 메모리가 거의 절반씩 줄어서, 한 번에 넣으면 5.48GB이던 것이 16번에 나누면 0.64GB가 됩니다. 메모리의 대부분이 배치 크기에 비례하는 **활성화**였다는 뜻이에요. 0.64GB 아래로는 잘 안 내려가는데, 그 바닥에는 세 가지가 남아 있어요. 조각 크기와 상관없이 GPU에 올려 둔 입력 256장(약 0.14GB), 파라미터·기울기·모멘텀(합쳐서 약 0.13GB), 그리고 16장짜리 조각 하나의 활성화(16장 16.4절에서 잰 대로 약 0.33GB)입니다. 바닥의 절반 넘게가 여전히 활성화라서, 조각을 더 잘게 나누면 조금 더 내려가지만 GPU가 노는 시간이 늘어나요.

시간은 어떨까요? 256장을 한 걸음 처리하는 데 97~108ms로 거의 그대로입니다. 이 책의 GPU에서는 ResNet18이 사진 16장만 받아도 GPU를 충분히 바쁘게 만들 만큼 계산이 크기 때문이에요. 모델이 작거나 나눈 조각이 너무 작으면 GPU가 놀아서 느려지니(16장 16.11절의 배치 크기와 처리량), "메모리에 들어가는 가장 큰 조각"으로 나누는 것이 요령입니다.

17.3 활성화 체크포인팅: 적어 두지 말고 다시 계산하기

시험 공부 노트 대신 풀이를 다시 하기

17.1절에서 활성화가 배치 크기에 비례해 커진다고 했습니다. 트랜스포머처럼 층이 깊고 입력이 길면 활성화가 파라미터보다 훨씬 커져요. 12장 12.9절에서 어텐션 가중치 표가 글자 수의 제곱으로 커지던 것 기억하시죠?

활성화 체크포인팅(activation checkpointing) 은 수학 문제를 풀 때 중간 계산을 모두 적어 두는 대신, 중요한 지점(층의 입력)만 적어 두고 나중에 필요해지면 그 지점부터 **다시 풀어 보는** 방법입니다. forward 때는 층의 입력만 저장하고 중간 결과는 버렸다가, backward가 그 층에 도착하면 forward를 한 번 더 해서 중간 결과를 되살려요. 메모리를 아끼는 대신 계산을 더 하는 거죠. 12장의 인코더 층을 12개 쌓은 모델로 재 보겠습니다.

```
import statistics
import time
```

```
import torch
from torch import nn
from torch.utils.checkpoint import checkpoint

assert torch.cuda.is_available(), "이 예제는 NVIDIA GPU가 있어야 합니다"
장치 = torch.device("cuda")
GB = 1024 ** 3


class 트랜스포머(nn.Module):
    """12장의 인코더 층을 쌓은 모델. 체크포인트=True면 층마다 활성화를 버리고 다시 계산한다."""

    def __init__(self, 층수: int, 체크포인트: bool) -> None:
        super().__init__()
        self.층들 = nn.ModuleList(
            nn.TransformerEncoderLayer(1024, 16, 4096, dropout=0.1, batch_first=True)
            for _ in range(층수))
        self.체크포인트 = 체크포인트

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        for 층 in self.층들:
            if self.체크포인트 and self.training:
                x = checkpoint(층, x, use_reentrant=False)   # 층의 중간 결과를 저장하지 않는다
            else:
                x = 층(x)
        return x


x = torch.randn(16, 512, 1024, device=장치)       # 예제용 입력: 16문장 × 512토큰
결과 = {}
for 체크포인트 in (False, True):
    torch.manual_seed(0)
    모델 = 트랜스포머(12, 체크포인트).to(장치)
    시간들 = []
    for 번째 in range(6):
        모델.zero_grad()
```

```
        torch.cuda.synchronize()
        torch.cuda.reset_peak_memory_stats()
        시작 = time.perf_counter()
        torch.manual_seed(번째)                    # 드롭아웃 난수를 두 방식에서 똑같이
        모델(x).pow(2).mean().backward()
        torch.cuda.synchronize()
        시간들.append(time.perf_counter() - 시작)
    결과[체크포인트] = [p.grad.clone() for p in 모델.parameters()]
    개수 = sum(p.numel() for p in 모델.parameters())
    print(f"체크포인팅 {'켬' if 체크포인트 else '끔'}: 최고 메모리 "
          f"{torch.cuda.max_memory_allocated() / GB:5.2f} GB, "
          f"forward+backward {statistics.median(시간들[1:]) * 1000:4.0f} ms")
    del 모델

print(f"파라미터 {개수:,}개 (float32로 {개수 * 4 / GB:.2f} GB)")
같은가 = all(torch.allclose(a, b, atol=1e-6) for a, b in zip(결과[False], 결과[True]))
print("두 방식의 기울기가 같은가:", 같은가)
```

```
체크포인팅 끔: 최고 메모리  7.78 GB, forward+backward  241 ms
체크포인팅 켬: 최고 메모리  2.43 GB, forward+backward  322 ms
파라미터 151,154,688개 (float32로 0.56 GB)
두 방식의 기울기가 같은가: True
```

체크포인팅을 켜니 최고 메모리가 7.78GB에서 2.43GB로 3분의 1 아래로 줄었습니다. 대신 forward+backward 시간은 241ms에서 322ms로 약 34% 늘었어요. forward를 사실상 두 번 하니 그만큼 느려진 겁니다. 이 책의 실행 환경에서 잰 값이라 숫자는 조금씩 다를 수 있지만, "메모리를 크게 줄이고 시간을 조금 더 낸다"는 거래 관계는 어디서나 같습니다.

마지막 줄도 중요합니다. 두 방식의 기울기가 **같아요.** 드롭아웃처럼 난수를 쓰는 층이 있어도, `checkpoint`는 처음 forward 때의 난수 상태를 기억했다가 다시 계산할 때 되돌려 놓기 때문입니다. 결과는 그대로이고 메모리와 시간만 바꾼 거죠.

쓰는 법은 한 줄입니다. `x = 층(x)` 대신 `x = checkpoint(층, x, use_reentrant=False)`라고 쓰면 그 층이 체크포인트 구간이 돼요. `use_reentrant=False`는 PyTorch가 권하는 새 방식이라 명시해 두는 것이 좋습니다. Hugging Face 모델은 `model.gradient_checkpointing_enable()` 한 줄로 같은 일을 해 줍니다.

방법	줄어드는 것	대가	결과
그래디언트 누적	활성화 (배치를 나눠서)	거의 없음 (조각이 너무 작으면 느려짐)	큰 배치와 같음 (배치 정규화 제외)
활성화 체크포인팅	활성화 (저장하지 않고 다시 계산)	forward 한 번 더 (이 예제에서 +34%)	같음
옵티마이저 바꾸기 (Adam → SGD)	옵티마이저 상태	학습이 느리거나 불안정할 수 있음	달라짐
16비트 계산 (16장 16.6절)	활성화·계산 시간	정밀도 관리	거의 같음

17.4 여러 GPU에 일을 나누는 방법들

GPU 하나로 할 수 있는 일을 다 했는데도 모자라면 이제 친구를 부를 차례입니다. 나누는 방법은 크게 두 갈래예요. 식당 주방으로 비유해 보겠습니다.

- **데이터 병렬(data parallel)**: 같은 레시피를 가진 요리사 여럿이 **손님을 나눠 받습니다.** 요리사마다 모델 전체를 갖고 데이터만 나눠 맡아요. 각자 계산한 기울기를 마지막에 평균 내서 모두 같은 방향으로 고칩니다. 가장 쉽고 가장 많이 씁니다.
- **모델 병렬(model parallel)**: 요리 하나를 **공정별로 나눠** 맡습니다. 한 사람은 재료 손질, 한 사람은 굽기, 한 사람은 담기처럼요. 모델이 GPU 하나에 아예 안 들어갈 때 씁니다.

방법	나누는 것	GPU마다 가진 것	언제 쓰나
DDP (17.7절)	데이터	모델 전체	모델이 GPU 하나에 들어갈 때. 가장 먼저 시도
FSDP·ZeRO (17.11절)	데이터 + 파라미터·기울기·옵티마이저 상태	모델의 일부 조각	학습 메모리(17.1절의 4배)가 GPU 하나를 넘을 때

방법	나누는 것	GPU마다 가진 것	언제 쓰나
텐서 병렬	층 안의 행렬을 쪼갬	행렬의 일부	아주 큰 층. GPU끼리 빠른 연결(NVLink)이 있을 때
파이프라인 병렬	층을 순서대로 나눔	앞쪽 층, 뒤쪽 층	아주 깊은 모델. 여러 서버에 걸칠 때

이 장에서는 DDP를 중심으로 직접 돌려 보고, FSDP는 짧게 맛만 봅니다. 텐서 병렬과 파이프라인 병렬은 수백 장의 GPU로 거대 모델을 학습하는 곳의 기술이라 개념만 알아 두면 충분해요.

17.5 분산의 기본 낱말: 프로세스, rank, all-reduce

GPU마다 프로세스 하나

PyTorch의 분산 학습은 **GPU 하나에 프로세스(실행 중인 파이썬 프로그램) 하나**를 띄우는 방식입니다. 1권 17장에서 `ProcessPoolExecutor`로 계산을 여러 프로세스에 나눴던 것과 비슷해요. 다만 이번에는 프로세스들이 같은 코드를 동시에 실행하면서, 정해진 순간에 서로 숫자를 주고받습니다. 이때 쓰는 낱말 몇 개만 알면 됩니다.

낱말	뜻	이 장의 서버에서
world size	함께 일하는 프로세스의 수	2
rank	프로세스마다 붙은 번호 (0부터)	0, 1
local rank	그 컴퓨터 안에서의 번호. 몇 번 GPU를 쓸지 정할 때 쓴다	0, 1
백엔드	통신을 맡는 라이브러리. NVIDIA GPU는 `nccl`, CPU는 `gloo`	`nccl`

낱말	뜻	이 장의 서버에서
all-reduce	모두의 값을 모아 합(또는 평균)을 내서 **모두에게** 돌려주는 통신	

여기서 가장 중요한 것이 **all-reduce**입니다. 요리사 둘이 각자 맛을 본 점수를 불러 주고, 둘 다 합계를 받아 적는 것과 같아요. DDP가 기울기를 평균 낼 때 쓰는 통신이 바로 이것입니다. 직접 해 보겠습니다. 아래 코드를 `b06_첫_allreduce.py`로 저장하세요. 이 장의 분산 예제는 모두 이렇게 파일로 저장한 뒤, 아래 명령의 파일 이름만 바꿔서 실행합니다.

```
import os
from datetime import timedelta

import torch
import torch.distributed as dist


def 분산_시작() -> torch.device:
    """torchrun이 넣어 준 환경 변수로 프로세스 그룹에 들어간다."""
    if torch.cuda.is_available():
        장치 = torch.device("cuda", int(os.environ["LOCAL_RANK"]))  # 프로세스마다 GPU 하나
        torch.cuda.set_device(장치)
        dist.init_process_group("nccl", timeout=timedelta(seconds=60), device_id=장치)
    else:
        장치 = torch.device("cpu")                                  # GPU가 없으면 CPU끼리
        dist.init_process_group("gloo", timeout=timedelta(seconds=60))
    return 장치


def 차례로_말하기(말: str) -> None:
    """rank 0부터 차례로 한 줄씩 출력한다 (출력이 뒤섞이지 않게)."""
    for 차례 in range(dist.get_world_size()):
        if dist.get_rank() == 차례:
            print(말, flush=True)
        dist.barrier()                                              # 모두 여기까지 올 때까지 기다린다
```

```
장치 = 분산_시작()
rank, 전체 = dist.get_rank(), dist.get_world_size()
if rank == 0:
    print(f"백엔드 {dist.get_backend()}, 프로세스 {전체}개", flush=True)
dist.barrier()

x = torch.full((3,), float(rank + 1), device=장치)                # rank 0은 1, rank 1은 2
차례로_말하기(f"[rank {rank}] 내 값        {x.tolist()} ({장치})")
dist.all_reduce(x, op=dist.ReduceOp.SUM)                          # 모두의 값을 더해 모두에게
차례로_말하기(f"[rank {rank}] all_reduce 뒤 {x.tolist()}")

평균 = torch.tensor([float(rank + 1)], device=장치)
dist.all_reduce(평균)                                              # 더한 뒤
평균 /= 전체                                                       # 프로세스 수로 나누면 평균
차례로_말하기(f"[rank {rank}] 평균          {평균.item()}")

dist.destroy_process_group()                                      # 끝낼 때 정리
```

이 파일은 `python`으로 바로 실행하지 않고 `torchrun`으로 띄웁니다. `torchrun`은 PyTorch와 함께 설치되는 명령이라, 지금까지 `uv run python`으로 실행했듯이 `uv run torchrun`으로 부릅니다. 터미널에서 이렇게 실행하세요.

```
# 이 컴퓨터(노드 하나)에서 프로세스 2개를 띄운다. GPU가 있으면 프로세스마다 GPU 하나씩
uv run torchrun --nproc_per_node=2 --master_addr=127.0.0.1 --master_port=29500 b06_첫_allreduce.py
```

```
백엔드 nccl, 프로세스 2개
[rank 0] 내 값        [1.0, 1.0, 1.0] (cuda:0)
[rank 1] 내 값        [2.0, 2.0, 2.0] (cuda:1)
[rank 0] all_reduce 뒤 [3.0, 3.0, 3.0]
[rank 1] all_reduce 뒤 [3.0, 3.0, 3.0]
[rank 0] 평균          1.5
[rank 1] 평균          1.5
```

rank 0은 `[1, 1, 1]`, rank 1은 `[2, 2, 2]`를 갖고 있었는데, `all_reduce` 한 줄 뒤에 **둘 다** `[3, 3,`

3]이 됐습니다. 합을 프로세스 수로 나누면 평균 1.5고요. 이것이 DDP의 핵심 동작입니다. GPU마다 다른 데이터로 구한 기울기를 all-reduce로 더해 GPU 수로 나누면, 모든 GPU가 똑같은 평균 기울기를 갖게 되고, 똑같이 고쳐서 계속 같은 모델을 유지해요.

코드에서 볼 점은 이렇습니다.

- **torchrun이 하는 일**: `--nproc_per_node=2`만큼 프로세스를 띄우고, 프로세스마다 `RANK`, `LOCAL_RANK`, `WORLD_SIZE`와 서로 만날 주소(`MASTER_ADDR`, `MASTER_PORT`)를 환경 변수로 넣어 줍니다. `init_process_group`은 이 환경 변수를 읽어서 서로를 찾아요. 실행하면 `OMP_NUM_THREADS`를 1로 정했다는 경고가 나오는데, 프로세스 여럿이 CPU 스레드를 서로 다투지 않게 하려는 조치라 무시해도 됩니다(이 장의 출력에서는 뺐습니다).
- **timeout과 device_id**: 상대가 오지 않으면 영원히 기다리는 대신 정해진 시간 뒤에 오류를 냅니다. `device_id`는 이 프로세스가 쓸 GPU를 미리 알려 줘서 문제가 생겼을 때 멈추지 않고 오류로 끝나게 도와요. 17.12절에서 왜 중요한지 봅니다.
- **barrier()**: 모든 프로세스가 여기까지 올 때까지 기다리는 약속 장소입니다. `차례로_말하기`는 이걸로 rank 순서대로 한 줄씩 출력하게 했어요. 프로세스들은 동시에 달리기 때문에, 그냥 `print`하면 출력 순서가 실행할 때마다 뒤섞입니다.
- **destroy_process_group()**: 끝날 때 통신 자원을 정리합니다. 빼먹으면 종료할 때 경고가 나와요.

GPU가 없으면 `gloo` 백엔드로 CPU 프로세스끼리 통신합니다. 이 책의 맥에서 같은 명령을 실행하니 백엔드가 `gloo`, 장치가 `cpu`로 바뀌었을 뿐 숫자는 똑같았어요.

컴퓨터 여러 대로 넓히려면

지금은 컴퓨터 한 대 안의 GPU 두 장이지만, 같은 코드가 컴퓨터 여러 대로도 그대로 넓어집니다. 달라지는 것은 `torchrun`에 주는 값뿐이에요. `--nnodes`에 컴퓨터(노드) 수를, `--node_rank`에 이 컴퓨터의 번호를 주고, `--master_addr`에는 127.0.0.1 대신 0번 컴퓨터의 네트워크 주소를 줍니다. 그러면 `torchrun`이 컴퓨터마다 GPU 수만큼 프로세스를 띄우고, 전체 rank를 0부터 차례로 매겨요. 코드에서 `LOCAL_RANK`(이 컴퓨터 안의 번호)로 GPU를 고르고 `RANK`(전체 번호)로 rank 0을 가린 것도 이 때문입니다. 컴퓨터 한 대에서는 두 값이 같아서 구분이 안 되지만, 두 대째부터는 달라지거든요.

다만 컴퓨터 사이의 네트워크는 17.6절에서 볼 GPU 사이 연결보다 대개 더 느립니다. 그래서 여러 대로 넓히는 일은 전용 고속 네트워크가 깔린 GPU 클러스터에서 주로 하고, 이 책에서는 컴퓨터 한 대까지만 확인했습니다.

Q. 예전 코드에서 `nn.DataParallel`을 봤는데, 그건 뭔가요? 프로세스 하나가 스레드로 여러 GPU를 부리는 예전 방식입니다. `model = nn.DataParallel(model)` 한 줄이면 돼서 간단해 보이지만, 매 걸음 입력을 GPU 0에서 나눠 보내고 결과를 다시 GPU 0으로 모으느라 GPU 0만 바쁘고, 1권 15장에서 본 GIL 때문에 파이썬 쪽도 막힙니다. PyTorch 문서도 컴퓨터 한 대에서 여러 GPU를 쓸 때조차 `DistributedDataParallel`을 쓰라고 권해요. 이 장에서 DDP만 다루는 이유입니다.

17.6 통신은 공짜가 아니다

친구를 불렀으면 "다 옮겼어?"를 주고받는 데 얼마나 걸리는지 알아야 합니다. 이 서버에서 all-reduce가 크기별로 얼마나 걸리는지 재 보겠습니다. 비교를 위해 같은 GPU 안에서 256MB를 복사하는 시간도 함께 쟀어요.

```
import os
import statistics
import time
from datetime import timedelta

import torch
import torch.distributed as dist

assert torch.cuda.is_available(), "이 예제는 NVIDIA GPU가 2개 있어야 합니다"
장치 = torch.device("cuda", int(os.environ["LOCAL_RANK"]))
torch.cuda.set_device(장치)
dist.init_process_group("nccl", timeout=timedelta(seconds=60), device_id=장치)
rank = dist.get_rank()
MB = 1024 ** 2


def 재기(일, 횟수: int = 10) -> float:
    """준비 운동 3번 뒤 횟수만큼 재서 중앙값(초)을 돌려준다."""
    for _ in range(3):
        일()
    torch.cuda.synchronize()
    시간들 = []
```

```
    for _ in range(횟수):
        시작 = time.perf_counter()
        일()
        torch.cuda.synchronize()
        시간들.append(time.perf_counter() - 시작)
    return statistics.median(시간들)


if rank == 0:
    print(f"GPU {dist.get_world_size()}개: {torch.cuda.get_device_name(장치)}")
for 크기 in (1, 4, 16, 64, 256):
    x = torch.ones(크기 * MB // 4, device=장치)                # float32 크기 MB짜리
    초 = 재기(lambda: dist.all_reduce(x))
    if rank == 0:
        print(f"all_reduce {크기:3} MB: {초 * 1000:7.2f} ms ({크기 / 1024 / 초:5.2f} GB/s)")

if rank == 0:                                                  # 비교: 같은 GPU 안에서 복사
    x = torch.ones(256 * MB // 4, device=장치)
    y = torch.empty_like(x)
    초 = 재기(lambda: y.copy_(x))
    print(f"GPU 안 복사 256 MB: {초 * 1000:7.2f} ms ({256 / 1024 / 초:5.0f} GB/s)")
dist.barrier()
dist.destroy_process_group()
```

```
GPU 2개: NVIDIA GeForce RTX 4090
all_reduce   1 MB:    0.56 ms ( 1.74 GB/s)
all_reduce   4 MB:    2.10 ms ( 1.86 GB/s)
all_reduce  16 MB:    8.19 ms ( 1.91 GB/s)
all_reduce  64 MB:   32.46 ms ( 1.93 GB/s)
all_reduce 256 MB:  127.29 ms ( 1.96 GB/s)
GPU 안 복사 256 MB:    0.59 ms (  424 GB/s)
```

이 책의 실행 환경에서 잰 값입니다. all-reduce는 크기에 거의 정비례해서 1MB에 0.56ms, 256MB에 127ms가 걸렸고, 속도로 따지면 초당 2GB 남짓입니다. 같은 GPU 안에서 256MB를 복사하는 데는 0.59ms, 200배 넘게 빨라요. GPU 안의 메모리는 고속도로이고, GPU 사이는 좁은 골목길인 셈입니다.

왜 이렇게 느릴까요? 이 서버의 구성 때문입니다.

항목	이 책의 서버
GPU	RTX 4090 24GB × 2
GPU 0의 자리	PCIe 4세대 8차선
GPU 1의 자리	PCIe 3세대 4차선 (이론상 한 방향 약 4GB/s)
GPU 사이 연결	NVLink 없음. CPU(PCIe 호스트 브리지)를 거쳐 통신 (`nvidia-smi topo -m`에서 `PHB`)

두 GPU가 주고받는 길이 좁은 쪽(GPU 1의 3세대 4차선)에 맞춰지니, 이론 한계도 초당 4GB 안팎이에요. 데이터센터용 GPU 서버는 GPU끼리 NVLink로 직접 이어서 이 길이 수십~수백 배 넓습니다. 여러분의 컴퓨터는 `nvidia-smi topo -m` 명령으로 GPU 사이 연결을 확인할 수 있어요.

이 숫자를 기억해 두세요. 10장에서 ResNet18의 파라미터가 44.6MB였죠(정답 10가지짜리로 바꾸면 42.7MB). DDP는 걸음마다 기울기 전체, 즉 이만큼을 all-reduce합니다. 이 서버에서는 약 20ms예요. 한 걸음의 계산이 20ms보다 훨씬 길면 통신 시간이 묻히고, 짧으면 통신이 발목을 잡습니다. 17.9절에서 실제로 확인합니다.

17.7 DistributedDataParallel로 학습하기

학습 코드에서 바뀌는 곳

이제 9장 9.5절의 옷 사진 CNN을 GPU 두 장으로 학습해 보겠습니다. 6장 6.11절의 학습 루프와 비교하면 바뀌는 곳이 다섯 군데뿐이에요.

```
import os
import time
from datetime import timedelta

import torch
import torch.distributed as dist
from torch import nn
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data import DataLoader
```

```
from torch.utils.data.distributed import DistributedSampler
from torchvision import datasets, transforms

데이터_폴더 = os.environ.get("BOOK_DATA_DIR", "data")


class 옷CNN(nn.Module):
    """9장 9.5절의 작은 CNN."""

    def __init__(self) -> None:
        super().__init__()
        self.특징 = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        )
        self.분류 = nn.Sequential(
            nn.Flatten(), nn.Linear(64 * 7 * 7, 128), nn.ReLU(), nn.Linear(128, 10),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.분류(self.특징(x))


def 분산_시작() -> torch.device:
    """torchrun이 넣어 준 환경 변수로 프로세스 그룹에 들어간다."""
    if torch.cuda.is_available():
        장치 = torch.device("cuda", int(os.environ["LOCAL_RANK"]))
        torch.cuda.set_device(장치)
        dist.init_process_group("nccl", timeout=timedelta(seconds=120), device_id=장치)
    else:
        장치 = torch.device("cpu")
        dist.init_process_group("gloo", timeout=timedelta(seconds=120))
    return 장치


def 옷_데이터(학습: bool) -> datasets.FashionMNIST:
```

```
    return datasets.FashionMNIST(데이터_폴더, train=학습, download=True,
                                 transform=transforms.ToTensor())


def 정확도(모델: nn.Module, 장치: torch.device) -> float:
    """시험 데이터 1만 장의 정확도 (rank 0에서만 부른다)."""
    모델.eval()
    맞힘 = 0
    with torch.no_grad():
        for x, y in DataLoader(옷_데이터(False), batch_size=1000):
            맞힘 += (모델(x.to(장치)).argmax(1) == y.to(장치)).sum().item()
    return 맞힘 / 10_000


장치 = 분산_시작()
rank, 전체 = dist.get_rank(), dist.get_world_size()
if rank == 0:
    옷_데이터(True), 옷_데이터(False)          # 내려받기는 한 프로세스만
dist.barrier()                               # 다 받을 때까지 나머지는 기다린다

학습셋 = 옷_데이터(True)
샘플러 = DistributedSampler(학습셋, shuffle=True, seed=0)   # 데이터를 프로세스 수만큼 나눈다
로더 = DataLoader(학습셋, batch_size=128, sampler=샘플러)    # GPU마다 128장, 합쳐서 256장
for 차례 in range(전체):
    if rank == 차례:
        print(f"[rank {rank}] 맡은 샘플 {len(샘플러):,}개, 배치 {len(로더)}개, "
              f"앞 번호 {list(샘플러)[:4]}", flush=True)
    dist.barrier()

torch.manual_seed(0)                         # 모든 프로세스가 같은 초깃값으로 시작
모델 = DDP(옷CNN().to(장치))                  # 기울기를 all_reduce로 평균 내 주는 포장지
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
손실함수 = nn.CrossEntropyLoss()

for 에폭 in range(1, 4):
    샘플러.set_epoch(에폭)                   # 에폭마다 섞는 순서를 바꾼다
```

```
    모델.train()
    합계 = torch.zeros(2, device=장치)        # [손실 합, 개수]
    시작 = time.perf_counter()
    for x, y in 로더:
        x, y = x.to(장치), y.to(장치)
        옵티마이저.zero_grad()
        손실 = 손실함수(모델(x), y)
        손실.backward()                       # 여기서 GPU끼리 기울기를 평균 낸다
        옵티마이저.step()
        합계[0] += 손실.detach() * len(y)
        합계[1] += len(y)
    dist.all_reduce(합계)                     # 모든 프로세스의 손실을 모은다
    if rank == 0:
        print(f"에폭 {에폭}: 학습 손실 {합계[0] / 합계[1]:.4f} "
              f"({int(합계[1]):,}장, {time.perf_counter() - 시작:.1f}초)")

if rank == 0:                                 # 저장과 평가는 한 프로세스만
    print(f"시험 정확도: {정확도(모델.module, 장치):.2%}")
    torch.save({"에폭": 3, "모델": 모델.module.state_dict(),
                "옵티마이저": 옵티마이저.state_dict()}, "ddp_옷CNN.pt")
    print("저장한 키:", list(torch.load("ddp_옷CNN.pt")["모델"])[:2], "...")
dist.barrier()
dist.destroy_process_group()
```

```
[rank 0] 맡은 샘플 30,000개, 배치 235개, 앞 번호 [36044, 57327, 21567, 9481]
[rank 1] 맡은 샘플 30,000개, 배치 235개, 앞 번호 [10678, 55074, 15479, 43095]
에폭 1: 학습 손실 0.5870 (60,000장, 0.9초)
에폭 2: 학습 손실 0.3526 (60,000장, 0.8초)
에폭 3: 학습 손실 0.3029 (60,000장, 0.8초)
시험 정확도: 88.54%
저장한 키: ['특징.0.weight', '특징.0.bias'] ...
```

바뀐 곳을 차례로 보겠습니다.

1. **내려받기는 한 프로세스만**: 두 프로세스가 동시에 같은 파일을 내려받으면 서로 덮어써서 파일이 깨질 수 있습니다. rank 0만 받고, 나머지는 `barrier()`에서 기다렸다가 받아 둔 파일을 읽어요.

2. **`DistributedSampler`**: 데이터를 프로세스 수만큼 겹치지 않게 나눕니다. 출력을 보면 6만 장을 3만 장씩 나눠 맡았고, 두 rank의 앞 번호가 서로 다르죠. GPU마다 128장씩이니 한 걸음에 합쳐서 256장, 에폭마다 235걸음입니다. 샘플러를 줄 때는 `DataLoader`의 `shuffle`을 쓰지 않아요. 섞는 일도 샘플러가 합니다.
3. **`DDP(모델)`**: 모델을 DDP로 감쌉니다. 감쌀 때 rank 0의 파라미터를 모두에게 복사해서 출발점을 맞추고, `backward()` 도중에 기울기를 all-reduce로 평균 내 줘요. 학습 루프의 `zero_grad`, `backward`, `step`은 한 글자도 바뀌지 않았습니다.
4. **`set_epoch(에폭)`**: 샘플러는 에폭 번호를 씨앗에 섞어서 순서를 정합니다. 이걸 부르지 않으면 **매 에폭 같은 순서**로 데이터를 보게 돼요. 오류도 경고도 없이 조용히 학습이 조금 나빠지는 흔한 실수입니다.
5. **출력·평가·저장은 rank 0만**: 그냥 두면 모든 프로세스가 같은 줄을 출력하고 같은 파일에 동시에 씁니다. 손실은 GPU마다 다르니 `all_reduce`로 모은 뒤 rank 0이 출력했어요(6만 장이 모두 집계된 게 보이죠).

저장할 때는 `모델.module.state_dict()`처럼 DDP 포장지 안의 원래 모델을 꺼내서 저장했습니다. 그래야 키 이름이 `특징.0.weight`처럼 깔끔하게 나와서, DDP 없이 GPU 하나나 CPU에서 그대로 불러올 수 있어요. `모델.state_dict()`로 저장하면 키마다 `module.`이 붙어서 6장 방식으로 불러올 때 이름이 안 맞습니다. 다른 장치에서 불러올 때는 4장 4.9절의 `map_location`을 쓰고요.

Q. 이어서 학습하려면 체크포인트에 무엇을 더 넣어야 하나요? 자세한 것은 다음 18장 18.3절에서 다룹니다. 요점은 모델·옵티마이저·스케줄러·난수 상태를 담고, 다시 시작할 때 **모든 rank가** 그 파일을 읽어 같은 상태에서 출발해야 해요. 분산이라서 더할 것은 둘입니다. 샘플러에 `set_epoch`로 이어 갈 에폭 번호를 주는 것, 그리고 rank 0이 저장을 마치기 전에 다른 rank가 파일을 읽지 않도록 저장 뒤에 `barrier()`를 두는 것이에요. 위 코드의 마지막 `dist.barrier()`가 그 자리입니다.

시험 정확도 88.54%는 9장의 작은 CNN을 3에폭 학습한 결과로 그럴듯합니다. 실행마다 끝자리가 조금씩 다를 수 있어요. GPU의 일부 계산은 순서가 매번 조금씩 달라지기 때문입니다(8장 8.10절).

17.8 1 GPU와 같은 결과일까

DDP로 학습한 모델은 GPU 하나로 학습한 것과 같은 모델일까요? 같은 데이터, 같은 초깃값, 같은 **전체 배치 크기**라면 같아야 합니다. GPU 두 장이 16개씩 맡는 것과 GPU 하나가 32개를 한 번에 받는 것을 비교해 보겠습니다.

```
import os
from datetime import timedelta

import torch
import torch.distributed as dist
from torch import nn
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data import DataLoader, TensorDataset
from torch.utils.data.distributed import DistributedSampler

assert torch.cuda.is_available(), "이 예제는 NVIDIA GPU가 2개 있어야 합니다"
장치 = torch.device("cuda", int(os.environ["LOCAL_RANK"]))
torch.cuda.set_device(장치)
dist.init_process_group("nccl", timeout=timedelta(seconds=60), device_id=장치)
rank, 전체 = dist.get_rank(), dist.get_world_size()

torch.manual_seed(0)
데이터 = TensorDataset(torch.randn(256, 20), torch.randn(256, 1))   # 예제용으로 만든 데이터
전체_배치 = 32


def 새_모델() -> nn.Module:
    torch.manual_seed(1)
    return nn.Sequential(nn.Linear(20, 64), nn.ReLU(), nn.Linear(64, 1)).to(장치)


def 학습(모델: nn.Module, 로더: DataLoader, 분산: bool) -> list[float]:
    """한 에폭 동안 걸음마다 (전체 배치의) 손실을 기록한다."""
    옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.1)
    기록 = []
    for x, y in 로더:
        옵티마이저.zero_grad()
        손실 = nn.functional.mse_loss(모델(x.to(장치)), y.to(장치))
        손실.backward()
        옵티마이저.step()
        손실 = 손실.detach()
        if 분산:
```

```
            dist.all_reduce(손실)          # 두 GPU의 손실 평균 = 전체 배치의 손실
            손실 /= 전체
        기록.append(round(손실.item(), 5))
    return 기록


# 2 GPU: GPU마다 16개씩, 합쳐서 32개
샘플러 = DistributedSampler(데이터, shuffle=False)
ddp = DDP(새_모델())
ddp_기록 = 학습(ddp, DataLoader(데이터, batch_size=전체_배치 // 전체, sampler=샘플러), 분산=True)

if rank == 0:                              # 비교용: 1 GPU에서 32개씩
    하나 = 새_모델()
    하나_기록 = 학습(하나, DataLoader(데이터, batch_size=전체_배치), 분산=False)
    print("2 GPU × 16개:", ddp_기록)
    print("1 GPU × 32개:", 하나_기록)
    차이 = max((a - b).abs().max().item()
               for a, b in zip(ddp.module.parameters(), 하나.parameters()))
    print(f"8걸음 뒤 파라미터 최대 차이: {차이:.1e}")
dist.barrier()
dist.destroy_process_group()
```

```
2 GPU × 16개: [1.21992, 1.15777, 1.85537, 1.33644, 0.62719, 0.9723, 0.89722, 1.04313]
1 GPU × 32개: [1.21992, 1.15777, 1.85537, 1.33644, 0.62719, 0.9723, 0.89722, 1.04313]
8걸음 뒤 파라미터 최대 차이: 3.7e-08
```

여덟 걸음의 손실이 다섯째 자리까지 똑같고, 파라미터 차이는 3.7e-08로 `float32`의 끝자리 오차뿐입니다. 17.2절의 그래디언트 누적과 같은 원리예요. 16개짜리 평균 기울기 둘을 다시 평균 내면 32개짜리 평균 기울기가 되니까요. **DDP는 GPU 여러 장에 걸친 그래디언트 누적**이라고 생각해도 됩니다. 누적은 시간 순서로 나눠 담고, DDP는 GPU별로 나눠 담는 거죠.

샘플러에 `shuffle=False`를 줬기 때문에 비교가 정확히 맞아떨어졌습니다. `DistributedSampler`는 번호를 rank 0에 0, 2, 4, ..., rank 1에 1, 3, 5, ...처럼 번갈아 나눠 줘서, 첫 걸음에 두 GPU가 받은 32개가 GPU 하나가 받은 0~31번과 같은 묶음이 됐어요.

여기서 실무에서 자주 헷갈리는 점이 하나 나옵니다. 실습 코드를 GPU 하나에서 배치 32로 쓰다가 그대로 GPU 두 장에 올리면, GPU마다 32개씩 **전체 배치가 64**가 됩니다. 같은 에폭 동안 걸음 수는

절반이 되고 한 걸음의 평균 기울기는 더 매끈해져요. 그러면 같은 학습률로는 결과가 달라집니다. GPU를 늘릴 때는 둘 중 하나를 정하세요.

- **전체 배치를 유지한다**: GPU당 배치를 GPU 수로 나눈다. 결과가 GPU 하나와 같다(이 절과 17.13절 실습의 방식).
- **전체 배치를 키운다**: 처리량을 최대로 쓰는 대신 학습률을 다시 맞춘다. 배치가 k배면 학습률도 k배 가까이 키우고 처음 몇 걸음은 작게 시작하는(워밍업, 8장 8.7절의 스케줄러) 방식이 흔히 쓰이지만, 언제나 통하는 공식은 아니니 검증 성적으로 확인해야 해요.

17.9 GPU를 늘리면 몇 배 빨라질까

두 배일까요? 17.6절의 통신 비용을 생각하면 그렇지 않을 것 같습니다. 모델과 입력 크기를 바꿔 가며 GPU 하나와 두 장의 처리량(1초에 학습하는 사진 수)을 비교해 보겠습니다. GPU당 배치는 고정하고, 입력은 GPU 안에서 만든 난수라서 데이터 로더나 디스크는 끼어들지 않아요. 순수하게 계산과 통신만 재는 셈입니다.

```
import os
import statistics
import time
from datetime import timedelta

import torch
import torch.distributed as dist
from torch import nn
from torch.nn.parallel import DistributedDataParallel as DDP
from torchvision.models import resnet18, resnet50

assert torch.cuda.is_available(), "이 예제는 NVIDIA GPU가 2개 있어야 합니다"
장치 = torch.device("cuda", int(os.environ["LOCAL_RANK"]))
torch.cuda.set_device(장치)
dist.init_process_group("nccl", timeout=timedelta(seconds=300), device_id=장치)
rank = dist.get_rank()


class 작은CNN(nn.Module):
    """9장의 옷CNN과 같은 구조 (파라미터 42만 개)."""
```

```
    def __init__(self) -> None:
        super().__init__()
        self.층들 = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Flatten(), nn.Linear(64 * 7 * 7, 128), nn.ReLU(), nn.Linear(128, 10))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.층들(x)


def 초당_장수(모델: nn.Module, 배치: int, 크기: int, 채널: int) -> float:
    """GPU 하나가 1초에 학습하는 사진 수 (준비 운동 3걸음 뒤 10걸음의 중앙값)."""
    x = torch.randn(배치, 채널, 크기, 크기, device=장치)        # 예제용 입력
    y = torch.randint(0, 10, (배치,), device=장치)
    옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.01)
    시간들 = []
    for 걸음 in range(13):
        torch.cuda.synchronize()
        시작 = time.perf_counter()
        옵티마이저.zero_grad()
        nn.functional.cross_entropy(모델(x), y).backward()
        옵티마이저.step()
        torch.cuda.synchronize()
        if 걸음 >= 3:
            시간들.append(time.perf_counter() - 시작)
    return 배치 / statistics.median(시간들)


실험들 = [  # (이름, 모델 만들기, GPU당 배치, 사진 크기, 채널)
    ("작은CNN 28px", 작은CNN, 256, 28, 1),
    ("ResNet18 64px", lambda: resnet18(num_classes=10), 128, 64, 3),
    ("ResNet18 224px", lambda: resnet18(num_classes=10), 128, 224, 3),
    ("ResNet50 224px", lambda: resnet50(num_classes=10), 64, 224, 3),
]
```

```
if rank == 0:
    print(f"{'모델':14} {'기울기 MB':>8} {'1 GPU':>7} {'2 GPU':>7} {'배율':>5}")
for 이름, 만들기, 배치, 크기, 채널 in 실험들:
    torch.manual_seed(0)
    모델 = 만들기().to(장치)
    MB = sum(p.numel() for p in 모델.parameters()) * 4 / 1024 ** 2
    하나 = 초당_장수(모델, 배치, 크기, 채널) if rank == 0 else 0.0   # GPU 0 혼자
    dist.barrier()
    둘 = 초당_장수(DDP(모델), 배치, 크기, 채널) * dist.get_world_size()  # 두 GPU 합계
    if rank == 0:
        print(f"{이름:14} {MB:8.1f} {하나:7,.0f} {둘:7,.0f} {둘 / 하나:5.2f}")
dist.barrier()
dist.destroy_process_group()
```

```
모델              기울기 MB   1 GPU   2 GPU    배율
작은CNN 28px          1.6 270,920 296,665  1.10
ResNet18 64px      42.7  17,768  10,517  0.59
ResNet18 224px     42.7   2,445   3,953  1.62
ResNet50 224px     89.8     790   1,195  1.51
```

이 책의 실행 환경에서 다른 프로그램이 GPU를 쓰지 않을 때 잰 값이고, 두 번 재서 거의 같은 값을 확인했습니다. 배율이 2배에 가까운 것이 하나도 없어요. 그리고 두 번째 줄은 GPU 두 장이 **오히려 0.59배, 더 느립니다.** 한 줄씩 해석해 보겠습니다.

- **작은CNN 28px (1.10배)**: 기울기가 1.6MB밖에 안 돼서 통신은 1ms 남짓입니다. 그런데 한 걸음의 계산도 1ms 정도로 아주 짧아요(256장 ÷ 초당 27만 장). 걸음마다 오가는 확인 절차가 계산만큼 길어서 이득이 거의 없습니다.
- **ResNet18 64px (0.59배)**: 기울기 42.7MB를 주고받는 데 이 서버에서 20ms 넘게 걸리는데, GPU 하나의 한 걸음 계산은 7ms 정도(128장 ÷ 초당 1만 7,768장)입니다. 7ms 일하고 20ms 기다리니 혼자 할 때보다 느려졌어요. 친구를 불렀더니 엘리베이터 앞 줄이 일하는 시간보다 길어진 겁니다.
- **ResNet18 224px (1.62배)**: 같은 모델, 같은 기울기 크기인데 사진이 커서 한 걸음 계산이 50ms 남짓으로 길어졌습니다. 통신 20ms가 상대적으로 작아지니 이득이 났어요.
- **ResNet50 224px (1.51배)**: 계산은 더 길지만 기울기도 89.8MB로 두 배라서 통신도 늘었습니다.

정리하면 GPU를 늘린 이득은 **"한 걸음의 계산 시간 대 통신 시간"** 의 비율로 정해집니다. 모델이 크고 입력이 크고 배치가 클수록 계산이 길어져 이득이 커지고, GPU 사이 길이 좁을수록 이득이 줄어요.

그런데 ResNet18 224px를 보면 계산 52ms에 통신 20ms를 더하면 72ms여야 하는데, 2 GPU의 한 걸음은 65ms 정도(256장 ÷ 초당 3,953장)입니다. 통신 일부가 계산 뒤에 숨은 거예요. DDP는 기울기를 25MB 안팎의 **버킷(bucket)** 으로 묶어 두고, backward가 뒤쪽 층의 기울기를 다 구하면 앞쪽 층을 계산하는 동안 그 버킷부터 미리 보냅니다. 앞 사람이 짐을 들고 계단을 오르는 동안 뒷사람이 다음 짐을 싸는 셈이죠. 그래서 계산이 길수록 통신이 더 많이 숨습니다.

Q

Q. 그럼 이 서버에서는 2 GPU를 쓰지 말아야 하나요? 모델에 따라 다릅니다. 위 표처럼 계산이 큰 학습에서는 1.5~1.6배 빨라지니 충분히 쓸 만해요. 계산이 작은 학습이라면 다음 절의 방법으로 통신 횟수를 줄이거나, 아예 GPU마다 **다른 실험**(예를 들어 학습률이 다른 두 실험)을 하나씩 돌리는 편이 낫습니다. 18장 18.5절의 하이퍼파라미터 탐색처럼 실험이 여러 개라면 이쪽이 통신 없이 정확히 2배예요.

17.10 no_sync: 누적과 DDP 합치기

ResNet18 64px처럼 통신이 병목일 때 쓸 수 있는 방법이 있습니다. 17.2절의 그래디언트 누적을 DDP와 합치는 거예요. 그런데 그냥 합치면 손해를 봅니다. DDP는 `backward()`를 부를 때마다 all-reduce를 하니까, 4번 누적하면 통신도 4번 하거든요. 사실 평균은 **누적이 끝난 뒤 한 번만** 내면 충분합니다. 장바구니를 다 채운 뒤 한 번만 계산대에 가면 되는 것처럼요. DDP의 `no_sync()` 안에서 부른 `backward()`는 기울기를 쌓기만 하고 통신을 건너뜁니다.

```
import os
import statistics
import time
from contextlib import nullcontext
from datetime import timedelta

import torch
import torch.distributed as dist
from torch import nn
from torch.nn.parallel import DistributedDataParallel as DDP
from torchvision.models import resnet18
```

```
assert torch.cuda.is_available(), "이 예제는 NVIDIA GPU가 2개 있어야 합니다"
장치 = torch.device("cuda", int(os.environ["LOCAL_RANK"]))
torch.cuda.set_device(장치)
dist.init_process_group("nccl", timeout=timedelta(seconds=300), device_id=장치)
rank, 전체 = dist.get_rank(), dist.get_world_size()

누적 = 4
torch.manual_seed(rank)                                  # GPU마다 다른 예제용 입력
조각들 = [(torch.randn(128, 3, 64, 64, device=장치), torch.randint(0, 10, (128,), device=장치))
        for _ in range(누적)]


def 한_걸음(모델: DDP, 옵티마이저: torch.optim.Optimizer, no_sync: bool) -> None:
    """작은 배치 4개의 기울기를 모은 뒤 한 번 고친다."""
    옵티마이저.zero_grad()
    for i, (x, y) in enumerate(조각들):
        마지막 = i == 누적 - 1
        문맥 = 모델.no_sync() if no_sync and not 마지막 else nullcontext()
        with 문맥:                                       # no_sync 안에서는 all_reduce를 건너뛴다
            (nn.functional.cross_entropy(모델(x), y) / 누적).backward()
    옵티마이저.step()


def 새_모델() -> tuple[DDP, torch.optim.Optimizer]:
    torch.manual_seed(0)
    모델 = DDP(resnet18(num_classes=10).to(장치))
    return 모델, torch.optim.SGD(모델.parameters(), lr=0.01)


for no_sync in (False, True):
    모델, 옵티마이저 = 새_모델()
    시간들 = []
    for 걸음 in range(13):
        torch.cuda.synchronize()
        시작 = time.perf_counter()
```

```
        한_걸음(모델, 옵티마이저, no_sync)
        torch.cuda.synchronize()
        if 걸음 >= 3:
            시간들.append(time.perf_counter() - 시작)
    장수 = 128 * 누적 * 전체 / statistics.median(시간들)
    if rank == 0:
        print(f"누적 {누적}번, no_sync {'쓺 ' if no_sync else '안 쓺'}: "
              f"한 걸음 {statistics.median(시간들) * 1000:4.0f} ms, 두 GPU 합계 {장수:6,.0f}장/초")


def 한_걸음_뒤(no_sync: bool) -> list[torch.Tensor]:
    """새 모델로 한 걸음만 고친 뒤의 파라미터."""
    모델, 옵티마이저 = 새_모델()
    한_걸음(모델, 옵티마이저, no_sync)
    return [p.detach().clone() for p in 모델.parameters()]


def 최대_차이(a: list[torch.Tensor], b: list[torch.Tensor]) -> float:
    return max((x - y).abs().max().item() for x, y in zip(a, b))


기준, 다시, 건너뜀 = 한_걸음_뒤(False), 한_걸음_뒤(False), 한_걸음_뒤(True)
if rank == 0:
    print(f"한 걸음 뒤 파라미터 최대 차이: 같은 방식 두 번 {최대_차이(기준, 다시):.1e}, "
          f"no_sync 쓴 쪽과 {최대_차이(기준, 건너뜀):.1e}")
dist.destroy_process_group()
```

```
누적 4번, no_sync 안 쓺: 한 걸음  100 ms, 두 GPU 합계 10,284장/초
누적 4번, no_sync 쓺 : 한 걸음   46 ms, 두 GPU 합계 22,033장/초
한 걸음 뒤 파라미터 최대 차이: 같은 방식 두 번 7.5e-09, no_sync 쓴 쪽과 7.5e-09
```

같은 누적 4번인데 `no_sync`를 쓰니 한 걸음이 100ms에서 46ms로 줄고, 처리량이 초당 1만 284장에서 2만 2,033장으로 두 배 넘게 늘었습니다. 통신을 4번에서 1번으로 줄인 덕분이에요. 17.9절에서 GPU 하나가 초당 1만 7,768장이었으니, 0.59배이던 2 GPU가 이제는 1.24배가 됐습니다. 전체 배치는 1,024장(128장 × 4번 × 2 GPU)으로 커졌으니 17.8절에서 말한 학습률 조정은 함께 생각해야 하고요.

결과도 확인했습니다. 한 걸음 고친 뒤의 파라미터 차이가 `no_sync`를 쓴 쪽과 7.5e-09로, 같은 방식을 두 번 돌렸을 때의 차이(7.5e-09)와 같아요. GPU 계산의 끝자리 흔들림 말고는 차이가 없다는 뜻입니다. 이렇게 "기준이 되는 흔들림"을 함께 재 두면, 차이가 버그 때문인지 실수 계산의 잡음인지 구분할 수 있습니다.

코드에서는 `no_sync()`를 **마지막 조각을 뺀** 나머지에만 씌운 것이 핵심입니다. 마지막 조각까지 씌우면 통신을 한 번도 안 해서 GPU마다 다른 기울기로 고치게 되고, 두 GPU의 모델이 조용히 갈라져요. `nullcontext()`는 "아무것도 하지 않는 `with`"라서, 조건에 따라 `with`를 쓰거나 안 쓰는 코드를 한 줄로 쓸 수 있게 해 줍니다.

17.11 FSDP: 모델까지 나눠 담기

모두가 같은 책을 들고 있을 필요는 없다

DDP는 GPU마다 모델 전체와 기울기 전체, 옵티마이저 상태 전체를 똑같이 들고 있습니다. 17.1절의 "파라미터의 4배"가 GPU마다 그대로 있는 거죠. 7B 모델이라면 GPU마다 104GB라서 DDP로는 아무리 GPU를 늘려도 학습할 수 없어요.

FSDP(Fully Sharded Data Parallel) 는 이 중복을 없앱니다. 두 사람이 백과사전 한 질을 나눠 들고 다니다가, 어떤 권이 필요한 순간에만 잠깐 빌려 보고 돌려주는 방식이에요. 평소에는 GPU마다 파라미터·기울기·옵티마이저 상태의 **1/GPU 수**만 갖고 있다가, 어떤 층을 계산할 차례가 되면 그 층의 파라미터를 모두에게서 모아(all-gather) 계산하고 바로 버립니다. 이 아이디어를 처음 널리 알린 것이 DeepSpeed라는 라이브러리의 **ZeRO**이고, 무엇까지 나누느냐에 따라 1단계(옵티마이저 상태), 2단계(+ 기울기), 3단계(+ 파라미터)로 부릅니다. FSDP는 3단계에 해당해요.

PyTorch 2.14의 `fully_shard`로 17.3절의 트랜스포머를 DDP와 FSDP로 각각 학습해 보겠습니다.

```
import os
import statistics
import time
from datetime import timedelta

import torch
import torch.distributed as dist
from torch import nn
from torch.distributed.fsdp import fully_shard
```

```
from torch.nn.parallel import DistributedDataParallel as DDP

assert torch.cuda.is_available(), "이 예제는 NVIDIA GPU가 2개 있어야 합니다"
장치 = torch.device("cuda", int(os.environ["LOCAL_RANK"]))
torch.cuda.set_device(장치)
dist.init_process_group("nccl", timeout=timedelta(seconds=300), device_id=장치)
rank = dist.get_rank()
GB = 1024 ** 3


def 새_모델() -> nn.Sequential:
    """인코더 층 12개 (파라미터 약 1억 5천만 개). 두 방식이 같은 초깃값으로 시작한다."""
    torch.manual_seed(0)
    return nn.Sequential(*[nn.TransformerEncoderLayer(1024, 16, 4096, dropout=0.0,
    ↪  batch_first=True)
                           for _ in range(12)]).to(장치)


def 감싸기(방식: str) -> nn.Module:
    모델 = 새_모델()
    if 방식 == "DDP":
        return DDP(모델)                    # GPU마다 모델 전체를 복사해 둔다
    for 층 in 모델:
        fully_shard(층)                     # 층마다 파라미터를 GPU 수만큼 나눠 맡는다
    return fully_shard(모델)


torch.manual_seed(rank)
x = torch.randn(8, 256, 1024, device=장치)          # 예제용 입력, GPU마다 다르게
for 방식 in ("DDP", "FSDP"):
    torch.cuda.empty_cache()
    torch.cuda.reset_peak_memory_stats()
    시작_메모리 = torch.cuda.memory_allocated()
    모델 = 감싸기(방식)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-4)
    손실들, 시간들 = [], []
```

```
    for _ in range(5):
        torch.cuda.synchronize()
        시작 = time.perf_counter()
        옵티마이저.zero_grad()
        손실 = 모델(x).pow(2).mean()
        손실.backward()
        옵티마이저.step()
        torch.cuda.synchronize()
        시간들.append(time.perf_counter() - 시작)
        손실 = 손실.detach()
        dist.all_reduce(손실)                          # 두 GPU 손실의 평균을 기록
        손실들.append(round(손실.item() / 2, 5))
    남은 = (torch.cuda.memory_allocated() - 시작_메모리) / GB   # 파라미터·기울기·Adam 상태
    if rank == 0:
        print(f"{방식:4}: GPU 하나에 남은 것 {남은:4.2f} GB, 최고
        ↪  {torch.cuda.max_memory_allocated() / GB:4.2f} GB, "
             f"한 걸음 {statistics.median(시간들[1:]) * 1000:4.0f} ms")
        print(f"      손실 {손실들}")
    del 모델, 옵티마이저
dist.destroy_process_group()
```

```
DDP : GPU 하나에 남은 것 2.83 GB, 최고 3.60 GB, 한 걸음  328 ms
      손실 [0.99999, 0.99971, 0.99929, 0.99889, 0.99851]
FSDP: GPU 하나에 남은 것 1.08 GB, 최고 2.28 GB, 한 걸음  580 ms
      손실 [0.99999, 0.99971, 0.99929, 0.99889, 0.99851]
```

학습이 끝난 뒤 GPU 하나에 남아 있는 것(파라미터·기울기·Adam 상태)이 DDP는 2.83GB, FSDP는 1.08GB입니다. 파라미터 1억 5천만 개 × 16바이트 = 2.25GB를 FSDP는 반으로 나눠 들었고, DDP는 그 2.25GB에 통신용 버킷(기울기 크기만큼, 약 0.56GB)까지 더 들고 있어요. 다섯 걸음의 손실이 두 방식에서 똑같으니, 같은 학습을 메모리만 덜 쓰고 한 겁니다.

대신 한 걸음이 328ms에서 580ms로 느려졌습니다. 층마다 파라미터를 모으고 기울기를 다시 나누느라 통신량이 DDP보다 많은데, 이 서버의 GPU 사이 길이 좁기 때문이에요. 그래서 FSDP는 **모델이 DDP로 안 들어갈 때** 꺼내는 도구입니다. 들어간다면 DDP가 더 빠르고 간단해요. GPU 사이 연결이 빠른 서버일수록 FSDP의 손해는 작아집니다.

FSDP로 저장할 때는 각 GPU가 조각만 갖고 있어서 `state_dict()`가 조각난 텐서(DTensor)로 나옵니

다. 하나의 파일로 모으거나 조각별로 저장하는 방법은 PyTorch의 `torch.distributed.checkpoint` 문서를 따르세요. 이 장에서는 거기까지 확인하지 않았습니다.

직접 짜지 않는 방법들

실무에서는 지금까지의 코드를 대신 써 주는 도구를 많이 씁니다. Hugging Face의 **Accelerate**는 학습 루프를 거의 그대로 둔 채 `accelerate launch` 명령 하나로 DDP나 FSDP로 실행하게 해 주고, `transformers`의 `Trainer`도 `torchrun`으로 실행하면 알아서 DDP로 학습합니다. 그래도 이 장에서 본 rank·샘플러·all-reduce·`no_sync`의 원리를 알고 있어야, 도구가 이상하게 느리거나 멈출 때 어디를 볼지 알 수 있어요.

17.12 흔한 오류와 대처

한 프로세스만 통신을 부르면

분산 학습에서 가장 무서운 오류는 오류 메시지가 **안 나오는** 것입니다. 통신은 모든 프로세스가 같은 순서로 같은 함수를 불러야 이뤄지는데, 한 프로세스만 부르면 그 프로세스는 상대가 올 때까지 하염없이 기다려요. 일부러 일으켜 보겠습니다. CPU끼리 통신하는 `gloo`로 만들었으니 어느 컴퓨터에서나 실행할 수 있어요.

```
import time
from datetime import timedelta

import torch
import torch.distributed as dist

# CPU끼리(gloo) 통신. 기다리는 한계를 5초로 짧게 잡는다
dist.init_process_group("gloo", timeout=timedelta(seconds=5))
rank = dist.get_rank()
x = torch.ones(1)

if rank == 0:                       # 실수: rank 0만 all_reduce를 부른다
    시작 = time.perf_counter()
    try:
        dist.all_reduce(x)
    except RuntimeError as e:
```

```
        print(f"[rank 0] {time.perf_counter() - 시작:.0f}초 만에 오류:", str(e).split("] ", 1)[-1])
        ↪ # 파일 위치는 떼고
else:
    time.sleep(8)                  # rank 1은 다른 일을 하느라 통신에 오지 않는다
    print("[rank 1] 끝", flush=True)
```

```
[rank 0] 5초 만에 오류: Timed out waiting 5000ms for recv operation to complete
[rank 1] 끝
```

rank 0만 `all_reduce`를 불렀고, rank 1은 오지 않았습니다. `timeout`을 5초로 정해 뒀기 때문에 rank 0은 5초 뒤 "받기를 기다리다 시간이 다 됐다"는 오류를 냈어요. 기본 시간 한계는 `nccl`이 10분, `gloo`가 30분이라서, `timeout`을 주지 않으면 이 프로그램은 오랫동안 아무 말 없이 멈춰 있습니다. GPU 사용률은 0%인데 프로세스는 살아 있는 상태요.

실제 코드에서는 이런 실수가 이렇게 생깁니다.

- `if rank == 0:` 안에서 평가하다가, 그 안에서 `all_reduce`나 DDP 모델의 forward처럼 **통신이 들어 있는 일**을 부른다. 평가는 `모델.module`처럼 DDP 포장지를 벗긴 모델로 한다(17.7절).
- 프로세스마다 배치 수가 다르다. `DistributedSampler`는 데이터 수가 나누어떨어지지 않으면 몇 개를 반복해서 배치 수를 맞춰 주지만, 직접 데이터를 나눴다면 한쪽이 먼저 끝나고 다른 쪽은 기다린다.
- 한 프로세스에서만 예외가 나서 죽었다. 남은 프로세스는 죽은 상대를 기다린다. `torchrun`은 한 프로세스가 죽으면 나머지도 정리하지만, 직접 띄운 프로세스라면 남아 있을 수 있다.

처음부터 서로를 못 찾을 때

이 장의 예제를 처음 이 서버에서 실행했을 때, 프로세스들이 서로를 찾지 못하고 멈췄습니다. 로그에는 이런 줄이 반복됐어요.

```
[c10d] The server socket on [::ffff:0.0.15.250]:34045 has timed out, will retry.
```

`0.0.15.250`이라는 이상한 주소가 보이죠? 이 서버의 호스트 이름이 숫자 `4090`인데, 이걸 주소로 해석하는 과정에서 정수 4090이 IP 주소 0.0.15.250(15 × 256 + 250 = 4090)으로 바뀌어 버린 거예요. `torchrun`은 주소를 따로 주지 않으면 호스트 이름으로 서로를 찾습니다. 그래서 이 장의 모든 명령에 `--master_addr=127.0.0.1`(이 컴퓨터 자신)을 줬어요. 앞의 멈춤 예제를 이 서버에서 실행했을 때도 `gloo`가 "호스트 이름을 주소로 바꿀 수 없어서 루프백 주소(127.0.0.1)를 쓴다"는 경고를 냈는

데, 같은 원인입니다. 컴퓨터 한 대에서만 돌린다면 `--master_addr=127.0.0.1`을 습관처럼 붙여 두면 이런 일을 피할 수 있어요.

그 밖의 흔한 문제

증상	원인	대처
`Address already in use`	같은 포트를 다른 실행(또는 죽지 않은 지난 실행)이 쓰는 중	`–master_port`를 다른 번호로. 남은 프로세스 정리
출력이 두 번씩 나온다	모든 rank가 출력함	`if rank == 0:` 안에서 출력
에폭마다 손실 곡선이 똑같은 모양	`set_epoch`을 안 불러 순서가 매번 같음	에폭마다 `샘플러.set_epoch(에폭)`
불러온 가중치의 키 이름이 안 맞음	`module.`이 붙은 DDP 모델을 저장함	`모델.module.state_dict()`로 저장
모든 프로세스가 GPU 0에 몰림	`torch.cuda.set_device`를 안 함	`LOCAL_RANK`로 장치를 정하고 `set_device`
`NVIDIA driver on your system is too old`	설치한 PyTorch의 CUDA 버전을 드라이버가 지원하지 않음	드라이버가 지원하는 CUDA용 빌드로 다시 설치 (부록 A.4절)
끝났는데 GPU 메모리가 차 있다	죽지 않은 프로세스가 남음	`nvidia-smi`로 남은 프로세스 번호(PID)를 보고 종료

표의 여섯째 줄은 16장 첫머리에서 본, 이 서버에서 실제로 겪은 오류입니다. 새 서버에서는 학습 코드보다 `nvidia-smi`의 CUDA 버전과 `torch.version.cuda`부터 확인하세요.

17.13 실습: 옷 분류기를 세 가지 방법으로 학습하기

이 장에서 배운 것을 한 파일에 모아 보겠습니다. 9장의 옷 사진 CNN을 **한 걸음에 반영하는 사진 수를 256장으로 고정**한 채 세 가지 방법으로 학습하고, 정확도·시간·메모리를 비교합니다.

1. GPU 하나로 256장씩
2. GPU 하나로 64장씩 4번 누적
3. GPU 두 장으로 128장씩 (DDP)

같은 파일을 python으로 실행하면 프로세스 하나로, torchrun으로 실행하면 분산 학습으로 돌아가게 만들었습니다. torchrun이 넣어 주는 RANK 환경 변수가 있는지로 구분해요. 7장 7.10절에서 본 것처럼 프로세스를 여럿 띄우는 프로그램이니 if __name__ == "__main__": 아래에서 시작합니다. 아래 코드를 b13_실습.py로 저장하세요. 뒤에서 이 파일의 부품을 다시 가져다 쓰니 이름을 그대로 두는 것이 좋아요.

```
"""17장 실습: 옷 사진 CNN을 1 GPU, 그래디언트 누적, DDP 2 GPU로 학습해 비교한다.

python으로 실행하면 프로세스 하나, torchrun으로 실행하면 분산 학습이 된다.
"""
import argparse
import os
import time
from contextlib import nullcontext
from datetime import timedelta

import torch
import torch.distributed as dist
from torch import nn
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data import DataLoader
from torch.utils.data.distributed import DistributedSampler
from torchvision import datasets, transforms

데이터_폴더 = os.environ.get("BOOK_DATA_DIR", "data")
전체_배치 = 256                                  # 한 걸음에 반영하는 사진 수는 언제나 256장


class 옷CNN(nn.Module):
    """9장 9.5절의 작은 CNN."""

    def __init__(self) -> None:
        super().__init__()
        self.특징 = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        )
```

```
        self.분류 = nn.Sequential(
            nn.Flatten(), nn.Linear(64 * 7 * 7, 128), nn.ReLU(), nn.Linear(128, 10),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.분류(self.특징(x))


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")


def 준비() -> tuple[torch.device, bool]:
    """torchrun으로 실행됐으면 분산을 시작하고, 아니면 장치만 고른다."""
    if "RANK" not in os.environ:
        return 장치_고르기(), False
    if torch.cuda.is_available():
        장치 = torch.device("cuda", int(os.environ["LOCAL_RANK"]))
        torch.cuda.set_device(장치)
        dist.init_process_group("nccl", timeout=timedelta(seconds=120), device_id=장치)
    else:
        장치 = torch.device("cpu")
        dist.init_process_group("gloo", timeout=timedelta(seconds=120))
    return 장치, True


def 옷_데이터(학습: bool) -> datasets.FashionMNIST:
    return datasets.FashionMNIST(데이터_폴더, train=학습, download=True,
                                 transform=transforms.ToTensor())
```

```
def 정확도(모델: nn.Module, 장치: torch.device) -> float:
    모델.eval()
    맞힘 = 0
    with torch.no_grad():
        for x, y in DataLoader(옷_데이터(False), batch_size=1000):
            맞힘 += (모델(x.to(장치)).argmax(1) == y.to(장치)).sum().item()
    return 맞힘 / 10_000


def 한_에폭(모델: nn.Module, 로더: DataLoader, 옵티마이저: torch.optim.Optimizer,
           장치: torch.device, 누적: int, 분산: bool) -> torch.Tensor:
    """한 에폭 학습하고 [손실 합, 개수]를 돌려준다. 배치를 누적 횟수만큼 나눠 기울기를 모은다."""
    모델.train()
    합계 = torch.zeros(2, device=장치)
    for x, y in 로더:
        x, y = x.to(장치), y.to(장치)
        옵티마이저.zero_grad()
        for i, (xb, yb) in enumerate(zip(x.chunk(누적), y.chunk(누적))):
            건너뛰기 = 분산 and i < 누적 - 1                  # 마지막 조각에서만 통신
            with 모델.no_sync() if 건너뛰기 else nullcontext():
                손실 = nn.functional.cross_entropy(모델(xb), yb)
                (손실 / 누적).backward()
            합계[0] += 손실.detach() * len(yb)
            합계[1] += len(yb)
        옵티마이저.step()
    return 합계


def main() -> None:
    인자 = argparse.ArgumentParser()
    인자.add_argument("--누적", type=int, default=1)
    인자.add_argument("--에폭", type=int, default=5)
    설정 = 인자.parse_args()

    장치, 분산 = 준비()
    rank = dist.get_rank() if 분산 else 0
```

```
    전체 = dist.get_world_size() if 분산 else 1
    if rank == 0:
        옷_데이터(True), 옷_데이터(False)              # 내려받기는 한 프로세스만
    if 분산:
        dist.barrier()

    학습셋 = 옷_데이터(True)
    GPU당 = 전체_배치 // 전체
    if 분산:
        샘플러 = DistributedSampler(학습셋, shuffle=True, seed=0)
        로더 = DataLoader(학습셋, batch_size=GPU당, sampler=샘플러)
    else:
        샘플러 = None
        로더 = DataLoader(학습셋, batch_size=GPU당, shuffle=True,
                          generator=torch.Generator().manual_seed(0))

    torch.manual_seed(0)
    모델 = 옷CNN().to(장치)
    if 분산:
        모델 = DDP(모델)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
    if 장치.type == "cuda":
        torch.cuda.reset_peak_memory_stats(장치)

    손실들 = []
    시작 = time.perf_counter()
    for 에폭 in range(설정.에폭):
        if 샘플러 is not None:
            샘플러.set_epoch(에폭)
        합계 = 한_에폭(모델, 로더, 옵티마이저, 장치, 설정.누적, 분산)
        if 분산:
            dist.all_reduce(합계)
        손실들.append(round((합계[0] / 합계[1]).item(), 4))
    걸린_시간 = time.perf_counter() - 시작

    if rank == 0:
```

```
        원본 = 모델.module if 분산 else 모델
        조각 = f" ({GPU당 // 설정.누적}장 × {설정.누적}번 누적)" if 설정.누적 > 1 else ""
        메모리 = (f"{torch.cuda.max_memory_allocated(장치) / 1024 ** 2:.0f} MB"
                if 장치.type == "cuda" else "-")
        print(f"[{장치.type} {전체}개 × {GPU당}장{조각}]")
        print(f"  에폭별 학습 손실 {손실들}")
        print(f"  시험 정확도 {정확도(원본, 장치):.2%}, 학습 {걸린_시간:.1f}초, GPU당 최고 메모리
        ↪ {메모리}")
    if 분산:
        dist.destroy_process_group()


if __name__ == "__main__":
    main()
```

세 가지 방법으로 실행합니다. `--누적` 같은 명령줄 인자는 표준 라이브러리의 `argparse`로 받았어요. `add_argument`로 옵션 이름과 자료형, 기본값을 정해 두면 `parse_args()`가 명령줄을 읽어 `설정.누적`처럼 꺼내 쓰게 해 줍니다. 설정을 명령줄로 바꾸는 방법은 다음 18장 18.2절에서 자세히 다뤄요.

```
uv run python b13_실습.py          # 1) GPU 하나, 256장씩
uv run python b13_실습.py --누적 4  # 2) GPU 하나, 64장씩 4번 누적
uv run torchrun --nproc_per_node=2 --master_addr=127.0.0.1 --master_port=29500 b13_실습.py   # 3)
↪ GPU 2개 × 128장
```

```
[cuda 1개 × 256장]
  에폭별 학습 손실 [0.5886, 0.3498, 0.2963, 0.2714, 0.2485]
  시험 정확도 89.43%, 학습 6.5초, GPU당 최고 메모리 104 MB
[cuda 1개 × 256장 (64장 × 4번 누적)]
  에폭별 학습 손실 [0.5891, 0.3505, 0.297, 0.2718, 0.2486]
  시험 정확도 89.56%, 학습 8.8초, GPU당 최고 메모리 45 MB
[cuda 2개 × 128장]
  에폭별 학습 손실 [0.5912, 0.3494, 0.3027, 0.2752, 0.2531]
  시험 정확도 89.76%, 학습 4.0초, GPU당 최고 메모리 66 MB
```

이 책의 실행 환경(RTX 4090 두 장)에서 잰 값입니다. 표로 정리하면 이렇습니다.

방법	시험 정확도	학습 시간 (5에폭)	GPU당 최고 메모리
GPU 1개 × 256장	89.43%	6.5초	104MB
GPU 1개 × (64장 × 4번 누적)	89.56%	8.8초	45MB
GPU 2개 × 128장	89.76%	4.0초	66MB

- **정확도**: 세 방법 모두 89%대로 비슷합니다. 1번과 2번은 같은 순서로 같은 256장을 봤으니 에폭별 손실이 0.5886과 0.5891, 0.2485와 0.2486처럼 거의 같아요. 17.2절에서는 CPU라 끝자리까지 같았는데, 여기서는 GPU 계산의 끝자리 흔들림이 다섯 에폭 동안 조금 쌓였습니다. 3번은 `DistributedSampler`가 데이터를 섞는 순서가 달라서 손실이 조금 다르게 흘렀지만 같은 수준에 도착했어요.
- **메모리**: 누적한 2번이 104MB에서 45MB로 가장 적습니다. 한 번에 64장만 올리니까요. DDP는 GPU마다 128장이라 그 중간이고요.
- **시간**: 누적은 6.5초에서 8.8초로 느려졌습니다. 이 작은 CNN에게 64장은 GPU를 바쁘게 만들기에 너무 적어서, 17.2절의 ResNet18과 달리 조각을 나눈 만큼 손해를 봤어요. DDP는 4.0초로 1.6배 빨라졌는데, 17.9절에서 작은CNN의 순수 계산은 1.1배밖에 안 빨라졌던 것과 다르죠? 이 실습은 GPU 안의 난수가 아니라 `DataLoader`가 CPU에서 사진을 꺼내 오는데, 프로세스가 둘이면 **데이터를 꺼내는 일도 둘이 나눠** 하기 때문입니다.

사진도 모델도 작아서, 이 실습에서는 GPU 계산보다 데이터를 꺼내는 쪽이 시간을 더 잡아먹고 있을 것 같아요. 짐작으로 끝내지 말고 재 보겠습니다. 실습의 부품을 그대로 가져와, 한 걸음을 "배치를 기다린 시간"과 "GPU로 옮겨 계산·통신을 끝낸 시간"으로 나눠 재는 코드입니다. `b13_실습.py`와 같은 폴더에 `b16_시간_나누기.py`로 저장하고, GPU 하나는 `uv run python b16_시간_나누기.py`, 두 개는 위의 3번과 같은 `uv run torchrun` 명령으로 실행했어요. GPU가 없으면 동기화를 건너뛰고 CPU나 MPS에서도 돌아갑니다.

```
"""b13 실습의 한 걸음을 '데이터 기다림'과 'GPU 계산·통신'으로 나눠 잰다."""
import time

import torch
import torch.distributed as dist
from torch import nn
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data import DataLoader
```

```
from torch.utils.data.distributed import DistributedSampler

from b13_실습 import 옷_데이터, 옷CNN, 전체_배치, 준비      # 실습의 부품을 그대로 쓴다

에폭_수 = 5


def main() -> None:
    장치, 분산 = 준비()
    rank = dist.get_rank() if 분산 else 0
    전체 = dist.get_world_size() if 분산 else 1
    if rank == 0:
        옷_데이터(True)
    if 분산:
        dist.barrier()
    학습셋 = 옷_데이터(True)
    GPU당 = 전체_배치 // 전체
    샘플러 = DistributedSampler(학습셋, shuffle=True, seed=0) if 분산 else None
    로더 = DataLoader(학습셋, batch_size=GPU당, sampler=샘플러, shuffle=샘플러 is None)
    torch.manual_seed(0)
    모델 = 옷CNN().to(장치)
    if 분산:
        모델 = DDP(모델)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
    손실_함수 = nn.CrossEntropyLoss()
    데이터_시간 = 계산_시간 = 0.0
    걸음 = 0
    시작 = time.perf_counter()
    for 에폭 in range(에폭_수):
        if 샘플러 is not None:
            샘플러.set_epoch(에폭)
        t0 = time.perf_counter()
        for x, y in 로더:
            t1 = time.perf_counter()                  # 여기까지가 배치를 기다린 시간
            x, y = x.to(장치), y.to(장치)
            옵티마이저.zero_grad()
```

```
            손실_함수(모델(x), y).backward()
            옵티마이저.step()
            if 장치.type == "cuda":                # GPU 일이 정말 끝날 때까지 (16장 16.2절)
                torch.cuda.synchronize()
            elif 장치.type == "mps":
                torch.mps.synchronize()
            t2 = time.perf_counter()                # 여기까지가 계산과 통신 시간
            데이터_시간 += t1 - t0
            계산_시간 += t2 - t1
            걸음 += 1
            t0 = time.perf_counter()
    전체_시간 = time.perf_counter() - 시작
    if rank == 0:
        print(f"[GPU {전체}개 × {GPU당}장] 걸음 {걸음}번, 전체 {전체_시간:.1f}초 = "
              f"데이터 기다림 {데이터_시간:.1f}초 + GPU 계산·통신 {계산_시간:.1f}초 "
              f"(걸음당 데이터 {데이터_시간 / 걸음 * 1000:.1f}ms, 계산 {계산_시간 / 걸음 *
              ↪ 1000:.1f}ms)")
    if 분산:
        dist.destroy_process_group()


if __name__ == "__main__":
    main()
```

```
[GPU 1개 × 256장] 걸음 1175번, 전체 6.8초 = 데이터 기다림 5.4초 + GPU 계산·통신 1.5초 (걸음당 데이
↪ 터 4.6ms, 계산 1.3ms)
[GPU 2개 × 128장] 걸음 1175번, 전체 4.9초 = 데이터 기다림 2.7초 + GPU 계산·통신 2.2초 (걸음당 데이
↪ 터 2.3ms, 계산 1.9ms)
```

짐작이 맞았습니다. GPU 하나일 때 6.8초 중 5.4초, 거의 4분의 3이 **배치를 기다린 시간**이에요. GPU는 걸음마다 1.3ms만 일하고 4.6ms를 놀았습니다. 프로세스가 둘이 되자 각자 절반의 사진만 꺼내니 기다림이 2.7초로 절반이 됐고, 대신 계산·통신 시간은 1.5초에서 2.2초로 늘었어요. 17.6절의 통신 비용이 걸음마다 붙었기 때문입니다. 늘어난 0.7초보다 줄어든 2.7초가 커서 전체로는 빨라진 거예요. 걸음마다 `synchronize`로 기다리게 해서 위 표보다 조금씩 느리게 나오지만, 무엇이 시간을 잡아먹는지는 분명하죠. 그렇다면 GPU를 늘리기 전에 GPU 하나에서 7장 7.10절의 `num_workers`부터 늘려 보는 게 순서입니다. 16장 16.13절 실습에서 일꾼 8명으로 5.2배 빨라진 것과 같은 이야기예요.

이렇게 숫자를 나란히 놓고 보면 "GPU를 늘리면 빨라진다", "누적하면 느려진다" 같은 한 줄 요약이 모두 반쯤만 맞는다는 걸 알 수 있습니다. 어디가 병목인지(계산, 통신, 데이터)를 재 보고 고르는 것이 이 장에서 가져갈 가장 중요한 습관이에요.

17.14 정리와 다음 장 미리보기

이 장에서는 GPU 메모리를 아끼는 법과 GPU를 늘리는 법을 직접 재 가며 익혔습니다.

- **학습 메모리의 내역**: 파라미터 + 기울기 + 옵티마이저 상태 + 활성화. Adam이면 활성화를 빼고도 파라미터의 4배(float32 기준 파라미터당 16바이트). 메모리 오류는 가장 높은 순간에 난다
- **그래디언트 누적**: 손실을 누적 횟수로 나누고, step·zero_grad는 누적이 끝난 뒤 한 번. 큰 배치와 같은 기울기(배치 정규화 제외), 활성화 메모리가 조각 크기만큼 준다
- **활성화 체크포인팅**: 층의 입력만 저장하고 backward 때 다시 계산. 이 장의 트랜스포머에서 메모리 7.78GB → 2.43GB, 시간 +34%, 기울기는 같다
- **분산의 낱말**: GPU마다 프로세스 하나, rank·world size, 백엔드(`nccl`/`gloo`), all-reduce, barrier. `uv run torchrun --nproc_per_node=N --master_addr=127.0.0.1`
- DDP: `DistributedSampler`와 `set_epoch`, `DDP(모델)`, 출력·평가·저장은 rank 0만, 저장은 `모델.module.state_dict()`. 같은 전체 배치면 1 GPU와 같은 결과
- **통신 비용**: 이 책의 서버(NVLink 없음, PCIe 3세대 4차선)에서 all-reduce는 초당 약 2GB. 이득은 계산 시간 대 통신 시간으로 정해져서, 작은 계산에서는 2 GPU가 오히려 느렸다(0.59배). DDP는 버킷으로 통신을 계산 뒤에 숨긴다
- **no_sync**: 누적과 DDP를 합칠 때 마지막 조각에서만 통신. 이 장에서 처리량이 두 배 넘게 늘었다
- FSDP·ZeRO: 파라미터·기울기·옵티마이저 상태까지 나눠 담아 GPU당 메모리를 줄이고, 그 대가로 통신이 늘어난다. 텐서 병렬·파이프라인 병렬은 거대 모델용
- **흔한 오류**: 한 프로세스만 통신을 부르면 멈춘다(`timeout`으로 오류로 바꾸기), 호스트 이름 문제는 `--master_addr`, 드라이버와 CUDA 빌드 맞추기

GPU를 여러 장 쓰기 시작하면 실험 하나하나가 비싸집니다. 몇 시간짜리 학습이 중간에 끊기기도 하고, 같은 설정을 GPU 개수만 바꿔 다시 돌리기도 하죠. 그럴수록 "그 결과가 어떤 설정, 어떤 코드, 어떤 환경에서 나왔는지"를 남겨 두는 일이 중요해집니다. 다음 18장에서는 설정을 코드 밖으로 빼고, 체크포인트에서 똑같이 이어 가고, 실험 기록을 남기고 비교하는 **실험 관리와 재현성**을 다룹니다.

18

실험 관리와 재현성

학습 목표

- 실험 하나를 이루는 것(설정, 코드, 데이터, 환경, 난수, 결과)을 나열하고 왜 모두 남겨야 하는지 설명하기
- 설정을 dataclass·YAML·명령줄 인자로 코드 밖에 두고, 우선순위에 따라 덮어쓰기
- 옵티마이저·스케줄러·난수 상태까지 담은 체크포인트로 중단된 학습을 똑같이 이어 가기
- JSON Lines, TensorBoard, MLflow로 실험 기록을 남기고 꺼내 비교하기
- Optuna로 하이퍼파라미터를 자동으로 탐색하고, 가망 없는 시도를 중간에 멈추기
- 여러 씨앗으로 돌려 평균과 표준편차로 보고하고, 같은 씨앗끼리 짝지어 차이를 판단하기
- 데이터·코드·환경의 지문을 남기고 재현성 보고서를 만들기

들어가며: 그 86%는 어떤 설정이었을까요?

석 달 전에 만든 모델이 86% 남짓을 냈다고 해 봅시다. 팀장님이 묻습니다. "그때 학습률이 얼마였죠? 드롭아웃은 넣었나요? 데이터는 지금 거랑 같은 거예요?" 폴더를 열어 보면 이런 파일들이 보입니다.

```
model.pt
model_final.pt
```

```
model_final_v2.pt
model_final_v2_lr0.001_진짜최종.pt
train.py
train_copy.py
결과정리(수정).xlsx
```

어느 파일이 86%를 냈는지, 그때 `train.py`가 지금의 `train.py`와 같았는지 아무도 모릅니다. 요리로 치면 맛있게 끓인 찌개의 레시피를 적어 두지 않은 셈이에요. 한 번 맛있게 끓였다는 기억만 남고, 다시 끓이면 그 맛이 안 납니다.

이 책에서도 비슷한 일을 이미 겪었어요. 6장 6.11절에서 같은 모델, 같은 씨앗으로 학습했는데 한 번은 86.26%, 한 번은 86.07%가 나왔죠. 평가를 언제 했느냐에 따라 데이터를 섞는 순서가 달라졌기 때문이었습니다. 8장 8.10절에서는 씨앗을 고정하는 체크리스트를 만들었고요. 이 장은 그 체크리스트를 **실험 전체를 관리하는 습관**으로 넓힙니다.

이 장에서 할 일은 크게 세 가지입니다. 첫째, 설정·체크포인트·기록을 남겨서 **같은 실험을 다시 할 수 있게** 만듭니다. 둘째, 하이퍼파라미터 탐색과 여러 씨앗 실험을 자동으로 돌려서 **실험을 많이, 체계적으로** 합니다. 셋째, 데이터·코드·환경의 지문을 붙인 보고서로 **결과를 믿을 수 있게** 만듭니다.

Q

Q. 이 장에 필요한 패키지는 무엇인가요? PyTorch와 torchvision에 더해 `uv add tensorboard mlflow optuna pyyaml scikit-learn`을 실행하세요. 이 장의 예제는 파이썬 3.12, PyTorch 2.14.1, torchvision 0.29.1, TensorBoard 2.21.0, MLflow 3.17.2, Optuna 5.0.0, PyYAML 6.0.3, scikit-learn 1.9.1에서 실행해 확인했습니다. 결과를 똑같이 재현하려고 학습은 모두 CPU에서 했어요. 기록 파일과 데이터베이스는 예제를 실행한 폴더 안에 만들어집니다.

18.1 실험 하나를 이루는 것

정확도 86%라는 숫자 하나 뒤에는 무엇이 숨어 있을까요? 같은 숫자를 다시 얻으려면 무엇이 같아야 하는지 하나씩 꼽아 보면 이렇습니다.

구성 요소	예	바뀌면 생기는 일	이 장에서 남기는 방법
설정	학습률, 배치 크기, 에폭, 드롭아웃	결과가 바로 달라진다	dataclass·YAML (18.2절)

구성 요소	예	바뀌면 생기는 일	이 장에서 남기는 방법
코드	모델 구조, 학습 루프, 전처리	같은 설정이어도 다른 실험이 된다	git 커밋, 파일 지문 (18.7절)
데이터	어떤 파일을 어떻게 나눴나	점수의 기준 자체가 바뀐다	데이터 지문 (18.7절)
환경	파이썬·PyTorch 버전, CPU·GPU	끝자리가 달라지거나 오류가 난다	환경 기록, `uv.lock` (18.7절)
난수	씨앗, 섞는 순서, 드롭아웃	같은 설정이어도 1%p쯤 흔들린다	씨앗 고정, 여러 씨앗 (18.3절, 18.6절)
결과	에폭별 손실·정확도, 모델 파일	무엇이 좋았는지 비교할 수 없다	기록 도구, 체크포인트 (18.3절, 18.4절)

여섯 가지 중 하나라도 빠지면 그 실험은 "한 번 그랬던 일"이 됩니다. 과학 실험실에서 실험 노트를 쓰는 이유와 같아요. 노트에는 결과만 적지 않고 시약의 제조 번호, 온도, 날짜를 함께 적습니다. 그래야 결과가 이상할 때 원인을 거꾸로 짚어 갈 수 있으니까요.

재현성에도 단계가 있습니다. 가장 쉬운 것은 **내 컴퓨터에서 같은 코드를 다시 돌리면 같은 숫자가 나오는 것**입니다. 씨앗과 섞는 순서를 고정하면 됩니다. 그다음은 **다른 컴퓨터에서 돌려도 비슷한 숫자가 나오는 것**이에요. 8장 8.10절에서 본 것처럼 CPU와 GPU, PyTorch 버전이 다르면 끝자리가 달라질 수 있어서, 환경을 기록해 두고 "비슷한"의 범위를 알아야 합니다. 가장 어려운 것은 **다른 사람이 같은 결론에 이르는 것**입니다. 씨앗 하나에서 우연히 나온 차이를 "개선"이라고 보고하면, 다른 사람이 다시 해 봤을 때 그 개선이 사라져요. 이 장의 뒤쪽에서 이 세 번째 단계를 위해 여러 씨앗으로 실험하는 법을 다룹니다.

18.2 설정을 코드 밖으로

dataclass로 설정 묶기

설정값이 코드 곳곳에 흩어져 있으면 무엇을 바꿨는지 추적하기 어렵습니다. 학습률은 옵티마이저를 만드는 줄에, 배치 크기는 `DataLoader`를 만드는 줄에, 에폭은 `for` 문에 숨어 있죠. 먼저 이 값들을 **한 상자에** 모으겠습니다. 1권 8장의 `dataclass`가 딱 맞는 그릇이에요.

```
import json
from dataclasses import asdict, dataclass, fields, replace


@dataclass(frozen=True)
class 학습설정:
    """실험 하나를 정하는 값들. 기본값이 곧 '기준 실험'이다."""
    모델: str = "mlp"
    학습률: float = 1e-3
    배치_크기: int = 64
    에폭: int = 2
    드롭아웃: float = 0.0
    씨앗: int = 0


기준 = 학습설정()
print(기준)

# 기준에서 한두 개만 바꾼 실험들: 원본은 그대로 두고 새 설정을 만든다
후보들 = [replace(기준, 학습률=학습률) for 학습률 in (3e-4, 1e-3, 3e-3)]
후보들.append(replace(기준, 모델="cnn", 드롭아웃=0.3))
for 설정 in 후보들:
    print(설정.모델, 설정.학습률, 설정.드롭아웃)

try:
    기준.학습률 = 0.1                    # frozen=True: 실험 도중에 몰래 바꿀 수 없다
except Exception as e:
    print("바꾸기 실패:", type(e).__name__)

# 딕셔너리로 바꾸면 JSON으로 저장할 수 있다
글 = json.dumps(asdict(후보들[-1]), ensure_ascii=False)
print(글)
다시 = 학습설정(**json.loads(글))
print("저장했다 불러와도 같은가:", 다시 == 후보들[-1])
print("필드 이름:", [f.name for f in fields(학습설정)])
```

```
학습설정(모델='mlp', 학습률=0.001, 배치_크기=64, 에폭=2, 드롭아웃=0.0, 씨앗=0)
mlp 0.0003 0.0
mlp 0.001 0.0
mlp 0.003 0.0
cnn 0.001 0.3
바꾸기 실패: FrozenInstanceError
{"모델": "cnn", "학습률": 0.001, "배치_크기": 64, "에폭": 2, "드롭아웃": 0.3, "씨앗": 0}
저장했다 불러와도 같은가: True
필드 이름: ['모델', '학습률', '배치_크기', '에폭', '드롭아웃', '씨앗']
```

기본값으로 만든 `학습설정()`이 곧 **기준 실험**입니다. 다른 실험은 `dataclasses.replace`로 기준에서 한두 값만 바꿔 만들었어요. `replace`는 원본을 고치지 않고 바뀐 값을 가진 새 설정을 돌려줍니다. 그래서 "기준 대비 무엇을 바꿨는가"가 코드에 그대로 드러나요.

`frozen=True`도 눈여겨보세요. 설정을 만든 뒤에는 값을 바꿀 수 없게 막아 둡니다. 학습 도중에 누군가 `설정.학습률 = 0.1`을 실행하면, 기록에 남은 학습률과 실제로 쓴 학습률이 달라지는 사고가 나거든요. 바꾸려 하면 `FrozenInstanceError`가 납니다.

마지막 부분은 `asdict`로 설정을 딕셔너리로 바꿔 JSON 글자로 만들고, 그 글자에서 다시 설정을 만든 것입니다. 둘이 `==`로 같다고 나왔어요. 설정을 파일로 저장했다가 그대로 되살릴 수 있다는 뜻이고, 이것이 실험 기록의 첫걸음입니다.

YAML 파일에서 읽기

실험을 바꿀 때마다 파이썬 코드를 고치는 대신 **설정 파일**만 바꾸면 편합니다. 딥러닝 프로젝트에서는 사람이 읽고 쓰기 쉬운 YAML 형식을 많이 써요. `이름: 값`을 한 줄에 하나씩 적고, `#` 뒤는 주석입니다. PyYAML 패키지의 `yaml.safe_load`로 읽으면 딕셔너리가 됩니다.

```
from dataclasses import dataclass, fields
from pathlib import Path

import yaml



@dataclass(frozen=True)
class 학습설정:
    모델: str = "mlp"
```

```
    학습률: float = 1e-3
    배치_크기: int = 64
    에폭: int = 2
    드롭아웃: float = 0.0
    씨앗: int = 0


Path("설정_cnn.yaml").write_text("""\
# 9장 CNN에 드롭아웃을 더한 실험
모델: cnn
학습률: 3e-4
드롭아웃: 0.3
""", encoding="utf-8")

읽은값 = yaml.safe_load(Path("설정_cnn.yaml").read_text(encoding="utf-8"))
print(읽은값)
print("학습률의 자료형:", type(읽은값["학습률"]).__name__)


def 설정_만들기(값들: dict) -> 학습설정:
    """딕셔너리를 검사해서 설정으로 바꾼다. 모르는 이름과 틀린 자료형을 잡아낸다."""
    필드 = {f.name: f.type for f in fields(학습설정)}
    모르는_이름 = set(값들) - set(필드)
    if 모르는_이름:
        raise KeyError(f"모르는 설정 이름: {sorted(모르는_이름)}")
    바꾼값 = {}
    for 이름, 값 in 값들.items():
        자료형 = 필드[이름]                  # str, float, int
        if 자료형 is float and isinstance(값, (str, int)):
            값 = float(값)                   # "3e-4" 같은 글자도 실수로
        if not isinstance(값, 자료형):
            raise TypeError(f"{이름}은(는) {자료형.__name__}이어야 하는데 {값!r}")
        바꾼값[이름] = 값
    return 학습설정(**바꾼값)
```

```
print(설정_만들기(읽은값))
for 잘못된_값 in [{"학습율": 0.01}, {"에폭": "열"}]:
    try:
        설정_만들기(잘못된_값)
    except (KeyError, TypeError) as e:
        print(type(e).__name__, e)
```

```
{'모델': 'cnn', '학습률': '3e-4', '드롭아웃': 0.3}
학습률의 자료형: str
학습설정(모델='cnn', 학습률=0.0003, 배치_크기=64, 에폭=2, 드롭아웃=0.3, 씨앗=0)
KeyError "모르는 설정 이름: ['학습율']"
TypeError 에폭은(는) int이어야 하는데 '열'
```

출력의 두 번째 줄을 보세요. `학습률: 3e-4`라고 적었는데 실수가 아니라 **글자(str)** `'3e-4'`로 읽혔습니다. 같은 파일의 `드롭아웃: 0.3`은 실수로 잘 읽혔는데 말이죠. PyYAML이 따르는 YAML 1.1 규칙에서는 지수 표기에 소수점이 있어야(`3.0e-4`) 실수로 보기 때문입니다. 이 글자를 그대로 옵티마이저에 넘기면 학습을 시작하는 순간 오류가 나거나, 더 나쁘면 엉뚱한 곳에서 조용히 틀린 값으로 쓰일 수 있어요.

그래서 파일에서 읽은 값은 **검사를 거쳐서** 설정으로 바꿉니다. `설정_만들기`는 dataclass의 필드 목록(`fields`)과 각 필드의 자료형을 보고 세 가지를 합니다. 모르는 이름이 있으면 `KeyError`(마지막 두 줄의 `학습율`처럼 오타를 잡아 줘요), 실수 자리에 글자나 정수가 오면 실수로 바꾸기, 그래도 자료형이 맞지 않으면 `TypeError`입니다. 오타 난 설정 이름을 조용히 무시하면 "학습률을 바꿨는데 결과가 그대로"인 실험을 몇 시간씩 돌리게 됩니다. 1권 8장 8.5절의 Pydantic을 쓰면 이런 검사를 더 짧게 쓸 수 있어요.

명령줄에서 덮어쓰기

설정 파일을 만들어 두고도, 학습률 하나만 바꿔서 빨리 돌려 보고 싶을 때가 많습니다. 그럴 때마다 YAML 파일을 복사해 고치기는 번거롭죠. 실행할 때 `uv run python 학습.py --설정 설정_cnn.yaml --학습률 0.001`처럼 **명령줄 인자**로 덮어쓰게 해 두면 편합니다. 표준 라이브러리의 `argparse`가 이 일을 합니다.

```
import argparse
from dataclasses import asdict, dataclass, fields, replace
from pathlib import Path

import yaml


@dataclass(frozen=True)
class 학습설정:
    모델: str = "mlp"
    학습률: float = 1e-3
    배치_크기: int = 64
    에폭: int = 2
    드롭아웃: float = 0.0
    씨앗: int = 0


def 명령줄_해석기() -> argparse.ArgumentParser:
    """설정의 필드마다 --이름 옵션을 하나씩 만든다. 기본값은 None(=말하지 않음)."""
    해석기 = argparse.ArgumentParser(prog="학습.py", description="옷 분류 실험")
    해석기.add_argument("--설정", type=Path, help="YAML 설정 파일")
    for f in fields(학습설정):
        해석기.add_argument(f"--{f.name}", type=f.type, default=None)
    return 해석기


def 설정_읽기(명령줄: list[str]) -> 학습설정:
    """기본값 < YAML 파일 < 명령줄 순서로 덮어쓴다."""
    인자 = vars(명령줄_해석기().parse_args(명령줄))
    설정 = 학습설정()                                          # 1) 기본값
    파일 = 인자.pop("설정")
    if 파일:
        파일값 = yaml.safe_load(파일.read_text(encoding="utf-8"))
        설정 = replace(설정, **{k: type(getattr(설정, k))(v) for k, v in 파일값.items()})   # 2) 파일
    준값 = {k: v for k, v in 인자.items() if v is not None}
    return replace(설정, **준값)                               # 3) 명령줄
```

```
Path("설정_cnn.yaml").write_text("모델: cnn\n학습률: 3e-4\n드롭아웃: 0.3\n", encoding="utf-8")

print(설정_읽기([]))
print(설정_읽기(["--설정", "설정_cnn.yaml"]))
print(설정_읽기(["--설정", "설정_cnn.yaml", "--학습률", "0.001", "--씨앗", "7"]))
print(asdict(설정_읽기(["--에폭", "5"])))
명령줄_해석기().print_usage()
```

```
학습설정(모델='mlp', 학습률=0.001, 배치_크기=64, 에폭=2, 드롭아웃=0.0, 씨앗=0)
학습설정(모델='cnn', 학습률=0.0003, 배치_크기=64, 에폭=2, 드롭아웃=0.3, 씨앗=0)
학습설정(모델='cnn', 학습률=0.001, 배치_크기=64, 에폭=2, 드롭아웃=0.3, 씨앗=7)
{'모델': 'mlp', '학습률': 0.001, '배치_크기': 64, '에폭': 5, '드롭아웃': 0.0, '씨앗': 0}
usage: 학습.py [-h] [--설정 설정] [--모델 모델] [--학습률 학습률] [--배치_크기 배치_크기] [--에폭
↪ 에폭]
            [--드롭아웃 드롭아웃] [--씨앗 씨앗]
```

`명령줄_해석기`는 dataclass의 필드를 돌면서 `--학습률`, `--에폭` 같은 옵션을 자동으로 만듭니다. 필드를 하나 늘리면 옵션도 저절로 늘어나요. 옵션의 기본값을 `None`으로 둔 것이 요령입니다. `None`이면 "명령줄에서 말하지 않았다"는 뜻이라 아래 단계의 값을 그대로 두고, 값이 있을 때만 덮어씁니다.

출력을 위에서부터 보면 덮어쓰는 순서가 보입니다. 아무 인자도 없으면 기본값, `--설정`을 주면 YAML의 세 값이 기본값을 덮고, 거기에 `--학습률 0.001 --씨앗 7`을 더하면 그 둘만 다시 덮어요. **기본값 < 설정 파일 < 명령줄**, 오른쪽으로 갈수록 힘이 셉니다. 1권 17장의 "설정 관리와 환경별 구성"에서 기본값 위에 환경별 파일을 덮어쓰던 것과 같은 생각이에요. 마지막 두 줄의 사용법 안내는 `-h`를 주면 볼 수 있는 내용으로, `argparse`가 만들어 줍니다.

여기서 하나만 주의하세요. 명령줄로 덮어쓴 값은 파일에 남지 않습니다. 그래서 다음에 볼 실험 폴더에는 **최종으로 합쳐진 설정**을 꼭 저장해야 해요.

이 장의 실험 도구

이제 이 장의 예제들이 함께 쓸 도구를 한 파일에 모으겠습니다. 8장의 `학습도구.py`, 9장의 `옷도구.py`와 같은 방식이에요. 아래 내용을 `실험도구.py`로 저장하세요.

```
"""18장 예제에서 함께 쓰는 설정·데이터·모델·학습 도구."""
import os
import random
from dataclasses import dataclass

import numpy as np
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
from torchvision import datasets

데이터_폴더 = os.environ.get("BOOK_DATA_DIR", "data")    # 2장부터 써 온 데이터 폴더 설정


@dataclass
class 학습설정:
    """실험 하나를 정하는 값들. 코드와 이 값이 같으면 결과도 같아야 한다."""
    모델: str = "mlp"            # "mlp"(6장) 또는 "cnn"(9장)
    학습률: float = 1e-3
    배치_크기: int = 64
    에폭: int = 2
    드롭아웃: float = 0.0
    가중치_감쇠: float = 0.0
    학습_개수: int = 55_000      # 노트북에서 빨리 돌려 볼 때는 줄인다
    씨앗: int = 0


def 씨앗_고정(씨앗: int) -> None:
    """8장 8.10절: 파이썬·NumPy·PyTorch의 난수 씨앗을 한꺼번에 고정한다."""
    random.seed(씨앗)
    np.random.seed(씨앗)
    torch.manual_seed(씨앗)


def 옷_데이터(학습_개수: int = 55_000, 검증_개수: int = 5_000):
    """FashionMNIST를 텐서로 읽어 학습·검증·시험으로 나눈다. 나누는 방법은 언제나 같다."""
    학습셋 = datasets.FashionMNIST(데이터_폴더, train=True, download=True)
```

```
    시험셋 = datasets.FashionMNIST(데이터_폴더, train=False, download=True)
    X, y = 학습셋.data.float().unsqueeze(1) / 255, 학습셋.targets
    순서 = torch.randperm(len(X), generator=torch.Generator().manual_seed(0))
    검증, 학습 = 순서[:검증_개수], 순서[검증_개수:검증_개수 + 학습_개수]
    X_시험 = 시험셋.data.float().unsqueeze(1) / 255
    return (X[학습], y[학습]), (X[검증], y[검증]), (X_시험, 시험셋.targets)


def 모델_만들기(설정: 학습설정) -> nn.Sequential:
    """설정에 적힌 이름으로 6장의 MLP나 9장의 CNN을 만든다."""
    if 설정.모델 == "mlp":
        몸통 = [nn.Flatten(), nn.Linear(28 * 28, 256), nn.ReLU(), nn.Linear(256, 128), nn.ReLU()]
        마지막 = 128
    elif 설정.모델 == "cnn":
        몸통 = [nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
                nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
                nn.Flatten(), nn.Linear(64 * 7 * 7, 128), nn.ReLU()]
        마지막 = 128
    else:
        raise ValueError(f"모르는 모델 이름: {설정.모델}")
    if 설정.드롭아웃 > 0:
        몸통.append(nn.Dropout(설정.드롭아웃))
    return nn.Sequential(*몸통, nn.Linear(마지막, 10))


def 로더_만들기(X: torch.Tensor, y: torch.Tensor, 배치_크기: int, 씨앗: int) -> DataLoader:
    """섞는 순서를 자기 생성기로 정하는 DataLoader (4장 4.1절, 7장)."""
    생성기 = torch.Generator().manual_seed(씨앗)
    return DataLoader(TensorDataset(X, y), batch_size=배치_크기, shuffle=True, generator=생성기)


def 한_에폭_학습(모델: nn.Module, 로더: DataLoader, 옵티마이저: torch.optim.Optimizer) -> float:
    """한 바퀴 학습하고 평균 손실을 돌려준다."""
    모델.train()
    손실_합, 개수 = 0.0, 0
    for x, y in 로더:
```

```
        옵티마이저.zero_grad()
        손실 = nn.functional.cross_entropy(모델(x), y)
        손실.backward()
        옵티마이저.step()
        손실_합 += 손실.item() * len(y)
        개수 += len(y)
    return 손실_합 / 개수


@torch.no_grad()
def 평가(모델: nn.Module, X: torch.Tensor, y: torch.Tensor) -> tuple[float, float]:
    """손실과 정확도를 돌려준다. 메모리를 아끼려고 1,000장씩 나눠 계산한다."""
    모델.eval()
    출력 = torch.cat([모델(X[i:i + 1000]) for i in range(0, len(X), 1000)])
    return nn.functional.cross_entropy(출력, y).item(), (출력.argmax(dim=1) ==
    ↪ y).float().mean().item()
```

`학습설정`은 앞의 설정에 `가중치_감쇠`(8장 8.2절)와 `학습_개수`를 더한 것입니다. 노트북에서 실험을 여러 번 빨리 돌리려고 학습 사진 수를 줄일 수 있게 했어요. 이번에는 `frozen=True`를 빼서 실습 중에 값을 바꿔 보기 쉽게 했지만, 실제 프로젝트에서는 붙여 두기를 권합니다.

`옷_데이터`는 FashionMNIST 학습 사진 6만 장을 고정된 순서로 섞어서 앞의 5,000장을 검증용, 그다음 `학습_개수`장을 학습용으로 씁니다. 시험 데이터 1만 장은 따로 두고요. 사진마다 변환을 거치지 않고 처음부터 텐서로 바꿔 7장 7.2절의 `TensorDataset`에 담기 때문에 빠릅니다. `모델_만들기`는 설정의 `모델` 값을 보고 6장의 MLP나 9장의 CNN을 같은 구조로 만들어요. `로더_만들기`는 섞는 순서를 자기 생성기로 정하는 `DataLoader`이고, 이 생성기가 다음 절의 주인공입니다.

설정마다 실험 폴더 하나

설정이 정해지면 실험 결과를 담을 폴더를 만듭니다. 폴더 이름에 날짜를 붙이는 사람도 많지만, 여기서는 **설정의 지문**을 붙여 보겠습니다. 설정이 같으면 지문도 같으니, 같은 실험을 두 번 돌리는 일을 막을 수 있어요.

```
import hashlib
import json
from dataclasses import asdict, replace
```

```
from pathlib import Path

import yaml

from 실험도구 import 학습설정


def 설정_지문(설정: 학습설정) -> str:
    """설정의 모든 값을 정해진 순서로 글자로 만든 뒤 해시한 앞 8글자."""
    글 = json.dumps(asdict(설정), sort_keys=True, ensure_ascii=False)
    return hashlib.sha256(글.encode("utf-8")).hexdigest()[:8]


def 실험_폴더(설정: 학습설정, 뿌리: Path = Path("runs")) -> Path | None:
    """설정마다 폴더를 하나 만들고 설정을 적어 둔다. 이미 있으면 None."""
    폴더 = 뿌리 / f"{설정.모델}_lr{설정.학습률:g}_s{설정.씨앗}_{설정_지문(설정)}"
    if 폴더.exists():
        return None                                  # 같은 실험을 두 번 하지 않는다
    폴더.mkdir(parents=True)
    (폴더 / "설정.yaml").write_text(yaml.safe_dump(asdict(설정), allow_unicode=True,
↪   sort_keys=False),
                                    encoding="utf-8")
    return 폴더


기준 = 학습설정()
print("기준 지문     :", 설정_지문(기준))
print("다시 만들어도 :", 설정_지문(학습설정()))
print("씨앗만 바꿔도 :", 설정_지문(replace(기준, 씨앗=1)))

for 설정 in [기준, replace(기준, 학습률=3e-3), 기준]:
    폴더 = 실험_폴더(설정)
    if 폴더 is None:
        print("이미 한 실험이라 건너뜀")
    else:
        print("새 폴더:", 폴더)
```

```
print((Path("runs") / f"mlp_lr0.001_s0_{설정_지문(기준)}" / "설
↪ 정.yaml").read_text(encoding="utf-8"))
```

```
기준 지문     : 865b758e
다시 만들어도 : 865b758e
씨앗만 바꿔도 : 60be0fbc
새 폴더: runs/mlp_lr0.001_s0_865b758e
새 폴더: runs/mlp_lr0.003_s0_8f62ef74
이미 한 실험이라 건너뜀
모델: mlp
학습률: 0.001
배치_크기: 64
에폭: 2
드롭아웃: 0.0
가중치_감쇠: 0.0
학습_개수: 55000
씨앗: 0
```

`설정_지문`은 설정을 JSON 글자로 바꾼 뒤 SHA-256 해시의 앞 8글자를 씁니다. `hashlib`은 어떤 글자든 받아서 고정된 길이의 "지문"을 만드는 표준 라이브러리예요. 입력이 한 글자만 달라도 지문이 완전히 달라지고, 같은 입력은 언제 어디서 계산해도 같은 지문이 나옵니다. `sort_keys=True`로 이름 순서를 고정한 것도 중요해요. 같은 설정인데 딕셔너리 순서 때문에 지문이 달라지면 안 되니까요.

결과를 보면 다시 만든 기본 설정은 지문이 같고(`865b758e`), 씨앗만 바꿔도 지문이 달라졌습니다. 세 번째로 기준 설정을 다시 넣었을 때는 폴더가 이미 있어서 건너뛰었어요. 폴더 안의 `설정.yaml`에는 합쳐진 최종 설정이 모두 들어 있습니다. 나중에 이 폴더만 보고도 "이 결과는 어떤 설정이었나"에 답할 수 있어요.

18.3 체크포인트와 재개

모델과 옵티마이저만으로는 부족합니다

몇 시간짜리 학습이 중간에 멈추는 일은 생각보다 흔합니다. 노트북 덮개를 닫거나, 전원이 나가거나, 공용 GPU 서버의 사용 시간이 끝나거나요. 6장 6.10절에서 에폭 번호, 모델 상태, 옵티마이저 상태

를 체크포인트에 담아 이어서 학습하는 법을 봤습니다. 그런데 정말 그것으로 충분할까요? 4에폭을 한 번에 학습한 결과와, 2에폭 하고 6장처럼 저장한 뒤 프로그램을 새로 켜서 2에폭을 더 한 결과를 비교해 보겠습니다. 8장 8.7절의 코사인 스케줄러와 8.3절의 드롭아웃을 넣었어요.

```
import torch
from torch.optim.lr_scheduler import CosineAnnealingLR

from 실험도구 import 로더_만들기, 모델_만들기, 씨앗_고정, 옷_데이터, 평가, 학습설정, 한_에폭_학습

설정 = 학습설정(드롭아웃=0.2, 에폭=4, 학습_개수=10_000)
(X_학습, y_학습), (X_검증, y_검증), _ = 옷_데이터(설정.학습_개수)


def 학습_준비():
    """프로그램을 새로 켰을 때처럼 모든 것을 처음부터 만든다."""
    씨앗_고정(설정.씨앗)
    모델 = 모델_만들기(설정)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=설정.학습률)
    스케줄러 = CosineAnnealingLR(옵티마이저, T_max=설정.에폭)
    로더 = 로더_만들기(X_학습, y_학습, 설정.배치_크기, 설정.씨앗)
    return 모델, 옵티마이저, 스케줄러, 로더


def 에폭_돌기(시작: int, 끝: int, 모델, 옵티마이저, 스케줄러, 로더) -> list[float]:
    """시작~끝 에폭을 학습하고, 에폭마다 쓴 학습률을 돌려준다."""
    쓴_학습률 = []
    for 에폭 in range(시작, 끝 + 1):
        쓴_학습률.append(round(옵티마이저.param_groups[0]["lr"], 6))
        한_에폭_학습(모델, 로더, 옵티마이저)
        스케줄러.step()
    return 쓴_학습률


# 1) 한 번에 4에폭
모델, 옵티마이저, 스케줄러, 로더 = 학습_준비()
학습률들 = 에폭_돌기(1, 4, 모델, 옵티마이저, 스케줄러, 로더)
한번에 = 모델.state_dict()
```

```
print(f"한 번에 4에폭       : 검증 정확도 {평가(모델, X_검증, y_검증)[1]:.2%}, 학습률 {학습률들}")

# 2) 2에폭 하고 6장 6.10절처럼 저장 → 프로그램을 새로 켜서 불러오고 2에폭 더
모델, 옵티마이저, 스케줄러, 로더 = 학습_준비()
앞 = 에폭_돌기(1, 2, 모델, 옵티마이저, 스케줄러, 로더)
torch.save({"에폭": 2, "모델": 모델.state_dict(), "옵티마이저": 옵티마이저.state_dict()}, "중
↪ 간.pt")

모델, 옵티마이저, 스케줄러, 로더 = 학습_준비()          # 새로 켠 프로그램
상자 = torch.load("중간.pt", weights_only=True)
모델.load_state_dict(상자["모델"])
옵티마이저.load_state_dict(상자["옵티마이저"])
뒤 = 에폭_돌기(상자["에폭"] + 1, 4, 모델, 옵티마이저, 스케줄러, 로더)
print(f"2에폭 + 이어서 2에폭: 검증 정확도 {평가(모델, X_검증, y_검증)[1]:.2%}, 학습률 {앞 + 뒤}")
같은가 = all(torch.equal(한번에[k], v) for k, v in 모델.state_dict().items())
print("가중치가 같은가:", 같은가)
```

```
한 번에 4에폭       : 검증 정확도 84.32%, 학습률 [0.001, 0.000854, 0.0005, 0.000146]
2에폭 + 이어서 2에폭: 검증 정확도 84.24%, 학습률 [0.001, 0.000854, 0.0005, 0.000427]
가중치가 같은가: False
```

두 결과의 검증 정확도가 84.32%와 84.24%로 다르고, 가중치도 같지 않습니다. 이어서 한 학습은 한 번에 한 학습을 **재현하지 못했어요.** 원인은 세 가지입니다.

- **스케줄러**: 출력의 학습률 목록을 보세요. 한 번에 학습하면 네 번째 에폭의 학습률이 0.000146인데, 이어서 학습하면 0.000427입니다. 새로 만든 스케줄러는 자기가 몇 번째 에폭에 있는지 모르고, 코사인 곡선의 처음부터 다시 계산했기 때문이에요.
- **섞는 순서**: 새로 만든 `DataLoader`의 생성기가 처음 상태로 돌아가서, 3에폭째에 1에폭과 같은 순서로 데이터를 섞었습니다. 한 번에 학습했다면 3에폭째에는 세 번째 순서가 나왔겠죠.
- **드롭아웃의 난수**: 드롭아웃이 어떤 칸을 끌지는 PyTorch의 전역 난수가 정합니다. 새 프로그램에서 씨앗을 다시 심었으니, 3에폭째의 난수 줄이 한 번에 학습할 때와 달라졌어요.

차이가 0.08%p뿐이라 별일 아닌 것 같지만, 이 차이가 있는 한 "중간에 끊겼던 실험"과 "끊기지 않은 실험"을 같은 실험이라고 말할 수 없습니다. 끊긴 횟수와 위치에 따라 결과가 바뀌는 실험은 재현할 수 없어요.

다시 켜도 똑같이 이어 가기

그럼 이 세 가지 상태까지 모두 담으면 되겠죠. 스케줄러는 옵티마이저처럼 `state_dict()`가 있고, `DataLoader`의 생성기와 난수 발생기는 `get_state()`로 지금 상태를 꺼낼 수 있습니다. 내친김에 실제로 학습 도중 "정전"을 일으켜 보겠습니다.

```
import os
import random
from pathlib import Path

import numpy as np
import torch
from torch.optim.lr_scheduler import CosineAnnealingLR

from 실험도구 import 로더_만들기, 모델_만들기, 씨앗_고정, 옷_데이터, 평가, 학습설정, 한_에폭_학습

설정 = 학습설정(드롭아웃=0.2, 에폭=4, 학습_개수=10_000)
(X_학습, y_학습), (X_검증, y_검증), _ = 옷_데이터(설정.학습_개수)


def 난수_상태() -> dict:
    """세 난수 발생기의 지금 상태. NumPy 상태는 weights_only로 읽을 수 있게 리스트로."""
    _, 표, 위치, 가우스, 남은값 = np.random.get_state()
    return {"torch": torch.get_rng_state(), "numpy": (표.tolist(), 위치, 가우스, 남은값),
            "random": random.getstate()}


def 난수_되돌리기(상태: dict) -> None:
    torch.set_rng_state(상태["torch"])
    표, 위치, 가우스, 남은값 = 상태["numpy"]
    np.random.set_state(("MT19937", np.array(표, dtype=np.uint32), 위치, 가우스, 남은값))
    random.setstate(상태["random"])


def 안전하게_저장(내용: dict, 파일: Path) -> None:
    """임시 파일에 다 쓴 다음 이름을 바꾼다. 쓰다가 꺼져도 예전 파일은 멀쩡하다."""
    임시 = 파일.with_suffix(".tmp")
```

```
    torch.save(내용, 임시)
    os.replace(임시, 파일)


def 학습(폴더: Path, 멈출_에폭: int | None = None) -> torch.nn.Module:
    """폴더에 마지막 체크포인트가 있으면 거기서부터 이어서 학습한다."""
    씨앗_고정(설정.씨앗)
    모델 = 모델_만들기(설정)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=설정.학습률)
    스케줄러 = CosineAnnealingLR(옵티마이저, T_max=설정.에폭)
    로더 = 로더_만들기(X_학습, y_학습, 설정.배치_크기, 설정.씨앗)
    시작, 최고 = 1, 0.0
    마지막 = 폴더 / "last.pt"
    if 마지막.exists():
        상자 = torch.load(마지막, weights_only=True)
        모델.load_state_dict(상자["모델"])
        옵티마이저.load_state_dict(상자["옵티마이저"])
        스케줄러.load_state_dict(상자["스케줄러"])
        로더.generator.set_state(상자["섞기_생성기"])
        난수_되돌리기(상자["난수"])
        시작, 최고 = 상자["에폭"] + 1, 상자["최고_정확도"]
        print(f"  {상자['에폭']}에폭 체크포인트에서 이어 갑니다")
    for 에폭 in range(시작, 설정.에폭 + 1):
        한_에폭_학습(모델, 로더, 옵티마이저)
        스케줄러.step()
        if 에폭 == 멈출_에폭:
            raise RuntimeError(f"{에폭}에폭 도중 정전!")
        정확도 = 평가(모델, X_검증, y_검증)[1]
        if 정확도 > 최고:
            최고 = 정확도
            안전하게_저장({"모델": 모델.state_dict(), "에폭": 에폭, "정확도": 정확도}, 폴더 /
↪   "best.pt")
        안전하게_저장({"에폭": 에폭, "모델": 모델.state_dict(), "옵티마이저": 옵티마이
↪   저.state_dict(),
                    "스케줄러": 스케줄러.state_dict(), "섞기_생성기": 로더.generator.get_state(),
                    "난수": 난수_상태(), "최고_정확도": 최고}, 마지막)
```

```
        print(f"  에폭 {에폭}: 검증 정확도 {정확도:.2%}, 학습률 {스케줄러.get_last_lr()[0]:.6f}")
    return 모델


for 이름 in ("한번에", "이어서"):
    Path(이름).mkdir(exist_ok=True)
print("[한 번에]")
한번에 = 학습(Path("한번에"))
print("[중간에 꺼졌다가 다시]")
try:
    학습(Path("이어서"), 멈출_에폭=3)
except RuntimeError as e:
    print("  오류:", e)
이어서 = 학습(Path("이어서"))
같은가 = all(torch.equal(a, b) for a, b in zip(한번에.state_dict().values(), 이어
↪  서.state_dict().values()))
print("가중치가 같은가:", 같은가)
상자 = torch.load("이어서/last.pt", weights_only=True)
print("체크포인트에 담은 것:", list(상자))
print(f"파일 크기: last.pt {os.path.getsize('이어서/last.pt') / 1024:.0f}KB, "
      f"best.pt {os.path.getsize('이어서/best.pt') / 1024:.0f}KB")
```

```
[한 번에]
  에폭 1: 검증 정확도 77.44%, 학습률 0.000854
  에폭 2: 검증 정확도 81.70%, 학습률 0.000500
  에폭 3: 검증 정확도 82.64%, 학습률 0.000146
  에폭 4: 검증 정확도 84.32%, 학습률 0.000000
[중간에 꺼졌다가 다시]
  에폭 1: 검증 정확도 77.44%, 학습률 0.000854
  에폭 2: 검증 정확도 81.70%, 학습률 0.000500
  오류: 3에폭 도중 정전!
  2에폭 체크포인트에서 이어 갑니다
  에폭 3: 검증 정확도 82.64%, 학습률 0.000146
  에폭 4: 검증 정확도 84.32%, 학습률 0.000000
가중치가 같은가: True
```

```
체크포인트에 담은 것: ['에폭', '모델', '옵티마이저', '스케줄러', '섞기_생성기', '난수', '최고_정확
↪ 도']
파일 크기: last.pt 2781KB, best.pt 921KB
```

3에폭 학습은 마쳤지만 아직 체크포인트를 저장하기 전, 즉 3에폭을 마무리하는 도중에 오류를 일으켜 프로그램이 죽은 상황을 흉내 냈습니다. 다시 `학습`을 부르자 `last.pt`를 찾아 2에폭 체크포인트에서 이어 갔고, 3·4에폭의 검증 정확도와 학습률이 한 번에 학습한 것과 똑같습니다. 마지막에 가중치를 비교하니 `True`예요. 이제 몇 번을 끊겼다 이어 가도 같은 실험입니다.

체크포인트에 담은 일곱 가지 중 새로 더한 것은 이렇습니다.

담은 것	꺼내기	되돌리기	빠지면
스케줄러	`스케줄러.state_dict()`	`load_state_dict`	학습률이 처음부터 다시 계산됨
섞기 생성기	`로더.generator.get_state()`	`set_state`	섞는 순서가 1에폭부터 반복됨
전역 난수	`torch.get_rng_state()` 등	`torch.set_rng_state()` 등	드롭아웃·증강의 무작위가 달라짐
최고 점수	숫자 하나	그대로	조기 종료·최고 모델 판단이 틀어짐

코드에서 짚어 둘 곳이 세 군데 있습니다.

- **NumPy 난수 상태**: `np.random.get_state()`는 NumPy 배열을 담은 튜플을 돌려주는데, 이것을 그대로 저장하면 `torch.load(..., weights_only=True)`가 거부합니다. 6장 6.10절에서 본 안전장치가 NumPy 배열을 "텐서도 기본 자료형도 아닌 것"으로 보기 때문이에요. 그래서 `난수_상태`에서 `.tolist()`로 바꿔 저장하고, 되돌릴 때 다시 배열로 만들었습니다. 안전장치를 끄는(`weights_only=False`) 쉬운 길보다 이쪽이 낫습니다.
- **안전하게 저장하기**: `안전하게_저장`은 `.tmp` 파일에 다 쓴 다음 `os.replace`로 이름을 바꿉니다. `torch.save`가 파일을 반쯤 쓰다가 전원이 나가면 체크포인트가 깨져서 열리지 않는데, 이렇게 하면 마지막 순간까지 예전 `last.pt`가 멀쩡히 남아 있어요. 이름 바꾸기는 한 번에 일어나서 "반쯤 바뀐" 상태가 없습니다.
- **last와 best**: `last.pt`는 이어 가기용으로 매 에폭 덮어쓰고, `best.pt`는 검증 정확도가 좋아질

때만 모델만 저장합니다. 8장 8.8절의 조기 종료에서 "가장 좋았던 순간의 가중치"를 메모리에 두던 것을 파일로 옮긴 셈이에요. 파일 크기를 보면 `last.pt`가 `best.pt`의 세 배쯤 됩니다. Adam이 파라미터마다 두 가지 평균을 기억해서(6장 6.10절), 옵티마이저 상태가 모델보다 두 배 크기 때문입니다.

Q. 체크포인트는 얼마나 자주 저장해야 하나요? "잃어도 괜찮은 시간"으로 정하세요. 한 에폭이 몇 분이면 에폭마다, 한 에폭이 몇 시간이면 몇백 스텝마다 저장합니다. 스텝 단위로 저장할 때는 에폭 번호 대신 "몇 번째 배치까지 했는지"도 담아야 하고, 그 에폭 안에서 이미 지나간 배치를 건너뛰어야 정확히 이어집니다. 디스크가 걱정되면 `last.pt` 하나만 덮어쓰고, `best.pt`와 에폭별 파일은 꼭 필요한 것만 남기세요.

18.4 실험 기록 남기기

직접 만드는 기록장: JSON Lines

학습 중의 손실과 정확도를 `print`로만 보면 터미널을 닫는 순간 사라집니다. 가장 간단한 기록 방법은 **에폭마다 한 줄씩 파일에 덧붙이는 것**이에요. 한 줄에 JSON 하나를 쓰는 이 형식을 JSON Lines라고 부르고, 확장자는 보통 `.jsonl`입니다.

```
import json
from pathlib import Path

import torch

from 실험도구 import 로더_만들기, 모델_만들기, 씨앗_고정, 옷_데이터, 평가, 학습설정, 한_에폭_학습



class 기록장:
    """에폭마다 한 줄씩 JSON으로 덧붙여 쓰는 기록 파일 (JSON Lines)."""

    def __init__(self, 파일: Path) -> None:
        파일.parent.mkdir(parents=True, exist_ok=True)
        self.파일 = 파일.open("a", encoding="utf-8")

    def 적기(self, **값) -> None:
```

```
        self.파일.write(json.dumps(값, ensure_ascii=False) + "\n")
        self.파일.flush()                  # 학습이 중간에 죽어도 여기까지는 남는다

    def __enter__(self) -> "기록장":
        return self

    def __exit__(self, *오류) -> None:
        self.파일.close()


설정 = 학습설정(에폭=4, 학습_개수=10_000)
(X_학습, y_학습), (X_검증, y_검증), _ = 옷_데이터(설정.학습_개수)
씨앗_고정(설정.씨앗)
모델 = 모델_만들기(설정)
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=설정.학습률)
로더 = 로더_만들기(X_학습, y_학습, 설정.배치_크기, 설정.씨앗)

with 기록장(Path("runs/기록연습/기록.jsonl")) as 기록:
    for 에폭 in range(1, 설정.에폭 + 1):
        학습_손실 = 한_에폭_학습(모델, 로더, 옵티마이저)
        검증_손실, 검증_정확도 = 평가(모델, X_검증, y_검증)
        기록.적기(에폭=에폭, 학습_손실=round(학습_손실, 4), 검증_손실=round(검증_손실, 4),
                  검증_정확도=round(검증_정확도, 4))

줄들 = Path("runs/기록연습/기록.jsonl").read_text(encoding="utf-8").splitlines()
print(줄들[0])
기록들 = [json.loads(줄) for 줄 in 줄들]
최고 = max(기록들, key=lambda r: r["검증_정확도"])
print(f"{len(기록들)}줄 기록, 가장 좋았던 에폭 {최고['에폭']}: {최고['검증_정확도']:.2%}")
```

```
{"에폭": 1, "학습_손실": 0.8527, "검증_손실": 0.6118, "검증_정확도": 0.7756}
4줄 기록, 가장 좋았던 에폭 4: 84.52%
```

`기록장`은 파일을 덧붙이기 모드(`"a"`)로 열고, `적기`를 부를 때마다 JSON 한 줄을 씁니다. 1권 12장에서 본 `with` 문으로 쓸 수 있게 `__enter__`와 `__exit__`를 넣어서 학습이 끝나면 파일이 저절로 닫혀요. JSON 하나짜리 파일 대신 줄마다 따로 쓰는 이유는 **중간에 죽어도 거기까지는 남기 때문**입니

다. `flush()`로 매번 디스크에 밀어내니, 3에폭째에 프로그램이 죽어도 1·2에폭의 기록은 파일에 있어요. 읽을 때는 한 줄씩 `json.loads`하면 딕셔너리 목록이 됩니다.

덧붙이기 모드라서 같은 파일로 다시 실행하면 기록이 뒤에 이어 붙습니다. 18.2절처럼 실험마다 폴더를 따로 두면 이 문제가 없어요. 이 정도 기록장만 있어도 혼자 하는 작은 프로젝트에는 충분합니다. 그런데 실험이 수십 개가 되면 곡선을 겹쳐 보고 표로 정렬하는 일이 번거로워지죠. 그때 쓰는 것이 기록 도구입니다.

TensorBoard: 곡선을 겹쳐 보기

TensorBoard는 원래 TensorFlow와 함께 나온 오픈소스 시각화 도구인데, PyTorch에 기록용 `SummaryWriter`가 들어 있어서 그대로 쓸 수 있습니다. 학습률 두 가지로 학습하며 기록해 보겠습니다.

```
from dataclasses import asdict, replace

import torch
from tensorboard.backend.event_processing.event_accumulator import EventAccumulator
from torch.utils.tensorboard import SummaryWriter

from 실험도구 import 로더_만들기, 모델_만들기, 씨앗_고정, 옷_데이터, 평가, 학습설정, 한_에폭_학습

기준 = 학습설정(에폭=4, 학습_개수=10_000)
(X_학습, y_학습), (X_검증, y_검증), _ = 옷_데이터(기준.학습_개수)

for 학습률 in (1e-3, 1e-2):
    설정 = replace(기준, 학습률=학습률)
    씨앗_고정(설정.씨앗)
    모델 = 모델_만들기(설정)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=설정.학습률)
    로더 = 로더_만들기(X_학습, y_학습, 설정.배치_크기, 설정.씨앗)
    with SummaryWriter(f"tb/lr{학습률:g}") as 쓰개:          # 실험마다 폴더 하나
        for 에폭 in range(1, 설정.에폭 + 1):
            학습_손실 = 한_에폭_학습(모델, 로더, 옵티마이저)
            검증_손실, 검증_정확도 = 평가(모델, X_검증, y_검증)
            쓰개.add_scalar("손실/학습", 학습_손실, 에폭)      # 이름의 / 앞이 같으면 한 칸에 모인다
            쓰개.add_scalar("손실/검증", 검증_손실, 에폭)
```

```
            쓰개.add_scalar("정확도/검증", 검증_정확도, 에폭)
        쓰개.add_histogram("가중치/첫층", 모델[1].weight, 에폭)
        쓰개.add_hparams(asdict(설정), {"최종/검증_정확도": 검증_정확도}, run_name=".")

# 파일에 무엇이 들어갔는지 TensorBoard의 읽기 도구로 확인
for 폴더 in ("tb/lr0.001", "tb/lr0.01"):
    읽개 = EventAccumulator(폴더).Reload()
    print(폴더, "→", 읽개.Tags()["scalars"])
    print("  검증 정확도:", [round(e.value, 4) for e in 읽개.Scalars("정확도/검증")])
```

```
tb/lr0.001 → ['손실/학습', '손실/검증', '정확도/검증', '최종/검증_정확도']
  검증 정확도: [0.7756, 0.8282, 0.834, 0.8452]
tb/lr0.01 → ['손실/학습', '손실/검증', '정확도/검증', '최종/검증_정확도']
  검증 정확도: [0.799, 0.8014, 0.8302, 0.822]
```

`SummaryWriter("tb/lr0.001")`처럼 실험마다 폴더를 하나씩 주고, `add_scalar(이름, 값, 에폭)`으로 숫자를 남깁니다. 이름의 `/` 앞부분이 같으면 화면에서 한 칸에 모여요. `손실/학습`과 `손실/검증`은 "손실" 칸에 나란히 놓입니다. `add_histogram`은 첫 층 가중치의 분포를, `add_hparams`는 설정과 최종 성적을 표로 남겨요.

TensorBoard의 기록은 사람이 읽기 어려운 이진 파일이라, 마지막 부분에서 TensorBoard의 읽기 도구(`EventAccumulator`)로 무엇이 들어갔는지 확인했습니다. 학습률 0.001은 정확도가 꾸준히 올라 84.52%가 됐고, 0.01은 첫 에폭에 79.90%로 더 빨리 올랐지만 4에폭째에 83.02%에서 82.20%로 도로 내려갔어요. 학습률이 크면 처음엔 빠르지만 끝에서 흔들린다는 8장 8.7절의 이야기 그대로입니다.

화면으로 보려면 터미널에서 TensorBoard를 띄웁니다. 이 책의 환경에서 실제로 실행한 명령과 출력이에요.

```
$ uv run tensorboard --logdir tb
TensorFlow installation not found - running with reduced feature set.
Serving TensorBoard on localhost; to expose to the network, use a proxy or pass --bind_all
TensorBoard 2.21.0 at http://localhost:6006/ (Press CTRL+C to quit)
```

첫 줄은 TensorFlow 없이 PyTorch 기록만 보여 준다는 안내라 신경 쓰지 않아도 됩니다. 웹 브라우저로 `http://localhost:6006/`을 열면 Scalars(또는 Time Series) 탭에서 두 실험의 곡선이 색을 달

리해 겹쳐 그려지고, Histograms 탭에서 가중치 분포를, HParams 탭에서 설정별 성적을 표로 볼 수 있어요. 학습이 진행되는 동안에도 몇 초 간격으로 새 기록을 읽어 와 곡선이 자랍니다. 다 보면 터미널에서 `Ctrl+C`로 끕니다.

MLflow: 실험을 표로 찾고 비교하기

TensorBoard가 곡선을 보는 데 강하다면, MLflow는 실험을 **데이터베이스처럼 저장하고 찾는 데** 강한 오픈소스 도구입니다. 설정(파라미터), 에폭별 숫자(메트릭), 메모(태그), 모델 같은 파일(아티팩트)을 실험 단위로 묶어 두고, 나중에 조건으로 찾아 표로 꺼낼 수 있어요. 기록은 1권 18장에서 배운 SQLite 파일 하나에 담기게 하겠습니다.

```
from dataclasses import asdict, replace

import mlflow
import torch

from 실험도구 import 로더_만들기, 모델_만들기, 씨앗_고정, 옷_데이터, 평가, 학습설정, 한_에폭_학습

mlflow.set_tracking_uri("sqlite:///mlflow.db")        # 기록은 이 폴더의 SQLite 파일 하나에
mlflow.set_experiment("옷분류-학습률")
기준 = 학습설정(에폭=4, 학습_개수=10_000)
(X_학습, y_학습), (X_검증, y_검증), _ = 옷_데이터(기준.학습_개수)

for 학습률 in (3e-4, 1e-3, 3e-3, 1e-2):
    설정 = replace(기준, 학습률=학습률)
    with mlflow.start_run(run_name=f"mlp lr={학습률:g}"):
        mlflow.log_params(asdict(설정))                  # 설정 전부
        mlflow.set_tag("메모", "학습률만 바꿔 본 실험")
        씨앗_고정(설정.씨앗)
        모델 = 모델_만들기(설정)
        옵티마이저 = torch.optim.Adam(모델.parameters(), lr=설정.학습률)
        로더 = 로더_만들기(X_학습, y_학습, 설정.배치_크기, 설정.씨앗)
        for 에폭 in range(1, 설정.에폭 + 1):
            학습_손실 = 한_에폭_학습(모델, 로더, 옵티마이저)
            검증_손실, 검증_정확도 = 평가(모델, X_검증, y_검증)
            mlflow.log_metrics({"학습_손실": 학습_손실, "검증_손실": 검증_손실,
                                "검증_정확도": 검증_정확도}, step=에폭)
```

```
        torch.save(모델.state_dict(), "모델.pt")
        mlflow.log_artifact("모델.pt")                  # 파일도 실험에 붙여 둔다

# 기록을 표로 꺼내 비교하기 (pandas DataFrame)
표 = mlflow.search_runs(experiment_names=["옷분류-학습률"], order_by=["metrics.검증_정확도 DESC"])
for _, 줄 in 표.iterrows():
    print(f"{줄['tags.mlflow.runName']:13} 학습률 {줄['params.학습률']:>6}  "
          f"검증 정확도 {줄['metrics.검증_정확도']:.2%}  검증 손실 {줄['metrics.검증_손실']:.4f}")

# 실험 하나의 에폭별 기록과 붙여 둔 파일
최고 = 표.iloc[0]
이력 = mlflow.MlflowClient().get_metric_history(최고.run_id, "검증_정확도")
print("에폭별 검증 정확도:", [(m.step, round(m.value, 4)) for m in 이력])
print("붙여 둔 파일:", [f.path for f in mlflow.MlflowClient().list_artifacts(최고.run_id)])
```

```
mlp lr=0.003  학습률  0.003  검증 정확도 84.60%  검증 손실 0.4289
mlp lr=0.001  학습률  0.001  검증 정확도 84.52%  검증 손실 0.4395
mlp lr=0.0003 학습률 0.0003  검증 정확도 82.82%  검증 손실 0.4883
mlp lr=0.01   학습률   0.01  검증 정확도 82.20%  검증 손실 0.5008
에폭별 검증 정확도: [(1, 0.8068), (2, 0.8116), (3, 0.825), (4, 0.846)]
붙여 둔 파일: ['모델.pt']
```

`set_experiment`로 실험 묶음의 이름을 정하고, `with mlflow.start_run():` 안에서 기록하면 그 블록이 실험 하나가 됩니다. `log_params(asdict(설정))` 한 줄로 18.2절의 설정을 통째로 남기는 것이 dataclass로 설정을 묶어 둔 덕분이에요. `log_metrics`는 에폭마다, `log_artifact`는 모델 파일을 실험에 붙입니다. 기록은 `mlflow.db`(SQLite)에, 붙인 파일은 `mlruns` 폴더에 저장돼요.

뒷부분이 MLflow의 진가입니다. `search_runs`가 실험 묶음 전체를 pandas 표(1권 20장)로 돌려주고, `order_by`로 마지막 검증 정확도가 높은 순서대로 정렬했어요. 학습률 0.003이 84.60%, 0.001이 84.52%로 거의 같고, 0.0003은 너무 느려서, 0.01은 너무 커서 낮습니다. 가장 좋은 실험의 에폭별 기록과 붙여 둔 파일도 꺼내 봤어요. 실험이 수백 개로 늘어나도 `filter_string="params.모델 = 'cnn'"`처럼 조건을 걸어 찾을 수 있습니다.

MLflow에도 화면이 있습니다. 같은 폴더에서 이렇게 띄워요(이 책의 환경에서 실행한 출력의 마지막 줄만 옮겼습니다).

```
$ uv run mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5001
... INFO:     Uvicorn running on http://127.0.0.1:5001 (Press CTRL+C to quit)
```

`--backend-store-uri`로 기록이 든 SQLite 파일을 알려 줍니다. 기본 포트는 5000인데, macOS에서는 AirPlay 수신 기능(제어 센터)이 5000번을 쓰고 있을 수 있어서 이 책에서는 `--port 5001`로 바꿨어요. 브라우저에서 `http://127.0.0.1:5001`을 열면 실험 목록이 표로 나오고, 실험 여러 개를 골라 곡선과 설정을 나란히 비교할 수 있습니다. `Uvicorn`이라는 이름이 낯익다면 맞아요, 1권 19장에서 FastAPI 서버를 띄울 때 본 그 서버입니다.

도구	강한 곳	기록이 저장되는 곳	이럴 때
JSON Lines (직접)	단순함, 의존성 없음	실험 폴더의 `.jsonl`	혼자 하는 작은 실험
TensorBoard	곡선 겹쳐 보기, 분포 그림	이벤트 파일 폴더	학습 과정을 실시간으로 볼 때
MLflow	설정·결과 검색과 비교, 파일 보관	SQLite 등 데이터베이스 + 파일 폴더	실험이 많고 여럿이 함께 볼 때

어느 도구를 쓰든 원칙은 같습니다. **설정 전체, 에폭별 숫자, 최종 결과, 모델 파일을 실험 하나로 묶어 둔다.** 도구는 나중에 바꿀 수 있지만, 기록하지 않은 실험은 되살릴 수 없어요.

18.5 하이퍼파라미터 탐색 자동화

Optuna: 지난 결과를 보고 다음 값을 고르기

3장 3.10절에서 `GridSearchCV`로 후보 조합을 모두 시험해 봤습니다. 딥러닝에서는 학습 한 번이 무거워서 조합을 다 돌리기 어렵죠. 학습률·드롭아웃·가중치 감쇠·배치 크기를 네 가지씩만 골라도 256번입니다. 그래서 정해진 횟수 안에서 **영리하게 고르는** 방법을 씁니다.

Optuna는 이 일을 해 주는 오픈소스 라이브러리입니다. 우리는 "값을 받아 학습하고 점수를 돌려주는 함수"만 짜면, Optuna가 값을 골라 이 함수를 여러 번 부르며 점수를 키워 가요. 무작위로 고르는 방법과 Optuna의 기본 방법(TPE)을 같은 30번으로 비교해 보겠습니다.

```
import optuna
import torch
```

```
from 실험도구 import 로더_만들기, 모델_만들기, 씨앗_고정, 옷_데이터, 평가, 학습설정, 한_에폭_학습

optuna.logging.set_verbosity(optuna.logging.WARNING)     # 시도마다 찍는 안내 끄기
(X_학습, y_학습), (X_검증, y_검증), _ = 옷_데이터(학습_개수=20_000)


def 목적함수(시도: optuna.Trial) -> float:
    """Optuna가 고른 값으로 학습하고 검증 정확도를 돌려준다. 이 값을 키우는 것이 목표."""
    설정 = 학습설정(
        학습률=시도.suggest_float("학습률", 1e-4, 1e-1, log=True),     # 0.0001~0.1, 로그 눈금
        드롭아웃=시도.suggest_float("드롭아웃", 0.0, 0.5),
        가중치_감쇠=시도.suggest_float("가중치_감쇠", 1e-6, 1e-1, log=True),
        배치_크기=시도.suggest_categorical("배치_크기", [32, 64, 128, 256]),
        에폭=3, 학습_개수=20_000,
    )
    씨앗_고정(설정.씨앗)
    모델 = 모델_만들기(설정)
    옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=설정.학습률, weight_decay=설정.가중치_감
↪   쇠)
    로더 = 로더_만들기(X_학습, y_학습, 설정.배치_크기, 설정.씨앗)
    for _ in range(설정.에폭):
        한_에폭_학습(모델, 로더, 옵티마이저)
    return 평가(모델, X_검증, y_검증)[1]


for 이름, 고르개 in [("무작위", optuna.samplers.RandomSampler(seed=0)),
                     ("TPE", optuna.samplers.TPESampler(seed=0))]:
    연구 = optuna.create_study(direction="maximize", sampler=고르개)
    연구.optimize(목적함수, n_trials=30)
    값들 = [t.value for t in 연구.trials]
    print(f"[{이름}] 최고 {연구.best_value:.2%} ({연구.best_trial.number + 1}번째 시도)")
    print("  최고 설정:", {k: (round(v, 5) if isinstance(v, float) else v) for k, v in 연
↪   구.best_params.items()})
    print("  처음 10번:", " ".join(f"{v:.3f}" for v in 값들[:10]))
    print("  나중 20번:", " ".join(f"{v:.3f}" for v in 값들[10:]))
```

```
    print(f"  나중 20번 평균 {sum(값들[10:]) / 20:.2%}, 80% 미만 {sum(v < 0.8 for v in 값들)}번")
```

```
[무작위] 최고 85.64% (8번째 시도)
  최고 설정: {'학습률': 0.00123, '드롭아웃': 0.2851, '가중치_감쇠': 0.00016, '배치_크기': 32}
  처음 10번: 0.840 0.614 0.784 0.809 0.845 0.814 0.818 0.856 0.826 0.824
  나중 20번: 0.428 0.801 0.849 0.840 0.789 0.850 0.833 0.780 0.811 0.850 0.417 0.578 0.793 0.849
↪  0.830 0.854 0.851 0.855 0.838 0.815
  나중 20번 평균 77.55%, 80% 미만 8번
[TPE] 최고 86.14% (24번째 시도)
  최고 설정: {'학습률': 0.00099, '드롭아웃': 0.43098, '가중치_감쇠': 0.00065, '배치_크기': 32}
  처음 10번: 0.840 0.614 0.784 0.809 0.845 0.814 0.818 0.856 0.826 0.824
  나중 20번: 0.843 0.854 0.826 0.763 0.857 0.851 0.855 0.852 0.818 0.845 0.848 0.843 0.854 0.861
↪  0.859 0.848 0.855 0.827 0.858 0.859
  나중 20번 평균 84.38%, 80% 미만 3번
```

목적함수는 **시도**(trial)에게 값을 물어 설정을 만듭니다. `suggest_float("학습률", 1e-4, 1e-1, log=True)`는 "0.0001부터 0.1 사이에서 하나 골라 줘, 단 로그 눈금으로"라는 뜻이에요. 학습률은 0.0001과 0.001의 차이가 0.09와 0.1의 차이보다 훨씬 중요하니, 자릿수마다 공평하게 고르라고 `log=True`를 줍니다. `suggest_categorical`은 목록에서 하나를 고르고요. 고른 값으로 18.2절의 **학습설정**을 만드니 나머지 학습 코드는 그대로입니다.

두 방법의 **처음 10번이 똑같습니다.** TPE도 처음에는 아는 것이 없어서 무작위로 고르고, 씨앗(`seed=0`)이 같으니 같은 값이 나온 거예요. 차이는 그다음부터입니다. 무작위는 계속 아무 데나 골라서 나중 20번에도 42.8%, 41.7% 같은 실패작이 섞이고 평균이 77.55%입니다. TPE는 지난 결과에서 "점수가 좋았던 값 근처"를 더 자주 골라서 나중 20번의 평균이 84.38%, 최고도 86.14%로 더 높아요. 같은 30번을 돌려도 좋은 곳을 더 많이 들여다본 셈입니다.

TPE가 찾은 최고 설정은 학습률 약 0.001, 드롭아웃 0.43, 배치 크기 32였어요. 학습률 0.001은 6장에서 "Adam의 출발점"으로 권한 값과 거의 같습니다. 탐색은 이렇게 경험칙을 확인해 주기도 하고, 드롭아웃처럼 감으로 정하기 어려운 값을 대신 골라 주기도 합니다.

가망 없는 시도는 중간에 멈추기

30번 중에는 첫 에폭부터 성적이 형편없는 시도가 섞여 있습니다. 그런 시도를 끝까지 학습하는 것은 낭비죠. 수능 모의고사 성적이 계속 바닥인 과목에 학원비를 끝까지 내지 않는 것과 비슷합니다. Optuna는 에폭마다 중간 성적을 보고받아, 다른 시도들보다 못하면 **가지치기**(pruning)로 멈추게

할 수 있어요.

```
import optuna
import torch

from 실험도구 import 로더_만들기, 모델_만들기, 씨앗_고정, 옷_데이터, 평가, 학습설정, 한_에폭_학습

optuna.logging.set_verbosity(optuna.logging.WARNING)
(X_학습, y_학습), (X_검증, y_검증), _ = 옷_데이터(학습_개수=20_000)
최대_에폭 = 5


def 목적함수(시도: optuna.Trial) -> float:
    설정 = 학습설정(
        학습률=시도.suggest_float("학습률", 1e-4, 1e-1, log=True),
        드롭아웃=시도.suggest_float("드롭아웃", 0.0, 0.5),
        가중치_감쇠=시도.suggest_float("가중치_감쇠", 1e-6, 1e-1, log=True),
        배치_크기=시도.suggest_categorical("배치_크기", [32, 64, 128, 256]),
        에폭=최대_에폭, 학습_개수=20_000,
    )
    씨앗_고정(설정.씨앗)
    모델 = 모델_만들기(설정)
    옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=설정.학습률, weight_decay=설정.가중치_감
↪   쇠)
    로더 = 로더_만들기(X_학습, y_학습, 설정.배치_크기, 설정.씨앗)
    for 에폭 in range(1, 설정.에폭 + 1):
        한_에폭_학습(모델, 로더, 옵티마이저)
        정확도 = 평가(모델, X_검증, y_검증)[1]
        시도.report(정확도, 에폭)            # 중간 성적 알리기
        시도.set_user_attr("에폭", 에폭)
        if 시도.should_prune():              # 지금까지의 시도들 중간값보다 못하면
            raise optuna.TrialPruned()       # 여기서 그만
    return 정확도


연구 = optuna.create_study(
    study_name="옷-가지치기", storage="sqlite:///탐색.db",       # 기록을 파일에 남긴다
```

```
    direction="maximize", sampler=optuna.samplers.TPESampler(seed=0),
    pruner=optuna.pruners.MedianPruner(n_startup_trials=5, n_warmup_steps=2),
)
연구.optimize(목적함수, n_trials=40)

끝까지 = [t for t in 연구.trials if t.state == optuna.trial.TrialState.COMPLETE]
잘림 = [t for t in 연구.trials if t.state == optuna.trial.TrialState.PRUNED]
쓴_에폭 = sum(t.user_attrs["에폭"] for t in 연구.trials)
print(f"끝까지 학습 {len(끝까지)}번, 중간에 멈춤 {len(잘림)}번")
print(f"학습한 에폭 {쓴_에폭} / 가지치기가 없었다면 {len(연구.trials) * 최대_에폭}")
print(f"최고 검증 정확도 {연구.best_value:.2%} ({연구.best_trial.number + 1}번째 시도)")
print("멈춘 에폭별 개수:", {e: sum(t.user_attrs['에폭'] == e for t in 잘림) for e in range(1, 최대_
↪ 에폭 + 1)})
```

```
끝까지 학습 25번, 중간에 멈춤 15번
학습한 에폭 158 / 가지치기가 없었다면 200
최고 검증 정확도 86.64% (39번째 시도)
멈춘 에폭별 개수: {1: 0, 2: 12, 3: 3, 4: 0, 5: 0}
```

달라진 곳은 학습 루프 안의 세 줄입니다. 에폭마다 `시도.report(정확도, 에폭)`로 중간 성적을 알리고, `시도.should_prune()`이 참이면 `optuna.TrialPruned` 예외를 던져 그 시도를 끝냅니다. 판단은 `MedianPruner`가 해요. "같은 에폭에서 지금까지 시도들의 중간값보다 못하면 멈춰라"는 규칙입니다. `n_startup_trials=5`는 처음 5번은 비교할 대상이 없으니 끝까지 학습시키라는 뜻이고, `n_warmup_steps=2`는 첫 에폭 성적만 보고 섣불리 자르지 말고 2에폭부터 판단하라는 뜻이에요.

결과를 보면 40번 중 15번이 중간에 멈췄고, 대부분 2에폭에서 잘렸습니다. 학습한 에폭은 158번으로, 가지치기가 없었다면 했을 200번보다 약 21% 적어요. 그러면서 최고 검증 정확도는 86.64%로 앞의 30번 탐색(3에폭)보다 높습니다. 에폭을 5로 늘렸으니 당연한 면도 있지만, 늘어난 비용의 일부를 가지치기가 되찾아 준 거죠. 단, 처음엔 느리고 나중에 좋아지는 설정(예를 들어 학습률이 아주 작은 것)이 억울하게 잘릴 수 있다는 점은 알고 쓰세요.

탐색 기록 다시 열기와 중요도

`create_study`에 `storage="sqlite:///탐색.db"`를 줬기 때문에 40번의 시도가 SQLite 파일에 남아 있습니다. 탐색을 몇 시간씩 돌리다 끊겨도 기록은 그대로예요. 이 기록을 다시 열어 보겠습니다.

```
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)

# 앞 절에서 탐색.db에 남긴 연구를 이름으로 다시 연다 (프로그램을 껐다 켜도 그대로)
연구 = optuna.load_study(study_name="옷-가지치기", storage="sqlite:///탐색.db")
print(f"불러온 시도 {len(연구.trials)}개, 최고 {연구.best_value:.2%}")

# 시도 기록을 pandas 표로: 끝까지 학습한 것 중 위에서 다섯
표 = 연구.trials_dataframe(attrs=("number", "value", "params", "state"))
표 = 표[표.state == "COMPLETE"].sort_values("value", ascending=False).head(5)
for _, 줄 in 표.iterrows():
    print(f"{줄.number + 1:2}번째 {줄.value:.2%}  학습률 {줄.params_학습률:.5f}  드롭아웃
    ↪ {줄.params_드롭아웃:.2f}  "
          f"감쇠 {줄.params_가중치_감쇠:.1e}  배치 {줄.params_배치_크기}")

# 어떤 값이 결과를 가장 많이 흔들었나
중요도 = optuna.importance.get_param_importances(연구,
↪ evaluator=optuna.importance.FanovaImportanceEvaluator(seed=0))
for 이름, 값 in 중요도.items():
    print(f"중요도 {이름}: {값:.2f}")
```

```
불러온 시도 40개, 최고 86.64%
39번째 86.64%  학습률 0.00082  드롭아웃 0.13  감쇠 3.4e-06  배치 32
35번째 86.60%  학습률 0.00086  드롭아웃 0.15  감쇠 1.3e-06  배치 32
28번째 86.60%  학습률 0.00092  드롭아웃 0.16  감쇠 2.5e-06  배치 32
19번째 86.44%  학습률 0.00071  드롭아웃 0.18  감쇠 2.0e-06  배치 32
33번째 86.32%  학습률 0.00077  드롭아웃 0.08  감쇠 3.0e-03  배치 32
중요도 학습률: 0.67
중요도 드롭아웃: 0.17
중요도 배치_크기: 0.15
중요도 가중치_감쇠: 0.00
```

`load_study`로 이름과 저장소를 주면 연구를 그대로 되살립니다. 여기서 `optimize`를 다시 부르면 41번째 시도부터 이어서 탐색해요. 반대로 앞 예제를 처음부터 다시 실행하고 싶으면 `탐색.db`를 지우세요. 같은 이름의 연구가 이미 있으면 `create_study`가 오류를 냅니다(`load_if_exists=True`를 주

면 이어 붙입니다).

`trials_dataframe`은 시도 기록을 pandas 표로 줍니다. 위에서 다섯 개를 보면 모두 학습률 0.0007~0.0009, 배치 크기 32이고, 가중치 감쇠는 0.000001부터 0.003까지 제각각이에요. 마지막 줄의 **중요도**가 이 인상을 숫자로 보여 줍니다. 결과의 흔들림 중 학습률이 0.67을 설명하고, 가중치 감쇠는 0.00으로 거의 상관이 없었어요. 다음 탐색에서는 가중치 감쇠를 빼고 학습률 범위를 좁히는 식으로 계획을 세울 수 있습니다. 이 중요도 계산(fANOVA)은 내부에서 scikit-learn의 랜덤 포레스트(3장)를 쓰기 때문에 scikit-learn이 설치되어 있어야 해요.

그런데 위의 다섯 줄을 다시 보세요. 86.64%, 86.60%, 86.60%, 86.44%, 86.32%. 1등과 2등의 차이가 0.04%p입니다. 39번째 시도가 35번째보다 정말 좋은 설정일까요? 씨앗을 바꾸면 순위가 뒤집히지 않을까요? 다음 절이 이 질문에 답합니다.

18.6 여러 씨앗으로 보고하기

6장의 86.07%를 다시 보기

6장 6.11절에서 옵티마이저를 비교할 때 Adam(lr=0.001)이 86.07%, 모멘텀을 더한 SGD가 84.12%였습니다. 그리고 "0.1~0.2%p 차이로 우열을 가리면 안 된다, 씨앗을 바꿔 여러 번 돌려 봐야 한다"고 했죠. 그 실험을 씨앗 다섯 개로 다시 해 보겠습니다. 학습률을 두 배로 키운 Adam도 함께 넣었어요.

```
import os
import statistics

import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
from torchvision import datasets

데이터_폴더 = os.environ.get("BOOK_DATA_DIR", "data")

학습셋 = datasets.FashionMNIST(데이터_폴더, train=True, download=True)
시험셋 = datasets.FashionMNIST(데이터_폴더, train=False, download=True)
학습_로더 = DataLoader(TensorDataset(학습셋.data.float().unsqueeze(1) / 255, 학습셋.targets),
                    batch_size=64, shuffle=True)
```

```
X_시험, y_시험 = 시험셋.data.float().unsqueeze(1) / 255, 시험셋.targets


def 학습하고_재기(옵티마이저_만들기, 씨앗: int, 에폭수: int = 2) -> float:
    """6장 6.11절의 옵티마이저 비교와 같은 실험. 씨앗만 바꿀 수 있게 했다."""
    torch.manual_seed(씨앗)
    모델 = nn.Sequential(nn.Flatten(), nn.Linear(28 * 28, 256), nn.ReLU(),
                         nn.Linear(256, 128), nn.ReLU(), nn.Linear(128, 10))
    옵티마이저 = 옵티마이저_만들기(모델.parameters())
    for _ in range(에폭수):
        for x, y in 학습_로더:
            옵티마이저.zero_grad()
            nn.functional.cross_entropy(모델(x), y).backward()
            옵티마이저.step()
    with torch.no_grad():
        return (모델(X_시험).argmax(dim=1) == y_시험).float().mean().item()


후보 = {
    "Adam lr=0.001": lambda p: torch.optim.Adam(p, lr=0.001),
    "Adam lr=0.002": lambda p: torch.optim.Adam(p, lr=0.002),
    "SGD+momentum ": lambda p: torch.optim.SGD(p, lr=0.01, momentum=0.9),
}
씨앗들 = range(5)
결과 = {이름: [학습하고_재기(만들기, 씨앗) for 씨앗 in 씨앗들] for 이름, 만들기 in 후보.items()}

for 이름, 값들 in 결과.items():
    print(f"{이름}  " + " ".join(f"{v:.2%}" for v in 값들)
          + f"  → 평균 {statistics.mean(값들):.2%} ± {statistics.stdev(값들):.2%}")

# 같은 씨앗끼리 짝지어 차이 보기
for 상대 in ("Adam lr=0.002", "SGD+momentum "):
    차이 = [(a - b) * 100 for a, b in zip(결과["Adam lr=0.001"], 결과[상대])]     # %p 단위
    평균, 표준편차 = statistics.mean(차이), statistics.stdev(차이)
    표준오차 = 표준편차 / len(차이) ** 0.5            # 평균이 흔들리는 정도
    print(f"Adam lr=0.001 - {상대.strip()}: " + " ".join(f"{d:+.2f}" for d in 차이))
```

```
    print(f"  평균 {평균:+.2f}%p, 표준편차 {표준편차:.2f}%p, 표준오차 {표준오차:.2f}%p, "
          f"평균 ÷ 표준오차 = {평균 / 표준오차:.1f}")
```

```
Adam lr=0.001  86.07% 85.54% 85.97% 86.41% 86.69%  → 평균 86.14% ± 0.44%
Adam lr=0.002  85.73% 85.53% 86.22% 86.28% 86.58%  → 평균 86.07% ± 0.43%
SGD+momentum   84.12% 85.02% 84.77% 84.58% 85.16%  → 평균 84.73% ± 0.41%
Adam lr=0.001 - Adam lr=0.002: +0.34 +0.01 -0.25 +0.13 +0.11
  평균 +0.07%p, 표준편차 0.21%p, 표준오차 0.10%p, 평균 ÷ 표준오차 = 0.7
Adam lr=0.001 - SGD+momentum: +1.95 +0.52 +1.20 +1.83 +1.53
  평균 +1.41%p, 표준편차 0.57%p, 표준오차 0.26%p, 평균 ÷ 표준오차 = 5.5
```

씨앗 0의 결과가 6장과 똑같이 86.07%, 84.12%입니다. 사진을 변환 없이 텐서로 바꿔 넣었을 뿐 같은 실험이라 같은 숫자가 나왔어요. 이제 씨앗 0만 봤다면 어떤 결론을 냈을지 생각해 보세요. Adam lr=0.001이 86.07%, lr=0.002가 85.73%이니 "0.001이 0.34%p 낫다"고 했을 겁니다. 그런데 씨앗 다섯 개의 평균은 86.14%와 86.07%로 0.07%p 차이뿐이고, 씨앗 2에서는 오히려 0.002가 0.25%p 높아요.

같은 설정 안에서 씨앗만 바꿔도 결과가 얼마나 흔들리는지는 **표준편차**가 알려 줍니다. 세 설정 모두 0.4%p 남짓이에요. 씨앗 하나 바꾸는 것만으로 85.54%에서 86.69%까지, 1%p 넘게 오르내립니다. 이 흔들림보다 작은 차이는 씨앗 하나짜리 실험으로는 가릴 수 없어요. `statistics`는 1권 17장에서 본 표준 라이브러리로, `mean`과 `stdev`(n - 1로 나누는 표준편차, 4장 4.10절)를 줍니다.

같은 씨앗끼리 짝지어 보기

두 설정을 비교할 때는 평균끼리만 보지 말고 **같은 씨앗끼리 짝지어** 차이를 보면 더 많은 것이 보입니다. 같은 씨앗이면 처음 가중치와 섞는 순서가 같아서, "운이 좋은 씨앗"의 효과가 양쪽에 똑같이 들어가거든요. 짝마다 차이를 빼면 그 운이 상쇄되고 설정의 차이만 남습니다.

출력의 마지막 네 줄이 그 계산입니다. Adam lr=0.001에서 lr=0.002를 빼면 +0.34, +0.01, -0.25, +0.13, +0.11로 부호가 오락가락해요. 평균 차이 +0.07%p를 **표준오차**(차이의 표준편차 ÷ √개수, "평균이 흔들리는 정도") 0.10%p로 나누면 0.7입니다. 평균 차이가 흔들림보다도 작다는 뜻이에요. 반대로 SGD+momentum과의 차이는 다섯 씨앗 모두 양수이고, 평균 +1.41%p를 표준오차 0.26%p로 나누면 5.5입니다. 이건 우연으로 보기 어렵죠.

이 "평균 ÷ 표준오차"가 통계에서 말하는 t 값입니다. 짝이 다섯 개일 때는 이 값이 대략 2.8을 넘으면 "우연이라고 보기 어렵다"(95% 기준)고 판단해요. 엄밀한 검정은 SciPy의 `scipy.stats.ttest_rel`이

해 주지만, 이 장에서는 다음 세 가지 습관이면 충분합니다.

- 결과는 **평균 ± 표준편차**와 씨앗 개수로 보고한다. "86.14 ± 0.44% (씨앗 5개)"처럼요.
- 두 설정을 비교할 때는 **같은 씨앗 목록**을 쓰고 짝지은 차이를 본다. 차이의 부호가 씨앗마다 바뀌면 결론을 미룬다.
- 탐색(18.5절)에서 뽑은 1등 설정은 **씨앗을 바꿔 다시 돌려** 확인한다. 수십 번 중 1등은 실력에 운이 더해진 값이라 다시 돌리면 대개 조금 내려갑니다.

Q

Q. 씨앗은 몇 개면 충분한가요? 정답은 없지만 흔히 3~5개로 시작합니다. 표준오차는 씨앗 수의 제곱근에 반비례해서, 씨앗을 4배로 늘려야 흔들림이 절반으로 줄어요. 그래서 "가리고 싶은 차이"가 크면 3개로도 충분하고, 0.1%p 차이를 가리려면 훨씬 많이 필요합니다. 학습 한 번이 며칠씩 걸리는 큰 모델은 씨앗을 여러 개 돌리기 어려워서, 대신 검증 데이터를 크게 잡고 차이가 큰 실험만 믿는 쪽으로 타협하기도 합니다.

18.7 데이터·코드·환경 버전 기록

데이터의 지문

설정과 씨앗이 같아도 **데이터가 바뀌면** 다른 실험입니다. 누군가 레이블 몇 개를 고쳤거나, 전처리 스크립트가 바뀌었거나, 내려받은 파일이 깨졌을 수 있어요. 데이터를 통째로 실험 폴더에 복사해 둘 수는 없으니, 18.2절에서 설정에 쓴 지문을 데이터에도 붙입니다.

```
import hashlib
import os
from pathlib import Path

import torch

from 실험도구 import 옷_데이터


def 파일_지문(파일: Path, 조각: int = 1 << 20) -> str:
    """큰 파일도 1MB씩 읽어 가며 SHA-256 해시를 낸다."""
    해시 = hashlib.sha256()
    with 파일.open("rb") as f:
```

```
        while 덩어리 := f.read(조각):
            해시.update(덩어리)
    return 해시.hexdigest()


def 텐서_지문(*텐서들: torch.Tensor) -> str:
    """텐서 안의 숫자(와 모양)가 같으면 같은 지문."""
    해시 = hashlib.sha256()
    for t in 텐서들:
        해시.update(str(tuple(t.shape)).encode())
        해시.update(t.contiguous().numpy().tobytes())
    return 해시.hexdigest()


for 파일 in sorted(Path(os.environ.get("BOOK_DATA_DIR", "data"), "FashionMNIST",
↪  "raw").glob("*-ubyte")):
    print(f"{파일.name:28} {파일.stat().st_size:>10,}바이트  {텐서_지문(파일)[:16]}")

(X_학습, y_학습), (X_검증, y_검증), _ = 옷_데이터(학습_개수=10_000)
print("학습 데이터 지문      :", 텐서_지문(X_학습, y_학습)[:16])
print("다시 나눠도           :", 텐서_지문(*옷_데이터(학습_개수=10_000)[0])[:16])
X_한점 = X_학습.clone()
X_한점[0, 0, 14, 14] += 1 / 255                    # 사진 한 장의 점 하나를 한 단계 밝게
print("점 하나만 바꿔도      :", 텐서_지문(X_한점, y_학습)[:16])
print("검증 데이터 지문      :", 텐서_지문(X_검증, y_검증)[:16])
```

```
t10k-images-idx3-ubyte        7,840,016바이트  5b4141f0afbad91e
t10k-labels-idx1-ubyte           10,008바이트  0402a96d92fd2663
train-images-idx3-ubyte      47,040,016바이트  c59f468a2f672dc8
train-labels-idx1-ubyte          60,008바이트  bad3541b69d91243
학습 데이터 지문      : 0a802f4d1fdbc90b
다시 나눠도           : 0a802f4d1fdbc90b
점 하나만 바꿔도      : 8fe268f623ee965d
검증 데이터 지문      : 49ad3f4da16478c6
```

`파일_지문`은 큰 파일을 1MB씩 나눠 읽으며 해시를 갱신합니다. 4,700만 바이트짜리 학습 사진 파

일을 한꺼번에 메모리에 올리지 않아도 돼요(`:=`는 1권 3장에서 본, 값을 대입하면서 그 값을 바로 쓰는 연산자입니다). 이 지문을 실험 기록에 함께 남겨 두면, 나중에 "같은 데이터였나?"를 지문 비교 한 번으로 확인할 수 있습니다. 공개 데이터셋은 배포하는 쪽에서 지문(체크섬)을 함께 알려 주기도 해서, 내려받은 파일이 온전한지 확인하는 데에도 씁니다.

텐서_지문은 파일 대신 **전처리를 마친 텐서**의 지문입니다. 원본 파일이 같아도 나누는 방법이나 정규화가 바뀌면 모델이 보는 데이터가 달라지니, 실제로 학습에 들어가는 숫자의 지문이 더 정확해요. 다시 나눠도 지문이 같고, 사진 한 장의 점 하나를 1/255만큼 밝게 했을 뿐인데 지문이 완전히 달라졌습니다. 데이터 파일 자체를 버전별로 보관하고 싶다면 git과 함께 쓰는 DVC 같은 오픈소스 도구도 있습니다.

코드의 버전: git 커밋

코드는 git 커밋 번호가 가장 좋은 지문입니다. 실험을 시작할 때 지금의 커밋 번호를 기록해 두면 나중에 그 시점의 코드로 정확히 돌아갈 수 있어요. 단, **커밋하지 않은 수정**이 있으면 커밋 번호만으로는 그 코드를 되살릴 수 없으니 그 여부도 함께 남깁니다. 연습용 저장소를 임시 폴더에 만들어 확인해 보겠습니다.

```
import os
import subprocess
import tempfile
from pathlib import Path


def 코드_버전(폴더: Path) -> dict:
    """git 커밋 번호와, 커밋하지 않은 수정이 남아 있는지를 알려 준다."""
    def git(*명령) -> str:
        return subprocess.run(["git", *명령], cwd=폴더, capture_output=True, text=True,
                              check=True).stdout.strip()
    try:
        커밋 = git("rev-parse", "HEAD")
    except (subprocess.CalledProcessError, FileNotFoundError):
        return {"커밋": None, "수정됨": None}        # git 저장소가 아니거나 git이 없음
    바뀐_파일 = git("status", "--porcelain")         # 커밋 안 한 변경이 있으면 한 줄 이상
    return {"커밋": 커밋[:12], "수정됨": bool(바뀐_파일)}
```

```
# 연습용 저장소를 임시 폴더에 만들어 본다 (날짜·시간대·이름을 고정해 커밋 번호가 늘 같게)
고정 = {"GIT_AUTHOR_NAME": "책", "GIT_AUTHOR_EMAIL": "book@example.com",
        "GIT_AUTHOR_DATE": "2026-01-01T00:00:00+09:00", "GIT_COMMITTER_NAME": "책",
        "GIT_COMMITTER_EMAIL": "book@example.com", "GIT_COMMITTER_DATE":
        ↪ "2026-01-01T00:00:00+09:00"}
os.environ.update(고정)
with tempfile.TemporaryDirectory() as 임시:
    폴더 = Path(임시)
    print("git 저장소가 아닐 때:", 코드_버전(폴더))
    subprocess.run(["git", "init", "-q", "-b", "main"], cwd=폴더, check=True)
    (폴더 / "학습.py").write_text("학습률 = 0.001\n", encoding="utf-8")
    subprocess.run(["git", "add", "학습.py"], cwd=폴더, check=True)
    subprocess.run(["git", "commit", "-q", "-m", "첫 실험 코드"], cwd=폴더, check=True)
    print("커밋한 직후        :", 코드_버전(폴더))
    (폴더 / "학습.py").write_text("학습률 = 0.003\n", encoding="utf-8")   # 커밋 없이 고침
    print("커밋 없이 고친 뒤  :", 코드_버전(폴더))
```

```
git 저장소가 아닐 때: {'커밋': None, '수정됨': None}
커밋한 직후        : {'커밋': 'bc4959b5a3f4', '수정됨': False}
커밋 없이 고친 뒤  : {'커밋': 'bc4959b5a3f4', '수정됨': True}
```

`코드_버전`은 1권 17장에서 본 `subprocess`로 git 명령을 실행합니다. `git rev-parse HEAD`는 지금 커밋 번호를, `git status --porcelain`은 커밋하지 않은 변경 목록을 한 줄씩 돌려줘요. 목록이 비어 있지 않으면 "수정됨"입니다. 커밋한 직후에는 `수정됨: False`였다가, 파일을 고치자 커밋 번호는 그대로인데 `수정됨: True`가 됐습니다. 이 상태로 실험하면 "커밋 bc4959b5a3f4의 코드"라고 기록해도 실제 코드는 그와 다르다는 뜻이에요. 그래서 실험 전에 커밋하는 습관을 들이거나, 수정된 상태라면 바뀐 내용(`git diff`)까지 실험 폴더에 저장해 둡니다.

예제에서 작성자 이름과 날짜를 환경 변수로 고정한 것은 이 책의 출력과 여러분의 출력이 같게 나오도록 하기 위해서입니다. git의 커밋 번호는 내용뿐 아니라 작성자와 시각까지 넣어 계산하거든요. 날짜 끝의 `+09:00`(한국 시간)도 빼면 안 됩니다. 시간대를 적지 않으면 git이 컴퓨터에 설정된 시간대를 붙여서, 다른 시간대의 컴퓨터에서는 커밋 번호가 달라져요. 실제 프로젝트에서는 이 부분이 필요 없습니다. git 저장소가 아닌 폴더에서는 오류 대신 `None`을 돌려주게 해서, 기록 코드 때문에 학습이 멈추지 않게 했어요.

실행 환경

마지막은 환경입니다. 같은 코드와 데이터라도 PyTorch 버전이나 장치가 다르면 끝자리가 달라질 수 있어요(8장 8.10절).

```
import hashlib
import importlib.metadata
import platform
import sys
from pathlib import Path

import torch


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")


def 환경_기록(패키지들: list[str], 잠금_파일: Path = Path("uv.lock")) -> dict:
    """결과에 영향을 줄 수 있는 실행 환경을 딕셔너리 하나로."""
    기록 = {
        "파이썬": platform.python_version(),
        "운영체제": f"{platform.system()} {platform.release()} ({platform.machine()})",
        "패키지": {이름: importlib.metadata.version(이름) for 이름 in 패키지들},
        "쓸 수 있는 장치": str(장치_고르기()),
        "CPU 스레드": torch.get_num_threads(),
        "CUDA": torch.version.cuda,                        # CUDA가 없는 PyTorch면 None
    }
    if 잠금_파일.exists():                                   # uv.lock이 있으면 그 지문까지
        기록["uv.lock"] = hashlib.sha256(잠금_파일.read_bytes()).hexdigest()[:16]
    return 기록
```

```
for 키, 값 in 환경_기록(["torch", "numpy", "torchvision", "optuna", "mlflow"]).items():
    print(f"{키}: {값}")
print("실행한 명령:", " ".join([Path(sys.argv[0]).name, *sys.argv[1:]]))
```

```
파이썬: 3.12.10
운영체제: Darwin 24.6.0 (arm64)
패키지: {'torch': '2.14.1', 'numpy': '2.5.4', 'torchvision': '0.29.1', 'optuna': '5.0.0', 'mlflow':
↪ '3.17.2'}
쓸 수 있는 장치: mps
CPU 스레드: 12
CUDA: None
실행한 명령: b16_env.py
```

`platform`과 `importlib.metadata`로 파이썬·운영체제·패키지 버전을, PyTorch에게 쓸 수 있는 장치와 CPU 스레드 수, CUDA 버전을 물었습니다. 이 책의 환경은 Apple M4 Max라 장치는 `mps`, CUDA는 `None`으로 나와요. 마지막 줄은 실행한 파일 이름과 명령줄 인자라서, 여러분 화면에는 이 코드를 저장한 파일 이름이 나옵니다. CPU 스레드 수를 남기는 이유는 CPU 계산도 스레드 수에 따라 덧셈 순서가 바뀌어 끝자리가 달라질 수 있기 때문입니다.

패키지 버전을 하나하나 적는 것보다 확실한 방법은 1권 14장의 `uv.lock`입니다. 설치된 모든 패키지의 정확한 버전이 담겨 있으니, 실험할 때의 `uv.lock`을 git에 커밋해 두고 그 지문을 기록하면 환경 전체를 되살릴 수 있어요. `환경_기록`은 실행한 폴더에 `uv.lock`이 있으면 그 지문까지 남기는데, 이 예제는 `uv.lock`이 없는 폴더에서 실행해서 그 줄이 빠졌습니다.

재현성 보고서에 들어갈 것

지금까지 남긴 것들을 모으면 실험 하나의 **재현성 보고서**가 됩니다. 논문의 "실험 설정" 절이나 회사의 실험 보고서에 들어가는 내용이 바로 이것이에요.

항목	내용	출처
결과	지표의 평균 ± 표준편차, 씨앗 개수와 목록	18.6절
비교	기준 실험과 같은 씨앗끼리 짝지은 차이	18.6절

항목	내용	출처
설정	합쳐진 최종 설정 전체(파일 + 명령줄)	18.2절
선택 과정	탐색 범위와 횟수, 무엇으로 골랐나(검증), 시험은 한 번	18.5절
데이터	데이터 지문, 나누는 방법과 개수	18.7절
코드	커밋 번호와 수정 여부	18.7절
환경	파이썬·PyTorch 버전, 장치, `uv.lock` 지문	18.7절

18.8 실습: 작은 실험 묶음과 재현성 보고서

이 장에서 배운 것을 모두 써서, 작은 실험 묶음을 체계적으로 돌리고 보고서까지 만들어 보겠습니다. 질문은 이렇습니다. "학습 사진이 1만 장뿐일 때, 9장의 CNN에 무엇을 바꾸면 나아질까?" 6장 MLP를 비교 기준으로 함께 넣고, 실험마다 씨앗 세 개로 돌립니다.

1. 실험 목록을 YAML로 적는다: 기준 설정, 씨앗 목록, 기준에서 바꿀 값들 (18.2절)
2. 실험마다 설정 지문으로 폴더를 만들고, 설정·환경·데이터 지문·코드 지문을 남긴다 (18.2절, 18.7절)
3. 에폭별 기록은 JSON Lines로, 검증 정확도가 가장 좋았던 모델은 `best.pt`로 (18.3절, 18.4절)
4. 이미 끝난 실험은 건너뛴다. 중간에 끊겨도 다시 실행하면 남은 것만 돈다
5. 모든 폴더를 모아 평균 ± 표준편차와 짝지은 차이로 보고서를 만들고, 고른 실험만 시험 데이터로 확인한다 (18.6절)

먼저 실험을 돌립니다. 학습 1만 장, 4에폭, CPU라서 이 책의 환경에서 2분쯤 걸렸어요. 이 코드는 `Path(__file__)`로 자기 파일의 지문을 남기기 때문에, 노트북 칸에 붙여 넣지 말고 `실험도구.py`와 같은 폴더에 파일(예: `실습_실행.py`)로 저장해 `uv run python 실습_실행.py`로 실행하세요.

```
import hashlib
import json
import platform
```

```
from dataclasses import asdict, replace
from pathlib import Path

import torch
import yaml

import 실험도구
from 실험도구 import 로더_만들기, 모델_만들기, 씨앗_고정, 옷_데이터, 평가, 학습설정, 한_에폭_학습

실험묶음 = yaml.safe_load("""
기준: {모델: cnn, 에폭: 4, 학습_개수: 10000}
씨앗들: [0, 1, 2]
실험들:
  6장 MLP: {모델: mlp}
  9장 CNN: {}
  CNN + 드롭아웃 0.3: {드롭아웃: 0.3}
  CNN + 학습률 0.003: {학습률: 0.003}
""")


def 지문(글: str | bytes) -> str:
    return hashlib.sha256(글 if isinstance(글, bytes) else 글.encode()).hexdigest()[:12]


def 실험_하나(이름: str, 설정: 학습설정, 데이터, 뿌리: Path = Path("runs/실습")) -> None:
    """실험 폴더 하나에 설정·환경·에폭별 기록·최고 모델·결과를 남긴다. 이미 끝났으면 건너뛴다."""
    폴더 = 뿌리 / f"{설정.모델}_{지문(json.dumps(asdict(설정), sort_keys=True))}"
    if (폴더 / "결과.json").exists():
        return
    폴더.mkdir(parents=True, exist_ok=True)
    (X_학습, y_학습), (X_검증, y_검증), _ = 데이터
    (폴더 / "설정.yaml").write_text(yaml.safe_dump({"이름": 이름, **asdict(설정)},
↪   allow_unicode=True,
                                                   sort_keys=False), encoding="utf-8")
    환경 = {"파이썬": platform.python_version(), "torch": torch.__version__, "장치": "cpu",
            "코드_지문": 지문(Path(실험도구.__file__).read_bytes() + Path(__file__).read_bytes()),
```

```
            "데이터_지문": 지문(X_학습.numpy().tobytes() + y_학습.numpy().tobytes())}
    (폴더 / "환경.json").write_text(json.dumps(환경, ensure_ascii=False, indent=1),
↪   encoding="utf-8")

    씨앗_고정(설정.씨앗)
    모델 = 모델_만들기(설정)
    옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=설정.학습률, weight_decay=설정.가중치_감
↪   쇠)
    로더 = 로더_만들기(X_학습, y_학습, 설정.배치_크기, 설정.씨앗)
    최고 = {"검증_정확도": 0.0}
    with (폴더 / "기록.jsonl").open("w", encoding="utf-8") as 기록:
        for 에폭 in range(1, 설정.에폭 + 1):
            학습_손실 = 한_에폭_학습(모델, 로더, 옵티마이저)
            검증_손실, 검증_정확도 = 평가(모델, X_검증, y_검증)
            기록.write(json.dumps({"에폭": 에폭, "학습_손실": 학습_손실, "검증_손실": 검증_손실,
                                   "검증_정확도": 검증_정확도}) + "\n")
            if 검증_정확도 > 최고["검증_정확도"]:
                최고 = {"검증_정확도": 검증_정확도, "에폭": 에폭}
                torch.save(모델.state_dict(), 폴더 / "best.pt")
    (폴더 / "결과.json").write_text(json.dumps(최고, ensure_ascii=False), encoding="utf-8")


데이터 = 옷_데이터(학습_개수=실험묶음["기준"]["학습_개수"])
기준 = 학습설정(**실험묶음["기준"])
for 이름, 바꿀값 in 실험묶음["실험들"].items():
    for 씨앗 in 실험묶음["씨앗들"]:
        실험_하나(이름, replace(기준, **바꿀값, 씨앗=씨앗), 데이터)
    print(f"[완료] {이름}")
폴더들 = sorted(p for p in Path("runs/실습").iterdir() if p.is_dir())
print(f"실험 폴더 {len(폴더들)}개, 예: {폴더들[0].name} →", sorted(p.name for p in 폴더들
↪   [0].iterdir()))
```

```
[완료] 6장 MLP
[완료] 9장 CNN
[완료] CNN + 드롭아웃 0.3
[완료] CNN + 학습률 0.003
```

```
실험 폴더 12개, 예: cnn_0c631cdec376 → ['best.pt', '결과.json', '기록.jsonl', '설정.yaml', '환
↪ 경.json']
```

실험 목록이 파이썬 코드가 아니라 YAML 글자로 적혀 있다는 점을 보세요. **실험들**의 각 줄은 "기준에서 바꿀 값"만 담고 있어서, 새 실험을 추가하려면 한 줄만 더 쓰면 됩니다. 4가지 실험 × 씨앗 3개로 폴더 12개가 생겼고, 각 폴더에는 **설정.yaml**, **환경.json**, **기록.jsonl**, **best.pt**, **결과.json**이 들어 있어요. **결과.json**은 학습이 끝까지 돌았을 때만 마지막에 쓰기 때문에, 이 파일이 있는지로 "끝난 실험"을 판단합니다. 같은 코드를 다시 실행하면 12개 모두 건너뛰고, 중간에 끊겼다면 끝나지 않은 실험만 다시 돌아요.

환경.json의 코드 지문은 **실험도구.py**와 이 스크립트 파일의 내용을 합쳐 해시한 것입니다. git을 쓰는 프로젝트라면 18.7절의 커밋 번호를 함께 넣으면 됩니다. 이 장의 첫머리에서 본 **model_final_v2_lr0.001_진짜최종.pt** 대신, 이제 모든 모델 파일 옆에 그 모델을 만든 설정과 환경이 함께 있습니다.

이제 폴더들을 모아 보고서를 만듭니다.

```
import json
import statistics
from collections import defaultdict
from pathlib import Path

import torch
import yaml

from 실험도구 import 모델_만들기, 옷_데이터, 평가, 학습설정

# 1) 실험 폴더를 모두 읽어 이름별로 묶는다
묶음 = defaultdict(list)
for 폴더 in sorted(p for p in Path("runs/실습").iterdir() if p.is_dir()):
    설정값 = yaml.safe_load((폴더 / "설정.yaml").read_text(encoding="utf-8"))
    결과 = json.loads((폴더 / "결과.json").read_text(encoding="utf-8"))
    환경 = json.loads((폴더 / "환경.json").read_text(encoding="utf-8"))
    묶음[설정값.pop("이름")].append({"설정": 설정값, "결과": 결과, "환경": 환경, "폴더": 폴더})

# 2) 이름마다 씨앗 순으로 정렬해 평균과 표준편차를 내고, 검증 정확도 순으로
```

```
정확도 = {}
for 이름, 실행들 in 묶음.items():
    실행들.sort(key=lambda r: r["설정"]["씨앗"])
    정확도[이름] = [r["결과"]["검증_정확도"] * 100 for r in 실행들]
순서 = sorted(정확도, key=lambda 이름: statistics.mean(정확도[이름]), reverse=True)

보고서 = ["# 재현성 보고서: FashionMNIST 작은 실험 묶음", "",
          "¦ 실험 ¦ 씨앗 0, 1, 2의 검증 정확도(%) ¦ 평균 ± 표준편차 ¦ 9장 CNN과 같은 씨앗끼리 차이
          ↪  (%p) ¦",
          "¦---¦---¦---¦---¦"]
for 이름 in 순서:
    값들 = 정확도[이름]
    차이 = [a - b for a, b in zip(값들, 정확도["9장 CNN"])]
    보고서.append(f"¦ {이름} ¦ {', '.join(f'{v:.2f}' for v in 값들)} ¦ "
                  f"{statistics.mean(값들):.2f} ± {statistics.stdev(값들):.2f} ¦ "
                  f"{', '.join(f'{d:+.2f}' for d in 차이)} ¦")

# 3) 검증으로 고른 실험 하나만, 시험 데이터로 마지막에 한 번
고른_이름 = 순서[0]
실행들 = 묶음[고른_이름]
_, _, (X_시험, y_시험) = 옷_데이터()
시험 = []
for r in 실행들:
    모델 = 모델_만들기(학습설정(**r["설정"]))
    모델.load_state_dict(torch.load(r["폴더"] / "best.pt", weights_only=True))
    시험.append(평가(모델, X_시험, y_시험)[1] * 100)
환경 = 실행들[0]["환경"]
같은_환경 = all(r["환경"] == 환경 for 실행 in 묶음.values() for r in 실행)
보고서 += ["", f"- 고른 실험: {고른_이름} (검증 평균이 가장 높음)",
           f"- 시험 정확도: {statistics.mean(시험):.2f} ± {statistics.stdev(시험):.2f}% (씨앗
           ↪ {len(시험)}개)",
           f"- 공통 설정: {json.dumps({k: v for k, v in 실행들[0]['설정'].items() if k != '씨앗'},
           ↪ ensure_ascii=False)}",
           f"- 환경: 파이썬 {환경['파이썬']}, PyTorch {환경['torch']}, 장치 {환경['장치']}",
           f"- 코드 지문 {환경['코드_지문']}, 데이터 지문 {환경['데이터_지문']}, 모든 실험이 같은
           ↪ 환경: {같은_환경}"]
```

```
Path("runs/실습/보고서.md").write_text("\n".join(보고서) + "\n", encoding="utf-8")
print("\n".join(보고서))
```

```
# 재현성 보고서: FashionMNIST 작은 실험 묶음

| 실험 | 씨앗 0, 1, 2의 검증 정확도(%) | 평균 ± 표준편차 | 9장 CNN과 같은 씨앗끼리 차이(%p) |
|---|---|---|---|
| CNN + 학습률 0.003 | 87.74, 88.00, 86.28 | 87.34 ± 0.93 | +0.96, +1.44, +0.66 |
| 9장 CNN | 86.78, 86.56, 85.62 | 86.32 ± 0.62 | +0.00, +0.00, +0.00 |
| CNN + 드롭아웃 0.3 | 86.90, 86.44, 85.50 | 86.28 ± 0.71 | +0.12, -0.12, -0.12 |
| 6장 MLP | 84.52, 82.86, 83.48 | 83.62 ± 0.84 | -2.26, -3.70, -2.14 |

- 고른 실험: CNN + 학습률 0.003 (검증 평균이 가장 높음)
- 시험 정확도: 87.08 ± 0.88% (씨앗 3개)
- 공통 설정: {"모델": "cnn", "학습률": 0.003, "배치_크기": 64, "에폭": 4, "드롭아웃": 0.0, "가중치_
↪ 감쇠": 0.0, "학습_개수": 10000}
- 환경: 파이썬 3.12.10, PyTorch 2.14.1, 장치 cpu
- 코드 지문 56314e6a70dd, 데이터 지문 e48b61768093, 모든 실험이 같은 환경: True
```

보고서는 마크다운 표로 만들어 `runs/실습/보고서.md`에 저장했습니다. 그대로 문서나 발표 자료에 붙일 수 있어요. 결과를 읽어 보겠습니다.

- **6장 MLP는 확실히 뒤처집니다.** 평균 83.62%로 9장 CNN보다 2.7%p 낮고, 같은 씨앗끼리 차이도 -2.26, -3.70, -2.14로 셋 다 음수예요. 9장에서 본 CNN의 장점이 학습 사진이 적을 때도 그대로입니다.
- **드롭아웃 0.3은 효과가 없었습니다.** 평균이 86.28%로 기준 86.32%와 거의 같고, 짝지은 차이가 +0.12, -0.12, -0.12로 부호가 오락가락합니다. 각 폴더의 `기록.jsonl`을 열어 보면, 9장 CNN의 4에폭째 학습 손실은 0.35~0.38, 검증 손실은 0.36~0.46으로 크게 벌어지지 않았어요. 과적합이 뚜렷해지기 전이라(8장 8.1절) 드롭아웃이 할 일이 적었던 것으로 보입니다. "효과 없음"도 기록해 둘 가치가 있는 결과입니다.
- **학습률 0.003은 나아졌습니다.** 평균 87.34%로 기준보다 1.02%p 높고, 짝지은 차이가 +0.96, +1.44, +0.66으로 셋 다 양수예요. 씨앗 세 개뿐이라 단정하기는 이르지만, 짧게 학습할 때는 조금 큰 학습률로 빨리 내려가는 쪽이 유리했다고 볼 수 있습니다.

표준편차가 0.6~0.9%p로 18.6절의 MLP 실험보다 큰 것도 눈에 띕니다. 검증 사진이 5,000장이라 시험 1만 장보다 적고, 학습 사진도 1만 장뿐이라 씨앗에 따라 더 흔들린 거예요. 실제로 CNN 세 실

험에서는 모두 씨앗 2가 가장 낮습니다. 이렇게 씨앗의 운이 실험 전체에 함께 들어가기 때문에, 평균끼리 비교하는 것보다 짝지은 차이가 더 또렷한 답을 줍니다.

마지막으로 검증 평균으로 고른 "CNN + 학습률 0.003"만 시험 데이터로 확인했더니 87.08 ± 0.88%였습니다. 검증 평균 87.34%와 비슷하니, 검증으로 고른 결과를 믿을 만해요. 시험 데이터는 이렇게 **고른 다음 한 번만** 엽니다(7장 7.8절). 9장 9.5절의 같은 CNN이 91.01%를 냈던 것과 비교하면 4%p 쯤 낮은데, 그때는 학습 사진 6만 장으로 5에폭을 학습했으니 이상한 일이 아니에요. 보고서의 공통 설정 줄에 학습 1만 장, 4에폭이 적혀 있으니, 이 보고서를 받은 사람도 숫자를 오해하지 않습니다. 이것이 보고서에 설정을 함께 적는 이유예요.

더 해 볼 만한 것들을 남겨 둡니다.

- 실험 목록에 `CNN + 학습률 0.003 + 드롭아웃 0.3`과 `에폭: 8`인 실험을 더해 보세요. 에폭을 늘리면 드롭아웃의 효과가 달라지는지 확인할 수 있습니다. 이미 돈 12개는 건너뛰고 새 실험만 돌 거예요.
- 18.5절의 Optuna로 CNN의 학습률과 드롭아웃을 탐색한 뒤, 찾은 설정을 이 실험 목록에 넣어 씨앗 세 개로 다시 확인해 보세요. 탐색의 1등이 다시 돌려도 1등인지 보는 것이 이 장의 핵심입니다.
- `실험_하나`에 18.4절의 MLflow 기록을 더해서, 12개 실험을 `mlflow ui`에서 곡선으로 비교해 보세요.

18.9 정리와 다음 장 미리보기

이 장에서 실험을 다시 할 수 있고, 많이 할 수 있고, 믿을 수 있게 만드는 습관을 익혔습니다.

- **실험의 구성 요소**: 설정, 코드, 데이터, 환경, 난수, 결과. 하나라도 빠지면 다시 할 수 없다
- **설정**: `dataclass(frozen=True, replace, asdict)`로 묶고, YAML에서 읽을 때는 이름과 자료형을 검사한다(`3e-4`는 글자로 읽힌다). 기본값 < 설정 파일 < 명령줄(`argparse`) 순서로 덮어쓰고, 합쳐진 최종 설정을 저장한다
- **실험 폴더**: 설정 지문(SHA-256)으로 이름을 붙여 같은 실험을 두 번 하지 않는다
- **체크포인트**: 모델·옵티마이저에 더해 스케줄러, 섞기 생성기, 전역 난수, 최고 점수까지. NumPy 난수 상태는 리스트로 바꿔 `weights_only=True`를 지킨다. 임시 파일에 쓰고 `os.replace`, `last.pt`와 `best.pt`
- **기록**: JSON Lines(덧붙이고 `flush`), TensorBoard(`SummaryWriter`, 곡선 겹쳐 보기), MLflow(파라미터·메트릭·아티팩트, `search_runs`로 표 비교)
- **탐색**: Optuna의 목적함수, `suggest_float(log=True)`, TPE가 무작위보다 좋은 곳을 더 자

주 본다. `report·should_prune`과 `MedianPruner`로 가지치기, SQLite 저장소로 이어 가기, 중요도로 다음 탐색 계획

- **여러 씨앗**: 평균 ± 표준편차와 씨앗 개수로 보고. 같은 씨앗끼리 짝지은 차이와 표준오차로 판단. 탐색의 1등은 다시 돌려 확인
- **버전 기록**: 데이터 지문(파일·텐서), git 커밋과 수정 여부, 파이썬·패키지 버전과 장치, `uv.lock`

지금까지 우리 모델은 늘 PyTorch 안에서 학습하고 PyTorch 안에서 예측했습니다. 그런데 이 장의 실습에서 만든 `best.pt`를 휴대폰 앱이나 다른 언어로 만든 서비스에 넣으려면 어떻게 해야 할까요? 다음 19장에서는 모델을 ONNX라는 공통 형식으로 내보내 PyTorch 없이 실행하고, 숫자를 작은 정수로 바꿔 담는 **양자화**로 모델을 가볍고 빠르게 만듭니다. 그때도 "정확도가 얼마나 떨어졌나"를 재는 일이 핵심인데, 이 장에서 익힌 기록과 비교 습관이 그대로 쓰여요.

19

모델 내보내기와 경량화: ONNX와 양자화

학습 목표

- 학습한 모델을 PyTorch 밖에서 돌려야 하는 이유와, ONNX 파일에 무엇이 담기는지 설명하기
- `torch.onnx.export`로 모델을 내보내고, 묶음 크기 같은 차원을 자유롭게 열어 두기
- ONNX Runtime으로 추론해 PyTorch 출력과 비교하고, torch 없이 모델 돌리기
- 내보낼 수 없는 코드(데이터에 따라 갈리는 `if`, 파이썬 쪽 계산)를 알아보고 고치기
- 그래프를 들여다보며 배치 정규화 접어 넣기와 연산 합치기 같은 최적화를 확인하기
- 배율·영점·정수 행렬 곱·배율 단위로 양자화의 원리를 설명하기
- torchao와 ONNX Runtime으로 양자화하고 크기·속도·정확도 표 만들기
- 가지치기와 지식 증류의 아이디어를 작은 실험으로 확인하기

들어가며: 모델을 다 만들었는데, 어디서 돌리죠?

9장 9.10절에서 만든 작은 ResNet은 옷 사진을 93.04% 맞혔습니다. 이 모델을 친구가 만드는 쇼핑 앱에 넣고 싶다고 해 봅시다. 친구가 묻습니다. "그 모델 파일 주면 돼? 우리 앱은 파이썬이 아니라 다른 언어로 짰는데."

지금 가진 것은 `fashion_resnet.pt` 하나예요. 6장 6.10절에서 본 것처럼 이 파일에는 숫자(`state_dict`)만 있고, 그 숫자를 어떤 순서로 계산할지는 우리가 짠 파이썬 클래스 `작은_ResNet`에 들어 있습니다. 그러니 이 파일을 쓰려면 파이썬과 PyTorch, 그리고 우리 코드가 모두 필요해요. PyTorch가 얼마나 큰지부터 재 보겠습니다.

```
from importlib import metadata

# 설치된 패키지 파일이 디스크에서 차지하는 크기 (이 책의 맥, 파이썬 3.12)
for 이름 in ["torch", "onnxruntime", "onnx", "numpy"]:
    배포판 = metadata.distribution(이름)
    크기 = sum(파일.size or 0 for 파일 in 배포판.files or [])
    print(f"{이름:12} {배포판.version:8} {크기 / 1024**2:6.1f} MB")
```

```
torch        2.14.1    498.4 MB
onnxruntime  1.31.0     74.4 MB
onnx         1.23.2     25.0 MB
numpy        2.5.4      19.6 MB
```

이 맥에서 PyTorch는 약 500MB입니다. NVIDIA GPU용 리눅스 판은 CUDA 라이브러리가 따로 붙어서 몇 GB가 되기도 해요. 그런데 이 장에서 쓸 추론 엔진 **ONNX Runtime**은 74MB 남짓이고, 무엇보다 PyTorch 없이 돌아갑니다. 파이썬이 아닌 C++, C#, Java, JavaScript에서도 쓸 수 있어요.

음악에 비유해 보겠습니다. 작곡가가 피아노 앞에서 곡을 만들었다고 그 곡을 피아노로만 연주할 수 있는 건 아니죠. **악보**로 적어 두면 바이올린 연주자도, 오케스트라도 같은 곡을 연주합니다. PyTorch는 곡을 만드는(학습하는) 작업실이고, 이 장에서 만들 **ONNX 파일은 모델의 악보**예요. 악보를 받은 쪽은 작곡가의 작업실이 없어도 자기 악기(런타임)로 연주합니다.

이 장의 목표는 두 가지입니다.

- **밖으로**: 모델을 ONNX 파일로 내보내고, ONNX Runtime으로 PyTorch와 같은 답을 내는지 확인합니다.
- **작게**: 14장 14.6절에서 원리만 흉내 냈던 **양자화**를 진짜 도구로 해서, 모델을 4분의 1 크기로 줄이고 더 빠르게 돌립니다.

Q

Q. 이 장에 필요한 패키지는 어떻게 설치하나요? 프로젝트 폴더에서 `uv add onnx onnxscript onnxruntime torchao`를 실행하세요. `onnx`는 ONNX 파일을 읽고 쓰는 라이브러리, `onnxscript`는 PyTorch의 내보내기 기능이 안에서 쓰는 도구(없으면 내보내기가 오류를 냅니다), `onnxruntime`은 추론 엔진, `torchao`는 19.9절의 PyTorch 양자화 도구입니다. 이 장은 파이썬 3.12, PyTorch 2.14.1, onnx 1.23.2, onnxscript 0.7.2, onnxruntime 1.31.0, torchao 0.18.0에서 실행해 확인했어요. 라이선스는 onnx

가 Apache-2.0, onnxscript와 onnxruntime이 MIT, torchao가 BSD-3-Clause입니다. 9장의 `옷도구.py`, `fashion_cnn.pt`(9.5절), `fashion_resnet.pt`(9.10절)를 이 장의 폴더에 함께 두세요.

19.1 ONNX: 모델의 악보

ONNX(Open Neural Network Exchange)는 신경망을 담는 공개 파일 형식입니다. 여러 회사와 단체가 함께 만든 표준이라, 한 도구에서 만든 ONNX 파일을 다른 도구에서 읽을 수 있어요. ONNX 파일 하나에는 네 가지가 들어 있습니다.

- **계산 그래프**: `Conv` → `Relu` → `MaxPool` → … 처럼 어떤 연산을 어떤 순서로 이어서 하는지. 그래프의 마디 하나하나를 **노드(node)** 라고 부릅니다.
- **가중치**: 학습으로 정해진 숫자들. ONNX에서는 **이니셜라이저(initializer)** 라고 불러요.
- **입출력 정보**: 입력과 출력의 이름, 모양, 자료형.
- **연산자 판(opset) 번호**: `Conv`나 `Relu` 같은 연산의 정확한 뜻은 판마다 조금씩 다를 수 있어서, 이 파일이 몇 번째 판의 규칙으로 적혔는지 함께 적어 둡니다.

악보에 음표의 뜻이 정해져 있듯, ONNX는 `Conv`, `Gemm`(행렬 곱 + 덧셈) 같은 표준 연산자 수백 개의 뜻을 정해 두었습니다. 그래서 런타임은 파이썬 코드를 몰라도 그래프만 보고 계산할 수 있어요.

하나 더 중요한 점이 있습니다. 3장 3.11절과 6장 6.10절에서 "모르는 곳에서 받은 pickle 파일(`.pt`, joblib)은 여는 순간 그 안의 코드가 실행될 수 있다"고 했죠. ONNX 파일은 pickle이 아니라 **프로토콜 버퍼(protobuf)** 라는 데이터 형식이라, 파일을 읽는다고 그 안의 파이썬 코드가 실행되지는 않습니다. 담긴 것은 연산자 이름과 숫자뿐이에요. 다른 사람, 다른 언어와 모델을 주고받을 때 ONNX를 많이 쓰는 이유 중 하나입니다.

어디서 돌리나	무엇으로 돌리나 (예)
파이썬 서버(PyTorch 없이)	ONNX Runtime (20장에서 FastAPI와 함께)
다른 언어로 짠 프로그램	ONNX Runtime의 C++, C#, Java, JavaScript API
휴대폰, 작은 기기	ONNX Runtime 모바일판, ExecuTorch, LiteRT, Core ML (19.12절)

어디서 돌리나	무엇으로 돌리나 (예)
특정 회사의 가속기	TensorRT(NVIDIA), OpenVINO(Intel) 같은 전용 도구

19.2 torch.onnx.export로 내보내기

첫 내보내기

9장 9.5절의 CNN(`fashion_cnn.pt`, 시험 정확도 91.01%)부터 내보내 보겠습니다. 필요한 것은 모델과, 모델에 넣을 **예시 입력** 하나예요.

```
from pathlib import Path

import torch

from 옷도구 import 옷CNN

모델 = 옷CNN()
모델.load_state_dict(torch.load("fashion_cnn.pt", weights_only=True))   # 9장 9.5절의 CNN
모델.eval()                                          # 내보내기 전에 꼭 평가 모드로 (6장 6.8절)

예시_입력 = torch.zeros(1, 1, 28, 28)               # 사진 한 장 모양이면 값은 무엇이든 된다
결과 = torch.onnx.export(모델, (예시_입력,), "fashion_cnn_first.onnx", verbose=False)
print(type(결과).__name__)

for 파일 in ["fashion_cnn.pt", "fashion_cnn_first.onnx", "fashion_cnn_first.onnx.data"]:
    print(f"{파일:28} {Path(파일).stat().st_size / 1024:7.1f} KB")
```

```
ONNXProgram
fashion_cnn.pt                1650.8 KB
fashion_cnn_first.onnx          16.1 KB
fashion_cnn_first.onnx.data   1646.6 KB
```

`torch.onnx.export`가 `ONNXProgram`이라는 객체를 돌려주고 파일 두 개를 만들었습니다. 무슨 일이

있었는지 순서대로 보면 이렇습니다.

1. **그래프 따기**: 예시 입력을 모델에 한 번 흘려보내면서, 어떤 텐서 연산이 어떤 순서로 일어나는지 기록합니다. PyTorch의 `torch.export`가 이 일을 해요. 예시 입력의 값은 상관없고 **모양과 자료형**만 중요하기 때문에 `torch.zeros`를 넣었습니다.
2. **분해**: 기록한 PyTorch 연산을 더 기본적인 연산으로 풀어 둡니다.
3. **번역**: 기본 연산을 ONNX 표준 연산자로 바꿉니다. 이때 `onnxscript`가 쓰여요.
4. **최적화**: 쓸데없는 노드를 지우고 합칠 수 있는 것을 합칩니다(19.6절에서 확인합니다).

이 방식은 PyTorch 2.9부터 기본값이 됐습니다. 그 전에는 TorchScript라는 다른 방식으로 그래프를 땄는데, 지금도 `dynamo=False`를 주면 쓸 수 있지만 "곧 없어질 기능"이라는 경고가 나와요. 인터넷의 옛 예제에서 이 옵션이나 `torch.jit`가 보이면 오래된 방식이라고 생각하면 됩니다.

Q

Q. 내보내기 전에 왜 `model.eval()`을 하나요? 6장 6.8절에서 본 것처럼 드롭아웃과 배치 정규화는 학습 모드와 평가 모드에서 다르게 동작합니다. 내보내기는 그 순간의 동작을 그래프로 굳히니, 학습 모드에서 내보내면 드롭아웃이 켜진 채로 굳어 버려요. 실제로 학습 모드인 모델을 내보내면 PyTorch가 "평가 모드로 바꾸는 것을 권한다"는 경고를 냅니다. 배포할 모델은 언제나 평가 모드로 내보내세요.

가중치는 어디에: .onnx와 .onnx.data

파일 크기를 보세요. `fashion_cnn_first.onnx`는 16KB밖에 안 되고, 1.6MB짜리 가중치는 옆의 `.onnx.data` 파일에 따로 들어갔습니다. `torch.onnx.export`의 `external_data` 옵션이 기본으로 켜져 있어서 그래요. ONNX가 쓰는 프로토콜 버퍼는 파일 하나가 2GB를 넘을 수 없어서, 큰 모델은 그래프와 가중치를 나눠 담아야 하거든요. 14장의 LLM 같은 모델이라면 꼭 필요한 기능입니다.

대신 두 파일은 **항상 함께 다녀야** 합니다. `.onnx`만 복사해 가면 가중치가 없어서 열리지 않아요. 우리 모델은 작으니 `external_data=False`로 한 파일에 담겠습니다. 담은 뒤에는 `onnx.checker`로 규격에 맞는지 검사하고, 파일에 적힌 정보를 읽어 볼게요.

```
from pathlib import Path

import onnx
import torch

from 옷도구 import 옷CNN
```

```
모델 = 옷CNN()
모델.load_state_dict(torch.load("fashion_cnn.pt", weights_only=True))
모델.eval()
torch.onnx.export(모델, (torch.zeros(1, 1, 28, 28),), "fashion_cnn.onnx",
                  verbose=False, external_data=False)        # 가중치까지 한 파일에
print(f"fashion_cnn.onnx {Path('fashion_cnn.onnx').stat().st_size / 1024:.1f} KB")

모형 = onnx.load("fashion_cnn.onnx")
onnx.checker.check_model(모형)                    # ONNX 규격에 맞는지 검사 (틀리면 예외)
print("규격 검사 통과")
print("만든 도구:", 모형.producer_name, 모형.producer_version)
print("연산자 판(opset):", [(o.domain or "ai.onnx", o.version) for o in 모형.opset_import])


def 모양(값: onnx.ValueInfoProto) -> list[int | str]:
    """입출력 정보에서 차원 크기(또는 이름)를 꺼낸다."""
    return [d.dim_value or d.dim_param for d in 값.type.tensor_type.shape.dim]


print("입력:", [(i.name, 모양(i)) for i in 모형.graph.input])
print("출력:", [(o.name, 모양(o)) for o in 모형.graph.output])
```

```
fashion_cnn.onnx 1662.3 KB
규격 검사 통과
만든 도구: pytorch 2.14.1
연산자 판(opset): [('ai.onnx', 20)]
입력: [('x', [1, 1, 28, 28])]
출력: [('linear_1', [1, 10])]
```

가중치까지 한 파일에 들어가니 1,662KB가 됐습니다. `.pt` 파일과 거의 같은 크기죠. 같은 숫자 42만 개를 float32로 담았으니 당연합니다. 파일에는 이 파일을 만든 도구(PyTorch 2.14.1)와 연산자 판 번호(20)가 적혀 있어요. 런타임은 이 번호를 보고 자기가 그 판을 지원하는지 판단합니다.

입출력 정보도 보세요. 입력 이름은 `forward(self, x)`의 인자 이름을 따서 `x`, 출력 이름은 마지막 층에서 따온 `linear_1`이 됐고, 모양은 예시 입력 그대로 `[1, 1, 28, 28]`로 **고정**되었습니다. 이름과

모양은 19.4절에서 손보겠습니다.

19.3 ONNX Runtime으로 추론하기

같은 답을 내는지 확인하기

악보가 제대로 적혔는지 확인하는 방법은 연주해 보는 것이죠. ONNX Runtime으로 파일을 열어 시험 사진 한 장을 넣고, PyTorch 모델의 답과 비교하겠습니다. ONNX Runtime에서 모델을 연 것을 **세션(session)** 이라고 불러요.

```
import numpy as np
import onnxruntime as ort
import torch

from 옷도구 import 옷CNN, 시험_데이터

모델 = 옷CNN()
모델.load_state_dict(torch.load("fashion_cnn.pt", weights_only=True))
모델.eval()

세션 = ort.InferenceSession("fashion_cnn.onnx", providers=["CPUExecutionProvider"])
입력, 출력 = 세션.get_inputs()[0], 세션.get_outputs()[0]
print("입력:", 입력.name, 입력.shape, 입력.type)
print("출력:", 출력.name, 출력.shape, 출력.type)

X, y = 시험_데이터()
사진 = X[:1]                                    # 시험 사진 첫 장, (1, 1, 28, 28)
점수_onnx = 세션.run(None, {입력.name: 사진.numpy()})[0]    # 넘파이 배열로 넣고 넘파이로 받는다
with torch.no_grad():
    점수_torch = 모델(사진).numpy()

이름 = ["T셔츠", "바지", "풀오버", "드레스", "코트", "샌들", "셔츠", "운동화", "가방", "앵클부츠"]
np.set_printoptions(precision=3, suppress=True)
print("ONNX Runtime:", 점수_onnx[0])
print("PyTorch     :", 점수_torch[0])
print("가장 큰 차이:", np.abs(점수_onnx - 점수_torch).max())
print("예측:", 이름[점수_onnx.argmax()], "/ 정답:", 이름[y[0]])
```

```
입력: x [1, 1, 28, 28] tensor(float)
출력: linear_1 [1, 10] tensor(float)
ONNX Runtime: [-3.732 -8.279 -7.15  -7.188 -8.329 -0.003 -5.677  2.529 -5.237  8.533]
PyTorch     : [-3.732 -8.279 -7.15  -7.188 -8.329 -0.003 -5.677  2.529 -5.237  8.533]
가장 큰 차이: 9.059906e-06
예측: 앵클부츠 / 정답: 앵클부츠
```

`InferenceSession`에 파일 경로와 **실행 장치(execution provider)** 목록을 주면 세션이 만들어집니다. 여기서는 CPU를 골랐어요. 세션은 입력과 출력의 이름·모양·자료형을 알려 주는데, `tensor(float)`가 PyTorch의 `float32`입니다.

`run`의 사용법이 PyTorch와 조금 다릅니다. 첫 인자는 받고 싶은 출력 이름 목록인데 `None`이면 전부 돌려주고, 둘째 인자는 `{입력 이름: 넘파이 배열}` 딕셔너리예요. 결과도 넘파이 배열의 **리스트**라서 `[0]`으로 첫 출력을 꺼냈습니다. ONNX Runtime은 텐서가 아니라 1권 20장의 넘파이 배열을 주고받아요.

두 점수는 소수 셋째 자리까지 똑같고, 가장 큰 차이는 백만분의 몇 수준입니다. 완전히 0이 아닌 이유는 4장 4.3절에서 본 float32의 끝자리 오차 때문이에요. 같은 곱셈과 덧셈이라도 **더하는 순서**가 다르면 끝자리가 조금 달라지는데, PyTorch와 ONNX Runtime은 계산을 나누고 모으는 방식이 서로 다릅니다. 이 정도 차이는 정상이고, 예측(앵클부츠)은 당연히 같아요.

torch 없이 돌리기

이제 이 장의 핵심을 확인할 차례입니다. 정말 PyTorch 없이 돌아갈까요? 아래 코드는 `torch`도 `torchvision`도 불러오지 않습니다. FashionMNIST 시험 사진은 6장 6.11절에서 받아 둔 원본 파일을 넘파이로 직접 읽어요. 원본 파일은 앞부분에 개수와 크기를 적은 머리말(사진 파일 16바이트, 정답 파일 8바이트)이 있고, 그 뒤로 픽셀 값이 한 바이트씩 이어집니다.

```
import sys

import numpy as np
import onnxruntime as ort

# torch 없이 FashionMNIST 원본 파일을 넘파이로 읽는다 (앞부분 16바이트, 8바이트는 머리말)
사진 = np.fromfile("data/FashionMNIST/raw/t10k-images-idx3-ubyte", dtype=np.uint8, offset=16)
정답 = np.fromfile("data/FashionMNIST/raw/t10k-labels-idx1-ubyte", dtype=np.uint8, offset=8)
```

```
사진 = 사진.reshape(-1, 1, 28, 28).astype(np.float32) / 255     # ToTensor()와 같은 0~1 실수

세션 = ort.InferenceSession("fashion_cnn.onnx", providers=["CPUExecutionProvider"])
입력이름 = 세션.get_inputs()[0].name
예측 = np.array([세션.run(None, {입력이름: 사진[i:i + 1]})[0].argmax()
                 for i in range(len(사진))])              # 한 장씩 1만 번
print(f"사진 {len(사진):,}장, 정확도 {(예측 == 정답).mean():.2%}")
print("torch를 불러왔나:", "torch" in sys.modules)
```

```
사진 10,000장, 정확도 91.01%
torch를 불러왔나: False
```

정확도 91.01%. 9장에서 PyTorch로 잰 숫자와 똑같습니다. 마지막 줄의 `sys.modules`는 지금까지 불러온 모듈의 목록인데, 거기에 `torch`가 없어요. 이 프로그램이 돌아가는 데 필요한 것은 넘파이, ONNX Runtime, 그리고 `fashion_cnn.onnx` 파일 하나뿐입니다.

전처리를 눈여겨보세요. `/ 255`로 0~1 사이 실수로 바꾸는 일은 학습 때 `ToTensor()`가 해 주던 일입니다. 모델 바깥에서 하던 전처리는 ONNX 파일에 들어가지 않으니, **배포하는 쪽에서 똑같이 다시 짜야** 해요. 10장 10.2절에서 "모델이 배운 기준 그대로" 입력을 맞춰야 한다고 했던 것이 여기서도 그대로 적용됩니다. 전처리를 틀리면 어떻게 되는지는 19.10절에서 뜻밖의 모습으로 다시 보게 됩니다.

그런데 사진 1만 장을 한 장씩 1만 번 넣었습니다. 묶어서 한 번에 넣으면 안 될까요?

19.4 입력 모양 정하기

모양이 고정된 모델

앞에서 만든 `fashion_cnn.onnx`에 사진 64장 묶음을 넣어 보겠습니다. 실수로 float64 배열을 넣는 경우도 함께 보죠.

```
import numpy as np
import onnxruntime as ort

세션 = ort.InferenceSession("fashion_cnn.onnx", providers=["CPUExecutionProvider"])
입력이름 = 세션.get_inputs()[0].name
```

```
for 설명, 입력 in [("64장 묶음", np.zeros((64, 1, 28, 28), dtype=np.float32)),
                   ("float64", np.zeros((1, 1, 28, 28), dtype=np.float64))]:
    try:
        세션.run(None, {입력이름: 입력})
    except Exception as e:
        print(f"[{설명}] {type(e).__name__}")
        print("  ", str(e).replace("\n", " ").split(" : ")[-1])
```

```
[64장 묶음] InvalidArgument
  Got invalid dimensions for input: x for the following indices  index: 0 Got: 64 Expected: 1
↪  Please fix either the inputs/outputs or the model.
[float64] InvalidArgument
  Unexpected input data type. Actual: (tensor(double)) , expected: (tensor(float))
```

두 번 다 `InvalidArgument` 오류입니다. 첫 번째는 "0번 차원에 64가 왔는데 1이어야 한다", 두 번째는 "double(float64)이 왔는데 float(float32)이어야 한다"는 뜻이에요. 4장 4.10절의 요령대로 숫자와 자료형 이름부터 읽으면 바로 보입니다. 넘파이의 기본 실수가 float64라서 두 번째 실수는 정말 자주 일어나요(4장 4.1절). 배포 코드에서는 `astype(np.float32)`를 습관처럼 붙이세요.

첫 번째 오류의 원인은 내보낼 때 넣은 예시 입력이 `(1, 1, 28, 28)`이었기 때문입니다. `torch.export`는 예시 입력의 모양을 보고 "이 모델은 언제나 사진 1장을 받는다"고 기록했어요.

자유로운 차원 열어 두기

묶음 크기는 실행할 때마다 달라질 수 있어야 합니다. 이렇게 크기를 미리 정하지 않는 차원을 **동적 차원(dynamic dimension)** 이라고 하고, `dynamic_shapes`로 알려 줘요. 입력과 출력의 이름도 알아보기 쉽게 바꾸겠습니다.

```
import onnx
import onnxruntime as ort
import torch

from 옷도구 import 옷CNN, 시험_데이터

모델 = 옷CNN()
```

```
모델.load_state_dict(torch.load("fashion_cnn.pt", weights_only=True))
모델.eval()

묶음 = torch.export.Dim("batch")                  # 이름 붙인 "자유로운 차원"
torch.onnx.export(모델, (torch.zeros(1, 1, 28, 28),), "fashion_cnn_batch.onnx",
                  verbose=False, external_data=False,
                  input_names=["사진"], output_names=["점수"],
                  dynamic_shapes={"x": {0: 묶음}})       # forward(x)의 0번 차원은 자유

모형 = onnx.load("fashion_cnn_batch.onnx")
for 값 in [*모형.graph.input, *모형.graph.output]:
    print(값.name, [d.dim_value or d.dim_param for d in 값.type.tensor_type.shape.dim])

세션 = ort.InferenceSession("fashion_cnn_batch.onnx", providers=["CPUExecutionProvider"])
X, y = 시험_데이터()
for 장수 in [1, 7, 64]:
    print(f"{장수:>5}장 →", 세션.run(None, {"사진": X[:장수].numpy()})[0].shape)
점수 = 세션.run(["점수"], {"사진": X.numpy()})[0]          # 1만 장을 한 번에
print(f"1만 장 한 번에: {점수.shape}, 정확도 {(점수.argmax(axis=1) == y.numpy()).mean():.2%}")
```

```
사진 ['batch', 1, 28, 28]
점수 ['batch', 10]
    1장 → (1, 10)
    7장 → (7, 10)
   64장 → (64, 10)
1만 장 한 번에: (10000, 10), 정확도 91.01%
```

입력 모양이 `['batch', 1, 28, 28]`, 출력이 `['batch', 10]`이 됐습니다. 0번 자리에 숫자 대신 이름이 적혀 있으면 "아무 크기나 된다"는 뜻이에요. 실제로 1장, 7장, 64장을 넣어도 모두 돌아가고, 1만 장을 한 번에 넣어 앞 절과 같은 91.01%를 얻었습니다. `run`의 첫 인자에 `["점수"]`처럼 출력 이름을 적으면 그 출력만 받아요.

`dynamic_shapes`는 "어느 입력의 몇 번 차원이 자유로운가"를 적는 딕셔너리입니다. 열쇠 `"x"`는 `forward(self, x)`의 **인자 이름**이고, 값 `{0: 묶음}`은 그 인자의 0번 차원이 `묶음`이라는 이름의 자유로운 차원이라는 뜻이에요. 입력이 여럿이면 열쇠도 여럿이 됩니다. 11~13장의 문장 모델이라면 `{0: 묶음, 1: 길이}`처럼 문장 길이 차원도 열어 두겠죠. `input_names`, `output_names`는 ONNX 파일 안

에서 쓸 이름일 뿐이라 한글도 됩니다. 다만 다른 언어로 짠 프로그램이 이 파일을 읽을 때는 영어 이름이 다루기 편해서, 20장 서빙에서는 같은 방식으로 내보내되 입출력 이름만 영어(`image`, `logits`)로 바꿉니다.

> **Q. 다른 예제에서는 `dynamic_axes={"input": {0: "batch"}}`라고 쓰던데요?** 예전 TorchScript 방식에서 쓰던 옵션입니다. 지금 방식에서도 받아 주기는 하지만 안에서 `dynamic_shapes`로 바꿔 쓰면서 "권하지 않는다"는 경고를 내요. 새로 짜는 코드에서는 `dynamic_shapes`를 쓰세요. 인자 이름 대신 순서로 적고 싶으면 `dynamic_shapes=({0: 묶음},)`처럼 튜플로 써도 됩니다.

19.5 내보낼 수 있는 코드, 없는 코드

데이터에 따라 갈리는 if

19.2절에서 `torch.export`가 "예시 입력을 한 번 흘려보내면서 일어나는 텐서 연산을 기록한다"고 했습니다. 그렇다면 입력에 따라 다른 길로 가는 모델은 어떻게 될까요? 길이 하나만 기록될 텐데요. "입력의 합이 양수면 두 배, 아니면 1을 빼는" 작은 모델을 세 가지로 짜서 내보내 보겠습니다.

```
import numpy as np
import onnx
import onnxruntime as ort
import torch
from torch import nn


class 파이썬_if(nn.Module):
    """합이 양수면 두 배, 아니면 1을 뺀다: 파이썬 if로."""

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        if x.sum() > 0:
            return x * 2
        return x - 1


class 텐서_where(nn.Module):
```

```
    """같은 일을 torch.where로: 두 결과를 다 계산하고 칸마다 고른다 (4장 4.4절)."""

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.where(x.sum() > 0, x * 2, x - 1)


class 텐서_cond(nn.Module):
    """같은 일을 torch.cond로: 조건에 따라 둘 중 한 갈래만 계산한다."""

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.cond(x.sum() > 0, lambda x: x * 2, lambda x: x - 1, (x,))


for 모델 in [파이썬_if(), 텐서_where(), 텐서_cond()]:
    이름 = type(모델).__name__
    try:
        torch.onnx.export(모델.eval(), (torch.ones(3),), "branch.onnx",
                          verbose=False, external_data=False)
    except Exception as e:
        print(f"{이름}: 내보내기 실패 ({type(e).__name__} ← {type(e.__cause__).__name__})")
        continue
    연산 = [노드.op_type for 노드 in onnx.load("branch.onnx").graph.node]
    세션 = ort.InferenceSession("branch.onnx", providers=["CPUExecutionProvider"])
    입력이름 = 세션.get_inputs()[0].name
    결과 = [세션.run(None, {입력이름: np.array(v, dtype=np.float32)})[0].tolist()
            for v in ([1, 2, 3], [-1, -2, -3])]
    print(f"{이름}: {연산} → {결과}")
```

```
파이썬_if: 내보내기 실패 (TorchExportError ← GuardOnDataDependentSymNode)
텐서_where: ['ReduceSum', 'Greater', 'Mul', 'Sub', 'Where'] → [[2.0, 4.0, 6.0], [-2.0, -3.0, -4.0]]
텐서_cond: ['ReduceSum', 'Greater', 'If'] → [[2.0, 4.0, 6.0], [-2.0, -3.0, -4.0]]
```

파이썬 `if`로 짠 모델은 내보내기에 실패했습니다. 오류의 속 원인 `GuardOnDataDependentSymNode`는 "데이터 값에 따라 정해지는 것에 기대고 있다"는 뜻이에요. `x.sum() > 0`이 참인지 거짓인지는 실제 숫자를 넣어 봐야 알 수 있는데, 그래프에는 두 갈래 중 하나만 적을 수 있으니 내보내기가 거절한 겁니다. 조용히 한쪽 길만 굳혀 버리는 것보다 이렇게 실패하는 편이 훨씬 안전해요.

나머지 둘은 성공했고, 양수·음수 입력 모두에서 올바른 답을 냈습니다.

- `torch.where` 는 4장 4.4절에서 본 그 함수예요. 두 갈래(`x * 2`와 `x - 1`)를 **모두 계산한 뒤** 조건에 따라 고릅니다. 그래프에 `Mul`, `Sub`, `Where`가 다 들어 있죠. 갈래가 가벼울 때 가장 간단한 해결책입니다.
- `torch.cond` 는 조건에 따라 **한 갈래만 계산하는** 연산이에요. ONNX 그래프에 `If` 노드 하나가 생기고, 그 안에 두 갈래가 작은 그래프로 들어갑니다. 갈래가 무거워서 둘 다 계산하기 아까울 때 씁니다.

파이썬 코드는 내보낼 때 한 번만 돈다

반대로 실패하지 않아서 더 조심해야 하는 경우도 있습니다. `forward` 안에 텐서 연산이 아닌 **파이썬 코드**가 섞여 있을 때예요.

```
import numpy as np
import onnx
import onnxruntime as ort
import torch
from torch import nn


class 굳는_모델(nn.Module):
    """forward 안에 파이썬 코드가 섞인 모델."""

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        print("  (forward 실행됨)")
        잡음 = torch.from_numpy(np.random.rand(3).astype(np.float32))   # 넘파이 난수
        for _ in range(3):                                              # 파이썬 반복문
            x = x + 1
        return x + 잡음


print("내보내는 중:")
torch.onnx.export(굳는_모델().eval(), (torch.zeros(3),), "frozen.onnx",
                  verbose=False, external_data=False)
그래프 = onnx.load("frozen.onnx").graph
print("연산:", [노드.op_type for 노드 in 그래프.node])
```

```
print("저장된 상수:", [(값.name, list(값.dims)) for 값 in 그래프.initializer])

세션 = ort.InferenceSession("frozen.onnx", providers=["CPUExecutionProvider"])
입력이름 = 세션.get_inputs()[0].name
for 회 in range(1, 3):
    print(f"{회}번째 실행:", 세션.run(None, {입력이름: np.zeros(3, dtype=np.float32)})[0])
```

```
내보내는 중:
  (forward 실행됨)
연산: ['Add', 'Add', 'Add', 'Add']
저장된 상수: [('clone', [3]), ('scalar_tensor_default', [])]
1번째 실행: [3.3831162 3.3439608 3.2308836]
2번째 실행: [3.3831162 3.3439608 3.2308836]
```

세 가지를 보세요.

- `print`는 내보낼 때 **한 번** 실행됐고, ONNX 그래프에는 남지 않았습니다. 세션을 두 번 실행할 때는 아무것도 찍히지 않았죠.
- 파이썬 `for` 반복문은 세 번 돌면서 `Add`를 세 개 기록했습니다. 반복문 자체가 아니라 **펼쳐진 결과**가 그래프에 남은 거예요. 반복 횟수가 입력과 상관없이 정해져 있다면 문제없습니다.
- 가장 위험한 것은 넘파이 난수입니다. `np.random.rand(3)`은 텐서 연산이 아니라서, 내보낼 때 한 번 뽑힌 숫자가 `clone`이라는 **상수**로 굳어 버렸어요. 1번째와 2번째 실행의 결과가 똑같죠. 매번 다른 잡음을 더하려던 의도가 조용히 사라졌는데 오류는 하나도 없습니다.

정리하면, `forward` 안에서는 **텐서 연산만으로** 계산하는 것이 내보내기의 기본 규칙입니다. 넘파이, `math`, `random` 같은 파이썬 쪽 계산은 상수로 굳고, 텐서 값에 따라 갈리는 `if`는 실패해요. 모양에 따라 갈리는 `if`(예: `if x.shape[1] > 100`)는 예시 입력의 모양 기준으로 한쪽이 굳으니, 동적 차원을 열어 둔 모델에서는 특히 조심해야 합니다. 내보낸 뒤에는 항상 **여러 입력으로 PyTorch와 결과를 비교**하는 습관을 들이세요. 19.3절에서 한 비교가 바로 그 검사입니다.

19.6 그래프 들여다보기

이 장의 도구 모음

지금까지 내보내기 코드를 매번 길게 썼습니다. 19.2~19.4절에서 정한 것(한 파일, 평가 모드, 묶음 차원 열기, 입출력 이름)을 함수 하나로 묶고, 9장의 작은 ResNet 설계도와 시간 재는 함수도 함께 `배포도구.py`에 모아 두겠습니다. 9장의 `옷도구.py`처럼 이 장의 나머지 예제가 이 파일을 `import`해요.

```
"""19장 예제에서 함께 쓰는 모델 설계도·내보내기·시간 재기 도구."""
import statistics
import time
from collections.abc import Callable
from pathlib import Path

import numpy as np
import onnxruntime as ort
import torch
import torch.nn.functional as F
from torch import nn

from 옷도구 import 옷CNN, 시험_데이터  # noqa: F401  (9장 9.5절의 CNN과 시험 데이터)


class 잔차블록(nn.Module):
    """9장 9.8절의 잔차 블록."""

    def __init__(self, 입력: int, 출력: int, 보폭: int = 1) -> None:
        super().__init__()
        self.합성곱1 = nn.Conv2d(입력, 출력, 3, stride=보폭, padding=1, bias=False)
        self.정규화1 = nn.BatchNorm2d(출력)
        self.합성곱2 = nn.Conv2d(출력, 출력, 3, padding=1, bias=False)
        self.정규화2 = nn.BatchNorm2d(출력)
        if 보폭 != 1 or 입력 != 출력:
            self.지름길 = nn.Sequential(nn.Conv2d(입력, 출력, 1, stride=보폭, bias=False),
                                        nn.BatchNorm2d(출력))
        else:
            self.지름길 = nn.Identity()
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        고칠양 = self.정규화2(self.합성곱2(F.relu(self.정규화1(self.합성곱1(x)))))
        return F.relu(고칠양 + self.지름길(x))


def 작은_ResNet() -> nn.Sequential:
    """9장 9.10절의 작은 ResNet 설계도."""
    return nn.Sequential(
        nn.Conv2d(1, 32, 3, padding=1, bias=False), nn.BatchNorm2d(32), nn.ReLU(),
        잔차블록(32, 32), 잔차블록(32, 64, 보폭=2), 잔차블록(64, 128, 보폭=2),
        nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Dropout(0.3), nn.Linear(128, 10),
    )


def 불러오기(설계도: Callable[[], nn.Module], 파일: str) -> nn.Module:
    """저장한 가중치를 설계도에 채우고 평가 모드로 돌려준다."""
    모델 = 설계도()
    모델.load_state_dict(torch.load(파일, weights_only=True))
    return 모델.eval()


def ONNX로_내보내기(모델: nn.Module, 경로: str, 최적화: bool = True) -> None:
    """19.2~19.4절의 정리: 묶음 크기가 자유로운 한 파일짜리 ONNX로 내보낸다."""
    torch.onnx.export(모델, (torch.zeros(1, 1, 28, 28),), 경로, verbose=False,
                      external_data=False, optimize=최적화,
                      input_names=["사진"], output_names=["점수"],
                      dynamic_shapes=({0: torch.export.Dim("batch")},))


def 세션_만들기(경로: str, 스레드: int = 4) -> ort.InferenceSession:
    """스레드 수를 정해 CPU에서 돌리는 ONNX Runtime 세션."""
    설정 = ort.SessionOptions()
    설정.intra_op_num_threads = 스레드
    return ort.InferenceSession(경로, 설정, providers=["CPUExecutionProvider"])
```

```
def 세션_정확도(세션: ort.InferenceSession, X: np.ndarray, y: np.ndarray) -> float:
    """사진 묶음 전체를 한 번에 넣어 정확도를 잰다."""
    점수 = 세션.run(None, {"사진": X})[0]
    return float((점수.argmax(axis=1) == y).mean())


def 시간_재기(함수: Callable[[], object], 반복: int = 50, 준비: int = 5) -> float:
    """준비 운동 뒤 여러 번 재서 한 번 실행 시간의 중앙값(밀리초)을 돌려준다."""
    for _ in range(준비):
        함수()
    기록 = []
    for _ in range(반복):
        시작 = time.perf_counter()
        함수()
        기록.append((time.perf_counter() - 시작) * 1000)
    return statistics.median(기록)


def 파일_크기(*경로: str) -> float:
    """파일 크기의 합(KB). .onnx와 옆의 .onnx.data를 함께 셀 때 쓴다."""
    return sum(Path(p).stat().st_size for p in 경로 if Path(p).exists()) / 1024
```

`작은_ResNet`과 `잔차블록`은 9장 9.10절의 코드 그대로입니다. 9장에서는 실습 코드 안에 있었는데, 저장한 `fashion_resnet.pt`를 불러오려면 같은 설계도가 필요해서 옮겨 왔어요. 이것이 6장 6.10절에서 말한 `state_dict` 방식의 약속입니다. 숫자만 저장했으니 설계도는 코드로 갖고 있어야 하죠. 반대로 ONNX 파일은 설계도(그래프)와 숫자를 함께 담으니, ONNX로 내보낸 다음부터는 이 클래스가 필요 없습니다.

`세션_만들기`는 ONNX Runtime이 쓸 스레드 수를 정하는 함수이고, `시간_재기`는 19.7절에서 설명합니다.

노드와 이니셜라이저

`onnx.load`로 연 파일은 파이썬 객체라서 안을 직접 들여다볼 수 있습니다. 9.5절 CNN의 그래프를 노드 순서대로 찍어 보겠습니다.

```
import onnx

from 배포도구 import ONNX로_내보내기, 불러오기, 옷CNN

모델 = 불러오기(옷CNN, "fashion_cnn.pt")
ONNX로_내보내기(모델, "fashion_cnn.onnx")
그래프 = onnx.load("fashion_cnn.onnx").graph

print("[계산 순서]")
for 노드 in 그래프.node:
    print(f"  {노드.op_type:8} {', '.join(노드.input)} → {', '.join(노드.output)}")

print("[저장된 숫자]")
개수 = 0
for 값 in 그래프.initializer:
    배열 = onnx.numpy_helper.to_array(값)
    개수 += 배열.size
    print(f"  {값.name:22} {str(배열.shape):16} {배열.dtype}")
print(f"합계 {개수:,}개 / PyTorch 파라미터 {sum(p.numel() for p in 모델.parameters()):,}개")
```

```
[계산 순서]
  Conv     사진, 특징.0.weight, 특징.0.bias → conv2d
  Relu     conv2d → relu
  MaxPool  relu → max_pool2d
  Conv     max_pool2d, 특징.3.weight, 특징.3.bias → conv2d_1
  Relu     conv2d_1 → relu_1
  MaxPool  relu_1 → max_pool2d_1
  Reshape  max_pool2d_1, val_7 → view
  Gemm     view, 분류.1.weight, 분류.1.bias → linear
  Relu     linear → relu_2
  Gemm     relu_2, 분류.3.weight, 분류.3.bias → 점수
[저장된 숫자]
  특징.0.weight            (32, 1, 3, 3)    float32
  특징.0.bias              (32,)            float32
  특징.3.weight            (64, 32, 3, 3)   float32
  특징.3.bias              (64,)            float32
```

```
  분류.1.weight            (128, 3136)      float32
  분류.1.bias              (128,)           float32
  분류.3.weight            (10, 128)        float32
  분류.3.bias              (10,)            float32
  val_7                 (2,)             int64
합계 421,644개 / PyTorch 파라미터 421,642개
```

"계산 순서"가 바로 악보입니다. 노드마다 연산자 이름과, 어떤 값을 받아(왼쪽) 어떤 값을 만드는지(오른쪽)가 적혀 있어요. 사진이 `Conv` → `Relu` → `MaxPool`을 두 번 지나고, `Reshape`로 한 줄로 펴진 뒤(`nn.Flatten`), `Gemm`(행렬 곱 + 편향) 두 번을 거쳐 **점수**가 됩니다. 9장 9.5절에서 따라가 본 모양 그대로죠. `Conv`의 입력에 **`특징.0.weight`** 같은 이름이 보이는데, PyTorch `state_dict`의 열쇠 이름이 그대로 이니셜라이저의 이름이 된 거예요.

이니셜라이저 숫자를 모두 세면 421,644개로, PyTorch 파라미터 421,642개보다 두 개 많습니다. 범인은 `val_7`이에요. `Reshape`에 "몇 행 몇 열로 펼지"를 알려 주는 정수 두 개(int64)가 상수로 들어간 겁니다. 학습할 숫자는 정확히 같아요.

이렇게 글자로 보는 것 말고 그림으로 보고 싶다면 **Netron**이라는 오픈소스 뷰어가 많이 쓰입니다. ONNX 파일을 열면 노드를 상자로, 값의 흐름을 화살표로 그려 줘요.

배치 정규화는 어디로 갔을까

이번에는 작은 ResNet을 내보내 보겠습니다. 이 모델에는 배치 정규화가 9개 있어요. `optimize` 옵션을 끈 것과 켠 것(기본값)을 비교합니다.

```
from collections import Counter


import numpy as np
import onnx
import torch

from 배포도구 import ONNX로_내보내기, 불러오기, 세션_만들기, 작은_ResNet

모델 = 불러오기(작은_ResNet, "fashion_resnet.pt")        # 9장 9.10절의 작은 ResNet (93.04%)
x = torch.randn(8, 1, 28, 28)
with torch.no_grad():
    기준 = 모델(x).numpy()
```

```
for 최적화 in [False, True]:
    경로 = f"fashion_resnet_opt{int(최적화)}.onnx"
    ONNX로_내보내기(모델, 경로, 최적화=최적화)
    연산 = Counter(노드.op_type for 노드 in onnx.load(경로).graph.node)
    차이 = np.abs(세션_만들기(경로).run(None, {"사진": x.numpy()})[0] - 기준).max()
    print(f"optimize={최적화}: 노드 {sum(연산.values())}개, 최대 차이 {차이:.1e}")
    print("  ", dict(연산.most_common(6)))
```

```
optimize=False: 노드 86개, 최대 차이 1.5e-05
   {'Expand': 18, 'Constant': 14, 'Shape': 10, 'CastLike': 9, 'Conv': 9, 'BatchNormalization': 9}
optimize=True: 노드 22개, 최대 차이 1.5e-05
   {'Conv': 9, 'Relu': 7, 'Add': 3, 'ReduceMean': 1, 'Reshape': 1, 'Gemm': 1}
```

최적화를 끄면 노드가 86개이고, `BatchNormalization`이 9개 보입니다. `Expand`, `Constant`, `Shape` 같은 것은 번역하면서 생긴 자질구레한 보조 노드예요. 최적화를 켜면 노드가 22개로 줄고, **`BatchNormalization`이 모두 사라졌습니다.** 그런데 결과는 둘 다 PyTorch와 최대 차이가 십만분의 1~2 수준으로 같아요. 배치 정규화 계산이 어디로 갔을까요?

8장 8.4절에서 배치 정규화는 "평균을 빼고 표준편차로 나눈 뒤 감마를 곱하고 베타를 더한다"고 했습니다. 평가 모드에서는 평균과 분산이 학습 중에 모아 둔 **고정된 숫자**(`running_mean`, `running_var`)라서, 결국 채널마다 "어떤 수를 곱하고 어떤 수를 더하는" 계산이 돼요. 그런데 합성곱도 채널마다 곱하고 더하는 계산이니, 둘을 하나로 합칠 수 있습니다. 손으로 해 보겠습니다.

```
import torch
import torch.nn.functional as F

from 배포도구 import 불러오기, 작은_ResNet

모델 = 불러오기(작은_ResNet, "fashion_resnet.pt")
합성곱, 정규화 = 모델[0], 모델[1]                    # 첫 Conv2d(1→32)와 BatchNorm2d(32)

# 평가 모드의 배치 정규화: (x - 평균) / sqrt(분산 + eps) * 감마 + 베타 → 채널마다 곱하고 더하기
배율 = 정규화.weight / torch.sqrt(정규화.running_var + 정규화.eps)       # (32,)
새_가중치 = 합성곱.weight * 배율.view(-1, 1, 1, 1)                       # 출력 채널마다 곱하기
```

```
새_편향 = 정규화.bias - 정규화.running_mean * 배율                    # 원래 Conv는 bias=False

x = torch.randn(4, 1, 28, 28)
with torch.no_grad():
    두_단계 = 정규화(합성곱(x))
    한_단계 = F.conv2d(x, 새_가중치, 새_편향, padding=1)
print("합성곱 가중치:", tuple(새_가중치.shape), "편향:", tuple(새_편향.shape))
print("두 결과의 최대 차이:", (두_단계 - 한_단계).abs().max().item())
```

```
합성곱 가중치: (32, 1, 3, 3) 편향: (32,)
두 결과의 최대 차이: 5.7220458984375e-06
```

배치 정규화의 "곱할 수"(배율)를 합성곱 가중치에 미리 곱해 두고, "더할 수"를 편향으로 만들었습니다. 이렇게 만든 합성곱 한 번이 원래의 합성곱 + 배치 정규화 두 단계와 백만분의 몇 차이로 같은 답을 냅니다. 이것을 **배치 정규화 접어 넣기(folding)** 라고 해요. 9장 9.7절에서 "배치 정규화 앞의 합성곱은 `bias=False`로 둔다"고 했는데, 접어 넣고 나면 배치 정규화의 베타가 그 합성곱의 편향 자리로 들어가는 셈입니다. 추론할 때는 층 하나가 통째로 사라지니 그만큼 빨라져요. 학습 중에는 평균과 분산이 배치마다 바뀌니 이 계산을 할 수 없고, 평가 모드로 굳힌 뒤에만 가능하다는 점도 기억해 두세요.

ONNX Runtime이 한 번 더 다듬는다

내보낸 그래프는 ONNX Runtime이 세션을 만들 때 한 번 더 다듬습니다. 단계별로 실제로 돌릴 그래프를 파일로 저장하게 해서 비교해 보겠습니다.

```
from collections import Counter

import onnx
import onnxruntime as ort

from 배포도구 import ONNX로_내보내기, 불러오기, 작은_ResNet

ONNX로_내보내기(불러오기(작은_ResNet, "fashion_resnet.pt"), "fashion_resnet.onnx")
단계들 = {"끄기": ort.GraphOptimizationLevel.ORT_DISABLE_ALL,
          "기본": ort.GraphOptimizationLevel.ORT_ENABLE_BASIC,
```

```
        "확장": ort.GraphOptimizationLevel.ORT_ENABLE_EXTENDED,
        "전부": ort.GraphOptimizationLevel.ORT_ENABLE_ALL}           # 세션의 기본값
for 이름, 단계 in 단계들.items():
    설정 = ort.SessionOptions()
    설정.graph_optimization_level = 단계
    설정.optimized_model_filepath = "fashion_resnet_ort.onnx"    # 실제로 돌릴 그래프를 저장
    ort.InferenceSession("fashion_resnet.onnx", 설정, providers=["CPUExecutionProvider"])
    연산 = Counter(노드.op_type for 노드 in onnx.load("fashion_resnet_ort.onnx").graph.node)
    print(f"{이름}: 노드 {sum(연산.values()):2}개 {dict(연산.most_common(4))}")
```

```
끄기: 노드 22개 {'Conv': 9, 'Relu': 7, 'Add': 3, 'ReduceMean': 1}
기본: 노드 22개 {'Conv': 9, 'Relu': 7, 'Add': 3, 'ReduceMean': 1}
확장: 노드 18개 {'Conv': 5, 'FusedConv': 4, 'Add': 3, 'Relu': 3}
전부: 노드 12개 {'FusedConv': 7, 'Conv': 2, 'ReduceMean': 1, 'Reshape': 1}
```

"끄기"와 "기본"은 22개 그대로입니다. 기본 단계가 하는 정리(상수 계산 미리 하기, 쓸모없는 노드 지우기)는 PyTorch가 내보내면서 이미 해 두었기 때문이에요. "확장" 단계부터 `FusedConv`가 나타납니다. `Conv` 바로 뒤의 `Relu`를 합성곱 안으로 합친 노드예요. 합성곱 결과를 메모리에 썼다가 다시 읽어서 ReLU를 하는 대신, 쓰기 전에 바로 ReLU를 해 버리니 메모리를 오가는 횟수가 줄어듭니다. "전부" 단계에서는 잔차 연결의 `Add`까지 합성곱에 합쳐져 노드가 12개가 됐어요.

세션의 기본값이 이 "전부" 단계라서, 우리가 따로 설정하지 않아도 ONNX Runtime은 이렇게 다듬은 그래프로 계산합니다. 다만 "전부" 단계는 그 컴퓨터의 CPU에 맞춘 메모리 배치(레이아웃)까지 정하기 때문에, 이 단계의 그래프를 저장하면 "같은 환경에서만 쓰라"는 경고가 나와요. 그래서 배포할 때는 다듬기 전의 `.onnx` 파일을 나눠 주고, 다듬기는 각 컴퓨터의 런타임에 맡기는 것이 보통입니다.

19.7 얼마나 빨라졌나: 속도 재기

재는 방법부터

이제 PyTorch와 ONNX Runtime의 속도를 비교할 차례입니다. 4장 4.8절에서 GPU 시간을 잴 때 "준비 운동을 하고 재라"고 했죠. CPU에서도 첫 몇 번은 메모리를 잡고 캐시를 채우느라 느리기 때문에, `배포도구.py`의 `시간_재기`는 준비 운동 5번 뒤에 50번을 재서 **중앙값**을 돌려줍니다. 평균 대신 중앙값을 쓰는 건, 다른 프로그램이 잠깐 끼어들어 한두 번 크게 느려진 기록에 흔들리지 않게 하려

는 거예요. 두 엔진의 스레드 수도 4개로 똑같이 맞췄습니다. 스레드 수가 다르면 공정한 비교가 아니니까요.

```
import platform

import torch

from 배포도구 import (ONNX로_내보내기, 불러오기, 세션_만들기, 시간_재기, 시험_데이터,
                     옷CNN, 작은_ResNet)

torch.set_num_threads(4)                          # PyTorch와 ONNX Runtime 모두 스레드 4개
print("환경:", platform.machine(), platform.python_version(), "torch", torch.__version__)
X, _ = 시험_데이터()

print(f"{'모델':6} {'묶음':>4} {'PyTorch':>10} {'ORT':>10} {'배':>5}")
for 이름, 설계도, 파일 in [("CNN", 옷CNN, "fashion_cnn.pt"),
                          ("ResNet", 작은_ResNet, "fashion_resnet.pt")]:
    모델 = 불러오기(설계도, 파일)
    ONNX로_내보내기(모델, "speed.onnx")
    세션 = 세션_만들기("speed.onnx", 스레드=4)
    for 장수 in [1, 64]:
        x = X[:장수]
        x_np = x.numpy()
        with torch.inference_mode():
            파이토치 = 시간_재기(lambda: 모델(x))
        런타임 = 시간_재기(lambda: 세션.run(None, {"사진": x_np}))
        print(f"{이름:6} {장수:>4} {파이토치:8.3f}ms {런타임:8.3f}ms {파이토치 / 런타임:5.1f}")
```

```
환경: arm64 3.12.10 torch 2.14.1
모델       묶음    PyTorch        ORT     배
CNN           1    0.111ms    0.060ms   1.8
CNN          64    3.683ms    1.650ms   2.2
ResNet        1    0.663ms    0.286ms   2.3
ResNet       64   36.491ms   15.151ms   2.4
```

이 숫자는 이 책의 실행 환경(Apple M4 Max, CPU, 스레드 4개)에서 잰 중앙값입니다. 같은 컴퓨터에서도 다른 프로그램이 함께 돌고 있으면 실행마다 꽤 흔들리고(특히 PyTorch 쪽이 그랬습니다),

다른 컴퓨터에서는 크게 달라질 수 있어요. 그래도 흐름은 분명합니다. **같은 모델, 같은 숫자인데 ONNX Runtime이 두 배쯤 빠릅니다.** 사진 1장을 넣을 때도, 64장을 넣을 때도 그래요.

어디서 이런 차이가 날까요? 앞 절에서 본 대로 ONNX Runtime은 그래프 전체를 미리 보고 배치 정규화를 접고, 합성곱과 ReLU와 덧셈을 합치고, 그 CPU에 맞는 계산 방식을 골라 둡니다. 반면 PyTorch의 기본 실행 방식은 파이썬이 `forward`를 한 줄씩 따라가며 연산을 하나씩 부르는 방식이라, 연산과 연산 사이에 매번 파이썬을 거치고 중간 결과를 메모리에 씁니다. 사진 1장처럼 계산이 작을수록 이 "사이 시간"의 비중이 커져요.

또 하나 보이는 것은 묶음의 힘입니다. ResNet에서 1장은 약 0.3ms, 64장은 약 15ms이니 한 장당 0.23ms쯤이에요. 묶어서 넣으면 한 장씩 넣을 때보다 장당 시간이 줄어듭니다. 19.3절처럼 1만 장을 한 장씩 넣는 대신 19.4절에서 묶음 차원을 열어 둔 이유죠. 20장에서 서버를 만들 때 여러 요청을 묶어 처리하는 **배치 추론**이 이 원리를 씁니다.

> **Q. PyTorch도 그래프를 미리 보고 합치면 되지 않나요?** 그런 기능이 있습니다. `torch.compile`이 바로 "PyTorch 모델을 그래프로 따서 합치고 다듬는" 도구예요. 학습과 GPU에서 특히 힘을 발휘하는데, 16장 GPU와 성능에서 다룹니다. 이 장에서는 "PyTorch가 없는 곳에서 돌린다"는 목표가 먼저라 ONNX Runtime을 썼습니다.

19.8 양자화 다시 보기: 눈금, 영점, 정수 계산

14장의 흉내에서 진짜 양자화로

14장 14.6절에서 양자화를 "자에 눈금을 새기는 것"에 비유하고, 가중치를 행마다 -127~127 눈금에 맞췄다가 다시 실수로 되돌려 봤습니다. 그때 두 가지 숙제를 남겼어요. 실수로 되돌려 넣었으니 **메모리가 줄지 않았고**, 행마다 배율을 하나만 써서 **int4가 망가졌다**는 것이었죠. 진짜 양자화 도구는 정수와 배율만 저장하고, 계산도 가능한 한 정수로 합니다. 그러려면 개념 세 가지가 더 필요해요.

영점: 0 이상인 값을 위한 눈금

가중치는 0을 가운데 두고 양수와 음수가 섞여 있어서 -127~127 눈금이 잘 맞았습니다. 그런데 층과 층 사이를 흐르는 값(**활성값**)은 사정이 달라요. ReLU를 지난 값은 모두 0 이상이거든요. 0 이상인 값에 -127~127 눈금을 쓰면 음수 쪽 눈금 절반을 버리게 됩니다. 이럴 때는 0~255 눈금(uint8)을 쓰고, "실수 0이 몇 번 눈금에 놓이는지"를 따로 기억합니다. 이 번호를 **영점(zero point)** 이라고 해요.

```
import torch

torch.manual_seed(0)
# ReLU를 지난 값: 0 이상만 있다 (예제용으로 만든 0~6 사이 값)
활성값 = torch.rand(10_000) ** 2 * 6


def 대칭_int8(x: torch.Tensor) -> tuple[torch.Tensor, float]:
    """14장 14.6절 방식: 0을 가운데 두고 -127~127 눈금."""
    배율 = x.abs().max().item() / 127
    return (x / 배율).round().clamp(-127, 127), 배율


def 비대칭_uint8(x: torch.Tensor) -> tuple[torch.Tensor, float, int]:
    """최솟값~최댓값을 0~255 눈금에 맞추고, 실수 0이 놓일 눈금(영점)을 기억한다."""
    작은, 큰 = min(x.min().item(), 0.0), max(x.max().item(), 0.0)   # 실수 0은 꼭 범위 안에
    배율 = (큰 - 작은) / 255
    영점 = round(-작은 / 배율)
    return (x / 배율 + 영점).round().clamp(0, 255), 배율, 영점


정수, 배율 = 대칭_int8(활성값)
print(f"대칭 int8   : 배율 {배율:.5f}, 쓰인 눈금 {정수.unique().numel():3}개, "
      f"평균 오차 {(정수 * 배율 - 활성값).abs().mean():.5f}")
정수, 배율, 영점 = 비대칭_uint8(활성값)
print(f"비대칭 uint8: 배율 {배율:.5f}, 쓰인 눈금 {정수.unique().numel():3}개, "
      f"평균 오차 {((정수 - 영점) * 배율 - 활성값).abs().mean():.5f}, 영점 {영점}")

# 음수도 있는 값이라면 영점이 가운데쯤으로 온다
_, 배율, 영점 = 비대칭_uint8(torch.tensor([-1.0, 0.0, 3.0]))
print(f"-1~3 범위: 배율 {배율:.5f}, 영점 {영점} (실수 0 → 정수 {영점})")
```

```
대칭 int8   : 배율 0.04724, 쓰인 눈금 128개, 평균 오차 0.01163
비대칭 uint8: 배율 0.02353, 쓰인 눈금 256개, 평균 오차 0.00576, 영점 0
-1~3 범위: 배율 0.01569, 영점 64 (실수 0 → 정수 64)
```

예제용으로 0~6 사이의 값 1만 개를 만들었습니다. 대칭 int8 방식은 256개 눈금 중 0~127, 128

개만 썼어요. 비대칭 uint8 방식은 256개를 다 쓰니 눈금 간격(배율)이 절반이 되고, 평균 오차도 0.01163에서 0.00576으로 절반이 됐습니다. 같은 1바이트로 두 배 정밀하게 담은 거예요.

이 값들은 0에서 시작하니 영점은 0입니다. 마지막 줄처럼 -1~3 범위라면 영점이 64가 돼요. 정수 64가 실수 0을 뜻하고, 정수 0은 실수 -1 근처, 정수 255는 3 근처를 뜻합니다. 실수 = (정수 - 영점) × 배율, 이 한 줄이 양자화의 전부예요.

정수끼리 곱하고 마지막에 한 번만 되돌리기

양자화가 빨라지는 비밀은 **계산 자체를 정수로** 한다는 데 있습니다. 6장 6.1절의 층 하나, 즉 행렬곱 `a @ w`를 정수로 해 보겠습니다.

```
import torch

torch.manual_seed(0)
a = torch.rand(4, 64) * 3                       # 입력(활성값): 0 이상
w = torch.randn(64, 8) * 0.1                    # 가중치: 0 근처, 양수와 음수


def 양자화(x: torch.Tensor, 작은: int, 큰: int) -> tuple[torch.Tensor, float, int]:
    """x의 범위를 정수 작은~큰 눈금에 맞춘다 (19.8절의 비대칭 방식)."""
    lo, hi = min(x.min().item(), 0.0), max(x.max().item(), 0.0)
    배율 = (hi - lo) / (큰 - 작은)
    영점 = round(작은 - lo / 배율)
    return (x / 배율 + 영점).round().clamp(작은, 큰).to(torch.int32), 배율, 영점


qa, sa, za = 양자화(a, 0, 255)                  # 입력은 uint8
qw, sw, zw = 양자화(w, -128, 127)               # 가중치는 int8
# 정수끼리 곱해 int32에 모으고, 마지막에 한 번만 배율을 곱해 실수로 되돌린다
정수_누적 = (qa - za) @ (qw - zw)               # int32 행렬 곱
결과 = 정수_누적.float() * (sa * sw)

정답 = a @ w
print("정수 누적값의 자료형:", 정수_누적.dtype, "최댓값:", 정수_누적.abs().max().item())
print("실수 행렬 곱 :", 정답[0, :4])
print("정수 행렬 곱 :", 결과[0, :4])
```

```
print(f"상대 오차: {((결과 - 정답).norm() / 정답.norm()).item():.4f}")
```

```
정수 누적값의 자료형: torch.int32 최댓값: 177979
실수 행렬 곱 : tensor([-1.0971,  3.5936, -2.0597, -1.3047])
정수 행렬 곱 : tensor([-1.0872,  3.5964, -2.0531, -1.3087])
상대 오차: 0.0035
```

입력 a는 uint8로, 가중치 w는 int8로 바꿨습니다. 실수 하나는 (정수 - 영점) × 배율이니, 두 실수의 곱은 (qa - za) × (qw - zw) × sa × sw예요. 그런데 배율 sa × sw는 모든 칸에 같은 숫자라서 덧셈 밖으로 꺼낼 수 있습니다. 그래서 **정수끼리 곱하고 더하는 일을 먼저 다 하고**, 맨 마지막에 배율을 한 번만 곱하면 돼요.

곱해서 더한 값은 17만이 넘어서 int8에 담을 수 없으니, 32비트 정수(int32)에 모읍니다. 결과는 실수 행렬 곱과 상대 오차 0.35% 차이로 거의 같아요. CPU와 휴대폰 칩, GPU에는 이런 8비트 정수 곱셈을 float32보다 몇 배 많이 한꺼번에 하는 명령이 있습니다. 읽어 올 숫자도 4분의 1로 줄고요. 양자화가 크기와 속도를 함께 줄일 수 있는 이유입니다.

배율을 얼마나 잘게 둘까

14장에서 int4가 망가진 것은 행마다 배율을 하나만 썼기 때문이라고 했습니다. 배율을 두는 단위를 **텐서마다**(텐서 전체에 하나), **채널마다**(출력 채널이나 행마다 하나), **묶음마다**(숫자 몇십 개씩 하나)로 바꿔 가며 9.5절 CNN의 가중치를 양자화해 보겠습니다.

```
import torch

from 배포도구 import 불러오기, 옷CNN


def 왕복(w: torch.Tensor, 비트: int, 묶음: str) -> tuple[torch.Tensor, int]:
    """대칭 양자화했다가 되돌린 값과, 그때 쓴 배율의 개수."""
    최대 = 2 ** (비트 - 1) - 1
    if 묶음 == "텐서마다":
        조각 = w.reshape(1, -1)                    # 텐서 전체에 배율 하나
    elif 묶음 == "채널마다":
        조각 = w.reshape(w.shape[0], -1)           # 출력 채널(행)마다 하나
```

```
    else:
        조각 = w.reshape(-1, 32)                    # 숫자 32개마다 하나
    배율 = 조각.abs().amax(dim=1, keepdim=True) / 최대
    되돌림 = (조각 / 배율).round().clamp(-최대, 최대) * 배율
    return 되돌림.reshape(w.shape), 배율.numel()


모델 = 불러오기(옷CNN, "fashion_cnn.pt")
for 이름, w in [("합성곱 2층 (64, 32, 3, 3)", 모델.특징[3].weight.detach()),
                ("완전 연결 (128, 3136)", 모델.분류[1].weight.detach())]:
    print(이름)
    for 묶음 in ["텐서마다", "채널마다", "32개씩"]:
        오차 = []
        for 비트 in (8, 4):
            되돌림, 개수 = 왕복(w, 비트, 묶음)
            오차.append(((되돌림 - w).norm() / w.norm()).item())   # 상대 오차
        print(f"  {묶음:5} 배율 {개수:>6,}개   int8 오차 {오차[0]:.4f}   int4 오차 {오차[1]:.4f}")
```

```
합성곱 2층 (64, 32, 3, 3)
  텐서마다  배율      1개   int8 오차 0.0120   int4 오차 0.2175
  채널마다  배율     64개   int8 오차 0.0084   int4 오차 0.1530
  32개씩  배율    576개   int8 오차 0.0058   int4 오차 0.1055
완전 연결 (128, 3136)
  텐서마다  배율      1개   int8 오차 0.0235   int4 오차 0.3642
  채널마다  배율    128개   int8 오차 0.0115   int4 오차 0.2077
  32개씩  배율 12,544개   int8 오차 0.0058   int4 오차 0.1057
```

배율을 잘게 둘수록 오차가 줄어듭니다. 큰 완전 연결층에서 int8은 텐서마다 2.35% → 채널마다 1.15% → 32개씩 0.58%, int4는 36.4% → 20.8% → 10.6%예요. 이유는 14장의 자 비유 그대로입니다. 행 하나에 아주 큰 값이 하나 있으면 그 행 전체의 눈금 간격이 넓어지는데, 묶음을 작게 나누면 큰 값의 영향이 그 묶음 안에만 머물러요.

대신 배율도 저장해야 합니다. 완전 연결층을 32개씩 나누면 배율이 12,544개가 생겨요. 배율을 float32로 저장하면 숫자 32개(int4로 16바이트)마다 4바이트가 더 붙는 셈입니다. 그래서 보통 int8은 **가중치는 채널마다, 활성값은 텐서마다** 배율을 두고, int4처럼 눈금이 거친 방식은 묶음마다 배율을 둬요. 14장의 LLM을 4비트로 줄이는 GPTQ, AWQ 같은 방법들은 여기에 더해 "오차가 덜 나

도록 눈금에 맞추는 요령"을 쓰는 것입니다.

19.9 PyTorch 안에서 양자화하기: torchao

한 줄로 바꾸기

원리를 알았으니 도구를 써 보겠습니다. PyTorch 쪽 양자화 도구는 **torchao**예요. `quantize_` 함수에 모델과 설정을 넘기면, 모델 안의 `nn.Linear`를 제자리에서 양자화된 것으로 바꿉니다. 가중치만 int8로 담는 방식과, 입력(활성값)까지 실행할 때 int8로 바꿔 정수 계산을 하는 방식을 비교해 보겠습니다.

```
import copy
import io

import torch
from torchao.quantization import (Int8DynamicActivationInt8WeightConfig, Int8WeightOnlyConfig,
                                  quantize_)

from 배포도구 import 불러오기, 시간_재기, 시험_데이터, 옷CNN

torch.set_num_threads(4)
X, y = 시험_데이터()
원본 = 불러오기(옷CNN, "fashion_cnn.pt")


def 저장_크기(모델: torch.nn.Module) -> float:
    """state_dict를 torch.save했을 때의 크기(KB). 파일 대신 메모리 버퍼에 써 본다."""
    버퍼 = io.BytesIO()
    torch.save(모델.state_dict(), 버퍼)
    return len(버퍼.getvalue()) / 1024


for 이름, 설정 in [("float32 (원본)", None),
                   ("가중치만 int8", Int8WeightOnlyConfig()),
                   ("가중치·입력 int8", Int8DynamicActivationInt8WeightConfig())]:
    모델 = copy.deepcopy(원본)
```

```
    if 설정 is not None:
        quantize_(모델, 설정)                    # 모델 안의 nn.Linear를 제자리에서 바꾼다
    with torch.inference_mode():
        정확도 = (모델(X).argmax(dim=1) == y).float().mean().item()
        한장 = 시간_재기(lambda: 모델(X[:1]))
        묶음 = 시간_재기(lambda: 모델(X[:64]))
    print(f"{이름}: {저장_크기(모델):,.0f}KB, 정확도 {정확도:.2%}, 1장 {한장:.3f}ms, 64장 {묶
    ↪ 음:.3f}ms")

w = 모델.분류[1].weight                          # 마지막으로 바꾼 모델의 큰 Linear
print(type(w).__name__, "정수:", w.qdata.dtype, tuple(w.qdata.shape),
      "배율:", tuple(w.scale.shape))
```

```
float32 (원본): 1,650KB, 정확도 91.01%, 1장 0.114ms, 64장 3.490ms
가중치만 int8: 472KB, 정확도 91.02%, 1장 0.198ms, 64장 3.590ms
가중치·입력 int8: 473KB, 정확도 90.98%, 1장 0.288ms, 64장 3.808ms
Int8Tensor 정수: torch.int8 (128, 3136) 배율: (128, 1)
```

크기는 1,650KB에서 472KB로 3.5배쯤 줄었고, 정확도는 91.01%에서 91.02%, 90.98%로 사실상 그대로입니다. 4배가 아닌 이유는 합성곱층은 바꾸지 않았기 때문이에요(`quantize_`는 기본으로 `nn.Linear`만 바꿉니다). 9장 9.5절에서 본 대로 이 모델은 큰 `Linear` 하나가 파라미터의 95%를 차지하니 거의 다 줄어든 셈이죠. 마지막 줄을 보면 그 `Linear`의 가중치가 `Int8Tensor`로 바뀌었고, 안에 int8 정수 (128, 3136)와 **행마다 하나씩인** 배율 (128, 1)을 갖고 있습니다. 앞 절의 "채널마다" 방식이에요.

그런데 속도는 오히려 **느려졌습니다.** 이 맥의 CPU에서 PyTorch를 그대로 실행하는 방식으로는, 정수 칸의 가중치를 다루는 추가 작업이 정수 계산으로 아낀 시간보다 컸던 거예요. 14장 14.6절 Q&A에서 "양자화하면 빨라지는가는 장비와 방법에 따라 다르다"고 했던 것을 실제로 확인한 셈입니다. 크기와 메모리는 확실히 줄었으니, 메모리가 모자라 모델이 안 들어가는 상황이라면 이것만으로도 쓸모가 있어요. 속도까지 얻으려면 그 장비에서 빠른 정수 계산을 쓰는 실행 방식이 함께 필요합니다. 다음 절의 ONNX Runtime이 그 예입니다.

Q. 다른 책에서는 `torch.ao.quantization.quantize_dynamic`을 쓰던데요? PyTorch에 예전부터 있던 양자화 도구입니다. PyTorch 2.14에도 남아 있지만, 쓰면

Q "torch.ao.quantization은 더 이상 권하지 않으며 torchao의 quantize_로 옮기라"는 경고가 나와요. 맥에서는 torch.backends.quantized.engine = "qnnpack"을 먼저 설정해야 동작하는 등 손도 많이 갑니다. 새로 짜는 코드라면 torchao를 쓰세요.

19.10 ONNX Runtime으로 양자화하기

동적 양자화

ONNX Runtime에도 양자화 도구가 들어 있습니다. ONNX 파일을 받아 양자화된 ONNX 파일을 내놓으니, 내보내기가 끝난 뒤의 마지막 단계로 쓰기 좋아요. 먼저 **동적 양자화(dynamic quantization)**입니다. 가중치는 미리 int8로 바꿔 저장하고, 활성값은 실행할 때마다 그 입력을 보고 배율과 영점을 그때그때 정해요.

```
from collections import Counter

import onnx
from onnxruntime.quantization import QuantType, quantize_dynamic
from onnxruntime.quantization.shape_inference import quant_pre_process

from 배포도구 import (ONNX로_내보내기, 불러오기, 세션_만들기, 세션_정확도, 시간_재기,
                    시험_데이터, 옷CNN, 파일_크기)

ONNX로_내보내기(불러오기(옷CNN, "fashion_cnn.pt"), "fashion_cnn.onnx")
quant_pre_process("fashion_cnn.onnx", "fashion_cnn_pre.onnx")          # 양자화 전 준비(모양 추론
↪ 등)
quantize_dynamic("fashion_cnn_pre.onnx", "fashion_cnn_dynamic.onnx",
                 weight_type=QuantType.QInt8)                           # 가중치를 int8로

X, y = 시험_데이터()
X, y = X.numpy(), y.numpy()
for 경로 in ["fashion_cnn.onnx", "fashion_cnn_dynamic.onnx"]:
    세션 = 세션_만들기(경로)
    print(f"{경로:26} {파일_크기(경로):7.1f}KB  정확도 {세션_정확도(세션, X, y):.2%}  "
          f"64장 {시간_재기(lambda: 세션.run(None, {'사진': X[:64]})):.3f}ms")
연산 = Counter(노드.op_type for 노드 in onnx.load("fashion_cnn_dynamic.onnx").graph.node)
```

```
print(dict(연산.most_common()))
```

```
fashion_cnn.onnx            1662.5KB  정확도 91.01%  64장 1.652ms
fashion_cnn_dynamic.onnx     425.1KB  정확도 91.03%  64장 2.683ms
{'Mul': 8, 'DynamicQuantizeLinear': 4, 'Cast': 4, 'Add': 4, 'Reshape': 3, 'Relu': 3, 'ConvInteger':
↪  2, 'MaxPool': 2, 'MatMulInteger': 2}
```

`quant_pre_process`는 양자화 전에 그래프를 정리하고 각 값의 모양을 적어 두는 준비 단계입니다. ONNX Runtime 문서가 양자화 전에 해 두기를 권하는 단계예요. 결과를 보면 크기가 1,662KB에서 425KB로 4분의 1이 됐고, 정확도는 91.03%로 그대로입니다.

그런데 64장 묶음의 시간이 1.7ms에서 2.7ms로 **늘었습니다.** 맨 아래 연산자 목록이 이유를 말해 줘요. `DynamicQuantizeLinear`가 실행할 때마다 활성값의 최솟값·최댓값을 찾아 배율과 영점을 계산하고, `ConvInteger`, `MatMulInteger`가 정수 계산을 한 뒤, `Cast`, `Mul`, `Add`로 다시 실수로 되돌립니다. 층마다 이 앞뒤 작업이 붙으니 작은 CNN에서는 정수 계산으로 아낀 것보다 더 들었어요. ONNX Runtime 문서도 동적 양자화는 RNN이나 트랜스포머 계열 모델에, 정적 양자화는 CNN에 권합니다. 트랜스포머는 거대한 행렬 곱(`MatMul`)이 대부분이라 가중치를 읽는 시간이 크게 줄어드는 이득이 앞뒤 작업보다 크거든요.

정적 양자화와 보정 데이터

정적 양자화(static quantization) 는 활성값의 배율과 영점도 **미리 정해** 둡니다. 실행할 때 계산하지 않으니 앞뒤 작업이 줄어들죠. 그런데 미리 정하려면 "층마다 활성값이 대개 어느 범위에 있는지"를 알아야 해요. 그래서 실제 입력과 닮은 사진을 조금 흘려보내 범위를 재는데, 이 과정을 **보정**(calibration), 그 사진들을 **보정 데이터**라고 합니다. 시험 전에 모의고사를 몇 번 쳐 보고 점수 분포를 보는 것과 비슷해요.

```
from collections import Counter

import numpy as np
import onnx
from onnxruntime.quantization import (CalibrationDataReader, QuantFormat, QuantType,
                                      quantize_static)
from onnxruntime.quantization.shape_inference import quant_pre_process
from torchvision import datasets
```

```
from 배포도구 import (ONNX로_내보내기, 불러오기, 세션_만들기, 세션_정확도, 시간_재기,
                    시험_데이터, 옷CNN, 파일_크기)


class 보정_데이터(CalibrationDataReader):
    """보정용 사진 묶음을 32장씩 하나씩 건네주는 읽개."""

    def __init__(self, 사진: np.ndarray, 크기: int = 32) -> None:
        self.묶음들 = iter([{"사진": 사진[i:i + 크기]} for i in range(0, len(사진), 크기)])

    def get_next(self) -> dict[str, np.ndarray] | None:
        return next(self.묶음들, None)              # 다 건네면 None


# 보정에는 시험 데이터가 아니라 학습 데이터의 일부를 쓴다
학습셋 = datasets.FashionMNIST("data", train=True, download=True)
보정용 = (학습셋.data[:512].float().unsqueeze(1) / 255).numpy()

ONNX로_내보내기(불러오기(옷CNN, "fashion_cnn.pt"), "fashion_cnn.onnx")
quant_pre_process("fashion_cnn.onnx", "fashion_cnn_pre.onnx")
quantize_static("fashion_cnn_pre.onnx", "fashion_cnn_static.onnx", 보정_데이터(보정용),
                quant_format=QuantFormat.QDQ, per_channel=True,
                activation_type=QuantType.QUInt8, weight_type=QuantType.QInt8)

X, y = 시험_데이터()
X, y = X.numpy(), y.numpy()
for 경로 in ["fashion_cnn.onnx", "fashion_cnn_static.onnx"]:
    세션 = 세션_만들기(경로)
    print(f"{경로:24} {파일_크기(경로):7.1f}KB  정확도 {세션_정확도(세션, X, y):.2%}  "
          f"1장 {시간_재기(lambda: 세션.run(None, {'사진': X[:1]})):.3f}ms  "
          f"64장 {시간_재기(lambda: 세션.run(None, {'사진': X[:64]})):.3f}ms")
연산 = Counter(노드.op_type for 노드 in onnx.load("fashion_cnn_static.onnx").graph.node)
print(dict(연산.most_common()))
```

```
fashion_cnn.onnx          1662.5KB  정확도 91.01%  1장 0.067ms  64장 2.146ms
fashion_cnn_static.onnx    430.7KB  정확도 91.09%  1장 0.026ms  64장 0.994ms
{'DequantizeLinear': 16, 'QuantizeLinear': 8, 'Conv': 2, 'MaxPool': 2, 'Gemm': 2, 'Reshape': 1}
```

보정 데이터는 `CalibrationDataReader`를 이어받은 클래스로 건넵니다. `get_next`가 부를 때마다 `{입력 이름: 묶음}`을 하나씩 돌려주고, 다 주면 `None`을 돌려주면 돼요. 1권 15장의 반복자와 같은 생각입니다. 보정에는 학습 데이터의 512장을 썼어요. **시험 데이터로 보정하면 안 됩니다.** 보정도 데이터를 보고 숫자를 정하는 일이니, 시험 데이터를 쓰면 2장 2.3절의 데이터 누수가 돼요.

결과가 좋습니다. 크기는 431KB로 동적 양자화와 비슷하게 4분의 1이 됐고, 정확도는 91.09%로 원본(91.01%)과 거의 같으며, 시간은 사진 1장이 0.067ms에서 0.026ms로, 64장이 2.1ms에서 1.0ms로 **줄었습니다.**

연산자 목록에는 `QuantizeLinear`와 `DequantizeLinear`가 짝지어 있고, 가운데의 `Conv`, `Gemm`은 원래 이름 그대로입니다. 이런 모양을 **QDQ 형식**이라고 해요. "여기서 정수로 바꿨다가(Q) 다시 실수로 되돌린다(DQ)"는 표시만 그래프에 붙여 두는 방식인데, 19.6절에서 본 ONNX Runtime의 그래프 다듬기가 세션을 만들 때 이 표시를 보고 `DQ → Conv → Q` 묶음을 **정수 합성곱 하나**로 합칩니다. 표시만 붙인 그래프라 다른 런타임도 이해하기 쉬워서, ONNX Runtime이 기본으로 쓰는 형식이에요. `per_channel=True`는 19.8절의 "가중치는 채널마다" 배율을 쓰라는 옵션입니다. `activation_type=QuantType.QUInt8`과 `weight_type=QuantType.QInt8`은 19.8절에서 본 대로 활성값은 0~255 눈금(uint8)과 영점으로, 가중치는 int8로 담으라는 뜻이에요. 활성값 쪽을 적지 않으면 ONNX Runtime은 활성값도 int8로 담으니, 이 장의 뒤쪽 예제에서도 두 옵션을 그대로 적어 둡니다.

보정 데이터를 잘못 고르면

보정 데이터는 얼마나, 어떤 것을 써야 할까요? 이번에는 작은 ResNet으로 보정 데이터를 바꿔 가며 실험해 보겠습니다.

```
import numpy as np
from onnxruntime.quantization import (CalibrationDataReader, QuantFormat, QuantType,
                                      quantize_static)
from onnxruntime.quantization.shape_inference import quant_pre_process
from torchvision import datasets

from 배포도구 import ONNX로_내보내기, 불러오기, 세션_만들기, 세션_정확도, 시험_데이터, 작은_ResNet
```

```
class 보정_데이터(CalibrationDataReader):
    """19.10절과 같은 읽개."""

    def __init__(self, 사진: np.ndarray, 크기: int = 32) -> None:
        self.묶음들 = iter([{"사진": 사진[i:i + 크기]} for i in range(0, len(사진), 크기)])

    def get_next(self) -> dict[str, np.ndarray] | None:
        return next(self.묶음들, None)


학습셋 = datasets.FashionMNIST("data", train=True, download=True)
진짜 = (학습셋.data[:512].float().unsqueeze(1) / 255).numpy()
보정_후보 = {
    "학습 사진 512장": 진짜,
    "학습 사진 8장": 진짜[:8],
    "새까만 사진 64장": np.zeros_like(진짜[:64]),
    "0~255 그대로 64장": 진짜[:64] * 255,        # 전처리(/255)를 빼먹은 실수
}

ONNX로_내보내기(불러오기(작은_ResNet, "fashion_resnet.pt"), "fashion_resnet.onnx")
quant_pre_process("fashion_resnet.onnx", "fashion_resnet_pre.onnx")
X, y = 시험_데이터()
X, y = X.numpy(), y.numpy()
print(f"float32 원본: 정확도 {세션_정확도(세션_만들기('fashion_resnet.onnx'), X, y):.2%}")
for 이름, 사진 in 보정_후보.items():
    quantize_static("fashion_resnet_pre.onnx", "fashion_resnet_calib.onnx", 보정_데이터(사진),
                    quant_format=QuantFormat.QDQ, per_channel=True,
                    activation_type=QuantType.QUInt8, weight_type=QuantType.QInt8)
    정확도 = 세션_정확도(세션_만들기("fashion_resnet_calib.onnx"), X, y)
    print(f"[{이름}] 보정 후 정확도 {정확도:.2%}")
```

```
float32 원본: 정확도 93.04%
[학습 사진 512장] 보정 후 정확도 93.07%
[학습 사진 8장] 보정 후 정확도 92.98%
```

```
[새까만 사진 64장] 보정 후 정확도 54.86%
[0~255 그대로 64장] 보정 후 정확도 9.85%
```

학습 사진 512장으로 보정하면 93.07%로 원본(93.04%)과 같고, **8장**만으로도 92.98%입니다. 활성값의 범위만 재면 되니 많이 필요하지 않아요. 문제는 데이터의 **종류**입니다. 새까만 사진으로 보정하면 54.86%로 무너지고, `/ 255`를 빼먹어 0~255 값을 그대로 넣으면 9.85%, 열 가지 중 찍는 수준이 됩니다. 범위를 엉뚱하게 재면 실제 입력이 눈금 밖으로 나가 잘려 버리거나, 눈금이 너무 거칠어지기 때문이에요.

무서운 점은 이 과정에서 **오류가 하나도 나지 않는다**는 것입니다. 양자화는 조용히 끝나고, 정확도를 재 봐야 비로소 알 수 있어요. 그래서 양자화 뒤에는 반드시 원본과 같은 시험 데이터로 정확도를 비교해야 합니다. 그리고 19.3절에서 말한 전처리 이야기가 여기서도 그대로예요. 보정 데이터도, 배포 후의 입력도 **학습 때와 똑같이 전처리**해야 합니다.

19.11 더 작게 만드는 다른 방법: 가지치기와 지식 증류

양자화는 "숫자 하나를 담는 칸"을 줄였습니다. 모델을 작게 만드는 길은 두 가지가 더 있어요. 숫자의 **개수**를 줄이는 가지치기와, **작은 모델을 새로 가르치는** 지식 증류입니다. 둘 다 깊이 들어가면 한 장씩 필요한 주제라, 아이디어만 실험으로 확인하겠습니다.

가지치기: 작은 가중치를 0으로

정원사가 웃자란 가지를 쳐내듯, **가지치기(pruning)** 는 절댓값이 작은 가중치를 0으로 만듭니다. 0에 가까운 가중치는 결과에 거의 기여하지 않을 테니까요. PyTorch에는 `torch.nn.utils.prune`이 있습니다.

```
import gzip
import io

import torch
import torch.nn.utils.prune as prune

from 배포도구 import 불러오기, 시험_데이터, 옷CNN

X, y = 시험_데이터()
```

```
def 크기들(모델: torch.nn.Module) -> tuple[float, float]:
    """state_dict의 저장 크기와 gzip으로 압축한 크기(KB)."""
    버퍼 = io.BytesIO()
    torch.save(모델.state_dict(), 버퍼)
    return len(버퍼.getvalue()) / 1024, len(gzip.compress(버퍼.getvalue())) / 1024


for 비율 in [0.0, 0.5, 0.8, 0.9]:
    모델 = 불러오기(옷CNN, "fashion_cnn.pt")
    대상 = [(모델.특징[0], "weight"), (모델.특징[3], "weight"),
            (모델.분류[1], "weight"), (모델.분류[3], "weight")]
    if 비율 > 0:
        # 네 층의 가중치를 한데 모아 절댓값이 작은 것부터 '비율'만큼 0으로
        prune.global_unstructured(대상, pruning_method=prune.L1Unstructured, amount=비율)
        for 층, 이름 in 대상:
            prune.remove(층, 이름)                # 가림판을 떼고 0을 가중치에 굳힌다
    영 = sum((층.weight == 0).sum().item() for 층, _ in 대상)
    전체 = sum(층.weight.numel() for 층, _ in 대상)
    with torch.no_grad():
        정확도 = (모델(X).argmax(dim=1) == y).float().mean().item()
    저장, 압축 = 크기들(모델)
    print(f"가지치기 {비율:3.0%}: 0인 가중치 {영 / 전체:5.1%}, 정확도 {정확도:.2%}, "
          f"저장 {저장:,.0f}KB, gzip {압축:,.0f}KB")
```

```
가지치기  0%: 0인 가중치  0.0%, 정확도 91.01%, 저장 1,650KB, gzip 1,532KB
가지치기 50%: 0인 가중치 50.0%, 정확도 90.99%, 저장 1,650KB, gzip 894KB
가지치기 80%: 0인 가중치 80.0%, 정확도 89.37%, 저장 1,650KB, gzip 447KB
가지치기 90%: 0인 가중치 90.0%, 정확도 78.15%, 저장 1,650KB, gzip 283KB
```

`global_unstructured`는 네 층의 가중치를 한데 모아 절댓값이 작은 것부터 정한 비율만큼 0으로 만들고, `prune.remove`는 그 결과를 가중치에 굳힙니다. 가중치의 절반을 0으로 만들어도 정확도는 91.01%에서 90.99%로 그대로고, 80%를 0으로 만들어도 89.37%예요. 90%에서는 78.15%로 크게 떨어집니다.

그런데 **저장 크기가 1,650KB로 그대로**입니다. 0도 float32 숫자 하나라서 4바이트를 차지하거든

요. 계산량도 그대로예요. 0을 곱하는 것도 곱셈이니까요. 대신 gzip으로 압축하면 50%에서 894KB, 80%에서 447KB로 줄어듭니다. 같은 0이 잔뜩 반복되니 압축이 잘 되는 거죠. 가지치기로 실제 속도를 얻으려면 0을 건너뛰는 특별한 계산 방식이나, 채널이나 필터를 **통째로** 잘라 내서 층 자체를 작게 만드는 방식이 필요합니다. 그래서 실무에서는 압축된 크기만 필요할 때 쓰거나, 통째로 잘라 낸 뒤 다시 조금 학습시키는 식으로 씁니다.

지식 증류: 선생의 답안지로 배우기

지식 증류(knowledge distillation) 는 큰 모델(선생)의 출력을 보고 작은 모델(학생)을 가르치는 방법입니다. 정답 레이블은 "이건 셔츠"라는 한 가지만 알려 주지만, 선생의 점수는 "셔츠일 확률 80%, T셔츠일 확률 17%"처럼 **무엇과 얼마나 닮았는지**까지 알려 주죠. 학생은 그 닮음의 정도까지 따라 배웁니다.

9.10절의 작은 ResNet(파라미터 308,074개)을 선생으로, 파라미터가 9,098개뿐인 작은 CNN을 학생으로 하겠습니다. 학생에게는 학습 사진 **2,000장**만 주고, 정답만으로 배운 학생과 선생의 점수도 함께 따라 배운 학생을 씨앗 세 개로 비교해요(18장에서 다루는 "여러 씨앗으로 돌려 평균 내기"를 미리 해 보는 셈입니다).

```
import torch
import torch.nn.functional as F
from torch import nn
from torchvision import datasets

from 배포도구 import 불러오기, 시험_데이터, 작은_ResNet

torch.set_num_threads(4)
학습셋 = datasets.FashionMNIST("data", train=True, download=True)
X학습 = 학습셋.data[:2000].float().unsqueeze(1) / 255      # 학생에게는 2,000장만
y학습 = 학습셋.targets[:2000]
X, y = 시험_데이터()

선생 = 불러오기(작은_ResNet, "fashion_resnet.pt")          # 6만 장으로 배운 선생 (308,074개)
with torch.no_grad():
    선생_점수 = 선생(X학습)                                  # 선생의 답안지를 미리 받아 둔다


def 학생() -> nn.Sequential:
```

```
    """채널을 확 줄인 작은 CNN."""
    return nn.Sequential(
        nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        nn.Flatten(), nn.Linear(16 * 7 * 7, 10),
    )


def 학습시키기(증류: bool, 씨앗: int, 온도: float = 4.0, 비중: float = 0.9) -> float:
    """학생을 30에폭 학습하고 시험 정확도를 돌려준다. 증류면 선생의 점수도 따라 배운다."""
    torch.manual_seed(씨앗)
    모델 = 학생()
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=2e-3)
    for _ in range(30):
        for 번호 in torch.randperm(2000).split(64):
            출력 = 모델(X학습[번호])
            손실 = F.cross_entropy(출력, y학습[번호])
            if 증류:   # 온도로 부드럽게 만든 두 확률 분포의 차이 (KL 발산)
                따라하기 = F.kl_div(F.log_softmax(출력 / 온도, dim=1),
                                    F.softmax(선생_점수[번호] / 온도, dim=1),
                                    reduction="batchmean") * 온도 ** 2
                손실 = 비중 * 따라하기 + (1 - 비중) * 손실
            옵티마이저.zero_grad()
            손실.backward()
            옵티마이저.step()
    with torch.no_grad():
        return (모델(X).argmax(dim=1) == y).float().mean().item()


print(f"학생 파라미터 {sum(p.numel() for p in 학생().parameters()):,}개")
결과 = {"정답만": [], "증류": []}
for 씨앗 in range(3):
    for 방식 in 결과:
        결과[방식].append(학습시키기(방식 == "증류", 씨앗))
for 방식, 값 in 결과.items():
    print(f"{방식:4}: " + ", ".join(f"{v:.2%}" for v in 값) + f"  평균 {sum(값) / len(값):.2%}")
```

```
학생 파라미터 9,098개
정답만 : 84.21%, 83.60%, 82.23%  평균 83.35%
증류   : 85.09%, 84.11%, 85.21%  평균 84.80%
```

세 씨앗 모두 증류한 학생이 더 높았고, 평균은 83.35%에서 84.80%로 1.45%p 올랐습니다. 학생은 같은 2,000장을 봤지만, 선생의 점수에는 선생이 6만 장에서 배운 "옷들이 서로 얼마나 닮았나"가 담겨 있어서 그것까지 전해 받은 거예요.

코드에서 두 가지를 보세요. 하나는 **온도**입니다. 14장 14.3절에서 온도로 확률 분포를 평평하게 만들었죠. 선생의 점수를 온도 4로 나눠 소프트맥스를 하면 1등이 아닌 클래스의 확률도 드러나서 닮음의 정도가 잘 보입니다. 다른 하나는 **손실을 섞는 비중**이에요. 선생 따라 하기(KL 발산)에 0.9, 정답 맞히기에 0.1을 줬습니다. 온도 제곱을 곱한 것은 온도로 나누면서 작아진 기울기 크기를 되돌리려는 관례예요.

정직하게 덧붙이면, 학생에게도 6만 장을 다 주고 3에폭씩 같은 비교를 했을 때는 증류의 이득이 보이지 않았습니다. 세 씨앗의 평균이 정답만 배운 학생 88.52%, 증류한 학생 87.60%로 오히려 조금 낮았어요. 데이터가 넉넉하면 학생이 정답만으로도 충분히 배우기 때문이에요. 증류는 학생이 볼 수 있는 레이블 데이터가 적거나, 선생이 학생보다 훨씬 많은 것을 알 때 빛납니다. 오늘날 작은 LLM 중 상당수가 큰 LLM을 선생으로 이런 방식의 학습을 거쳐 만들어져요.

19.12 휴대폰과 작은 기기에서: 런타임 고르기

ONNX Runtime은 실행 장치를 바꿔 끼울 수 있습니다. 19.3절에서 `providers=["CPUExecutionProvider"]`라고 쓴 자리예요. 이 컴퓨터에서 쓸 수 있는 것을 확인해 보겠습니다.

```
import onnxruntime as ort

print("ONNX Runtime", ort.__version__)
print("이 컴퓨터에서 쓸 수 있는 실행 장치:", ort.get_available_providers())
```

```
ONNX Runtime 1.31.0
이 컴퓨터에서 쓸 수 있는 실행 장치: ['CoreMLExecutionProvider', 'AzureExecutionProvider',
↪  'CPUExecutionProvider']
```

이 맥에서는 CPU 말고도 `CoreMLExecutionProvider`가 보입니다. 애플 기기의 Core ML을 통해 GPU

나 신경망 엔진을 쓰는 실행 장치예요. NVIDIA GPU가 있는 컴퓨터에 GPU용 ONNX Runtime을 설치하면 `CUDAExecutionProvider`가 나타납니다. 모델 파일은 그대로 두고 `providers` 목록만 바꾸면 된다는 것이 ONNX Runtime의 장점이에요. 다만 실행 장치마다 지원하는 연산과 속도가 다르니, 바꿨다면 19.3절처럼 결과를 비교하고 19.7절처럼 시간을 다시 재야 합니다.

휴대폰이나 작은 기기에서 돌릴 때는 런타임 선택지가 더 있습니다. 이름과 한 줄 소개만 정리해 둘게요.

런타임	한 줄 소개
ONNX Runtime (모바일·웹)	이 장에서 쓴 엔진의 안드로이드·iOS판과 웹 브라우저판. ONNX 파일을 그대로 쓴다
ExecuTorch	PyTorch가 만든 기기용 런타임. `torch.export`로 딴 그래프를 기기용 파일로 바꿔 돌린다
LiteRT	구글의 기기용 런타임. 예전 이름은 TensorFlow Lite
Core ML	애플 기기의 머신러닝 프레임워크. coremltools로 PyTorch 모델을 바꿔 넣는다
TensorRT	NVIDIA GPU용 추론 최적화 도구. ONNX 파일을 받아 그 GPU에 맞게 다듬는다
OpenVINO	인텔 CPU·GPU용 추론 도구. ONNX 파일을 읽을 수 있다

어느 것을 고를지는 결국 **어디서 돌릴 것인가**가 정합니다. 파이썬 서버나 여러 운영체제를 함께 지원해야 한다면 ONNX Runtime이 무난하고, 특정 기기나 가속기의 성능을 끝까지 뽑아야 한다면 그 회사의 도구를 씁니다. 어느 쪽이든 이 장에서 익힌 과정, 즉 **내보내기 → 같은 답인지 확인 → 양자화 → 정확도와 시간 다시 재기**는 똑같아요.

19.13 실습: 작은 ResNet을 배포용으로 만들기

내보내고, 양자화하고, 같은 잣대로 비교하기

이 장에서 배운 것을 모아 9장 9.10절의 작은 ResNet을 배포용 파일로 만들겠습니다. 할 일은 세 가지예요.

1. ONNX로 내보내고 PyTorch와 같은 답을 내는지 확인하기
2. 학습 사진 512장으로 보정해 정적 양자화하기(가중치는 int8·채널마다 배율, 활성값은 uint8, QDQ 형식)
3. PyTorch float32, ONNX float32, ONNX int8을 **같은 잣대**로 비교하기: 파일 크기, 시험 정확도, 셔츠 정확도(9장의 약점), float32 PyTorch와 답이 같은 비율, 4칸 밀었을 때 정확도(9장 9.6절), 1장·64장 시간

```
import numpy as np
import torch
from onnxruntime.quantization import (CalibrationDataReader, QuantFormat, QuantType,
                                      quantize_static)
from onnxruntime.quantization.shape_inference import quant_pre_process
from torchvision import datasets

from 배포도구 import (ONNX로_내보내기, 불러오기, 세션_만들기, 시간_재기, 시험_데이터,
                     작은_ResNet, 파일_크기)
from 옷도구 import 오른쪽으로_밀기


class 보정_데이터(CalibrationDataReader):
    """19.10절과 같은 읽개."""

    def __init__(self, 사진: np.ndarray, 크기: int = 32) -> None:
        self.묶음들 = iter([{"사진": 사진[i:i + 크기]} for i in range(0, len(사진), 크기)])

    def get_next(self) -> dict[str, np.ndarray] | None:
        return next(self.묶음들, None)


torch.set_num_threads(4)
X, y = 시험_데이터()
X_np, y_np = X.numpy(), y.numpy()
```

```
밀린_np = 오른쪽으로_밀기(X, 4).numpy()                # 4칸 밀어 본 시험 사진

# 1) 내보내고 확인하기
모델 = 불러오기(작은_ResNet, "fashion_resnet.pt")
ONNX로_내보내기(모델, "fashion_resnet.onnx")
with torch.inference_mode():
    기준 = 모델(X).numpy()
원본 = 세션_만들기("fashion_resnet.onnx")
print(f"[확인] PyTorch와 최대 차이 {np.abs(원본.run(None, {'사진': X_np})[0] - 기준).max():.1e}")

# 2) 학습 사진 512장으로 보정해 정적 양자화
학습셋 = datasets.FashionMNIST("data", train=True, download=True)
보정용 = (학습셋.data[:512].float().unsqueeze(1) / 255).numpy()
quant_pre_process("fashion_resnet.onnx", "fashion_resnet_pre.onnx")
quantize_static("fashion_resnet_pre.onnx", "fashion_resnet_int8.onnx", 보정_데이터(보정용),
                quant_format=QuantFormat.QDQ, per_channel=True,
                activation_type=QuantType.QUInt8, weight_type=QuantType.QInt8)
정수 = 세션_만들기("fashion_resnet_int8.onnx")

# 3) 세 가지 실행 방법을 같은 잣대로 비교
후보 = {
    "PyTorch float32": ("fashion_resnet.pt", lambda x: 모델(torch.from_numpy(x)).numpy()),
    "ONNX float32": ("fashion_resnet.onnx", lambda x: 원본.run(None, {"사진": x})[0]),
    "ONNX int8": ("fashion_resnet_int8.onnx", lambda x: 정수.run(None, {"사진": x})[0]),
}
print(f"{'방법':16}{'크기':>8}{'정확도':>8}{'셔츠':>7}{'일치':>8}{'4칸':>7}{'1장':>9}{'64장':>9}")
for 이름, (파일, 실행) in 후보.items():
    with torch.inference_mode():
        예측 = 실행(X_np).argmax(axis=1)
        밀림 = (실행(밀린_np).argmax(axis=1) == y_np).mean()
        한장 = 시간_재기(lambda: 실행(X_np[:1]))
        묶음 = 시간_재기(lambda: 실행(X_np[:64]))
    셔츠 = (예측[y_np == 6] == 6).mean()
    일치 = (예측 == 기준.argmax(axis=1)).mean()            # float32 PyTorch와 같은 답의 비율
    print(f"{이름:16}{파일_크기(파일):6.0f}KB{(예측 == y_np).mean():8.2%}{셔츠:7.1%}"
          f"{일치:8.2%}{밀림:7.1%}{한장:7.3f}ms{묶음:7.2f}ms")
```

```
[확인] PyTorch와 최대 차이 1.9e-05
방법                    크기     정확도     셔츠      일치     4칸       1장      64장
PyTorch float32   1227KB  93.04%  77.4% 100.00%  92.0%  0.586ms  36.73ms
ONNX float32      1235KB  93.04%  77.4% 100.00%  92.0%  0.278ms  15.09ms
ONNX int8          349KB  93.07%  77.2%  99.55%  92.0%  0.129ms   5.52ms
```

표를 한 칸씩 읽어 보겠습니다.

- **크기**: int8 모델은 349KB로 원본(1,227KB)의 4분의 1이 조금 넘습니다. 정확히 4분의 1이 아닌 것은 배율과 영점, 그리고 QDQ 표시 노드가 함께 들어 있기 때문이에요.
- **정확도**: 93.04% → 93.07%. 양자화로 잃은 것이 없습니다. 셔츠 정확도도 77.4%에서 77.2%로 거의 같고, 4칸 밀었을 때도 셋 다 92.0%예요. 9장에서 얻은 성질이 양자화 뒤에도 그대로 남았습니다.
- **일치**: 정확도가 같다고 답까지 다 같은 것은 아닙니다. int8 모델은 float32와 99.55%의 사진에서 같은 답을 냈어요. 1만 장 중 45장은 답이 바뀌었는데, 맞던 것이 틀리게 된 것과 틀리던 것이 맞게 된 것이 비슷해서 정확도는 그대로인 거죠. 경계에 걸친 애매한 사진들입니다. 실무에서 양자화를 검증할 때는 정확도와 함께 이 일치율도 보면 좋아요.
- **시간**: 이 책의 실행 환경(Apple M4 Max, CPU, 스레드 4개)에서 ONNX float32가 PyTorch보다 두 배 남짓 빠르고(19.7절과 같은 흐름이에요), int8은 거기서 다시 2~3배 빨라졌습니다. PyTorch와 견주면 1장은 4.5배, 64장은 6.7배 빠른 셈이에요. 다만 이 표는 컴퓨터가 한가할 때 잰 것입니다. 다른 프로그램이 CPU를 많이 쓰던 때 같은 실습을 돌렸더니 64장 시간이 PyTorch 101ms, ONNX float32 26ms, int8 7.7ms까지 늘었어요. 한가할 때 세 번 더 돌려 보면 PyTorch는 36~38ms, ONNX Runtime은 15ms 남짓, int8은 4.9~5.5ms로 거의 일정했습니다. 그래서 "몇 배 빨라졌다"는 숫자 하나보다, **같은 시각에 같은 잣대로 나란히 잰 표**와 그 흔들림의 폭을 함께 남기는 것이 정직한 보고입니다. 어느 실행에서든 순서는 같았어요. int8 ONNX가 가장 빠르고, PyTorch가 가장 느렸습니다.

배포하는 쪽의 코드

마지막으로, 이 파일을 받은 쪽에서 쓸 코드를 짜 보겠습니다. torch 없이, 넘파이와 ONNX Runtime만으로 0~255 정수 사진을 받아 확률 높은 후보 세 개를 돌려주는 작은 클래스예요. 전처리(`/ 255`)와 소프트맥스(6장 6.2절)를 직접 넣었습니다.

```
import sys

import numpy as np
import onnxruntime as ort

이름 = ["T셔츠", "바지", "풀오버", "드레스", "코트", "샌들", "셔츠", "운동화", "가방", "앵클부츠"]


class 옷_분류기:
    """ONNX 파일 하나와 넘파이만으로 옷 사진을 분류한다 (torch 없이)."""

    def __init__(self, 경로: str) -> None:
        설정 = ort.SessionOptions()
        설정.intra_op_num_threads = 4
        self.세션 = ort.InferenceSession(경로, 설정, providers=["CPUExecutionProvider"])

    def 예측(self, 사진: np.ndarray, 몇_개: int = 3) -> list[list[tuple[str, float]]]:
        """0~255 정수 사진 (장 수, 28, 28)을 받아 장마다 확률 높은 순서로 몇_개를 돌려준다."""
        x = 사진.reshape(-1, 1, 28, 28).astype(np.float32) / 255     # 학습 때와 같은 전처리
        점수 = self.세션.run(None, {"사진": x})[0]
        확률 = np.exp(점수 - 점수.max(axis=1, keepdims=True))         # 소프트맥스 (6장 6.2절)
        확률 /= 확률.sum(axis=1, keepdims=True)
        순서 = np.argsort(-확률, axis=1)[:, :몇_개]
        return [[(이름[k], float(p[k])) for k in 줄] for p, 줄 in zip(확률, 순서)]


사진 = np.fromfile("data/FashionMNIST/raw/t10k-images-idx3-ubyte", dtype=np.uint8, offset=16)
정답 = np.fromfile("data/FashionMNIST/raw/t10k-labels-idx1-ubyte", dtype=np.uint8, offset=8)
사진 = 사진.reshape(-1, 28, 28)

분류기 = 옷_분류기("fashion_resnet_int8.onnx")
for 번호, 후보 in zip([0, 4, 7], 분류기.예측(사진[[0, 4, 7]])):
    print(f"{번호}번 (정답 {이름[정답[번호]]}): " + ", ".join(f"{n} {p:.1%}" for n, p in 후보))
print("torch를 불러왔나:", "torch" in sys.modules)
```

```
0번 (정답 앵클부츠): 앵클부츠 99.9%, 운동화 0.1%, 샌들 0.0%
4번 (정답 셔츠): 셔츠 80.3%, T셔츠 17.4%, 코트 1.4%
7번 (정답 셔츠): 셔츠 99.8%, 코트 0.2%, 풀오버 0.0%
torch를 불러왔나: False
```

0번 사진은 앵클부츠를 99.9%로 확신했고, 7번 셔츠도 99.8%로 맞혔습니다. 눈여겨볼 것은 4번 사진이에요. 정답은 셔츠인데 셔츠 80.3%, T셔츠 17.4%로 나눠 걸었습니다. 9장 내내 이 모델을 괴롭힌 셔츠와 T셔츠의 혼동이 확률로 드러난 거예요. 앱에서는 이렇게 1등만 보여 주지 말고 확신이 낮을 때는 후보를 함께 보여 주는 식으로 쓸 수 있습니다. 마지막 줄의 `False`가 말해 주듯 이 코드는 PyTorch 없이 돌아갑니다. 앞에서 본 74MB짜리 ONNX Runtime과 349KB짜리 파일 하나면 충분해요.

19.14 정리와 다음 장 미리보기

이 장에서 모델을 PyTorch 밖으로 꺼내고 작게 줄였습니다.

- **ONNX**: 계산 그래프(노드) + 가중치(이니셜라이저) + 입출력 정보 + 연산자 판(opset)을 담는 공개 형식. pickle이 아니라 열 때 코드가 실행되지 않는다
- **내보내기**: `torch.onnx.export(모델, (예시_입력,), 경로)`. PyTorch 2.9부터 `torch.export` 기반이 기본. 평가 모드로, 작은 모델은 `external_data=False`로 한 파일에, `dynamic_shapes`로 묶음 차원 열기, `input_names·output_names`로 이름 붙이기
- **확인**: ONNX Runtime의 `InferenceSession`과 `run`(넘파이 배열을 주고받는다), PyTorch와 최대 차이 비교, 전처리는 배포하는 쪽에서 다시 짠다
- **내보낼 수 있는 코드**: `forward`는 텐서 연산만으로. 값에 따라 갈리는 `if`는 실패하니 `torch.where`나 `torch.cond`로, 파이썬 쪽 계산(넘파이 난수 등)은 상수로 굳는다
- **그래프 최적화**: 배치 정규화 접어 넣기, `Conv·Relu·Add` 합치기(`FusedConv`). 같은 모델이 ONNX Runtime에서 두 배쯤 빨랐다
- **속도 재기**: 준비 운동 뒤 여러 번 재서 중앙값, 스레드 수 맞추기, 실행 환경 밝히기
- **양자화 원리**: 실수 = (정수 - 영점) × 배율. 활성값은 uint8과 영점, 정수끼리 곱해 int32에 모으고 마지막에 배율 한 번. 배율은 텐서·채널·묶음마다, 잘게 둘수록 오차가 작다
- **양자화 도구**: torchao의 `quantize_`(크기는 줄었지만 이 CPU에서 빨라지지는 않음), ONNX Runtime의 동적 양자화(트랜스포머 계열에 알맞음)와 정적 양자화(CNN에 알맞음, 보정 데이터 필요, QDQ 형식). 보정 데이터는 학습 데이터에서, 학습 때와 같은 전처리로
- **그 밖의 방법**: 가지치기(0을 늘리지만 크기·속도는 그대로, 압축은 잘 됨), 지식 증류(선생의

점수로 작은 학생 가르치기)

- **실습 결과**: 작은 ResNet을 4분의 1 크기, 정확도 그대로, 이 책의 실행 환경에서 PyTorch보다 4~7배(1장 4.5배, 64장 6.7배) 빠른 파일로 만들었다. 시간은 같은 시각에 나란히 재고 흔들림의 폭도 함께 남긴다

모델이 파일 하나와 작은 런타임만으로 돌아가게 됐으니, 이제 이것을 다른 사람이 쓸 수 있게 열어 줄 차례입니다. 다음 20장에서는 1권 19장의 FastAPI 위에 모델을 올려 **웹 서비스**로 만들어요. 서버가 시작될 때 모델을 한 번만 불러오고, 업로드한 사진을 받아 분류하고, 여러 요청을 묶어 처리하고, 이 장에서 만든 ONNX 파일로 백엔드를 바꿔 끼운 뒤, Docker로 감싸 어디서든 띄울 수 있게 만듭니다.

20

FastAPI로 모델 서빙: 내 모델을 웹 서비스로

학습 목표

- 모델을 요청마다 불러오면 생기는 문제를 재 보고, `lifespan`으로 서버가 켜질 때 한 번만 올리기
- Pydantic으로 추론 요청과 응답의 모양을 정하고, 업로드한 사진을 학습 때와 같은 방법으로 전처리하기
- 여러 장을 한 번에 처리하는 배치 추론과, 따로 온 요청을 모아 한 번에 계산하는 방법 만들기
- CPU를 오래 쓰는 추론이 이벤트 루프를 막는 모습을 보고, 스레드로 넘겨 해결하기
- 헬스 체크·버전 정보·처리 시간 헤더를 달고, 지연 시간을 백분위수로 재기
- 19장의 ONNX 파일로 백엔드를 바꿔 끼우고, LLM의 답을 토큰마다 흘려보내기
- pytest로 서비스를 시험하고, uv와 Docker로 어디서나 같은 모습으로 띄우기

들어가며: 모델 파일을 친구에게 보내면 끝일까요?

9장에서 옷 사진을 91% 맞히는 CNN을 만들어 `fashion_cnn.pt`로 저장했습니다. 친구가 쇼핑 앱을 만드는데 "사진을 올리면 옷 종류를 자동으로 채워 주는 기능"에 이 모델을 쓰고 싶대요. 파일을 보내 주면 될까요?

19장 들어가며에서 본 것처럼 `.pt` 파일만으로는 아무것도 할 수 없습니다. 친구 앱에 파이썬과

PyTorch, 우리가 짠 `옷CNN` 클래스와 전처리 코드까지 옮겨야 하고, 모델을 고칠 때마다 친구 앱을 다시 배포해야 해요.

1권 19장의 식당을 떠올려 보세요. 손님은 주방에 들어가 요리하지 않습니다. 주문서를 내면 주방이 음식을 내줄 뿐이에요. 모델도 **주방에 두고 주문을 받으면** 됩니다. 친구 앱은 사진을 HTTP 요청에 담아 보내고, 우리 서버는 모델로 계산한 답을 JSON으로 돌려주는 거죠. 앱이 어떤 언어로 짜였든 HTTP만 보낼 줄 알면 되고, 모델을 바꿔도 서버만 다시 띄우면 됩니다. 이렇게 학습한 모델을 다른 프로그램이 쓸 수 있게 내놓는 일을 **모델 서빙(model serving)** 이라고 합니다.

경로 함수, `Depends`, Pydantic 검증, `TestClient`는 1권 19장에서 이미 써 봤어요. 그런데 주방에 들어오는 것이 블로그 글이 아니라 **모델**이 되면 새로운 고민이 생깁니다.

- 모델은 불러오는 데 오래 걸리고 메모리를 많이 씁니다. 언제 불러와 어디에 둘까요?
- 사진 파일을 학습할 때와 같은 모양으로 어떻게 바꿀까요?
- 추론이 CPU를 붙잡는 동안 다른 손님은 기다려야 할까요?
- 모아서 계산하면 빠른데 손님은 하나씩 옵니다. 어떻게 모을까요?
- 다른 컴퓨터에 옮겨도 똑같이 돌게 하려면 무엇을 챙길까요?

이 질문들에 하나씩 답한 뒤, 마지막 실습에서 9장 옷 분류기를 테스트까지 갖춘 서비스로 완성하고 Docker 이미지로 감쌉니다.

Q

Q. 이 장에 필요한 패키지는 어떻게 설치하나요? 프로젝트 폴더에서 `uv add "fastapi[standard]" pillow onnxruntime`을 실행하세요. `fastapi[standard]`에는 서버 `uvicorn`, 파일 업로드에 필요한 `python-multipart`, `TestClient`가 쓰는 HTTP 라이브러리가 함께 들어 있어요(1권 19.2절). `pillow`는 사진 파일을 여는 라이브러리, `onnxruntime`은 19장의 추론 엔진입니다. 20.10절은 14장의 `transformers`도 씁니다. 이 장의 예제는 파이썬 3.12, PyTorch 2.14.1, FastAPI 0.156.4, Pydantic 2.14.1, uvicorn 0.54.0, python-multipart 0.0.32, Pillow 13.0.0, ONNX Runtime 1.31.0, transformers 5.20.2, pytest 9.1.1에서 실행해 확인했고, 컨테이너는 Docker 28.3.2와 uv 0.9.16으로 만들었습니다.

20.1 모델을 서버에 올리기

이 장의 공통 도구

이 장의 예제는 거의 모두 9장 9.5절의 CNN을 불러와 씁니다. 9장의 `옷도구.py`에 들어 있는 설계도 `옷CNN`을 가져와서, 저장한 가중치를 **추론용으로** 불러오는 함수를 하나 만들어 두겠습니다. 아래 내용을 `서빙도구.py`로 저장하고, 같은 폴더에 9장의 `옷도구.py`와 `fashion_cnn.pt`를 두세요.

```
"""20장 예제에서 함께 쓰는 도구: 9장 9.5절 CNN을 추론용으로 불러오기, 시험 사진을 파일로 만들기."""
import io
from pathlib import Path

import torch
from torchvision import datasets

from 옷도구 import 옷CNN

클래스_이름 = ["T셔츠", "바지", "풀오버", "드레스", "코트",
          "샌들", "셔츠", "운동화", "가방", "앵클부츠"]


def 모델_불러오기(경로: str | Path = "fashion_cnn.pt") -> 옷CNN:
    """저장된 가중치를 CPU로 불러와(4장 4.9절) 평가 모드로 돌려준다(6장 6.8절)."""
    모델 = 옷CNN()
    모델.load_state_dict(torch.load(경로, map_location="cpu", weights_only=True))
    return 모델.eval()


def 시험_사진_파일(개수: int) -> list[tuple[bytes, int]]:
    """FashionMNIST 시험 사진을 앞에서부터 개수만큼, PNG 파일 내용과 정답 번호로 돌려준다."""
    시험셋 = datasets.FashionMNIST("data", train=False, download=True)
    파일들 = []
    for i in range(개수):
        사진, 정답 = 시험셋[i]
        버퍼 = io.BytesIO()                    # 디스크 대신 메모리에 파일을 쓴다
        사진.save(버퍼, format="PNG")
        파일들.append((버퍼.getvalue(), 정답))
    return 파일들
```

`map_location="cpu"`는 GPU에서 저장한 파일도 CPU로 불러오게 하고(4장 4.9절),

`weights_only=True`는 텐서 말고 다른 파이썬 객체가 든 파일을 거부합니다. 여러 사람이 쓰는 서버에서는 안전한 쪽이 기본이어야 해요. `.eval()`도 빠뜨리면 안 됩니다. 드롭아웃이나 배치 정규화가 있는 모델을 학습 모드로 서빙하면 같은 사진에도 답이 흔들리거나 정확도가 떨어져요(6장 6.8절).

요청마다 불러오면 생기는 일

가장 단순한 서빙은 경로 함수 안에서 모델을 불러와 계산하는 것입니다. 요리 주문이 들어올 때마다 주방을 새로 차리는 셈이죠. 미리 차려 둔 주방과 얼마나 차이가 날까요? 아직 사진 업로드를 만들지 않았으니, FashionMNIST 시험 사진의 **번호**를 주소로 받아 그 사진을 분류하는 API로 비교하겠습니다.

```
import statistics
import time

import torch
from fastapi import FastAPI
from fastapi.testclient import TestClient
from torchvision import datasets
from torchvision.transforms.functional import to_tensor

from 서빙도구 import 모델_불러오기, 클래스_이름

시험셋 = datasets.FashionMNIST("data", train=False, download=True)
app = FastAPI()
미리_올린_모델 = 모델_불러오기()            # 프로그램이 시작될 때 한 번


def 예측(모델: torch.nn.Module, index: int) -> dict:
    사진, 정답 = 시험셋[index]
    with torch.no_grad():
        점수 = 모델(to_tensor(사진).unsqueeze(0))
    return {"예측": 클래스_이름[점수.argmax().item()], "정답": 클래스_이름[정답]}


@app.get("/slow/{index}")
def 느린_예측(index: int) -> dict:
```

```
    return 예측(모델_불러오기(), index)       # 요청이 올 때마다 파일에서 다시 불러온다


@app.get("/fast/{index}")
def 빠른_예측(index: int) -> dict:
    return 예측(미리_올린_모델, index)         # 이미 올려 둔 모델을 쓴다


client = TestClient(app)
print(client.get("/fast/0").json())
for 주소 in ["/slow", "/fast"]:
    걸린_시간 = []
    for i in range(50):
        시작 = time.perf_counter()
        client.get(f"{주소}/{i}")
        걸린_시간.append((time.perf_counter() - 시작) * 1000)
    print(f"{주소:5}: 요청 하나에 {statistics.median(걸린_시간):.2f}ms (중앙값)")
```

```
{'예측': '앵클부츠', '정답': '앵클부츠'}
/slow: 요청 하나에 3.73ms (중앙값)
/fast: 요청 하나에 1.85ms (중앙값)
```

0번 시험 사진을 앵클부츠로 맞혔으니 서버는 제대로 돕니다. 시간을 보면 요청마다 모델을 불러오는 `/slow`가 미리 올려 둔 `/fast`보다 두 배쯤 걸렸어요. 이 모델은 1.6MB짜리라 차이가 1~2ms에 그치지만, 모델이 커지면 이야기가 달라집니다. 이 책의 실행 환경에서 14장의 0.5B 언어 모델은 파일이 이미 메모리 캐시에 올라와 있는데도 불러오는 데만 0.2~0.4초가 걸렸어요. 계산은 시작도 안 했는데요. 게다가 요청 열 개가 동시에 들어오면 2GB짜리 모델이 메모리에 열 벌 올라갑니다.

이 비교로 원칙 하나를 얻습니다. **모델은 서버가 켜질 때 한 번 불러오고, 요청은 그 모델을 같이 쓴다.** 위 코드의 `/fast`는 모델을 모듈 맨 위의 전역 변수에 올려서 이 원칙을 지켰어요. 그런데 이 방법에는 문제가 있습니다. 다음 절에서 그 문제와 FastAPI가 준비한 정식 방법을 보겠습니다.

20.2 서버가 켜질 때 한 번만: lifespan

전역 변수로 올리면 무엇이 불편할까요?

모듈 맨 위에서 모델을 불러오면 `import`하는 순간 모델이 올라갑니다. 테스트 코드가 경로 함수 하나만 확인하려고 `import`해도 무거운 모델이 올라오고, 모델 파일이 없는 컴퓨터에서는 `import` 자체가 실패해요. 서버가 꺼질 때 GPU 메모리를 비우거나 연결을 닫는 정리 작업을 넣을 자리도 없습니다.

FastAPI는 이를 위해 **lifespan**(수명)이라는 자리를 마련해 두었습니다. 1권 15장의 `@asynccontextmanager`로 함수를 하나 만들면, `yield` 앞은 서버가 요청을 받기 **직전에**, `yield` 뒤는 서버가 꺼질 때 실행돼요. 식당으로 치면 문 열기 전 준비와 문 닫은 뒤 정리입니다.

```
from contextlib import asynccontextmanager

import torch
from fastapi import FastAPI, Request
from fastapi.testclient import TestClient

from 서빙도구 import 모델_불러오기, 클래스_이름


@asynccontextmanager
async def lifespan(app: FastAPI):
    print("[시작] 모델을 올립니다")
    app.state.모델 = 모델_불러오기()          # 서버가 시작될 때 딱 한 번
    yield                                    # 여기서 요청을 받는다
    print("[종료] 모델을 내립니다")
    del app.state.모델


app = FastAPI(lifespan=lifespan)


@app.get("/predict/blank")
def 빈_사진_예측(request: Request) -> dict:
    모델 = request.app.state.모델             # 시작할 때 올려 둔 모델 꺼내기
    with torch.no_grad():
        점수 = 모델(torch.zeros(1, 1, 28, 28))
    return {"예측": 클래스_이름[점수.argmax().item()]}
```

```
with TestClient(app) as client:              # with 블록에 들어갈 때 시작, 나올 때 종료
    print(client.get("/predict/blank").json())
    print(client.get("/predict/blank").json())
print("with 블록 밖")

client = TestClient(app)                     # with 없이 쓰면 lifespan이 돌지 않는다
try:
    client.get("/predict/blank")
except AttributeError as e:
    print("오류:", e)
```

```
[시작] 모델을 올립니다
{'예측': '샌들'}
{'예측': '샌들'}
[종료] 모델을 내립니다
with 블록 밖
오류: 'State' object has no attribute '모델'
```

출력 순서를 보세요. `[시작]`이 요청보다 먼저, `[종료]`가 마지막 요청 뒤에 찍혔습니다. 요청이 두 번 왔지만 모델은 한 번만 올라갔어요.

- **`FastAPI(lifespan=lifespan)`**: 앱을 만들 때 수명 함수를 넘깁니다.
- **`app.state`**: 앱에 딸린 보관함입니다. 아무 이름이나 붙여 값을 넣어 둘 수 있어요. 시작할 때 넣은 모델을 경로 함수에서 `request.app.state.모델`로 꺼내 씁니다. 매개변수에 `request: Request`를 적으면 FastAPI가 지금 요청 객체를 넣어 줘요.
- **빈 사진의 예측**: 아무것도 없는 검은 사진을 넣었는데 모델은 "샌들"이라고 답했습니다. 분류 모델은 열 가지 중 하나를 반드시 고르게 만들어져서 "모르겠다"고 하지 못해요. 서빙할 때는 이 점을 기억해 두세요. 엉뚱한 사진에도 자신 있게 틀린 답을 냅니다.

마지막 부분이 이 절에서 가장 중요합니다. `TestClient`를 `with` 없이 만들어 요청하면 lifespan이 실행되지 않아서 `app.state`에 모델이 없다는 `AttributeError`가 나요. 1권 19장에서는 lifespan이 없었으니 `with` 없이 써도 괜찮았지만, 이제부터는 **`with TestClient(app) as client:`** 로 써야 서버가 시작·종료되는 과정까지 흉내 냅니다. 실제 서버인 uvicorn은 언제나 lifespan을 실행하니, 이 실수는 테스트에서만 나타나서 더 헷갈려요.

Q. 예전 코드에서 `@app.on_event("startup")`을 봤는데, 그건 뭔가요? lifespan이 생기기 전에 쓰던 방법입니다. 시작할 때 할 일과 끝날 때 할 일을 서로 다른 함수 두 개에 나눠 적었는데, 그러면 시작할 때 만든 값을 끝날 때 함수로 넘기기 번거로웠어요. 지금 쓰면 "on_event는 더 이상 권하지 않으니 lifespan을 쓰라"는 경고(`DeprecationWarning`)가 나와요. 하나의 함수 안에서 `yield` 앞뒤로 준비와 정리가 짝을 이루니, 1권 12장의 `with` 문처럼 "열었으면 반드시 닫는다"가 눈에 보입니다.

20.3 주문서 양식: 요청과 응답 스키마

옷 분류기에 넣을 값은 28 × 28 크기에 0~1 사이 실수여야 합니다. 1권 19.5절에서 Pydantic으로 "주문서 양식"을 정해 두면 잘못된 주문서는 경로 함수에 닿기 전에 422로 돌려보낸다고 했죠. 모델 서빙에서는 이 검사가 더 중요해요. 모양이 틀린 입력이 모델에 들어가면 4장 4.10절의 모양 오류가 서버 안에서 터져 500이 되거나, 더 나쁘게는 오류 없이 엉뚱한 답이 나가거든요.

이번에는 사진을 숫자 목록(28줄 × 28칸)으로 받고, 응답도 Pydantic 모델로 정해 보겠습니다. 응답 모양을 정해 두면 자동 문서(1권 19.4절)에 "이런 모양이 돌아온다"가 적혀서, API를 쓰는 친구가 코드를 읽지 않아도 됩니다.

```
from contextlib import asynccontextmanager
from typing import Annotated

import torch
from fastapi import FastAPI, Request
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field
from torchvision import datasets
from torchvision.transforms.functional import to_tensor

from 서빙도구 import 모델_불러오기, 클래스_이름

픽셀 = Annotated[float, Field(ge=0.0, le=1.0)]                # 칸 하나: 0~1 사이 실수
한_줄 = Annotated[list[픽셀], Field(min_length=28, max_length=28)]


class PixelInput(BaseModel):
```

```
    pixels: Annotated[list[한_줄], Field(min_length=28, max_length=28)]   # 28줄 × 28칸


class Prediction(BaseModel):
    label: str
    class_id: int
    probability: float


@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.모델 = 모델_불러오기()
    yield


app = FastAPI(lifespan=lifespan)


@app.post("/predict")
def 예측(입력: PixelInput, request: Request) -> Prediction:
    x = torch.tensor(입력.pixels).reshape(1, 1, 28, 28)
    with torch.no_grad():
        확률 = request.app.state.모델(x).softmax(dim=1)[0]
    번호 = 확률.argmax().item()
    return Prediction(label=클래스_이름[번호], class_id=번호,
                      probability=round(확률[번호].item(), 4))


사진, 정답 = datasets.FashionMNIST("data", train=False, download=True)[0]
픽셀값 = to_tensor(사진)[0].tolist()                                  # 28 × 28 리스트
with TestClient(app) as client:
    응답 = client.post("/predict", json={"pixels": 픽셀값})
    print(응답.status_code, 응답.json(), "정답:", 클래스_이름[정답])

    잘못된_입력 = {"pixels": 픽셀값[:27]}                               # 27줄뿐
    잘못된_입력["pixels"][3][5] = 2.0                                  # 1보다 큰 값
```

```
    for 차례 in range(2):
        응답 = client.post("/predict", json=잘못된_입력)
        print(응답.status_code, [(오류["loc"], 오류["msg"]) for 오류 in 응답.json()["detail"]])
        잘못된_입력["pixels"][3][5] = 0.5                          # 값을 고쳐서 다시 보내기
    print(sorted(client.get("/openapi.json").json()["components"]["schemas"]))
```

```
200 {'label': '앵클부츠', 'class_id': 9, 'probability': 0.9973} 정답: 앵클부츠
422 [(['body', 'pixels', 3, 5], 'Input should be less than or equal to 1')]
422 [(['body', 'pixels'], 'List should have at least 28 items after validation, not 27')]
['HTTPValidationError', 'PixelInput', 'Prediction', 'ValidationError']
```

0번 시험 사진을 보냈더니 앵클부츠일 확률 0.9973이라는 답이 응답 모델 `Prediction`의 모양 그대로 왔습니다. 경로 함수가 `-> Prediction`으로 응답 모양을 밝혔으니(1권 19.6절), 마지막 줄처럼 자동 문서의 스키마 목록에도 `PixelInput`과 `Prediction`이 올라 있어요.

입력 양식은 `Annotated`로 층층이 쌓았습니다. 칸 하나는 0~1 사이 실수, 한 줄은 그런 칸이 정확히 28개, 사진은 그런 줄이 정확히 28개예요. JSON 필드 이름(`pixels`, `label`)과 모델 클래스 이름(`PixelInput`, `Prediction`)은 1권 19.5절 Q&A에서 본 이유로 영어로 지었습니다.

잘못된 요청 두 번의 결과는 조금 흥미롭습니다. 27줄짜리 사진에 3번 줄 5번 칸이 2.0인 요청을 보냈는데, 첫 응답은 **칸 값**만 지적하고 줄 수는 말하지 않았어요. 칸 값을 고쳐 다시 보내자 그제야 "28개 이상이어야 하는데 27개"라고 알려 줍니다. Pydantic은 목록 안의 값을 먼저 검사하고, 그게 통과해야 목록의 길이를 셉니다. 그러니 422 응답을 받고 한 곳만 고쳤는데 또 다른 오류가 나와도 당황하지 마세요. `loc`의 `['body', 'pixels', 3, 5]`처럼 오류 위치가 정확히 찍히니 하나씩 고치면 됩니다.

다만 이 방법은 사진 한 장을 숫자 784개짜리 JSON으로 보내야 해서, 앱에서 쓰기에 번거롭고 크기도 큽니다. 실제 앱은 사진 **파일**을 그대로 올리죠. 다음 절에서 파일을 받아 보겠습니다.

20.4 사진 파일 받기

UploadFile로 받기

파일은 JSON 본문이 아니라 **multipart/form-data**라는 형식으로 옵니다. 웹 페이지의 "파일 선택" 버튼이 보내는 형식이에요. FastAPI에서는 매개변수 타입을 `UploadFile`로, 기본값을 `File(...)`로 적으면 됩니다. 받은 파일은 Pillow로 열어서 학습 데이터와 같은 **흑백 28 × 28**로 바꾸고, 9장처럼

`to_tensor`로 0~1 사이 실수 텐서를 만듭니다.

```
import io
from contextlib import asynccontextmanager

import torch
from fastapi import FastAPI, File, HTTPException, Request, UploadFile
from fastapi.testclient import TestClient
from PIL import Image, UnidentifiedImageError
from torchvision import datasets
from torchvision.transforms.functional import to_tensor

from 서빙도구 import 모델_불러오기, 클래스_이름

최대_바이트 = 1_000_000                      # 사진 한 장은 1MB까지만 받는다


def 사진_전처리(데이터: bytes) -> torch.Tensor:
    """올라온 사진 파일을 학습 때와 같은 (1, 1, 28, 28) 텐서로 바꾼다."""
    try:
        사진 = Image.open(io.BytesIO(데이터))
        사진 = 사진.convert("L").resize((28, 28))          # 흑백, 28 × 28
    except UnidentifiedImageError:
        raise HTTPException(status_code=400, detail="사진 파일이 아닙니다")
    return to_tensor(사진).unsqueeze(0)                    # 0~1 실수, 배치 차원 추가


@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.모델 = 모델_불러오기()
    yield


app = FastAPI(lifespan=lifespan)


@app.post("/predict/image")
```

```
def 사진_예측(request: Request, file: UploadFile = File(...)) -> dict:
    데이터 = file.file.read(최대_바이트 + 1)          # 한도보다 1바이트만 더 읽어 본다
    if len(데이터) > 최대_바이트:
        raise HTTPException(status_code=413, detail="파일이 너무 큽니다")
    with torch.no_grad():
        확률 = request.app.state.모델(사진_전처리(데이터)).softmax(dim=1)[0]
    return {"파일": file.filename, "예측": 클래스_이름[확률.argmax().item()],
            "확률": round(확률.max().item(), 4)}


def png_바이트(사진: Image.Image) -> bytes:
    버퍼 = io.BytesIO()
    사진.save(버퍼, format="PNG")
    return 버퍼.getvalue()


시험셋 = datasets.FashionMNIST("data", train=False, download=True)
사진, 정답 = 시험셋[1]
큰_컬러 = 사진.convert("RGB").resize((280, 280))              # 열 배 크게, 컬러로 바꾼 사진
보낼_것들 = {
    "옷.png": png_바이트(사진),
    "큰_컬러.png": png_바이트(큰_컬러),
    "메모.txt": "사진이 아니에요".encode(),
    "큰_파일.png": bytes(2_000_000),
}
with TestClient(app) as client:
    print("정답:", 클래스_이름[정답])
    for 이름, 내용 in 보낼_것들.items():
        응답 = client.post("/predict/image", files={"file": (이름, 내용)})
        print(f"{이름:10} {len(내용):>9,}바이트 → {응답.status_code} {응답.json()}")
```

```
정답: 풀오버
옷.png            594바이트 → 200 {'파일': '옷.png', '예측': '풀오버', '확률': 0.9998}
큰_컬러.png      21,698바이트 → 200 {'파일': '큰_컬러.png', '예측': '풀오버', '확률': 0.999}
메모.txt            22바이트 → 400 {'detail': '사진 파일이 아닙니다'}
큰_파일.png   2,000,000바이트 → 413 {'detail': '파일이 너무 큽니다'}
```

네 가지 파일을 보냈습니다.

- **옷.png**: 1번 시험 사진(풀오버)을 PNG로 저장한 594바이트짜리 파일입니다. 숫자 784개짜리 JSON보다 훨씬 작죠.
- **큰_컬러.png**: 같은 사진을 열 배 키우고 컬러로 바꿨는데도 `convert("L")`(흑백)과 `resize((28, 28))`를 거쳐 같은 답이 나왔어요.
- **메모.txt**: 사진이 아닌 파일은 Pillow가 `UnidentifiedImageError`를 냅니다. 이걸 잡아서 `400 Bad Request`로 바꿨어요. 잡지 않으면 FastAPI는 이를 서버 잘못으로 보고 500을 돌려줍니다. 손님이 잘못 보낸 것은 4xx, 서버가 잘못한 것은 5xx라는 1권 19.1절의 약속을 지키려면 예외를 알맞은 상태 코드로 바꿔 줘야 해요.
- **큰_파일.png**: 1MB가 넘는 파일은 `413`(내용이 너무 큼)으로 거절했습니다. 파일을 통째로 읽지 않고 **한도보다 1바이트만 더** 읽어 본 것이 요령이에요. 1바이트라도 더 읽혔다면 한도를 넘은 파일입니다. 이렇게 하면 누군가 10GB짜리 파일을 보내도 **메모리에 올리는 것**은 1MB 남짓뿐이에요. 다만 경로 함수가 실행될 때는 FastAPI(정확히는 그 밑의 Starlette)가 요청 본문을 이미 끝까지 받아서, 큰 파일은 임시 파일로 디스크에 써 둔 뒤입니다. 받는 것 자체를 막으려면 요청 머리의 `Content-Length`를 먼저 보고 거절하는 미들웨어(20.7절)를 두거나, 서버 앞에 둔 프록시에서 크기를 제한합니다(nginx라면 `client_max_body_size` 설정).

경로 함수를 `async def`가 아닌 그냥 `def`로 쓰고 파일을 `file.file.read()`로 읽은 데에는 이유가 있습니다. 20.6절에서 자세히 봐요.

학습 때와 같은 전처리

서빙에서 가장 흔하고 찾기 어려운 버그는 **학습할 때와 서빙할 때의 전처리가 다른 것**입니다. 10장 10.2절에서 사전학습 모델에 ImageNet의 평균과 표준편차를 맞춰 넣어야 했던 것과 같은 이야기예요. 오류는 나지 않고 정확도만 조용히 떨어지니 더 무섭습니다.

FashionMNIST는 **검은 배경에 밝은 옷**입니다. 그런데 쇼핑몰 상품 사진은 대개 흰 배경에 옷이 있죠. 이런 사진이 들어오면 어떻게 될까요? 시험 사진 1,000장으로 두 가지 실수를 흉내 내 보겠습니다. 이번에는 텐서 대신 NumPy 배열로 전처리하는데, 이유는 20.8절에서 밝혀집니다.

```
import numpy as np
import torch
from PIL import Image, ImageOps
from torchvision import datasets
```

```
from 서빙도구 import 모델_불러오기

모델 = 모델_불러오기()
시험셋 = datasets.FashionMNIST("data", train=False, download=True)
사진들 = [시험셋[i][0] for i in range(1000)]
정답 = torch.tensor([시험셋[i][1] for i in range(1000)])


def 정확도(전처리, 사진_목록: list[Image.Image]) -> float:
    x = torch.stack([전처리(사진) for 사진 in 사진_목록])
    with torch.no_grad():
        return (모델(x).argmax(dim=1) == 정답).float().mean().item()


def 학습때와_같이(사진: Image.Image) -> torch.Tensor:
    배열 = np.asarray(사진.convert("L").resize((28, 28)), dtype=np.float32) / 255
    return torch.from_numpy(배열).unsqueeze(0)


def 나누기_빠뜨림(사진: Image.Image) -> torch.Tensor:
    배열 = np.asarray(사진.convert("L").resize((28, 28)), dtype=np.float32)   # 0~255 그대로
    return torch.from_numpy(배열).unsqueeze(0)


def 배경_맞추기(사진: Image.Image) -> torch.Tensor:
    x = 학습때와_같이(사진)
    테두리 = torch.cat([x[0, 0], x[0, -1], x[0, :, 0], x[0, :, -1]])
    return 1 - x if 테두리.mean() > 0.5 else x      # 배경이 밝으면 뒤집어 검은 배경으로


흰_배경 = [ImageOps.invert(사진) for 사진 in 사진들]   # 쇼핑몰 사진처럼 흰 배경에 어두운 옷
print(f"학습 때와 같은 전처리         : {정확도(학습때와_같이, 사진들):.1%}")
print(f"255로 나누기를 빠뜨림         : {정확도(나누기_빠뜨림, 사진들):.1%}")
print(f"흰 배경 사진, 그대로          : {정확도(학습때와_같이, 흰_배경):.1%}")
print(f"흰 배경 사진, 배경 맞추기     : {정확도(배경_맞추기, 흰_배경):.1%}")
print(f"원래 사진에 배경 맞추기를 써도: {정확도(배경_맞추기, 사진들):.1%}")
```

```
학습 때와 같은 전처리          : 90.7%
255로 나누기를 빠뜨림          : 84.5%
흰 배경 사진, 그대로           : 28.4%
흰 배경 사진, 배경 맞추기      : 90.7%
원래 사진에 배경 맞추기를 써도: 90.7%
```

학습 때와 같은 전처리로는 1,000장 중 90.7%를 맞힙니다. 첫 번째 실수인 **255로 나누기를 빠뜨리면** 84.5%로 떨어졌어요. 값이 255배나 커졌는데 생각보다 덜 떨어졌죠? 그래서 더 위험합니다. 오류 없이 6%p를 잃었으니, 정확도를 따로 재 보지 않으면 알아챌 길이 없어요.

진짜 문제는 **흰 배경 사진**입니다. 흑백을 뒤집은 사진을 그대로 넣었더니 28.4%, 열 가지 중 아무거나 찍는 10%보다 조금 나은 수준까지 무너졌어요. 모델은 9장에서 배운 "밝은 부분의 모양"을 보는데, 배경 전체가 밝아졌으니 엉뚱한 것을 보게 된 거죠.

`배경_맞추기`는 사진 테두리의 평균 밝기를 보고, 0.5보다 밝으면 배경이 흰 사진으로 보고 뒤집습니다. 이 한 단계로 흰 배경 사진의 정확도가 원래와 같은 90.7%로 돌아왔어요. 그리고 마지막 줄처럼 원래 사진에 써도 정확도가 그대로입니다. FashionMNIST 사진은 테두리가 모두 어둡기 때문이에요. 이렇게 "학습 데이터처럼 생기지 않은 입력을 학습 데이터처럼 바꾸는" 단계는, 서빙할 곳의 실제 입력을 보고 나서야 알 수 있습니다. 그래서 서비스를 열고 나서도 어떤 사진이 들어오는지 꾸준히 들여다봐야 해요.

Q

Q. 전처리 코드를 학습 코드와 서빙 코드에 따로 두면 안 되나요? 따로 두면 언젠가는 어긋납니다. 학습 쪽에서 크기를 32 × 32로 바꿨는데 서빙 쪽은 28 × 28 그대로 둔다든지요. 가장 좋은 방법은 전처리 함수를 **한 곳에** 두고 학습과 서빙이 같이 `import`하는 것입니다. 그럴 수 없다면 이 절처럼 "같은 사진을 두 전처리에 넣으면 같은 결과가 나오는가"를 확인하는 테스트를 20.11절처럼 만들어 두세요. 10장 10.2절의 `weights.transforms()`처럼 모델과 전처리를 한 묶음으로 배포하는 것도 같은 생각입니다.

20.5 여러 장을 한 번에: 배치 추론

한 장씩 vs 묶어서

4장 4.2절에서 모델은 사진을 한 장씩이 아니라 **배치**로 묶어 처리한다고 했습니다. 그래야 계산 장치가 쉬지 않으니까요. 서빙에서는 이게 얼마나 차이를 낼까요? 사진 512장을 배치 크기를 바꿔 가며

처리하고 한 장당 시간을 재 보겠습니다. 4장 4.8절처럼 준비 운동을 한 뒤 일곱 번 재서 중앙값을 썼어요.

```
import statistics
import time

import torch

from 서빙도구 import 모델_불러오기

torch.manual_seed(0)
모델 = 모델_불러오기()
사진들 = torch.rand(512, 1, 28, 28)                    # 시간만 잴 것이라 아무 사진이나


def 시간_재기(배치_크기: int, 반복: int = 7) -> float:
    """사진 512장을 배치_크기씩 나눠 처리할 때, 사진 한 장당 걸린 시간(ms)의 중앙값."""
    기록 = []
    with torch.no_grad():
        모델(사진들[:배치_크기])                          # 준비 운동 (4장 4.8절)
        for _ in range(반복):
            시작 = time.perf_counter()
            for i in range(0, len(사진들), 배치_크기):
                모델(사진들[i:i + 배치_크기])
            기록.append((time.perf_counter() - 시작) * 1000 / len(사진들))
    return statistics.median(기록)


print("배치 크기  한 장당 시간  한 장씩 대비")
기준 = 시간_재기(1)
for 크기 in [1, 8, 32, 128, 512]:
    걸림 = 기준 if 크기 == 1 else 시간_재기(크기)
    print(f"{크기:>7}  {걸림:9.3f}ms  {기준 / 걸림:8.1f}배")
```

```
배치 크기  한 장당 시간  한 장씩 대비
      1      0.273ms       1.0배
```

```
   8      0.095ms       2.9배
  32      0.079ms       3.5배
 128      0.036ms       7.6배
 512      0.028ms       9.9배
```

이 책의 실행 환경(Apple M4 Max, CPU)에서 잰 값입니다. 한 장씩 넣으면 한 장에 0.27ms쯤 걸리던 것이, 128장씩 묶으면 0.04ms 아래로, 512장을 한꺼번에 넣으면 0.03ms 아래로 줄었어요. 같은 일을 열 배 가까이 빨리 한 셈입니다. 모델을 한 번 부를 때마다 드는 고정 비용(파이썬 함수 호출, 층마다 계산을 준비하는 일)을 여러 장이 나눠 내고, 큰 행렬 곱은 CPU의 여러 코어와 벡터 연산을 더 잘 활용하기 때문이에요. GPU라면 차이가 훨씬 더 큽니다(16장).

따로 온 요청을 모아서 한 번에

그렇다면 손님이 사진 여러 장을 한 번에 보내게 하면 되겠죠? 매개변수를 `list[UploadFile]`로 받으면 같은 이름으로 온 파일들이 목록이 됩니다. 20.11절 실습의 `/predict/batch`가 그렇게 만든 것인데, 한 요청이 너무 많은 장수를 보내면 그 계산이 끝날 때까지 다른 손님이 기다려야 하니 장수에 상한(32장)을 두었어요. 그런데 실제 손님은 대개 사진을 **한 장씩, 따로따로** 보냅니다. 앱 사용자 64명이 동시에 사진을 하나씩 올리는 식이죠. 손님 쪽을 바꿀 수 없다면 서버 쪽에서 모으면 됩니다. 버스 정류장을 떠올려 보세요. 버스는 손님 한 명이 올 때마다 출발하지 않고, 자리가 차거나 정해진 시간이 되면 출발합니다. 서버도 첫 요청이 오면 아주 잠깐(여기서는 5ms) 더 기다리면서 다른 요청을 모았다가, 32장이 차거나 시간이 다 되면 모델을 한 번 부르는 거예요. 이런 방법을 **동적 배칭**(dynamic batching)이라고 합니다.

asyncio의 `Queue`(대기 줄)와, 1권 15장에서 어웨이터블의 하나로 이름만 봤던 `Future`(나중에 답이 채워질 빈자리)로 만들 수 있습니다. 동시에 64명이 요청하는 상황은 `TestClient`로 만들기 어려워서, `TestClient`가 안에서 쓰는 `httpx2`의 비동기 클라이언트를 직접 썼어요. `ASGITransport`는 네트워크를 거치지 않고 앱에 바로 요청을 넣는 연결입니다.

```
import asyncio
import io
from contextlib import asynccontextmanager

import httpx2
import numpy as np
import torch
```

```
from fastapi import FastAPI, File, Request, UploadFile
from fastapi.concurrency import run_in_threadpool
from PIL import Image

from 서빙도구 import 모델_불러오기, 시험_사진_파일, 클래스_이름


class 배치_모으기:
    """따로 온 요청을 잠깐 모았다가 모델을 한 번만 부른다."""

    def __init__(self, 모델: torch.nn.Module, 최대_장수: int = 32,
                 최대_대기: float = 0.005) -> None:
        self.모델, self.최대_장수, self.최대_대기 = 모델, 최대_장수, 최대_대기
        self.큐: asyncio.Queue = asyncio.Queue()
        self.배치_기록: list[int] = []

    async def 예측(self, x: np.ndarray) -> int:
        결과_자리 = asyncio.get_running_loop().create_future()   # 나중에 답이 들어올 빈자리
        await self.큐.put((x, 결과_자리))
        return await 결과_자리                                    # 답이 채워질 때까지 기다린다

    async def 계속_돌리기(self) -> None:
        루프 = asyncio.get_running_loop()
        while True:
            묶음 = [await self.큐.get()]                          # 첫 요청이 올 때까지 기다리고
            마감 = 루프.time() + self.최대_대기                    # 그때부터 잠깐만 더 모은다
            while len(묶음) < self.최대_장수 and (남은 := 마감 - 루프.time()) > 0:
                try:
                    묶음.append(await asyncio.wait_for(self.큐.get(), 남은))
                except TimeoutError:
                    break
            x = torch.from_numpy(np.stack([x for x, _ in 묶음]))
            번호들 = await run_in_threadpool(self.계산, x)        # 계산은 스레드에서 (20.6절)
            for (_, 결과_자리), 번호 in zip(묶음, 번호들):
                결과_자리.set_result(번호)
            self.배치_기록.append(len(묶음))
```

```
    def 계산(self, x: torch.Tensor) -> list[int]:
        with torch.no_grad():
            return self.모델(x).argmax(dim=1).tolist()


@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.모으기 = 배치_모으기(모델_불러오기())
    일꾼 = asyncio.create_task(app.state.모으기.계속_돌리기())   # 뒤에서 계속 도는 작업
    yield
    일꾼.cancel()


app = FastAPI(lifespan=lifespan)


@app.post("/predict/image")
async def 사진_예측(request: Request, file: UploadFile = File(...)) -> dict:
    # 28 × 28짜리 작은 사진이라 이벤트 루프에서 바로 전처리했다. 무거운 전처리라면 20.6절처럼 스레
    ↪ 드로 넘긴다
    사진 = Image.open(io.BytesIO(await file.read())).convert("L").resize((28, 28))
    x = np.asarray(사진, dtype=np.float32)[None] / 255
    return {"label": 클래스_이름[await request.app.state.모으기.예측(x)]}


async def 손님_64명() -> None:
    파일들 = 시험_사진_파일(64)
    async with lifespan(app):                                  # TestClient 대신 직접 시작·종료
        전송 = httpx2.ASGITransport(app=app)
        async with httpx2.AsyncClient(transport=전송, base_url="http://test") as client:
            요청들 = [client.post("/predict/image", files={"file": ("x.png", 내용)})
                      for 내용, _ in 파일들]
            응답들 = await asyncio.gather(*요청들)
        맞힘 = sum(r.json()["label"] == 클래스_이름[답] for r, (_, 답) in zip(응답들, 파일들))
        기록 = app.state.모으기.배치_기록
```

```
        print(f"요청 {len(응답들)}개 → 모델 호출 {len(기록)}번, 배치 크기 {기록}")
        print(f"64장 중 {맞힘}장 정답")


asyncio.run(손님_64명())
```

```
요청 64개 → 모델 호출 2번, 배치 크기 [32, 32]
64장 중 58장 정답
```

요청 64개가 들어왔는데 모델은 **두 번**만 불렸고, 한 번에 32장씩 처리했습니다. 64장 중 58장(90.6%)을 맞혀 정확도도 그대로예요. 흐름을 따라가 보겠습니다.

1. 경로 함수 `사진_예측`은 사진을 전처리한 뒤 `모으기.예측(x)`를 `await`합니다. `예측`은 빈자리(`Future`)를 하나 만들어 사진과 함께 대기 줄에 넣고, 그 자리에 답이 채워질 때까지 기다려요. 기다리는 동안 이벤트 루프는 다른 요청을 받습니다.
2. lifespan에서 `asyncio.create_task`로 띄워 둔 `계속_돌리기`가 뒤에서 계속 돕니다. 대기 줄에서 첫 사진을 꺼내면 마감 시각을 정하고, 마감 전까지 최대 32장을 더 꺼내 모아요.
3. 모은 사진을 `np.stack`으로 한 배치로 쌓아 모델을 한 번 부르고, 답을 각자의 빈자리에 `set_result`로 채웁니다. 그러면 1번에서 기다리던 경로 함수들이 깨어나 응답을 돌려줘요.

최대 대기 시간은 손님이 손해를 보는 시간입니다. 요청이 드물 때는 첫 손님이 5ms를 그냥 기다리니까요. 대신 요청이 몰리면 모델 호출이 수십 분의 1로 줄어 처리량이 오릅니다. 작은 모델에서는 이득이 크지 않지만, 한 번 부르는 비용이 큰 GPU 모델에서는 서빙 도구들이 기본으로 갖추는 기능이에요.

20.6 오래 걸리는 계산과 async

이벤트 루프를 막으면

1권 19.2절에서 `async def` 경로 함수는 이벤트 루프에서 직접 실행되고, 그냥 `def`는 FastAPI가 따로 마련한 작업 스레드에서 실행된다고 했습니다. 그리고 1권 15장에서 `async def` 안에서 `time.sleep()`처럼 멈추는 코드를 부르면 이벤트 루프 전체가 멈춘다고 했죠. 모델 추론도 그런 코드입니다. `await`할 곳 없이 CPU로 계산만 하니까요.

직접 확인해 보겠습니다. 요청 하나에 사진 3,000장을 계산하는 무거운 예측 API를 세 가지 방식으

로 만들고, 예측 요청 네 개를 보낸 직후에 가벼운 /health 요청을 하나 보냅니다. /health의 답이 언제 오는지가 관심사예요.

```
import asyncio
import time

import httpx2
import torch
from fastapi import FastAPI
from fastapi.concurrency import run_in_threadpool

from 서빙도구 import 모델_불러오기

모델 = 모델_불러오기()
사진들 = torch.rand(3000, 1, 28, 28)       # 요청 하나에 계산이 꽤 걸리게 3,000장


def 무거운_추론() -> int:
    with torch.no_grad():
        return 모델(사진들).argmax(dim=1)[0].item()


def 앱_만들기(방식: str) -> FastAPI:
    app = FastAPI()

    if 방식 == "async def에서 바로":
        @app.get("/predict")
        async def 예측() -> dict:
            return {"label": 무거운_추론()}          # 이벤트 루프 위에서 계산
    elif 방식 == "그냥 def":
        @app.get("/predict")
        def 예측() -> dict:
            return {"label": 무거운_추론()}          # FastAPI가 스레드에서 실행
    else:
        @app.get("/predict")
        async def 예측() -> dict:
            return {"label": await run_in_threadpool(무거운_추론)}   # 계산만 스레드로
```

```
    @app.get("/health")
    async def 상태() -> dict:
        return {"status": "ok"}

    return app


async def 시험하기(방식: str) -> None:
    전송 = httpx2.ASGITransport(app=앱_만들기(방식))
    async with httpx2.AsyncClient(transport=전송, base_url="http://test") as client:
        async def 상태_확인() -> float:
            await asyncio.sleep(0.01)                # 예측 요청들이 먼저 시작하게
            await client.get("/health")
            return time.perf_counter() - 시작        # 처음부터 /health 답을 받기까지

        시작 = time.perf_counter()
        예측들 = [client.get("/predict") for _ in range(4)]
        *_, 상태_시각 = await asyncio.gather(*예측들, 상태_확인())
        끝 = time.perf_counter() - 시작
        print(f"{방식:18}: 예측 4개 끝 {끝:.2f}초, /health 답 {상태_시각:.2f}초")


무거운_추론()                                        # 준비 운동
for 방식 in ["async def에서 바로", "그냥 def", "run_in_threadpool"]:
    asyncio.run(시험하기(방식))
```

```
async def에서 바로    : 예측 4개 끝 0.37초, /health 답 0.37초
그냥 def            : 예측 4개 끝 0.34초, /health 답 0.02초
run_in_threadpool : 예측 4개 끝 0.35초, /health 답 0.03초
```

- **async def에서 바로**: `/health`의 답이 예측 네 개가 모두 끝난 뒤에야 왔습니다. 계산하는 동안 이벤트 루프가 멈춰서, "살아 있니?"라는 한 마디에도 대답하지 못한 거예요. 실제 서비스라면 감시 프로그램이 이 서버를 "죽었다"고 판단해 재시작해 버릴 수도 있습니다.
- **그냥 def**: FastAPI가 경로 함수를 작업 스레드에서 돌리니, 이벤트 루프는 자유롭게 `/health`에 바로 답했어요.

- **run_in_threadpool**: `async def` 안에서 다른 비동기 작업(`await`)을 써야 하는데 무거운 계산도 해야 할 때는, 계산 부분만 `await run_in_threadpool(함수)`로 작업 스레드에 넘깁니다. 1권 15장의 `asyncio.to_thread`와 같은 일을 FastAPI가 쓰는 스레드 묶음에서 해 주는 함수예요. 20.5절의 동적 배칭도 이 방법으로 모델을 불렀습니다.

규칙으로 정리하면 이렇습니다. **모델 추론을 하는 경로 함수는 그냥 `def`로 쓴다. `async def`가 꼭 필요하면 추론만 `run_in_threadpool`로 넘긴다.** 20.4절에서 파일을 `await file.read()` 대신 `file.file.read()`로 읽고 `def`로 쓴 것도 이 때문이었어요.

한 가지 더 보세요. 세 방식 모두 예측 네 개가 끝나는 데는 비슷하게 0.3초 남짓 걸렸습니다. 스레드로 넘겼다고 계산이 빨라지지는 않아요. PyTorch는 큰 계산 하나에 이미 CPU 코어를 여럿 쓰고 있어서, 스레드를 늘려도 코어를 나눠 쓸 뿐입니다. 스레드의 역할은 계산을 빠르게 하는 것이 아니라 **계산하는 동안에도 서버가 다른 손님에게 대답할 수 있게** 하는 거예요.

> **Q. PyTorch는 GIL 때문에 스레드에서 느려지지 않나요?** 1권 15장에서 파이썬 코드는 GIL 때문에 한 번에 한 스레드만 실행된다고 했죠. 그런데 PyTorch와 ONNX Runtime의 무거운 계산은 C++ 코드 안에서 일어나고, 그동안은 GIL을 내려놓습니다. 그래서 한 스레드가 계산하는 동안 다른 스레드가 요청을 해석하고 응답을 만들 수 있어요. 위 실험에서 `/health`가 바로 답한 것이 그 증거입니다.

20.7 살아 있나, 준비됐나, 무슨 모델인가

헬스 체크와 버전 정보

띄워 둔 서버가 잘 도는지는 대개 Docker나 클라우드의 감시 프로그램이 몇 초마다 물어서 확인합니다. 그래서 서빙 서버에는 이런 주소를 둬요.

- **`/health`**: "살아 있니?" 프로세스가 응답할 수 있으면 200. 응답이 없으면 감시 프로그램이 서버를 재시작합니다.
- **`/ready`**: "손님 받을 준비 됐니?" 모델까지 올라와야 200, 아니면 `503 Service Unavailable`. 준비가 안 된 서버에는 요청을 보내지 않습니다.
- **`/version`**: "지금 무슨 모델로 답하고 있니?" 앱 버전, 모델 파일과 그 **지문**(해시), 라이브러리 버전을 알려 줍니다.

마지막이 의외로 중요합니다. "어제부터 답이 이상하다"는 연락을 받았을 때, 지금 서버에 올라간 모델이 정확히 어떤 파일인지 모르면 원인을 찾을 수가 없어요. 18장에서 데이터와 모델 파일을 해시로

기록한 것처럼, 서버도 자기가 올린 모델 파일의 sha256 해시를 알려 주게 하겠습니다. 덤으로 모든 응답 헤더에 처리 시간과 모델 지문을 붙이는 **미들웨어**도 달아요. 미들웨어는 모든 요청이 경로 함수에 가기 전과 후에 거치는 관문입니다.

```
import hashlib
import time
from contextlib import asynccontextmanager
from pathlib import Path

import torch
from fastapi import FastAPI, Request, Response
from fastapi.testclient import TestClient

from 서빙도구 import 모델_불러오기

앱_버전 = "1.0.0"
모델_경로 = Path("fashion_cnn.pt")


def 파일_지문(경로: Path) -> str:
    """파일 내용의 sha256 해시 앞 12글자: 내용이 한 바이트만 달라도 바뀐다."""
    return hashlib.sha256(경로.read_bytes()).hexdigest()[:12]


@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.모델 = 모델_불러오기(모델_경로)
    app.state.모델_지문 = 파일_지문(모델_경로)
    yield


app = FastAPI(lifespan=lifespan, version=앱_버전)


@app.middleware("http")
async def 처리_시간_기록(request: Request, call_next):
    시작 = time.perf_counter()
```

```
    응답 = await call_next(request)                    # 실제 경로 함수 실행
    응답.headers["X-Process-Time-ms"] = f"{(time.perf_counter() - 시작) * 1000:.2f}"
    응답.headers["X-Model-Version"] = getattr(request.app.state, "모델_지문", "none")
    return 응답


@app.get("/health")
def 살아있나() -> dict:
    return {"status": "ok"}                            # 프로세스가 응답할 수 있는가


@app.get("/ready")
def 준비됐나(request: Request, response: Response) -> dict:
    if not hasattr(request.app.state, "모델"):         # 모델이 올라왔는가
        response.status_code = 503
        return {"ready": False}
    return {"ready": True}


@app.get("/version")
def 버전(request: Request) -> dict:
    return {"app": 앱_버전, "model_file": 모델_경로.name,
            "model_sha256": request.app.state.모델_지문, "torch": torch.__version__}


아직 = TestClient(app)                                 # lifespan 전: 모델이 아직 없다
print("시작 전:", 아직.get("/health").json(), 아직.get("/ready").status_code)
with TestClient(app) as client:
    print("시작 후:", client.get("/health").json(), client.get("/ready").status_code)
    응답 = client.get("/version")
    print(응답.json())
    print({k: v for k, v in 응답.headers.items() if k.startswith("x-")})
```

```
시작 전: {'status': 'ok'} 503
시작 후: {'status': 'ok'} 200
{'app': '1.0.0', 'model_file': 'fashion_cnn.pt', 'model_sha256': 'fa2fe8ec0e5a', 'torch': '2.14.1'}
```

```
{'x-process-time-ms': '0.58', 'x-model-version': 'fa2fe8ec0e5a'}
```

- **시작 전**: 20.2절의 함정을 일부러 다시 썼습니다. lifespan이 돌기 전에는 `/health`는 200이지만 `/ready`는 503이에요. 프로세스는 살아 있지만 모델이 아직 없다는 뜻입니다. 큰 모델을 불러오는 데 몇십 초씩 걸리는 서버라면 이 구분이 꼭 필요해요.
- **시작 후**: 둘 다 200입니다.
- `/version`: 모델 파일의 sha256 앞 12글자 `fa2fe8ec0e5a`가 찍혔습니다. 파일 내용이 한 바이트만 달라도 이 값이 완전히 바뀌니, 두 서버가 같은 모델을 쓰는지 이 값만 비교하면 됩니다. 9장을 다른 환경에서 학습했다면 여러분의 값은 다를 거예요.
- **헤더**: `@app.middleware("http")`로 만든 `처리_시간_기록`이 `call_next(request)`로 경로 함수를 실행하기 전후의 시간을 재서 `X-Process-Time-ms` 헤더에 적고, `X-Model-Version`도 붙였습니다. 본문을 바꾸지 않고도 모든 응답에 정보를 실을 수 있어요. 이름 앞의 `X-`는 표준이 아닌, 우리가 정한 헤더라는 관례입니다.

20.8 ONNX Runtime으로 백엔드 바꾸기

서버 안에서 PyTorch 빼기

19장에서 ONNX 파일로 내보낸 모델은 PyTorch 없이 ONNX Runtime만으로 같은 답을 냈습니다. 서버에 쓰면 좋은 점이 셋이에요. 같은 CPU에서 더 빠르고(19장), 설치할 것이 훨씬 작고(20.11절의 Docker 이미지), pickle이 아니라서 열 때 코드가 실행될 걱정이 없습니다.

내보내는 방법은 19장 19.2절과 19.4절 그대로입니다. 서빙에 맞게 두 가지만 정했어요. 묶음 크기(`batch`)를 열어 둬서 20.5절의 배치 추론이 되게 하되 `min=1, max=1024`로 범위를 적어 두었고(서버가 받는 장수 상한보다 넉넉하게 잡은 값입니다), 입출력 이름은 다른 언어로 짠 프로그램도 이 파일을 그대로 열 수 있게 19장의 `사진`·`점수` 대신 영어 `image`와 `logits`로 붙였습니다.

```
from pathlib import Path

import numpy as np
import onnxruntime as ort
import torch

from 서빙도구 import 모델_불러오기
```

```
모델 = 모델_불러오기("fashion_cnn.pt")
배치 = torch.export.Dim("batch", min=1, max=1024)          # 첫 차원(장 수)은 바뀔 수 있다
torch.onnx.export(모델, (torch.zeros(1, 1, 28, 28),), "fashion_cnn.onnx", dynamo=True,
                  input_names=["image"], output_names=["logits"],
                  dynamic_shapes={"x": {0: 배치}},
                  external_data=False, verbose=False)   # 가중치까지 한 파일에, 진행 메시지 끄기
for 파일 in sorted(Path(".").glob("fashion_cnn.*")):
    print(f"{파일.name:17} {파일.stat().st_size / 1024:7.1f} KB")

세션 = ort.InferenceSession("fashion_cnn.onnx", providers=["CPUExecutionProvider"])
print("입력:", [(i.name, i.shape) for i in 세션.get_inputs()])
print("출력:", [(o.name, o.shape) for o in 세션.get_outputs()])

torch.manual_seed(0)
for 장수 in [1, 64]:
    x = torch.rand(장수, 1, 28, 28)
    with torch.no_grad():
        파이토치 = 모델(x).numpy()
    런타임 = 세션.run(["logits"], {"image": x.numpy()})[0]
    같은_답 = (파이토치.argmax(1) == 런타임.argmax(1)).all()
    print(f"{장수:2}장: 최대 오차 {np.abs(파이토치 - 런타임).max():.2e}, 예측 모두 같음 {같은_답}")
```

```
fashion_cnn.onnx   1662.5 KB
fashion_cnn.pt     1650.8 KB
입력: [('image', ['batch', 1, 28, 28])]
출력: [('logits', ['batch', 10])]
 1장: 최대 오차 5.13e-06, 예측 모두 같음 True
64장: 최대 오차 4.53e-06, 예측 모두 같음 True
```

`external_data=False`로 가중치까지 파일 하나에 담았더니 `.pt`와 거의 같은 1.6MB입니다. 입력과 출력의 첫 차원이 `batch`라는 이름으로 열려 있고, 1장과 64장 모두 PyTorch와의 차이가 100만분의 5 정도라서 예측은 하나도 다르지 않아요. 이 정도 차이가 왜 생기는지는 19장 19.3절에서 설명했습니다.

같은 약속, 다른 구현

이제 서버가 PyTorch 모델과 ONNX 모델 중 무엇이든 쓸 수 있게 하고 싶습니다. 핵심은 경로 함수가 "어떤 엔진으로 계산하는지"를 몰라도 되게 하는 거예요. 그러려면 두 엔진 사이에 **약속**을 하나 정합니다. "(N, 1, 28, 28) NumPy 배열을 받아 (N, 10) 점수 배열을 돌려주는 `점수` 메서드가 있다." 경로 함수는 이 약속만 믿고 부르고, 어느 엔진을 쓸지는 코드가 아니라 환경 변수로 정해요. 1권 19.7절에서 비밀 값을 코드 밖에 둔 것과 같은 생각입니다. 이 설계는 20.11절 실습의 `backends.py`에서 코드로 만나요.

여기서 짚어 둘 것은 약속의 재료가 텐서가 아니라 **NumPy 배열**이라는 점입니다. 20.4절에서 전처리를 NumPy로 바꿔 둔 이유가 이거예요. 전처리나 약속에 torch가 끼어 있으면, ONNX 엔진을 고르더라도 서버에 PyTorch를 깔아야 하거든요.

속도는 19장 19.7절에서 잰 것처럼 이 CNN에서 ONNX Runtime이 PyTorch보다 두 배쯤 빨랐습니다. 그런데 20.7절의 처리 시간 헤더를 다시 보면 요청 하나가 0.58ms였어요. 모델 계산이 두 배 빨라지면 서버도 두 배 빨라질까요? 다음 절에서 재 봅니다.

20.9 지연 시간 재기: 평균에 속지 않기

서빙의 성적표는 정확도만이 아닙니다. 손님이 느끼는 것은 **지연 시간**(latency), 즉 요청을 보내고 답을 받기까지 걸린 시간이에요. 그런데 지연 시간을 평균 하나로 보고하면 위험합니다. 100명 중 99명은 1ms 만에 답을 받고 한 명은 1초를 기다렸다면, 평균은 11ms쯤으로 괜찮아 보이지만 그 한 명은 앱이 멈췄다고 느낄 테니까요.

그래서 서빙에서는 **백분위수**로 봅니다. p50(중앙값)은 "절반의 요청이 이보다 빨랐다", p95는 "95%가 이보다 빨랐다", p99는 "99%가 이보다 빨랐다"는 뜻이에요. p99가 느린 서비스는 하루 100만 번 요청을 받으면 그중 1만 번을 오래 기다리게 합니다. 사진 500장을 차례로 보내 보겠습니다. 서버 안에서 전처리와 모델 계산에 걸린 시간은 `Server-Timing` 이라는 표준 헤더에 담아 돌려받았어요. 브라우저 개발자 도구도 이 헤더를 읽어 그래프로 보여 줍니다.

```
import io
import statistics
import time
from contextlib import asynccontextmanager


import numpy as np
```

```
import onnxruntime as ort
from fastapi import FastAPI, File, Request, Response, UploadFile
from fastapi.testclient import TestClient
from PIL import Image

from 서빙도구 import 시험_사진_파일


@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.세션 = ort.InferenceSession("fashion_cnn.onnx", providers=["CPUExecutionProvider"])
    yield


app = FastAPI(lifespan=lifespan)


@app.post("/predict/image")
def 사진_예측(request: Request, response: Response, file: UploadFile = File(...)) -> dict:
    t0 = time.perf_counter()
    사진 = Image.open(io.BytesIO(file.file.read())).convert("L").resize((28, 28))
    x = np.asarray(사진, dtype=np.float32)[None, None] / 255
    t1 = time.perf_counter()
    점수 = request.app.state.세션.run(["logits"], {"image": x})[0]
    t2 = time.perf_counter()
    # 서버 안에서 단계별로 걸린 시간을 표준 헤더(Server-Timing)로 알려 준다
    전처리_ms, 모델_ms = (t1 - t0) * 1000, (t2 - t1) * 1000
    response.headers["Server-Timing"] = f"pre;dur={전처리_ms:.3f}, model;dur={모델_ms:.3f}"
    return {"class_id": int(점수.argmax())}


전체, 전처리, 모델 = [], [], []
with TestClient(app) as client:
    for 내용, _ in 시험_사진_파일(500):
        시작 = time.perf_counter()
        응답 = client.post("/predict/image", files={"file": ("x.png", 내용)})
```

```
        전체.append((time.perf_counter() - 시작) * 1000)
        단계 = dict(p.split(";dur=") for p in 응답.headers["server-timing"].split(", "))
        전처리.append(float(단계["pre"]))
        모델.append(float(단계["model"]))

print(f"첫 요청: {전체[0]:.2f}ms")
나머지 = 전체[1:]
백분위 = statistics.quantiles(나머지, n=100)                # 1~99번째 백분위수
print(f"나머지 {len(나머지)}개: 평균 {statistics.mean(나머지):.2f}ms, p50 {백분위[49]:.2f}ms, "
      f"p95 {백분위[94]:.2f}ms, p99 {백분위[98]:.2f}ms, 최대 {max(나머지):.2f}ms")
print(f"그중 서버 안 (중앙값): 전처리 {statistics.median(전처리):.3f}ms, "
      f"모델 {statistics.median(모델):.3f}ms")
```

```
첫 요청: 3.79ms
나머지 499개: 평균 1.23ms, p50 1.13ms, p95 1.85ms, p99 2.15ms, 최대 2.60ms
그중 서버 안 (중앙값): 전처리 0.053ms, 모델 0.073ms
```

이 책의 실행 환경에서 잰 값이라 실행할 때마다 조금씩 다르지만, 모양은 늘 비슷합니다.

- **첫 요청이 가장 느립니다.** 처음 쓰는 코드 경로를 불러오고, 메모리를 준비하는 일이 첫 요청에 몰려요. 서버를 띄운 직후 첫 손님이 느린 답을 받지 않게, lifespan에서 가짜 입력으로 모델을 한 번 돌려 두는 **준비 운동**(4장 4.8절)을 하기도 합니다.
- **평균은 중앙값보다 큽니다.** 가끔 느린 요청이 평균을 끌어올리기 때문이에요. p99와 최대값은 중앙값의 두 배쯤까지 벌어졌어요. 그 사이 운영체제가 다른 일을 했거나, 파이썬이 메모리를 정리했거나 하는 이유로 생기는 꼬리예요.
- **모델은 아주 작은 부분입니다.** 서버 안에서 전처리와 모델 계산을 합쳐도 0.13ms쯤인데, 요청 하나 전체는 1ms가 넘었어요. 나머지는 HTTP 요청을 만들고 해석하고, multipart 본문에서 파일을 꺼내고, 응답을 JSON으로 만드는 데 쓰였습니다. 이렇게 작은 모델에서는 모델을 더 빠르게 만들기보다 **요청 횟수를 줄이는 것**(20.5절의 배치)이 훨씬 효과적이에요. 반대로 14장의 언어 모델처럼 계산이 수백 ms씩 걸리는 모델이라면 모델 쪽을 줄여야 하고요. 어디를 고칠지는 이렇게 **재 보고** 정합니다.

`TestClient`는 네트워크를 거치지 않으니, 여기서 잰 것은 서버 프로그램 자체의 시간입니다. 진짜 네트워크를 거친 시간과 여러 손님이 동시에 몰릴 때의 모습은 20.11절에서 실제 서버를 띄워 재 봅니다.

20.10 LLM의 답을 흘려보내기: 스트리밍

만드는 대로 보내기

옷 분류기는 1ms 만에 답이 나오지만, 14장의 언어 모델은 토큰을 하나씩 만들어서 답 하나에 몇 초가 걸립니다. 다 만든 뒤 한꺼번에 보내면 사용자는 그동안 빈 화면만 봐요. 챗봇 서비스들이 글자가 타자 치듯 나타나게 하는 이유가 여기 있습니다. 전체 시간은 같아도 첫 글자가 빨리 보이면 기다림이 훨씬 짧게 느껴지거든요.

FastAPI의 `StreamingResponse` 는 1권 15장의 제너레이터를 받아서, 값이 하나 `yield`될 때마다 바로 보냅니다. 받는 쪽은 `client.stream(...)`으로 요청을 열고 `iter_text()`로 조각이 오는 대로 꺼내요. 일반 함수(`def`)로 만든 제너레이터라면 `StreamingResponse`가 20.6절의 작업 스레드에서 돌려주니, 안에서 오래 걸리는 계산을 해도 이벤트 루프를 막지 않습니다.

14장 언어 모델의 답 흘려보내기

문제는 14장의 `generate`가 답을 **끝까지 다 만들어야** 돌아오는 함수라는 점이에요. `transformers`는 이를 위해 `TextIteratorStreamer` 를 줍니다. `generate`에 스트리머를 넘기면 토큰이 나올 때마다 글자로 바꿔 스트리머에 넣어 주고, 우리는 스트리머를 반복문으로 꺼내 읽어요. 단, `generate`는 끝날 때까지 돌아오지 않으니 **다른 스레드**(1권 17장의 `threading.Thread`)에서 돌리고, 지금 스레드에서는 스트리머를 읽습니다. 요리사가 요리하는 동안 서빙 직원이 나온 접시부터 나르는 셈이에요. 모델은 14장과 같은 Qwen2.5-0.5B-Instruct(Apache-2.0)이고, 14장처럼 CPU에서 탐욕 디코딩으로 돌렸습니다.

```
import threading
import time
from collections.abc import Iterator
from contextlib import asynccontextmanager

import torch
from fastapi import FastAPI, Request
from fastapi.responses import StreamingResponse
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field
from transformers import AutoModelForCausalLM, AutoTokenizer, TextIteratorStreamer
```

```
모델_이름 = "Qwen/Qwen2.5-0.5B-Instruct"                    # 14장의 모델


@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.토크나이저 = AutoTokenizer.from_pretrained(모델_이름)
    app.state.모델 = AutoModelForCausalLM.from_pretrained(모델_이름, dtype=torch.float32).eval()
    app.state.생성_자물쇠 = threading.Lock()                    # 한 번에 한 요청만 생성
    yield


app = FastAPI(lifespan=lifespan)


class ChatRequest(BaseModel):
    question: str = Field(min_length=1, max_length=500)
    max_new_tokens: int = Field(default=60, ge=1, le=256)


def 토큰_흘리기(app: FastAPI, 요청: ChatRequest) -> Iterator[str]:
    토크나이저, 모델 = app.state.토크나이저, app.state.모델
    대화 = [{"role": "user", "content": 요청.question}]
    입력 = 토크나이저.apply_chat_template(대화, add_generation_prompt=True,
                                          return_tensors="pt", return_dict=True)
    스트리머 = TextIteratorStreamer(토크나이저, skip_prompt=True, skip_special_tokens=True)
    with app.state.생성_자물쇠:
        # generate는 끝까지 돌아야 돌아오므로 다른 스레드에서 돌리고, 나온 글자는 스트리머로 받는다
        일꾼 = threading.Thread(target=모델.generate, kwargs=dict(
            **입력, streamer=스트리머, max_new_tokens=요청.max_new_tokens, do_sample=False))
        일꾼.start()
        yield from 스트리머                                    # 조각이 나오는 대로 내보내기
        일꾼.join()


@app.post("/chat")
def 대화(요청: ChatRequest, request: Request) -> StreamingResponse:
```

```
    return StreamingResponse(토큰_흘리기(request.app, 요청), media_type="text/plain;
    ↪ charset=utf-8")


with TestClient(app) as client:
    시작 = time.perf_counter()
    조각들, 첫_조각 = [], None
    질문 = {"question": "파이썬의 리스트를 한 문장으로 설명해 줘.", "max_new_tokens": 60}
    with client.stream("POST", "/chat", json=질문) as 응답:
        for 조각 in 응답.iter_text():
            if 첫_조각 is None:
                첫_조각 = time.perf_counter() - 시작
            조각들.append(조각)
    전체 = time.perf_counter() - 시작
    print(f"첫 조각까지 {첫_조각:.2f}초, 전체 {전체:.2f}초, 조각 {len(조각들)}개")
    print("처음 다섯 조각:", 조각들[:5])
    print("".join(조각들))
    print(client.post("/chat", json={"question": ""}).status_code)
```

```
첫 조각까지 0.18초, 전체 1.23초, 조각 24개
처음 다섯 조각: ['파이썬의 ', '리스트는 ', 'Python에서 ', '사용하는 ', '데이터 ']
파이썬의 리스트는 Python에서 사용하는 데이터 타입 중 하나로, 다양한 프로그래밍 패턴과 기능을 제공합
↪ 니다. 리스트는 원소가 변경될 수 있는 자료형이며, 이 원소들은 순서대로 저장됩니다.

리스트의
422
```

첫 조각이 0.2초쯤에 도착했고, 60토큰짜리 답 전체는 1초 남짓 걸렸습니다(이 책의 실행 환경, CPU). 스트리밍이 없었다면 사용자는 1초를 빈 화면으로 기다렸을 거예요. 답이 길어 몇십 초씩 걸리는 경우라면 차이가 훨씬 크게 느껴집니다. 조각은 토큰 하나가 아니라 대개 **단어 하나**씩 왔어요. 스트리머가 토큰을 모아 두었다가 띄어쓰기가 나올 때 내보내기 때문입니다. 한국어 한 글자가 여러 토큰으로 쪼개지는 경우(14장 14.1절)에 깨진 글자를 보내지 않으려는 배려예요.

답은 "한 문장으로"라는 요청을 무시하고 길게 이어지다가 60토큰에서 잘렸습니다. `max_new_tokens`에 걸린 것이고, 작은 모델이 지시를 잘 따르지 못하는 것은 14장 14.5절에서 본 그대로예요. 빈 질문은 `ChatRequest`의 `min_length=1`에 걸려 422가 됐습니다.

코드에서 눈여겨볼 곳이 두 군데 있습니다.

- **생성_자물쇠**: `threading.Lock`은 한 번에 한 스레드만 들어갈 수 있는 자물쇠입니다. 큰 모델을 여러 스레드가 동시에 돌리면 메모리가 늘고 코어를 빼앗아 모두 느려지니, 생성은 한 번에 하나만 하고 뒤에 온 요청은 기다리게 했어요. 여러 요청을 한꺼번에 생성하는 일은 20.5절의 동적 배칭을 언어 모델에 맞게 확장한 전용 서빙 도구들의 몫입니다.
- **`yield from` 스트리머**: 1권 15장의 `yield from`으로, 스트리머가 내놓는 조각을 그대로 바깥으로 넘깁니다. 다 넘긴 뒤 `join()`으로 생성 스레드가 끝나기를 기다려요.

20.11 실습: 옷 사진 분류 서비스 완성하기

지금까지 만든 조각을 모아 9장 옷 분류기를 **한 벌의 서비스**로 완성하겠습니다. 역할별로 파일을 나누고(1권 14장), pytest로 시험하고, 실제 서버에 부하를 걸어 보고, Docker 이미지로 감싸는 데까지 갑니다.

프로젝트 구조

```
fashion_service/
├── pyproject.toml        # 의존성 (uv)
├── uv.lock               # 정확한 버전 목록 (uv lock이 만든다)
├── .python-version       # 3.12
├── Dockerfile
├── .dockerignore
├── app/
│   ├── __init__.py       # 빈 파일 (1권 14장의 패키지 표시)
│   ├── fashion_model.py  # 9장 옷CNN 설계도와 불러오기 (torch 백엔드용)
│   ├── preprocess.py     # 사진 → (1, 28, 28) 배열
│   ├── backends.py       # ONNX / PyTorch 예측기
│   └── main.py           # FastAPI 앱
├── models/
│   ├── fashion_cnn.pt    # 9장 9.5절에서 저장한 파일
│   └── fashion_cnn.onnx  # scripts/export_onnx.py가 만든다
├── scripts/
│   ├── export_onnx.py
│   └── load_test.py
```

```
└── tests/
    ├── data/              # 시험 사진 두 장: ankle_boot.png, pullover.png
    └── test_main.py
```

`tests/data/`의 두 사진은 9장에서 쓴 FashionMNIST 시험 데이터의 0번(앵클부츠)과 1번(풀오버) 사진을 PNG로 저장한 것입니다. 9장처럼 `시험 = datasets.FashionMNIST("data", train=False)`로 불러온 뒤 `시험[0][0].save("tests/data/ankle_boot.png")`, `시험[1][0].save("tests/data/pullover.png")`를 실행하면 똑같은 파일이 만들어지고, 예제 코드 꾸러미에도 들어 있어요. `models/fashion_cnn.pt`는 9장 9.5절에서 저장한 파일을 복사해 넣으면 됩니다.

명령줄에 `app.main:app`처럼 모듈 경로를 적고 Docker로 옮길 폴더라서 파일과 폴더 이름은 영어로, 파일 **안의** 이름은 지금까지처럼 한글로 지었습니다.

`app/fashion_model.py`에는 9장의 `옷CNN` 클래스와 20.1절의 `모델_불러오기`가 들어 있습니다. 이번에는 9장 `옷도구.py`에서 `import`하지 않고 클래스를 **직접 옮겨 담았어요**. 서비스 폴더는 이 폴더만 복사해 가도 돌아가야 하니까요. 내용이 9장과 같으니 여기 다시 싣지는 않을게요.

전처리와 백엔드

전처리는 20.4절에서 확인한 두 가지 교훈, "학습 때와 같은 방법으로"와 "흰 배경이면 뒤집기"를 모두 담았습니다. torch 없이 NumPy와 Pillow만 써요.

```
"""올라온 사진 파일을 모델 입력 배열로 바꾼다 (torch 없이 NumPy와 Pillow만 쓴다)."""
import io

import numpy as np
from PIL import Image, UnidentifiedImageError

최대_바이트 = 1_000_000                # 파일 크기 한도: 1MB
최대_픽셀 = 4096 * 4096                # 가로 × 세로 한도: 압축 폭탄 막기


class 사진오류(ValueError):
    """사진으로 읽을 수 없거나 받을 수 없는 파일."""


def 사진_전처리(데이터: bytes) -> np.ndarray:
```

```
    """사진 파일을 학습 때와 같은 (1, 28, 28) float32 배열로 바꾼다."""
    try:
        사진 = Image.open(io.BytesIO(데이터))
        if 사진.width * 사진.height > 최대_픽셀:
            raise 사진오류("사진의 가로·세로가 너무 큽니다")
        사진 = 사진.convert("L").resize((28, 28))
    except Image.DecompressionBombError as e:      # 아주 큰 그림은 Image.open이 먼저 알아채고 멈춘다
        raise 사진오류("사진의 가로·세로가 너무 큽니다") from e
    except (UnidentifiedImageError, OSError) as e:
        raise 사진오류("사진 파일이 아닙니다") from e
    배열 = np.asarray(사진, dtype=np.float32) / 255
    테두리 = np.concatenate([배열[0], 배열[-1], 배열[:, 0], 배열[:, -1]])
    if 테두리.mean() > 0.5:              # 배경이 밝으면 뒤집어서 학습 데이터처럼 검은 배경으로
        배열 = 1 - 배열
    return 배열[None]
```

20.4절에서 하지 않던 검사가 하나 늘었습니다. PNG 같은 형식은 압축이 잘되는 그림을 아주 작은 파일에 담을 수 있어서, 1MB 한도를 통과한 파일이 열어 보니 수만 × 수만 픽셀이라 메모리를 다 먹어 버리는 공격(압축 폭탄)이 가능해요. `Image.open`은 파일 머리말만 읽고 실제 픽셀은 아직 풀지 않으니, 그 사이에 가로 × 세로를 확인해 막았습니다. 가로 × 세로가 터무니없이 크면(약 1억 8천만 픽셀이 넘으면) Pillow가 `Image.open`에서 먼저 `DecompressionBombError`를 내며 멈추는데, 이 오류는 `OSError`가 아니라서 따로 잡아 같은 `사진오류`로 바꿨어요. 잡지 않으면 500이 나갑니다. 전처리에서 난 문제는 HTTP를 모르는 `사진오류`로 던지고, 상태 코드로 바꾸는 일은 `main.py`에 맡겼어요. 전처리 함수를 HTTP 없이도 쓰고 시험할 수 있게요.

```
"""모델 계산을 맡는 백엔드: 같은 약속(점수)을 지키는 두 가지 구현."""
import os
from pathlib import Path
from typing import Protocol

import numpy as np


class 예측기(Protocol):
    """(N, 1, 28, 28) float32 배열을 받아 (N, 10) 점수를 돌려준다."""
    이름: str
```

```
    모델_파일: Path

    def 점수(self, x: np.ndarray) -> np.ndarray: ...


class ONNX예측기:
    이름 = "onnx"

    def __init__(self, 폴더: Path) -> None:
        import onnxruntime as ort

        self.모델_파일 = 폴더 / "fashion_cnn.onnx"
        옵션 = ort.SessionOptions()
        옵션.intra_op_num_threads = int(os.environ.get("ORT_THREADS", "0"))   # 0이면 코어 수만큼
        self.세션 = ort.InferenceSession(self.모델_파일, 옵션, providers=["CPUExecutionProvider"])

    def 점수(self, x: np.ndarray) -> np.ndarray:
        return self.세션.run(["logits"], {"image": x})[0]


class 파이토치예측기:
    이름 = "torch"

    def __init__(self, 폴더: Path) -> None:
        import torch                            # torch 백엔드를 고를 때만 필요하다

        from app.fashion_model import 모델_불러오기

        self.모델_파일 = 폴더 / "fashion_cnn.pt"
        self.torch, self.모델 = torch, 모델_불러오기(self.모델_파일)

    def 점수(self, x: np.ndarray) -> np.ndarray:
        with self.torch.no_grad():
            return self.모델(self.torch.from_numpy(x)).numpy()
```

```
def 예측기_만들기(이름: str, 폴더: Path) -> 예측기:
    백엔드들 = {"onnx": ONNX예측기, "torch": 파이토치예측기}
    if 이름 not in 백엔드들:
        raise ValueError(f"알 수 없는 백엔드: {이름} (가능: {', '.join(백엔드들)})")
    return 백엔드들[이름](폴더)
```

20.8절에서 말한 약속을 코드로 옮긴 파일입니다.

- **예측기(Protocol)**: 1권 13장의 `Protocol`로 약속을 적었습니다. `이름`, `모델_파일`, 그리고 배열을 받아 배열을 돌려주는 **점수**. 두 백엔드 클래스는 **예측기**를 상속하지 않고 이 셋을 갖추기만 했어요. `Protocol`은 실행할 때 검사하는 것이 아니라 읽는 사람과 타입 검사 도구를 위한 약속이라, 뒤의 테스트에서는 경로 함수가 실제로 부르는 **점수**만 갖춘 가짜도 끼워 쓸 수 있습니다.
- **import를 클래스 안에서**: `onnxruntime`과 `torch`를 파일 맨 위가 아니라 각 클래스의 `__init__` 안에서 불러옵니다. 그 백엔드를 고를 때만 그 라이브러리가 필요하니, ONNX만 쓰는 서버는 PyTorch를 깔지 않아도 이 파일을 불러올 수 있어요.
- **예측기_만들기**: 이름으로 백엔드를 고릅니다. 모르는 이름이면 서버가 시작될 때 바로 오류를 내서, 설정 실수를 손님이 오기 전에 알게 해요.
- **ORT_THREADS**: ONNX Runtime이 계산에 쓸 스레드 수를 환경 변수로 정할 수 있게 했습니다. 기본값 0은 "알아서 코어 수만큼"이에요. 이 손잡이가 왜 필요한지는 조금 뒤 부하 시험에서 보여 드릴게요.

경로 함수

```
"""옷 사진 분류 서비스: 9장 옷CNN을 HTTP로 내놓는다."""
import hashlib
import logging
import os
import time
from contextlib import asynccontextmanager
from pathlib import Path

import numpy as np
from fastapi import Depends, FastAPI, File, HTTPException, Request, UploadFile
from pydantic import BaseModel
```

```
from app.backends import 예측기, 예측기_만들기
from app.preprocess import 사진_전처리, 사진오류, 최대_바이트

앱_버전 = "1.0.0"
최대_장수 = 32
클래스_이름 = ["T셔츠", "바지", "풀오버", "드레스", "코트",
          "샌들", "셔츠", "운동화", "가방", "앵클부츠"]
로그 = logging.getLogger("uvicorn.error")           # uvicorn이 찍는 로그에 함께 나오게


class ClassScore(BaseModel):
    label: str
    probability: float


class Prediction(BaseModel):
    file: str
    label: str
    class_id: int
    probability: float
    top3: list[ClassScore]


@asynccontextmanager
async def lifespan(app: FastAPI):
    백엔드 = os.environ.get("MODEL_BACKEND", "onnx")
    폴더 = Path(os.environ.get("MODEL_DIR", "models"))
    app.state.예측기 = 예측기_만들기(백엔드, 폴더)
    app.state.모델_지문 = hashlib.sha256(app.state.예측기.모델_파일.read_bytes()).hexdigest()[:12]
    로그.info("모델 준비 완료: backend=%s, sha256=%s", 백엔드, app.state.모델_지문)
    yield


app = FastAPI(title="옷 사진 분류 API", version=앱_버전, lifespan=lifespan)
```

```
@app.middleware("http")
async def 처리_시간_기록(request: Request, call_next):
    시작 = time.perf_counter()
    응답 = await call_next(request)
    응답.headers["X-Process-Time-ms"] = f"{(time.perf_counter() - 시작) * 1000:.2f}"
    return 응답


def 예측기_얻기(request: Request) -> 예측기:
    """경로 함수가 쓸 예측기. 테스트에서는 이 함수를 가짜로 바꿔 끼운다."""
    return request.app.state.예측기


def 파일_읽기(file: UploadFile) -> np.ndarray:
    데이터 = file.file.read(최대_바이트 + 1)
    if len(데이터) > 최대_바이트:
        raise HTTPException(status_code=413, detail=f"{file.filename}: 1MB보다 큽니다")
    try:
        return 사진_전처리(데이터)
    except 사진오류 as e:
        raise HTTPException(status_code=400, detail=f"{file.filename}: {e}")


def 결과_만들기(이름들: list[str], 점수: np.ndarray) -> list[Prediction]:
    확률 = np.exp(점수 - 점수.max(axis=1, keepdims=True))      # 소프트맥스 (6장)
    확률 /= 확률.sum(axis=1, keepdims=True)
    결과 = []
    for 이름, p in zip(이름들, 확률):
        상위 = [ClassScore(label=클래스_이름[i], probability=round(float(p[i]), 4))
                for i in p.argsort()[::-1][:3]]
        결과.append(Prediction(file=이름, label=상위[0].label, class_id=int(p.argmax()),
                               probability=상위[0].probability, top3=상위))
    return 결과
```

```
@app.get("/health")
def 살아있나() -> dict:
    return {"status": "ok"}


@app.get("/version")
def 버전(request: Request) -> dict:
    예측기_ = request.app.state.예측기
    return {"app": 앱_버전, "backend": 예측기_.이름, "model_file": 예측기_.모델_파일.name,
            "model_sha256": request.app.state.모델_지문}


@app.post("/predict/image")
def 사진_예측(file: UploadFile = File(...), 예측기_: 예측기 = Depends(예측기_얻기)) -> Prediction:
    x = 파일_읽기(file)[None]                                  # (1, 1, 28, 28)
    return 결과_만들기([file.filename or ""], 예측기_.점수(x))[0]


@app.post("/predict/batch")
def 여러_장_예측(files: list[UploadFile] = File(...),
                 예측기_: 예측기 = Depends(예측기_얻기)) -> list[Prediction]:
    if len(files) > 최대_장수:
        raise HTTPException(status_code=413, detail=f"한 번에 {최대_장수}장까지 보낼 수 있습니다")
    x = np.stack([파일_읽기(f) for f in files])                # (N, 1, 28, 28)
    return 결과_만들기([f.filename or "" for f in files], 예측기_.점수(x))
```

20.2절부터 20.7절까지의 조각이 모두 들어 있습니다. 새로 달라진 곳만 짚어 볼게요.

- **설정은 환경 변수로**: 백엔드(`MODEL_BACKEND`, 기본 onnx)와 모델 폴더(`MODEL_DIR`, 기본 `models`)를 코드 밖에서 정합니다. Docker로 띄울 때 이 값을 바꿔 줄 수 있어요.
- **로그**: 1권 19장에서는 확인용으로 `print`를 썼지만, 서버에서는 `logging`을 씁니다. `uvicorn.error`는 uvicorn이 자기 메시지를 찍는 로거의 이름이에요(이름과 달리 오류만 찍는 곳은 아닙니다). 여기에 쓰면 uvicorn의 `INFO:` 로그 사이에 함께 나옵니다.
- **`Depends(예측기_얻기)`**: 20.2~20.7절에서는 경로 함수가 `request.app.state`에서 모델을 직접 꺼냈는데, 이번에는 꺼내는 일을 의존성 함수(1권 19.4절)로 뺐습니다. 이렇게 해 두면 테스트에서 이 함수만 **가짜로 바꿔 끼울 수** 있어요. 바로 다음에 씁니다.
- **소프트맥스는 NumPy로**: 백엔드가 점수만 돌려주니 확률은 직접 계산했습니다. 가장 큰 점수

를 먼저 빼는 것은 6장 6.2절에서 `torch.softmax`가 안에서 쓴다고 했던, `exp`가 넘치지 않게 하는 요령 그대로예요.

- **배치 상한**: 32장을 넘으면 413, 파일마다 1MB가 넘으면 413, 사진이 아니면 400. 오류 메시지에 파일 이름을 넣어서 여러 장 중 어느 파일이 문제인지 알 수 있게 했습니다.

ONNX 파일 만들기

서비스는 ONNX 파일로 돌지만, 그 파일은 PyTorch가 있어야 만들 수 있습니다. 내보내기와 확인을 스크립트 하나로 묶어 두면, 모델을 다시 학습할 때마다 같은 방법으로 만들 수 있어요.

```
"""models/fashion_cnn.pt를 ONNX로 내보내고, 두 결과가 같은지 확인한다."""
from pathlib import Path

import numpy as np
import onnxruntime as ort
import torch

from app.fashion_model import 모델_불러오기

폴더 = Path("models")
모델 = 모델_불러오기(폴더 / "fashion_cnn.pt")
배치 = torch.export.Dim("batch", min=1, max=1024)
torch.onnx.export(모델, (torch.zeros(1, 1, 28, 28),), 폴더 / "fashion_cnn.onnx", dynamo=True,
                  input_names=["image"], output_names=["logits"], dynamic_shapes={"x": {0: 배치}},
                  external_data=False, verbose=False)

x = torch.rand(16, 1, 28, 28, generator=torch.Generator().manual_seed(0))
with torch.no_grad():
    기대값 = 모델(x).numpy()
세션 = ort.InferenceSession(폴더 / "fashion_cnn.onnx", providers=["CPUExecutionProvider"])
오차 = np.abs(세션.run(["logits"], {"image": x.numpy()})[0] - 기대값).max()
assert 오차 < 1e-4, f"PyTorch와 ONNX 결과가 다릅니다: {오차}"
print(f"내보내기 완료: {폴더 / 'fashion_cnn.onnx'} (최대 오차 {오차:.1e})")
```

내보낸 뒤 PyTorch와 ONNX Runtime에 같은 사진 16장을 넣어 보고, 차이가 0.0001을 넘으면 `assert`로 멈춥니다. 잘못 만든 파일이 서비스에 올라가는 것을 이 자리에서 막는 거예요. 프로젝트 폴더에서 이렇게 실행합니다. `--extra torch`는 `pyproject.toml`의 선택 의존성(바로 뒤에 봅니다)

까지 설치하라는 뜻이고, `-m scripts.export_onnx`로 실행해야 스크립트 안에서 `app` 패키지를 찾을 수 있어요.

```
uv run --extra torch python -m scripts.export_onnx
```

```
내보내기 완료: models/fashion_cnn.onnx (최대 오차 3.8e-06)
```

실행하면 이 줄 앞에 "torchvision이 설치되어 있지 않아 일부 연산 등록을 건너뛴다"는 경고가 몇 줄 나오는데, 우리 모델은 torchvision 연산을 쓰지 않으니 무시해도 됩니다.

테스트: 진짜 모델과 가짜 모델

이제 서비스가 약속대로 동작하는지 pytest로 확인하겠습니다. 1권 13장에서 `test_`로 시작하는 함수의 `assert`를 pytest가 모아 실행한다고 했죠. 여기서는 pytest의 **픽스처(fixture)** 를 하나 더 씁니다. `@pytest.fixture`를 붙인 함수를 테스트 함수의 매개변수 이름으로 적으면, pytest가 그 함수를 실행해 결과를 넣어 줘요. FastAPI의 `Depends`와 비슷한 생각입니다. 픽스처 안에서 `yield`를 쓰면 `yield` 뒤는 테스트가 끝난 뒤에 실행되니 정리 작업도 넣을 수 있고요.

테스트를 두 종류로 나눴습니다.

- **진짜 모델로**: 실제 ONNX 파일로 앵클부츠·풀오버 사진을 맞히는지, 흰 배경 사진도 맞히는지 확인합니다. 모델과 전처리가 함께 제대로 도는지 보는 테스트예요.
- **가짜 모델로**: 언제나 "드레스"라고 답하는 `가짜예측기`를 끼우고, 응답 모양과 오류 처리를 확인합니다. 모델 파일이 바뀌어도 결과가 흔들리지 않고, 모델이 무거워도 빨리 돌아요. `app.dependency_overrides[예측기_얻기] = 가짜예측기`가 "`예측기_얻기`를 의존하는 곳에는 가짜를 넣어라"라는 뜻입니다.

```
"""옷 사진 분류 API 테스트. 실행: uv run --extra torch pytest -q"""
import io
from collections.abc import Iterator
from pathlib import Path

import numpy as np
import pytest
from fastapi.testclient import TestClient
from PIL import Image, ImageOps
```

```
from app.main import app, 예측기_얻기

자료 = Path(__file__).parent / "data"


class 가짜예측기:
    """언제나 3번(드레스)이 가장 높은 점수를 내는 가짜. 모델 없이 API 모양만 시험한다."""
    이름 = "fake"

    def 점수(self, x: np.ndarray) -> np.ndarray:
        점수 = np.zeros((len(x), 10), dtype=np.float32)
        점수[:, 3] = 5.0
        return 점수


@pytest.fixture
def client() -> Iterator[TestClient]:
    with TestClient(app) as c:                      # with로 열어야 lifespan이 돈다
        yield c


@pytest.fixture
def fake_client(client: TestClient) -> Iterator[TestClient]:
    app.dependency_overrides[예측기_얻기] = 가짜예측기  # 진짜 대신 가짜를 끼운다
    yield client
    app.dependency_overrides.clear()                # 다른 테스트에 번지지 않게 되돌린다


def 사진_파일(이름: str) -> tuple[str, bytes]:
    return 이름, (자료 / 이름).read_bytes()


def test_health_and_version_after_startup(client: TestClient) -> None:
    assert client.get("/health").json() == {"status": "ok"}
    assert client.get("/version").json()["backend"] == "onnx"
```

```
def test_predict_image_with_real_model_returns_correct_label(client: TestClient) -> None:
    for 이름, 정답 in [("ankle_boot.png", "앵클부츠"), ("pullover.png", "풀오버")]:
        응답 = client.post("/predict/image", files={"file": 사진_파일(이름)})
        assert 응답.status_code == 200
        assert 응답.json()["label"] == 정답
        assert "x-process-time-ms" in 응답.headers


def test_predict_image_with_white_background_still_correct(client: TestClient) -> None:
    사진 = ImageOps.invert(Image.open(자료 / "ankle_boot.png"))     # 흰 배경으로 뒤집은 사진
    버퍼 = io.BytesIO()
    사진.save(버퍼, format="PNG")
    응답 = client.post("/predict/image", files={"file": ("white.png", 버퍼.getvalue())})
    assert 응답.json()["label"] == "앵클부츠"


def test_predict_image_with_fake_model_returns_response_shape(fake_client: TestClient) -> None:
    본문 = fake_client.post("/predict/image", files={"file": 사진_파일("pullover.png")}).json()
    assert 본문["label"] == "드레스" and 본문["class_id"] == 3
    assert len(본문["top3"]) == 3
    assert sum(항목["probability"] for 항목 in 본문["top3"]) <= 1.0


def test_predict_image_when_not_image_returns_400(fake_client: TestClient) -> None:
    응답 = fake_client.post("/predict/image", files={"file": ("memo.txt", b"hello")})
    assert 응답.status_code == 400
    assert "사진 파일이 아닙니다" in 응답.json()["detail"]


def test_predict_image_when_too_large_returns_413(fake_client: TestClient) -> None:
    응답 = fake_client.post("/predict/image", files={"file": ("big.png", bytes(1_000_001))})
    assert 응답.status_code == 413
```

```
def test_predict_batch_keeps_order_and_limits_count(fake_client: TestClient) -> None:
    두_장 = [("files", 사진_파일("ankle_boot.png")), ("files", 사진_파일("pullover.png"))]
    본문 = fake_client.post("/predict/batch", files=두_장).json()
    assert [항목["file"] for 항목 in 본문] == ["ankle_boot.png", "pullover.png"]
    서른세_장 = [("files", 사진_파일("pullover.png"))] * 33
    assert fake_client.post("/predict/batch", files=서른세_장).status_code == 413


def test_backends_torch_and_onnx_agree() -> None:
    pytest.importorskip("torch")                    # torch가 없는 환경에서는 건너뛴다
    from app.backends import 예측기_만들기
    from app.preprocess import 사진_전처리

    파일들 = ["ankle_boot.png", "pullover.png"]
    x = np.stack([사진_전처리((자료 / 이름).read_bytes()) for 이름 in 파일들])
    점수들 = [예측기_만들기(이름, Path("models")).점수(x) for 이름 in ["torch", "onnx"]]
    assert np.abs(점수들[0] - 점수들[1]).max() < 1e-4
```

테스트 이름은 "무엇을, 어떤 상황에서, 어떻게 되어야 하는가"가 드러나게 지어서, 실패하면 이름만 보고 어디가 깨졌는지 알 수 있게 했습니다. 마지막 테스트는 torch 백엔드와 ONNX 백엔드의 점수가 같은지 보는데, torch가 설치되지 않은 환경에서는 `pytest.importorskip`이 이 테스트를 실패가 아니라 **건너뜀**으로 처리합니다. 실행해 보겠습니다.

```
uv run --extra torch pytest -q
```

```
........                                                                 [100%]
8 passed in 1.05s
```

점 여덟 개가 테스트 여덟 개의 통과를 뜻합니다. 실패한 테스트가 있으면 점 대신 `F`가 찍히고, 아래에 어느 `assert`에서 무엇이 달랐는지 자세히 나와요.

서버로 띄우기

테스트를 통과했으니 진짜 서버를 띄워 보겠습니다. 터미널 하나에서 서버를 띄우고,

```
uv run uvicorn app.main:app --port 8000
```

다른 터미널에서 `curl`로 요청을 보냅니다. `curl`은 맥과 리눅스, 윈도우 10 이상에 기본으로 들어 있는 명령줄 HTTP 도구예요. `-F "file=@경로"`는 파일을 multipart로 올리라는 뜻입니다.

```
curl -s localhost:8000/version
curl -s -F "file=@tests/data/ankle_boot.png" localhost:8000/predict/image
curl -s -i -F "files=@tests/data/ankle_boot.png" -F "files=@tests/data/pullover.png"
↪ localhost:8000/predict/batch
```

```
{"app":"1.0.0","backend":"onnx","model_file":"fashion_cnn.onnx","model_sha256":"265e35c606cf"}
{"file":"ankle_boot.png","label":"앵클부츠","class_id":9,"probability":0.9973,"top3":[{"label":"앵
↪ 클부츠","probability":0.9973},{"label":"운동화","probability":0.0025},{"label":"샌
↪ 들","probability":0.0002}]}
HTTP/1.1 200 OK
date: Wed, 21 Oct 2026 07:01:33 GMT
server: uvicorn
content-length: 430
content-type: application/json
x-process-time-ms: 1.51

[{"file":"ankle_boot.png","label":"앵클부츠","class_id":9,"probability":0.9973,"top3":
↪ [{"label":"앵클부츠","probability":0.9973},{"label":"운동화","probability":0.0025},
↪ {"label":"샌들","probability":0.0002}]},{"file":"pullover.png","label":"풀오
↪ 버","class_id":2,"probability":0.9998,"top3":[{"label":"풀오버","probability":0.9998},
↪ {"label":"T셔츠","probability":0.0001},{"label":"셔츠","probability":0.0}]}]
```

`/version`의 모델 지문이 20.7절과 다르죠? 그때는 `.pt` 파일의 해시였고 지금은 `.onnx` 파일의 해시라서 그렇습니다. 같은 모델이라도 파일이 다르면 지문이 다르니, 지문과 함께 파일 이름도 알려 주는 것이 좋아요. 세 번째 요청은 `-i`로 응답 헤더까지 보이게 했는데, 미들웨어가 붙인 `x-process-time-ms`가 보입니다. 서버를 띄운 터미널에는 이런 로그가 찍혀요. 마지막 네 줄은 요청을 다 보낸 뒤 서버를 끌 때(`Ctrl+C`) 나온 것입니다.

```
INFO:     Started server process [58418]
INFO:     Waiting for application startup.
INFO:     모델 준비 완료: backend=onnx, sha256=265e35c606cf
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

```
INFO:     127.0.0.1:58890 - "GET /version HTTP/1.1" 200 OK
INFO:     127.0.0.1:58892 - "POST /predict/image HTTP/1.1" 200 OK
INFO:     127.0.0.1:58894 - "POST /predict/batch HTTP/1.1" 200 OK
INFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [58418]
```

`Waiting for application startup`과 `Application startup complete` 사이에 우리 lifespan의 로그가 찍혔습니다. uvicorn은 lifespan의 `yield` 앞부분이 끝나야 손님을 받기 시작해요. 마지막 `Uvicorn running on` 줄도 lifespan 뒤에 찍혔죠? 모델이 다 올라온 뒤에야 포트를 열어 손님을 받는다는 뜻이라, 20.7절의 `/ready`가 503을 내는 순간이 uvicorn에서는 생기지 않습니다. 그 대신 모델이 커서 시작이 오래 걸리면 그동안은 포트가 아예 닫혀 있으니, 감시 프로그램이 '응답 없음'으로 오해하지 않게 시작에 걸리는 시간을 알려 줘야 해요(Docker의 `HEALTHCHECK`라면 `--start-period` 옵션). 끌 때는 `Waiting for application shutdown` 줄에서 lifespan의 `yield` 뒷부분이 실행돼요.

부하를 걸어 보기

손님이 한 명씩 올 때와 여럿이 몰릴 때는 어떻게 다를까요? 서버에 동시에 요청을 보내는 작은 스크립트를 만들었습니다. 20.5절의 `asyncio.gather`로 "손님" 코루틴 여러 개를 동시에 돌리고, 손님마다 번호표가 떨어질 때까지 요청을 계속 보내요. `TestClient`가 쓰는 `httpx2`의 비동기 클라이언트로 진짜 네트워크 요청을 보냅니다.

```
"""서버에 동시에 요청을 보내 처리량과 지연 시간을 잰다. 인자: 주소 [동시 손님 수] [총 요청 수]"""
import asyncio
import statistics
import sys
import time
from pathlib import Path

import httpx2


async def 부하_주기(주소: str, 동시: int, 총: int) -> None:
    사진 = Path("tests/data/ankle_boot.png").read_bytes()
    걸린_시간: list[float] = []
```

```
    남은_번호 = iter(range(총))

    async def 손님(client: httpx2.AsyncClient) -> None:
        for _ in 남은_번호:                          # 번호표가 떨어질 때까지 계속 요청
            시작 = time.perf_counter()
            응답 = await client.post("/predict/image", files={"file": ("x.png", 사진)})
            응답.raise_for_status()
            걸린_시간.append((time.perf_counter() - 시작) * 1000)

    async with httpx2.AsyncClient(base_url=주소, timeout=10) as client:
        시작 = time.perf_counter()
        await asyncio.gather(*[손님(client) for _ in range(동시)])
        전체 = time.perf_counter() - 시작
    백분위 = statistics.quantiles(걸린_시간, n=100)
    print(f"동시 {동시}명, 요청 {총}개: 초당 {총 / 전체:,.0f}개, "
          f"p50 {백분위[49]:.1f}ms, p95 {백분위[94]:.1f}ms, p99 {백분위[98]:.1f}ms")


if __name__ == "__main__":
    주소 = sys.argv[1]
    동시 = int(sys.argv[2]) if len(sys.argv) > 2 else 16
    총 = int(sys.argv[3]) if len(sys.argv) > 3 else 2000
    asyncio.run(부하_주기(주소, 동시, 총))
```

서버를 띄워 둔 채로 `uv run python -m scripts.load_test http://127.0.0.1:8000 16 3000`처럼 실행합니다(동시 손님 16명, 요청 3,000개). uvicorn의 `--workers` 옵션으로 서버 프로세스 수를, 환경 변수 `ORT_THREADS`로 ONNX Runtime의 계산 스레드 수를 바꿔 가며 쟀습니다. 아래는 이 책의 실행 환경(Apple M4 Max, 코어 16개)에서 잰 값이에요. 같은 컴퓨터에서 부하를 주는 프로그램도 함께 돌았으니 절대값보다는 서로의 비교로 보세요.

```
ORT_THREADS=1 uv run uvicorn app.main:app --port 8000 --workers 4
```

서버 설정	동시 1명: 초당 요청	동시 1명: p99	동시 16명: 초당 요청	동시 16명: p50	동시 16명: p99
워커 1개, 스레드 기본값	656	1.8ms	1,424	10.9ms	19.2ms
워커 1개, 스레드 1개	611	1.9ms	1,377	11.3ms	20.2ms
워커 4개, 스레드 기본값	643	2.5ms	699	21.7ms	43.1ms
워커 4개, 스레드 1개	613	1.9ms	2,065	7.4ms	13.4ms

표에서 세 가지를 읽을 수 있습니다.

- **손님이 한 명이면 설정은 상관없습니다.** 어느 설정이든 초당 600개 남짓, p99도 2.5ms 안쪽이에요. 한 번에 요청 하나뿐이니 워커와 스레드를 늘려도 놉니다.
- **손님이 몰리면 줄이 생깁니다.** 워커 하나로 16명을 받으면 초당 1,400개쯤에서 막히고 중앙값 지연이 11ms로 늘었어요. 늘어난 것은 **줄 서서 기다리는 시간**입니다.
- **워커를 늘렸더니 오히려 느려졌습니다.** 워커 네 개에 스레드를 기본값으로 두니 초당 699개로 워커 하나일 때의 절반이 됐어요. 프로세스 네 개가 저마다 "코어 수만큼" 스레드를 만들어 모두 16개 코어를 차지하려고 다투면서, 계산보다 자리다툼에 시간을 쓴 겁니다. 이런 상태를 **과다 구독(oversubscription)** 이라고 해요. 프로세스마다 스레드를 하나로 줄이자 초당 2,065개, p99 13.4ms로 가장 좋아졌습니다.

20.6절에서 본 것처럼 PyTorch와 ONNX Runtime은 스스로 코어를 나눠 쓰니, 그 위에 프로세스나 스레드를 더 얹을 때는 **곱한 수가 코어 수를 넘지 않게** 맞춰야 해요. 그래서 다음에 만들 Docker 이미지에는 `ORT_THREADS=1`을 기본으로 넣었습니다. 몇 개의 워커가 가장 좋은지는 모델과 컴퓨터마다 다르니, 이 스크립트로 직접 재서 정하세요.

Docker로 감싸기

이제 서비스를 회사 서버나 클라우드로 옮겨야 합니다. 그 컴퓨터에는 파이썬 3.12가 없을 수도, 다른 버전의 numpy가 깔려 있을 수도, 운영체제가 리눅스일 수도 있어요. "내 컴퓨터에서는 되는데요"가 나오는 순간이죠.

Docker는 이 문제를 이사 비유로 풉니다. 짐을 하나하나 새집에 맞춰 다시 사는 대신, 방 하나를 통째로 **컨테이너**에 담아 옮기는 거예요. 운영체제의 필요한 부분, 파이썬, 패키지, 우리 코드, 모델 파일까지 한 상자에 담으니 어느 컴퓨터에 내려놓아도 똑같이 돕니다. 용어 두 개만 알면 돼요.

- **이미지(image)**: 상자의 설계도이자 완성품입니다. `Dockerfile`이라는 글 파일에 "무엇을 어떤 순서로 담을지" 적으면 `docker build`가 이미지를 만들어요. 한 번 만든 이미지는 바뀌지 않습니다.
- **컨테이너(container)**: 이미지를 실제로 실행한 것입니다. 이미지 하나로 컨테이너를 여러 개 띄울 수 있어요. 클래스(이미지)와 객체(컨테이너)의 관계와 비슷합니다(1권 11장).

Docker는 docker.com에서 Docker Desktop을 받아 설치하면 됩니다. 먼저 의존성을 정리한 `pyproject.toml`을 보겠습니다.

```
[project]
name = "fashion-service"
version = "1.0.0"
requires-python = ">=3.12"
dependencies = [            # 서빙에 꼭 필요한 것만 (torch 없음)
    "fastapi>=0.156",
    "uvicorn>=0.54",
    "python-multipart>=0.0.32",
    "numpy>=2.5",
    "pillow>=13.0",
    "onnxruntime>=1.31",
]

[project.optional-dependencies]
torch = ["torch>=2.14", "onnx>=1.23", "onnxscript>=0.7"]   # torch 백엔드와 ONNX 내보내기용

[dependency-groups]
dev = ["pytest>=8.4", "httpx2>=2.15"]     # httpx2: TestClient가 쓰는 HTTP 라이브러리

[tool.uv]
package = false             # 설치할 패키지가 아니라 실행할 앱이다

[tool.pytest.ini_options]
pythonpath = ["."]          # 테스트에서 app 패키지를 찾을 수 있게
```

```
# 리눅스(도커)에서는 CUDA가 빠진 CPU 전용 torch를 받는다
[[tool.uv.index]]
name = "pytorch-cpu"
url = "https://download.pytorch.org/whl/cpu"
explicit = true

[tool.uv.sources]
torch = [{ index = "pytorch-cpu", marker = "sys_platform == 'linux'" }]
```

이 파일이 이미지 크기를 좌우합니다.

- **dependencies에는 서빙에 꼭 필요한 것만**: FastAPI, uvicorn, 업로드용 python-multipart, NumPy, Pillow, ONNX Runtime. **torch가 없습니다.**
- **torch는 선택 의존성으로**: `[project.optional-dependencies]`의 `torch` 묶음은 `--extra torch`를 줄 때만 설치됩니다. ONNX 내보내기와 torch 백엔드에만 필요하니까요.
- **테스트 도구는 개발 그룹으로**: `[dependency-groups]`의 `dev`는 `uv run`이 개발할 때 기본으로 설치하고, Docker 이미지를 만들 때는 `--no-dev`로 뺍니다.
- **리눅스용 torch는 CPU 전용으로**: 마지막 두 표는 "리눅스에서는 torch를 PyTorch의 CPU 전용 저장소에서 받으라"는 설정입니다. 왜 필요한지는 조금 뒤 이미지 크기로 보여 드릴게요. `explicit = true`는 torch 말고 다른 패키지는 이 저장소에서 찾지 말라는 뜻입니다.

`uv lock`을 실행하면 이 조건을 만족하는 정확한 버전이 `uv.lock`에 기록됩니다. 이미지 안에서 같은 잠금 파일로 설치하니, 내 노트북과 똑같은 버전이 들어가요(18장의 재현성). 이제 `Dockerfile`입니다.

```
# 1단계: uv로 가상 환경을 만든다 (빌드 도구는 이 단계에만 있다)
FROM python:3.12-slim AS builder
COPY --from=ghcr.io/astral-sh/uv:0.9.16 /uv /bin/uv
WORKDIR /app
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy
ARG EXTRAS=""
COPY pyproject.toml uv.lock .python-version ./
RUN uv sync --frozen --no-dev $EXTRAS

# 2단계: 실행에 필요한 것만 옮겨 담는다
```

```
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /app/.venv /app/.venv
COPY app ./app
COPY models ./models
ENV PATH="/app/.venv/bin:$PATH" MODEL_BACKEND=onnx ORT_THREADS=1
USER nobody
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s \
    CMD ["python", "-c", "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/
    ↪ health')"]
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

`FROM`이 두 번 나옵니다. **다단계 빌드(multi-stage build)** 라고 해요.

- **1단계(`builder`)**: 파이썬 3.12가 들어 있는 가벼운 리눅스 이미지(`python:3.12-slim`)에서 시작해, uv 공식 이미지에서 `uv` 실행 파일만 복사해 옵니다. `pyproject.toml`과 `uv.lock`을 먼저 복사해서 `uv sync --frozen --no-dev`로 가상 환경을 만들어요. `--frozen`은 잠금 파일을 고치지 말고 적힌 그대로 설치하라는 뜻입니다. `ARG EXTRAS`는 빌드할 때 `--extra torch`를 끼워 넣을 수 있게 열어 둔 자리예요.
- **2단계**: 새 `python:3.12-slim`에서 다시 시작해, 1단계에서 만든 **가상 환경 폴더만** 복사해 옵니다. uv와 설치 중에 쌓인 캐시는 1단계에 두고 오니 최종 이미지가 가벼워져요. 그다음 코드와 모델을 복사합니다.

나머지 줄도 하나씩 이유가 있습니다.

- **복사 순서**: Docker는 줄마다 결과를 **층(layer)** 으로 저장해 두고 바뀌지 않은 줄은 다음 빌드에서 그대로 씁니다. 잘 바뀌지 않는 의존성 설치를 먼저, 자주 바뀌는 코드 복사를 나중에 두면 코드만 고쳤을 때 설치를 건너뛰어요.
- **`ENV`**: 가상 환경의 `bin`을 `PATH` 앞에 넣어 `uvicorn`과 `python`이 가상 환경 것으로 실행되게 하고, 백엔드와 스레드 수의 기본값을 정합니다. `docker run -e MODEL_BACKEND=...`로 바꿀 수 있어요.
- **`USER nobody`**: 서버를 관리자(root)가 아닌, 아무 권한 없는 사용자로 실행합니다. 서버에 구멍이 나도 피해를 줄이는 기본 습관이에요.
- **`HEALTHCHECK`**: Docker가 30초마다 컨테이너 안에서 `/health`를 불러 봅니다. 응답이 없으면 컨테이너 상태가 `unhealthy`로 바뀌어요. 20.7절의 헬스 체크를 쓰는 곳이 바로 여기입니다. 이

미지에 `curl`이 없어서 파이썬 표준 라이브러리의 `urllib`로 불렀어요.

- `CMD`: 컨테이너가 시작될 때 실행할 명령입니다. `--host 0.0.0.0`이 중요해요. 기본값 `127.0.0.1`은 "같은 컨테이너 안에서 온 요청만 받는다"는 뜻이라, 컨테이너 밖에서는 접속할 수 없습니다.

`.dockerignore`에는 이미지에 넣지 않을 것을 적습니다. 특히 내 노트북의 `.venv`가 들어가면 맥용 패키지가 리눅스 이미지에 섞이니 꼭 빼야 해요.

```
.venv
__pycache__
.pytest_cache
tests
scripts
```

이제 빌드합니다. `-t`는 이미지에 붙일 이름표(이름:태그)예요. 비교를 위해 torch를 넣은 이미지도 함께 만들었습니다.

```
docker build -t fashion-service:onnx .
docker build --build-arg EXTRAS="--extra torch" -t fashion-service:torch-cpu .
docker images fashion-service
```

```
REPOSITORY        TAG         IMAGE ID       CREATED                  SIZE
fashion-service   torch-cpu   d795993dd16f   Less than a second ago   1.51GB
fashion-service   onnx        9af313c3c83e   1 second ago             445MB
fashion-service   torch       b626eed4c12e   17 minutes ago           9.22GB
```

목록에 이미지가 셋 있습니다. 세 번째 `torch`는 `pyproject.toml`에 CPU 전용 저장소 설정을 넣기 **전에** 같은 방법으로 만든 이미지예요. 크기를 비교해 보세요.

이미지	들어 있는 것	크기
`fashion-service:onnx`	ONNX Runtime만	445MB
`fashion-service:torch-cpu`	+ CPU 전용 PyTorch	1.51GB
`fashion-service:torch`	+ 보통 PyTorch (CUDA 라이브러리 포함)	9.22GB

리눅스용 PyTorch를 그냥 설치하면 NVIDIA GPU용 CUDA 라이브러리가 함께 따라와서 이미지가 9GB를 넘었습니다. 빌드 기록을 보면 `nvidia-`로 시작하는 패키지가 줄줄이 설치돼요. GPU가 없는 서버에는 쓸모없는 짐이죠. CPU 전용 저장소에서 받으니 1.51GB로 줄었고, PyTorch를 아예 빼고 ONNX Runtime만 넣으니 445MB입니다. 같은 일을 하는 서비스가 스무 배 차이 나는 거예요. 이미지가 작으면 서버로 내려받는 시간, 저장 공간, 새 서버를 띄우는 시간이 모두 줄어듭니다. 19장에서 ONNX로 내보낸 수고가 여기서 보상받는 셈이에요.

`docker images`로 보면 바탕 이미지 `python:3.12-slim`만 해도 203MB이니, ONNX 이미지에서 우리 몫은 절반 남짓입니다. 컨테이너 안에서 `du`로 재 보니 가상 환경 폴더가 ONNX 이미지는 170MB, torch-cpu 이미지는 979MB였어요.

이제 ONNX 이미지로 컨테이너를 띄워 봅니다.

```
docker run -d --name fashion -p 8000:8000 fashion-service:onnx
docker ps --filter name=fashion --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
curl -s -F "file=@tests/data/pullover.png" localhost:8000/predict/image
sleep 30                                  # 30초 기다렸다가 상태를 다시 본다
docker ps --filter name=fashion --format "table {{.Names}}\t{{.Status}}"
```

```
e809272f1af563b436d7f6fe191d09022a4d6a56036a803304204bb487328e63
NAMES     IMAGE                  STATUS                            PORTS
fashion   fashion-service:onnx   Up 4 seconds (health: starting)   0.0.0.0:8000->8000/tcp,
↪  [::]:8000->8000/tcp
{"file":"pullover.png","label":"풀오버","class_id":2,"probability":0.9998,"top3":[{"label":"풀오
↪  버","probability":0.9998},{"label":"T셔츠","probability":0.0001},{"label":"셔
↪  츠","probability":0.0}]}
NAMES     STATUS
fashion   Up 34 seconds (healthy)
```

`-d`는 컨테이너를 뒤에서 돌리라는 뜻이고(첫 줄의 긴 글자가 새 컨테이너의 번호입니다), `-p 8000:8000`은 "내 컴퓨터의 8000번 포트로 온 요청을 컨테이너의 8000번 포트로 넘겨라"라는 뜻이에요. 컨테이너는 자기만의 네트워크를 가진 작은 컴퓨터라서, 이렇게 문을 이어 줘야 밖에서 들어갈 수 있습니다. 처음 상태는 `health: starting`이었다가, 30초 뒤에는 첫 헬스 체크를 통과해 `healthy`로 바뀌었습니다. 예측은 맥에서 띄웠을 때와 같은 답이 왔어요. 컨테이너 로그도 볼 수 있습니다.

```
docker logs fashion
```

```
INFO:     Started server process [1]
INFO:     Waiting for application startup.
2026-10-21 07:02:03.082696426 [W:onnxruntime:Default, telemetry_1ds.cc:600 operator()] Failed to
↪  persist telemetry device ID; using an in-memory identifier
INFO:     모델 준비 완료: backend=onnx, sha256=265e35c606cf
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     192.168.65.1:34036 - "POST /predict/image HTTP/1.1" 200 OK
INFO:     127.0.0.1:50952 - "GET /health HTTP/1.1" 200 OK
```

모델 지문이 맥에서 띄웠을 때와 같은 `265e35c606cf`입니다. 같은 모델 파일이 상자에 담겨 왔다는 확인이에요. 셋째 줄의 ONNX Runtime 경고는 `USER nobody`로 실행해서 생긴 것입니다. ONNX Runtime이 사용 통계에 쓸 기기 번호를 홈 폴더에 저장하려다 권한이 없어 못 했다는 뜻이라, 서비스에는 영향이 없어요. 마지막 두 줄의 요청 주소도 보세요. 우리가 `curl`로 보낸 예측 요청은 `192.168.65.1`, 즉 컨테이너 밖에서 들어왔고, `/health` 요청은 `127.0.0.1`, 컨테이너 안에서 왔습니다. `HEALTHCHECK`가 30초마다 부른 것이 이 줄이에요.

다 쓴 컨테이너와 이미지는 지워 둡니다. 이미지는 디스크를 꽤 차지하니까요.

```
docker rm -f fashion
docker rmi fashion-service:onnx fashion-service:torch-cpu fashion-service:torch
```

`docker rm -f`는 돌고 있는 컨테이너를 멈추고 지우고, `docker rmi`는 이미지를 지웁니다.

Q

Q. 이 이미지를 클라우드에 올리려면 어떻게 하나요? 이미지를 **레지스트리**(이미지 저장소)에 `docker push`로 올리고, 서버에서 `docker pull`로 받아 `docker run`하는 것이 기본 흐름입니다. 클라우드 회사들은 저마다 레지스트리와 "이미지를 주면 알아서 띄워 주는" 서비스를 갖추고 있어요. 다만 회사마다 방법이 달라서 이 책에서는 다루지 않습니다. 이 장에서 만든 것처럼 `/health`가 있고, 설정을 환경 변수로 받고, 포트 하나로 HTTP를 받는 이미지라면 어느 곳에 올려도 거의 그대로 돌아가요. 21장 종합 프로젝트도 Docker 이미지까지 만듭니다.

20.12 정리와 다음 장 미리보기

이 장에서 학습한 모델을 다른 프로그램이 쓸 수 있는 웹 서비스로 만들었습니다.

- **불러오기**: 서버가 켜질 때 한 번. `lifespan`의 `yield` 앞에서 올려 `app.state`에 둔다. `map_location="cpu"`, `weights_only=True`, `.eval()`. 테스트는 `with TestClient(app)`로 열어야 lifespan이 돈다
- **스키마와 파일**: `Annotated`와 `Field`로 모양·범위 검사, 응답 모델은 자동 문서에. `UploadFile`, 한도보다 1바이트만 더 읽어 크기 검사, 사진이 아니면 400, 너무 크면 413
- **전처리**: 학습 때와 똑같이. 흰 배경 사진을 그대로 넣으면 90.7%가 28.4%로 무너졌다. 서빙할 곳의 실제 입력을 보고 맞추기
- **배치**: 묶을수록 한 장당 빠르다(512장이면 열 배쯤). 여러 장 엔드포인트에는 상한을, 따로 온 요청은 대기 줄과 `Future`로 잠깐 모아 한 번에(동적 배칭)
- **async**: 추론하는 경로 함수는 그냥 `def`로, `async def`라면 추론만 `run_in_threadpool`로. 그래야 계산 중에도 `/health`에 답한다
- **운영 정보**: `/health`(살아 있나), `/ready`(준비됐나), `/version`(모델 파일 해시), 미들웨어로 처리 시간 헤더
- **백엔드 바꾸기**: NumPy 배열을 주고받는 `Protocol` 약속, 환경 변수로 고르기. ONNX Runtime이면 torch 없이
- **지연 시간**: 평균 말고 p50·p95·p99. 작은 모델은 모델보다 HTTP 처리가 비싸다. 재 보고 고칠 곳을 정한다
- **스트리밍**: `StreamingResponse`와 제너레이터, LLM은 `TextIteratorStreamer` + 생성 스레드. 첫 조각까지의 시간이 체감 속도
- **테스트**: pytest 픽스처로 `TestClient` 열기, `dependency_overrides`로 가짜 모델 끼우기, 없는 패키지는 `importorskip`
- **부하와 워커**: 워커 수 × 계산 스레드 수가 코어 수를 넘으면 오히려 느려진다(과다 구독)
- **Docker**: 다단계 빌드로 uv 가상 환경만 옮겨 담기, 의존성 먼저·코드 나중, root가 아닌 사용자, `HEALTHCHECK`, `--host 0.0.0.0`. torch를 빼면 9.22GB가 445MB로

이 장으로 Part 4 실무 편이 끝났습니다. 데이터를 준비하고(Part 1), 모델을 만들고(Part 2, 3), 실험을 관리하고, 가볍게 만들고, 서비스로 내놓는 데까지(Part 4) 한 바퀴를 돌았어요. 다음 21장은 이 책의 마지막 장인 종합 프로젝트입니다. PyTorch 임베딩으로 취향을 배우는 **맛집 추천 AI**를 처음부터 만들고, 이 장에서 익힌 방법 그대로 FastAPI 서비스와 테스트, Docker 이미지까지 완성합니다. 각 장에서 따로 배운 조각들이 하나의 제품으로 맞물리는 모습을 보게 될 거예요.

21

종합 프로젝트: 맛집 추천 AI

학습 목표

- 추천을 "순위 맞히기" 문제로 정의하고, Recall@K와 NDCG@K로 인기순 기준선과 비교하기
- 숨은 취향을 넣은 예제용 데이터를 만들고, 시간 순서로 나눠 미래를 엿보지 않게 평가하기
- `nn.Embedding`으로 행렬 분해 모델을 만들고, 음성 샘플링과 BCE·BPR 손실로 학습하기
- 학습된 식당 임베딩으로 비슷한 식당을 찾고, 임베딩이 숨은 성질을 배웠는지 확인하기
- 특징을 함께 쓰는 투-타워 모델로 새 식당·새 사용자 문제(콜드 스타트)를 다루기
- 18장 방식으로 설정·씨앗·결과를 기록하고 여러 씨앗으로 모델을 고르기
- 벡터를 미리 계산해 두고 torch 없는 FastAPI 추천 API를 만들어 테스트와 Docker 이미지까지 완성하기

들어가며: "오늘 점심 뭐 먹지?"

점심시간마다 같은 고민을 합니다. 이럴 때 가장 믿음직한 건 내 입맛을 아는 친구예요. "너 매운 거 좋아하잖아, 성수에 새로 생긴 마라탕집 가 봐." 친구는 내가 어디를 다녀왔고 무엇을 좋아했는지 기억해 두었다가, 내가 아직 안 가 본 곳 중에서 좋아할 만한 곳을 골라 줍니다.

배달 앱, 지도 앱, 쇼핑몰, 동영상 서비스의 첫 화면이 하는 일이 바로 이것입니다. 수많은 사용자의 기록에서 "이런 사람은 이런 곳을 좋아하더라"를 배워, 한 사람에게 맞춘 목록을 보여 주는 **추천 시**

스템(recommender system) 이에요. 1장 1.1절에서 "규칙을 짜기 어려운 문제"의 예로 추천을 들었는데, 이번에는 그걸 직접 만듭니다.

이 장은 2권의 마지막 장이자 종합 프로젝트입니다. 지금까지 장마다 따로 배운 조각을 하나의 제품으로 엮어요.

단계	이 장에서 하는 일	다시 쓰는 장
문제 정의	무엇이 좋은 추천인가, 순위 지표	1장(작업 흐름), 3장 3.7절(재현율)
데이터	예제용 데이터 만들기, 시간 순서로 나누기	2장 2.3절(누수), 2.9절(시간 데이터)
기준선	무작위, 인기순, 동네 인기순	3장(기준선과 비교하기)
모델	행렬 분해, BPR 손실, 투-타워	4장(행렬 곱), 6장, 8장, 11장 11.11절·13장 13.7절(임베딩)
실험	설정·씨앗·결과 기록, 여러 씨앗 비교	18장
서빙	벡터 미리 계산, FastAPI, pytest, Docker	19장, 20장

Q

Q. 이 장에 필요한 패키지는 무엇인가요? 모델을 만드는 앞부분(21.1~21.11절)은 `uv add torch numpy pandas`면 됩니다. 마지막 실습의 서비스는 따로 만든 프로젝트 폴더에서 `uv add fastapi uvicorn numpy`로 시작하고, 테스트 도구는 `uv add --dev pytest httpx2`로 더해요(21.12절). 이 장의 예제는 파이썬 3.12, PyTorch 2.14.1, NumPy 2.5.4, pandas 3.0.6에서, 서비스는 FastAPI 0.156.8(20장의 0.156.4보다 조금 뒤에 잠근 판), Pydantic 2.14.1, uvicorn 0.54.0, pytest 9.1.1, httpx2 2.16.0에서 실행해 확인했고, 컨테이너는 Docker 28.3.2(엔진)와 uv 0.9.16으로 만들었습니다. 모델이 작아서 학습은 모두 CPU로 몇 초면 끝나고, 결과를 똑같이 재현하려고 일부러 CPU에서 돌렸어요.

21.1 좋은 추천이란 무엇일까요?

별점을 맞히는 문제일까요?

추천을 처음 떠올리면 "이 사람이 이 식당에 별점 몇 점을 줄까?"를 맞히는 문제로 생각하기 쉽습니다. 3장의 회귀처럼 예측한 별점과 실제 별점의 차이(RMSE)를 줄이면 될 것 같죠. 그런데 사용자가 보는 화면을 떠올려 보세요. 앱은 별점 예측값을 보여 주지 않습니다. **목록 맨 위의 열 곳 남짓**을 보여 줄 뿐이에요. 사용자는 거기서 마음에 드는 곳을 고르거나, 없으면 앱을 닫습니다.

그러니 중요한 것은 "별점을 소수점까지 맞혔나"가 아니라 "사용자가 실제로 가서 좋아할 곳이 **목록 위쪽에** 있었나"입니다. 4.2점짜리 식당을 4.0점으로 예측해도 목록 1등에 올렸다면 성공이고, 별점을 정확히 맞혀도 그 식당이 목록 50등에 있었다면 사용자는 끝내 보지 못해요. 추천은 **순위를 맞히는 문제**입니다.

데이터도 별점만 있는 게 아닙니다. 별점이나 "좋아요"처럼 사용자가 직접 남긴 평가를 **명시적 피드백**, 방문·클릭·구매처럼 행동으로 드러난 기록을 **암묵적 피드백**이라고 해요. 별점을 남기는 사람은 드물어도 방문 기록은 모두에게 쌓이니, 실제 서비스는 대개 암묵적 피드백을 중심으로 배웁니다. 이 장에서도 방문 기록을 쓰되, 다녀와서 1~2점을 준 곳은 "좋아한다"는 신호가 아니니 빼고 **3점 이상 준 방문**만 "좋았던 방문"으로 칩니다.

정리하면 이 장의 과제는 이렇습니다. **어느 날까지의 기록을 보고, 그 뒤 기간에 사용자가 처음 가서 좋았던 식당을 상위 K개 안에 넣어라.** 이미 가 본 식당은 후보에서 뺍니다. 단골집을 다시 추천하는 건 쉽지만, 친구에게 바라는 건 "아직 안 가 본 좋은 곳"이니까요.

Recall@K와 NDCG@K

순위를 채점하는 지표 두 가지를 쓰겠습니다.

- Recall@K: 정답 식당 중 상위 K개 안에 들어간 비율입니다. 3장 3.7절의 **재현율**("진짜 양성 중 찾아낸 비율")을 "상위 K개 안에서"로 좁힌 것이에요. 정답이 두 곳인데 한 곳만 목록 열 칸 안에 있으면 Recall@10은 0.5입니다.
- NDCG@K: Recall은 1등에 있든 10등에 있든 똑같이 셉니다. 그런데 사용자는 위에서부터 보니 1등에 있는 편이 훨씬 좋죠. NDCG(정규화 할인 누적 이득)는 맞힌 식당마다 `1 / log2(순위 + 1)`을 더합니다. 1등이면 1, 2등이면 0.63, 3등이면 0.5, 10등이면 0.29로 아래로 갈수록 덜 쳐 줘요. 이 합(DCG)을 "정답을 가장 위에 모두 놓았을 때의 합"(IDCG)으로 나누면 0~1 사이의 값이 됩니다.

이 장에서 함께 쓸 지표 도구를 `순위지표.py`로 저장하세요. 앞쪽 두 함수가 지금 설명한 지표이고, 뒤쪽의 `평가셋`, `평가셋_만들기`, `평가하기`는 여러 사용자를 한꺼번에 채점하는 도구로 21.3절에서 설명합니다.

```
"""21장 순위 지표: 추천 목록 하나를 채점하고, 여러 사용자를 한꺼번에 평가한다."""
from dataclasses import dataclass

import numpy as np
import pandas as pd

좋은_평점 = 3                                # 3점 이상 준 방문을 '좋았던 방문'으로 본다


def recall_at_k(추천: list[int], 정답: set[int], k: int) -> float:
    """정답 식당 중 상위 k개 안에 든 비율."""
    return len(set(추천[:k]) & 정답) / len(정답)


def ndcg_at_k(추천: list[int], 정답: set[int], k: int) -> float:
    """맞힌 식당이 위에 있을수록 높은 점수. 가장 좋은 순서일 때 1."""
    dcg = sum(1 / np.log2(순위 + 2) for 순위, 식당 in enumerate(추천[:k]) if 식당 in 정답)
    idcg = sum(1 / np.log2(순위 + 2) for 순위 in range(min(len(정답), k)))
    return dcg / idcg


@dataclass
class 평가셋:
    사용자: np.ndarray          # 평가할 사용자 번호들
    정답: list[set[int]]        # 사용자마다 '처음 가서 좋았던' 식당들
    제외: np.ndarray            # (사용자, 식당) 참이면 후보에서 뺀다


def 평가셋_만들기(이전: pd.DataFrame, 대상: pd.DataFrame, 식당: pd.DataFrame) -> 평가셋:
    """이전 기록이 있는 사용자가 대상 기간에 '처음 가서 좋았던' 식당을 맞히는 문제를 만든다."""
    가본_곳 = 이전.groupby("사용자")["식당"].apply(set)
    좋았던 = 대상[대상["평점"] >= 좋은_평점]
    사용자들, 정답들 = [], []
    for u, 식당들 in 좋았던.groupby("사용자")["식당"]:
        새_곳 = set(식당들) - 가본_곳.get(u, set())
        if u in 가본_곳.index and 새_곳:          # 이전 기록이 있고, 새로 간 곳이 있는 사람만
            사용자들.append(u)
```

```
            정답들.append(새_곳)
    마지막_날 = 대상["날짜"].max()
    제외 = np.zeros((len(사용자들), len(식당)), dtype=bool)
    제외[:, 식당["개업일"].to_numpy() > 마지막_날] = True     # 아직 문을 열지 않은 식당
    for 줄, u in enumerate(사용자들):
        제외[줄, list(가본_곳[u])] = True                    # 이미 가 본 식당
    return 평가셋(np.array(사용자들), 정답들, 제외)


def 평가하기(점수: np.ndarray, 셋: 평가셋, k: int = 10) -> dict[str, float]:
    """점수: (평가 사용자 수, 식당 수). 제외할 칸을 지우고 상위 k개로 두 지표의 평균을 낸다."""
    점수 = np.where(셋.제외, -np.inf, 점수)
    상위 = np.argsort(-점수, axis=1, kind="stable")[:, :k]
    return {f"Recall@{k}": float(np.mean([recall_at_k(list(줄), 답, k) for 줄, 답 in zip(상위, 셋.
    ↪ 정답)])),
            f"NDCG@{k}": float(np.mean([ndcg_at_k(list(줄), 답, k) for 줄, 답 in zip(상위, 셋.정
            ↪ 답)]))}
```

손으로 확인해 보겠습니다. 어떤 사용자가 다음 달에 처음 가서 좋았던 곳이 "성수 고기"와 "종로 분식" 두 곳이라고 해 봅시다. 같은 다섯 식당을 순서만 바꾼 목록 셋을 채점해요.

```
from 순위지표 import ndcg_at_k, recall_at_k

이름 = {41: "성수 고기", 17: "종로 분식", 102: "강남 카페", 233: "홍대 일식", 5: "잠실 한식"}
정답 = {41, 17}                       # 이 사용자가 다음 달에 처음 가서 좋았던 두 곳

목록들 = {
    "가": [41, 102, 17, 233, 5],      # 정답이 1등, 3등
    "나": [102, 233, 5, 17, 41],      # 같은 다섯 곳, 정답이 4등, 5등
    "다": [17, 41, 102, 233, 5],      # 정답이 1등, 2등
}
for 이름표, 목록 in 목록들.items():
    print(f"목록 {이름표}: {[이름[i] for i in 목록[:3]]} ...")
    for k in (3, 5):
        print(f"   k={k}: Recall {recall_at_k(목록, 정답, k):.3f}, NDCG {ndcg_at_k(목록, 정답,
        ↪ k):.3f}")
```

```
목록 가: ['성수 고기', '강남 카페', '종로 분식'] ...
   k=3: Recall 1.000, NDCG 0.920
   k=5: Recall 1.000, NDCG 0.920
목록 나: ['강남 카페', '홍대 일식', '잠실 한식'] ...
   k=3: Recall 0.000, NDCG 0.000
   k=5: Recall 1.000, NDCG 0.501
목록 다: ['종로 분식', '성수 고기', '강남 카페'] ...
   k=3: Recall 1.000, NDCG 1.000
   k=5: Recall 1.000, NDCG 1.000
```

- **목록 가**는 정답을 1등과 3등에 두었습니다. 상위 3개 안에 정답이 둘 다 있으니 Recall@3은 1입니다. NDCG는 DCG가 1/log2(2) + 1/log2(4) = 1 + 0.5 = 1.5, 가장 좋은 경우(1등, 2등)의 IDCG가 1 + 0.631 = 1.631이라서 1.5 ÷ 1.631 = 0.920이에요.
- **목록 나**는 같은 다섯 곳인데 정답을 4등과 5등에 두었습니다. 상위 3개만 보는 사용자에게는 아무것도 맞히지 못한 목록이라 Recall@3과 NDCG@3이 모두 0이에요. k를 5로 늘리면 Recall@5는 1이 되지만 NDCG@5는 0.501에 그칩니다. 둘 다 찾았지만 너무 아래에 있다는 뜻이죠.
- **목록 다**는 정답을 1등과 2등에 두어 두 지표 모두 1입니다.

목록 가와 다는 Recall로는 구분되지 않고 NDCG로만 구분됩니다. 그래서 두 지표를 함께 보고, 모델을 고를 때는 순서까지 따지는 NDCG를 주로 봐요.

Q

Q. 정확도로는 왜 안 되나요? 식당 400곳 중 사용자가 다음 달에 처음 가는 곳은 한두 곳뿐입니다. "아무 데도 안 간다"고 찍으면 정확도는 99.5%가 넘어요. 3장 3.7절의 불균형 데이터에서 정확도가 속이던 것과 같은 이야기입니다. 게다가 정확도는 순서를 보지 않습니다. 추천에서는 언제나 "위쪽 몇 칸"을 채점하는 지표를 써야 해요.

21.2 예제용 데이터 만들기

왜 직접 만들까요?

실제 맛집 리뷰 데이터를 쓰면 좋겠지만 두 가지 문제가 있습니다. 첫째, **개인정보**입니다. 누가 언제 어느 식당에 갔는지는 그 사람의 동선이에요. 이름을 지워도 방문 기록 몇 개만으로 사람을 짐작할 수 있습니다. 둘째, **이용 약관과 라이선스**입니다. 플랫폼의 리뷰를 모아 책에 싣거나 나눠 주는 것은 대개 허락되지 않아요. 그래서 이 장은 **예제용으로 직접 만든 데이터**를 씁니다. 동네와 음식 이름은

친숙하게 붙였지만 식당, 사용자, 방문 기록은 모두 가짜예요.

직접 만들면 좋은 점도 있습니다. 데이터 속에 어떤 취향 구조가 들어 있는지 **정답을 알고 있어서**, 나중에 모델이 그 구조를 배웠는지 확인할 수 있어요(21.7절). 대신 진짜 데이터보다 훨씬 깔끔하니, 이 장의 숫자를 실제 서비스의 성능으로 읽으면 안 됩니다.

숨은 취향을 심기

데이터를 만드는 규칙은 이렇습니다.

	모델이 볼 수 있는 것	모델이 볼 수 없는 것(숨은 열)
식당 400곳	이름, 지역(8곳), 음식 종류(8가지), 가격대(1~3), 개업일	매운 정도, 분위기(조용함~시끌벅적), 품질
사용자 3,000명	사는 동네, 가입일	직장 지역, 매운맛 선호, 분위기 선호, 가격 민감도, 음식 종류 선호, 활동량
방문 기록	누가, 언제(0~364일), 어디에, 평점 1~5	

사용자마다 식당마다 **끌림 점수**를 계산합니다. 좋아하는 음식 종류이면, 매운맛 선호와 식당의 매운 정도가 맞으면, 분위기 취향이 맞으면, 사는 동네나 직장 근처이면, 품질이 좋으면 점수가 오르고, 가격에 민감한 사람에게 비싼 곳은 점수가 내려가요. 방문할 때마다 그날 문을 연 식당 중에서 끌림이 큰 곳을 **확률적으로** 고릅니다. 늘 1등만 고르면 사람이 아니죠.

확률적으로 고르는 데는 **굼벨(Gumbel) 트릭**을 썼습니다. 점수마다 굼벨 분포에서 뽑은 잡음을 더한 뒤 가장 큰 것을 고르면, 점수에 소프트맥스를 씌운 확률대로 뽑은 것과 같아진다는 성질이에요. 14장에서 다음 토큰을 확률대로 뽑던 것과 같은 일을, 반복문 없이 배열 계산 한 번으로 하는 요령입니다. 평점은 고른 식당의 끌림이 클수록 높게 주되 잡음을 섞어 1~5점으로 만들었어요. 끌림이 큰 곳을 골랐어도 그날따라 별로였을 수 있으니까요.

마지막으로 두 가지 "늦게 온 손님"을 넣었습니다. 식당 40곳은 300일 이후에, 사용자 300명은 300일 이후에 처음 등장해요. 21.9절의 콜드 스타트 문제를 위해서입니다. 아래 내용을 `맛집데이터.py`로 저장하세요. `숨은_`으로 시작하는 열은 정답 확인용이고, 모델에는 절대 넣지 않습니다.

```
"""21장 예제용 맛집 데이터: 숨은 취향을 넣어 만든 가짜 사용자·식당·방문 기록 (실제 데이터가 아니다)."""
from dataclasses import dataclass

import numpy as np
import pandas as pd

지역들 = ["강남", "홍대", "성수", "종로", "여의도", "잠실", "이태원", "신촌"]
음식들 = ["한식", "중식", "일식", "양식", "분식", "고기", "해산물", "카페"]
음식별_매운맛 = np.array([0.6, 0.7, 0.2, 0.2, 0.7, 0.4, 0.4, 0.05])
기간 = 365                                  # 1년치 기록 (0일 ~ 364일)


@dataclass
class 맛집데이터:
    식당: pd.DataFrame
    사용자: pd.DataFrame
    방문: pd.DataFrame


def 식당_만들기(rng: np.random.Generator, 개수: int, 새_식당: int) -> pd.DataFrame:
    음식 = rng.integers(0, len(음식들), 개수)
    지역 = rng.integers(0, len(지역들), 개수)
    개업일 = np.zeros(개수, dtype=int)
    개업일[-새_식당:] = rng.integers(300, 350, 새_식당)   # 마지막 몇 곳은 나중에 문을 연다
    이름 = [f"{지역들[j]} {음식들[c]} {i:03d}" for i, (j, c) in enumerate(zip(지역, 음식))]
    return pd.DataFrame({
        "식당": np.arange(개수), "이름": 이름, "지역": 지역, "음식": 음식,
        "가격대": rng.integers(1, 4, 개수), "개업일": 개업일,
        # '숨은_'으로 시작하는 열은 정답 확인용이다. 모델에는 절대 주지 않는다
        "숨은_매운맛": np.clip(음식별_매운맛[음식] + rng.normal(0, 0.25, 개수), 0, 1),
        "숨은_분위기": rng.uniform(-1, 1, 개수),            # -1 조용함 ~ +1 시끌벅적
        "숨은_품질": rng.normal(0, 1, 개수),
    })
```

```
def 사용자_만들기(rng: np.random.Generator, 개수: int, 새_사용자: int) -> pd.DataFrame:
    가입일 = np.zeros(개수, dtype=int)
    가입일[-새_사용자:] = rng.integers(300, 340, 새_사용자)   # 마지막 몇 명은 늦게 가입한다
    return pd.DataFrame({
        "사용자": np.arange(개수), "동네": rng.integers(0, len(지역들), 개수),
        "가입일": 가입일,
        "숨은_직장": rng.integers(0, len(지역들), 개수),
        "숨은_매운맛선호": rng.normal(0, 1, 개수),
        "숨은_분위기선호": rng.normal(0, 1, 개수),
        "숨은_가격민감": rng.uniform(0, 1, 개수),
        "숨은_활동량": rng.lognormal(np.log(18), 0.6, 개수),
    })


def 끌림_계산(식당: pd.DataFrame, 사용자: pd.DataFrame, 음식_선호: np.ndarray) -> np.ndarray:
    """사용자 × 식당 끌림 점수. 숨은 취향과 식당의 숨은 성질이 맞을수록 크다."""
    u, r = 사용자, 식당
    끌림 = (1.5 * 음식_선호[:, r["음식"]]
          + 1.6 * np.outer(u["숨은_매운맛선호"], r["숨은_매운맛"] - 0.4)
          + 1.2 * np.outer(u["숨은_분위기선호"], r["숨은_분위기"])
          - 1.0 * np.outer(u["숨은_가격민감"], r["가격대"] - 2)
          + 0.8 * r["숨은_품질"].to_numpy())
    끌림 += 2.2 * (u["동네"].to_numpy()[:, None] == r["지역"].to_numpy())
    끌림 += 1.6 * (u["숨은_직장"].to_numpy()[:, None] == r["지역"].to_numpy())
    return 끌림


def 데이터_만들기(씨앗: int = 0, 식당_수: int = 400, 사용자_수: int = 3000) -> 맛집데이터:
    rng = np.random.default_rng(씨앗)
    식당 = 식당_만들기(rng, 식당_수, 새_식당=40)
    사용자 = 사용자_만들기(rng, 사용자_수, 새_사용자=300)
    끌림 = 끌림_계산(식당, 사용자, rng.normal(0, 1, (사용자_수, len(음식들))))
    개업일 = 식당["개업일"].to_numpy()
    기록 = []
    for u, 가입 in enumerate(사용자["가입일"]):
        횟수 = max(3, int(사용자["숨은_활동량"][u] * (기간 - 가입) / 기간))
```

```
        날짜 = np.sort(rng.integers(가입, 기간, 횟수))
        # 날마다 그날 문을 연 식당 중에서 끌림이 큰 곳을 확률적으로 고른다 (굼벨 트릭)
        점수 = 끌림[u] + rng.gumbel(size=(횟수, 식당_수))
        점수[개업일[None, :] > 날짜[:, None]] = -np.inf
        고른_곳 = 점수.argmax(axis=1)
        기록.append(pd.DataFrame({"사용자": u, "식당": 고른_곳, "날짜": 날짜, "끌림": 끌림[u, 고른_
↪ 곳]}))
    방문 = pd.concat(기록).sort_values(["날짜", "사용자"]).reset_index(drop=True)
    # 평점: 끌림이 클수록 높게, 잡음을 섞어 1~5점으로
    z = (방문["끌림"] - 방문["끌림"].mean()) / 방문["끌림"].std()
    방문["평점"] = np.clip(np.round(3.6 + 0.9 * z + rng.normal(0, 0.8, len(방문))), 1,
↪ 5).astype(int)
    방문 = 방문.drop(columns="끌림")
    return 맛집데이터(식당, 사용자, 방문)


def 시간으로_나누기(방문: pd.DataFrame, 검증_시작: int = 300,
                  시험_시작: int = 330) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    """날짜로 자른다(2장 2.9절): 학습 < 검증_시작 <= 검증 < 시험_시작 <= 시험."""
    학습 = 방문[방문["날짜"] < 검증_시작]
    검증 = 방문[(방문["날짜"] >= 검증_시작) & (방문["날짜"] < 시험_시작)]
    시험 = 방문[방문["날짜"] >= 시험_시작]
    return 학습, 검증, 시험
```

끌림_계산의 `np.outer(사용자 쪽, 식당 쪽)`은 4장 4.6절의 브로드캐스팅과 같은 계산으로, 사용자 3,000명 × 식당 400곳의 표를 한 번에 만듭니다. 만든 데이터를 살펴보겠습니다. 1권 20장에서 쓴 pandas 그대로예요.

```
import numpy as np

from 맛집데이터 import 데이터_만들기

데이터 = 데이터_만들기(씨앗=0)
식당, 사용자, 방문 = 데이터.식당, 데이터.사용자, 데이터.방문
print(f"식당 {len(식당)}곳, 사용자 {len(사용자):,}명, 방문 {len(방문):,}번")
```

```
보이는_열 = [열 for 열 in 식당.columns if not 열.startswith("숨은_")]
print(식당[보이는_열].head(3).to_string(index=False))
print(방문.head(3).to_string(index=False))

방문수 = 방문.groupby("사용자").size()
print(f"사용자 한 명의 방문: 중앙값 {방문수.median():.0f}번, 가장 많은 사람 {방문수.max()}번")
식당별 = 방문["식당"].value_counts()
print(f"방문이 가장 많은 식당 10%가 받은 방문: {식당별.head(len(식당) // 10).sum() / len(방
↪ 문):.1%}")
print(f"다시 간 방문의 비율: {방문.duplicated(['사용자', '식당']).mean():.1%}")
print("평점 분포:", 방문["평점"].value_counts().sort_index().to_dict())
채워진_칸 = 방문[["사용자", "식당"]].drop_duplicates().shape[0]
print(f"사용자 × 식당 표에서 채워진 칸: {채워진_칸:,} / {len(사용자) * len(식당):,} "
      f"= {채워진_칸 / (len(사용자) * len(식당)):.2%}")
```

```
식당 400곳, 사용자 3,000명, 방문 58,330번
 식당         이름  지역  음식  가격대  개업일
  0 신촌 해산물 000   7   6    1    0
  1  성수 고기 001   2   5    2    0
  2 이태원 분식 002   6   4    3    0
 사용자  식당  날짜  평점
  78 143   0   4
 189 181   0   5
 211 134   0   4
사용자 한 명의 방문: 중앙값 16번, 가장 많은 사람 146번
방문이 가장 많은 식당 10%가 받은 방문: 34.1%
다시 간 방문의 비율: 28.5%
평점 분포: {1: 2358, 2: 8305, 3: 16626, 4: 17916, 5: 13125}
사용자 × 식당 표에서 채워진 칸: 41,709 / 1,200,000 = 3.48%
```

- **식당 표**: 모델이 볼 수 있는 열만 골라 찍었습니다. 지역과 음식은 번호로 들어 있고(0번 지역은 강남, 7번은 신촌), 이름에도 지역과 음식이 붙어 있어요.
- **방문 수는 사람마다 크게 다릅니다.** 중앙값은 16번인데 가장 많이 간 사람은 146번이에요. 소수의 열성 사용자가 기록의 상당 부분을 만드는 건 실제 서비스에서도 흔한 모습입니다.
- **인기는 한쪽으로 쏠립니다.** 방문이 많은 식당 10%(40곳)가 전체 방문의 34.1%를 받았어요. 이렇게 소수가 많이 받고 나머지가 길게 늘어선 모양을 **긴 꼬리(long tail)** 라고 합니다. 인기

순 추천이 생각보다 잘 맞는 이유이자, 꼬리 쪽 식당은 좀처럼 추천되지 못하는 이유예요.

- **다시 간 방문이 28.5%입니다.** 이 장의 과제는 "처음 가는 곳"이니 이런 방문은 정답에서 빠집니다.
- **표는 거의 비어 있습니다.** 사용자 × 식당 120만 칸 중 한 번이라도 간 칸은 3.48%뿐이에요. 추천은 이 빈칸 대부분을 채워 넣는 일입니다.

21.3 시간 순서로 나누기

미래를 학습에 쓰지 않기

2장 2.9절에서 시간 순서가 있는 데이터는 **섞지 않고 시점으로 자른다**고 했습니다. 추천도 마찬가지예요. 실제 서비스는 오늘까지의 기록으로 내일을 추천하니까요. 무작위로 섞어 나누면 "다음 달에 문을 연 식당"이나 "다음 달에 가입한 사용자"의 기록이 학습에 섞여, 정작 서비스에서 가장 어려운 상황을 평가에서 볼 수 없게 됩니다. 2장 2.3절의 데이터 누수 그대로예요.

시간으로_나누기는 0~299일을 학습, 300~329일을 검증, 330~364일을 시험으로 자릅니다. 검증 기간으로 학습을 언제 멈출지와 어떤 모델이 좋은지를 정하고, 시험 기간은 마지막 비교에만 써요.

```
import pandas as pd

from 맛집데이터 import 데이터_만들기, 시간으로_나누기
from 순위지표 import 평가셋_만들기

데이터 = 데이터_만들기(씨앗=0)
학습, 검증, 시험 = 시간으로_나누기(데이터.방문)
for 이름, 조각 in [("학습", 학습), ("검증", 검증), ("시험", 시험)]:
    print(f"{이름}: {조각['날짜'].min():3}일 ~ {조각['날짜'].max():3}일, 방문 {len(조각):6,}번")

처음_보는_사용자 = set(시험["사용자"]) - set(학습["사용자"])
처음_보는_식당 = set(시험["식당"]) - set(학습["식당"])
print(f"시험 기간 사용자 중 학습 기록이 없는 사람: {len(처음_보는_사용자)}명")
print(f"시험 기간 식당 중 학습 기록이 없는 곳: {len(처음_보는_식당)}곳")

검증셋 = 평가셋_만들기(학습, 검증, 데이터.식당)
시험셋 = 평가셋_만들기(pd.concat([학습, 검증]), 시험, 데이터.식당)
for 이름, 셋 in [("검증", 검증셋), ("시험", 시험셋)]:
```

```
    정답_수 = sum(len(답) for 답 in 셋.정답)
    print(f"{이름} 평가: 사용자 {len(셋.사용자):,}명, 맞힐 식당 평균 {정답_수 / len(셋.사용
    ↪ 자):.2f}곳, "
          f"후보에서 빼는 칸 평균 {셋.제외.sum(axis=1).mean():.1f}곳")
```

```
학습:   0일 ~ 299일, 방문 46,963번
검증: 300일 ~ 329일, 방문  4,996번
시험: 330일 ~ 364일, 방문  6,371번
시험 기간 사용자 중 학습 기록이 없는 사람: 298명
시험 기간 식당 중 학습 기록이 없는 곳: 40곳
검증 평가: 사용자 1,412명, 맞힐 식당 평균 1.43곳, 후보에서 빼는 칸 평균 34.0곳
시험 평가: 사용자 1,707명, 맞힐 식당 평균 1.58곳, 후보에서 빼는 칸 평균 14.1곳
```

학습 기간이 전체 방문의 80%쯤이고 검증과 시험이 나머지를 나눠 가졌습니다. 시험 기간 사용자 중 298명은 학습 기록이 아예 없고, 시험 기간에 손님을 받은 식당 중 40곳도 학습 기간에는 없던 곳이에요. 무작위로 나눴다면 보이지 않았을 숫자입니다.

여러 사용자를 한꺼번에 채점하기

여기서 21.1절 `순위지표.py`의 뒷부분이 쓰입니다. `평가셋_만들기(이전, 대상, 식당)`는 이렇게 문제를 만들어요.

1. **누구를 채점하나**: `이전` 기간에 기록이 있고, `대상` 기간에 **처음 가서 좋았던** 식당이 하나라도 있는 사용자.
2. **정답**: 그 사용자가 대상 기간에 처음 가서 3점 이상을 준 식당들.
3. **후보에서 뺄 곳(`제외`)**: 이전 기간에 이미 가 본 식당과, 대상 기간이 끝날 때까지 문을 열지 않은 식당.

`평가하기(점수, 셋)`는 모델이 낸 (사용자 수 × 식당 수) 점수표를 받아, 뺄 칸을 `-inf`로 지우고 위에서부터 k개를 골라 두 지표의 평균을 냅니다. 모델이 무엇이든 점수표만 만들어 넘기면 같은 방법으로 채점되니, 기준선과 모델을 공정하게 비교할 수 있어요.

출력 마지막 두 줄을 보면 검증에서는 1,412명, 시험에서는 1,707명을 채점하고, 한 사람이 맞혀야 할 식당은 평균 1.5곳 남짓입니다. 후보에서 빼는 곳은 검증이 평균 34.0곳, 시험이 14.1곳인데, 검증에는 330일 이후에 문을 여는 식당 20곳이 더 빠지기 때문이에요. 나머지 14곳 남짓이 사용자마다 이미 가 본 식당입니다.

Q

Q. 시험할 때 검증 기간의 기록은 어떻게 쓰나요? 이 장에서는 모델 비교를 공정하게 하려고 모든 모델을 **학습 기간 기록으로만** 학습합니다. 다만 시험 문제를 만들 때는 검증 기간까지를 "이전 기록"으로 봐서, 검증 기간에 이미 간 곳을 다시 정답으로 세지 않게 했어요. 비교가 끝나고 서비스에 올릴 모델은 검증 기간까지 모두 써서 다시 학습합니다 (21.11절).

21.4 기준선부터: 인기순과 동네 인기순

모델을 만들기 전에 **간단한 방법이 몇 점을 내는지**부터 재 둡니다. 3장에서 평균만 찍는 기준선과 비교했던 것처럼요. 기준선이 없으면 모델이 0.14를 냈을 때 그게 좋은 건지 알 수가 없습니다.

- **무작위**: 아무렇게나 섞은 목록. 바닥 점수입니다.
- **인기순**: 이전 기간에 방문이 많았던 식당부터. 모든 사람에게 같은 목록이에요.
- **최근 30일 인기순**: 요즘 뜨는 곳을 반영하려고 마지막 30일만 셉니다.
- **동네 인기순**: 사용자가 사는 동네의 식당을 먼저, 그 안에서는 인기순. 가입할 때 적은 동네 하나만 쓰는 아주 단순한 개인화입니다.

```
import numpy as np
import pandas as pd

from 맛집데이터 import 데이터_만들기, 시간으로_나누기
from 순위지표 import 평가셋_만들기, 평가하기

데이터 = 데이터_만들기(씨앗=0)
학습, 검증, 시험 = 시간으로_나누기(데이터.방문)
셋들 = {"검증": 평가셋_만들기(학습, 검증, 데이터.식당),
       "시험": 평가셋_만들기(pd.concat([학습, 검증]), 시험, 데이터.식당)}
식당_지역 = 데이터.식당["지역"].to_numpy()
사용자_동네 = 데이터.사용자["동네"].to_numpy()


def 기준선_점수(이름: str, 이전: pd.DataFrame, 사용자: np.ndarray) -> np.ndarray:
    """(사용자 수, 식당 수) 점수표. 클수록 위에 추천한다."""
    인기 = np.bincount(이전["식당"], minlength=len(식당_지역)).astype(float)
    if 이름 == "무작위":
```

```
        return np.random.default_rng(0).random((len(사용자), len(식당_지역)))
    if 이름 == "최근 30일 인기순":
        최근 = 이전[이전["날짜"] >= 이전["날짜"].max() - 29]
        인기 = np.bincount(최근["식당"], minlength=len(식당_지역)).astype(float)
    점수 = np.tile(인기, (len(사용자), 1))
    if 이름 == "동네 인기순":           # 내 동네 식당을 먼저, 그 안에서는 인기순
        점수 += 1e6 * (식당_지역[None, :] == 사용자_동네[사용자][:, None])
    return 점수


print("검증 Recall@10  NDCG@10 ¦ 시험 Recall@10  NDCG@10 ¦ 방법")
for 이름 in ["무작위", "인기순", "최근 30일 인기순", "동네 인기순"]:
    칸 = []
    for 셋_이름, 셋 in 셋들.items():
        이전 = 학습 if 셋_이름 == "검증" else pd.concat([학습, 검증])
        결과 = 평가하기(기준선_점수(이름, 이전, 셋.사용자), 셋)
        칸.append(f"{결과['Recall@10']:14.4f} {결과['NDCG@10']:8.4f}")
    print(" ¦ ".join(칸), "¦", 이름)
```

```
검증 Recall@10  NDCG@10 ¦ 시험 Recall@10  NDCG@10 ¦ 방법
        0.0291   0.0146 ¦         0.0234   0.0120 ¦ 무작위
        0.0977   0.0524 ¦         0.0875   0.0487 ¦ 인기순
        0.0986   0.0528 ¦         0.0896   0.0476 ¦ 최근 30일 인기순
        0.1851   0.1021 ¦         0.1745   0.0946 ¦ 동네 인기순
```

표가 한글 폭 때문에 어긋나지 않게 방법 이름을 줄 끝에 두었습니다. 시험 NDCG@10을 보면 무작위 0.0120, 인기순 0.0487, 동네 인기순 0.0946입니다. 인기순만으로도 무작위의 네 배이고, 동네 하나를 더 썼을 뿐인데 다시 두 배가 됐어요. 최근 30일 인기순은 전체 인기순과 거의 같습니다. 이 예제 데이터에는 "요즘 뜨는 곳" 같은 유행을 넣지 않았기 때문이에요. 실제 데이터라면 차이가 날 수 있습니다.

이제 목표가 생겼습니다. 모델은 **동네 인기순(시험 NDCG@10 0.0946)을 넘어야** 만든 보람이 있어요. 이 숫자를 넘지 못하는 모델이라면, 복잡한 모델 대신 동네 인기순을 서비스하는 편이 낫습니다.

21.5 행렬 분해: 사용자와 식당을 화살표로

빈칸 많은 표를 작은 표 두 개로

21.2절의 사용자 × 식당 표를 다시 떠올려 보세요. 3,000줄 × 400칸 중 3.48%만 채워져 있었습니다. 이 표의 빈칸을 채우는 고전적인 방법이 **행렬 분해(matrix factorization)** 예요. 큰 표를 **작은 표 두 개의 곱**으로 나타냅니다.

- 사용자마다 숫자 32개짜리 화살표: 3,000 × 32 표
- 식당마다 숫자 32개짜리 화살표: 400 × 32 표
- 사용자 u와 식당 i의 점수 = 두 화살표의 **내적**

4장 4.6절의 행렬 곱으로 쓰면 `사용자표 @ 식당표.T`가 (3,000 × 400) 점수표 전체가 됩니다. 4장 4.11절에서 도시의 기온 패턴을 화살표로 보고 방향이 비슷한지 쟀던 것, 13장 13.7절에서 토큰마다 화살표를 하나씩 준 **임베딩**과 같은 생각이에요. 다만 이번에는 화살표의 숫자를 사람이 정하지 않고, "간 곳의 점수는 높게"를 맞추도록 학습합니다. 학습이 끝나면 매운 걸 좋아하는 사용자의 화살표와 매운 식당의 화살표가 비슷한 쪽을 가리키게 될 거라고 기대하는 거죠. 번호로 화살표를 꺼내는 표는 11장 11.11절에서 처음 만난 `nn.Embedding`이 그대로 맡습니다.

안 간 곳은 싫어하는 곳일까요?

학습 데이터에는 "간 곳"만 있습니다. 그런데 모델에게 "간 곳은 높게"만 가르치면, 모든 식당에 높은 점수를 주는 게으른 답이 정답이 돼요. "안 간 곳은 낮게"도 알려 줘야 합니다.

문제는 안 간 곳이 곧 싫어하는 곳은 아니라는 점입니다. 몰라서 못 갔을 수도, 아직 기회가 없었을 수도 있어요. 그래도 360곳 중 무작위로 하나를 고르면, 그곳은 대개 그 사람이 별로 끌리지 않는 곳일 겁니다. 그래서 배치마다 사용자의 "간 곳" 하나에 **무작위로 뽑은 식당 하나**를 짝지어 "이쪽은 낮게"라고 가르칩니다. 이를 **음성 샘플링(negative sampling)** 이라고 해요. 가끔은 사용자가 좋아할 곳이 음성으로 뽑히기도 하지만, 그런 잡음은 감수합니다. 음성 후보는 학습 기간에 문을 연 360곳으로 한정했어요. 아직 문을 열지 않은 식당을 "안 간 곳"으로 가르치면 안 되니까요.

학습 도구

모델과 학습 루프를 `추천학습.py`로 저장하세요.

```
"""21장 추천 모델 학습 도구: 행렬 분해 모델, 음성 샘플링, BCE·BPR 손실, 학습 루프."""
import copy
```

```
from collections.abc import Callable
from dataclasses import dataclass

import pandas as pd
import torch
from torch import nn

from 순위지표 import 좋은_평점


@dataclass(frozen=True)
class 학습설정:
    손실: str = "bpr"           # "bce" 또는 "bpr"
    차원: int = 32
    에폭: int = 50
    배치_크기: int = 1024
    학습률: float = 0.005
    가중치_감쇠: float = 1e-4
    씨앗: int = 0


class 내적추천(nn.Module):
    """점수 = 사용자 벡터 · 식당 벡터. 행렬 분해와 투-타워가 함께 쓰는 틀."""

    def forward(self, 사용자: torch.Tensor, 식당: torch.Tensor) -> torch.Tensor:
        return (self.사용자_벡터(사용자) * self.식당_벡터(식당)).sum(dim=-1)

    def 모든_점수(self, 사용자: torch.Tensor) -> torch.Tensor:
        """사용자들 × 모든 식당의 점수표."""
        return self.사용자_벡터(사용자) @ self.식당_벡터(torch.arange(self.식당_수)).T


class 행렬분해(내적추천):
    def __init__(self, 사용자_수: int, 식당_수: int, 차원: int = 32) -> None:
        super().__init__()
        self.식당_수 = 식당_수
```

```
        self.사용자 = nn.Embedding(사용자_수, 차원)
        self.식당 = nn.Embedding(식당_수, 차원)
        self.편향 = nn.Embedding(식당_수, 1)            # 식당마다 '누구에게나 끌리는 정도'
        nn.init.normal_(self.사용자.weight, std=0.1)
        nn.init.normal_(self.식당.weight, std=0.1)
        nn.init.zeros_(self.편향.weight)

    def 사용자_벡터(self, 사용자: torch.Tensor) -> torch.Tensor:
        u = self.사용자(사용자)
        return torch.cat([u, torch.ones_like(u[..., :1])], dim=-1)    # 끝에 1을 붙이면

    def 식당_벡터(self, 식당: torch.Tensor) -> torch.Tensor:
        return torch.cat([self.식당(식당), self.편향(식당)], dim=-1)   # 내적에 편향이 더해진다


def 좋았던_쌍(방문: pd.DataFrame) -> torch.Tensor:
    """좋았던 방문을 (사용자, 식당) 짝으로. 같은 곳을 여러 번 가도 한 번만 센다."""
    좋았던 = 방문[방문["평점"] >= 좋은_평점][["사용자", "식당"]].drop_duplicates()
    return torch.tensor(좋았던.to_numpy())


def 손실_계산(모델: 내적추천, 사용자: torch.Tensor, 양성: torch.Tensor,
              음성: torch.Tensor, 종류: str) -> torch.Tensor:
    양성_점수, 음성_점수 = 모델(사용자, 양성), 모델(사용자, 음성)
    if 종류 == "bce":     # 간 곳은 1, 무작위로 뽑은 곳은 0으로 맞히는 분류 문제
        점수 = torch.cat([양성_점수, 음성_점수])
        정답 = torch.cat([torch.ones_like(양성_점수), torch.zeros_like(음성_점수)])
        return nn.functional.binary_cross_entropy_with_logits(점수, 정답)
    # bpr: 간 곳의 점수가 무작위로 뽑은 곳보다 높기만 하면 된다
    return -nn.functional.logsigmoid(양성_점수 - 음성_점수).mean()


def 학습하기(모델: 내적추천, 쌍: torch.Tensor, 후보_식당: torch.Tensor, 설정: 학습설정,
             검증: Callable[[내적추천], float] | None = None) -> list[dict]:
    """미니배치마다 음성 식당을 무작위로 뽑아 학습한다. 검증 점수가 가장 좋던 에폭으로 되돌린다."""
    생성기 = torch.Generator().manual_seed(설정.씨앗)
```

```
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=설정.학습률, weight_decay=설정.가중치_감쇠)
    기록, 최고, 최고_상태 = [], -1.0, None
    for 에폭 in range(1, 설정.에폭 + 1):
        모델.train()
        순서 = torch.randperm(len(쌍), generator=생성기)
        합계 = 0.0
        for 시작 in range(0, len(쌍), 설정.배치_크기):
            u, 양성 = 쌍[순서[시작:시작 + 설정.배치_크기]].T
            음성 = 후보_식당[torch.randint(len(후보_식당), (len(u),), generator=생성기)]
            손실 = 손실_계산(모델, u, 양성, 음성, 설정.손실)
            옵티마이저.zero_grad()
            손실.backward()
            옵티마이저.step()
            합계 += 손실.item() * len(u)
        모델.eval()
        점수 = 검증(모델) if 검증 else float("nan")
        기록.append({"에폭": 에폭, "손실": 합계 / len(쌍), "검증": 점수})
        if 점수 > 최고:
            최고, 최고_상태 = 점수, copy.deepcopy(모델.state_dict())
    if 최고_상태 is not None:
        모델.load_state_dict(최고_상태)          # 조기 종료처럼 가장 좋던 때로 (8장 8.8절)
    return 기록
```

- **내적추천**: 점수를 "사용자 벡터 · 식당 벡터"로 내는 모델들의 공통 틀입니다. `forward`는 짝 하나하나의 점수를, **모든_점수**는 사용자들 × 모든 식당의 점수표를 냅니다. 21.8절의 투-타워도 이 틀을 물려받아요.
- **편향 붙이기**: **행렬분해**에는 식당마다 숫자 하나짜리 **편향**을 두었습니다. "누구에게나 끌리는 정도", 즉 인기를 담는 칸이에요. 사용자 벡터 끝에 1을, 식당 벡터 끝에 편향을 붙이면 내적 = 화살표끼리의 내적 + 편향이 됩니다. 점수가 언제나 순수한 내적 하나로 나오니, 21.11절에서 서빙할 때 계산이 단순해져요.
- **좋았던_쌍**: 3점 이상 받은 방문을 (사용자, 식당) 짝으로 만들고, 같은 곳을 여러 번 간 기록은 한 번만 셉니다.
- **학습하기**: 6장 6.7절의 학습 루프에 음성 샘플링을 더한 모양입니다. 섞는 순서와 음성 뽑기는 4장 4.1절의 나만 쓰는 `torch.Generator`로 고정했고, Adam의 `weight_decay`(8장 8.2절)로 화살표가 너무 길어지지 않게 했어요. 에폭마다 검증 점수를 재서 가장 좋던 때의 `state_dict`를 기억했다가 끝에 되돌립니다. 8장 8.8절의 조기 종료에서 멈추는 부분만 뺀 것

이에요.

- **손실_계산**: 지금은 앞쪽의 "bce"만 봅니다. 간 곳은 1, 뽑은 곳은 0으로 맞히는 이진 분류이고, 6장 6.5절의 BCEWithLogits와 같은 손실이에요. 뒤쪽 "bpr"은 다음 절에서 봅니다.

이제 학습해 보겠습니다.

```
import numpy as np
import torch

from 맛집데이터 import 데이터_만들기, 시간으로_나누기
from 순위지표 import 평가셋_만들기, 평가하기
from 추천학습 import 행렬분해, 학습설정, 학습하기, 좋았던_쌍

데이터 = 데이터_만들기(씨앗=0)
학습, 검증, 시험 = 시간으로_나누기(데이터.방문)
검증셋 = 평가셋_만들기(학습, 검증, 데이터.식당)

쌍 = 좋았던_쌍(학습)                                          # (사용자, 식당) 짝
후보_식당 = torch.tensor(np.flatnonzero(데이터.식당["개업일"] < 300))  # 음성으로 뽑을 식당
print("좋았던 짝:", tuple(쌍.shape), "음성 후보:", len(후보_식당), "곳")

torch.manual_seed(0)
모델 = 행렬분해(len(데이터.사용자), len(데이터.식당), 차원=32)
print("파라미터:", f"{sum(p.numel() for p in 모델.parameters()):,}개")


def 검증_NDCG(모델: 행렬분해) -> float:
    with torch.no_grad():
        점수 = 모델.모든_점수(torch.tensor(검증셋.사용자)).numpy()
    return 평가하기(점수, 검증셋)["NDCG@10"]


기록 = 학습하기(모델, 쌍, 후보_식당, 학습설정(손실="bce"), 검증_NDCG)
for 줄 in [기록[0]] + 기록[9::10]:
    print(f"에폭 {줄['에폭']:2}: 손실 {줄['손실']:.4f}, 검증 NDCG@10 {줄['검증']:.4f}")
최고 = max(기록, key=lambda 줄: 줄["검증"])
print(f"가장 좋았던 에폭 {최고['에폭']}: 검증 NDCG@10 {최고['검증']:.4f}")
```

```
좋았던 짝: (27006, 2) 음성 후보: 360 곳
파라미터: 109,200개
에폭  1: 손실 0.6866, 검증 NDCG@10 0.0443
에폭 10: 손실 0.6001, 검증 NDCG@10 0.1003
에폭 20: 손실 0.5455, 검증 NDCG@10 0.1218
에폭 30: 손실 0.5304, 검증 NDCG@10 0.1302
에폭 40: 손실 0.5224, 검증 NDCG@10 0.1305
에폭 50: 손실 0.5198, 검증 NDCG@10 0.1312
가장 좋았던 에폭 49: 검증 NDCG@10 0.1337
```

좋았던 짝은 27,006개이고, 파라미터는 사용자 3,000 × 32 + 식당 400 × 32 + 편향 400 = 109,200개입니다. 학습은 CPU에서 몇 초면 끝나요. 검증 NDCG@10은 1에폭의 0.0443에서 시작해 49에폭에 0.1337까지 올랐습니다. 21.4절의 동네 인기순은 검증에서 0.1021이었으니 넘어서긴 했어요. 그런데 손실이 0.52 근처에서 거의 줄지 않고, 검증 점수도 30에폭쯤부터는 제자리걸음입니다. 더 나은 방법이 없을까요?

21.6 순위를 직접 배우기: BPR

BCE 손실은 모델에게 "간 곳은 확률 1, 뽑은 곳은 확률 0"이라는 **절대적인 답**을 요구합니다. 그런데 우리가 원하는 건 순위예요. "이 사람에게 A가 B보다 위"이기만 하면 점수가 3이든 30이든 상관없습니다. 시험에 비유하면 BCE는 모든 과목의 점수를 정확히 맞히라는 것이고, 우리가 원하는 건 등수만 맞히는 거예요.

BPR(Bayesian Personalized Ranking) 은 이 생각을 그대로 손실로 옮겼습니다. 같은 사용자의 간 곳(양성)과 뽑은 곳(음성)을 짝지어 `-log sigmoid(양성 점수 - 음성 점수)`를 줄입니다. 두 점수의 **차이**만 보니, 양성이 음성보다 충분히 위에 있으면 손실이 0에 가까워지고, 음성이 더 위에 있으면 크게 벌을 받아요. 시그모이드와 로그를 함께 계산하는 `logsigmoid`를 쓴 것은 6장 6.5절에서 `BCEWithLogits`가 넘침을 막던 것과 같은 이유입니다. `추천학습.py`의 `손실_계산`에서 마지막 줄이 BPR이에요. 같은 씨앗으로 두 손실을 비교해 보겠습니다.

```
import numpy as np
import pandas as pd
import torch

from 맛집데이터 import 데이터_만들기, 시간으로_나누기
```

```
from 순위지표 import 평가셋_만들기, 평가하기
from 추천학습 import 행렬분해, 학습설정, 학습하기, 좋았던_쌍

데이터 = 데이터_만들기(씨앗=0)
학습, 검증, 시험 = 시간으로_나누기(데이터.방문)
검증셋 = 평가셋_만들기(학습, 검증, 데이터.식당)
시험셋 = 평가셋_만들기(pd.concat([학습, 검증]), 시험, 데이터.식당)
쌍 = 좋았던_쌍(학습)
후보_식당 = torch.tensor(np.flatnonzero(데이터.식당["개업일"] < 300))


def 점수표(모델: 행렬분해, 사용자: np.ndarray) -> np.ndarray:
    with torch.no_grad():
        return 모델.모든_점수(torch.tensor(사용자)).numpy()


기록들, 시험_결과 = {}, {}
for 손실 in ["bce", "bpr"]:
    torch.manual_seed(0)
    모델 = 행렬분해(len(데이터.사용자), len(데이터.식당), 차원=32)
    기록들[손실] = 학습하기(모델, 쌍, 후보_식당, 학습설정(손실=손실),
                      lambda m: 평가하기(점수표(m, 검증셋.사용자), 검증셋)["NDCG@10"])
    시험_결과[손실] = 평가하기(점수표(모델, 시험셋.사용자), 시험셋)

print("에폭   BCE 검증 NDCG   BPR 검증 NDCG")
for 에폭 in [1, 5, 10, 20, 30, 40, 50]:
    print(f"{에폭:4} {기록들['bce'][에폭 - 1]['검증']:15.4f} {기록들['bpr'][에폭 - 1]['검
    ↪ 증']:15.4f}")
for 손실, 결과 in 시험_결과.items():
    print(f"{손실.upper()} 시험: Recall@10 {결과['Recall@10']:.4f}, NDCG@10 {결과['NDCG@10']:.4f}")
```

```
에폭   BCE 검증 NDCG   BPR 검증 NDCG
   1          0.0443          0.0424
   5          0.0796          0.0905
  10          0.1003          0.1461
  20          0.1218          0.1759
```

```
  30          0.1302          0.1813
  40          0.1305          0.1776
  50          0.1312          0.1802
BCE 시험: Recall@10 0.1783, NDCG@10 0.1082
BPR 시험: Recall@10 0.2310, NDCG@10 0.1409
```

같은 모델, 같은 씨앗, 같은 음성 뽑기인데 손실만 바꿨더니 BPR이 5에폭부터 앞서기 시작해 30에폭에 0.18을 넘었습니다. BCE는 50에폭이 지나도 0.13 근처예요. 시험 기간에서도 BPR의 NDCG@10이 0.1409로 BCE의 0.1082보다 30% 높고, 동네 인기순(0.0946)보다는 49% 높습니다. 반면 BCE는 시험에서 동네 인기순을 겨우 넘었어요. 손실 하나가 "기준선과 비슷한 모델"과 "확실히 나은 모델"을 갈랐습니다.

BCE가 불리한 이유는 BCE가 맞히려는 문제가 우리 목표와 다르기 때문입니다. BCE는 모든 사용자의 점수를 "0이냐 1이냐"라는 같은 눈금에 맞추느라 힘을 씁니다. 사람마다 다른 점수의 높낮이는 순위에 아무 영향이 없는데도요. BPR은 그 힘을 "누가 누구보다 위인가"에만 쓰니 순위 지표가 더 좋아집니다. 손실을 고를 때는 **평가 지표와 같은 것을 가르치는 손실**을 고르라는 교훈이에요. 이제부터는 BPR을 기본으로 씁니다. 다만 한 씨앗의 결과이니, 21.10절에서 씨앗을 바꿔 다시 확인하겠습니다.

21.7 학습된 임베딩 들여다보기

비슷한 식당 찾기

학습이 끝난 식당 화살표에는 무엇이 담겼을까요? 13장 13.7절에서 KLUE BERT의 임베딩 표로 닮은 토큰을 찾은 것처럼, 코사인 유사도로 닮은 식당을 찾아보겠습니다. 이번에는 정답지(숨은 열)가 있으니 이웃들의 숨은 매운맛과 분위기도 함께 찍어요. 같은 동네 안에서 찾는 것과, 다른 동네에서 찾는 것을 나눠 봤습니다. "성수에서 즐겨 가던 곳과 비슷한 곳을 홍대에서 찾기" 같은 기능을 떠올리면 됩니다.

```
import numpy as np
import torch

from 맛집데이터 import 데이터_만들기, 시간으로_나누기
from 추천학습 import 행렬분해, 학습설정, 학습하기, 좋았던_쌍
```

```
데이터 = 데이터_만들기(씨앗=0)
학습, _, _ = 시간으로_나누기(데이터.방문)
식당 = 데이터.식당
torch.manual_seed(0)
모델 = 행렬분해(len(데이터.사용자), len(식당), 차원=32)
학습하기(모델, 좋았던_쌍(학습), torch.tensor(np.flatnonzero(식당["개업일"] < 300)),
        학습설정(손실="bpr"))

벡터 = 모델.식당.weight.detach()                          # (식당 수, 32)
단위 = 벡터 / 벡터.norm(dim=1, keepdim=True)              # 4장 4.11절, 13장 13.7절처럼
아는_식당 = torch.tensor((식당["개업일"] < 300).to_numpy())  # 학습 기간에 연 식당만


def 보여_주기(번호들: list[int], 유사도: torch.Tensor) -> None:
    for i in 번호들:
        줄 = 식당.loc[i]
        print(f"  유사도 {유사도[i]:.2f} ¦ 가격대 {줄['가격대']} ¦ 매운맛 {줄['숨은_매운맛']:.2f} ¦
        ↪ "
              f"분위기 {줄['숨은_분위기']:+.2f} ¦ {줄['이름']}")


for 기준 in [17, 5]:
    줄 = 식당.loc[기준]
    print(f"[기준] 가격대 {줄['가격대']} ¦ 매운맛 {줄['숨은_매운맛']:.2f} ¦ "
          f"분위기 {줄['숨은_분위기']:+.2f} ¦ {줄['이름']}")
    유사도 = 단위 @ 단위[기준]
    유사도[기준] = -1                                      # 자기 자신은 빼고
    유사도[~아는_식당] = -1                                # 학습 기간에 없던 식당도 빼고
    print(" 가장 비슷한 곳")
    보여_주기(유사도.topk(3).indices.tolist(), 유사도)
    다른_동네 = 유사도.clone()
    다른_동네[torch.tensor((식당["지역"] == 줄["지역"]).to_numpy())] = -1
    print(" 다른 동네에서 가장 비슷한 곳")
    보여_주기(다른_동네.topk(3).indices.tolist(), 유사도)
```

```
[기준] 가격대 1 ¦ 매운맛 0.72 ¦ 분위기 +0.37 ¦ 종로 분식 017
 가장 비슷한 곳
  유사도 0.86 ¦ 가격대 1 ¦ 매운맛 0.68 ¦ 분위기 +0.74 ¦ 종로 분식 035
  유사도 0.86 ¦ 가격대 2 ¦ 매운맛 0.55 ¦ 분위기 +0.93 ¦ 종로 분식 140
  유사도 0.82 ¦ 가격대 3 ¦ 매운맛 0.61 ¦ 분위기 +0.60 ¦ 종로 분식 334
 다른 동네에서 가장 비슷한 곳
  유사도 0.57 ¦ 가격대 2 ¦ 매운맛 0.94 ¦ 분위기 +0.13 ¦ 홍대 분식 263
  유사도 0.54 ¦ 가격대 2 ¦ 매운맛 0.84 ¦ 분위기 +0.49 ¦ 이태원 분식 049
  유사도 0.52 ¦ 가격대 1 ¦ 매운맛 0.66 ¦ 분위기 +0.54 ¦ 잠실 분식 066
[기준] 가격대 3 ¦ 매운맛 0.49 ¦ 분위기 +0.76 ¦ 잠실 한식 005
 가장 비슷한 곳
  유사도 0.90 ¦ 가격대 1 ¦ 매운맛 0.71 ¦ 분위기 +0.56 ¦ 잠실 한식 330
  유사도 0.82 ¦ 가격대 2 ¦ 매운맛 0.61 ¦ 분위기 -0.13 ¦ 잠실 한식 027
  유사도 0.80 ¦ 가격대 1 ¦ 매운맛 0.34 ¦ 분위기 -0.00 ¦ 잠실 한식 188
 다른 동네에서 가장 비슷한 곳
  유사도 0.60 ¦ 가격대 1 ¦ 매운맛 0.51 ¦ 분위기 +0.81 ¦ 이태원 한식 107
  유사도 0.60 ¦ 가격대 3 ¦ 매운맛 0.77 ¦ 분위기 +0.77 ¦ 종로 한식 284
  유사도 0.58 ¦ 가격대 1 ¦ 매운맛 0.88 ¦ 분위기 +0.25 ¦ 여의도 한식 305
```

맵고 활기찬 "종로 분식 017"과 가장 비슷한 곳은 모두 종로의 분식집입니다. 같은 동네 사람들이 같은 날 찾는 곳이니 화살표가 가까워지는 게 당연해요. 흥미로운 건 **다른 동네** 쪽입니다. 모델은 식당의 음식 종류를 본 적이 없는데도(행렬 분해는 번호만 봅니다) 홍대·이태원·잠실의 **분식집**을 찾아왔고, 셋 모두 매운맛이 0.66~0.94로 맵고 분위기도 활기찬 쪽(+)입니다. 비싸고 시끌벅적한 한식집 "잠실 한식 005"의 다른 동네 이웃도 모두 한식이고, 분위기가 모두 시끌벅적한 쪽(+)이에요.

임베딩은 숨은 성질을 배웠을까요?

몇 개만 골라 보면 우연일 수도 있습니다. 숫자로 확인해 보겠습니다. 식당 화살표 32개 숫자로 숨은 성질을 **직선 하나로** 맞힐 수 있는지 재는 거예요. 화살표 안에 그 성질의 "방향"이 있다면 직선으로 잘 맞힐 겁니다. 짝수 번호 식당으로 직선을 맞추고 홀수 번호 식당에서 3장 3.8절의 R^2를 쟀습니다. 비교 대상은 모델이 볼 수 있는 특징(지역·음식 원-핫과 가격대)으로 맞혔을 때예요. 이렇게 학습된 표현에 단순한 직선 모델을 얹어 무엇이 담겼는지 확인하는 방법을 **선형 탐침**(linear probe) 이라고 합니다. `torch.linalg.lstsq`는 최소 제곱으로 직선을 맞춰 주는 함수예요.

```
import numpy as np
import torch

from 맛집데이터 import 데이터_만들기, 시간으로_나누기
from 추천학습 import 행렬분해, 학습설정, 학습하기, 좋았던_쌍

데이터 = 데이터_만들기(씨앗=0)
학습, _, _ = 시간으로_나누기(데이터.방문)
식당 = 데이터.식당
torch.manual_seed(0)
모델 = 행렬분해(len(데이터.사용자), len(식당), 차원=32)
학습하기(모델, 좋았던_쌍(학습), torch.tensor(np.flatnonzero(식당["개업일"] < 300)),
        학습설정(손실="bpr"))


def 설명력(X: torch.Tensor, y: torch.Tensor, 배울: np.ndarray, 볼: np.ndarray) -> float:
    """배울 식당들로 X → y 직선을 맞추고(최소 제곱), 볼 식당들에서 R²를 잰다(3장 3.8절)."""
    X = torch.cat([X, torch.ones(len(X), 1)], dim=1)       # 절편 칸
    w = torch.linalg.lstsq(X[배울], y[배울, None]).solution
    오차 = y[볼] - (X[볼] @ w)[:, 0]
    return (1 - 오차.pow(2).sum() / (y[볼] - y[볼].mean()).pow(2).sum()).item()


아는 = np.flatnonzero(식당["개업일"] < 300)
배울, 볼 = 아는[아는 % 2 == 0], 아는[아는 % 2 == 1]              # 짝수 번호로 배우고 홀수로 확인
원핫 = torch.nn.functional.one_hot
보이는_특징 = torch.cat([원핫(torch.tensor(식당["지역"].to_numpy()), 8),
                    원핫(torch.tensor(식당["음식"].to_numpy()), 8),
                    torch.tensor(식당["가격대"].to_numpy())[:, None]], dim=1).float()
임베딩 = 모델.식당.weight.detach()
편향 = 모델.편향.weight.detach()

print("보이는 특징  학습된 임베딩  임베딩+편향 ¦ 맞힐 숨은 성질")
for 열 in ["숨은_매운맛", "숨은_분위기", "숨은_품질"]:
    y = torch.tensor(식당[열].to_numpy()).float()
    결과 = [설명력(X, y, 배울, 볼) for X in (보이는_특징, 임베딩, torch.cat([임베딩, 편향], dim=1))]
    print(f"{결과[0]:11.3f} {결과[1]:14.3f} {결과[2]:12.3f} ¦ {열}")
```

```
방문수 = np.bincount(학습["식당"], minlength=len(식당))[아는]
print(f"편향과 log(방문 수)의 상관계수: {np.corrcoef(편향[아는, 0], np.log1p(방문수))[0, 1]:.3f}")
```

```
보이는 특징  학습된 임베딩  임베딩+편향 ┆ 맞힐 숨은 성질
      0.452         0.766        0.766 ┆ 숨은_매운맛
     -0.124         0.847        0.846 ┆ 숨은_분위기
     -0.133         0.068        0.796 ┆ 숨은_품질
편향과 log(방문 수)의 상관계수: 0.987
```

결과가 꽤 놀랍습니다.

- **분위기**: 보이는 특징으로는 R^2가 -0.124, 아무것도 못 맞힙니다. 분위기는 지역·음식·가격과 상관없이 정했으니까요. 그런데 학습된 임베딩으로는 0.847입니다. 모델은 "분위기"라는 말을 들어 본 적도 없는데, 사람들의 방문 기록만 보고 화살표 안에 분위기 축을 만들어 냈어요.
- **매운맛**: 음식 종류가 매운 정도를 어느 정도 알려 주니(분식·중식은 맵다) 보이는 특징으로도 0.452가 나오지만, 임베딩은 0.766으로 그보다 훨씬 잘 맞힙니다. 같은 분식집 안에서도 더 매운 곳과 덜 매운 곳의 차이까지 배웠다는 뜻이에요.
- **품질**: 화살표만으로는 0.068로 거의 못 맞히는데, 편향을 더하면 0.796으로 뜁니다. 품질은 "누구에게나 끌리는 정도"라서 사람마다 다른 취향을 담는 화살표가 아니라 편향 칸에 들어갔어요. 마지막 줄처럼 편향은 방문 수의 로그와 0.987의 상관을 보입니다. 21.5절에서 "편향은 인기를 담는 칸"이라고 한 것이 그대로 확인됐어요.

진짜 데이터에는 이런 정답지가 없으니 이렇게 깔끔하게 확인할 수는 없습니다. 그래도 "가격대처럼 알고 있는 성질이 임베딩에 담겼나"를 선형 탐침으로 재 보는 것은 실무에서도 모델을 이해하는 좋은 방법이에요.

21.8 특징을 함께 쓰는 투-타워 모델

번호만 아는 모델의 한계

행렬 분해는 사용자와 식당을 **번호로만** 압니다. 학습 기간에 한 번도 방문 기록이 없는 식당은 화살표가 학습되지 않으니, 21.3절의 새 식당 40곳에는 아무 말도 할 수 없어요. 그런데 새 식당이라도 "성수에 있는 매운 중식집, 가격대 1"이라는 정보는 문을 여는 날부터 압니다. 이 정보를 쓸 수 없을까요?

투-타워(two-tower) 모델은 사용자와 식당에 탑을 하나씩 세웁니다. 사용자 탑은 사용자 번호와 동네를, 식당 탑은 식당 번호와 지역·음식·가격대를 받아 각자 화살표 하나를 만들고, 점수는 행렬 분해처럼 두 화살표의 내적이에요. 탑 안은 6장의 작은 신경망(`nn.Linear` - ReLU - `nn.Linear`)입니다. 두 탑이 서로를 보지 않고 각자 화살표를 만든다는 점이 중요한데, 그 덕분에 21.11절에서 화살표를 미리 계산해 둘 수 있어요.

번호 가리기

특징을 넣는다고 끝이 아닙니다. 학습할 때 번호가 늘 함께 들어가면 모델은 편한 번호에만 기대고 특징은 무시할 수 있어요. 그러면 번호 없이 특징만 들어오는 새 식당에서는 엉뚱한 답이 나옵니다. 그래서 학습 중에 30%의 확률로 번호를 **"처음 보는 번호"(표의 맨 끝 줄)로 바꿔 넣습니다.** 8장 8.3절의 드롭아웃을 번호에 적용한 셈이에요. 모델은 번호가 가려진 경우에도 맞히려고 특징에서 쓸 만한 정보를 뽑아내게 됩니다. 평가 모드(`eval()`)에서는 가리지 않아요.

```
"""21장 투-타워 모델: 사용자 탑과 식당 탑이 각자 번호와 특징으로 벡터를 만든다."""
import pandas as pd
import torch
from torch import nn

from 맛집데이터 import 음식들, 지역들
from 추천학습 import 내적추천


class 투타워(내적추천):
    def __init__(self, 사용자: pd.DataFrame, 식당: pd.DataFrame, 차원: int = 32,
                 번호_가리기: float = 0.3) -> None:
        super().__init__()
        self.식당_수, self.번호_가리기 = len(식당), 번호_가리기
        self.모르는_사용자, self.모르는_식당 = len(사용자), len(식당)     # 맨 끝 번호 = '처음 보는'
        # 모델이 볼 수 있는 특징: 사용자는 동네, 식당은 지역·음식·가격대 (숨은_ 열은 쓰지 않는다)
        self.register_buffer("동네", torch.tensor(사용자["동네"].to_numpy()))
        self.register_buffer("식당_특징", torch.tensor(식당[["지역", "음식", "가격
        ↪ 대"]].to_numpy()))
        self.사용자_번호 = nn.Embedding(len(사용자) + 1, 차원)
        self.식당_번호 = nn.Embedding(len(식당) + 1, 차원)
        self.동네_표 = nn.Embedding(len(지역들), 차원)
```

```
        self.지역_표 = nn.Embedding(len(지역들), 차원)
        self.음식_표 = nn.Embedding(len(음식들), 차원)
        self.가격_표 = nn.Embedding(4, 차원)
        self.사용자_탑 = nn.Sequential(nn.Linear(2 * 차원, 2 * 차원), nn.ReLU(), nn.Linear(2 * 차
        ↪ 원, 차원))
        self.식당_탑 = nn.Sequential(nn.Linear(4 * 차원, 2 * 차원), nn.ReLU(), nn.Linear(2 * 차원,
        ↪ 차원))

    def 가리기(self, 번호: torch.Tensor, 모르는_번호: int) -> torch.Tensor:
        """학습 중에는 일부 번호를 '처음 보는' 번호로 바꿔, 특징만으로도 맞히게 훈련한다."""
        if not self.training:
            return 번호
        return torch.where(torch.rand(번호.shape) < self.번호_가리기, 모르는_번호, 번호)

    def 사용자_벡터(self, 사용자: torch.Tensor, 번호_쓰기: bool = True) -> torch.Tensor:
        번호 = self.가리기(사용자, self.모르는_사용자) if 번호_쓰기 else torch.full_like(사용자,
↪ self.모르는_사용자)
        return self.사용자_탑(torch.cat([self.사용자_번호(번호), self.동네_표(self.동네[사용자])],
        ↪ dim=-1))

    def 식당_벡터(self, 식당: torch.Tensor, 번호_쓰기: bool = True) -> torch.Tensor:
        번호 = self.가리기(식당, self.모르는_식당) if 번호_쓰기 else torch.full_like(식당, self.모
↪ 르는_식당)
        지역, 음식, 가격 = self.식당_특징[식당].unbind(dim=-1)
        return self.식당_탑(torch.cat([self.식당_번호(번호), self.지역_표(지역),
                                    self.음식_표(음식), self.가격_표(가격)], dim=-1))
```

`register_buffer`는 학습하지 않는 텐서(여기서는 사용자의 동네와 식당의 특징 표)를 모델에 달아 두는 방법입니다. 파라미터는 아니지만 `state_dict`에 함께 저장되고 `.to(장치)`로 함께 옮겨져요. `번호_쓰기=False`를 주면 진짜 번호 대신 "처음 보는 번호"를 넣어, 특징만으로 화살표를 만듭니다. 21.9절의 새 식당과 새 사용자에게 쓸 길이에요.

같은 조건(BPR, 60에폭, 씨앗 0)에서 행렬 분해와 비교해 보겠습니다.

```
import numpy as np
import pandas as pd
import torch
```

```
from 맛집데이터 import 데이터_만들기, 시간으로_나누기
from 순위지표 import 평가셋_만들기, 평가하기
from 추천학습 import 내적추천, 행렬분해, 학습설정, 학습하기, 좋았던_쌍
from 투타워 import 투타워

데이터 = 데이터_만들기(씨앗=0)
학습, 검증, 시험 = 시간으로_나누기(데이터.방문)
검증셋 = 평가셋_만들기(학습, 검증, 데이터.식당)
시험셋 = 평가셋_만들기(pd.concat([학습, 검증]), 시험, 데이터.식당)
쌍 = 좋았던_쌍(학습)
후보_식당 = torch.tensor(np.flatnonzero(데이터.식당["개업일"] < 300))


def 점수표(모델: 내적추천, 사용자: np.ndarray) -> np.ndarray:
    with torch.no_grad():
        return 모델.모든_점수(torch.tensor(사용자)).numpy()


모델들 = {"행렬 분해": lambda: 행렬분해(len(데이터.사용자), len(데이터.식당)),
        "투-타워": lambda: 투타워(데이터.사용자, 데이터.식당)}
for 이름, 만들기 in 모델들.items():
    torch.manual_seed(0)
    모델 = 만들기()
    기록 = 학습하기(모델, 쌍, 후보_식당, 학습설정(손실="bpr", 에폭=60),
                lambda m: 평가하기(점수표(m, 검증셋.사용자), 검증셋)["NDCG@10"])
    최고 = max(기록, key=lambda 줄: 줄["검증"])
    결과 = 평가하기(점수표(모델, 시험셋.사용자), 시험셋)
    print(f"{이름}: 파라미터 {sum(p.numel() for p in 모델.parameters()):,}개, "
          f"검증 곡선 {[round(기록[i]['검증'], 3) for i in (0, 9, 19, 39, 59)]}")
    print(f"   가장 좋던 에폭 {최고['에폭']} → 시험 Recall@10 {결과['Recall@10']:.4f}, "
          f"NDCG@10 {결과['NDCG@10']:.4f}")
```

```
행렬 분해: 파라미터 109,200개, 검증 곡선 [0.042, 0.146, 0.176, 0.178, 0.182]
   가장 좋던 에폭 57 → 시험 Recall@10 0.2365, NDCG@10 0.1451
투-타워: 파라미터 126,336개, 검증 곡선 [0.077, 0.106, 0.128, 0.158, 0.162]
```

```
가장 좋던 에폭 55 → 시험 Recall@10 0.2421, NDCG@10 0.1392
```

투-타워는 탑 안의 신경망만큼 파라미터가 조금 더 많습니다(126,336개). 검증 곡선을 보면 1에폭부터 0.077로 행렬 분해(0.042)보다 높게 출발해요. 첫 에폭부터 "같은 동네, 같은 음식"이라는 특징을 쓸 수 있으니까요. 그런데 끝은 행렬 분해가 조금 앞섭니다. 시험 NDCG@10은 행렬 분해 0.1451, 투-타워 0.1392이고(행렬 분해가 21.6절의 0.1409와 다른 것은 이번에 60에폭으로 늘려 학습했기 때문이에요), Recall@10은 거꾸로 투-타워가 조금 높아요. 이미 기록이 충분한 사용자와 식당에게는 특징이 번호 이상의 정보를 주지 못한 셈입니다. 이 차이가 진짜인지는 21.10절에서 씨앗을 바꿔 보겠습니다. 투-타워가 진가를 보이는 곳은 다음 절이에요.

21.9 콜드 스타트: 처음 보는 식당과 사용자

기록이 없어서 추천을 시작하지 못하는 문제를 **콜드 스타트(cold start)** 라고 합니다. 차가운 엔진에 시동이 잘 안 걸리는 데서 온 말이에요. 두 가지가 있습니다.

새 식당

학습 기간 뒤에 문을 연 식당 40곳만 놓고 문제를 만들었습니다. 학습 기록이 있는 사용자가 시험 기간에 새 식당에 처음 가서 좋았다면, 새 식당 40곳 중 그곳을 상위 5개 안에 넣는지 채점해요. 비교하는 방법은 다섯 가지입니다. 행렬 분해에는 흔히 쓰는 응급 처치도 하나 붙였어요. 새 식당의 화살표를 **같은 지역·같은 음식 식당들의 화살표 평균**으로 채우는 방법입니다.

```
import numpy as np
import pandas as pd
import torch

from 맛집데이터 import 데이터_만들기, 시간으로_나누기
from 순위지표 import 좋은_평점, ndcg_at_k, recall_at_k
from 추천학습 import 행렬분해, 학습설정, 학습하기, 좋았던_쌍
from 투타워 import 투타워

데이터 = 데이터_만들기(씨앗=0)
식당, 사용자 = 데이터.식당, 데이터.사용자
학습, 검증, 시험 = 시간으로_나누기(데이터.방문)
쌍, 후보_식당 = 좋았던_쌍(학습), torch.tensor(np.flatnonzero(식당["개업일"] < 300))
```

```
새_식당 = np.flatnonzero(식당["개업일"] >= 300)            # 학습 기간 뒤에 문을 연 40곳

# 문제: 학습 기록이 있는 사용자가 시험 기간에 '새 식당'에 처음 가서 좋았다. 새 식당 40곳 중에서 맞
↪ 히기
가본_곳 = pd.concat([학습, 검증]).groupby("사용자")["식당"].apply(set)
좋았던 = 시험[(시험["평점"] >= 좋은_평점) & 시험["식당"].isin(새_식당)]
문제들 = [(u, set(g) - 가본_곳[u]) for u, g in 좋았던.groupby("사용자")["식당"]
        if u in set(학습["사용자"]) and set(g) - 가본_곳[u]]
print(f"평가할 사용자 {len(문제들)}명, 후보는 새 식당 {len(새_식당)}곳")


def 채점(점수_함수, k: int = 5) -> str:
    """점수_함수(사용자) → 새 식당 40곳의 점수. 상위 k개로 Recall과 NDCG 평균."""
    목록들 = [list(새_식당[np.argsort(-점수_함수(u), kind="stable")]) for u, _ in 문제들]
    r = np.mean([recall_at_k(목록, 답, k) for 목록, (_, 답) in zip(목록들, 문제들)])
    n = np.mean([ndcg_at_k(목록, 답, k) for 목록, (_, 답) in zip(목록들, 문제들)])
    return f"Recall@{k} {r:.4f}, NDCG@{k} {n:.4f}"


torch.manual_seed(0)
행렬 = 행렬분해(len(사용자), len(식당))
학습하기(행렬, 쌍, 후보_식당, 학습설정(손실="bpr"))
torch.manual_seed(0)
타워 = 투타워(사용자, 식당)
학습하기(타워, 쌍, 후보_식당, 학습설정(손실="bpr", 에폭=60))
타워.eval()

지역, 음식, 동네 = 식당["지역"].to_numpy(), 식당["음식"].to_numpy(), 사용자["동네"].to_numpy()
with torch.no_grad():
    행렬_사용자 = 행렬.사용자_벡터(torch.arange(len(사용자)))
    행렬_식당 = 행렬.식당_벡터(torch.arange(len(식당)))
    평균_식당 = 행렬_식당.clone()                    # 새 식당 = 같은 지역·같은 음식 식당들의 평균
    for i in 새_식당:
        같은 = np.flatnonzero((지역 == 지역[i]) & (음식 == 음식[i]) & (식당["개업일"] < 300))
        평균_식당[i] = 행렬_식당[같은].mean(dim=0)
    타워_사용자 = 타워.사용자_벡터(torch.arange(len(사용자)))
```

```
    타워_새_식당 = 타워.식당_벡터(torch.tensor(새_식당), 번호_쓰기=False)   # 특징만으로

rng = np.random.default_rng(0)
print(채점(lambda u: rng.random(len(새_식당))), "¦", "무작위")
print(채점(lambda u: (지역[새_식당] == 동네[u]) + rng.random(len(새_식당)) * 0.1), "¦", "내 동네 식
↪ 당 먼저")
print(채점(lambda u: (행렬_사용자[u] @ 행렬_식당[새_식당].T).numpy()), "¦", "행렬 분해 (배운 적 없
↪ 음)")
print(채점(lambda u: (행렬_사용자[u] @ 평균_식당[새_식당].T).numpy()), "¦", "행렬 분해 + 비슷한 곳
↪ 평균")
print(채점(lambda u: (타워_사용자[u] @ 타워_새_식당.T).numpy()), "¦", "투-타워 (특징만)")
```

```
평가할 사용자 295명, 후보는 새 식당 40곳
Recall@5 0.1232, NDCG@5 0.0719 ¦ 무작위
Recall@5 0.4554, NDCG@5 0.2807 ¦ 내 동네 식당 먼저
Recall@5 0.1169, NDCG@5 0.0654 ¦ 행렬 분해 (배운 적 없음)
Recall@5 0.5819, NDCG@5 0.3851 ¦ 행렬 분해 + 비슷한 곳 평균
Recall@5 0.6192, NDCG@5 0.4293 ¦ 투-타워 (특징만)
```

- **행렬 분해(배운 적 없음)**: NDCG@5가 0.0654로 무작위(0.0719)와 다를 바 없습니다. 새 식당의 화살표는 학습되지 않은 숫자라서, 순위가 사실상 제비뽑기예요.
- **내 동네 식당 먼저**: 0.2807. 새 식당이라도 동네는 아니까, 이것만으로 네 배 가까이 좋아집니다.
- **행렬 분해 + 비슷한 곳 평균**: 0.3851. 사람이 정한 규칙("같은 지역·음식이면 비슷할 것")으로 빈 화살표를 채웠더니 꽤 쓸 만해졌어요.
- **투-타워(특징만)**: 0.4293으로 가장 좋습니다. 번호 없이 특징만 넣었는데, 지역·음식·가격대를 **어떻게 섞을지를 데이터에서 배운** 덕분이에요. 규칙으로 평균 낸 방법은 가격대를 쓰지 않았고, 지역과 음식이 둘 다 같은 식당만 평균 냈습니다.

새 사용자

이번에는 학습 기간 뒤에 가입한 사용자입니다. 가입할 때 적은 동네와, 시험 전(검증 기간)에 다녀온 곳이 있으면 그것만 압니다. 276명 중 144명은 다녀온 곳이 한두 곳 있고, 나머지는 동네만 알아요. 네 가지를 비교했습니다. 마지막 방법은 "동네 인기순"에서 출발해, 다녀온 곳이 있으면 21.7절의 **비슷한 식당** 점수를 더해 올리는 방법이에요.

```
import numpy as np
import pandas as pd
import torch

from 맛집데이터 import 데이터_만들기, 시간으로_나누기
from 순위지표 import 좋은_평점, ndcg_at_k, recall_at_k
from 추천학습 import 학습설정, 학습하기, 좋았던_쌍
from 투타워 import 투타워

데이터 = 데이터_만들기(씨앗=0)
식당, 사용자 = 데이터.식당, 데이터.사용자
학습, 검증, 시험 = 시간으로_나누기(데이터.방문)
이전 = pd.concat([학습, 검증])                          # 시험 직전까지 쌓인 기록

# 문제: 학습 기간 뒤에 가입한 사용자. 가입 때 적은 동네와, 그 뒤 검증 기간에 다녀온 곳(있으면)만 안
↪ 다
새_사용자 = sorted(set(시험["사용자"]) - set(학습["사용자"]))
가본_곳 = 이전.groupby("사용자")["식당"].apply(set)
좋았던 = 시험[시험["평점"] >= 좋은_평점].groupby("사용자")["식당"].apply(set)
문제들 = [(u, 좋았던[u] - 가본_곳.get(u, set()), 가본_곳.get(u, set()))
        for u in 새_사용자 if u in 좋았던.index and 좋았던[u] - 가본_곳.get(u, set())]
print(f"새 사용자 {len(문제들)}명, 그중 시험 전에 다녀온 곳이 있는 사람 {sum(bool(h) for *_, h in
↪ 문제들)}명")

torch.manual_seed(0)
타워 = 투타워(사용자, 식당)
학습하기(타워, 좋았던_쌍(학습), torch.tensor(np.flatnonzero(식당["개업일"] < 300)),
        학습설정(손실="bpr", 에폭=60))
타워.eval()
with torch.no_grad():
    아는_식당 = torch.tensor((식당["개업일"] < 300).to_numpy())[:, None]
    전체 = torch.arange(len(식당))
    식당_벡터 = torch.where(아는_식당, 타워.식당_벡터(전체), 타워.식당_벡터(전체, 번호_쓰기=False))
    단위 = 식당_벡터 / 식당_벡터.norm(dim=1, keepdim=True)
    비슷함 = (단위 @ 단위.T).numpy()
    동네만_사용자 = 타워.사용자_벡터(torch.arange(len(사용자)), 번호_쓰기=False)
```

```
인기 = np.log1p(np.bincount(이전["식당"], minlength=len(식당)))
지역, 동네 = 식당["지역"].to_numpy(), 사용자["동네"].to_numpy()


def 채점(점수_함수, k: int = 10) -> str:
    rec, ndcg = [], []
    for u, 답, 가본 in 문제들:
        점수 = 점수_함수(u, 가본).astype(float)
        점수[list(가본)] = -np.inf                          # 이미 가 본 곳은 빼고
        목록 = list(np.argsort(-점수, kind="stable")[:k])
        rec.append(recall_at_k(목록, 답, k))
        ndcg.append(ndcg_at_k(목록, 답, k))
    return f"Recall@{k} {np.mean(rec):.4f}, NDCG@{k} {np.mean(ndcg):.4f}"


def 동네_인기(u: int, 가본: set[int]) -> np.ndarray:
    return 인기 + 3.0 * (지역 == 동네[u])                   # 내 동네면 점수를 크게 더한다


def 동네_인기_비슷한_곳(u: int, 가본: set[int]) -> np.ndarray:
    점수 = 동네_인기(u, 가본)
    if 가본:                                               # 다녀온 곳이 있으면 그와 비슷한 곳을 더
    ↪ 올린다
        점수 = 점수 + 2.0 * 비슷함[list(가본)].mean(axis=0)
    return 점수


print(채점(lambda u, h: 인기), "¦ 인기순")
print(채점(동네_인기), "¦ 동네 인기순")
print(채점(lambda u, h: (동네만_사용자[u] @ 식당_벡터.T).numpy()), "¦ 투-타워 (동네만)")
print(채점(동네_인기_비슷한_곳), "¦ 동네 인기순 + 다녀온 곳과 비슷한 곳")
```

```
새 사용자 276명, 그중 시험 전에 다녀온 곳이 있는 사람 144명
Recall@10 0.1262, NDCG@10 0.0748 ¦ 인기순
Recall@10 0.3038, NDCG@10 0.1865 ¦ 동네 인기순
```

```
Recall@10 0.2917, NDCG@10 0.1743 ¦ 투-타워 (동네만)
Recall@10 0.3056, NDCG@10 0.1979 ¦ 동네 인기순 + 다녀온 곳과 비슷한 곳
```

이번에는 결과가 기대와 다릅니다. 동네만 넣은 투-타워(NDCG@10 0.1743)가 단순한 **동네 인기순(0.1865)보다 낮아요.** 투-타워의 사용자 탑이 번호 없이 받는 정보는 동네 하나뿐인데, 그 동네 하나에서 뽑아낼 수 있는 것은 동네 인기순이 이미 다 쓰고 있습니다. 게다가 동네 인기순의 인기 숫자는 검증 기간까지 센 최신 값이지만, 모델은 학습 기간까지만 봤어요. 다녀온 곳과 비슷한 곳을 더한 방법이 0.1979로 가장 좋았지만, 다녀온 곳이 있는 사람이 절반뿐이라 차이가 크지는 않습니다.

이 결과를 감추지 않는 이유가 있습니다. **신경망이 언제나 이기는 건 아닙니다.** 정보가 거의 없는 상황에서는 잘 만든 간단한 규칙이 더 낫거나 같은 경우가 많아요. 그래서 실제 서비스는 상황마다 방법을 섞습니다. 기록이 충분하면 모델로, 기록이 없으면 규칙으로, 한두 곳 다녀왔으면 그 둘을 섞어서요. 21.12절의 서비스도 이렇게 만듭니다.

Q

Q. 가입할 때 취향을 물어보면 더 나아지지 않을까요? 그렇습니다. 많은 서비스가 가입할 때 "좋아하는 음식을 골라 주세요" 같은 질문을 하는 이유가 이것이에요. 질문으로 얻은 답을 사용자 탑의 특징으로 넣으면 동네 하나보다 훨씬 많은 정보가 생깁니다. 다만 질문이 길면 가입하다 떠나는 사람이 늘어서, 몇 개를 물을지도 데이터로 정해야 하는 문제예요.

21.10 실험 기록과 여러 씨앗

지금까지의 비교는 모두 씨앗 하나짜리였습니다. 18장 18.6절에서 본 것처럼 씨앗 하나의 차이로는 결론을 내리면 안 되죠. 서비스에 올릴 모델을 고르기 전에, 세 후보(행렬 분해·BCE, 행렬 분해·BPR, 투-타워·BPR)를 씨앗 세 개로 다시 돌리고 18장 방식으로 기록하겠습니다.

- 실험 설정은 `frozen=True` dataclass로(18장 18.2절), 설정의 지문은 SHA-256으로 남깁니다.
- 데이터의 지문도 남깁니다(18장 18.7절). pandas의 `hash_pandas_object`가 표의 줄마다 해시를 내 주니, 그것을 다시 SHA-256으로 묶었어요.
- 결과는 한 줄에 실험 하나씩 JSON Lines 파일에 덧붙이고, 줄마다 `flush`합니다(18장 18.4절).

```
import hashlib
import json
import statistics
```

```
from dataclasses import asdict, dataclass
from pathlib import Path

import numpy as np
import pandas as pd
import torch

from 맛집데이터 import 데이터_만들기, 시간으로_나누기
from 순위지표 import 평가셋_만들기, 평가하기
from 추천학습 import 내적추천, 행렬분해, 학습설정, 학습하기, 좋았던_쌍
from 투타워 import 투타워


@dataclass(frozen=True)
class 실험:
    모델: str                  # "행렬분해" 또는 "투타워"
    손실: str                  # "bce" 또는 "bpr"
    씨앗: int
    에폭: int = 60
    데이터_씨앗: int = 0


데이터 = 데이터_만들기(씨앗=0)
데이터_지문 = hashlib.sha256(pd.util.hash_pandas_object(데이터.방
↪ 문).to_numpy().tobytes()).hexdigest()
학습, 검증, 시험 = 시간으로_나누기(데이터.방문)
검증셋 = 평가셋_만들기(학습, 검증, 데이터.식당)
시험셋 = 평가셋_만들기(pd.concat([학습, 검증]), 시험, 데이터.식당)
쌍, 후보_식당 = 좋았던_쌍(학습), torch.tensor(np.flatnonzero(데이터.식당["개업일"] < 300))


def 점수표(모델: 내적추천, 사용자: np.ndarray) -> np.ndarray:
    with torch.no_grad():
        return 모델.모든_점수(torch.tensor(사용자)).numpy()
```

```
def 돌리기(설정: 실험) -> dict:
    torch.manual_seed(설정.씨앗)
    모델 = (행렬분해(len(데이터.사용자), len(데이터.식당)) if 설정.모델 == "행렬분해"
            else 투타워(데이터.사용자, 데이터.식당))
    기록 = 학습하기(모델, 쌍, 후보_식당, 학습설정(손실=설정.손실, 에폭=설정.에폭, 씨앗=설정.씨앗),
                    lambda m: 평가하기(점수표(m, 검증셋.사용자), 검증셋)["NDCG@10"])
    최고 = max(기록, key=lambda 줄: 줄["검증"])
    return {"최고_에폭": 최고["에폭"], "검증_NDCG@10": 최고["검증"],
            **{f"시험_{k}": v for k, v in 평가하기(점수표(모델, 시험셋.사용자), 시험셋).items()}}


기록_파일 = Path("실험기록") / "21장_비교.jsonl"
기록_파일.parent.mkdir(exist_ok=True)
설정들 = [실험(모델, 손실, 씨앗) for 모델, 손실 in [("행렬분해", "bce"), ("행렬분해", "bpr"), ("투
↪ 타워", "bpr")]
          for 씨앗 in (0, 1, 2)]
with 기록_파일.open("w", encoding="utf-8") as f:
    for 설정 in 설정들:
        글 = json.dumps(asdict(설정), ensure_ascii=False, sort_keys=True)
        줄 = {"설정_지문": hashlib.sha256(글.encode()).hexdigest()[:10], "설정": asdict(설정),
              "데이터_지문": 데이터_지문[:16], "torch": torch.__version__, **돌리기(설정)}
        f.write(json.dumps(줄, ensure_ascii=False) + "\n")
        f.flush()                                  # 중간에 끊겨도 끝난 실험은 남는다 (18장 18.4절)

결과 = pd.DataFrame([json.loads(줄) for 줄 in 기록_파일.read_text(encoding="utf-8").splitlines()])
결과["이름"] = [f"{s['모델']}·{s['손실']}" for s in 결과["설정"]]
print("기록", len(결과), "줄, 데이터 지문", 결과["데이터_지문"][0], "\n")
print("시험 NDCG@10 (씨앗 0, 1, 2)          평균 ± 표준편차  ¦ 실험")
for 이름, 묶음 in 결과.groupby("이름", sort=False):
    값 = 묶음["시험_NDCG@10"].tolist()
    print(" ".join(f"{v:.4f}" for v in 값), f"  {statistics.mean(값):.4f} ±
    ↪ {statistics.stdev(값):.4f}  ¦ {이름}")

for 상대 in ["행렬분해·bce", "투타워·bpr"]:
    차이 = (결과[결과["이름"] == 상대]["시험_NDCG@10"].to_numpy()
          - 결과[결과["이름"] == "행렬분해·bpr"]["시험_NDCG@10"].to_numpy())
```

```
    print(f"{상대} - 행렬분해·bpr (같은 씨앗끼리): {np.round(차이, 4).tolist()}")
```

```
기록 9 줄, 데이터 지문 13a9083f48c9341e

시험 NDCG@10 (씨앗 0, 1, 2)          평균 ± 표준편차  ┆ 실험
0.1058 0.1083 0.1050   0.1064 ± 0.0017  ┆ 행렬분해·bce
0.1451 0.1403 0.1422   0.1425 ± 0.0024  ┆ 행렬분해·bpr
0.1392 0.1454 0.1416   0.1421 ± 0.0031  ┆ 투타워·bpr
행렬분해·bce - 행렬분해·bpr (같은 씨앗끼리): [-0.0394, -0.0321, -0.0372]
투타워·bpr - 행렬분해·bpr (같은 씨앗끼리): [-0.0059, 0.005, -0.0006]
```

씨앗 세 개의 평균 ± 표준편차로 보면 행렬 분해·BCE 0.1064, 행렬 분해·BPR 0.1425, 투-타워·BPR 0.1421입니다. 같은 씨앗끼리 짝지은 차이(18장 18.6절)를 보면 결론이 분명해요.

- **BCE와 BPR**: 세 씨앗 모두 BCE가 0.03~0.04 낮습니다. 부호가 한 번도 바뀌지 않고, 차이가 씨앗 사이의 흔들림(표준편차 0.002~0.003)보다 열 배 이상 커요. BPR이 낫다는 21.6절의 결론은 믿어도 됩니다.
- **투-타워와 행렬 분해**: 차이가 -0.0059, +0.0050, -0.0006으로 부호가 오락가락합니다. 21.8절에서 씨앗 0만 보고 "행렬 분해가 조금 앞선다"고 했던 차이는 운이었어요. 기록이 있는 사용자에게는 **둘이 같다**고 보는 게 맞습니다.

그렇다면 서비스에는 어느 쪽을 올릴까요? 기록이 있는 사용자에게는 둘이 같고, 새 식당에는 투-타워가 확실히 낫습니다(21.9절). 그래서 **투-타워를 고릅니다.** 이렇게 "평균적인 점수"만이 아니라 "어떤 상황에서 누가 나은가"를 함께 보고 고르는 것이 실무의 모델 선택이에요. 실험 기록 파일에는 설정, 데이터 지문, PyTorch 버전, 가장 좋던 에폭, 검증·시험 점수가 줄마다 남아 있으니, 석 달 뒤에 "왜 투-타워였지?"라는 질문을 받아도 답할 수 있습니다.

21.11 서빙 준비: 벡터를 미리 계산해 두기

요청마다 탑을 돌릴 필요가 있을까요?

투-타워의 점수는 "사용자 탑의 화살표 · 식당 탑의 화살표"입니다. 그런데 두 탑의 화살표는 **모델을 다시 학습하기 전까지 바뀌지 않아요.** 사용자 0번의 화살표는 오늘 아침에 계산하든 저녁에 계산하든 같습니다. 그렇다면 요청이 올 때마다 탑을 돌릴 이유가 없죠. 학습이 끝나면 모든 사용자와 식당의 화살표를 **한 번만 계산해 표로 저장**하고, 서버는 그 표에서 꺼내 내적만 하면 됩니다. 두 탑이 서로

를 보지 않게 만든 이유가 여기 있어요(21.8절).

세 가지 방법으로 요청 하나에 걸리는 시간을 재 보겠습니다. 20장 20.9절처럼 준비 운동 뒤에 여러 번 재서 중앙값을 썼어요.

```
import statistics
import time

import numpy as np
import torch

from 맛집데이터 import 데이터_만들기, 시간으로_나누기
from 추천학습 import 학습설정, 학습하기, 좋았던_쌍
from 투타워 import 투타워

데이터 = 데이터_만들기(씨앗=0)
학습, _, _ = 시간으로_나누기(데이터.방문)
torch.manual_seed(0)
타워 = 투타워(데이터.사용자, 데이터.식당)
학습하기(타워, 좋았던_쌍(학습), torch.tensor(np.flatnonzero(데이터.식당["개업일"] < 300)),
         학습설정(에폭=3))                          # 시간만 잴 것이라 짧게
타워.eval()
전체_식당 = torch.arange(len(데이터.식당))


def 모델로_바로(u: int) -> list[int]:
    """요청마다 두 탑을 모두 돌린다."""
    with torch.no_grad():
        점수 = 타워.사용자_벡터(torch.tensor([u])) @ 타워.식당_벡터(전체_식당).T
    return 점수[0].topk(10).indices.tolist()


with torch.no_grad():
    식당_행렬 = 타워.식당_벡터(전체_식당)                      # (400, 32): 한 번만
    사용자_행렬 = 타워.사용자_벡터(torch.arange(len(데이터.사용자)))  # (3000, 32): 한 번만


def 식당만_미리(u: int) -> list[int]:
```

```
    with torch.no_grad():
        점수 = 타워.사용자_벡터(torch.tensor([u])) @ 식당_행렬.T
    return 점수[0].topk(10).indices.tolist()


식당_np, 사용자_np = 식당_행렬.numpy(), 사용자_행렬.numpy()


def 모두_미리_numpy(u: int) -> list[int]:
    점수 = 식당_np @ 사용자_np[u]                          # (400,): 곱셈 한 번
    상위 = np.argpartition(-점수, 10)[:10]                 # 상위 10개만 골라 놓고
    return 상위[np.argsort(-점수[상위])].tolist()          # 그 10개만 정렬


print("세 방법의 답이 같은가:", all(모델로_바로(u) == 식당만_미리(u) == 모두_미리_numpy(u)
                                for u in range(100)))
for 이름, 함수 in [("요청마다 두 탑 모두", 모델로_바로), ("식당 벡터만 미리", 식당만_미리),
                   ("모두 미리 + NumPy", 모두_미리_numpy)]:
    for u in range(50):
        함수(u)                                            # 준비 운동
    시간 = []
    for u in range(2000):
        시작 = time.perf_counter()
        함수(u % 3000)
        시간.append((time.perf_counter() - 시작) * 1e6)
    print(f"요청 하나에 {statistics.median(시간):7.1f}µs (중앙값) ┆ {이름}")
print(f"미리 계산한 벡터: 사용자 {사용자_np.nbytes / 1024:.0f}KB, 식당 {식당_np.nbytes /
↪ 1024:.0f}KB")
```

```
세 방법의 답이 같은가: True
요청 하나에    72.4µs (중앙값) ┆ 요청마다 두 탑 모두
요청 하나에    25.8µs (중앙값) ┆ 식당 벡터만 미리
요청 하나에     8.4µs (중앙값) ┆ 모두 미리 + NumPy
미리 계산한 벡터: 사용자 375KB, 식당 50KB
```

이 책의 실행 환경(Apple M4 Max, CPU)에서 잰 값입니다. 세 방법의 상위 10개는 모두 같은데, 요

청마다 두 탑을 돌리면 72µs, 식당 화살표만 미리 계산해 두면 26µs, 모두 미리 계산해 NumPy로 내적만 하면 8µs입니다. 아홉 배 가까이 빨라졌어요. 실행할 때마다 앞자리 숫자는 조금씩 달라지지만 순서는 늘 같습니다. 상위 10개를 고를 때는 400개를 모두 정렬하지 않고 `np.argpartition`으로 상위 10개만 골라낸 뒤 그 10개만 정렬했습니다. 식당이 수만 곳으로 늘면 이 차이가 커져요.

속도보다 더 큰 이득은 **서버에 PyTorch가 필요 없어진다**는 점입니다. 저장한 표는 사용자 375KB, 식당 50KB짜리 NumPy 배열이고, 내적은 NumPy만으로 충분해요. 20장 20.8절에서 ONNX Runtime으로 torch를 빼서 이미지가 작아진 것처럼, 이번에는 아예 NumPy만으로 서빙합니다.

Q

Q. 식당이 수백만 곳이면 어떻게 하나요? 요청마다 수백만 번 내적하고 정렬하면 느려집니다. 그때는 **근사 최근접 이웃 검색(approximate nearest neighbor, ANN)**을 씁니다. 화살표들을 미리 비슷한 것끼리 묶어 색인을 만들어 두고, 요청이 오면 가까운 묶음만 뒤지는 방법이에요. 정확한 상위 10개 대신 "거의 정확한" 상위 10개를 훨씬 빨리 찾습니다. 이 일을 전문으로 하는 라이브러리와 벡터 데이터베이스가 따로 있어요. 4권 「에이전트 엔지니어링 편」에서 문서를 벡터로 바꿔 찾는 RAG를 만들 때 같은 문제를 다시 만납니다.

서비스에 넘길 파일 만들기

이제 서비스에 넘길 파일을 만듭니다. 두 가지를 정했어요.

- **다시 학습하기**: 지금까지는 공정한 비교를 위해 학습 기간(0~299일)만 썼지만, 서비스에 올릴 모델은 서비스를 여는 날(330일) 전의 기록을 **모두** 써서 다시 학습합니다. 최근 기록일수록 소중하니까요. 검증 기간까지 써 버렸으니 검증 점수로 멈출 때를 정할 수 없어서, 21.8절에서 투-타워의 검증 점수가 가장 좋았던 55에폭으로 정해 두고 돌렸습니다.
- **두 파일로 나누기**: 숫자 표는 `vectors.npz`(NumPy의 배열 묶음 형식)에, 식당 이름·지역·방문 수와 사용자의 동네·가 본 곳 같은 정보는 `catalog.json`에 담습니다. 서비스는 이 두 파일만 있으면 돌아가요. JSON의 키는 20장처럼 다른 프로그램도 읽기 쉽게 영어로 붙였습니다.

```
import hashlib
import json
from pathlib import Path

import numpy as np
import torch
```

```
from 맛집데이터 import 데이터_만들기, 음식들, 지역들
from 추천학습 import 학습설정, 학습하기, 좋았던_쌍
from 투타워 import 투타워

기준일 = 330                                    # 서비스를 여는 날: 이날 전의 기록을 모두 쓴다
데이터 = 데이터_만들기(씨앗=0)
식당, 사용자 = 데이터.식당, 데이터.사용자
지난_기록 = 데이터.방문[데이터.방문["날짜"] < 기준일]
쌍 = 좋았던_쌍(지난_기록)
영업중 = (식당["개업일"] < 기준일).to_numpy()

torch.manual_seed(0)
타워 = 투타워(사용자, 식당)
학습하기(타워, 쌍, torch.tensor(np.flatnonzero(영업중)), 학습설정(손실="bpr", 에폭=55))  # 검증 없
↪ 이 정한 에폭만큼
타워.eval()

아는_사용자 = np.isin(np.arange(len(사용자)), 쌍[:, 0].numpy())
아는_식당 = torch.tensor(np.isin(np.arange(len(식당)), 쌍[:, 1].numpy()))[:, None]
with torch.no_grad():
    전체 = torch.arange(len(식당))
    식당_벡터 = torch.where(아는_식당, 타워.식당_벡터(전체), 타워.식당_벡터(전체, 번호_쓰기=False))
    사용자_벡터 = 타워.사용자_벡터(torch.arange(len(사용자)))

폴더 = Path("restaurant_service/artifacts")
폴더.mkdir(parents=True, exist_ok=True)
np.savez(폴더 / "vectors.npz", user_vectors=사용자_벡터.numpy(), restaurant_vectors=식당_벡
↪ 터.numpy(),
         known_users=아는_사용자)
방문수 = np.bincount(지난_기록["식당"], minlength=len(식당))
가본_곳 = 지난_기록.groupby("사용자")["식당"].apply(lambda s: sorted(set(s)))
목록 = {
    "as_of_day": 기준일, "areas": 지역들,
    "restaurants": [{"id": int(r.식당), "name": r.이름, "area": 지역들[r.지역], "cuisine": 음식들
    ↪ [r.음식],
```

```
                    "price": int(r.가격대), "open": bool(영업중[r.식당]), "visits": int(방문수[r.식
                     ↪ 당])}
                    for r in 식당.itertuples()],
    "users": [{"home": 지역들[동네], "visited": [int(i) for i in 가본_곳.get(u, [])]}
              for u, 동네 in enumerate(사용자["동네"])],
}
(폴더 / "catalog.json").write_text(json.dumps(목록, ensure_ascii=False), encoding="utf-8")

# 확인: NumPy로 계산한 상위 10개가 PyTorch 모델의 상위 10개와 같은가
읽은 = np.load(폴더 / "vectors.npz")
with torch.no_grad():
    모델_점수 = 사용자_벡터 @ 식당_벡터.T
같음 = all(np.array_equal(np.argsort(-(읽은["restaurant_vectors"] @ 읽은["user_vectors"][u]))[:10],
                          모델_점수[u].topk(10).indices.numpy()) for u in range(200))
print(f"아는 사용자 {아는_사용자.sum():,}명 / {len(사용자):,}명, 영업 중인 식당 {영업중.sum()}곳 "
      f"(그중 기록 없는 새 식당 {(영업중 & ~아는_식당[:, 0].numpy()).sum()}곳)")
print("NumPy와 PyTorch의 상위 10개가 같은가:", 같음)
for 파일 in sorted(폴더.iterdir()):
    print(f"{파일.name:14} {파일.stat().st_size / 1024:7.1f} KB  sha256 {hashlib.sha256(파
     ↪ 일.read_bytes()).hexdigest()[:12]}")
```

```
아는 사용자 2,846명 / 3,000명, 영업 중인 식당 380곳 (그중 기록 없는 새 식당 6곳)
NumPy와 PyTorch의 상위 10개가 같은가: True
catalog.json     320.0 KB  sha256 3a47045bcb03
vectors.npz      428.7 KB  sha256 ab1b2963a93c
```

서비스를 여는 날 기준으로 화살표를 가진 사용자는 2,846명이고, 나머지 154명은 아직 좋았던 방문이 없어서 21.9절의 새 사용자 방법으로 추천합니다. 영업 중인 380곳 중 6곳은 기록이 없는 새 식당이라 투-타워의 특징만으로 화살표를 만들었어요. 저장한 파일을 다시 읽어 NumPy로 계산한 상위 10개가 PyTorch 모델의 상위 10개와 사용자 200명 모두 같다는 것도 확인했습니다. 20장 실습의 ONNX 내보내기 스크립트처럼, 만든 파일이 원래 모델과 같은 답을 내는지 **만든 자리에서** 확인하는 습관이에요. 두 파일의 SHA-256은 서비스의 `/version`에서 다시 만납니다.

21.12 실습: 맛집 추천 API 완성하기

이제 20장 20.11절의 구조를 그대로 따라 추천 서비스를 완성합니다.

프로젝트 구조

```
restaurant_service/
├── pyproject.toml        # 의존성: fastapi, uvicorn, numpy (torch 없음)
├── uv.lock
├── .python-version       # 3.12
├── Dockerfile
├── .dockerignore
├── app/
│   ├── __init__.py
│   ├── recommender.py    # 미리 계산한 벡터로 추천하기 (NumPy만)
│   └── main.py           # FastAPI 앱
├── artifacts/
│   ├── vectors.npz       # 21.11절에서 만든 화살표 표
│   └── catalog.json      # 식당·사용자 정보
└── tests/
    └── test_main.py
```

20장과 달리 `scripts/` 폴더가 없습니다. 화살표 표를 만드는 일은 PyTorch가 있는 학습 쪽(21.11절의 코드)에서 끝내고, 서비스는 결과 파일만 받아요. **학습하는 곳과 서빙하는 곳의 경계가 파일 두 개**라는 점이 이 설계의 핵심입니다.

추천기

```
"""미리 계산한 벡터로 추천한다 (torch 없이 NumPy만 쓴다)."""
import json
from pathlib import Path
from typing import Protocol

import numpy as np

추천목록 = list[tuple[int, float]]                # (식당 번호, 점수)를 점수 높은 순으로
```

```
class 추천기(Protocol):
    지역들: list[str]
    식당: list[dict]

    def 사용자_수(self) -> int: ...
    def 추천(self, 사용자: int, k: int, 지역: str | None = None) -> tuple[str, 추천목록]: ...
    def 새_사용자_추천(self, 동네: str, 가본_곳: list[int], k: int, 지역: str | None = None) -> 추
    ↪ 천목록: ...
    def 비슷한_식당(self, 식당: int, k: int) -> 추천목록: ...


class 벡터추천기:
    def __init__(self, 폴더: Path) -> None:
        벡터 = np.load(폴더 / "vectors.npz")
        self.사용자_벡터, self.식당_벡터 = 벡터["user_vectors"], 벡터["restaurant_vectors"]
        self.아는_사용자 = 벡터["known_users"]
        목록 = json.loads((폴더 / "catalog.json").read_text(encoding="utf-8"))
        self.지역들, self.식당, self.사용자 = 목록["areas"], 목록["restaurants"], 목록["users"]
        self.영업중 = np.array([r["open"] for r in self.식당])
        self.식당_지역 = np.array([self.지역들.index(r["area"]) for r in self.식당])
        self.인기 = np.log1p([r["visits"] for r in self.식당])
        단위 = self.식당_벡터 / np.linalg.norm(self.식당_벡터, axis=1, keepdims=True)
        self.비슷함 = 단위 @ 단위.T                    # 식당 × 식당 코사인 유사도, 미리 한 번

    def 사용자_수(self) -> int:
        return len(self.사용자)

    def _상위(self, 점수: np.ndarray, 뺄_곳: list[int], k: int, 지역: str | None) -> 추천목록:
        점수 = np.where(self.영업중, 점수, -np.inf)       # 문을 닫았거나 아직 안 연 곳은 빼고
        점수[뺄_곳] = -np.inf                              # 이미 가 본 곳(또는 자기 자신)도 빼고
        if 지역 is not None:
            점수[self.식당_지역 != self.지역들.index(지역)] = -np.inf
        상위 = np.argsort(-점수, kind="stable")[:k]
        return [(int(i), round(float(점수[i]), 4)) for i in 상위 if np.isfinite(점수[i])]
```

```
    def 추천(self, 사용자: int, k: int, 지역: str | None = None) -> tuple[str, 추천목록]:
        정보 = self.사용자[사용자]
        if not self.아는_사용자[사용자]:                # 학습 기록이 없으면 새 사용자처럼
            return "new_user", self.새_사용자_추천(정보["home"], 정보["visited"], k, 지역)
        점수 = self.식당_벡터 @ self.사용자_벡터[사용자]
        return "personal", self._상위(점수, 정보["visited"], k, 지역)

    def 새_사용자_추천(self, 동네: str, 가본_곳: list[int], k: int, 지역: str | None = None) -> 추
    ↪ 천목록:
        점수 = self.인기 + 3.0 * (self.식당_지역 == self.지역들.index(동네))     # 동네 인기순
        if 가본_곳:                                                              # + 다녀온 곳과 비슷한 곳
            점수 = 점수 + 2.0 * self.비슷함[가본_곳].mean(axis=0)
        return self._상위(점수, 가본_곳, k, 지역)

    def 비슷한_식당(self, 식당: int, k: int) -> 추천목록:
        return self._상위(self.비슷함[식당], [식당], k, None)
```

- **추천기(Protocol)**: 20장 20.11절의 **예측기**처럼 경로 함수가 믿을 약속입니다. 테스트에서는 이 약속을 지키는 가짜를 끼워요.
- **__init__**: 서버가 켜질 때 두 파일을 읽고, 식당끼리의 코사인 유사도 표(400 × 400)도 **미리 한 번** 계산해 둡니다. 비슷한 식당 요청에는 표에서 한 줄을 꺼내기만 하면 돼요.
- **_상위**: 모든 추천이 거치는 마지막 관문입니다. 영업하지 않는 곳, 이미 가 본 곳을 지우고, 지역이 주어지면 그 지역만 남긴 뒤 상위 k개를 고릅니다.
- **추천**: 화살표가 있는 사용자는 내적으로(personal), 없는 사용자는 21.9절에서 가장 좋았던 "동네 인기순 + 다녀온 곳과 비슷한 곳"으로(new_user) 추천합니다. 어떤 방법을 썼는지 이름을 함께 돌려줘서, 응답만 보고도 알 수 있게 했어요.

경로 함수

```
"""맛집 추천 서비스: 미리 계산한 사용자·식당 벡터로 추천한다."""
import hashlib
import logging
import os
import time
```

```
from contextlib import asynccontextmanager
from pathlib import Path
from typing import Annotated, Literal

from fastapi import Depends, FastAPI, HTTPException, Query, Request
from pydantic import BaseModel, Field

from app.recommender import 벡터추천기, 추천기, 추천목록

앱_버전 = "1.0.0"
Area = Literal["강남", "홍대", "성수", "종로", "여의도", "잠실", "이태원", "신촌"]  # catalog.json
↪ 과 같아야 한다
로그 = logging.getLogger("uvicorn.error")


class Restaurant(BaseModel):
    id: int
    name: str
    area: str
    cuisine: str
    price: int
    score: float


class Recommendations(BaseModel):
    strategy: str                       # personal, new_user, similar
    items: list[Restaurant]


class NewUserRequest(BaseModel):
    home: Area
    visited: list[int] = Field(default=[], max_length=50)
    k: int = Field(default=10, ge=1, le=50)


@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
    폴더 = Path(os.environ.get("ARTIFACT_DIR", "artifacts"))
    app.state.추천기 = 벡터추천기(폴더)
    app.state.지문 = hashlib.sha256((폴더 / "vectors.npz").read_bytes()).hexdigest()[:12]
    로그.info("벡터 준비 완료: 사용자 %d명, 식당 %d곳, sha256=%s",
              app.state.추천기.사용자_수(), len(app.state.추천기.식당), app.state.지문)
    yield


app = FastAPI(title="맛집 추천 API", version=앱_버전, lifespan=lifespan)


@app.middleware("http")
async def 처리_시간_기록(request: Request, call_next):
    시작 = time.perf_counter()
    응답 = await call_next(request)
    응답.headers["X-Process-Time-ms"] = f"{(time.perf_counter() - 시작) * 1000:.2f}"
    return 응답


def 추천기_얻기(request: Request) -> 추천기:
    """경로 함수가 쓸 추천기. 테스트에서는 가짜로 바꿔 끼운다."""
    return request.app.state.추천기


def 응답_만들기(추천기_: 추천기, 전략: str, 목록: 추천목록) -> Recommendations:
    식당들 = [Restaurant(**{키: 추천기_.식당[i][키] for 키 in ("id", "name", "area", "cuisine",
↪   "price")},
                         score=점수) for i, 점수 in 목록]
    return Recommendations(strategy=전략, items=식당들)


def 식당_번호_확인(추천기_: 추천기, 번호들: list[int]) -> None:
    for 번호 in 번호들:
        if not 0 <= 번호 < len(추천기_.식당):
            raise HTTPException(status_code=404, detail=f"식당 {번호}을(를) 찾을 수 없습니다")
```

```
@app.get("/health")
def 살아있나() -> dict:
    return {"status": "ok"}


@app.get("/version")
def 버전(request: Request) -> dict:
    return {"app": 앱_버전, "vectors_sha256": request.app.state.지문}


@app.get("/users/{user_id}/recommendations")
def 사용자_추천(user_id: int, k: Annotated[int, Query(ge=1, le=50)] = 10, area: Area | None = None,
              추천기_: 추천기 = Depends(추천기_얻기)) -> Recommendations:
    if not 0 <= user_id < 추천기_.사용자_수():
        raise HTTPException(status_code=404, detail=f"사용자 {user_id}을(를) 찾을 수 없습니다")
    전략, 목록 = 추천기_.추천(user_id, k, area)
    return 응답_만들기(추천기_, 전략, 목록)


@app.post("/recommendations/new-user")
def 새_사용자_추천(요청: NewUserRequest, 추천기_: 추천기 = Depends(추천기_얻기)) ->
↪  Recommendations:
    식당_번호_확인(추천기_, 요청.visited)
    return 응답_만들기(추천기_, "new_user", 추천기_.새_사용자_추천(요청.home, 요청.visited, 요
    ↪  청.k))


@app.get("/restaurants/{restaurant_id}/similar")
def 비슷한_식당(restaurant_id: int, k: Annotated[int, Query(ge=1, le=50)] = 5,
              추천기_: 추천기 = Depends(추천기_얻기)) -> Recommendations:
    식당_번호_확인(추천기_, [restaurant_id])
    return 응답_만들기(추천기_, "similar", 추천기_.비슷한_식당(restaurant_id, k))
```

20장의 서비스와 같은 뼈대(lifespan, `Depends`로 꺼내는 추천기, 처리 시간 미들웨어, `/health`·`/version`) 위에 경로 세 개를 얹었습니다.

경로	하는 일
`GET /users/{user_id}/recommendations?k=10&area=성수`	사용자에게 아직 안 가 본 식당 k곳. `area`를 주면 그 지역에서만
`POST /recommendations/new-user`	앱에 처음 온 사람: 동네와 다녀온 곳(있으면)으로 추천
`GET /restaurants/{restaurant_id}/similar?k=5`	비슷한 식당 k곳

- **`Area = Literal[...]`**: 지역은 여덟 곳 중 하나여야 합니다. `Literal`로 적어 두면 Pydantic이 다른 값을 422로 돌려보내고, 자동 문서에도 고를 수 있는 값이 표시돼요. 다만 지역 목록이 `catalog.json`에도 있으니 둘이 어긋날 위험이 생깁니다. 바로 다음 테스트가 그 위험을 막아요.
- **`Query(ge=1, le=50)`**: 1권 19.3절에서 `/restaurants?지역=서울`로 맛집을 거르던 쿼리 매개변수에 범위 검사를 붙였습니다. 누군가 `k=1000000`을 보내 응답을 키우는 일을 막아요.
- **404**: 없는 사용자 번호나 식당 번호는 `HTTPException(404)`로 답합니다. 검사하지 않으면 NumPy 배열 인덱싱에서 `IndexError`가 나 500이 되거나, 음수 번호는 오류 없이 **맨 뒤 사용자**의 추천이 나가요(4장 4.4절의 음수 인덱스). 오류가 안 나서 더 위험한 경우입니다.

테스트

20장처럼 진짜 파일로 하는 테스트와 가짜 추천기로 하는 테스트를 나눴습니다.

```
"""맛집 추천 API 테스트. 실행: uv run pytest -q"""
from collections.abc import Iterator
from typing import get_args

import pytest
from fastapi.testclient import TestClient

from app.main import Area, app, 추천기_얻기
from app.recommender import 추천목록
```

```
class 가짜추천기:
    """식당 세 곳만 아는 가짜. 진짜 벡터 없이 API의 모양과 오류 처리만 시험한다."""
    지역들 = list(get_args(Area))
    식당 = [{"id": i, "name": f"가짜 식당 {i}", "area": "성수", "cuisine": "한식", "price": 1}
            for i in range(3)]

    def 사용자_수(self) -> int:
        return 2

    def 추천(self, 사용자: int, k: int, 지역: str | None = None) -> tuple[str, 추천목록]:
        return "personal", [(2, 0.9), (0, 0.5)][:k]

    def 새_사용자_추천(self, 동네: str, 가본_곳: list[int], k: int, 지역: str | None = None) -> 추
    ↪ 천목록:
        return [(i, 0.1) for i in range(3) if i not in 가본_곳][:k]

    def 비슷한_식당(self, 식당: int, k: int) -> 추천목록:
        return [(i, 0.3) for i in range(3) if i != 식당][:k]


@pytest.fixture
def client() -> Iterator[TestClient]:
    with TestClient(app) as c:                      # with로 열어야 lifespan이 돈다
        yield c


@pytest.fixture
def fake_client(client: TestClient) -> Iterator[TestClient]:
    app.dependency_overrides[추천기_얻기] = 가짜추천기
    yield client
    app.dependency_overrides.clear()


def test_area_literal_matches_catalog(client: TestClient) -> None:
    assert list(get_args(Area)) == app.state.추천기.지역들
```

```
def test_recommendations_for_known_user_exclude_visited(client: TestClient) -> None:
    가본_곳 = set(app.state.추천기.사용자[0]["visited"])
    본문 = client.get("/users/0/recommendations", params={"k": 10}).json()
    assert 본문["strategy"] == "personal"
    assert len(본문["items"]) == 10
    assert not {항목["id"] for 항목 in 본문["items"]} & 가본_곳
    점수 = [항목["score"] for 항목 in 본문["items"]]
    assert 점수 == sorted(점수, reverse=True)


def test_recommendations_with_area_filter_return_only_that_area(client: TestClient) -> None:
    본문 = client.get("/users/0/recommendations", params={"area": "이태원"}).json()
    assert {항목["area"] for 항목 in 본문["items"]} == {"이태원"}


def test_recommendations_for_user_without_history_fall_back_to_new_user(client: TestClient) ->
↪ None:
    추천기_ = app.state.추천기
    처음_온_사람 = next(u for u in range(추천기_.사용자_수()) if not 추천기_.아는_사용자[u])
    본문 = client.get(f"/users/{처음_온_사람}/recommendations").json()
    assert 본문["strategy"] == "new_user"
    assert len(본문["items"]) == 10


def test_similar_restaurants_exclude_itself(client: TestClient) -> None:
    본문 = client.get("/restaurants/17/similar", params={"k": 5}).json()
    번호들 = [항목["id"] for 항목 in 본문["items"]]
    assert len(번호들) == 5 and 17 not in 번호들


def test_new_user_request_excludes_visited(fake_client: TestClient) -> None:
    본문 = fake_client.post("/recommendations/new-user",
                            json={"home": "성수", "visited": [0], "k": 5}).json()
    assert [항목["id"] for 항목 in 본문["items"]] == [1, 2]
```

```
def test_unknown_user_or_restaurant_returns_404(fake_client: TestClient) -> None:
    assert fake_client.get("/users/99/recommendations").status_code == 404
    assert fake_client.get("/restaurants/99/similar").status_code == 404
    응답 = fake_client.post("/recommendations/new-user", json={"home": "성수", "visited": [99]})
    assert 응답.status_code == 404


def test_invalid_k_or_area_returns_422(fake_client: TestClient) -> None:
    assert fake_client.get("/users/0/recommendations", params={"k": 0}).status_code == 422
    assert fake_client.get("/users/0/recommendations", params={"area": "부산"}).status_code == 422
    assert fake_client.post("/recommendations/new-user", json={"home": "부산"}).status_code == 422
```

진짜 파일로 하는 테스트는 추천의 **약속**을 확인합니다. 이미 가 본 곳은 나오지 않는다, 점수 순서대로 나온다, 지역을 주면 그 지역만 나온다, 기록 없는 사용자는 `new_user`로 넘어간다, 비슷한 식당에 자기 자신은 없다. 숫자 하나하나는 모델을 다시 학습하면 바뀌지만, 이 약속들은 어떤 모델이 와도 지켜져야 하니까요. 첫 테스트 `test_area_literal_matches_catalog`는 코드의 `Area` 목록과 `catalog.json`의 지역 목록이 같은지 봅니다. 나중에 지역을 하나 더 넣고 코드 고치기를 잊으면 이 테스트가 알려 줘요. 가짜 추천기로는 404와 422 같은 오류 처리와 응답 모양을 봅니다.

```
uv run pytest -q
```

```
........                                                                 [100%]
8 passed in 0.17s
```

서버로 띄우기

20장과 같이 서버를 띄우고 `curl`로 요청을 보냅니다.

```
uv run uvicorn app.main:app --port 8000
```

```
curl -s localhost:8000/version
curl -s "localhost:8000/users/0/recommendations?k=3"
```

```
curl -s -G localhost:8000/users/0/recommendations --data-urlencode "k=2" --data-urlencode "area=이
↪ 태원"
curl -s "localhost:8000/restaurants/17/similar?k=3"
curl -s -X POST localhost:8000/recommendations/new-user -H "Content-Type: application/json" \
    -d '{"home": "성수", "visited": [17], "k": 3}'
curl -s "localhost:8000/users/0/recommendations?k=100"
```

```
{"app":"1.0.0","vectors_sha256":"ab1b2963a93c"}
{"strategy":"personal","items":[{"id":271,"name":"신촌 양식 271","area":"신촌","cuisine":"양
↪ 식","price":1,"score":2.5672},{"id":201,"name":"신촌 양식 201","area":"신촌","cuisine":"양
↪ 식","price":1,"score":2.2646},{"id":166,"name":"홍대 중식 166","area":"홍대","cuisine":"중
↪ 식","price":1,"score":2.1025}]}
{"strategy":"personal","items":[{"id":149,"name":"이태원 양식 149","area":"이태원","cuisine":"양
↪ 식","price":2,"score":-1.1206},{"id":121,"name":"이태원 양식 121","area":"이태
↪ 원","cuisine":"양식","price":2,"score":-1.6019}]}
{"strategy":"similar","items":[{"id":342,"name":"종로 분식 342","area":"종로","cuisine":"분
↪ 식","price":3,"score":0.9644},{"id":140,"name":"종로 분식 140","area":"종로","cuisine":"분
↪ 식","price":2,"score":0.9641},{"id":334,"name":"종로 분식 334","area":"종로","cuisine":"분
↪ 식","price":3,"score":0.9428}]}
{"strategy":"new_user","items":[{"id":194,"name":"성수 분식 194","area":"성수","cuisine":"분
↪ 식","price":1,"score":9.1637},{"id":59,"name":"성수 고기 059","area":"성수","cuisine":"고
↪ 기","price":1,"score":9.1107},{"id":242,"name":"성수 중식 242","area":"성수","cuisine":"중
↪ 식","price":1,"score":9.068}]}
{"detail":[{"type":"less_than_equal","loc":["query","k"],"msg":"Input should be less than or equal
↪ to 50","input":"100","ctx":{"le":50}}]}
```

- **`/version`**: 21.11절에서 찍은 `vectors.npz`의 지문 `ab1b2963a93c`가 그대로 나옵니다.
- **사용자 0번**: 홍대에 사는 사람인데 신촌 양식집 둘과 홍대 중식집을 추천받았습니다. 정답지를 열어 보면 이 사람의 숨은 직장은 신촌이고, 다녀온 36번 중 19번이 양식집이에요. 모델에게 직장은 알려 준 적이 없는데 방문 기록에서 배워 낸 거죠.
- **지역 걸기**: 주소에 한글을 넣을 때는 `-G --data-urlencode`로 보내면 `curl`이 알맞게 바꿔 줍니다(서버 로그에는 `area=%ec%9d%b4...`처럼 찍혀요). 그대로 넣으면 `curl`이 한글을 바꾸지 않아 서버가 잘못된 요청으로 거절합니다. 이태원 식당의 점수는 음수인데, 내적 점수는 사용자 안에서의 **순서**만 뜻이 있으니 괜찮아요.
- **비슷한 식당**: 21.7절의 "종로 분식 017"과 비슷한 곳으로 종로 분식집들이 나왔습니다. 21.7

절은 행렬 분해, 지금은 투-타워의 화살표라 순서는 조금 다릅니다.

- **새 사용자**: 성수에 살고 종로 분식 017에 다녀온 사람에게 성수의 분식·고기·중식집을 추천했어요. 아래 Docker 절에서 다녀온 곳이 없는 성수 사람(2706번)이 받은 목록과 견주면, 고기 059와 중식 242는 원래 동네 인기순에서 온 것이고, 분식 194가 1위로 올라온 것이 "종로 분식 017과 비슷한 곳" 점수를 더한 효과예요.
- **k=100**: 범위를 넘으니 422와 함께 어디가 틀렸는지(`["query","k"]`) 알려 줍니다.

Docker로 감싸기

의존성은 20장보다 더 단순합니다.

```
[project]
name = "restaurant-service"
version = "1.0.0"
requires-python = ">=3.12"
dependencies = [           # 미리 계산한 벡터로 추천하니 torch가 필요 없다
    "fastapi>=0.156",
    "uvicorn>=0.54",
    "numpy>=2.5",
]

[dependency-groups]
dev = ["pytest>=8.4", "httpx2>=2.15"]

[tool.uv]
package = false

[tool.pytest.ini_options]
pythonpath = ["."]
```

`Dockerfile`도 20장과 거의 같습니다.

```
FROM python:3.12-slim AS builder
COPY --from=ghcr.io/astral-sh/uv:0.9.16 /uv /bin/uv
WORKDIR /app
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy
COPY pyproject.toml uv.lock .python-version ./
```

```
RUN uv sync --frozen --no-dev

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /app/.venv /app/.venv
COPY app ./app
COPY artifacts ./artifacts
ENV PATH="/app/.venv/bin:$PATH" ARTIFACT_DIR=/app/artifacts
USER nobody
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s \
    CMD ["python", "-c", "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/
    ↪ health')"]
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

20장과 달라진 곳은 세 군데예요. torch 선택 의존성이 없으니 `ARG EXTRAS`를 뺐고, 모델 파일 대신 `artifacts` 폴더를 복사하며, 그 위치를 `ARTIFACT_DIR` 환경 변수로 알려 줍니다. `HEALTHCHECK`에는 `--start-period=10s`를 더해, 서버가 켜지는 동안의 실패는 세지 않게 했어요(20장 20.11절에서 말한 "시작에 걸리는 시간 알려 주기"). `.dockerignore`에는 20장과 같이 `.venv`, `__pycache__`, `.pytest_cache`, `tests`를 적었습니다.

```
docker build -t restaurant-service:1.0.0 .
docker images restaurant-service
```

```
REPOSITORY          TAG       IMAGE ID       CREATED         SIZE
restaurant-service  1.0.0     dc01e6d29f34   2 seconds ago   318MB
```

이미지는 318MB입니다. 20장의 ONNX 이미지(445MB)보다도 작아요. 컨테이너 안에서 재 보니 가상 환경 폴더가 85MB, 화살표 표와 정보 파일이 756KB였습니다. 나머지는 바탕 이미지 `python:3.12-slim`의 몫이에요. 20장에서 CPU 전용 PyTorch를 넣은 이미지가 1.51GB였던 것과 비교해 보세요. 이제 띄워 봅니다. 두 번째 요청의 2706번은 311일에 가입했지만 서비스를 여는 날까지 한 곳도 다녀오지 않은 사용자예요.

```
docker run -d --name restaurant -p 8000:8000 restaurant-service:1.0.0
docker ps --filter name=restaurant --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
```

```
curl -s "localhost:8000/users/0/recommendations?k=3"
curl -s "localhost:8000/users/2706/recommendations?k=3"
sleep 15
docker ps --filter name=restaurant --format "table {{.Names}}\t{{.Status}}"
docker logs restaurant
```

```
bb7549d4c72823b8c1d4d9e57a6a0838ded62a8600c3ebd96d639b5c4705affa
NAMES        IMAGE                      STATUS                            PORTS
restaurant   restaurant-service:1.0.0   Up 2 seconds (health: starting)   0.0.0.0:8000->8000/tcp,
↪ [::]:8000->8000/tcp
{"strategy":"personal","items":[{"id":271,"name":"신촌 양식 271","area":"신촌","cuisine":"양
↪ 식","price":1,"score":2.5672},{"id":201,"name":"신촌 양식 201","area":"신촌","cuisine":"양
↪ 식","price":1,"score":2.2646},{"id":166,"name":"홍대 중식 166","area":"홍대","cuisine":"중
↪ 식","price":1,"score":2.1025}]}
{"strategy":"new_user","items":[{"id":242,"name":"성수 중식 242","area":"성수","cuisine":"중
↪ 식","price":1,"score":9.5439},{"id":59,"name":"성수 고기 059","area":"성수","cuisine":"고
↪ 기","price":1,"score":9.0283},{"id":251,"name":"성수 중식 251","area":"성수","cuisine":"중
↪ 식","price":1,"score":8.9532}]}
NAMES        STATUS
restaurant   Up 17 seconds (healthy)
```

```
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     벡터 준비 완료: 사용자 3000명, 식당 400곳, sha256=ab1b2963a93c
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     192.168.65.1:51870 - "GET /users/0/recommendations?k=3 HTTP/1.1" 200 OK
INFO:     192.168.65.1:42865 - "GET /users/2706/recommendations?k=3 HTTP/1.1" 200 OK
INFO:     127.0.0.1:41204 - "GET /health HTTP/1.1" 200 OK
```

컨테이너 안에서도 사용자 0번의 추천이 맥에서 띄웠을 때와 똑같고, 기록이 없는 2706번은 `new_user` 방법으로 성수의 인기 식당을 받았습니다. 상태는 `health: starting`에서 `healthy`로 바뀌었고, 로그의 벡터 지문도 같은 `ab1b2963a93c`예요. 다 쓴 컨테이너와 이미지는 지웁니다.

```
docker rm -f restaurant
docker rmi restaurant-service:1.0.0
```

Q. 모델을 다시 학습할 때마다 이미지를 새로 만들어야 하나요? 지금 구조에서는 그렇습니다. `artifacts`를 이미지 안에 복사했으니까요. 대신 이미지와 모델이 한 묶음이라 "이 이미지는 이 모델"이 분명하고, 되돌리기도 쉬워요. 모델을 자주 바꾼다면 이미지에는 코드만 넣고, 화살표 파일은 `docker run -v 내_폴더:/app/artifacts:ro`처럼 바깥 폴더를 연결해 넣는 방법도 있습니다. 어느 쪽이든 `/version`의 지문으로 지금 어떤 파일이 올라가 있는지 확인할 수 있게 해 두면, "어제부터 추천이 이상해요"라는 연락에 답할 수 있어요.

21.13 정리와 2권을 마치며

이 장에서 맛집 추천 AI를 문제 정의부터 Docker 이미지까지 만들었습니다.

- **문제 정의**: 추천은 별점이 아니라 순위를 맞히는 문제. 처음 가서 좋았던 곳을 상위 K개에 넣는가를 Recall@K(들었나)와 NDCG@K(얼마나 위에)로 채점한다
- **데이터**: 개인정보와 라이선스 때문에 예제용 데이터를 만들었고, 숨은 취향을 정답지로 남겼다. 시간 순서로 나눠 새 식당·새 사용자를 평가에서 볼 수 있게 했다
- **기준선**: 무작위 0.0120, 인기순 0.0487, 동네 인기순 0.0946(시험 NDCG@10). 모델은 이것을 넘어야 의미가 있다
- **행렬 분해**: 사용자·식당마다 `nn.Embedding` 화살표, 점수는 내적, 편향은 인기. 안 간 곳은 무작위로 뽑아 음성으로(음성 샘플링)
- **손실**: 순위 문제에는 순위를 가르치는 BPR. 시험 NDCG@10이 BCE 0.1064, BPR 0.1425(씨앗 세 개 평균)
- **임베딩 들여다보기**: 코사인 유사도로 비슷한 식당, 선형 탐침으로 숨은 분위기(R^2 0.847)와 매운맛(0.766)이 화살표 안에 있음을 확인
- **투-타워**: 번호 + 특징, 번호 가리기로 특징만으로도 맞히게. 기록 있는 사용자에게는 행렬 분해와 같고, 새 식당에서는 가장 좋다(NDCG@5 0.4293)
- **새 사용자**: 동네만 아는 모델은 동네 인기순을 넘지 못했다. 정보가 적을 때는 규칙과 섞는다
- **실험 관리**: 설정·데이터 지문·결과를 JSON Lines로, 같은 씨앗끼리 짝지은 차이로 판단
- **서빙**: 화살표는 다시 학습하기 전까지 바뀌지 않으니 미리 계산해 파일로. 요청은 NumPy 내

적 한 번(72µs → 8µs), 서버에서 torch를 빼서 이미지 318MB. 약속을 확인하는 테스트, 기록이 없으면 다른 방법으로 넘어가기

2권을 마치며

2권을 시작할 때 1장에서 "if 문으로 스팸을 거를 수 있을까?"라고 물었습니다. 규칙을 짜는 대신 데이터로 배우는 프로그램이라는 생각에서 출발해, 1부에서 데이터를 나누고 평가하는 감각을, 2부에서 텐서와 자동 미분과 학습 루프라는 도구를, 3부에서 CNN·RNN·트랜스포머·생성 모델이라는 여러 구조를, 4부에서 실험을 관리하고 모델을 가볍게 만들어 서비스로 내놓는 일을 익혔어요. 그리고 이 장에서 그 모두를 하나로 엮었습니다.

돌아보면 2권 전체에 걸쳐 되풀이된 습관이 몇 가지 있습니다. 모델보다 **기준선을 먼저** 잰다. 데이터를 나눌 때 **미래를 엿보지 않는다**. 점수 하나보다 **여러 씨앗의 평균과 흔들림**을 본다. 결과가 기대보다 나쁘면 **감추지 않고 이유를 찾는다**. 그리고 서비스로 내놓을 때는 **약속을 테스트로 적어 둔다**. 도구와 모델은 몇 년이면 바뀌지만, 이 습관들은 오래 갑니다.

다음 3권 「로봇 편」에서는 지능이 화면 밖으로 나가 몸을 얻습니다. 파이썬과 MuJoCo 시뮬레이터를 중심으로(ROS 2는 한 파트에서 다룹니다), 로봇 팔과 몸의 좌표계와 기구학부터 시작해요. 카메라와 센서로 주변을 보는 인지에는 9장의 CNN이, 사람의 자세와 동작을 이해하는 로봇에는 3부의 모델 구조들이 다시 쓰입니다. 경로를 계획하고 제어하는 법을 익힌 뒤에는 강화 학습과 모방 학습으로 로봇이 스스로 배우게 하는데, 여기서 6장의 학습 루프와 18장의 실험 관리가 그대로 필요해요. 14장의 언어 모델이 로봇과 만나 말로 지시를 받아 움직이는 로봇 파운데이션 모델도 만나고, 마지막으로 시뮬레이션에서 배운 것을 현실의 로봇으로 옮기는 문제를 다룹니다.

4권 「에이전트 엔지니어링 편」에서는 14장에서 만난 언어 모델이 주인공이 됩니다. 모델을 직접 학습하는 대신 LLM API로 부르고, 모델이 답을 말하는 데서 그치지 않고 도구를 써서(MCP) 일을 끝내는 **에이전트**를 만들어요. 14장 실습에서 "적은 데이터의 LoRA는 말투와 형식을 가르치는 데 강하고, 새로운 사실을 넣는 데는 약하다"는 결론을 얻었죠. 그 약점을 메우는 방법이 4권의 RAG입니다. 사실을 모델에 외우게 하지 않고, 질문과 관련된 문서를 찾아 건네주는 거예요. 이때 문서를 화살표로 바꿔 가까운 것을 찾는 일은 이 장에서 식당을 화살표로 바꿔 비슷한 곳을 찾은 것과 같은 계산입니다. 4권은 여기에 기억, 여러 에이전트의 협업, 에이전트를 평가하고 안전하게 만드는 법을 더하고, 완성한 에이전트를 20장과 이 장에서 익힌 FastAPI로 배포합니다. 여기까지 따라온 여러분이라면 충분히 준비됐습니다.

부록 A. 실행 환경 만들기

새 컴퓨터에서 이 책의 예제를 처음 돌리려는 참이라고 해 봅시다. 무엇부터 설치해야 할까요? 1권에서는 파이썬과 uv만 있으면 거의 모든 예제가 돌아갔습니다. 2권은 조금 다릅니다. PyTorch는 내 컴퓨터에 어떤 그래픽 카드가 있느냐에 따라 **설치해야 하는 파일 자체가 달라지는** 라이브러리이고, Hugging Face에서 내려받는 모델은 한 개에 수백 MB에서 1GB가 넘습니다.

이 부록은 그런 준비를 한곳에 모았습니다. 장마다 Q&A 상자에 흩어 둔 설치 안내를 표 하나로 묶고, 운영체제와 그래픽 카드별로 PyTorch를 까는 방법, 설치가 제대로 됐는지 확인하는 스크립트, 자주 만나는 설치 문제, 그리고 내려받은 모델과 데이터가 어디에 쌓이는지까지 다룹니다. 처음 시작할 때 한 번 훑어보고, 문제가 생겼을 때 다시 펼쳐 보세요.

이 부록의 명령과 출력은 이 책을 쓴 컴퓨터(Apple M4 Max 맥북, macOS, 파이썬 3.12, uv 0.9.16)에서 직접 실행해 확인했습니다. 이 컴퓨터에는 NVIDIA 그래픽 카드가 없어서, NVIDIA GPU에 관한 내용은 PyTorch와 uv의 공식 설치 안내와 이 컴퓨터에서 할 수 있는 확인(설치 파일 목록 조회, 다른 운영체제용 의존성 계산)으로 적었습니다. 다만 16·17장은 RTX 4090 두 장이 달린 리눅스 서버에서 실행했고, 그때 실제로 겪은 설치 문제를 A.4절에 함께 적었어요. 2026년 10월에 확인한 내용이라, 여러분이 읽을 때는 버전 번호가 바뀌어 있을 수 있습니다.

A.1 이 책에서 쓰는 패키지와 확인한 버전

2권은 장마다 필요한 패키지가 조금씩 늘어납니다. 아래 표는 각 장 첫머리의 Q&A 상자에 적은 설치 명령과, 그 장의 예제를 실행해 확인한 버전을 모은 것입니다. "더하는 패키지"는 앞 장까지 설치한 것에 **새로 더하는** 것만 적었어요.

장	더하는 패키지	본문에서 밝힌 버전
1장 머신러닝이란	`numpy`, `scikit-learn`	scikit-learn 1.9

장	더하는 패키지	본문에서 밝힌 버전
2장 데이터 준비	`pandas`	scikit-learn 1.9, pandas 3.0
3장 고전 머신러닝	(2장과 같음, `joblib`·`scipy`는 함께 설치됨)	scikit-learn 1.9
4장 텐서	`torch`	PyTorch 2.14
5장 자동 미분	(4장과 같음, 데이터는 scikit-learn 내장)	—
6장 nn.Module	`torchvision`	—
7장 데이터 파이프라인	`pillow`	PyTorch 2.14, torchvision 0.29
8장 학습 잘 시키기	`matplotlib`	—
9장 CNN	(6·8장과 같음)	—
10장 전이 학습	(6장과 같음)	PyTorch 2.14.1, torchvision 0.29.1
11장 RNN·LSTM	`statsmodels`	PyTorch 2.14, statsmodels 0.15, pandas 3.0, matplotlib 3.11
12장 트랜스포머	(4장과 같음)	PyTorch 2.14
13장 한국어 NLP	`transformers`, `datasets`, `tokenizers`, `kiwipiepy`	transformers 5.20.2, datasets 5.1.0, tokenizers 0.23.3, kiwipiepy 0.24.1, scikit-learn 1.9.1
14장 LLM과 LoRA	`peft` (`accelerate`가 함께 설치됨)	transformers 5.20.2, peft 0.21.2, accelerate 1.15.0
15장 생성 모델	(6·8장과 같음)	torchvision 0.29.1, matplotlib 3.11.2

장	더하는 패키지	본문에서 밝힌 버전
16장 GPU와 성능	(6장과 같음, `torch.compile`용 `triton`은 리눅스에서 함께 설치됨)	PyTorch 2.14.1+cu126, torchvision 0.29.1, triton 3.8.0 (RTX 4090 서버)
17장 분산 학습	(6장과 같음, 통신 라이브러리 NCCL은 리눅스용 PyTorch에 들어 있음)	PyTorch 2.14.1+cu126, NCCL 2.29.3 (RTX 4090 두 장)
18장 실험 관리	`tensorboard`, `mlflow`, `optuna`, `pyyaml`	TensorBoard 2.21.0, MLflow 3.17.2, Optuna 5.0.0, PyYAML 6.0.3
19장 ONNX와 양자화	`onnx`, `onnxscript`, `onnxruntime`, `torchao`	onnx 1.23.2, onnxscript 0.7.2, onnxruntime 1.31.0, torchao 0.18.0
20장 모델 서빙	`fastapi[standard]`, `pytest`	FastAPI 0.156.4, Pydantic 2.14.1, uvicorn 0.54.0, python-multipart 0.0.32, Pillow 13.0.0, pytest 9.1.1
21장 종합 프로젝트	(앞 장들과 같음, 서비스 테스트에 `httpx2`)	pandas 3.0.6, FastAPI 0.156.8, Pydantic 2.14.1, pytest 9.1.1, httpx2 2.16.0

모든 장은 **파이썬 3.12**에서 실행했고, PyTorch는 2.14.1입니다. "—"는 그 장 본문에 버전을 따로 적지 않았다는 뜻인데, 같은 시기에 같은 환경에서 실행했으니 바로 위아래 장과 같은 버전이라고 보면 됩니다. 다만 20·21장의 실습 서비스는 프로젝트마다 따로 만든 잠금 파일(`uv.lock`)로 설치해서, 표에 적은 FastAPI와 pytest 판이 서로 조금 다릅니다.

Q

Q. 표의 버전과 똑같이 맞춰야 하나요? 꼭 그럴 필요는 없습니다. 판 번호(가운데 숫자)가 같으면 예제는 거의 그대로 돌아가요. 다만 학습 결과의 소수점 아래 숫자, 경고 문구, 오류 메시지 문장은 버전에 따라 달라질 수 있습니다. 본문과 숫자를 한 자리까지 맞춰 보고 싶다면 `uv add "torch==2.14.1"`처럼 버전을 고정해서 설치하세요.

A.2 uv로 2권 프로젝트 만들기

1권 1장에서 만든 것처럼 프로젝트 폴더 하나를 만들고 그 안에서 모든 예제를 실행하는 것이 가장 편합니다. 1권 부록 A에서 소개한 `uv init`으로 시작해 볼까요?

```
$ uv init pytorch-book --python 3.12
Initialized project `pytorch-book` at `.../pytorch-book`
$ cd pytorch-book
$ cat .python-version
3.12
```

`--python 3.12`를 주면 uv가 `.python-version` 파일에 3.12를 적어 두고, 이 폴더에서 `uv run`을 할 때마다 파이썬 3.12를 씁니다. 컴퓨터에 3.12가 없으면 uv가 알아서 내려받아요.

이제 2권의 중심인 PyTorch부터 넣습니다. macOS와 리눅스라면 이 한 줄이면 됩니다(윈도우에서 NVIDIA GPU를 쓰려면 A.4절을 먼저 보세요).

```
$ uv add torch torchvision numpy
...
 + torch==2.14.1
 + torchvision==0.29.1
 + typing-extensions==4.16.0
$ uv run python -c "import torch; print(torch.__version__)"
2.14.1
```

장을 따라가면서 필요한 패키지를 그때그때 더해도 되고, 처음부터 한꺼번에 넣어도 됩니다. 아래는 2권의 1~15장, 18~21장에서 쓰는 패키지를 모두 넣는 명령이에요. 이 책의 컴퓨터에서 실행해 보니 패키지 191개가 서로 충돌 없이 한 프로젝트에 들어갔고, 가상 환경 폴더(`.venv`)는 약 1.5GB가 됐습니다.

```
uv add torch torchvision numpy scikit-learn pandas matplotlib statsmodels pillow
uv add transformers datasets tokenizers kiwipiepy peft
uv add tensorboard mlflow optuna pyyaml
uv add onnx onnxscript onnxruntime torchao "fastapi[standard]"
uv add --dev pytest
```

`pytest`만 `--dev`로 넣은 이유는 1권 부록 A에서 본 그대로입니다. 테스트 도구는 개발할 때만 필요하고, 완성한 프로그램을 배포할 때는 필요 없으니까요. 20장에서 Docker 이미지를 만들 때 이 구분

이 실제로 쓰입니다.

> **Q. 이미 1권 프로젝트가 있는데 거기에 더해도 되나요?** 됩니다. 다만 2권 패키지는 크기가 커서(위 명령 전체로 약 1.5GB), 1권 프로젝트와 폴더를 나눠 두는 편이 정리하기 쉽습니다. 프로젝트를 지우고 싶을 때 폴더째 지우면 가상 환경까지 함께 사라지는 것도 uv 프로젝트의 장점이에요.

A.3 macOS: Apple 실리콘과 MPS

맥이라면 설치는 가장 간단합니다. `uv add torch`가 설치하는 PyTorch에 Apple 실리콘(M1~M4 등)의 GPU를 쓰는 **MPS(Metal Performance Shaders)** 기능이 이미 들어 있거든요. 따로 드라이버나 도구를 설치할 필요가 없습니다. 이 책의 예제도 대부분 이 방식으로, 맥북 한 대에서 실행했어요.

조건이 두 가지 있습니다. PyTorch 2.14.1의 맥용 설치 파일은 이름이 `macosx_14_0_arm64`로 끝나는 것 하나뿐이에요. 즉 **Apple 실리콘 맥**이어야 하고, **macOS 14(Sonoma) 이상**이어야 합니다. 이 책의 컴퓨터에서 인텔 맥(x86_64)을 대상으로 설치할 버전을 계산하게 해 봤더니, uv는 인텔 맥용 파일이 있는 마지막 버전인 **2.2.2**를 골랐습니다. 2.2.2는 이 책의 예제가 쓰는 기능 일부가 없는 오래된 버전이라, 인텔 맥이라면 A.5절의 Colab을 쓰는 편이 낫습니다.

MPS는 거의 모든 예제에서 잘 동작하지만, CUDA와 다른 점이 몇 가지 있습니다.

- **64비트 실수(float64)를 쓸 수 없습니다.** 아래 A.7절에서 실제 오류를 보여 드립니다.
- 아직 MPS에서 지원하지 않는 연산을 만나면 오류가 납니다. 이때 환경 변수 `PYTORCH_ENABLE_MPS_FALLBACK=1`을 주면 그 연산만 CPU에서 계산하도록 할 수 있어요(PyTorch 공식 문서의 MPS 환경 변수 목록). 대신 그 부분은 느려집니다.
- 같은 코드라도 CPU와 결과가 소수점 아래에서 조금씩 다를 수 있습니다. 그래서 이 책에서 정확한 숫자를 인용하는 작은 학습 예제는 대부분 CPU에서 돌렸고, MPS로 돌린 예제에는 그렇다고 밝혀 두었어요.

```
# MPS에서 지원하지 않는 연산을 CPU로 돌려서 계산하게 하기 (macOS·리눅스 셸)
PYTORCH_ENABLE_MPS_FALLBACK=1 uv run python 학습.py
```

A.4 리눅스·윈도우와 NVIDIA GPU: CUDA

NVIDIA 그래픽 카드가 있는 컴퓨터라면 이야기가 조금 복잡해집니다. PyTorch는 NVIDIA의 계산 도구인 CUDA의 버전별로 설치 파일을 따로 만들고, 운영체제마다 기본으로 설치되는 파일도 다르기 때문이에요. 순서대로 보겠습니다.

먼저 그래픽 카드 드라이버

CUDA로 계산하려면 NVIDIA 그래픽 카드 드라이버가 설치되어 있어야 합니다. 드라이버가 설치되어 있으면 터미널에서 `nvidia-smi` 명령이 그래픽 카드 이름과 드라이버 버전, 그 드라이버가 지원하는 CUDA 버전을 보여 줍니다. 이 명령이 없다고 나오면 NVIDIA 홈페이지에서 그래픽 카드에 맞는 드라이버부터 설치하세요. PyTorch 설치 파일에는 CUDA 계산에 필요한 라이브러리가 함께 들어 있어서, **CUDA 개발 도구(CUDA Toolkit)를 따로 설치할 필요는 없습니다.** 필요한 것은 드라이버뿐이에요.

기본 설치는 무엇을 깔까요?

uv 공식 문서의 PyTorch 안내에 따르면, 아무 설정 없이 `uv add torch`를 하면 PyPI(파이썬 패키지 공용 저장소)에서 설치하는데, PyPI의 PyTorch는 **리눅스에서는 GPU(CUDA)용, 윈도우와 macOS에서는 CUDA가 빠진 빌드**입니다(macOS 빌드에는 A.3절의 MPS가 들어 있어요). 이 책의 컴퓨터에서 운영체제별로 PyTorch 2.14.1이 함께 끌어오는 패키지를 계산해 봤더니 정말 그랬어요.

```
$ uv pip compile req.in --python-version 3.12 --python-platform x86_64-unknown-linux-gnu
...
nvidia-cuda-runtime==13.0.96
nvidia-cudnn-cu13==9.24.0.43
...
torch==2.14.1
triton==3.8.0
$ uv pip compile req.in --python-version 3.12 --python-platform x86_64-pc-windows-msvc
...
torch==2.14.1
```

`req.in`에는 `torch==2.14.1` 한 줄만 적었습니다. 리눅스용 결과에는 `nvidia-`로 시작하는 CUDA 13 라이브러리가 줄줄이 따라오고, 윈도우용 결과에는 하나도 없어요. 정리하면 이렇습니다.

운영체제	`uv add torch`로 설치되는 것	NVIDIA GPU를 쓰려면
리눅스	CUDA 13용 PyTorch	드라이버가 CUDA 13을 지원하면 그대로 사용
윈도우	CPU 전용 PyTorch	PyTorch 인덱스를 지정해서 다시 설치 (아래)
macOS	MPS가 들어 있는 PyTorch	해당 없음(맥용 CUDA 빌드는 없음)

PyTorch 인덱스 지정하기

윈도우에서 GPU를 쓰려면, 또는 리눅스에서 드라이버가 오래되어 더 낮은 CUDA 버전이 필요하면, PyTorch가 직접 운영하는 설치 파일 저장소(**인덱스**)를 지정합니다. 주소는 `https://download.pytorch.org/whl/` 뒤에 CUDA 버전을 붙인 꼴이에요. `cu126`은 CUDA 12.6, `cu130`은 CUDA 13.0이라는 뜻입니다.

어떤 CUDA 버전용 파일이 있는지는 PyTorch 버전마다 다릅니다. 2026년 10월에 이 인덱스들을 직접 조회해 보니 PyTorch 2.14.1은 `cpu`, `cu126`, `cu130`, `cu132`에 리눅스용과 윈도우용 파일이 있었고, `cu128`에는 없었어요. 여러분이 설치할 때는 PyTorch 홈페이지의 설치 안내(Get Started) 페이지에서 운영체제, 패키지 도구, CUDA 버전을 고르면 나오는 명령을 기준으로 삼으세요.

uv 프로젝트에서는 `pyproject.toml`에 인덱스를 적어 둡니다. 아래는 uv 공식 문서의 예를 CUDA 13.0으로 바꾼 것이에요.

```
[[tool.uv.index]]
name = "pytorch-cu130"
url = "https://download.pytorch.org/whl/cu130"
explicit = true

[tool.uv.sources]
torch = [
  { index = "pytorch-cu130", marker = "sys_platform == 'linux' or sys_platform == 'win32'" },
]
torchvision = [
  { index = "pytorch-cu130", marker = "sys_platform == 'linux' or sys_platform == 'win32'" },
```

```
]
```

설정이 두 부분으로 나뉘어 있죠?

- `[[tool.uv.index]]`는 "이런 이름의 저장소가 있다"고 알려 줍니다. `explicit = true`는 **이 저장소를 지정한 패키지에만 쓰라**는 뜻이에요. 이 줄이 없으면 numpy나 pandas까지 이 저장소에서 찾으려 할 수 있습니다.
- `[tool.uv.sources]`는 `torch`와 `torchvision`을 그 저장소에서 가져오라고 정합니다. `marker`의 조건 덕분에 리눅스와 윈도우에서만 이 저장소를 쓰고, 맥에서는 PyPI의 MPS용 PyTorch를 그대로 씁니다.

이렇게 적은 뒤 `uv add torch torchvision`(이미 넣었다면 `uv lock`)을 실행하면 됩니다. 이 책의 컴퓨터에서 이 설정으로 잠금 파일을 만들어 보니, `uv.lock`에 맥용 `torch 2.14.1`(PyPI)과 리눅스·윈도우용 `torch 2.14.1+cu130`(PyTorch 인덱스)이 함께 기록됐어요. 잠금 파일 하나로 세 운영체제를 모두 챙기는 셈입니다. 16·17장을 실행한 리눅스 서버에서는 이 `cu130` 빌드가 바로 돌지 않았습니다. 서버의 드라이버(550 버전)가 CUDA 12.4까지만 지원해서 `NVIDIA driver on your system is too old` 오류가 났거든요. 그래서 위 설정의 `cu130`을 모두 `cu126`으로 바꿔 `torch 2.14.1+cu126`을 설치했고, 이 빌드로 두 GPU 모두에서 학습이 돌았습니다. `nvidia-smi`가 보여 주는 CUDA 버전보다 높은 빌드를 깔면 이렇게 되니, 드라이버를 올릴 수 없는 서버라면 그보다 낮거나 가까운 버전의 인덱스를 고르세요.

pip을 쓰는 환경이라면

Colab이나 회사 서버처럼 uv 프로젝트가 아닌 곳에서는 PyTorch 설치 안내 페이지의 pip 명령을 씁니다. 그 페이지가 보여 주는 명령은 이런 꼴이에요.

```
pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu130
```

uv에는 이 고르기를 대신해 주는 기능도 있습니다. `uv pip install --torch-backend=auto torch`는 설치된 NVIDIA 드라이버(AMD·인텔 GPU도)를 조사해서 가장 잘 맞는 PyTorch 인덱스를 고르고, 맞는 GPU가 없으면 CPU 전용 인덱스를 씁니다(uv 공식 문서). 이 책의 uv 0.9.16에도 이 옵션이 있었고, 맥에서 시험 삼아 실행하니 PyPI의 `torch==2.14.1`을 골랐어요. 이 기능은 `uv add`가 아니라 `uv pip` 명령에서만 쓸 수 있습니다.

Q

Q. 설치한 PyTorch가 GPU용인지 어떻게 아나요? PyTorch 인덱스에서 받은 파일은 버전 뒤에 `+cu130`, `+cpu`처럼 꼬리표가 붙습니다(설치 파일 이름이 `torch-2.14.1+cu130-...`). 파이썬 안에서는 `torch.version.cuda`를 보세요. CPU 전용이나 맥용 빌드는 `None`이고, CUDA용 빌드는 CUDA 버전 문자열이 나옵니다. 그다음 `torch.cuda.is_available()`이 `True`인지 확인하면 드라이버까지 연결된 것입니다. A.6절의 스크립트가 이 두 가지를 함께 출력해요.

A.5 GPU가 없을 때: Colab과 클라우드 GPU

GPU가 없어도 이 책은 따라갈 수 있습니다. 4장 Q&A에서 말했듯 예제를 노트북 CPU나 맥의 MPS에서 몇 분 안에 끝나도록 크기를 맞췄거든요. 그래도 15장 확산 모델이나 13·14장의 미세조정처럼 계산이 많은 예제를 빠르게 돌리고 싶다면 GPU를 빌려 쓰는 방법이 있습니다.

가장 쉬운 방법은 1권 1장의 "설치가 어렵다면: Colab에서 시작하기"에서 만난 **Google Colab**입니다. Colab은 인터넷 너머의 컴퓨터를 빌려 쓰는 서비스라, 메뉴에서 런타임 유형을 GPU로 바꾸면 NVIDIA GPU가 달린 컴퓨터를 쓸 수 있어요. 무료 사용량에는 한도가 있고, 어떤 GPU를 얼마나 오래 쓸 수 있는지는 구글이 수시로 바꾸니 그때그때 Colab 안내를 확인하세요. Colab에는 PyTorch가 대개 미리 설치되어 있어서 `import torch`부터 바로 시작할 수 있고, 없는 패키지는 노트북 칸에 `!pip install 패키지`로 넣습니다.

Colab을 쓸 때 기억할 점이 세 가지 있습니다.

- 빌린 컴퓨터는 한동안 쓰지 않으면 **반납되고, 그 안의 파일도 함께 사라집니다.** 학습한 모델(`.pt` 파일)은 구글 드라이브에 저장하세요.
- 장치를 고르는 코드는 바꿀 필요가 없습니다. 아래 A.6절의 `장치_고르기()`가 Colab에서는 `cuda`를 고릅니다.
- Colab의 PyTorch 버전은 이 책의 버전과 다를 수 있습니다. 첫 칸에서 `torch.__version__`을 확인해 두세요.

Colab보다 오래, 크게 쓰려면 클라우드 회사(AWS, Google Cloud, Azure 등)나 GPU 대여 전문 서비스에서 GPU 서버를 시간 단위로 빌릴 수 있습니다. 서비스마다 화면과 요금이 달라서 이 책에서 따라 하기를 싣지는 않았지만, 일반적으로 지킬 점은 같아요.

- **쓰지 않을 때는 반드시 꺼 두세요.** 대부분 켜져 있는 시간만큼 요금이 나갑니다.
- 빌린 서버는 대개 리눅스입니다. A.4절의 리눅스 설치가 그대로 적용되고, 서버에 드라이버가

미리 설치된 이미지를 고르면 `nvidia-smi`부터 바로 됩니다.

- 코드와 데이터는 git이나 클라우드 저장소로 옮기고, 결과는 끄기 전에 내려받으세요. 18장의 실험 기록 습관이 이때 빛을 봅니다.

A.6 장치 고르기와 설치 확인 스크립트

장치_고르기() 다시 보기

4장 4.8절에서 만든 함수를 다시 한번 보겠습니다. 이 책의 거의 모든 장이 이 함수를 그대로 씁니다.

```
def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")
```

확인하는 순서가 곧 우선순위입니다. NVIDIA GPU가 있으면 `cuda`, 없고 Apple 실리콘이면 `mps`, 둘 다 없으면 `cpu`예요. 이 함수 덕분에 같은 코드가 맥, 윈도우, 리눅스, Colab 어디서나 고칠 곳 없이 돌아갑니다. 쓰는 법은 6장 6.9절에서 본 세 단계 그대로입니다.

```
import torch
from torch import nn


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")


장치 = 장치_고르기()
```

```
torch.manual_seed(0)
모델 = nn.Linear(4, 2).to(장치)            # 1) 모델을 장치로 (6장 6.9절)
x = torch.randn(8, 4).to(장치)             # 2) 데이터도 같은 장치로
출력 = 모델(x)
print(출력.device, tuple(출력.shape))
print(출력.argmax(dim=1).cpu().tolist())    # 3) 출력은 CPU로 꺼내서 파이썬 값으로
```

```
mps:0 (8, 2)
[1, 0, 0, 1, 1, 0, 0, 1]
```

모델과 데이터를 같은 장치로 옮기고, 결과를 파이썬 값으로 쓸 때는 `.cpu()`로 꺼냅니다. 이 컴퓨터에서는 `mps:0`, 즉 첫 번째 MPS 장치에서 계산됐어요. 셋 중 하나라도 빠뜨리면 4장 4.8절에서 본 "같은 장치에 있어야 한다"는 오류를 만납니다.

설치 확인 스크립트

새 컴퓨터에 환경을 만들었거나 Colab에 처음 접속했다면 아래 스크립트부터 실행해 보세요. 파이썬과 PyTorch 버전, CUDA·MPS를 쓸 수 있는지, 고른 장치에서 실제로 계산이 되는지를 한 번에 봅니다. `설치_확인.py`로 저장하고 `uv run python 설치_확인.py`로 실행하면 됩니다.

```
# 설치_확인.py: 새 컴퓨터에서 가장 먼저 실행해 보는 점검 스크립트
import platform
import sys

import torch


def 장치_고르기() -> torch.device:
    """쓸 수 있는 가장 빠른 장치를 고른다."""
    if torch.cuda.is_available():
        return torch.device("cuda")       # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")        # Apple 실리콘 GPU
    return torch.device("cpu")
```

```
print("파이썬     :", sys.version.split()[0])
print("운영체제   :", platform.system(), platform.machine())
print("PyTorch    :", torch.__version__)
print("CUDA 빌드  :", torch.version.cuda)           # CPU·MPS용 빌드면 None
print("CUDA 사용  :", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU 이름   :", torch.cuda.get_device_name(0))
print("MPS 사용   :", torch.backends.mps.is_available())

장치 = 장치_고르기()
print("고른 장치  :", 장치)

# 같은 계산을 CPU와 고른 장치에서 해 보고 결과를 비교한다
torch.manual_seed(0)
a = torch.randn(512, 512)
b = torch.randn(512, 512)
cpu_결과 = a @ b
장치_결과 = (a.to(장치) @ b.to(장치)).cpu()
print("행렬 곱 모양:", tuple(장치_결과.shape))
print("CPU와 같은가:", torch.allclose(cpu_결과, 장치_결과, atol=1e-3))
```

이 책의 컴퓨터에서 실행한 결과입니다.

```
파이썬     : 3.12.10
운영체제   : Darwin arm64
PyTorch    : 2.14.1
CUDA 빌드  : None
CUDA 사용  : False
MPS 사용   : True
고른 장치  : mps
행렬 곱 모양: (512, 512)
CPU와 같은가: True
```

읽는 법은 이렇습니다.

- **CUDA 빌드**가 `None`이면 설치된 PyTorch에 CUDA 기능이 아예 없다는 뜻입니다. 맥에서는 정상이에요. NVIDIA GPU가 있는 윈도우 컴퓨터에서 `None`이 나오면 CPU 전용 PyTorch가 설치

된 것이니 A.4절대로 다시 설치하세요.

- **CUDA 빌드**에 버전이 나오는데 **CUDA 사용**이 `False`라면 PyTorch는 맞게 설치됐지만 드라이버와 연결되지 않은 경우입니다. `nvidia-smi`가 되는지, 드라이버가 그 CUDA 버전을 지원하는지 확인하세요.
- 마지막 두 줄은 실제로 그 장치에서 행렬 곱을 하고 CPU 결과와 비교한 것입니다. GPU는 계산 순서가 달라 소수점 아래 끝자리가 조금 다를 수 있어서 `allclose`에 여유(`atol=1e-3`)를 줬어요. `True`면 그 장치로 학습할 준비가 된 것입니다.

NVIDIA GPU가 있는 컴퓨터라면 `GPU 이름` 줄이 하나 더 나오고 `고른 장치`가 `cuda`가 될 것입니다. 이 책의 컴퓨터에서는 확인할 수 없었던 출력이라 결과를 싣지 않았어요.

A.7 자주 만나는 설치 문제

설치하다 막히는 지점은 대개 정해져 있습니다. 이 책의 컴퓨터에서 직접 재현해 본 것만 모았어요.

파이썬 버전이 맞지 않을 때

PyTorch 2.14.1은 파이썬 3.10부터 3.14까지만 설치 파일을 냅니다. 파이썬 3.9 환경에서 설치하려고 하면 uv가 이렇게 알려 줘요.

```
hint: You require CPython 3.9 (`cp39`), but we only found wheels
for `torch` (v2.14.1) with the following Python implementation tags:
`cp310`, `cp311`, `cp312`, `cp313`, `cp314`
```

"`cp39`가 필요한데 `cp310`~`cp314`용 파일만 찾았다"는 뜻입니다. `cp312`가 CPython 3.12라는 뜻이에요. 해결은 간단합니다. 프로젝트의 파이썬을 바꾸면 돼요(`uv python pin 3.12`). 1권 부록 A에서 본 대로 uv가 3.12를 알아서 내려받습니다.

운영체제용 설치 파일이 없을 때

같은 종류의 오류가 운영체제 때문에 나기도 합니다. A.3절에서 본 인텔 맥이나 macOS 13 이하에서 `torch==2.14.1`을 설치하려 하면 "이 플랫폼용 파일만 있다"는 안내가 나와요.

```
hint: Wheels are available for `torch` (v2.14.1) on the following
platforms: `manylinux_2_28_aarch64`, `manylinux_2_28_x86_64`,
`macosx_14_0_arm64`, `win_amd64`
```

목록이 곧 설치할 수 있는 곳입니다. 리눅스(인텔·AMD 계열 `x86_64`와 ARM 계열 `aarch64`), macOS 14 이상의 Apple 실리콘, 64비트 윈도우예요. 여기에 없는 컴퓨터라면 버전을 고정하지 않았을 때 uv가 설치할 수 있는 옛 버전을 고르는데, 그러면 이 책의 예제가 돌지 않을 수 있습니다. 이럴 때는 A.5절의 Colab이 가장 빠른 해결책이에요.

GPU가 있는데 CUDA가 False일 때

NVIDIA GPU가 달린 윈도우 컴퓨터에서 `uv add torch`만 했다면, A.4절에서 본 대로 CPU 전용 PyTorch가 설치되어 있습니다. A.6절 스크립트의 `CUDA 빌드`가 `None`으로 나오는 것으로 확인할 수 있어요. `pyproject.toml`에 PyTorch 인덱스를 적고 다시 설치하세요. `CUDA 빌드`에 버전이 나오는데도 `False`라면 드라이버 문제입니다.

MPS에서 float64 오류

맥에서 가장 자주 만나는 오류입니다. NumPy 배열의 기본 실수 자료형은 float64인데, 이걸 `torch.from_numpy`로 바꾸면 자료형이 그대로 따라오고, MPS는 float64를 지원하지 않아요.

```
import numpy as np
import torch

배열 = np.array([0.1, 0.2, 0.3])            # NumPy의 기본 실수 자료형은 float64
t = torch.from_numpy(배열)
print("자료형:", t.dtype)

if torch.backends.mps.is_available():
    try:
        t.to("mps")
    except TypeError as 오류:
        print("오류:", 오류)
    print("고친 뒤:", t.float().to("mps").dtype)  # float32로 바꾼 다음 옮긴다
```

```
자료형: torch.float64
오류: Cannot convert a MPS Tensor to float64 dtype as the MPS framework doesn't support float64.
↪  Please use float32 instead.
고친 뒤: torch.float32
```

오류 문장이 해결책까지 알려 줍니다. "float32를 쓰라"죠. `.float()`로 float32로 바꾼 뒤 옮기면 됩

니다. `.float()`는 4장 4.3절에서 본 `.to(torch.float32)`를 줄여 쓴 것이에요. pandas에서 꺼낸 값, scikit-learn 데이터셋도 대개 float64이니 텐서로 만들 때 `dtype=torch.float32`를 붙이는 습관을 들이면 이 오류를 만나지 않아요.

NumPy 없이 PyTorch만 설치했을 때

PyTorch는 NumPy 없이도 설치되고 대부분 동작합니다. 그런데 `.numpy()`처럼 NumPy와 주고받는 기능을 쓰는 순간 오류가 나요. 이 책의 컴퓨터에서 `uv run --with torch`로 NumPy 없는 환경을 만들어 `torch.ones(2).numpy()`를 실행했더니 이렇게 나왔습니다.

```
RuntimeError: Numpy is not available
```

사실 신호는 그 전에 이미 옵니다. NumPy가 없는 환경에서는 `import torch`를 할 때 `UserWarning: Failed to initialize NumPy: No module named 'numpy'`라는 경고가 먼저 찍혀요. 경고라서 지나치기 쉬운데, 이 문장을 봤다면 NumPy가 빠진 것입니다.

`uv add numpy`로 해결됩니다. A.2절의 명령처럼 처음부터 `torch`와 `numpy`를 함께 넣어 두면 만날 일이 없어요.

장치가 섞였을 때

설치 문제는 아니지만 처음 GPU를 쓸 때 가장 많이 만나는 오류라 함께 적어 둡니다. 4장 4.8절에서 본 `Expected all tensors to be on the same device`예요. 모델과 데이터, 그리고 손실 계산에 들어가는 정답 텐서까지 모두 같은 장치에 있어야 합니다. A.6절의 세 단계(모델 옮기기, 데이터 옮기기, 결과 꺼내기)를 확인하세요.

A.8 내려받은 모델은 어디에 쌓일까: 캐시와 디스크 용량

Hugging Face 캐시와 HF_HOME

13·14·20장에서 `from_pretrained("klue/bert-base")`처럼 모델을 불러오면, 처음 한 번만 인터넷에서 내려받고 그다음부터는 컴퓨터에 저장해 둔 것을 씁니다. 이렇게 다시 쓰려고 저장해 두는 곳을 **캐시(cache)** 라고 해요. 10장의 torchvision 사전학습 가중치도 마찬가지로 캐시에 저장됩니다. 지금 내 컴퓨터의 캐시 위치는 이렇게 확인할 수 있어요.

```
import os
from pathlib import Path

import torch
from huggingface_hub import constants


def 짧게(경로: str | Path) -> str:
    """사용자 폴더 부분을 ~로 줄여서 보여 준다."""
    return str(경로).replace(str(Path.home()), "~")


print("HF_HOME 지정  :", os.environ.get("HF_HOME", "(지정 안 함)"))
print("Hugging Face  :", 짧게(constants.HF_HOME))
print("  모델 캐시   :", 짧게(constants.HF_HUB_CACHE))
print("torchvision   :", 짧게(torch.hub.get_dir()))
print("데이터 폴더   :", os.environ.get("BOOK_DATA_DIR", "data"))
```

```
HF_HOME 지정  : (지정 안 함)
Hugging Face  : ~/.cache/huggingface
  모델 캐시   : ~/.cache/huggingface/hub
torchvision   : ~/.cache/torch/hub
데이터 폴더   : data
```

아무것도 지정하지 않으면 Hugging Face는 사용자 폴더의 `.cache/huggingface`를, torchvision은 `.cache/torch`를 씁니다. 모델 파일은 그 안의 `hub` 폴더에 들어가요. 위치를 바꾸고 싶으면 환경 변수 **`HF_HOME`** 을 지정합니다. 사용자 폴더가 있는 디스크가 작아서 외장 디스크에 두고 싶을 때 유용해요(torchvision 쪽은 환경 변수 `TORCH_HOME`이 같은 역할을 합니다).

```
# macOS·리눅스: 이번 실행에만 적용
HF_HOME=/Volumes/외장/hf uv run python 미세조정.py
```

```
# 윈도우 PowerShell: 이 창에서만 적용
$env:HF_HOME = "D:\hf"
uv run python 미세조정.py
```

계속 쓰려면 셸 설정 파일(macOS의 `~/.zshrc` 등)이나 윈도우의 환경 변수 설정에 넣어 두세요. 1권 19장과 2권 20장에서처럼, 설정은 코드에 박지 않고 환경 변수로 바꾸는 것이 원칙입니다.

얼마나 차지할까

이 책을 쓰면서 1~15장, 18~21장의 예제를 모두 실행하고 나니 캐시와 데이터 폴더가 이만큼 쌓였습니다. Hugging Face 모델은 `hf cache ls` 명령(아래)으로, 나머지는 폴더 크기를 재서 적었어요.

무엇	쓰는 장	크기
Qwen2.5-0.5B-Instruct (언어 모델)	14·20장	약 1,000MB
klue/bert-base (한국어 BERT)	13장	약 446MB
xlm-roberta-base, bert-base-multilingual-cased (토크나이저만)	13장	약 17MB
KLUE 데이터셋 (뉴스 제목 주제 분류)	13장	약 5MB(내려받은 파일) + 15MB(변환본)
torchvision 사전학습 가중치 (ResNet18, MobileNetV3-Small, EfficientNet-B0)	10장	45MB + 9.8MB + 20MB
EuroSAT 위성 사진 (압축 파일과 푼 파일)	10장	약 223MB
FashionMNIST	6·9·15·18~20장	약 82MB
NSMC 네이버 영화 리뷰	13장	약 19MB
가상 환경 `.venv` (A.2절의 패키지 전체)	전체	약 1.5GB

모두 합치면 4GB 가까이 됩니다. 여기에 장마다 저장하는 체크포인트와 13장에서 미세조정한 모델(이 책의 컴퓨터에서 약 420MB)이 더해지니, **여유 공간을 10GB쯤 확보해 두면** 넉넉해요.

Hugging Face 캐시에 무엇이 있는지는 `huggingface_hub`에 들어 있는 `hf` 명령으로 볼 수 있습니다. `transformers`를 설치하면 함께 설치돼요. 이 책의 캐시를 조회한 결과입니다.

```
$ uv run hf cache ls
id  size    last_accessed   last_modified   refs
dataset/klue/klue   5.0M    4 hours ago 4 hours ago ['main']
model/FacebookAI/xlm-roberta-base   14.2M   4 hours ago 4 hours ago ['main']
model/Qwen/Qwen2.5-0.5B-Instruct    999.6M  2 hours ago 4 hours ago ['main']
model/google-bert/bert-base-multilingual-cased  3.0M    4 hours ago 4 hours ago ['main']
model/klue/bert-base    445.7M  3 hours ago 4 hours ago ['main']

Found 5 repo(s) for a total of 5 revision(s) and 1.5G on disk.
```

다 본 모델은 `hf cache rm model/Qwen/Qwen2.5-0.5B-Instruct`처럼 지울 수 있습니다. 지워도 다음에 필요하면 다시 내려받을 뿐이니 걱정하지 않아도 돼요. torchvision 가중치와 데이터 폴더는 그냥 파일이라 폴더째 지우면 됩니다.

A.9 데이터 폴더 관례: BOOK_DATA_DIR

이 책의 예제는 인터넷에서 받는 데이터셋을 현재 폴더 아래 `data/`에 저장합니다. 6장의 `datasets.FashionMNIST("data", ...)`처럼요. 그런데 장마다 폴더를 따로 만들어 예제를 실행하면 같은 FashionMNIST를 장마다 다시 받게 됩니다. 그래서 2·3·12장의 예제는 데이터 폴더를 환경변수 `BOOK_DATA_DIR` 로 바꿀 수 있게 썼어요. 규칙은 하나뿐입니다. "`BOOK_DATA_DIR`이 있으면 그 폴더, 없으면 `data`."

```
import os
from pathlib import Path


def 데이터_폴더() -> Path:
    """예제 데이터를 둘 폴더. BOOK_DATA_DIR이 있으면 그곳, 없으면 ./data"""
    return Path(os.environ.get("BOOK_DATA_DIR", "data"))


os.environ.pop("BOOK_DATA_DIR", None)           # 지정하지 않았을 때
print(데이터_폴더())
```

```
os.environ["BOOK_DATA_DIR"] = "/Volumes/외장/책데이터"  # 지정했을 때 (예)
print(데이터_폴더())
print(데이터_폴더() / "FashionMNIST")
```

```
data
/Volumes/외장/책데이터
/Volumes/외장/책데이터/FashionMNIST
```

`os.environ.get(이름, 기본값)`은 1권 17장에서 본 그대로, 환경 변수가 없으면 기본값을 돌려줍니다. 지정하지 않으면 상대 경로 `data`이니 지금 실행하는 폴더 아래에, 지정하면 그 폴더에 데이터가 모여요. 다른 장의 예제도 `"data"`라고 적힌 자리를 `데이터_폴더()`로 바꾸면 모든 장이 한 폴더를 같이 쓰게 됩니다.

```
# 모든 장의 예제가 같은 데이터 폴더를 쓰게 하기 (macOS·리눅스)
export BOOK_DATA_DIR="$HOME/book-data"
uv run python 06_옷분류.py
```

정리하면, 이 책의 예제가 컴퓨터에 남기는 것은 세 곳입니다. 패키지는 프로젝트의 `.venv`, 모델은 `HF_HOME`과 torchvision 캐시, 데이터는 `BOOK_DATA_DIR`(기본 `data/`)이에요. 디스크가 부족하거나 컴퓨터를 옮길 때 이 세 곳만 기억하면 됩니다.

부록 B. 꼭 필요한 수학 직관

딥러닝을 공부하려면 수학을 얼마나 알아야 할까요? 서점에 가면 선형대수, 미적분, 확률과 통계 책이 줄지어 있어서 "이걸 다 떼야 시작할 수 있나" 싶어집니다. 다행히 이 책을 따라가는 데 필요한 수학은 생각보다 적어요. 이 부록은 **본문에 실제로 나온 수학만** 골라서, 공식을 증명하는 대신 "이게 무슨 뜻이고 왜 쓰는지"를 코드로 확인하며 정리합니다.

본문을 읽다가 내적, 기울기, 로그, 소프트맥스 같은 말에서 걸리면 이 부록의 해당 절을 펼쳐 보세요. 절마다 마지막에 그 개념이 나오는 장을 적어 두었습니다. 거꾸로, 이 부록을 먼저 한 번 훑고 본문으로 들어가도 좋아요.

몇 가지 약속이 있습니다.

- 수식은 인쇄 글꼴 문제 때문에 그리스 문자나 위 첨자 없이 **말과 파이썬 코드로** 씁니다. "x의 제곱"은 `x ** 2`, "e의 x제곱"은 `exp(x)`, 자연로그는 `log(x)`로 적어요. 본문의 장들과 같은 방식입니다.
- 모든 코드는 PyTorch 2.14.1과 NumPy로 이 책의 컴퓨터에서 실행했고, 출력은 그 결과를 그대로 옮겼습니다. 무작위 값을 쓰는 예제는 씨앗을 고정했어요.
- 엄밀한 정의보다 직관을 우선합니다. 더 정확한 내용이 궁금하면 각 개념의 영어 이름으로 찾아보면 좋은 자료가 많아요.

B.1 벡터: 숫자 묶음은 화살표다

숫자 묶음을 화살표로 보기

서울의 오늘 날씨를 (기온 3도, 습도 4)라는 숫자 두 개로 적었다고 해 봅시다. 이 두 숫자를 가로 3칸, 세로 4칸 간 곳을 가리키는 **화살표**로 그릴 수 있어요. 이렇게 숫자를 순서대로 늘어놓은 묶음을 **벡터**(vector) 라고 합니다. 4장에서 만든 1차원 텐서가 바로 벡터예요.

숫자가 두 개면 종이 위의 화살표, 세 개면 방 안의 화살표입니다. 그 이상은 그림으로 그릴 수 없지

만 계산은 똑같아요. 1장 1.2절에서 붓꽃 한 송이를 "4차원 공간의 점 하나"로 봤고, 13장의 단어 임베딩은 768차원 화살표였습니다. 차원이 몇이든 "길이"와 "방향"이라는 두 성질을 가진다는 점만 기억하면 됩니다.

```
import torch

# 두 도시의 (기온, 습도)를 화살표로 본다
서울 = torch.tensor([3.0, 4.0])
부산 = torch.tensor([6.0, 8.0])

길이 = (서울 ** 2).sum().sqrt()            # 피타고라스: 제곱해서 더하고 제곱근
print("서울 화살표의 길이:", 길이.item(), torch.linalg.norm(서울).item())
print("두 점 사이 거리   :", torch.linalg.norm(부산 - 서울).item())
print("두 배로 늘리기    :", (서울 * 2).tolist())
```

```
서울 화살표의 길이: 5.0 5.0
두 점 사이 거리   : 5.0
두 배로 늘리기    : [6.0, 8.0]
```

화살표의 **길이**는 피타고라스 정리 그대로, 각 숫자를 제곱해서 더한 뒤 제곱근을 씌운 값입니다. 3과 4면 9 + 16 = 25, 제곱근은 5죠. `torch.linalg.norm`이 같은 계산을 해 줍니다. 길이를 **노름(norm)**이라고도 불러요. 두 점 사이의 거리는 "한 점에서 다른 점으로 가는 화살표"의 길이입니다. 1장 1.5절의 최근접 이웃이 이웃을 고를 때 쓴 거리가 이것이에요(유클리드 거리). 8장 8.2절의 weight decay가 작게 만드는 "가중치의 크기"도, 8.9절의 그래디언트 클리핑이 잘라 내는 "기울기의 길이"도 모두 이 노름입니다.

내적: 같은 자리끼리 곱해서 더하기

두 벡터를 하나의 숫자로 비교하는 가장 기본적인 방법이 **내적(dot product)** 입니다. 계산은 단순해요. 같은 자리끼리 곱하고, 모두 더합니다.

```
import torch

a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])
```

```
print("같은 자리끼리 곱하기:", (a * b).tolist())
print("그걸 모두 더하기   :", (a * b).sum().item())   # 이것이 내적
print("@ 로 한 번에       :", (a @ b).item())
print("torch.dot          :", torch.dot(a, b).item())
```

```
같은 자리끼리 곱하기: [4.0, 10.0, 18.0]
그걸 모두 더하기   : 32.0
@ 로 한 번에       : 32.0
torch.dot          : 32.0
```

1 × 4 + 2 × 5 + 3 × 6 = 32입니다. PyTorch에서는 `@` 기호로 한 번에 쓸 수 있어요. 그런데 이 숫자가 무엇을 뜻할까요? 길이는 그대로 두고 방향만 바꿔 가며 내적을 재 보면 보입니다.

```
import torch

기준 = torch.tensor([1.0, 0.0])
비교 = {
    "같은 방향": torch.tensor([2.0, 0.0]),
    "45도"     : torch.tensor([1.0, 1.0]),
    "직각"     : torch.tensor([0.0, 3.0]),
    "반대 방향": torch.tensor([-2.0, 0.0]),
}
for 이름, v in 비교.items():
    print(f"{이름}: 내적 {(기준 @ v).item():.1f}")
```

```
같은 방향: 내적 2.0
45도: 내적 1.0
직각: 내적 0.0
반대 방향: 내적 -2.0
```

방향이 같으면 크고, 직각이면 0, 반대면 음수입니다. **내적은 두 화살표가 같은 쪽을 가리키는 정도**예요. 12장 12.1절의 어텐션이 질문(쿼리)과 키의 내적으로 "얼마나 관련 있는지" 점수를 매기는 이유가 이것입니다.

코사인 유사도: 길이를 빼고 방향만 비교하기

그런데 내적은 길이에도 영향을 받습니다. 방향이 똑같아도 화살표가 10배 길면 내적도 10배가 되죠. 방향만 비교하고 싶다면 내적을 두 길이의 곱으로 나눠 줍니다. 이것이 **코사인 유사도(cosine similarity)** 예요. 결과는 언제나 -1에서 1 사이이고, 1이면 같은 방향, 0이면 직각, -1이면 정반대입니다.

```
import torch
import torch.nn.functional as F

a = torch.tensor([1.0, 1.0])
b = torch.tensor([10.0, 10.0])     # a와 방향이 같고 길이만 10배
c = torch.tensor([1.0, 0.0])       # a와 45도
d = torch.tensor([-1.0, -1.0])     # a와 정반대


def 코사인(x: torch.Tensor, y: torch.Tensor) -> float:
    """내적을 두 길이의 곱으로 나눈다."""
    return ((x @ y) / (x.norm() * y.norm())).item()


for 이름, v in [("b", b), ("c", c), ("d", d)]:
    print(f"a와 {이름}: 내적 {(a @ v).item():6.1f}  코사인 {코사인(a, v):+.4f}")
print("내장 함수(a와 c):", round(F.cosine_similarity(a, c, dim=0).item(), 4))

# 길이를 1로 맞춘(단위 벡터) 뒤 행렬 곱 한 번이면 모든 쌍의 코사인이 나온다
단위 = F.normalize(torch.stack([a, b, c, d]), dim=1)
for 줄 in (단위 @ 단위.T).tolist():
    print([f"{값:+.2f}" for 값 in 줄])
```

```
a와 b: 내적   20.0  코사인 +1.0000
a와 c: 내적    1.0  코사인 +0.7071
a와 d: 내적   -2.0  코사인 -1.0000
내장 함수(a와 c): 0.7071
['+1.00', '+1.00', '+0.71', '-1.00']
['+1.00', '+1.00', '+0.71', '-1.00']
['+0.71', '+0.71', '+1.00', '-0.71']
```

```
['-1.00', '-1.00', '-0.71', '+1.00']
```

a와 b는 길이가 10배 다르지만 코사인 유사도는 1입니다. 방향만 봤으니까요. 마지막 부분이 실전에서 쓰는 방법입니다. 먼저 모든 벡터의 길이를 1로 맞추고(`F.normalize`, 이런 벡터를 **단위 벡터**라고 해요), 행렬 곱 한 번(다음 절)으로 모든 쌍의 내적을 구하면 그것이 곧 모든 쌍의 코사인 유사도입니다. 4장 4.11절에서 기후가 닮은 도시를 찾을 때, 13장 13.7절에서 뜻이 비슷한 단어를 찾을 때 이 방법을 썼어요.

개념	처음 나오는 곳	다시 쓰이는 곳
벡터, 점으로 보는 샘플	1장 1.2절, 1.5절	4장 4.2절, 13장 13.7절
거리(유클리드 거리)	1장 1.5절	3장 3.6절(k-means)
노름(벡터의 길이)	8장 8.2절(가중치 크기)	8장 8.9절(그래디언트 클리핑)
내적	12장 12.1절(계산은 4장 4.11절의 행렬 곱 안에 이미 있음)	12장 12.2절, 13장 13.7절, 21장 21.5절
코사인 유사도	4장 4.11절	10장 10.4절, 12장 12.6절, 13장 13.7절, 21장 21.7절

B.2 행렬 곱과 모양 규칙

모양 규칙 하나만 기억하기

행렬은 숫자를 가로세로 표로 늘어놓은 것, 즉 2차원 텐서입니다. 딥러닝 코드에서 가장 많이 하는 계산이 행렬 곱(`@`)이고, 가장 많이 나는 오류도 행렬 곱의 모양 오류예요. 규칙은 하나뿐입니다. **"앞 행렬의 열 수와 뒤 행렬의 행 수가 같아야 하고, 결과는 (앞의 행 수, 뒤의 열 수)가 된다."** 가운데 숫자가 맞물려 사라진다고 기억하면 쉬워요.

```
import torch

A = torch.ones(2, 3)
B = torch.ones(3, 4)
print("(2, 3) @ (3, 4) ->", tuple((A @ B).shape))
```

```
try:
    A @ torch.ones(2, 4)
except RuntimeError as 오류:
    print("오류:", 오류)

# 맨 앞 차원은 배치로 보고 따로따로 곱한다
배치 = torch.ones(8, 2, 3)
print("(8, 2, 3) @ (3, 4) ->", tuple((배치 @ B).shape))
```

```
(2, 3) @ (3, 4) -> (2, 4)
오류: mat1 and mat2 shapes cannot be multiplied (2x3 and 2x4)
(8, 2, 3) @ (3, 4) -> (8, 2, 4)
```

`(2, 3) @ (3, 4)`는 가운데 3이 맞물려 `(2, 4)`가 됩니다. `(2, 3) @ (2, 4)`는 가운데가 3과 2로 달라서 오류가 나요. 오류 메시지의 `2x3 and 2x4`를 보고 가운데 두 숫자부터 확인하는 습관을 들이세요(4장 4.10절). 마지막 줄처럼 3차원 이상이면 **맨 뒤 두 차원끼리** 곱하고, 앞쪽 차원은 배치로 보고 따로따로 계산합니다. 12장 12.5절의 멀티헤드 어텐션이 `(배치, 헤드, 글자, 칸)` 모양으로 한꺼번에 계산할 수 있는 것도 이 덕분이에요.

행렬 곱은 내적을 한꺼번에 하는 것

그럼 행렬 곱의 결과 한 칸은 무엇일까요? **앞 행렬의 한 줄과 뒤 행렬의 한 열의 내적**입니다.

```
import torch

x = torch.tensor([[1.0, 2.0, 3.0],
                  [0.0, 1.0, 0.0]])      # 샘플 2개, 특성 3개
W = torch.tensor([[1.0, 0.0],
                  [0.0, 1.0],
                  [1.0, 1.0]])           # 특성 3개 -> 출력 2개

결과 = x @ W
print(결과)
# 결과[i, j]는 x의 i번째 줄과 W의 j번째 열의 내적
print("손으로 [0, 1]:", (x[0] @ W[:, 1]).item())
```

```
tensor([[4., 5.],
        [0., 1.]])
손으로 [0, 1]: 5.0
```

결과의 `[0, 1]` 칸 5는 `x`의 0번 줄 `[1, 2, 3]`과 `W`의 1번 열 `[0, 1, 1]`의 내적(0 + 2 + 3)이에요. 그러니 `(샘플 수, 특성 수) @ (특성 수, 출력 수)`는 "샘플마다, 출력마다 내적 하나씩"을 한 번에 계산합니다. 반복문으로 하면 샘플 수 × 출력 수만큼 돌아야 할 일을 한 줄로 하는 거죠.

nn.Linear 속의 행렬 곱

6장 6.1절의 `nn.Linear`는 이 행렬 곱에 편향을 더한 것입니다. 한 가지 헷갈리는 점은 PyTorch가 가중치를 `(출력 수, 입력 수)` 모양으로 저장한다는 거예요. 그래서 계산할 때 전치(`.T`, 가로세로 뒤집기)를 합니다.

```
import torch
from torch import nn

torch.manual_seed(0)
층 = nn.Linear(3, 2)                    # 입력 3칸 -> 출력 2칸
x = torch.randn(5, 3)                   # 샘플 5개

print("가중치 모양:", tuple(층.weight.shape), " 편향 모양:", tuple(층.bias.shape))
직접 = x @ 층.weight.T + 층.bias          # 입력 @ 가중치(전치) + 편향
print("같은가:", torch.allclose(층(x), 직접))
print("파라미터 수:", sum(p.numel() for p in 층.parameters()), "= 3 x 2 + 2")
```

```
가중치 모양: (2, 3)  편향 모양: (2,)
같은가: True
파라미터 수: 8 = 3 x 2 + 2
```

`층(x)`와 `x @ 층.weight.T + 층.bias`가 같습니다. 파라미터 수도 모양에서 바로 나와요. 가중치가 입력 수 × 출력 수, 편향이 출력 수만큼이니 3 × 2 + 2 = 8개입니다. 6장 6.4절에서 모델의 파라미터를 셀 때, 9장에서 합성곱과 완전 연결층의 파라미터 수를 비교할 때, 14장 14.7절에서 LoRA가 큰 행렬 하나를 작은 행렬 둘의 곱으로 바꿔 파라미터를 줄일 때 모두 이 모양 계산을 씁니다.

개념	처음 나오는 곳	다시 쓰이는 곳
행렬 곱과 모양 규칙	4장 4.6절	6장 6.1절, 12장 12.1절, 12.5절, 21장 21.5절
전치	4장 4.5절	6장 6.1절, 12장 12.1절
배치 행렬 곱	4장 4.6절	12장 12.5절
파라미터 수 세기	6장 6.4절	9장 9.3절, 11장 11.6절, 14장 14.7절

B.3 미분과 기울기

기울기: 조금 움직였을 때 얼마나 변하나

산길을 걷는다고 해 봅시다. 한 걸음 앞으로 갔더니 높이가 0.3m 올라갔다면, 그 자리의 경사는 "높이 변화 ÷ 걸어간 거리"로 잴 수 있죠. 5장 5.1절에서 본 **기울기**가 바로 이것입니다. 함수에서 입력을 아주 조금 움직였을 때 출력이 얼마나 변하는지의 비율이에요. 이 비율을 구하는 것을 **미분**이라고 합니다.

```
def f(x: float) -> float:
    return x ** 2

# 기울기 = 높이 변화 / 움직인 거리. 움직이는 거리를 점점 줄여 본다
x = 3.0
for h in [1.0, 0.1, 0.01, 0.001]:
    print(f"h={h:<6} 기울기 ≈ {(f(x + h) - f(x)) / h:.4f}")
print("x ** 2의 기울기 공식 2 * x =", 2 * x)
```

```
h=1.0    기울기 ≈ 7.0000
h=0.1    기울기 ≈ 6.1000
h=0.01   기울기 ≈ 6.0100
h=0.001  기울기 ≈ 6.0010
x ** 2의 기울기 공식 2 * x = 6.0
```

`x ** 2`에서 x = 3일 때, 움직이는 거리 h를 1, 0.1, 0.01로 줄여 갈수록 비율이 6에 다가갑니다. 이

"아주 조금 움직였을 때의 비율"이 x = 3에서의 기울기 6이에요. 미분 공식(`x ** 2`의 기울기는 `2 * x`)은 이 값을 바로 알려 주는 지름길일 뿐입니다. 5장 5.3절에서 손으로 구한 기울기 공식을 이렇게 직접 잰 값과 비교해 검사하기도 했어요(기울기 검사).

기울기의 부호와 크기에는 각각 뜻이 있습니다.

- **부호**: 양수면 오른쪽으로 갈수록 올라가는 오르막, 음수면 내리막입니다.
- **크기**: 클수록 가파릅니다. 바닥(가장 낮은 곳)에서는 0이에요.

변수가 여럿일 때와 경사하강법

모델에는 숫자(파라미터)가 수천, 수백만 개 있습니다. 그럴 때는 **파라미터마다 따로** 기울기를 잽니다. "다른 것은 그대로 두고 이것 하나만 조금 움직이면 손실이 얼마나 변하나"를 파라미터마다 구하는 거예요. 이 기울기들을 모은 것이 **그래디언트(gradient)** 이고, 손실이 가장 가파르게 올라가는 방향을 가리킵니다.

```
import torch

# 변수가 둘인 함수: 손실 = (w * 2 + b - 7) ** 2
w = torch.tensor(1.0, requires_grad=True)
b = torch.tensor(0.5, requires_grad=True)
손실 = (w * 2 + b - 7) ** 2
손실.backward()
print("손실:", 손실.item())
print("w 방향 기울기:", w.grad.item(), " b 방향 기울기:", b.grad.item())

# 경사하강법 한 걸음: 기울기의 반대쪽으로 조금
학습률 = 0.05
with torch.no_grad():
    w -= 학습률 * w.grad
    b -= 학습률 * b.grad
print("한 걸음 뒤 손실:", ((w * 2 + b - 7) ** 2).item())
```

```
손실: 20.25
w 방향 기울기: -18.0  b 방향 기울기: -9.0
한 걸음 뒤 손실: 5.0625
```

`w`와 `b` 둘 다 기울기가 음수이니, 둘 다 키우면 손실이 줄어든다는 뜻입니다. 그래서 **기울기의 반대 방**

향으로, 학습률만큼 조금 움직였더니 손실이 20.25에서 5.0625로 줄었어요. 이 한 걸음을 반복하는 것이 5장의 **경사하강법**이고, 6장의 `optimizer.step()`이 하는 일의 기본형입니다. 학습률이 너무 크면 골짜기 건너편으로 튀어 발산하고, 너무 작으면 한참 걸려요(5장 5.10절).

연쇄 법칙: 기울기는 곱해서 전달된다

신경망은 계산을 여러 단계로 이어 붙인 것입니다. 그럼 처음 입력이 마지막 손실에 주는 영향은 어떻게 구할까요? 톱니바퀴를 떠올려 보세요. 첫 바퀴가 1바퀴 돌 때 둘째가 3바퀴 돌고, 둘째가 1바퀴 돌 때 셋째가 2바퀴 돈다면, 첫 바퀴 1바퀴에 셋째는 3 × 2 = 6바퀴 돕니다. **단계마다의 기울기를 곱하면** 전체 기울기가 됩니다. 이것이 5장 5.6절의 **연쇄 법칙(chain rule)** 이에요.

```
import torch

# y = (3 * x + 1) ** 2 를 두 단계로: u = 3 * x + 1, y = u ** 2
x = torch.tensor(2.0, requires_grad=True)
u = 3 * x + 1
y = u ** 2
y.backward()

du_dx = 3                # u는 x가 1 움직이면 3 움직인다
dy_du = 2 * u.item()     # y는 u가 1 움직이면 2 * u 움직인다
print("단계별 기울기의 곱:", dy_du * du_dx)
print("PyTorch가 구한 값 :", x.grad.item())
```

```
단계별 기울기의 곱: 42.0
PyTorch가 구한 값 : 42.0
```

x = 2에서 u = 7이고, u는 x의 3배로 움직이고(기울기 3), y는 u의 2 × 7 = 14배로 움직입니다(기울기 14). 둘을 곱한 42가 PyTorch가 `backward()`로 구한 값과 같아요. 역전파는 이 곱셈을 마지막 단계부터 거꾸로 차례차례 해 나가는 것입니다.

곱해서 전달된다는 성질에는 함정이 있습니다. 단계가 많아지면 1보다 작은 수가 수십 번 곱해져서 기울기가 사라지거나, 1보다 큰 수가 곱해져 폭주해요. 11장 11.4절에서 0.9를 100번 곱하면 0.0000266, 1.1을 100번 곱하면 1만 3천이 되는 것을 봤죠. 8장의 가중치 초기화와 배치 정규화, 9장 9.8절의 잔차 연결, 11장의 LSTM은 모두 이 곱셈이 1 근처에 머물도록 만드는 장치입니다.

개념	처음 나오는 곳	다시 쓰이는 곳
기울기(미분)	5장 5.1절	5장 5.3절(기울기 검사)
그래디언트, 경사하강법	5장 5.2절, 5.3절	6장 6.6절, 8장 8.7절
학습률	5장 5.2절, 5.10절	6장 6.6절, 8장 8.7절, 18장 18.5절
연쇄 법칙, 역전파	5장 5.6절	8장 8.6절, 11장 11.4절
기울기 소실과 폭주	8장 8.6절	9장 9.8절, 11장 11.4절

B.4 지수와 로그

exp와 log는 서로 되돌리는 짝

지수 함수 `exp(x)`는 약 2.718(이 수를 e라고 불러요)을 x번 곱한 값입니다. **로그** `log(x)`는 그 반대로, "e를 몇 번 곱해야 x가 되나"를 답해요. 그래서 둘은 서로를 되돌립니다. 이 책에서 그냥 `log`라고 하면 e를 밑으로 하는 자연로그이고, PyTorch의 `torch.log`, 파이썬의 `math.log`가 모두 자연로그예요.

```
import math

print("exp(0) =", math.exp(0), " exp(1) =", round(math.exp(1), 4))
print("exp(2 + 3) == exp(2) * exp(3):", math.isclose(math.exp(5), math.exp(2) * math.exp(3)))
print("log(2 * 3) == log(2) + log(3):", math.isclose(math.log(6), math.log(2) + math.log(3)))
print("log(exp(4)) =", math.log(math.exp(4)))
for p in [1.0, 0.5, 0.1, 0.01]:
    print(f"log({p}) = {math.log(p):.3f}")
```

```
exp(0) = 1.0  exp(1) = 2.7183
exp(2 + 3) == exp(2) * exp(3): True
log(2 * 3) == log(2) + log(3): True
log(exp(4)) = 4.0
log(1.0) = 0.000
log(0.5) = -0.693
log(0.1) = -2.303
```

```
log(0.01) = -4.605
```

딥러닝에서 쓰는 성질은 세 가지면 충분합니다.

- **exp는 언제나 양수입니다.** 입력이 -1000이어도 0보다 커요. 그래서 아무 점수나 양수로 바꿀 때 씁니다(B.5절의 소프트맥스, 15장 15.6절에서 로그분산을 표준편차로 바꿀 때).
- **log는 곱셈을 덧셈으로 바꿉니다.** `log(2 * 3)`은 `log(2) + log(3)`이에요. 작은 확률 여러 개를 곱하는 계산을 로그의 덧셈으로 바꾸면 훨씬 안전해집니다(아래).
- **1보다 작은 수의 log는 음수이고, 0에 가까울수록 급격히 작아집니다.** `log(0.5)`는 -0.693, `log(0.01)`은 -4.605예요. 이 숫자들은 6장 6.5절의 교차 엔트로피 표에 그대로 나옵니다(B.6절).

로그 눈금: 곱하기로 늘어나는 값 고르기

학습률을 0.0001에서 0.1 사이에서 고른다고 해 봅시다. 후보를 고르게 7개 뽑으면 어떻게 될까요?

```
import numpy as np

# 0.0001에서 0.1 사이에서 후보 7개 고르기
print("고르게  :", np.round(np.linspace(1e-4, 1e-1, 7), 4))
print("로그 눈금:", np.round(np.logspace(-4, -1, 7), 5))
print("log10   :", np.round(np.log10(np.logspace(-4, -1, 7)), 2))
```

```
고르게  : [0.0001 0.0168 0.0334 0.0501 0.0667 0.0834 0.1   ]
로그 눈금: [0.0001  0.00032 0.001   0.00316 0.01    0.03162 0.1    ]
log10   : [-4.  -3.5 -3.  -2.5 -2.  -1.5 -1. ]
```

고르게 뽑으면 0.0001 다음이 바로 0.0168입니다. 0.001이나 0.003 같은 작은 값은 하나도 시험하지 못해요. 학습률은 "0.001이냐 0.002냐"보다 "0.001이냐 0.01이냐"가 중요한 값이라, **같은 비율로 늘어나게** 고르는 편이 맞습니다. 이것이 **로그 눈금**이에요. 마지막 줄처럼 로그 눈금으로 고른 값에 `log10`(10을 밑으로 하는 로그)을 씌우면 -4, -3.5, -3, ...으로 고르게 늘어섭니다. 8장 8.11절의 학습률 범위 테스트 그래프가 가로축을 로그 눈금으로 그렸고, 18장 18.5절의 Optuna가 `suggest_float(..., log=True)`로 학습률을 고른 것도 같은 이유예요.

작은 확률은 로그로 더한다

언어 모델은 문장의 확률을 토큰마다의 확률을 곱해서 구합니다. 그런데 0.01을 200번 곱하면 어떻게 될까요?

```
import math

import torch

# 확률 0.01을 200번 곱하면? (토큰 200개짜리 문장의 확률을 생각해 보자)
p = torch.full((200,), 0.01)
print("그냥 곱하기 (float32):", torch.prod(p).item())
print("로그를 더하기        :", torch.log(p).sum().item())
print("손으로 200 * log(0.01):", 200 * math.log(0.01))
```

```
그냥 곱하기 (float32): 0.0
로그를 더하기        : -921.0341796875
손으로 200 * log(0.01): -921.0340371976182
```

진짜 답은 10의 -400제곱인데, float32가 나타낼 수 있는 가장 작은 수보다 훨씬 작아서 0이 돼 버렸습니다. 이런 현상을 **언더플로(underflow)** 라고 해요. 로그를 씌워 더하면 -921.03이라는 멀쩡한 숫자가 나옵니다. 곱셈이 덧셈으로 바뀌었기 때문이죠. 그래서 확률을 다루는 손실 함수는 거의 언제나 **로그 확률**로 계산합니다. 6장의 `nn.CrossEntropyLoss`가 안에서 소프트맥스 대신 로그 소프트맥스를 쓰는 이유(8장 8.11절의 "소프트맥스 두 번" 버그)도 여기에 있어요.

개념	처음 나오는 곳	다시 쓰이는 곳
지수 함수 exp	3장 3.3절(시그모이드)	6장 6.2절, 15장 15.6절
로그, -log	4장 4.4절(정답 확률의 -log)	6장 6.5절, 8장 8.11절
로그 눈금	8장 8.11절	18장 18.5절
거듭 곱하기(지수적 증가·감소)	8장 8.6절	11장 11.4절

B.5 확률과 소프트맥스

점수를 확률로: 소프트맥스

분류 모델의 마지막 층은 클래스마다 점수(**로짓**)를 하나씩 냅니다. 이 점수는 음수도 되고 합도 제멋대로라 확률로 읽을 수가 없어요. 확률이 되려면 두 조건을 지켜야 합니다. **모두 0 이상이고, 다 더하면 1.** 이 두 조건을 만드는 함수가 **소프트맥스(softmax)** 입니다. 하는 일은 두 단계예요. exp를 씌워 모두 양수로 만들고, 그 합으로 나눠 합을 1로 맞춥니다.

```
import torch

점수 = torch.tensor([2.0, 1.0, 0.1])          # 로짓
지수 = torch.exp(점수)
확률 = 지수 / 지수.sum()
print("exp 씌우기:", 지수.round(decimals=4))
print("나눠서 합 1:", 확률.round(decimals=4), 확률.sum().item())
print("내장 함수  :", torch.softmax(점수, dim=0).round(decimals=4))
print("모두 +100 :", torch.softmax(점수 + 100, dim=0).round(decimals=4))
```

```
exp 씌우기: tensor([7.3891, 2.7183, 1.1052])
나눠서 합 1: tensor([0.6590, 0.2424, 0.0986]) 0.9999999403953552
내장 함수  : tensor([0.6590, 0.2424, 0.0986])
모두 +100 : tensor([0.6590, 0.2424, 0.0986])
```

점수 2.0, 1.0, 0.1이 확률 0.659, 0.242, 0.099가 됐습니다. 이 확률 0.6590은 6장 6.5절에서 교차 엔트로피를 손으로 풀 때 나온 숫자와 같아요. 합계가 정확히 1이 아니라 0.99999994로 찍힌 것은 float32의 반올림 오차입니다(4장 4.3절).

마지막 줄을 보세요. 점수에 모두 100을 더해도 확률이 그대로입니다. 소프트맥스는 **점수의 차이만** 보거든요. 이 성질이 다음의 안전장치에 쓰입니다.

큰 점수와 안전한 계산

exp는 입력이 조금만 커져도 엄청나게 커집니다. float32에서는 exp(89)쯤이면 이미 나타낼 수 있는 가장 큰 수를 넘어서 `inf`(무한대)가 돼요.

```
import torch

점수 = torch.tensor([1000.0, 999.0, 0.0])
```

```
지수 = torch.exp(점수)
print("그대로 exp:", 지수, "->", 지수 / 지수.sum())

# 가장 큰 값을 빼고 계산해도 결과는 같다 (모두에 같은 수를 빼도 소프트맥스는 그대로)
빼고 = torch.exp(점수 - 점수.max())
print("최댓값 빼기:", (빼고 / 빼고.sum()).round(decimals=4))
print("내장 함수  :", torch.softmax(점수, dim=0).round(decimals=4))
```

```
그대로 exp: tensor([inf, inf, 1.]) -> tensor([nan, nan, 0.])
최댓값 빼기: tensor([0.7311, 0.2689, 0.0000])
내장 함수  : tensor([0.7311, 0.2689, 0.0000])
```

그대로 계산하면 `inf / inf`가 되어 `nan`(숫자가 아님)이 나옵니다. 그런데 점수에서 가장 큰 값을 빼고 계산하면 exp에 들어가는 가장 큰 수가 0이 되니 넘칠 일이 없어요. 앞에서 본 대로 모두에 같은 수를 빼도 결과는 같고요. `torch.softmax`는 안에서 이렇게 계산하기 때문에 6장 6.2절에서 본 것처럼 `[1000, 0]` 같은 점수도 문제없습니다. 직접 소프트맥스를 짤 일이 있다면(20장 20.11절에서 NumPy로 짰듯이) 최댓값 빼기를 잊지 마세요.

온도: 확률을 날카롭게, 또는 평평하게

14장 14.3절에서 언어 모델이 다음 토큰을 뽑을 때 쓴 **온도**는 소프트맥스에 넣기 전에 점수를 나누는 숫자입니다.

```
import torch

로짓 = torch.tensor([2.0, 1.0, 0.1])
for 온도 in [0.5, 1.0, 2.0]:
    print(f"온도 {온도}:", [f"{p:.3f}" for p in torch.softmax(로짓 / 온도, dim=0).tolist()])
```

```
온도 0.5: ['0.864', '0.117', '0.019']
온도 1.0: ['0.659', '0.242', '0.099']
온도 2.0: ['0.502', '0.304', '0.194']
```

온도가 1보다 작으면 점수 차이가 커져서 확률이 1등에게 쏠리고(0.864), 1보다 크면 차이가 줄어 확률이 고르게 퍼집니다(0.502, 0.304, 0.194). 12장 12.2절의 "점수를 d의 제곱근으로 나누기"도 같

은 원리예요. 점수가 너무 커져서 소프트맥스가 한 칸으로 쏠리는 것을 막으려고 나눕니다.

시그모이드: 두 칸짜리 소프트맥스

3장 3.3절의 로지스틱 회귀와 6장의 이진 분류에서 쓴 **시그모이드(sigmoid)** 는 점수 하나를 0과 1 사이의 확률로 바꿉니다. 식은 `1 / (1 + exp(-z))`예요. 사실 시그모이드는 소프트맥스의 특별한 경우입니다.

```
import torch

z = torch.tensor([-4.0, -1.0, 0.0, 1.0, 4.0])
print("시그모이드       :", [f"{p:.4f}" for p in torch.sigmoid(z).tolist()])
print("1 / (1 + exp(-z)):", [f"{p:.4f}" for p in (1 / (1 + torch.exp(-z))).tolist()])

# 두 칸 소프트맥스 [z, 0]의 첫 칸 = 시그모이드(z)
두칸 = torch.stack([z, torch.zeros_like(z)], dim=1)
print("두 칸 소프트맥스 :", [f"{p:.4f}" for p in torch.softmax(두칸, dim=1)[:, 0].tolist()])
```

```
시그모이드       : ['0.0180', '0.2689', '0.5000', '0.7311', '0.9820']
1 / (1 + exp(-z)): ['0.0180', '0.2689', '0.5000', '0.7311', '0.9820']
두 칸 소프트맥스 : ['0.0180', '0.2689', '0.5000', '0.7311', '0.9820']
```

세 줄이 똑같죠? 점수 `[z, 0]`에 소프트맥스를 씌운 첫 칸이 시그모이드(z)입니다. "양성일 점수 z, 음성일 점수 0"인 두 칸짜리 분류라는 뜻이에요. 그래서 이진 분류에는 출력 하나와 시그모이드(`BCEWithLogitsLoss`)를, 클래스가 셋 이상이면 출력 여러 개와 소프트맥스(`CrossEntropyLoss`)를 씁니다. 점수가 0이면 확률 0.5, 4면 0.982, -4면 0.018이에요.

개념	처음 나오는 곳	다시 쓰이는 곳
확률(0 이상, 합 1)	3장 3.3절	6장 6.5절, 14장 14.2절
로짓	6장 6.2절	6장 6.5절, 14장 14.2절
소프트맥스	6장 6.2절	12장 12.1절, 14장 14.3절, 20장 20.11절
시그모이드	3장 3.3절	6장 6.2절, 11장 11.5절(LSTM의 문)
온도	14장 14.3절	12장 12.2절(같은 원리), 19장 19.11절(지식 증류)

B.6 교차 엔트로피: 정답 확률에 매기는 벌점

분류 모델이 얼마나 틀렸는지는 어떻게 숫자로 잴까요? 아이디어는 간단합니다. **정답 칸에 준 확률이 높을수록 잘한 것**이에요. 그 확률에 `-log`를 씌우면 벌점이 됩니다. 확률이 1이면 벌점 0, 확률이 0에 가까울수록 벌점이 끝없이 커지죠(B.4절의 로그 성질). 이것을 샘플마다 구해 평균 낸 것이 **교차 엔트로피**(cross entropy) 손실입니다.

```
import math

import torch
import torch.nn.functional as F

로짓 = torch.tensor([[2.0, 1.0, 0.1],
                     [0.5, 2.5, 0.3]])     # 샘플 2개, 클래스 3개
정답 = torch.tensor([0, 2])                # 두 번째 샘플은 확률이 낮은 칸이 정답

확률 = torch.softmax(로짓, dim=1)
정답_확률 = 확률[torch.arange(2), 정답]
print("정답 칸의 확률:", [f"{p:.4f}" for p in 정답_확률.tolist()])
print("-log 평균     :", round((-torch.log(정답_확률)).mean().item(), 4))
print("F.cross_entropy:", round(F.cross_entropy(로짓, 정답).item(), 4))
print("클래스 10개를 찍을 때 손실 -log(1/10):", round(-math.log(1 / 10), 4))
```

```
정답 칸의 확률: ['0.6590', '0.0889']
-log 평균     : 1.4185
F.cross_entropy: 1.4185
클래스 10개를 찍을 때 손실 -log(1/10): 2.3026
```

첫 샘플은 정답 칸에 0.659를 줘서 벌점이 작고, 둘째 샘플은 정답 칸에 0.089밖에 주지 않아서 벌점이 큽니다. 두 벌점의 평균 1.4185가 `F.cross_entropy`의 결과와 같아요. `F.cross_entropy`에는 확률이 아니라 **로짓을 그대로** 넣는다는 점을 기억하세요. 소프트맥스는 안에서 해 줍니다.

마지막 줄은 학습을 시작할 때 쓸모 있는 숫자입니다. 아무것도 배우지 않은 모델은 클래스 10개에 고르게 0.1씩 주니, 손실이 `-log(0.1)`, 약 2.303에서 시작해요. 6장 6.5절 Q&A와 8장 8.11절에서 첫 손실이 이 근처인지 확인하라고 한 이유입니다. 클래스가 k개면 `log(k)` 근처에서 시작해야 정상이에요.

이름에 “엔트로피”가 붙은 것은 정보 이론에서 온 말이라서인데, 이 책을 따라가는 데는 “정답 확률의 -log 평균”이라는 뜻이면 충분합니다. 15장 15.6절의 VAE에 나오는 **KL 발산**도 같은 집안의 개념으로, 두 확률 분포가 얼마나 다른지를 재는 값이에요. VAE에서는 잠재 벡터의 분포가 표준 정규분포(평균 0, 표준편차 1)에서 너무 멀어지지 않게 하는 벌점으로 쓰였습니다.

개념	처음 나오는 곳	다시 쓰이는 곳
교차 엔트로피	4장 4.4절(손으로 계산), 6장 6.5절	8장 8.11절, 9장 9.5절, 11장 11.3절, 12장 12.8절, 14장
첫 손실 `log(클래스 수)`	6장 6.5절	8장 8.11절
이진 교차 엔트로피	6장 6.2절, 6.5절	15장 15.6절
KL 발산	15장 15.6절	19장 19.11절(지식 증류)

B.7 평균·분산·표준편차와 표준화

흔들림을 숫자 하나로: 분산과 표준편차

두 반의 시험 평균이 똑같이 70점이라고 같은 반일까요? 한 반은 모두 70점 근처이고, 다른 반은 50점부터 90점까지 흩어져 있을 수 있습니다. 평균만으로는 이 차이가 보이지 않아요. 흩어진 정도를 재는 숫자가 **분산(variance)** 과 **표준편차(standard deviation)** 입니다.

```
import torch

점수_a = torch.tensor([70.0, 71.0, 69.0, 70.0, 70.0])
점수_b = torch.tensor([50.0, 90.0, 60.0, 80.0, 70.0])
for 이름, x in [("A반", 점수_a), ("B반", 점수_b)]:
    평균 = x.mean()
    분산 = ((x - 평균) ** 2).mean()          # 평균에서 떨어진 거리의 제곱을 평균
    print(f"{이름} 평균 {평균.item():.1f}  분산 {분산.item():6.1f}  표준편차 {분
    ↪  산.sqrt().item():5.2f}")

print("torch.std 기본값  :", round(점수_b.std().item(), 2))           # n - 1로 나눔
print("correction=0     :", round(점수_b.std(correction=0).item(), 2))    # n으로 나눔
```

```
A반 평균 70.0  분산    0.4  표준편차  0.63
B반 평균 70.0  분산  200.0  표준편차 14.14
torch.std 기본값  : 15.81
correction=0     : 14.14
```

분산은 "각 값이 평균에서 떨어진 거리"를 제곱해서 평균 낸 것입니다. 제곱하는 이유는 위로 떨어진 것과 아래로 떨어진 것이 서로 지워지지 않게 하려는 거예요. 그런데 제곱을 했으니 단위가 "점의 제곱"이 되어 버리죠. 그래서 제곱근을 씌워 원래 단위로 돌린 것이 표준편차입니다. B반은 평균에서 대략 14점쯤 흩어져 있다고 읽으면 돼요.

마지막 두 줄은 자주 헷갈리는 부분입니다. `torch.std`는 기본으로 개수(n) 대신 **n - 1로 나눠서** 15.81이 나왔어요. 갖고 있는 데이터가 전체가 아니라 일부(표본)라고 보고 조금 크게 추정하는 방식입니다. `correction=0`을 주면(n에서 1 대신 0을 빼라는 뜻) n으로 나눠서 직접 계산한 14.14와 같아집니다. NumPy의 `np.std`는 반대로 기본이 n이라 PyTorch와 결과가 다를 수 있어요. 데이터가 많으면 둘의 차이는 무시할 만큼 작아집니다.

표준화: 단위가 다른 특성을 같은 눈금으로

키(cm)와 몸무게(kg)를 함께 쓰는 모델을 생각해 봅시다. 키는 160~180, 몸무게는 55~80이라 숫자의 범위가 다릅니다. 1장 1.8절에서 봤듯 거리를 재는 모델은 범위가 큰 특성에 휘둘리고, 경사하강법도 눈금이 다른 특성이 섞이면 길쭉한 골짜기에서 헤매요(2장 2.6절). 그래서 **각 특성에서 평균을 빼고 표준편차로 나눕니다.** 이것이 **표준화(standardization)** 예요.

```
import torch

키_cm = torch.tensor([160.0, 170.0, 180.0, 175.0, 165.0])
몸무게_kg = torch.tensor([55.0, 65.0, 80.0, 72.0, 60.0])
표 = torch.stack([키_cm, 몸무게_kg], dim=1)        # (사람 5, 특성 2)

평균 = 표.mean(dim=0)
표준편차 = 표.std(dim=0)
표준화 = (표 - 평균) / 표준편차                    # 브로드캐스팅 (4장 4.6절)
print(표준화.round(decimals=3))
print("열 평균:", 표준화.mean(dim=0).round(decimals=3).tolist(),
      " 열 표준편차:", 표준화.std(dim=0).round(decimals=3).tolist())
```

```
tensor([[-1.2650, -1.1560],
        [ 0.0000, -0.1420],
        [ 1.2650,  1.3790],
        [ 0.6320,  0.5680],
        [-0.6320, -0.6490]])
열 평균: [0.0, -0.0]  열 표준편차: [1.0, 1.0]
```

표준화한 뒤에는 모든 열이 평균 0, 표준편차 1이 됩니다. 이제 값은 "평균보다 표준편차 몇 개만큼 크거나 작은가"를 뜻해요. 키 180cm는 1.265, 즉 평균보다 표준편차 1.3개쯤 큽니다. 열 평균의 `-0.0`은 아주 작은 음수가 반올림된 것이라 0과 같아요.

표준화는 이 책 여러 곳에 다른 이름으로 나옵니다.

- 2장 2.6절의 `StandardScaler`는 이 계산을 하는 scikit-learn 도구이고, 평균과 표준편차는 **학습 데이터로만** 구해야 합니다(2장 2.3절의 데이터 누수).
- 7장과 10장 10.2절의 `transforms.Normalize(평균, 표준편차)`는 사진의 채널마다 같은 계산을 합니다. 사전학습 모델을 쓸 때는 그 모델이 학습한 ImageNet의 평균과 표준편차를 써야 해요.
- 8장 8.4절의 **배치 정규화**와 12장 12.7절의 **층 정규화**는 신경망 중간에서 이 표준화를 하는 층입니다. 배치 정규화는 배치 안의 샘플끼리, 층 정규화는 샘플 하나 안의 숫자끼리 평균과 표준편차를 구한다는 점이 달라요.

더하면 커지고, 평균 내면 작아진다

마지막으로 본문 여러 곳에 숨어 있는 규칙 하나를 확인해 보겠습니다. 서로 상관없이 흔들리는 숫자를 여러 개 더하거나 평균 내면 흔들림이 어떻게 될까요?

```
import torch

torch.manual_seed(0)
# 표준편차 1인 무작위 수 n개를 "더하기"와 "평균 내기"를 10000번씩 해 보고 흔들림을 잰다
for n in [4, 64, 512]:
    묶음 = torch.randn(10000, n)
    합_흔들림 = 묶음.sum(dim=1).std().item()
    평균_흔들림 = 묶음.mean(dim=1).std().item()
    print(f"n={n:3}  합의 표준편차 {합_흔들림:6.2f} (n의 제곱근 {n ** 0.5:5.2f})"
```

```
        f"  평균의 표준편차 {평균_흔들림:.4f} (1 / n의 제곱근 {1 / n ** 0.5:.4f})")
```

```
n=  4  합의 표준편차   2.02 (n의 제곱근  2.00)  평균의 표준편차 0.5050 (1 / n의 제곱근 0.5000)
n= 64  합의 표준편차   7.96 (n의 제곱근  8.00)  평균의 표준편차 0.1244 (1 / n의 제곱근 0.1250)
n=512  합의 표준편차  22.37 (n의 제곱근 22.63)  평균의 표준편차 0.0437 (1 / n의 제곱근 0.0442)
```

표준편차 1인 수를 n개 **더하면** 흔들림이 n의 제곱근배로 커지고, n개를 **평균 내면** n의 제곱근분의 1로 작아집니다. n배가 아니라 n의 제곱근배라는 점이 핵심이에요. 이 규칙이 세 장에 나옵니다.

- **12장 12.2절**: 내적은 숫자 d개를 곱해서 **더한** 값이라, d가 커지면 점수의 표준편차가 d의 제곱근배로 커집니다. 그래서 점수를 d의 제곱근으로 나눠 흔들림을 1로 되돌렸어요.
- **8장 8.6절**: 층 하나는 입력 256개에 가중치를 곱해 더하니, 가중치의 표준편차가 1이면 출력이 크게 불어납니다(ReLU가 절반을 0으로 만들어 실제로는 층마다 10배쯤). Kaiming 초기화는 가중치의 표준편차를 입력 수의 제곱근에 맞춰 줄여서 흔들림을 유지해요.
- **18장 18.6절**: 씨앗을 바꿔 n번 돌린 결과의 **평균**은 한 번 돌린 결과보다 덜 흔들립니다. 그 흔들림, 즉 "표준편차 ÷ n의 제곱근"이 **표준오차**예요. 씨앗을 4번에서 16번으로 늘리면 표준오차가 절반이 됩니다.

오차를 재는 법: MAE, MSE, RMSE

평균과 제곱근은 회귀 모델의 오차를 잴 때도 씁니다. 3장 3.8절에서 본 세 지표를 작은 예로 다시 보면 이렇습니다.

```
import torch

정답 = torch.tensor([3.0, 5.0, 2.0, 7.0])
예측 = torch.tensor([2.5, 5.0, 4.0, 8.0])
오차 = 예측 - 정답
print("오차      :", 오차.tolist())
print("MAE  (절댓값 평균)      :", 오차.abs().mean().item())
print("MSE  (제곱 평균)        :", (오차 ** 2).mean().item())
print("RMSE (MSE의 제곱근)     :", round((오차 ** 2).mean().sqrt().item(), 4))
```

```
오차      : [-0.5, 0.0, 2.0, 1.0]
MAE (절댓값 평균)       : 0.875
MSE (제곱 평균)         : 1.3125
RMSE (MSE의 제곱근)      : 1.1456
```

MAE는 오차의 절댓값 평균(0.875), **MSE**는 오차 제곱의 평균(1.3125), **RMSE**는 MSE에 제곱근을 씌워 원래 단위로 돌린 것(1.1456)입니다. 분산과 표준편차의 관계와 똑같죠. 제곱을 하기 때문에 MSE와 RMSE는 **큰 오차 하나에 크게 반응**합니다. 위에서 오차 2.0인 샘플 하나가 MSE의 대부분을 차지해요. 6장 6.5절의 `nn.MSELoss`가 이 MSE이고, 5장 5.13절과 11장 11.10절에서는 결과를 읽기 쉽게 RMSE로 보고했습니다.

개념	처음 나오는 곳	다시 쓰이는 곳
평균, 표준편차	1장 1.8절	4장 4.11절, 18장 18.6절
표준화	1장 1.8절	2장 2.6절, 4장 4.11절, 5장 5.3절
채널별 정규화(Normalize)	7장 7.6절	10장 10.2절
배치 정규화, 층 정규화	8장 8.4절, 12장 12.7절	9장 9.7절, 10장 10.6절
합과 평균의 흔들림(제곱근 규칙)	8장 8.6절, 12장 12.2절	18장 18.6절(표준오차)
MAE, MSE, RMSE	3장 3.8절	5장 5.13절, 6장 6.5절, 11장 11.10절

B.8 확률 분포에서 뽑기: 정규분포와 샘플링

정규분포: 가운데가 두툼한 종 모양

4장 4.1절부터 거의 모든 장에서 `torch.randn`으로 무작위 숫자를 만들었습니다. 이 숫자들은 아무렇게나 나오는 게 아니라 **정규분포(normal distribution)** 를 따릅니다. 키나 시험 점수처럼 평균 근처에 많이 몰리고 양 끝으로 갈수록 드물어지는, 종 모양의 분포예요. `randn`은 그중에서도 평균 0, 표준편차 1인 **표준 정규분포**에서 뽑습니다.

```
import torch

torch.manual_seed(0)
z = torch.randn(100000)                    # 평균 0, 표준편차 1인 정규분포에서 뽑기
print(f"평균 {z.mean().item():+.3f}  표준편차 {z.std().item():.3f}")
for k in [1, 2, 3]:
    print(f"평균에서 표준편차 {k}개 안쪽: {(z.abs() < k).float().mean().item():.1%}")

# 평균 5, 표준편차 2인 정규분포가 필요하면: 평균 + 표준편차 × z
x = 5 + 2 * z
print(f"옮기고 늘린 뒤: 평균 {x.mean().item():.3f}  표준편차 {x.std().item():.3f}")
```

```
평균 -0.002  표준편차 1.001
평균에서 표준편차 1개 안쪽: 68.1%
평균에서 표준편차 2개 안쪽: 95.5%
평균에서 표준편차 3개 안쪽: 99.7%
옮기고 늘린 뒤: 평균 4.995  표준편차 2.002
```

10만 개를 뽑았더니 평균이 거의 0, 표준편차가 거의 1입니다. 그리고 평균에서 표준편차 1개 안쪽에 약 68%, 2개 안쪽에 약 95%, 3개 안쪽에 99.7%가 들어 있어요. 이 비율은 정규분포라면 어디서나 같아서, "표준편차 3개 밖은 거의 없다"는 감각으로 쓰기 좋습니다.

마지막 줄이 중요합니다. 평균 5, 표준편차 2인 정규분포가 필요하면 새로 만들 필요 없이 **표준 정규분포에서 뽑은 값에 2를 곱하고 5를 더하면** 됩니다. B.7절의 표준화를 거꾸로 한 계산이죠. 15장 15.6절의 VAE가 쓴 **재매개변수화**가 바로 이것입니다. "평균 + 표준편차 × 무작위 값"으로 쓰면 무작위 값은 따로 뽑아 두고 평균과 표준편차는 평범한 계산이 되니, 뽑기 과정이 끼어 있어도 기울기가 흐를 수 있어요. 15.8절의 확산 모델이 사진에 섞는 잡음도 `randn`으로 뽑은 정규분포 잡음입니다.

확률대로 뽑기

언어 모델이 다음 토큰을 고를 때는 두 가지 방법이 있습니다. 확률이 가장 높은 것을 고르거나(`argmax`, 탐욕), 확률대로 뽑거나(샘플링). 확률대로 뽑는다는 게 어떤 뜻인지 B.5절의 확률로 확인해 볼까요?

```
import torch

torch.manual_seed(0)
확률 = torch.tensor([0.659, 0.242, 0.099])          # B.5절 소프트맥스 결과
뽑기 = torch.multinomial(확률, num_samples=10000, replacement=True)
횟수 = torch.bincount(뽑기, minlength=3)
print("뽑힌 횟수:", 횟수.tolist())
print("뽑힌 비율:", [f"{c / 10000:.3f}" for c in 횟수.tolist()])
print("argmax   :", 확률.argmax().item(), "(언제나 같은 답)")
```

```
뽑힌 횟수: [6545, 2423, 1032]
뽑힌 비율: ['0.654', '0.242', '0.103']
argmax   : 0 (언제나 같은 답)
```

`torch.multinomial`로 1만 번 뽑았더니 0번이 65.4%, 1번이 24.2%, 2번이 10.3%로, 주어진 확률 0.659, 0.242, 0.099와 거의 같은 비율로 나왔습니다. 확률이 낮은 2번도 열 번에 한 번쯤은 뽑혀요. 그래서 샘플링은 실행할 때마다 다른 답을 내고, `argmax`는 언제나 같은 답을 냅니다. 11장 11.11절의 속담 짓기, 12장 12.9절의 글 만들기, 14장 14.3절의 디코딩이 모두 이 `multinomial`을 썼고, 온도(B.5절)로 확률을 날카롭게 하거나 평평하게 해서 뽑히는 정도를 조절했습니다.

개념	처음 나오는 곳	다시 쓰이는 곳
정규분포, `randn`	4장 4.1절	8장 8.6절(초기화), 15장 15.6절, 15.8절
재매개변수화(평균 + 표준편차 × 무작위 값)	15장 15.6절	—
확률대로 뽑기(샘플링)	11장 11.11절	12장 12.9절, 14장 14.3절

B.9 한눈에 보기

이 부록에서 다룬 개념을 한 표로 모았습니다. 본문에서 막혔을 때 어느 절을 펼치면 되는지 찾는 용도로 쓰세요.

본문에서 만나는 말	한 줄 직관	부록
노름, 거리	화살표의 길이, 두 점 사이 화살표의 길이	B.1
내적	같은 자리끼리 곱해 더하기. 같은 쪽을 가리키는 정도	B.1
코사인 유사도	길이를 빼고 방향만 비교. -1~1	B.1
행렬 곱 `@`	내적을 한꺼번에. 가운데 숫자가 맞물려 사라진다	B.2
기울기, 그래디언트	조금 움직였을 때 손실이 변하는 비율. 오르막 방향	B.3
경사하강법	기울기 반대쪽으로 학습률만큼 한 걸음	B.3
연쇄 법칙, 역전파	단계마다의 기울기를 곱해 거꾸로 전달	B.3
exp, log	서로 되돌리는 짝. log는 곱을 덧셈으로	B.4
로그 눈금	같은 비율로 늘어나게 고르기	B.4
소프트맥스	exp를 씌우고 합으로 나눠 확률로	B.5
온도	점수를 나눠 확률을 날카롭게 또는 평평하게	B.5
시그모이드	두 칸짜리 소프트맥스	B.5
교차 엔트로피	정답 확률의 -log 평균. 시작은 `log(클래스 수)`	B.6
분산, 표준편차	평균에서 흩어진 정도	B.7
표준화, 정규화	평균을 빼고 표준편차로 나누기	B.7
표준오차	평균의 흔들림. 표준편차 ÷ n의 제곱근	B.7

본문에서 만나는 말	한 줄 직관	부록
MAE, RMSE	오차의 절댓값 평균, 제곱 평균의 제곱근	B.7
정규분포, `randn`	평균 근처에 몰린 종 모양. 표준편차 2개 안에 약 95%	B.8
재매개변수화	평균 + 표준편차 × 무작위 값으로 뽑기	B.8
샘플링, `multinomial`	확률대로 뽑기. 할 때마다 답이 다르다	B.8

부록 C. 용어집

책을 읽다가 “이 말이 어디서 나왔더라?” 싶을 때가 있죠. 딥러닝은 새 용어가 유난히 많은 분야입니다. 같은 것을 가리키는 말이 둘 셋씩 있기도 하고(기울기와 그래디언트, 레이블과 라벨), 같은 말이 장마다 다른 것을 가리키기도 해요(4장의 보폭과 9장의 보폭, 10장의 머리와 12장의 헤드). 이 부록은 2권에 나온 핵심 용어를 한곳에 모은 작은 사전입니다.

읽는 법은 간단합니다.

- **용어**는 이 책 본문에서 쓴 이름 그대로입니다. 한글 용어는 가나다순으로, 약자나 이름을 영문 그대로 부르는 용어(Adam, LSTM, ONNX 등)는 C.2절에 알파벳순으로 모았어요.
- **영문**은 그 용어를 영어 자료에서 찾을 때 쓰는 이름입니다. 공식 문서나 논문을 찾아볼 때 이 이름으로 검색하면 됩니다.
- **뜻**은 한 줄 요약입니다. 정확한 설명과 예제는 해당 장에 있어요.
- **장**은 그 용어를 **처음 설명한 장**입니다. 대부분 본문에서 굵게 표시하며 소개한 곳이고, 그 앞 장에서 “몇 장에서 배웁니다”처럼 이름만 미리 나온 경우는 실제로 설명한 장을 적었습니다. 장 번호는 원고를 검색해 확인했어요.

수학 용어(내적, 코사인 유사도, 소프트맥스, 표준화 등)는 부록 B에 직관과 코드가 함께 있으니 같이 보세요.

C.1 한글 용어

ㄱ

용어	영문	뜻	장
가중치	weight	입력에 곱하는 학습 대상 숫자. 층의 `weight` 행렬	6장
가중치 감쇠	weight decay	가중치가 커질수록 벌점을 줘 작게 유지하는 정규화. L2 정규화와 비슷	3장
가중치 초기화	weight initialization	학습 전에 가중치에 처음 넣는 값을 정하는 방법. Kaiming 초기화 등	8장
가지치기	pruning	가망 없는 탐색 시도를 일찍 멈추는 것(18장), 또는 중요도가 낮은 가중치를 0으로 만드는 것(19장)	18장
강화 학습	reinforcement learning	행동하고 보상을 받으며 더 나은 행동을 배우는 머신러닝 갈래	1장
검증 데이터	validation data	설정을 고르고 학습을 점검하는 데 쓰는 데이터. 시험 데이터와 따로 둔다	1장
결정 트리	decision tree	예/아니오 질문을 이어 붙여 답을 고르는 모델. 스무고개	3장
결측치	missing value	데이터에서 비어 있는 값. 채우거나 버려야 한다	2장

용어	영문	뜻	장
경사하강법	gradient descent	기울기의 반대 방향으로 조금씩 움직여 손실을 줄이는 방법	5장
계산 그래프	computational graph	텐서가 어떤 계산을 거쳐 만들어졌는지 PyTorch가 기록한 연결 구조	5장
고전 머신러닝	classical machine learning	신경망이 아닌 머신러닝 방법. 선형 모델, 트리, k-means 등	3장
고정 메모리	pinned memory	운영체제가 옮기지 않도록 고정한 CPU 메모리. GPU로 복사를 빠르게 하고 non_blocking과 함께 쓴다	16장
과다 구독	oversubscription	프로세스와 스레드 수의 곱이 CPU 코어 수를 넘어 서로 방해하는 상태	20장
과소적합	underfitting	모델이 학습 데이터조차 잘 맞히지 못하는 상태	1장
과적합	overfitting	학습 데이터는 외워서 잘 맞히지만 새 데이터에서는 못하는 상태	1장

용어	영문	뜻	장
교차 검증	cross validation	데이터를 여러 조각으로 나눠 번갈아 검증하고 점수를 평균 내는 방법	3장
교차 어텐션	cross-attention	쿼리는 한 문장에서, 키와 값은 다른 문장에서 가져오는 어텐션	12장
교차 엔트로피	cross entropy	정답 클래스에 준 확률의 -log 평균. 분류의 기본 손실	6장
군집	clustering	정답 없이 비슷한 샘플끼리 묶는 일. k-means가 대표적	3장
그래디언트	gradient	파라미터마다의 기울기를 모은 것. 손실이 가장 가파르게 커지는 방향	5장
그래디언트 누적	gradient accumulation	작은 배치 여러 번의 기울기를 더해 두었다가 한 번에 가중치를 고쳐 큰 배치 효과를 내는 방법	17장
그래디언트 부스팅	gradient boosting	앞 나무가 틀린 만큼을 다음 나무가 고쳐 가며 쌓는 앙상블	3장
그래디언트 클리핑	gradient clipping	기울기의 길이가 정한 값을 넘으면 방향은 두고 길이만 줄이는 방법	8장

용어	영문	뜻	장
그래프 끊김	graph break	torch.compile이 따라가지 못하는 코드에서 그래프를 끊고 파이썬으로 돌아가는 일	16장
기울기	slope, gradient	입력을 조금 움직였을 때 출력이 변하는 비율. 미분값	5장
기울기 소실	vanishing gradient	역전파에서 기울기가 거듭 곱해지며 0에 가까워져 앞쪽 층이 배우지 못하는 현상	8장
기울기 폭주	exploding gradient	역전파에서 기울기가 거듭 곱해지며 터지듯 커지는 현상	8장
기준 모델	baseline	새 방법과 비교하려고 먼저 만드는 단순한 모델	3장
긴 꼬리	long tail	소수가 대부분을 차지하고 나머지가 길게 늘어선 분포. 인기 쏠림	21장

ㄴ

용어	영문	뜻	장
내보내기	export	PyTorch 모델을 ONNX 같은 다른 형식의 파일로 저장하는 일	19장

용어	영문	뜻	장
내적	dot product	두 벡터의 같은 자리끼리 곱해 모두 더한 값. 닮은 정도	12장
노드	node	ONNX 계산 그래프에서 연산 하나를 나타내는 칸	19장

ㄷ

용어	영문	뜻	장
다단계 빌드	multi-stage build	Docker에서 빌드용 단계와 실행용 단계를 나눠 이미지를 작게 만드는 방법	20장
다중공선성	multicollinearity	서로 비슷하게 움직이는 특성이 함께 들어 있는 상태. 계수를 곧이곧대로 해석하기 어려워진다	3장
다층 퍼셉트론	multi-layer perceptron, MLP	선형층과 활성화 함수를 여러 겹 쌓은 기본 신경망	6장
대규모 언어 모델	large language model, LLM	아주 많은 글로 다음 토큰 예측을 학습한 큰 트랜스포머 모델	14장
데이터 누수	data leakage	평가에 쓸 정보가 학습 과정에 새어 들어가 점수가 부풀려지는 일	2장

용어	영문	뜻	장
데이터 병렬	data parallel	GPU마다 모델 사본을 두고 데이터를 나눠 학습한 뒤 기울기를 모으는 방법	17장
데이터 증강	data augmentation	뒤집기, 자르기처럼 뜻을 유지하며 샘플을 조금씩 바꿔 데이터를 늘리는 방법	7장
데이터 파이프라인	data pipeline	데이터를 읽고 손질해 배치로 모델에 나르는 과정 전체	7장
동적 배칭	dynamic batching	서버가 짧은 시간 동안 들어온 요청을 모아 한 배치로 추론하는 방법	20장
동적 양자화	dynamic quantization	가중치는 미리, 활성값은 실행할 때마다 눈금을 정해 정수로 바꾸는 양자화	19장
동적 차원	dynamic dimension	내보낸 모델에서 크기를 미리 정하지 않고 실행할 때마다 달라질 수 있게 열어 둔 차원. 묶음 크기 등	19장
동적 패딩	dynamic padding	배치마다 그 배치에서 가장 긴 길이까지만 채우는 패딩	13장
드롭아웃	dropout	학습 중 무작위로 일부 값을 0으로 끄는 정규화	6장

용어	영문	뜻	장
디코더	decoder	인코더가 만든 표현을 받아 결과(글, 사진)를 만들어 내는 부분	12장
디코딩	decoding	언어 모델의 확률에서 다음 토큰을 고르는 방법. 탐욕, 샘플링 등	14장

ㄹ

용어	영문	뜻	장
랜덤 포레스트	random forest	무작위로 다르게 학습한 결정 트리 여러 그루의 투표로 답하는 모델	3장
랭크	rank	LoRA에서 작은 행렬의 폭. 고칠 수 있는 정도를 정한다. 분산 학습의 프로세스 번호 rank와는 다른 말(17장)	14장
레이블	label	모델이 맞혀야 하는 정답. 라벨이라고도 쓴다	1장
로그 눈금	log scale	값이 같은 비율로 늘어나게 매긴 눈금. 학습률 탐색에 쓴다	18장
로지스틱 회귀	logistic regression	선형 점수에 시그모이드를 씌워 확률로 답하는 분류 모델	3장
로짓	logit	소프트맥스나 시그모이드에 넣기 전의 점수	6장

ㅁ

용어	영문	뜻	장
머리	head	사전학습 모델에서 문제에 맞게 바꿔 다는 마지막 층	10장
머신러닝	machine learning	규칙을 직접 짜지 않고 데이터에서 규칙을 배우게 하는 방법	1장
멀티헤드 어텐션	multi-head attention	어텐션을 여러 개 나란히 돌려 서로 다른 관점으로 보게 하는 구조	12장
명시적 피드백	explicit feedback	별점이나 좋아요처럼 사용자가 직접 남긴 평가	21장
모델	model	입력을 받아 예측을 내는 계산 틀. 학습으로 파라미터를 정한다	1장
모델 병렬	model parallel	모델 자체를 여러 GPU에 나눠 담는 방법. 텐서 병렬, 파이프라인 병렬 등	17장
모델 서빙	model serving	학습한 모델을 서버에 올려 요청에 예측을 돌려주는 일	20장
모드 붕괴	mode collapse	GAN의 생성자가 몇 가지 비슷한 결과만 만들게 되는 현상	15장
몸통	backbone	사전학습 모델에서 특징을 뽑는 앞부분	10장

용어	영문	뜻	장
문맥 임베딩	contextual embedding	같은 단어라도 문장 속 쓰임에 따라 달라지는 임베딩	13장
미니배치	mini-batch	데이터 전체 대신 일부를 뽑아 기울기를 구하는 묶음	5장
미들웨어	middleware	모든 요청이 경로 함수에 가기 전과 후에 거치는 관문	20장
미세조정	fine-tuning	사전학습 모델을 우리 데이터로 조금 더 학습시키는 것	10장

ㅂ

용어	영문	뜻	장
배율	scale	양자화에서 정수 눈금 하나가 뜻하는 실수 크기. 실수 = (정수 - 영점) × 배율	14장
배치	batch	한 번에 모델에 넣는 샘플 묶음. 텐서의 맨 앞 차원	4장
배치 정규화	batch normalization	배치 안 샘플끼리의 평균과 표준편차로 중간값을 표준화하는 층	8장

용어	영문	뜻	장
배치 정규화 접어 넣기	batch norm folding	평가 모드의 배치 정규화를 앞 합성곱의 가중치와 편향에 미리 합쳐 층 하나를 없애는 최적화	19장
백분위수	percentile	값을 줄 세웠을 때 몇 %가 그보다 작은지로 정한 값. 지연 시간의 p50, p95, p99	20장
범주형	categorical	숫자가 아니라 종류로 나뉘는 값. 색, 지역 등	2장
병목	bottleneck	가장 좁은 곳. 오토인코더의 압축 층, 또는 전체 속도를 늦추는 단계	7장
보간	interpolation	두 점 사이를 조금씩 옮겨 가며 중간 값을 만드는 것	15장
보정	calibration	정적 양자화에서 대표 데이터를 흘려 활성값의 눈금을 정하는 단계	19장
보폭	stride	합성곱 필터가 한 번에 건너뛰는 칸 수. 텐서 메모리의 칸 간격이기도 하다	4장
분류	classification	정해진 종류 중 하나를 고르는 문제	1장

용어	영문	뜻	장
분류기 없는 가이던스	classifier-free guidance	조건이 있을 때와 없을 때의 예측 차이를 키워 조건을 더 잘 따르게 하는 확산 모델 기법	15장
불균형 데이터	imbalanced data	클래스마다 샘플 수 차이가 큰 데이터	2장
브로드캐스팅	broadcasting	모양이 다른 텐서끼리 계산할 때 작은 쪽을 늘려 맞추는 규칙	4장
비동기 실행	asynchronous execution	CPU가 GPU에 일을 맡기고 끝나길 기다리지 않고 다음 줄로 넘어가는 방식. 시간을 잴 때 동기화가 필요한 이유	16장
비지도 학습	unsupervised learning	정답 없이 데이터의 구조를 찾는 머신러닝 갈래	1장

ㅅ

용어	영문	뜻	장
사전학습 모델	pretrained model	남이 큰 데이터로 미리 학습해 공개한 모델	10장
상호 상관	cross-correlation	필터를 뒤집지 않고 겹쳐 곱해 더하는 연산. PyTorch 합성곱의 실제 계산	9장

용어	영문	뜻	장
샘플	sample	데이터 표의 한 행. 사진 한 장, 손님 한 명	1장
샘플러	sampler	DataLoader가 어떤 순서로 샘플을 꺼낼지 정하는 객체	7장
생성 모델	generative model	데이터와 비슷한 새 샘플을 만들어 내는 모델	15장
생성자	generator	GAN에서 가짜 샘플을 만드는 신경망	15장
서브워드	subword	단어보다 작고 글자보다 큰 토큰 단위	13장
선형 탐침	linear probe	학습된 표현 위에 직선 모델을 얹어 어떤 정보가 담겼는지 재는 방법	21장
선형 회귀	linear regression	특성에 가중치를 곱해 더한 직선(평면)으로 숫자를 예측하는 모델	3장
세션	session	ONNX Runtime에서 모델을 불러와 추론을 실행하는 객체	19장
셀 상태	cell state	LSTM에서 기억을 따로 실어 나르는 통로	11장
셀프 어텐션	self-attention	문장이 자기 자신의 다른 위치를 보는 어텐션	12장
소프트맥스	softmax	점수에 exp를 씌우고 합으로 나눠 합이 1인 확률로 바꾸는 함수	6장

용어	영문	뜻	장
손실	loss	모델이 얼마나 틀렸는지를 나타내는 숫자. 학습은 이것을 줄인다	1장
손실 함수	loss function	예측과 정답으로 손실을 계산하는 함수. MSE, 교차 엔트로피 등	6장
수용 영역	receptive field	출력 한 칸이 입력 그림에서 보는 범위	9장
순전파	forward pass	입력에서 출력까지 계산을 차례로 하는 것	6장
순환 신경망	recurrent neural network, RNN	순서 데이터를 한 단계씩 읽으며 은닉 상태를 고쳐 쓰는 신경망	11장
스케일링	scaling	특성의 눈금(범위)을 맞추는 전처리. 표준화 등	2장
스트리밍	streaming	결과를 다 만들기 전에 만들어진 부분부터 흘려보내는 응답 방식	20장
시계열	time series	시간 순서대로 잰 값의 나열	11장
시그모이드	sigmoid	점수 하나를 0과 1 사이 확률로 바꾸는 S자 함수	3장
실루엣 점수	silhouette score	군집이 얼마나 잘 나뉘었는지 -1~1로 재는 지표	3장

용어	영문	뜻	장
실행 장치	execution provider	ONNX Runtime이 계산을 맡길 하드웨어 백엔드. CPU, CUDA 등	19장

ㅇ

용어	영문	뜻	장
암묵적 피드백	implicit feedback	방문, 클릭, 구매처럼 행동으로 드러난 기록	21장
양방향	bidirectional	순서를 앞에서 뒤로, 뒤에서 앞으로 모두 읽는 RNN 구성	11장
양자화	quantization	32비트 실수 가중치를 8비트 정수 등으로 바꿔 모델을 줄이는 방법	14장
어댑터	adapter	원래 모델은 얼려 두고 덧붙여 학습하는 작은 부품. LoRA 가중치	14장
어텐션	attention	쿼리와 키의 닮은 정도로 값들을 섞어 꺼내는 계산. 부드러운 딕셔너리	12장
언더플로	underflow	숫자가 너무 작아 자료형이 나타내지 못하고 0이 되어 버리는 일	16장
에폭	epoch	학습 데이터 전체를 한 바퀴 다 보는 단위	5장

용어	영문	뜻	장
역전파	backpropagation	연쇄 법칙으로 손실의 기울기를 출력 쪽에서 입력 쪽으로 거꾸로 구하는 방법	5장
연쇄 법칙	chain rule	이어진 계산의 기울기는 단계마다의 기울기를 곱한 것이라는 규칙	5장
영점	zero point	양자화에서 실수 0에 대응하는 정수 값	19장
예측	prediction	학습한 모델이 새 입력에 내놓는 답	1장
오토인코더	autoencoder	입력을 작게 줄였다가 다시 되살리도록 학습하는 신경망	15장
온도	temperature	소프트맥스 전에 점수를 나누는 값. 작으면 날카롭고 크면 고르게	14장
옵티마이저	optimizer	기울기를 보고 파라미터를 고치는 규칙. SGD, Adam 등	6장
옵티마이저 상태	optimizer state	옵티마이저가 파라미터마다 따로 기억하는 값. Adam은 파라미터의 2배	17장
워밍업	warm-up	학습 초반에 학습률을 작게 시작해 점점 올리는 것	8장

용어	영문	뜻	장
원-핫 인코딩	one-hot encoding	종류마다 칸을 하나씩 두고 해당 칸만 1로 두는 표현	2장
은닉 상태	hidden state	RNN이 지금까지 읽은 내용을 요약해 들고 다니는 벡터	11장
음성 샘플링	negative sampling	가지 않은 항목을 무작위로 뽑아 '이쪽은 낮게'라고 가르치는 방법	21장
이니셜라이저	initializer	ONNX 파일 안에 상수로 저장된 가중치	19장
이동 등변성	translation equivariance	입력을 밀면 출력의 특징 맵도 그만큼 밀리는 합성곱의 성질	9장
이동 불변성	translation invariance	입력을 조금 밀어도 출력이 바뀌지 않는 성질	9장
이미지	image	Docker에서 프로그램과 실행 환경을 통째로 묶은 꾸러미. 컨테이너의 설계도	20장
이상 탐지	anomaly detection	평소와 다른 샘플을 찾아내는 일. 재구성 오차 등을 쓴다	15장
인공지능	artificial intelligence, AI	사람의 지적 행동을 컴퓨터로 흉내 내는 분야 전체	1장
인과 마스크	causal mask	각 위치가 자기보다 뒤의 위치를 보지 못하게 가리는 마스크	12장

용어	영문	뜻	장
인코더	encoder	입력을 읽어 쓸모 있는 표현(특징, 잠재 벡터)으로 바꾸는 부분	12장
일반화	generalization	학습에 없던 새 데이터에서도 잘 맞히는 능력	1장
임베딩	embedding	번호(토큰)를 학습되는 숫자 벡터로 바꾸는 표. `nn.Embedding`	11장

ㅈ

용어	영문	뜻	장
자동 미분	autograd	PyTorch가 계산 기록을 따라 기울기를 자동으로 구하는 기능	5장
자료형	dtype	텐서 숫자의 종류. float32, int64 등	4장
잔차 연결	residual connection	층의 입력을 출력에 그대로 더하는 지름길 연결. ResNet의 핵심	9장
잠재 벡터	latent vector	인코더가 입력을 압축해 만든 작은 벡터	15장
잡음 제거 오토인코더	denoising autoencoder	잡음 섞인 입력에서 깨끗한 원본을 되살리도록 학습한 오토인코더	15장
장치	device	계산을 하는 곳. CPU, CUDA(NVIDIA GPU), MPS(Apple GPU)	4장

용어	영문	뜻	장
재구성 오차	reconstruction error	오토인코더의 입력과 되살린 결과의 차이	15장
재매개변수화	reparameterization trick	평균 + 표준편차 × 무작위 값으로 뽑아 뽑기 과정에도 기울기가 흐르게 하는 VAE 기법	15장
재현성	reproducibility	같은 코드와 설정으로 같은 결과를 다시 얻을 수 있는 성질	8장
재현율	recall	실제 양성 중 모델이 양성이라고 맞힌 비율	3장
전역 평균 풀링	global average pooling	특징 맵 한 장을 평균 하나로 줄이는 풀링	9장
전이 학습	transfer learning	다른 문제로 학습한 모델의 지식을 새 문제에 빌려 쓰는 방법	10장
정밀도	precision	모델이 양성이라고 한 것 중 실제 양성인 비율	3장
정적 양자화	static quantization	보정 데이터로 활성값 눈금까지 미리 정해 두는 양자화	19장
정확도	accuracy	전체 중 맞힌 비율	1장
조기 종료	early stopping	검증 성적이 더 나아지지 않으면 학습을 멈추는 방법	8장
준비 운동	warm-up	시간을 재기 전에 몇 번 미리 돌려 처음 한 번만 드는 비용을 빼는 것	4장

용어	영문	뜻	장
즉시 실행 모드	eager mode	파이썬 코드를 한 줄씩 읽고 바로 실행하는 PyTorch의 기본 방식	16장
지도 학습	supervised learning	입력과 정답 쌍으로 배우는 머신러닝 갈래	1장
지식 증류	knowledge distillation	큰 모델(선생)의 출력 확률을 따라 하도록 작은 모델(학생)을 학습시키는 방법	19장
지역 최솟값	local minimum	주변보다는 낮지만 전체에서 가장 낮지는 않은 곳	5장
지연 시간	latency	요청을 보내고 답을 받기까지 걸린 시간	20장

ㅊ

용어	영문	뜻	장
채널	channel	그림의 겹. 색(RGB)이나 합성곱이 만든 특징 맵 한 장	9장
채팅 템플릿	chat template	대화를 모델이 학습한 형식의 문자열로 바꾸는 규칙	14장
처리량	throughput	정해진 시간에 처리하는 양. 초당 처리한 샘플 수 등	16장

용어	영문	뜻	장
체크포인트	checkpoint	학습 중간 상태(모델, 옵티마이저, 에폭 등)를 저장한 파일	6장
추천 시스템	recommender system	사용자들의 기록에서 취향을 배워 사람마다 맞춘 목록을 보여 주는 시스템	21장
층	layer	신경망을 이루는 계산 단위 하나. 선형층, 합성곱층 등	6장
층 정규화	layer normalization	샘플 하나 안의 숫자끼리 표준화하는 층. 트랜스포머에서 쓴다	12장

ㅋ

용어	영문	뜻	장
캐싱 할당자	caching allocator	PyTorch가 GPU 메모리를 받아 둔 채 다시 나눠 쓰는 관리 방식. 예약 메모리가 할당 메모리보다 큰 이유	16장
커널	kernel	합성곱에서 그림 위를 움직이는 작은 가중치 격자. 필터와 같은 말. GPU에서는 GPU가 한 번에 실행하는 계산 하나를 가리킨다(16장)	9장

용어	영문	뜻	장
커널 실행 비용	kernel launch overhead	GPU에 계산 하나를 맡길 때마다 드는 고정 비용. 작은 일을 여러 번 맡기면 커진다	16장
커널 융합	kernel fusion	여러 계산을 GPU 커널 하나로 합쳐 메모리를 오가는 횟수를 줄이는 것	16장
컨테이너	container	이미지로부터 실행된, 격리된 프로그램 실행 환경	20장
코사인 유사도	cosine similarity	두 벡터의 방향이 얼마나 같은지 -1~1로 잰 값	4장
콜드 스타트	cold start	기록이 없는 새 사용자나 새 항목에게 추천을 시작하기 어려운 문제	21장
클래스 가중치	class weight	드문 클래스를 틀리면 손실을 더 크게 주는 가중치	2장

ㅌ

용어	영문	뜻	장
텐서 코어	Tensor Core	16비트 등 낮은 정밀도 행렬 곱을 아주 빠르게 하는 NVIDIA GPU의 전용 계산 장치	16장

용어	영문	뜻	장
토크나이저	tokenizer	글을 토큰으로 자르고 번호로 바꾸는 도구	13장
토큰	token	모델이 읽는 글의 단위 조각. 글자, 형태소, 서브워드 등	13장
투-타워 모델	two-tower model	사용자 탑과 항목 탑이 각자 번호와 특징으로 벡터를 만들고 내적으로 점수를 내는 추천 모델	21장
트랜스포머	Transformer	어텐션을 중심으로 쌓은 신경망 구조. LLM의 바탕	12장
특성	feature	샘플을 설명하는 값 하나. 데이터 표의 열	1장
특징 맵	feature map	합성곱 필터 하나가 그림 전체에서 반응한 정도를 담은 지도	9장
특징 추출	feature extraction	사전학습 몸통은 얼리고 그 출력(특징)만 써서 머리를 학습하는 방법	10장

표

용어	영문	뜻	장
파라미터	parameter	학습으로 정해지는 모델의 숫자. 가중치와 편향	1장
파라미터 공유	parameter sharing	같은 필터를 그림의 모든 위치에서 함께 쓰는 것	9장
판별 모델	discriminative model	입력을 보고 답(클래스, 숫자)을 고르는 모델	15장
판별자	discriminator	GAN에서 진짜와 가짜를 가려내는 신경망	15장
패딩	padding	모양을 맞추려고 가장자리나 끝에 채워 넣는 값	7장
편향	bias	층의 출력에 더하는 학습 대상 숫자	6장
평가 모드	evaluation mode	`model.eval()`. 드롭아웃을 끄고 배치 정규화가 저장된 통계를 쓰게 한다	8장
표준오차	standard error	평균이 흔들리는 정도. 표준편차 ÷ 개수의 제곱근	18장
표준편차	standard deviation	값들이 평균에서 흩어진 정도. 분산의 제곱근	1장
표준화	standardization	평균을 빼고 표준편차로 나눠 평균 0, 표준편차 1로 맞추는 것	1장

용어	영문	뜻	장
풀링	pooling	구역마다 최댓값이나 평균 하나만 남겨 그림을 줄이는 연산	9장
프로파일러	profiler	프로그램의 어느 부분에서 시간이 드는지 재 주는 도구. torch.profiler	16장
프롬프트	prompt	언어 모델에 주는 입력 글	14장
피드포워드	feed-forward	트랜스포머 블록에서 위치마다 따로 적용하는 작은 MLP	12장
픽스처	fixture	pytest에서 테스트마다 준비물을 만들어 넘겨주는 함수	20장
필터	filter	합성곱에서 찾을 무늬를 담은 작은 가중치 격자	9장

ㅎ

용어	영문	뜻	장
하이퍼파라미터	hyperparameter	학습 전에 사람이 정하는 설정. 학습률, 층 수 등	1장
학습	learning, training	데이터로 모델의 파라미터를 정하는 과정. 훈련	1장

용어	영문	뜻	장
학습 곡선	learning curve	에폭에 따른 학습·검증 손실(정확도)을 그린 그래프	8장
학습률	learning rate	경사하강법에서 한 번에 움직이는 크기	5장
학습률 스케줄러	learning rate scheduler	학습이 진행됨에 따라 학습률을 바꾸는 도구	8장
합성곱	convolution	작은 필터를 그림 위로 옮기며 곱해 더해 무늬를 찾는 연산	9장
합성곱 신경망	convolutional neural network, CNN	합성곱 층을 쌓아 그림을 다루는 신경망	9장
행렬 분해	matrix factorization	사용자 × 항목 표를 작은 표 두 개(사용자 벡터, 항목 벡터)의 곱으로 나타내는 추천 방법	21장
헤드	head	멀티헤드 어텐션에서 나란히 도는 어텐션 하나하나	12장
헬스 체크	health check	감시 프로그램이 서버가 살아 있는지 주기적으로 묻는 요청. `/health`	20장
형태소	morpheme	뜻을 가진 가장 작은 말의 단위	13장
혼동 행렬	confusion matrix	실제 클래스와 예측 클래스의 조합별 개수를 센 표	3장

용어	영문	뜻	장
혼합 정밀도	mixed precision	계산마다 알맞은 자료형을 골라 16비트와 32비트를 섞어 쓰는 학습·추론 방법	16장
확률적 경사하강법	stochastic gradient descent, SGD	무작위로 뽑은 미니배치의 기울기로 한 걸음씩 가는 경사하강법	5장
확산 모델	diffusion model	잡음을 조금씩 걷어 내며 그림을 만들어 가는 생성 모델	15장
환각	hallucination	언어 모델이 그럴듯하지만 사실이 아닌 내용을 지어내는 것	14장
활성화 체크포인팅	activation checkpointing	순전파의 중간 결과를 일부만 저장하고 역전파 때 다시 계산해 메모리를 아끼는 방법	17장
활성화 함수	activation function	층 사이에 넣어 직선을 구부리는 함수. ReLU 등	6장
회귀	regression	숫자를 예측하는 문제	1장

C.2 영문으로 쓰는 용어

약자나 이름을 영문 그대로 부르는 용어는 알파벳순으로 따로 모았습니다.

용어	풀어 쓴 이름	뜻	장
Adam	—	기울기의 평균과 크기를 함께 기억해 파라미터마다 걸음을 조절하는 옵티마이저	6장
AdamW	—	weight decay를 기울기와 따로 적용하는 Adam	8장
all-reduce	—	모든 프로세스의 텐서를 더해(또는 평균 내) 그 결과를 모두에게 똑같이 나눠 주는 통신	17장
autocast	—	연산마다 16비트로 할지 32비트로 할지 PyTorch가 골라 주는 혼합 정밀도 도구	16장
bfloat16	brain floating point 16	float32와 같은 범위에 촘촘함만 줄인 16비트 실수. 언더플로 걱정이 적다	16장
BPE	Byte Pair Encoding	자주 붙어 나오는 조각을 합쳐 가며 서브워드 사전을 만드는 방법	13장
BPR	Bayesian Personalized Ranking	간 곳의 점수가 무작위로 뽑은 곳보다 높아지도록 점수 차이로 가르치는 순위 손실	21장
BPTT	Backpropagation Through Time	RNN을 시간 방향으로 펼쳐 역전파하는 것	11장

용어	풀어 쓴 이름	뜻	장
collate_fn	—	DataLoader가 샘플 여러 개를 배치 하나로 묶는 함수	7장
CUDA	—	NVIDIA GPU에서 계산하는 도구 모음. PyTorch의 장치 이름 `cuda`	4장
DataLoader	—	Dataset에서 샘플을 꺼내 배치로 묶어 나르는 PyTorch 도구	7장
Dataset	—	번호를 주면 샘플 하나를 돌려주는 약속을 지키는 PyTorch 클래스	7장
DDP	DistributedDataParallel	프로세스마다 모델 사본을 두고 기울기를 all-reduce로 맞추는 PyTorch 데이터 병렬 도구	17장
DDPM	Denoising Diffusion Probabilistic Models	잡음 예측을 학습해 그림을 만드는 대표적인 확산 모델	15장
DistributedSampler	—	분산 학습에서 프로세스마다 겹치지 않게 데이터를 나눠 주는 샘플러	17장
Docker	—	프로그램과 실행 환경을 이미지로 묶어 어디서나 같이 실행하게 하는 도구	20장

용어	풀어 쓴 이름	뜻	장
F1 점수	F1 score	정밀도와 재현율의 조화 평균	3장
FastAPI	—	타입 힌트로 요청과 응답을 검사하는 파이썬 웹 프레임워크	20장
FlashAttention	—	어텐션 점수 표를 통째로 만들지 않고 조각으로 계산해 빠르고 메모리를 아끼는 방법	16장
FSDP	Fully Sharded Data Parallel	파라미터·기울기·옵티마이저 상태까지 GPU마다 나눠 담는 데이터 병렬	17장
GAN	Generative Adversarial Network	생성자와 판별자가 겨루며 학습하는 생성 모델	15장
GPU	Graphics Processing Unit	단순한 계산 수천 개를 동시에 하는 그래픽 처리 장치	4장
GradScaler	—	float16 학습에서 손실을 키웠다가 되돌려 작은 기울기가 0이 되는 것을 막는 도구	16장
GRU	Gated Recurrent Unit	LSTM보다 문을 줄인 순환 신경망 셀	11장
Hugging Face	—	사전학습 모델과 데이터셋을 공유하는 플랫폼. transformers 라이브러리	13장

용어	풀어 쓴 이름	뜻	장
k-means	—	중심 k개를 두고 가까운 샘플을 묶고 중심을 옮기기를 반복하는 군집 방법	3장
k-최근접 이웃	k-nearest neighbors, k-NN	가장 가까운 학습 샘플 k개의 다수결(평균)로 답하는 모델	1장
Kaiming 초기화	Kaiming initialization	ReLU 층을 지나도 값의 크기가 유지되도록 정한 가중치 초기화. He 초기화	8장
KL 발산	Kullback-Leibler divergence	두 확률 분포가 얼마나 다른지 재는 값	15장
KV 캐시	key-value cache	생성할 때 앞 토큰들의 키와 값을 저장해 다시 계산하지 않게 하는 것	14장
lifespan	—	FastAPI에서 서버가 켜질 때와 꺼질 때 할 일을 `yield` 앞뒤로 적는 함수. 모델을 한 번만 올릴 때 쓴다	20장
LoRA	Low-Rank Adaptation	큰 가중치는 얼리고 작은 행렬 두 개의 곱만 학습해 고치는 미세조정 방법	14장
LSTM	Long Short-Term Memory	셀 상태와 문으로 긴 기억을 다루는 순환 신경망 셀	11장
MAE	mean absolute error	오차 절댓값의 평균	3장

용어	풀어 쓴 이름	뜻	장
MLflow	—	실험의 설정, 지표, 파일을 기록하고 비교하는 도구	18장
MPS	Metal Performance Shaders	Apple 실리콘 맥의 GPU. PyTorch의 장치 이름 `mps`	4장
MSE	mean squared error	오차 제곱의 평균. 회귀의 기본 손실	3장
NCCL	NVIDIA Collective Communications Library	NVIDIA GPU끼리 all-reduce 같은 통신을 하는 라이브러리. 분산 학습의 기본 백엔드	17장
NDCG@K	normalized discounted cumulative gain	맞힌 항목이 목록 위쪽에 있을수록 높게 쳐 주는 순위 지표. 가장 좋은 순서면 1	21장
nn.Module	—	PyTorch에서 모든 층과 모델이 상속하는 기본 클래스	6장
no_sync	—	DDP에서 기울기 통신을 잠시 미루는 문맥. 그래디언트 누적과 함께 쓴다	17장
NSMC	Naver Sentiment Movie Corpus	네이버 영화 리뷰 감성 분류 데이터	13장
NVLink	—	NVIDIA GPU끼리 PCIe보다 훨씬 빠르게 잇는 전용 연결	17장

용어	풀어 쓴 이름	뜻	장
ONNX	Open Neural Network Exchange	프레임워크와 상관없이 모델을 담는 공개 파일 형식	19장
ONNX Runtime	—	ONNX 모델을 실행하는 추론 엔진	19장
OOM	Out Of Memory	메모리가 모자라 계산을 이어 갈 수 없다는 오류	16장
OOV	out-of-vocabulary	토크나이저 사전에 없는 말	13장
opset	operator set	ONNX 파일이 따르는 연산자 규칙의 판 번호	19장
Optuna	—	하이퍼파라미터 탐색을 자동으로 해 주는 라이브러리	18장
PCIe	PCI Express	CPU와 GPU 같은 장치를 잇는 컴퓨터 안의 통로. 세대와 폭(x8, x16)에 따라 빠르기가 다르다	16장
Pipeline	—	scikit-learn에서 전처리와 모델을 한 줄로 묶는 도구	2장
QDQ	QuantizeLinear / DequantizeLinear	정수로 바꿨다 되돌리는 표시 노드를 그래프에 붙여 양자화를 나타내는 ONNX 형식	19장

용어	풀어 쓴 이름	뜻	장
Recall@K	recall at K	정답 중 상위 K개 안에 든 비율. 재현율을 목록 위쪽 K칸으로 좁힌 것	21장
ReLU	Rectified Linear Unit	음수는 0, 양수는 그대로 내보내는 활성화 함수	6장
ResNet	Residual Network	잔차 연결로 아주 깊이 쌓을 수 있게 한 CNN 구조	9장
RMSE	root mean squared error	MSE의 제곱근. 원래 단위로 읽는 오차	3장
ROC-AUC	area under the ROC curve	문턱을 바꿔 가며 그린 ROC 곡선 아래 넓이. 1에 가까울수록 좋다	3장
R^2	coefficient of determination	평균만 쓰는 예측보다 오차를 얼마나 줄였는지 나타내는 회귀 지표	3장
scikit-learn	—	고전 머신러닝 방법과 전처리, 평가 도구를 모은 파이썬 라이브러리	1장
SDPA	scaled_dot_product_attention	PyTorch가 장치에 맞는 빠른 계산법을 골라 주는 어텐션 함수	16장
state_dict	—	모델이나 옵티마이저의 숫자를 이름별로 담은 딕셔너리. 저장에 쓴다	6장
tanh	hyperbolic tangent	값을 -1과 1 사이로 누르는 S자 활성화 함수	6장

용어	풀어 쓴 이름	뜻	장
TensorBoard	—	학습 중 손실과 지표를 그래프로 보여 주는 기록 도구	18장
TF32	TensorFloat-32	float32 행렬 곱을 텐서 코어로 빠르게 하려고 촘촘함을 조금 줄인 계산 방식	16장
TFLOPS	tera floating-point operations per second	1초에 실수 계산을 몇 조 번 하는지 나타내는 단위	16장
top-k / top-p	—	확률 상위 k개, 또는 누적 확률 p까지의 토큰 중에서만 뽑는 디코딩 방법	14장
torch.compile	—	모델을 미리 그래프로 따서 다듬고 커널을 합쳐 빠르게 만드는 PyTorch 도구	16장
torchrun	—	분산 학습용 프로세스를 GPU 수만큼 띄우고 rank 같은 정보를 넣어 주는 실행 명령	17장
torchvision	—	사진 데이터셋, 변환, 사전학습 모델을 담은 PyTorch 꾸러미	6장
VAE	Variational Autoencoder	잠재 공간을 점 대신 분포로 학습해 새 샘플을 뽑을 수 있게 한 오토인코더	15장

용어	풀어 쓴 이름	뜻	장
world size	—	분산 학습에 참여하는 프로세스 수. 각 프로세스는 0부터 world size - 1까지의 rank를 받는다	17장
ZeRO	Zero Redundancy Optimizer	데이터 병렬에서 옵티마이저 상태 등을 GPU마다 나눠 담아 중복을 없애는 방법	17장

영어 자료를 찾아볼 때 한 가지 팁을 덧붙이면, 같은 개념이라도 분야마다 부르는 이름이 조금씩 다릅니다. 예를 들어 "가중치 감쇠"는 영어 자료에서 weight decay와 L2 regularization이 섞여 쓰이고(8장에서 둘이 Adam에서는 다르게 동작한다는 것을 봤죠), "특성"은 feature, "특징"도 feature입니다. 이 책은 대체로 표 데이터의 열은 **특성**, 신경망이 뽑아낸 숫자 묶음은 **특징**이라고 나눠 불렀어요. 검색 결과가 이상하면 표의 영문 이름과 함께 PyTorch, scikit-learn 같은 도구 이름을 붙여 찾아보세요.